

```
=====
FILE: catalogs\__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: catalogs\db2_naming_catalog.py
ENCODING: utf-8
=====
```

```
"""
DB2 naming catalog.

This file owns DB2 naming conventions so resolvers and generators do not
hardcode table suffixes or cursor suffix rules.
"""
```

```
DB2_TABLE_SUFFIX_EQUIVALENTS = [
    ("_TB", "_TV"),
    ("_TV", "_TB"),
    ("TB", "TV"),
    ("TV", "TB"),
]
```

```
DB2_CURSOR_SUFFIX = "C1"
```

```
DB2_DEFAULT_CURSOR_NAME = "DB2CURC1"
```

```
DB2_HOST_REFERENCE_PREFIX = ":"
```

```
=====
FILE: catalogs\db2_sql_types.py
ENCODING: utf-8
=====
```

```
"""
DB2 SQL type catalog.

DCLGEN parsing logic must import SQL type starters from here.
"""
```

```
DB2_SQL_TYPE_STARTERS = {
    "CHAR",
    "CHARACTER",
    "VARCHAR",
    "LONG",
    "GRAPHIC",
    "VARGRAPHIC",
    "SMALLINT",
    "INTEGER",
    "INT",
    "BIGINT",
    "DECIMAL",
    "DEC",
    "NUMERIC",
    "NUM",
    "FLOAT",
    "REAL",
    "DOUBLE",
    "DATE",
    "TIME",
    "TIMESTAMP",
    "BLOB",
    "CLOB",
    "DBCLOB",
    "XML",
}
```

```

DB2_SQL_SKIP_WORDS = {
    "EXEC",
    "SQL",
    "DECLARE",
    "TABLE",
    "END-EXEC",
    "END",
    "NOT",
    "NULL",
    "WITH",
    "DEFAULT",
    "PRIMARY",
    "FOREIGN",
    "KEY",
    "CONSTRAINT",
    "UNIQUE",
    "CHECK",
    "REFERENCES",
    "CREATE",
    "IN",
    "IS",
    "THE",
    "DCLGEN",
    "COMMAND",
    "THAT",
    "MADE",
    "FOLLOWING",
    "STATEMENTS",
}

```

```

=====
FILE: catalogs\dclgen_schema.py
ENCODING: utf-8
=====

```

```

"""
DCLGEN schema catalog.

This file owns DCLGEN-related static names and conventions.
Parser and service classes must import from here instead of hardcoding
DCLGEN names internally.
"""

```

```

DCLGEN_GROUP_PREFIX = "DCL"

```

```

DCLGEN_COLUMN_FIELDS = [
    "table_name",
    "column_name",
    "db2_type",
    "cobol_host_name",
    "cobol_picture",
    "cobol_usage",
    "nullable",
]

```

```

DCLGEN_HOST_FIELD_SUFFIXES_TO_IGNORE = [
    "-NULL",
    "-LEN",
    "-TEXT",
]

```

```

=====
FILE: catalogs\input_file_roles.py
ENCODING: utf-8
=====

```

```

"""
Input file role catalog.

```

Input labels are centralized here so UI, runners, validators, and diagnostics
do not hardcode file role names.
"""

```
INPUT_ROLE_SHEET_MAPPING = "Sheet Mapping"
INPUT_ROLE_DCLGEN = "DCLGEN"
INPUT_ROLE_COPYBOOK = "Copybook"
INPUT_ROLE_IDMS_COBOL_SOURCE = "IDMS COBOL Source"
```

```
REQUIRED_INPUT_ROLES = [
    INPUT_ROLE_SHEET_MAPPING,
    INPUT_ROLE_DCLGEN,
    INPUT_ROLE_IDMS_COBOL_SOURCE,
]
```

```
OPTIONAL_INPUT_ROLES = [
    INPUT_ROLE_COPYBOOK,
]
```

```
=====
FILE: catalogs\output_sections.py
ENCODING: utf-8
=====
```

```
# LOCATION: catalogs/output_sections.py
# ACTION: REPLACE ENTIRE FILE
```

"""Output section titles and generated block markers.

Generators and composers must import section labels from this catalog
instead of hardcoding generated block titles inside logic classes.

Constants only. No regex, no runtime logic, no program / record / table /
cursor / host variable names.
"""

```
from __future__ import annotations
```

```
# =====
# Generated block markers
# =====
DB2_INFRASTRUCTURE_MARKER = (
    "DB2 SQLCA, SQL ERROR WORKING STORAGE, DCLGEN INCLUDES, AND CURSOR FLAGS"
)
```

```
DB2_SQL_ERROR_LOCATION_MARKER = "DB2 SQL ERROR LOCATION"
```

```
DB2_CURSOR_FLAGS_MARKER = "DB2 CURSOR END-OF-CURSOR FLAGS"
DB2_CURSOR_DECLARATIONS_MARKER = "DB2 CURSOR DECLARATIONS"
DB2_CURSOR_PARAGRAPH_MARKER = (
    "DB2 GENERATED CURSOR OPEN FETCH CLOSE PARAGRAPHS"
)
```

```
DB2_GENERATED_TIMESTAMP_MARKER = (
    "DB2 GENERATED TIMESTAMP AND AUDIT WORKING STORAGE"
)
```

```
# =====
# Marker comment geometry
# =====
# A marker comment must fit the 65-column body window (cols 8-72).
# DB2_INFRASTRUCTURE_MARKER is 70 characters, so the old
# "{title:<62}*" template overflowed and wrapped onto a second line:
#
#     *DB2 SQLCA, SQL ERROR WORKING STORAGE, DCLGEN INCLUDES, AND CURSOR
#     *FLAGS*
#
# The template no longer pads to a fixed trailing star, and long titles
# are truncated at the body width by the emitting helper rather than
# wrapping into a broken second comment line.
COMMENT_BODY_WIDTH = 65
COMMENT_MARKER_TEMPLATE = "{title}"
```

```

COMMENT_RULE_LINE = "*" + "-" * (COMMENT_BODY_WIDTH - 2) + "*"

# Short form used when the full marker will not fit.
DB2_INFRASTRUCTURE_MARKER_SHORT = "DB2 INFRASTRUCTURE - SQLCA, INCLUDES, FLAGS"

# =====
# Infrastructure include names
# =====
SQLERRWS_INCLUDE_NAME = "SQLERRWS"
SQLCA_INCLUDE_NAME = "SQLCA"
SQLERROR_INCLUDE_NAME = "SQLERROR"

# Includes that are DB2 infrastructure, never DCLGEN table includes.
INFRASTRUCTURE_INCLUDE_NAMES = frozenset(
    {
        SQLCA_INCLUDE_NAME,
        SQLERRWS_INCLUDE_NAME,
        SQLERROR_INCLUDE_NAME,
        "GEN",
    }
)

# Manual reference emits a single-line include, not a three-line block.
INCLUDE_SINGLE_LINE = True
INCLUDE_STATEMENT_TEMPLATE = "EXEC SQL INCLUDE {name} END-EXEC."

# =====
# SQL-LOCATION
# =====
SQL_LOCATION_FIELD_NAME = "SQL-LOCATION"
SQL_LOCATION_PICTURE = "PIC X(40) VALUE SPACES."

# --- SQL-LOCATION ownership -----
# SQLERRWS declares SQL-LOCATION. The converter must NOT declare it
# again: a duplicate data-name is a compile error, and the COBOL team's
# manual reference program carries no local 01 SQL-LOCATION entry.
#
# Consumed by:
#   src/idms_db2_phase2/generators/db2_infrastructure/
#   infrastructure_block_builder.py
DECLARE_SQL_LOCATION_FIELD = False

# =====
# DCLGEN
# =====
DCLGEN_GROUP_PREFIX = "DCL"
DCLGEN_INITIALIZE_TEMPLATE = "INITIALIZE {group}"

=====
FILE: catalogs\sheet_mapping_schema.py
ENCODING: utf-8
=====

"""
Sheet Mapping schema catalog.

This module owns all Sheet Mapping names, aliases, and model-field mapping.
Parser logic must not hardcode Sheet Mapping column labels.
"""

SHEET_MAPPING_COLUMN_IDMS_TO_DB2_MAPPING = "IDMS to DB2 Mapping"
SHEET_MAPPING_COLUMN_COBOL_RECORD_IDMS = "Cobol Record IDMS"
SHEET_MAPPING_COLUMN_COBOL_ZONE = "Cobol Zone"
SHEET_MAPPING_COLUMN_IDMS_KEY = "IDMS Key"
SHEET_MAPPING_COLUMN_IDMS_PIC_CLAUSE = "IDMS PIC Clause"
SHEET_MAPPING_COLUMN_LENGTH_OF_FIELD_BYTES = "Length of Field Bytes"
SHEET_MAPPING_COLUMN_FIELD_END_POSITION = "Field end position"
SHEET_MAPPING_COLUMN_DB2_KEY = "DB2 Key"
SHEET_MAPPING_COLUMN_NEW_DB2_RECORD = "New DB2 Record"
SHEET_MAPPING_COLUMN_NEW_DB2_FIELD_NAME = "New DB2 Field name"
SHEET_MAPPING_COLUMN_NEW_DB2_DATA_TYPE = "New DB2 Data Type"
SHEET_MAPPING_COLUMN_HOPEX_EXPRESSION_TYPE_REMARK = "Hopex Expression TypeRemark"
SHEET_MAPPING_COLUMN_REMARKS = "Remarks"
SHEET_MAPPING_COLUMN_RELATION = "Relation"

```

```

SHEET_MAPPING_COLUMN_REFERENCE_FIELD_NAME_COPYBOOK = "Reference Field Name (CopyBook) "
SHEET_MAPPING_COLUMN_REFERENCE_FIELD_PIC_CLAUSE = "Reference Field PIC Clause"
SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_TABLE = "Cross Application DB2 table"
SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_FIELD_NAME = "Cross Application DB2 Field Name"
SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_DATA_TYPE = "Cross Appln DB2 Data Type"
SHEET_MAPPING_COLUMN_BASETYPE = "Basetype"

```

```

SHEET_MAPPING_CANONICAL_COLUMNS = [
    SHEET_MAPPING_COLUMN_IDMS_TO_DB2_MAPPING,
    SHEET_MAPPING_COLUMN_COBOL_RECORD_IDMS,
    SHEET_MAPPING_COLUMN_COBOL_ZONE,
    SHEET_MAPPING_COLUMN_IDMS_KEY,
    SHEET_MAPPING_COLUMN_IDMS_PIC_CLAUSE,
    SHEET_MAPPING_COLUMN_LENGTH_OF_FIELD_BYTES,
    SHEET_MAPPING_COLUMN_FIELD_END_POSITION,
    SHEET_MAPPING_COLUMN_DB2_KEY,
    SHEET_MAPPING_COLUMN_NEW_DB2_RECORD,
    SHEET_MAPPING_COLUMN_NEW_DB2_FIELD_NAME,
    SHEET_MAPPING_COLUMN_NEW_DB2_DATA_TYPE,
    SHEET_MAPPING_COLUMN_HOPEX_EXPRESSION_TYPE_REMARK,
    SHEET_MAPPING_COLUMN_REMARKS,
    SHEET_MAPPING_COLUMN_RELATION,
    SHEET_MAPPING_COLUMN_REFERENCE_FIELD_NAME_COPYBOOK,
    SHEET_MAPPING_COLUMN_REFERENCE_FIELD_PIC_CLAUSE,
    SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_TABLE,
    SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_FIELD_NAME,
    SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_DATA_TYPE,
    SHEET_MAPPING_COLUMN_BASETYPE,
]

```

```

SHEET_MAPPING_FIELD_ALIASES = {
    SHEET_MAPPING_COLUMN_COBOL_RECORD_IDMS: [
        "Cobol Record IDMS",
        "COBOL RECORD IDMS",
        "Cobol Record",
        "COBOL Record",
        "IDMS Record",
        "Record IDMS",
        "Cobol Record Name",
        "IDMS to DB2 Mapping",
        "Cobol Recrd IDMS",
        "COBOL RECRD IDMS",
        "Cobol Rec IDMS",
        "COBOL REC IDMS",
        "Cobol Rord IDMS",
        "COBOL RCRD IDMS",
        "Cobol Recrd",
        "COBOL RECRD",
    ],
    SHEET_MAPPING_COLUMN_COBOL_ZONE: [
        "Cobol Zone",
        "COBOL Zone",
        "COBOL ZONE",
        "Cobol Field",
        "COBOL Field",
        "IDMS Field",
        "IDMS COBOL Zone",
        "IDMS COBOL Field",
        "Zone",
    ],
    SHEET_MAPPING_COLUMN_IDMS_KEY: [
        "IDMS Key",
        "IDMS KEY",
        "Key IDMS",
        "IDMS_Key",
    ],
    SHEET_MAPPING_COLUMN_IDMS_PIC_CLAUSE: [
        "IDMS PIC Clause",
        "IDMS PIC",
        "PIC Clause",
        "Picture",
        "PIC",
    ],
    SHEET_MAPPING_COLUMN_LENGTH_OF_FIELD_BYTES: [

```

```

        "Length of Field Bytes",
        "Length",
        "Field Length",
        "Length Bytes",
        "Length of Field",
    ],
    SHEET_MAPPING_COLUMN_FIELD_END_POSITION: [
        "Field end position",
        "Field End Position",
        "End Position",
        "Field End",
    ],
    SHEET_MAPPING_COLUMN_DB2_KEY: [
        "DB2 Key",
        "DB2 key",
        "DB2 KEY",
        "Key DB2",
        "DB2_key",
        "DB2_Key",
        "DB2KEY",
    ],
    SHEET_MAPPING_COLUMN_NEW_DB2_RECORD: [
        "New DB2 Record",
        "New DB2 Record ",
        "DB2 Record",
        "DB2 Table",
        "New DB2 Table",
        "Table",
        "DB2_Table",
        "New DB2_Record",
    ],
    SHEET_MAPPING_COLUMN_NEW_DB2_FIELD_NAME: [
        "New DB2 Field name",
        "New DB2 Field Name",
        "New DB2_Field name",
        "New DB2_Field Name",
        "New DB2 Field",
        "New DB2_Field",
        "DB2 Field",
        "DB2 Column",
        "New DB2 Column",
        "Column",
        "DB2_Field",
        "DB2_Column",
    ],
    SHEET_MAPPING_COLUMN_NEW_DB2_DATA_TYPE: [
        "New DB2 Data Type",
        "New DB2 DataType",
        "New DB2_Data Type",
        "DB2 Data Type",
        "DB2 Type",
        "Data Type",
    ],
    SHEET_MAPPING_COLUMN_HOPEX_EXPRESSION_TYPE_REMARK: [
        "Hopex Expression TypeRemark",
        "Hopex Expression Type",
        "Expression Type",
        "Hopex Remark",
    ],
    SHEET_MAPPING_COLUMN_REMARKS: [
        "Remarks",
        "Remark",
        "Comments",
        "Comment",
    ],
    SHEET_MAPPING_COLUMN_RELATION: [
        "Relation",
        "Relationship",
        "Set Relation",
    ],
    SHEET_MAPPING_COLUMN_REFERENCE_FIELD_NAME_COPYBOOK: [
        "Reference Field Name (CopyBook) ",
        "Reference Field Name (CopyBook)",
        "Reference Field Name",
        "CopyBook Field",
        "Copybook Field",
    ]

```

```

        "Reference Field",
    ],
    SHEET_MAPPING_COLUMN_REFERENCE_FIELD_PIC_CLAUSE: [
        "Reference Field PIC Clause",
        "Reference PIC Clause",
        "Reference PIC",
    ],
    SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_TABLE: [
        "Cross Application DB2 table",
        "Cross Application DB2 Table",
        "Cross App DB2 Table",
        "Cross Application Table",
    ],
    SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_FIELD_NAME: [
        "Cross Application DB2 Field Name",
        "Cross App DB2 Field Name",
        "Cross Application DB2 Field",
        "Cross App DB2 Field",
    ],
    SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_DATA_TYPE: [
        "Cross Appln DB2 Data Type",
        "Cross Application DB2 Data Type",
        "Cross App DB2 Data Type",
    ],
    SHEET_MAPPING_COLUMN_BASETYPE: [
        "Basetype",
        "Base Type",
        "BaseType",
    ],
    ],
}

SHEET_MAPPING_MODEL_FIELD_MAP = {
    "cobol_record_idms": SHEET_MAPPING_COLUMN_COBOL_RECORD_IDMS,
    "cobol_zone": SHEET_MAPPING_COLUMN_COBOL_ZONE,
    "idms_key": SHEET_MAPPING_COLUMN_IDMS_KEY,
    "idms_pic_clause": SHEET_MAPPING_COLUMN_IDMS_PIC_CLAUSE,
    "length_of_field_bytes": SHEET_MAPPING_COLUMN_LENGTH_OF_FIELD_BYTES,
    "field_end_position": SHEET_MAPPING_COLUMN_FIELD_END_POSITION,
    "db2_key": SHEET_MAPPING_COLUMN_DB2_KEY,
    "new_db2_record": SHEET_MAPPING_COLUMN_NEW_DB2_RECORD,
    "new_db2_field_name": SHEET_MAPPING_COLUMN_NEW_DB2_FIELD_NAME,
    "new_db2_data_type": SHEET_MAPPING_COLUMN_NEW_DB2_DATA_TYPE,
    "hopex_expression_type_remark": SHEET_MAPPING_COLUMN_HOPEX_EXPRESSION_TYPE_REMARK,
    "remarks": SHEET_MAPPING_COLUMN_REMARKS,
    "relation": SHEET_MAPPING_COLUMN_RELATION,
    "reference_field_name_copybook": SHEET_MAPPING_COLUMN_REFERENCE_FIELD_NAME_COPYBOOK,
    "reference_field_pic_clause": SHEET_MAPPING_COLUMN_REFERENCE_FIELD_PIC_CLAUSE,
    "cross_application_db2_table": SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_TABLE,
    "cross_application_db2_field_name": SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_FIELD_NAME,
    "cross_application_db2_data_type": SHEET_MAPPING_COLUMN_CROSS_APPLICATION_DB2_DATA_TYPE,
    "basetype": SHEET_MAPPING_COLUMN_BASETYPE,
}

SHEET_MAPPING_HEADER_DETECTION_GROUPS = [
    [
        SHEET_MAPPING_COLUMN_COBOL_RECORD_IDMS,
        "Cobol Recrd IDMS",
        "IDMS Record",
    ],
    [
        SHEET_MAPPING_COLUMN_NEW_DB2_RECORD,
        "DB2 Table",
        "DB2 Record",
    ],
    [
        SHEET_MAPPING_COLUMN_NEW_DB2_FIELD_NAME,
        "New DB2 Field name",
        "DB2 Column",
    ],
]

```

```
=====
FILE: code_review\__init__.py
ENCODING: utf-8
=====
```

```
"""Automated code review of generated DB2 COBOL.
```

```
Reads a converted .cbl, scores it against the site COBOL and DB2 standards,
and produces a Code Review Report.
```

```
Dependency direction is one-way: this package reads from the converter;
the converter never imports from here.
"""
```

```
from code_review.engine import (
    CheckResult, Outcome, ReviewContext, ReviewResult, Reviewer,
    blockers, is_accepted,
)
```

```
__all__ = [
    "ReviewContext", "Reviewer", "ReviewResult", "CheckResult",
    "Outcome", "is_accepted", "blockers",
]
```

```
=====
FILE: code_review\checks\__init__.py
ENCODING: utf-8
=====
```

```
[Empty file]
```

```
=====
FILE: code_review\checks\chk_01_line_layout.py
ENCODING: utf-8
=====
```

```
"""CHK-01 Fixed format 80-column layout."""
```

```
from __future__ import annotations
```

```
from code_review.engine.check_base import CRITICAL, Check
from code_review.standards import cobol_standards as std
```

```
class LineLayoutCheck(Check):
    CHECK_ID = "CHK-01"
    TITLE = "Fixed format 80-column layout"
    SEVERITY = CRITICAL
    ORDER = 10

    def review(self, ctx, view, record):
        lines = view.populated

        record.expect_none(
            "01", f"Every line is exactly {std.LINE_WIDTH} characters",
            [l for l in lines if len(l.raw) != std.LINE_WIDTH],
        )
        record.expect_none(
            "02", "Columns 1-6 hold a numeric sequence number",
            [l for l in lines if not l.raw[0:6].isdigit()],
        )
        record.expect_none(
            "03", "Column 7 holds a valid indicator",
            [l for l in lines if l.indicator not in std.VALID_INDICATORS],
        )
        record.expect_none(
            "04", f"Body stays within columns {std.BODY_FIRST_COLUMN}-{std.BODY_LAST_COLUMN}",
            [l for l in lines if l.overflow.strip()],
        )
        record.expect_none(
            "05", "Columns 73-80 hold a numeric sequence number",
            [l for l in lines if not l.raw[72:80].isdigit()],
        )
        record.expect_none(
```



```

        "06", "No sequence digits left inside the body",
        [l for l in lines if l.has_sequence_in_body],
    )

    left, right = view.sequence(), view.sequence(right=True)
    record.expect(
        "07", "Left sequence ascends by a constant step",
        view.constant_step(left) is not None,
        note=f"first values {left[:6]}",
    )
    record.expect(
        "08", "Right sequence ascends by a constant step",
        view.constant_step(right) is not None,
        note=f"first values {right[:6]}",
    )

    if std.ENFORCE_SEQUENCE_START:
        record.expect(
            "09", f"Left sequence starts at {std.LEFT_SEQUENCE_START}",
            bool(left) and left[0] == std.LEFT_SEQUENCE_START,
            note=f"found {left[0] if left else 'none'}",
        )
    else:
        record.skip("09", "Sequence start value", "Not enforced by the site standard.")

```

```

=====
FILE: code_review\checks\chk_02_db2_infrastructure.py
ENCODING: utf-8
=====

```

```

# LOCATION: code_review/checks/chk_02_db2_infrastructure.py
# ACTION: REPLACE ENTIRE FILE

```

```

"""CHK-02 DB2 infrastructure declarations."""

```

```

from __future__ import annotations

```

```

import re

```

```

from code_review.engine.check_base import CRITICAL, Check
from code_review.standards import cobol_standards as std

```

```

WS = re.compile(r"^WORKING-STORAGE\s+SECTION\s*\.?.$")
PD = re.compile(r"^PROCEDURE\s+DIVISION\b")
SQL_LOC = re.compile(r"^(?:01|05|77)\s+SQL-LOCATION\s+(?P<pic>PIC\s+.*)$")

```

```

WS_RUN = re.compile(r"\s+")
EXEC_SQL_START = "EXEC SQL"
EXEC_SQL_END = "END-EXEC"

```

```

DEFAULT_SOURCE_COPYBOOK = "SQLERRWS"

```

```

def norm(text: str) -> str:
    """Uppercase, trimmed, with every internal whitespace run collapsed."""
    return WS_RUN.sub(" ", str(text or "").strip()).upper()

```

```

def statements(lines: list[str]) -> list[str]:
    """Logical statements, with EXEC SQL ... END-EXEC. joined into one entry.

```

```

    Ordering is preserved, so positional criteria stay valid. A non-SQL line
    maps one-to-one onto a single statement.
    """

```

```

    out: list[str] = []
    buffer: list[str] = []
    inside = False

```

```

    for raw in lines:
        line = norm(raw)

```

```

        if not inside and line.startswith(EXEC_SQL_START):
            inside = True
            buffer = [line]
            if EXEC_SQL_END in line:

```

```

        out.append(" ".join(buffer))
        inside, buffer = False, []
        continue

    if inside:
        buffer.append(line)
        if EXEC_SQL_END in line:
            out.append(" ".join(buffer))
            inside, buffer = False, []
            continue

    out.append(line)

if buffer:
    out.append(" ".join(buffer))

return out

class Db2InfrastructureCheck(Check):
    CHECK_ID = "CHK-02"
    TITLE = "DB2 infrastructure declarations"
    SEVERITY = CRITICAL
    ORDER = 20

    def review(self, ctx, view, record):
        # ---- 01 required tokens -----
        record.expect_all(
            "01", "Required DB2 tokens are present",
            [t for t in std.REQUIRED_TOKENS if not view.has_code_token(t)],
        )

        code = statements([l.logical for l in view.code])

        # ---- 02 each required include exactly once -----
        wrong = [
            (t, code.count(norm(t)))
            for t in std.REQUIRED_LINES
            if code.count(norm(t)) != 1
        ]
        record.expect_all(
            "02", "Each required include is declared exactly once",
            [f"{t} (found {n})" for t, n in wrong],
        )

        # ---- 03 SQL-LOCATION ownership -----
        self._review_sql_location(code, record)

        # ---- 04 / 05 section markers -----
        ws_at = next((i for i, line in enumerate(code) if WS.match(line)), -1)
        pd_at = next((i for i, line in enumerate(code) if PD.match(line)), -1)

        record.expect("04", "WORKING-STORAGE SECTION is present", ws_at >= 0)
        record.expect("05", "PROCEDURE DIVISION is present", pd_at >= 0)

        # ---- 06 infrastructure placement -----
        if ws_at >= 0 and pd_at >= 0:
            late = []
            for required in std.REQUIRED_LINES:
                key = norm(required)
                at = code.index(key) if key in code else -1
                if at >= 0 and not (ws_at < at < pd_at):
                    late.append(required)

            record.expect_all(
                "06", "Infrastructure sits inside WORKING-STORAGE", late,
            )
        else:
            record.skip(
                "06", "Infrastructure placement", "Section markers not found.",
            )

        # ---- 07 marker uniqueness -----
        record.expect_none(
            "07", "No generated block marker is duplicated",
            [

```

```

        f"{m} x{view.count_of(m)}"
        for m in std.UNIQUE_MARKERS
        if view.count_of(m) > 1
    ],
)

# ---- 08 no TODO or warning marker -----
record.expect_none(
    "08", "No TODO or warning marker remains",
    [m for m in std.FORBIDDEN_MARKERS if view.has_token(m)],
)

# =====
# Criterion 03
# =====
def _review_sql_location(self, code: list[str], record) -> None:
    """SQL-LOCATION is declared, or deliberately supplied elsewhere.

    Two site standards are possible and both are legitimate:

    ENFORCE_SQL_LOCATION_DECLARATION = True
        the program declares 01 SQL-LOCATION itself.

    ENFORCE_SQL_LOCATION_DECLARATION = False
        SQLERRWS supplies it. A local declaration is then a
        DUPLICATE DATA-NAME and the compiler rejects the program,
        so a declaration found here is a FAILURE, not a pass.

    The flag is therefore tested BEFORE the field is looked for.
    Testing presence first made the second standard unreachable: the
    absent field failed the check before the flag was ever consulted.
    """
    title = f"{std.SQL_LOCATION_FIELD} is declared"

    pics = [
        m.group("pic").strip()
        for m in (SQL_LOC.match(line) for line in code)
        if m
    ]

    enforce = getattr(std, "ENFORCE_SQL_LOCATION_DECLARATION", True)

    if enforce:
        if pics:
            record.ok("03", title)
        else:
            record.fail("03", title, note="Field not found.")
        return

    copybook = getattr(
        std, "SQL_LOCATION_SOURCE_COPYBOOK", DEFAULT_SOURCE_COPYBOOK
    )

    if pics:
        record.fail(
            "03",
            f"{std.SQL_LOCATION_FIELD} is not declared locally",
            note=(
                f"{copybook} already declares {std.SQL_LOCATION_FIELD}. "
                f"A second declaration is a duplicate data-name. "
                f"Found: {' ', ' '.join(pics)}"
            ),
        )
        return

    record.skip(
        "03",
        title,
        f"{std.SQL_LOCATION_FIELD} is supplied by {copybook}, "
        f"not declared in the program.",
    )

```

```
=====
FILE: code_review\checks\chk_03_residual_idms.py
ENCODING: utf-8
=====
```

```
"""CHK-03 No residual IDMS statements."""
```

```
from __future__ import annotations
```

```
from code_review.engine.check_base import CRITICAL, Check
from code_review.standards import cobol_standards as std
```

```
class ResidualIdmsCheck(Check):
    CHECK_ID = "CHK-03"
    TITLE = "No residual IDMS statements"
    SEVERITY = CRITICAL
    ORDER = 30

    def review(self, ctx, view, record):
        record.expect_none(
            "01", "No IDMS executable verbs remain",
            [l for l in view.code
             if l.logical.startswith(std.IDMS_EXECUTABLE_PREFIXES)],
        )
        record.expect_none(
            "02", "No IDMS declarative statements remain",
            [l for l in view.code
             if l.logical.startswith(std.IDMS_DECLARATIVE_PREFIXES)],
        )
        record.expect_none(
            "03", "No IDMS status or abort references remain",
            [l for l in view.code
             if any(t in l.logical for t in std.IDMS_TOKENS)],
        )
        record.expect_none(
            "04", "No USAGE-MODE clauses remain",
            [l for l in view.code
             if any(c in l.logical for c in std.IDMS_CLAUSES)],
        )
```

```
=====
FILE: code_review\checks\chk_04_program_header.py
ENCODING: utf-8
=====
```

```
"""CHK-04 Compiler option and PROGRAM-ID."""
```

```
from __future__ import annotations
```

```
import re
```

```
from code_review.engine.check_base import CRITICAL, Check
from code_review.standards import cobol_standards as std
```

```
CBL = re.compile(r"^\s*\b.*$")
PROG = re.compile(r"^\s*PROGRAM-ID\s*(?P<name>[A-Z0-9-]+\s*\.\s*$)")
ENDP = re.compile(r"^\s*END\s+PROGRAM\s+(?P<name>[A-Z0-9-]+\s*\.\s*$)")

# The level number is CAPTURED, not hardcoded, so CS_PROGRAM_LEVELS governs
# it and decision D-2 can be settled from standards/ alone. The manual
# reference declares CS-PROGRAM as a 10-level inside 01 WS-TE-WORK.
#
# NOTE: the previous pattern read "PIC\s+X$8$", where $ is an end-of-string
# anchor, not a literal parenthesis. It could never match, which is why
# criterion 06 always skipped with "CS-PROGRAM not declared".
CSPG = re.compile(
    r"^(?P<level>\d{2})\s+CS-PROGRAM\s+"
    r"PIC\s+X\s*$\s*8\s*$\s+"
    r"VALUE\s+['\"]\s*(?P<v>[A-Z0-9-]*?)\s*['\"]\s*$",
    flags=re.IGNORECASE,
)

SITE = re.compile(r"^(VM)([A-Z0-9]?)(BD)([A-Z0-9]+)$")
```

```

def derive(name: str) -> str:
    text = str(name or "").strip().upper()
    m = SITE.match(text)
    return f"{std.NAME_PREFIX}{std.NAME_TARGET_MARKER}{m.group(2)}{m.group(4)}" if m else text

class ProgramHeaderCheck(Check):
    CHECK_ID = "CHK-04"
    TITLE = "Compiler option and PROGRAM-ID"
    SEVERITY = CRITICAL
    ORDER = 40

    def review(self, ctx, view, record):
        options = [l for l in view.code if CBL.match(l.logical)]
        record.expect(
            "01", f"Compiler option is {std.COMPILER_OPTION_LINE}",
            bool(options) and options[0].logical == std.COMPILER_OPTION_LINE.upper(),
            note=f"found '{options[0].logical}'" if options else "no CBL line found",
            findings=options[:1],
        )
        record.expect("02", "Exactly one compiler option line", len(options) == 1,
            note=f"found {len(options)}")

        actual = view.first_group(PROG, "name")
        record.expect("03", "PROGRAM-ID is present", bool(actual))

        expected = ctx.target_program_id.upper() or (
            derive(ProgramHeaderCheck._source_id(ctx)) if ctx.source_cobol else ""
        )
        if expected:
            record.expect("04", f"PROGRAM-ID is {expected}", actual == expected,
                note=f"found '{actual}'")
        else:
            record.skip("04", "PROGRAM-ID matches derived name",
                "No source program supplied for comparison.")

        # ---- 05 END PROGRAM
        #
        # The manual reference program emits no END PROGRAM statement, so this
        # is gated rather than always scored. See ENFORCE_END_PROGRAM.
        if not getattr(std, "ENFORCE_END_PROGRAM", True):
            record.skip("05", "END PROGRAM matches PROGRAM-ID",
                "END PROGRAM is not part of the site standard.")
        else:
            end_name = view.first_group(ENDP, "name")
            if end_name:
                record.expect("05", "END PROGRAM matches PROGRAM-ID",
                    end_name == actual,
                    note=f"END PROGRAM '{end_name}' vs '{actual}'")
            else:
                record.skip("05", "END PROGRAM matches PROGRAM-ID",
                    "No END PROGRAM statement.")

        # ---- 06 CS-PROGRAM holds the PROGRAM-ID
        levels = getattr(std, "CS_PROGRAM_LEVELS", ("01", "77"))
        field = getattr(std, "CS_PROGRAM_FIELD", "CS-PROGRAM")

        matches = [
            m
            for m in (CSPG.match(l.logical) for l in view.code)
            if m and m.group("level") in levels
        ]

        if not matches:
            record.skip(
                "06", f"{field} holds the PROGRAM-ID",
                f"{field} not declared at level {'', '.join(levels)}.",
            )
        else:
            values = [m.group("v").strip().upper() for m in matches]
            wanted = str(actual or "").strip().upper()
            record.expect(
                "06", f"{field} holds the PROGRAM-ID",
                all(v == wanted for v in values),
                note=f"PROGRAM-ID '{actual}', {field} {values}",
            )

```

```

    )

    @staticmethod
    def _source_id(ctx) -> str:
        for raw in str(ctx.source_cobol or "").splitlines():
            body = raw[7:72] if len(raw) >= 80 else raw
            m = PROG.match(body.strip().upper())
            if m:
                return m.group("name")
        return ""

=====
FILE: code_review\checks\chk_05_sql_error_routing.py
ENCODING: utf-8
=====

# LOCATION: code_review/checks/chk_05_sql_error_routing.py
# ACTION: REPLACE ENTIRE FILE

"""CHK-05 SQL error paragraph and routing."""

from __future__ import annotations

import re

from code_review.engine.check_base import CRITICAL, Check
from code_review.standards import cobol_standards as std

PERFORM_SQL_ERROR = re.compile(r"\bPERFORM\s+(?P<name>SQLERROR|SQL-ERROR)\b")
PARA_HEADER = re.compile(r"^(?P<name>[A-Z0-9][A-Z0-9-]*)\.$")

# The manual reference moves the bare paragraph number:
#     MOVE 710     TO SQL-LOCATION
#
# The quoted-name form is accepted too, as is a data-name source such as
#     MOVE CS-PARAGRAPH TO SQL-LOCATION
#
# The terminator is optional because the site standard closes a paragraph
# with a lone period on its own line.
MOVE_SQL_LOCATION = re.compile(
    r"^(MOVE\s+(?:'[^']*'|\d+|[A-Z][A-Z0-9-]*)\s+TO\s+SQL-LOCATION\s*\.$)"
)

EXEC_SQL = re.compile(r"^(EXEC\s+SQL\b)")

class SqlErrorRoutingCheck(Check):
    CHECK_ID = "CHK-05"
    TITLE = "SQL error paragraph and routing"
    SEVERITY = CRITICAL
    ORDER = 50

    def relevant(self, ctx, view) -> bool:
        return view.has_code_token("EXEC SQL")

    def not_relevant_reason(self) -> str:
        return "Program contains no embedded SQL."

    def review(self, ctx, view, record):
        standard = std.SQL_ERROR_PARAGRAPH
        legacy = std.LEGACY_SQL_ERROR_PARAGRAPH

        headers = [
            m.group("name")
            for m in (PARA_HEADER.match(l.logical) for l in view.code)
            if m
        ]
        declared = [h for h in headers if h in (standard, legacy)]

        # The site standard routes through EXEC SQL INCLUDE SQLERROR, which
        # supplies the routine from the copybook. The manual reference
        # therefore declares no SQLERROR paragraph of its own, and writing
        # one risks a duplicate paragraph name.
        include_token = getattr(std, "SQL_ERROR_INCLUDE", "")
        included = bool(include_token) and view.has_code_token(include_token)

```

```

# ---- 01 declared or included -----
record.expect(
    "01", f"{standard} is declared or included",
    (standard in declared) or included,
    note=(
        f"paragraphs found: {declared or 'none'}; "
        f"include present: {included}"
    ),
)

# ---- 02 declared exactly once -----
if standard not in declared and included:
    record.skip(
        "02", f"Exactly one {standard} paragraph is declared",
        f"{standard} is supplied by {include_token}, not declared "
        f"in the program.",
    )
else:
    record.expect(
        "02", f"Exactly one {standard} paragraph is declared",
        declared.count(standard) == 1,
        note=f"found {declared.count(standard)}",
    )

# ---- 03 legacy name gone -----
record.expect_none(
    "03", f"Legacy {legacy} paragraph name is not used",
    view.exact(f"{legacy}."),
)

# ---- 04 / 05 PERFORM routing -----
performs = view.searching(PERFORM_SQL_ERROR)

record.expect_none(
    "04", f"Every PERFORM targets {standard}",
    [
        line for line in performs
        if self._target_of(line)
        and self._target_of(line) != standard
    ],
)

# A paragraph supplied by the copybook is not visible as a header in
# this program, so it cannot be counted as undeclared.
declared_set = set(headers)
if included:
    declared_set.add(standard)

record.expect_none(
    "05", "No PERFORM refers to an undeclared paragraph",
    [
        line for line in performs
        if self._target_of(line)
        and self._target_of(line) not in declared_set
    ],
)

# ---- 06 reachability -----
if not performs:
    record.skip(
        "06", f"{standard} is reachable",
        f"No PERFORM {standard} statement in the program.",
    )
else:
    record.ok("06", f"{standard} is reachable")

# ---- 07 SQL-LOCATION populated -----
record.expect(
    "07", "SQL-LOCATION is populated before SQL blocks",
    bool(view.matching(MOVE_SQL_LOCATION)),
    note="No MOVE to SQL-LOCATION found.",
)

# ---- 08 body matches the site standard -----
self._review_body(view, record, standard)

```

```

# =====
# Criterion 08
# =====
@staticmethod
def _review_body(view, record, standard: str) -> None:
    """The SQLERROR routine carries the site's agreed body.

    SQL_ERROR_BODY is the authority. Decision D-1 closed it in favour
    of the include form:

        EXEC SQL
            INCLUDE SQLERROR
        END-EXEC.

    A program that instead emits the older DISPLAY / CALL USERABEN
    body fails here. That is correct: two SQLERROR bodies in one
    codebase means retrieval and update output diverge, and the
    copybook routine is duplicated by a hand-rolled one.
    """
    title = f"{standard} body matches the site standard"
    body = [token.upper() for token in getattr(std, "SQL_ERROR_BODY", ())]

    if not body:
        record.skip(
            "08", title,
            "Site standard body not yet agreed (decision open).",
        )
        return

    record.expect_all(
        "08", title,
        [token for token in body if token not in view.code_text],
    )

# =====
# Helpers
# =====
@staticmethod
def _target_of(line) -> str:
    """Performed paragraph name, or '' when the line does not match.

    view.searching and PERFORM_SQL_ERROR.search can disagree if one
    inspects the raw line and the other the logical line. Calling
    .group() on None would abort the entire review over a formatting
    edge case, so a non-match is treated as 'nothing to judge'.
    """
    match = PERFORM_SQL_ERROR.search(line.logical)
    return match.group("name") if match else ""

```

```

=====
FILE: code_review\checks\chk_06_area_alignment.py
ENCODING: utf-8
=====

```

```

"""CHK-06 Area A and Area B alignment."""

```

```

from __future__ import annotations

```

```

import re

```

```

from code_review.engine.check_base import MINOR, Check
from code_review.standards import cobol_standards as std

```

```

DIVISION = re.compile(r"^(?P<name>[A-Z]+)\s+DIVISION\b")
SECTION = re.compile(r"^(?P<name>[A-Z0-9][A-Z0-9-]*)\s+SECTION\s*\.\?$")
PARAGRAPH = re.compile(r"^(?P<name>[A-Z0-9][A-Z0-9-]*)\s*\.$")
LEVEL = re.compile(r"^(?P<level>\d{2})\s+(?P<name>[A-Z0-9][A-Z0-9-]*)")
PROCEDURE_DIVISION = re.compile(r"^(?P<name>[A-Z0-9][A-Z0-9-]*)\s+PROCEDURE\b")

```

```

SQL_START = re.compile(rf"^{re.escape(std.SQL_BLOCK_START)}\b")
SQL_END = re.compile(rf"^{re.escape(std.SQL_BLOCK_END)}\b")
STATUS_START = re.compile(rf"^{re.escape(std.SQL_STATUS_BLOCK_START)}\b")
STATUS_END = re.compile(rf"^{re.escape(std.SQL_STATUS_BLOCK_END)}\b")

```



```

class AreaAlignmentCheck(Check):
    CHECK_ID = "CHK-06"
    TITLE = "Area A and Area B alignment"
    SEVERITY = MINOR
    ORDER = 60

    def relevant(self, ctx, view) -> bool:
        return any(line.is_fixed for line in view.code)

    def not_relevant_reason(self) -> str:
        return "No fixed-format lines to measure indentation against."

    def review(self, ctx, view, record):
        lines = [line for line in view.code if line.is_fixed]
        generated = self._generated_sql_lines(lines)
        generated_numbers = {line.number for line in generated}

        area_a = {
            line.number for line in lines
            if line.indent <= std.AREA_A_MAX_INDENT
        }
        area_b = {
            line.number for line in lines
            if line.indent >= std.AREA_B_MIN_INDENT
        }

        # ---- 01 division headers -----
        divisions = [
            line for line in lines
            if DIVISION.match(line.logical)
            and DIVISION.match(line.logical).group("name") in std.DIVISION_NAMES
        ]
        if not divisions:
            record.skip(
                "01",
                f"Division headers start in Area A "
                f"(cols {std.AREA_A_FIRST_COLUMN}-{std.AREA_A_LAST_COLUMN})",
                "No division header found.",
            )
        else:
            record.expect_none(
                "01",
                f"Division headers start in Area A "
                f"(cols {std.AREA_A_FIRST_COLUMN}-{std.AREA_A_LAST_COLUMN})",
                [l for l in divisions if l.number not in area_a],
            )

        # ---- 02 section headers -----
        sections = [line for line in lines if SECTION.match(line.logical)]
        if not sections:
            record.skip(
                "02", "Section headers start in Area A",
                "No section header found.",
            )
        else:
            record.expect_none(
                "02", "Section headers start in Area A",
                [l for l in sections if l.number not in area_a],
            )

        # ---- 03 paragraph headers -----
        paragraphs = self._paragraph_headers(lines, generated_numbers)
        if not paragraphs:
            record.skip(
                "03", "Paragraph headers start in Area A",
                "No paragraph header found.",
            )
        else:
            record.expect_none(
                "03", "Paragraph headers start in Area A",
                [l for l in paragraphs if l.number not in area_a],
            )

        # ---- 04 level 01 and 77 -----
        top_levels = [

```

```

        line for line in lines
        if LEVEL.match(line.logical)
        and LEVEL.match(line.logical).group("level")
        in std.AREA_A_LEVEL_NUMBERS
    ]
    if not top_levels:
        record.skip(
            "04",
            f"Level {' and '.join(std.AREA_A_LEVEL_NUMBERS)} items "
            f"start in Area A",
            "No level 01 or 77 item found.",
        )
    else:
        record.expect_none(
            "04",
            f"Level {' and '.join(std.AREA_A_LEVEL_NUMBERS)} items "
            f"start in Area A",
            [l for l in top_levels if l.number not in area_a],
        )

# ---- 05 subordinate levels -----
sub_levels = [
    line for line in lines
    if LEVEL.match(line.logical)
    and LEVEL.match(line.logical).group("level")
    in std.AREA_B_LEVEL_NUMBERS
]
if not sub_levels:
    record.skip(
        "05",
        f"Subordinate level items start in Area B "
        f"(col {std.AREA_B_FIRST_COLUMN} onward)",
        "No subordinate level item found.",
    )
else:
    record.expect_none(
        "05",
        f"Subordinate level items start in Area B "
        f"(col {std.AREA_B_FIRST_COLUMN} onward)",
        [l for l in sub_levels if l.number not in area_b],
    )

# ---- 06 procedure statements -----
#
# Generated DB2 block lines are excluded here and measured by
# criterion 07 instead, so decision D-8 has exactly one gate.
if not std.ENFORCE_AREA_B_STATEMENT_INDENT:
    record.skip(
        "06", "Procedure statements start in Area B",
        "Not enforced by the site standard.",
    )
else:
    statements = self._procedure_statements(lines, generated_numbers)
    if not statements:
        record.skip(
            "06", "Procedure statements start in Area B",
            "No hand-written PROCEDURE DIVISION statement found.",
        )
    else:
        record.expect_none(
            "06", "Procedure statements start in Area B",
            [l for l in statements if l.number not in area_b],
        )

# ---- 07 generated DB2 blocks -----
if not std.ENFORCE_AREA_B_SQL_INDENT:
    record.skip(
        "07", "Generated DB2 blocks sit in Area B",
        "Generated DB2 block indentation not yet agreed "
        "(decision open).",
    )
elif not generated:
    record.skip(
        "07", "Generated DB2 blocks sit in Area B",
        "Program contains no generated DB2 block.",
    )

```

```

else:
    record.expect_none(
        "07", "Generated DB2 blocks sit in Area B",
        [
            1 for 1 in generated
            if 1.indent < std.AREA_B_SQL_MIN_INDENT
        ],
    )

# ---- 08 continuation lines -----
continuations = [
    line for line in view.lines
    if line.is_fixed
    and line.indicator == std.CONTINUATION_INDICATOR
]
if not continuations:
    record.skip(
        "08", "Continuation lines sit in Area B",
        "No continuation line found.",
    )
else:
    record.expect_none(
        "08", "Continuation lines sit in Area B",
        [1 for 1 in continuations if 1.indent < std.AREA_B_MIN_INDENT],
    )

# ---- helpers -----
@staticmethod
def _paragraph_headers(lines, generated: set[int]) -> list:
    """Real paragraph headers only.

    A bare word followed by a period is a paragraph header only when it
    is not a COBOL verb or scope terminator, is not a section header,
    and is not inside a generated DB2 block.
    """
    out = []
    for line in lines:
        if line.number in generated:
            continue
        if SECTION.match(line.logical):
            continue
        match = PARAGRAPH.match(line.logical)
        if not match:
            continue
        if match.group("name") in std.NON_PARAGRAPH_WORDS:
            continue
        out.append(line)
    return out

@staticmethod
def _procedure_statements(lines, generated: set[int]) -> list:
    """Hand-written executable lines after PROCEDURE DIVISION."""
    out = []
    started = False
    for line in lines:
        if PROCEDURE_DIVISION.match(line.logical):
            started = True
            continue
        if not started:
            continue
        if line.number in generated:
            continue
        if DIVISION.match(line.logical) or SECTION.match(line.logical):
            continue
        match = PARAGRAPH.match(line.logical)
        if match and match.group("name") not in std.NON_PARAGRAPH_WORDS:
            continue
        out.append(line)
    return out

@staticmethod
def _generated_sql_lines(lines) -> list:
    """Every line of a generated DB2 block, terminators included.

    Covers two block shapes emitted by the converter:

```

```

EXEC SQL ... END-EXEC.
EVALUATE SQLCODE ... END-EVALUATE.
"""
out = []
in_exec = False
in_status = False

for line in lines:
    logical = line.logical

    if not in_exec and not in_status:
        if SQL_START.match(logical):
            in_exec = True
            out.append(line)
            if SQL_END.search(logical):
                in_exec = False
            continue
        if STATUS_START.match(logical):
            in_status = True
            out.append(line)
            if STATUS_END.search(logical):
                in_status = False
            continue
        continue

    out.append(line)
    if in_exec and SQL_END.search(logical):
        in_exec = False
    elif in_status and STATUS_END.search(logical):
        in_status = False

return out

```

```

=====
FILE: code_review\checks\chk_07_sequence_continuity.py
ENCODING: utf-8
=====

```

```

"""CHK-07 Sequence numbering continuity."""

```

```

from __future__ import annotations

```

```

from collections import Counter

```

```

from code_review.engine.check_base import MAJOR, Check
from code_review.standards import cobol_standards as std

```

```

class SequenceContinuityCheck(Check):

```

```

    CHECK_ID = "CHK-07"
    TITLE = "Sequence numbering continuity"
    SEVERITY = MAJOR
    ORDER = 70

```

```

    def relevant(self, ctx, view) -> bool:
        return any(line.is_fixed for line in view.lines)

```

```

    def not_relevant_reason(self) -> str:
        return "Program has no fixed-format sequenced lines."

```

```

    def review(self, ctx, view, record):
        left = view.sequence()
        right = view.sequence(right=True)
        fixed = [line for line in view.lines if line.is_fixed]

```

```

        # ---- 01 every populated line carries both sequences -----
        record.expect_none(
            "01", "Every populated line carries both sequence numbers",
            [line for line in view.populated if not line.is_fixed],
        )

```

```

        # ---- 02 / 03 left sequence ascends and steps -----
        self._ascending(record, "02", "Left sequence ascends", left)
        self._step(
            record, "03", "Left sequence",

```

```

        left, std.LEFT_SEQUENCE_STEP,
    )

# ---- 04 / 05 right sequence ascends and steps -----
self._ascending(record, "04", "Right sequence ascends", right)
self._step(
    record, "05", "Right sequence",
    right, std.RIGHT_SEQUENCE_STEP,
)

# ---- 06 / 07 start values -----
if not std.ENFORCE_SEQUENCE_START:
    record.skip(
        "06", "Left sequence start value",
        "Not enforced by the site standard.",
    )
    record.skip(
        "07", "Right sequence start value",
        "Not enforced by the site standard.",
    )
else:
    record.expect(
        "06", f"Left sequence starts at {std.LEFT_SEQUENCE_START}",
        bool(left) and left[0] == std.LEFT_SEQUENCE_START,
        note=f"found {left[0]} if left else 'none'",
    )
    record.expect(
        "07", f"Right sequence starts at {std.RIGHT_SEQUENCE_START}",
        bool(right) and right[0] == std.RIGHT_SEQUENCE_START,
        note=f"found {right[0]} if right else 'none'",
    )

# ---- 08 / 09 duplicates -----
self._unique(record, "08", "Left sequence", left)
self._unique(record, "09", "Right sequence", right)

# ---- 10 counts agree -----
record.expect(
    "10", "Left and right sequences cover the same lines",
    len(left) == len(right) == len(fixed),
    note=f"left {len(left)}, right {len(right)}, fixed {len(fixed)}",
)

# ---- helpers -----
@staticmethod
def _ascending(record, num: str, description: str, values: list[int]) -> None:
    if len(values) < 2:
        record.skip(num, description, "Fewer than two sequenced lines.")
        return
    breaks = [
        f"{a} then {b}"
        for a, b in zip(values, values[1:])
        if b <= a
    ]
    record.expect_all(
        num, description, breaks,
        note=(
            "not ascending at: " + ", ".join(breaks[:10])
            if breaks else ""
        ),
    )

@staticmethod
def _step(
    record,
    num: str,
    label: str,
    values: list[int],
    expected: int,
) -> None:
    description = f"{label} step is {expected}"

    if not std.ENFORCE_SEQUENCE_STEP:
        record.skip(num, description, "Step not enforced by the standard.")
        return
    if len(values) < 2:

```

```

        record.skip(num, description, "Fewer than two sequenced lines.")
        return

    wrong = [
        f"{a} -> {b} (step {b - a})"
        for a, b in zip(values, values[1:])
        if (b - a) != expected
    ]
    if wrong and std.ALLOW_SEQUENCE_GAPS:
        wrong = [
            item for item in wrong
            if item.endswith("(step 0)") or "-" in item.split("step ")[-1]
        ]
    record.expect_all(
        num, description, wrong,
        note=(
            f"{len(wrong)} irregular step(s): " + ", ".join(wrong[:10])
            if wrong else ""
        ),
    )
)

```

@staticmethod

```

def _unique(record, num: str, label: str, values: list[int]) -> None:
    description = f"{label} numbers are unique"
    if not values:
        record.skip(num, description, "No sequenced lines.")
        return
    duplicates = [
        f"{value} x{count}"
        for value, count in sorted(Counter(values).items())
        if count > 1
    ]
    record.expect_all(
        num, description, duplicates,
        note=(
            f"{len(duplicates)} duplicated value(s)"
            if duplicates else ""
        ),
    )
)

```

```

=====
FILE: code_review\checks\chk_08_dclgen_include_integrity.py
ENCODING: utf-8
=====

```

"""CHK-08 DCLGEN include and copybook integrity."""

from __future__ import annotations

import re

from collections import Counter

from code_review.engine.check_base import CRITICAL, Check

from code_review.standards import cobol_standards as std

```

INCLUDE = re.compile(
    r"^\EXEC\s+SQL\s+INCLUDE\s+(?P<name>[A-Z0-9][A-Z0-9_-]*)\s+END-EXEC\s*\.\? $"
)
COPY = re.compile(r"^\COPY\s+(?P<name>[A-Z0-9][A-Z0-9_-]*)\b")
WS_SECTION = re.compile(r"^\WORKING-STORAGE\s+SECTION\s*\.\? $"
PROCEDURE_DIVISION = re.compile(r"^\PROCEDURE\s+DIVISION\b")
WS_RUN = re.compile(r"\s+")

```

def norm(text: str) -> str:

"""Uppercase, trimmed, internal whitespace runs collapsed."""

```

    return WS_RUN.sub(" ", str(text or "").strip().upper())

```

def statements(lines: list[str]) -> list[str]:

"""Logical statements, with EXEC SQL ... END-EXEC. joined into one entry.

Order is preserved, so positional criteria remain valid.

"""

```

out: list[str] = []

```

```

buffer: list[str] = []
inside = False

for raw in lines:
    line = norm(raw)

    if not inside and line.startswith(std.SQL_BLOCK_START):
        inside = True
        buffer = [line]
        if std.SQL_BLOCK_END in line:
            out.append(" ".join(buffer))
            inside, buffer = False, []
            continue

    if inside:
        buffer.append(line)
        if std.SQL_BLOCK_END in line:
            out.append(" ".join(buffer))
            inside, buffer = False, []
            continue

    out.append(line)

if buffer:
    out.append(" ".join(buffer))
return out

def is_infrastructure(name: str) -> bool:
    """SQLCA, SQLERRWS, SQLERROR and GEN are not DCLGEN table includes."""
    return str(name or "").strip().upper() in std.EXCLUDED_INCLUDE_NAMES

class DclgenIncludeIntegrityCheck(Check):
    CHECK_ID = "CHK-08"
    TITLE = "DCLGEN include and copybook integrity"
    SEVERITY = CRITICAL
    ORDER = 80

    def relevant(self, ctx, view) -> bool:
        return view.has_code_token(std.SQL_BLOCK_START)

    def not_relevant_reason(self) -> str:
        return "Program contains no embedded SQL."

    def review(self, ctx, view, record):
        code = statements([line.logical for line in view.code])

        included = [
            INCLUDE.match(line).group("name").upper()
            for line in code
            if INCLUDE.match(line)
        ]
        dclgen_includes = [
            name for name in included if not is_infrastructure(name)
        ]
        include_set = set(dclgen_includes)

        groups = self._referenced_groups(view)
        referenced_tables = {self._table_of(group) for group in groups}

        copies = [
            COPY.match(line).group("name").upper()
            for line in code
            if COPY.match(line)
        ]

        # ---- 01 no DCLGEN include is unused -----
        if not dclgen_includes:
            record.skip(
                "01", "Every DCLGEN include is actually used",
                "Program declares no DCLGEN include.",
            )
        else:
            record.expect_all(
                "01", "Every DCLGEN include is actually used",

```

```

        sorted(
            name for name in include_set
            if name not in referenced_tables
        ),
    ),
)

# ---- 02 each include declared exactly once -----
duplicates = [
    f"{name} x{count}"
    for name, count in sorted(Counter(included).items())
    if count > 1
]
record.expect_all(
    "02", "Each INCLUDE is declared exactly once", duplicates,
    note=f"{len(duplicates)} duplicated include(s)" if duplicates else "",
)

# ---- 03 DCLGEN includes sit inside WORKING-STORAGE -----
#
# Only DCLGEN table includes are placement-checked. Infrastructure
# includes are exempt on purpose: the converter emits
# "EXEC SQL INCLUDE SQLERROR END-EXEC." inside the PROCEDURE
# DIVISION SQLERROR paragraph, which is correct.
if not std.ENFORCE_INCLUDE_IN_WORKING_STORAGE:
    record.skip(
        "03", "DCLGEN INCLUDE statements sit inside WORKING-STORAGE",
        "Placement not enforced by the site standard.",
    )
elif not dclgen_includes:
    record.skip(
        "03", "DCLGEN INCLUDE statements sit inside WORKING-STORAGE",
        "Program declares no DCLGEN include.",
    )
else:
    ws_at = next(
        (i for i, line in enumerate(code) if WS_SECTION.match(line)),
        -1,
    )
    pd_at = next(
        (
            i for i, line in enumerate(code)
            if PROCEDURE_DIVISION.match(line)
        ),
        -1,
    )
    if ws_at < 0 or pd_at < 0:
        record.skip(
            "03",
            "DCLGEN INCLUDE statements sit inside WORKING-STORAGE",
            "Section markers not found.",
        )
    else:
        late = []
        for index, line in enumerate(code):
            match = INCLUDE.match(line)
            if not match:
                continue
            if is_infrastructure(match.group("name")):
                continue
            if not (ws_at < index < pd_at):
                late.append(line)
        record.expect_all(
            "03",
            "DCLGEN INCLUDE statements sit inside WORKING-STORAGE",
            late,
        )

# ---- 04 every referenced DCL group has an include -----
if not groups:
    record.skip(
        "04",
        f"Every referenced {std.DCLGEN_GROUP_PREFIX} group has an "
        f"INCLUDE",
        "No DCLGEN host group referenced.",
    )
else:

```



```

        record.expect_all(
            "04",
            f"Every referenced {std.DCLGEN_GROUP_PREFIX} group has an "
            f"INCLUDE",
            sorted(
                group for group in groups
                if self._table_of(group) not in include_set
            ),
        )
    )

# ---- 05 include names are real DCLGEN tables -----
tables = self._dclgen_tables(ctx)
if not tables:
    record.skip(
        "05", "Every INCLUDE names a known DCLGEN table",
        "No DCLGEN metadata supplied for this run.",
    )
elif not dclgen_includes:
    record.skip(
        "05", "Every INCLUDE names a known DCLGEN table",
        "Program declares no DCLGEN include.",
    )
else:
    record.expect_all(
        "05", "Every INCLUDE names a known DCLGEN table",
        sorted(name for name in include_set if name not in tables),
    )

# ---- 06 host variables belong to an included table -----
hosts = self._dclgen_hosts(ctx)
if not hosts:
    record.skip(
        "06", "Every DCLGEN host variable used has its table included",
        "No DCLGEN metadata supplied for this run.",
    )
else:
    used_tables = {
        table for host, table in hosts.items()
        if view.has_code_token(host)
    }
    record.expect_all(
        "06", "Every DCLGEN host variable used has its table included",
        sorted(
            table for table in used_tables
            if table not in include_set
        ),
    )

# ---- 07 no forbidden COPY remains -----
record.expect_all(
    "07", "No forbidden COPY member remains",
    [
        name for name in copies
        if name.startswith(std.FORBIDDEN_COPY_PREFIXES)
    ],
)

# ---- 08 no COPY member is declared twice -----
if not copies:
    record.skip(
        "08", "Each COPY member is declared exactly once",
        "Program declares no COPY statement.",
    )
else:
    repeated = [
        f"{name} x{count}"
        for name, count in sorted(Counter(copies).items())
        if count > 1
    ]
    record.expect_all(
        "08", "Each COPY member is declared exactly once", repeated,
    )

# ---- 09 COPY members resolve to a copybook record -----
#
# The supplied copybook metadata is a flat list of FIELD names and

```

```

# carries no member or include name, so a COPY member can only be
# matched against 01-level record names. That equivalence is not
# yet agreed. See open decision D-9.
records_01 = self._copybook_records(ctx)
if not std.ENFORCE_COPY_MEMBER_RESOLUTION:
    record.skip(
        "09", "Every COPY member resolves to a copybook record",
        "Copybook member names are not part of the supplied "
        "metadata (decision open).",
    )
elif not records_01:
    record.skip(
        "09", "Every COPY member resolves to a copybook record",
        "No copybook 01-level record supplied for this run.",
    )
elif not copies:
    record.skip(
        "09", "Every COPY member resolves to a copybook record",
        "Program declares no COPY statement.",
    )
else:
    record.expect_all(
        "09", "Every COPY member resolves to a copybook record",
        sorted(
            name for name in set(copies)
            if name not in records_01
        ),
    )
)

# ---- helpers -----
@staticmethod
def _table_of(group: str) -> str:
    """DCLDZBEFFTV -> DZBEFFTV."""
    text = str(group or "").strip().upper()
    prefix = std.DCLGEN_GROUP_PREFIX.upper()
    return text[len(prefix):] if text.startswith(prefix) else text

@staticmethod
def _referenced_groups(view) -> set[str]:
    prefix = std.DCLGEN_GROUP_PREFIX.upper()
    pattern = re.compile(rf"\b{prefix}[A-Z0-9_-]+\b")
    return set(pattern.findall(view.code_text))

@staticmethod
def _dclgen_tables(ctx) -> set[str]:
    return {
        str(getattr(column, "table_name", "") or "").strip().upper()
        for column in (ctx.dclgen_columns or [])
        if str(getattr(column, "table_name", "") or "").strip()
    }

@staticmethod
def _dclgen_hosts(ctx) -> dict[str, str]:
    out: dict[str, str] = {}
    for column in ctx.dclgen_columns or []:
        host = str(
            getattr(column, "cobol_host_name", "") or ""
        ).strip().upper()
        table = str(
            getattr(column, "table_name", "") or ""
        ).strip().upper()
        if host and table:
            out[host] = table
    return out

@staticmethod
def _copybook_records(ctx) -> set[str]:
    """01-level record names from the supplied copybook fields."""
    out: set[str] = set()
    for field in ctx.copybook_fields or []:
        level = str(getattr(field, "level", "") or "").strip()
        name = str(getattr(field, "name", "") or "").strip().upper()
        if name and level == std.COPYBOOK_RECORD_LEVEL:
            out.add(name)
    return out

```

```
=====
FILE: code_review\checks\chk_09_record_retrieval_conversion.py
ENCODING: utf-8
=====
```

```
"""CHK-09 Record retrieval conversion."""
```

```
from __future__ import annotations
```

```
from code_review.engine import sql_blocks as sql
from code_review.engine.check_base import MAJOR, Check
from code_review.standards import cobol_standards as std
```

```
class RecordRetrievalConversionCheck(Check):
    CHECK_ID = "CHK-09"
    TITLE = "Record retrieval conversion"
    SEVERITY = MAJOR
    ORDER = 90

    def relevant(self, ctx, view) -> bool:
        return view.has_code_token(std.SQL_BLOCK_START)

    def not_relevant_reason(self) -> str:
        return "Program contains no embedded SQL."

    def review(self, ctx, view, record):
        # Restart SQL is scored by CHK-19, CHK-20 and CHK-21.
        blocks = [
            b for b in sql.sql_blocks(view)
            if not sql.targets_restart(b, std.RESTART_TABLE_NAME_HINTS)
        ]
        selects = [b for b in blocks if b.verb == "SELECT"]
        retrieval = [b for b in blocks if b.verb in std.RETRIEVAL_SQL_VERBS]

        # ---- 01 nothing was skipped for missing metadata -----
        record.expect_none(
            "01", "No business retrieval was skipped for missing metadata",
            [
                line for line in view.comments
                if any(m in line.logical for m in std.MISSING_MAPPING_MARKERS)
                and not any(
                    hint in line.logical
                    for hint in std.RESTART_CONTROL_HINTS
                )
            ],
        )

        # ---- 02 SELECT has FROM -----
        if not selects:
            record.skip(
                "02", "Every SELECT names a source table",
                "Program contains no business singleton SELECT.",
            )
        else:
            record.expect_none(
                "02", "Every SELECT names a source table",
                [b for b in selects if not b.has("FROM")],
            )

        # ---- 03 SELECT has INTO -----
        if not selects:
            record.skip(
                "03", "Every SELECT retrieves into host variables",
                "Program contains no business singleton SELECT.",
            )
        else:
            record.expect_none(
                "03", "Every SELECT retrieves into host variables",
                [b for b in selects if not b.has("INTO")],
            )

        # ---- 04 SELECT has WHERE -----
        if not std.ENFORCE_SELECT_WHERE:
            record.skip(
```

```

        "04", "Every SELECT is key qualified",
        "Not enforced by the site standard.",
    )
elif not selects:
    record.skip(
        "04", "Every SELECT is key qualified",
        "Program contains no business singleton SELECT.",
    )
else:
    record.expect_none(
        "04", "Every SELECT is key qualified",
        [b for b in selects if not b.has("WHERE")],
    )

# ---- 05 SQL-LOCATION set before retrieval -----
if not std.ENFORCE_SQL_LOCATION_BEFORE_SQL:
    record.skip(
        "05",
        f"{std.SQL_LOCATION_FIELD} is set before each retrieval",
        "Not enforced by the site standard.",
    )
elif not retrieval:
    record.skip(
        "05",
        f"{std.SQL_LOCATION_FIELD} is set before each retrieval",
        "Program contains no business retrieval SQL.",
    )
else:
    record.expect_none(
        "05",
        f"{std.SQL_LOCATION_FIELD} is set before each retrieval",
        [
            b for b in retrieval
            if not self._location_set_before(view, b)
        ],
    )

# ---- 06 no empty converted block -----
empty = self._empty_paragraphs(view)
if not empty:
    record.ok("06", "No paragraph was left empty by the conversion")
else:
    record.expect_none(
        "06", "No paragraph was left empty by the conversion", empty,
    )

# ---- helpers -----
@staticmethod
def _location_set_before(view, block) -> bool:
    """True when SQL-LOCATION is set within the look-back window.

    Both move forms are accepted:

        MOVE '710-OPEN-DZBEFFC1' TO SQL-LOCATION
        MOVE 710                TO SQL-LOCATION

    The manual reference uses the bare paragraph number. Matching
    only the quoted form failed every generated cursor paragraph
    while the field was in fact being set correctly.
    """
    prefixes = getattr(
        std,
        "SQL_LOCATION_MOVE_PREFIXES",
        (std.SQL_LOCATION_MOVE_PREFIX, ),
    )
    window = sql.lines_before(view, block, std.SQL_LOCATION_LOOKBACK)

    return any(
        line.startswith(tuple(prefixes))
        and std.SQL_LOCATION_MOVE_SUFFIX in line
        for line in window
    )

@staticmethod
def _empty_paragraphs(view) -> list:
    """Paragraph headers immediately followed by another header."""
    code = view.code

```

```

out: list = []
for index, line in enumerate(code[:-1]):
    logical = line.logical
    if not logical.endswith(".") or " " in logical:
        continue
    name = logical[:-1]
    if not name or name in std.NON_PARAGRAPH_WORDS:
        continue
    nxt = code[index + 1].logical
    if nxt.endswith(".") and " " not in nxt:
        following = nxt[:-1]
        if following and following not in std.NON_PARAGRAPH_WORDS:
            out.append(line)
return out

```

```

=====
FILE: code_review\checks\chk_10_record_modification_conversion.py
ENCODING: utf-8
=====

```

```

"""CHK-10 Record modification conversion."""

```

```

from __future__ import annotations

```

```

from code_review.engine import sql_blocks as sql
from code_review.engine.check_base import CRITICAL, UPDATE, Check
from code_review.standards import cobol_standards as std

```

```

class RecordModificationConversionCheck(Check):
    CHECK_ID = "CHK-10"
    TITLE = "Record modification conversion"
    SEVERITY = CRITICAL
    APPLIES_TO = UPDATE
    ORDER = 100

    def relevant(self, ctx, view) -> bool:
        return view.has_code_token(std.SQL_BLOCK_START)

    def not_relevant_reason(self) -> str:
        return "Program contains no embedded SQL."

    def review(self, ctx, view, record):
        # Restart writes carry their own QUERYNOs and are scored by CHK-20.
        blocks = [
            b for b in sql.sql_blocks(view)
            if not sql.targets_restart(b, std.RESTART_TABLE_NAME_HINTS)
        ]
        writes = [b for b in blocks if b.verb in std.WRITE_SQL_VERBS]
        inserts = [b for b in writes if b.verb == "INSERT"]
        updates = [b for b in writes if b.verb == "UPDATE"]
        deletes = [b for b in writes if b.verb == "DELETE"]

        # ---- 01 no business write was skipped -----
        record.expect_none(
            "01", "No business write was skipped for missing metadata",
            [
                line for line in view.comments
                if any(m in line.logical for m in std.MISSING_MAPPING_MARKERS)
                and not any(
                    hint in line.logical
                    for hint in std.RESTART_CONTROL_HINTS
                )
            ],
        )

        # ---- 02 the program actually writes -----
        if not writes:
            record.skip(
                "02", "Update program issues at least one business write",
                "No business INSERT, UPDATE or DELETE found. Only restart "
                "writes were generated.",
            )
        else:
            record.ok(

```

```

        "02", "Update program issues at least one business write",
    )

# ---- 03 INSERT names a target -----
if not inserts:
    record.skip(
        "03", "Every INSERT names a target table",
        "Program contains no business INSERT.",
    )
else:
    record.expect_none(
        "03", "Every INSERT names a target table",
        [
            b for b in inserts
            if not all(b.has(c) for c in std.INSERT_REQUIRED_CLAUSES)
        ],
    )

# ---- 04 UPDATE has SET and WHERE -----
if not updates:
    record.skip(
        "04", "Every UPDATE has SET and WHERE",
        "Program contains no business UPDATE.",
    )
else:
    record.expect_none(
        "04", "Every UPDATE has SET and WHERE",
        [
            b for b in updates
            if not all(b.has(c) for c in std.UPDATE_REQUIRED_CLAUSES)
        ],
    )

# ---- 05 DELETE is key qualified -----
if not deletes:
    record.skip(
        "05", "Every DELETE is key qualified",
        "Program contains no business DELETE.",
    )
else:
    record.expect_none(
        "05", "Every DELETE is key qualified",
        [
            b for b in deletes
            if not all(b.has(c) for c in std.DELETE_REQUIRED_CLAUSES)
        ],
    )

# ---- 06 QUERYNO present -----
if not std.ENFORCE_QUERYNO:
    record.skip(
        "06", f"Every business write carries QUERYNO "
              f"{std.UPDATE_QUERYNO}",
        "QUERYNO not enforced by the site standard.",
    )
elif not writes:
    record.skip(
        "06", f"Every business write carries QUERYNO "
              f"{std.UPDATE_QUERYNO}",
        "Program contains no business write statement.",
    )
else:
    expected = f"QUERYNO {std.UPDATE_QUERYNO}"
    record.expect_none(
        "06", f"Every business write carries QUERYNO "
              f"{std.UPDATE_QUERYNO}",
        [b for b in writes if expected not in b.text],
    )

# ---- 07 SQL-LOCATION set before each write -----
if not std.ENFORCE_SQL_LOCATION_BEFORE_SQL:
    record.skip(
        "07", f"{std.SQL_LOCATION_FIELD} is set before each "
              f"business write",
        "Not enforced by the site standard.",
    )

```

```

elif not writes:
    record.skip(
        "07", f"{std.SQL_LOCATION_FIELD} is set before each "
        f"business write",
        "Program contains no business write statement.",
    )
else:
    record.expect_none(
        "07", f"{std.SQL_LOCATION_FIELD} is set before each "
        f"business write",
        [
            b for b in writes
            if not self._location_set_before(view, b)
        ],
    )

# ---- helpers -----
@staticmethod
def _location_set_before(view, block) -> bool:
    window = sql.lines_before(view, block, std.SQL_LOCATION_LOOKBACK)
    return any(
        line.startswith(std.SQL_LOCATION_MOVE_PREFIX)
        and std.SQL_LOCATION_MOVE_SUFFIX in line
        for line in window
    )

=====
FILE: code_review\checks\chk_11_sqlcode_commit_conversion.py
ENCODING: utf-8
=====

"""CHK-11 SQLCODE and COMMIT conversion."""

from __future__ import annotations

from code_review.engine import sql_blocks as sql
from code_review.engine.check_base import MAJOR, Check
from code_review.standards import cobol_standards as std

EVALUATE_ANY = "EVALUATE"
PERIOD = "."

class SqlcodeCommitConversionCheck(Check):
    CHECK_ID = "CHK-11"
    TITLE = "SQLCODE and COMMIT conversion"
    SEVERITY = MAJOR
    ORDER = 110

    def relevant(self, ctx, view) -> bool:
        return view.has_code_token(std.SQL_BLOCK_START)

    def not_relevant_reason(self) -> str:
        return "Program contains no embedded SQL."

    def review(self, ctx, view, record):
        lines = list(view.code)
        code = [sql.norm(line.logical) for line in lines]
        blocks = sql.sql_blocks(view)

        executable = [
            b for b in blocks
            if b.verb in std.EXECUTABLE_SQL_VERBS
            and b.verb not in std.SQLCODE_EXEMPT_VERBS
        ]
        spans = self._evaluate_spans(code)
        sql_indexes = self._sql_block_indexes(blocks)

        # ---- 01 no legacy SQLCODE form -----
        record.expect_none(
            "01", "No legacy SQLCODE test remains",
            [
                line for line in lines
                if any(
                    form in sql.norm(line.logical)

```

```

        for form in std.LEGACY_SQLCODE_FORMS
    )
],
)

# ---- 02 EVALUATE SQLCODE is the standard form -----
if not std.USE_EVALUATE_SQLCODE:
    record.skip(
        "02", f"SQLCODE handling uses {std.SQLCODE_EVALUATE_START}",
        "Not enforced by the site standard.",
    )
elif not executable:
    record.skip(
        "02", f"SQLCODE handling uses {std.SQLCODE_EVALUATE_START}",
        "Program contains no executable SQL.",
    )
else:
    record.expect(
        "02", f"SQLCODE handling uses {std.SQLCODE_EVALUATE_START}",
        bool(spans),
        note="No EVALUATE SQLCODE block found.",
    )

# ---- 03 every EVALUATE SQLCODE closes -----
#
# Scope is resolved by walking forward with a depth counter. A
# plain count of END-EVALUATE lines is wrong: the generated date
# realignment emits EVALUATE TRUE ... END-EVALUATE, so closes
# outnumber the SQLCODE opens even when every block is balanced.
if not spans:
    record.skip(
        "03", f"Every {std.SQLCODE_EVALUATE_START} block is closed",
        "No EVALUATE SQLCODE block found.",
    )
else:
    record.expect_all(
        "03", f"Every {std.SQLCODE_EVALUATE_START} block is closed",
        [
            f"line {lines[start].number} opens an unclosed block"
            for start, end in spans
            if end < 0
        ],
    )

# ---- 04 every EVALUATE has WHEN OTHER -----
if not spans:
    record.skip(
        "04", f"Every SQLCODE block handles "
            f"{std.SQLCODE_WHEN_OTHER}",
        "No EVALUATE SQLCODE block found.",
    )
else:
    record.expect_all(
        "04", f"Every SQLCODE block handles "
            f"{std.SQLCODE_WHEN_OTHER}",
        [
            f"line {lines[start].number}"
            for start, end in spans
            if not self._span_has(
                code, start, end, std.SQLCODE_WHEN_OTHER,
            )
        ],
    )

# ---- 05 WHEN OTHER routes to the error paragraph -----
if not spans:
    record.skip(
        "05", f"{std.SQLCODE_WHEN_OTHER} performs "
            f"{std.SQL_ERROR_PARAGRAPH}",
        "No EVALUATE SQLCODE block found.",
    )
else:
    perform = f"PERFORM {std.SQL_ERROR_PARAGRAPH}"
    record.expect_all(
        "05", f"{std.SQLCODE_WHEN_OTHER} performs "
            f"{std.SQL_ERROR_PARAGRAPH}",
    )

```



```

        [
            f"line {lines[start].number}"
            for start, end in spans
            if not self._span_has(code, start, end, perform)
        ],
    )

# ---- 06 every executable SQL block is checked -----
if not executable:
    record.skip(
        "06", "Every executable SQL block checks SQLCODE",
        "Program contains no executable SQL.",
    )
else:
    record.expect_none(
        "06", "Every executable SQL block checks SQLCODE",
        [b for b in executable if not self._checked_after(view, b)],
    )

# ---- 07 end of cursor uses SQLCODE = 100 -----
if not any(b.verb == "FETCH" for b in blocks):
    record.skip(
        "07", f"End of cursor uses {std.END_OF_CURSOR_CONDITION}",
        "Program contains no cursor FETCH.",
    )
else:
    record.expect(
        "07", f"End of cursor uses {std.END_OF_CURSOR_CONDITION}",
        view.has_code_token(std.END_OF_CURSOR_CONDITION)
        or any(
            line.startswith(std.SQLCODE_NOT_FOUND_BRANCH)
            for line in code
        ),
        note="Neither SQLCODE = 100 nor WHEN 100 found.",
    )

# ---- 08 FINISH became COMMIT -----
source = sql.norm(ctx.source_cobol)
if not ctx.has_source:
    record.skip(
        "08", f"{std.IDMS_COMMIT_VERB} converted to "
            f"{std.COMMIT_STATEMENT}",
        "No source program supplied for comparison.",
    )
elif std.IDMS_COMMIT_VERB not in source:
    record.skip(
        "08", f"{std.IDMS_COMMIT_VERB} converted to "
            f"{std.COMMIT_STATEMENT}",
        f"Source contains no {std.IDMS_COMMIT_VERB} statement.",
    )
else:
    record.expect(
        "08", f"{std.IDMS_COMMIT_VERB} converted to "
            f"{std.COMMIT_STATEMENT}",
        any(b.verb == std.COMMIT_STATEMENT for b in blocks),
        note=f"Source has {std.IDMS_COMMIT_VERB} but no generated "
            f"{std.COMMIT_STATEMENT} block was found.",
    )

# ---- 09 no period inside an EVALUATE block -----
#
# In COBOL a period terminates the sentence and closes every open
# scope. A period on a statement inside EVALUATE therefore ends
# the EVALUATE early and leaves END-EVALUATE with no matching
# opener, which the compiler rejects.
if not std.ENFORCE_NO_PERIOD_IN_EVALUATE:
    record.skip(
        "09", f"No period terminates a statement inside "
            f"{std.SQLCODE_EVALUATE_START}",
        "Not enforced by the site standard.",
    )
elif not spans:
    record.skip(
        "09", f"No period terminates a statement inside "
            f"{std.SQLCODE_EVALUATE_START}",
        "No EVALUATE SQLCODE block found.",
    )

```

```

    )
else:
    record.expect_none(
        "09", f"No period terminates a statement inside "
            f"{std.SQLCODE_EVALUATE_START}",
        self._interior_periods(lines, code, spans, sql_indexes),
    )

# ---- helpers -----
@staticmethod
def _evaluate_spans(code: list[str]) -> list[tuple[int, int]]:
    """Start and end index of each EVALUATE SQLCODE block.

    End is -1 when the block never closes. Nested EVALUATE blocks of
    any kind are counted, so an inner EVALUATE TRUE cannot be mistaken
    for the outer block's terminator.
    """
    spans: list[tuple[int, int]] = []
    for index, line in enumerate(code):
        if not line.startswith(std.SQLCODE_EVALUATE_START):
            continue
        depth = 0
        end = -1
        for cursor in range(index, len(code)):
            current = code[cursor]
            if current.startswith(std.SQLCODE_EVALUATE_END):
                depth -= 1
                if depth <= 0:
                    end = cursor
                    break
            elif current.startswith(EVALUATE_ANY):
                depth += 1
        spans.append((index, end))
    return spans

@staticmethod
def _sql_block_indexes(blocks) -> set[int]:
    """Code indexes occupied by EXEC SQL blocks.

    END-EXEC. legitimately carries a period, so those lines must not
    be reported by criterion 09.
    """
    out: set[int] = set()
    for block in blocks:
        out.update(range(block.first_index, block.last_index + 1))
    return out

@staticmethod
def _interior_periods(
    lines: list,
    code: list[str],
    spans: list[tuple[int, int]],
    sql_indexes: set[int],
) -> list:
    """Lines inside an EVALUATE that end with a period."""
    out: list = []
    seen: set[int] = set()

    for start, end in spans:
        stop = end if end >= 0 else len(code)
        for index in range(start + 1, stop):
            if index in sql_indexes or index in seen:
                continue
            if code[index].endswith(PERIOD):
                seen.add(index)
                out.append(lines[index])
    return out

@staticmethod
def _span_has(code: list[str], start: int, end: int, token: str) -> bool:
    stop = end if end >= 0 else len(code)
    wanted = sql.norm(token)
    return any(wanted in line for line in code[start + 1:stop])

@staticmethod
def _checked_after(view, block) -> bool:

```

```

window = sql.lines_after(view, block, std.SQLCODE_LOOKAHEAD)
return any(
    line.startswith(std.SQLCODE_EVALUATE_START)
    or line.startswith("IF SQLCODE")
    or line.startswith(std.SQLCODE_NOT_FOUND_BRANCH)
    for line in window
)

```

```

=====
FILE: code_review\checks\chk_12_cursor_declaration.py
ENCODING: utf-8
=====

```

```

"""CHK-12 Cursor declaration standard."""

```

```

from __future__ import annotations

from code_review.engine import cursors
from code_review.engine.check_base import CRITICAL, RETRIEVAL, Check
from code_review.standards import cobol_standards as std

```

```

class CursorDeclarationCheck(Check):
    CHECK_ID = "CHK-12"
    TITLE = "Cursor declaration standard"
    SEVERITY = CRITICAL
    APPLIES_TO = RETRIEVAL
    ORDER = 120

    def relevant(self, ctx, view) -> bool:
        return bool(cursors.declare_blocks(view))

    def not_relevant_reason(self) -> str:
        return "Program declares no DB2 cursor."

    def review(self, ctx, view, record):
        blocks = cursors.declare_blocks(view)
        declared = cursors.declared_cursors(view)

        # ---- 01 WITH HOLD -----
        if not std.ENFORCE_CURSOR_WITH_HOLD:
            record.skip(
                "01", "Every cursor is declared WITH HOLD",
                "Not enforced by the site standard.",
            )
        else:
            record.expect_none(
                "01", "Every cursor is declared WITH HOLD",
                [b for b in blocks if not b.has("WITH HOLD")],
            )

        # ---- 02 FOR READ ONLY -----
        if not std.ENFORCE_CURSOR_FOR_READ_ONLY:
            record.skip(
                "02", "Every cursor is declared FOR READ ONLY",
                "Not enforced by the site standard.",
            )
        else:
            record.expect_none(
                "02", "Every cursor is declared FOR READ ONLY",
                [b for b in blocks if not b.has("FOR READ ONLY")],
            )

        # ---- 03 SELECT and FROM -----
        record.expect_none(
            "03", "Every cursor declaration has SELECT and FROM",
            [
                b for b in blocks
                if not all(b.has(c) for c in std.CURSOR_SELECT_CLAUSES)
            ],
        )

        # ---- 04 explicit columns -----
        if not std.ENFORCE_CURSOR_EXPLICIT_COLUMNS:
            record.skip(

```

```

        "04", "No cursor selects every column",
        "Not enforced by the site standard.",
    )
else:
    record.expect_none(
        "04", "No cursor selects every column",
        [b for b in blocks if std.CURSOR_SELECT_ALL in b.text],
    )

# ---- 05 each cursor declared once -----
names = [
    cursors.DECLARE.search(b.text).group("cursor").upper()
    for b in blocks
    if cursors.DECLARE.search(b.text)
]
repeated = sorted({n for n in names if names.count(n) > 1})
record.expect_all(
    "05", "Each cursor is declared exactly once", repeated,
)

# ---- 06 declarations precede PROCEDURE DIVISION -----
pd_line = self._procedure_division_line(view)
if pd_line <= 0:
    record.skip(
        "06", "Cursor declarations precede PROCEDURE DIVISION",
        "PROCEDURE DIVISION not found.",
    )
else:
    record.expect_none(
        "06", "Cursor declarations precede PROCEDURE DIVISION",
        [b for b in blocks if b.line_number > pd_line],
    )

# ---- 07 every opened cursor is declared -----
opened = cursors.cursors_used(view, "OPEN")
if not opened:
    record.skip(
        "07", "Every opened cursor is declared",
        "No EXEC SQL OPEN found.",
    )
else:
    record.expect_all(
        "07", "Every opened cursor is declared",
        sorted(name for name in opened if name not in declared),
    )

# ---- 08 declaration block marker present -----
record.expect(
    "08", f"Generated block marker '{std.CURSOR_DECLARATION_MARKER}' "
        f"is present",
    view.has_token(std.CURSOR_DECLARATION_MARKER),
    note="Marker comment not found.",
)

# ---- helpers -----
@staticmethod
def _procedure_division_line(view) -> int:
    for line in view.code:
        if line.logical.startswith("PROCEDURE DIVISION"):
            return line.number
    return -1

```

```

=====
FILE: code_review\checks\chk_13_cursor_paragraph_structure.py
ENCODING: utf-8
=====

```

```

"""CHK-13 Cursor paragraph structure."""

```

```

from __future__ import annotations

from code_review.engine import cursors
from code_review.engine.check_base import MAJOR, RETRIEVAL, Check
from code_review.standards import cobol_standards as std

```

```

class CursorParagraphStructureCheck(Check):
    CHECK_ID = "CHK-13"
    TITLE = "Cursor paragraph structure"
    SEVERITY = MAJOR
    APPLIES_TO = RETRIEVAL
    ORDER = 130

    def relevant(self, ctx, view) -> bool:
        return bool(cursors.cursor_paragraphs(view))

    def not_relevant_reason(self) -> str:
        return "Program contains no generated cursor paragraph."

    def review(self, ctx, view, record):
        sets = cursors.paragraph_sets(view)
        paragraphs = cursors.cursor_paragraphs(view)
        declared = cursors.declared_cursors(view)
        performed = cursors.performed_names(view)

        # ---- 01 every declared cursor has a full set -----
        if not declared:
            record.skip(
                "01", f"Every cursor has "
                f"{'', '.join(std.CURSOR_OPERATIONS)} paragraphs",
                "Program declares no DB2 cursor.",
            )
        else:
            missing = []
            for name in sorted(declared):
                have = sets.get(name, {})
                for operation in std.CURSOR_OPERATIONS:
                    if operation not in have:
                        missing.append(f"{name} {operation}")
            record.expect_all(
                "01", f"Every cursor has "
                f"{'', '.join(std.CURSOR_OPERATIONS)} paragraphs",
                missing,
            )

        # ---- 02 numbering offsets -----
        wrong = []
        for name, group in sorted(sets.items()):
            open_para = group.get("OPEN")
            if open_para is None:
                continue
            base = open_para.number
            expected = {
                "FETCH": base + std.CURSOR_FETCH_OFFSET,
                "CLOSE": base + std.CURSOR_CLOSE_OFFSET,
            }
            for operation, number in expected.items():
                para = group.get(operation)
                if para is not None and para.number != number:
                    wrong.append(
                        f"{name} {operation} is {para.number}, "
                        f"expected {number}"
                    )
            record.expect_all(
                "02", f"FETCH is OPEN plus {std.CURSOR_FETCH_OFFSET}, "
                f"CLOSE is OPEN plus {std.CURSOR_CLOSE_OFFSET}",
                wrong,
            )

        # ---- 03 open numbers come from the agreed set -----
        if not std.ENFORCE_CURSOR_OPEN_NUMBERS:
            record.skip(
                "03", "Cursor OPEN numbers come from the site set",
                "Not enforced by the site standard.",
            )
        else:
            allowed = set(std.CURSOR_OPEN_NUMBERS)
            record.expect_all(
                "03", f"Cursor OPEN numbers come from "
                f"{std.CURSOR_OPEN_NUMBERS}",
                [

```

```

        f"{group['OPEN'].cursor} opens at {group['OPEN'].number}"
        for group in sets.values()
        if "OPEN" in group and group["OPEN"].number not in allowed
    ],
)

# ---- 04 each paragraph declared once -----
names = [p.name for p in paragraphs]
record.expect_all(
    "04", "Each cursor paragraph is declared exactly once",
    sorted({n for n in names if names.count(n) > 1}),
)

# ---- 05 every cursor paragraph sets SQL-LOCATION -----
if not std.ENFORCE_SQL_LOCATION_BEFORE_SQL:
    record.skip(
        "05", f"Every cursor paragraph sets "
              f"{std.SQL_LOCATION_FIELD}",
        "Not enforced by the site standard.",
    )
else:
    record.expect_none(
        "05", f"Every cursor paragraph sets "
              f"{std.SQL_LOCATION_FIELD}",
        [
            para for para in paragraphs
            if not any(
                std.SQL_LOCATION_MOVE_SUFFIX in body
                for body in cursors.paragraph_body(view, para)
            )
        ],
    )

# ---- 06 every cursor paragraph is reachable -----
record.expect_all(
    "06", "Every cursor paragraph is reachable",
    [
        para.name for para in paragraphs
        if para.name not in performed
    ],
)

# ---- 07 generated block marker present -----
record.expect(
    "07", f"Generated block marker '{std.CURSOR_PARAGRAPH_MARKER}' "
          f"is present",
    view.has_token(std.CURSOR_PARAGRAPH_MARKER),
    note="Marker comment not found.",
)

```

```

=====
FILE: code_review\checks\chk_14_end_of_cursor_handling.py
ENCODING: utf-8
=====

```

```

"""CHK-14 End-of-cursor handling in the fetch paragraph."""

```

```

from __future__ import annotations

```

```

from code_review.engine import cursors, sql_blocks as sql
from code_review.engine.check_base import MAJOR, RETRIEVAL, Check
from code_review.standards import cobol_standards as std

```

```

class EndOfCursorHandlingCheck(Check):
    CHECK_ID = "CHK-14"
    TITLE = "End-of-cursor handling"
    SEVERITY = MAJOR
    APPLIES_TO = RETRIEVAL
    ORDER = 140

```

```

def relevant(self, ctx, view) -> bool:
    return bool(cursors.cursor_paragraphs(view))

```

```

def not_relevant_reason(self) -> str:

```

```

return "Program contains no generated cursor paragraph."

def review(self, ctx, view, record):
    sets = cursors.paragraph_sets(view)
    fetches = [
        group["FETCH"] for group in sets.values() if "FETCH" in group
    ]
    opens = [
        group["OPEN"] for group in sets.values() if "OPEN" in group
    ]

    # ---- 01 FETCH handles WHEN 100 -----
    if not fetches:
        record.skip(
            "01", f"Every FETCH paragraph handles "
                f"{std.SQLCODE_NOT_FOUND_BRANCH}",
            "Program contains no FETCH paragraph.",
        )
    else:
        record.expect_none(
            "01", f"Every FETCH paragraph handles "
                f"{std.SQLCODE_NOT_FOUND_BRANCH}",
            [
                para for para in fetches
                if not self._body_has(
                    view, para, std.SQLCODE_NOT_FOUND_BRANCH,
                )
            ],
        )

    # ---- 02 WHEN 100 sets the EOC flag -----
    if not fetches:
        record.skip(
            "02", "Every FETCH paragraph sets the end-of-cursor flag",
            "Program contains no FETCH paragraph.",
        )
    else:
        record.expect_none(
            "02", "Every FETCH paragraph sets the end-of-cursor flag",
            [
                para for para in fetches
                if not self._body_has(
                    view,
                    para,
                    f"SET {para.cursor}{std.CURSOR_EOC_SUFFIX} TO TRUE",
                )
            ],
        )

    # ---- 03 OPEN resets the flag -----
    if not opens:
        record.skip(
            "03", "Every OPEN paragraph resets the end-of-cursor flag",
            "Program contains no OPEN paragraph.",
        )
    else:
        record.expect_none(
            "03", "Every OPEN paragraph resets the end-of-cursor flag",
            [
                para for para in opens
                if not self._body_has(
                    view,
                    para,
                    f"SET {para.cursor}{std.CURSOR_NOT_EOC_SUFFIX} "
                    f"TO TRUE",
                )
            ],
        )

    # ---- 04 loops test the flag, not SQLCODE -----
    if not std.ENFORCE_EOC_FLAG_LOOP:
        record.skip(
            "04", "Fetch loops test the end-of-cursor flag",
            "Not enforced by the site standard.",
        )
    else:

```

```

        record.expect_none(
            "04", "Fetch loops test the end-of-cursor flag",
            [
                line for line in view.code
                if any(
                    legacy in sql.norm(line.logical)
                    for legacy in std.LEGACY_EOC_CONDITIONS
                )
            ],
        )

# ---- 05 each fetch is performed UNTIL its flag -----
if not fetches:
    record.skip(
        "05", "Every FETCH paragraph is performed until its flag",
        "Program contains no FETCH paragraph.",
    )
else:
    code = [sql.norm(line.logical) for line in view.code]
    missing = []
    for para in fetches:
        wanted = (
            f"PERFORM {para.name} UNTIL "
            f"{para.cursor}{std.CURSOR_EOC_SUFFIX}"
        )
        if not any(line.startswith(wanted) for line in code):
            missing.append(wanted)
    record.expect_all(
        "05", "Every FETCH paragraph is performed until its flag",
        missing,
    )

# ---- 06 flag block marker present -----
record.expect(
    "06", f"Generated block marker '{std.CURSOR_FLAGS_MARKER}' "
        f"is present",
    view.has_token(std.CURSOR_FLAGS_MARKER),
    note="Marker comment not found.",
)

# ---- helpers -----
@staticmethod
def _body_has(view, para, token: str) -> bool:
    wanted = sql.norm(token)
    body = cursors.paragraph_body(
        view, para, stop_words=std.NON_PARAGRAPH_WORDS,
    )
    return any(wanted in line for line in body)

```

```

=====
FILE: code_review\checks\chk_15_nested_cursor_processing.py
ENCODING: utf-8
=====

```

```

"""CHK-15 Nested cursor processing."""

```

```

from __future__ import annotations

```

```

from code_review.engine import cursors, sql_blocks as sql
from code_review.engine.check_base import MAJOR, RETRIEVAL, Check
from code_review.standards import cobol_standards as std

```

```

class NestedCursorProcessingCheck(Check):
    CHECK_ID = "CHK-15"
    TITLE = "Nested cursor processing"
    SEVERITY = MAJOR
    APPLIES_TO = RETRIEVAL
    ORDER = 150

    def relevant(self, ctx, view) -> bool:
        return len(cursors.paragraph_sets(view)) > 1

    def not_relevant_reason(self) -> str:
        return "Program uses fewer than two cursors."

```



```

def review(self, ctx, view, record):
    sets = cursors.paragraph_sets(view)
    parents, children = self._split(sets)
    performed = cursors.performed_names(view)

    # ---- 01 child cursors are numbered above the threshold -----
    record.expect(
        "01",
        f"Nested cursors are numbered at or above "
        f"{std.CHILD_CURSOR_MINIMUM_NUMBER}",
        bool(children),
        note=(
            f"{len(sets)} cursor(s) found, none numbered at or above "
            f"{std.CHILD_CURSOR_MINIMUM_NUMBER}."
        ),
    )

    # ---- 02 every cursor set is complete -----
    incomplete = []
    for name, group in sorted(sets.items()):
        for operation in std.CURSOR_OPERATIONS:
            if operation not in group:
                incomplete.append(f"{name} {operation}")
    record.expect_all(
        "02", "Every nested cursor has a complete paragraph set",
        incomplete,
    )

    # ---- 03 every child cursor is closed -----
    if not children:
        record.skip(
            "03", "Every nested cursor is closed",
            "No nested cursor found.",
        )
    else:
        record.expect_all(
            "03", "Every nested cursor is closed",
            [
                group["CLOSE"].name
                for group in children.values()
                if "CLOSE" in group
                and group["CLOSE"].name not in performed
            ],
        )

    # ---- 04 parent and child use distinct flags -----
    flags = [
        f"{name}{std.CURSOR_EOC_SUFFIX}" for name in sorted(sets)
    ]
    record.expect(
        "04", "Parent and nested cursors use distinct flags",
        len(set(flags)) == len(flags),
        note=f"flags: {' ', ' '.join(flags)}",
    )

    # ---- 05 each cursor is opened exactly once -----
    repeated = []
    for name, group in sorted(sets.items()):
        open_para = group.get("OPEN")
        if open_para is None:
            continue
        count = len(performed.get(open_para.name, []))
        if count > 1:
            repeated.append(f"{open_para.name} performed {count} times")
    record.expect_all(
        "05", "Each cursor is opened from exactly one place", repeated,
    )

    # ---- 06 child fetch loop has an early stop -----
    if not std.ENFORCE_CHILD_FETCH_EARLY_STOP:
        record.skip(
            "06", "Nested fetch loops carry an early-stop condition",
            "Early-stop condition not yet agreed (decision open).",
        )
    elif not children:

```

```

        record.skip(
            "06", "Nested fetch loops carry an early-stop condition",
            "No nested cursor found.",
        )
    else:
        code = [sql.norm(line.logical) for line in view.code]
        missing = []
        for group in children.values():
            fetch = group.get("FETCH")
            if fetch is None:
                continue
            loop = f"PERFORM {fetch.name} UNTIL"
            found = any(
                line.startswith(loop)
                and std.CHILD_FETCH_EARLY_STOP_FIELD in line
                for line in code
            )
            if not found:
                missing.append(fetch.name)
        record.expect_all(
            "06", "Nested fetch loops carry an early-stop condition",
            missing,
        )

```

```

# ---- helpers -----

```

```

@staticmethod

```

```

def _split(sets):

```

```

    parents, children = {}, {}

```

```

    for name, group in sets.items():

```

```

        open_para = group.get("OPEN")

```

```

        number = open_para.number if open_para else 0

```

```

        if number >= std.CHILD_CURSOR_MINIMUM_NUMBER:

```

```

            children[name] = group

```

```

        else:

```

```

            parents[name] = group

```

```

    return parents, children

```

```

=====
FILE: code_review\checks\chk_16_conservative_update_columns.py

```

```

ENCODING: utf-8
=====

```

```

"""CHK-16 Conservative update column selection."""

```

```

from __future__ import annotations

```

```

import re

```

```

from code_review.engine import metadata as meta, sql_blocks as sql

```

```

from code_review.engine.check_base import CRITICAL, UPDATE, Check

```

```

from code_review.standards import cobol_standards as std

```

```

UPDATE_TABLE = re.compile(r"\bUPDATE\s+(?P<table>[A-Z0-9_]+\b)")

```

```

SET_CLAUSE = re.compile(r"\bSET\b(?:P<body>.*?)\bWHERE\b")

```

```

ASSIGNMENT = re.compile(r"(?P<column>[A-Z][A-Z0-9_]*)\s*=")

```

```

class ConservativeUpdateColumnsCheck(Check):

```

```

    CHECK_ID = "CHK-16"

```

```

    TITLE = "Conservative update column selection"

```

```

    SEVERITY = CRITICAL

```

```

    APPLIES_TO = UPDATE

```

```

    ORDER = 160

```

```

    def relevant(self, ctx, view) -> bool:

```

```

        return any(b.verb == "UPDATE" for b in sql.sql_blocks(view))

```

```

    def not_relevant_reason(self) -> str:

```

```

        return "Program issues no UPDATE statement."

```

```

    def review(self, ctx, view, record):

```

```

        updates = [b for b in sql.sql_blocks(view) if b.verb == "UPDATE"]

```

```

        plans = [(b, self._table(b), self._set_columns(b)) for b in updates]

```

```

        # ---- 01 every UPDATE sets at least one column -----

```

```

record.expect_none(
    "01", "Every UPDATE sets at least one column",
    [block for block, _table, columns in plans if not columns],
)

# ---- 02 no insert-only audit column in SET -----
offenders = []
for block, _table, columns in plans:
    bad = [
        column for column in columns
        if column.startswith(std.INSERT_ONLY_AUDIT_PREFIXES)
    ]
    if bad:
        offenders.append(f"{block.line_number}: {' '.join(bad)}")
record.expect_all(
    "02",
    f"No UPDATE sets an insert-only audit column "
    f"{std.INSERT_ONLY_AUDIT_PREFIXES}",
    offenders,
)

# ---- 03 key columns stay out of SET -----
keys = meta.all_primary_key_columns(ctx, std.KEY_TEXT_MARKERS)
if not keys:
    record.skip(
        "03", "No UPDATE sets a primary key column",
        "No Sheet Mapping key metadata supplied for this run.",
    )
else:
    offenders = []
    for block, _table, columns in plans:
        bad = sorted(set(columns) & keys)
        if bad:
            offenders.append(
                f"{block.line_number}: {' '.join(bad)}"
            )
    record.expect_all(
        "03", "No UPDATE sets a primary key column", offenders,
    )

# ---- 04 the update is not broad -----
if not std.ENFORCE_CONSERVATIVE_UPDATE:
    record.skip(
        "04", "No UPDATE rewrites the whole row",
        "Not enforced by the site standard.",
    )
elif not meta.dclgen_tables(ctx):
    record.skip(
        "04", "No UPDATE rewrites the whole row",
        "No DCLGEN metadata supplied for this run.",
    )
else:
    offenders = []
    for block, table, columns in plans:
        available = meta.columns_for_table(ctx, table)
        if not available or not columns:
            continue
        ratio = len(set(columns) & available) / len(available)
        if ratio >= std.BROAD_UPDATE_RATIO:
            offenders.append(
                f"{table} sets {len(columns)} of {len(available)} "
                f"columns"
            )
    record.expect_all(
        "04", "No UPDATE rewrites the whole row", offenders,
    )

# ---- 05 every SET column exists in DCLGEN -----
if not meta.dclgen_tables(ctx):
    record.skip(
        "05", "Every updated column exists in DCLGEN",
        "No DCLGEN metadata supplied for this run.",
    )
else:
    offenders = []
    for block, table, columns in plans:

```

```

        available = meta.columns_for_table(ctx, table)
        if not available:
            continue
        bad = sorted(set(columns) - available)
        if bad:
            offenders.append(f"{table}: {' '.join(bad)}")
    record.expect_all(
        "05", "Every updated column exists in DCLGEN", offenders,
    )

    # ---- 06 no incomplete-metadata marker remains -----
    record.expect_none(
        "06", "No update was skipped for incomplete metadata",
        [
            line for line in view.comments
            if any(m in line.logical for m in std.MISSING_MAPPING_MARKERS)
        ],
    )

```

```

# ---- helpers -----

```

```

@staticmethod
def _table(block) -> str:
    match = UPDATE_TABLE.search(block.text)
    return match.group("table") if match else ""

```

```

@staticmethod
def _set_columns(block) -> list[str]:
    match = SET_CLAUSE.search(block.text)
    if not match:
        return []
    return [
        found.group("column")
        for found in ASSIGNMENT.finditer(match.group("body"))
    ]

```

```

=====
FILE: code_review\checks\chk_17_update_where_key_integrity.py
ENCODING: utf-8
=====

```

```

"""CHK-17 Update WHERE key integrity."""

```

```

from __future__ import annotations

```

```

import re

```

```

from code_review.engine import metadata as meta, sql_blocks as sql
from code_review.engine.check_base import CRITICAL, UPDATE, Check
from code_review.standards import cobol_standards as std

```

```

WHERE_CLAUSE = re.compile(r"\bWHERE\b(?:P<body>.*?)(?:\bQUERYNO\b|$)")
COMPARISON = re.compile(r"(?P<column>[A-Z][A-Z0-9_]*)\s*=\s*(?P<value>\S+)")
TARGET = re.compile(
    r"\b(?:UPDATE|DELETE\s+FROM)\s+(?P<table>[A-Z0-9_+])\b"
)

```

```

class UpdateWhereKeyIntegrityCheck(Check):

```

```

    CHECK_ID = "CHK-17"
    TITLE = "Update WHERE key integrity"
    SEVERITY = CRITICAL
    APPLIES_TO = UPDATE
    ORDER = 170

```

```

    def relevant(self, ctx, view) -> bool:
        return any(
            b.verb in ("UPDATE", "DELETE")
            and not sql.targets_restart(b, std.RESTART_TABLE_NAME_HINTS)
            for b in sql.sql_blocks(view)
        )

```

```

    def not_relevant_reason(self) -> str:
        return "Program issues no business UPDATE or DELETE statement."

```

```

    def review(self, ctx, view, record):

```

```

# Restart writes are keyed on restart-table columns that are not
# business records in Sheet Mapping. CHK-20 owns them.
blocks = [
    b for b in sql.sql_blocks(view)
    if b.verb in ("UPDATE", "DELETE")
    and not sql.targets_restart(b, std.RESTART_TABLE_NAME_HINTS)
]
plans = [
    (b, self._where_columns(b), self._where_body(b)) for b in blocks
]

keys_by_record = self._keys_by_record(ctx)
all_keys = set().union(*keys_by_record.values()) if keys_by_record else set()
foreign = meta.all_foreign_key_columns(ctx)

# ---- 01 every write is key qualified -----
record.expect_none(
    "01", "Every business UPDATE and DELETE has a WHERE clause",
    [block for block, _columns, body in plans if not body],
)

# ---- 02 WHERE references a key-style column -----
if not all_keys:
    record.skip(
        "02", "Every WHERE references a primary key column",
        "No Sheet Mapping key metadata supplied for this run.",
    )
else:
    record.expect_none(
        "02", "Every WHERE references a primary key column",
        [
            block for block, columns, body in plans
            if body and not (set(columns) & all_keys)
        ],
    )

# ---- 03 composite keys are complete -----
#
# Keys are evaluated per record, not pooled. Several IDMS records
# can map to one DB2 table, each with its own composite key, so a
# pooled set would demand columns from an unrelated record.
if not keys_by_record:
    record.skip(
        "03", "Composite keys appear in full",
        "No Sheet Mapping key metadata supplied for this run.",
    )
else:
    record.expect_all(
        "03", "Composite keys appear in full",
        self._incomplete_keys(plans, keys_by_record),
    )

# ---- 04 no foreign key in WHERE -----
if not std.ENFORCE_WHERE_NO_FOREIGN_KEY:
    record.skip(
        "04", "No WHERE uses a foreign key column",
        "Not enforced by the site standard.",
    )
elif not foreign:
    record.skip(
        "04", "No WHERE uses a foreign key column",
        "No Sheet Mapping relationship metadata supplied for "
        "this run.",
    )
else:
    offenders = []
    for block, columns, _body in plans:
        bad = sorted(set(columns) & foreign)
        if bad:
            offenders.append(
                f"line {block.line_number}: {' ', '.join(bad)}"
            )
    record.expect_all(
        "04", "No WHERE uses a foreign key column", offenders,
    )

```

```

# ---- 05 comparisons use host variables -----
if not std.ENFORCE_WHERE_HOST_VARIABLES:
    record.skip(
        "05", "Every WHERE comparison uses a host variable",
        "Not enforced by the site standard.",
    )
else:
    offenders = []
    for block, _columns, body in plans:
        if not body:
            continue
        literals = [
            found.group(0)
            for found in COMPARISON.finditer(body)
            if std.HOST_REFERENCE_MARKER not in found.group("value")
        ]
        if literals:
            offenders.append(
                f"line {block.line_number}: {' '.join(literals[:3])}"
            )
    record.expect_all(
        "05", "Every WHERE comparison uses a host variable",
        offenders,
    )

# ---- 06 no always-true predicate -----
record.expect_none(
    "06", "No WHERE uses an always-true predicate",
    [
        block for block, _columns, body in plans
        if any(
            predicate in body
            for predicate in std.FORBIDDEN_WHERE_PREDICATES
        )
    ],
)

# ---- helpers -----
@staticmethod
def _keys_by_record(ctx) -> dict[str, set[str]]:
    out: dict[str, set[str]] = {}
    for name in meta.mapped_records(ctx):
        columns = set(
            meta.primary_key_columns(ctx, name, std.KEY_TEXT_MARKERS)
        )
        if columns:
            out[name] = columns
    return out

@classmethod
def _incomplete_keys(cls, plans, keys_by_record) -> list[str]:
    out: list[str] = []
    for block, columns, body in plans:
        if not body:
            continue
        used = set(columns)
        candidates = [
            (name, keys)
            for name, keys in keys_by_record.items()
            if keys & used
        ]
        if not candidates:
            continue
        if any(keys <= used for _name, keys in candidates):
            continue
        name, keys = max(
            candidates, key=lambda item: len(item[1] & used)
        )
        missing = sorted(keys - used)
        out.append(
            f"{cls._table(block)} line {block.line_number} "
            f"for record {name} missing {' '.join(missing)}"
        )
    return out

@staticmethod

```

```

def _where_body(block) -> str:
    match = WHERE_CLAUSE.search(block.text)
    return match.group("body").strip() if match else ""

@classmethod
def _where_columns(cls, block) -> list[str]:
    body = cls._where_body(block)
    return [
        found.group("column") for found in COMPARISON.finditer(body)
    ]

@staticmethod
def _table(block) -> str:
    match = TARGET.search(block.text)
    return match.group("table") if match else ""

=====
FILE: code_review\checks\chk_18_audit_and_date_handling.py
ENCODING: utf-8
=====

"""CHK-18 Audit column and date handling."""

from __future__ import annotations

import re

from code_review.engine import metadata as meta, sql_blocks as sql
from code_review.engine.check_base import MAJOR, UPDATE, Check
from code_review.standards import cobol_standards as std

UPDATE_TABLE = re.compile(r"\bUPDATE\s+(?P<table>[A-Z0-9_]+)\b")
SET_CLAUSE = re.compile(r"\bSET\b(?:P<body>.*?)\bWHERE\b")
ASSIGNMENT = re.compile(r"(?P<column>[A-Z][A-Z0-9_]*)\s*=")
MOVE_TO = re.compile(r"MOVE\s+(?P<source>.+?)\s+TO\s+(?P<target>.+?)\s*\.$")

class AuditAndDateHandlingCheck(Check):
    CHECK_ID = "CHK-18"
    TITLE = "Audit column and date handling"
    SEVERITY = MAJOR
    APPLIES_TO = UPDATE
    ORDER = 180

    def relevant(self, ctx, view) -> bool:
        return any(b.verb == "UPDATE" for b in sql.sql_blocks(view))

    def not_relevant_reason(self) -> str:
        return "Program issues no UPDATE statement."

    def review(self, ctx, view, record):
        updates = [b for b in sql.sql_blocks(view) if b.verb == "UPDATE"]
        plans = [(b, self._table(b), self._set_columns(b)) for b in updates]

        # ---- 01 update audit columns are set -----
        if not meta.dclgen_tables(ctx):
            record.skip(
                "01", "Every UPDATE sets its update audit columns",
                "No DCLGEN metadata supplied for this run.",
            )
        else:
            offenders = []
            for block, table, columns in plans:
                expected = meta.columns_with_prefix(
                    ctx, table, std.UPDATE_AUDIT_PREFIXES,
                )
                missing = sorted(expected - set(columns))
                if expected and missing:
                    offenders.append(f"{table}: {' '.join(missing)}")
            record.expect_all(
                "01", "Every UPDATE sets its update audit columns", offenders,
            )

        # ---- 02 insert-only audit columns stay out -----
        offenders = []

```

```

for block, table, columns in plans:
    bad = [
        column for column in columns
        if column.startswith(std.INSERT_ONLY_AUDIT_PREFIXES)
    ]
    if bad:
        offenders.append(f"{table}: {' '.join(bad)}")
record.expect_all(
    "02",
    f"No UPDATE sets {' '.join(std.INSERT_ONLY_AUDIT_PREFIXES)}",
    offenders,
)

# ---- 03 audit host moves precede the UPDATE -----
if not std.ENFORCE_AUDIT_MOVES_BEFORE_UPDATE:
    record.skip(
        "03", "Audit host moves precede each UPDATE",
        "Not enforced by the site standard.",
    )
elif not meta.dclgen_tables(ctx):
    record.skip(
        "03", "Audit host moves precede each UPDATE",
        "No DCLGEN metadata supplied for this run.",
    )
else:
    offenders = []
    for block, table, _columns in plans:
        expected = meta.columns_with_prefix(
            ctx, table, std.UPDATE_AUDIT_PREFIXES,
        )
        if not expected:
            continue
        hosts = meta.hosts_for_table(ctx, table)
        wanted = {
            hosts[column] for column in expected if column in hosts
        }
        if not wanted:
            continue
        window = " ".join(
            sql.lines_before(view, block, std.AUDIT_MOVE_LOOKBACK)
        )
        missing = sorted(
            host for host in wanted if host not in window
        )
        if missing:
            offenders.append(f"{table}: {' '.join(missing)}")
    record.expect_all(
        "03", "Audit host moves precede each UPDATE", offenders,
    )

# ---- 04 date staging fields are used -----
if not std.ENFORCE_DATE_STAGING:
    record.skip(
        "04", "Date values pass through the staging fields",
        "Not enforced by the site standard.",
    )
else:
    date_hosts = self._date_hosts(ctx)
    if not date_hosts:
        record.skip(
            "04", "Date values pass through the staging fields",
            "No DCLGEN date column supplied for this run.",
        )
    else:
        offenders = [
            line for line in view.code
            if self._raw_date_move(line, date_hosts)
        ]
        record.expect_none(
            "04", "Date values pass through the staging fields",
            offenders,
        )

# ---- 05 date sentinel is declared -----
date_hosts = self._date_hosts(ctx)
if not date_hosts:

```



```

        record.skip(
            "05", f"DB2 date sentinel {std.DB2_DATE_SENTINEL} is used",
            "No DCLGEN date column supplied for this run.",
        )
    else:
        record.expect(
            "05", f"DB2 date sentinel {std.DB2_DATE_SENTINEL} is used",
            view.has_code_token(std.DB2_DATE_SENTINEL),
            note="Sentinel not found. Empty source dates may move "
                "SPACES into a DB2 date column.",
        )

# ---- 06 timestamp source field is used -----
if not any(
    meta.columns_with_prefix(ctx, table, ("TS_UPDATE",))
    for _block, table, _columns in plans
):
    record.skip(
        "06", f"Update timestamps come from "
            f"{std.TIMESTAMP_SOURCE_FIELD}",
        "No TS_UPDATE column supplied for this run.",
    )
else:
    record.expect(
        "06", f"Update timestamps come from "
            f"{std.TIMESTAMP_SOURCE_FIELD}",
        view.has_code_token(std.TIMESTAMP_SOURCE_FIELD),
        note=f"{std.TIMESTAMP_SOURCE_FIELD} not referenced.",
    )

# ---- helpers -----
@staticmethod
def _table(block) -> str:
    match = UPDATE_TABLE.search(block.text)
    return match.group("table") if match else ""

@staticmethod
def _set_columns(block) -> list[str]:
    match = SET_CLAUSE.search(block.text)
    if not match:
        return []
    return [
        found.group("column")
        for found in ASSIGNMENT.finditer(match.group("body"))
    ]

@staticmethod
def _date_hosts(ctx) -> set[str]:
    out: set[str] = set()
    for table in meta.dclgen_tables(ctx):
        columns = meta.columns_with_prefix(
            ctx, table, std.DATE_COLUMN_PREFIXES,
        )
        hosts = meta.hosts_for_table(ctx, table)
        out.update(
            hosts[column] for column in columns if column in hosts
        )
    return out

@staticmethod
def _raw_date_move(line, date_hosts: set[str]) -> bool:
    """A MOVE into a DB2 date host that bypasses the staging fields."""
    match = MOVE_TO.match(sql.norm(line.logical))
    if not match:
        return False
    target = match.group("target")
    if not any(host in target for host in date_hosts):
        return False
    source = match.group("source")
    if any(field in source for field in std.DATE_STAGING_FIELDS):
        return False
    return std.TIMESTAMP_SOURCE_FIELD not in source

```

```
=====
FILE: code_review\checks\chk_19_restart_paragraph_structure.py
ENCODING: utf-8
=====
```

```
"""CHK-19 Restart paragraph structure."""
```

```
from __future__ import annotations
```

```
import re
```

```
from code_review.engine import cursors, sql_blocks as sql
from code_review.engine.check_base import MAJOR, UPDATE, Check
from code_review.standards import cobol_standards as std
```

```
PARAGRAPH = re.compile(r"^(?P<name>[A-Z0-9][A-Z0-9-]*)\.$")
RESTART_SQL_PARAGRAPH = re.compile(
    r"^(?P<number>\d{3})-(?P<operation>SELECT|UPDATE|INSERT)-"
    r"(?P<table>[A-Z0-9]+)\.$"
)
```

```
class RestartParagraphStructureCheck(Check):
```

```
    CHECK_ID = "CHK-19"
```

```
    TITLE = "Restart paragraph structure"
```

```
    SEVERITY = MAJOR
```

```
    APPLIES_TO = UPDATE
```

```
    ORDER = 190
```

```
    def relevant(self, ctx, view) -> bool:
```

```
        return bool(self._declared(view))
```

```
    def not_relevant_reason(self) -> str:
```

```
        return (
```

```
            "No generated restart paragraph found. The legacy IDMS restart "
```

```
            "flow is preserved, pending Sheet Mapping input (decision D-3)."
```

```
        )
```

```
    def review(self, ctx, view, record):
```

```
        declared = self._declared(view)
```

```
        performed = cursors.performed_names(view)
```

```
        expected = set(std.RESTART_PARAGRAPHS.values())
```

```
        # ---- 01 the standard set is complete -----
```

```
        record.expect_all(
```

```
            "01", "Every standard restart paragraph is declared",
```

```
            sorted(expected - declared.keys()),
```

```
        )
```

```
        # ---- 02 each declared once -----
```

```
        repeated = sorted(
```

```
            name for name, lines in declared.items()
```

```
            if name in expected and len(lines) > 1
```

```
        )
```

```
        record.expect_all(
```

```
            "02", "Each restart paragraph is declared exactly once", repeated,
```

```
        )
```

```
        # ---- 03 each is reachable -----
```

```
        record.expect_all(
```

```
            "03", "Every restart paragraph is reachable",
```

```
            sorted(
```

```
                name for name in declared
```

```
                if name in expected
```

```
                and name != std.RESTART_PARAGRAPHS["control"]
```

```
                and name not in performed
```

```
            ),
```

```
        )
```

```
        # ---- 04 restart SQL paragraphs exist -----
```

```
        sql_paragraphs = self._restart_sql_paragraphs(view)
```

```
        if not sql_paragraphs:
```

```
            record.skip(
```

```
                "04", "Restart SELECT, UPDATE and INSERT paragraphs exist",
```

```
                "No restart SQL paragraph generated.",
```

```
            )
```

```

else:
    found = {p["operation"] for p in sql_paragraphs}
    record.expect_all(
        "04", "Restart SELECT, UPDATE and INSERT paragraphs exist",
        sorted(set(std.RESTART_SQL_OPERATIONS) - found),
    )

# ---- 05 restart SQL paragraphs are numbered consistently ----
if not sql_paragraphs:
    record.skip(
        "05", f"Restart SQL paragraphs use the "
        f"{std.RESTART_SQL_PARAGRAPH_PREFIX} prefix",
        "No restart SQL paragraph generated.",
    )
else:
    record.expect_all(
        "05", f"Restart SQL paragraphs use the "
        f"{std.RESTART_SQL_PARAGRAPH_PREFIX} prefix",
        [
            p["name"] for p in sql_paragraphs
            if not p["name"].startswith(
                std.RESTART_SQL_PARAGRAPH_PREFIX
            )
        ],
    )

# ---- 06 the abend paragraph calls the abend routine -----
abend = std.RESTART_PARAGRAPHS["abend"]
if abend not in declared:
    record.skip(
        "06", f"{abend} calls {std.RESTART_ABEND_CALL}",
        "Abend paragraph not declared.",
    )
else:
    body = self._paragraph_body(view, declared[abend][0])
    record.expect(
        "06", f"{abend} calls {std.RESTART_ABEND_CALL}",
        any(std.RESTART_ABEND_CALL in line for line in body),
        note="Abend call not found in the paragraph body.",
    )

# ---- 07 the control paragraph drives the select -----
control = std.RESTART_PARAGRAPHS["control"]
if control not in declared:
    record.skip(
        "07", f"{control} performs a restart SELECT",
        "Control paragraph not declared.",
    )
else:
    body = self._paragraph_body(view, declared[control][0])
    record.expect(
        "07", f"{control} performs a restart SELECT",
        any(
            line.startswith("PERFORM")
            and "SELECT" in line
            for line in body
        ),
        note="No PERFORM of a restart SELECT paragraph found.",
    )

# ---- helpers -----
@staticmethod
def _declared(view) -> dict[str, list]:
    out: dict[str, list] = {}
    for line in view.code:
        match = PARAGRAPH.match(sql.norm(line.logical))
        if not match:
            continue
        name = match.group("name")
        if name in std.NON_PARAGRAPH_WORDS:
            continue
        if name in std.RESTART_PARAGRAPHS.values() or (
            name.startswith(std.RESTART_SQL_PARAGRAPH_PREFIX)
        ):
            out.setdefault(name, []).append(line)
    return out

```

```

@staticmethod
def _restart_sql_paragraphs(view) -> list[dict]:
    out = []
    for line in view.code:
        match = RESTART_SQL_PARAGRAPH.match(sql.norm(line.logical))
        if match:
            out.append({
                "name": sql.norm(line.logical).rstrip("."),
                "operation": match.group("operation"),
                "table": match.group("table"),
                "line": line,
            })
    return out

@staticmethod
def _paragraph_body(view, header) -> list[str]:
    code = view.code
    start = next(
        (i for i, line in enumerate(code) if line.number == header.number),
        -1,
    )
    if start < 0:
        return []
    out = []
    for line in code[start + 1:]:
        logical = sql.norm(line.logical)
        if PARAGRAPH.match(logical) and (
            logical[:-1] not in std.NON_PARAGRAPH_WORDS
        ):
            break
        out.append(logical)
    return out

```

```

=====
FILE: code_review\checks\chk_20_restart_sql_metadata.py
ENCODING: utf-8
=====

```

```

"""CHK-20 Restart SQL generated from DCLGEN metadata."""

```

```

from __future__ import annotations

```

```

import re

```

```

from code_review.engine import metadata as meta, sql_blocks as sql
from code_review.engine.check_base import MAJOR, UPDATE, Check
from code_review.standards import cobol_standards as std

```

```

TABLE = re.compile(
    r"\b(?:FROM|UPDATE|INSERT\s+INTO)\s+(?P<table>[A-Z0-9_]+\b"
)
HOST_DOT = re.compile(r"(:?(?P<group>[A-Z0-9-]+\.)?(?P<field>[A-Z0-9-]+)")

```

```

class RestartSqlMetadataCheck(Check):

```

```

    CHECK_ID = "CHK-20"
    TITLE = "Restart SQL generated from DCLGEN metadata"
    SEVERITY = MAJOR
    APPLIES_TO = UPDATE
    ORDER = 200

```

```

    def relevant(self, ctx, view) -> bool:
        return bool(self._restart_blocks(view))

```

```

    def not_relevant_reason(self) -> str:
        return (
            "No restart SQL generated. The legacy IDMS restart flow is "
            "preserved, pending Sheet Mapping input (decision D-3)."
        )

```

```

    def review(self, ctx, view, record):
        blocks = self._restart_blocks(view)
        tables = {self._table(b) for b in blocks} - {""}

```

```

# ---- 01 restart SQL targets one table -----
record.expect(
    "01", "Restart SQL targets a single restart table",
    len(tables) == 1,
    note=f"tables: {' ', '.join(sorted(tables)) or 'none'"}",
)

# ---- 02 the table is a recognisable restart table -----
record.expect_all(
    "02", f"The restart table name carries a "
        f"{' ' or '.join(std.RESTART_TABLE_NAME_HINTS)} marker",
    sorted(
        table for table in tables
        if not any(
            hint in table for hint in std.RESTART_TABLE_NAME_HINTS
        )
    ),
)

# ---- 03 the table exists in DCLGEN -----
known = meta.dclgen_tables(ctx)
if not known:
    record.skip(
        "03", "The restart table exists in DCLGEN",
        "No DCLGEN metadata supplied for this run.",
    )
else:
    record.expect_all(
        "03", "The restart table exists in DCLGEN",
        sorted(table for table in tables if table not in known),
    )

# ---- 04 every restart column exists in DCLGEN -----
if not known:
    record.skip(
        "04", "Every restart column exists in DCLGEN",
        "No DCLGEN metadata supplied for this run.",
    )
else:
    offenders = []
    for block in blocks:
        table = self._table(block)
        available = meta.columns_for_table(ctx, table)
        if not available:
            continue
        bad = sorted(self._columns(block) - available)
        if bad:
            offenders.append(f"{table}: {' ', '.join(bad)}")
    record.expect_all(
        "04", "Every restart column exists in DCLGEN", offenders,
    )

# ---- 05 host references use a DCLGEN group -----
prefix = std.DCLGEN_GROUP_PREFIX.upper()
offenders = []
for block in blocks:
    for match in HOST_DOT.finditer(block.text):
        group = match.group("group")
        if not group.startswith(prefix):
            offenders.append(f"{group}.{' ', '.join(match.group('field'))}")
record.expect_all(
    "05", f"Every restart host reference uses a "
        f"{std.DCLGEN_GROUP_PREFIX} group",
    sorted(set(offenders)),
)

# ---- 06 QUERYNO -----
if not std.ENFORCE_RESTART_QUERYNO:
    record.skip(
        "06", "Every restart statement carries its QUERYNO",
        "Restart QUERYNO values conflict between "
        "update_postprocess_rules and update_restart_rules "
        "(decision D-3).",
    )
else:
    expected = {

```

```

        "SELECT": std.RESTART_SELECT_QUERYNO,
        "UPDATE": std.RESTART_UPDATE_QUERYNO,
        "INSERT": std.RESTART_INSERT_QUERYNO,
    }
    offenders = []
    for block in blocks:
        wanted = expected.get(block.verb, "")
        if not wanted:
            continue
        if f"QUERYNO {wanted}" not in block.text:
            offenders.append(
                f"{block.verb} at line {block.line_number}"
            )
    record.expect_all(
        "06", "Every restart statement carries its QUERYNO",
        offenders,
    )

# ---- 07 SQL-LOCATION before each restart statement -----
#
# Without this, a restart SQL failure reports whatever location the
# previous business statement left in SQL-LOCATION, sending the
# operator to the wrong paragraph.
if not std.ENFORCE_SQL_LOCATION_BEFORE_SQL:
    record.skip(
        "07", f"{std.SQL_LOCATION_FIELD} is set before each restart "
        f"statement",
        "Not enforced by the site standard.",
    )
else:
    record.expect_none(
        "07", f"{std.SQL_LOCATION_FIELD} is set before each restart "
        f"statement",
        [
            b for b in blocks
            if not self._location_set_before(view, b)
        ],
    )

# ---- helpers -----
@classmethod
def _restart_blocks(cls, view) -> list:
    return [
        block for block in sql.sql_blocks(view)
        if block.verb in std.RESTART_SQL_OPERATIONS
        and sql.targets_restart(block, std.RESTART_TABLE_NAME_HINTS)
    ]

@staticmethod
def _table(block) -> str:
    match = TABLE.search(block.text)
    return match.group("table") if match else ""

@staticmethod
def _columns(block) -> set[str]:
    return {
        match.group("field").replace("-", "_")
        for match in HOST_DOT.finditer(block.text)
    }

@staticmethod
def _location_set_before(view, block) -> bool:
    window = sql.lines_before(view, block, std.SQL_LOCATION_LOOKBACK)
    return any(
        line.startswith(std.SQL_LOCATION_MOVE_PREFIX)
        and std.SQL_LOCATION_MOVE_SUFFIX in line
        for line in window
    )

```

```

=====
FILE: code_review\checks\chk_21_restart_safe_skip.py
ENCODING: utf-8
=====

```

```

""""CHK-21 Unresolved restart metadata safe skip."""

```

```

from __future__ import annotations

from code_review.engine import metadata as meta, sql_blocks as sql
from code_review.engine.check_base import CRITICAL, UPDATE, Check
from code_review.standards import cobol_standards as std

class RestartSafeSkipCheck(Check):
    """Certifies that a deferred restart conversion was deferred safely.

    The authority rule is absolute: the converter must never invent a DB2
    name. When the restart table is absent from the supplied metadata the
    correct behaviour is to skip, document the skip, and leave the legacy
    IDMS flow untouched. This check proves that happened, which is what
    turns decision D-3 from an undocumented gap into an auditable one.
    """

    CHECK_ID = "CHK-21"
    TITLE = "Unresolved restart metadata safe skip"
    SEVERITY = CRITICAL
    APPLIES_TO = UPDATE
    ORDER = 210

    def relevant(self, ctx, view) -> bool:
        return bool(
            self._restart_in_source(ctx) or self._restart_blocks(view)
        )

    def not_relevant_reason(self) -> str:
        return "Program has no restart or control record."

    def review(self, ctx, view, record):
        resolved = self._resolved_restart_tables(ctx)
        blocks = self._restart_blocks(view)
        generated = bool(blocks)

        # ---- 01 no fabrication -----
        if resolved:
            record.skip(
                "01", "No restart SQL is generated without metadata",
                f"Restart table {' '.join(sorted(resolved))} is resolved "
                f"in the supplied metadata, so generation is authorised.",
            )
        elif not ctx.has_metadata:
            record.skip(
                "01", "No restart SQL is generated without metadata",
                "No Sheet Mapping or DCLGEN metadata supplied for this run.",
            )
        else:
            record.expect_none(
                "01", "No restart SQL is generated without metadata", blocks,
            )

        # ---- 02 the skip is documented -----
        if generated:
            record.skip(
                "02", "The restart skip is documented in the output",
                "Restart flow was generated, so no skip is expected.",
            )
        else:
            record.expect(
                "02", "The restart skip is documented in the output",
                self._skip_documented(view),
                note="No restart skip comment found. A silent skip leaves "
                "the reader unable to tell converted from deferred.",
            )

        # ---- 03 the legacy flow is preserved -----
        if generated:
            record.skip(
                "03", "The legacy restart flow is preserved",
                "Restart flow was generated, so replacement is expected.",
            )
        elif not ctx.has_source:
            record.skip(

```

```

        "03", "The legacy restart flow is preserved",
        "No source program supplied for comparison.",
    )
else:
    record.expect_all(
        "03", "The legacy restart flow is preserved",
        self._lost_restart_tokens(ctx, view),
    )

# ---- 04 no half-built restart paragraph set -----
present = [
    name for name in std.RESTART_PARAGRAPHS.values()
    if view.paragraph_exists(name)
]
if not present:
    record.ok("04", "No partial restart paragraph set was emitted")
else:
    record.expect_all(
        "04", "No partial restart paragraph set was emitted",
        sorted(set(std.RESTART_PARAGRAPHS.values()) - set(present)),
    )

# ---- 05 no warning marker remains in code -----
record.expect_none(
    "05", "No unresolved-metadata warning marker remains in code",
    [
        line for line in view.code
        if any(m in line.logical for m in std.FORBIDDEN_MARKERS)
    ],
)

# ---- helpers -----
@staticmethod
def _restart_blocks(view) -> list:
    """Generated SQL that reads or writes a restart or control table."""
    return [
        block for block in sql.sql_blocks(view)
        if block.verb in std.RESTART_SQL_OPERATIONS
        and sql.targets_restart(block, std.RESTART_TABLE_NAME_HINTS)
    ]

@staticmethod
def _resolved_restart_tables(ctx) -> set[str]:
    """Restart tables the supplied metadata actually confirms."""
    known = meta.dclgen_tables(ctx) | meta.mapped_tables(ctx)
    return {
        table for table in known
        if any(
            hint in table for hint in std.RESTART_TABLE_NAME_HINTS
        )
    }

@staticmethod
def _restart_tokens(text: str) -> set[str]:
    upper = sql.norm(text)
    return {
        hint for hint in std.RESTART_CONTROL_HINTS if hint in upper
    }

@classmethod
def _restart_in_source(cls, ctx) -> set[str]:
    return cls._restart_tokens(ctx.source_cobol)

@staticmethod
def _skip_documented(view) -> bool:
    return any(
        marker in line.logical
        for line in view.comments
        for marker in std.RESTART_SKIP_MARKERS
    )

@classmethod
def _lost_restart_tokens(cls, ctx, view) -> list[str]:
    """Restart markers present in the source but gone from the output.

    When nothing was generated the legacy flow must survive intact.

```


A token that vanished means the converter removed working logic without replacing it.

```
"""
output = view.all_text
return sorted(
    token for token in cls._restart_in_source(ctx)
    if token not in output
)
```

```
=====
FILE: code_review\checks\chk_22_naming_authority.py
ENCODING: utf-8
=====
```

```
"""CHK-22 DB2 table and column naming authority."""
```

```
from __future__ import annotations
```

```
import re
```

```
from code_review.engine import metadata as meta, sql_blocks as sql
from code_review.engine.check_base import CRITICAL, Check
from code_review.standards import cobol_standards as std
```

```
TABLE = re.compile(
    r"\b(?:FROM|UPDATE|INSERT\s+INTO|DELETE\s+FROM)\s+(?P<table>[A-Z0-9_]+\b"
)
HOST_DOT = re.compile(r"(:?P<group>[A-Z0-9-]+\.)?(?P<field>[A-Z0-9-]+)")
```

```
class NamingAuthorityCheck(Check):
```

```
    CHECK_ID = "CHK-22"
    TITLE = "DB2 table and column naming authority"
    SEVERITY = CRITICAL
    ORDER = 220
```

```
    def relevant(self, ctx, view) -> bool:
        return view.has_code_token(std.SQL_BLOCK_START)
```

```
    def not_relevant_reason(self) -> str:
        return "Program contains no embedded SQL."
```

```
    def review(self, ctx, view, record):
        blocks = sql.sql_blocks(view)
        tables = {
            match.group("table")
            for block in blocks
            for match in TABLE.finditer(block.text)
        }
```

```
        mapped = meta.mapped_tables(ctx)
        declared = meta.dclgen_tables(ctx)
        authority = mapped | declared
```

```
        # ---- 01 every table comes from an authority -----
        if not std.ENFORCE_TABLE_AUTHORITY:
            record.skip(
                "01", "Every DB2 table comes from Sheet Mapping or DCLGEN",
                "Not enforced by the site standard.",
            )
        elif not authority:
            record.skip(
                "01", "Every DB2 table comes from Sheet Mapping or DCLGEN",
                "No Sheet Mapping or DCLGEN metadata supplied for this run.",
            )
        else:
            record.expect_all(
                "01", "Every DB2 table comes from Sheet Mapping or DCLGEN",
                sorted(table for table in tables if table not in authority),
            )
```

```
        # ---- 02 every table used has a DCLGEN -----
        if not declared:
            record.skip(
                "02", "Every DB2 table used has a DCLGEN",
```

```

        "No DCLGEN metadata supplied for this run.",
    )
else:
    record.expect_all(
        "02", "Every DB2 table used has a DCLGEN",
        sorted(table for table in tables if table not in declared),
    )

# ---- 03 every column comes from DCLGEN -----
if not std.ENFORCE_COLUMN_AUTHORITY:
    record.skip(
        "03", "Every DB2 column comes from DCLGEN",
        "Not enforced by the site standard.",
    )
elif not declared:
    record.skip(
        "03", "Every DB2 column comes from DCLGEN",
        "No DCLGEN metadata supplied for this run.",
    )
else:
    offenders = []
    for block in blocks:
        match = TABLE.search(block.text)
        if not match:
            continue
        table = match.group("table")
        available = meta.columns_for_table(ctx, table)
        if not available:
            continue
        used = {
            found.group("field").replace("-", "_")
            for found in HOST_DOT.finditer(block.text)
        }
        bad = sorted(used - available)
        if bad:
            offenders.append(f"{table}: {' '.join(bad)}")
    record.expect_all(
        "03", "Every DB2 column comes from DCLGEN", offenders,
    )

# ---- 04 mapped tables resolve to a DCLGEN -----
if not mapped or not declared:
    record.skip(
        "04", "Every Sheet Mapping table used resolves to a DCLGEN",
        "Sheet Mapping or DCLGEN metadata not supplied for this run.",
    )
else:
    used_mapped = tables & mapped
    record.expect_all(
        "04", "Every Sheet Mapping table used resolves to a DCLGEN",
        sorted(used_mapped - declared),
    )

# ---- 05 no DCL group without a table -----
prefix = std.DCLGEN_GROUP_PREFIX.upper()
groups = {
    match.group("group")
    for block in blocks
    for match in HOST_DOT.finditer(block.text)
}
if not declared:
    record.skip(
        "05", f"Every {std.DCLGEN_GROUP_PREFIX} group names a known "
        f"table",
        "No DCLGEN metadata supplied for this run.",
    )
else:
    record.expect_all(
        "05", f"Every {std.DCLGEN_GROUP_PREFIX} group names a known "
        f"table",
        sorted(
            group for group in groups
            if group.startswith(prefix)
            and group[len(prefix):] not in declared
        ),
    )

```

```
=====
FILE: code_review\checks\chk_23_host_variable_exactness.py
ENCODING: utf-8
=====
```

```
"""CHK-23 DCLGEN host variable name exactness."""

from __future__ import annotations

import re

from code_review.engine import metadata as meta, sql_blocks as sql
from code_review.engine.check_base import CRITICAL, Check
from code_review.standards import cobol_standards as std

HOST_DOT = re.compile(r"(:(?P<group>[A-Z0-9-]+)\.(?P<field>[A-Z0-9-]+)")
QUALIFIED = re.compile(
    r"\b(?:P<field>[A-Z][A-Z0-9-]*)\s+(?:OF|IN)\s+(?:P<group>[A-Z][A-Z0-9-]*)\b"
)

class HostVariableExactnessCheck(Check):
    CHECK_ID = "CHK-23"
    TITLE = "DCLGEN host variable name exactness"
    SEVERITY = CRITICAL
    ORDER = 230

    def relevant(self, ctx, view) -> bool:
        return bool(meta.dclgen_tables(ctx))

    def not_relevant_reason(self) -> str:
        return "No DCLGEN metadata supplied for this run."

    def review(self, ctx, view, record):
        prefix = std.DCLGEN_GROUP_PREFIX.upper()
        known_hosts = self._all_hosts(ctx)
        group_hosts = self._hosts_by_group(ctx)

        sql_refs = self._sql_references(view)
        cobol_refs = self._cobol_references(view, prefix)

        # ---- 01 SQL host names exist in DCLGEN -----
        if not std.ENFORCE_HOST_NAME_EXACTNESS:
            record.skip(
                "01", "Every SQL host variable exists in DCLGEN",
                "Not enforced by the site standard.",
            )
        elif not sql_refs:
            record.skip(
                "01", "Every SQL host variable exists in DCLGEN",
                "No qualified SQL host reference found.",
            )
        else:
            record.expect_all(
                "01", "Every SQL host variable exists in DCLGEN",
                sorted({
                    f"{group}.{field}"
                    for group, field in sql_refs
                    if not self._resolves(field, known_hosts)
                }),
            )

        # ---- 02 SQL host belongs to its group -----
        if not sql_refs:
            record.skip(
                "02", "Every SQL host variable belongs to its group",
                "No qualified SQL host reference found.",
            )
        else:
            record.expect_all(
                "02", "Every SQL host variable belongs to its group",
                sorted({
                    f"{group}.{field}"
                    for group, field in sql_refs
                }),
            )
```

```

        if self._resolves(field, known_hosts)
        and not self._resolves(
            field, group_hosts.get(group, set())
        )
    ),
)

# ---- 03 COBOL qualified names exist in DCLGEN -----
if not cobol_refs:
    record.skip(
        "03", f"Every {std.DCLGEN_GROUP_PREFIX} qualified reference "
            f"exists in DCLGEN",
        f"No {std.DCLGEN_GROUP_PREFIX} qualified reference found.",
    )
else:
    record.expect_all(
        "03", f"Every {std.DCLGEN_GROUP_PREFIX} qualified reference "
            f"exists in DCLGEN",
        sorted({
            f"{field} OF {group}"
            for group, field in cobol_refs
            if not self._resolves(field, known_hosts)
        }),
    )

# ---- 04 COBOL qualified name belongs to its group -----
if not cobol_refs:
    record.skip(
        "04", f"Every {std.DCLGEN_GROUP_PREFIX} qualified reference "
            f"belongs to its group",
        f"No {std.DCLGEN_GROUP_PREFIX} qualified reference found.",
    )
else:
    record.expect_all(
        "04", f"Every {std.DCLGEN_GROUP_PREFIX} qualified reference "
            f"belongs to its group",
        sorted({
            f"{field} OF {group}"
            for group, field in cobol_refs
            if self._resolves(field, known_hosts)
            and not self._resolves(
                field, group_hosts.get(group, set())
            )
        }),
    )

# ---- 05 every group is a real DCLGEN group -----
groups = {group for group, _ in sql_refs} | {
    group for group, _ in cobol_refs
}
tables = meta.dclgen_tables(ctx)
record.expect_all(
    "05", f"Every {std.DCLGEN_GROUP_PREFIX} group matches a DCLGEN "
        f"table",
    sorted(
        group for group in groups
        if group.startswith(prefix)
        and group[len(prefix):] not in tables
    ),
)

# ---- helpers -----
@staticmethod
def _resolves(field: str, hosts: set[str]) -> bool:
    """True when a name is a DCLGEN host or a VARCHAR subfield of one.

    DCLGEN declares a VARCHAR column as a group with two 49-level
    children. DclgenParser keeps only fields carrying a PIC or USAGE,
    so neither the group nor its children reach the review. A name
    ending in a known VARCHAR suffix whose stem is a real host is
    therefore valid, not a defect in the generated code.
    """
    if field in hosts:
        return True
    for suffix in std.VARCHAR_SUBFIELD_SUFFIXES:
        if field.endswith(suffix) and field[: -len(suffix)] in hosts:

```

```

        return True
    return False

    @staticmethod
    def _all_hosts(ctx) -> set[str]:
        out: set[str] = set()
        for table in meta.dclgen_tables(ctx):
            out.update(meta.hosts_for_table(ctx, table).values())
        return out

    @staticmethod
    def _hosts_by_group(ctx) -> dict[str, set[str]]:
        prefix = std.DCLGEN_GROUP_PREFIX.upper()
        out: dict[str, set[str]] = {}
        for table in meta.dclgen_tables(ctx):
            out[f"{prefix}{table}"] = set(
                meta.hosts_for_table(ctx, table).values()
            )
        return out

    @staticmethod
    def _sql_references(view) -> set[tuple[str, str]]:
        return {
            (match.group("group"), match.group("field"))
            for block in sql.sql_blocks(view)
            for match in HOST_DOT.finditer(block.text)
        }

    @staticmethod
    def _cobol_references(view, prefix: str) -> set[tuple[str, str]]:
        out: set[tuple[str, str]] = set()
        for line in view.code:
            for match in QUALIFIED.finditer(sql.norm(line.logical)):
                if match.group("group").startswith(prefix):
                    out.add((match.group("group"), match.group("field")))
        return out

```

```

=====
FILE: code_review\checks\chk_24_picture_and_length.py
ENCODING: utf-8
=====

```

```

"""CHK-24 Picture clause and length validation."""

from __future__ import annotations

import re

from code_review.engine import metadata as meta
from code_review.engine.check_base import MAJOR, Check
from code_review.standards import cobol_standards as std

DIGITS = re.compile(r"^\d+$")
DB2_SIZED = re.compile(
    r"^(?P<type>[A-Z]+)\s*(\s*(?P<size>\d+)", re.IGNORECASE
)

PIC_KEYWORDS = ("PICTURE", "PIC")
IS_KEYWORD = "IS"
LENGTH_SYMBOLS = "X9AZ"
IGNORED_SYMBOLS = "SV"

# DB2 types whose host length is fixed by the type itself.
FIXED_TYPE_LENGTHS = {
    "TIMESTAMP": 26,
    "DATE": 10,
    "TIME": 8,
}
CHARACTER_TYPES = ("CHAR", "CHARACTER", "GRAPHIC")

def _strip_picture_prefix(text: str) -> str:
    value = str(text or "").strip().upper()
    for keyword in PIC_KEYWORDS:
        if value.startswith(keyword):

```

```

        value = value[len(keyword):].strip()
        break
    if value.startswith(IS_KEYWORD):
        value = value[len(IS_KEYWORD):].strip()
    return value

def picture_length(picture: str) -> int:
    """Character length implied by a COBOL picture clause.

    Deliberately regex free. An explicit scan cannot silently drop a
    parenthesised repeat count, which is what makes PIC X(10) measure 1.

    S is a sign and V an implied decimal point, so neither occupies a
    stored position.
    """
    text = _strip_picture_prefix(picture).replace(" ", "").rstrip(".")
    if not text:
        return 0

    total = 0
    index = 0
    size = len(text)

    while index < size:
        symbol = text[index]

        if symbol in IGNORED_SYMBOLS:
            index += 1
            continue

        if symbol not in LENGTH_SYMBOLS:
            index += 1
            continue

        index += 1

        # Optional repeat count, for example X(10).
        if index < size and text[index] == "(":
            close = text.find(")", index)
            digits = text[index + 1:close] if close > index else ""
            if digits.isdigit():
                total += int(digits)
                index = close + 1
                continue

        total += 1

    return total

def picture_category(picture: str) -> str:
    """ALPHANUMERIC, NUMERIC or empty when undecidable."""
    text = _strip_picture_prefix(picture)
    if not text:
        return ""
    if "X" in text or "A" in text:
        return "ALPHANUMERIC"
    if "9" in text:
        return "NUMERIC"
    return ""

def db2_type_length(db2_type: str) -> int:
    """Host length implied by a DB2 type, 0 when not decidable.

    Fixed types are matched longest name first. TIMESTAMP starts with
    TIME, so insertion order would otherwise resolve a 26-character
    timestamp host to 8.
    """
    text = str(db2_type or "").strip().upper()
    if not text:
        return 0

    for name in sorted(FIXED_TYPE_LENGTHS, key=len, reverse=True):
        if text.startswith(name):

```

```

        return FIXED_TYPE_LENGTHS[name]

match = DB2_SIZED.match(text)
if match and match.group("type").startswith(CHARACTER_TYPES):
    return int(match.group("size"))
return 0

def is_varchar(db2_type: str) -> bool:
    """A group host whose pictures live on its 49-level children."""
    return str(db2_type or "").strip().upper().startswith(std.VARCHAR_TYPES)

def has_comparable_length(db2_type: str) -> bool:
    """True when stored bytes and picture positions mean the same thing."""
    return str(db2_type or "").strip().upper().startswith(
        std.COMPARABLE_LENGTH_TYPES
    )

class PictureAndLengthCheck(Check):
    CHECK_ID = "CHK-24"
    TITLE = "Picture clause and length validation"
    SEVERITY = MAJOR
    ORDER = 240

    def relevant(self, ctx, view) -> bool:
        return bool(ctx.dclgen_columns)

    def not_relevant_reason(self) -> str:
        return "No DCLGEN metadata supplied for this run."

    def review(self, ctx, view, record):
        columns = list(ctx.dclgen_columns or [])

        # ---- 01 every scalar column has a picture -----
        #
        # VARCHAR columns are exempt: the host is a group and groups
        # carry no PICTURE of their own.
        record.expect_all(
            "01", "Every scalar DCLGEN column declares a picture clause",
            sorted({
                f"{meta.field(c, 'table_name')}}."
                f"{meta.field(c, 'column_name')}"
                for c in columns
                if not meta.field(c, "cobol_picture")
                and not is_varchar(meta.field(c, "db2_type"))
            }),
        )

        # ---- 02 every DCLGEN column has a host name -----
        record.expect_all(
            "02", "Every DCLGEN column declares a host variable",
            sorted({
                f"{meta.field(c, 'table_name')}}."
                f"{meta.field(c, 'column_name')}"
                for c in columns
                if not meta.field(c, "cobol_host_name")
            }),
        )

        # ---- 03 picture clauses resolve to a length -----
        record.expect_all(
            "03", "Every picture clause resolves to a length",
            sorted({
                f"{meta.field(c, 'cobol_host_name')}} "
                f"PIC {meta.field(c, 'cobol_picture')}"
                for c in columns
                if meta.field(c, "cobol_picture")
                and picture_length(meta.field(c, "cobol_picture")) == 0
            }),
        )

        # ---- 04 Sheet Mapping length agrees with DCLGEN -----
        if not std.ENFORCE_PICTURE_LENGTH_MATCH:
            record.skip(

```

```

        "04", "Character column lengths agree between Sheet "
            "Mapping and DCLGEN",
        "Not enforced by the site standard.",
    )
elif not ctx.sheet_mapping_rows:
    record.skip(
        "04", "Character column lengths agree between Sheet "
            "Mapping and DCLGEN",
        "No Sheet Mapping metadata supplied for this run.",
    )
else:
    record.expect_all(
        "04", "Character column lengths agree between Sheet "
            "Mapping and DCLGEN",
        self._length_mismatches(ctx),
    )

# ---- 05 source picture agrees with DCLGEN picture -----
if not ctx.sheet_mapping_rows:
    record.skip(
        "05", "Source picture length agrees with the DCLGEN picture",
        "No Sheet Mapping metadata supplied for this run.",
    )
else:
    record.expect_all(
        "05", "Source picture length agrees with the DCLGEN picture",
        self._source_picture_mismatches(ctx),
    )

# ---- 06 category agreement -----
if not ctx.sheet_mapping_rows:
    record.skip(
        "06", "Source and DB2 field categories agree",
        "No Sheet Mapping metadata supplied for this run.",
    )
else:
    record.expect_all(
        "06", "Source and DB2 field categories agree",
        self._category_mismatches(ctx),
    )

# ---- 07 host variables used declare a picture -----
used = [
    column for column in columns
    if meta.field(column, "cobol_host_name")
    and view.has_code_token(meta.field(column, "cobol_host_name"))
    and not is_varchar(meta.field(column, "db2_type"))
]
if not used:
    record.skip(
        "07", "Every scalar host variable used declares a picture",
        "No scalar DCLGEN host variable referenced in the program.",
    )
else:
    record.expect_all(
        "07", "Every scalar host variable used declares a picture",
        sorted({
            meta.field(column, "cobol_host_name")
            for column in used
            if not meta.field(column, "cobol_picture")
        }),
    )

# ---- 08 picture agrees with its own DB2 type -----
#
# DCLGEN compared against DCLGEN, so a disagreement can only mean
# the picture was parsed incorrectly.
if not any(meta.field(c, "db2_type") for c in columns):
    record.skip(
        "08", "Every DCLGEN picture agrees with its DB2 type",
        "No DB2 type recorded in the supplied DCLGEN metadata.",
    )
else:
    record.expect_all(
        "08", "Every DCLGEN picture agrees with its DB2 type",
        self._type_mismatches(ctx),
    )

```



```

    )

# ---- helpers -----
@staticmethod
def _target(row) -> tuple[str, str]:
    table = meta.field(
        row, "new_db2_record", "cross_application_db2_table",
    )
    column = meta.field(
        row, "new_db2_field_name", "cross_application_db2_field_name",
    )
    return table, column

@staticmethod
def _db2_type_for(ctx, table: str, column: str) -> str:
    wanted_table = meta.norm_name(table)
    wanted_column = meta.norm_name(column)
    for item in ctx.dclgen_columns or []:
        if meta.field(item, "table_name") != wanted_table:
            continue
        if meta.field(item, "column_name") != wanted_column:
            continue
        return meta.field(item, "db2_type")
    return ""

@classmethod
def _length_mismatches(cls, ctx) -> list[str]:
    """Compared only where bytes and positions mean the same thing."""
    out: list[str] = []
    for row in ctx.sheet_mapping_rows or []:
        table, column = cls._target(row)
        declared = meta.field(row, "length_of_field_bytes")
        if not table or not column or not DIGITS.match(declared):
            continue
        if not has_comparable_length(cls._db2_type_for(ctx, table, column)):
            continue
        actual = picture_length(meta.picture_for(ctx, table, column))
        if not actual:
            continue
        if abs(actual - int(declared)) > std.PICTURE_LENGTH_TOLERANCE:
            out.append(
                f"{table}.{column}: Sheet Mapping {declared}, "
                f"DCLGEN {actual}"
            )
    return sorted(set(out))

@classmethod
def _source_picture_mismatches(cls, ctx) -> list[str]:
    out: list[str] = []
    for row in ctx.sheet_mapping_rows or []:
        table, column = cls._target(row)
        if not table or not column:
            continue
        if not has_comparable_length(cls._db2_type_for(ctx, table, column)):
            continue
        source_pic = meta.field(
            row, "idms_pic_clause", "reference_field_pic_clause",
        )
        if not source_pic:
            continue
        source_length = picture_length(source_pic)
        target_length = picture_length(
            meta.picture_for(ctx, table, column)
        )
        if not source_length or not target_length:
            continue
        if abs(source_length - target_length) > (
            std.PICTURE_LENGTH_TOLERANCE
        ):
            out.append(
                f"{table}.{column}: source {source_pic} "
                f"({source_length}) vs DCLGEN ({target_length})"
            )
    return sorted(set(out))

@classmethod

```

```

def _category_mismatches(cls, ctx) -> list[str]:
    out: list[str] = []
    for row in ctx.sheet_mapping_rows or []:
        table, column = cls._target(row)
        if not table or not column:
            continue
        source_pic = meta.field(
            row, "idms_pic_clause", "reference_field_pic_clause",
        )
        source = picture_category(source_pic)
        target = picture_category(meta.picture_for(ctx, table, column))
        if not source or not target:
            continue
        if source != target:
            out.append(f"{table}.{column}: {source} to {target}")
    return sorted(set(out))

@staticmethod
def _type_mismatches(ctx) -> list[str]:
    out: list[str] = []
    for column in ctx.dclgen_columns or []:
        picture = meta.field(column, "cobol_picture")
        db2_type = meta.field(column, "db2_type")
        if not picture or not db2_type:
            continue
        if is_varchar(db2_type):
            continue
        expected = db2_type_length(db2_type)
        if not expected:
            continue
        actual = picture_length(picture)
        if not actual:
            continue
        if abs(actual - expected) > std.PICTURE_LENGTH_TOLERANCE:
            out.append(
                f"{meta.field(column, 'table_name')}. "
                f"{meta.field(column, 'column_name')}: "
                f"{db2_type} implies {expected}, "
                f"PIC {picture} gives {actual}"
            )
    return sorted(set(out))

```

```

=====
FILE: code_review\checks\chk_25_copybook_and_record_usage.py
ENCODING: utf-8
=====

```

```

"""CHK-25 Copybook and record usage validation."""

```

```

from __future__ import annotations

```

```

import re

```

```

from code_review.engine import metadata as meta, sql_blocks as sql
from code_review.engine.check_base import MAJOR, Check
from code_review.standards import cobol_standards as std

```

```

QUALIFIED = re.compile(
    r"\b(?:P<field>[A-Z][A-Z0-9-]*)\s+(?:OF|IN)\s+(?:P<group>[A-Z][A-Z0-9-]*)\b"
)
COPY = re.compile(r"^COPY\s+(?:P<name>[A-Z0-9][A-Z0-9-]*)\b")
DATA_ITEM = re.compile(
    r"^(?:P<level>0[1-9]|[1-4][0-9]|66|77|88)\s+(?:P<name>[A-Z][A-Z0-9-]*)\b"
)
LITERAL = re.compile(r"'[^']*'|\\"[^\"]*"")

```

```

def without_literals(text: str) -> str:
    """Blank out quoted literals.

```

```

    COBOL qualification only applies to identifiers. Display text such as
    DISPLAY 'DATE OF ERROR' is prose, not a field reference, and must not
    be parsed as one.
    """

```

```

    return LITERAL.sub(" ", str(text or ""))

```

```

class CopybookAndRecordUsageCheck(Check):
    CHECK_ID = "CHK-25"
    TITLE = "Copybook and record usage validation"
    SEVERITY = MAJOR
    ORDER = 250

    def relevant(self, ctx, view) -> bool:
        return bool(ctx.sheet_mapping_rows) or bool(ctx.copybook_fields)

    def not_relevant_reason(self) -> str:
        return "No Sheet Mapping or copybook metadata supplied for this run."

    def review(self, ctx, view, record):
        prefix = std.DCLGEN_GROUP_PREFIX.upper()
        records = meta.mapped_records(ctx)
        qualifiers = self._qualifiers(view)
        copies = self._copies(view)
        copybook = self._copybook_fields(ctx)
        declared = self._declared_names(view)

        # ---- 01 no IDMS record survives as a qualifier -----
        if not std.ENFORCE_NO_IDMS_QUALIFIER:
            record.skip(
                "01", "No IDMS record name survives as a field qualifier",
                "Not enforced by the site standard.",
            )
        elif not records:
            record.skip(
                "01", "No IDMS record name survives as a field qualifier",
                "No Sheet Mapping record metadata supplied for this run.",
            )
        else:
            record.expect_all(
                "01", "No IDMS record name survives as a field qualifier",
                sorted({
                    f"{field} OF {group}"
                    for field, group in qualifiers
                    if group in records
                }),
            )

        # ---- 02 every qualifier resolves to a declared group -----
        if not qualifiers:
            record.skip(
                "02",
                "Every qualified reference resolves to a declared group",
                "Program contains no qualified field reference.",
            )
        else:
            known = (
                {f"{prefix}{table}" for table in meta.dclgen_tables(ctx)}
                | copybook
                | set(copies)
                | declared
            )
            record.expect_all(
                "02",
                "Every qualified reference resolves to a declared group",
                sorted({
                    f"{field} OF {group}"
                    for field, group in qualifiers
                    if group not in known
                }),
            )

        # ---- 03 every mapped record is converted or documented -----
        if not records:
            record.skip(
                "03",
                "Every referenced IDMS record was converted or documented",
                "No Sheet Mapping record metadata supplied for this run.",
            )
        elif not ctx.has_source:
            record.skip(

```

```

        "03",
        "Every referenced IDMS record was converted or documented",
        "No source program supplied for comparison.",
    )
else:
    record.expect_all(
        "03",
        "Every referenced IDMS record was converted or documented",
        self._undocumented(ctx, view, records),
    )

# ---- 04 no unmapped record marker remains -----
record.expect_none(
    "04", "No unmapped business record marker remains",
    [
        line for line in view.comments
        if any(m in line.logical for m in std.MISSING_MAPPING_MARKERS)
        and not any(
            hint in line.logical
            for hint in std.RESTART_CONTROL_HINTS
        )
    ],
)

# ---- 05 copybook-qualified fields exist -----
if not copybook:
    record.skip(
        "05", "Every copybook-qualified field exists in the copybook",
        "No copybook metadata supplied for this run.",
    )
elif not qualifiers:
    record.skip(
        "05", "Every copybook-qualified field exists in the copybook",
        "Program contains no qualified field reference.",
    )
else:
    record.expect_all(
        "05", "Every copybook-qualified field exists in the copybook",
        sorted({
            f"{field} OF {group}"
            for field, group in qualifiers
            if group in copybook
            and not group.startswith(prefix)
            and field not in copybook
            and field not in declared
        })),
    )

# ---- 06 every COPY member is referenced -----
if not copies:
    record.skip(
        "06", "Every COPY member is referenced",
        "Program declares no COPY statement.",
    )
else:
    referenced = {group for _field, group in qualifiers}
    record.expect_all(
        "06", "Every COPY member is referenced",
        sorted(
            name for name in copies
            if name not in referenced
            and not view.has_code_token(name)
        ),
    )

# ---- helpers -----
@staticmethod
def _qualifiers(view) -> set[tuple[str, str]]:
    out: set[tuple[str, str]] = set()
    for line in view.code:
        body = without_literals(sql.norm(line.logical))
        for match in QUALIFIED.finditer(body):
            out.add((match.group("field"), match.group("group")))
    return out

@staticmethod

```

```

def _copies(view) -> set[str]:
    out: set[str] = set()
    for line in view.code:
        match = COPY.match(sql.norm(line.logical))
        if match:
            out.add(match.group("name"))
    return out

@staticmethod
def _copybook_fields(ctx) -> set[str]:
    return {
        meta.field(item, "name")
        for item in (ctx.copybook_fields or [])
        if meta.field(item, "name")
    }

@staticmethod
def _declared_names(view) -> set[str]:
    """Every data name the program declares in its own DATA DIVISION."""
    out: set[str] = set()
    for line in view.code:
        match = DATA_ITEM.match(sql.norm(line.logical))
        if match:
            out.add(match.group("name"))
    return out

@staticmethod
def _table_variants(table: str) -> set[str]:
    """Every equivalent DB2 table spelling, TB and TV included."""
    text = str(table or "").strip().upper()
    if not text:
        return set()
    out = {text}
    for source, target in std.DB2_TABLE_SUFFIX_EQUIVALENTS:
        if text.endswith(source):
            out.add(f"{text[: -len(source)]}{target}")
    return out

@classmethod
def _undocumented(cls, ctx, view, records: set[str]) -> list[str]:
    source = sql.norm(ctx.source_cobol)
    comments = " ".join(line.logical for line in view.comments)

    out: list[str] = []
    for name in sorted(name for name in records if name in source):
        table = meta.table_for_record(ctx, name)
        if any(
            view.has_code_token(item)
            for item in cls._table_variants(table)
        ):
            continue
        if name in comments:
            continue
        out.append(f"{name} (table {table or 'unmapped'})")
    return out

```

```

=====
FILE: code_review\engine\__init__.py
ENCODING: utf-8
=====

```

```

from code_review.engine.context import ReviewContext
from code_review.engine.models import CheckResult, Outcome, ReviewResult
from code_review.engine.reviewer import Reviewer, blockers, is_accepted

__all__ = [
    "ReviewContext", "Reviewer", "ReviewResult", "CheckResult",
    "Outcome", "is_accepted", "blockers",
]

```

```
=====
FILE: code_review\engine\check_base.py
ENCODING: utf-8
=====
```

```
"""Base class and recorder for a single code review check."""
```

```
from __future__ import annotations
```

```
from code_review.engine.cobol_view import CobolView, Line
from code_review.engine.context import ReviewContext
from code_review.engine.models import Criterion, Finding, Outcome
```

```
CRITICAL = "Critical"
```

```
MAJOR = "Major"
```

```
MINOR = "Minor"
```

```
BOTH = "BOTH"
```

```
RETRIEVAL = "RETRIEVAL"
```

```
UPDATE = "UPDATE"
```

```
def _finding(item) -> Finding:
    if isinstance(item, Line):
        return Finding(line_number=item.number, text=item.body.rstrip())
    if isinstance(item, Finding):
        return item
    return Finding(text=str(item))
```

```
class Recorder:
```

```
    """Collects criterion outcomes for one check."""
```

```
    def __init__(self, check_id: str, limit: int = 10) -> None:
```

```
        self.check_id = check_id
```

```
        self.limit = limit
```

```
        self.criteria: list[Criterion] = []
```

```
    def _add(self, num, desc, outcome, note="", findings=None) -> None:
```

```
        self.criteria.append(
```

```
            Criterion(
```

```
                criterion_id=f"{self.check_id}.{num}",
```

```
                description=desc,
```

```
                outcome=outcome,
```

```
                note=note,
```

```
                findings=[_finding(f) for f in list(findings or [])[: self.limit]],
```

```
            )
```

```
        )
```

```
    def ok(self, num, desc) -> None:
```

```
        self._add(num, desc, Outcome.PASS)
```

```
    def fail(self, num, desc, note="", findings=None) -> None:
```

```
        self._add(num, desc, Outcome.FAIL, note=note, findings=findings)
```

```
    def skip(self, num, desc, reason="") -> None:
```

```
        self._add(num, desc, Outcome.SKIPPED, note=reason)
```

```
    def expect(self, num, desc, condition, note="", findings=None) -> None:
```

```
        if condition:
```

```
            self.ok(num, desc)
```

```
        else:
```

```
            self.fail(num, desc, note=note, findings=findings)
```

```
    def expect_none(self, num, desc, offenders, note="") -> None:
```

```
        items = list(offenders or [])
```

```
        self.expect(
```

```
            num, desc,
```

```
            condition=not items,
```

```
            note=note or f"{len(items)} occurrence(s)",
```

```
            findings=items,
```

```
        )
```

```
    def expect_all(self, num, desc, missing, note="") -> None:
```

```
        items = list(missing or [])
```

```
        self.expect(
```

```

        num, desc,
        condition=not items,
        note=note or ("missing: " + ", ".join(str(i) for i in items[:10])),
        findings=items,
    )

```

```

class Check:
    """Base class for every code review check."""

    CHECK_ID: str = ""
    TITLE: str = ""
    SEVERITY: str = MAJOR
    APPLIES_TO: str = BOTH
    NEEDS: tuple[str, ...] = ("converted_cobol",)
    ORDER: int = 0

    def applies(self, ctx: ReviewContext) -> bool:
        scope = str(self.APPLIES_TO).upper()
        return scope == BOTH or scope == str(ctx.program_kind).upper()

    def relevant(self, ctx: ReviewContext, view: CobolView) -> bool:
        """Override when the check only makes sense for some programs."""
        return True

    def not_relevant_reason(self) -> str:
        return "Program does not contain the construct this check inspects."

    def review(self, ctx: ReviewContext, view: CobolView, record: Recorder) -> None:
        raise NotImplementedError

```

```

=====
FILE: code_review\engine\cobol_view.py
ENCODING: utf-8
=====

```

```

"""Read-only fixed-format COBOL inspection.

Layout: cols 1-6 left sequence, col 7 indicator,
        cols 8-72 body, cols 73-80 right sequence.
"""

```

```

from __future__ import annotations

```

```

import re
from dataclasses import dataclass

```

```

WIDTH = 80
IND = 6
BODY_A, BODY_B = 7, 72
RSEQ_A, RSEQ_B = 72, 80
COMMENTS = ("*", "/")
VALID_INDICATORS = (" ", "*", "/", "D", "-")
AREA_B_MIN = 4

```

```

_SEQ_IN_BODY = re.compile(r"^\s*\d{6}\s")

```

```

@dataclass(frozen=True)
class Line:
    number: int
    raw: str

    @property
    def is_fixed(self) -> bool:
        return (
            len(self.raw) >= WIDTH
            and self.raw[0:6].isdigit()
            and self.raw[RSEQ_A:RSEQ_B].isdigit()
        )

    @property
    def left_sequence(self) -> str:
        return self.raw[0:6] if self.is_fixed else ""

```

```

@property
def right_sequence(self) -> str:
    return self.raw[RSEQ_A:RSEQ_B] if self.is_fixed else ""

@property
def indicator(self) -> str:
    return self.raw[IND] if len(self.raw) > IND else " "

@property
def body(self) -> str:
    return self.raw[BODY_A:BODY_B] if self.is_fixed else self.raw

@property
def logical(self) -> str:
    return self.body.strip().upper()

@property
def indent(self) -> int:
    b = self.body
    return len(b) - len(b.lstrip())

@property
def is_blank(self) -> bool:
    return not self.body.strip()

@property
def is_comment(self) -> bool:
    return self.indicator in COMMENTS or self.body.lstrip().startswith(COMMENTS)

@property
def is_code(self) -> bool:
    return not self.is_blank and not self.is_comment

@property
def has_sequence_in_body(self) -> bool:
    return bool(_SEQ_IN_BODY.match(self.body))

@property
def overflow(self) -> str:
    return self.raw[BODY_B:RSEQ_A]

class CobolView:
    def __init__(self, text: str) -> None:
        clean = str(text or "").replace("\r\n", "\n").replace("\r", "\n")
        self.lines = [Line(i, v) for i, v in enumerate(clean.split("\n"), start=1)]

    @property
    def populated(self) -> list[Line]:
        return [l for l in self.lines if l.raw.strip()]

    @property
    def code(self) -> list[Line]:
        return [l for l in self.lines if l.is_code]

    @property
    def comments(self) -> list[Line]:
        return [l for l in self.lines if l.is_comment]

    @property
    def code_text(self) -> str:
        return "\n".join(l.logical for l in self.code)

    @property
    def all_text(self) -> str:
        return "\n".join(l.logical for l in self.lines)

    @property
    def is_empty(self) -> bool:
        return not any(l.raw.strip() for l in self.lines)

    # ---- queries -----
    def has_code_token(self, token: str) -> bool:
        return str(token).upper() in self.code_text

    def has_token(self, token: str) -> bool:

```



```

        return str(token).upper() in self.all_text

def code_with(self, token: str) -> list[Line]:
    t = str(token).upper()
    return [l for l in self.code if t in l.logical]

def starting_with(self, prefix: str) -> list[Line]:
    p = str(prefix).upper()
    return [l for l in self.code if l.logical.startswith(p)]

def matching(self, pattern: re.Pattern) -> list[Line]:
    return [l for l in self.code if pattern.match(l.logical)]

def searching(self, pattern: re.Pattern) -> list[Line]:
    return [l for l in self.code if pattern.search(l.logical)]

def exact(self, text: str) -> list[Line]:
    t = str(text).strip().upper()
    return [l for l in self.code if l.logical == t]

def count_of(self, text: str) -> int:
    return self.all_text.count(str(text).upper())

def first_group(self, pattern: re.Pattern, group: str) -> str:
    for l in self.code:
        m = pattern.match(l.logical)
        if m:
            return m.group(group)
    return ""

def paragraph_exists(self, name: str) -> bool:
    return bool(self.exact(f"{str(name).upper()}."))

# ---- sequence -----
def sequence(self, right: bool = False) -> list[int]:
    out = []
    for l in self.lines:
        if not l.is_fixed:
            continue
        v = l.right_sequence if right else l.left_sequence
        if v.isdigit():
            out.append(int(v))
    return out

@staticmethod
def constant_step(values: list[int]) -> int | None:
    if len(values) < 2:
        return None
    step = values[1] - values[0]
    if step <= 0:
        return None
    return step if all(b - a == step for a, b in zip(values, values[1:])) else None

```

```

=====
FILE: code_review\engine\context.py
ENCODING: utf-8
=====

```

"""The single input object for a code review run. Data only.

Everything a check is allowed to see arrives here. Checks never read files,
never call the converter and never reach into session state.

Dependency direction is one-way: this module imports nothing from the
converter, and the converter never imports from code_review.
"""

```
from __future__ import annotations
```

```
from dataclasses import dataclass, field
```

```

# -----
# Program kinds
#

```

```

# Values are UPPERCASE because Check.applies() compares them against
# check_base.RETRIEVAL / UPDATE / BOTH using str(...).upper().
# -----
KIND_RETRIEVAL = "RETRIEVAL"
KIND_UPDATE = "UPDATE"
KIND_UNKNOWN = "UNKNOWN"

VALID_KINDS: tuple[str, ...] = (KIND_RETRIEVAL, KIND_UPDATE, KIND_UNKNOWN)

def normalise_kind(value: str) -> str:
    """Returns a known kind, or KIND_UNKNOWN when the value is not recognised."""
    kind = str(value or "").strip().upper()
    return kind if kind in VALID_KINDS else KIND_UNKNOWN

@dataclass
class ReviewContext:
    """Immutable-by-convention input for one program review."""

    program_name: str = ""
    program_kind: str = KIND_UNKNOWN
    target_program_id: str = ""
    source_file: str = ""
    converted_cobol: str = ""
    source_cobol: str = ""
    sheet_mapping_rows: list = field(default_factory=list)
    dclgen_columns: list = field(default_factory=list)
    copybook_fields: list = field(default_factory=list)

    def __post_init__(self) -> None:
        self.program_name = str(self.program_name or "")
        self.program_kind = normalise_kind(self.program_kind)
        self.target_program_id = str(self.target_program_id or "")
        self.source_file = str(self.source_file or "")
        self.converted_cobol = str(self.converted_cobol or "")
        self.source_cobol = str(self.source_cobol or "")
        self.sheet_mapping_rows = list(self.sheet_mapping_rows or [])
        self.dclgen_columns = list(self.dclgen_columns or [])
        self.copybook_fields = list(self.copybook_fields or [])

    # ---- availability -----
    def has(self, name: str) -> bool:
        """True when the named input was supplied and is non-empty."""
        if not hasattr(self, str(name)):
            return False
        value = getattr(self, str(name))
        if isinstance(value, str):
            return bool(value.strip())
        if isinstance(value, (list, tuple, dict, set)):
            return bool(value)
        return value is not None

    def missing(self, names) -> list[str]:
        """Inputs a check declared in NEEDS that were not supplied.

        Reviewer._one() calls this and reports BLOCKED instead of FAIL.
        """
        return [str(n) for n in (names or ()) if not self.has(str(n))]

    # ---- convenience -----
    @property
    def is_retrieval(self) -> bool:
        return self.program_kind == KIND_RETRIEVAL

    @property
    def is_update(self) -> bool:
        return self.program_kind == KIND_UPDATE

    @property
    def has_source(self) -> bool:
        return bool(self.source_cobol.strip())

    @property
    def has_metadata(self) -> bool:
        return bool(

```

```

        self.sheet_mapping_rows or self.dclgen_columns or self.copybook_fields
    )

    def describe(self) -> str:
        return (
            f"{self.program_name or '(unnamed)'} "
            f"[{self.program_kind}] -> {self.source_file or '(in memory)'}"
        )

```

```

__all__ = [
    "ReviewContext",
    "KIND_RETRIEVAL",
    "KIND_UPDATE",
    "KIND_UNKNOWN",
    "VALID_KINDS",
    "normalise_kind",
]

```

```

=====
FILE: code_review\engine\cursors.py
ENCODING: utf-8
=====

```

```

"""Shared cursor parsing for checks.

```

```

Recognises the two shapes the converter emits:

```

```

EXEC SQL
    DECLARE <cursor> CURSOR WITH HOLD FOR
    SELECT ... FROM <table> ... FOR READ ONLY
END-EXEC.

710-OPEN-<cursor>.
720-FETCH-<cursor>.
730-CLOSE-<cursor>.

```

```

Read-only. Knows nothing about standards or wording.
"""

```

```

from __future__ import annotations

```

```

import re
from dataclasses import dataclass

```

```

from code_review.engine import sql_blocks as sql

```

```

DECLARE = re.compile(
    r"\bDECLARE\s+(?P<cursor>[A-Z][A-Z0-9-]*)\s+CURSOR\b"
)
PARAGRAPH = re.compile(
    r"^(?P<number>\d{3,6})-"
    r"(?P<operation>OPEN|FETCH|CLOSE)-"
    r"(?P<cursor>[A-Z0-9][A-Z0-9-]*)\.$"
)
PERFORM = re.compile(
    r"^PERFORM\s+(?P<name>[A-Z0-9][A-Z0-9-]*)\b(?P<rest>.*)$"
)
CURSOR_VERB = re.compile(
    r"\b(?P<operation>OPEN|FETCH|CLOSE)\s+(?P<cursor>[A-Z][A-Z0-9-]*)\b"
)

```

```

# COBOL scope terminators that look like a paragraph header because they
# are a single word followed by a period. These are language facts, not
# site standards, so they live here rather than in standards/. A caller
# may pass a fuller list through stop_words.

```

```

SCOPE_TERMINATORS = frozenset({
    "CONTINUE", "END-EXEC", "END-EVALUATE", "END-IF", "END-PERFORM",
    "END-READ", "END-SEARCH", "END-STRING", "END-WRITE", "EXIT",
    "GOBACK", "STOP",
})

```

```

@dataclass(frozen=True)
class CursorParagraph:

```

```

"""One generated cursor paragraph header."""

number: int
operation: str
cursor: str
line_number: int
name: str

def render(self) -> str:
    return str(self)

def __str__(self) -> str:
    """One readable evidence line.

    The Recorder turns anything that is not a Line or Finding into
    Finding(text=str(item)), so without this the whole dataclass repr
    would land in the report.
    """
    return f"line {self.line_number:>5} : {self.name}"

def declared_cursors(view) -> dict[str, sql.SqlBlock]:
    """Cursor name -> its DECLARE block."""
    out: dict[str, sql.SqlBlock] = {}
    for block in sql.sql_blocks(view):
        if block.verb != "DECLARE":
            continue
        match = DECLARE.search(block.text)
        if match:
            out.setdefault(match.group("cursor").upper(), block)
    return out

def declare_blocks(view) -> list[sql.SqlBlock]:
    return [b for b in sql.sql_blocks(view) if b.verb == "DECLARE"]

def cursor_paragraphs(view) -> list[CursorParagraph]:
    """Every generated cursor paragraph header, in source order."""
    out: list[CursorParagraph] = []
    for line in view.code:
        logical = sql.norm(line.logical)
        match = PARAGRAPH.match(logical)
        if not match:
            continue
        out.append(CursorParagraph(
            number=int(match.group("number")),
            operation=match.group("operation").upper(),
            cursor=match.group("cursor").upper(),
            line_number=line.number,
            name=logical.rstrip("."),
        ))
    return out

def paragraph_sets(view) -> dict[str, dict[str, CursorParagraph]]:
    """Cursor name -> {OPEN: para, FETCH: para, CLOSE: para}."""
    out: dict[str, dict[str, CursorParagraph]] = {}
    for para in cursor_paragraphs(view):
        out.setdefault(para.cursor, {})[para.operation] = para
    return out

def performed_names(view) -> dict[str, list]:
    """Performed paragraph name -> the lines that perform it."""
    out: dict[str, list] = {}
    for line in view.code:
        match = PERFORM.match(sql.norm(line.logical))
        if match:
            out.setdefault(match.group("name").upper(), []).append(line)
    return out

def cursors_used(view, operation: str) -> set[str]:
    """Cursor names appearing in an EXEC SQL OPEN / FETCH / CLOSE."""
    wanted = str(operation or "").upper()

```

```

out: set[str] = set()
for block in sql.sql_blocks(view):
    if block.verb != wanted:
        continue
    match = CURSOR_VERB.search(block.text)
    if match:
        out.add(match.group("cursor").upper())
return out

def is_paragraph_header(logical: str, stop_words) -> bool:
    """A single word plus a period that is not a scope terminator.

    END-EXEC. and CONTINUE. look identical to a paragraph header at the
    token level. Treating them as one truncates every paragraph body at
    the first EXEC SQL block.
    """
    text = str(logical or "").strip()
    if not text.endswith(".") or " " in text:
        return False
    name = text[:-1]
    if not name:
        return False
    if name in SCOPE_TERMINATORS:
        return False
    return name not in (stop_words or frozenset())

def paragraph_body(view, para: CursorParagraph, stop_words=None) -> list[str]:
    """Normalised lines of a paragraph, up to the next paragraph header."""
    words = stop_words if stop_words is not None else frozenset()
    code = view.code
    start = next(
        (
            index for index, line in enumerate(code)
            if line.number == para.line_number
        ),
        -1,
    )
    if start < 0:
        return []

    out: list[str] = []
    for line in code[start + 1:]:
        logical = sql.norm(line.logical)
        if PARAGRAPH.match(logical):
            break
        if is_paragraph_header(logical, words):
            break
        out.append(logical)
    return out

```

```

=====
FILE: code_review\engine\metadata.py
ENCODING: utf-8
=====

```

"""Read-only classification of the metadata carried on a ReviewContext.

Sheet Mapping is the authority for DB2 table and column names.
DCLGEN is the authority for host variable spelling and picture clauses.

Every function tolerates missing metadata by returning an empty result, so
callers can skip a criterion rather than fail it. Prefix tuples are passed
in by the caller, keeping this module free of site standards.
"""

```
from __future__ import annotations
```

```
import re
```

```

WS_RUN = re.compile(r"\s+")
KEY_FOREIGN = "FOREIGN"
KEY_FK = " FK "

```

```

def norm_name(value) -> str:
    """Uppercase, trimmed, whitespace collapsed."""
    return WS_RUN.sub(" ", str(value or "").strip().upper())

def field(row, *names, default: str = "") -> str:
    """First populated attribute from a metadata row."""
    for name in names:
        value = norm_name(getattr(row, name, ""))
        if value:
            return value
    return default

# ---- DCLGEN -----
def dclgen_tables(ctx) -> set[str]:
    return {
        field(column, "table_name")
        for column in (ctx.dclgen_columns or [])
        if field(column, "table_name")
    }

def columns_for_table(ctx, table: str) -> set[str]:
    wanted = norm_name(table)
    return {
        field(column, "column_name")
        for column in (ctx.dclgen_columns or [])
        if field(column, "table_name") == wanted
        and field(column, "column_name")
    }

def hosts_for_table(ctx, table: str) -> dict[str, str]:
    """DB2 column -> COBOL host variable, for one table."""
    wanted = norm_name(table)
    out: dict[str, str] = {}
    for column in ctx.dclgen_columns or []:
        if field(column, "table_name") != wanted:
            continue
        name = field(column, "column_name")
        host = field(column, "cobol_host_name")
        if name and host:
            out[name] = host
    return out

def columns_with_prefix(ctx, table: str, prefixes) -> set[str]:
    """DCLGEN columns of a table whose name starts with any prefix."""
    wanted = tuple(norm_name(p) for p in (prefixes or ())) if norm_name(p)
    if not wanted:
        return set()
    return {
        name for name in columns_for_table(ctx, table)
        if name.startswith(wanted)
    }

def picture_for(ctx, table: str, column: str) -> str:
    wanted_table = norm_name(table)
    wanted_column = norm_name(column)
    for item in ctx.dclgen_columns or []:
        if field(item, "table_name") != wanted_table:
            continue
        if field(item, "column_name") != wanted_column:
            continue
        return field(item, "cobol_picture")
    return ""

# ---- Sheet Mapping -----
def rows_for_record(ctx, record: str) -> list:
    wanted = norm_name(record)
    return [
        row for row in (ctx.sheet_mapping_rows or [])

```

```

        if field(row, "cobol_record_idms") == wanted
    ]

def mapped_records(ctx) -> set[str]:
    return {
        field(row, "cobol_record_idms")
        for row in (ctx.sheet_mapping_rows or [])
        if field(row, "cobol_record_idms")
    }

def table_for_record(ctx, record: str) -> str:
    for row in rows_for_record(ctx, record):
        table = field(row, "new_db2_record", "cross_application_db2_table")
        if table:
            return table
    return ""

def mapped_tables(ctx) -> set[str]:
    return {
        field(row, "new_db2_record", "cross_application_db2_table")
        for row in (ctx.sheet_mapping_rows or [])
        if field(row, "new_db2_record", "cross_application_db2_table")
    }

def mapped_columns(ctx) -> set[str]:
    return {
        field(row, "new_db2_field_name", "cross_application_db2_field_name")
        for row in (ctx.sheet_mapping_rows or [])
        if field(row, "new_db2_field_name", "cross_application_db2_field_name")
    }

def _column_of(row) -> str:
    return field(
        row, "new_db2_field_name", "cross_application_db2_field_name",
    )

def _is_foreign(row) -> bool:
    text = " ".join([field(row, "db2_key"), field(row, "relation")])
    return KEY_FOREIGN in text or KEY_FK in f" {text} "

def primary_key_columns(ctx, record: str, key_markers) -> list[str]:
    """Composite PK and CALC key columns. FOREIGN columns are excluded."""
    markers = tuple(norm_name(m) for m in (key_markers or ()) if norm_name(m))
    out: list[str] = []
    seen: set[str] = set()

    for row in rows_for_record(ctx, record):
        column = _column_of(row)
        if not column or column in seen:
            continue
        if _is_foreign(row):
            continue
        text = " ".join([field(row, "idms_key"), field(row, "db2_key")])
        if not any(marker in text for marker in markers):
            continue
        seen.add(column)
        out.append(column)
    return out

def foreign_key_columns(ctx, record: str) -> list[str]:
    out: list[str] = []
    seen: set[str] = set()
    for row in rows_for_record(ctx, record):
        if not _is_foreign(row):
            continue
        column = _column_of(row)
        if column and column not in seen:
            seen.add(column)

```

```

        out.append(column)
    return out

def all_primary_key_columns(ctx, key_markers) -> set[str]:
    out: set[str] = set()
    for record in mapped_records(ctx):
        out.update(primary_key_columns(ctx, record, key_markers))
    return out

def all_foreign_key_columns(ctx) -> set[str]:
    out: set[str] = set()
    for record in mapped_records(ctx):
        out.update(foreign_key_columns(ctx, record))
    return out

=====
FILE: code_review\engine\models.py
ENCODING: utf-8
=====

"""Code review result models. Data only.

No file access, no printing, no wording. Every counter the report needs is
derived here so writers stay presentation-only.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from datetime import datetime
from enum import Enum

class Outcome(str, Enum):
    PASS = "PASS"
    FAIL = "FAIL"
    SKIPPED = "SKIPPED"
    BLOCKED = "BLOCKED"

# Severity of an outcome when several must be reduced to one.
_RANK = {
    Outcome.PASS: 0,
    Outcome.SKIPPED: 1,
    Outcome.BLOCKED: 2,
    Outcome.FAIL: 3,
}

def worst(items: list[Outcome]) -> Outcome:
    """Strict reduction: the most severe outcome in the list.

    Kept for callers that genuinely want strictness. CheckResult.outcome
    deliberately does NOT use this, because an informational SKIPPED must
    not mask passing criteria.
    """
    return max(items, key=lambda o: _RANK[o]) if items else Outcome.SKIPPED

@dataclass
class Finding:
    line_number: int = 0
    text: str = ""
    message: str = ""

    def render(self) -> str:
        if self.line_number:
            return f"line {self.line_number:>5} : {self.text}"
        return self.text or self.message

@dataclass
class Criterion:

```



```

criterion_id: str
description: str
outcome: Outcome
note: str = ""
findings: list[Finding] = field(default_factory=list)

```

```

@dataclass
class CheckResult:
    check_id: str
    title: str
    severity: str
    criteria: list[Criterion] = field(default_factory=list)

    @property
    def outcome(self) -> Outcome:
        """A check is SKIPPED only when nothing was actually scored.

        Precedence: FAIL, then BLOCKED, then PASS, then SKIPPED.

        A check with eight passing criteria and one informational skip is
        a PASS. Ranking SKIPPED above PASS made the report under-state
        quality, because a site standard that is deliberately not enforced
        turned a whole check grey.
        """
        outcomes = [c.outcome for c in self.criteria]
        if not outcomes:
            return Outcome.SKIPPED
        if Outcome.FAIL in outcomes:
            return Outcome.FAIL
        if Outcome.BLOCKED in outcomes:
            return Outcome.BLOCKED
        if Outcome.PASS in outcomes:
            return Outcome.PASS
        return Outcome.SKIPPED

    def _count(self, outcome: Outcome) -> int:
        return sum(1 for c in self.criteria if c.outcome is outcome)

    @property
    def passed(self) -> int:
        return self._count(Outcome.PASS)

    @property
    def failed(self) -> int:
        return self._count(Outcome.FAIL)

    @property
    def skipped(self) -> int:
        return self._count(Outcome.SKIPPED)

    @property
    def blocked(self) -> int:
        return self._count(Outcome.BLOCKED)

    @property
    def total(self) -> int:
        return len(self.criteria)

    @property
    def failures(self) -> list[Criterion]:
        return [c for c in self.criteria if c.outcome is Outcome.FAIL]

    @property
    def skips(self) -> list[Criterion]:
        return [c for c in self.criteria if c.outcome is Outcome.SKIPPED]

    @property
    def note(self) -> str:
        for c in self.criteria:
            if c.note:
                return c.note
        return ""

```

```

@dataclass

```

```

class ReviewResult:
    program_name: str = ""
    program_kind: str = ""
    source_file: str = ""
    reviewed_at: datetime = field(default_factory=datetime.now)
    checks: list[CheckResult] = field(default_factory=list)

    # ---- check level -----
    def _by(self, outcome: Outcome) -> list[CheckResult]:
        return [c for c in self.checks if c.outcome is outcome]

    @property
    def total(self) -> int:
        return len(self.checks)

    @property
    def passed(self) -> int:
        return len(self._by(Outcome.PASS))

    @property
    def failed(self) -> int:
        return len(self._by(Outcome.FAIL))

    @property
    def skipped(self) -> int:
        return len(self._by(Outcome.SKIPPED))

    @property
    def blocked(self) -> int:
        return len(self._by(Outcome.BLOCKED))

    @property
    def failed_checks(self) -> list[CheckResult]:
        return self._by(Outcome.FAIL)

    def failed_at(self, severity: str) -> list[CheckResult]:
        s = str(severity or "").upper()
        return [c for c in self.failed_checks if c.severity.upper() == s]

    # ---- criterion level -----
    @property
    def criteria_passed(self) -> int:
        return sum(c.passed for c in self.checks)

    @property
    def criteria_failed(self) -> int:
        return sum(c.failed for c in self.checks)

    @property
    def criteria_skipped(self) -> int:
        return sum(c.skipped for c in self.checks)

    @property
    def criteria_blocked(self) -> int:
        return sum(c.blocked for c in self.checks)

    @property
    def criteria_total(self) -> int:
        return sum(c.total for c in self.checks)

=====
FILE: code_review\engine\registry.py
ENCODING: utf-8
=====

"""Auto-discovers checks. Drop a module in code_review/checks to add one."""

from __future__ import annotations

import importlib
import inspect
import pkgutil

from code_review.engine.check_base import Check

```

```

try:
    from code_review.standards.pipeline import CHECK_ORDER, DISABLED_CHECKS, ENABLED_CHECKS
except ImportError:
    CHECK_ORDER, DISABLED_CHECKS, ENABLED_CHECKS = (), frozenset(), ()

PACKAGE = "code_review.checks"

def _classes() -> list[type[Check]]:
    package = importlib.import_module(PACKAGE)
    found = []
    for info in pkgutil.iter_modules(list(package.__path__)):
        if info.name.startswith("_"):
            continue
        module = importlib.import_module(f"{PACKAGE}.{info.name}")
        for _, member in inspect.getmembers(module, inspect.isclass):
            if (
                issubclass(member, Check)
                and member is not Check
                and member.__module__ == module.__name__
                and member.CHECK_ID
            ):
                found.append(member)
    return found

def _enabled(check_id: str) -> bool:
    if check_id in DISABLED_CHECKS:
        return False
    return check_id in ENABLED_CHECKS if ENABLED_CHECKS else True

def _key(check: Check):
    if CHECK_ORDER and check.CHECK_ID in CHECK_ORDER:
        return (CHECK_ORDER.index(check.CHECK_ID), check.CHECK_ID)
    return (check.ORDER or 9999, check.CHECK_ID)

def load_checks() -> list[Check]:
    unique = {c.CHECK_ID: c() for c in _classes() if _enabled(c.CHECK_ID)}
    return sorted(unique.values(), key=_key)

def check_ids() -> list[str]:
    return [c.CHECK_ID for c in load_checks()]

```

```

=====
FILE: code_review\engine\reviewer.py
ENCODING: utf-8
=====

```

```

"""Runs the check pipeline against one ReviewContext."""

```

```

from __future__ import annotations

```

```

from code_review.engine.check_base import Check, Recorder
from code_review.engine.cobol_view import CobolView
from code_review.engine.context import ReviewContext
from code_review.engine.models import CheckResult, Criterion, Outcome, ReviewResult
from code_review.engine.registry import load_checks

```

```

try:
    from code_review.standards.pipeline import BLOCKING_SEVERITIES, EVIDENCE_LIMIT
except ImportError:
    BLOCKING_SEVERITIES, EVIDENCE_LIMIT = ("Critical",), 10

```

```

class Reviewer:
    def __init__(self, checks: list[Check] | None = None) -> None:
        self.checks = checks if checks is not None else load_checks()

    def review(self, ctx: ReviewContext) -> ReviewResult:
        view = CobolView(ctx.converted_cobol)
        result = ReviewResult(
            program_name=ctx.program_name,

```

```

        program_kind=ctx.program_kind,
        source_file=ctx.source_file,
    )
    for check in self.checks:
        result.checks.append(self._one(check, ctx, view))
    return result

def _one(self, check: Check, ctx: ReviewContext, view: CobolView) -> CheckResult:
    out = CheckResult(check.CHECK_ID, check.TITLE, check.SEVERITY)

    def single(outcome: Outcome, note: str) -> CheckResult:
        out.criteria.append(
            Criterion(f"{check.CHECK_ID}.00", check.TITLE, outcome, note=note)
        )
        return out

    if not check.applies(ctx):
        return single(Outcome.SKIPPED, f"Applies to {check.APPLIES_TO} programs only.")

    missing = ctx.missing(check.NEEDS)
    if missing:
        return single(Outcome.BLOCKED, "Not supplied: " + ", ".join(missing))

    if view.is_empty:
        return single(Outcome.BLOCKED, "Converted COBOL is empty.")

    try:
        if not check.relevant(ctx, view):
            return single(Outcome.SKIPPED, check.not_relevant_reason())
        recorder = Recorder(check.CHECK_ID, limit=EVIDENCE_LIMIT)
        check.review(ctx, view, recorder)
        out.criteria.extend(recorder.criteria)
    except Exception as exc: # noqa: BLE001
        return single(Outcome.BLOCKED, f"Check raised an error: {exc}")

    if not out.criteria:
        return single(Outcome.SKIPPED, "No criteria recorded.")
    return out

def is_accepted(result: ReviewResult) -> bool:
    return not any(result.failed_at(s) for s in BLOCKING_SEVERITIES)

def blockers(result: ReviewResult) -> list[CheckResult]:
    seen = {}
    for severity in BLOCKING_SEVERITIES:
        for check in result.failed_at(severity):
            seen[check.check_id] = check
    return [seen[k] for k in sorted(seen)]

=====
FILE: code_review\engine\sql_blocks.py
ENCODING: utf-8
=====

"""Shared EXEC SQL block parsing for checks.

Fixed-format COBOL splits one logical SQL statement across many physical
lines, and pads with whitespace. Checks that reason about SQL need a
normalised, whole-statement view, so that logic lives here once instead of
being copied into every check.

Read-only. Knows nothing about standards or wording.
"""

from __future__ import annotations

import re
from dataclasses import dataclass, field

WS_RUN = re.compile(r"\s+")
EXEC_SQL = "EXEC SQL"
END_EXEC = "END-EXEC"
VERB = re.compile(r"^(EXEC\s+SQL\s+(?P<verb>[A-Z-]+\s+)\s+)"
```

```
EVIDENCE_WIDTH = 90
```

```
def norm(text: str) -> str:
    """Uppercase, trimmed, internal whitespace runs collapsed."""
    return WS_RUN.sub(" ", str(text or "").strip().upper())

def logical_statements(lines: list[str]) -> list[str]:
    """Logical statements, with EXEC SQL ... END-EXEC. joined into one entry.

    Order is preserved one-to-one for non-SQL lines, so positional
    reasoning stays valid.
    """
    out: list[str] = []
    buffer: list[str] = []
    inside = False

    for raw in lines:
        line = norm(raw)

        if not inside and line.startswith(EXEC_SQL):
            inside = True
            buffer = [line]
            if END_EXEC in line:
                out.append(" ".join(buffer))
                inside, buffer = False, []
            continue

        if inside:
            buffer.append(line)
            if END_EXEC in line:
                out.append(" ".join(buffer))
                inside, buffer = False, []
            continue

        out.append(line)

    if buffer:
        out.append(" ".join(buffer))
    return out
```

```
@dataclass(frozen=True)
class SqlBlock:
    """One EXEC SQL ... END-EXEC. statement."""

    first_index: int
    last_index: int
    text: str
    lines: list = field(default_factory=list)

    @property
    def verb(self) -> str:
        match = VERB.match(self.text)
        return match.group("verb") if match else ""

    @property
    def line_number(self) -> int:
        return self.lines[0].number if self.lines else 0

    def has(self, keyword: str) -> bool:
        return f" {norm(keyword)} " in f" {self.text} "

    def render(self) -> str:
        return str(self)

    def __str__(self) -> str:
        """One readable evidence line.

        The Recorder turns anything that is not a Line or Finding into
        Finding(text=str(item)), so without this a block would print its
        whole dataclass repr into the report.
        """
        body = self.text
        if len(body) > EVIDENCE_WIDTH:
```

```

        body = f"{body[:EVIDENCE_WIDTH]}..."
        return f"line {self.line_number:>5} : {body}"

def sql_blocks(view) -> list[SqlBlock]:
    """Every EXEC SQL block in the code lines of a CobolView."""
    code = view.code
    out: list[SqlBlock] = []
    buffer: list = []
    start = -1
    inside = False

    for index, line in enumerate(code):
        logical = norm(line.logical)

        if not inside and logical.startswith(EXEC_SQL):
            inside = True
            start = index
            buffer = [line]
            if END_EXEC in logical:
                out.append(_build(start, index, buffer))
                inside, buffer, start = False, [], -1
                continue

            if inside:
                buffer.append(line)
                if END_EXEC in logical:
                    out.append(_build(start, index, buffer))
                    inside, buffer, start = False, [], -1

        if buffer and start >= 0:
            out.append(_build(start, len(code) - 1, buffer))
    return out

def _build(first: int, last: int, lines: list) -> SqlBlock:
    text = norm(" ".join(line.logical for line in lines))
    return SqlBlock(
        first_index=first, last_index=last, text=text, lines=list(lines),
    )

def targets_restart(block: SqlBlock, hints) -> bool:
    """True when a block reads or writes a restart or control table.

    Restart SQL is owned by CHK-19, CHK-20 and CHK-21. The business
    conversion checks must exclude it, or one deferred decision is
    reported several times and blocks delivery twice over.
    """
    text = block.text
    return any(str(hint or "").upper() in text for hint in (hints or ()))

def lines_before(view, block: SqlBlock, count: int) -> list[str]:
    """Normalised code lines immediately before a block."""
    code = view.code
    start = max(0, block.first_index - int(count))
    return [norm(line.logical) for line in code[start:block.first_index]]

def lines_after(view, block: SqlBlock, count: int) -> list[str]:
    """Normalised code lines immediately after a block."""
    code = view.code
    end = min(len(code), block.last_index + 1 + int(count))
    return [norm(line.logical) for line in code[block.last_index + 1:end]]

=====
FILE: code_review\engine\sql_location.py
ENCODING: utf-8
=====

# LOCATION: code_review/engine/sql_location.py
# ACTION: CREATE NEW FILE

"""Shared SQL-LOCATION detection for CHK-05, CHK-09, CHK-10 and CHK-20.
```

The manual reference program moves the bare paragraph number:

```
MOVE 710                TO SQL-LOCATION
```

The quoted paragraph-name form is equally valid and is what the restart template emits:

```
MOVE '710-OPEN-DZBEFFC1' TO SQL-LOCATION
```

Both must be recognised. Testing only for a "MOVE '" prefix makes the manual reference itself fail CHK-09.05, CHK-10.05 and CHK-20.07, which is by definition a false positive.

Read-only. Knows nothing about programs, cursors, tables or host variables.
"""

```
from __future__ import annotations
```

```
import re
```

```
from code_review.engine import sql_blocks as sql
from code_review.standards import cobol_standards as std
```

```
# MOVE <bare-number|'quoted-literal'> TO SQL-LOCATION
```

```
# The terminator is optional: the site standard closes a paragraph with a
# lone period line, so the MOVE itself carries no period.
```

```
MOVE_SQL_LOCATION = re.compile(
    r"^MOVE\s+(?:'[\^']*'|\d+)\s+TO\s+"
    + re.escape(std.SQL_LOCATION_FIELD)
    + r"\s*\.??"
)
```

```
def is_location_move(line: str) -> bool:
```

```
    """True when the logical line sets SQL-LOCATION in an accepted form."""
    return bool(MOVE_SQL_LOCATION.match(sql.norm(line)))
```

```
def set_before(view, block, lookback: int | None = None) -> bool:
```

```
    """True when SQL-LOCATION is set within the look-back window of a block."""
    window = sql.lines_before(
        view,
        block,
        std.SQL_LOCATION_LOOKBACK if lookback is None else lookback,
    )
    return any(is_location_move(line) for line in window)
```

```
=====
FILE: code_review\README.md
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: code_review\report\__init__.py
ENCODING: utf-8
=====
```

```
from code_review.report import csv_writer, text_writer
```

```
__all__ = ["text_writer", "csv_writer"]
```

```
=====
FILE: code_review\report\csv_writer.py
ENCODING: utf-8
=====
```

```
"""Writes a ReviewResult as CSV.
```

```
write()          one row per criterion, for a single program
write_summary()  one row per program, for a batch run
```

```
Presentation only. Headers come from code_review.standards wording.
"""
```

```
from __future__ import annotations
```

```
import csv
from pathlib import Path
```

```
from code_review.engine.models import ReviewResult
from code_review.engine.reviewer import is_accepted
from code_review.standards import wording as w
```

```
TEXT_ENCODING = "utf-8"
FILE_SUFFIX = ".csv"
```

```
def rows(result: ReviewResult) -> list[list[str]]:
    """One row per criterion."""
    when = result.reviewed_at.strftime(w.CSV_TIMESTAMP_FORMAT)
    out: list[list[str]] = []
    for check in result.checks:
        for crit in check.criteria:
            out.append([
                result.program_name,
                when,
                result.source_file,
                check.check_id,
                check.severity,
                check.title,
                crit.criterion_id,
                crit.description,
                crit.outcome.value,
                crit.note,
                w.CSV_LINE_SEPARATOR.join(
                    str(f.line_number)
                    for f in crit.findings
                    if f.line_number
                ),
            ])
    return out
```

```
def summary_rows(results: list[ReviewResult]) -> list[list[str]]:
    """One row per reviewed program."""
    out: list[list[str]] = []
    for result in results:
        out.append([
            result.program_name,
            result.program_kind,
            result.reviewed_at.strftime(w.CSV_TIMESTAMP_FORMAT),
            result.source_file,
            result.passed,
            result.failed,
            result.skipped,
            result.blocked,
            w.ACCEPTED if is_accepted(result) else w.REJECTED,
        ])
    return out
```

```
def write(result: ReviewResult, folder: Path, stem: str) -> Path:
    folder.mkdir(parents=True, exist_ok=True)
    path = Path(folder) / f"{stem}.{FILE_SUFFIX}"
    with path.open("w", newline="", encoding=TEXT_ENCODING) as handle:
        writer = csv.writer(handle)
        writer.writerow(w.CSV_HEADER)
        writer.writerows(rows(result))
    return path
```

```
def write_summary(
    results: list[ReviewResult],
    folder: Path,
    stem: str,
) -> Path:
```



```

folder.mkdir(parents=True, exist_ok=True)
path = Path(folder) / f"{stem}{FILE_SUFFIX}"
with path.open("w", newline="", encoding=TEXT_ENCODING) as handle:
    writer = csv.writer(handle)
    writer.writerow(w.CSV_SUMMARY_HEADER)
    writer.writerows(summary_rows(results))
return path

```

```

=====
FILE: code_review\report\text_writer.py
ENCODING: utf-8
=====

```

```

"""Renders a ReviewResult as the human-readable Code Review Report.

```

```

Presentation only. Every string comes from code_review.standards wording
and every number comes from code_review.engine.models.
"""

```

```

from __future__ import annotations

```

```

from pathlib import Path

```

```

from code_review.engine.models import Outcome, ReviewResult
from code_review.engine.reviewer import blockers, is_accepted
from code_review.standards import wording as w

```

```

TEXT_ENCODING = "utf-8"
FILE_SUFFIX = ".txt"

```

```

def render(result: ReviewResult) -> str:
    out: list[str] = []
    add = out.append

```

```

    # ---- header -----
    add(w.REPORT_TITLE)
    add(w.RULE)
    add(w.F_PROGRAM.format(value=result.program_name or w.VALUE_UNNAMED))
    add(w.F_KIND.format(value=result.program_kind or w.VALUE_UNKNOWN))
    add(w.F_SOURCE.format(value=result.source_file or w.VALUE_IN_MEMORY))
    add(w.F_WHEN.format(
        value=result.reviewed_at.strftime(w.CSV_TIMESTAMP_FORMAT),
    ))
    add(w.F_TOTALS.format(
        passed=result.passed,
        failed=result.failed,
        skipped=result.skipped,
        blocked=result.blocked,
    ))
    add(w.F_CRITERIA.format(
        passed=result.criteria_passed,
        failed=result.criteria_failed,
        skipped=result.criteria_skipped,
        blocked=result.criteria_blocked,
    ))
    add(w.F_RESULT.format(
        value=w.ACCEPTED if is_accepted(result) else w.REJECTED,
    ))

```

```

    blocking = blockers(result)
    if blocking:
        add(w.F_BLOCKING.format(
            value=", ".join(c.check_id for c in blocking),
        ))

```

```

    # ---- detail -----
    add("")
    add(w.H_DETAIL)
    add(w.RULE)

```

```

    for check in result.checks:
        add(w.F_CHECK.format(
            symbol=w.SYMBOL[check.outcome.value],
            check_id=check.check_id,

```

```

        severity=check.severity,
        title=check.title,
    ))
    for crit in check.criteria:
        if crit.outcome is Outcome.PASS:
            continue
        add(w.F_CRIT.format(
            criterion_id=crit.criterion_id,
            description=crit.description,
        ))
        if crit.note:
            add(w.F_NOTE.format(note=crit.note))
        for finding in crit.findings:
            add(w.F_FIND.format(finding=finding.render()))

# ---- failures -----
add("")
add(w.H_FAILURES)
add(w.RULE)

failed = result.failed_checks
if not failed:
    add(w.NO_FAILURES)
else:
    for check in failed:
        add(w.F_FAIL_CHECK.format(
            check_id=check.check_id,
            severity=check.severity,
            title=check.title,
        ))
        for crit in check.failures:
            add(w.F_FAIL_CRIT.format(
                criterion_id=crit.criterion_id,
                description=crit.description,
            ))
            if crit.note:
                add(w.F_FAIL_NOTE.format(note=crit.note))

add("")
return "\n".join(out)

def write(result: ReviewResult, folder: Path, stem: str) -> Path:
    folder.mkdir(parents=True, exist_ok=True)
    path = Path(folder) / f"{stem}{FILE_SUFFIX}"
    path.write_text(render(result), encoding=TEXT_ENCODING)
    return path

=====
FILE: code_review\runner\__init__.py
ENCODING: utf-8
=====

"""Command-line entry points for the code review."""

=====
FILE: code_review\runner\_bootstrap.py
ENCODING: utf-8
=====

"""Shared constants and small file helpers for the code review runners.

No sys.path work happens in this module.

Each runner performs its own inline sys.path bootstrap at the very top of
its file, before any code_review import, because running

    python code_review\runner\review_file.py

puts only code_review\runner on sys.path. Doing it here as well would be
a second, competing mechanism that runs too late to help.
"""

from __future__ import annotations

```

```

from datetime import datetime
from pathlib import Path

# ---- Locations -----
PROJECT_ROOT = Path(__file__).resolve().parents[2]
SRC_DIR = PROJECT_ROOT / "src"
REPORT_DIR = PROJECT_ROOT / "code_review" / "report" / "output"

# ---- Formats -----
TIMESTAMP_FORMAT = "%d-%m-%Y_%H%M%S"
COBOL_GLOB = "*.cbl"
TEXT_ENCODING = "utf-8"

# ---- Exit codes -----
EXIT_ACCEPTED = 0
EXIT_REJECTED = 1
EXIT_ERROR = 2

def stamp() -> str:
    """Timestamp used in report file names."""
    return datetime.now().strftime(TIMESTAMP_FORMAT)

def read_text(path: Path) -> str:
    """Read a COBOL or text file without ever raising on bad bytes."""
    return Path(path).read_text(encoding=TEXT_ENCODING, errors="replace")

=====
FILE: code_review\runner\review_file.py
ENCODING: utf-8
=====

"""Review a generated DB2 COBOL file, or every .cbl in a folder.

Needs no Sheet Mapping, DCLGEN or Copybook. Checks that require metadata
report SKIPPED instead of failing.

python code_review\runner\review_file.py --path C:\\S\\S-Input\\Output\\Retrieval\\VMDZ4420.cbl
python code_review\runner\review_file.py --folder C:\\S\\S-Input\\Output\\Retrieval
python code_review\runner\review_file.py --folder C:\\S\\S-Input\\Output\\Retrieval --latest
python code_review\runner\review_file.py --path out.cbl --source in.txt --kind UPDATE

Exit codes: 0 accepted, 1 rejected, 2 error or nothing to review.
"""

from __future__ import annotations

# -----
# Path bootstrap. Plain inline code, and it MUST run before any code_review
# import, because "python code_review\runner\review_file.py" puts only
# code_review\runner on sys.path.
# -----
import sys
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[2]
SRC_DIR = PROJECT_ROOT / "src"

for _c in (PROJECT_ROOT, SRC_DIR):
    if str(_c) not in sys.path:
        sys.path.insert(0, str(_c))
# -----

import argparse # noqa: E402

from code_review.engine.context import ( # noqa: E402
    KIND_RETRIEVAL,
    KIND_UNKNOWN,
    KIND_UPDATE,
    ReviewContext,
)
from code_review.engine.viewer import ( # noqa: E402
    Reviewer,

```

```

        blockers,
        is_accepted,
    )
from code_review.report import csv_writer, text_writer # noqa: E402
from code_review.report.text_writer import render # noqa: E402
from code_review.runner._bootstrap import ( # noqa: E402
    COBOL_GLOB,
    EXIT_ACCEPTED,
    EXIT_ERROR,
    EXIT_REJECTED,
    REPORT_DIR,
    read_text,
    stamp,
)
from code_review.standards import wording as w # noqa: E402

VALID_KINDS = (KIND_RETRIEVAL, KIND_UPDATE, KIND_UNKNOWN)

class NothingToReview(Exception):
    """The folder exists but holds no generated .cbl."""

    def __init__(self, folder: Path) -> None:
        self.folder = Path(folder)
        super().__init__(w.MSG_NO_OUTPUT_FOUND.format(folder=self.folder))

def parse_arguments() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Code review a generated DB2 COBOL file."
    )
    source = parser.add_mutually_exclusive_group(required=True)
    source.add_argument("--path", help="a single .cbl file to review")
    source.add_argument("--folder", help="a folder of .cbl files to review")
    parser.add_argument(
        "--latest",
        action="store_true",
        help="with --folder, review only the newest file",
    )
    parser.add_argument(
        "--source",
        help="original IDMS COBOL source file, optional",
    )
    parser.add_argument(
        "--kind",
        default=KIND_UNKNOWN,
        help="RETRIEVAL, UPDATE or UNKNOWN (default UNKNOWN)",
    )
    parser.add_argument(
        "--report",
        default=str(REPORT_DIR),
        help="folder to write the reports into",
    )
    parser.add_argument(
        "--quiet",
        action="store_true",
        help="print one summary line per file instead of the full report",
    )
    return parser.parse_args()

def resolve_targets(args: argparse.Namespace) -> list[Path]:
    if args.path:
        path = Path(args.path)
        if not path.is_file():
            raise FileNotFoundError(w.MSG_FILE_NOT_FOUND.format(path=path))
        return [path]

    folder = Path(args.folder)
    if not folder.exists():
        raise FileNotFoundError(w.MSG_FOLDER_NOT_FOUND.format(folder=folder))
    if not folder.is_dir():
        raise FileNotFoundError(w.MSG_PATH_NOT_FOLDER.format(folder=folder))

    files = sorted(
        (item for item in folder.glob(COBOL_GLOB) if item.is_file()),
    )

```

```

        key=lambda item: item.stat().st_mtime,
        reverse=True,
    )
    if not files:
        raise NothingToReview(folder)

    if args.latest:
        return files[:1]
    return sorted(files, key=lambda item: item.name.lower())

def resolve_kind(value: str) -> str:
    kind = str(value or "").strip().upper()
    return kind if kind in VALID_KINDS else KIND_UNKNOWN

def build_context(cbl_path: Path, kind: str, source_text: str) -> ReviewContext:
    return ReviewContext(
        program_name=cbl_path.stem,
        program_kind=kind,
        target_program_id="",
        source_file=cbl_path.name,
        converted_cobol=read_text(cbl_path),
        source_cobol=source_text,
    )

def print_banner(args, targets: list[Path], kind: str, report_dir: Path) -> None:
    print("")
    print(w.H_RUN_FILE)
    print(w.RULE)
    if args.folder:
        print(w.MSG_REVIEWED_FOLDER.format(folder=Path(args.folder)))
    print(w.MSG_FILE_COUNT.format(count=len(targets)))
    print(w.MSG_PROGRAM_KIND.format(kind=kind))
    print(w.MSG_REPORT_FOLDER.format(folder=report_dir))
    print("")

def print_nothing_to_review(exc: NothingToReview) -> int:
    print("")
    print(w.MSG_NO_OUTPUT_TITLE)
    print(w.RULE)
    print(w.MSG_NO_OUTPUT_FOUND.format(folder=exc.folder))
    print("")
    print(w.MSG_NO_OUTPUT_HINT)
    print("")
    return EXIT_ERROR

def run() -> int:
    args = parse_arguments()

    report_dir = Path(args.report)
    report_dir.mkdir(parents=True, exist_ok=True)

    kind = resolve_kind(args.kind)

    source_text = ""
    if args.source:
        source_path = Path(args.source)
        if not source_path.is_file():
            raise FileNotFoundError(
                w.MSG_SOURCE_NOT_FOUND.format(path=source_path)
            )
        source_text = read_text(source_path)

    targets = resolve_targets(args)

    reviewer = Reviewer()
    when = stamp()
    results = []

    print_banner(args, targets, kind, report_dir)

    for cbl_path in targets:

```

```

result = reviewer.review(build_context(cbl_path, kind, source_text))
results.append(result)

stem = f"{cbl_path.stem}_{when}"
text_writer.write(result, report_dir, stem)
csv_writer.write(result, report_dir, stem)

if args.quiet:
    ids = ", ".join(c.check_id for c in blockers(result))
    print(w.MSG_QUIET_LINE.format(
        verdict=w.ACCEPTED if is_accepted(result) else w.REJECTED,
        name=cbl_path.name,
        suffix=w.MSG_QUIET_BLOCKING.format(ids=ids) if ids else "",
    ))
else:
    print(render(result))

if len(results) > 1:
    csv_writer.write_summary(results, report_dir, f"summary_{when}")

accepted = sum(1 for item in results if is_accepted(item))
rejected = len(results) - accepted

print(w.RULE)
print(w.MSG_SUMMARY.format(
    reviewed=len(results), accepted=accepted, rejected=rejected,
))
print(w.MSG_REPORTS_WRITTEN.format(folder=report_dir))
print("")

return EXIT_ACCEPTED if rejected == 0 else EXIT_REJECTED

def main() -> None:
    try:
        raise SystemExit(run())
    except SystemExit:
        raise
    except NothingToReview as exc:
        raise SystemExit(print_nothing_to_review(exc))
    except FileNotFoundError as exc:
        print(w.MSG_ERROR.format(message=exc))
        raise SystemExit(EXIT_ERROR)
    except Exception as exc: # noqa: BLE001
        print(w.MSG_ERROR.format(message=exc))
        raise SystemExit(EXIT_ERROR)

if __name__ == "__main__":
    main()

=====
FILE: code_review\runner\review_retrieval.py
ENCODING: utf-8
=====

"""Convert every retrieval program, then code review each generated file.

Reuses the existing retrieval conversion pipeline unchanged. The converted
text is reviewed straight from the file that was just written, so each
program is always matched with its own output.

    set PYTHONPATH=src
    python code_review\runner\review_retrieval.py
    python code_review\runner\review_retrieval.py --quiet

Exit codes: 0 accepted, 1 rejected, 2 error.
"""

from __future__ import annotations

# -----
# Path bootstrap. Must run before any project or code_review import.
# -----
import sys

```

```

from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[2]
SRC_DIR = PROJECT_ROOT / "src"

for _candidate in (PROJECT_ROOT, SRC_DIR):
    if str(_candidate) not in sys.path:
        sys.path.insert(0, str(_candidate))
# -----

import argparse # noqa: E402

from code_review.engine.context import KIND_RETRIEVAL, ReviewContext # noqa: E402
from code_review.engine.reviewer import Reviewer, blockers, is_accepted # noqa: E402
from code_review.report import csv_writer, text_writer # noqa: E402
from code_review.report.text_writer import render # noqa: E402
from code_review.runner._bootstrap import ( # noqa: E402
    EXIT_ACCEPTED,
    EXIT_ERROR,
    EXIT_REJECTED,
    REPORT_DIR,
    read_text,
    stamp,
)

from config.path_settings import ( # noqa: E402
    DEFAULT_RETRIEVAL_OUTPUT_DIR,
    DEFAULT_RETRIEVAL_PROGRAM_DIR,
    LOGS_DIR,
)

from idms_db2_phase2.infrastructure.logger_factory import ( # noqa: E402
    LoggerFactory,
)

from idms_db2_phase2.testing.retrieval.retrieval_input_loader import ( # noqa: E402
    load_shared_inputs,
)

from idms_db2_phase2.testing.retrieval.retrieval_input_selector import ( # noqa: E402
    selected_program_paths,
    validate_inputs,
)

from idms_db2_phase2.testing.retrieval.retrieval_program_converter import ( # noqa: E402
    convert_one_program,
    resolve_target_program_id,
)

LOGGER_NAME = "idms_db2_retrieval_code_review"

def parse_arguments() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Convert retrieval programs, then code review the output."
    )
    parser.add_argument(
        "--report",
        default=str(REPORT_DIR),
        help="folder to write the reports into",
    )
    parser.add_argument(
        "--quiet",
        action="store_true",
        help="print one summary line per program instead of the full report",
    )
    return parser.parse_args()

def build_context(
    program_path: Path,
    output_path: Path,
    sheet_rows: list,
    dclgen_columns: list,
    copybook_fields: list,
) -> ReviewContext:
    source_text = read_text(program_path)
    return ReviewContext(
        program_name=program_path.stem,
        program_kind=KIND_RETRIEVAL,
    )

```

```

        target_program_id=resolve_target_program_id(source_text),
        source_file=Path(output_path).name,
        converted_cobol=read_text(output_path),
        source_cobol=source_text,
        sheet_mapping_rows=sheet_rows,
        dclgen_columns=dclgen_columns,
        copybook_fields=copybook_fields,
    )

def run() -> int:
    args = parse_arguments()

    report_dir = Path(args.report)
    report_dir.mkdir(parents=True, exist_ok=True)

    validate_inputs()

    logger = LoggerFactory.create_logger(name=LOGGER_NAME, logs_dir=LOGS_DIR)

    sheet_rows, dclgen_columns, copybook_fields, diagnostics = load_shared_inputs(
        logger
    )

    programs = selected_program_paths()
    if not programs:
        raise FileNotFoundError(
            f"No retrieval programs found in: {DEFAULT_RETRIEVAL_PROGRAM_DIR}"
        )

    reviewer = Reviewer()
    when = stamp()
    results = []
    failed_conversions: list[str] = []

    print("")
    print("Retrieval Code Review")
    print("-" * 74)
    print(f"Program Folder : {DEFAULT_RETRIEVAL_PROGRAM_DIR}")
    print(f"Program Count : {len(programs)}")
    print(f"Output Folder : {DEFAULT_RETRIEVAL_OUTPUT_DIR}")
    print(f"Report Folder : {report_dir}")
    print("")

    for program_path in programs:
        try:
            output_path = convert_one_program(
                program_path=program_path,
                sheet_rows=sheet_rows,
                dclgen_columns=dclgen_columns,
                copybook_fields=copybook_fields,
                shared_diagnostics=diagnostics,
                output_dir=DEFAULT_RETRIEVAL_OUTPUT_DIR,
                project_root=PROJECT_ROOT,
                src_dir=SRC_DIR,
                logger=logger,
            )
        except Exception as exc: # noqa: BLE001
            message = f"Conversion failed for {program_path.name}: {exc}"
            print(f"ERROR: {message}")
            logger.exception(message)
            failed_conversions.append(program_path.name)
            continue

        context = build_context(
            program_path=program_path,
            output_path=Path(output_path),
            sheet_rows=sheet_rows,
            dclgen_columns=dclgen_columns,
            copybook_fields=copybook_fields,
        )

        result = reviewer.review(context)
        results.append(result)

        stem = f"{program_path.stem}_{when}"

```



```

text_writer.write(result, report_dir, stem)
csv_writer.write(result, report_dir, stem)

if args.quiet:
    verdict = "ACCEPTED" if is_accepted(result) else "REJECTED"
    failing = ", ".join(c.check_id for c in blockers(result))
    suffix = f" blocking: {failing}" if failing else ""
    print(f"{verdict:<9} {program_path.stem}{suffix}")
else:
    print(render(result))

if len(results) > 1:
    csv_writer.write_summary(results, report_dir, f"summary_retrieval_{when}")

accepted = sum(1 for item in results if is_accepted(item))
rejected = len(results) - accepted

print("-" * 74)
print(f"Reviewed : {len(results)}    Accepted : {accepted}    Rejected : {rejected}")
if failed_conversions:
    print(f"Conversion failed : {' '.join(failed_conversions)}")
print(f"Reports written to: {report_dir}")
print("")

if failed_conversions:
    return EXIT_ERROR
return EXIT_ACCEPTED if rejected == 0 else EXIT_REJECTED

def main() -> None:
    try:
        raise SystemExit(run())
    except SystemExit:
        raise
    except FileNotFoundError as exc:
        print(f"ERROR: {exc}")
        raise SystemExit(EXIT_ERROR)
    except Exception as exc: # noqa: BLE001
        print(f"ERROR: {exc}")
        raise SystemExit(EXIT_ERROR)

if __name__ == "__main__":
    main()

=====
FILE: code_review\runner\review_update.py
ENCODING: utf-8
=====

"""Convert every update program, then code review each generated file.

Reuses the existing update conversion pipeline unchanged. The converted
text is reviewed straight from the file that was just written, so each
program is always matched with its own output.

    set PYTHONPATH=src
    python code_review\runner\review_update.py
    python code_review\runner\review_update.py --quiet

Exit codes: 0 accepted, 1 rejected, 2 error or nothing to convert.
"""

from __future__ import annotations

# -----
# Path bootstrap.
#
# This block MUST stay at the very top, above every code_review and converter
# import. Running "python code_review\runner\review_update.py" puts only
# code_review\runner on sys.path, so without this the package cannot be seen
# and the first import fails with ModuleNotFoundError.
# -----
import sys
from pathlib import Path

```

```

PROJECT_ROOT = Path(__file__).resolve().parents[2]
SRC_DIR = PROJECT_ROOT / "src"

for _c in (PROJECT_ROOT, SRC_DIR):
    if str(_c) not in sys.path:
        sys.path.insert(0, str(_c))
# -----

import argparse # noqa: E402

from code_review.engine.context import KIND_UPDATE, ReviewContext # noqa: E402
from code_review.engine.reviewer import ( # noqa: E402
    Reviewer,
    blockers,
    is_accepted,
)
from code_review.report import csv_writer, text_writer # noqa: E402
from code_review.report.text_writer import render # noqa: E402
from code_review.runner._bootstrap import ( # noqa: E402
    EXIT_ACCEPTED,
    EXIT_ERROR,
    EXIT_REJECTED,
    REPORT_DIR,
    read_text,
    stamp,
)
from code_review.standards import wording as w # noqa: E402
from config.path_settings import ( # noqa: E402
    DEFAULT_UPDATE_OUTPUT_DIR,
    DEFAULT_UPDATE_PROGRAM_DIR,
    LOGS_DIR,
)
from idms_db2_phase2.infrastructure.logger_factory import ( # noqa: E402
    LoggerFactory,
)
from idms_db2_phase2.testing.update.update_input_loader import ( # noqa: E402
    load_shared_inputs,
)
from idms_db2_phase2.testing.update.update_input_selector import ( # noqa: E402
    selected_program_paths,
    validate_inputs,
)
from idms_db2_phase2.testing.update.update_program_converter import ( # noqa: E402
    convert_one_program,
    resolve_target_program_id,
)

LOGGER_NAME = "idms_db2_update_code_review"

class NothingToConvert(Exception):
    """The program folder exists but holds no program to convert."""

    def __init__(self, folder: Path, kind_label: str) -> None:
        self.folder = Path(folder)
        self.kind_label = kind_label
        super().__init__(
            w.MSG_NO_PROGRAMS_FOUND.format(kind=kind_label, folder=self.folder)
        )

def parse_arguments() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description="Convert update programs, then code review the output."
    )
    parser.add_argument(
        "--report",
        default=str(REPORT_DIR),
        help="folder to write the reports into",
    )
    parser.add_argument(
        "--quiet",
        action="store_true",
        help="print one summary line per program instead of the full report",
    )

```

```

return parser.parse_args()

def build_context(
    program_path: Path,
    output_path: Path,
    sheet_rows: list,
    dclgen_columns: list,
    copybook_fields: list,
) -> ReviewContext:
    source_text = read_text(program_path)
    return ReviewContext(
        program_name=program_path.stem,
        program_kind=KIND_UPDATE,
        target_program_id=resolve_target_program_id(source_text),
        source_file=Path(output_path).name,
        converted_cobol=read_text(output_path),
        source_cobol=source_text,
        sheet_mapping_rows=sheet_rows,
        dclgen_columns=dclgen_columns,
        copybook_fields=copybook_fields,
    )

def print_banner(programs: list[Path], report_dir: Path) -> None:
    print("")
    print(w.H_RUN_UPDATE)
    print(w.RULE)
    print(w.MSG_PROGRAM_FOLDER.format(folder=DEFAULT_UPDATE_PROGRAM_DIR))
    print(w.MSG_PROGRAM_COUNT.format(count=len(programs)))
    print(w.MSG_OUTPUT_FOLDER.format(folder=DEFAULT_UPDATE_OUTPUT_DIR))
    print(w.MSG_REPORT_FOLDER.format(folder=report_dir))
    print("")

def print_nothing_to_convert(exc: NothingToConvert) -> int:
    print("")
    print(w.MSG_NO_PROGRAMS_TITLE)
    print(w.RULE)
    print(w.MSG_NO_PROGRAMS_FOUND.format(
        kind=exc.kind_label, folder=exc.folder,
    ))
    print("")
    print(w.MSG_NO_PROGRAMS_HINT)
    print("")
    return EXIT_ERROR

def run() -> int:
    args = parse_arguments()

    report_dir = Path(args.report)
    report_dir.mkdir(parents=True, exist_ok=True)

    # The update selector takes the output folder. The retrieval one does not.
    validate_inputs(DEFAULT_UPDATE_OUTPUT_DIR)
    logger = LoggerFactory.create_logger(name=LOGGER_NAME, logs_dir=LOGS_DIR)

    sheet_rows, dclgen_columns, copybook_fields, diagnostics = (
        load_shared_inputs(logger)
    )

    programs = selected_program_paths()
    if not programs:
        raise NothingToConvert(
            DEFAULT_UPDATE_PROGRAM_DIR, w.KIND_LABEL_UPDATE,
        )

    reviewer = Reviewer()
    when = stamp()
    results = []
    failed_conversions: list[str] = []

    print_banner(programs, report_dir)

    for program_path in programs:

```

```

try:
    output_path = convert_one_program(
        program_path=program_path,
        sheet_rows=sheet_rows,
        dclgen_columns=dclgen_columns,
        copybook_fields=copybook_fields,
        shared_diagnostics=diagnostics,
        output_dir=DEFAULT_UPDATE_OUTPUT_DIR,
        project_root=PROJECT_ROOT,
        src_dir=SRC_DIR,
        logger=logger,
    )
except Exception as exc: # noqa: BLE001
    message = w.MSG_CONVERSION_ERROR.format(
        name=program_path.name, reason=exc,
    )
    print(w.MSG_ERROR.format(message=message))
    logger.exception(message)
    failed_conversions.append(program_path.name)
    continue

result = reviewer.review(build_context(
    program_path=program_path,
    output_path=Path(output_path),
    sheet_rows=sheet_rows,
    dclgen_columns=dclgen_columns,
    copybook_fields=copybook_fields,
))
results.append(result)

stem = f"{program_path.stem}_{when}"
text_writer.write(result, report_dir, stem)
csv_writer.write(result, report_dir, stem)

if args.quiet:
    ids = ", ".join(c.check_id for c in blockers(result))
    print(w.MSG_QUIET_LINE.format(
        verdict=w.ACCEPTED if is_accepted(result) else w.REJECTED,
        name=program_path.stem,
        suffix=w.MSG_QUIET_BLOCKING.format(ids=ids) if ids else "",
    ))
else:
    print(render(result))

if len(results) > 1:
    csv_writer.write_summary(
        results, report_dir, f"summary_update_{when}",
    )

accepted = sum(1 for item in results if is_accepted(item))
rejected = len(results) - accepted

print(w.RULE)
print(w.MSG_SUMMARY.format(
    reviewed=len(results), accepted=accepted, rejected=rejected,
))
if failed_conversions:
    print(w.MSG_CONVERSION_FAILED.format(
        names=", ".join(failed_conversions),
    ))
print(w.MSG_REPORTS_WRITTEN.format(folder=report_dir))
print("")

if failed_conversions:
    return EXIT_ERROR
return EXIT_ACCEPTED if rejected == 0 else EXIT_REJECTED

def main() -> None:
    try:
        raise SystemExit(run())
    except SystemExit:
        raise
    except NothingToConvert as exc:
        raise SystemExit(print_nothing_to_convert(exc))
    except FileNotFoundError as exc:

```

```

        print(w.MSG_ERROR.format(message=exc))
        raise SystemExit(EXIT_ERROR)
    except Exception as exc: # noqa: BLE001
        print(w.MSG_ERROR.format(message=exc))
        raise SystemExit(EXIT_ERROR)

if __name__ == "__main__":
    main()

=====
FILE: code_review\standards\__init__.py
ENCODING: utf-8
=====

"""Expected values and pipeline control. Owned by the COBOL team."""

=====
FILE: code_review\standards\_wording_probe.py
ENCODING: utf-8
=====

# LOCATION: tools/rules_probe.py
# ACTION: CREATE NEW FILE
"""Developer probe: verify every `from rules.X import (...)` name exists.

Run from the project root:

    $env:PYTHONPATH = "src;."
    python tools\rules_probe.py

Exit codes: 0 all names resolve, 1 one or more are missing.
"""

from __future__ import annotations

import ast
import importlib
import sys
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
SRC_DIR = PROJECT_ROOT / "src"

for _candidate in (PROJECT_ROOT, SRC_DIR):
    if str(_candidate) not in sys.path:
        sys.path.insert(0, str(_candidate))

SCAN_DIRS = ("src", "rules", "patterns", "catalogs", "code_review")
WATCHED_PACKAGES = ("rules", "patterns", "catalogs")

def missing_names() -> list[str]:
    problems: list[str] = []
    cache: dict[str, object] = {}

    for folder in SCAN_DIRS:
        base = PROJECT_ROOT / folder
        if not base.exists():
            continue

        for path in sorted(base.rglob("*.py")):
            try:
                tree = ast.parse(path.read_text(encoding="utf-8"))
            except SyntaxError as exc:
                problems.append(f"{path} -> syntax error: {exc}")
                continue

            for node in ast.walk(tree):
                if not isinstance(node, ast.ImportFrom):
                    continue
                module = node.module or ""
                if not module.startswith(WATCHED_PACKAGES):
                    continue

```

```

        if module not in cache:
            try:
                cache[module] = importlib.import_module(module)
            except Exception as exc: # noqa: BLE001
                problems.append(f"{path} -> cannot import {module}: {exc}")
                cache[module] = None

        target = cache.get(module)
        if target is None:
            continue

        for alias in node.names:
            if alias.name == "*":
                continue
            if not hasattr(target, alias.name):
                problems.append(
                    f"{path} -> {module}.{alias.name} does not exist"
                )

    return problems

def main() -> None:
    problems = missing_names()

    if not problems:
        print("rules/patterns/catalogs imports OK")
        raise SystemExit(0)

    for problem in problems:
        print(problem)
    print(f"\n{len(problems)} missing name(s)")
    raise SystemExit(1)

if __name__ == "__main__":
    main()

```

```

=====
FILE: code_review\standards\cobol_standards.py
ENCODING: utf-8
=====

```

```

"""Expected values. THIS FILE IS OWNED BY THE COBOL TEAM.

```

```

Constants only. No regex, no logic, no program names, no DB2 table names.
Change a value here and every check that reads it follows.
"""

```

```

from __future__ import annotations

```

```

# ---- CHK-01 line layout -----

```

```

LINE_WIDTH = 80
BODY_FIRST_COLUMN = 8
BODY_LAST_COLUMN = 72
VALID_INDICATORS = (" ", "*", "/", "D", "-")
ENFORCE_SEQUENCE_START = False
LEFT_SEQUENCE_START = 10
RIGHT_SEQUENCE_START = 10000

```

```

# ---- CHK-02 DB2 infrastructure -----

```

```

REQUIRED_TOKENS = ("EXEC SQL", "SQLCA", "END-EXEC")
REQUIRED_LINES = (
    "EXEC SQL INCLUDE SQLCA END-EXEC.",
    "EXEC SQL INCLUDE SQLERRWS END-EXEC.",
)
SQL_LOCATION_FIELD = "SQL-LOCATION"
SQL_LOCATION_PICTURE = "PIC X(40) VALUE SPACES."
UNIQUE_MARKERS = (
    "DB2 SQLCA, SQL ERROR WORKING STORAGE, DCLGEN INCLUDES, AND CURSOR FLAGS",
    "DB2 CURSOR DECLARATIONS",
    "DB2 GENERATED CURSOR OPEN FETCH CLOSE PARAGRAPHS",
)
FORBIDDEN_MARKERS = (

```

```

"TODO", "TODO-HOST-VARIABLE", "DB2 WARNING",
"UNABLE TO DECLARE CURSOR", "NO FETCH HOST VARIABLES MAPPED",
)

# ---- CHK-03 residual IDMS -----
IDMS_EXECUTABLE_PREFIXES = (
    "OBTAIN", "STORE ", "MODIFY ", "ERASE ", "FIND CURRENT", "FIND FIRST",
    "BIND ", "READY ", "CONNECT ", "DISCONNECT ", "FINISH",
)
IDMS_DECLARATIVE_PREFIXES = (
    "IDMS-CONTROL SECTION", "PROTOCOL", "SCHEMA SECTION",
    "IDMS-RECORDS WITHIN", "COPY IDMS",
)
IDMS_TOKENS = ("IDMS-STATUS", "IDMS-ABORT", "DB-REC-NOT-FOUND", "DB-END-OF-SET")
IDMS_CLAUSES = ("USAGE-MODE IS UPDATE", "USAGE-MODE IS RETRIEVAL")

# ---- CHK-04 program header -----
COMPILER_OPTION_LINE = "CBL ARITH(EXTEND)"
NAME_PREFIX = "VM"
NAME_SOURCE_MARKER = "BD"
NAME_TARGET_MARKER = "DZ"
CS_PROGRAM_LEVELS = ("01", "77")

# ---- CHK-05 SQL error routing -----
SQL_ERROR_PARAGRAPH = "SQLERROR"
LEGACY_SQL_ERROR_PARAGRAPH = "SQL-ERROR"

# ---- CHK-06 Area A / Area B alignment -----
AREA_A_FIRST_COLUMN = 8
AREA_A_LAST_COLUMN = 11
AREA_B_FIRST_COLUMN = 12
AREA_A_MAX_INDENT = 3
AREA_B_MIN_INDENT = 4

DIVISION_NAMES = ("IDENTIFICATION", "ENVIRONMENT", "DATA", "PROCEDURE")
AREA_A_LEVEL_NUMBERS = ("01", "77")
AREA_B_LEVEL_NUMBERS = (
    "02", "03", "04", "05", "06", "07", "08", "09",
    "10", "11", "12", "15", "20", "25", "30", "35", "40", "45", "49",
    "66", "88",
)
CONTINUATION_INDICATOR = "-"
ENFORCE_AREA_B_STATEMENT_INDENT = True

# Generated DB2 block indentation. See open decision D-8.
ENFORCE_AREA_B_SQL_INDENT = False
AREA_B_SQL_MIN_INDENT = AREA_B_MIN_INDENT

SQL_BLOCK_START = "EXEC SQL"
SQL_BLOCK_END = "END-EXEC"
SQL_STATUS_BLOCK_START = "EVALUATE SQLCODE"
SQL_STATUS_BLOCK_END = "END-EVALUATE"

# NOTE: the converter holds an equivalent list in patterns/. Copied here
# on purpose. code_review must never import from the converter.
NON_PARAGRAPH_WORDS = frozenset({
    "ACCEPT", "ADD", "ALTER", "CALL", "CANCEL", "CLOSE", "COMMIT",
    "COMPUTE", "CONTINUE", "DELETE", "DISPLAY", "DIVIDE", "EJECT", "ELSE",
    "END-ADD", "END-CALL", "END-DELETE", "END-DIVIDE", "END-EVALUATE",
    "END-EXEC", "END-IF", "END-MULTIPLY", "END-PERFORM", "END-READ",
    "END-RETURN", "END-REWRITE", "END-SEARCH", "END-START", "END-STRING",
    "END-SUBTRACT", "END-UNSTRING", "END-WRITE", "EVALUATE", "EXEC",
    "EXIT", "FETCH", "GOBACK", "IF", "INITIALIZE", "INSPECT", "MOVE",
    "MULTIPLY", "NEXT", "OPEN", "PERFORM", "READ", "RETURN", "REWRITE",
    "ROLLBACK", "SEARCH", "SET", "SKIP1", "SKIP2", "SKIP3", "SORT",
    "SPACE", "SPACES", "START", "STOP", "STRING", "SUBTRACT", "UNSTRING",
    "WHEN", "WRITE",
})

# ---- CHK-07 sequence numbering continuity -----
LEFT_SEQUENCE_STEP = 10
RIGHT_SEQUENCE_STEP = 10000
ENFORCE_SEQUENCE_STEP = True
ALLOW_SEQUENCE_GAPS = False

# ---- CHK-08 DCLGEN include and copybook integrity -----

```

```

DCLGEN_GROUP_PREFIX = "DCL"
EXCLUDED_INCLUDE_NAMES = frozenset({"GEN", "SQLCA", "SQLERRWS", "SQLERROR"})
INCLUDE_STATEMENT_TEMPLATE = "EXEC SQL INCLUDE {name} END-EXEC."
FORBIDDEN_COPY_PREFIXES = ("IDMS",)
ENFORCE_INCLUDE_IN_WORKING_STORAGE = True

# COPY member validation. See open decision D-9.
ENFORCE_COPY_MEMBER_RESOLUTION = False
COPYBOOK_RECORD_LEVEL = "01"

# ---- Shared conversion markers -----
MISSING_MAPPING_MARKERS = (
    "CONVERSION SKIPPED",
    "MISSING SHEET MAPPING",
    "MISSING DCLGEN",
    "MISSING SELECT",
    "MISSING INSERT",
    "MISSING UPDATE",
    "MISSING DELETE",
    "MISSING MAPPING",
    "MISSING KEY COLUMN METADATA",
    "MISSING INSERT COLUMNS",
    "INCOMPLETE CONSERVATIVE UPDATE METADATA",
    "MANUAL REDESIGN",
)

SQL_LOCATION_MOVE_PREFIX = "MOVE '"
SQL_LOCATION_MOVE_SUFFIX = "'TO SQL-LOCATION"
SQL_LOCATION_LOOKBACK = 6
ENFORCE_SQL_LOCATION_BEFORE_SQL = True

CONTINUE_STATEMENT = "CONTINUE"
NON_EXECUTABLE_SQL_VERBS = ("INCLUDE", "DECLARE", "WHENEVER", "BEGIN", "END")

# ---- Shared DB2 naming equivalences -----
DB2_TABLE_SUFFIX_EQUIVALENTS = (
    ("_TB", "_TV"),
    ("_TV", "_TB"),
    ("TB", "TV"),
    ("TV", "TB"),
)
VARCHAR_SUBFIELD_SUFFIXES = ("-LEN", "-TEXT")

# ---- CHK-09 record retrieval conversion -----
RETRIEVAL_SQL_VERBS = ("SELECT", "OPEN", "FETCH", "CLOSE")
SELECT_REQUIRED_CLAUSES = ("FROM", "INTO", "WHERE")
ENFORCE_SELECT_WHERE = True

# ---- CHK-10 record modification conversion -----
WRITE_SQL_VERBS = ("INSERT", "UPDATE", "DELETE")
INSERT_REQUIRED_CLAUSES = ("INTO",)
UPDATE_REQUIRED_CLAUSES = ("SET", "WHERE")
DELETE_REQUIRED_CLAUSES = ("WHERE",)
UPDATE_QUERYNO = "442"
ENFORCE_QUERYNO = True

# ---- CHK-11 SQLCODE and COMMIT conversion -----
USE_EVALUATE_SQLCODE = True
SQLCODE_EVALUATE_START = "EVALUATE SQLCODE"
SQLCODE_EVALUATE_END = "END-EVALUATE"
SQLCODE_WHEN_OTHER = "WHEN OTHER"
SQLCODE_ZERO_BRANCHES = ("WHEN 0", "WHEN ZERO")
SQLCODE_NOT_FOUND_BRANCH = "WHEN 100"
END_OF_CURSOR_CONDITION = "SQLCODE = 100"
SQLCODE_LOOKAHEAD = 3

LEGACY_SQLCODE_FORMS = (
    "IF SQLCODE NOT = 0",
    "IF SQLCODE NOT= 0",
    "IF NOT SQLCODE = 0",
    "IF NOT SQLCODE = 100",
)

COMMIT_STATEMENT = "COMMIT"
IDMS_COMMIT_VERB = "FINISH"
EXECUTABLE_SQL_VERBS = (

```



```

        "SELECT", "INSERT", "UPDATE", "DELETE",
        "OPEN", "FETCH", "CLOSE", "COMMIT", "ROLLBACK",
    )
    SQLCODE_EXEMPT_VERBS = ("COMMIT", "ROLLBACK")

# A period inside an EVALUATE terminates the sentence and closes the
# scope, orphaning END-EVALUATE. Only the statement immediately before
# END-EVALUATE may carry one, and conventionally not even that.
ENFORCE_NO_PERIOD_IN_EVALUATE = True

# ---- CHK-12 cursor declaration standard -----
CURSOR_DECLARE_VERB = "DECLARE"
CURSOR_REQUIRED_CLAUSES = ("WITH HOLD", "FOR READ ONLY")
CURSOR_SELECT_CLAUSES = ("SELECT", "FROM")
CURSOR_SELECT_ALL = "SELECT *"
ENFORCE_CURSOR_WITH_HOLD = True
ENFORCE_CURSOR_FOR_READ_ONLY = True
ENFORCE_CURSOR_EXPLICIT_COLUMNS = True
CURSOR_DECLARATION_MARKER = "DB2 CURSOR DECLARATIONS"

# ---- CHK-13 cursor paragraph structure -----
CURSOR_OPERATIONS = ("OPEN", "FETCH", "CLOSE")
CURSOR_FETCH_OFFSET = 10
CURSOR_CLOSE_OFFSET = 20
CURSOR_OPEN_NUMBERS = (710, 810, 910)
ENFORCE_CURSOR_OPEN_NUMBERS = True
CURSOR_PARAGRAPH_MARKER = "DB2 GENERATED CURSOR OPEN FETCH CLOSE PARAGRAPHS"

# ---- CHK-14 end of cursor handling -----
CURSOR_EOC_SUFFIX = "-EOC"
CURSOR_NOT_EOC_SUFFIX = "-NOT-EOC"
CURSOR_FLAGS_MARKER = "DB2 CURSOR END-OF-CURSOR FLAGS"
LEGACY_EOC_CONDITIONS = (
    "UNTIL SQLCODE = 100",
    "IF NOT SQLCODE = 100",
)
ENFORCE_EOC_FLAG_LOOP = True

# ---- CHK-15 nested cursor processing -----
CHILD_CURSOR_MINIMUM_NUMBER = 800
CHILD_FETCH_EARLY_STOP_FIELD = "SW-STATUS-D"
# See open decision D-10.
ENFORCE_CHILD_FETCH_EARLY_STOP = False

# ---- CHK-16 conservative update column selection -----
UPDATE_AUDIT_PREFIXES = (
    "TS_UPDATE", "ID_USERID", "NR_USERID", "ID_USER", "NR_USER",
)
INSERT_ONLY_AUDIT_PREFIXES = ("TS_CREATE",)
BROAD_UPDATE_RATIO = 0.8
ENFORCE_CONSERVATIVE_UPDATE = True

# ---- CHK-17 update WHERE key integrity -----
KEY_TEXT_MARKERS = ("PRIMARY", "KEY", "CALC")
IDENTITY_KEY_PREFIXES = ("NS_ID", "NR_ID", "ID_", "CO_ID", "NR_IS")
HOST_REFERENCE_MARKER = ":"
FORBIDDEN_WHERE_PREDICATES = ("1 = 1", "1=1")
ENFORCE_WHERE_HOST_VARIABLES = True
ENFORCE_WHERE_NO_FOREIGN_KEY = True

# ---- CHK-18 audit column and date handling -----
DATE_COLUMN_PREFIXES = ("DA_", "DT_")
DB2_DATE_SENTINEL = "00010101"
DATE_STAGING_FIELDS = ("DA-CCYYMMDD", "DA-CCYYMMDD-R", "DA-DD-MM-CCYY")
TIMESTAMP_SOURCE_FIELD = "TS-TIMESTAMP"
USER_SOURCE_FIELD = "CS-PROGRAM"
AUDIT_MOVE_LOOKBACK = 12
ENFORCE_AUDIT_MOVES_BEFORE_UPDATE = True
ENFORCE_DATE_STAGING = True

# ---- CHK-19 restart paragraph structure -----
RESTART_PARAGRAPHS = {
    "control": "700-RESTART-CONTROL",
    "restart_found": "710-JOB-IS-RESTART",
    "write_restart": "720-WRITE-RESTART-REC",
    "commit": "800-PROCESS-COMMIT",
}

```

```

    "abend": "810000-CALL-USERABEN",
}
RESTART_SQL_PARAGRAPH_PREFIX = "700-"
RESTART_SQL_OPERATIONS = ("SELECT", "UPDATE", "INSERT")
RESTART_ABEND_CALL = "CALL USERABEN"
RESTART_STATUS_COMPLETE = "1"
RESTART_STATUS_INCOMPLETE = "0"

# ---- CHK-20 restart SQL from DCLGEN metadata -----
RESTART_TABLE_NAME_HINTS = ("RST", "RESTART")
# See open decision D-3.
ENFORCE_RESTART_QUERYNO = False
RESTART_SELECT_QUERYNO = "376"
RESTART_UPDATE_QUERYNO = "398"
RESTART_INSERT_QUERYNO = ""

# ---- CHK-21 unresolved restart metadata safe skip -----
RESTART_CONTROL_HINTS = (
    "RECAP", "RESTART", "RST", "CONTROL", "CTRL", "CHECKPOINT", "CHKPT",
)
RESTART_SKIP_MARKERS = (
    "MANUAL DB2 REDESIGN",
    "WAS NOT CONVERTED",
    "RESTART DCLGEN NOT RESOLVED",
    "RESTART FLOW NOT GENERATED",
    "MISSING SHEET MAPPING AND DCLGEN METADATA",
)

# ---- CHK-22 DB2 table and column naming authority -----
SQL_TABLE_KEYWORDS = ("FROM", "UPDATE", "INSERT INTO", "DELETE FROM")
ENFORCE_TABLE_AUTHORITY = True
ENFORCE_COLUMN_AUTHORITY = True

# ---- CHK-23 DCLGEN host variable name exactness -----
HOST_QUALIFIER_KEYWORDS = ("OF", "IN")
ENFORCE_HOST_NAME_EXACTNESS = True

# ---- CHK-24 picture clause and length validation -----
ENFORCE_PICTURE_LENGTH_MATCH = True
PICTURE_LENGTH_TOLERANCE = 0

# Sheet Mapping records STORED BYTES. A COBOL picture records POSITIONS.
# The two coincide only for character data: a DB2 DATE stores in 4 bytes
# but its host is PIC X(10), and a packed numeric stores in fewer bytes
# than it has digits. Comparing anything else produces noise.
COMPARABLE_LENGTH_TYPES = ("CHAR", "CHARACTER", "GRAPHIC")

# A VARCHAR host is a group with two 49-level children carrying the
# pictures. A group item has no PICTURE of its own, so its absence is
# correct COBOL rather than a defect.
VARCHAR_TYPES = (
    "VARCHAR", "VARGRAPHIC", "LONG VARCHAR", "LONG VARGRAPHIC",
    "CLOB", "DBCLOB", "BLOB",
)

# ---- CHK-25 copybook and record usage validation -----
ENFORCE_NO_IDMS_QUALIFIER = True

# =====
# DECISIONS CLOSED BY THE MANUAL REFERENCE PROGRAM (VMDZ4420)
#
# Appended deliberately: these assignments override any earlier value in
# this file. Fold them into their CHK- sections when convenient.
# =====

# ---- CHK-04 compiler option and PROGRAM-ID
#
# The manual reference declares CS-PROGRAM as a subordinate 10-level inside
# 01 WS-TE-WORK, not as 01 or 77. Decision D-2 closed by evidence.
CS_PROGRAM_FIELD = "CS-PROGRAM"
CS_PROGRAM_PICTURE = "PIC X(8)"
CS_PROGRAM_LEVELS = (
    "01", "02", "03", "04", "05", "06", "07", "08", "09", "10", "77",
)

# The manual reference emits no END PROGRAM statement.

```

```

ENFORCE_END_PROGRAM = False

# ---- CHK-05 SQL error paragraph and routing
#
# Manual reference body:
#     EXEC SQL
#         INCLUDE SQLERROR
#     END-EXEC.
# It declares no SQLERROR paragraph of its own; the copybook supplies the
# routine. Decision D-1 closed by evidence. Set to () to skip criterion 08.
SQL_ERROR_BODY = ("EXEC SQL", "INCLUDE SQLERROR", "END-EXEC")
SQL_ERROR_INCLUDE = "INCLUDE SQLERROR"

# ---- CHK-06 Area A and Area B alignment
#
# Manual reference places EXEC SQL, EVALUATE SQLCODE and END-EVALUATE at
# column 12. Decision D-8 closed by evidence.
ENFORCE_AREA_B_SQL_INDENT = True

# ---- CHK-15 nested cursor processing
#
# Manual reference:
#     PERFORM 820-FETCH-DZEVEFC1 UNTIL DZEVEFC1-EOC OR
#                                     SW-STATUS-D = 'Y'
# Decision D-10 closed by evidence.
ENFORCE_CHILD_FETCH_EARLY_STOP = True

# ---- CHK-20 restart SQL from DCLGEN metadata
#
# rules/update_restart_rules.py and this file agree: 376 / 398 / 413.
# The recorded conflict was a blank INSERT value here, not a disagreement
# between modules. Decision D-3 closed.
ENFORCE_RESTART_QUERYNO = True
RESTART_SELECT_QUERYNO = "376"
RESTART_UPDATE_QUERYNO = "398"
RESTART_INSERT_QUERYNO = "413"
# ---- CHK-02 SQL-LOCATION ownership
#
# SQLERRWS supplies SQL-LOCATION. A local declaration is a duplicate
# data-name. Decision closed by the manual reference program.
ENFORCE_SQL_LOCATION_DECLARATION = False

# =====
# DECISIONS CLOSED BY THE MANUAL REFERENCE PROGRAM (VMDZ4420)
#
# Appended deliberately: these assignments override any earlier value in
# this file.
# =====

# ---- CHK-02.03 SQL-LOCATION ownership
#
# SQLERRWS declares SQL-LOCATION. A local 01 declaration is a DUPLICATE
# DATA-NAME, which the compiler rejects. The manual reference program
# carries no 01 SQL-LOCATION entry at all: it moves into the field and
# lets the copybook own it.
#
# The converter was changed to match (catalogs/output_sections.py,
# DECLARE_SQL_LOCATION_FIELD = False). This flag lets CHK-02.03 follow.
ENFORCE_SQL_LOCATION_DECLARATION = False

# ---- CHK-09.05 SQL-LOCATION move form
#
# Manual reference:
#     MOVE 710    TO SQL-LOCATION
#
# The old single prefix "MOVE '" recognised only the quoted paragraph-name
# form, so every generated cursor paragraph failed the check even though
# it sets SQL-LOCATION correctly. Both forms are now accepted.
SQL_LOCATION_MOVE_PREFIXES = ("MOVE '", "MOVE ")

# Retained for any caller still reading the singular name.
SQL_LOCATION_MOVE_PREFIX = "MOVE "
SQL_LOCATION_SOURCE_COPYBOOK = "SQLERRWS"

```

```
=====
FILE: code_review\standards\pipeline.py
ENCODING: utf-8
=====
```

"""Controls which checks run. Edit this file only.

Drop a module in code_review/checks to ADD a check. List its id in
DISABLED_CHECKS to REMOVE it. There is no central list of class names to
maintain: code_review/engine/registry.py discovers checks with pkgutil.
"""

```
from __future__ import annotations
```

```
# Empty tuple means: run everything the registry discovers.
# Populate it only to run a deliberate subset, for example during triage.
ENABLED_CHECKS: tuple[str, ...] = ()
```

```
# List ids here to switch a check off without deleting its file.
DISABLED_CHECKS: frozenset[str] = frozenset()
```

```
# Report order. Any check not listed falls back to its own ORDER attribute,
# then to its id, so a newly dropped-in check still sorts sensibly before
# anyone edits this file.
```

```
CHECK_ORDER: tuple[str, ...] = (
    # Layout and structure
    "CHK-01", "CHK-06", "CHK-07",
    # DB2 infrastructure and error routing
    "CHK-02", "CHK-05", "CHK-08",
    # Conversion completeness
    "CHK-03", "CHK-04", "CHK-09", "CHK-11",
    # Cursor family
    "CHK-12", "CHK-13", "CHK-14", "CHK-15",
    # Update family
    "CHK-10", "CHK-16", "CHK-17", "CHK-18",
    # Restart family
    "CHK-19", "CHK-20", "CHK-21",
    # Authority family
    "CHK-22", "CHK-23", "CHK-24", "CHK-25",
)
```

```
# A failure at any of these severities makes the verdict REJECTED.
# Severities are Critical, Major, Minor. See open decision D-5.
BLOCKING_SEVERITIES: tuple[str, ...] = ("Critical",)
```

```
# Maximum evidence lines recorded per criterion.
EVIDENCE_LIMIT = 10
```

```
=====
FILE: code_review\standards\wording.py
ENCODING: utf-8
=====
```

"""Display text only.

No logic, no regex, no paths, no program names. Every string the code
review prints, writes or renders is declared here so wording stays in one
place and the COBOL team can reword the report without touching logic.
"""

```
from __future__ import annotations
```

```
# ---- Outcome symbols -----
SYMBOL = {
    "PASS": "PASS ",
    "FAIL": "FAIL ",
    "SKIPPED": "SKIP ",
    "BLOCKED": "BLOCK",
}
```

```
# ---- Placeholders -----
VALUE_UNNAMED = "(unnamed)"
```

```

VALUE_UNKNOWN = "(unknown)"
VALUE_IN_MEMORY = "(in memory)"

# ---- Report header -----
REPORT_TITLE = "Code Review Report"
RULE = "-" * 74

F_PROGRAM = "Program          : {value}"
F_KIND = "Program Kind       : {value}"
F_SOURCE = "Reviewed File    : {value}"
F_WHEN = "Reviewed          : {value}"
F_TOTALS = (
    "Checks          : {passed} passed, {failed} failed, "
    "{skipped} skipped, {blocked} blocked"
)
F_CRITERIA = (
    "Criteria          : {passed} passed, {failed} failed, "
    "{skipped} skipped, {blocked} blocked"
)
F_RESULT = "Result          : {value}"
F_BLOCKING = "Blocking       : {value}"

ACCEPTED = "ACCEPTED"
REJECTED = "REJECTED"

# ---- Report body -----
H_DETAIL = "Detail"
H_FAILURES = "Failures"
NO_FAILURES = "No failures."

F_CHECK = "{symbol} {check_id} {severity:<8} {title}"
F_CRIT = "          {criterion_id} {description}"
F_FIND = "          {finding}"
F_NOTE = "          note : {note}"

F_FAIL_CHECK = "{check_id} {severity:<8} {title}"
F_FAIL_CRIT = "          {criterion_id} {description}"
F_FAIL_NOTE = "          {note}"

# ---- CSV -----
CSV_HEADER = (
    "Program", "Reviewed", "File", "Check", "Severity", "Title",
    "Criterion", "Description", "Outcome", "Note", "Lines",
)
CSV_SUMMARY_HEADER = (
    "Program", "Kind", "Reviewed", "File",
    "Passed", "Failed", "Skipped", "Blocked", "Result",
)
CSV_TIMESTAMP_FORMAT = "%d-%m-%Y %H:%M:%S"
CSV_LINE_SEPARATOR = " "

# ---- Streamlit panel -----
UI_TITLE = "Code Review"
UI_NOT_READY = "Generate DB2 COBOL first, then run the code review."
UI_RUN = "Run Code Review"
UI_RUN_HINT = "Click Run Code Review to score the generated program."
UI_ACCEPTED = "Accepted. No blocking failures."
UI_REJECTED = "Rejected. Blocking checks: {ids}"
UI_UNAVAILABLE = "Code review unavailable: {reason}"

UI_TARGET = "Ready to review `{file}` as a **{kind}** program."
UI_STALE = (
    "The generated program changed since this report was produced. "
    "The stale report was discarded. Run the code review again."
)
UI_SCOPE = (
    "This panel reviews the program currently in the session, not files "
    "on disk. Use review_file.py to review a specific .cbl."
)

UI_PROGRAM = "Program **{name}** | Kind **{kind}** | File `{file}`"
UI_METRIC_PASSED = "Checks passed"
UI_METRIC_FAILED = "Checks failed"
UI_METRIC_SKIPPED = "Checks skipped"
UI_METRIC_BLOCKED = "Checks blocked"
UI_CRITERIA = (

```

```

        "Criteria: {passed} passed, {failed} failed, "
        "{skipped} skipped, {blocked} blocked"
    )

    UI_FILTER_FAILURES = "Show failures only"
    UI_FILTER_SKIPS = "Show skip reasons"
    UI_CHECK_LABEL = "{outcome} | {check_id} | {severity} | {title}"
    UI_CRITERION = "`{mark}` **{criterion_id}** {description}"
    UI_MARK_PASS = "OK "

    UI_DOWNLOAD_TEXT = "Download report (text)"
    UI_DOWNLOAD_CSV = "Download report (csv)"
    UI_FILE_TEXT = "{stem}_code_review.txt"
    UI_FILE_CSV = "{stem}_code_review.csv"
    UI_MIME_TEXT = "text/plain"
    UI_MIME_CSV = "text/csv"
    UI_DEFAULT_STEM = "review"

    # ---- Runner banners -----
    H_RUN_FILE = "Code Review"
    H_RUN_RETRIEVAL = "Retrieval Code Review"
    H_RUN_UPDATE = "Update Code Review"

    MSG_REVIEWED_FOLDER = "Reviewed Folder      : {folder}"
    MSG_PROGRAM_FOLDER = "Program Folder      : {folder}"
    MSG_PROGRAM_COUNT = "Program Count       : {count}"
    MSG_OUTPUT_FOLDER = "Output Folder       : {folder}"
    MSG_REPORT_FOLDER = "Report Folder       : {folder}"
    MSG_FILE_COUNT = "Files                : {count}"
    MSG_PROGRAM_KIND = "Program Kind        : {kind}"

    # ---- Runner summary -----
    MSG_SUMMARY = "Reviewed : {reviewed}   Accepted : {accepted}   Rejected : {rejected}"
    MSG_CONVERSION_FAILED = "Conversion failed : {names}"
    MSG_REPORTS_WRITTEN = "Reports written to : {folder}"
    MSG_QUIET_LINE = "{verdict:<9} {name}{suffix}"
    MSG_QUIET_BLOCKING = " blocking: {ids}"

    # ---- Runner errors -----
    MSG_ERROR = "ERROR: {message}"
    MSG_FILE_NOT_FOUND = "File not found: {path}"
    MSG_FOLDER_NOT_FOUND = "Folder not found: {folder}"
    MSG_PATH_NOT_FOLDER = "Path is not a folder: {folder}"
    MSG_SOURCE_NOT_FOUND = "Source file not found: {path}"
    MSG_CONVERSION_ERROR = "Conversion failed for {name}: {reason}"

    # ---- Nothing generated yet -----
    MSG_NO_OUTPUT_TITLE = "Nothing to review."
    MSG_NO_OUTPUT_FOUND = (
        "No .cbl files found in: {folder}\n"
        "The folder exists but is empty, so no DB2 COBOL has been generated yet."
    )
    MSG_NO_OUTPUT_HINT = (
        "Next step: run the conversion first, then run the code review again.\n"
        "  set PYTHONPATH=src\n"
        "  python code_review\\runner\\review_retrieval.py    (retrieval programs)\n"
        "  python code_review\\runner\\review_update.py        (update programs)"
    )

    # ---- Nothing to convert -----
    MSG_NO_PROGRAMS_TITLE = "Nothing to convert."
    MSG_NO_PROGRAMS_FOUND = (
        "No {kind} programs found in: {folder}\n"
        "The folder exists but holds no program file to convert."
    )
    MSG_NO_PROGRAMS_HINT = (
        "Next step: place at least one .txt, .cbl or .cob program in that\n"
        "folder, then run this command again."
    )

    KIND_LABEL_RETRIEVAL = "retrieval"
    KIND_LABEL_UPDATE = "update"

```

```
=====
FILE: code_review\ui\__init__.py
ENCODING: utf-8
=====
```

```
"""Streamlit panel for the code review."""
```

```
=====
FILE: code_review\ui\review_panel.py
ENCODING: utf-8
=====
```

```
"""Streamlit panel. Reviews whatever is currently in session state.
```

```
Reads only from session state and writes only to it. Imports nothing from
the converter, so the one-way dependency rule holds even here.
"""
```

```
from __future__ import annotations
```

```
import csv
import hashlib
import io
import re
```

```
import streamlit as st
```

```
from code_review.engine.context import (
    KIND_RETRIEVAL,
    KIND_UPDATE,
    ReviewContext,
)
```

```
from code_review.engine.models import Outcome
from code_review.engine.reviewer import Reviewer, blockers, is_accepted
from code_review.report import csv_writer, text_writer
from code_review.standards import wording as w
```

```
IDMS_WRITE = re.compile(
    r"^\s*(?:\d{6}\s+)?(STORE|MODIFY|ERASE)\b", re.IGNORECASE | re.MULTILINE
)
STATE_KEY = "code_review_result"
STATE_STAMP = "code_review_stamp"
TEXT_ENCODING = "utf-8"
```

```
def _kind(source_text: str) -> str:
    """UPDATE when the source contains an IDMS write verb."""
    return (
        KIND_UPDATE
        if IDMS_WRITE.search(str(source_text or ""))
        else KIND_RETRIEVAL
    )
```

```
def _fingerprint() -> str:
    """Identity of the program currently held in session state.
```

```
The converter resets its own generated-output keys when new inputs are
loaded, but it knows nothing about the review result. Without this,
a report produced for the retrieval program stays on screen after the
update program is generated.
"""
```

```
converted = str(st.session_state.get("converted_cobol", "") or "")
digest = hashlib.sha1(
    converted.encode(TEXT_ENCODING, errors="replace")
).hexdigest()
return "|".join([
    str(st.session_state.get("idms_cobol_source_name", "") or ""),
    str(st.session_state.get("converted_cobol_file_name", "") or ""),
    str(len(converted)),
    digest,
])
```

```
def build_context() -> ReviewContext:
```

```

source = st.session_state.get("idms_cobol_text", "")
return ReviewContext(
    program_name=(
        st.session_state.get("idms_cobol_source_name", "")
        or w.VALUE_UNNAMED
    ),
    program_kind=_kind(source),
    target_program_id="",
    source_file=st.session_state.get("converted_cobol_file_name", ""),
    converted_cobol=st.session_state.get("converted_cobol", ""),
    source_cobol=source,
    sheet_mapping_rows=st.session_state.get("sheet_mapping_rows", []),
    dclgen_columns=st.session_state.get("dclgen_columns", []),
    copybook_fields=st.session_state.get("copybook_fields", []),
)

def render_review_panel() -> None:
    st.markdown(f"## {w.UI_TITLE}")

    if not st.session_state.get("converted_cobol"):
        _discard()
        st.info(w.UI_NOT_READY)
        return

    current = _fingerprint()
    result = _current_result(current)

    st.caption(w.UI_TARGET.format(
        file=st.session_state.get("converted_cobol_file_name", "")
        or w.VALUE_IN_MEMORY,
        kind=_kind(st.session_state.get("idms_cobol_text", "")),
    ))

    left, _right = st.columns([1, 3])
    with left:
        if st.button(w.UI_RUN, type="primary", key="code_review_run"):
            result = Reviewer().review(build_context())
            st.session_state[STATE_KEY] = result
            st.session_state[STATE_STAMP] = current

    if result is None:
        st.info(w.UI_RUN_HINT)
        st.caption(w.UI_SCOPE)
        return

    _render_verdict(result)
    _render_totals(result)
    _render_detail(result)
    _render_downloads(result)
    st.caption(w.UI_SCOPE)

# ---- state -----
def _discard() -> None:
    st.session_state.pop(STATE_KEY, None)
    st.session_state.pop(STATE_STAMP, None)

def _current_result(current: str):
    """The stored report, but only when it belongs to this program."""
    result = st.session_state.get(STATE_KEY)
    if result is None:
        return None
    if st.session_state.get(STATE_STAMP) == current:
        return result
    _discard()
    st.warning(w.UI_STALE)
    return None

# ---- sections -----
def _render_verdict(result) -> None:
    st.markdown(w.UI_PROGRAM.format(
        name=result.program_name or w.VALUE_UNNAMED,
        kind=result.program_kind or w.VALUE_UNKNOWN,
    ))

```



```

        file=result.source_file or w.VALUE_IN_MEMORY,
    ))

    if is_accepted(result):
        st.success(w.UI_ACCEPTED)
    else:
        st.error(w.UI_REJECTED.format(
            ids=", ".join(c.check_id for c in blockers(result))
        ))

def _render_totals(result) -> None:
    c1, c2, c3, c4 = st.columns(4)
    c1.metric(w.UI_METRIC_PASSED, result.passed)
    c2.metric(w.UI_METRIC_FAILED, result.failed)
    c3.metric(w.UI_METRIC_SKIPPED, result.skipped)
    c4.metric(w.UI_METRIC_BLOCKED, result.blocked)

    st.caption(w.UI_CRITERIA.format(
        passed=result.criteria_passed,
        failed=result.criteria_failed,
        skipped=result.criteria_skipped,
        blocked=result.criteria_blocked,
    ))

def _render_detail(result) -> None:
    col_left, col_right = st.columns(2)
    with col_left:
        failures_only = st.checkbox(
            w.UI_FILTER_FAILURES, value=False, key="code_review_filter",
        )
    with col_right:
        show_skips = st.checkbox(
            w.UI_FILTER_SKIPS, value=True, key="code_review_skips",
        )

    for check in result.checks:
        if failures_only and check.outcome is not Outcome.FAIL:
            continue

        label = w.UI_CHECK_LABEL.format(
            outcome=check.outcome.value,
            check_id=check.check_id,
            severity=check.severity,
            title=check.title,
        )
        with st.expander(label, expanded=check.outcome is Outcome.FAIL):
            _render_criteria(check, show_skips)

def _render_criteria(check, show_skips: bool) -> None:
    for crit in check.criteria:
        if crit.outcome is Outcome.PASS:
            continue
        if crit.outcome is Outcome.SKIPPED and not show_skips:
            continue

        st.markdown(w.UI_CRITERION.format(
            mark=crit.outcome.value,
            criterion_id=crit.criterion_id,
            description=crit.description,
        ))
        if crit.note:
            st.caption(crit.note)
        if crit.findings:
            st.code(
                "\n".join(f.render() for f in crit.findings),
                language="text",
            )

def _render_downloads(result) -> None:
    st.markdown("---")
    stem = (result.program_name or w.UI_DEFAULT_STEM).replace(" ", "_")

```

```

d1, d2 = st.columns(2)
with d1:
    st.download_button(
        w.UI_DOWNLOAD_TEXT,
        data=text_writer.render(result),
        file_name=w.UI_FILE_TEXT.format(stem=stem),
        mime=w.UI_MIME_TEXT,
        key="code_review_download_text",
    )
with d2:
    buffer = io.StringIO()
    writer = csv.writer(buffer)
    writer.writerow(w.CSV_HEADER)
    writer.writerows(csv_writer.rows(result))
    st.download_button(
        w.UI_DOWNLOAD_CSV,
        data=buffer.getvalue(),
        file_name=w.UI_FILE_CSV.format(stem=stem),
        mime=w.UI_MIME_CSV,
        key="code_review_download_csv",
    )

```

```

=====
FILE: config\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: config\app_settings.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: config\logging_settings.py
ENCODING: utf-8
=====

```

```

"""
Logging settings.

Log files are written to the project logs folder with date-time names.
"""

LOG_FILE_PREFIX = "idms_db2_conversion"

LOG_DATE_TIME_FORMAT = "%d-%m-%Y-%H%M%S"

LOG_MESSAGE_FORMAT = "%(asctime)s | %(levelname)s | %(name)s | %(message)s"

LOG_DATE_FORMAT = "%d-%m-%Y %H:%M:%S"

DEFAULT_LOG_LEVEL = "INFO"

```

```

=====
FILE: config\path_settings.py
ENCODING: utf-8
=====

```

```

r"""
Path settings for local command-line runs.

These paths are intentionally centralized here instead of being embedded in
runner, parser, or conversion logic.

Folder-based local input layout:

C:\S\S-Input
Mapping Sheet

```

```

    Retrieval
        *.csv / *.xlsx
    Update
        *.csv / *.xlsx
    DCLGen
        *.txt / *.cbl / *.cob / *.cpy
    Copybook
        *.txt / *.cbl / *.cob / *.cpy
    Program
        Retrieval
            *.txt / *.cbl / *.cob
        Update
            *.txt / *.cbl / *.cob
    Output
        Retrieval
        Update
"""

from __future__ import annotations

from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
SRC_DIR = PROJECT_ROOT / "src"
LOGS_DIR = PROJECT_ROOT / "logs"

DEFAULT_INPUT_DIR = Path(r"C:\S\S-Input")

DEFAULT_MAPPING_SHEET_DIR = DEFAULT_INPUT_DIR / "Mapping Sheet"
DEFAULT_RETRIEVAL_MAPPING_SHEET_DIR = DEFAULT_MAPPING_SHEET_DIR / "Retrieval"
DEFAULT_UPDATE_MAPPING_SHEET_DIR = DEFAULT_MAPPING_SHEET_DIR / "Update"

DEFAULT_DCLGEN_DIR = DEFAULT_INPUT_DIR / "DCLGen"
DEFAULT_COPYBOOK_DIR = DEFAULT_INPUT_DIR / "Copybook"

DEFAULT_PROGRAM_DIR = DEFAULT_INPUT_DIR / "Program"
DEFAULT_RETRIEVAL_PROGRAM_DIR = DEFAULT_PROGRAM_DIR / "Retrieval"
DEFAULT_UPDATE_PROGRAM_DIR = DEFAULT_PROGRAM_DIR / "Update"

DEFAULT_OUTPUT_DIR = DEFAULT_INPUT_DIR / "Output"
DEFAULT_RETRIEVAL_OUTPUT_DIR = DEFAULT_OUTPUT_DIR / "Retrieval"
DEFAULT_UPDATE_OUTPUT_DIR = DEFAULT_OUTPUT_DIR / "Update"

SUPPORTED_MAPPING_EXTENSIONS = (
    ".csv",
    ".xlsx",
)

SUPPORTED_TEXT_EXTENSIONS = (
    ".txt",
    ".cbl",
    ".cob",
    ".cpy",
)

DEFAULT_SHEET_MAPPING_PATH = DEFAULT_MAPPING_SHEET_DIR / "Excel_Sheet_mapping.csv"
DEFAULT_RETRIEVAL_SOURCE_PATH = DEFAULT_RETRIEVAL_PROGRAM_DIR / "Retrieval.txt"
DEFAULT_UPDATE_SOURCE_PATH = DEFAULT_UPDATE_PROGRAM_DIR / "Update.txt"

def existing_files(
    paths: list[Path],
) -> list[Path]:
    return [
        path
        for path in paths
        if path.exists() and path.is_file()
    ]

def files_in_folder(
    folder: Path,
    extensions: tuple[str, ...],
) -> list[Path]:
    if not folder.exists() or not folder.is_dir():

```

```

        return []

    normalized_extensions = tuple(
        extension.lower()
        for extension in extensions
    )

    return sorted(
        [
            path
            for path in folder.iterdir()
            if path.is_file()
            and path.suffix.lower() in normalized_extensions
        ],
        key=lambda item: item.name.lower(),
    )

def mapping_sheet_paths() -> list[Path]:
    return files_in_folder(
        DEFAULT_MAPPING_SHEET_DIR,
        SUPPORTED_MAPPING_EXTENSIONS,
    )

def retrieval_mapping_sheet_paths() -> list[Path]:
    mode_paths = files_in_folder(
        DEFAULT_RETRIEVAL_MAPPING_SHEET_DIR,
        SUPPORTED_MAPPING_EXTENSIONS,
    )

    if mode_paths:
        return mode_paths

    return mapping_sheet_paths()

def update_mapping_sheet_paths() -> list[Path]:
    mode_paths = files_in_folder(
        DEFAULT_UPDATE_MAPPING_SHEET_DIR,
        SUPPORTED_MAPPING_EXTENSIONS,
    )

    if mode_paths:
        return mode_paths

    return mapping_sheet_paths()

def dclgen_paths() -> list[Path]:
    return files_in_folder(
        DEFAULT_DCLGEN_DIR,
        SUPPORTED_TEXT_EXTENSIONS,
    )

def copybook_paths() -> list[Path]:
    return files_in_folder(
        DEFAULT_COPYBOOK_DIR,
        SUPPORTED_TEXT_EXTENSIONS,
    )

def retrieval_program_paths() -> list[Path]:
    return files_in_folder(
        DEFAULT_RETRIEVAL_PROGRAM_DIR,
        SUPPORTED_TEXT_EXTENSIONS,
    )

def update_program_paths() -> list[Path]:
    return files_in_folder(
        DEFAULT_UPDATE_PROGRAM_DIR,
        SUPPORTED_TEXT_EXTENSIONS,
    )

```

```

def ensure_output_dirs() -> None:
    DEFAULT_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    DEFAULT_RETRIEVAL_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    DEFAULT_UPDATE_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

DEFAULT_DCLGEN_CANDIDATE_PATHS = dclgen_paths()
DEFAULT_COPYBOOK_CANDIDATE_PATHS = copybook_paths()

# LOCATION: config/path_settings.py
# ACTION: APPEND at the end of the file

# ---- LRF (Logical Record Facility) subschema input -----
# Separate folder, exactly like Mapping Sheet / DCLGen / Copybook:
# C:\S\S-Input\LRF\Sub Schema.txt
DEFAULT_LRF_DIR = DEFAULT_INPUT_DIR / "LRF"

SUPPORTED_LRF_EXTENSIONS = (
    ".txt",
    ".sub",
    ".sch",
    ".cpy",
    ".cbl",
)

def lrf_paths() -> list[Path]:
    """Return all LRF subschema files from the LRF folder."""
    return files_in_folder(
        DEFAULT_LRF_DIR,
        SUPPORTED_LRF_EXTENSIONS,
    )

def ensure_lrf_dir() -> None:
    """Create the optional LRF folder when it does not exist."""
    DEFAULT_LRF_DIR.mkdir(parents=True, exist_ok=True)

DEFAULT_LRF_CANDIDATE_PATHS = lrf_paths()

```

```

=====
FILE: debug_indent_in.txt
ENCODING: utf-8
=====

```

000010 IDENTIFICATION DIVISION.	00010000
000020*=====	00020000
000030 PROGRAM-ID. VMDZ7200.	00030000
000040 AUTHOR. J-M BOOGAERTS 88/07.	00040000
000050	00050000
000060*****	00060000
000070*	00070000
000080* ARCHIVAGE : EXTRACT DES FORMULAIRES A ARCHIVER	00080000
000090* ===== (SOUSCRIPTIONS ET ANNULATIONS SOUSCRIPTIONS)	00090000
000100*	00100000
000110* WULTEPUTTE KRISTINE : (10/7/91) :	00110000
000120*	00120000
000130* TOEVOEGEN VAN NIEUWE PLAGES VOOR BCI1	00130000
000140*	00140000
000150* DE SCHEPPER KOEN : (18/12/92) :	00150000
000160*	00160000
000170* AANPASSING VOOR SLIK : UITBREIDING FORM 270 -> 390	00170000
000180*	00180000
000190* CLEMENT J. (06/93) : TOEVOEGEN PLAGES	00190000
000200*	00200000
000210* DE SCHEPPER KOEN (04/02/94) :	00210000
000220* PROGRAMMA VOLLEDIG HERSCHREVEN : GEEN SELECTIE OP	00220000
000230* WAARDECODE MEER.	00230000
000240*	00240000
000250* POMPONIO S (03/97)	00250000
000260* AANPASSING EURO2000 : PARMDATE EN 8 POSITIONS	00260000
000270* RECORD VMBFAS 388 -> 432 BYTES	00270000
000280*	00280000

000290* J-M BOOGAERTS 2005/02	00290000
000300* INTÉGRATION ARCHIVAGE IUNS P8	00300000
000310*	00310000
000320*****	00320000
000330	00330000
000340 ENVIRONMENT DIVISION.	00340000
000350*=====	00350000
000360 CONFIGURATION SECTION.	00360000
000370*-----	00370000
000380 SPECIAL-NAMES.	00380000
000390 DECIMAL-POINT IS COMMA.	00390000
000400*	00400000
000410 INPUT-OUTPUT SECTION.	00410000
000420*-----	00420000
000430 FILE-CONTROL.	00430000
000440 SELECT DAARCH ASSIGN TO UT-S-DD005.	00440000
000450 SELECT FORM ASSIGN TO UT-S-DD010.	00450000
000460*	00460000
000470*-----	00470000
000480/	00480000
000490 DATA DIVISION.	00490000
000500*=====	00500000
000510 FILE SECTION.	00510000
000520*-----	00520000
000530* ----- DATE ARCHIVAGE -----	00530000
000540*	00540000
000550 FD DAARCH RECORDING F	00550000
000560 RECORD CONTAINS 20	00560000
000570 BLOCK CONTAINS 0	00570000
000580 LABEL RECORD STANDARD.	00580000
000590 01 DAARCH-REC PIC X(20).	00590000
000600*	00600000
000610* ----- GESELECTEERDE FORMULIEREN -----	00610000
000620*	00620000
000630 FD FORM RECORDING F	00630000
000640 RECORD CONTAINS 530	00640000
000650 BLOCK CONTAINS 0	00650000
000660 LABEL RECORD STANDARD.	00660000
000670 01 FORM-REC PIC X(530).	00670000
000680/	00680000
000690*-----	00690000
000700*	00700000
000710 WORKING-STORAGE SECTION.	00710000
000720*-----	00720000
000730*	00730000
000740 77 USERABEN PIC X(8) VALUE 'USERABEN'.	00740000
000750 77 WS-TELLER PIC S9(11) COMP-3 VALUE ZERO.	00750000
000760*	00760000
000770 01 W-NR-IS-FORM-AS PIC 9(9).	00770000
000780 01 FILLER REDEFINES W-NR-IS-FORM-AS.	00780000
000790 05 W-NR-ID-ISSUE PIC 9(7).	00790000
000800 05 W-NR-CD-ISSUE PIC 9(2).	00800000
000810*	00810000
000820 01 REC-FORM.	00820000
000830 05 F-FORM.	00830000
000830*DB2 MATERIALISED IDMS RECORD VMBFAS	00830000
000830 06 VMBFAS.	00830000
000830 07 KY-VMBFAS.	00830000
000830 08 NR-RK-GOOD-V02.	00830000
000830 09 CT-RK-TGDSERV.	00830000
000830 10 CT-RK-TGDSV PIC X(4).	00830000
000830 10 CT-RK-TGOOD REDEFINES CT-RK-TGDSV	00830000
000830 PIC 9(4).	00830000
000830 09 GR-ID-GOOD-V02.	00830000
000830 10 NR-CIO-FORM-AS PIC 9(8) COMP-3.	00830000
000830 10 DA-CR-FORM-AS PIC 9(8) COMP-3.	00830000
000830 10 NR-ID-FORM-AS PIC 9(6) COMP-3.	00830000
000830 10 CT-OP-FORM-AS.	00830000
000830 11 CT-OPD-FORM-AS PIC S9(3) COMP-3.	00830000
000830 11 CT-OPG-FORM-AS PIC 9(3) COMP-3.	00830000
000830 10 FILLER PIC X(2).	00830000
000830 07 AT-FORM-AS.	00830000
000830 08 NR-CIW-FORM-AS PIC X(8).	00830000
000830 08 DA-VA-FORM-AS.	00830000
000830 09 DA-VA-FORM-AS-Y PIC 9(4).	00830000
000830 09 DA-VA-FORM-AS-M PIC 9(2).	00830000

000830	09 DA-VA-FORM-AS-D	PIC 9(2).	00830000
000830	08 CO-VM-FORM-AS	PIC X(1).	00830000
000830	08 NR-IS-FORM-AS	PIC 9(9).	00830000
000830	08 RA-RT-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	08 RA-CP-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	08 RA-BASE-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	08 DA-UB-FORM-AS.		00830000
000830	09 DA-UB-FORM-AS-Y	PIC 9(4).	00830000
000830	09 DA-UB-FORM-AS-M	PIC 9(2).	00830000
000830	09 DA-UB-FORM-AS-D	PIC 9(2).	00830000
000830	08 DA-UE-FORM-AS.		00830000
000830	09 DA-UE-FORM-AS-Y	PIC 9(4).	00830000
000830	09 DA-UE-FORM-AS-M	PIC 9(2).	00830000
000830	09 DA-UE-FORM-AS-D	PIC 9(2).	00830000
000830	08 DU-ST-FORM-AS	PIC 9(3).	00830000
000830	08 RA-PS-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	08 RA-PR-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	08 AM-CNS-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08 AM-PS-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08 NB-FI-FORM-AS	PIC 9(3).	00830000
000830	08 AM-FIN-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08 CO-RI-FORM-AS	PIC X(1).	00830000
000830	08 NR-IDR-FORM-AS	PIC 9(6).	00830000
000830	08 DA-UER-FORM-AS.		00830000
000830	09 DA-UER-FORM-AS-Y	PIC 9(4).	00830000
000830	09 DA-UER-FORM-AS-M	PIC 9(2).	00830000
000830	09 DA-UER-FORM-AS-D	PIC 9(2).	00830000
000830	08 AM-FBA-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08 AM-FBB-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08 CO-RM-FORM-AS	PIC X(1).	00830000
000830	08 RA-FB-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	08 NB-FB-FORM-AS	PIC 9(2).	00830000
000830	08 AM-FBN-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08 RA-TI-FORM-AS	PIC 9(2)V9(2).	00830000
000830	08 AM-CND-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08 AM-CNDN-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08 AM-CNSN-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08 NR-CD-FORM-AS	PIC 9(4).	00830000
000830	08 CT-SL-FORM-AS	PIC X(1).	00830000
000830	08 NR-ST-FORM-AS	PIC 9(13).	00830000
000830	08 CX-TW-FORM-AS	PIC X(1).	00830000
000830	08 NR-DS-FORM-AS	PIC 9(12).	00830000
000830	08 NR-IDX-FORM-AS	PIC 9(6).	00830000
000830	08 CO-LA-FORM-AS	PIC X(1).	00830000
000830	08 AM-INTER-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08 NB-GROUP-FORM-AS	PIC 9(3).	00830000
000830	08 NR-PRGRP-FORM-AS	PIC 9(6).	00830000
000830	08 NR-NEGRP-FORM-AS	PIC 9(6).	00830000
000830	08 NR-FIGRP-FORM-AS	PIC 9(6).	00830000
000830	08 NR-SEGRP-FORM-AS	PIC 9(3).	00830000
000830	07 NS-IDCP-FORM-AS	PIC 9(3).	00830000
000830	07 AM-CNSSP-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07 RA-CPTOT-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	07 CO-CP-FORM-AS	PIC X.	00830000
000830	07 CO-PERIN-FORM-AS	PIC X.	00830000
000830	07 NS-ID-FORM-AS	PIC 9(2).	00830000
000830	07 CO-DEV-FORM-AS	PIC X(3).	00830000
000830	07 NB-KRDEV-FORM-AS	PIC S9(6)V9(5).	00830000
000830	07 AM-BSDEV-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07 CO-TYP-FORM-AS	PIC X(3).	00830000
000830	07 CO-SUBTP-FORM-AS	PIC X(3).	00830000
000830	07 CO-CONT-FORM-AS	PIC X(3).	00830000
000830	07 CO-TPVGL-FORM-AS	PIC X(2).	00830000
000830	07 AM-VGL-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07 NR-PROD-FORM-AS	PIC X(4).	00830000
000830	07 DA-ACTVR-FORM-AS.		00830000
000830	08 DY-ACTVR-FORM-AS	PIC 9(4).	00830000
000830	08 DM-ACTVR-FORM-AS	PIC 9(2).	00830000
000830	08 DD-ACTVR-FORM-AS	PIC 9(2).	00830000
000830	07 DA-INVTR-FORM-AS.		00830000
000830	08 DY-INVTR-FORM-AS	PIC 9(4).	00830000
000830	08 DM-INVTR-FORM-AS	PIC 9(2).	00830000
000830	08 DD-INVTR-FORM-AS	PIC 9(2).	00830000
000830	07 RA-QZTTI-FORM-AS	PIC S9(3)V999 COMP-3.	00830000
000830	07 RA-DGTTI-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	07 NB-DGTTI-FORM-AS	PIC 9(2).	00830000

000830	07	AM-DGTTI-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07	ST-DGTTI-FORM-AS	PIC 9(1).	00830000
000830	07	ST-PRRW-FORM-AS	PIC 9(1).	00830000
000830	07	RA-GIFT-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	07	AM-GIFT-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07	ST-VOORH-FORM-AS	PIC 9(1).	00830000
000830	07	DA-CPTA-FORM-AS.		00830000
000830	08	DY-CPTA-FORM-AS	PIC 9(4).	00830000
000830	08	DM-CPTA-FORM-AS	PIC 9(2).	00830000
000830	08	DD-CPTA-FORM-AS	PIC 9(2).	00830000
000830	07	CO-EXTOB-FORM-AS	PIC X(1).	00830000
000830	07	CO-PLTOB-FORM-AS	PIC X(1).	00830000
000830	07	RA-TOB-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	07	AM-TOB-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07	RA-TARPH-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	07	AM-TARPH-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07	AM-BASE-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07	AM-FLK-FORM-AS	PIC S9(3)V9(2) COMP-3.	00830000
000830	07	FILLER	PIC X(0014).	00830000
000830	06	FILLER	PIC X(46).	00830000
000840	05	F-SW-ARCH	PIC X.	00840000
000850	05	F-SW-VMBFGD	PIC X.	00850000
000860	05	F-TELLER	PIC S9(11) COMP-3.	00860000
000870	05	F-NR-IS	PIC 9(7).	00870000
000880	05	F-NR-ST	PIC 9(13).	00880000
000890	05	FILLER	PIC X(24).	00890000
000900*				00900000
000910		COPY VMBZ270I.		00910000
000920*				00920000
000930	01	WS-DATUMS.		00930000
000940	02	DATE-YMD.		00940000
000950	03	D-YEAR	PIC 9(2).	00950000
000960	03	D-MONTH	PIC 9(2).	00960000
000970	03	D-DAY	PIC 9(2).	00970000
000980	02	DATE-YMD8.		00980000
000990	03	D-CENT	PIC 9(2).	00990000
001000	03	D-YEAR	PIC 9(2).	01000000
001010	03	D-MONTH	PIC 9(2).	01010000
001020	03	D-DAY	PIC 9(2).	01020000
001030	02	DATE-DMY8.		01030000
001040	03	D-DAY	PIC 9(2).	01040000
001050	03	D-MONTH	PIC 9(2).	01050000
001060	03	D-CENT	PIC 9(2).	01060000
001070	03	D-YEAR	PIC 9(2).	01070000
001080*				01080000
001090/				01090000
001100*				01100000
001110*				01110000
001120*				01120000
001130*				01130000
001140*				01140000
001150*				01150000
001160/				01160000
001170				01170000
001180*DB2		SQLCA, SQL ERROR WORKING STORAGE, DCLGEN INCLUDES, AND CURSOR		01180000
001190*FLAGS*				01190000
001200		EXEC SQL INCLUDE SQLERRWS END-EXEC.		01200000
001210		EXEC SQL INCLUDE SQLCA END-EXEC.		01210000
001220		EXEC SQL INCLUDE DZBFAS TV END-EXEC.		01220000
001230				01230000
001240*DB2		CURSOR END-OF-CURSOR FLAGS		* 01240000
001250	01	WS-DZBFASC1-FLAG	PIC X.	01250000
001260	88	DZBFASC1-NOT-EOC	VALUE 'N'.	01260000
001270	88	DZBFASC1-EOC	VALUE 'Y'.	01270000
001280				01280000
001290				01290000
001300*DB2		CURSOR DECLARATIONS		* 01300000
001310		EXEC SQL		01310000
001320		DECLARE DZBFASC1 CURSOR WITH HOLD FOR		01320000
001330		SELECT		01330000
001340		DA_CPTAFS_479BFAS		01340000
001350		, DA_CRFMAS_479BFAS		01350000
001360		, NR_ISFMAS_479BFAS		01360000
001370		, NR_STFMAS_479BFAS		01370000
001380		, CT_RKTGDSV_479BFAS		01380000
001390		, NR_CIOFMAS_479BFAS		01390000

001400	, NR_IDFMAS_479BFAS			01400000
001410	, CT_OPD_479BFAS			01410000
001420	, CT_OPG_479BFAS			01420000
001430	, NR_ID_479BFAS			01430000
001440	FROM DZBFASTV			01440000
001450	FOR READ ONLY			01450000
001460	END-EXEC.			01460000
001470				01470000
001480	*DB2 DATE CONVERSION WORKING STORAGE			01480000
001490	01 WS-DATUMVELDEN.			01490000
001500	03 DA-CCYYMMDD.			01500000
001510	05 CC	PIC 99	VALUE ZERO.	01510000
001520	05 YY	PIC 99	VALUE ZERO.	01520000
001530	05 MM	PIC 99	VALUE ZERO.	01530000
001540	05 DD	PIC 99	VALUE ZERO.	01540000
001550	03 DA-CCYYMMDD-R REDEFINES DA-CCYYMMDD.			01550000
001560	05 CCYY	PIC 9999.		01560000
001570	05 MM	PIC 99.		01570000
001580	05 DD	PIC 99.		01580000
001590	03 DA-DD-MM-CCYY.			01590000
001600	05 DD	PIC 9(02).		01600000
001610	05 TE-MARKER5	PIC X	VALUE '.'	01610000
001620	05 MM	PIC 9(02).		01620000
001630	05 TE-MARKER6	PIC X	VALUE '.'	01630000
001640	05 CCYY	PIC 9(04).		01640000
001650	02 HELP-DA-CPTAFS-479BFAS	PIC 9(8)	VALUE ZEROES.	01650000
001660	02 HELP-DA-CRFMAS-479BFAS	PIC 9(8)	VALUE ZEROES.	01660000
001670				01670000
001680	LINKAGE SECTION.			01680000
001690	01 PARAM-ZONEDO.			01690000
001700	02 L-PARAM	PIC S9(4)	COMP.	01700000
001710	02 DATUMLS.			01710000
001720	03 DATE-IND	PIC X.		01720000
001730	03 DATE8-LS	PIC 9(8).		01730000
001740	/			01740000
001750	PROCEDURE DIVISION USING PARAM-ZONEDO.			01750000
001760	*DB2: Initialize generated DCLGEN host groups.			01760000
001770	INITIALIZE DCLDZBFASTV.			01770000
001780	*=====			01780000
001790	PERFORM BEGIN-PROGRAMMA THRU BEGIN-PROGRAMMA-EXIT.			01790000
001800	PERFORM BEHANDELING THRU BEHANDELING-EXIT			01800000
001810	UNTIL DZBFASC1-EOC.			01810000
001820	PERFORM 730-CLOSE-DZBFASC1.			01820000
001830	PERFORM EINDE-PROGRAMMA THRU EINDE-PROGRAMMA-EXIT.			01830000
001840	*			01840000
001850	GOBACK.			01850000
001860	*			01860000
001870	*			01870000
001880	BEGIN-PROGRAMMA.			01880000
001890	*-----			01890000
001900	IF DATE-IND = '0'			01900000
001910	ACCEPT DATE-YMD	FROM DATE		01910000
001920	MOVE CORR DATE-YMD	TO DATE-DMY8		01920000
001930	IF D-YEAR OF DATE-YMD > 60			01930000
001940	MOVE 19	TO D-CENT OF DATE-DMY8		01940000
001950	ELSE			01950000
001960	MOVE 20	TO D-CENT OF DATE-DMY8		01960000
001970	END-IF			01970000
001980	ELSE			01980000
001990	MOVE DATE8-LS	TO DATE-DMY8		01990000
002000	MOVE CORR DATE-DMY8	TO DATE-YMD		02000000
002010	END-IF.			02010000
002020	MOVE CORR DATE-DMY8	TO DATE-YMD8.		02020000
002030				02030000
002040	*			02040000
002050	MOVE SPACES TO REC-FORM.			02050000
002060	*			02060000
002070	OPEN INPUT DAARCH			02070000
002080	OUTPUT FORM.			02080000
002090	READ DAARCH INTO VMBZ270I-REC			02090000
002100	AT END			02100000
002110	DISPLAY 'VM7BD200: ** DATE ARCHIVAGE NOT OK **'			02110000
002120	CALL USERABEN.			02120000
002130	DISPLAY 'DATE ARCH : ' DA-ARCH-YMD.			02130000
002140	CLOSE DAARCH.			02140000
002150	*			02150000

002160	MOVE 'VM7BD200'	TO PROGRAM-NAME.	02160000
002170*			02170000
002180*DB2:	Expanded LRF FIRST VMBTL03-R01 WHERE SWEEP-VMBFAS -> VMBFAS		02180000
002190*	WITHIN AR-VMBFRM1.		02190000
002200*DB2:	Converted OBTAIN FIRST VMBFAS WITHIN AR-VMBFRM1.		02200000
002210	PERFORM 710-OPEN-DZBFASC1.		02210000
002220	PERFORM 720-FETCH-DZBFASC1.		02220000
002230*			02230000
002240	BEGIN-PROGRAMMA-EXIT.		02240000
002250*			02250000
002260	EXIT.		02260000
002270*			02270000
002280*			02280000
002290	BEHADELING.		02290000
002300*-----			02300000
002310*DB2:	realigned 2 DB2 DATE host(s) before comparison.		02310000
002320	MOVE ZEROES TO DA-CCYMMDD		02320000
002330	MOVE ZEROES TO HELP-DA-CPTAFS-479BFAS		02330000
002340	MOVE DA-CPTAFS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY		02340000
002350	EVALUATE TRUE		02350000
002360	WHEN DA-DD-MM-CCYY = '01.01.0001'		02360000
002370	MOVE ZEROES TO DA-CCYMMDD-R		02370000
002380	WHEN DA-DD-MM-CCYY = '31.12.9999'		02380000
002390	MOVE '99999999' TO DA-CCYMMDD-R		02390000
002400	WHEN OTHER		02400000
002410	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF DA-CCYMMDD-R		02410000
002420	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYMMDD-R		02420000
002430	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYMMDD-R		02430000
002440	END-EVALUATE		02440000
002450	MOVE DA-CCYMMDD-R TO HELP-DA-CPTAFS-479BFAS		02450000
002460	MOVE ZEROES TO DA-CCYMMDD		02460000
002470	MOVE ZEROES TO HELP-DA-CRFMAS-479BFAS		02470000
002480	MOVE DA-CRFMAS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY		02480000
002490	EVALUATE TRUE		02490000
002500	WHEN DA-DD-MM-CCYY = '01.01.0001'		02500000
002510	MOVE ZEROES TO DA-CCYMMDD-R		02510000
002520	WHEN DA-DD-MM-CCYY = '31.12.9999'		02520000
002530	MOVE '99999999' TO DA-CCYMMDD-R		02530000
002540	WHEN OTHER		02540000
002550	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF DA-CCYMMDD-R		02550000
002560	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYMMDD-R		02560000
002570	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYMMDD-R		02570000
002580	END-EVALUATE		02580000
002590	MOVE DA-CCYMMDD-R TO HELP-DA-CRFMAS-479BFAS		02590000
002600	IF (HELP-DA-CPTAFS-479BFAS < DA-ARCH-YMD AND HELP-DA-CPTAFS-	02600000	
002610	(HELP-DA-CRFMAS-479BFAS < DA-ARCH-YMD AND HELP-DA-CPTAFS-402610000		
002620	ADD 1 TO WS-TELLER	02620000	
002630	MOVE WS-TELLER TO F-TELLER	02630000	
002640*DB2:	Materialised VMBFAS from DCLDZBFASTV - 83 field move(s).	02640000	
002640	MOVE CT-RKTGDSV-479BFAS OF DCLDZBFASTV TO	02640000	
002640	CT-RK-TGDSV OF VMBFAS	02640000	
002640	MOVE NR-CIOFAS-479BFAS OF DCLDZBFASTV TO	02640000	
002640	NR-CIO-FORM-AS OF VMBFAS	02640000	
002640	MOVE DA-CRFMAS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000	
002640	EVALUATE TRUE	02640000	
002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000	
002640	MOVE ZEROES TO DA-CCYMMDD-R	02640000	
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000	
002640	MOVE '99999999' TO DA-CCYMMDD-R	02640000	
002640	WHEN OTHER	02640000	
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000	
002640	DA-CCYMMDD-R	02640000	
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYMMDD-R	02640000	
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYMMDD-R	02640000	
002640	END-EVALUATE	02640000	
002640	MOVE DA-CCYMMDD-R TO DA-CR-FORM-AS OF VMBFAS	02640000	
002640	MOVE NR-IDFAS-479BFAS OF DCLDZBFASTV TO	02640000	
002640	NR-ID-FORM-AS OF VMBFAS	02640000	
002640	MOVE CT-OPD-479BFAS OF DCLDZBFASTV TO	02640000	
002640	CT-OPD-FORM-AS OF VMBFAS	02640000	
002640	MOVE CT-OPG-479BFAS OF DCLDZBFASTV TO	02640000	
002640	CT-OPG-FORM-AS OF VMBFAS	02640000	
002640	MOVE NR-CIWFAS-479BFAS OF DCLDZBFASTV TO	02640000	
002640	NR-CIW-FORM-AS OF VMBFAS	02640000	
002640	MOVE DA-VAFAS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000	
002640	EVALUATE TRUE	02640000	

002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000
002640	MOVE ZEROES TO DA-CCYYMMDD-R	02640000
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000
002640	MOVE '99999999' TO DA-CCYYMMDD-R	02640000
002640	WHEN OTHER	02640000
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000
002640	DA-CCYYMMDD-R	02640000
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMMDD-R	02640000
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMMDD-R	02640000
002640	END-EVALUATE	02640000
002640	MOVE DA-CCYYMMDD-R TO DA-VA-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-VMFAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-VM-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-ISFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-IS-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-RTFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-RT-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-CPFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-CP-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-BASFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-BASE-FORM-AS OF VMBFAS	02640000
002640	MOVE DA-UBFMAS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000
002640	EVALUATE TRUE	02640000
002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000
002640	MOVE ZEROES TO DA-CCYYMMDD-R	02640000
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000
002640	MOVE '99999999' TO DA-CCYYMMDD-R	02640000
002640	WHEN OTHER	02640000
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000
002640	DA-CCYYMMDD-R	02640000
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMMDD-R	02640000
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMMDD-R	02640000
002640	END-EVALUATE	02640000
002640	MOVE DA-CCYYMMDD-R TO DA-UB-FORM-AS OF VMBFAS	02640000
002640	MOVE DA-UEFMAS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000
002640	EVALUATE TRUE	02640000
002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000
002640	MOVE ZEROES TO DA-CCYYMMDD-R	02640000
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000
002640	MOVE '99999999' TO DA-CCYYMMDD-R	02640000
002640	WHEN OTHER	02640000
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000
002640	DA-CCYYMMDD-R	02640000
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMMDD-R	02640000
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMMDD-R	02640000
002640	END-EVALUATE	02640000
002640	MOVE DA-CCYYMMDD-R TO DA-UE-FORM-AS OF VMBFAS	02640000
002640	MOVE DU-STFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	DU-ST-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-PSFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-PS-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-PRFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-PR-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-CNSFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-CNS-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-PSFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-PS-FORM-AS OF VMBFAS	02640000
002640	MOVE NB-FIFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NB-FI-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-FINFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-FIN-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-RIFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-RI-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-IDRFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-IDR-FORM-AS OF VMBFAS	02640000
002640	MOVE DA-UERFMAS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000
002640	EVALUATE TRUE	02640000
002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000
002640	MOVE ZEROES TO DA-CCYYMMDD-R	02640000
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000
002640	MOVE '99999999' TO DA-CCYYMMDD-R	02640000
002640	WHEN OTHER	02640000
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000
002640	DA-CCYYMMDD-R	02640000
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMMDD-R	02640000
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMMDD-R	02640000

002640	END-EVALUATE	02640000
002640	MOVE DA-CCYMMDD-R TO DA-UER-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-FBAFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-FBA-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-FBBFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-FBB-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-RMFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-RM-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-FBFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-FB-FORM-AS OF VMBFAS	02640000
002640	MOVE NB-FBFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NB-FB-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-FBNFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-FBN-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-TIFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-TI-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-CNDFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-CND-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-CNDNFAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-CNDN-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-CNSNFAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-CNSN-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-CDFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-CD-FORM-AS OF VMBFAS	02640000
002640	MOVE CT-SLFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CT-SL-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-STFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-ST-FORM-AS OF VMBFAS	02640000
002640	MOVE CX-TWFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CX-TW-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-DSFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-DS-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-IDXFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-IDX-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-LAFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-LA-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-INTFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-INTER-FORM-AS OF VMBFAS	02640000
002640	MOVE NB-GRPFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NB-GROUP-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-PRGRPFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-PRGRP-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-NEGRPFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-NEGRP-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-FIGRPFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-FIGRP-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-SEGRPFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-SEGRP-FORM-AS OF VMBFAS	02640000
002640	MOVE NS-IDCPFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NS-IDCP-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-CNSSPFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-CNSSP-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-CPTOTFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-CPTOT-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-CPFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-CP-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-PERINFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-PERIN-FORM-AS OF VMBFAS	02640000
002640	MOVE NS-IDFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NS-ID-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-DEVFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-DEV-FORM-AS OF VMBFAS	02640000
002640	MOVE NB-KRDEVFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NB-KRDEV-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-BSDECFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-BSDEV-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-TYPFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-TYP-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-SUBTPFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-SUBTP-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-CONTFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-CONT-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-TPVGLFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-TPVGL-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-VGLFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-VGL-FORM-AS OF VMBFAS	02640000

002640	MOVE NR-PRDFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-PROD-FORM-AS OF VMBFAS	02640000
002640	MOVE DA-ACTVRFS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000
002640	EVALUATE TRUE	02640000
002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000
002640	MOVE ZEROES TO DA-CCYYMMDD-R	02640000
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000
002640	MOVE '99999999' TO DA-CCYYMMDD-R	02640000
002640	WHEN OTHER	02640000
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000
002640	DA-CCYYMMDD-R	02640000
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMMDD-R	02640000
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMMDD-R	02640000
002640	END-EVALUATE	02640000
002640	MOVE DA-CCYYMMDD-R TO DA-ACTVR-FORM-AS OF VMBFAS	02640000
002640	MOVE DA-INVTRFS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000
002640	EVALUATE TRUE	02640000
002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000
002640	MOVE ZEROES TO DA-CCYYMMDD-R	02640000
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000
002640	MOVE '99999999' TO DA-CCYYMMDD-R	02640000
002640	WHEN OTHER	02640000
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000
002640	DA-CCYYMMDD-R	02640000
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMMDD-R	02640000
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMMDD-R	02640000
002640	END-EVALUATE	02640000
002640	MOVE DA-CCYYMMDD-R TO DA-INVTR-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-QZTTIFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-QZTTI-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-DGTTIFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-DGTTI-FORM-AS OF VMBFAS	02640000
002640	MOVE NB-DGTTIFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NB-DGTTI-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-DGTTIFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-DGTTI-FORM-AS OF VMBFAS	02640000
002640	MOVE ST-DGTTIFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	ST-DGTTI-FORM-AS OF VMBFAS	02640000
002640	MOVE ST-PRRWFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	ST-PRRW-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-GIFTFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-GIFT-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-GIFTFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-GIFT-FORM-AS OF VMBFAS	02640000
002640	MOVE ST-VOORHFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	ST-VOORH-FORM-AS OF VMBFAS	02640000
002640	MOVE DA-CPTAFS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000
002640	EVALUATE TRUE	02640000
002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000
002640	MOVE ZEROES TO DA-CCYYMMDD-R	02640000
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000
002640	MOVE '99999999' TO DA-CCYYMMDD-R	02640000
002640	WHEN OTHER	02640000
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000
002640	DA-CCYYMMDD-R	02640000
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMMDD-R	02640000
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMMDD-R	02640000
002640	END-EVALUATE	02640000
002640	MOVE DA-CCYYMMDD-R TO DA-CPTA-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-EXTOBFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-EXTOB-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-PLTOBFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-PLTOB-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-TOBFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-TOB-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-TOBFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-TOB-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-TARPHFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-TARPH-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-TARPH-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-TARPH-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-BASEFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-BASE-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-FLKFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-FLK-FORM-AS OF VMBFAS	02640000
002650	MOVE NR-ISFMAS-479BFAS OF DCLDZBFASTV	02650000

002660		TO W-NR-IS-FORM-AS	02660000
002670	MOVE W-NR-ID-ISSUE TO F-NR-IS		02670000
002680	MOVE NR-STFMAS-479BFAS OF DCLDZBFASTV		02680000
002690		TO F-NR-ST	02690000
002700	WRITE FORM-REC FROM REC-FORM		02700000
002710	END-IF.		02710000
002720*			02720000
002730*DB2:	Expanded LRF NEXT VMBTL03-R01 WHERE SWEEP-VMBFAS -> VMBFAS		02730000
002740*	WITHIN AR-VMBFRM1.		02740000
002750*DB2:	Converted OBTAIN NEXT VMBFAS WITHIN AR-VMBFRM1.		02750000
002760	PERFORM 720-FETCH-DZBFASC1.		02760000
002770*			02770000
002780	BEHANDELING-EXIT.		02780000
002790*			02790000
002800	EXIT.		02800000
002810*			02810000
002820*			02820000
002830	EINDE-PROGRAMMA.		02830000
002840*	-----		02840000
002850	MOVE 'COMMIT' TO SQL-LOCATION.		02850000
002860	EXEC SQL		02860000
002870	COMMIT		02870000
002880	END-EXEC.		02880000
002890	CLOSE FORM.		02890000
002900*			02900000
002910	EINDE-PROGRAMMA-EXIT.		02910000
002920*			02920000
002930	EXIT.		02930000
002940/			02940000
002950*DB2-KEEP:	statement follows a paragraph EXIT and is unreachable.		02950000
002950*DB2-KEEP	CLOSE FORM.		02950000
002960*			02960000
002970			02970000
002980*DB2	GENERATED CURSOR OPEN FETCH CLOSE PARAGRAPHS	*	02980000
002990			02990000
003000	710-OPEN-DZBFASC1.		03000000
003010	MOVE 710 TO SQL-LOCATION		03010000
003020			03020000
003030	EXEC SQL		03030000
003040	OPEN DZBFASC1		03040000
003050	END-EXEC		03050000
003060			03060000
003070	EVALUATE SQLCODE		03070000
003080	WHEN ZERO		03080000
003090	SET DZBFASC1-NOT-EOC TO TRUE		03090000
003100	WHEN OTHER		03100000
003110	DISPLAY 'ERROR WHILE OPENING CURSOR DZBFASC1'		03110000
003120	PERFORM SQLERROR		03120000
003130	END-EVALUATE		03130000
003140	.		03140000
003150			03150000
003160	720-FETCH-DZBFASC1.		03160000
003170	MOVE 720 TO SQL-LOCATION		03170000
003180			03180000
003190	EXEC SQL		03190000
003200	FETCH DZBFASC1		03200000
003210	INTO :DCLDZBFASTV.DA-CPTAFS-479BFAS		03210000
003220	, :DCLDZBFASTV.DA-CRFMAS-479BFAS		03220000
003230	, :DCLDZBFASTV.NR-ISFMAS-479BFAS		03230000
003240	, :DCLDZBFASTV.NR-STFMAS-479BFAS		03240000
003250	, :DCLDZBFASTV.CT-RKTGDSV-479BFAS		03250000
003260	, :DCLDZBFASTV.NR-CIOFMAS-479BFAS		03260000
003270	, :DCLDZBFASTV.NR-IDFMAS-479BFAS		03270000
003280	, :DCLDZBFASTV.CT-OPD-479BFAS		03280000
003290	, :DCLDZBFASTV.CT-OPG-479BFAS		03290000
003300	, :DCLDZBFASTV.NR-ID-479BFAS		03300000
003310	END-EXEC		03310000
003320			03320000
003330	EVALUATE SQLCODE		03330000
003340	WHEN ZERO		03340000
003350	CONTINUE		03350000
003360	WHEN 100		03360000
003370	SET DZBFASC1-EOC TO TRUE		03370000
003380	WHEN OTHER		03380000
003390	DISPLAY 'ERROR WHILE FETCHING CURSOR DZBFASC1'		03390000
003400	PERFORM SQLERROR		03400000

003410	END-EVALUATE	03410000
003420	.	03420000
003430		03430000
003440	730-CLOSE-DZBFASC1.	03440000
003450	MOVE 730 TO SQL-LOCATION	03450000
003460		03460000
003470	EXEC SQL	03470000
003480	CLOSE DZBFASC1	03480000
003490	END-EXEC	03490000
003500		03500000
003510	EVALUATE SQLCODE	03510000
003520	WHEN ZERO	03520000
003530	CONTINUE	03530000
003540	WHEN OTHER	03540000
003550	DISPLAY 'ERROR WHILE CLOSING CURSOR DZBFASC1'	03550000
003560	PERFORM SQLERROR	03560000
003570	END-EVALUATE	03570000
003580	.	03580000
003590		03590000
003600	*SQLERROR ROUTINE.	03600000
003610	EXEC SQL	03610000
003620	INCLUDE SQLERROR	03620000
003630	END-EXEC.	03630000

=====

FILE: debug_indent_out.txt

ENCODING: utf-8

=====

000010	IDENTIFICATION DIVISION.	00010000
000020	*=====	00020000
000030	PROGRAM-ID. VMDZ7200.	00030000
000040	AUTHOR. J-M BOOGAERTS 88/07.	00040000
000050		00050000
000060	*****	00060000
000070	*	00070000
000080	ARCHIVAGE : EXTRACT DES FORMULAIRES A ARCHIVER	00080000
000090	===== (SOUSCRIPTIONS ET ANNULATIONS SOUSCRIPTIONS)	00090000
000100	*	00100000
000110	WULTEPUTTE KRISTINE : (10/7/91) :	00110000
000120	*	00120000
000130	TOEVOEGEN VAN NIEUWE PLAGES VOOR BCI1	00130000
000140	*	00140000
000150	DE SCHEPPER KOEN : (18/12/92) :	00150000
000160	*	00160000
000170	AANPASSING VOOR SLIK : UITBREIDING FORM 270 -> 390	00170000
000180	*	00180000
000190	CLEMENT J. (06/93) : TOEVOEGEN PLAGES	00190000
000200	*	00200000
000210	DE SCHEPPER KOEN (04/02/94) :	00210000
000220	PROGRAMMA VOLLEDIG HERSCHREVEN : GEEN SELECTIE OP	00220000
000230	WAARDECODE MEER.	00230000
000240	*	00240000
000250	POMPONIO S (03/97)	00250000
000260	AANPASSING EURO2000 : PARMDATE EN 8 POSITIONS	00260000
000270	RECORD VMBFAS 388 -> 432 BYTES	00270000
000280	*	00280000
000290	J-M BOOGAERTS 2005/02	00290000
000300	INTÉGRATION ARCHIVAGE IUNS P8	00300000
000310	*	00310000
000320	*****	00320000
000330		00330000
000340	ENVIRONMENT DIVISION.	00340000
000350	*=====	00350000
000360	CONFIGURATION SECTION.	00360000
000370	*-----	00370000
000380	SPECIAL-NAMES.	00380000
000390	DECIMAL-POINT IS COMMA.	00390000
000400	*	00400000
000410	INPUT-OUTPUT SECTION.	00410000
000420	*-----	00420000
000430	FILE-CONTROL.	00430000
000440	SELECT DAARCH ASSIGN TO UT-S-DD005.	00440000
000450	SELECT FORM ASSIGN TO UT-S-DD010.	00450000
000460	*	00460000

000470*	-----				00470000
000480/					00480000
000490	DATA DIVISION.				00490000
000500*	=====				00500000
000510	FILE SECTION.				00510000
000520*	-----				00520000
000530*	----- DATE ARCHIVAGE -----				00530000
000540*					00540000
000550	FD DAARCH	RECORDING F			00550000
000560		RECORD CONTAINS 20			00560000
000570		BLOCK CONTAINS 0			00570000
000580		LABEL RECORD STANDARD.			00580000
000590	01 DAARCH-REC		PIC X(20).		00590000
000600*					00600000
000610*	----- GESELECTEERDE FORMULIEREN -----				00610000
000620*					00620000
000630	FD FORM	RECORDING F			00630000
000640		RECORD CONTAINS 530			00640000
000650		BLOCK CONTAINS 0			00650000
000660		LABEL RECORD STANDARD.			00660000
000670	01 FORM-REC		PIC X(530).		00670000
000680/					00680000
000690*	-----				00690000
000700*					00700000
000710	WORKING-STORAGE SECTION.				00710000
000720*	-----				00720000
000730*					00730000
000740	77 USERABEN	PIC X(8)	VALUE 'USERABEN'.		00740000
000750	77 WS-TELLER	PIC S9(11)	COMP-3 VALUE ZERO.		00750000
000760*					00760000
000770	01 W-NR-IS-FORM-AS	PIC 9(9).			00770000
000780	01 FILLER REDEFINES W-NR-IS-FORM-AS.				00780000
000790	05 W-NR-ID-ISSUE	PIC 9(7).			00790000
000800	05 W-NR-CD-ISSUE	PIC 9(2).			00800000
000810*					00810000
000820	01 REC-FORM.				00820000
000830	05 F-FORM.				00830000
000830*	DB2 MATERIALISED IDMS RECORD VMBFAS				00830000
000830	06 VMBFAS.				00830000
000830	07 KY-VMBFAS.				00830000
000830	08 NR-RK-GOOD-V02.				00830000
000830	09 CT-RK-TGDSERV.				00830000
000830	10 CT-RK-TGDSV	PIC X(4).			00830000
000830	10 CT-RK-TGOOD	REDEFINES CT-RK-TGDSV			00830000
000830		PIC 9(4).			00830000
000830	09 GR-ID-GOOD-V02.				00830000
000830	10 NR-CIO-FORM-AS	PIC 9(8) COMP-3.			00830000
000830	10 DA-CR-FORM-AS	PIC 9(8) COMP-3.			00830000
000830	10 NR-ID-FORM-AS	PIC 9(6) COMP-3.			00830000
000830	10 CT-OP-FORM-AS.				00830000
000830	11 CT-OPD-FORM-AS	PIC S9(3) COMP-3.			00830000
000830	11 CT-OPG-FORM-AS	PIC 9(3) COMP-3.			00830000
000830	10 FILLER	PIC X(2).			00830000
000830	07 AT-FORM-AS.				00830000
000830	08 NR-CIW-FORM-AS	PIC X(8).			00830000
000830	08 DA-VA-FORM-AS.				00830000
000830	09 DA-VA-FORM-AS-Y	PIC 9(4).			00830000
000830	09 DA-VA-FORM-AS-M	PIC 9(2).			00830000
000830	09 DA-VA-FORM-AS-D	PIC 9(2).			00830000
000830	08 CO-VM-FORM-AS	PIC X(1).			00830000
000830	08 NR-IS-FORM-AS	PIC 9(9).			00830000
000830	08 RA-RT-FORM-AS	PIC S9(3)V9(3) COMP-3.			00830000
000830	08 RA-CP-FORM-AS	PIC S9(3)V9(3) COMP-3.			00830000
000830	08 RA-BASE-FORM-AS	PIC S9(3)V9(3) COMP-3.			00830000
000830	08 DA-UB-FORM-AS.				00830000
000830	09 DA-UB-FORM-AS-Y	PIC 9(4).			00830000
000830	09 DA-UB-FORM-AS-M	PIC 9(2).			00830000
000830	09 DA-UB-FORM-AS-D	PIC 9(2).			00830000
000830	08 DA-UE-FORM-AS.				00830000
000830	09 DA-UE-FORM-AS-Y	PIC 9(4).			00830000
000830	09 DA-UE-FORM-AS-M	PIC 9(2).			00830000
000830	09 DA-UE-FORM-AS-D	PIC 9(2).			00830000
000830	08 DU-ST-FORM-AS	PIC 9(3).			00830000
000830	08 RA-PS-FORM-AS	PIC S9(3)V9(3) COMP-3.			00830000
000830	08 RA-PR-FORM-AS	PIC S9(3)V9(3) COMP-3.			00830000
000830	08 AM-CNS-FORM-AS	PIC S9(13)V99 COMP-3.			00830000

000830	08	AM-PS-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08	NB-FI-FORM-AS	PIC 9(3).	00830000
000830	08	AM-FIN-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08	CO-RI-FORM-AS	PIC X(1).	00830000
000830	08	NR-IDR-FORM-AS	PIC 9(6).	00830000
000830	08	DA-UER-FORM-AS.		00830000
000830	09	DA-UER-FORM-AS-Y	PIC 9(4).	00830000
000830	09	DA-UER-FORM-AS-M	PIC 9(2).	00830000
000830	09	DA-UER-FORM-AS-D	PIC 9(2).	00830000
000830	08	AM-FBA-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08	AM-FBB-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08	CO-RM-FORM-AS	PIC X(1).	00830000
000830	08	RA-FB-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	08	NB-FB-FORM-AS	PIC 9(2).	00830000
000830	08	AM-FBN-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08	RA-TI-FORM-AS	PIC 9(2)V9(2).	00830000
000830	08	AM-CND-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08	AM-CNDN-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08	AM-CNSN-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08	NR-CD-FORM-AS	PIC 9(4).	00830000
000830	08	CT-SL-FORM-AS	PIC X(1).	00830000
000830	08	NR-ST-FORM-AS	PIC 9(13).	00830000
000830	08	CX-TW-FORM-AS	PIC X(1).	00830000
000830	08	NR-DS-FORM-AS	PIC 9(12).	00830000
000830	08	NR-IDX-FORM-AS	PIC 9(6).	00830000
000830	08	CO-LA-FORM-AS	PIC X(1).	00830000
000830	08	AM-INTER-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	08	NB-GROUP-FORM-AS	PIC 9(3).	00830000
000830	08	NR-PRGRP-FORM-AS	PIC 9(6).	00830000
000830	08	NR-NEGRP-FORM-AS	PIC 9(6).	00830000
000830	08	NR-FIGRP-FORM-AS	PIC 9(6).	00830000
000830	08	NR-SEGRP-FORM-AS	PIC 9(3).	00830000
000830	07	NS-IDCP-FORM-AS	PIC 9(3).	00830000
000830	07	AM-CNSSP-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07	RA-CPTOT-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	07	CO-CP-FORM-AS	PIC X.	00830000
000830	07	CO-PERIN-FORM-AS	PIC X.	00830000
000830	07	NS-ID-FORM-AS	PIC 9(2).	00830000
000830	07	CO-DEV-FORM-AS	PIC X(3).	00830000
000830	07	NB-KRDEV-FORM-AS	PIC S9(6)V9(5).	00830000
000830	07	AM-BSDEV-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07	CO-TYP-FORM-AS	PIC X(3).	00830000
000830	07	CO-SUBTP-FORM-AS	PIC X(3).	00830000
000830	07	CO-CONT-FORM-AS	PIC X(3).	00830000
000830	07	CO-TPVGL-FORM-AS	PIC X(2).	00830000
000830	07	AM-VGL-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07	NR-PROD-FORM-AS	PIC X(4).	00830000
000830	07	DA-ACTVR-FORM-AS.		00830000
000830	08	DY-ACTVR-FORM-AS	PIC 9(4).	00830000
000830	08	DM-ACTVR-FORM-AS	PIC 9(2).	00830000
000830	08	DD-ACTVR-FORM-AS	PIC 9(2).	00830000
000830	07	DA-INVTR-FORM-AS.		00830000
000830	08	DY-INVTR-FORM-AS	PIC 9(4).	00830000
000830	08	DM-INVTR-FORM-AS	PIC 9(2).	00830000
000830	08	DD-INVTR-FORM-AS	PIC 9(2).	00830000
000830	07	RA-QZTTI-FORM-AS	PIC S9(3)V999 COMP-3.	00830000
000830	07	RA-DGTTI-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	07	NB-DGTTI-FORM-AS	PIC 9(2).	00830000
000830	07	AM-DGTTI-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07	ST-DGTTI-FORM-AS	PIC 9(1).	00830000
000830	07	ST-PRRW-FORM-AS	PIC 9(1).	00830000
000830	07	RA-GIFT-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	07	AM-GIFT-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07	ST-VOORH-FORM-AS	PIC 9(1).	00830000
000830	07	DA-CPTA-FORM-AS.		00830000
000830	08	DY-CPTA-FORM-AS	PIC 9(4).	00830000
000830	08	DM-CPTA-FORM-AS	PIC 9(2).	00830000
000830	08	DD-CPTA-FORM-AS	PIC 9(2).	00830000
000830	07	CO-EXTOB-FORM-AS	PIC X(1).	00830000
000830	07	CO-PLTOB-FORM-AS	PIC X(1).	00830000
000830	07	RA-TOB-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	07	AM-TOB-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07	RA-TARPH-FORM-AS	PIC S9(3)V9(3) COMP-3.	00830000
000830	07	AM-TARPH-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07	AM-BASE-FORM-AS	PIC S9(13)V99 COMP-3.	00830000
000830	07	AM-FLK-FORM-AS	PIC S9(3)V9(2) COMP-3.	00830000

000830	07 FILLER	PIC X(0014).	00830000
000830	06 FILLER	PIC X(46).	00830000
000840	05 F-SW-ARCH	PIC X.	00840000
000850	05 F-SW-VMBFGD	PIC X.	00850000
000860	05 F-TELLER	PIC S9(11) COMP-3.	00860000
000870	05 F-NR-IS	PIC 9(7).	00870000
000880	05 F-NR-ST	PIC 9(13).	00880000
000890	05 FILLER	PIC X(24).	00890000
000900*			00900000
000910	COPY VMBZ270I.		00910000
000920*			00920000
000930	01 WS-DATUMS.		00930000
000940	02 DATE-YMD.		00940000
000950	03 D-YEAR	PIC 9(2).	00950000
000960	03 D-MONTH	PIC 9(2).	00960000
000970	03 D-DAY	PIC 9(2).	00970000
000980	02 DATE-YMD8.		00980000
000990	03 D-CENT	PIC 9(2).	00990000
001000	03 D-YEAR	PIC 9(2).	01000000
001010	03 D-MONTH	PIC 9(2).	01010000
001020	03 D-DAY	PIC 9(2).	01020000
001030	02 DATE-DMY8.		01030000
001040	03 D-DAY	PIC 9(2).	01040000
001050	03 D-MONTH	PIC 9(2).	01050000
001060	03 D-CENT	PIC 9(2).	01060000
001070	03 D-YEAR	PIC 9(2).	01070000
001080*			01080000
001090/			01090000
001100*			01100000
001110*			01110000
001120*			01120000
001130*			01130000
001140*			01140000
001150*			01150000
001160/			01160000
001170			01170000
001180*DB2	SQLCA, SQL ERROR WORKING STORAGE, DCLGEN INCLUDES, AND CURSOR		01180000
001190*FLAGS*			01190000
001200	EXEC SQL INCLUDE SQLERRWS END-EXEC.		01200000
001210	EXEC SQL INCLUDE SQLCA END-EXEC.		01210000
001220	EXEC SQL INCLUDE DZBFAS TV END-EXEC.		01220000
001230			01230000
001240*DB2	CURSOR END-OF-CURSOR FLAGS		* 01240000
001250	01 WS-DZBFASC1-FLAG	PIC X.	01250000
001260	88 DZBFASC1-NOT-EOC	VALUE 'N'.	01260000
001270	88 DZBFASC1-EOC	VALUE 'Y'.	01270000
001280			01280000
001290			01290000
001300*DB2	CURSOR DECLARATIONS		* 01300000
001310	EXEC SQL		01310000
001320	DECLARE DZBFASC1 CURSOR WITH HOLD FOR		01320000
001330	SELECT		01330000
001340	DA_CPTAFS_479BFAS		01340000
001350	, DA_CRFMAS_479BFAS		01350000
001360	, NR_ISFMAS_479BFAS		01360000
001370	, NR_STFMAS_479BFAS		01370000
001380	, CT_RKTGDSV_479BFAS		01380000
001390	, NR_CIOFMAS_479BFAS		01390000
001400	, NR_IDFMAS_479BFAS		01400000
001410	, CT_OPD_479BFAS		01410000
001420	, CT_OPG_479BFAS		01420000
001430	, NR_ID_479BFAS		01430000
001440	FROM DZBFAS TV		01440000
001450	FOR READ ONLY		01450000
001460	END-EXEC.		01460000
001470			01470000
001480*DB2	DATE CONVERSION WORKING STORAGE		01480000
001490	01 WS-DATUMVELDEN.		01490000
001500	03 DA-CCYYMMDD.		01500000
001510	05 CC	PIC 99	VALUE ZERO.
001520	05 YY	PIC 99	VALUE ZERO.
001530	05 MM	PIC 99	VALUE ZERO.
001540	05 DD	PIC 99	VALUE ZERO.
001550	03 DA-CCYYMMDD-R REDEFINES DA-CCYYMMDD.		01550000
001560	05 CCYY	PIC 9999.	01560000
001570	05 MM	PIC 99.	01570000

001580	05 DD	PIC 99.	01580000
001590	03 DA-DD-MM-CCYY.		01590000
001600	05 DD	PIC 9(02).	01600000
001610	05 TE-MARKER5	PIC X VALUE '.'	01610000
001620	05 MM	PIC 9(02).	01620000
001630	05 TE-MARKER6	PIC X VALUE '.'	01630000
001640	05 CCYY	PIC 9(04).	01640000
001650	02 HELP-DA-CPTAFS-479BFAS	PIC 9(8) VALUE ZEROES.	01650000
001660	02 HELP-DA-CRFMAS-479BFAS	PIC 9(8) VALUE ZEROES.	01660000
001670			01670000
001680	LINKAGE SECTION.		01680000
001690	01 PARAM-ZONEDO.		01690000
001700	02 L-PARAM	PIC S9(4) COMP.	01700000
001710	02 DATUMLS.		01710000
001720	03 DATE-IND	PIC X.	01720000
001730	03 DATE8-LS	PIC 9(8).	01730000
001740	/		01740000
001750	PROCEDURE DIVISION USING PARAM-ZONEDO.		01750000
001760	*DB2: Initialize generated DCLGEN host groups.		01760000
001770	INITIALIZE DCLDZBFAS TV.		01770000
001780	*=====		01780000
001790	PERFORM BEGIN-PROGRAMMA THRU BEGIN-PROGRAMMA-EXIT.		01790000
001800	PERFORM BEHANDELING THRU BEHANDELING-EXIT		01800000
001810	UNTIL DZBFASC1-EOC.		01810000
001820	PERFORM 730-CLOSE-DZBFASC1.		01820000
001830	PERFORM EINDE-PROGRAMMA THRU EINDE-PROGRAMMA-EXIT.		01830000
001840	*		01840000
001850	GOBACK.		01850000
001860	*		01860000
001870	*		01870000
001880	BEGIN-PROGRAMMA.		01880000
001890	*-----		01890000
001900	IF DATE-IND = '0'		01900000
001910	ACCEPT DATE-YMD	FROM DATE	01910000
001920	MOVE CORR DATE-YMD	TO DATE-DMY8	01920000
001930	IF D-YEAR OF DATE-YMD > 60		01930000
001940	MOVE 19	TO D-CENT OF DATE-DMY8	01940000
001950	ELSE		01950000
001960	MOVE 20	TO D-CENT OF DATE-DMY8	01960000
001970	END-IF		01970000
001980	ELSE		01980000
001990	MOVE DATE8-LS	TO DATE-DMY8	01990000
002000	MOVE CORR DATE-DMY8	TO DATE-YMD	02000000
002010	END-IF.		02010000
002020	MOVE CORR DATE-DMY8	TO DATE-YMD8.	02020000
002030			02030000
002040	*		02040000
002050	MOVE SPACES TO REC-FORM.		02050000
002060	*		02060000
002070	OPEN INPUT DAARCH		02070000
002080	OUTPUT FORM.		02080000
002090	READ DAARCH INTO VMBZ270I-REC		02090000
002100	AT END		02100000
002110	DISPLAY 'VM7BD200: ** DATE ARCHIVAGE NOT OK **'		02110000
002120	CALL USERABEN.		02120000
002130	DISPLAY 'DATE ARCH : ' DA-ARCH-YMD.		02130000
002140	CLOSE DAARCH.		02140000
002150	*		02150000
002160	MOVE 'VM7BD200'	TO PROGRAM-NAME.	02160000
002170	*		02170000
002180	*DB2: Expanded LRF FIRST VMBTL03-R01 WHERE SWEEP-VMBFAS -> VMBFAS		02180000
002190	* WITHIN AR-VMBFRM1.		02190000
002200	*DB2: Converted OBTAIN FIRST VMBFAS WITHIN AR-VMBFRM1.		02200000
002210	PERFORM 710-OPEN-DZBFASC1.		02210000
002220	PERFORM 720-FETCH-DZBFASC1.		02220000
002230	*		02230000
002240	BEGIN-PROGRAMMA-EXIT.		02240000
002250	*		02250000
002260	EXIT.		02260000
002270	*		02270000
002280	*		02280000
002290	BEHANDELING.		02290000
002300	*-----		02300000
002310	*DB2: realigned 2 DB2 DATE host(s) before comparison.		02310000
002320	MOVE ZEROES TO DA-CCYYMDD		02320000
002330	MOVE ZEROES TO HELP-DA-CPTAFS-479BFAS		02330000

002340	MOVE DA-CPTAFS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02340000
002350	EVALUATE TRUE	02350000
002360	WHEN DA-DD-MM-CCYY = '01.01.0001'	02360000
002370	MOVE ZEROES TO DA-CCYYMDD-R	02370000
002380	WHEN DA-DD-MM-CCYY = '31.12.9999'	02380000
002390	MOVE '99999999' TO DA-CCYYMDD-R	02390000
002400	WHEN OTHER	02400000
002410	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF DA-CCYYMDD-R	02410000
002420	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMDD-R	02420000
002430	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMDD-R	02430000
002440	END-EVALUATE	02440000
002450	MOVE DA-CCYYMDD-R TO HELP-DA-CPTAFS-479BFAS	02450000
002460	MOVE ZEROES TO DA-CCYYMDD	02460000
002470	MOVE ZEROES TO HELP-DA-CRFMAS-479BFAS	02470000
002480	MOVE DA-CRFMAS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02480000
002490	EVALUATE TRUE	02490000
002500	WHEN DA-DD-MM-CCYY = '01.01.0001'	02500000
002510	MOVE ZEROES TO DA-CCYYMDD-R	02510000
002520	WHEN DA-DD-MM-CCYY = '31.12.9999'	02520000
002530	MOVE '99999999' TO DA-CCYYMDD-R	02530000
002540	WHEN OTHER	02540000
002550	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF DA-CCYYMDD-R	02550000
002560	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMDD-R	02560000
002570	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMDD-R	02570000
002580	END-EVALUATE	02580000
002590	MOVE DA-CCYYMDD-R TO HELP-DA-CRFMAS-479BFAS	02590000
002600	IF (HELP-DA-CPTAFS-479BFAS < DA-ARCH-YMD AND HELP-DA-CPTAFS-	02600000
002610	(HELP-DA-CRFMAS-479BFAS < DA-ARCH-YMD AND HELP-DA-CPTAFS-	02610000
002620	ADD 1 TO WS-TELLER	02620000
002630	MOVE WS-TELLER TO F-TELLER	02630000
002640	*DB2: Materialised VMBFAS from DCLDZBFASTV - 83 field move(s).	02640000
002640	MOVE CT-RKTGDSV-479BFAS OF DCLDZBFASTV TO	02640000
002640	CT-RK-TGDSV OF VMBFAS	02640000
002640	MOVE NR-CIOFAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-CIO-FORM-AS OF VMBFAS	02640000
002640	MOVE DA-CRFMAS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000
002640	EVALUATE TRUE	02640000
002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000
002640	MOVE ZEROES TO DA-CCYYMDD-R	02640000
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000
002640	MOVE '99999999' TO DA-CCYYMDD-R	02640000
002640	WHEN OTHER	02640000
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000
002640	DA-CCYYMDD-R	02640000
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMDD-R	02640000
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMDD-R	02640000
002640	END-EVALUATE	02640000
002640	MOVE DA-CCYYMDD-R TO DA-CR-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-IDFAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-ID-FORM-AS OF VMBFAS	02640000
002640	MOVE CT-OPD-479BFAS OF DCLDZBFASTV TO	02640000
002640	CT-OPD-FORM-AS OF VMBFAS	02640000
002640	MOVE CT-OPG-479BFAS OF DCLDZBFASTV TO	02640000
002640	CT-OPG-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-CIWFAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-CIW-FORM-AS OF VMBFAS	02640000
002640	MOVE DA-VAFAS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000
002640	EVALUATE TRUE	02640000
002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000
002640	MOVE ZEROES TO DA-CCYYMDD-R	02640000
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000
002640	MOVE '99999999' TO DA-CCYYMDD-R	02640000
002640	WHEN OTHER	02640000
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000
002640	DA-CCYYMDD-R	02640000
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMDD-R	02640000
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMDD-R	02640000
002640	END-EVALUATE	02640000
002640	MOVE DA-CCYYMDD-R TO DA-VA-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-VMFAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-VM-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-ISFAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-IS-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-RTFAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-RT-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-CPFAS-479BFAS OF DCLDZBFASTV TO	02640000

002640	RA-CP-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-BASFAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-BASE-FORM-AS OF VMBFAS	02640000
002640	MOVE DA-UBFMAS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000
002640	EVALUATE TRUE	02640000
002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000
002640	MOVE ZEROES TO DA-CCYMMDD-R	02640000
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000
002640	MOVE '99999999' TO DA-CCYMMDD-R	02640000
002640	WHEN OTHER	02640000
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000
002640	DA-CCYMMDD-R	02640000
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYMMDD-R	02640000
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYMMDD-R	02640000
002640	END-EVALUATE	02640000
002640	MOVE DA-CCYMMDD-R TO DA-UB-FORM-AS OF VMBFAS	02640000
002640	MOVE DA-UEFMAS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000
002640	EVALUATE TRUE	02640000
002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000
002640	MOVE ZEROES TO DA-CCYMMDD-R	02640000
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000
002640	MOVE '99999999' TO DA-CCYMMDD-R	02640000
002640	WHEN OTHER	02640000
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000
002640	DA-CCYMMDD-R	02640000
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYMMDD-R	02640000
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYMMDD-R	02640000
002640	END-EVALUATE	02640000
002640	MOVE DA-CCYMMDD-R TO DA-UE-FORM-AS OF VMBFAS	02640000
002640	MOVE DU-STFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	DU-ST-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-PSFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-PS-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-PRFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-PR-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-CNSFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-CNS-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-PSFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-PS-FORM-AS OF VMBFAS	02640000
002640	MOVE NB-FIFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NB-FI-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-FINFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-FIN-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-RIFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-RI-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-IDRFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-IDR-FORM-AS OF VMBFAS	02640000
002640	MOVE DA-UERFMAS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000
002640	EVALUATE TRUE	02640000
002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000
002640	MOVE ZEROES TO DA-CCYMMDD-R	02640000
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000
002640	MOVE '99999999' TO DA-CCYMMDD-R	02640000
002640	WHEN OTHER	02640000
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000
002640	DA-CCYMMDD-R	02640000
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYMMDD-R	02640000
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYMMDD-R	02640000
002640	END-EVALUATE	02640000
002640	MOVE DA-CCYMMDD-R TO DA-UER-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-FBAFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-FBA-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-FBBFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-FBB-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-RMFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-RM-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-FBFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-FB-FORM-AS OF VMBFAS	02640000
002640	MOVE NB-FBFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NB-FB-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-FBNFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-FBN-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-TIFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-TI-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-CNDFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-CND-FORM-AS OF VMBFAS	02640000

002640	MOVE AM-CNDNFAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-CNDN-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-CNSNFAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-CNSN-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-CDFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-CD-FORM-AS OF VMBFAS	02640000
002640	MOVE CT-SLFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CT-SL-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-STFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-ST-FORM-AS OF VMBFAS	02640000
002640	MOVE CX-TWFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CX-TW-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-DSFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-DS-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-IDXFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-IDX-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-LAFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-LA-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-INTFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-INTER-FORM-AS OF VMBFAS	02640000
002640	MOVE NB-GRPFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NB-GROUP-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-PRGRPFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-PRGRP-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-NEGRPFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-NEGRP-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-FIGRPFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-FIGRP-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-SEGRPFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-SEGRP-FORM-AS OF VMBFAS	02640000
002640	MOVE NS-IDCPFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NS-IDCP-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-CNSSPFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-CNSSP-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-CPTOTFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-CPTOT-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-CPFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-CP-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-PERINFAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-PERIN-FORM-AS OF VMBFAS	02640000
002640	MOVE NS-IDFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NS-ID-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-DEVFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-DEV-FORM-AS OF VMBFAS	02640000
002640	MOVE NB-KRDEVFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NB-KRDEV-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-BSDECFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-BSDEV-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-TYPFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-TYP-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-SUBTPFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-SUBTP-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-CONTFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-CONT-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-TPVGLFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-TPVGL-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-VGLFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-VGL-FORM-AS OF VMBFAS	02640000
002640	MOVE NR-PRDFMAS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NR-PROD-FORM-AS OF VMBFAS	02640000
002640	MOVE DA-ACTVRFS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000
002640	EVALUATE TRUE	02640000
002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000
002640	MOVE ZEROES TO DA-CCYYMMDD-R	02640000
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000
002640	MOVE '99999999' TO DA-CCYYMMDD-R	02640000
002640	WHEN OTHER	02640000
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000
002640	DA-CCYYMMDD-R	02640000
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMMDD-R	02640000
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMMDD-R	02640000
002640	END-EVALUATE	02640000
002640	MOVE DA-CCYYMMDD-R TO DA-ACTVR-FORM-AS OF VMBFAS	02640000
002640	MOVE DA-INVTRFS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000
002640	EVALUATE TRUE	02640000
002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000

002640	MOVE ZEROES TO DA-CCYMMDD-R	02640000
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000
002640	MOVE '99999999' TO DA-CCYMMDD-R	02640000
002640	WHEN OTHER	02640000
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000
002640	DA-CCYMMDD-R	02640000
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYMMDD-R	02640000
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYMMDD-R	02640000
002640	END-EVALUATE	02640000
002640	MOVE DA-CCYMMDD-R TO DA-INVTR-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-QZTTIFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-QZTTI-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-DGTTIFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-DGTTI-FORM-AS OF VMBFAS	02640000
002640	MOVE NB-DGTTIFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	NB-DGTTI-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-DGTTIFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-DGTTI-FORM-AS OF VMBFAS	02640000
002640	MOVE ST-DGTTIFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	ST-DGTTI-FORM-AS OF VMBFAS	02640000
002640	MOVE ST-PRRWFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	ST-PRRW-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-GIFTFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-GIFT-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-GIFTFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-GIFT-FORM-AS OF VMBFAS	02640000
002640	MOVE ST-VOORHFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	ST-VOORH-FORM-AS OF VMBFAS	02640000
002640	MOVE DA-CPTAFS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY	02640000
002640	EVALUATE TRUE	02640000
002640	WHEN DA-DD-MM-CCYY = '01.01.0001'	02640000
002640	MOVE ZEROES TO DA-CCYMMDD-R	02640000
002640	WHEN DA-DD-MM-CCYY = '31.12.9999'	02640000
002640	MOVE '99999999' TO DA-CCYMMDD-R	02640000
002640	WHEN OTHER	02640000
002640	MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF	02640000
002640	DA-CCYMMDD-R	02640000
002640	MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYMMDD-R	02640000
002640	MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYMMDD-R	02640000
002640	END-EVALUATE	02640000
002640	MOVE DA-CCYMMDD-R TO DA-CPTA-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-EXTOBFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-EXTOB-FORM-AS OF VMBFAS	02640000
002640	MOVE CO-PLTOBFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	CO-PLTOB-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-TOBFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-TOB-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-TOBFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-TOB-FORM-AS OF VMBFAS	02640000
002640	MOVE RA-TARPHFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	RA-TARPH-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-TARPH-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-TARPH-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-BASEFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-BASE-FORM-AS OF VMBFAS	02640000
002640	MOVE AM-FLKFS-479BFAS OF DCLDZBFASTV TO	02640000
002640	AM-FLK-FORM-AS OF VMBFAS	02640000
002650	MOVE NR-ISFMAS-479BFAS OF DCLDZBFASTV	02650000
002660	TO W-NR-IS-FORM-AS	02660000
002670	MOVE W-NR-ID-ISSUE TO F-NR-IS	02670000
002680	MOVE NR-STFMAS-479BFAS OF DCLDZBFASTV	02680000
002690	TO F-NR-ST	02690000
002700	WRITE FORM-REC FROM REC-FORM	02700000
002710	END-IF.	02710000
002720*		02720000
002730*DB2:	Expanded LRF NEXT VMBTL03-R01 WHERE SWEEP-VMBFAS -> VMBFAS	02730000
002740*	WITHIN AR-VMBFRM1.	02740000
002750*DB2:	Converted OBTAIN NEXT VMBFAS WITHIN AR-VMBFRM1.	02750000
002760	PERFORM 720-FETCH-DZBFASC1.	02760000
002770*		02770000
002780	BEHANDELING-EXIT.	02780000
002790*		02790000
002800	EXIT.	02800000
002810*		02810000
002820*		02820000
002830	EINDE-PROGRAMMA.	02830000

002840*-----	02840000
002850 MOVE 'COMMIT' TO SQL-LOCATION.	02850000
002860 EXEC SQL	02860000
002870 COMMIT	02870000
002880 END-EXEC.	02880000
002890 CLOSE FORM.	02890000
002900*	02900000
002910 EINDE-PROGRAMMA-EXIT.	02910000
002920*	02920000
002930 EXIT.	02930000
002940/	02940000
002950*DB2-KEEP: statement follows a paragraph EXIT and is unreachable.	02950000
002950*DB2-KEEP CLOSE FORM.	02950000
002960*	02960000
002970	02970000
002980*DB2 GENERATED CURSOR OPEN FETCH CLOSE PARAGRAPHS	* 02980000
002990	02990000
003000 710-OPEN-DZBFASC1.	03000000
003010 MOVE 710 TO SQL-LOCATION	03010000
003020	03020000
003030 EXEC SQL	03030000
003040 OPEN DZBFASC1	03040000
003050 END-EXEC	03050000
003060	03060000
003070 EVALUATE SQLCODE	03070000
003080 WHEN ZERO	03080000
003090 SET DZBFASC1-NOT-EOC TO TRUE	03090000
003100 WHEN OTHER	03100000
003110 DISPLAY 'ERROR WHILE OPENING CURSOR DZBFASC1'	03110000
003120 PERFORM SQLERROR	03120000
003130 END-EVALUATE	03130000
003140 .	03140000
003150	03150000
003160 720-FETCH-DZBFASC1.	03160000
003170 MOVE 720 TO SQL-LOCATION	03170000
003180	03180000
003190 EXEC SQL	03190000
003200 FETCH DZBFASC1	03200000
003210 INTO :DCLDZBFASTV.DA-CPTAFS-479BFAS	03210000
003220 , :DCLDZBFASTV.DA-CRFMAS-479BFAS	03220000
003230 , :DCLDZBFASTV.NR-ISFMAS-479BFAS	03230000
003240 , :DCLDZBFASTV.NR-STFMAS-479BFAS	03240000
003250 , :DCLDZBFASTV.CT-RKTGDSV-479BFAS	03250000
003260 , :DCLDZBFASTV.NR-CIOFMAS-479BFAS	03260000
003270 , :DCLDZBFASTV.NR-IDFMAS-479BFAS	03270000
003280 , :DCLDZBFASTV.CT-OPD-479BFAS	03280000
003290 , :DCLDZBFASTV.CT-OPG-479BFAS	03290000
003300 , :DCLDZBFASTV.NR-ID-479BFAS	03300000
003310 END-EXEC	03310000
003320	03320000
003330 EVALUATE SQLCODE	03330000
003340 WHEN ZERO	03340000
003350 CONTINUE	03350000
003360 WHEN 100	03360000
003370 SET DZBFASC1-EOC TO TRUE	03370000
003380 WHEN OTHER	03380000
003390 DISPLAY 'ERROR WHILE FETCHING CURSOR DZBFASC1'	03390000
003400 PERFORM SQLERROR	03400000
003410 END-EVALUATE	03410000
003420 .	03420000
003430	03430000
003440 730-CLOSE-DZBFASC1.	03440000
003450 MOVE 730 TO SQL-LOCATION	03450000
003460	03460000
003470 EXEC SQL	03470000
003480 CLOSE DZBFASC1	03480000
003490 END-EXEC	03490000
003500	03500000
003510 EVALUATE SQLCODE	03510000
003520 WHEN ZERO	03520000
003530 CONTINUE	03530000
003540 WHEN OTHER	03540000
003550 DISPLAY 'ERROR WHILE CLOSING CURSOR DZBFASC1'	03550000
003560 PERFORM SQLERROR	03560000
003570 END-EVALUATE	03570000
003580 .	03580000

003590		03590000
003600	*SQLERROR ROUTINE.	03600000
003610	EXEC SQL	03610000
003620	INCLUDE SQLERROR	03620000
003630	END-EXEC.	03630000

```
=====
FILE: Mail
ENCODING: utf-8
=====
```

Hi Team,

The Python conversion fixes have been implemented and the latest output was cross-checked against the Iteration-1 feedback.

For the retrieval program, the reported issues are fixed:

COBOL output is now generated in proper 80-column format.
 Right sequence number is placed in columns 73-80.
 Comment indicator is placed in column 7.
 Compiler option is converted to CBL ARITH(EXTEND).
 PROGRAM-ID is replaced correctly.
 DB2 and DCLGEN includes are generated.
 Cursor declarations now include WITH HOLD.
 DCLGEN initialization is generated.
 Cursor EOC handling is moved to fetch paragraphs.
 DB2 date realignment logic is generated.
 DCLGEN field names are corrected.
 SQL-ERROR is replaced with SQLERROR, and EXEC SQL INCLUDE SQLERROR is included.
 For the update program, FFRECB is not available in the Sheet Mapping as a valid mapped business record. So, as discussed, Python does not hardcode FFRECB to any restart DB2 table. The logic now skips FFRECB conversion and adds a manual -redesign comment.

This is intentional because the update restart logic needs proper business mapping and cannot be generated safely from field mapping alone.

Latest outputs are shared for validation.

```
=====
FILE: main.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: manual_conventions\__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: manual_conventions\DEFERRED_DECISIONS.md
ENCODING: utf-8
=====
```

```
# LOCATION: manual_conventions/DEFERRED_DECISIONS.md
# ACTION: CREATE / REPLACE – team-review tracking template

# Deferred Conversion Decisions – COBOL Team Review
```

This document tracks COBOL-shop conventions observed in **hand-written manual DB2 conversions** that the automated converter does **NOT** apply automatically.

The converter follows one strict principle:

- > Convert ONLY IDMS database statements and mapped field references.
- > Every other business line stays byte-for-byte identical to the input.

Each item below is a convention the manual programmer applied as a **rewrite**

preference**, not an IDMS→DB2 conversion. The team reviews each one and decides whether to (a) leave it manual, (b) automate it as a new `rules/` entry, or (c) block it pending inputs.

How to use this document

For each item:

1. Discuss with the COBOL team.
2. Fill in **Decision**, **Rationale**, **Owner**, **Target date**.
3. If **Automate** → create a `rules/` entry + a test, then update Status to DONE.
4. If **Keep manual** → mark Status CLOSED; no code change.
5. If **Blocked** → note the blocker (e.g. missing Sheet Mapping input).

Decision values: `AUTOMATE` | `KEEP MANUAL` | `BLOCKED` | `PENDING`

Status values: `OPEN` | `IN REVIEW` | `DONE` | `CLOSED`

Decision Log

D-1 Paragraph numbering

- **Observed:** `VERWERKING` → `1000-VERWERKING`, `600-GET-TIMESTAMP` → `820000-GET-TIMESTAMP`
- **Converter behavior:** keeps original paragraph names
- **Risk:** style only (no behavior change)
- **Decision:** _____
- **Rationale:** _____
- **Owner:** _____
- **Target date:** _____
- **Status:** OPEN

D-2 Processing counters

- **Observed:** adds `WS-TELLER`, `WS-NB-INPUT-UPD-I`, `WS-NB-BFAR-UPD-I` + increment logic
- **Converter behavior:** not added
- **Risk:** HIGH – adds NEW business logic (changes program behavior)
- **Decision:** _____
- **Rationale:** _____
- **Owner:** _____
- **Target date:** _____
- **Status:** OPEN

D-3 Restart-table SQL (DZ01RSTV)

- **Observed:** builds `700-SELECT/UPDATE/INSERT-DZ01RSTV` + `700-RESTART-CONTROL`
- **Converter behavior:** emits one "manual redesign required" comment, keeps original lines
- **Risk:** BLOCKED – restart table not in Sheet Mapping input
- **Blocker:** Add restart table (e.g. `DZ01RSTB`) to Sheet Mapping. Then it becomes a normal auto-conversion (no manual rule needed).
- **Decision:** BLOCKED (pending Sheet Mapping input)
- **Rationale:** Converter must not invent DB2 names (authority rule).
- **Owner:** _____
- **Target date:** _____
- **Status:** OPEN

D-4 Commit-flow restructure

- **Observed:** extracts `800-PROCESS-COMMIT` paragraph; restructures commit-every-100 logic
- **Converter behavior:** keeps original inline commit flow
- **Risk:** HIGH – restructures working business flow
- **Decision:** _____
- **Rationale:** _____
- **Owner:** _____
- **Target date:** _____
- **Status:** OPEN

D-5 Header / REMARKS rewrite

- **Observed:** new AUTHOR, DATE-WRITTEN, rewritten REMARKS block
- **Converter behavior:** keeps original header unchanged
- **Risk:** BLOCKED by existing rule: "Do not corrupt COBOL headers such as

```

DATE-WRITTEN, AUTHOR, or PROGRAM-ID."
- **Decision:** KEEP MANUAL (recommended – headers are human documentation)
- **Rationale:** Automating header rewrites conflicts with the header-preservation rule.
- **Owner:** _____
- **Target date:** _____
- **Status:** OPEN

```

D-6 EOF flag style

```

- **Observed:** `SW-EOF PIC X` → `88-level VMDZ205I-EOF / VMDZ205I-NOT-EOF`
- **Converter behavior:** keeps original `SW-EOF`
- **Risk:** style only (behavior equivalent if mapped carefully)
- **Decision:** _____
- **Rationale:** _____
- **Owner:** _____
- **Target date:** _____
- **Status:** OPEN

```

D-7 Multiple date-staging groups

```

- **Observed:** adds `DA-YYMMDD`, `DA-DDMMCCYY` in addition to `DA-CCYYMMDD`
- **Converter behavior:** generates only `DA-CCYYMMDD` + `DA-CCYYMMDD-R` + `DA-DD-MM-CCYY`
- **Risk:** LOW – extra staging fields; only needed if a specific date format is used
- **Decision:** _____
- **Rationale:** _____
- **Owner:** _____
- **Target date:** _____
- **Status:** OPEN

```

Summary Table (fill during review)

ID	Convention	Decision	Owner	Target	Status
D-1	Paragraph numbering				OPEN
D-2	Processing counters				OPEN
D-3	Restart-table SQL	BLOCKED			OPEN
D-4	Commit-flow restructure				OPEN
D-5	Header / REMARKS rewrite	KEEP MAN.			OPEN
D-6	EOF flag style				OPEN
D-7	Date-staging groups				OPEN

Promotion checklist (when a decision = AUTOMATE)

When the team approves automating a convention:

- [] Add the constant/template to the appropriate `rules/\*.py` file
- [] Add regex (if needed) to the appropriate `patterns/\*.py` file
- [] Wire it into the relevant composer/generator (logic only)
- [] Add a unit test proving the convention is applied correctly
- [] Add a test proving non-target lines stay unchanged
- [] Update this document: set Decision=AUTOMATE, Status=DONE
- [] Move the documentation note out of `manual\_conventions/` if fully promoted

Guiding principle (do not violate)

```

> The automated converter is a **surgical IDMS→DB2 translator**, not a program
> rewriter. It converts IDMS database access and mapped field references, and
> preserves all other business logic byte-for-byte. Any convention that adds
> new logic, renames working paragraphs, or restructures flow must be an
> explicit, team-approved rule – never an implicit default.

```

```

=====
FILE: manual_conventions\paragraph_numbering_conventions.py
ENCODING: utf-8
=====

```

```

# LOCATION: manual_conventions/paragraph_numbering_conventions.py

```

ACTION: CREATE NEW FILE (documentation-only; NOT wired into the converter)

"""

Paragraph numbering conventions (DEFERRED - pending COBOL team review).

Observed in manual DB2 rewrites. NOT applied by the automated converter,
which preserves original paragraph names per the business-flow rule.

These are documented here so the team can later decide on a standard scheme.
Do NOT import these into active conversion logic until approved.

"""

from __future__ import annotations

Example scheme observed in one manual rewrite (NOT authoritative).

Left as data for future discussion only.

```
OBSERVED_PARAGRAPH_NUMBERING = {
    "main_process": "1000-",    # e.g. VERWERKING -> 1000-VERWERKING
    "commit_process": "800-",   # e.g. 800-PROCESS-COMMIT
    "restart_control": "700-",   # e.g. 700-RESTART-CONTROL
    "timestamp": "820000-",     # e.g. 820000-GET-TIMESTAMP
}
```

DEFERRED_NOTE = (

"Paragraph renumbering restructures business flow and is not an IDMS->DB2 "
"conversion. Requires COBOL team agreement on a consistent scheme before "
"any automation."

)

=====

FILE: manual_conventions\README.md

ENCODING: utf-8

=====

Manual Conventions (Deferred – Pending COBOL Team Review)

This folder documents COBOL-shop conventions observed in **hand-written**
manual DB2 conversions that the automated converter does **NOT** apply.

Why these are separate

The automated converter follows one strict principle:

- > Convert ONLY IDMS database statements and mapped field references.
- > Every other business line stays byte-for-byte identical to the input.

The conventions listed here are **human-rewrite preferences**, not IDMS->DB2
conversions. They restructure or re-style working business logic. Applying
them automatically would violate the "preserve business logic" rule and risk
introducing bugs.

Status: DEFERRED

These are documented here for a **future discussion with the COBOL team**.
Once the team confirms which conventions should be standardized, selected
items may be promoted into active `rules/` (with tests) – one at a time.

What is NOT here (already automated)

The following ARE done automatically by the converter:

- OBTAIN CALC / STORE / MODIFY / ERASE -> DB2 SQL
- FINISH -> COMMIT
- IDMS control statement removal
- Mapped field reference rewrite (FIELD OF RECORD -> DCLGEN host)
- DA_/DT_ date column conversion
- DCLGEN INCLUDE, timestamp WS, date WS generation
- SQLCODE handling (EVALUATE SQLCODE), QUERYNO

What IS here (deferred conventions)

See the files in this folder. Each documents one convention category.

```
=====
FILE: manual_conventions\working_storage_conventions.py
ENCODING: utf-8
=====
```

```
# LOCATION: manual_conventions/working_storage_conventions.py
# ACTION: CREATE NEW FILE (documentation-only)
```

```
"""
```

```
Working-storage conventions (DEFERRED - pending COBOL team review).
```

```
Observed additions in manual rewrites that the converter does NOT generate,
because they add new business logic rather than converting IDMS access.
```

```
"""
```

```
from __future__ import annotations
```

```
# Counters observed in a manual rewrite (NOT auto-generated).
```

```
OBSERVED_PROCESSING_COUNTERS = [
```

```
    "WS-TELLER",
    "WS-NB-INPUT-UPD-I",
    "WS-NB-BFAR-UPD-I",
```

```
]
```

```
# EOF flag style observed (88-level instead of raw switch).
```

```
OBSERVED_EOF_FLAG_STYLE = {
    "group": "WS-SWITCHES",
    "field": "SW-<record>",
    "conditions": ["<record>-NOT-EOF VALUE 'N'", "<record>-EOF VALUE 'Y'"],
}
```

```
DEFERRED_NOTE = (
```

```
    "Processing counters and 88-level EOF flags are program-design choices, "
    "not IDMS->DB2 conversions. Requires COBOL team sign-off; #2 changes "
    "behavior, #6 is style only."
```

```
)
```

```
=====
FILE: patterns\__init__.py
ENCODING: utf-8
=====
```

```
[Empty file]
```

```
=====
FILE: patterns\batch_console_patterns.py
ENCODING: utf-8
=====
```

```
# LOCATION: patterns/batch_console_patterns.py
# ACTION: REPLACE ENTIRE FILE
```

```
"""Batch console output-scraping patterns.
```

```
Patterns only. No labels, no rules, no layout constants.
```

```
The batch orchestrator launches each runner as a SUBPROCESS so that a
change inside a runner cannot break the orchestrator. Everything it
reports is therefore read back out of the runner's own console output,
using the wording those runners already emit from
rules/retrieval_runner_messages.py, rules/update_runner_messages.py and
code_review/standards/wording.py.
```

```
Every pattern is optional: a value that cannot be found renders as '-'
and never fails the batch.
```

```
"""
```

```
from __future__ import annotations
```

```
import re
```

```
# =====
# Metadata counts
# =====
```

```

MAPPING_ROWS_PATTERN = re.compile(
    r"Sheet\s+Mapping\s+rows\s*[:=]\s*(?P<count>\d+)",
    flags=re.IGNORECASE,
)

DCLGEN_COLUMNS_PATTERN = re.compile(
    r"DCLGEN\s+total\s+columns\s*[:=]\s*(?P<count>\d+)",
    flags=re.IGNORECASE,
)

COPYBOOK_FIELDS_PATTERN = re.compile(
    r"Copybook\s+total\s+fields\s*[:=]\s*(?P<count>\d+)",
    flags=re.IGNORECASE,
)

PROGRAM_COUNT_PATTERN = re.compile(
    r"Program\s+Count\s*[:=]\s*(?P<count>\d+)",
    flags=re.IGNORECASE,
)

# =====
# Artefacts
# =====
OUTPUT_FILE_PATTERN = re.compile(
    r"Output\s+file\s+created\s*[:=]\s*(?P<path>.+)$",
    flags=re.IGNORECASE | re.MULTILINE,
)

REPORT_FOLDER_PATTERN = re.compile(
    r"Reports\s+written\s+to\s*[:=]\s*(?P<path>.+)$",
    flags=re.IGNORECASE | re.MULTILINE,
)

# =====
# Verdicts
# =====
REVIEW_SUMMARY_PATTERN = re.compile(
    r"Reviewed\s*[:=]\s*(?P<reviewed>\d+)\s+"
    r"Accepted\s*[:=]\s*(?P<accepted>\d+)\s+"
    r"Rejected\s*[:=]\s*(?P<rejected>\d+)",
    flags=re.IGNORECASE,
)

# The review runners' --quiet line:
# REJECTED VMDZ4420_db2.cbl blocking: CHK-02
QUIET_VERDICT_PATTERN = re.compile(
    r"^(?P<verdict>ACCEPTED|REJECTED)\s+(?P<name>\S+)(?P<rest>.*)$",
    flags=re.IGNORECASE | re.MULTILINE,
)

# The blocking check ids on that line. This is the single fact the COBOL
# team needs from a rejected review, so it is extracted explicitly rather
# than left buried in the runner's log.
BLOCKING_PATTERN = re.compile(
    r"blocking\s*[:=]\s*(?P<ids>[A-Z0-9][A-Z0-9,\s-]*)",
    flags=re.IGNORECASE,
)

NOTHING_TO_DO_PATTERN = re.compile(
    r"Nothing\s+to\s+(convert|review)\b",
    flags=re.IGNORECASE,
)

CONVERSION_FAILED_PATTERN = re.compile(
    r"Conversion\s+failed\s*[:=]\s*(?P<names>.+)$",
    flags=re.IGNORECASE | re.MULTILINE,
)

# =====
# Log filtering
# =====
# LoggerFactory writes "%(asctime)s | %(levelname)s | %(name)s |
# %(message)s" to a StreamHandler, which defaults to stderr. Dumping the
# stderr tail therefore showed five INFO lines instead of the failure.
# This pattern lets the console keep only the levels that matter.
LOG_LINE_PATTERN = re.compile(

```

```

r"^(?P<stamp>\d{2}-\d{2}-\d{4}\s+\d{2}:\d{2}:\d{2})\s*\\s*"
r"(?P<level>[A-Z]+\s*\\s*"
r"(?P<logger>[^\|]+\?)\s*\\s*"
r"(?P<message>.*)$"
)

ERROR_LINE_PATTERN = re.compile(
    r"^ERROR:\s*(?P<message>.+)$",
    flags=re.IGNORECASE | re.MULTILINE,
)

TRACEBACK_PATTERN = re.compile(
    r"^(?P<line>\s*(Traceback $most recent call last$|\w+Error:|\w+Exception:).*)$",
    flags=re.MULTILINE,
)

=====
FILE: patterns\batch_runner_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

# Regex patterns shared by the batch/update/retrieval runners.
# No business rules, no runtime logic, no hardcoded program/record names.

# Matches a COBOL COPY statement and captures the copybook/record name.
COPY_NAME_PATTERN = re.compile(
    r"\bCOPY\s+([A-Z0-9-]+\b)",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\cobol_cleanup_patterns.py
ENCODING: utf-8
=====

"""
COBOL cleanup regex patterns.

Regex patterns belong in patterns/, not in composer classes. These patterns
are shared by the COBOL post-conversion cleanup helper classes.
"""

import re

EXEC_SQL_INCLUDE_PATTERN = re.compile(
    r"\bINCLUDE\s+(?P<include>[A-Z0-9]+\b)",
    flags=re.IGNORECASE,
)

DCL_GROUP_PATTERN = re.compile(
    r"\bDCL(?P<table>[A-Z0-9]+\b)",
    flags=re.IGNORECASE,
)

WORKING_STORAGE_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s*)?WORKING-STORAGE\s+SECTION\.",
    flags=re.IGNORECASE,
)

LINKAGE_SECTION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s*)?LINKAGE\s+SECTION\.",
    flags=re.IGNORECASE,
)

ERROR_STATUS_MOVE_PATTERN = re.compile(
    r"^\s*MOVE\s+['\"]?[A-Z0-9]+['\"]?\s+TO\s+ERROR-STATUS\.\s*$",
    flags=re.IGNORECASE,
)

WS_STATUS_PATTERN = re.compile(

```

```

        r"^\\s*10\\s+WS-STATUS\\b",
        flags=re.IGNORECASE,
    )

SW_STATUS_D_PIC_PATTERN = re.compile(
    r"\\bSW-STATUS-D\\b.+\\bPIC\\b",
    flags=re.IGNORECASE,
)

SW_STATUS_D_MOVE_N_PATTERN = re.compile(
    r"^\\s*MOVE\\s+[ '\\\\"]N[ '\\\\"]\\s+TO\\s+SW-STATUS-D\\.?.\\s*$",
    flags=re.IGNORECASE,
)

FETCH_UNTIL_EOC_PATTERN = re.compile(
    r"^(?P<prefix>\\s*PERFORM\\s+)"
    r"(?P<fetch>[0-9]+-FETCH-[A-Z0-9-]+)"
    r"\\s+UNTIL\\s+(?P<eoc>[A-Z0-9-]+-EOC)\\.?.\\s*$",
    flags=re.IGNORECASE,
)

FETCH_UNTIL_EOC_OR_SW_PATTERN = re.compile(
    r"^(?P<prefix>\\s*PERFORM\\s+)"
    r"(?P<fetch>[0-9]+-FETCH-[A-Z0-9-]+)"
    r"\\s+UNTIL\\s+(?P<eoc>[A-Z0-9-]+-EOC)\\s+OR\\s+SW-STATUS-D\\s*$",
    flags=re.IGNORECASE,
)

FETCH_UNTIL_EOC_FULL_SW_PATTERN = re.compile(
    r"^(?P<prefix>\\s*PERFORM\\s+)"
    r"(?P<fetch>[0-9]+-FETCH-[A-Z0-9-]+)"
    r"\\s+UNTIL\\s+(?P<eoc>[A-Z0-9-]+-EOC)"
    r"\\s+OR\\s+SW-STATUS-D\\s*=?\\s*[ '\\\\"]?Y[ '\\\\"]?\\.?.\\s*$",
    flags=re.IGNORECASE,
)

SW_STATUS_Y_CONTINUATION_PATTERN = re.compile(
    r"^\\s*=?\\s*[ '\\\\"]?Y[ '\\\\"]?\\.?.\\s*$",
    flags=re.IGNORECASE,
)

WRITE_PATTERN = re.compile(
    r"^\\s*WRITE\\s+(?P<record>[A-Z0-9-]+)\\.?.\\s*$",
    flags=re.IGNORECASE,
)

INITIALIZE_PATTERN = re.compile(
    r"^\\s*INITIALIZE\\s+(?P<record>[A-Z0-9-]+)\\.?.\\s*$",
    flags=re.IGNORECASE,
)

MOVE_START_PATTERN = re.compile(
    r"^\\s*MOVE\\b",
    flags=re.IGNORECASE,
)

MOVE_TO_OUTPUT_FIELD_PATTERN = re.compile(
    r"^\\s*MOVE\\b.+\\bTO\\s+(?P<target>UIT-[A-Z0-9-]+)\\.?.\\s*$",
    flags=re.IGNORECASE,
)

TO_OUTPUT_FIELD_PATTERN = re.compile(
    r"^\\s*TO\\s+(?P<target>UIT-[A-Z0-9-]+)\\.?.\\s*$",
    flags=re.IGNORECASE,
)

DB2_DATE_MOVE_START_PATTERN = re.compile(
    r"^\\s*MOVE\\s+(?P<field>(?:DA|DT)-[A-Z0-9-]+)\\s+OF\\s+"
    r"(?P<group>DCL[A-Z0-9-]+)\\s*$",
    flags=re.IGNORECASE,
)

DB2_DATE_MOVE_ONE_LINE_PATTERN = re.compile(
    r"^\\s*MOVE\\s+(?P<field>(?:DA|DT)-[A-Z0-9-]+)\\s+OF\\s+"
    r"(?P<group>DCL[A-Z0-9-]+)\\s+TO\\s+"
    r"(?P<target>(?:UIT|OUT)-(?:DA|DT)-[A-Z0-9-]+)\\.?.\\s*$",

```



```

    flags=re.IGNORECASE,
)

TO_UIT_DATE_FIELD_PATTERN = re.compile(
    r"^\s*TO\s+(\?<target>(\?:UIT|OUT)-(\?:DA|DT)-[A-Z0-9-]+\)\.?\s*$",
    flags=re.IGNORECASE,
)

FETCH_PARAGRAPH_NUMBER_PATTERN = re.compile(
    r"^(?P<number>\d+)-FETCH-",
    flags=re.IGNORECASE,
)

# LOCATION: patterns/cobol_cleanup_patterns.py
# ACTION: APPEND these patterns to the existing file (add at the end)

import re # already imported at top; keep single import

# --- Category G: residual IDMS comment-noise cleanup ---

# Generated advisory comments left by the transformer when removing IDMS code.
RESIDUAL_IDMS_COMMENT_PATTERN = re.compile(
    r"^\s*\*\s*DB2:\s*(\"
    r"Removed\s+residual\s+IDMS\b\"
    r"\"|Removed\s+IDMS\b\"
    r"\"|Removed\s+orphan\s+IDMS\b\"
    r"\"|IDMS\s+record\s+[A-Z0-9-]+\s+was\s+not\s+converted\"
    r"\"|Missing\s+Sheet\s+Mapping\s+and\s+DCLGEN\s+metadata\"
    r"\"|Restart/control\s+logic\s+requires\s+manual\s+DB2\s+redesign\"
    r"\"|IDMS\s+FINISH\s+converted\s+to\s+COMMIT\"
    r\")\",
    flags=re.IGNORECASE,
)

# Continuation of a wrapped comment line (a comment line that is NOT a new
# DB2: marker and NOT a business comment separator like *---- or *****).
RESIDUAL_IDMS_COMMENT_CONTINUATION_PATTERN = re.compile(
    r"^\s*\*\s*(?!DB2:)(?![-*])\s\",
    flags=re.IGNORECASE,
)

# A lone CONTINUE. statement.
LONE_CONTINUE_PATTERN = re.compile(
    r"^\s*CONTINUE\.\s*$\",
    flags=re.IGNORECASE,
)

# Block-control keywords whose branch must not be left empty.
BLOCK_OPENER_PATTERN = re.compile(
    r"^\s*(IF|ELSE|WHEN|EVALUATE)\b\",
    flags=re.IGNORECASE,
)

# LOCATION: patterns/cobol_cleanup_patterns.py
# ACTION: APPEND this pattern (add near the other Category G patterns)

# Generated OBTAIN CALC removal marker (comment pair emitted before a direct
# UPDATE). Manual standard removes these since the MOVES flow into the UPDATE.
REMOVED_OBTAIN_CALC_COMMENT_PATTERN = re.compile(
    r"^\s*\*\s*DB2:\s*(\"
    r"Removed\s+OBTAIN\s+CALC\s+SELECT\s+for\b\"
    r"\"|Direct\s+UPDATE\s+will\s+use\s+mapped\s+composite\s+key\"
    r\")\",
    flags=re.IGNORECASE,
)

# LOCATION: patterns/cobol_cleanup_patterns.py
# ACTION: REPLACE these two patterns

# Generated advisory comments left by the transformer when removing IDMS code.
# NOTE: matches only \"* DB2:\" (colon after DB2). \"*DB2-KEEP:\" is deliberately
# NOT matched, so Option B keep-and-comment markers survive cleanup.
RESIDUAL_IDMS_COMMENT_PATTERN = re.compile(
    r"^\s*\*\s*DB2:\s*(\"
    r"Removed\s+residual\s+IDMS\b\"
    r"\"|Removed\s+IDMS\b\"

```

```

r"|Removed\s+orphan\s+IDMS\b"
r"|IDMS\s+record\s+[A-Z0-9-]+\s+was\s+not\s+converted"
r"|Missing\s+Sheet\s+Mapping\s+and\s+DCLGEN\s+metadata"
r"|Restart/control\s+logic\s+requires\s+manual\s+DB2\s+redesign"
r"|IDMS\s+FINISH\s+converted\s+to\s+COMMIT"
r")",
flags=re.IGNORECASE,
)

# Continuation of a wrapped residual comment. Must NOT match:
# - a new DB2: marker
# - a DB2-KEEP marker or a kept commented IDMS verb (Option B)
# - a business separator (*--- or ****)
RESIDUAL_IDMS_COMMENT_CONTINUATION_PATTERN = re.compile(
    r"^\s*\s*(?!DB2:)(?!DB2-KEEP)(?![-*])\s",
    flags=re.IGNORECASE,
)

# --- Output write placement patterns (appended) -----
#
# Regex only. No business rules, program names, paragraph names, record
# names, table names, cursor names or host variables.

PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^(?P<name>[A-Z0-9][A-Z0-9-]*)\.$",
    flags=re.IGNORECASE,
)

IF_START_PATTERN = re.compile(
    r"^IF\b",
    flags=re.IGNORECASE,
)

END_IF_PATTERN = re.compile(
    r"^END-IF\.$",
    flags=re.IGNORECASE,
)

WRITE_STATEMENT_PATTERN = re.compile(
    r"^WRITE\s+[A-Z0-9][A-Z0-9-]*",
    flags=re.IGNORECASE,
)

WHEN_ZERO_PATTERN = re.compile(
    r"^WHEN\s+ZERO\s*$",
    flags=re.IGNORECASE,
)

# PERFORM of any generated cursor paragraph, e.g. PERFORM 830-CLOSE-DZEVEFC1
PERFORM_CURSOR_PARAGRAPH_PATTERN = re.compile(
    r"^PERFORM\s+\d{3}-(?:OPEN|FETCH|CLOSE)-[A-Z0-9][A-Z0-9-]*\b",
    flags=re.IGNORECASE,
)

# PERFORM of the CLOSE paragraph of one cursor.
PERFORM_CLOSE_CURSOR_PATTERN = re.compile(
    r"^PERFORM\s+(?P<number>\d{3})-CLOSE-(?P<cursor>[A-Z0-9][A-Z0-9-]*)"
    r"\s*\.$",
    flags=re.IGNORECASE,
)

# PERFORM of a business paragraph (no UNTIL, no TIMES, no THRU).
PERFORM_PARAGRAPH_PATTERN = re.compile(
    r"^PERFORM\s+(?P<paragraph>[A-Z0-9][A-Z0-9-]*)\s*\.$",
    flags=re.IGNORECASE,
)

# --- Paragraph structure patterns (appended) -----

PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^(?P<name>[A-Z0-9][A-Z0-9-]*)\.$",
    flags=re.IGNORECASE,
)

SECTION_HEADER_PATTERN = re.compile(
    r"^(?P<name>[A-Z0-9][A-Z0-9-]*)\s+SECTION\s*\.$",
    flags=re.IGNORECASE,
)

```

```

)

PROCEDURE_DIVISION_LINE_PATTERN = re.compile(
    r"^\s*PROCEDURE\s+DIVISION\b",
    flags=re.IGNORECASE,
)

# =====
# APPENDED: patterns for the generated-COBOL safety cleanup passes
#
# Consumed by:
#   composers/cleanup/output_write_placement_cleanup.py
#   composers/cleanup/paragraph_terminator_cleanup.py
#
# Regex only. No business rules, no program / paragraph / record / table /
# cursor / host variable names.
# =====

# --- Paragraph structure -----

PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^\s*(?P<name>[A-Z0-9][A-Z0-9-]*)\s*\.\"",
    flags=re.IGNORECASE,
)

SECTION_HEADER_PATTERN = re.compile(
    r"^\s*(?P<name>[A-Z0-9][A-Z0-9-]*)\s+SECTION\s*\s*\.\"",
    flags=re.IGNORECASE,
)

PROCEDURE_DIVISION_LINE_PATTERN = re.compile(
    r"^\s*PROCEDURE\s+DIVISION\b",
    flags=re.IGNORECASE,
)

# --- Output write placement -----

IF_START_PATTERN = re.compile(
    r"^\s*IF\b",
    flags=re.IGNORECASE,
)

END_IF_PATTERN = re.compile(
    r"^\s*END-IF\s*\.?\"",
    flags=re.IGNORECASE,
)

WRITE_STATEMENT_PATTERN = re.compile(
    r"^\s*WRITE\s+[A-Z0-9][A-Z0-9-]*",
    flags=re.IGNORECASE,
)

WHEN_ZERO_PATTERN = re.compile(
    r"^\s*WHEN\s+ZERO\s*\"",
    flags=re.IGNORECASE,
)

# PERFORM of any generated cursor paragraph, e.g. PERFORM 830-CLOSE-DZEVEFC1
PERFORM_CURSOR_PARAGRAPH_PATTERN = re.compile(
    r"^\s*PERFORM\s+\d{3}-(:OPEN|FETCH|CLOSE)-[A-Z0-9][A-Z0-9-]*\b",
    flags=re.IGNORECASE,
)

# PERFORM of the CLOSE paragraph of one cursor.
PERFORM_CLOSE_CURSOR_PATTERN = re.compile(
    r"^\s*PERFORM\s+(?P<number>\d{3})-CLOSE-(?P<cursor>[A-Z0-9][A-Z0-9-]*)"
    r"\s*\s*\.?\"",
    flags=re.IGNORECASE,
)

# PERFORM of a business paragraph (no UNTIL, no TIMES, no THRU).
PERFORM_PARAGRAPH_PATTERN = re.compile(
    r"^\s*PERFORM\s+(?P<paragraph>[A-Z0-9][A-Z0-9-]*)\s*\s*\.?\"",
    flags=re.IGNORECASE,
)

```

```
=====
FILE: patterns\cobol_patterns.py
ENCODING: utf-8
=====
```

```
"""
COBOL regex patterns.
```

```
These patterns are shared by parsers, analyzers, transformers, composers,
and formatters. Keep regex definitions centralized here so runtime modules
do not define their own duplicate COBOL patterns.
```

```
"""
```

```
import re
```

```
PROGRAM_ID_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?PROGRAM-ID\.\s*([A-Z0-9- ]+)\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
DIVISION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?"
    r"(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)"
    r"\s+DIVISION\b.*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE,
)
```

```
IDENTIFICATION_DIVISION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?IDENTIFICATION\s+DIVISION\b.*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
ENVIRONMENT_DIVISION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?ENVIRONMENT\s+DIVISION\b.*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
DATA_DIVISION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?DATA\s+DIVISION\b.*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
PROCEDURE_DIVISION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?PROCEDURE\s+DIVISION\b.*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
WORKING_STORAGE_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?WORKING-STORAGE\s+SECTION\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
LINKAGE_SECTION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?LINKAGE\s+SECTION\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)
```

```
SECTION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?[A-Z0-9][A-Z0-9-]*\s+SECTION\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE,
)
```

```
PARAGRAPH_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?(?<P<name>[A-Z0-9][A-Z0-9-]*)\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE,
```

```

)

END_PROGRAM_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?END\\s+PROGRAM\\b.*\\.?\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

STOP_RUN_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?STOP\\s+RUN\\.?\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE,
)

GOBACK_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?GOBACK\\.?\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE,
)

COMMENT_OR_SKIP_PATTERN = re.compile(
    r"^\\s*(\\*|/|EJECT\\b|SKIP[0-9]*\\b)",
    flags=re.IGNORECASE,
)

COMMENT_PATTERN = re.compile(
    r"^\\s*\\*",
    flags=re.IGNORECASE,
)

PAGE_EJECT_PATTERN = re.compile(
    r"^\\s*/\\s*$",
    flags=re.IGNORECASE,
)

SEQUENCE_ONLY_PATTERN = re.compile(
    r"^\\s*(\\d{6}|\\d{8})\\s*$",
    flags=re.IGNORECASE,
)

LEFT_SEQUENCE_PATTERN = re.compile(
    r"^\\s*(?P<seq>\\d{6})(?P<body>\\s+.*)*$",
    flags=re.IGNORECASE,
)

RIGHT_SEQUENCE_PATTERN = re.compile(
    r"(?P<body>.*?)\\s+(?P<right>\\d{8})\\s*$",
    flags=re.IGNORECASE,
)

DATA_ENTRY_START_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?"
    r"(?P<level>0[1-9]|[1-4][0-9]|66|77|88)"
    r"\\s+(?P<name>[A-Z0-9][A-Z0-9-]*)\\b"
    r"(?P<rest>.*)*$",
    flags=re.IGNORECASE,
)

FILE_DESCRIPTOR_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?(FD|SD)\\s+",
    flags=re.IGNORECASE,
)

SELECT_FILE_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?SELECT\\s+",
    flags=re.IGNORECASE,
)

```

```

SQL_ERROR_PARAGRAPH_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?(SQL-ERROR|SQLERROR)\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

SQL_ERROR_DOT_PATTERN = re.compile(
    r"\bPERFORM\s+(SQL-ERROR|SQLERROR)\s*\.\?",
    flags=re.IGNORECASE,
)

DB2_INFRASTRUCTURE_MARKER_PATTERN = re.compile(
    r"DB2 SQLCA, SQL ERROR WORKING STORAGE, DCLGEN INCLUDES, AND CURSOR FLAGS",
    flags=re.IGNORECASE,
)

NON_PARAGRAPH_SINGLE_WORDS = {
    "ACCEPT",
    "ADD",
    "ALTER",
    "CALL",
    "CANCEL",
    "CLOSE",
    "COMMIT",
    "COMPUTE",
    "CONTINUE",
    "DELETE",
    "DISPLAY",
    "DIVIDE",
    "ELSE",
    "END-IF",
    "END-EVALUATE",
    "END-PERFORM",
    "END-READ",
    "END-STRING",
    "END-UNSTRING",
    "END-WRITE",
    "END-EXEC",
    "EJECT",
    "EXIT",
    "GOBACK",
    "IF",
    "INITIALIZE",
    "INSPECT",
    "MOVE",
    "MULTIPLY",
    "NEXT",
    "OPEN",
    "PERFORM",
    "READ",
    "RETURN",
    "REWRITE",
    "ROLLBACK",
    "SEARCH",
    "SET",
    "SKIP1",
    "SKIP2",
    "SKIP3",
    "SORT",
    "SPACE",
    "SPACES",
    "START",
    "STOP",
    "STRING",
    "SUBTRACT",
    "UNSTRING",
    "WHEN",
    "WRITE",
}

```

```
=====
FILE: patterns\cobol_transformer_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
import re
```

```
RIGHT_SEQUENCE_PATTERN = re.compile(
    r"^(?P<body>.*?)(?:\s+(?P<right>\d{8}))\s*$",
    flags=re.IGNORECASE,
)
```

```
LEFT_SEQUENCE_PATTERN = re.compile(
    r"^(?P<left>\s*\d{6}\s+)(?P<body>.*?)$",
    flags=re.IGNORECASE,
)
```

```
RIGHT_SEQUENCE_WITH_SPACES_PATTERN = re.compile(
    r"^(?P<body>.*?)(?P<spaces>\s+)(?P<right>\d{8})\s*$",
    flags=re.IGNORECASE,
)
```

```
IDMS_DECLARATIVE_PATTERNS = [
    re.compile(
        r"^IDMS-CONTROL\s+SECTION\?.?$",
        flags=re.IGNORECASE,
    ),
    re.compile(
        r"^PROTOCOL\b.*$",
        flags=re.IGNORECASE,
    ),
    re.compile(
        r"^IDMS-RECORDS\s+WITHIN\s+WORKING-STORAGE\s+SECTION\?.?$",
        flags=re.IGNORECASE,
    ),
    re.compile(
        r"^SCHEMA\s+SECTION\?.?$",
        flags=re.IGNORECASE,
    ),
    re.compile(
        r"^DB\s+[A-Z0-9-]+\s+WITHIN\s+[A-Z0-9-]+\?.?$",
        flags=re.IGNORECASE,
    ),
    re.compile(
        r"^COPY\s+IDMS\b.*$",
        flags=re.IGNORECASE,
    ),
]
```

```
IDMS_EXECUTABLE_PATTERNS = [
    re.compile(
        r"^BIND\b.*$",
        flags=re.IGNORECASE,
    ),
    re.compile(
        r"^FIND\s+CURRENT\b.*$",
        flags=re.IGNORECASE,
    ),
    re.compile(
        r"^FINISH\?.?$",
        flags=re.IGNORECASE,
    ),
]
```

```
IDMS_ABORT_PARAGRAPH_PATTERN = re.compile(
    r"^IDMS-ABORT\?.?$",
    flags=re.IGNORECASE,
)
```

```

EXIT_LINE_PATTERN = re.compile(
    r"^EXIT\.$",
    flags=re.IGNORECASE,
)

FINISH_STATEMENT_PATTERN = re.compile(
    r"^FINISH\.$",
    flags=re.IGNORECASE,
)

BIND_STATEMENT_PATTERN = re.compile(
    r"^BIND\b",
    flags=re.IGNORECASE,
)

FIND_CURRENT_STATEMENT_PATTERN = re.compile(
    r"^FIND\s+CURRENT\b",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\copybook_patterns.py
ENCODING: utf-8
=====

"""
Copybook regex patterns.

Copybook parser logic must import these patterns instead of defining regex
directly inside parser classes.
"""

import re

COPYBOOK_FIELD_PATTERN = re.compile(
    r"^\s*(?P<level>0[1-9]|[1-4][0-9]|66|77|88)\s+"
    r"(?P<name>[A-Z0-9-]+)"
    r"(?P<rest>.*)$",
    flags=re.IGNORECASE,
)

COPYBOOK_PIC_PATTERN = re.compile(
    r"\bPIC(?:TURE)?\s+"
    r"(?P<pic>[SXA9VZ0-9\(\)\+\-\.\,\/]+)",
    flags=re.IGNORECASE,
)

COPYBOOK_USAGE_PATTERN = re.compile(
    r"\b(?:USAGE\s+IS\s+|USAGE\s+)?"
    r"(?P<usage>COMP-3|COMPUTATIONAL-3|COMP|COMPUTATIONAL|BINARY|DISPLAY|PACKED-DECIMAL)\b",
    flags=re.IGNORECASE,
)

COPYBOOK_OCCURS_PATTERN = re.compile(
    r"\bOCCURS\s+(?P<occurs>[0-9]+\s+(?:TIMES\b)?",
    flags=re.IGNORECASE,
)

COPYBOOK_COMMENT_OR_SKIP_PATTERN = re.compile(
    r"^\s*(?:\*|/|EJECT\b|SKIP[0-9]*\b)",
    flags=re.IGNORECASE,
)

```



```
=====
FILE: patterns\counter_declaration_patterns.py
ENCODING: utf-8
=====
```

```
# LOCATION: patterns/counter_declaration_patterns.py
# ACTION: CREATE NEW FILE
```

```
"""Counter declaration regex patterns.
```

```
This file must contain regex patterns only.
```

```
No token lists, business rules, conversion rules or layout constants.
Patterns are matched against an already-stripped logical line.
"""
```

```
from __future__ import annotations
```

```
import re
```

```
# An increment of a generated row counter.
```

```
# ADD 1 TO WS-NB-OUTPUT-COUNT
```

```
COUNTER_INCREMENT_PATTERN = re.compile(
    r"^(ADD\s+\d+\s+TO\s+(?P<name>WS-NB-[A-Z0-9][A-Z0-9-]*)\s*\.\?$",
    flags=re.IGNORECASE,
)
```

```
# Any COBOL data description entry, used to collect declared names.
```

```
DATA_ENTRY_PATTERN = re.compile(
    r"^(?P<level>\d{2})\s+(?P<name>[A-Z0-9][A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)
```

```
# An 01-level group header.
```

```
GROUP_HEADER_PATTERN = re.compile(
    r"^01\s+(?P<name>[A-Z0-9][A-Z0-9-]*)\s*\.\$",
    flags=re.IGNORECASE,
)
```

```
# A subordinate entry, i.e. any level other than 01 / 77 / 88.
```

```
SUBORDINATE_ENTRY_PATTERN = re.compile(
    r"^(?P<level>0[2-9]|[1-4][0-9]|49)\s+(?P<name>[A-Z0-9][A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)
```

```
WORKING_STORAGE_PATTERN = re.compile(
    r"^(WORKING-STORAGE\s+SECTION\s*\.\$",
    flags=re.IGNORECASE,
)
```

```
SECTION_OR_DIVISION_PATTERN = re.compile(
    r"^(?P<name>[A-Z0-9-]+\s+SECTION|[A-Z]+\s+DIVISION)\b",
    flags=re.IGNORECASE,
)
```

```
STOP_RUN_PATTERN = re.compile(
    r"^(STOP\s+RUN\s*\.\?$",
    flags=re.IGNORECASE,
)
```

```
=====
FILE: patterns\cursor_close_guarantee_patterns.py
ENCODING: utf-8
=====
```

```
# LOCATION: src/idms_db2_phase2/composers/cursor_close_guarantee_composer.py
# ACTION: CREATE NEW FILE
```

```
"""Guarantees that every generated cursor is looped and closed.
```

```
Runs AFTER CursorFlowComposer. It is a no-op for any cursor that the
INLINE or SPAN strategy already handled correctly, and a last-resort
repair for every cursor they could not correlate.
```

```
WHY THIS EXISTS
```

CursorFlowComposer recognises two driving-loop idioms. An LRF-expanded program matches neither, so it logged `no\_loop\_for\_cursor` as a WARNING and shipped:

```
PERFORM 710-OPEN-DZBFASC1.
PERFORM 720-FETCH-DZBFASC1.      <- primes once, never loops
...
BEHANDELING.
...
PERFORM 720-FETCH-DZBFASC1.      <- orphan repeat fetch
```

with 730-CLOSE-DZBFASC1 declared but never performed. The cursor is opened, read twice, and leaked. The manual reference is:

```
PERFORM 710-OPEN-DZBFASC1.
PERFORM 720-FETCH-DZBFASC1 UNTIL DZBFASC1-EOC.
PERFORM 730-CLOSE-DZBFASC1.
```

with the FETCH paragraph driving the business paragraph:

```
EVALUATE SQLCODE
  WHEN ZERO
    PERFORM BEHANDELING THRU BEHANDELING-EXIT
```

THREE REPAIR PATHS

-
- A. A fetch PERFORM already carries UNTIL -> INLINE already ran.
Only normalise a legacy exit condition and guarantee the CLOSE.
 - B. A business SPAN loop drives the paragraph that held the OBTAIN NEXT
-> SPAN shape. Normalise its exit condition and guarantee the CLOSE.
The in-loop fetch is REQUIRED here and is never removed.
 - C. No loop at all -> synthesise the manual shape, bind the FETCH
paragraph to the business paragraph, and remove the orphan repeat
fetch.

CLEAN ARCHITECTURE

Regex lives in patterns/cursor_close_guarantee_patterns.py.
Constants live in rules/cursor_close_guarantee_rules.py.
No program, paragraph, record, table, cursor or host variable name is
hardcoded here.
"""

```
from __future__ import annotations

from dataclasses import dataclass, field

from idms_db2_phase2.composers.cursor_flow_line_formatter import (
    CursorFlowLineFormatter,
)
from patterns.cursor_close_guarantee_patterns import (
    CONTINUE_PATTERN,
    CURSOR_PARAGRAPH_HEADER_PATTERN,
    PARAGRAPH_HEADER_PATTERN,
    PERFORM_CURSOR_PARAGRAPH_PATTERN,
    PERFORM_INLINE_WITH_UNTIL_PATTERN,
    PERFORM_PARAGRAPH_PATTERN,
    PERFORM_SPAN_PATTERN,
    PERFORM_SPAN_WITH_UNTIL_PATTERN,
    SECTION_HEADER_PATTERN,
    UNTIL_ONLY_PATTERN,
    WHEN_ZERO_PATTERN,
)
from rules.cursor_close_guarantee_rules import (
    CURSOR_CLOSE_GUARANTEE_MESSAGES,
    ENFORCE_CURSOR_CLOSE_GUARANTEE,
    ENFORCE_FETCH_DRIVES_BUSINESS_PARAGRAPH,
    EOC_CONDITION_TEMPLATE,
    EXIT_PARAGRAPH_SUFFIX,
    LEGACY_EOC_CONDITIONS,
    NON_PARAGRAPH_SINGLE_WORDS,
    OPERATION_CLOSE,
    OPERATION_FETCH,
```

```

OPERATION_OPEN,
PERFORM_BUSINESS_TEMPLATE,
PERFORM_BUSINESS_THRU_TEMPLATE,
PERFORM_CLOSE_TEMPLATE,
PERFORM_FETCH_UNTIL_TEMPLATE,
PREFER_THRU_EXIT_PARAGRAPH,
REQUIRED_OPERATIONS,
STATEMENT_TERMINATOR,
SYNTHESISE_MISSING_DRIVING_LOOP,
UNTIL_LOOKAHEAD_LIMIT,
UNTIL_TEMPLATE,
)

COMMENT_INDICATORS = ("*", "/")
UNTIL_KEYWORD = "UNTIL"

#
# Value objects
#
@dataclass
class ParagraphHeader:
    """One paragraph header line."""

    index: int = -1
    name: str = ""

@dataclass
class CursorParagraphSet:
    """The OPEN / FETCH / CLOSE paragraph names of one cursor."""

    cursor: str = ""
    names: dict = field(default_factory=dict)

    @property
    def is_complete(self) -> bool:
        return all(operation in self.names for operation in REQUIRED_OPERATIONS)

    @property
    def open_name(self) -> str:
        return self.names.get(OPERATION_OPEN, "")

    @property
    def fetch_name(self) -> str:
        return self.names.get(OPERATION_FETCH, "")

    @property
    def close_name(self) -> str:
        return self.names.get(OPERATION_CLOSE, "")

@dataclass
class CursorPerform:
    """One PERFORM of a generated cursor paragraph."""

    index: int = -1
    operation: str = ""
    paragraph: str = ""
    tail: str = ""

    @property
    def has_until(self) -> bool:
        return UNTIL_KEYWORD in str(self.tail or "").upper()

@dataclass
class LoopShape:
    """A business driving loop found in the generated flow."""

    perform_index: int = -1
    until_index: int = -1
    end_index: int = -1
    condition: str = ""
    terminated: bool = False

```

```

class CursorCloseGuaranteeComposer:
    """Last-resort guarantee that a generated cursor loops and closes."""

    def __init__(
        self,
        line_formatter: CursorFlowLineFormatter | None = None,
    ) -> None:
        self.line_formatter = line_formatter or CursorFlowLineFormatter()
        self.messages: list[str] = []

    #
    # Public entry point
    #
    def compose(self, text: str) -> str:
        self.messages = []

        if not text:
            return ""

        if not ENFORCE_CURSOR_CLOSE_GUARANTEE:
            return str(text)

        lines = self._normalize_line_endings(text).splitlines()

        if not lines:
            return ""

        cursor_sets = self._cursor_paragraph_sets(lines)

        for cursor in sorted(cursor_sets):
            cursor_set = cursor_sets[cursor]

            if not cursor_set.is_complete:
                self._log("incomplete_paragraph_set", cursor=cursor)
                continue

            lines = self._repair_cursor(lines, cursor_set)

        return "\n".join(lines).rstrip() + "\n"

    #
    # Discovery
    #
    def _cursor_paragraph_sets(
        self,
        lines: list[str],
    ) -> dict:
        """Map every cursor to its generated OPEN / FETCH / CLOSE names."""
        found: dict = {}

        for line in lines:
            logical = self._logical(line)

            if not logical:
                continue

            match = CURSOR_PARAGRAPH_HEADER_PATTERN.match(logical)

            if not match:
                continue

            cursor = match.group("cursor").upper()
            operation = match.group("operation").upper()
            number = int(match.group("number"))
            name = f"{number:03d}-{operation}-{cursor}"

            entry = found.setdefault(
                cursor,
                CursorParagraphSet(cursor=cursor, names={}),
            )
            entry.names.setdefault(operation, name)

        return found

    def _paragraph_headers(

```

```

        self,
        lines: list[str],
    ) -> list[ParagraphHeader]:
        """Every real paragraph header, cursor paragraphs included."""
        headers: list[ParagraphHeader] = []

        for index, line in enumerate(lines):
            logical = self._logical(line)

            if not logical or self._is_comment(logical):
                continue

            if SECTION_HEADER_PATTERN.match(logical):
                continue

            match = PARAGRAPH_HEADER_PATTERN.match(logical)

            if not match:
                continue

            name = match.group("name").upper()

            if name in NON_PARAGRAPH_SINGLE_WORDS:
                continue

            headers.append(ParagraphHeader(index=index, name=name))

        return headers

def _cursor_performs(
    self,
    lines: list[str],
    cursor: str,
) -> list[CursorPerform]:
    """Every PERFORM of a generated paragraph belonging to one cursor."""
    output: list[CursorPerform] = []

    for index, line in enumerate(lines):
        logical = self._logical(line)

        if not logical or self._is_comment(logical):
            continue

        match = PERFORM_CURSOR_PARAGRAPH_PATTERN.match(logical)

        if not match:
            continue

        if match.group("cursor").upper() != cursor:
            continue

        operation = match.group("operation").upper()
        number = int(match.group("number"))

        output.append(
            CursorPerform(
                index=index,
                operation=operation,
                paragraph=f"{number:03d}-{operation}-{cursor}",
                tail=str(match.group("tail") or "").strip(),
            )
        )

    return output

#
# Repair
#
def _repair_cursor(
    self,
    lines: list[str],
    cursor_set: CursorParagraphSet,
) -> list[str]:
    cursor = cursor_set.cursor
    eoc_condition = EOC_CONDITION_TEMPLATE.format(cursor=cursor)

```

```

performs = self._cursor_performs(lines, cursor)

open_performs = [p for p in performs if p.operation == OPERATION_OPEN]
fetch_performs = [p for p in performs if p.operation == OPERATION_FETCH]
close_performs = [p for p in performs if p.operation == OPERATION_CLOSE]

if not open_performs:
    self._log("no_open_perform", cursor=cursor)
    return lines

# ---- Path A: an INLINE loop already exists.
driving = next((p for p in fetch_performs if p.has_until), None)

if driving is not None:
    lines = self._normalise_condition_on_perform_line(
        lines=lines,
        index=driving.index,
        condition=eoc_condition,
        cursor=cursor,
    )
    return self._guarantee_close(
        lines=lines,
        after_index=driving.index,
        cursor_set=cursor_set,
        close_performs=close_performs,
    )

if not fetch_performs:
    self._log("no_open_perform", cursor=cursor)
    return lines

priming = fetch_performs[0]
repeats = fetch_performs[1:]

headers = self._paragraph_headers(lines)

business = ""
if repeats:
    business = self._owner_paragraph(headers, repeats[0].index)

# ---- Path B: a business SPAN / inline loop drives that paragraph.
loop = self._find_business_loop(lines, business) if business else None

if loop is not None:
    lines = self._normalise_condition_at(
        lines=lines,
        loop=loop,
        condition=eoc_condition,
        cursor=cursor,
    )
    return self._guarantee_close(
        lines=lines,
        after_index=loop.end_index,
        cursor_set=cursor_set,
        close_performs=close_performs,
    )

# ---- Path C: nothing drives the cursor. Synthesise the manual shape.
if not SYNTHESISE_MISSING_DRIVING_LOOP:
    return lines

return self._synthesise_loop(
    lines=lines,
    cursor_set=cursor_set,
    priming=priming,
    repeats=repeats,
    business=business,
    close_performs=close_performs,
    eoc_condition=eoc_condition,
)

def _synthesise_loop(
    self,
    lines: list[str],
    cursor_set: CursorParagraphSet,
    priming: CursorPerform,

```

```

        repeats: list[CursorPerform],
        business: str,
        close_performs: list[CursorPerform],
        eoc_condition: str,
    ) -> list[str]:
        """Build the manual OPEN / FETCH-UNTIL / CLOSE shape."""
        cursor = cursor_set.cursor
        output = list(lines)

        # 1. Bind the FETCH paragraph to the business paragraph.
        if business and ENFORCE_FETCH_DRIVES_BUSINESS_PARAGRAPH:
            output = self._bind_fetch_to_business(
                lines=output,
                fetch_paragraph=cursor_set.fetch_name,
                business=business,
            )
        elif not business:
            self._log("business_paragraph_unknown", cursor=cursor)

        # 2. Remove every orphan repeat fetch, highest index first.
        for repeat in sorted(repeats, key=lambda item: item.index, reverse=True):
            output = self._remove_statement(output, repeat.index)
            self._log("repeat_fetch_removed", fetch=repeat.paragraph)

        # 3. Replace the priming fetch with the driving loop plus the CLOSE.
        reference = output[priming.index]

        replacement = [
            self.line_formatter.format_like_line(
                reference_line=reference,
                replacement_body=PERFORM_FETCH_UNTIL_TEMPLATE.format(
                    fetch=cursor_set.fetch_name,
                    condition=eoc_condition,
                ),
            ),
        ]

        if not close_performs:
            replacement.append(
                self.line_formatter.format_like_line(
                    reference_line=reference,
                    replacement_body=PERFORM_CLOSE_TEMPLATE.format(
                        close=cursor_set.close_name,
                    ),
                )
            )
        else:
            self._log("close_present", close=cursor_set.close_name)

        output[priming.index : priming.index + 1] = replacement

        self._log(
            "loop_synthesised",
            cursor=cursor,
            fetch=cursor_set.fetch_name,
            condition=eoc_condition,
            close=cursor_set.close_name,
        )

        return output

def _bind_fetch_to_business(
    self,
    lines: list[str],
    fetch_paragraph: str,
    business: str,
) -> list[str]:
    """Rewrite WHEN ZERO / CONTINUE inside the FETCH paragraph."""
    headers = self._paragraph_headers(lines)
    span = self._paragraph_span(headers, fetch_paragraph, len(lines))

    if span is None:
        return lines

    start, end = span
    output = list(lines)

```

```

body = PERFORM_BUSINESS_TEMPLATE.format(paragraph=business)

if PREFER_THRU_EXIT_PARAGRAPH:
    exit_name = f"{business}{EXIT_PARAGRAPH_SUFFIX}"
    if any(header.name == exit_name for header in headers):
        body = PERFORM_BUSINESS_THRU_TEMPLATE.format(
            paragraph=business,
            through=exit_name,
        )

index = start
while index < end:
    logical = self._logical(output[index])

    if not WHEN_ZERO_PATTERN.match(logical):
        index += 1
        continue

    target = self._next_executable_index(output, index + 1, end)

    if target < 0:
        index += 1
        continue

    if not CONTINUE_PATTERN.match(self._logical(output[target])):
        index += 1
        continue

    output[target] = self.line_formatter.format_like_line(
        reference_line=output[target],
        replacement_body=body,
    )

    self._log(
        "business_paragraph_bound",
        fetch=fetch_paragraph,
        paragraph=business,
    )

    index = target + 1

return output

#
# CLOSE guarantee
#
def _guarantee_close(
    self,
    lines: list[str],
    after_index: int,
    cursor_set: CursorParagraphSet,
    close_performs: list[CursorPerform],
) -> list[str]:
    if close_performs:
        self._log("close_present", close=cursor_set.close_name)
        return lines

    if not 0 <= after_index < len(lines):
        return lines

    reference = lines[after_index]
    close_line = self.line_formatter.format_like_line(
        reference_line=reference,
        replacement_body=PERFORM_CLOSE_TEMPLATE.format(
            close=cursor_set.close_name,
        ),
    )

    output = list(lines)
    output.insert(after_index + 1, close_line)

    self._log(
        "close_inserted",
        close=cursor_set.close_name,
        cursor=cursor_set.cursor,

```



```

    )

    return output

#
# Loop discovery
#
def _find_business_loop(
    self,
    lines: list[str],
    business: str,
) -> LoopShape | None:
    """Find the loop that drives the business paragraph, if any."""
    target = str(business or "").upper()

    if not target:
        return None

    for index, line in enumerate(lines):
        logical = self._logical(line)

        if not logical or self._is_comment(logical):
            continue

        # A generated cursor PERFORM is never a business driving loop.
        if PERFORM_CURSOR_PARAGRAPH_PATTERN.match(logical):
            continue

        match = PERFORM_SPAN_WITH_UNTIL_PATTERN.match(logical)
        if match and match.group("paragraph").upper() == target:
            return LoopShape(
                perform_index=index,
                until_index=index,
                end_index=index,
                condition=match.group("condition").strip(),
                terminated=logical.rstrip().endswith(STATEMENT_TERMINATOR),
            )

        match = PERFORM_INLINE_WITH_UNTIL_PATTERN.match(logical)
        if match and match.group("paragraph").upper() == target:
            return LoopShape(
                perform_index=index,
                until_index=index,
                end_index=index,
                condition=match.group("condition").strip(),
                terminated=logical.rstrip().endswith(STATEMENT_TERMINATOR),
            )

        match = PERFORM_SPAN_PATTERN.match(logical)
        if match and match.group("paragraph").upper() == target:
            shape = self._until_on_following_line(lines, index)
            if shape is not None:
                return shape
            continue

        match = PERFORM_PARAGRAPH_PATTERN.match(logical)
        if match and match.group("paragraph").upper() == target:
            shape = self._until_on_following_line(lines, index)
            if shape is not None:
                return shape

    return None

def _until_on_following_line(
    self,
    lines: list[str],
    perform_index: int,
) -> LoopShape | None:
    """A PERFORM whose UNTIL clause sits on a continuation line."""
    scanned = 0
    index = perform_index + 1

    while index < len(lines) and scanned < UNTIL_LOOKAHEAD_LIMIT:
        logical = self._logical(lines[index])

        if not logical or self._is_comment(logical):

```

```

        index += 1
        continue

    scanned += 1
    match = UNTIL_ONLY_PATTERN.match(logical)

    if match:
        return LoopShape(
            perform_index=perform_index,
            until_index=index,
            end_index=index,
            condition=match.group("condition").strip(),
            terminated=logical.rstrip().endswith(STATEMENT_TERMINATOR),
        )

    index += 1

    return None

#
# Exit-condition normalisation
#
def _normalise_condition_on_perform_line(
    self,
    lines: list[str],
    index: int,
    condition: str,
    cursor: str,
) -> list[str]:
    """Rewrite the UNTIL tail of a PERFORM <fetch> UNTIL <cond> line."""
    if not 0 <= index < len(lines):
        return lines

    logical = self._logical(lines[index])
    head, _separator, tail = self._partition_until(logical)

    if not tail:
        return lines

    current = tail.rstrip(STATEMENT_TERMINATOR).strip()

    if self._compress(current) == self._compress(condition):
        return lines

    if not self._is_legacy_condition(current):
        return lines

    terminated = logical.rstrip().endswith(STATEMENT_TERMINATOR)
    body = f"{head.rstrip()} {UNTIL_TEMPLATE.format(condition=condition)}"

    if terminated:
        body = f"{body}{STATEMENT_TERMINATOR}"

    output = list(lines)
    output[index] = self.line_formatter.format_like_line(
        reference_line=output[index],
        replacement_body=body,
    )

    self._log("condition_rewritten", cursor=cursor, condition=condition)

    return output

def _normalise_condition_at(
    self,
    lines: list[str],
    loop: LoopShape,
    condition: str,
    cursor: str,
) -> list[str]:
    """Rewrite a legacy loop exit condition to the cursor EOC flag."""
    index = loop.until_index

    if not 0 <= index < len(lines):
        return lines

```

```

    if self._compress(loop.condition) == self._compress(condition):
        return lines

    if not self._is_legacy_condition(loop.condition):
        return lines

    logical = self._logical(lines[index])

    if UNTIL_ONLY_PATTERN.match(logical):
        body = UNTIL_TEMPLATE.format(condition=condition)
    else:
        head, _separator, _tail = self._partition_until(logical)
        body = f"{head.rstrip()} {UNTIL_TEMPLATE.format(condition=condition)}"

    if loop.terminated:
        body = f"{body}{STATEMENT_TERMINATOR}"

    output = list(lines)
    output[index] = self.line_formatter.format_like_line(
        reference_line=output[index],
        replacement_body=body,
    )

    self._log("condition_rewritten", cursor=cursor, condition=condition)

    return output

@staticmethod
def _partition_until(logical: str) -> tuple[str, str, str]:
    """Split a logical line on the UNTIL keyword, preserving case."""
    text = str(logical or "")
    position = text.upper().find(UNTIL_KEYWORD)

    if position < 0:
        return text, "", ""

    head = text[:position]
    tail = text[position + len(UNTIL_KEYWORD) :].strip()

    return head, UNTIL_KEYWORD, tail

def _is_legacy_condition(self, condition: str) -> bool:
    current = self._compress(condition)

    if not current:
        return False

    return any(
        current == self._compress(candidate)
        for candidate in LEGACY_EOC_CONDITIONS
    )

#
# Statement removal
#
def _remove_statement(
    self,
    lines: list[str],
    index: int,
) -> list[str]:
    """Delete one generated statement without orphaning its sentence."""
    if not 0 <= index < len(lines):
        return lines

    removed_logical = self._logical(lines[index])
    carried_period = removed_logical.rstrip().endswith(STATEMENT_TERMINATOR)

    output = list(lines)
    del output[index]

    if not carried_period:
        return output

    previous = self._previous_executable_index(output, index - 1)

    if previous < 0:

```

```

        return output

    previous_logical = self._logical(output[previous])

    if previous_logical.rstrip().endswith(STATEMENT_TERMINATOR):
        return output

    output[previous] = self.line_formatter.format_like_line(
        reference_line=output[previous],
        replacement_body=f"{previous_logical.rstrip()}{STATEMENT_TERMINATOR}",
    )

    return output

#
# Paragraph helpers
#
@staticmethod
def _owner_paragraph(
    headers: list[ParagraphHeader],
    index: int,
) -> str:
    """Name of the paragraph that physically contains a line index."""
    owner = ""

    for header in headers:
        if header.index >= index:
            break
        owner = header.name

    return owner

@staticmethod
def _paragraph_span(
    headers: list[ParagraphHeader],
    name: str,
    total: int,
) -> tuple[int, int] | None:
    """(first body line, first line after the paragraph)."""
    target = str(name or "").upper()

    for position, header in enumerate(headers):
        if header.name != target:
            continue

        start = header.index + 1
        end = (
            headers[position + 1].index
            if position + 1 < len(headers)
            else total
        )
        return start, min(end, total)

    return None

def _next_executable_index(
    self,
    lines: list[str],
    start: int,
    end: int,
) -> int:
    for index in range(start, min(end, len(lines))):
        logical = self._logical(lines[index])

        if not logical or self._is_comment(logical):
            continue

        return index

    return -1

def _previous_executable_index(
    self,
    lines: list[str],
    start: int,
) -> int:

```

```

        for index in range(min(start, len(lines) - 1), -1, -1):
            logical = self._logical(lines[index])

            if not logical or self._is_comment(logical):
                continue

            if PARAGRAPH_HEADER_PATTERN.match(logical):
                return -1

        return index

    return -1

#
# Low-level helpers
#
def _logical(self, line: str) -> str:
    return self.line_formatter.logical(line)

@staticmethod
def _is_comment(logical: str) -> bool:
    text = str(logical or "").strip()
    return bool(text) and text[0] in COMMENT_INDICATORS

@staticmethod
def _compress(text: str) -> str:
    return " ".join(str(text or "").split()).upper()

@staticmethod
def _normalize_line_endings(text: str) -> str:
    return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

def _log(self, key: str, **values) -> None:
    template = CURSOR_CLOSE_GUARANTEE_MESSAGES.get(key, "")

    if template:
        self.messages.append(template.format(**values))

```

```
__all__ = ["CursorCloseGuaranteeComposer"]
```

```

=====
FILE: patterns\cursor_flow_patterns.py
ENCODING: utf-8
=====

```

```

"""
Cursor flow regex patterns.

Regex patterns belong in patterns/, not in composer or service classes.
These patterns are shared by the cursor flow composer helper classes.
"""

```

```

import re

PERFORM_CURSOR_PATTERN = re.compile(
    r"^\s*PERFORM\s+"
    r"(?P<number>\d{3})-"
    r"(?P<operation>OPEN|FETCH|CLOSE)-"
    r"(?P<cursor>[A-Z0-9-]+)"
    r"\.\?\s*$",
    flags=re.IGNORECASE,
)

PERFORM_BUSINESS_PATTERN = re.compile(
    r"^\s*PERFORM\s+"
    r"(?P<paragraph>[A-Z0-9][A-Z0-9-]*)"
    r"\.\?\s*$",
    flags=re.IGNORECASE,
)

UNTIL_SQLCODE_100_PATTERN = re.compile(
    r"^\s*UNTIL\s+SQLCODE\s*=\s*100\.\?\s*$",
    flags=re.IGNORECASE,
)

```

```

CURSOR_PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^\s*"
    r"(?P<number>\d{3})-"
    r"(?P<operation>OPEN|FETCH|CLOSE)-"
    r"(?P<cursor>[A-Z0-9-]+)"
    r"\.\s*$",
    flags=re.IGNORECASE,
)

ANY_PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^\s*[A-Z0-9][A-Z0-9-]*\.\s*$",
    flags=re.IGNORECASE,
)

WHEN_ZERO_PATTERN = re.compile(
    r"^\s*WHEN\s+ZERO\s*$",
    flags=re.IGNORECASE,
)

CONTINUE_PATTERN = re.compile(
    r"^\s*CONTINUE\.\s*$",
    flags=re.IGNORECASE,
)

CONVERTED_OBTAIN_NEXT_COMMENT_PATTERN = re.compile(
    r"^\s*\*\s*DB2:\s*Converted\s+OBTAIN\s+NEXT\b",
    flags=re.IGNORECASE,
)

CONVERTED_OBTAIN_COMMENT_PATTERN = re.compile(
    r"^\s*\*\s*DB2:\s*Converted\s+OBTAIN\b",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\cursor_loop_patterns.py
ENCODING: utf-8
=====

# LOCATION: patterns/cursor_loop_patterns.py
# ACTION: CREATE NEW FILE
"""Regex for COBOL driving-loop idioms around a generated cursor.

Regex only. No rules, no constants, no names.
patterns/cursor_flow_patterns.py owns the ORIGINAL inline idiom and is
left untouched so the proven path cannot regress.
"""

from __future__ import annotations

import re

_NAME = r"[A-Z0-9][A-Z0-9-]*"

# PERFORM <para> THRU <para> UNTIL <condition>.      (all on one line)
PERFORM_SPAN_WITH_UNTIL_PATTERN = re.compile(
    rf"^\s*PERFORM\s+(?P<paragraph>{_NAME})\s+(?:THRU|THROUGH)\s+"
    rf"(?P<through>{_NAME})\s+UNTIL\s+(?P<condition>.+?)\s*\.\s*$",
    flags=re.IGNORECASE,
)

# PERFORM <para> THRU <para>                                (UNTIL on a later line)
PERFORM_SPAN_PATTERN = re.compile(
    rf"^\s*PERFORM\s+(?P<paragraph>{_NAME})\s+(?:THRU|THROUGH)\s+"
    rf"(?P<through>{_NAME})\s*\.\s*$",
    flags=re.IGNORECASE,
)

# PERFORM <para> UNTIL <condition>.                        (inline, one line)
PERFORM_INLINE_WITH_UNTIL_PATTERN = re.compile(
    rf"^\s*PERFORM\s+(?P<paragraph>{_NAME})\s+UNTIL\s+"
    rf"(?P<condition>.+?)\s*\.\s*$",
    flags=re.IGNORECASE,
)

```

```

# PERFORM <para>.                                     (UNTIL on a later line)
PERFORM_SIMPLE_PATTERN = re.compile(
    rf"^\bPERFORM\s+(?P<paragraph>{_NAME})\s*\.\s*$",
    flags=re.IGNORECASE,
)

# UNTIL <condition>.                                   (continuation line)
UNTIL_LINE_PATTERN = re.compile(
    rf"^\bUNTIL\s+(?P<condition>.+?)\s*(?P<terminator>\.?)\s*$",
    flags=re.IGNORECASE,
)

# PERFORM nnn-OPEN|FETCH|CLOSE-<cursor>.
PERFORM_CURSOR_PARAGRAPH_PATTERN = re.compile(
    rf"^\bPERFORM\s+(?P<number>\d{{3,6}})-"
    rf"(?P<operation>OPEN|FETCH|CLOSE)-"
    rf"(?P<cursor>{_NAME})\s*\.\s*$",
    flags=re.IGNORECASE,
)

# A cursor end-of-cursor condition name.
EOC_CONDITION_PATTERN = re.compile(
    rf"^\b{_NAME}-EOC$",
    flags=re.IGNORECASE,
)

# Paragraph / section headers, used to bound a scan.
PARAGRAPH_HEADER_PATTERN = re.compile(
    rf"^\b(?P<name>{_NAME})\s*\.\s*$",
    flags=re.IGNORECASE,
)

DIVISION_OR_SECTION_PATTERN = re.compile(
    rf"^\b[A-Z0-9-]+\s+(DIVISION|SECTION)\b",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\cursor_paragraph_patterns.py
ENCODING: utf-8
=====

# LOCATION: patterns/cursor_paragraph_patterns.py
# ACTION: CREATE NEW FILE

"""
Cursor paragraph regex patterns.

This module contains regex patterns and pattern templates only.
No business rules, conversion rules, COBOL layout text, or hardcoded
program, record, table, cursor, or host variable names belong here.
"""

from __future__ import annotations
import re

# A PERFORM of an unnumbered cursor paragraph, e.g. PERFORM OPEN-DZBEFFC1.
#
# (?![A-Z0-9-]) prevents OPEN-CUR1 from matching inside OPEN-CUR12.
# {operation} and {cursor} are injected via .format with re.escape.
UNNUMBERED_PERFORM_PATTERN_TEMPLATE = (
    rf"(?P<prefix>\bPERFORM\s+)"
    rf"{operation}-{cursor}"
    rf"(?![A-Z0-9-])"
    rf"(?P<suffix>\.?)"
)

# A PERFORM of a numbered cursor paragraph, e.g. PERFORM 710-OPEN-DZBEFFC1.
NUMBERED_PERFORM_PATTERN_TEMPLATE = (
    rf"(?P<prefix>\bPERFORM\s+)"
    rf"\d{{3}}-{operation}-{cursor}"
    rf"(?![A-Z0-9-])"
    rf"(?P<suffix>\.?)"
)

```

```

)

# A PERFORM of a superseded paragraph name carried on the cursor spec.
LEGACY_PERFORM_PATTERN_TEMPLATE = (
    r"(?P<prefix>\bPERFORM\s+)"
    r"{paragraph}"
    r"(?![A-Z0-9-])"
    r"(?P<suffix>\.?)"
)

# Replacement body shared by all three PERFORM rewrites.
PERFORM_REPLACEMENT_TEMPLATE = r"\g<prefix>{paragraph}\g<suffix>"

# PERFORM SQL-ERROR with an OPTIONAL terminator.
#
# The terminator is captured so the caller can carry it through unchanged.
# Substituting a fixed "PERFORM SQLERROR." would add a period to statements
# inside EVALUATE blocks and orphan END-EVALUATE.
LEGACY_SQL_ERROR_PERFORM_PATTERN = re.compile(
    r"\bPERFORM\s+SQL-ERROR(?:P<dot>\s*\.)?",
    flags=re.IGNORECASE,
)

# A SQL-ERROR paragraph header on its own fixed-format line.
# A header always ends in a period, so renaming it is safe.
LEGACY_SQL_ERROR_HEADER_PATTERN = re.compile(
    r"(?m)^(?P<indent>\s*)(?:\d{6}\s+)?SQL-ERROR\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE,
)

LEGACY_SQL_ERROR_HEADER_REPLACEMENT = r"\g<indent>SQLERROR."

```

```

=====
FILE: patterns\cursor_rules.py
ENCODING: utf-8
=====

```

```

"""
Cursor generation rules.
"""

```

```

CURSOR_AUTHORITY_RULES = [
    "Cursor names are derived from Sheet Mapping DB2 record/table names.",
    "Cursor names are not derived from IDMS set names.",
    "Generated cursor OPEN, FETCH, and CLOSE paragraphs are placed near the end.",
    "SQL-ERROR paragraph is required when cursor code performs SQL error handling.",
]

```

```

CURSOR_PARAGRAPH_NUMBERING = {
    1: {
        "open": 710,
        "fetch": 720,
        "close": 730,
    },
    2: {
        "open": 810,
        "fetch": 820,
        "close": 830,
    },
    3: {
        "open": 910,
        "fetch": 920,
        "close": 930,
    },
}

```

```

def cursor_paragraph_number(
    cursor_order: int,
    operation: str,
) -> int:
    operation_key = str(operation or "").strip().lower()

```



```

if cursor_order not in CURSOR_PARAGRAPH_NUMBERING:
    raise ValueError(f"Unsupported cursor order: {cursor_order}")

if operation_key not in CURSOR_PARAGRAPH_NUMBERING[cursor_order]:
    raise ValueError(f"Unsupported cursor operation: {operation}")

return CURSOR_PARAGRAPH_NUMBERING[cursor_order][operation_key]

```

```

=====
FILE: patterns\db2_date_patterns.py
ENCODING: utf-8
=====

```

```

# LOCATION: patterns/db2_date_patterns.py
# ACTION: REPLACE ENTIRE FILE

```

```

"""DB2 date comparison regex patterns.

```

These patterns support conservative DB2 date realignment before comparing DB2 date host fields with COBOL numeric date fields.

CORRECTION 1 - the host had to be the first token after IF

```

-----
DB2_DATE_COMPARISON_PATTERN was anchored as

```

```

^\\s*IF\\s+(?:DA|DT)-...

```

so it matched only a bare single-operand IF. A parenthesised or compound condition such as

```

IF (DA-CPTAFS-479BFAS OF DCLDZBFASTV < DA-ARCH-YMD
AND DA-CPTAFS-479BFAS OF DCLDZBFASTV NOT = '00000000') OR

```

never matched, the detector returned an empty field list, and the whole pass was skipped in silence.

CORRECTION 2 - a business field name was hardcoded in patterns/

```

-----
The old condition group was

```

```

(?P<condition>.+\\bPARMDATE\\b.*)

```

PARMDATE is a host variable belonging to ONE program. rules/authority_rules forbids hardcoding host variables anywhere, and any program comparing a DATE host against a differently named field was silently skipped. The comparison operand is no longer named; a DATE host is now recognised by its DCLGEN-qualified shape alone.

CORRECTION 3 - multi-line conditions

```

-----

```

Matching is performed against an ASSEMBLED logical condition, not a single physical line. Db2DateConditionAssembler owns the assembly.

Regex only. No rules, no constants, no program / record / table / DCLGEN / host variable names.

```

"""

```

```

from __future__ import annotations

```

```

import re

```

```

#

```

```

# Structure

```

```

#

```

```

WORKING_STORAGE_SECTION_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?WORKING-STORAGE\\s+SECTION\\.\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE,
)

```

```

LINKAGE_SECTION_PATTERN = re.compile(
    r"^\\s*(?:\\d{6}\\s+)?LINKAGE\\s+SECTION\\.\\s*(?:\\d{8})?\\s*$",
    flags=re.IGNORECASE,
)

```

```

PROCEDURE_DIVISION_PATTERN = re.compile(

```

```

r"^\s*(?:\d{6}\s+)?PROCEDURE\s+DIVISION\b.*\.\s*(?:\d{8})?\s*$",
flags=re.IGNORECASE,
)

#
# Condition span detection
#
# Any line that opens a condition. The operands are NOT constrained here;
# the assembler decides how far the condition runs.
IF_START_PATTERN = re.compile(
    r"^(?P<indent>\s*)IF\b(?P<rest>.*)$",
    flags=re.IGNORECASE,
)

# A line that plainly continues a condition rather than starting a
# statement: it opens with a boolean operator or a parenthesis.
CONDITION_CONTINUATION_PATTERN = re.compile(
    r"^\s*(?:(:AND|OR|NOT)\b|\(\)",
    flags=re.IGNORECASE,
)

# A line that plainly ends a condition by starting a statement.
STATEMENT_START_PATTERN = re.compile(
    r"^\s*(?:ACCEPT|ADD|CALL|CANCEL|CLOSE|COMPUTE|CONTINUE|DELETE|DISPLAY"
    r"|DIVIDE|ELSE|END-IF|EVALUATE|EXEC|EXIT|GO|GOBACK|IF|INITIALIZE"
    r"|INSPECT|MERGE|MOVE|MULTIPLY|OPEN|PERFORM|READ|RELEASE|RETURN"
    r"|REWRITE|SEARCH|SET|SORT|START|STOP|STRING|SUBTRACT|UNSTRING"
    r"|WRITE)\b",
    flags=re.IGNORECASE,
)

# A condition line ending on a dangling boolean operator.
TRAILING_BOOLEAN_PATTERN = re.compile(
    r"\b(?:AND|OR|NOT)\s*$",
    flags=re.IGNORECASE,
)

#
# DATE host operands
#
# A DCLGEN-qualified DB2 date host, anywhere in a condition:
#   DA-CPTAFS-479BFAS OF DCLDZBFASTV
#
# finditer over the ASSEMBLED condition yields every operand, so a
# compound condition reports all of them instead of only the first.
DB2_DATE_HOST_OPERAND_PATTERN = re.compile(
    r"\b(?:P<field>(?:DA|DT)-[A-Z0-9-]+)"
    r"\s+OF\s+"
    r"(?P<group>DCL[A-Z0-9-]+\b",
    flags=re.IGNORECASE,
)

# The same operand written in SQL host form, which must NEVER be rewritten.
SQL_HOST_OPERAND_PATTERN = re.compile(
    r":\s*(?:DCL[A-Z0-9-]+\.\.)(?:DA|DT)-[A-Z0-9-]+",
    flags=re.IGNORECASE,
)

# A comparison operator, used to prove the operand is actually compared
# and not merely referenced inside a MOVE that happens to sit on the line.
COMPARISON_OPERATOR_PATTERN = re.compile(
    r"(?:<|=|>|=|<>|=|<|>|\bNOT\s+=|\bEQUAL\b|\bGREATER\b|\bLESS\b)",
    flags=re.IGNORECASE,
)

#
# Embedded SQL guard
#
EXEC_SQL_START_PATTERN = re.compile(
    r"^\s*EXEC\s+SQL\b",
    flags=re.IGNORECASE,
)

END_EXEC_PATTERN = re.compile(
    r"\bEND-EXEC\b",
    flags=re.IGNORECASE,
)

```

```

)

#
# Working storage
#
DATE_HELPER_FIELD_PATTERN = re.compile(
    r"\bHELP-(?:DA|DT)-[A-Z0-9-]+\b",
    flags=re.IGNORECASE,
)

DATE_WORKING_STORAGE_MARKER_PATTERN = re.compile(
    r"DB2 DATE (?:COMPARISON|CONVERSION) WORKING STORAGE",
    flags=re.IGNORECASE,
)

DB2_SHARED_DATE_HELPER_USAGE_PATTERN = re.compile(
    r"\b(?:DA-CCYYMMDD-R|DA-CCYYMMDD|DA-DD-MM-CCYY)\b",
    flags=re.IGNORECASE,
)

DB2_DATE_WORKING_STORAGE_BASE_PATTERN = re.compile(
    r"\b(?:01\s+WS-DATUMVELDEN|DA-CCYYMMDD|DA-DD-MM-CCYY)\b",
    flags=re.IGNORECASE,
)

#
# DEPRECATED
#
# Retained only so an unmigrated import does not raise. It matches the
# narrow single-operand shape and must not be used for detection.
DB2_DATE_COMPARISON_PATTERN = re.compile(
    r"^(?P<indent>\s*)IF\s+"
    r"(?P<field>(?:DA|DT)-[A-Z0-9-]+)"
    r"\s+OF\s+"
    r"(?P<group>DCL[A-Z0-9-]+)"
    r"\s+"
    r"(?P<condition>.+)$",
    flags=re.IGNORECASE,
)

__all__ = [
    "WORKING_STORAGE_SECTION_PATTERN",
    "LINKAGE_SECTION_PATTERN",
    "PROCEDURE_DIVISION_PATTERN",
    "IF_START_PATTERN",
    "CONDITION_CONTINUATION_PATTERN",
    "STATEMENT_START_PATTERN",
    "TRAILING_BOOLEAN_PATTERN",
    "DB2_DATE_HOST_OPERAND_PATTERN",
    "SQL_HOST_OPERAND_PATTERN",
    "COMPARISON_OPERATOR_PATTERN",
    "EXEC_SQL_START_PATTERN",
    "END_EXEC_PATTERN",
    "DATE_HELPER_FIELD_PATTERN",
    "DATE_WORKING_STORAGE_MARKER_PATTERN",
    "DB2_SHARED_DATE_HELPER_USAGE_PATTERN",
    "DB2_DATE_WORKING_STORAGE_BASE_PATTERN",
    "DB2_DATE_COMPARISON_PATTERN",
]

=====
FILE: patterns\db2_include_patterns.py
ENCODING: utf-8
=====

"""DB2 INCLUDE scanning patterns.

Patterns only. No rules, no layout constants.

Both emitted shapes must be recognised, because duplicate detection that
sees only one of them is what produced DZBFARTV twice:

EXEC SQL INCLUDE DZBFARTV END-EXEC.

EXEC SQL

```

```

        INCLUDE DZBFARTV
    END-EXEC.
"""

from __future__ import annotations

import re

# The single-line form, complete on one logical line.
INCLUDE_SINGLE_PATTERN = re.compile(
    r"^EXEC\s+SQL\s+INCLUDE\s+(?P<name>[A-Z0-9][A-Z0-9_-]*)\s+END-EXEC\s*\.\?$",
    flags=re.IGNORECASE,
)

# The INCLUDE body line of the three-line block form.
INCLUDE_BODY_PATTERN = re.compile(
    r"^INCLUDE\s+(?P<name>[A-Z0-9][A-Z0-9_-]*)\s*\.\?$",
    flags=re.IGNORECASE,
)

# Any INCLUDE name, whichever shape carries it.
INCLUDE_NAME_PATTERN = re.compile(
    r"\bINCLUDE\s+(?P<name>[A-Z0-9][A-Z0-9_-]*)\b",
    flags=re.IGNORECASE,
)

EXEC_SQL_OPEN_PATTERN = re.compile(
    r"^EXEC\s+SQL\b",
    flags=re.IGNORECASE,
)

END_EXEC_PATTERN = re.compile(
    r"^END-EXEC\s*\.\?$",
    flags=re.IGNORECASE,
)

# A DCLGEN host group reference. Owned here so every consumer can rely on
# the 'group' capture name.
DCLGEN_GROUP_PATTERN = re.compile(
    r"\b(?:P<group>DCL[A-Z0-9_-]+)\b",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\db2_patterns.py
ENCODING: utf-8
=====

"""
DB2 infrastructure and generated block regex patterns.

DB2 infrastructure generators, validators, transformers, and composers
must import these patterns instead of defining DB2 regex internally.

This file must contain regex patterns only.

Do not place validation token lists, business rules, conversion rules,
or layout constants in this file.
"""

import re

SQLCA_TOKEN_PATTERN = re.compile(
    r"\bSQLCA\b",
    flags=re.IGNORECASE,
)

SQLERRWS_TOKEN_PATTERN = re.compile(
    r"\bSQLERRWS\b",
    flags=re.IGNORECASE,
)

SQL_LOCATION_PATTERN = re.compile(
    r"\bSQL-LOCATION\b",
    flags=re.IGNORECASE,
)

```

```

)

SQL_ERROR_PARAGRAPH_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?(SQL-ERROR|SQLERROR)\s*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

SQL_ERROR_PERFORM_PATTERN = re.compile(
    r"^\bPERFORM\s+(SQL-ERROR|SQLERROR)\s*\.\s*$",
    flags=re.IGNORECASE,
)

DB2_CURSOR_BLOCK_MARKER_PATTERN = re.compile(
    r"DB2 GENERATED CURSOR OPEN FETCH CLOSE PARAGRAPHS",
    flags=re.IGNORECASE,
)

DB2_INFRASTRUCTURE_MARKER_PATTERN = re.compile(
    r"DB2 SQLCA, SQL ERROR WORKING STORAGE, DCLGEN INCLUDES, AND CURSOR FLAGS",
    flags=re.IGNORECASE,
)

DB2_CURSOR_DECLARATIONS_MARKER_PATTERN = re.compile(
    r"DB2 GENERATED CURSOR DECLARATIONS",
    flags=re.IGNORECASE,
)

DB2_CURSOR_FLAGS_MARKER_PATTERN = re.compile(
    r"DB2 GENERATED CURSOR FLAGS",
    flags=re.IGNORECASE,
)

DB2_SQL_ERROR_LOCATION_MARKER_PATTERN = re.compile(
    r"DB2 GENERATED SQL ERROR LOCATION",
    flags=re.IGNORECASE,
)

PROCEDURE_DIVISION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?PROCEDURE\s+DIVISION\b.*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

LINKAGE_SECTION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?LINKAGE\s+SECTION\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

WORKING_STORAGE_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?WORKING-STORAGE\s+SECTION\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

DATA_DIVISION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?DATA\s+DIVISION\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

END_PROGRAM_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?END\s+PROGRAM\b.*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

EXEC_SQL_START_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?EXEC\s+SQL\b",
    flags=re.IGNORECASE,
)

EXEC_SQL_END_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?END-EXEC\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE,
)

SQL_INCLUDE_PATTERN = re.compile(
    r"^\bINCLUDE\s+(?P<include_name>[A-Z0-9_-]+\b",
    flags=re.IGNORECASE,
)

```

```

SQLCA_INCLUDE_PATTERN = re.compile(
    r"\bINCLUDE\s+SQLCA\b",
    flags=re.IGNORECASE,
)

SQLERRWS_INCLUDE_PATTERN = re.compile(
    r"\bINCLUDE\s+SQLERRWS\b",
    flags=re.IGNORECASE,
)

DECLARE_CURSOR_PATTERN = re.compile(
    r"\bDECLARE\s+(?P<cursor_name>[A-Z0-9_-]+\s+Cursors\b",
    flags=re.IGNORECASE,
)

OPEN_CURSOR_PATTERN = re.compile(
    r"\bOPEN\s+(?P<cursor_name>[A-Z0-9_-]+\b",
    flags=re.IGNORECASE,
)

FETCH_CURSOR_PATTERN = re.compile(
    r"\bFETCH\s+(?P<cursor_name>[A-Z0-9_-]+\b",
    flags=re.IGNORECASE,
)

CLOSE_CURSOR_PATTERN = re.compile(
    r"\bCLOSE\s+(?P<cursor_name>[A-Z0-9_-]+\b",
    flags=re.IGNORECASE,
)

SQLCODE_PATTERN = re.compile(
    r"\bSQLCODE\b",
    flags=re.IGNORECASE,
)

SQLCODE_SUCCESS_PATTERN = re.compile(
    r"\bSQLCODE\s*=\s*0\b",
    flags=re.IGNORECASE,
)

SQLCODE_NOT_FOUND_PATTERN = re.compile(
    r"\bSQLCODE\s*=\s*100\b",
    flags=re.IGNORECASE,
)

SQLCODE_ERROR_PATTERN = re.compile(
    r"\bSQLCODE\s*(?:NOT\s*=\s*0|<\s*0|>\s*0)\b",
    flags=re.IGNORECASE,
)

COMMIT_PATTERN = re.compile(
    r"\bCOMMIT\b",
    flags=re.IGNORECASE,
)

ROLLBACK_PATTERN = re.compile(
    r"\bROLLBACK\b",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\dclgen_patterns.py
ENCODING: utf-8
=====

"""
DCLGEN regex patterns.

Parser classes must import these patterns instead of defining regex inside
parser classes.
"""

import re

```

```

DECLARE_TABLE_PATTERN = re.compile(
    r"EXEC\s+SQL\s+DECLARE\s+([A-Z0-9_\. #\$@]+)\s+TABLE",
    flags=re.IGNORECASE,
)

DECLARE_TABLE_ALT_PATTERN = re.compile(
    r"\bDECLARE\s+([A-Z0-9_\. #\$@]+)\s+TABLE\b",
    flags=re.IGNORECASE,
)

DCLGEN_TABLE_COMMENT_PATTERN = re.compile(
    r"\bDCLGEN\s+TABLE\s*$(?=[A-Z0-9_\. #\$@]+)\s*$",
    flags=re.IGNORECASE,
)

COBOL_GROUP_PATTERN = re.compile(
    r"^\s*\*\s*\01\s+(DCL[A-Z0-9- ]+)\.?\s*$",
    flags=re.IGNORECASE,
)

COBOL_FIELD_PATTERN = re.compile(
    r"^\s*(0[2-9]|[1-4][0-9]|77)\s+"
    r"([A-Z][A-Z0-9-]*)\b(?P<body>.*)$",
    flags=re.IGNORECASE,
)

DCLGEN_PIC_PATTERN = re.compile(
    r"\bPIC(?:TURE)?\s+(?:IS\s+)?"
    r"(?P<pic>[A-Z0-9SV() .,+ \- ]+)",
    flags=re.IGNORECASE,
)

DCLGEN_USAGE_PATTERN = re.compile(
    r"\b(?:USAGE\s+(?:IS\s+)?)"
    r"(COMP-3|COMP|COMP-1|COMP-2|COMP-4|COMP-5|BINARY|PACKED-DECIMAL|DISPLAY)\b",
    flags=re.IGNORECASE,
)

SQL_NAME_CLEANUP_PATTERN = re.compile(
    r"^[A-Z0-9_#\$@]+",
    flags=re.IGNORECASE,
)

MULTIPLE_UNDERSCORE_PATTERN = re.compile(
    r"_+",
    flags=re.IGNORECASE,
)

VALID_SQL_COLUMN_NAME_PATTERN = re.compile(
    r"^[A-Z][A-Z0-9_#\$@]*$",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\dynamic_metadata_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

PROGRAM_RE = re.compile(
    r"\bPROGRAM-ID\s*\.\s*(?=[A-Z0-9_-])\s*\.",

```

```

    flags=re.IGNORECASE | re.DOTALL,
)

SELECT_RE = re.compile(
    r"\bSELECT\s+([A-Z0-9_-]+\s+ASSIGN\s+TO\s+([A-Z0-9_-]+\s*\."),
    flags=re.IGNORECASE | re.DOTALL,
)

READ_RE = re.compile(
    r"\bREAD\s+([A-Z0-9_-]+)(?:\s+INTO\s+([A-Z0-9_-]+))?",
    flags=re.IGNORECASE | re.DOTALL,
)

COPY_RE = re.compile(
    r"\bCOPY\s+([A-Z0-9_-]+\s*\."),
    flags=re.IGNORECASE,
)

FIELD_USE_RE = re.compile(
    r"\b([A-Z][A-Z0-9_-]+\s+(?:OF|IN)\s+([A-Z][A-Z0-9_-]+\s+b",
    flags=re.IGNORECASE,
)

MOVE_FIELD_RE = re.compile(
    r"\bMOVE\s+([A-Z][A-Z0-9_-]+\s+b",
    flags=re.IGNORECASE,
)

SELECT_LINE_RE = re.compile(
    r"\bSELECT\s+([A-Z0-9_-]+\s+ASSIGN\s+TO\s+([A-Z0-9_-]+\s*\."),
    flags=re.IGNORECASE,
)

FD_LINE_RE = re.compile(
    r"\bFD\s+([A-Z0-9_-]+\s+b",
    flags=re.IGNORECASE,
)

RECORD_CONTAINS_RE = re.compile(
    r"\bRECORD\s+CONTAINS\s+(\d+)\s+CHARACTERS\b",
    flags=re.IGNORECASE,
)

FD_RECORD_01_RE = re.compile(
    r"^\s*01\s+([A-Z0-9_-]+\s+b",
    flags=re.IGNORECASE,
)

SECTION_END_RE = re.compile(
    r"\b(WORKING-STORAGE|LINKAGE|PROCEDURE)\s+SECTION\b",
    flags=re.IGNORECASE,
)

READ_LINE_RE = re.compile(
    r"\bREAD\s+([A-Z0-9_-]+)(?:\s+INTO\s+([A-Z0-9_-]+))?",
    flags=re.IGNORECASE,
)

EOF_UNTIL_RE = re.compile(
    r"\bUNTIL\s+([A-Z][A-Z0-9-]*)\s*=\s*['\"]Y['\"]",
    flags=re.IGNORECASE,
)

EOF_IF_RE = re.compile(
    r"\bIF\s+(?:NOT\s+)?([A-Z][A-Z0-9-]*)\s*=\s*['\"]Y['\"]",
    flags=re.IGNORECASE,
)

EOF_MOVE_RE = re.compile(
    r"\bMOVE\s+['\"]Y['\"]\s+TO\s+([A-Z][A-Z0-9-]*)\s+b",
    flags=re.IGNORECASE,
)

PIC_LENGTH_RE = re.compile(
    r"[X9]$(\d+)$[X9]",
    flags=re.IGNORECASE,
)

```


)

```
=====
FILE: patterns\field_reference_patterns.py
ENCODING: utf-8
=====
```

```
import re
```

```
QUALIFIED_REFERENCE_PATTERN = re.compile(
    r"\b(?:P<field>[A-Z][A-Z0-9-]*)\s+"
    r"(?:P<qualifier>OF|IN)\s+"
    r"(?:P<record>[A-Z][A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)

REDEFINES_PATTERN = re.compile(
    r"\b(?:P<alias>[A-Z][A-Z0-9-]*)\s+REDEFINES\s+"
    r"(?:P<base>[A-Z][A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)

STRING_LITERAL_PATTERN = re.compile(
    r"'[^\']*'|\"[^\"]*\"",
    flags=re.IGNORECASE,
)
```

```
=====
FILE: patterns\field_reference_rewriter_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
"""
```

```
Regex patterns for field reference rewriting.
```

```
This module contains regex patterns only.
"""
```

```
import re
```

```
QUALIFIED_REFERENCE_PATTERN = re.compile(
    r"\b(?:P<field>[A-Z][A-Z0-9-]*)\s+"
    r"(?:P<qualifier>OF|IN)\s+"
    r"(?:P<record>[A-Z][A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)

EXEC_SQL_PATTERN = re.compile(
    r"^\s*EXEC\s+SQL\b",
    flags=re.IGNORECASE,
)

END_EXEC_PATTERN = re.compile(
    r"^\s*END-EXEC\.\s*$",
    flags=re.IGNORECASE,
)

DIVISION_PATTERN = re.compile(
    r"^\s*(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)\s+DIVISION\b",
    flags=re.IGNORECASE,
)

PARAGRAPH_PATTERN = re.compile(
    r"^\s*(?:P<name>[A-Z0-9-]*)\.\s*$",
    flags=re.IGNORECASE,
)
```

```

STRING_PATTERN = re.compile(
    r"'[^']*'",
    flags=re.IGNORECASE,
)

HOST_REFERENCE_OF_PATTERN = re.compile(
    r":?\\s*(?P<host>[A-Z][A-Z0-9-]*)\\s+OF\\s+(?P<group>DCL[A-Z0-9-]*)",
    flags=re.IGNORECASE,
)

HOST_REFERENCE_DOT_PATTERN = re.compile(
    r":?\\s*(?P<group>DCL[A-Z0-9-]*)\\.(?P<host>[A-Z][A-Z0-9-]*)",
    flags=re.IGNORECASE,
)

INITIALIZE_DCL_PATTERN = re.compile(
    r"\\bINITIALIZE\\s+(?P<group>DCL[A-Z0-9-]*)\\b",
    flags=re.IGNORECASE,
)

GENERATED_OBTAIN_COMMENT_PATTERN = re.compile(
    r"CONVERTED\\s+(?:OBTAIN|FIND)\\s+(?:FIRST|NEXT|CALC)?\\s*"
    r"(?P<record>[A-Z][A-Z0-9-]*)",
    flags=re.IGNORECASE,
)

MOVE_SPACES_TO_RECORD_PATTERN = re.compile(
    r"\\bMOVE\\s+(?:SPACE|SPACES|ZEROS|ZEROS)\\s+TO\\s+"
    r"(?P<record>[A-Z][A-Z0-9-]*)\\b",
    flags=re.IGNORECASE,
)

LEVEL_NUMBER_FIELD_PATTERN = re.compile(
    r"^\\s*(0[1-9]|[1-4][0-9]|66|77|88)\\s+([A-Z][A-Z0-9-]*)\\b",
    flags=re.IGNORECASE,
)

REDEFINES_FIELD_PATTERN = re.compile(
    r"^\\s*([A-Z][A-Z0-9-]*)\\s+REDEFINES\\s+([A-Z][A-Z0-9-]*)",
    flags=re.IGNORECASE,
)

REDEFINES_BASE_PATTERN = re.compile(
    r"\\bREDEFINES\\s+([A-Z][A-Z0-9-]*)\\b",
    flags=re.IGNORECASE,
)

COBOL_IDENTIFIER_PATTERN = re.compile(
    r"[A-Z][A-Z0-9-]*",
    flags=re.IGNORECASE,
)

OF_IN_BEFORE_PATTERN = re.compile(
    r"\\b(OF|IN)\\s*$",
    flags=re.IGNORECASE,
)

OF_IN_AFTER_PATTERN = re.compile(
    r"^\\s+(OF|IN)\\s+[A-Z0-9-]+",
    flags=re.IGNORECASE,
)

```

```

DCL_DOT_REFERENCE_PATTERN = re.compile(
    r":\s*DCL[A-Z0-9-]+\.",
    flags=re.IGNORECASE,
)

HOST_OF_DCL_REFERENCE_PATTERN = re.compile(
    r":\s*[A-Z0-9-]+\s+OF\s+DCL[A-Z0-9-]+",
    flags=re.IGNORECASE,
)

# Splits a cleaned host reference "HOST OF GROUP" on the OF keyword.
# Used by the host-reference resolver so no regex lives in logic.
HOST_REFERENCE_OF_SPLIT_PATTERN = re.compile(
    r"\s+OF\s+",
    flags=re.IGNORECASE,
)

```

```

=====
FILE: patterns\field_usage_analysis_patterns.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

"""
Field usage analyzer regex patterns.

This module contains regex patterns only.
No analyzer logic or business rules belong here.
"""

import re

QUALIFIED_REFERENCE_PATTERN = re.compile(
    r"\b(?:P<field>[A-Z][A-Z0-9-]*)\s+(?:OF|IN)\s+"
    r"(?:P<record>[A-Z][A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)

DCLGEN_OF_PATTERN = re.compile(
    r":?\s*(?:P<field>[A-Z][A-Z0-9-]*)\s+OF\s+"
    r"(?:P<group>DCL[A-Z0-9-]+)",
    flags=re.IGNORECASE,
)

DCLGEN_DOT_PATTERN = re.compile(
    r":?\s*(?:P<group>DCL[A-Z0-9-]+)\.(?:P<field>[A-Z][A-Z0-9-]*)",
    flags=re.IGNORECASE,
)

MOVE_PATTERN = re.compile(
    r"\bMOVE\s+(?:P<source>.+?)\s+TO\s+(?:P<target>.+?)(?:\.|$)",
    flags=re.IGNORECASE,
)

CONDITION_PATTERN = re.compile(
    r"^\s*(IF|WHEN|UNTIL|EVALUATE)\b",
    flags=re.IGNORECASE,
)

DIVISION_PATTERN = re.compile(
    r"^\s*(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)\s+DIVISION\b",
    flags=re.IGNORECASE,
)

COMMENT_PATTERN = re.compile(
    r"^\s*[\/]",

```

```

        flags=re.IGNORECASE,
    )

EXEC_SQL_START_PATTERN = re.compile(
    r"^EXEC\s+SQL\b",
    flags=re.IGNORECASE,
)

EXEC_SQL_END_PATTERN = re.compile(
    r"^END-EXEC\b",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\field_usage_patterns.py
ENCODING: utf-8
=====

"""
Field usage regex patterns.

This module contains regex patterns only.

Runtime analyzers, transformers, validators, composers, and services must
import these patterns instead of defining duplicate regex internally.

Rules:
- No business rules in this file.
- No service logic in this file.
- No parsing orchestration in this file.
- Regex patterns only.
"""

import re

QUALIFIED_REFERENCE_PATTERN = re.compile(
    r"\b(?:P<field>[A-Z][A-Z0-9-]*)\s+(?:OF|IN)\s+"
    r"(?:P<record>[A-Z][A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)

DCLGEN_OF_PATTERN = re.compile(
    r":\s+(?:P<field>[A-Z][A-Z0-9-]*)\s+OF\s+"
    r"(?:P<group>DCL[A-Z0-9-]+)",
    flags=re.IGNORECASE,
)

DCLGEN_DOT_PATTERN = re.compile(
    r":\s+(?:P<group>DCL[A-Z0-9-]+)\.(?:P<field>[A-Z][A-Z0-9-]*)",
    flags=re.IGNORECASE,
)

MOVE_STATEMENT_PATTERN = re.compile(
    r"^\s*MOVE\s+(?:P<source>.+?)\s+TO\s+(?:P<target>.+?)(?:\s*\$)",
    flags=re.IGNORECASE,
)

CONDITION_STATEMENT_PATTERN = re.compile(
    r"^\s*(IF|WHEN|UNTIL|EVALUATE)\b",
    flags=re.IGNORECASE,
)

OUTPUT_STATEMENT_PATTERN = re.compile(
    r"^\s*(DISPLAY|WRITE|REWRITE|RETURN|GOBACK|EXIT|CALL)\b",
    flags=re.IGNORECASE,
)

COMMENT_OR_BLANK_PATTERN = re.compile(
    r"^\s*(?:$|\s*)",
    flags=re.IGNORECASE,
)

```

```

EXEC_SQL_START_PATTERN = re.compile(
    r"^\\s*EXEC\\s+SQL\\b",
    flags=re.IGNORECASE,
)

EXEC_SQL_END_PATTERN = re.compile(
    r"^\\s*END-EXEC\\.?\\s*$",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\\final_feedback_fix_patterns.py
ENCODING: utf-8
=====

"""
Final feedback fix regex patterns.

This module contains regex patterns only.

No business rules.
No runtime logic.
No hardcoded program names, table names, fields, or cursor names.
"""

from __future__ import annotations

import re

PROGRAM_ID_PATTERN = re.compile(
    r"^\\s*(?:\\d{6})?\\s*PROGRAM-ID\\.\\s+"
    r"(?P<program>[A-Z0-9-]+)\\.\"",
    flags=re.IGNORECASE | re.MULTILINE,
)

MOVE_TO_PROGRAM_NAME_PATTERN = re.compile(
    r"(?P<prefix>\\bMOVE\\s+)"
    r"(?P<program>[A-Z0-9-]+)"
    r"(?P<suffix>'\\s+TO\\s+PROGRAM-NAME\\s*\\.?)\"",
    flags=re.IGNORECASE,
)

PROCEDURE_DIVISION_TOKEN_PATTERN = re.compile(
    r"\\bPROCEDURE\\s+DIVISION\\b",
    flags=re.IGNORECASE,
)

DIVISION_SECTION_HEADER_PATTERN = re.compile(
    r"^(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)\\s+DIVISION\\b|"
    r"^[A-Z0-9-]+\\s+SECTION\\.\\$|"
    r"^FILE-CONTROL\\.\\$|"
    r"^INPUT-OUTPUT\\s+SECTION\\.\\$|"
    r"^FILE\\s+SECTION\\.\\$|"
    r"^WORKING-STORAGE\\s+SECTION\\.\\$|"
    r"^LINKAGE\\s+SECTION\\.\\$",
    flags=re.IGNORECASE,
)

PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^[A-Z0-9][A-Z0-9-]*\\.\\$",
    flags=re.IGNORECASE,
)

EXEC_SQL_START_PATTERN = re.compile(
    r"^EXEC\\s+SQL\\b",
    flags=re.IGNORECASE,
)

END_EXEC_PATTERN = re.compile(
    r"\\bEND-EXEC\\b",
    flags=re.IGNORECASE,
)

DECLARE_CURSOR_PATTERN = re.compile(

```

```

        r"\bDECLARE\s+(?P<cursor>[A-Z0-9- ]+)\s+CURLSOR\b",
        flags=re.IGNORECASE,
    )

    FETCH_CURSOR_PATTERN = re.compile(
        r"\bFETCH\s+(?P<cursor>[A-Z0-9- ]+)\b",
        flags=re.IGNORECASE,
    )

    SELECT_KEYWORD_PATTERN = re.compile(
        r"^SELECT$",
        flags=re.IGNORECASE,
    )

    SELECT_ITEM_PATTERN = re.compile(
        r"^\s*,?\s*(?P<column>[A-Z0-9_ ]+)\s*$",
        flags=re.IGNORECASE,
    )

    FROM_PATTERN = re.compile(
        r"\bFROM\b",
        flags=re.IGNORECASE,
    )

    WHERE_PATTERN = re.compile(
        r"\bWHERE\b",
        flags=re.IGNORECASE,
    )

    ORDER_BY_PATTERN = re.compile(
        r"\bORDER\s+BY\b",
        flags=re.IGNORECASE,
    )

    FOR_READ_ONLY_PATTERN = re.compile(
        r"\bFOR\s+READ\s+ONLY\b",
        flags=re.IGNORECASE,
    )

    HOST_REFERENCE_PATTERN = re.compile(
        r":(?P<group>DCL[A-Z0-9 ]+)\.(?P<host>[A-Z0-9- ]+)",
        flags=re.IGNORECASE,
    )

    ASC_DESC_PATTERN = re.compile(
        r"\b(ASC|DESC)\b",
        flags=re.IGNORECASE,
    )

    SQL_NAME_TOKEN_PATTERN = re.compile(
        r"^[A-Z0-9_ ]+$",
        flags=re.IGNORECASE,
    )

    DATE_HELPER_EUR_PATTERN = re.compile(
        r"\bDA-DD-MM-CCYY\b",
        flags=re.IGNORECASE,
    )

    DATE_LOW_EUR_LITERAL_PATTERN = re.compile(
        r"'01\.\01\.\0001'",
        flags=re.IGNORECASE,
    )

    DATE_HIGH_EUR_LITERAL_PATTERN = re.compile(
        r"'31\.\12\.\9999'",
        flags=re.IGNORECASE,
    )

    # --- Appended: patterns previously inlined in FinalFeedbackFixComposer ---

    STRING_START_PATTERN = re.compile(r"^STRING\b", flags=re.IGNORECASE)
    STRING_END_PATTERN = re.compile(r"^END-STRING\.\?$", flags=re.IGNORECASE)
    STRING_INTRO_PATTERN = re.compile(r"^INTO\b", flags=re.IGNORECASE)

    PROCEDURE_DIVISION_PATTERN = re.compile(

```

```

        r"^PROCEDURE\s+DIVISION\b", flags=re.IGNORECASE
    )
    DIVISION_PATTERN = re.compile(
        r"^[A-Z][A-Z0-9-]*\s+DIVISION\.?$", flags=re.IGNORECASE
    )
    SECTION_PATTERN = re.compile(
        r"^[A-Z][A-Z0-9-]*\s+SECTION\.?$", flags=re.IGNORECASE
    )
    PARAGRAPH_PATTERN = re.compile(
        r"^[A-Z0-9][A-Z0-9-]*\s\.?$", flags=re.IGNORECASE
    )

    # Note: EXEC_SQL_START_PATTERN and END_EXEC_PATTERN already exist in this file.
    # The composer's local EXEC_SQL_END_PATTERN (r"^END-EXEC\.$") is renamed to
    # avoid clashing with the existing END_EXEC_PATTERN (r"\bEND-EXEC\b").
    EXEC_SQL_END_LINE_PATTERN = re.compile(r"^END-EXEC\.$", flags=re.IGNORECASE)

    IF_START_PATTERN = re.compile(r"^IF\s+", flags=re.IGNORECASE)
    ELSE_PATTERN = re.compile(r"^ELSE\.$", flags=re.IGNORECASE)
    END_IF_PATTERN = re.compile(r"^END-IF\.$", flags=re.IGNORECASE)
    EVALUATE_START_PATTERN = re.compile(r"^EVALUATE\s+", flags=re.IGNORECASE)
    WHEN_PATTERN = re.compile(r"^WHEN\s+", flags=re.IGNORECASE)
    END_EVALUATE_PATTERN = re.compile(r"^END-EVALUATE\.$", flags=re.IGNORECASE)

    DB2_COLON_COMMENT_BODY_PATTERN = re.compile(r"^\s*\s*DB2:", flags=re.IGNORECASE)
    DB2_SPACE_COMMENT_BODY_PATTERN = re.compile(
        r"^\s*\s*DB2\s+", flags=re.IGNORECASE
    )

```

```

=====
FILE: patterns\fixed_format_patterns.py
ENCODING: utf-8
=====

```

```

"""
Fixed-format COBOL regex patterns.
These patterns are used by fixed-format composers and helpers.
Keep regex definitions here instead of defining them inside composer classes.
"""

```

```

import re
LEFT_SEQUENCE_PATTERN = re.compile(
    r"^\s*(?P<left>\d{6})(?P<rest>.*)$",
    flags=re.IGNORECASE,
)
RIGHT_SEQUENCE_PATTERN = re.compile(
    r"^(?P<body>.*?)(?P<right>\d{8})\s*$",
    flags=re.IGNORECASE,
)
SEQUENCE_ONLY_PATTERN = re.compile(
    r"^\s*(\d{6}|\d{8})\s*$",
    flags=re.IGNORECASE,
)
DIVISION_PATTERN = re.compile(
    r"^(IDENTIFICATION|ENVIRONMENT|DATA|PROCEDURE)\s+DIVISION\b.*\s\.$",
    flags=re.IGNORECASE,
)
SECTION_PATTERN = re.compile(
    r"^[A-Z0-9-]+\s+SECTION\.$",
    flags=re.IGNORECASE,
)
PARAGRAPH_PATTERN = re.compile(
    r"^[A-Z0-9][A-Z0-9-]*\s\.$",
    flags=re.IGNORECASE,
)
AREA_A_PREFIX_PATTERN = re.compile(
    (
        r"^(
        r"PROGRAM-ID\." |
        r"AUTHOR\." |
        r"INSTALLATION\." |
        r"DATE-WRITTEN\." |
        r"DATE-COMPILED\." |
        r"SECURITY\." |
        r"FD\s+" |
        r"SD\s+"

```

```

r"01\s+|"
r"66\s+|"
r"77\s+"
r")"
),
flags=re.IGNORECASE,
)
DATA_LEVEL_PATTERN = re.compile(
r"^(0[1-9]|[1-4][0-9]|66|77|88)\s+",
flags=re.IGNORECASE,
)
EXEC_SQL_START_PATTERN = re.compile(
r"^EXEC\s+SQL\b",
flags=re.IGNORECASE,
)
EXEC_SQL_END_PATTERN = re.compile(
r"^END-EXEC\.$",
flags=re.IGNORECASE,
)
BOOLEAN_OPERATOR_END_PATTERN = re.compile(
r"\b(AND|OR)\s*$",
flags=re.IGNORECASE,
)
IF_BOOLEAN_SPLIT_PATTERN = re.compile(
r"(\s+AND\s+|\s+OR\s+)",
flags=re.IGNORECASE,
)
MOVE_STATEMENT_PATTERN = re.compile(
r"^MOVE\s+(?P<src>.+?)\s+TO\s+(?P<tgt>.+?\.$)",
flags=re.IGNORECASE,
)
DEBUG_LINE_PATTERN = re.compile(
r"^[Dd]\s+",
flags=re.IGNORECASE,
)

# --- Fixed-format line parser sequence patterns (appended) ---

# A true left sequence number: exactly six digits in columns 1-6 followed by
# whitespace and a body.
TRUE_LEFT_SEQUENCE_PATTERN = re.compile(
r"^(?P<left>\d{6})(?P<body>\s+.*$)",
flags=re.IGNORECASE,
)

# Right sequence number separated from the body by spaces.
RIGHT_SEQUENCE_WITH_SPACES_PATTERN = re.compile(
r"^(?P<body>.*?)(?P<spaces>\s+)(?P<right>\d{8})\s*$",
flags=re.IGNORECASE,
)

# Right sequence number accidentally attached to the body (no spaces).
TRAILING_TIGHT_RIGHT_SEQUENCE_PATTERN = re.compile(
r"^(?P<body>.*\S)(?P<right>\d{8})\s*$",
flags=re.IGNORECASE,
)

# A single COBOL identifier token (used to detect body endings).
COBOL_IDENTIFIER_TOKEN_PATTERN = re.compile(
r"[A-Z][A-Z0-9-]*",
flags=re.IGNORECASE,
)

=====
FILE: patterns\fixed_format_rules.py
ENCODING: utf-8
=====

"""
Fixed-format COBOL layout rules.
These constants define the final physical COBOL layout:
- Columns 1-6 : left sequence number
- Column 7 : indicator area
- Columns 8-72 : COBOL body
- Columns 73-80 : right sequence number

```



```

""
TOTAL_WIDTH = 80
BODY_WIDTH = 65
LEFT_SEQUENCE_WIDTH = 6
INDICATOR_WIDTH = 1
RIGHT_SEQUENCE_WIDTH = 8
DEFAULT_LEFT_START = 10
DEFAULT_LEFT_STEP = 10
# COBOL team expected:
# 000010 body 00010000
# 000020 body 00020000
DEFAULT_RIGHT_START = 10000
DEFAULT_RIGHT_STEP = 10000
AREA_A_INDENT = ""
AREA_B_INDENT = " "
SQL_INDENT = " "
SQL_CONTINUATION_INDENT = " "
SPACE_INDICATOR = " "
COMMENT_INDICATOR = "*"
PAGE_INDICATOR = "/"
DEBUG_INDICATOR = "D"
CONTINUATION_INDICATOR = "-"
VALID_INDICATORS = {
SPACE_INDICATOR,
COMMENT_INDICATOR,
PAGE_INDICATOR,
DEBUG_INDICATOR,
"d",
CONTINUATION_INDICATOR,
}
NON_PARAGRAPH_WORDS = {
"ACCEPT",
"ADD",
"CALL",
"CLOSE",
"COMMIT",
"CONTINUE",
"DISPLAY",
"ELSE",
"END-EXEC",
"END-EVALUATE",
"END-IF",
"END-PERFORM",
"EVALUATE",
"EXEC",
"EXIT",
"FETCH",
"IF",
"INITIALIZE",
"MOVE",
"OPEN",
"PERFORM",
"READ",
"SET",
"STOP",
"WHEN",
"WRITE",
}
PROCEDURE_VERBS = (
"ACCEPT ",
"ADD ",
"CALL ",
"CLOSE ",
"COMMIT",
"CONTINUE",
"DISPLAY ",
"ELSE",
"END-EVALUATE",
"END-IF",
"EVALUATE ",
"EXEC SQL",
"EXIT",
"IF ",
"INITIALIZE ",
"MOVE ",
"OPEN ",

```

```

"PERFORM ",
"READ ",
"SET ",
"STOP ",
"WHEN ",
"WRITE ",
)
SQL_LEVEL_1_KEYWORDS = (
"SELECT ",
"INTO ",
"FROM ",
"WHERE ",
"ORDER BY ",
"GROUP BY ",
"HAVING ",
"FETCH ",
"OPEN ",
"CLOSE ",
"COMMIT",
"ROLLBACK",
"INSERT ",
"UPDATE ",
"DELETE ",
"SET ",
"VALUES ",
"FOR READ ONLY",
"QUERYNO ",
)
SQL_LEVEL_2_KEYWORDS = (
"AND ",
"OR ",
",",
)
AREA_A_HEADER_PREFIXES = (
"PROGRAM-ID.",
"AUTHOR.",
"INSTALLATION.",
"DATE-WRITTEN.",
"DATE-COMPILED.",
"SECURITY.",
)
AREA_A_DATA_LEVELS = {
"01",
"66",
"77",
}
AREA_B_DATA_LEVELS = {
"02",
"03",
"04",
"05",
"06",
"07",
"08",
"09",
"10",
"11",
"12",
"13",
"14",
"15",
"16",
"17",
"18",
"19",
"20",
"21",
"22",
"23",
"24",
"25",
"26",
"27",
"28",
"29",
"30",

```

```

"31",
"32",
"33",
"34",
"35",
"36",
"37",
"38",
"39",
"40",
"41",
"42",
"43",
"44",
"45",
"46",
"47",
"48",
"49",
"88",
}

```

```

=====
FILE: patterns\fixed_format_wrap_patterns.py
ENCODING: utf-8
=====

```

```

# LOCATION: patterns/fixed_format_wrap_patterns.py
# ACTION: CREATE NEW FILE

```

```

"""Fixed-format wrapping regex. Patterns only."""

```

```

from __future__ import annotations

```

```

import re

```

```

# MOVE <source> TO <target>, with an optional terminator.

```

```

MOVE_TO_PATTERN = re.compile(
    r"^(MOVE\s+(?P<source>.+?)\s+TO\s+(?P<target>.+?)(?P<dot>\s*\.)?$",
    flags=re.IGNORECASE,
)

```

```

# A paragraph header on its own logical line.

```

```

PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^[A-Z0-9][A-Z0-9-]*\.$",
    flags=re.IGNORECASE,
)

```

```

# Division and section headers, which also close a preceding sentence.

```

```

DIVISION_OR_SECTION_PATTERN = re.compile(
    r"^(?:[A-Z]+\s+DIVISION\b|[A-Z0-9][A-Z0-9-]*\s+SECTION\s*\.)",
    flags=re.IGNORECASE,
)

```

```

=====
FILE: patterns\idms_patterns.py
ENCODING: utf-8
=====

```

```

"""

```

```

IDMS regex patterns.

```

```

COBOL parser and IDMS transformers must import these patterns instead of
defining IDMS operation regex inside parser or transformer classes.
"""

```

```

import re

```

```

OBTAIN_CALC_PATTERN = re.compile(
    r"\bOBTAIN\s+(?:KEEP\s+)?(?P<record>[A-Z0-9-]+\s+CALC\b",
    flags=re.IGNORECASE,
)

```

```

OBTAIN_CALC_REVERSED_PATTERN = re.compile(
    r"\bOBTAIN\s+(?:KEEP\s+)?CALC\s+(?P<record>[A-Z0-9-]+\)\b",
    flags=re.IGNORECASE,
)

OBTAIN_FIRST_NEXT_PATTERN = re.compile(
    r"\bOBTAIN\s+(?:KEEP\s+)?(?P<mode>FIRST|NEXT)\s+"
    r"(?P<record>[A-Z0-9-]+\)\s+WITHIN\s+(?P<set>[A-Z0-9-]+\)\b",
    flags=re.IGNORECASE,
)

OBTAIN_OWNER_PATTERN = re.compile(
    r"\bOBTAIN\s+OWNER\s+WITHIN\s+(?P<set>[A-Z0-9-]+\)\b",
    flags=re.IGNORECASE,
)

FIND_FIRST_PATTERN = re.compile(
    r"\bFIND\s+FIRST\s+(?P<record>[A-Z0-9-]+\)?\s*WITHIN\s+(?P<set>[A-Z0-9-]+\)\b",
    flags=re.IGNORECASE,
)

STORE_PATTERN = re.compile(
    r"\bSTORE\s+(?P<record>[A-Z0-9-]+\)\b",
    flags=re.IGNORECASE,
)

MODIFY_PATTERN = re.compile(
    r"\bMODIFY\s+(?P<record>[A-Z0-9-]+\)\b",
    flags=re.IGNORECASE,
)

ERASE_PATTERN = re.compile(
    r"\bERASE\s+(?P<record>[A-Z0-9-]+\)\b",
    flags=re.IGNORECASE,
)

READY_PATTERN = re.compile(
    r"\s*READY\b",
    flags=re.IGNORECASE,
)

READY_UPDATE_PATTERN = re.compile(
    r"\bREADY\s+AREA\s+(?P<record>[A-Z0-9-]+\).*UPDATE\b",
    flags=re.IGNORECASE,
)

FINISH_PATTERN = re.compile(
    r"\s*FINISH\b",
    flags=re.IGNORECASE,
)

COMMIT_PATTERN = re.compile(
    r"\s*COMMIT\b",
    flags=re.IGNORECASE,
)

BIND_STATEMENT_PATTERN = re.compile(
    r"\s*BIND\b",
    flags=re.IGNORECASE,
)

FIND_CURRENT_PATTERN = re.compile(
    r"\s*FIND\s+CURRENT\b",

```

```

        flags=re.IGNORECASE,
    )

CONNECT_STATEMENT_PATTERN = re.compile(
    r"^\\s*CONNECT\b",
    flags=re.IGNORECASE,
)

DISCONNECT_STATEMENT_PATTERN = re.compile(
    r"^\\s*DISCONNECT\b",
    flags=re.IGNORECASE,
)

USAGE_MODE_PATTERN = re.compile(
    r"\\bUSAGE-MODE\\s+IS\\s+(UPDATE|RETRIEVAL)\\b",
    flags=re.IGNORECASE,
)

IDMS_STATUS_PERFORM_PATTERN = re.compile(
    r"\\bPERFORM\\b.*\\bIDMS-STATUS\\b",
    flags=re.IGNORECASE,
)

IDMS_ABORT_PERFORM_PATTERN = re.compile(
    r"\\bPERFORM\\b.*\\bIDMS-ABORT\\b",
    flags=re.IGNORECASE,
)

DB_REC_NOT_FOUND_TOKEN_PATTERN = re.compile(
    r"\\bDB-REC-NOT-FOUND\\b",
    flags=re.IGNORECASE,
)

DB_END_OF_SET_TOKEN_PATTERN = re.compile(
    r"\\bDB-END-OF-SET\\b",
    flags=re.IGNORECASE,
)

ON_DB_REC_NOT_FOUND_PATTERN = re.compile(
    r"^\\s*ON\\s+DB-REC-NOT-FOUND\\s+(?P<statement>.+?)\\.?\\s*$",
    flags=re.IGNORECASE,
)

IDMS_DECLARATIVE_OR_CONTROL_PATTERNS = [
    re.compile(r"^\\s*IDMS-CONTROL\\s+SECTION\\b", flags=re.IGNORECASE),
    re.compile(r"^\\s*PROTOCOL\\b", flags=re.IGNORECASE),
    re.compile(r"^\\s*IDMS-RECORDS\\s+WITHIN\\b", flags=re.IGNORECASE),
    re.compile(r"^\\s*SCHEMA\\s+SECTION\\b", flags=re.IGNORECASE),
    re.compile(r"^\\s*DB\\s+[A-Z0-9-]+\\s+WITHIN\\s+[A-Z0-9-]+\\b", flags=re.IGNORECASE),
    re.compile(r"^\\s*COPY\\s+IDMS\\b", flags=re.IGNORECASE),
]

# LOCATION: patterns/idms_patterns.py
# ACTION: APPEND at the end of the file

# PROTOCOL continuation. The statement is
#
#     PROTOCOL. MODE IS BATCH-AUTOSTATUS DEBUG
#     IDMS-RECORDS MANUAL.
#
# Removing only the PROTOCOL line leaves "IDMS-RECORDS MANUAL." behind as
# residual IDMS, which CHK-03 reports and the compiler rejects.
IDMS_RECORDS_CLAUSE_PATTERN = re.compile(
    r"^\\s*IDMS-RECORDS\\s+(?:MANUAL|AUTOMATIC|WITHIN\\b.*)\\s*\\.?\\s*$",
    flags=re.IGNORECASE,
)

```

```

MODE_IS_CLAUSE_PATTERN = re.compile(
    r"^s*MODE\s+IS\s+[A-Z0-9-]+",
    flags=re.IGNORECASE,
)

IDMS_DECLARATIVE_OR_CONTROL_PATTERNS.extend(
    [
        IDMS_RECORDS_CLAUSE_PATTERN,
        MODE_IS_CLAUSE_PATTERN,
    ]
)

=====
FILE: patterns\lrf_patterns.py
ENCODING: utf-8
=====

# LOCATION: patterns/lrf_patterns.py
# ACTION: CREATE NEW FILE
"""Regex for the IDMS subschema Logical Record Facility (LRF) source."""

from __future__ import annotations

import re

SUBSCHEMA_HEADER_PATTERN = re.compile(
    r"^s*ADD\s+SUBSCHEMA\s+NAME\s+IS\s+(?P<subschema>[A-Z0-9-]+)"
    r"s*OF\s+SCHEMA\s+NAME\s+IS\s+(?P<schema>[A-Z0-9-]+)",
    re.IGNORECASE,
)

LOGICAL_RECORD_PATTERN = re.compile(
    r"^s*ADD\s+LOGICAL\s+RECORD\s+(?P<lr>[A-Z0-9-]+)",
    re.IGNORECASE,
)

ELEMENTS_ARE_PATTERN = re.compile(r"^s*ELEMENTS\s+ARE\s*$", re.IGNORECASE)
COMMENTS_PATTERN = re.compile(r"^s*COMMENTS\s*$", re.IGNORECASE)
QUOTED_TEXT_PATTERN = re.compile(r"'(?P<text>[^']*)'")

PATH_GROUP_PATTERN = re.compile(
    r"^s*ADD\s+PATH-GROUP\s+(?P<verb>[A-Z])\s+(?P<lr>[A-Z0-9-]+)",
    re.IGNORECASE,
)

SELECT_KEYWORD_PATTERN = re.compile(
    r"^s*SELECT\s+FOR\s+KEYWORD\s+(?P<keyword>[A-Z0-9-]+)",
    re.IGNORECASE,
)

FIND_OBTAIN_PATTERN = re.compile(
    r"^s*(?P<verb>FIND|OBTAIN)\s+(?P<scope>CURRENT|EACH|FIRST|NEXT|OWNER)?"
    r"s*(?P<record>[A-Z0-9-]+)"
    r"(?:\s+WITHIN\s+(?P<within>[A-Z0-9-]+))?",
    re.IGNORECASE,
)

ERASE_PATTERN = re.compile(r"^s*ERASE\s+(?P<record>[A-Z0-9-]+)", re.IGNORECASE)

IF_SET_EMPTY_PATTERN = re.compile(
    r"^s*IF\s+(?P<set_name>[A-Z0-9-])\s+IS\s+(?P<negate>NOT\s+)?EMPTY",
    re.IGNORECASE,
)

WHERE_PATTERN = re.compile(r"^s*WHERE\s+(?P<clause>.+?)s*$", re.IGNORECASE)

ON_STATUS_PATTERN = re.compile(
    r"^s*ON\s+(?P<status>\d{4})\s+(?P<action>.+?)s*$",
    re.IGNORECASE,
)

ELEMENT_NAME_PATTERN = re.compile(r"^s*(?P<record>[A-Z][A-Z0-9-]*)s*$")

# The program-side hook: COPY IDMS LR VMBTL03-R01.
COPY_IDMS_LR_PATTERN = re.compile(

```

```

        r"^\\s*COPY\\s+IDMS\\s+LR\\s+(?P<lr>[A-Z0-9-]+)\\s*\\.\"",
        re.IGNORECASE,
    )

# LR-qualified field reference: KY-SIFORM OF VMBSIAS OF LR
LR_QUALIFIED_FIELD_PATTERN = re.compile(
    r"(?P<field>[A-Z0-9-]+)\\s+OF\\s+(?P<record>[A-Z0-9-]+)\\s+OF\\s+LR",
    re.IGNORECASE,
)

=====
FILE: patterns\\lrf_program_patterns.py
ENCODING: utf-8
=====

# LOCATION: patterns/lrf_program_patterns.py
# ACTION: CREATE NEW FILE
"""Regex for LRF syntax appearing in the COBOL PROGRAM (not the subschema).

patterns/lrf_patterns.py owns the SUBSCHEMA source regex.
This module owns what the program itself writes.
"""

from __future__ import annotations

import re

# OBTAIN FIRST VMBTL03-R01 WHERE SWEEP-VMBFAS.
# OBTAIN NEXT VMBTL03-R01 WHERE VMBFAS-BY-VMBSIAS.
LR_OBTAIN_WHERE_PATTERN = re.compile(
    r"^\\s*OBTAIN\\s+(?:KEEP\\s+)?(?:P<mode>FIRST|NEXT|EACH|LAST|PRIOR)\\s+"
    r"(?P<lr>[A-Z][A-Z0-9-]*)\\s+"
    r"WHERE\\s+(?P<keyword>[A-Z][A-Z0-9-]*)\\s*\\.?\\s*$",
    flags=re.IGNORECASE,
)

# ERASE VMBTL03-R01 WHERE ERASE-VMBSIAS.
LR_ERASE_WHERE_PATTERN = re.compile(
    r"^\\s*ERASE\\s+(?P<lr>[A-Z][A-Z0-9-]*)\\s+"
    r"WHERE\\s+(?P<keyword>[A-Z][A-Z0-9-]*)\\s*\\.?\\s*$",
    flags=re.IGNORECASE,
)

# MODIFY / STORE logical record forms.
LR_MODIFY_WHERE_PATTERN = re.compile(
    r"^\\s*(?P<verb>MODIFY|STORE)\\s+(?P<lr>[A-Z][A-Z0-9-]*)\\s+"
    r"WHERE\\s+(?P<keyword>[A-Z][A-Z0-9-]*)\\s*\\.?\\s*$",
    flags=re.IGNORECASE,
)

# LR-STATUS = 'VMBFAS-EOA' / LR-STATUS NOT = 'VMBFAS-EOA'
LR_STATUS_CONDITION_PATTERN = re.compile(
    r"\\bLR-STATUS\\b\\s*(?P<operator>NOT\\s*=|\\s|=|NOT\\s+EQUAL\\s+TO|EQUAL\\s+TO)\\s*"
    r"'(?:P<literal>[\\^']*)'",
    flags=re.IGNORECASE,
)

# Any bare LR-STATUS reference (MOVE, DISPLAY, IF ... etc.)
LR_STATUS_TOKEN_PATTERN = re.compile(
    r"\\bLR-STATUS\\b",
    flags=re.IGNORECASE,
)

# KY-SIFORM OF VMBSIAS OF LR -> KY-SIFORM OF VMBSIAS
OF_LR_SUFFIX_PATTERN = re.compile(
    r"(?P<head>\\b[A-Z][A-Z0-9-]*\\s+OF\\s+[A-Z][A-Z0-9-]*)\\s+OF\\s+LR\\b",
    flags=re.IGNORECASE,
)

# A trailing bare "OF LR" with no record qualifier.
BARE_OF_LR_PATTERN = re.compile(
    r"\\s+OF\\s+LR\\b",
    flags=re.IGNORECASE,
)

```

```
# COPY IDMS SUBSCHEMA-LR-CTRL.
COPY_SUBSCHEMA_LR_CTRL_PATTERN = re.compile(
    r"^\s*COPY\s+IDMS\s+SUBSCHEMA-LR-CTRL\s*\.",
    flags=re.IGNORECASE,
)
```

```
=====
FILE: patterns\mapping_scope_patterns.py
ENCODING: utf-8
=====
```

```
# LOCATION: patterns/mapping_scope_patterns.py
# ACTION: CREATE NEW FILE

"""Mapping scope regex patterns. Patterns only."""
```

```
from __future__ import annotations

import re

# Trailing table qualifier on a mapping validation message:
# Mapping validation: DB2 column mapping is missing. Table=DZ01FFTB
MESSAGE_TABLE_PATTERN = re.compile(
    r"\bTable\s*=\s*(?P<table>[A-Z0-9_]+\)\b",
    flags=re.IGNORECASE,
)

# A record or table name token inside COBOL source.
SOURCE_TOKEN_PATTERN = re.compile(
    r"[A-Z][A-Z0-9_-]{2,}",
    flags=re.IGNORECASE,
)
```

```
=====
FILE: patterns\naming_patterns.py
ENCODING: utf-8
=====
```

```
"""
Naming patterns and naming helpers.

Name normalization logic must use these patterns instead of repeating
regex cleanup expressions across services.
"""
```

```
import re

NON_DB2_NAME_CHARACTER_PATTERN = re.compile(
    r"^[A-Z0-9_]+",
    flags=re.IGNORECASE,
)

NON_COMPACT_NAME_CHARACTER_PATTERN = re.compile(
    r"^[A-Z0-9_]+",
    flags=re.IGNORECASE,
)

MULTIPLE_UNDERSCORE_PATTERN = re.compile(
    r"_+",
    flags=re.IGNORECASE,
)

DCL_PREFIX_PATTERN = re.compile(
    r"^DCL_?",
    flags=re.IGNORECASE,
)

FOUR_DIGIT_RECORD_SUFFIX_PATTERN = re.compile(
    r"_[0-9]{4}$",
```



```

    flags=re.IGNORECASE,
)

=====
FILE: patterns\output_write_paragraph_patterns.py
ENCODING: utf-8
=====

# LOCATION: patterns/output_write_paragraph_patterns.py
# ACTION: CREATE NEW FILE

"""Output write paragraph extraction regex patterns.

This file must contain regex patterns only.

No token lists, business rules, conversion rules or layout constants.
All patterns are matched against an already-stripped logical line
(sequence numbers and indicator removed, uppercased).
"""

from __future__ import annotations

import re

# A paragraph header on a stripped logical line.
PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^(?P<name>[A-Z0-9][A-Z0-9-]*)\.$",
    flags=re.IGNORECASE,
)

PROCEDURE_DIVISION_PATTERN = re.compile(
    r"^PROCEDURE\s+DIVISION\b",
    flags=re.IGNORECASE,
)

DIVISION_PATTERN = re.compile(
    r"^(IDENTIFICATION|ENVIRONMENT|DATA)\s+DIVISION\b",
    flags=re.IGNORECASE,
)

# Block scope.
IF_START_PATTERN = re.compile(
    r"^IF\b",
    flags=re.IGNORECASE,
)

END_IF_PATTERN = re.compile(
    r"^END-IF\.?$",
    flags=re.IGNORECASE,
)

# The statement that identifies an output write block, and the record
# name it writes. The record name drives the generated paragraph name.
WRITE_STATEMENT_PATTERN = re.compile(
    r"^WRITE\s+(?P<record>[A-Z0-9][A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)

# A PERFORM of an already generated write paragraph, used to make the
# pass idempotent.
PERFORM_WRITE_PARAGRAPH_PATTERN = re.compile(
    r"^PERFORM\s+WRITE-[A-Z0-9][A-Z0-9-]*\b",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\pic_length_auto_fixer_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

"""
PIC length auto-fixer regex patterns.

```

This module contains regex patterns only.
 No transformer logic, parser logic, program names, table names, copybook names,
 DCLGEN names, or host variables belong here.

```
import re
```

```
PIC_DECLARATION_PATTERN = re.compile(
    r"^(?P<prefix>\s*\d+\s+)"
    r"(?P<name>[A-Z][A-Z0-9-]*)"
    r"(?P<middle>\s+PIC\s+)"
    r"(?P<pic>S?9|X)"
    r"$(?P<length>\d+)$"
    r"(?P<suffix>.*)$",
    flags=re.IGNORECASE,
)
```

```
MOVE_PATTERN = re.compile(
    r"\bMOVE\s+"
    r"(?P<source>[A-Z][A-Z0-9-]*)"
    r"\s+TO\s+"
    r"(?P<target>[A-Z][A-Z0-9-]*)"
    r"\b",
    flags=re.IGNORECASE,
)
```

```
=====
FILE: patterns\pic_patterns.py
ENCODING: utf-8
=====
```

"""

PIC and MOVE regex patterns.

PIC length auto-fixer and production validators must import these patterns
 instead of defining regex internally.

"""

```
import re
```

```
DATA_ENTRY_START_PATTERN = re.compile(
    r"^s*(?P<level>0[1-9]|[1-4][0-9]|77)\s+"
    r"(?P<name>[A-Z][A-Z0-9-]*)\b"
    r"(?P<rest>.*)$",
    flags=re.IGNORECASE,
)
```

```
NUMERIC_PIC_PATTERN = re.compile(
    r"\bPIC(?:TURE)?\s+(?:IS\s+)?"
    r"(?P<pic>S?9(?:$(?P<len>\d+)$)?)?"
    r"(?P<trailing>[^\n]*)"
    r"(?P<dot>\.)?",
    flags=re.IGNORECASE,
)
```

```
MOVE_PATTERN = re.compile(
    r"\bMOVE\s+"
    r"(?P<source>[A-Z][A-Z0-9-]*(?:\.[A-Z][A-Z0-9-]*)?)"
    r"(?:\s+OF\s+[A-Z][A-Z0-9-]*)?"
    r"\s+TO\s+"
    r"(?P<target>[A-Z][A-Z0-9-]*(?:\.[A-Z][A-Z0-9-]*)?)\b",
    flags=re.IGNORECASE,
)
```

```
=====
FILE: patterns\procedure_indent_patterns.py
ENCODING: utf-8
=====
```

```
# LOCATION: patterns/procedure_indent_patterns.py
# ACTION: CREATE NEW FILE
```

```
"""Procedure Division indentation regex patterns.
```

```
This file must contain regex patterns only.
```

```
No token lists, business rules, conversion rules or layout constants.
Every pattern is matched against an already-stripped, uppercased logical
line (sequence numbers and indicator removed).
"""
```

```
from __future__ import annotations
```

```
import re
```

```
# =====
# Division and section boundaries
# =====
PROCEDURE_DIVISION_PATTERN = re.compile(
    r"^\s*PROCEDURE\s+DIVISION\b",
    flags=re.IGNORECASE,
)
```

```
OTHER_DIVISION_PATTERN = re.compile(
    r"^(IDENTIFICATION|ENVIRONMENT|DATA)\s+DIVISION\b",
    flags=re.IGNORECASE,
)
```

```
SECTION_HEADER_PATTERN = re.compile(
    r"^[A-Z0-9][A-Z0-9-]*\s+SECTION\s*\.\?$",
    flags=re.IGNORECASE,
)
```

```
PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^(?P<name>[A-Z0-9][A-Z0-9-]*)\.\?$",
    flags=re.IGNORECASE,
)
```

```
# =====
# Block openers
# =====
IF_START_PATTERN = re.compile(
    r"^\s*IF\b",
    flags=re.IGNORECASE,
)
```

```
ELSE_PATTERN = re.compile(
    r"^\s*ELSE\b",
    flags=re.IGNORECASE,
)
```

```
EVALUATE_START_PATTERN = re.compile(
    r"^\s*EVALUATE\b",
    flags=re.IGNORECASE,
)
```

```
WHEN_PATTERN = re.compile(
    r"^\s*WHEN\b",
    flags=re.IGNORECASE,
)
```

```
# An inline PERFORM block: bare PERFORM, PERFORM UNTIL, PERFORM VARYING.
# 'PERFORM paragraph-name' does NOT open a block.
```

```
INLINE_PERFORM_PATTERN = re.compile(
    r"^\s*PERFORM\s*$"
    r"|^\s*PERFORM\s+UNTIL\b"
    r"|^\s*PERFORM\s+VARYING\b"
    r"|^\s*PERFORM\s+WITH\s+TEST\b",
    flags=re.IGNORECASE,
)
```

```

SCOPED_OPENER_PATTERN = re.compile(
    r"^(?P<verb>READ|SEARCH|STRING|UNSTRING)\b",
    flags=re.IGNORECASE,
)

# =====
# Block closers
# =====
SCOPE_TERMINATOR_PATTERN = re.compile(
    r"^(?P<token>END-[A-Z]+\s*\.\?$",
    flags=re.IGNORECASE,
)

# =====
# Embedded SQL
# =====
EXEC_SQL_START_PATTERN = re.compile(
    r"^EXEC\s+SQL\b",
    flags=re.IGNORECASE,
)

END_EXEC_PATTERN = re.compile(
    r"^END-EXEC\s*\.\?$",
    flags=re.IGNORECASE,
)

# =====
# Sentence terminator
# =====
LONE_PERIOD_PATTERN = re.compile(
    r"^\.$",
)

=====
FILE: patterns\record_materialisation_patterns.py
ENCODING: utf-8
=====

# LOCATION: patterns/record_materialisation_patterns.py
# ACTION: CREATE NEW FILE
"""Regex for IDMS record materialisation. Regex only."""

from __future__ import annotations

import re

_NAME = r"[A-Z][A-Z0-9-]*"

# MOVE VMBFAS TO F-FORM
WHOLE_RECORD_MOVE_PATTERN = re.compile(
    rf"^MOVE\s+(?P<record>{_NAME})\s+TO\s+(?P<target>{_NAME})\s*\.\s*$",
    flags=re.IGNORECASE,
)

# 05 F-FORM PIC X(478).
TARGET_FIELD_PATTERN_TEMPLATE = (
    r"^(?P<level>\d{{2}})\s+(?P<name>{name})\s+"
    r"(?P<picture>PIC(?:TURE)?\s+[\^.]+)\s*$"
)

# "03 NR-CIO-FORM-AS" -> level 03, name NR-CIO-FORM-AS
COBOL_ZONE_PATTERN = re.compile(
    rf"^\s*(?P<level>\d{{2}})?\s*(?P<name>{_NAME})\s*$",
    flags=re.IGNORECASE,
)

# "05 CT-RK-TGOOD REDEFINES CT-RK-TGDSV"
COBOL_ZONE_REDEFINES_PATTERN = re.compile(
    rf"^\s*(?P<level>\d{{2}})?\s*(?P<name>{_NAME})\s+REDEFINES\s+"
    rf"(?P<base>{_NAME})\s*$",
    flags=re.IGNORECASE,
)

# PIC 9(8) COMP-3 / PIC X(4) / PIC S9(13)V99 COMP-3

```

```

PICTURE_PATTERN = re.compile(
    r"PIC(?:TURE)?\s+(?P<picture>[XA9VZ0-9()\-.,/+*]+)"
    r"(?P<usage>\s+(?:COMP-3|COMP|COMPUTATIONAL-3|COMPUTATIONAL"
    r"|BINARY|PACKED-DECIMAL|DISPLAY))?",
    flags=re.IGNORECASE,
)

# Repeat factor inside a picture: 9(8), X(478)
PICTURE_REPEAT_PATTERN = re.compile(r"(?P<symbol>[XA9Z])$(?P<count>\d+)$")

WORKING_STORAGE_PATTERN = re.compile(
    r"^WORKING-STORAGE\s+SECTION\s*\.", flags=re.IGNORECASE,
)

PROCEDURE_DIVISION_PATTERN = re.compile(
    r"^PROCEDURE\s+DIVISION\b", flags=re.IGNORECASE,
)

# LOCATION: patterns/record_materialisation_patterns.py
# ACTION: APPEND at the end of the file

# Every character-position symbol of a PICTURE clause, with its optional
# repeat count.
#
#     X(478)    -> symbol X, repeat 478
#     XXX       -> symbol X three times, repeat None
#     S9(13)V99-> 9 with repeat 13, then 9 twice
#
# Used to size a PICTURE from the LINE, so the byte count never depends
# on which group another pattern happened to capture.
PICTURE_SYMBOL_PATTERN = re.compile(
    r"(?P<symbol>[X9AZ])(?:$(?P<repeat>\d+)$)?",
    flags=re.IGNORECASE,
)

# Everything after the PIC / PICTURE keyword on a data description entry.
PICTURE_CLAUSE_PATTERN = re.compile(
    r"\bPIC(?:TURE)?\s+(?:IS\s+)?(?:P<clause>[^\s]*)",
    flags=re.IGNORECASE,
)

# LOCATION: patterns/record_materialisation_patterns.py
# ACTION: REPLACE the previously appended block with this one

# One PICTURE symbol together with its optional repeat count.
#
# The existing PICTURE_SYMBOL_PATTERN captures the symbol ONLY. It has no
# 'repeat' group, so every clause using (n) collapsed to the number of
# literal symbols:
#
#     PIC 9999    -> 4    correct, four literal symbols
#     PIC 9(7)    -> 1    WRONG, the (7) was never read
#     PIC X(478)  -> 1    WRONG
#
# F-FORM therefore measured 1 byte, the length guard refused a 432-byte
# VMBFAS layout as "longer than the 1 byte F-FORM declares", and roughly
# 400 lines of generated output were discarded on every run.
#
# Whitespace inside the parentheses is tolerated: the mapping workbook
# contains clauses written as 'PIC X(4 )'.
PICTURE_SYMBOL_REPEAT_PATTERN = re.compile(
    r"(?P<symbol>[X9AZ])"
    r"(?:\s*$\s*(?P<repeat>\d+)\s*$)?",
    flags=re.IGNORECASE,
)

# The PICTURE clause itself, bounded so that no trailing clause is read
# as picture symbols.
#
#     77  USERABEN  PIC X(8)  VALUE 'USERABEN'.
#
# An unbounded capture would count the A of VALUE and the A of USERABEN
# and report 10 bytes instead of 8.
PICTURE_AFTER_KEYWORD_PATTERN = re.compile(
    r"\bPIC(?:TURE)?\s+(?:IS\s+)?"
    r"(?P<picture>.+?)"
    r"(?=\s+(?:USAGE|COMPUTATIONAL-3|COMPUTATIONAL|COMP-3|COMP"

```

```

r"|PACKED-DECIMAL|BINARY|DISPLAY|VALUE|OCCURS|REDEFINES)\b"
r"|\s*\.\s*$|$",
flags=re.IGNORECASE,
)

# USAGE clause on the same entry, used only to halve a COMP-3 length.
USAGE_CLAUSE_PATTERN = re.compile(
    r"\b(?:USAGE\s+(?:IS\s+)?)?"
    r"(?P<usage>COMPUTATIONAL-3|COMP-3|PACKED-DECIMAL|COMPUTATIONAL"
    r"|COMP|BINARY|DISPLAY)\b",
    flags=re.IGNORECASE,
)

```

```

=====
FILE: patterns\sequence_patterns.py
ENCODING: utf-8
=====

```

```

# LOCATION: patterns/sequence_patterns.py
# ACTION: REPLACE ENTIRE FILE

```

```

"""COBOL fixed-format sequence handling.

```

```

ROOT CAUSE CORRECTED IN THIS FILE
-----

```

```

The previous left-sequence pattern was:

```

```

LEFT_SEQUENCE_PATTERN = re.compile(
    r"^\s*(?P<seq>\\d{6})(?P<body>\\s+.*$)"
)

```

The body group demanded WHITESPACE immediately after the six digits.

In fixed-format COBOL, position 7 is the INDICATOR AREA. It holds a space on ordinary statement lines, but it legitimately holds:

```

'*'  comment
 '/'  comment + page eject
 '-'  continuation
 'D'  debug line

```

On any such line the pattern did not match, so the left sequence number was NEVER STRIPPED. Downstream consumers then saw the sequence number as COBOL text:

```

001690/                                00420000
-> strip_sequence_numbers(...) returned "001690/"
-> is_comment_or_blank("001690/") returned False
-> ParagraphTerminatorCleanup appended a period
-> "001690/." <- not valid COBOL

```

That single defect also hid trailing IF ... WRITE ... END-IF blocks from OutputWritePlacementCleanup, so a guarded output WRITE was never relocated out of the child-row paragraph and fired once per fetched child row.

Stripping is now COLUMN-BASED, which is what the COBOL standard actually specifies, with a regex fallback only for free-form text that carries no fixed-format frame.

```

CONTRACT (unchanged for callers)
-----

```

```

strip_sequence_numbers(line) returns the line with columns 1-6 and 73-80
removed, KEEPING column 7 onwards. A comment line therefore still starts
with its '*' or '/' indicator, which is what is_comment_or_blank() relies
on.

```

```

This file must contain regex patterns and pure text helpers only.
No business rules, no layout constants, no program names.
"""

```

```

from __future__ import annotations

```

```

import re

```

```

# =====

```

```

# Fixed-format geometry
# =====
LEFT_SEQUENCE_START = 0
LEFT_SEQUENCE_END = 6
INDICATOR_INDEX = 6
BODY_START = 7
BODY_END = 72
RIGHT_SEQUENCE_START = 72
RIGHT_SEQUENCE_END = 80

MIN_ADDRESSABLE_LENGTH = 7

# Every indicator the COBOL standard permits in column 7.
VALID_INDICATORS = (" ", "*", "/", "D", "d", "-")
COMMENT_INDICATORS = ("*", "/")
DEBUG_INDICATORS = ("D", "d")
CONTINUATION_INDICATOR = "-"

# =====
# Patterns
# =====
# Left sequence followed by the indicator area. The body group now accepts
# ANY indicator character, not whitespace only. Kept for callers that
# import it directly.
LEFT_SEQUENCE_PATTERN = re.compile(
    r"^(?P<seq>\d{6})(?P<body>[ */\-Dd].*)?$",
)

# Trailing eight-digit right sequence on a free-form line.
RIGHT_SEQUENCE_PATTERN = re.compile(
    r"^(?P<body>.*?)\s+(?P<right>\d{8})\s*$",
)

# A body that is nothing but a leaked six-digit sequence number, with an
# optional indicator character and an optional period.
SEQUENCE_ARTIFACT_PATTERN = re.compile(
    r"^\s*\d{6}\s*[ */\-Dd]? \s*\.\s*$",
)

# =====
# Helpers
# =====
def is_fixed_format_line(line: str) -> bool:
    """True when columns 1-6 are numeric and column 7 is a valid indicator.

    The right sequence is deliberately optional: hand-maintained members
    often leave it blank or truncate the line, and demanding it was what
    caused sequence numbers to be parsed as COBOL text.
    """
    text = str(line or "").rstrip("\n")

    if len(text) < MIN_ADDRESSABLE_LENGTH:
        return False

    if not text[LEFT_SEQUENCE_START:LEFT_SEQUENCE_END].isdigit():
        return False

    return text[INDICATOR_INDEX] in VALID_INDICATORS

def indicator_of(line: str) -> str:
    """Column 7 of a fixed-format line, or '' when there is none."""
    text = str(line or "").rstrip("\n")
    return text[INDICATOR_INDEX] if is_fixed_format_line(text) else ""

def is_comment_or_control(line: str) -> bool:
    """True for comment, page-eject and debug lines.

    Works on a RAW line, before any stripping, so a caller can classify
    correctly without relying on the stripped text starting with '*'.
    """
    text = str(line or "").rstrip("\n")

    if not text.strip():

```

```

        return False

    if is_fixed_format_line(text):
        return text[INDICATOR_INDEX] in COMMENT_INDICATORS + DEBUG_INDICATORS

    stripped = text.lstrip()
    return bool(stripped) and stripped[0] in COMMENT_INDICATORS

def is_sequence_artifact(line: str) -> bool:
    """True when the stripped body is only a leaked sequence number."""
    body = strip_sequence_numbers(line).strip()
    if not body:
        return False
    return bool(SEQUENCE_ARTIFACT_PATTERN.match(body))

def strip_left_sequence(line: str) -> str:
    """Remove columns 1-6, keeping the indicator and the body."""
    text = str(line or "").rstrip("\n")

    if is_fixed_format_line(text):
        return text[LEFT_SEQUENCE_END:]

    match = LEFT_SEQUENCE_PATTERN.match(text)
    if match:
        return match.group("body") or ""

    return text

def strip_right_sequence(line: str) -> str:
    """Remove columns 73-80."""
    text = str(line or "").rstrip("\n")

    if is_fixed_format_line(text) and len(text) > RIGHT_SEQUENCE_START:
        return text[:RIGHT_SEQUENCE_START]

    match = RIGHT_SEQUENCE_PATTERN.match(text)
    if match:
        return match.group("body")

    return text

def strip_sequence_numbers(line: str) -> str:
    """Remove columns 1-6 and 73-80, KEEPING column 7 onwards.

    A comment line therefore still begins with its '*' or '/' indicator,
    preserving the contract every existing caller depends on.
    """
    text = str(line or "").rstrip("\n")

    if is_fixed_format_line(text):
        padded = text.ljust(RIGHT_SEQUENCE_END)
        return padded[INDICATOR_INDEX:RIGHT_SEQUENCE_START].rstrip()

    return strip_right_sequence(strip_left_sequence(text)).rstrip()

def logical_body(line: str) -> str:
    """Columns 8-72 only, stripped. Indicator excluded."""
    text = str(line or "").rstrip("\n")

    if is_fixed_format_line(text):
        padded = text.ljust(RIGHT_SEQUENCE_END)
        return padded[BODY_START:BODY_END].strip()

    return strip_sequence_numbers(text).strip()

__all__ = [
    "LEFT_SEQUENCE_PATTERN",
    "RIGHT_SEQUENCE_PATTERN",
    "SEQUENCE_ARTIFACT_PATTERN",
    "VALID_INDICATORS",

```



```

"COMMENT_INDICATORS",
"DEBUG_INDICATORS",
"CONTINUATION_INDICATOR",
"is_fixed_format_line",
"indicator_of",
"is_comment_or_control",
"is_sequence_artifact",
"strip_left_sequence",
"strip_right_sequence",
"strip_sequence_numbers",
"logical_body",
]

```

```

=====
FILE: patterns\sheet_mapping_patterns.py
ENCODING: utf-8
=====

```

```

"""
Sheet Mapping regex patterns.

Sheet Mapping parser logic must import these patterns instead of defining
regex directly inside parser classes.
"""

```

```
import re
```

```

HEADER_NON_ALPHANUMERIC_PATTERN = re.compile(
    r"^[A-Z0-9]+",
    flags=re.IGNORECASE,
)

```

```

HEADER_WHITESPACE_PATTERN = re.compile(
    r"\s+",
    flags=re.IGNORECASE,
)

```

```

CELL_WHITESPACE_PATTERN = re.compile(
    r"[\t]+",
    flags=re.IGNORECASE,
)

```

```

=====
FILE: patterns\sql_error_patterns.py
ENCODING: utf-8
=====

```

```

# LOCATION: patterns/sql_error_patterns.py
# ACTION: CREATE NEW FILE
"""SQLERROR regex patterns.

```

```

This file must contain regex patterns only.
No token lists, business rules, conversion rules or layout constants.
"""

```

```
from __future__ import annotations
```

```
import re
```

```

# A SQLERROR or SQL-ERROR paragraph header anywhere in the text.
SQLERROR_PARAGRAPH_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?(?:SQL-ERROR|SQLERROR)\s*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

```

```

# The same header matched against an already-stripped logical line.
SQLERROR_HEADER_ONLY_PATTERN = re.compile(
    r"(?:SQL-ERROR|SQLERROR)\s*\.",
    flags=re.IGNORECASE,
)

```

Any single-word paragraph header on a stripped logical line.

```
PARAGRAPH_HEADER_PATTERN = re.compile(
    r"[A-Z0-9][A-Z0-9-]*\s*\.",
    flags=re.IGNORECASE,
)
```

END PROGRAM on a stripped logical line.

```
END_PROGRAM_BOUNDARY_PATTERN = re.compile(
    r"END\s+PROGRAM\b",
    flags=re.IGNORECASE,
)
```

```
=====
FILE: patterns\sql_patterns.py
```

```
ENCODING: utf-8
=====
```

```
"""
```

SQL regex patterns and formatting keyword groups.

SQL generators, formatters, validators, and timestamp/audit components must

import SQL patterns from here instead of defining regex internally.

```
"""
```

```
import re
```

```
EXEC_SQL_PATTERN = re.compile(
    r"^\s*EXEC\s+SQL\b",
    flags=re.IGNORECASE,
)
```

```
END_EXEC_PATTERN = re.compile(
    r"^\s*END-EXEC\.\s*$",
    flags=re.IGNORECASE,
)
```

```
INCLUDE_PATTERN = re.compile(
    r"\bINCLUDE\s+(?P<table>[A-Z][A-Z0-9_]*)\b",
    flags=re.IGNORECASE,
)
```

```
SELECT_SQL_PATTERN = re.compile(
    r"\bSELECT\b",
    flags=re.IGNORECASE,
)
```

```
FROM_SQL_PATTERN = re.compile(
    r"\bFROM\s+(?P<table>[A-Z][A-Z0-9_]*)\b",
    flags=re.IGNORECASE,
)
```

```
WHERE_SQL_PATTERN = re.compile(
    r"\bWHERE\b",
    flags=re.IGNORECASE,
)
```

```
UPDATE_SQL_PATTERN = re.compile(
    r"\bUPDATE\s+(?P<table>[A-Z][A-Z0-9_]*)\b",
    flags=re.IGNORECASE,
)
```

```
INSERT_SQL_PATTERN = re.compile(
    r"\bINSERT\s+INTO\s+(?P<table>[A-Z][A-Z0-9_]*)\b",
    flags=re.IGNORECASE,
)
```

```

DELETE_SQL_PATTERN = re.compile(
    r"\bDELETE\s+FROM\s+(?P<table>[A-Z][A-Z0-9_]*)\b",
    flags=re.IGNORECASE,
)

OPEN_CURSOR_PATTERN = re.compile(
    r"\bOPEN\s+(?P<cursor>[A-Z][A-Z0-9_]*)\b",
    flags=re.IGNORECASE,
)

FETCH_CURSOR_PATTERN = re.compile(
    r"\bFETCH\s+(?P<cursor>[A-Z][A-Z0-9_]*)\b",
    flags=re.IGNORECASE,
)

CLOSE_CURSOR_PATTERN = re.compile(
    r"\bCLOSE\s+(?P<cursor>[A-Z][A-Z0-9_]*)\b",
    flags=re.IGNORECASE,
)

HOST_REFERENCE_PATTERN = re.compile(
    r":\s*(?P<group>DCL[A-Z0-9-]*+)\s*\.\s*(?P<field>[A-Z][A-Z0-9_]*)",
    flags=re.IGNORECASE,
)

SQL_KEYWORDS_ZERO_INDENT = (
    "EXEC SQL",
    "END-EXEC",
)

SQL_KEYWORDS_LEVEL_1 = (
    "INCLUDE ",
    "DECLARE ",
    "SELECT",
    "INTO",
    "FROM ",
    "WHERE",
    "ORDER BY",
    "GROUP BY",
    "HAVING",
    "FETCH ",
    "OPEN ",
    "CLOSE ",
    "COMMIT",
    "ROLLBACK",
    "INSERT ",
    "UPDATE ",
    "DELETE ",
    "SET ",
    "VALUES",
    "FOR READ ONLY",
)

SQL_KEYWORDS_LEVEL_2 = (
    "AND ",
    "OR ",
)

```

```

=====
FILE: patterns\sqlcode_wrapper_patterns.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import re

# Regex patterns for SQLCODE wrapper cleanup.

```

```
# Regex only. No business rules, program names, or hardcoded records.
```

```
# One-line MOVE <source> TO <target> (used to split into two body lines).
```

```
MOVE_TO_PATTERN = re.compile(
    r"^MOVE\s+(?P<source>.+?)\s+TO\s+(?P<target>.+?.?)$",
    flags=re.IGNORECASE,
)
```

```
=====
FILE: patterns\structural_safety_patterns.py
```

```
ENCODING: utf-8
=====
```

```
# LOCATION: patterns/structural_safety_patterns.py
```

```
# ACTION: REPLACE ENTIRE FILE
```

```
"""Regex for the structural safety passes. Regex only.
```

```
CORRECTION - numbered paragraphs were invisible
```

```
-----
_NAME was [A-Z][A-Z0-9-]* , which requires a leading LETTER. Every
generated cursor paragraph is numbered:
```

```
710-OPEN-DZBFASC1.
```

```
720-FETCH-DZBFASC1.
```

```
730-CLOSE-DZBFASC1.
```

```
None of them matched PARAGRAPH_HEADER_PATTERN, so the unreachable-code
guard was never disarmed and commented the whole generated block.
```

```
Two name classes are now declared. A DATA NAME must start with a letter.
```

```
A PARAGRAPH NAME may start with a digit.
```

```
"""
```

```
from __future__ import annotations
```

```
import re
```

```
# A COBOL data-name: must begin with a letter.
```

```
_DATA_NAME = r"[A-Z][A-Z0-9-]*"
```

```
# A COBOL paragraph / section name: may begin with a digit.
```

```
_PARAGRAPH_NAME = r"[A-Z0-9][A-Z0-9-]*"
```

```
# ---- conditions
```

```
IF_START_PATTERN = re.compile(r"^IF\b", flags=re.IGNORECASE)
```

```
END_IF_PATTERN = re.compile(r"^END-IF\s*\.\s*$", flags=re.IGNORECASE)
```

```
# ---- data declarations
```

```
LEVEL_01_PATTERN = re.compile(
    rf"^\01\s+(?P<name>{_DATA_NAME})\s*(?P<terminator>\.?)",
    flags=re.IGNORECASE,
)
```

```
SUBORDINATE_LEVEL_PATTERN = re.compile(
    rf"^(?P<level>\0[2-9]|[1-4][0-9]|66|77|88)\s+(?P<name>{_DATA_NAME})\b",
    flags=re.IGNORECASE,
)
```

```
FD_PATTERN = re.compile(r"^FD\s+", flags=re.IGNORECASE)
```

```
FILE_SECTION_PATTERN = re.compile(
    r"^FILE\s+SECTION\s*\.", flags=re.IGNORECASE,
)
```

```
WORKING_STORAGE_PATTERN = re.compile(
    r"^WORKING-STORAGE\s+SECTION\s*\.", flags=re.IGNORECASE,
)
```

```
LINKAGE_SECTION_PATTERN = re.compile(
    r"^LINKAGE\s+SECTION\s*\.", flags=re.IGNORECASE,
)
```

```
PROCEDURE_DIVISION_PATTERN = re.compile(
    r"^PROCEDURE\s+DIVISION\b", flags=re.IGNORECASE,
)
```

```
# ---- record-layout evidence
```

```
RECORD_VERB_PATTERN = re.compile(
    rf"^(?P<verb>WRITE|READ|REWRITE|RELEASE|RETURN)\s+"
    rf"(?P<name>{_DATA_NAME})\b",
    flags=re.IGNORECASE,
)
```

```

)
WRITE_FROM_PATTERN = re.compile(
    rf"\bFROM\s+(\?P<name>{_DATA_NAME})\b", flags=re.IGNORECASE,
)
READ_INTO_PATTERN = re.compile(
    rf"\bINTO\s+(\?P<name>{_DATA_NAME})\b", flags=re.IGNORECASE,
)
MOVE_WHOLE_GROUP_PATTERN = re.compile(
    rf"^MOVE\s+(\?:SPACE|SPACES|ZERO|ZEROES|ZEROS)\s+TO\s+"
    rf"(\?P<name>{_DATA_NAME})\s*\.\s*$",
    flags=re.IGNORECASE,
)

# ---- orphaned whole-record references
MOVE_BARE_SOURCE_PATTERN = re.compile(
    rf"^MOVE\s+(\?P<name>{_DATA_NAME})\s+TO\s+"
    rf"(\?P<target>{_DATA_NAME})\s*\.\s*$",
    flags=re.IGNORECASE,
)
MOVE_BARE_TARGET_PATTERN = re.compile(
    rf"^MOVE\s+(\?P<source>{_DATA_NAME})\s+TO\s+"
    rf"(\?P<name>{_DATA_NAME})\s*\.\s*$",
    flags=re.IGNORECASE,
)
QUALIFIER_PATTERN = re.compile(
    rf"\b(?:OF|IN)\s+(\?P<record>{_DATA_NAME})\b", flags=re.IGNORECASE,
)

# ---- paragraph structure
PARAGRAPH_HEADER_PATTERN = re.compile(
    rf"^(?P<name>{_PARAGRAPH_NAME})\s*\.\s*$", flags=re.IGNORECASE,
)
SECTION_HEADER_PATTERN = re.compile(
    rf"^{_PARAGRAPH_NAME}\s+SECTION\s*\.\s*$", flags=re.IGNORECASE,
)
EXIT_STATEMENT_PATTERN = re.compile(
    rf"^(?P<word>EXIT|GOBACK|STOP\s+RUN)\s*\.\s*$", flags=re.IGNORECASE,
)

=====
FILE: patterns\update_business_sql_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

"""
Regex patterns for update-program business SQL standardization.

This module contains regex patterns only.
No conversion logic, business rules, file paths, program names, DB2 table names,
DCLGEN names, copybook names, or host variables belong here.
"""

CONVERTED_MODIFY_COMMENT_PATTERN = re.compile(
    rf"^\s*\*DB2:\s*Converted\s+MODIFY\s+for\s+(\?P<record>[A-Z0-9-]+)\s*\.\s*$",
    flags=re.IGNORECASE,
)

SQL_LOCATION_UPDATE_PATTERN = re.compile(
    rf"^\s*\*MOVE\s+['\"]UPDATE-(?P<record>[A-Z0-9-]+)['\"]\s+TO\s+SQL-LOCATION\s*\.\s*",
    flags=re.IGNORECASE,
)

EXEC_SQL_START_PATTERN = re.compile(
    rf"^\s*\*EXEC\s+SQL\b",
    flags=re.IGNORECASE,
)

EXEC_SQL_UPDATE_PATTERN = re.compile(
    rf"^\s*\*UPDATE\s+(\?P<table>[A-Z0-9_+])\b",
    flags=re.IGNORECASE,
)

```

```

END_EXEC_PATTERN = re.compile(
    r"^\\s*END-EXEC\\s*\\.?",
    flags=re.IGNORECASE,
)

EVALUATE_SQLCODE_PATTERN = re.compile(
    r"^\\s*EVALUATE\\s+SQLCODE\\b",
    flags=re.IGNORECASE,
)

WHEN_ZERO_PATTERN = re.compile(
    r"^\\s*WHEN\\s+0\\b",
    flags=re.IGNORECASE,
)

WHEN_OTHER_PATTERN = re.compile(
    r"^\\s*WHEN\\s+OTHER\\b",
    flags=re.IGNORECASE,
)

CONTINUE_PATTERN = re.compile(
    r"^\\s*CONTINUE\\s*\\.?",
    flags=re.IGNORECASE,
)

END_EVALUATE_PATTERN = re.compile(
    r"^\\s*END-EVALUATE\\s*\\.?",
    flags=re.IGNORECASE,
)

DCL_HOST_GROUP_PATTERN = re.compile(
    r"^\\bDCL[A-Z0-9]+\\b",
    flags=re.IGNORECASE,
)

PERFORM_UPDATE_PARAGRAPH_PATTERN = re.compile(
    r"^\\s*PERFORM\\s+1100-UPDATE-[A-Z0-9-]+\\s*\\.?",
    flags=re.IGNORECASE,
)

PERFORM_READ_FLAT_FILE_PATTERN = re.compile(
    r"^\\s*PERFORM\\s+READ-FLAT-FILE\\s*\\.?",
    flags=re.IGNORECASE,
)

PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^[A-Z0-9][A-Z0-9-]*\\.\\$",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\\update_char_literal_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

# Regex patterns for CHAR-literal quoting.
#
# Regex only. No business rules, conversion rules, file paths, program names,
# DB2 table names, DCLGEN names, copybook names, or host variables.

EXEC_SQL_START_PATTERN = re.compile(
    r"^EXEC\\s+SQL\\b",
    flags=re.IGNORECASE,
)

END_EXEC_PATTERN = re.compile(
    r"^END-EXEC\\b",
    flags=re.IGNORECASE,
)

```

```

# One-line: MOVE <digits> TO <host> OF <DCLGROUP>
# Captures an UNQUOTED numeric literal moved to a DCLGEN host reference.
MOVE_NUMLIT_TO_HOST_OF_GROUP_PATTERN = re.compile(
    r"^MOVE\s+(?P<literal>[0-9]+)\s+TO\s+"
    r"(?P<host>[A-Z][A-Z0-9-]*)\s+OF\s+"
    r"(?P<group>DCL[A-Z0-9-]+\.)\.$",
    flags=re.IGNORECASE,
)

# Wrapped continuation line: TO <host> OF <DCLGROUP>
# (the MOVE <digits> verb is on the previous physical line).
TO_HOST_OF_GROUP_PATTERN = re.compile(
    r"^TO\s+(?P<host>[A-Z][A-Z0-9-]*)\s+OF\s+"
    r"(?P<group>DCL[A-Z0-9-]+\.)\.$",
    flags=re.IGNORECASE,
)

# A bare "MOVE <digits>" line (first half of a wrapped MOVE).
MOVE_NUMLIT_ONLY_PATTERN = re.compile(
    r"^MOVE\s+(?P<literal>[0-9]+)\.?\.$",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\update_date_host_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

# Regex patterns for DB2 DATE-host raw-move detection.
#
# Regex only. No business rules, program names, record names, DB2 tables,
# DCLGEN names, copybook names, or host variables.

EXEC_SQL_START_PATTERN = re.compile(
    r"^EXEC\s+SQL\b",
    flags=re.IGNORECASE,
)

END_EXEC_PATTERN = re.compile(
    r"^END-EXEC\b",
    flags=re.IGNORECASE,
)

# One-line: MOVE <source> TO <host> OF <DCLGROUP>
MOVE_TO_HOST_OF_GROUP_PATTERN = re.compile(
    r"^MOVE\s+(?P<source>.+?)\s+TO\s+"
    r"(?P<host>[A-Z][A-Z0-9-]*)\s+OF\s+"
    r"(?P<group>DCL[A-Z0-9-]+\.)\.$",
    flags=re.IGNORECASE,
)

# Wrapped: "MOVE <source>" (first physical line of a split move).
MOVE_SOURCE_ONLY_PATTERN = re.compile(
    r"^MOVE\s+(?P<source>[A-Z0-9][A-Z0-9-]*)\.\.?\.$",
    flags=re.IGNORECASE,
)

# Wrapped continuation: "TO <host> OF <DCLGROUP>".
TO_HOST_OF_GROUP_PATTERN = re.compile(
    r"^TO\s+(?P<host>[A-Z][A-Z0-9-]*)\s+OF\s+"
    r"(?P<group>DCL[A-Z0-9-]+\.)\.$",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\update_final_cleanup_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

```

```

import re

# Regex patterns for final update-only COBOL cleanup.
#
# Regex only.
# No business rules, conversion rules, file paths, program names, DB2 table
# names, DCLGEN names, copybook names, or host variables.

CONVERTED_MODIFY_COMMENT_PATTERN = re.compile(
    r"DB2:\s*Converted\s+MODIFY\s+for\s+[A-Z0-9-]+\.\?",
    flags=re.IGNORECASE,
)

UPDATE_PARAGRAPH_PATTERN = re.compile(
    r"^1100-UPDATE-[A-Z0-9-]+\.$",
    flags=re.IGNORECASE,
)

PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^[A-Z0-9][A-Z0-9-]*\.$",
    flags=re.IGNORECASE,
)

EXEC_SQL_PATTERN = re.compile(
    r"^EXEC\s+SQL\b",
    flags=re.IGNORECASE,
)

END_EXEC_PATTERN = re.compile(
    r"^END-EXEC\.$",
    flags=re.IGNORECASE,
)

EVALUATE_SQLCODE_PATTERN = re.compile(
    r"^EVALUATE\s+SQLCODE\b",
    flags=re.IGNORECASE,
)

END_EVALUATE_PATTERN = re.compile(
    r"^END-EVALUATE\.$",
    flags=re.IGNORECASE,
)

WHEN_PATTERN = re.compile(
    r"^WHEN\b",
    flags=re.IGNORECASE,
)

PROCESS_COMMIT_IF_PATTERN = re.compile(
    r"^IF\s+[A-Z0-9-]+\s*>\s*[0-9]+\.\?",
    flags=re.IGNORECASE,
)

PERFORM_PATTERN = re.compile(
    r"^PERFORM\b",
    flags=re.IGNORECASE,
)

STOP_RUN_PATTERN = re.compile(
    r"^STOP\s+RUN\.$",
    flags=re.IGNORECASE,
)

# Existing non-SQL DCLGROUP.FIELD detector (preserved, backward-compatible).
NON_SQL_DCL_DOT_REFERENCE_PATTERN = re.compile(
    r"(?!:)\b(?:P<group>DCL[A-Z0-9-]+)\.(?:P<field>[A-Z][A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)

MOVE_SPACES_TO_BARE_RECORD_PATTERN = re.compile(
    r"^MOVE\s+SPACES?\s+TO\s+(?:P<record>[A-Z][A-Z0-9-]*)\.\?",
    flags=re.IGNORECASE,
)

INITIALIZE_DCL_PATTERN = re.compile(

```



```

        r"^[^INITIALIZE\s+(?P<group>DCL[A-Z0-9-]+\.)\.",
        flags=re.IGNORECASE,
    )

DCL_REFERENCE_PATTERN = re.compile(
    r"^\bDCL[A-Z0-9-]+\b",
    flags=re.IGNORECASE,
)

# -----
# ADDED: robust generic handling for non-SQL "MOVE ... TO DCLGROUP.FIELD".
#
# These patterns fix the remaining case where a non-SQL COBOL MOVE still uses
# DCLGEN dot notation (DCLGROUP.FIELD) instead of COBOL "FIELD OF DCLGROUP".
# They deliberately do NOT require a trailing \b, so long DCLGEN host names
# with embedded digit-runs (e.g. DA-INFSDGD-479BFAR) are captured in full.
# SQL host variables ("DCLGROUP.FIELD") are never matched by these because
# the cleanup pass skips lines inside EXEC SQL ... END-EXEC blocks.
# -----

# One-line: MOVE <source> TO DCLGROUP.FIELD
MOVE_TO_DCL_DOT_REFERENCE_PATTERN = re.compile(
    r"^[^MOVE\s+(?P<source>.+?)\s+TO\s+"
    r"^(?P<group>DCL[A-Z0-9-]+\.)\.(?P<field>[A-Z][A-Z0-9-]*)\.",
    flags=re.IGNORECASE,
)

# Wrapped continuation line: TO DCLGROUP.FIELD
# (the MOVE verb was emitted on the previous physical line by fixed-format
# wrapping).
TO_DCL_DOT_REFERENCE_PATTERN = re.compile(
    r"^[^TO\s+(?P<group>DCL[A-Z0-9-]+\.)\.(?P<field>[A-Z][A-Z0-9-]*)\.",
    flags=re.IGNORECASE,
)

# Hardened in-line fallback detector (no trailing \b) for any remaining
# non-SQL DCLGROUP.FIELD occurrence.
NON_SQL_DCL_DOT_REFERENCE_PATTERN = re.compile(
    r"^(?<!\s+)\b(?P<group>DCL[A-Z0-9-]+\.)\.(?P<field>[A-Z][A-Z0-9-]*)",
    flags=re.IGNORECASE,
)

```

```

=====
FILE: patterns\update_final_feedback_patterns.py
ENCODING: utf-8
=====

```

```

"""
Update final feedback regex patterns.

This module contains regex patterns only.

No business rules.
No service logic.
No hardcoded program names, record names, DB2 table names, DB2 column names,
DCLGEN names, or host variable names.
"""

```

```

from __future__ import annotations

```

```

import re

```

```

EXEC_SQL_START_PATTERN = re.compile(
    r"^[^EXEC\s+SQL\b",
    flags=re.IGNORECASE,
)

```

```

EXEC_SQL_END_PATTERN = re.compile(
    r"^[^END-EXEC\.",
    flags=re.IGNORECASE,
)

```

```

UPDATE_STATEMENT_PATTERN = re.compile(
    r"^[^UPDATE\s+(?P<table>[A-Z0-9_]+\b",

```

```

        flags=re.IGNORECASE,
    )

FROM_STATEMENT_PATTERN = re.compile(
    r"^FROM\s+(?P<table>[A-Z0-9_]+\)\b",
    flags=re.IGNORECASE,
)

SET_KEYWORD_PATTERN = re.compile(
    r"^SET$",
    flags=re.IGNORECASE,
)

WHERE_KEYWORD_PATTERN = re.compile(
    r"^WHERE$",
    flags=re.IGNORECASE,
)

SQL_SET_ASSIGNMENT_PATTERN = re.compile(
    r"^(?P<column>[A-Z][A-Z0-9_]+\s*=\s*:"
    r"(?P<group>DCL[A-Z0-9_]+\)\."
    r"(?P<host>[A-Z][A-Z0-9-]*)"
    r"(?P<comma>,)?$",
    flags=re.IGNORECASE,
)

NON_SQL_DCL_DOT_REFERENCE_PATTERN = re.compile(
    r"\b(?P<group>DCL[A-Z0-9_]+\)\."
    r"(?P<host>[A-Z][A-Z0-9-]*)\b",
    flags=re.IGNORECASE,
)

DCLGEN_OF_REFERENCE_PATTERN = re.compile(
    r"\b(?P<host>[A-Z][A-Z0-9-]*)\s+OF\s+"
    r"(?P<group>DCL[A-Z0-9_]+\)\b",
    flags=re.IGNORECASE,
)

MOVE_TO_DCL_OF_REFERENCE_PATTERN = re.compile(
    r"^MOVE\s+(?P<source>.+?)\s+TO\s+"
    r"(?P<host>[A-Z][A-Z0-9-]*)\s+OF\s+"
    r"(?P<group>DCL[A-Z0-9_]+\)\.?$",
    flags=re.IGNORECASE,
)

MOVE_TO_BARE_RECORD_PATTERN = re.compile(
    r"^MOVE\s+(?P<source>SPACES?|SPACE)\s+TO\s+"
    r"(?P<target>[A-Z][A-Z0-9-]*)\.\.?$",
    flags=re.IGNORECASE,
)

INITIALIZE_DCL_PATTERN = re.compile(
    r"^INITIALIZE\s+(?P<group>DCL[A-Z0-9_]+\)\.?$",
    flags=re.IGNORECASE,
)

SQL_LOCATION_DB_OPERATION_PATTERN = re.compile(
    r"^MOVE\s+'(?P<operation>SELECT|UPDATE|INSERT|DELETE)-"
    r"(?P<label>[A-Z0-9-]+\)\s+TO\s+SQL-LOCATION\.\.?$",
    flags=re.IGNORECASE,
)

CONVERTED_DB2_OPERATION_COMMENT_PATTERN = re.compile(
    r"^(?P<prefix>\?*s*DB2:\s+Converted\s+"
    r"(?P<operation>OBTAIN\s+CALC|MODIFY|STORE|ERASE|FIND|OBTAIN)"
    r"\s+for\s+)"
    r"(?P<label>[A-Z0-9-]+\)"
    r"(?P<suffix>\.\.?s*)$",
    flags=re.IGNORECASE,
)

```

```
=====
FILE: patterns\update_main_flow_patterns.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
import re
```

```
"""
```

```
Regex patterns for update main-flow postprocess.
```

```
This file contains regex patterns only.
```

```
No conversion logic, business rules, file paths, program names, DB2 table names,  
DCLGEN names, copybook names, or host variables belong here.
```

```
"""
```

```
OPEN_INPUT_LINE_PATTERN = re.compile(  
    r"\bOPEN\s+INPUT\s+(?P<file>[A-Z0-9-]+\s+)\b",  
    flags=re.IGNORECASE,  
)
```

```
STOP_RUN_LINE_PATTERN = re.compile(  
    r"\bSTOP\s+RUN\b\.\?",  
    flags=re.IGNORECASE,  
)
```

```
PROCEDURE_DIVISION_LINE_PATTERN = re.compile(  
    r"^PROCEDURE\s+DIVISION\b",  
    flags=re.IGNORECASE,  
)
```

```
DIVISION_LINE_PATTERN = re.compile(  
    r"^[A-Z][A-Z0-9-]*\s+DIVISION\b",  
    flags=re.IGNORECASE,  
)
```

```
SECTION_LINE_PATTERN = re.compile(  
    r"^[A-Z][A-Z0-9-]*\s+SECTION\b",  
    flags=re.IGNORECASE,  
)
```

```
PARAGRAPH_HEADER_LINE_PATTERN = re.compile(  
    r"^[A-Z0-9][A-Z0-9-]*\.\$",  
    flags=re.IGNORECASE,  
)
```

```
PERFORM_UNTIL_LINE_PATTERN = re.compile(  
    r"\bPERFORM\s+(?P<paragraph>[A-Z0-9][A-Z0-9-]*)\s+UNTIL\b",  
    flags=re.IGNORECASE,  
)
```

```
PERFORM_READ_FLAT_FILE_LINE_PATTERN = re.compile(  
    r"^PERFORM\s+READ-FLAT-FILE\.\?$",  
    flags=re.IGNORECASE,  
)
```

```
EXEC_SQL_LINE_PATTERN = re.compile(  
    r"\bEXEC\s+SQL\b",  
    flags=re.IGNORECASE,  
)
```

```
MOVE_LINE_PATTERN = re.compile(  
    r"^MOVE\b",  
    flags=re.IGNORECASE,  
)
```

```
PERFORM_LINE_PATTERN = re.compile(  
    r"^PERFORM\b",  
    flags=re.IGNORECASE,  
)
```

```
=====
FILE: patterns\update_program_postprocess_patterns.py
ENCODING: utf-8
=====
```

```
"""
```

```
Regex patterns for update-program postprocess conversion.
```

```
This module contains regex patterns only.
```

```
No conversion logic, business decisions, or hardcoded program-specific values  
belong here.
```

```
"""
```

```
from __future__ import annotations
```

```
import re
```

```
WORKING_STORAGE_SECTION_PATTERN = re.compile(  
    r"^\s*(?:\d{6}\s*)?WORKING-STORAGE\section\s*\.\s*(?:\d{8})?\s*$",  
    flags=re.IGNORECASE | re.MULTILINE,  
)
```

```
LINKAGE_SECTION_PATTERN = re.compile(  
    r"^\s*(?:\d{6}\s*)?LINKAGE\section\s*\.\s*(?:\d{8})?\s*$",  
    flags=re.IGNORECASE | re.MULTILINE,  
)
```

```
PROCEDURE_DIVISION_PATTERN = re.compile(  
    r"^\s*(?:\d{6}\s*)?PROCEDURE\section\s*\.\s*(?:\d{8})?\s*$",  
    flags=re.IGNORECASE | re.MULTILINE,  
)
```

```
END_PROGRAM_PATTERN = re.compile(  
    r"^\s*(?:\d{6}\s*)?END\section\s*[A-Z0-9_-]+\s*\.\s*(?:\d{8})?\s*$",  
    flags=re.IGNORECASE | re.MULTILINE,  
)
```

```
READ_FLAT_FILE_HEADER_PATTERN = re.compile(  
    r"^\s*(?:\d{6}\s*)?READ-FLAT-FILE\section\s*\.\s*(?:\d{8})?\s*$",  
    flags=re.IGNORECASE,  
)
```

```
PARAGRAPH_HEADER_PATTERN = re.compile(  
    r"^\s*(?:\d{6}\s*)?[A-Z0-9][A-Z0-9-]*\s*\.\s*(?:\d{8})?\s*$",  
    flags=re.IGNORECASE,  
)
```

```
COPY_INCLUDE_TOKEN_PATTERN_TEMPLATE = r"\bCOPY\s+{include_name}\s*\."
```

```
EXEC_SQL_INCLUDE_TOKEN_PATTERN_TEMPLATE = (  
    r"\bEXEC\s+SQL\s+INCLUDE\s+{include_name}\s+END-EXEC\s*\."
```

```
MALFORMED_SQLERROR_END_EVALUATE_PATTERN = re.compile(  
    r"\bPERFORM\s+SQLERROR\s*\.\s*END-EVALUATE\b",  
    flags=re.IGNORECASE,  
)
```

```
OLD_EOF_EQUALS_Y_PATTERN = re.compile(  
    r"\b(?:P<switch>[A-Z][A-Z0-9-]*)\s*=\s*['\"]Y['\"]",  
    flags=re.IGNORECASE,  
)
```

```
MOVE_Y_TO_EOF_PATTERN = re.compile(  
    r"\bMOVE\s+['\"]Y['\"]\s+TO\s+(?:P<switch>[A-Z][A-Z0-9-]*)\b",  
    flags=re.IGNORECASE,  
)
```

```
MOVE_N_TO_EOF_PATTERN = re.compile(  
    r"\bMOVE\s+['\"]N['\"]\s+TO\s+(?:P<switch>[A-Z][A-Z0-9-]*)\b",  
    flags=re.IGNORECASE,  
)
```

```
OPEN_INPUT_PATTERN = re.compile(  
    r"\bOPEN\s+INPUT\s+(?:P<file>[A-Z0-9-]+)\b",
```

```

        flags=re.IGNORECASE,
    )

PERFORM_UNTIL_EOF_PATTERN = re.compile(
    r"\bPERFORM\s+(?P<paragraph>[A-Z0-9-]+\s+UNTIL\s+"
    r"(?P<switch>[A-Z0-9-]+\s*=\s*['\"]Y['\"])",
    flags=re.IGNORECASE,
)

OLD_RESTART_RECORD_REFERENCE_PATTERN = re.compile(
    r"\b(?:OF|IN)\s+(?P<record>[A-Z0-9-]+\b",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\update_record_rename_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

# Regex patterns for update-program input-record rename.
#
# Regex only. No business rules, program names, record names, DB2 tables,
# DCLGEN names, copybook names, or host variables.

# A COPY statement for a specific record (name injected via .format with
# re.escape). Matches "COPY <name>" and optional trailing period.
COPY_RECORD_PATTERN_TEMPLATE = r"\bCOPY\s+{name}\b.?"

# A whole-word occurrence of a specific record name (name injected via
# .format with re.escape). Word boundaries prevent partial matches.
WORD_RECORD_PATTERN_TEMPLATE = r"(?![A-Z0-9-]){name}(?![A-Z0-9-])"

=====
FILE: patterns\update_restart_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

WORKING_STORAGE_SECTION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s*)?WORKING-STORAGE\s+SECTION\s*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

LINKAGE_SECTION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s*)?LINKAGE\s+SECTION\s*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

PROCEDURE_DIVISION_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s*)?PROCEDURE\s+DIVISION\b.*?\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

END_PROGRAM_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s*)?END\s+PROGRAM\s+[A-Z0-9_-]+\s*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE | re.MULTILINE,
)

READ_FLAT_FILE_HEADER_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s*)?READ-FLAT-FILE\s*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE,
)

PARAGRAPH_HEADER_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s*)?[A-Z0-9][A-Z0-9-]*\s*\.\s*(?:\d{8})?\s*$",
    flags=re.IGNORECASE,
)

```

```

)

COPY_INCLUDE_TOKEN_PATTERN_TEMPLATE = r"\bCOPY\s+{include_name}\s*\."

EXEC_SQL_INCLUDE_TOKEN_PATTERN_TEMPLATE = (
    r"\bEXEC\s+SQL\s+INCLUDE\s+{include_name}\s+END-EXEC\s*\."
)

MALFORMED_SQLERROR_END_EVALUATE_PATTERN = re.compile(
    r"\bPERFORM\s+SQLERROR\s*\.\s*END-EVALUATE\b",
    flags=re.IGNORECASE,
)

RESTART_COMMENT_PATTERN = re.compile(
    r"\bRESTART\b|\bRECHERCHE\s+FFRECB\b",
    flags=re.IGNORECASE,
)

OLD_RESTART_IF_PATTERN = re.compile(
    r"\bIF\s+SW-[A-Z0-9-]*REC[A-Z0-9-]*\s*=\s*['\"]Y['\"]",
    flags=re.IGNORECASE,
)

OLD_PREREAD_IF_EOF_PATTERN = re.compile(
    r"\bIF\s+NOT\s+(?P<switch>[A-Z][A-Z0-9-]*)\s*=\s*['\"]Y['\"]",
    flags=re.IGNORECASE,
)

OLD_PERFORM_UNTIL_EOF_PATTERN = re.compile(
    r"\bPERFORM\s+(?P<paragraph>[A-Z0-9-]*)\s+UNTIL\s+"
    r"(?P<switch>[A-Z0-9-]*)\s*=\s*['\"]Y['\"]",
    flags=re.IGNORECASE,
)

MOVE_TO_RESTART_FAMILY_PATTERN = re.compile(
    r"\bTO\s+[A-Z0-9-]+FF(?:-[A-Z0-9-]+)?(?:\s+OF\s+[A-Z0-9-]+)?\b",
    flags=re.IGNORECASE,
)

FFRECB_REFERENCE_PATTERN = re.compile(
    r"\bFFRECB\b",
    flags=re.IGNORECASE,
)

EXEC_SQL_START_PATTERN = re.compile(
    r"^\s*EXEC\s+SQL\b",
    flags=re.IGNORECASE,
)

END_EXEC_PATTERN = re.compile(
    r"^\s*END-EXEC\s*\.",
    flags=re.IGNORECASE,
)

EVALUATE_SQLCODE_PATTERN = re.compile(
    r"^\s*EVALUATE\s+SQLCODE\b",
    flags=re.IGNORECASE,
)

END_EVALUATE_PATTERN = re.compile(
    r"^\s*END-EVALUATE\s*\.",
    flags=re.IGNORECASE,
)

STOP_RUN_PATTERN = re.compile(
    r"^\s*STOP\s+RUN\s*\.",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\update_restart_skip_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

```

```

import re

# Regex patterns for update restart/control skip cleanup.
# Regex only. No business rules, program names, or hardcoded records.

CONVERTED_FOR_PATTERN = re.compile(
    r"^\\s*\\*?\\s*DB2:\\s*Converted\\s+"
    r"((?P<operation>OBTAIN\\s+CALC|STORE|MODIFY|ERASE|DELETE|UPDATE|INSERT) "
    r"\\s+(?:for\\s+)?(?P<record>[A-Z0-9-]+)\\.?\\s*$",
    flags=re.IGNORECASE,
)

SKIPPED_FOR_PATTERN = re.compile(
    r"^\\s*\\*?\\s*DB2:\\s*"
    r"((?P<operation>SELECT|INSERT|UPDATE|DELETE|STORE|MODIFY|OBTAIN\\s+CALC) "
    r"\\s+conversion\\s+skipped\\s+for\\s+(?P<record>[A-Z0-9-]+)\\.?\\s*$",
    flags=re.IGNORECASE,
)

REMOVED_OBTAIN_CALC_PATTERN = re.compile(
    r"^\\s*\\*?\\s*DB2:\\s*Removed\\s+OBTAIN\\s+CALC\\s+SELECT\\s+for\\s+"
    r"((?P<record>[A-Z0-9-]+)\\.?\\s*$",
    flags=re.IGNORECASE,
)

SQLCODE_IF_PATTERN = re.compile(
    r"^\\s*IF\\s+SQLCODE\\b",
    flags=re.IGNORECASE,
)

END_IF_PATTERN = re.compile(
    r"^\\s*END-IF\\.?\\s*$",
    flags=re.IGNORECASE,
)

CONTINUE_PATTERN = re.compile(
    r"^\\s*CONTINUE\\.?\\s*$",
    flags=re.IGNORECASE,
)

DB2_COMMENT_PATTERN = re.compile(
    r"^\\s*\\*?\\s*DB2:",
    flags=re.IGNORECASE,
)

=====
FILE: patterns\\update_storage_include_patterns.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

"""
Regex patterns for update-program storage/include cleanup.

This file contains regex patterns only.
No business rules, conversion rules, paths, program names, table names,
DCLGEN names, copybook names, or host variables belong here.
"""

PROGRAM_NAME_MOVE_PATTERN = re.compile(
    r"\\bMOVE\\s+[\\"](?P<program>[A-Z0-9-]+)[\\"]\\s+TO\\s+PROGRAM-NAME\\s*\\.?",
    flags=re.IGNORECASE,
)

CS_PROGRAM_01_PATTERN = re.compile(
    r"^((?P<prefix>\\s*)(?:01)\\s+CS-PROGRAM\\s+PIC\\s+X$8$\\s+VALUE\\s+"
    r"((?P<quote>[\\"])(?P<value>[A-Z0-9-]+)(?P=quote)\\s*\\.?\\s*$",
    flags=re.IGNORECASE,
)

DCL_HOST_REFERENCE_PATTERN = re.compile(

```

```

r"(\b(?:[A-Z0-9]+)|\b(?:[A-Z0-9]+)|\b(?:[A-Z0-9]+))\b",
flags=re.IGNORECASE,
)

EXEC_SQL_INCLUDE_PATTERN = re.compile(
r"\bEXEC\SQL\INCLUDE\s+(\b(?:[A-Z0-9]+)|\b(?:[A-Z0-9]+)|\b(?:[A-Z0-9]+))\b",
flags=re.IGNORECASE,
)

LEGACY_77_DECLARATION_PATTERN_TEMPLATE = (
r"\s*(?:\d{6})\s*77\s+{name}\b.*$"
)

WORKING_STORAGE_SECTION_LINE_PATTERN = re.compile(
r"\s*(?:\d{6})\s*WORKING-STORAGE\SECTION\s*\.\s*(?:\d{8})?\s*$",
flags=re.IGNORECASE,
)

LINKAGE_SECTION_LINE_PATTERN = re.compile(
r"\s*(?:\d{6})\s*LINKAGE\SECTION\s*\.\s*(?:\d{8})?\s*$",
flags=re.IGNORECASE,
)

PROCEDURE_DIVISION_LINE_PATTERN = re.compile(
r"\s*(?:\d{6})\s*PROCEDURE\DIVISION\b.*\.\s*(?:\d{8})?\s*$",
flags=re.IGNORECASE,
)

# --- Legacy IDMS abend/checkpoint marker removal (generic, by '##&&' prefix) ---
# These 77-level markers (e.g. STOP01 VALUE '##&&...') are IDMS-era abend
# labels with no DB2 meaning. Detected by the '##&&' literal prefix, never by
# a hardcoded program-specific field name.

# A 77-level header that declares only a PIC X(nn) with NO inline VALUE.
# (Wrapped case: the VALUE lives on the following line.)

# APPEND to rules/update_restart_rules.py
LEGACY_ABEND_MARKER_LITERAL_PREFIX = "##&&"

# APPEND to patterns/update_storage_include_patterns.py
LEGACY_ABEND_MARKER_77_ONELINE_PATTERN = re.compile(
r"\s*(?:\d{6})\s*77\s+[A-Z0-9-]+\s+PIC\s+X\d+\s+"
r"VALUE\s+'##&&[^\']*'\s*\.\s*(?:\d{8})?\s*$",
flags=re.IGNORECASE,
)

LEGACY_ABEND_MARKER_VALUE_LINE_PATTERN = re.compile(
r"\s*(?:\d{6})\s*VALUE\s+'##&&[^\']*'\s*\.\s*(?:\d{8})?\s*$",
flags=re.IGNORECASE,
)

LEGACY_ABEND_MARKER_77_HEADER_NO_VALUE_PATTERN = re.compile(
r"\s*(?:\d{6})\s*77\s+[A-Z0-9-]+\s+PIC\s+X\d+\s*(?:\d{8})?\s*$",
flags=re.IGNORECASE,
)

LEGACY_ABEND_MARKER_77_HEADER_NAMED_PATTERN = re.compile(
r"\s*(?:\d{6})\s*77\s+(\b(?:[A-Z0-9]+)|\b(?:[A-Z0-9]+)|\b(?:[A-Z0-9]+))\s+PIC\s+X\d+\s*(?:\d{8})?\s*$",
flags=re.IGNORECASE,
)

```

```

=====
FILE: patterns/validation_patterns.py
ENCODING: utf-8
=====

```

```

"""
Validation regex patterns.

Validators must import these patterns instead of defining regex internally.
"""

```

```
import re
```

```
TODO_HOST_VARIABLE_PATTERN = re.compile(
```



```

        r":\s*TODO-HOST-VARIABLE",
        flags=re.IGNORECASE,
    )

TODO_DB2_PATTERN = re.compile(
    r"TODO\s+DB2",
    flags=re.IGNORECASE,
)

ERROR_DB2_PATTERN = re.compile(
    r"ERROR\s+DB2\s*:",
    flags=re.IGNORECASE,
)

UNABLE_TO_DECLARE_CURSOR_PATTERN = re.compile(
    r"UNABLE\s+TO\s+DECLARE\s+Cursors",
    flags=re.IGNORECASE,
)

NO_FETCH_HOST_VARIABLES_PATTERN = re.compile(
    r"NO\s+FETCH\s+HOST\s+VARIABLES\s+MAPPED",
    flags=re.IGNORECASE,
)

FORBIDDEN_EXECUTABLE_IDMS_PATTERNS = [
    re.compile(r"^\bBIND\b", flags=re.IGNORECASE),
    re.compile(r"^\bREADY\b", flags=re.IGNORECASE),
    re.compile(r"^\bOBTAIN\b", flags=re.IGNORECASE),
    re.compile(r"^\bFIND\s+CURRENT\b", flags=re.IGNORECASE),
    re.compile(r"^\bFIND\s+FIRST\b", flags=re.IGNORECASE),
    re.compile(r"^\bSTORE\b", flags=re.IGNORECASE),
    re.compile(r"^\bMODIFY\b", flags=re.IGNORECASE),
    re.compile(r"^\bERASE\b", flags=re.IGNORECASE),
    re.compile(r"^\bCONNECT\b", flags=re.IGNORECASE),
    re.compile(r"^\bDISCONNECT\b", flags=re.IGNORECASE),
    re.compile(
        r"^\bPERFORM\s+[A-Z0-9-]*IDMS-STATUS\b",
        flags=re.IGNORECASE,
    ),
    re.compile(
        r"^\bPERFORM\s+[A-Z0-9-]*IDMS-ABORT\b",
        flags=re.IGNORECASE,
    ),
    re.compile(
        r"^\bUSAGE-MODE\s+IS\s+UPDATE\b",
        flags=re.IGNORECASE,
    ),
    re.compile(
        r"^\bUSAGE-MODE\s+IS\s+RETRIEVAL\b",
        flags=re.IGNORECASE,
    ),
]

FORBIDDEN_IDMS_DECLARATIVE_PATTERNS = [
    re.compile(
        r"^\bIDMS-CONTROL\s+SECTION\b",
        flags=re.IGNORECASE,
    ),
    re.compile(
        r"^\bPROTOCOL\b",
        flags=re.IGNORECASE,
    ),
    re.compile(
        r"^\bIDMS-RECORDS\s+WITHIN\s+WORKING-STORAGE\s+SECTION\b",
        flags=re.IGNORECASE,
    ),
    re.compile(
        r"^\bSCHEMA\s+SECTION\b",
        flags=re.IGNORECASE,
    ),
]

```

```

re.compile(
    r"^DB[A-Z0-9-]+\s+WITHIN\s+[A-Z0-9-]+\b",
    flags=re.IGNORECASE,
),
re.compile(
    r"^COPY\s+IDMS\b",
    flags=re.IGNORECASE,
),
re.compile(
    r"^COPY\s+IDMS\s+IDMS-STATUS\b",
    flags=re.IGNORECASE,
),
re.compile(
    r"^COPY\s+IDMS\s+SUBSCHEMA-BINDS\b",
    flags=re.IGNORECASE,
),
]

```

```

=====
FILE: patterns/validation_rules.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: pyproject.toml
ENCODING: utf-8
=====

```

```

[project]
name = "idms-db2-code-conversion"
version = "0.1.0"
description = "IDMS COBOL to DB2 COBOL Phase 2 converter"
readme = "README.md"
requires-python = ">=3.14"
dependencies = [
    "openpyxl>=3.1.5",
    "pdfplumber>=0.11.10",
    "pydantic>=2.13.4",
    "pydantic-settings>=2.15.0",
    "streamlit>=1.61.1",
]

```

```

[tool.setuptools]
package-dir = {"" = "src"}

```

```

[tool.setuptools.packages.find]
where = ["src"]

```

```

[dependency-groups]
dev = [
    "pytest>=9.1.1",
]

```

```

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src", "."]
addopts = "-q --tb=short"

```

```

=====
FILE: README.md
ENCODING: utf-8
=====

```

IDMS DB2 Phase 2 Converter

Standalone Phase 2 project for converting IDMS COBOL programs into DB2 embedded SQL COBOL.

Inputs

- Sheet Mapping Excel or CSV
- DCLGEN text file or multiple DCLGEN files

- Copybook text file
- Optional Copybook PDF
- IDMS COBOL source code

Output

- Converted DB2 COBOL code
- Validation messages
- Metadata overview
- Record summary
- Column summary
- Set / relationship summary
- Sheet Mapping preview

Clean Architecture Rule

Hardcoded names, aliases, regex patterns, and business rules must not live inside parsers or services.

Use:

- `catalogs/` for column names, aliases, SQL type lists, labels, and static catalog values
- `patterns/` for regex patterns
- `rules/` for business and conversion rules
- `src/idms\_db2\_phase2/parsers/` for parsing logic only
- `src/idms\_db2\_phase2/services/`, `generators/`, `resolvers/`, `transformers/`, and `validators/` for logic only

Run

From the project root:

```
```bash
set PYTHONPATH=src
python -m streamlit run src/idms_db2_phase2/app.py --server.port 8502
```

```
$env:PYTHONPATH = "src"
python src\idms_db2_phase2\testing\run_update.py
```

```
python src\idms_db2_phase2\testing\run_retrieval.py
python src\idms_db2_phase2\testing\run_update.py
```

```
python src\idms_db2_phase2\testing\batch_execution.py --mode retrieval
python src\idms_db2_phase2\testing\batch_execution.py --no-review
python src\idms_db2_phase2\testing\batch_execution.py --quiet
```

```
python -m pip install "pytest>=9.1.1"
$env:PYTHONPATH = "src;."
python -m pytest tests -q
```

check

```
=====
FILE: rules__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: rules\authority_rules.py
ENCODING: utf-8
=====
```

```
"""
Authority rules.
```

```
These are the central project authority rules confirmed by the existing
conversion service comments.
"""
```

```
INPUT_AUTHORITY_RULES = [
```

```

"Sheet Mapping is the authority for DB2 record/table names.",
"Sheet Mapping is the authority for DB2 column names.",
"DCLGEN is the authority for COBOL host variable names and PIC clauses.",
"Original COBOL is the authority for business flow.",
"Final output is resequenced in manual-style COBOL format.",
]

```

```

NO_HARDCODE_RULES = [
 "Do not hardcode business records in services.",
 "Do not hardcode DB2 tables in services.",
 "Do not hardcode DB2 columns in services.",
 "Do not hardcode DCLGEN names in services.",
 "Do not hardcode host variables in services.",
 "Do not hardcode parser column aliases inside parser classes.",
 "Do not hardcode regex patterns inside parser or service classes.",
]

```

```

=====
FILE: rules\batch_console_rules.py
ENCODING: utf-8
=====

```

```

LOCATION: rules/batch_console_rules.py
ACTION: CREATE NEW FILE

```

```

"""Batch execution console presentation rules.

```

```

Constants only. No regex, no runtime logic, no program / record / table
names.

```

```

The COBOL team watches this output in a terminal, not in a log file. It
therefore has to be short, aligned, and unambiguous about pass or fail.
Everything it can print is declared here so wording and width are changed
in one place.
"""

```

```

from __future__ import annotations

```

```

=====
Frame
=====
CONSOLE_WIDTH = 80
RULE_HEAVY = "=" * CONSOLE_WIDTH
RULE_LIGHT = "-" * (CONSOLE_WIDTH - 2)
INDENT = " "

```

```

TITLE = "IDMS -> DB2 BATCH EXECUTION"
STARTED_TEMPLATE = "Started {stamp}"
FINISHED_TEMPLATE = "Finished {stamp}"
TIMESTAMP_FORMAT = "%d-%m-%Y %H:%M:%S"

```

```

SECTION_INPUTS = "INPUTS"
SECTION_EXECUTION = "EXECUTION"
SECTION_METADATA = "METADATA"
SECTION_SUMMARY = "SUMMARY"
SECTION_ARTEFACTS = "ARTEFACTS"

```

```

=====
Progress bar
=====
BAR_WIDTH = 20

```

```

Block glyphs read better, but a legacy console code page cannot render
them. BatchConsole falls back to the ASCII pair automatically.
BAR_FILLED_UNICODE = "\u2588"
BAR_EMPTY_UNICODE = "\u2591"
BAR_FILLED_ASCII = "#"
BAR_EMPTY_ASCII = "-"

```

```

BAR_OPEN = "["
BAR_CLOSE = "]"

```

```

STEP_LABEL_WIDTH = 24
STEP_COUNTER_TEMPLATE = "[{index}/{total}]"

```

```

PULSE_INTERVAL_SECONDS = 0.12
PULSE_SEGMENT = 5

=====
Status
=====
STATUS_RUNNING = "RUNNING"
STATUS_OK = "OK"
STATUS_ACCEPTED = "ACCEPTED"
STATUS_REJECTED = "REJECTED"
STATUS_FAILED = "FAILED"
STATUS_SKIPPED = "SKIPPED"
STATUS_NOTHING = "NOTHING"

STATUS_WIDTH = 9

PASSING_STATUSES = (STATUS_OK, STATUS_ACCEPTED, STATUS_SKIPPED, STATUS_NOTHING)
FAILING_STATUSES = (STATUS_REJECTED, STATUS_FAILED)

Exit code -> status, for the code review runners.
0 accepted, 1 rejected, 2 error or nothing to do.
REVIEW_EXIT_STATUS = {
 0: STATUS_ACCEPTED,
 1: STATUS_REJECTED,
 2: STATUS_NOTHING,
}
CONVERT_EXIT_STATUS = {
 0: STATUS_OK,
}

=====
Steps
#
(key, label, module path, is_review)
Module paths are relative to the project root and are launched as
subprocesses, so a change to a runner's internals cannot break this
orchestrator.
=====
STEP_CONVERT_RETRIEVAL = "convert_retrieval"
STEP_CONVERT_UPDATE = "convert_update"
STEP_REVIEW_RETRIEVAL = "review_retrieval"
STEP_REVIEW_UPDATE = "review_update"

STEP_DEFINITIONS = (
 (
 STEP_CONVERT_RETRIEVAL,
 "Convert retrieval",
 "src/idms_db2_phase2/testing/run_retrieval.py",
 False,
),
 (
 STEP_CONVERT_UPDATE,
 "Convert update",
 "src/idms_db2_phase2/testing/run_update.py",
 False,
),
 (
 STEP_REVIEW_RETRIEVAL,
 "Review retrieval",
 "code_review/runner/review_retrieval.py",
 True,
),
 (
 STEP_REVIEW_UPDATE,
 "Review update",
 "code_review/runner/review_update.py",
 True,
),
)

MODE_RETRIEVAL = "retrieval"
MODE_UPDATE = "update"
MODE_ALL = "all"
MODES = (MODE_ALL, MODE_RETRIEVAL, MODE_UPDATE)

REVIEW_QUIET_ARG = "--quiet"

```

```

=====
Input labels
=====
LABEL_WIDTH = 22

LABEL_MAPPING_SHEET = "Mapping Sheet"
LABEL_DCLGEN = "DCLGen"
LABEL_COPYBOOK = "Copybook"
LABEL_RETRIEVAL_PROGRAMS = "Retrieval programs"
LABEL_UPDATE_PROGRAMS = "Update programs"

LABEL_MAPPING_ROWS = "Sheet Mapping rows"
LABEL_DCLGEN_COLUMNS = "DCLGEN columns"
LABEL_COPYBOOK_FIELDS = "Copybook fields"

LABEL_OUTPUT_FOLDER = "Output"
LABEL_REPORT_FOLDER = "Reports"
LABEL_LOG_FOLDER = "Logs"

COUNT_FILE_SINGULAR = "{count} file"
COUNT_FILE_PLURAL = "{count} files"
COUNT_NONE = "none"
VALUE_UNKNOWN = "-"

=====
Summary table
=====
SUMMARY_COLUMN_STEP = "Step"
SUMMARY_COLUMN_STATUS = "Status"
SUMMARY_COLUMN_SECONDS = "Seconds"

SUMMARY_HEADER_TEMPLATE = "{step:<28}{status:<12}{seconds:>9}"
SUMMARY_ROW_TEMPLATE = "{step:<28}{status:<12}{seconds:>9}"
SUMMARY_TOTAL_TEMPLATE = "Total {seconds:.1f}s Failures {failures}"

VERDICT_PASSED = "BATCH COMPLETED - all steps passed"
VERDICT_FAILED_SINGULAR = "BATCH REJECTED - {count} step needs attention"
VERDICT_FAILED_PLURAL = "BATCH REJECTED - {count} steps need attention"

=====
Messages
=====
MSG_STEP_SKIPPED = "Skipped by --mode {mode}."
MSG_RUNNER_MISSING = "Runner not found: {path}"
MSG_NO_PROGRAMS = "No programs found for this step."
MSG_DETAIL_HEADER = "Detail for failing steps"
MSG_HINT_REPORTS = (
 "Open the report folder above for the per-criterion breakdown."
)

Exit codes for the batch itself.
EXIT_OK = 0
EXIT_REJECTED = 1
EXIT_ERROR = 2

=====
Colour
#
Disabled automatically when stdout is not a terminal, or when NO_COLOR
is set. Never required for correctness: every status is also a word.
=====
ENABLE_COLOUR = True
NO_COLOUR_ENV = "NO_COLOR"

COLOUR_RESET = "\033[0m"
COLOUR_BOLD = "\033[1m"
COLOUR_DIM = "\033[2m"
COLOUR_GREEN = "\033[32m"
COLOUR_RED = "\033[31m"
COLOUR_YELLOW = "\033[33m"
COLOUR_CYAN = "\033[36m"

STATUS_COLOURS = {
 STATUS_OK: COLOUR_GREEN,
 STATUS_ACCEPTED: COLOUR_GREEN,

```

```

 STATUS_RUNNING: COLOUR_CYAN,
 STATUS_REJECTED: COLOUR_RED,
 STATUS_FAILED: COLOUR_RED,
 STATUS_SKIPPED: COLOUR_DIM,
 STATUS_NOTHING: COLOUR_YELLOW,
}

LOCATION: rules/batch_console_rules.py
ACTION: APPEND at the end of the file

=====
Step execution
=====
A runner that hangs must not hang the batch. Generous, because a large
program folder legitimately takes minutes.
STEP_TIMEOUT_SECONDS = 900

Note rendered beside a completed review step.
REVIEW_NOTE_TEMPLATE = "{reviewed} reviewed, {accepted} accepted, {rejected} rejected"

Fallback when the runner output does not name its report folder.
DEFAULT_REPORT_FOLDER = "code_review/report/output"

LOCATION: rules/batch_console_rules.py
ACTION: APPEND at the end of the file

=====
Failure detail
=====
Log levels worth showing in the console. INFO and DEBUG belong in the
log file, not in a summary a COBOL reviewer reads at a glance.
DETAIL_LOG_LEVELS = ("WARNING", "ERROR", "CRITICAL")

Hard cap so one broken step cannot flood the terminal.
DETAIL_MAX_LINES = 12

MSG_BLOCKING_TEMPLATE = "Blocking checks: {ids}"
MSG_VERDICT_TEMPLATE = "{verdict} {name}"
MSG_NO_DETAIL = "No error detail reported. See the log folder above."

LABEL_MODE = "Mode"

LOCATION: rules/batch_console_rules.py
ACTION: APPEND at the end of the file

=====
Line control
=====
Overwriting an animated line with a shorter one leaves the tail of the
old line in the terminal buffer. Writing spaces then returning is not
enough either: the spaces persist until something overwrites them, and
they end up in any copied transcript. An explicit erase-line is clean.
CARRIAGE_RETURN = "\r"
ERASE_LINE = "\033[2K"
CLEAR_PAD_WIDTH = CONSOLE_WIDTH + 40

=====
FILE: rules\batch_runner_messages.py
ENCODING: utf-8
=====

from __future__ import annotations

Batch runner message templates.
#
All console and diagnostic text lives here so wording stays centralized.
No regex, no runtime logic, no hardcoded program/record/table names.

---- Folder / input validation ----
FOLDER_NOT_FOUND_TEMPLATE = "{label} folder not found: {folder}"
PATH_NOT_FOLDER_TEMPLATE = "{label} path is not a folder: {folder}"
NO_PROGRAM_FILES_TEMPLATE = "No {mode} program files found in: {folder}"
NO_MAPPING_FILES_TEMPLATE = "No mapping sheet files found in: {folder}"
NO_MAPPING_IN_FOLDER_TEMPLATE = "No mapping sheet found in folder: {folder}"
NO_DCLGEN_FILES_TEMPLATE = "No DCLGEN files found in: {folder}"

```

```

UNSUPPORTED_MODE_TEMPLATE = "Unsupported batch mode: {mode}"

Input validation labels.
LABEL_MAPPING_SHEET = "Mapping Sheet"
LABEL_DCLGEN = "DCLgen"
LABEL_COPYBOOK = "Copybook"
LABEL_PROGRAM_TEMPLATE = "{mode} Program"

---- Shared input diagnostics ----
DIAG_START_LOAD_SHARED_INPUTS = "START LOAD SHARED INPUTS"
DIAG_MULTIPLE_MAPPING_TEMPLATE = (
 "Multiple mapping sheet files found. Using first sorted file: {first}. "
 "All found: {names}"
)
DIAG_MAPPING_FOLDER_TEMPLATE = "Mapping Sheet folder: {folder}"
DIAG_MAPPING_SELECTED_TEMPLATE = "Mapping Sheet selected: {path}"
DIAG_MAPPING_ROWS_TEMPLATE = "Mapping Sheet rows: {count}"
DIAG_DCLGEN_FOLDER_TEMPLATE = "DCLGEN folder: {folder}"
DIAG_DCLGEN_FILE_COUNT_TEMPLATE = "DCLGEN file count: {count}"
DIAG_DCLGEN_FILE_TEMPLATE = "DCLGEN {index}: {path}"
DIAG_DCLGEN_TEXT_LEN_TEMPLATE = "DCLGEN {index} text length: {length}"
DIAG_DCLGEN_TOTAL_COLUMNS_TEMPLATE = "DCLGEN total columns: {count}"
DIAG_COPYBOOK_FOLDER_TEMPLATE = "Copybook folder: {folder}"
DIAG_COPYBOOK_FILE_COUNT_TEMPLATE = "Copybook file count: {count}"
DIAG_COPYBOOK_FILE_TEMPLATE = "Copybook {index}: {path}"
DIAG_COPYBOOK_TEXT_LEN_TEMPLATE = "Copybook {index} text length: {length}"
DIAG_COPYBOOK_TOTAL_FIELDS_TEMPLATE = "Copybook total fields: {count}"

Source label used when parsing copybook text.
COPYBOOK_SOURCE_LABEL = "Copybook files"
DCLGEN_SOURCE_LABEL_TEMPLATE = "DCLGEN file {index}"

---- Per-program conversion diagnostics ----
DIAG_SOURCE_FILE_TEMPLATE = "{mode} COBOL source file: {path}"
DIAG_SOURCE_LEN_TEMPLATE = "{mode} COBOL source text length: {length}"

---- Update postprocess diagnostics ----
DIAG_UPDATE_ENHANCEMENT_DONE = "Update postprocess enhancement completed."
DIAG_UPDATE_AUDIT_MOVES = (
 "Update audit moves: inserted TS_UPDATE / USER-ID host moves before "
 "UPDATE SQL."
)
DIAG_UPDATE_SKIPPED_TEMPLATE = (
 "Update postprocess enhancement skipped. Original converted COBOL will "
 "be written. Reason: {reason}"
)

---- Result reporting ----
LOG_CONVERSION_COMPLETED_TEMPLATE = "{mode} conversion completed."
LOG_INPUT_PROGRAM_TEMPLATE = "Input program: {path}"
LOG_OUTPUT_FILE_TEMPLATE = "Output file: {path}"

PRINT_GENERATION_COMPLETED_TEMPLATE = "{mode} DB2 COBOL generation completed."
PRINT_INPUT_PROGRAM_TEMPLATE = "Input program: {path}"
PRINT_OUTPUT_CREATED_TEMPLATE = "Output file created: {path}"

HEADER_VALIDATION_MESSAGES = "Validation Messages"
HEADER_DIAGNOSTICS = "Diagnostics"
HEADER_BATCH_INPUT_SUMMARY = "Batch Input Summary"
NO_VALIDATION_MESSAGES = "No validation messages."
BULLET_TEMPLATE = "- {text}"

---- Batch summary lines ----
SUMMARY_PROJECT_ROOT_TEMPLATE = "Project Root : {value}"
SUMMARY_SRC_DIR_TEMPLATE = "SRC Directory : {value}"
SUMMARY_MODE_TEMPLATE = "Mode : {value}"
SUMMARY_MAPPING_FOLDER_TEMPLATE = "Mapping Folder : {value}"
SUMMARY_PROGRAM_FOLDER_TEMPLATE = "Program Folder : {value}"
SUMMARY_PROGRAM_COUNT_TEMPLATE = "Program Count : {value}"
SUMMARY_OUTPUT_FOLDER_TEMPLATE = "Output Folder : {value}"
SUMMARY_TARGET_PROGRAM_ID_TEMPLATE = "Target PROGRAM-ID: {value}"
PRESERVE_SOURCE_LABEL = "(preserve source)"

```



```
=====
FILE: rules\batch_runner_rules.py
ENCODING: utf-8
=====

from __future__ import annotations

Batch runner rule constants.
#
Constants only. No regex, no runtime logic, no program/record/table names.
Change a value here and it reflects across every batch runner.

BATCH_RUNNER_RULES = [
 "Mapping Sheet is selected from the mapping folder (first sorted file).",
 "DCLGEN and Copybook inputs are loaded once and shared across programs.",
 "Target PROGRAM-ID comes from the environment, then the mode default.",
 "Do not hardcode program names, DB2 tables, columns, or host variables.",
]

Supported batch mode names.
BATCH_MODE_UPDATE = "UPDATE"
BATCH_MODE_RETRIEVAL = "RETRIEVAL"

Environment variable key that overrides the target PROGRAM-ID.
TARGET_PROGRAM_ID_ENV_KEY = "TARGET_PROGRAM_ID"

Converter behavior defaults.
AUTO_FIX_PIC_LENGTH_MISMATCHES_DEFAULT = False

Output file naming.
OUTPUT_TIMESTAMP_FORMAT = "%d-%m-%Y_%H%M%S"
OUTPUT_FILE_EXTENSION = ".cbl"
OUTPUT_FILE_NAME_TEMPLATE = "{stem}_{timestamp}{extension}"

Text encoding used when reading/writing COBOL files.
COBOL_TEXT_ENCODING = "utf-8"

```

```
=====
FILE: rules\business_flow_rules.py
ENCODING: utf-8
=====

"""
Business flow preservation rules.

These constants describe how original COBOL business logic should be preserved.
No regex or runtime logic belongs in this file.
"""

```

```
BUSINESS_FLOW_RULES = [
 "Preserve original COBOL business logic unless replacing IDMS database statements.",
 "Do not rewrite unrelated COBOL statements.",
 "Do not corrupt COBOL headers such as DATE-WRITTEN, AUTHOR, or PROGRAM-ID.",
 "Replace only IDMS database access logic with DB2 embedded SQL logic.",
 "Retain paragraph structure unless a database replacement requires a local block.",
]

```

```
=====
FILE: rules\business_rules.py
ENCODING: utf-8
=====

```

[Empty file]

```
=====
FILE: rules\cobol_cleanup_rules.py
ENCODING: utf-8
=====

```

```
"""
COBOL cleanup rule constants.

This file contains COBOL post-conversion cleanup constant values and message

```

text only. No regex patterns and no runtime logic belong here.  
Rules belong in rules/, patterns in patterns/.  
"""

# Generated Working-Storage declaration for the DB2 status flag.

```
SW_STATUS_D_DECLARATION = (
 " 10 SW-STATUS-D PIC X VALUE 'N'."
)
```

# Generated flag group line used when no WS-STATUS anchor exists.

# NOTE: This literal is emitted into the converted COBOL output, so its value  
# is preserved exactly to avoid changing generated program behavior.

```
WS_DB2_FLAGS_GROUP = " 01 WS-DB2-FEEDBACK-FLAGS."
```

# Statement starters that stop the backward MOVE-block scan.

```
STOP_BACKWARD_SCAN_WORDS = (
 "IF ",
 "ELSE",
 "END-IF",
 "PERFORM ",
 "EVALUATE ",
 "WHEN ",
 "WRITE ",
 "EXEC SQL",
 "END-EXEC",
 "OPEN ",
 "CLOSE ",
 "READ ",
 "STOP ",
 "EXIT",
)
```

# Cursor paragraph number at/above which a fetch is treated as child/nested.

```
CHILD_FETCH_MINIMUM_NUMBER = 800
```

# Look-back / look-ahead window sizes.

```
MOVE_N_LOOKBACK_WINDOW = 3
```

```
OUTPUT_MOVE_SCAN_WINDOW = 60
```

```
INITIALIZE_LOOKBACK_WINDOW = 3
```

# LOCATION: rules/cobol\_cleanup\_rules.py

# ACTION: REPLACE the entire CLEANUP\_MESSAGES = { ... } dictionary with this

```
CLEANUP_MESSAGES = {
 "added_dclgen_include": (
 "Cleanup: added missing DCLGEN include {table}."
),
 "replaced_error_status": (
 "Cleanup: replaced ERROR-STATUS move with SW-STATUS-D flag."
),
 "declared_sw_status_d_ws": (
 "Cleanup: declared SW-STATUS-D working-storage flag."
),
 "declared_sw_status_d_block": (
 "Cleanup: declared SW-STATUS-D in generated flag block."
),
 "child_fetch_early_stop": (
 "Cleanup: added SW-STATUS-D early-stop to child fetch loop."
),
 "moved_initialize": (
 "Cleanup: moved INITIALIZE before output population for {record}."
),
 "converted_db2_date_move": (
 "Cleanup: converted DB2 date move before output write."
),
 "removed_residual_idms_comment": (
 "Cleanup: removed residual IDMS conversion comment noise."
),
 "removed_orphan_continue": (
 "Cleanup: removed redundant CONTINUE after removed IDMS statement."
),
}
```

# LOCATION: rules/update\_sql\_cleanup\_rules.py

# ACTION: APPEND these constants to the existing file (add at the end)

```

--- Category E: manual-standard UPDATE block ---

QUERYNO value appended to generated SQL statements (manual standard).
The manual programmer uses the line/paragraph number; we emit a stable
generated value so DB2 EXPLAIN can identify the statement.
UPDATE_QUERYNO = "442"

When True, generated UPDATE uses EVALUATE SQLCODE (manual standard) instead
of the older IF SQLCODE NOT = 0 form.
USE_EVALUATE_SQLCODE = True

--- Output write placement (appended) -----
#
A guarded output WRITE left inside the per-row paragraph fires once per
fetched child row instead of once per parent row. Manual reference places
it in the parent paragraph after the child cursor CLOSE, wrapped in the
early-stop guard.
ENFORCE_OUTPUT_WRITE_AFTER_CHILD_LOOP = True

FETCH_PARAGRAPH_CURSOR_SEPARATOR = "-FETCH-"
NESTED_INDENT_STEP = " "
WHEN_ZERO_PERFORM_SCAN_LIMIT = 5
OUTPUT_WRITE_MOVE_PASS_LIMIT = 8

EARLY_STOP_FLAG = "SW-STATUS-D"
EARLY_STOP_VALUE = "'Y'"
EARLY_STOP_GUARD_OPEN = "IF NOT {flag} = {value}"
EARLY_STOP_GUARD_CLOSE = "END-IF"

Single-word lines that end in a period but are not paragraph headers.
NON_PARAGRAPH_SINGLE_WORDS = frozenset({
 "CONTINUE", "END-EXEC", "END-EVALUATE", "END-IF", "END-PERFORM",
 "END-READ", "END-SEARCH", "END-STRING", "END-WRITE", "EXIT",
 "GOBACK", "STOP",
})

CLEANUP_MESSAGES["output_write_moved"] = (
 "Cleanup: moved output write block from {source} to {target} after the "
 "child cursor loop."
)

=====
APPENDED: constants for the generated-COBOL safety cleanup passes
#
Consumed by:
composers/cleanup/output_write_placement_cleanup.py
composers/cleanup/paragraph_terminator_cleanup.py
#
Constants only. No regex, no runtime logic, no program / record / table /
cursor / host variable names.
=====

--- Paragraph termination -----
#
A paragraph whose last sentence carries no period runs into the next
paragraph header and the compiler rejects the program. Any pass that
inserts or lifts a block can leave that state behind.
ENFORCE_PARAGRAPH_TERMINATION = True
PARAGRAPH_TERMINATOR = "."

Single-word lines that end in a period but are NOT paragraph headers.
Declared here so the cleanup family has no cross-rules dependency.
NON_PARAGRAPH_SINGLE_WORDS = frozenset({
 "CONTINUE",
 "END-EXEC",
 "END-EVALUATE",
 "END-IF",
 "END-PERFORM",
 "END-READ",
 "END-SEARCH",
 "END-STRING",
 "END-WRITE",
 "EXIT",
 "GOBACK",
 "STOP",
})

```

```

}))

--- Output write placement -----
#
Manual reference shape:
#
PERFORM <close-child>
#
IF NOT SW-STATUS-D = 'Y'
IF <status-field> = '<value>'
PERFORM <write-paragraph>
END-IF
END-IF.
#
A guarded WRITE left inside the per-row paragraph fires once per fetched
child row instead of once per parent row.
ENFORCE_OUTPUT_WRITE_AFTER_CHILD_LOOP = True

FETCH_PARAGRAPH_CURSOR_SEPARATOR = "-FETCH-"
NESTED_INDENT_STEP = " "
WHEN_ZERO_PERFORM_SCAN_LIMIT = 5
OUTPUT_WRITE_MOVE_PASS_LIMIT = 8

EARLY_STOP_FLAG = "SW-STATUS-D"
EARLY_STOP_VALUE = "'Y'"
EARLY_STOP_GUARD_OPEN = "IF NOT {flag} = {value}"
EARLY_STOP_GUARD_CLOSE = "END-IF"

--- Diagnostics -----
CLEANUP_MESSAGES["output_write_moved"] = (
 "Cleanup: moved output write block from {source} to {target} after the "
 "child cursor loop."
)
CLEANUP_MESSAGES["paragraph_terminated"] = (
 "Cleanup: closed unterminated sentence in paragraph {paragraph}."
)

=====
FILE: rules\cobol_statement_rules.py
ENCODING: utf-8
=====

"""
COBOL statement classification rules.

This file contains COBOL statement rule constants only.

Rules belong in rules/, not patterns/.

No regex patterns should be defined here.
No parser, service, transformer, generator, validator, or composer logic
should be placed here.

Use this file for generic COBOL statement classification values that are
shared by formatters, composers, parsers, transformers, or validators.
"""

NON_PARAGRAPH_SINGLE_WORDS = {
 "ACCEPT",
 "ADD",
 "ALTER",
 "CALL",
 "CANCEL",
 "CLOSE",
 "COMMIT",
 "COMPUTE",
 "CONTINUE",
 "DELETE",
 "DISPLAY",
 "DIVIDE",
 "ELSE",
 "END-ADD",
 "END-CALL",
 "END-DELETE",
 "END-DIVIDE",

```

```

"END-EVALUATE",
"END-EXEC",
"END-IF",
"END-MULTIPLY",
"END-PERFORM",
"END-READ",
"END-RETURN",
"END-REWRITE",
"END-SEARCH",
"END-START",
"END-STRING",
"END-SUBTRACT",
"END-UNSTRING",
"END-WRITE",
"EJECT",
"EVALUATE",
"EXEC",
"FETCH",
"EXIT",
"GOBACK",
"IF",
"INITIALIZE",
"INSPECT",
"MOVE",
"MULTIPLY",
"NEXT",
"OPEN",
"PERFORM",
"READ",
"RETURN",
"REWRITE",
"ROLLBACK",
"SEARCH",
"SET",
"SKIP1",
"SKIP2",
"SKIP3",
"SORT",
"SPACE",
"SPACES",
"START",
"STOP",
"STRING",
"SUBTRACT",
"UNSTRING",
"WHEN",
"WRITE",
}

```

```

NON_PARAGRAPH_WORDS = NON_PARAGRAPH_SINGLE_WORDS

```

```

PROCEDURE_VERBS = (
"ACCEPT ",
"ADD ",
"ALTER ",
"CALL ",
"CANCEL ",
"CLOSE ",
"COMMIT",
"COMPUTE ",
"CONTINUE",
"DELETE ",
"DISPLAY ",
"DIVIDE ",
"ELSE",
"END-ADD",
"END-CALL",
"END-DELETE",
"END-DIVIDE",
"END-EVALUATE",
"END-EXEC",
"END-IF",
"END-MULTIPLY",
"END-PERFORM",
"END-READ",
"END-RETURN",

```

```

"END-REWRITE",
"END-SEARCH",
"END-START",
"END-STRING",
"END-SUBTRACT",
"END-UNSTRING",
"END-WRITE",
"EVALUATE ",
"EXEC SQL",
"EXIT",
"FETCH ",
"GOBACK",
"IF ",
"INITIALIZE ",
"INSPECT ",
"MOVE ",
"MULTIPLY ",
"NEXT ",
"OPEN ",
"PERFORM ",
"READ ",
"RETURN ",
"REWRITE ",
"ROLLBACK",
"SEARCH ",
"SET ",
"SORT ",
"START ",
"STOP ",
"STRING ",
"SUBTRACT ",
"UNSTRING ",
"WHEN ",
"WRITE ",
)

SQL_LEVEL_1_KEYWORDS = (
"SELECT ",
"INTO ",
"FROM ",
"WHERE",
"ORDER BY ",
"GROUP BY ",
"HAVING ",
"FETCH ",
"OPEN ",
"CLOSE ",
"COMMIT",
"ROLLBACK",
"INSERT ",
"UPDATE ",
"DELETE ",
"SET ",
"VALUES ",
"FOR READ ONLY",
"QUERYNO ",
)

SQL_LEVEL_2_KEYWORDS = (
"AND ",
"OR ",
)

def is_non_paragraph_word(
 word: str,
) -> bool:
 """
 Return True when the supplied word is a COBOL verb or keyword that should
 not be treated as a paragraph name.
 """
 clean_word = str(word or "").strip().upper()
 return clean_word in NON_PARAGRAPH_WORDS

def starts_with_procedure_verb(

```

```

 statement: str,
) -> bool:
 """
 Return True when the statement starts with a known PROCEDURE DIVISION verb.
 """
 clean_statement = str(statement or "").strip().upper()

 if not clean_statement:
 return False

 return clean_statement.startswith(PROCEDURE_VERBS)

```

```

def starts_with_sql_level_1_keyword(
 statement: str,
) -> bool:
 """
 Return True when the SQL statement starts with a level-1 SQL keyword.
 """
 clean_statement = str(statement or "").strip().upper()

 if not clean_statement:
 return False

 return clean_statement.startswith(SQL_LEVEL_1_KEYWORDS)

```

```

def starts_with_sql_level_2_keyword(
 statement: str,
) -> bool:
 """
 Return True when the SQL statement starts with a level-2 SQL keyword.
 """
 clean_statement = str(statement or "").strip().upper()

 if not clean_statement:
 return False

 return clean_statement.startswith(SQL_LEVEL_2_KEYWORDS)

```

```

=====
FILE: rules\cobol_transformer_rules.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

"""
Rules/constants for COBOL source transformation.

No regex patterns or transformer logic belong here.
"""

```

```

COBOL_TRANSFORMER_RULES = [
 "Preserve original COBOL business flow.",
 "Convert CBL compiler option line to DB2-compatible option.",
 "Replace target PROGRAM-ID only when requested.",
 "Remove residual IDMS declarative/control lines.",
 "Remove or convert residual IDMS executable lines.",
 "Preserve sequence/spacing for one-line DB condition replacements.",
 "Remove orphan IDMS-ABORT paragraph after PERFORM IDMS-ABORT is removed.",
]

```

```

DB2_COMPILER_OPTION_LINE = "CBL ARITH(EXTEND)"

```

```

DEFAULT_SQL_ERROR_PARAGRAPH = "SQL-ERROR"

```

```

GENERATED_LINE_PREFIXES = (
 "* DB2:",
 "EXEC SQL",
 "END-EXEC",
)

```

```

"MOVE '",
"PERFORM ",
"END-IF",
"CONTINUE",
"SELECT",
"INSERT",
"UPDATE",
"DELETE",
"COMMIT",
"ROLLBACK",
)

```

```

=====
FILE: rules\conversion_rules.py
ENCODING: utf-8
=====

```

```

"""
General IDMS to DB2 conversion rules.
"""

```

```

CONVERSION_RULES = [
 "Preserve original COBOL business flow.",
 "Replace IDMS database operations with DB2-compatible SQL logic.",
 "Do not fabricate DB2 SQL when Sheet Mapping metadata is missing.",
 "When mapping metadata is missing, generate a clear DB2 conversion-skipped comment.",
 "When removing executable PROCEDURE DIVISION IDMS code, add CONTINUE.",
]

```

```

MISSING_MAPPING_RULES = [
 "FFRECAB conversion requires Sheet Mapping and DCLGEN metadata.",
 "If Sheet Mapping entry is missing, conversion must be skipped with a clear comment.",
 "Missing Sheet Mapping metadata is an input-data blocker, not a Python logic failure.",
]

```

```

LOCATION: rules/conversion_rules.py
ACTION: REPLACE the Option B constants (find UNMAPPED_RECORD_MARKER_TEMPLATE
and COMMENTED_IDMS_LINE_PREFIX, replace with these)

```

```

--- Option B: unmapped-record handling (keep-and-comment) ---
Marker text is deliberately distinct (DB2-KEEP, not "DB2:") so the
residual-comment cleanup (Category G) never matches and strips it.

```

```

UNMAPPED_RECORD_MARKER_TEMPLATE = (
 "*DB2-KEEP: {record} has no usable DB2 column mapping "
 "- map this IDMS logic manually."
)

```

```

Prefix used to comment out an un-convertible IDMS verb line (kept visible).
COMMENTED_IDMS_LINE_PREFIX = "*DB2-KEEP "

```

```

=====
FILE: rules\counter_declaration_rules.py
ENCODING: utf-8
=====

```

```

LOCATION: rules/counter_declaration_rules.py
ACTION: CREATE NEW FILE

```

```

"""Row counter declaration rules.

```

```

Closes open decision D-2 (processing counters) in favour of AUTOMATE,
per the COBOL team's manual reference program:

```

```

10 WS-NB-BEFF-COUNT PIC 9(7) COMP-3 VALUE ZEROES.
10 WS-NB-OUTPUT-COUNT PIC 9(7) COMP-3 VALUE ZEROES.
...
DISPLAY 'TOTAL BEFF RECORDS : ' WS-NB-BEFF-COUNT
DISPLAY 'TOTAL OUTPUT RECORDS : ' WS-NB-OUTPUT-COUNT

```

Declaration is REFERENCE-DRIVEN, not table-driven: the composer declares exactly the counters the PROCEDURE DIVISION actually increments. It can therefore never declare an unused field, and never miss one.



Constants only. No regex, no runtime logic, no program / record / table / cursor / host variable names.

"""

```
from __future__ import annotations

=====
Master switch
=====
ENFORCE_COUNTER_DECLARATION = True

=====
Declaration layout
=====
The manual reference declares counters as subordinates of the existing
01 work group, at the same level as its other children. The composer
reads that level from the group rather than assuming one.
COUNTER_DECLARATION_TEMPLATE = "{level} {name:<24} {picture}"
COUNTER_PICTURE = "PIC 9(7) COMP-3 VALUE ZEROES."
FALLBACK_CHILD_LEVEL = "10"

=====
End-of-run totals
=====
EMIT_COUNTER_TOTALS = True

COUNTER_DISPLAY_TEMPLATE = "DISPLAY '{label} : ' {name}"
COUNTER_LABEL_TEMPLATE = "TOTAL {token} RECORDS"
COUNTER_LABEL_WIDTH = 20

Totals are emitted immediately before the program terminates.
TOTALS_ANCHOR_STATEMENT = "STOP RUN"
TOTALS_INDENT = " "

=====
Diagnostics
=====
COUNTER_DECLARATION_MESSAGES = {
 "declared": (
 "Counters: declared {name} in {group}."
),
 "totals_added": (
 "Counters: added {count} end-of-run total display line(s)."
),
 "skipped_no_group": (
 "Counters: no WORKING-STORAGE 01 group found; {count} counter "
 "reference(s) left undeclared."
),
 "skipped_no_anchor": (
 "Counters: no {anchor} statement found; end-of-run totals skipped."
),
}
}
LOCATION: rules/counter_declaration_rules.py
ACTION: APPEND at the end of the file

#
Counter naming convention
#
A generated row counter is WS-NB-<token>-COUNT. The token is the
distinctive middle of the DB2 table name, or OUTPUT for the file
counter. Kept here so no composer carries a naming literal.
#
COUNTER_NAME_PREFIX = "WS-NB-"
COUNTER_NAME_SUFFIX = "-COUNT"
COUNTER_OUTPUT_TOKEN = "OUTPUT"

Tokens that identify an existing end-of-run total line.
TOTALS_DISPLAY_VERB = "DISPLAY"
TOTALS_LABEL_TOKEN = "TOTAL"
```

```
=====
FILE: rules\counter_rules.py
ENCODING: utf-8
=====
```

```
LOCATION: rules/counter_rules.py
ACTION: CREATE NEW FILE
"""Row counter and program-name constant rules.
```

```
Closes open decision D-2 (processing counters) in favour of AUTOMATE,
per the COBOL team's manual reference program.
```

```
Constants only. No regex, no runtime logic, no program / record / table /
cursor / host variable names.
"""
```

```
from __future__ import annotations
```

```

Program-name constant

EMIT_PROGRAM_NAME_CONSTANT = True
```

```
PROGRAM_CONSTANT_NAME = "CS-PROGRAM"
PROGRAM_CONSTANT_TEMPLATE = (
 "10 {name:<24} PIC X(8) VALUE '{program_id}'."
)
PROGRAM_NAME_TARGET_FIELD = "PROGRAM-NAME"
PROGRAM_NAME_MOVE_TEMPLATE = "MOVE {constant} TO {target}"
```

```

Row counters

EMIT_ROW_COUNTERS = True
```

```
COUNTER_NAME_TEMPLATE = "WS-NB-{token}-COUNT"
COUNTER_PICTURE = "PIC 9(7) COMP-3 VALUE ZEROES."
COUNTER_DECLARATION_TEMPLATE = "10 {name:<24} {picture}"
```

```
One counter per cursor (rows fetched) plus one for the output file.
OUTPUT_COUNTER_NAME = "WS-NB-OUTPUT-COUNT"
OUTPUT_COUNTER_TOKEN = "OUTPUT"
```

```
COUNTER_ADD_TEMPLATE = "ADD 1 TO {name}"
```

```

End-of-run totals

EMIT_COUNTER_TOTALS = True
```

```
COUNTER_DISPLAY_TEMPLATE = "DISPLAY '{label} : ' {name}"
COUNTER_LABEL_WIDTH = 20
COUNTER_LABEL_FETCH_TEMPLATE = "TOTAL {token} RECORDS"
COUNTER_LABEL_OUTPUT = "TOTAL OUTPUT RECORDS"
```

```
Totals are displayed after the output file is closed and before
STOP RUN, matching the manual reference.
COUNTER_TOTALS_ANCHOR = "CLOSE"
```

```

Diagnostics

```

```
COUNTER_MESSAGES = {
 "declared_counter": (
 "Counters: declared row counter {name}."
),
 "declared_program_constant": (
 "Counters: declared program-name constant {name} = '{program_id}'."
),
 "counter_increment_added": (
 "Counters: added increment for {name} in paragraph {paragraph}."
),
 "counter_totals_added": (
 "Counters: added {count} end-of-run total display line(s)."
),
}
```

```

=====
FILE: rules\cursor_close_guarantee_rules.py
ENCODING: utf-8
=====

LOCATION: rules/cursor_close_guarantee_rules.py
ACTION: CREATE NEW FILE

"""Cursor close-guarantee rules.

Constants only. No regex, no runtime logic, no program / paragraph /
record / table / cursor / host variable names.

WHY THIS EXISTS

`CursorFlowComposer` handles two loop idioms (INLINE and SPAN). When a
program matches neither - as every LRF-expanded program did - it logged
`no_loop_for_cursor` as a WARNING and emitted:

 PERFORM 710-OPEN-DZBFASC1.
 PERFORM 720-FETCH-DZBFASC1. <- primes once, never loops
 ...
 PERFORM 720-FETCH-DZBFASC1. <- orphan repeat fetch

with `730-CLOSE-DZBFASC1` declared but never performed. The cursor is
opened, read twice and leaked. This pass is the last-resort guarantee
that produces the manual reference shape:

 PERFORM 710-OPEN-DZBFASC1.
 PERFORM 720-FETCH-DZBFASC1 UNTIL DZBFASC1-EOC.
 PERFORM 730-CLOSE-DZBFASC1.

and drives the business paragraph from the FETCH paragraph:

 EVALUATE SQLCODE
 WHEN ZERO
 PERFORM BEHADELING THRU BEHADELING-EXIT
"""

from __future__ import annotations

#
Master switches
#
ENFORCE_CURSOR_CLOSE_GUARANTEE = True

Convert a primed-but-unlooped fetch into PERFORM <fetch> UNTIL <eoc>.
SYNTHESISE_MISSING_DRIVING_LOOP = True

Rewrite the FETCH paragraph WHEN ZERO branch to drive the business
paragraph that used to carry the OBTAIN NEXT.
ENFORCE_FETCH_DRIVES_BUSINESS_PARAGRAPH = True

Emit PERFORM <para> THRU <para>-EXIT when the exit paragraph exists.
PREFER_THRU_EXIT_PARAGRAPH = True

#
Cursor paragraph operations
#
OPERATION_OPEN = "OPEN"
OPERATION_FETCH = "FETCH"
OPERATION_CLOSE = "CLOSE"
REQUIRED_OPERATIONS = (OPERATION_OPEN, OPERATION_FETCH, OPERATION_CLOSE)

#
End-of-cursor condition
#
EOC_CONDITION_TEMPLATE = "{cursor}-EOC"

Conditions meaning "the legacy IDMS / raw SQLCODE end test is still here".
LEGACY_EOC_CONDITIONS = (
 "SQLCODE = 100",
 "SQLCODE=100",
 "SQLCODE = +100",

```

```

 "SQLCODE=+100",
 "DB-END-OF-SET",
 "DB-REC-NOT-FOUND",
)

#
Generated statements
#
PERFORM_FETCH_UNTIL_TEMPLATE = "PERFORM {fetch} UNTIL {condition}."
PERFORM_CLOSE_TEMPLATE = "PERFORM {close}."
PERFORM_BUSINESS_TEMPLATE = "PERFORM {paragraph}"
PERFORM_BUSINESS_THRU_TEMPLATE = "PERFORM {paragraph} THRU {through}"
UNTIL_TEMPLATE = "UNTIL {condition}"

EXIT_PARAGRAPH_SUFFIX = "-EXIT"
STATEMENT_TERMINATOR = "."

How far ahead of a PERFORM the UNTIL clause may sit on its own line.
UNTIL_LOOKAHEAD_LIMIT = 3

Single word lines ending in a period that are NOT paragraph headers.
NON_PARAGRAPH_SINGLE_WORDS = frozenset({
 "CONTINUE",
 "END-EXEC",
 "END-EVALUATE",
 "END-IF",
 "END-PERFORM",
 "END-READ",
 "END-SEARCH",
 "END-STRING",
 "END-WRITE",
 "EXIT",
 "GOBACK",
 "STOP",
})

#
Diagnostics
#
CURSOR_CLOSE_GUARANTEE_MESSAGES = {
 "loop_synthesised": (
 "Cursor close guarantee: cursor {cursor} had no driving loop; "
 "emitted PERFORM {fetch} UNTIL {condition} and PERFORM {close}."
),
 "condition_rewritten": (
 "Cursor close guarantee: rewrote the {cursor} loop exit condition "
 "to {condition}."
),
 "close_inserted": (
 "Cursor close guarantee: inserted PERFORM {close} after the "
 "{cursor} driving loop."
),
 "close_present": (
 "Cursor close guarantee: {close} is already performed; no "
 "insertion needed."
),
 "business_paragraph_bound": (
 "Cursor close guarantee: FETCH paragraph {fetch} now performs "
 "business paragraph {paragraph}."
),
 "repeat_fetch_removed": (
 "Cursor close guarantee: removed the orphan repeat PERFORM "
 "{fetch} left by the converted OBTAIN NEXT."
),
 "no_open_perform": (
 "Cursor close guarantee: cursor {cursor} has OPEN/FETCH/CLOSE "
 "paragraphs but is never opened; left untouched for review."
),
 "incomplete_paragraph_set": (
 "Cursor close guarantee: cursor {cursor} does not have a complete "
 "OPEN/FETCH/CLOSE paragraph set; skipped."
),
 "business_paragraph_unknown": (
 "Cursor close guarantee: could not identify the business "
 "paragraph for cursor {cursor}; WHEN ZERO left unchanged."
),
}

```

```

}

__all__ = [
 "ENFORCE_CURSOR_CLOSE_GUARANTEE",
 "SYNTHESISE_MISSING_DRIVING_LOOP",
 "ENFORCE_FETCH_DRIVES_BUSINESS_PARAGRAPH",
 "PREFER_THRU_EXIT_PARAGRAPH",
 "OPERATION_OPEN",
 "OPERATION_FETCH",
 "OPERATION_CLOSE",
 "REQUIRED_OPERATIONS",
 "EOC_CONDITION_TEMPLATE",
 "LEGACY_EOC_CONDITIONS",
 "PERFORM_FETCH_UNTIL_TEMPLATE",
 "PERFORM_CLOSE_TEMPLATE",
 "PERFORM_BUSINESS_TEMPLATE",
 "PERFORM_BUSINESS_THRU_TEMPLATE",
 "UNTIL_TEMPLATE",
 "EXIT_PARAGRAPH_SUFFIX",
 "STATEMENT_TERMINATOR",
 "UNTIL_LOOKAHEAD_LIMIT",
 "NON_PARAGRAPH_SINGLE_WORDS",
 "CURSOR_CLOSE_GUARANTEE_MESSAGES",
]

=====
FILE: rules\cursor_column_rules.py
ENCODING: utf-8
=====

from __future__ import annotations

"""
Rules for cursor SELECT and ORDER BY column resolution.

This module contains constants only.
No resolver logic, parser logic, or program-specific names belong here.
"""

CURSOR_COLUMN_SELECT_RULES = [
 "Prefer columns used by COBOL procedure field usage analysis.",
 "Include parent key columns required by child cursor relationships.",
 "Include child order key columns for child cursors.",
 "Exclude audit columns from cursor SELECT lists.",
 "Fallback to mapped DCLGEN-valid columns only when usage is unavailable.",
]

CURSOR_COLUMN_ORDER_RULES = [
 "Parent/root cursors do not receive ORDER BY unless mapping metadata indicates ordering.",
 "Child cursors may order by non-FK primary or sequence columns.",
 "DESC is applied only when metadata indicates sequence/event/latest-first semantics.",
]

ORDER_HINT_WORDS = (
 "ORDER",
 "ORDERBY",
 "SORT",
 "SORTKEY",
 "RANK",
)

DESC_HINT_WORDS = (
 "SEQ",
 "SEQUENCE",
 "IDMSKEY",
 "IDENTIFIERSEQ",
 "IDENTIFIERSEQSPECIAL",
 "EVENT",
 "EVPR",
 "EVEF",
 "AUTO",

```

```

 "INCREMENT",
)

```

```

=====
FILE: rules\cursor_declaration_rules.py
ENCODING: utf-8
=====

```

```

LOCATION: rules/cursor_declaration_rules.py
ACTION: CREATE NEW FILE
"""DB2 cursor DECLARE statement layout rules.

```

Layout follows the COBOL team's manual reference program:

```

EXEC SQL DECLARE DZBEFFC1 CURSOR WITH HOLD FOR
 SELECT CO_IDPRKSK_479BEFF
 , NS_IRMOSTK_479BEFF
 FROM DZBEFFTV
 FOR READ ONLY
 QUERYNO 176
END-EXEC

```

Constants only. No regex, no runtime logic, no program / record / table / cursor / host variable names.

```

from __future__ import annotations

```

```

Statement form

True -> EXEC SQL DECLARE <cursor> CURSOR WITH HOLD FOR (manual)
False -> EXEC SQL / DECLARE <cursor> CURSOR WITH HOLD FOR (two lines)
DECLARE_ON_EXEC_SQL_LINE = True

```

```

DECLARE_LINE_TEMPLATE = "EXEC SQL DECLARE {cursor} CURSOR WITH HOLD FOR"
SELECT_FIRST_TEMPLATE = "SELECT {column}"
SELECT_NEXT_TEMPLATE = ", {column}"
FROM_TEMPLATE = "FROM {table}"
WHERE_FIRST_TEMPLATE = "WHERE {condition}"
WHERE_NEXT_TEMPLATE = "AND {condition}"
ORDER_BY_TEMPLATE = "ORDER BY {columns}"
FOR_READ_ONLY = "FOR READ ONLY"
END_EXEC = "END-EXEC"

```

```

Body indents, relative to column 8

IND_EXEC = " " # column 12
IND_CLAUSE = " " # column 14
IND_SELECT_FIRST = " " # column 14
IND_SELECT_NEXT = " " # column 19 - aligns comma under SELECT
IND_WHERE_NEXT = " " # column 16 - aligns AND under WHERE
IND_QUERYNO = " " # column 21

```

```

QUERYNO

DB2 EXPLAIN identifies a statement by QUERYNO. The manual programmer
uses the source line number; the converter emits a stable derived
value so repeated runs are byte-identical.
EMIT_QUERYNO = True
QUERYNO_TEMPLATE = "QUERYNO {number}"
QUERYNO_BASE = 100
QUERYNO_STEP = 19

```

```

Parent / child classification

A cursor with no WHERE clause is a parent/root cursor and must not
carry ORDER BY. A cursor with a WHERE clause is a child cursor and
keeps its ORDER BY.
PARENT_CURSOR_KEEPS_ORDER_BY = False
CHILD_CURSOR_KEEPS_ORDER_BY = True

```

```

WHERE column ownership

The left operand of a child join predicate must be a column of the
cursor's own FROM table. Anything else produces SQLCODE -206 at bind.
ENFORCE_WHERE_COLUMN_OWNERSHIP = True
CHILD_JOIN_PREDICATE_TEMPLATE = "{child_column} = :{parent_group}.{parent_host}"
UNRESOLVED_JOIN_KEY_TEMPLATE = (
 "* DB2 WARNING: Unable to declare cursor {cursor}; join key "
 "{column} is not a column of {table}."
)

```

```

Termination

Every declaration except the last closes with a bare END-EXEC.
The last one in the block closes the WORKING-STORAGE sentence with a
period, matching the manual reference.
LAST_DECLARATION_TERMINATOR = "."

```

```

Diagnostics

DECLARATION_MESSAGES = {
 "declared_cursor": (
 "Cursor declaration: declared {cursor} on {table} "
 "with {count} column(s), QUERYNO {queryno}."
),
 "order_by_removed": (
 "Cursor declaration: removed ORDER BY from parent cursor {cursor}."
),
 "join_key_unresolved": (
 "Cursor declaration: join key {column} is not a column of {table}; "
 "cursor {cursor} was not declared."
),
}

```

```
LOCATION: rules/counter_declaration_rules.py
ACTION: APPEND at the end of the file

```

```
A batch program terminates with STOP RUN, or with GOBACK when it is
CALled with a parameter list. Both are valid totals anchors.
VM7BD200 ends with GOBACK, which is why totals were silently skipped.
TOTALS_ANCHOR_STATEMENTS = ("STOP RUN", "GOBACK")

```

```
=====
FILE: rules\cursor_flow_rules.py
ENCODING: utf-8
=====

```

```
"""
Cursor flow rule constants.

```

```
This file contains cursor flow constant values only.
No regex patterns and no runtime logic belong here.
Rules belong in rules/, patterns in patterns/.
"""

```

```
Maximum number of lines to look ahead when correlating a cursor OPEN
with its matching FETCH and business loop.
LOOKAHEAD_LIMIT = 120

```

```
Maximum look-ahead when locating the UNTIL SQLCODE = 100 clause.
UNTIL_LOOKAHEAD_LIMIT = 10

```

```
Dotted lines that must never be treated as paragraph headers.
NON_PARAGRAPH_DOTTED_LINES = {
 "CONTINUE.",
 "END-EXEC.",
 "END-EVALUATE.",
 "END-IF.",
 "END-PERFORM.",
 "EXIT.",
 "GOBACK.",
 "STOP.",
}

```

```

}

Statement prefixes that disqualify a line from being a paragraph header.
NON_PARAGRAPH_HEADER_PREFIXES = (
 "WHEN ",
 "END-",
 "EXEC ",
 "MOVE ",
 "DISPLAY ",
 "PERFORM ",
 "SET ",
 "OPEN ",
 "CLOSE ",
 "FETCH ",
 "INTO",
 ":",
)

Comment replacement inserted where a redundant OBTAIN NEXT is removed.
REMOVED_OBTAIN_NEXT_COMMENT = (
 "* DB2: Removed redundant OBTAIN NEXT after cursor EOC loop."
)

```

```

=====
FILE: rules\cursor_loop_rules.py
ENCODING: utf-8
=====

```

```

LOCATION: rules/cursor_loop_rules.py
ACTION: CREATE NEW FILE
"""Cursor driving-loop rules.

```

Constants only. No regex, no runtime logic, no program / paragraph / record / table / cursor / host variable names.

WHY THIS EXISTS

rules/cursor\_flow\_rules.py describes ONE loop idiom:

```

PERFORM 710-OPEN-<cursor>.
PERFORM 720-FETCH-<cursor>.
PERFORM <business>
 UNTIL SQLCODE = 100.

```

Older programs - and every LRF program, which predates that convention - drive the loop with a paragraph span:

```

PERFORM <business> THRU <business>-EXIT
 UNTIL LR-STATUS = 'VMBFAS-EOA'.

```

with the OPEN and the priming FETCH in a DIFFERENT paragraph, which may appear BEFORE or AFTER the loop in source order.

Restructuring a span loop would rewrite working business flow, which the business-flow rule forbids. The SPAN strategy is therefore deliberately conservative: it fixes the exit condition and guarantees the cursor is closed, and changes nothing else.

```

from __future__ import annotations

```

```

----- master
ENFORCE_SPAN_LOOP_REWRITE = True

```

```

A leftover OBTAIN NEXT may only be stripped when the loop it belonged to
was ACTUALLY converted. Stripping it without a plan deletes the only
FETCH inside the loop and the program never advances.
REQUIRE_PLAN_FOR_OBTAIN_NEXT_CLEANUP = True

```

```

----- loop kinds
LOOP_KIND_INLINE = "INLINE" # PERFORM <para> [UNTIL ...]
LOOP_KIND_SPAN = "SPAN" # PERFORM <para> THRU <para> [UNTIL ...]

```

```

----- correlation
Whole program is searched in BOTH directions. A span loop legitimately

```



```

precedes the paragraph that opens the cursor.
SEARCH_BOTH_DIRECTIONS = True

0 disables the distance ceiling. A ceiling only hides correlation bugs.
MAX_CORRELATION_DISTANCE = 0

Window used when checking whether a CLOSE is already performed.
CLOSE_PRESENCE_WINDOW = 40

----- exit conditions
Conditions that mean "the legacy end test is still in place".
LEGACY_EOC_CONDITIONS = (
 "SQLCODE = 100",
 "SQLCODE=100",
 "DB-END-OF-SET",
 "DB-REC-NOT-FOUND",
)

EOC_CONDITION_TEMPLATE = "{cursor}-EOC"
EOC_CONDITION_SUFFIX = "-EOC"

----- statements
UNTIL_TEMPLATE = "UNTIL {condition}"
UNTIL_TERMINATED_TEMPLATE = "UNTIL {condition}."
PERFORM_CLOSE_TEMPLATE = "PERFORM {paragraph}."

STATEMENT_TERMINATOR = "."

----- diagnostics
CURSOR_LOOP_MESSAGES = {
 "scanned": (
 "Cursor flow: found {inline} inline loop(s) and {span} span loop(s).",
),
 "span_plan": (
 "Cursor flow: correlated cursor {cursor} with span loop "
 "PERFORM {paragraph} THRU {through}.",
),
 "condition_rewritten": (
 "Cursor flow: rewrote span loop exit condition to {condition}.",
),
 "close_inserted": (
 "Cursor flow: inserted {paragraph} after the span loop.",
),
 "close_present": (
 "Cursor flow: {paragraph} already performed; no insertion needed.",
),
 "no_loop_for_cursor": (
 "Cursor flow: cursor {cursor} has no recognised driving loop; "
 "the generated OBTAIN NEXT was left in place for manual review.",
),
 "obtain_next_retained": (
 "Cursor flow: no cursor loop was converted, so generated fetch "
 "calls were retained.",
),
}

```

```

=====
FILE: rules\cursor_paragraph_rules.py
ENCODING: utf-8
=====

```

```

LOCATION: rules/cursor_paragraph_rules.py
ACTION: REPLACE ENTIRE FILE

```

```

"""Cursor paragraph generation rules.

```

```

This module contains constants only.

```

```

No regex patterns, parser logic, generator logic, or hardcoded program,
record, table, cursor, or host variable names belong here.

```

```

Layout follows the COBOL team's manual reference program:

```

```

* OPEN CURSOR

```

```

710-OPEN-DZBEFFC1.
D DISPLAY '710-OPEN-DZBEFFC1.'
 MOVE 710 TO SQL-LOCATION

EXEC SQL
 OPEN DZBEFFC1
END-EXEC

EVALUATE SQLCODE
 WHEN ZERO
 SET DZBEFFC1-NOT-EOC TO TRUE
 WHEN OTHER
 DISPLAY 'ERROR WHILE OPENING CURSOR DZBEFFC1'
 PERFORM SQLERROR
END-EVALUATE
.
"""

from __future__ import annotations

=====
Fixed process / error paragraph names (COBOL-shop standard)
=====
SQL_ERROR_PARAGRAPH_NAME = "SQLERROR"
LEGACY_SQL_ERROR_PARAGRAPH_NAME = "SQL-ERROR"
SQL_LOCATION_FIELD = "SQL-LOCATION"

=====
Termination rule
=====
The paragraph body is ONE COBOL sentence. No period on MOVE, END-EXEC
or END-EVALUATE; a single period on its own line closes the paragraph.
A period inside EVALUATE closes the scope early and orphans
END-EVALUATE.
LONE_PERIOD = "."

=====
SQL-LOCATION form
=====
RESOLVED by the COBOL team's manual reference program:
#
MOVE 710 TO SQL-LOCATION
#
SQL-LOCATION is supplied by the SQLERRWS copybook, NOT declared by the
converter.
#
code_review CHK-05 already accepts both forms:
MOVE_SQL_LOCATION = re.compile(
r"^MOVE\s+(?:'[^']*'|\d+)\s+TO\s+SQL-LOCATION\s*\.\?$"
)
so flipping this to True does not regress the review.
SQL_LOCATION_USES_PARAGRAPH_NUMBER = True
SQL_LOCATION_NUMBER_PAD = " "

The converter must NOT declare SQL-LOCATION: the copybook owns it.
DECLARE_SQL_LOCATION_FIELD = False

=====
Body indents, relative to column 8
=====
IND_STATEMENT = " " # column 12
IND_SQL_BODY = " " # column 14
IND_SQL_INTRO = " " # column 15
IND_SQL_INTRO_NEXT = " " # column 18
IND_WHEN = " " # column 14
IND_WHEN_BODY = " " # column 19
IND_WHEN_BODY_CONT = " " # DISPLAY align

=====
COBOL tokens
=====
TOKEN_EXEC_SQL = "EXEC SQL"
TOKEN_END_EXEC = "END-EXEC"
TOKEN_EVALUATE_SQLCODE = "EVALUATE SQLCODE"
TOKEN_END_EVALUATE = "END-EVALUATE"

```

```

TOKEN_WHEN_ZERO = "WHEN ZERO"
TOKEN_WHEN_100 = "WHEN 100"
TOKEN_WHEN_OTHER = "WHEN OTHER"
TOKEN_CONTINUE = "CONTINUE"
TOKEN_INT0 = "INT0"
TOKEN_HOST_SEPARATOR = ", "

=====
Cursor operations and flag suffixes
=====
CURSOR_OPERATIONS = ("OPEN", "FETCH", "CLOSE")
CURSOR_EOC_SUFFIX = "-EOC"
CURSOR_NOT_EOC_SUFFIX = "-NOT-EOC"
CURSOR_FLAG_TEMPLATE = "WS-{cursor}-FLAG"

=====
Generated statement templates
=====
SQL_LOCATION_NUMBER_TEMPLATE = "MOVE {number}{pad}TO {field}"
SQL_LOCATION_NAME_TEMPLATE = "MOVE '{name}' TO {field}"
PARAGRAPH_HEADER_TEMPLATE = "{paragraph}."
OPEN_STATEMENT_TEMPLATE = "OPEN {cursor}"
FETCH_STATEMENT_TEMPLATE = "FETCH {cursor}"
CLOSE_STATEMENT_TEMPLATE = "CLOSE {cursor}"
SET_NOT_EOC_TEMPLATE = "SET {cursor}{suffix} TO TRUE"
SET_EOC_TEMPLATE = "SET {cursor}{suffix} TO TRUE"
DISPLAY_ERROR_TEMPLATE = "DISPLAY '{text}'"
PERFORM_TEMPLATE = "PERFORM {paragraph}"
COMMENT_BLOCK_TEMPLATE = "{title:<62}*"
INT0_FIRST_TEMPLATE = "INT0 :{group}.{host}"
INT0_NEXT_TEMPLATE = ", :{group}.{host}"

=====
Manual reference banners
=====
BANNER_RULE = (
 "*-----*"
)
BANNER_TITLE_TEMPLATE = "*" {title}"

BANNER_OPEN_CURSOR = "OPEN CURSOR"
BANNER_FETCH_CURSOR = "FETCH CURSOR"
BANNER_CLOSE_CURSOR = "CLOSE CURSOR"

EMIT_PARAGRAPH_BANNERS = True

=====
Debug (column 7 = 'D') trace lines
=====
EMIT_DEBUG_TRACE_LINES = True
DEBUG_INDICATOR = "D"
DEBUG_TRACE_TEMPLATE = "DISPLAY '{paragraph}.'"

Sentinel prefix used to mark a body line that must carry the 'D'
indicator in column 7. The fixed-format writer strips it.
DEBUG_SENTINEL = "\x01D"

=====
Join-key diagnostics inside a child FETCH WHEN OTHER branch
=====
The manual reference displays every parent join key before performing
SQLERROR so the failing row can be identified in the job log.
EMIT_JOIN_KEY_DIAGNOSTICS = True
JOIN_KEY_DISPLAY_TEMPLATE = "DISPLAY '{label} = ' {host}"
JOIN_KEY_QUALIFIER_TEMPLATE = "OF {group}"
JOIN_KEY_LABEL_WIDTH = 13

=====
Error display text
=====
ERROR_OPEN_TEMPLATE = "ERROR WHILE OPENING CURSOR {cursor}"
ERROR_FETCH_TEMPLATE = "ERROR WHILE FETCHING CURSOR {cursor}"
ERROR_CLOSE_TEMPLATE = "ERROR WHILE CLOSING CURSOR {cursor}"

=====
Diagnostics

```

```

=====
DIAG_NO_CURSOR_OPERATIONS = (
 "DB2 cursor paragraphs: no cursor operations found."
)
DIAG_BLOCK_EXISTS = (
 "DB2 cursor paragraphs: generated cursor paragraph block already exists."
)
DIAG_GENERATED_TEMPLATE = (
 "DB2 cursor paragraphs: generated {count} cursor paragraph set(s)."
)
DIAG_NO_HOST_VARIABLES_TEMPLATE = (
 "DB2 cursor paragraphs: no FETCH host variables resolved for "
 "cursor {cursor} record {record} table {table}."
)

=====
FILE: rules\cursor_rules.py
ENCODING: utf-8
=====

"""
Cursor generation rules.

This file contains cursor authority rules and paragraph numbering rules.

Rules belong in rules/, not patterns/.

No regex patterns should be defined here.
No parser, service, transformer, or generator logic should be placed here.
"""

CURSOR_AUTHORITY_RULES = [
 "Cursor names are derived from Sheet Mapping DB2 record/table names.",
 "Cursor names are not derived from IDMS set names.",
 "Generated cursor OPEN, FETCH, and CLOSE paragraphs are placed near the end.",
 "SQL-ERROR paragraph is required when cursor code performs SQL error handling.",
]

CURSOR_PARAGRAPH_NUMBERING = {
 1: {
 "open": 710,
 "fetch": 720,
 "close": 730,
 },
 2: {
 "open": 810,
 "fetch": 820,
 "close": 830,
 },
 3: {
 "open": 910,
 "fetch": 920,
 "close": 930,
 },
}

def cursor_paragraph_number(
 cursor_order: int,
 operation: str,
) -> int:
 operation_key = str(operation or "").strip().lower()

 if cursor_order not in CURSOR_PARAGRAPH_NUMBERING:
 raise ValueError(f"Unsupported cursor order: {cursor_order}")

 if operation_key not in CURSOR_PARAGRAPH_NUMBERING[cursor_order]:
 raise ValueError(f"Unsupported cursor operation: {operation}")

 return CURSOR_PARAGRAPH_NUMBERING[cursor_order][operation_key]

```

```
=====
FILE: rules\cursor_select_minimizer_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
"""
```

```
Rules for cursor SELECT minimization.
```

```
This module contains constants only.
```

```
No regex patterns, service logic, parser logic, cursor names, table names,
columns, or host variables belong here.
```

```
"""
```

```
CURSOR_SELECT_MINIMIZER_RULES = [
 "Remove columns selected only for ORDER BY when safe.",
 "Do not hardcode cursor names, table names, columns, or host variables.",
 "Keep SELECT and FETCH INTO synchronized by position.",
 "Do not treat SQL keywords as selected columns.",
 "Normalize commas only inside SELECT item and FETCH INTO scopes.",
]
```

```
SQL_NON_COLUMN_KEYWORDS = {
 "SELECT",
 "FROM",
 "WHERE",
 "ORDER",
 "ORDER BY",
 "GROUP",
 "GROUP BY",
 "HAVING",
 "FOR",
 "FOR READ ONLY",
 "INTO",
 "END-EXEC",
 "EXEC SQL",
}
```

```
SELECT_FIRST_ITEM_BODY_PREFIX = " "
SELECT_NEXT_ITEM_BODY_PREFIX = " ", "
FETCH_HOST_BODY_PREFIX = " "
```

```
=====
FILE: rules\db2_date_conversion_rules.py
ENCODING: utf-8
=====
```

```
"""
```

```
DB2 date conversion rules.
```

```
This file contains reusable templates for DB2 date comparison conversion.
```

```
No program name is hardcoded.
```

```
No table name is hardcoded.
```

```
No DCLGEN group name is hardcoded.
```

```
No business field name is hardcoded.
```

```
The composer supplies dynamic values such as:
```

```
- field_name
- group_name
- helper
- condition
- indent
```

```
"""
```

```
DB2_DATE_COMPARISON_RULES = [
 "DB2 DATE values must be realigned before numeric comparison.",
 "Generate date conversion only when a DA-/DT- DCLGEN host field is compared with PARMDATE.",
 "Generated helper field names are derived from the detected DB2 host field name.",
 "Date conversion logic must not hardcode program names, table names, record names, or business fields.",
]
```

```

 "Initialize generated date helper fields before moving converted date values into them.",
]

```

```

DB2_DATE_COMPARISON_WS_MARKER = "DB2 DATE CONVERSION WORKING STORAGE"

```

```

DB2_DATE_BASE_WORKING_STORAGE_LINES = [
 f"* {DB2_DATE_COMPARISON_WS_MARKER}",
 "01 WS-DATUMVELDEN.",
 " 03 DA-CCYYMMDD.",
 " 05 CC PIC 99 VALUE ZERO.",
 " 05 YY PIC 99 VALUE ZERO.",
 " 05 MM PIC 99 VALUE ZERO.",
 " 05 DD PIC 99 VALUE ZERO.",
 " 03 DA-CCYYMMDD-R REDEFINES DA-CCYYMMDD.",
 " 05 CCYY PIC 9999.",
 " 05 MM PIC 99.",
 " 05 DD PIC 99.",
 " 03 DA-DD-MM-CCYY.",
 " 05 DD PIC 9(02).",
 " 05 TE-MARKER5 PIC X VALUE '.'. ",
 " 05 MM PIC 9(02).",
 " 05 TE-MARKER6 PIC X VALUE '.'. ",
 " 05 CCYY PIC 9(04).",
]

```

```

DB2_DATE_HELPER_FIELD_TEMPLATE = (
 " 02 {helper:<30} PIC 9(8) VALUE ZEROES."
)

```

```

DB2_DATE_LOW_VALUE_LITERAL = "01.01.0001"
DB2_DATE_HIGH_VALUE_LITERAL = "31.12.9999"
DB2_DATE_HIGH_NUMERIC_LITERAL = "99999999"

```

```

DB2_DATE_CONVERSION_LINE_TEMPLATES = [
 "{indent}MOVE ZEROES TO DA-CCYYMMDD",
 "{indent}MOVE ZEROES TO {helper}",
 "{indent}MOVE {field_name} OF {group_name} TO DA-DD-MM-CCYY",
 "{indent}EVALUATE TRUE",
 "{indent} WHEN DA-DD-MM-CCYY = '{low_value}'",
 "{indent} MOVE ZEROES TO DA-CCYYMMDD-R",
 "{indent} WHEN DA-DD-MM-CCYY = '{high_value}'",
 "{indent} MOVE '{high_numeric}' TO DA-CCYYMMDD-R",
 "{indent} WHEN OTHER",
 "{indent} MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF DA-CCYYMMDD-R",
 "{indent} MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMMDD-R",
 "{indent} MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMMDD-R",
 "{indent}END-EVALUATE",
 "{indent}MOVE DA-CCYYMMDD-R TO {helper}",
]

```

```

DB2_DATE_IF_REPLACEMENT_TEMPLATE = "{indent}IF {helper} {condition}"

```

```

LOCATION: rules/db2_date_conversion_rules.py
ACTION: APPEND at the end of the file

#
---- Compound / multi-line condition support (appended)
#
A DB2 DATE host is stored in external form DD.MM.CCYY (10 bytes). A
COBOL date field is normally CCYYMMDD, PIC 9(8). Comparing them directly
compares '0' against '2' and selects the wrong rows, so every DATE host
that appears in a comparison is first realigned into a HELP- helper.
#
ENFORCE_DATE_COMPARISON_CONVERSION = True

Physical lines scanned forward looking for the end of one condition.
CONDITION_SCAN_LIMIT = 24

Refuse rather than guess when the condition is larger than this.
MAX_DATE_OPERANDS_PER_CONDITION = 8

```

```

Replacement for a whole condition, however compound.
The old DB2_DATE_IF_REPLACEMENT_TEMPLATE handled one operand only.
DB2_DATE_IF_CONDITION_TEMPLATE = "{indent}IF {condition}"

Banner emitted immediately above a generated realignment block.
DB2_DATE_CONVERSION_BANNER_TEMPLATE = (
 "{indent}* DB2: realigned {count} DB2 DATE host(s) before comparison."
)

#
---- Loud refusal
#
A pass that cannot handle its input must say so. Returning the text
unchanged made a skipped conversion indistinguishable from a program
with no dates at all, which is how the wrong-row defect shipped.
#
EMIT_DATE_REFUSAL_COMMENT = True

DB2_DATE_REFUSAL_COMMENT_TEMPLATE = (
 "{indent}* DB2 WARNING: {field} is a DB2 DATE host compared against a "
 "non-DATE field; {reason}"
)

DB2_DATE_MESSAGES = {
 "converted": (
 "DB2 date compare: realigned {count} DATE host(s) in {conditions} "
 "condition(s).",
),
 "helpers_declared": (
 "DB2 date compare: declared {count} HELP- date helper field(s).",
),
 "condition_terminated": (
 "DB2 date compare: condition starting at line {line} was not "
 "closed within {limit} lines; left unchanged for manual review.",
),
 "too_many_operands": (
 "DB2 date compare: condition starting at line {line} carries "
 "{count} DATE operands, above the limit of {limit}; left unchanged "
 "for manual review.",
),
 "no_comparison_operator": (
 "DB2 date compare: {field} appears in an IF with no comparison "
 "operator; left unchanged.",
),
 "inside_exec_sql": (
 "DB2 date compare: DATE host references inside EXEC SQL are never "
 "rewritten.",
),
 "nothing_found": (
 "DB2 date compare: no DB2 DATE host is compared in this program.",
),
}

```

```

=====
FILE: rules\db2_include_rules.py
ENCODING: utf-8
=====

```

```

LOCATION: rules/db2_include_rules.py
ACTION: CREATE NEW FILE

```

```

""""DB2 INCLUDE emission rules.

```

Constants only. No regex, no runtime logic, no program / record / table names.

WHY THIS EXISTS

-----

Three components emitted EXEC SQL INCLUDE independently:

InfrastructureBlockBuilder	3-line block, Area B
DclgenIncludeCleanup	3-line block, 1 space
StorageWsInjector	single line, 1 space

Two shapes and no shared duplicate detection. The update injector looked for the SINGLE-LINE form, did not recognise the BLOCK form already emitted upstream, and injected DZBFARTV a second time. A duplicated DCLGEN include duplicates every data-name in the group, which the compiler rejects.

Every emitter now renders through IncludeRenderer using these constants, so the shape and the indent are decided in one place.

```
from __future__ import annotations
```

```
=====
Canonical form
=====
True -> EXEC SQL INCLUDE SQLCA END-EXEC. (one line)
False -> EXEC SQL / INCLUDE SQLCA / END-EXEC. (three lines)
#
The COBOL team's manual reference uses the single-line form for
includes and the block form for cursor declarations.
INCLUDE_SINGLE_LINE = True

INCLUDE_SINGLE_TEMPLATE = "EXEC SQL INCLUDE {name} END-EXEC."
INCLUDE_BLOCK_OPEN = "EXEC SQL"
INCLUDE_BLOCK_BODY_TEMPLATE = "INCLUDE {name}"
INCLUDE_BLOCK_CLOSE = "END-EXEC."

=====
Indentation, relative to column 8
=====
CHK-06.07 requires at least 4 body spaces, i.e. physical column 12.
IND_INCLUDE = " " # column 12
IND_INCLUDE_BODY = " " # column 14

=====
Infrastructure includes
=====
Never treated as DCLGEN table includes, and never auto-injected as one.
INFRASTRUCTURE_INCLUDES = frozenset(
 {"SQLCA", "SQLERRWS", "SQLERROR", "GEN"})
)

=====
Diagnostics
=====
INCLUDE_MESSAGES = {
 "rendered": "DB2 includes: rendered {count} include statement(s).",
 "skipped_duplicate": (
 "DB2 includes: {name} is already included; not injected again."
),
 "injected": "DB2 includes: injected missing include {name}.",
 "removed_duplicate": (
 "DB2 includes: removed {count} duplicate include statement(s) "
 "for {name}."
),
}
}
```

```
=====
FILE: rules\db2_infrastructure_rules.py
ENCODING: utf-8
=====
```

```
LOCATION: rules/db2_infrastructure_rules.py
ACTION: APPEND at the end of the file
```

```
=====
Area B indentation for generated DB2 blocks
=====
CHK-06.07 measures every generated DB2 block line and requires at
least 4 body spaces, i.e. physical column 12. The manual reference
places EXEC SQL, END-EXEC, EVALUATE SQLCODE and END-EVALUATE at
column 12.
#
InfrastructureBlockBuilder previously emitted "EXEC SQL" with no
indent at all, so all 26 generated infrastructure lines landed in
```



```

Area A and the check failed.
#
Body indents are relative to column 8.
ENFORCE_AREA_B_SQL_INDENT = True

IND_EXEC = " " # column 12 - EXEC SQL, END-EXEC
IND_SQL_BODY = " " # column 14 - DECLARE, SELECT, FROM
IND_SELECT_FIRST = " " # column 16 - first SELECT column
IND_SELECT_NEXT = " " # column 15 - comma aligns under the name
IND_WHERE_NEXT = " " # column 16 - conditions under WHERE
IND_88_LEVEL = " " # column 12 - 88 condition names

=====
SQL tokens
=====
TOKEN_EXEC_SQL = "EXEC SQL"
TOKEN_END_EXEC = "END-EXEC."
TOKEN_SELECT = "SELECT"
TOKEN_WHERE = "WHERE"
TOKEN_ORDER_BY = "ORDER BY"
TOKEN_FOR_READ_ONLY = "FOR READ ONLY"

DECLARE_TEMPLATE = "DECLARE {cursor} CURSOR WITH HOLD FOR"
FROM_TEMPLATE = "FROM {table}"
SELECT_ITEM_FIRST_TEMPLATE = "{column}"
SELECT_ITEM_NEXT_TEMPLATE = ", {column}"
SELECT_ALL = "*"

=====
Cursor end-of-cursor flags
=====
FLAG_NAME_TEMPLATE = "WS-{cursor}-FLAG"
NOT_EOC_TEMPLATE = "{cursor}-NOT-EOC"
EOC_TEMPLATE = "{cursor}-EOC"

FLAG_DECLARATION_TEMPLATE = "01 {name:<30} PIC X."
CONDITION_TEMPLATE = "88 {name:<26} VALUE '{value}'."

VALUE_NOT_EOC = "N"
VALUE_EOC = "Y"

=====
SQL-LOCATION
=====
SQL_LOCATION_DECLARATION_TEMPLATE = "01 {name:<30} {picture}"

=====
Warnings
=====
MISSING_CURSOR_NAME = (
 "* DB2 WARNING: Unable to declare cursor; missing cursor name."
)
MISSING_TABLE_TEMPLATE = (
 "* DB2 WARNING: Unable to declare cursor {cursor}; "
 "missing DB2 table mapping."
)

=====
FILE: rules\db2_output_date_conversion_rules.py
ENCODING: utf-8
=====

"""
DB2 output date conversion rules.

This file contains reusable output-date conversion templates only.
No parser, service, transformer, composer, or generator logic belongs here.

Purpose:
- Convert DB2 date host fields into the output-file numeric date format
 before writing output records.
- Keep conversion rules centralized and generic.

No program name is hardcoded.
No table name is hardcoded.

```

No DCLGEN group name is hardcoded.  
No business field name is hardcoded.

The composer supplies dynamic values:

- indent
- field\_name
- group\_name
- target\_name

"""

```
DB2_OUTPUT_DATE_CONVERSION_RULES = [
 "DB2 date values must be realigned before moving to output date fields.",
 "Generate output date conversion only when a DA-/DT- DCLGEN host field is moved to an output date field.",
 "Initialize shared DB2 date work fields before date realignment.",
 "Clear the output date target before moving the converted date value.",
 "Do not hardcode program names, table names, DCLGEN group names, or business field names.",
 "Use shared date work fields when already generated in WORKING-STORAGE.",
]
```

```
DB2_OUTPUT_DATE_LOW_VALUE_LITERAL = "01.01.0001"
DB2_OUTPUT_DATE_HIGH_VALUE_LITERAL = "31.12.9999"
DB2_OUTPUT_DATE_HIGH_NUMERIC_LITERAL = "99999999"
```

```
DB2_OUTPUT_DATE_CONVERSION_LINE_TEMPLATES = [
 "{indent}MOVE ZEROES TO DA-CCYYMMDD",
 "{indent}MOVE SPACES TO {target_name}",
 "{indent}MOVE {field_name} OF {group_name} TO DA-DD-MM-CCYY",
 "{indent}EVALUATE TRUE",
 "{indent} WHEN DA-DD-MM-CCYY = '{low_value}'",
 "{indent} MOVE ZEROES TO DA-CCYYMMDD-R",
 "{indent} WHEN DA-DD-MM-CCYY = '{high_value}'",
 "{indent} MOVE '{high_numeric}' TO DA-CCYYMMDD-R",
 "{indent} WHEN OTHER",
 "{indent} MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF DA-CCYYMMDD-R",
 "{indent} MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMMDD-R",
 "{indent} MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMMDD-R",
 "{indent}END-EVALUATE",
 "{indent}MOVE DA-CCYYMMDD-R TO {target_name}",
]
```

```
=====
```

FILE: rules\db2\_validation\_rules.py  
ENCODING: utf-8

```
=====
```

"""

DB2 validation rules.

This file contains DB2 validation constants only.

Rules belong in rules/, not patterns/.

No regex patterns should be defined here.  
No parser, service, transformer, generator, validator, or composer logic  
should be placed here.

Use this file for DB2 validation rule values that are shared by validators  
or other runtime modules.

"""

```
REQUIRED_DB2_TOKENS = (
 "EXEC SQL",
 "SQLCA",
 "END-EXEC",
)
```

```
=====
```

FILE: rules\field\_mapping\_rules.py  
ENCODING: utf-8

```
=====
```

"""

Field mapping rules.

These constants describe authority and rewrite behavior for IDMS-to-DB2 field mapping. No regex or runtime logic belongs in this file.

```
"""
FIELD_MAPPING_RULES = [
 "Sheet Mapping is the authority for DB2 column names.",
 "DCLGEN is the authority for COBOL host variable spelling.",
 "Do not invent DB2 field names.",
 "Do not use IDMS record names in final DB2 procedure logic when a Sheet Mapping and DCLGEN mapping exists.",
 "Rewrite only safe qualified references unless strong context exists.",
]

QUALIFIED_REFERENCE_REWRITE_RULES = [
 "Rewrite FIELD OF IDMS-RECORD when a Sheet Mapping and DCLGEN mapping exists.",
 "Rewrite FIELD IN IDMS-RECORD when a Sheet Mapping and DCLGEN mapping exists.",
 "Do not rewrite bare words globally.",
 "When mapping metadata is missing, leave a clear conversion-skipped comment.",
]
```

```
=====
FILE: rules\field_naming_rules.py
ENCODING: utf-8
=====
```

```
LOCATION: rules/field_naming_rules.py
ACTION: REPLACE ENTIRE FILE (add audit prefixes to your existing canonical_field_key)
```

```
"""
Deterministic field-naming rules.
```

```
Sheet Mapping is the authority: each row pairs a COBOL-side identifier
(`cobol_zone` / `reference_field_name_copybook`) with `new_db2_field_name`
(the DB2 column). Resolution is an exact canonical-key lookup on the SAME
row. No fuzzy / similarity matching is used.
"""
```

```
from __future__ import annotations
```

```
INSERT_ONLY_AUDIT_PREFIXES = (
 "TS_CREATE",
)
```

```
UPDATE_AUDIT_PREFIXES = (
 "TS_UPDATE",
 "ID_USERID",
 "NR_USERID",
 "ID_USER",
 "NR_USER",
)
```

```
def canonical_field_key(name: str) -> str:
 """
 Canonical comparison key: uppercase, strip a leading COBOL level number,
 and remove '-' and '_' separators.

 Examples:
 "03 CT-RK-TGDSV" -> "CTRKTGDSV"
 "DA-INFSD-GDIFAR" -> "DAINFSDGDIFAR"
 "NR_ID" -> "NRID"
 """
 text = str(name or "").strip().upper()

 parts = text.split()
 if parts and parts[0].isdigit():
 text = " ".join(parts[1:])

 return text.replace("-", "").replace("_", "").replace(" ", "")
```

```
=====
FILE: rules\field_reference_rewriter_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
"""
```

```
Rules/constants for IDMS field reference rewriting.
```

```
This module contains constants only.
```

```
No regex patterns, transformer logic, DB2 table names, DCLGEN names, or host
variables belong here.
```

```
"""
```

```
FIELD_REFERENCE_REWRITE_RULES = [
 "Rewrite qualified FIELD OF RECORD and FIELD IN RECORD references when Sheet Mapping and DCLGEN metadata resolve sa
fely.",
 "Rewrite bare references only when active record context is strong.",
 "Do not rewrite inside EXEC SQL blocks.",
 "Do not rewrite string literals.",
 "Do not rewrite non-PROCEDURE DIVISION business declarations.",
 "Protect date/work/control prefixes for bare references only.",
 "Do not hardcode program names, record names, DB2 tables, DB2 columns, DCLGEN groups, or host variables.",
]
```

```
PROTECTED_BARE_PREFIXES = (
```

```
 "WS-",
 "WK-",
 "W-",
 "SW-",
 "UIT-",
 "OUT-",
 "ES-",
 "D-",
 "DATE",
 "SQL",
 "DCL",
 "L-",
 "LS-",
 "PARAM",
 "ERROR-",
 "USER",
 "CS-",
 "TS-",
 "DA-",
 "HR-",
 "HELP-",
```

```
)
```

```
DCLGEN_GROUP_PREFIX = "DCL"
```

```
--- Host-reference formatting constants (used by the host resolver) ---
```

```
Separator between a host field and its DCLGEN group.
```

```
HOST_GROUP_SEPARATOR = " OF "
```

```
Leading character stripped from a raw host reference (e.g. ":HOST").
```

```
HOST_REFERENCE_LEADING_CHAR = ":"
```

```
Format template for a resolved "HOST OF GROUP" reference.
```

```
HOST_OF_GROUP_TEMPLATE = "{host} OF {group}"
```

```
Number of parts expected when splitting on the OF keyword.
```

```
HOST_OF_SPLIT_PART_COUNT = 2
```

```
LOCATION: rules/field_reference_rewriter_rules.py
```

```
ACTION: APPEND at the end of the file
```

```
#
```

```
---- Fixed-format safety (appended)
```

```
#
```

```
A qualified rewrite is almost always LONGER than what it replaces:
```

```
#
```

```
DA-CPTA-FORM-AS OF VMBFAS 25 characters
DA-CPTAFS-479BFAS OF DCLDZBFASTV 32 characters
#
Substituting that into the whole record pushed the right sequence
number from column 73 to column 80. Downstream passes then read the
first displaced digit as part of the body and emitted
#
AND HELP-DA-CPTAFS-479BFAS NOT = '00000000') OR 0
#
Rewriting is therefore confined to columns 8-72 and the record is
reassembled from its original sequence areas.
ENFORCE_FIXED_FORMAT_REWRITE = True

A rewritten body that no longer fits 65 columns is wrapped at word
boundaries rather than truncated or allowed to overflow.
ALLOW_REWRITE_WRAPPING = True

FIELD_REFERENCE_GEOMETRY_MESSAGES = {
 "wrapped": (
 "Field reference: rewritten statement exceeded column 72 and was "
 "wrapped onto {count} line(s).",
),
 "refused": (
 "Field reference: rewritten statement cannot be represented in "
 "columns 8-72 and was left unchanged for manual review: {body}"
),
}
}
```

```
=====
FILE: rules\field_usage_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

"""
Field usage analysis rules.

This module contains rule constants only.
No regex patterns, parser logic, analyzer logic, program names, DB2 table names,
DCLGEN names, or host variables belong here.
"""
```

```
FIELD_USAGE_ANALYSIS_RULES = [
 "Detect COBOL field usage without rewriting COBOL.",
 "Track IDMS qualified field references by mapped record.",
 "Track DCLGEN host references by resolved DCLGEN group context.",
 "Track MOVE source and target fields separately.",
 "Track condition fields from IF, WHEN, UNTIL, and EVALUATE statements.",
 "Do not hardcode program names, DB2 tables, DB2 columns, or host variables.",
]
```

```
DCLGEN_GROUP_PREFIX = "DCL"
```

```
OUTPUT_TARGET_PREFIXES = (
 "UIT-",
)
```

```
=====
FILE: rules\final_feedback_fix_rules.py
ENCODING: utf-8
=====
```

```
"""
Final feedback fix rules.

This module contains constants and rule values only.

No regex.
No parser logic.
No service logic.
```

```

No hardcoded business names.
"""

from __future__ import annotations

FINAL_FEEDBACK_FIX_RULES = [
 "Fix Procedure Division executable statements to begin in Area B.",
 "Synchronize PROGRAM-NAME from final PROGRAM-ID only in safe MOVE statements.",
 "Use configurable DB2 date external format for generated date conversion.",
 "Remove ORDER-BY-only columns from cursor SELECT and FETCH when safe.",
 "Do not hardcode program names, DB2 tables, columns, DCLGEN groups, or host variables.",
 "Do not remove existing business logic or already fixed feedback functionality.",
]

AREA_A_BODY_INDENT = ""
AREA_B_BODY_INDENT = " "
SQL_BODY_INDENT = " "

SUPPORTED_DB2_DATE_FORMATS = {
 "YYYY-MM-DD",
 "DD.MM.YYYY",
 "YYYYMMDD",
}

DATE_FORMAT_ALIASES = {
 "ISO": "YYYY-MM-DD",
 "YYYY-MM-DD": "YYYY-MM-DD",
 "EUR": "DD.MM.YYYY",
 "EUROPEAN": "DD.MM.YYYY",
 "DD.MM.YYYY": "DD.MM.YYYY",
 "DD.MM.CCYY": "DD.MM.YYYY",
 "NUMERIC": "YYYYMMDD",
 "YYYYMMDD": "YYYYMMDD",
 "CCYYMMDD": "YYYYMMDD",
}

Preserve current generated behavior unless the caller explicitly configures
a different site/DCLGEN date external format.
DEFAULT_DB2_DATE_EXTERNAL_FORMAT = "DD.MM.YYYY"

DB2_DATE_HELPER_BY_FORMAT = {
 "DD.MM.YYYY": "DA-DD-MM-CCYY",
 "YYYY-MM-DD": "DA-YYYY-MM-DD",
 "YYYYMMDD": "DA-YYYYMMDD",
}

DB2_DATE_LOW_LITERAL_BY_FORMAT = {
 "DD.MM.YYYY": "'01.01.0001'",
 "YYYY-MM-DD": "'0001-01-01'",
 "YYYYMMDD": "'00010101'",
}

DB2_DATE_HIGH_LITERAL_BY_FORMAT = {
 "DD.MM.YYYY": "'31.12.9999'",
 "YYYY-MM-DD": "'9999-12-31'",
 "YYYYMMDD": "'99991231'",
}

ORDER_BY_COLUMNS_IN_SELECT_DEFAULT = False

--- Appended: constants previously inlined in FinalFeedbackFixComposer ---

FINAL_FIX_STRING_START_BODY_INDENT = " "
FINAL_FIX_STRING_CONTINUATION_BODY_INDENT = " "
FINAL_FIX_STRING_INTO_BODY_INDENT = " "

FINAL_FIX_AREA_B_INDENT = " "
FINAL_FIX_NESTED_INDENT = " "

Reuse the shared NON_PARAGRAPH_SINGLE_WORDS from rules/cobol_statement_rules.py.

AREA_B_CONTINUATION_BODY_INDENT = " " # one level past the statement

```

```
=====
FILE: rules\fixed_format_rules.py
ENCODING: utf-8
=====
```

```
"""
Fixed-format COBOL layout rules.
These constants define final physical COBOL layout:
- Columns 1-6 : left sequence number
- Column 7 : indicator area
- Columns 8-72 : COBOL body
- Columns 73-80 : right sequence number
This file contains rules/constants only.
No regex should be defined here.
"""

TOTAL_WIDTH = 80
BODY_WIDTH = 65
LEFT_SEQUENCE_WIDTH = 6
INDICATOR_WIDTH = 1
RIGHT_SEQUENCE_WIDTH = 8
DEFAULT_LEFT_START = 10
DEFAULT_LEFT_STEP = 10
Generic fallback for manual-style COBOL when input has no valid
manual right sequence, or when input has old/small right sequence.
#
Example:
left 000010 -> right 00010000
left 000020 -> right 00020000
#
This is not program-specific hardcoding.
It is a layout fallback rule.
DEFAULT_RIGHT_START = 10000
DEFAULT_RIGHT_STEP = 10000
AREA_A_INDENT = ""
AREA_B_INDENT = " "
SQL_INDENT = " "
SQL_CONTINUATION_INDENT = " "
SPACE_INDICATOR = " "
COMMENT_INDICATOR = "*"
PAGE_INDICATOR = "/"
DEBUG_INDICATOR = "D"
CONTINUATION_INDICATOR = "-"
VALID_INDICATORS = {
 SPACE_INDICATOR,
 COMMENT_INDICATOR,
 PAGE_INDICATOR,
 DEBUG_INDICATOR,
 "d",
 CONTINUATION_INDICATOR,
}
NON_PARAGRAPH_WORDS = {
 "ACCEPT",
 "ADD",
 "ALTER",
 "CALL",
 "CANCEL",
 "CLOSE",
 "COMMIT",
 "COMPUTE",
 "CONTINUE",
 "DELETE",
 "DISPLAY",
 "DIVIDE",
 "ELSE",
 "END-ADD",
 "END-CALL",
 "END-DELETE",
 "END-DIVIDE",
 "END-EVALUATE",
 "END-EXEC",
 "END-IF",
 "END-MULTIPLY",
 "END-PERFORM",
 "END-READ",
 "END-RETURN",
 "END-REWRITE",
}
```

```

"END-SEARCH",
"END-START",
"END-STRING",
"END-SUBTRACT",
"END-UNSTRING",
"END-WRITE",
"EVALUATE",
"EXEC",
"EXIT",
"FETCH",
"GOBACK",
"IF",
"INITIALIZE",
"INSPECT",
"MOVE",
"MULTIPLY",
"NEXT",
"OPEN",
"PERFORM",
"READ",
"RETURN",
"REWRITE",
"ROLLBACK",
"SEARCH",
"SET",
"SKIP1",
"SKIP2",
"SKIP3",
"SORT",
"SPACE",
"SPACES",
"START",
"STOP",
"STRING",
"SUBTRACT",
"UNSTRING",
"WHEN",
"WRITE",
}
PROCEDURE_VERBS = (
"ACCEPT ",
"ADD ",
"ALTER ",
"CALL ",
"CANCEL ",
"CLOSE ",
"COMMIT",
"COMPUTE ",
"CONTINUE",
"DELETE ",
"DISPLAY ",
"DIVIDE ",
"ELSE",
"END-ADD",
"END-CALL",
"END-DELETE",
"END-DIVIDE",
"END-EVALUATE",
"END-EXEC",
"END-IF",
"END-MULTIPLY",
"END-PERFORM",
"END-READ",
"END-RETURN",
"END-REWRITE",
"END-SEARCH",
"END-START",
"END-STRING",
"END-SUBTRACT",
"END-UNSTRING",
"END-WRITE",
"EVALUATE ",
"EXEC SQL",
"EXIT",
"FETCH ",
"GOBACK",

```



```

"IF ",
"INITIALIZE ",
"INSPECT ",
"MOVE ",
"MULTIPLY ",
"NEXT ",
"OPEN ",
"PERFORM ",
"READ ",
"RETURN ",
"REWRITE ",
"ROLLBACK",
"SEARCH ",
"SET ",
"SORT ",
"START ",
"STOP ",
"STRING ",
"SUBTRACT ",
"UNSTRING ",
"WHEN ",
"WRITE ",
)
SQL_LEVEL_1_KEYWORDS = (
"SELECT ",
"INTO ",
"FROM ",
"WHERE ",
"ORDER BY ",
"GROUP BY ",
"HAVING ",
"FETCH ",
"OPEN ",
"CLOSE ",
"COMMIT",
"ROLLBACK",
"INSERT ",
"UPDATE ",
"DELETE ",
"SET ",
"VALUES ",
"FOR READ ONLY",
"QUERYNO ",
)
SQL_LEVEL_2_KEYWORDS = (
"AND ",
"OR ",
",",
)
AREA_A_HEADER_PREFIXES = (
"PROGRAM-ID.",
"AUTHOR.",
"INSTALLATION.",
"DATE-WRITTEN.",
"DATE-COMPILED.",
"SECURITY.",
)
AREA_A_DATA_LEVELS = {
"01",
"66",
"77",
}
AREA_B_DATA_LEVELS = {
"02",
"03",
"04",
"05",
"06",
"07",
"08",
"09",
"10",
"11",
"12",
"13",
"14",
}

```

```

"15",
"16",
"17",
"18",
"19",
"20",
"21",
"22",
"23",
"24",
"25",
"26",
"27",
"28",
"29",
"30",
"31",
"32",
"33",
"34",
"35",
"36",
"37",
"38",
"39",
"40",
"41",
"42",
"43",
"44",
"45",
"46",
"47",
"48",
"49",
"88",
}

```

```

--- Fixed-format line parser statement starters (appended) ---
COBOL statement starters used to confirm a trailing 8-digit token is a
right sequence number rather than part of the body. Text only.

```

```

COBOL_STATEMENT_STARTERS = (

```

```

 "ACCEPT ",
 "ADD ",
 "CALL ",
 "CLOSE ",
 "COMPUTE ",
 "CONTINUE",
 "DISPLAY ",
 "ELSE",
 "END-",
 "EVALUATE ",
 "EXEC ",
 "EXIT",
 "IF ",
 "INITIALIZE ",
 "MOVE ",
 "OPEN ",
 "PERFORM ",
 "READ ",
 "SET ",
 "STOP ",
 "TO ",
 "WHEN ",
 "WRITE ",

```

```

)

```

```

Marker used when detecting a generated right sequence (00000010 style).
RIGHT_SEQUENCE_GENERATED_PREFIX = "0000"

```

```

--- Division name + non-area-A indicator set (appended) ---
COBOL division that receives Area-B procedure indentation.
PROCEDURE_DIVISION_NAME = "PROCEDURE"

```

```

Indicators whose body must be preserved verbatim (not Area A/B reformatted):
comment (*), page (/), debug (D), and continuation (-).

```

```

PRESERVE_VERBATIM_INDICATORS = {
 COMMENT_INDICATOR,
 PAGE_INDICATOR,
 DEBUG_INDICATOR,
 CONTINUATION_INDICATOR,
}
--- Lowercase debug indicator (appended) ---
Some inputs use a lowercase 'd' in the indicator column; normalize to D.
DEBUG_INDICATOR_LOWER = "d"
--- Fixed-format wrapper tokens (appended) ---
COBOL statement prefixes and boolean operators used by the body wrapper.
WRAP_IF_PREFIX = "IF "
WRAP_MOVE_PREFIX = "MOVE "
WRAP_TO_PREFIX = "TO "
BOOLEAN_OPERATOR_WORDS = frozenset({"AND", "OR"})

```

```

=====
FILE: rules\fixed_format_wrap_rules.py
ENCODING: utf-8
=====

```

```

LOCATION: rules/fixed_format_wrap_rules.py
ACTION: CREATE NEW FILE

```

```

"""

```

```

Fixed-format statement wrapping rules.

```

```

Constants only. No regex, no runtime logic, no program, record, table,
DCLGEN, or host variable names.

```

```

A rewritten statement that no longer fits columns 8-72 must be WRAPPED,
never truncated and never silently skipped. Truncation produces an
undefined data name and fails compilation; skipping leaves an unqualified
reference that fails DB2 precompilation.
"""

```

```

from __future__ import annotations

```

```

Fixed-format geometry.
BODY_START_COLUMN = 8
BODY_END_COLUMN = 72
BODY_WIDTH = 65 # columns 8-72 inclusive

```

```

Continuation indent applied to wrapped fragments.
CONTINUATION_INDENT = " "

```

```

Statement tokens used by the MOVE splitter.
TOKEN_MOVE = "MOVE"
TOKEN_TO = "TO"
TOKEN_OF_DCL = " OF DCL"

```

```

COBOL sentence terminator.
PERIOD = "."

```

```

Statements that legitimately end a sentence and may carry the period.

```

```

SENTENCE_TERMINATORS = (
 "END-IF",
 "END-EVALUATE",
 "END-PERFORM",
 "END-READ",
 "END-WRITE",
 "END-EXEC",
 "CONTINUE",
)

```

```

Diagnostics.

```

```

DIAG_WRAPPED_TEMPLATE = (
 "Fixed format: wrapped rewritten statement that exceeded column "
 "{column}."
)

```

```

DIAG_PERIOD_RESTORED = (
 "Fixed format: restored sentence terminator lost when an enclosing "
 "block was removed."
)

```

```
=====
FILE: rules\generated_cobol_safety_rules.py
ENCODING: utf-8
=====
```

"""Constants for the generated-COBOL safety cleanup passes.

This module contains constants only.

No regex patterns, parser logic, service logic, or hardcoded program, record, table, cursor, or host variable names belong here.

Covers:

- paragraph sentence termination
  - output-write placement relative to a child cursor loop
  - Area B alignment of generated EXEC SQL / END-EXEC marker lines
- """

```
from __future__ import annotations
```

```
--- Paragraph termination -----
#
A COBOL paragraph is one or more sentences. If the last executable line of
a paragraph carries no period, the sentence runs into the next paragraph
header and the compiler rejects the program.
PARAGRAPH_TERMINATOR = "."
ENFORCE_PARAGRAPH_TERMINATION = True
```

```
--- Output-write placement -----
#
Manual reference shape:
#
PERFORM <open-child>
PERFORM <fetch-child> UNTIL <eoc> OR <early-stop-flag> = 'Y'
PERFORM <close-child>
#
IF NOT <early-stop-flag> = 'Y'
IF <status-field> = '<value>'
PERFORM <write-paragraph>
END-IF
END-IF.
#
A guarded WRITE left inside the per-row paragraph fires once per fetched
child row instead of once per parent row.
ENFORCE_OUTPUT_WRITE_AFTER_CHILD_LOOP = True
```

```
Working-storage flag written by the child-fetch early-stop cleanup.
EARLY_STOP_FLAG = "SW-STATUS-D"
EARLY_STOP_VALUE = "'Y'"
```

```
Guard emitted around a relocated write block. {flag} and {value} are
supplied by the service from the constants above.
EARLY_STOP_GUARD_OPEN = "IF NOT {flag} = {value}"
EARLY_STOP_GUARD_CLOSE = "END-IF"
```

```
Body indents, relative to column 8.
IND_STATEMENT = " " # column 12
IND_NESTED = " " # one nesting step
```

```
Look-ahead limit when scanning a paragraph for its trailing write block.
WRITE_BLOCK_SCAN_LIMIT = 200
```

```
--- Area B alignment of generated SQL markers -----
#
CHK-06.07 measures every generated DB2 block line and requires an indent of
at least 4 body spaces, i.e. physical column 12. The manual reference puts
EXEC SQL, END-EXEC, EVALUATE SQLCODE and END-EVALUATE at column 12.
ALIGN_GENERATED_SQL_MARKERS_EVERYWHERE = True
SQL_MARKER_MIN_INDENT = 4
SQL_MARKER_INDENT = " "
```

```
--- Diagnostics -----
SAFETY_MESSAGES = {
 "paragraph_terminated": (
 "Safety: closed unterminated sentence in paragraph {paragraph}."
),
}
```

```

 "write_block_moved": (
 "Safety: moved output write block from {source} to {target} after "
 "the child cursor loop."
),
 "sql_markers_aligned": (
 "Safety: aligned {count} generated EXEC SQL / END-EXEC marker "
 "line(s) to Area B."
),
 }
}

```

```

=====
FILE: rules\idms_transformer_messages.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

IDMS statement transformer message/generated-line templates.
All *DB2: text lives here so wording stays centralized.
No regex, no logic, no hardcoded program/record/table names.

---- Removed control / declarative ----
REMOVED_CONTROL_STATEMENT_TEMPLATE = (
 "*DB2: Removed residual IDMS control statement: {line}"
)
REMOVED_BIND_TEMPLATE = "*DB2: Removed IDMS BIND statement: {line}"
REMOVED_USAGE_READY_TEMPLATE = "*DB2: Removed IDMS usage/READY statement: {line}"
REMOVED_CONNECT_TEMPLATE = "*DB2: Removed IDMS CONNECT statement: {line}"
REMOVED_DISCONNECT_TEMPLATE = "*DB2: Removed IDMS DISCONNECT statement: {line}"
REMOVED_STATUS_ABORT_TEMPLATE = (
 "*DB2: Removed IDMS status/abort paragraph call: {line}"
)
REMOVED_FIND_CURRENT_TEMPLATE = (
 "*DB2: Removed IDMS FIND CURRENT statement: {line}"
)

---- FINISH / COMMIT ----
FINISH_OUTSIDE_PROCEDURE_TEMPLATE = (
 "*DB2: FINISH ignored outside PROCEDURE DIVISION: {line}"
)
FINISH_CONVERTED_TO_COMMIT = "*DB2: IDMS FINISH converted to COMMIT."
COMMIT_OUTSIDE_PROCEDURE_TEMPLATE = (
 "*DB2: COMMIT ignored outside PROCEDURE DIVISION: {line}"
)

---- OBTAIN CALC ----
OBTAIN_CALC_OUTSIDE_PROCEDURE_TEMPLATE = (
 "*DB2: OBTAIN CALC ignored outside PROCEDURE DIVISION: {line}"
)
REMOVED_OBTAIN_CALC_SELECT_TEMPLATE = (
 "*DB2: Removed OBTAIN CALC SELECT for {record}."
)
DIRECT_UPDATE_COMPOSITE_KEY = (
 "*DB2: Direct UPDATE will use mapped composite key WHERE clause."
)

---- OBTAIN FIRST / NEXT ----
OBTAIN_OUTSIDE_PROCEDURE_TEMPLATE = (
 "*DB2: OBTAIN ignored outside PROCEDURE DIVISION: {line}"
)
CONVERTED_OBTAIN_FIRST_TEMPLATE = (
 "*DB2: Converted OBTAIN FIRST {record} WITHIN {set_name}."
)
CONVERTED_OBTAIN_NEXT_TEMPLATE = (
 "*DB2: Converted OBTAIN NEXT {record} WITHIN {set_name}."
)

---- FIND FIRST ----
FIND_FIRST_OUTSIDE_PROCEDURE_TEMPLATE = (
 "*DB2: FIND FIRST ignored outside PROCEDURE DIVISION: {line}"
)
CONVERTED_FIND_FIRST_TEMPLATE = (
 "*DB2: Converted FIND FIRST {record} WITHIN {set_name}."
)

```

```

---- STORE / MODIFY / ERASE ----
STORE_MODIFY_ERASE_OUTSIDE_PROCEDURE_TEMPLATE = (
 "*DB2: {operation} ignored outside PROCEDURE DIVISION: {line}"
)

---- Generated PERFORM lines for cursor flow ----
PERFORM_OPEN_CURSOR_TEMPLATE = "PERFORM OPEN-{cursor}."
PERFORM_FETCH_CURSOR_TEMPLATE = "PERFORM FETCH-{cursor}."

---- Operation names (used in STORE/MODIFY/ERASE messaging) ----
OPERATION_STORE = "STORE"
OPERATION_MODIFY = "MODIFY"
OPERATION_ERASE = "ERASE"

=====
FILE: rules\idms_transformer_rules.py
ENCODING: utf-8
=====

from __future__ import annotations

IDMS statement transformer rule constants. Constants only.
No regex, no logic, no program/record/table names.

IDMS_TRANSFORMER_RULES = [
 "Convert only IDMS database statements and control statements.",
 "When removing executable PROCEDURE DIVISION IDMS code, add CONTINUE.",
 "Do not generate DB2 SQL for records without Sheet Mapping metadata.",
 "Do not add duplicate SQLCODE wrappers around generated SQL blocks.",
 "Do not hardcode program names, DB2 tables, columns, or host variables.",
]

COBOL division name that permits executable conversion.
PROCEDURE_DIVISION_NAME = "PROCEDURE"

Token used when an OBTAIN FIRST cursor keyword is detected.
OBTAIN_FIRST_KEYWORD = "FIRST"

Replacement condition for IDMS end-of-set / record-not-found tokens.
SQLCODE_END_TOKEN_REPLACEMENT = "SQLCODE = 100"

Generated CONTINUE line for emptied blocks.
CONTINUE_STATEMENT = "CONTINUE."

SQLCODE=100 guard block scaffolding (ON DB-REC-NOT-FOUND).
SQLCODE_100_IF_OPEN = "IF SQLCODE = 100"
SQLCODE_100_IF_CLOSE = "END-IF."
SQLCODE_100_CONTINUE_BODY = " CONTINUE"

=====
FILE: rules\lrf_conversion_rules.py
ENCODING: utf-8
=====

LOCATION: rules/lrf_conversion_rules.py
ACTION: CREATE NEW FILE
"""LRF -> classic IDMS expansion rules.

Constants only. No regex, no runtime logic, no program / record / table /
set / cursor / host variable names.

WHY THIS EXISTS

The converter already knows how to turn

 OBTAIN FIRST <record> WITHIN <set>.

into a DB2 cursor with OPEN / FETCH / CLOSE paragraphs, join predicates,
QUERYNO and an EOC flag.

It does NOT know the Logical Record Facility form

 OBTAIN FIRST <logical-record> WHERE <path-keyword>.
```

Teaching every downstream generator about logical records would duplicate that machinery. Instead, a single pre-pass rewrites the LRF form into the classic form using the parsed subschema path, and every existing rule applies unchanged.

"""

```
from __future__ import annotations
```

```
----- master
ENABLE_LRF_PATH_EXPANSION = True
```

```
Expand only when the driving element record resolves to a DB2 table.
REQUIRE_MAPPED_DRIVING_RECORD = True
```

```
----- expanded syntax
OBTAIN_FIRST_TEMPLATE = "OBTAIN FIRST {record} WITHIN {within}"
OBTAIN_NEXT_TEMPLATE = "OBTAIN NEXT {record} WITHIN {within}"
FIND_FIRST_TEMPLATE = "FIND FIRST {record} WITHIN {within}"
STATEMENT_TERMINATOR = "."
```

```
IDMS token the converter already maps to "SQLCODE = 100".
END_OF_SET_TOKEN = "DB-END-OF-SET"
RECORD_NOT_FOUND_TOKEN = "DB-REC-NOT-FOUND"
```

```
IDMS path status codes that mean "no more rows" / "not found".
STATUS_END_OF_SET = "0307"
STATUS_NOT_FOUND = "0326"
STATUS_SET_EMPTY = "1601"
```

```
----- LR-CTRL field
Declared by COPY IDMS SUBSCHEMA-LR-CTRL. Never emitted into DB2 output.
LR_STATUS_FIELD = "LR-STATUS"
LR_QUALIFIER = "OF LR"
```

```
----- comments
```

```
COMMENT_PREFIX = "*DB2: "
EXPANDED_OBTAIN_TEMPLATE = (
 "*DB2: Expanded LRF {mode} {lr} WHERE {keyword} "
 "-> {record} WITHIN {within}."
)
EXPANDED_PARENT_TEMPLATE = (
 "*DB2: LRF path {keyword} parent access on {record}."
)
SKIPPED_NO_PATH_TEMPLATE = (
 "*DB2-KEEP: LRF path keyword {keyword} not found in the supplied "
 "subschema - expand this access manually."
)
SKIPPED_NO_RECORD_TEMPLATE = (
 "*DB2-KEEP: LRF path {keyword} has no resolvable driving record "
 "- expand this access manually."
)
SKIPPED_UNMAPPED_TEMPLATE = (
 "*DB2-KEEP: LRF driving record {record} has no Sheet Mapping entry "
 "- expand this access manually."
)
REMOVED_LR_STATUS_TEMPLATE = (
 "*DB2: Replaced LR-STATUS test with the cursor end-of-cursor test."
)
```

```
----- diagnostics
```

```
LRF_EXPANSION_MESSAGES = {
 "expanded": (
 "LRF expansion: {mode} {lr} WHERE {keyword} expanded to "
 "{record} WITHIN {within}."
),
 "parent_emitted": (
 "LRF expansion: emitted parent access {record} WITHIN {within} "
 "for keyword {keyword}."
),
 "status_rewritten": (
 "LRF expansion: rewrote {count} LR-STATUS condition(s) to the "
 "IDMS end-of-set token."
),
 "qualifier_stripped": (
 "LRF expansion: stripped {count} 'OF LR' qualifier(s)."
),
}
```

```

"keyword_not_found": (
 "LRF expansion: keyword {keyword} is not declared in the supplied "
 "subschema; the statement was left for manual review."
),
"record_unmapped": (
 "LRF expansion: driving record {record} for keyword {keyword} has "
 "no Sheet Mapping entry; the statement was left for manual review."
),
"no_logical_records": (
 "LRF expansion: no logical record metadata supplied; LRF statements "
 "are left unchanged."
),
"summary": (
 "LRF expansion: expanded {expanded} statement(s), skipped {skipped}."
),
}

```

```

=====
FILE: rules\lrf_rules.py
ENCODING: utf-8
=====

```

```

LOCATION: rules/lrf_rules.py
ACTION: CREATE NEW FILE

```

```

from __future__ import annotations

```

```

LRF_RULES = [
 "Parse the subschema LRF source as metadata only; never execute it.",
 "A logical record is converted only when every element record is mapped.",
 "Do not fabricate a DB2 join when an element record has no Sheet Mapping entry.",
 "OBTAIN path groups drive read logic; ERASE path groups drive delete logic.",
 "Path status codes are diagnostics, not generated COBOL.",
]

```

```

Path-group verbs recognised in the subschema source.

```

```

PATH_GROUP_OBTAIN = "OBTAIN"
PATH_GROUP_ERASE = "ERASE"
PATH_GROUP_STORE = "STORE"
PATH_GROUP_MODIFY = "MODIFY"
SUPPORTED_PATH_GROUPS = (
 PATH_GROUP_OBTAIN,
 PATH_GROUP_ERASE,
 PATH_GROUP_STORE,
 PATH_GROUP_MODIFY,
)

```

```

IDMS status codes seen in the path descriptions.

```

```

STATUS_OK = "0000"
STATUS_END_OF_AREA = "0307"
STATUS_RECORD_NOT_FOUND = "0326"
STATUS_SET_NOT_EMPTY = "0230"
STATUS_SET_EMPTY = "1601"

```

```

UI / diagnostic labels.

```

```

LABEL_LRF = "LRF Subschema"
LABEL_LOGICAL_RECORDS = "Logical Records"
LRF_SOURCE_LABEL = "LRF files"

```

```

DIAG_LRF_NOT_UPLOADED = "LRF subschema file not uploaded."
DIAG_LRF_FOLDER_TEMPLATE = "LRF folder: {folder}"
DIAG_LRF_FILE_COUNT_TEMPLATE = "LRF file count: {count}"
DIAG_LRF_FILE_TEMPLATE = "LRF {index}: {path}"
DIAG_LRF_TEXT_LEN_TEMPLATE = "LRF {index} text length: {length}"
DIAG_LRF_TOTAL_RECORDS_TEMPLATE = "LRF logical records parsed: {count}"
DIAG_LRF_PATHS_TEMPLATE = "LRF paths parsed for {lr}: {count}"
DIAG_LRF_PARSE_FAILED_TEMPLATE = "LRF parse failed for {name}: {reason}"

```

```

LOG_LRF_FILE_COUNT = "LRF file count: %s"
LOG_LRF_TOTAL_RECORDS = "LRF logical records parsed: %s"

```

```

Validation wording (optional input - warning, never a blocker).

```

```

LRF_VALIDATION_MESSAGES = {
 "missing_lrf": (
 "Program uses COPY IDMS LR but no LRF subschema file was supplied. "

```



```

 "Logical record access will be skipped and flagged for manual review."
),
 "unmapped_element_template": (
 "Logical record {lr}: element record {record} has no Sheet Mapping "
 "entry. Join generation skipped - map this manually."
),
}

```

```

LOCATION: rules/lrf_rules.py
ACTION: APPEND at the end of the file

```

```

DIAG_LRF_READ_FAILED_TEMPLATE = "LRF read failed for {name}: {reason}"
DIAG_LRF_PARSED_ZERO = "LRF parsed logical records: 0"
LRF_UPLOAD_LABEL = "Optional LRF Subschema source file or files"
LRF_UPLOAD_HELP = (
 "Upload the subschema source containing ADD LOGICAL RECORD and "
 "ADD PATH-GROUP blocks (for example Sub Schema.txt). Required only "
 "for programs that use COPY IDMS LR."
)
LRF_METRIC_LABEL = "Logical Records"

```

```

LOCATION: rules/lrf_rules.py
ACTION: APPEND at the end of the file

```

```

DIAG_LRF_RECORDS_IN_SCOPE_TEMPLATE = (
 "LRF logical records in scope for this program: {count} ({names})"
)
DIAG_LRF_NO_RECORDS_IN_SCOPE = (
 "LRF logical records in scope for this program: 0. "
 "Logical record access, if any, will be reported for manual review."
)

```

```

=====
FILE: rules\manual_layout_rules.py
ENCODING: utf-8
=====

```

```

"""

```

Manual-style COBOL layout rules.

These constants describe final manual-style sequencing behavior.  
 No regex or runtime logic belongs in this file.

```

"""

```

```

MANUAL_LAYOUT_RULES = [
 "Final COBOL should be manual-style sequenced.",
 "Columns 1-6 contain the left sequence number.",
 "Columns 8-72 contain the COBOL body.",
 "Columns 73-80 contain the right sequence number.",
 "Generated output must preserve COBOL headers and division structure.",
]

```

```

=====
FILE: rules\manual_reference_rules.py
ENCODING: utf-8
=====

```

```

LOCATION: rules/manual_reference_rules.py
ACTION: CREATE NEW FILE
"""Manual COBOL reference standard constants.

```

Single source of truth for everything the COBOL team's manual reference  
 program dictates about generated output shape.

Constants only. No regex, no runtime logic, no program / record / table /  
 cursor / host variable names.

```

"""

```

```

from __future__ import annotations

```

```

Identification Division

```

```

REPLACE_IDENTIFICATION_HEADER = True

IDENTIFICATION_RULE_LINE = "*=====
PROGRAM_ID_TEMPLATE = "PROGRAM-ID. {program_id}."
AUTHOR_TEMPLATE = "AUTHOR. {author}."
DATE_WRITTEN_TEMPLATE = "DATE-WRITTEN. {date_written}."

Supplied by config, never hardcoded in a generator.
DEFAULT_AUTHOR = ""
DEFAULT_DATE_WRITTEN = ""

Comment banners

Full-width rule used above and below a generated paragraph group.
BANNER_RULE = "*-----*"
BANNER_TITLE_TEMPLATE = "* {title}"

BANNER_OPEN_CURSOR = "OPEN CURSOR"
BANNER_FETCH_CURSOR = "FETCH CURSOR"
BANNER_CLOSE_CURSOR = "CLOSE CURSOR"

Section banner used inside WORKING-STORAGE.
SECTION_BANNER_TEMPLATE = "*---- {title} {fill}*"
SECTION_BANNER_WIDTH = 64
SECTION_BANNER_FILL = "-"

SECTION_DB2_INCLUDE = "INCLUDE FOR DB2"
SECTION_DB2_VIEWS = "INCLUDE VIEWS"

Short banner used inside PROCEDURE DIVISION (4 stars, no closing star).
INLINE_BANNER_PREFIX = "****"
INLINE_BANNER_TEMPLATE = "**** {text}"

INLINE_BANNER_PARENT_READ = "READ PARENT TABLE {table}"
INLINE_BANNER_CHILD_READ = "READ CHILD TABLE {table}"
INLINE_BANNER_WRITE_CHECK = (
 "CHECK THE CONDITION FOR WRITING THE RECORD INTO OUTPUT FILE"
)

Cursor declaration banner.
CURSOR_BANNER_TITLE_TEMPLATE = "* DECLARE CURSOR FOR {short_name} TABLE"
CURSOR_BANNER_UNDERLINE = "* -----"
CURSOR_BANNER_BLANK = "*"

Debug (column 7 = 'D') trace lines

EMIT_DEBUG_TRACE_LINES = True
DEBUG_INDICATOR = "D"
DEBUG_TRACE_TEMPLATE = "DISPLAY '{paragraph}.'"

COMMIT policy

A retrieval program declares every cursor FOR READ ONLY and therefore
never needs an explicit COMMIT. Emitting one is harmless at run time
but diverges from the manual reference.
EMIT_COMMIT_IN_RETRIEVAL = False

Page eject

The manual reference separates major business paragraphs with a page
eject in the indicator column.
EMIT_PAGE_EJECT_BETWEEN_PARAGRAPHS = True
PAGE_EJECT_INDICATOR = "/"

=====
FILE: rules\mapping_scope_rules.py
ENCODING: utf-8
=====

LOCATION: rules/mapping_scope_rules.py

```

```
ACTION: CREATE NEW FILE
```

```
"""Mapping validation scope rules.
```

```
Constants only. No regex, no runtime logic, no program / record / table names.
```

```
WHY THIS EXISTS
```

```

```

```
MappingValidator walks the ENTIRE Sheet Mapping workbook and reports any table carrying rows but no usable DB2 columns. That is correct for a workbook audit and wrong for a per-program conversion report: a retrieval program is marked with two blocking errors for restart tables it never touches.
```

```
A mapping gap is only an ERROR for a table the program being converted actually references. For every other table it is an informational note about the workbook, and it belongs in the conversion log.
```

```
"""
```

```
from __future__ import annotations
```

```
ENFORCE_MAPPING_SCOPE_FILTER = True
```

```
Rewritten message for a gap on a table this program does not use.
```

```
The 'Mapping scope' prefix classifies as Info, so it drops into the # conversion log instead of the Errors band.
```

```
OUT_OF_SCOPE_TEMPLATE = (
```

```
 "Mapping scope: {table} carries no usable DB2 column mapping, but "
```

```
 "this program does not reference it. Workbook gap, not a conversion "
```

```
 "failure."
```

```
)
```

```
DIAG_SCOPE_APPLIED = (
```

```
 "Mapping scope: {out_of_scope} of {total} mapping gap(s) relate to "
```

```
 "tables this program does not reference."
```

```
)
```

```
=====
FILE: rules/name_derivation_rules.py
```

```
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
import re
```

```
Deterministic COBOL name-derivation rules.
```

```
#
```

```
Constants + a single pure regex here (this file is the authority for the # VM...BD... -> VMDZ... site transform). No program names, record names, # copybook names, DB2 tables, DCLGEN names, or host variables are hardcoded.
```

```
#
```

```
Rule (applies to PROGRAM-ID and to input record / copybook names):
```

```
VM <site>? BD <tail> -> VMDZ <site>? <tail>
```

```
#
```

```
Examples:
```

```
VM1BD567 -> VMDZ1567 (site '1' preserved after DZ)
```

```
VMBD205I -> VMDZ205I (no site char; BD simply becomes DZ)
```

```
#
```

```
Structure captured:
```

```
prefix = "VM" (fixed)
```

```
site = optional single character between VM and BD (e.g. "1")
```

```
marker = "BD" -> replaced by "DZ"
```

```
tail = everything after BD (e.g. "567", "205I")
```

```
NAME_DERIVATION_RULES = [
```

```
 "Derive DB2-era names from the VM...BD... source shape.",
```

```
 "Replace the BD marker with DZ.",
```

```
 "Preserve an optional single site character (VM<site>BD -> VMDZ<site>).",
```

```
 "Preserve the trailing characters unchanged.",
```

```
 "Do not hardcode any program, record, or copybook name.",
```

```
]
```

```
NAME_DERIVATION_PREFIX = "VM"
```

```
NAME_DERIVATION_SOURCE_MARKER = "BD"
NAME_DERIVATION_TARGET_MARKER = "DZ"
```

```
VM (site?) BD (tail)
site is at most one character; tail is the remainder.
VM_BD_NAME_PATTERN = re.compile(
 r"^(?P<prefix>VM)(?P<site>[A-Z0-9]?)"
 r"(?P<marker>BD)"
 r"(?P<tail>[A-Z0-9]+)$",
 flags=re.IGNORECASE,
)
```

```
=====
FILE: rules\output_write_paragraph_rules.py
ENCODING: utf-8
=====
```

```
LOCATION: rules/output_write_paragraph_rules.py
ACTION: CREATE NEW FILE
```

```
"""Output write paragraph extraction rules.
```

```
Constants only. No regex, no runtime logic, no program / record / table /
cursor / paragraph / host variable names.
```

```
Manual reference shape:
```

```
PERFORM 830-CLOSE-DZEVEFC1
```

```
IF NOT SW-STATUS-D = 'Y'
 IF WS-STATUS = 'C'
 PERFORM WRITE-UITRECORD
 ADD 1 TO WS-NB-OUTPUT-COUNT
 END-IF
END-IF.
```

```
WRITE-UITRECORD.
 INITIALIZE UITRECORD
 MOVE ... TO ...
 WRITE UITRECORD.
```

```
"""
```

```
from __future__ import annotations
```

```
=====
Master switch
=====
ENFORCE_OUTPUT_WRITE_PARAGRAPH = True
```

```
=====
Generated paragraph name
=====
Derived from the COBOL record written, never hardcoded:
WRITE UITRECORD -> WRITE-UITRECORD
WRITE_PARAGRAPH_TEMPLATE = "WRITE-{record}"
WRITE_PARAGRAPH_HEADER_TEMPLATE = "{paragraph}."
PERFORM_TEMPLATE = "PERFORM {paragraph}"
```

```
=====
Extraction safety
=====
A block is extracted only when it contains exactly this many WRITE
statements. Two writes in one guard is a business decision, not a
formatting one, and must stay where the programmer put it.
REQUIRED_WRITE_COUNT = 1
```

```
Minimum executable body lines that make extraction worthwhile.
MINIMUM_BODY_LINES = 2
```

```
Maximum lines scanned forward when matching an IF to its END-IF.
IF_SCAN_LIMIT = 200
```

```
Maximum extraction passes over one program.
EXTRACTION_PASS_LIMIT = 8
```

```
Single-word lines that end in a period but are NOT paragraph headers.
SCOPE_TERMINATOR_WORDS = frozenset(
 {
 "CONTINUE", "EXIT", "GOBACK", "STOP",
 "END-EXEC", "END-EVALUATE", "END-IF", "END-PERFORM",
 "END-READ", "END-SEARCH", "END-STRING", "END-WRITE",
 }
)
```

```
=====
Layout
=====
IND_STATEMENT = " " # column 12, Area B
PARAGRAPH_TERMINATOR = "."
BLANK_LINE = ""
```

```
Page eject placed before the generated paragraph, matching the manual
reference separation between major paragraphs.
EMIT_PAGE_EJECT_BEFORE_PARAGRAPH = True
PAGE_EJECT_INDICATOR = "/"
```

```
=====
Counter increment
=====
The manual reference counts written records at the CALL SITE, not
inside the write paragraph, so the counter reflects guard decisions.
EMIT_OUTPUT_COUNTER_INCREMENT = True
OUTPUT_COUNTER_NAME = "WS-NB-OUTPUT-COUNT"
COUNTER_ADD_TEMPLATE = "ADD 1 TO {name}"
```

```
=====
Diagnostics
=====
OUTPUT_WRITE_PARAGRAPH_MESSAGES = {
 "extracted": (
 "Output write: extracted record population from {source} into "
 "paragraph {paragraph}."
),
 "counter_added": (
 "Output write: added {counter} increment at the {paragraph} "
 "call site."
),
 "skipped_multiple_writes": (
 "Output write: {source} holds {count} WRITE statements; "
 "extraction skipped."
),
 "skipped_name_taken": (
 "Output write: paragraph {paragraph} already exists; "
 "extraction skipped."
),
 "skipped_too_small": (
 "Output write: block in {source} is too small to extract."
),
}
```

```
=====
FILE: rules\procedure_indent_rules.py
ENCODING: utf-8
=====
```

```
LOCATION: rules/procedure_indent_rules.py
ACTION: CREATE NEW FILE
```

```
""""Procedure Division indentation rules.
```

Constants only. No regex, no runtime logic, no program / paragraph / record / table / cursor / host variable names.

Indentation model, measured from the COBOL team's manual reference program. All values are BODY indents, relative to column 8.

009790	IF NOT SW-STATUS-D = 'Y'	body indent 4 (col 12)
009800	IF WS-STATUS = 'C'	body indent 7 (col 15)
009900	PERFORM WRITE-UITRECORD	body indent 10 (col 18)
010100	END-IF	body indent 7

010101	END-IF.	body indent 4
011300	EXEC SQL	body indent 4
011400	OPEN DZBEFFC1	body indent 6
011500	END-EXEC	body indent 4
011700	EVALUATE SQLCODE	body indent 4
011800	WHEN ZERO	body indent 6
011900	SET ... TO TRUE	body indent 11
012300	END-EVALUATE	body indent 4
012400	.	body indent 4

"""

```
from __future__ import annotations
```

```
=====
Master switch
=====
ENFORCE_PROCEDURE_INDENT = True

=====
Indent geometry, relative to column 8
=====
AREA_A_INDENT = 0 # column 8 - paragraph and section headers
AREA_B_INDENT = 4 # column 12 - statements at depth 0
NEST_STEP = 3 # each IF / inline PERFORM level

EVALUATE offsets, relative to the EVALUATE's own indent.
WHEN_OFFSET = 2 # WHEN sits 2 past EVALUATE
WHEN_BODY_OFFSET = 7 # statements under a WHEN sit 7 past EVALUATE

EXEC SQL body offset, relative to the EXEC SQL line's own indent.
SQL_BODY_OFFSET = 2

A lone period closing a paragraph.
LONE_PERIOD = "."
LONE_PERIOD_INDENT = AREA_B_INDENT

=====
Safety limits
=====
Deeper than this and the depth tracker has almost certainly lost sync
with a malformed program. Re-indenting further would do more harm than
good, so the pass falls back to leaving lines untouched.
MAX_NESTING_DEPTH = 12

=====
Block classification
=====
BLOCK_KIND_IF = "IF"
BLOCK_KIND_EVALUATE = "EVALUATE"
BLOCK_KIND_PERFORM = "PERFORM"
BLOCK_KIND_SQL = "SQL"
BLOCK_KIND_OTHER = "OTHER"

Scope terminators mapped to the opener they close.
SCOPE_TERMINATORS = {
 "END-IF": BLOCK_KIND_IF,
 "END-EVALUATE": BLOCK_KIND_EVALUATE,
 "END-PERFORM": BLOCK_KIND_PERFORM,
 "END-EXEC": BLOCK_KIND_SQL,
 "END-READ": BLOCK_KIND_OTHER,
 "END-SEARCH": BLOCK_KIND_OTHER,
 "END-STRING": BLOCK_KIND_OTHER,
 "END-UNSTRING": BLOCK_KIND_OTHER,
 "END-WRITE": BLOCK_KIND_OTHER,
 "END-CALL": BLOCK_KIND_OTHER,
 "END-ADD": BLOCK_KIND_OTHER,
 "END-SUBTRACT": BLOCK_KIND_OTHER,
 "END-MULTIPLY": BLOCK_KIND_OTHER,
 "END-DIVIDE": BLOCK_KIND_OTHER,
 "END-COMPUTE": BLOCK_KIND_OTHER,
}

Verbs that open an explicitly scoped block.
SCOPED_OPENERS = {
```

```

 "READ": BLOCK_KIND_OTHER,
 "SEARCH": BLOCK_KIND_OTHER,
 "STRING": BLOCK_KIND_OTHER,
 "UNSTRING": BLOCK_KIND_OTHER,
}

Single-word lines ending in a period that are NOT paragraph headers.
NON_PARAGRAPH_SINGLE_WORDS = frozenset(
 {
 "CONTINUE", "EXIT", "GOBACK", "STOP",
 "END-IF", "END-EVALUATE", "END-PERFORM", "END-EXEC",
 "END-READ", "END-SEARCH", "END-STRING", "END-UNSTRING",
 "END-WRITE", "END-CALL", "END-ADD", "END-SUBTRACT",
 "END-MULTIPLY", "END-DIVIDE", "END-COMPUTE",
 "ELSE", "EJECT", "SKIP1", "SKIP2", "SKIP3",
 }
)

=====
Diagnostics
=====
PROCEDURE_INDENT_MESSAGES = {
 "normalized": (
 "Procedure indent: normalized {count} statement line(s) to Area B."
),
 "wrapped": (
 "Procedure indent: wrapped {count} line(s) that no longer fit "
 "columns 8-72 after re-indenting."
),
 "skipped_depth": (
 "Procedure indent: nesting depth exceeded {limit} at line {line}; "
 "remaining lines left untouched."
),
 "skipped_unbalanced": (
 "Procedure indent: unbalanced scope terminator '{token}' in "
 "paragraph {paragraph}; block left untouched."
),
}

ENFORCE_AREA_ALIGNMENT_REFLOW = False

=====
FILE: rules\program_flow_rules.py
ENCODING: utf-8
=====

from __future__ import annotations

"""
Program flow analysis rules.

This module contains constants only.
No regex patterns, parser logic, analyzer logic, program names, DB2 table names,
DCLGEN names, or host variables belong here.
"""

PROGRAM_FLOW_ANALYSIS_RULES = [
 "Analyze COBOL flow for diagnostics only.",
 "Do not rewrite COBOL.",
 "Detect procedure paragraphs.",
 "Detect cursor-like IDMS operations from parsed operations.",
 "Detect output write statements.",
 "Detect basic date usage hints.",
 "Do not hardcode program names, DB2 tables, DB2 columns, DCLGEN groups, or host variables.",
]

CURSOR_OPERATIONS = {
 "OBTAIN_FIRST",
 "OBTAIN_NEXT",
 "FIND_FIRST",
}

```

```

WRITE_TOKENS = {
 "WRITE",
 "STORE",
 "MODIFY",
 "ERASE",
}

```

```

DATE_TOKENS = {
 "DATE",
 "TIME",
 "TIMESTAMP",
 "TS-",
}

```

```

=====
FILE: rules\record_materialisation_rules.py
ENCODING: utf-8
=====

```

```

LOCATION: rules/record_materialisation_rules.py

```

```

from __future__ import annotations

```

```

=====
Master switch
=====
ENFORCE_RECORD_MATERIALISATION = True

```

```

A record is materialised only when EVERY COBOL field in the Sheet
Mapping resolves to a DCLGEN host variable. One unresolved field means
the written record would carry a silent hole, so the whole conversion
is refused and the original move is left for manual review.
REQUIRE_COMPLETE_FIELD_COVERAGE = True

```

```

The materialised layout must occupy exactly the same number of bytes as
the PIC it replaces. A mismatch shifts every field after it in a
fixed-length record, which is silent data corruption.
ENFORCE_LENGTH_MATCH = True

```

```

Byte tolerance. 0 = exact. Raise only with COBOL-team agreement.
LENGTH_TOLERANCE = 0

```

```

=====
Layout generation
=====
LAYOUT_BASE_INDENT = " " # column 12, Area B
LAYOUT_LEVEL_STEP = " " # one nesting step
LAYOUT_NAME_WIDTH = 24

```

```

GROUP_ITEM_TEMPLATE = "{level} {name}."
ELEMENTARY_ITEM_TEMPLATE = "{level} {name:<{width}} {picture}"
REDEFINES_TEMPLATE = "{level} {name} REDEFINES {base}{picture}"

```

```

LAYOUT_MARKER_TEMPLATE = "DB2 MATERIALISED IDMS RECORD {record}"

```

```

A mapping row with no PIC clause is a group item.
GROUP_PICTURE_TOKENS = ("GROUP", "")

```

```

=====
Move generation
=====
MOVE_MARKER_TEMPLATE = (
 "*DB2: Materialised {record} from {group} - {count} field move(s)."
)
MOVE_TEMPLATE = "MOVE {host} TO"
MOVE_TARGET_TEMPLATE = " {field} OF {record}"
MOVE_INDENT = " "

```

```

EMIT_DATE_CONVERSION = True

```

```

DATE_LOW_VALUE_LITERAL = "01.01.0001"
DATE_HIGH_VALUE_LITERAL = "31.12.9999"
DATE_HIGH_NUMERIC_LITERAL = "99999999"

```



```

DATE_CONVERSION_LINE_TEMPLATES = [
 "MOVE {host} TO DA-DD-MM-CCYY",
 "EVALUATE TRUE",
 " WHEN DA-DD-MM-CCYY = '{low_value}'",
 " MOVE ZEROES TO DA-CCYYMMDD-R",
 " WHEN DA-DD-MM-CCYY = '{high_value}'",
 " MOVE '{high_numeric}' TO DA-CCYYMMDD-R",
 " WHEN OTHER",
 " MOVE CCYY OF DA-DD-MM-CCYY TO CCYY OF DA-CCYYMMDD-R",
 " MOVE MM OF DA-DD-MM-CCYY TO MM OF DA-CCYYMMDD-R",
 " MOVE DD OF DA-DD-MM-CCYY TO DD OF DA-CCYYMMDD-R",
 "END-EVALUATE",
 "MOVE DA-CCYYMMDD-R TO {field} OF {record}",
]

DB2 types that need the conversion block. TIMESTAMP and TIME receive a
formatted timestamp, never a DD.MM.CCYY date.
DATE_DB2_TYPE_PREFIX = "DATE"
NON_DATE_DB2_TYPE_PREFIXES = ("TIMESTAMP", "TIME")

=====
Diagnostics
=====
RECORD_MATERIALISATION_MESSAGES = {
 "materialised": (
 "Record materialisation: expanded {target} into the {record} "
 "layout ({fields} field(s), {bytes} bytes) and generated "
 "{moves} move statement(s).",
),
 "layout_inserted": (
 "Record materialisation: inserted the {record} layout under "
 "{target}.",
),
 "skipped_no_mapping": (
 "Record materialisation: {record} has no Sheet Mapping rows; the "
 "whole-record move was left for manual review.",
),
 "skipped_incomplete": (
 "Record materialisation: {record} has {missing} field(s) with no "
 "DCLGEN host variable ({names}); the whole-record move was left "
 "for manual review.",
),
 "skipped_length": (
 "Record materialisation: {record} layout is {actual} bytes but "
 "{target} declares {expected}; refused rather than shift the "
 "record.",
),
 "skipped_no_target": (
 "Record materialisation: target field {target} was not found in "
 "the DATA DIVISION.",
),
 "skipped_no_group": (
 "Record materialisation: no DCLGEN group resolved for {record}.",
),
}

---- A: length ----
ALLOW_LAYOUT_SHORTER_THAN_TARGET = True
EMIT_REMAINDER_FILLER = True
REMAINDER_FILLER_NAME = "FILLER"
REMAINDER_FILLER_PICTURE_TEMPLATE = "PIC X({bytes})"

---- C: sentinels ----
Field names that are storage placeholders, never data carriers.
FILLER_FIELD_NAMES = ("FILLER",)

Values a mapping sheet puts in the DB2 column cell meaning "no column".
Compared upper-cased after strip.
NON_COLUMN_SENTINELS = ("", "FILLER", "N/A", "NA", "-", "NONE")

---- E: scan ----
Runaway guard on the number of whole-record MOVES rewritten per program.
MAX_RECORD_MOVES = 64

Suffixes identifying a work/staging field rather than an IDMS record

```

```

(for example a LINKAGE SECTION date field). Suffix driven, so no record
name is hardcoded.
NON_RECORD_NAME_SUFFIXES = ("-LS", "-WS", "-R", "-X")

---- level rebasing ----
The record group sits one level below the target field it replaces.
RECORD_GROUP_LEVEL_STEP = 1
MIN_LEVEL = 1
MAX_LEVEL = 49

---- new messages ----
RECORD_MATERIALISATION_MESSAGES.update({
 "remainder_filler": (
 "Record materialisation: {record} occupies {actual} of {expected} "
 "byte(s) in {target}; {remainder} trailing byte(s) declared as "
 "FILLER so the record length is unchanged."
),
 "skipped_too_long": (
 "Record materialisation: {record} layout is {actual} bytes but "
 "{target} declares only {expected}; refused rather than shift the "
 "record."
),
 "skipped_not_a_record": (
 "Record materialisation: {record} is a work field, not a mapped "
 "IDMS record; the move was left unchanged."
),
 "declared_only": (
 "Record materialisation: {record} produced {fields} declared "
 "field(s) but 0 movable field(s); the layout was emitted and no "
 "data is populated. Check the Sheet Mapping DB2 column cells."
),
 "no_moves": (
 "Record materialisation: no whole-record MOVE statement was found "
 "in the PROCEDURE DIVISION; nothing to materialise."
),
 "scan_summary": (
 "Record materialisation: {examined} whole-record move(s) examined, "
 "{applied} materialised, {refused} left for manual review."
),
 "composer_absent": (
 "Record materialisation: the composer is not wired into this "
 "build; every whole-record MOVE was left unchanged."
),
})

---- Fixed-format geometry (cols 8-72) ----
LAYOUT_BODY_WIDTH = 65

Area A is columns 8-11; levels 02-49 must start at column 12 or later.
4 spaces from column 8 lands the level number at column 12.
LAYOUT_BASE_INDENT = " "

2 spaces per nesting level, not 4. VMBFAS nests 6 deep; a 4-space step
put a level-11 entry at column 29 and the picture past column 72.
LAYOUT_LEVEL_STEP = " "

Hard ceiling on nesting indent, so no depth can overflow the body.
LAYOUT_MAX_INDENT_DEPTH = 6

Column at which the PICTURE starts, relative to the start of the body.
Reduced automatically when the entry would not fit.
LAYOUT_NAME_WIDTH = 24
LAYOUT_MIN_NAME_GAP = 1

---- Entry templates ----
The trailing period is added by the generator, never by the template.
GROUP_ITEM_TEMPLATE = "{level} {name}."
ELEMENTARY_ITEM_TEMPLATE = "{level} {name:<{width}}{picture}"

DEFECT: the old template had no separator between {base} and {picture}.
REDEFINES_TEMPLATE = "{level} {name} REDEFINES {base}{gap}{picture}"

LAYOUT_MARKER_TEMPLATE = "DB2 MATERIALISED IDMS RECORD {record}"

MOVE_TEMPLATE = "MOVE {host} TO"

```

```

MOVE_TARGET_TEMPLATE = " {field} OF {record}"

---- REDEFINES wrapping ----
WRAP_REDEFINES_ENTRY = True

A single-line REDEFINES longer than this is split. 58 leaves headroom
inside the 65-column body for the period and the sequence gap.
REDEFINES_INLINE_LIMIT = 58

Head line carries level, name and the REDEFINES clause.
REDEFINES_HEAD_TEMPLATE = "{level} {name}{gap}REDEFINES {base}"

The picture continues on the next line, aligned under the data-name.
REDEFINES_CONTINUATION_GAP = 4

---- MOVE continuation ----
Applied by RecordMoveGenerator in code. Five spaces places the target
under the source, matching the manual reference.
MOVE_CONTINUATION_INDENT = " "

A MOVE short enough to fit on one line is emitted on one line.
MOVE_INLINE_LIMIT = 56

---- Diagnostic for a target with no readable PIC ----
RECORD_MATERIALISATION_MESSAGES.update({
 "no_declared_length": (
 "Record materialisation: {target} carries no readable PIC "
 "length, so the {record} remainder FILLER could not be sized. "
 "The record may be shorter than {target} declares - verify the "
 "written record length."
),
})

RECORD_MATERIALISATION_MESSAGES.update({
 "refusal_summary": (
 "Record materialisation: {count} record(s) refused - {details}. "
 "The whole-record MOVE was left in place and will be commented "
 "by the structural safety pass."
),
 "target_measured": (
 "Record materialisation: target {target} measured at {bytes} "
 "byte(s); {record} layout is {actual} byte(s)."
),
})

Emit the per-target measurement on every attempt. It is two numbers
and it makes a length refusal self-explanatory without a code read.
EMIT_TARGET_MEASUREMENT = True

=====
FILE: rules\retrieval_runner_messages.py
ENCODING: utf-8
=====

from __future__ import annotations

Retrieval runner message templates.
#
All console and diagnostic text lives here so wording stays centralized.
No regex, no runtime logic, no hardcoded program/record/table names.

---- Validation labels ----
LABEL_MAPPING_SHEET = "Mapping Sheet"
LABEL_SHEET_MAPPING = "Sheet Mapping"
LABEL_RETRIEVAL_PROGRAM = "Retrieval Program"
LABEL_DCLGEN_TEMPLATE = "DCLGEN {index}"
LABEL_COPYBOOK_TEMPLATE = "Copybook {index}"
LABEL_RETRIEVAL_PROGRAM_TEMPLATE = "Retrieval Program {index}"

---- Validation errors ----
FILE_NOT_FOUND_TEMPLATE = "{label} file not found: {path}"
PATH_NOT_FILE_TEMPLATE = "{label} path is not a file: {path}"
FOLDER_NOT_FOUND_TEMPLATE = "{label} folder not found: {path}"
PATH_NOT_FOLDER_TEMPLATE = "{label} path is not a folder: {path}"

```

```

NO_MAPPING_FILE_TEMPLATE = "No mapping sheet file found in: {folder}"
NO_PROGRAM_FILE_TEMPLATE = "No retrieval program file found in: {folder}"
NO_DCLGEN_CANDIDATE_TEMPLATE = (
 "At least one DCLGEN file path is required. No DCLGEN candidate file was "
 "found. Searched:\n{searched}"
)

---- Shared input diagnostics ----
DIAG_START_LOAD_INPUTS = "START LOAD INPUTS"
DIAG_MAPPING_FOLDER_TEMPLATE = "Sheet Mapping folder: {folder}"
DIAG_MAPPING_SELECTED_TEMPLATE = "Sheet Mapping selected: {path}"
DIAG_MAPPING_ROWS_TEMPLATE = "Sheet Mapping rows: {count}"
DIAG_DCLGEN_FILE_COUNT_TEMPLATE = "DCLGEN file count: {count}"
DIAG_DCLGEN_FILE_TEMPLATE = "DCLGEN {index}: {path}"
DIAG_DCLGEN_TEXT_LEN_TEMPLATE = "DCLGEN {index} text length: {length}"
DIAG_DCLGEN_TOTAL_COLUMNS_TEMPLATE = "DCLGEN total columns: {count}"
DIAG_COPYBOOK_FILE_COUNT_TEMPLATE = "Copybook file count: {count}"
DIAG_COPYBOOK_FILE_TEMPLATE = "Copybook {index}: {path}"
DIAG_COPYBOOK_TEXT_LEN_TEMPLATE = "Copybook {index} text length: {length}"
DIAG_COPYBOOK_TOTAL_FIELDS_TEMPLATE = "Copybook total fields: {count}"

Parser source labels.
COPYBOOK_SOURCE_LABEL = "Copybook files"
DCLGEN_SOURCE_LABEL_TEMPLATE = "DCLGEN file {index}"

---- Log format strings (printf-style for logger) ----
LOG_MAPPING_SELECTED = "Sheet Mapping selected: %s"
LOG_MAPPING_ROWS = "Sheet Mapping rows: %s"
LOG_DCLGEN_FILE_COUNT = "DCLGEN file count: %s"
LOG_DCLGEN_FILE = "DCLGEN %s: %s"
LOG_DCLGEN_TEXT_LEN = "DCLGEN %s text length: %s"
LOG_DCLGEN_TOTAL_COLUMNS = "DCLGEN total columns: %s"
LOG_COPYBOOK_FILE_COUNT = "Copybook file count: %s"
LOG_COPYBOOK_FILE = "Copybook %s: %s"
LOG_COPYBOOK_TEXT_LEN = "Copybook %s text length: %s"
LOG_COPYBOOK_TOTAL_FIELDS = "Copybook total fields: %s"

---- Per-program conversion ----
DIAG_SOURCE_FILE_TEMPLATE = "IDMS COBOL source file: {path}"
DIAG_SOURCE_LEN_TEMPLATE = "IDMS COBOL source text length: {length}"
DIAG_DERIVED_PROGRAM_ID_TEMPLATE = "Derived target PROGRAM-ID: {value}"

LOG_SOURCE_FILE = "IDMS COBOL source file: %s"
LOG_SOURCE_LEN = "IDMS COBOL source text length: %s"
LOG_DERIVED_PROGRAM_ID = "Derived target PROGRAM-ID: %s"
LOG_GENERATION_COMPLETED = "Retrieval DB2 COBOL generation completed."
LOG_OUTPUT_CREATED = "Output file created: %s"

PRESERVE_SOURCE_LABEL = "(preserve source)"

---- Console output ----
PRINT_GENERATION_COMPLETED = "Retrieval DB2 COBOL generation completed."
PRINT_INPUT_PROGRAM_TEMPLATE = "Input program: {path}"
PRINT_OUTPUT_CREATED_TEMPLATE = "Output file created: {path}"

HEADER_INPUT_SUMMARY = "Input Summary"
HEADER_SELECTED_INPUT_FILES = "Selected Input Files"
HEADER_VALIDATION_MESSAGES = "Validation Messages"
HEADER_DIAGNOSTICS = "Diagnostics"
HEADER_RETRIEVAL_BATCH_SUMMARY = "Retrieval Batch Summary"

RULE_INPUT_SUMMARY = "-----"
RULE_SELECTED_INPUT_FILES = "-----"
RULE_VALIDATION_MESSAGES = "-----"
RULE_DIAGNOSTICS = "-----"
RULE_RETRIEVAL_BATCH_SUMMARY = "-----"

SUMMARY_PROJECT_ROOT_TEMPLATE = "Project Root : {value}"
SUMMARY_SRC_DIR_TEMPLATE = "SRC Directory : {value}"
SUMMARY_SHEET_ROWS_TEMPLATE = "Sheet Mapping Rows : {value}"
SUMMARY_DCLGEN_COLUMNS_TEMPLATE = "DCLGEN Columns : {value}"
SUMMARY_COPYBOOK_FIELDS_TEMPLATE = "Copybook Fields : {value}"
SUMMARY_COBOL_LEN_TEMPLATE = "COBOL Text Length : {value}"
SUMMARY_TARGET_PROGRAM_ID_TEMPLATE = "Target PROGRAM-ID : {value}"

SELECTED_SHEET_MAPPING_TEMPLATE = "Sheet Mapping : {value}"

```

```

SELECTED_COBOL_SOURCE_TEMPLATE = "IDMS COBOL Source : {value}"
SELECTED_DCLGEN_HEADER = "DCLGEN Files:"
SELECTED_COPYBOOK_HEADER = "Copybook Files:"
SELECTED_COPYBOOK_NONE = "Copybook Files : None"
SELECTED_PATH_BULLET_TEMPLATE = " - {value}"

BATCH_PROGRAM_FOLDER_TEMPLATE = "Program Folder : {value}"
BATCH_PROGRAM_COUNT_TEMPLATE = "Program Count : {value}"
BATCH_OUTPUT_FOLDER_TEMPLATE = "Output Folder : {value}"

NO_VALIDATION_MESSAGES = "No validation messages."
BULLET_TEMPLATE = "- {text}"

```

```

=====
FILE: rules\retrieval_runner_rules.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

Retrieval runner rule constants.
#
Constants only. No regex, no runtime logic, no program/record/table names.
Change a value here and it reflects across the retrieval runner.

RETRIEVAL_RUNNER_RULES = [
 "Derive the target PROGRAM-ID from the source (VM...BD... -> VMDZ...).",
 "Preserve the source PROGRAM-ID when no PROGRAM-ID is detected.",
 "Load Sheet Mapping, DCLGEN, and Copybook inputs once and share them.",
 "Do not hardcode program names, DB2 tables, columns, or host variables.",
]

Converter behavior defaults.
AUTO_FIX_PIC_LENGTH_MISMATCHES_DEFAULT = False

Output file naming.
OUTPUT_TIMESTAMP_FORMAT = "%d-%m-%Y_%H%M%S"
OUTPUT_FILE_EXTENSION = ".cbl"
OUTPUT_FILE_NAME_TEMPLATE = "{stem}_{timestamp}{extension}"

Text encoding used when reading/writing COBOL files.
COBOL_TEXT_ENCODING = "utf-8"

Logger name for the retrieval conversion run.
RETRIEVAL_LOGGER_NAME = "idms_db2_retrieval_conversion"

```

```

=====
FILE: rules\sequence_artifact_rules.py
ENCODING: utf-8
=====

```

```

LOCATION: rules/sequence_artifact_rules.py
ACTION: CREATE NEW FILE

```

```

"""Sequence artifact cleanup rules.

```

```

Constants only. No regex, no runtime logic, no program / record / table /
cursor / paragraph / host variable names.
"""

```

```

from __future__ import annotations

=====
Master switch
=====
ENFORCE_SEQUENCE_ARTIFACT_CLEANUP = True

=====
Sentence repair
=====
Removing an artifact can strip the period that closed the preceding
sentence. Without repair the sentence runs into the next paragraph
header and the compiler rejects the program.
REPAIR_EXPOSED_SENTENCE = True

```

```
PARAGRAPH_TERMINATOR = "."
```

```
A period is added only after a statement that can legitimately end a
COBOL sentence. Adding one after, say, a WHEN branch would close an
EVALUATE scope early and orphan END-EVALUATE.
```

```
SENTENCE_CARRIERS = frozenset(
```

```
{
 "END-IF",
 "END-EVALUATE",
 "END-PERFORM",
 "END-READ",
 "END-SEARCH",
 "END-STRING",
 "END-WRITE",
 "CONTINUE",
 "WRITE",
 "MOVE",
 "ADD",
 "SUBTRACT",
 "COMPUTE",
 "PERFORM",
 "DISPLAY",
 "CALL",
 "INITIALIZE",
 "SET",
 "OPEN",
 "CLOSE",
 "READ",
 "REWRITE",
 "GOBACK",
 "STOP",
})
```

```
=====
```

```
Diagnostics
```

```
=====
```

```
SEQUENCE_ARTIFACT_MESSAGES = {
```

```
 "removed": (
 "Sequence artifact: removed stale source sequence line '{body}'."
),
 "terminated": (
 "Sequence artifact: restored sentence terminator after "
 "'{statement}'."
),
 "summary": (
 "Sequence artifact: removed {removed} artifact line(s), "
 "repaired {repaired} sentence(s)."
),
}
```

```
=====
```

```
FILE: rules\sheet_mapping_parser_rules.py
```

```
ENCODING: utf-8
```

```
=====
```

```
from __future__ import annotations
```

```
Sheet Mapping parser constants. Constants only.
```

```
No regex, no parser logic, no column/alias names (those live in catalogs).
```

```
--- File handling ---
```

```
CSV_TEXT_ENCODING = "utf-8-sig"
```

```
XLSX_SUFFIX = ".xlsx"
```

```
LEGACY_XLS_SUFFIX = ".xls"
```

```
--- Magic numbers ---
```

```
CSV_SAMPLE_LENGTH = 500 # chars of CSV shown in the sample diagnostic
```

```
HEADER_SCAN_ROW_LIMIT = 100 # rows scanned when locating the header row
```

```
HEADER_DEBUG_ROW_LIMIT = 10 # rows for which normalized cells are logged
```

```
--- Diagnostic messages ---
```

```
DIAG_NO_FILE = "No Sheet Mapping file supplied."
```

```
DIAG_FILE_NAME_TEMPLATE = "Sheet Mapping file name: {name}"
```

```
DIAG_FILE_SIZE_TEMPLATE = "Sheet Mapping file size bytes: {size}"
```

```

DIAG_UNSUPPORTED_XLS = (
 "Unsupported .xls file detected. Save the file as .xlsx or .csv."
)
DIAG_CSV_DECODED_LEN_TEMPLATE = "CSV/text decoded length: {length}"
DIAG_CSV_SAMPLE_TEMPLATE = "CSV/text sample: {sample}"
DIAG_CSV_EMPTY = "CSV/text Sheet Mapping is empty."
DIAG_CSV_NO_ROWS = "CSV/text Sheet Mapping has no rows."
DIAG_CSV_HEADERS_TEMPLATE = "CSV detected headers: {headers}"
DIAG_CSV_USEFUL_ROWS_TEMPLATE = "CSV parsed useful rows: {count}"
DIAG_XLSX_EMPTY = "XLSX Sheet Mapping is empty."
DIAG_XLSX_USEFUL_ROWS_TEMPLATE = "XLSX parsed useful rows: {count}"
DIAG_SHEET_NO_HEADER_TEMPLATE = (
 "Sheet {sheet}: no Sheet Mapping header row detected."
)
DIAG_SHEET_USEFUL_ROWS_TEMPLATE = "Sheet {sheet}: parsed useful rows: {count}"
DIAG_SHEET_NORMALIZED_CELLS_TEMPLATE = (
 "Sheet {sheet}: row {index} normalized cells: {cells}"
)

--- Population diagnostic spec ---
Each entry: (label, [row-attribute names]); a row counts if ANY attr is set.
POPULATION_DIAGNOSTIC_SPEC = [
 ("Sheet Mapping populated Cobol Record IDMS rows", ["cobol_record_idms"]),
 (
 "Sheet Mapping populated source field rows",
 ["cobol_zone", "reference_field_name_copybook"],
),
 (
 "Sheet Mapping populated DB2 table rows",
 ["new_db2_record", "cross_application_db2_table"],
),
 (
 "Sheet Mapping populated DB2 column rows",
 ["new_db2_field_name", "cross_application_db2_field_name"],
),
]

Useful-context diagnostic: each group must have at least one populated attr.
USEFUL_CONTEXT_LABEL = "Sheet Mapping useful source-to-target rows"
USEFUL_CONTEXT_GROUPS = [
 ["cobol_record_idms"],
 ["cobol_zone", "reference_field_name_copybook"],
 ["new_db2_record", "cross_application_db2_table"],
 ["new_db2_field_name", "cross_application_db2_field_name"],
]

```

```

=====
FILE: rules/sql_error_rules.py
ENCODING: utf-8
=====

```

```

LOCATION: rules/sql_error_rules.py
ACTION: CREATE NEW FILE
"""SQLERROR routine constants.

```

```

Constants only. No regex, no runtime logic, no program / record / table /
cursor / host variable names.
"""

```

```

from __future__ import annotations

```

```

SQL_ERROR_PARAGRAPH_NAME = "SQLERROR"
LEGACY_SQL_ERROR_PARAGRAPH_NAME = "SQL-ERROR"
SQL_ERROR_INCLUDE_NAME = "SQLERROR"

```

```

--- Paragraph header policy -----
Decision D-1 CLOSED by the COBOL team's manual reference program.
The SQLERROR copybook supplies the paragraph label; emitting a local
header duplicates it and the compiler rejects the program.
EMIT_SQL_ERROR_PARAGRAPH_HEADER = False

```

```

SQL_ERROR_BANNER = "* SQLERROR ROUTINE."

```

```

--- Body indents, relative to column 8 -----
IND_STATEMENT = " " # column 12

```

```

IND_SQL_BODY = " " # column 17

--- Old-body recognition -----
Tokens that identify a previously generated SQLERROR body so it can be
consumed and replaced. Covers both known historical forms:
(a) EXEC SQL / INCLUDE SQLERROR / END-EXEC
(b) DISPLAY 'SQL ERROR AT : ' / DISPLAY 'SQLCODE : ' / CALL USERABEN
OLD_BLOCK_BODY_TOKENS = (
 "EXEC SQL",
 "INCLUDE SQLERROR",
 "END-EXEC",
 "SQL ERROR AT",
 "SQLCODE",
 "SQL-LOCATION",
 "CALL USERABEN",
 "CONTINUE",
)

--- Diagnostics -----
SQL_ERROR_MESSAGES = {
 "routine_added": "SQLERROR: added generated SQLERROR routine.",
 "routine_replaced": "SQLERROR: replaced existing SQLERROR body with the include form.",
 "already_present": "SQLERROR: routine already present; not duplicated.",
 "renamed_legacy": (
 "SQLERROR: renamed {headers} legacy header(s) and {performs} "
 "legacy PERFORM(s) from SQL-ERROR to SQLERROR."
),
}

=====
FILE: rules\sql_generation_rules.py
ENCODING: utf-8
=====

LOCATION: rules/sql_generation_rules.py
ACTION: REPLACE ENTIRE FILE (adds the date sentinel constant)

"""
SQL generation constant tuples.

Constants only. No regex, no runtime logic.
"""

SELECT_EXCLUDE_PREFIXES = (
 "TS_CREATE",
 "TS_UPDATE",
 "ID_USERID",
 "NR_USERID",
)

FALLBACK_KEY_PREFIXES = (
 "CT_",
 "NR_",
 "NS_",
 "CO_",
)

DATE_COLUMN_PREFIXES = (
 "DA_",
 "DT_",
)

DB2 low-date sentinel used when a source date is ZEROES or SPACES.
Manual standard moves the numeric 00010101 (0001-01-01) into DA-CCYYMMDD
instead of moving SPACES to the target date field.
DB2_DATE_NULL_SENTINEL = "00010101"

Fields owned by the shared DB2 date conversion block. These are the
OUTPUT of a conversion, never a valid INPUT. Feeding DA-DD-MM-CCYY
(10 bytes) back into DA-CCYYMMDD (8 bytes) truncates and destroys
the date. See CHK-18.
DATE_STAGING_FIELDS = (
 "DA-CCYYMMDD",
 "DA-CCYYMMDD-R",
 "DA-DD-MM-CCYY",
)

```



)

```
=====
FILE: rules\sqlcode_wrapper_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

SQLCODE wrapper cleanup constants. Constants only.
No regex, no runtime logic, no program/record/table names.

--- Detection / comment texts ---
OUTER_IF_TEXT = "IF NOT SQLCODE = 100"
INNER_IF_TEXT = "IF SQLCODE = 100"
REMOVED_OBTAIN_NEXT_TEXT = (
 "DB2: REMOVED REDUNDANT OBTAIN NEXT AFTER CURSOR EOC LOOP"
)
CLEANUP_COMMENT_TEXT = (
 "DB2: Removed redundant SQLCODE wrapper after cursor EOC loop."
)

--- COBOL tokens ---
TOKEN_IF = "IF "
TOKEN_END_IF = "END-IF"
TOKEN_MOVE = "MOVE "
TOKEN_TO = "TO "
TOKEN_OF_DCL = " OF DCL"
DB2_COMMENT_PREFIX = "DB2:"
COMMENT_INDICATOR = "*"
PAGE_INDICATOR = "/"

--- Indentation / geometry ---
BASE_INDENT = " " # 4 spaces
CHILD_INDENT = " " # 8 spaces
CONTINUATION_INDENT = " " # 4 spaces added for wrapped continuations
BODY_WIDTH = 65
FULL_LINE_WIDTH = 80
SEQUENCE_AREA_WIDTH = 6
INDICATOR_COLUMN = 6 # 0-based index of the indicator char
BODY_START_COLUMN = 7
BODY_END_COLUMN = 72
RIGHT_SEQUENCE_START = 72
RIGHT_SEQUENCE_END = 80

--- Look-ahead windows ---
MARKER_LOOKAHEAD = 8 # rows scanned after outer IF for the removed marker
INNER_IF_LOOKAHEAD = 12 # rows scanned for the inner IF SQLCODE = 100

--- Comment prefix set (for skip decisions) ---
COMMENT_PREFIXES = ("*", "/")

COBOL sentence terminator.
#
The removed outer "END-IF." carried the sentence period. When the wrapper
is stripped, that terminator must be transferred to the recovered inner
body, or the paragraph runs into the next paragraph header.
PERIOD = "."

DIAG_PERIOD_RESTORED = (
 "Cleanup: restored sentence terminator lost with the removed "
 "SQLCODE wrapper."
)

=====
```

```
FILE: rules\structural_safety_rules.py
ENCODING: utf-8
=====
```

```
LOCATION: rules/structural_safety_rules.py
ACTION: REPLACE ENTIRE FILE
"""Structural safety rules for generated COBOL.
```

```
Constants only. No regex, no runtime logic, no program / paragraph /
```

record / table / cursor / host variable names.

#### WHY THIS EXISTS

-----

Three passes move or insert lines: the output-write extraction, the counter declaration and the layout composer. Each was written against one program shape. On an older program they produced source the compiler rejects:

- A1. an IF whose CONDITION spans lines was split across two paragraphs,
- A2. a whole-record MOVE survived after the record's copybook was removed, leaving an undefined data-name,
- A4. a duplicate statement was left after a paragraph EXIT, unreachable and outside any paragraph,
- A5. a counter was declared INSIDE the 01 output record, shifting every byte of a fixed-length record.

Every rule here makes a pass REFUSE rather than guess. Refusing leaves working COBOL and a diagnostic; guessing produces source that will not compile.

""

```
from __future__ import annotations
```

```
=====
```

```
A1 - condition integrity
```

```
=====
```

```
An extraction must never cut between the IF keyword and the end of its
condition. Multi-line conditions are normal in hand-written COBOL.
```

```
ENFORCE_CONDITION_INTEGRITY = True
```

```
Lines scanned forward looking for the end of a condition.
```

```
CONDITION_SCAN_LIMIT = 24
```

```
Tokens that continue a condition onto the next line.
```

```
CONDITION_CONTINUATION_TOKENS = (
```

```
 "AND",
```

```
 "OR",
```

```
 "NOT",
```

```
)
```

```
A statement verb proves the condition has ended.
```

```
CONDITION_TERMINATING_VERBS = (
```

```
 "ACCEPT", "ADD", "CALL", "CLOSE", "COMPUTE", "CONTINUE", "DELETE",
```

```
 "DISPLAY", "DIVIDE", "ELSE", "END-IF", "EVALUATE", "EXEC", "EXIT",
```

```
 "GO", "GOBACK", "IF", "INITIALIZE", "INSPECT", "MOVE", "MULTIPLY",
```

```
 "OPEN", "PERFORM", "READ", "RETURN", "REWRITE", "SEARCH", "SET",
```

```
 "STOP", "STRING", "SUBTRACT", "UNSTRING", "WRITE",
```

```
)
```

```
=====
```

```
A2 - orphaned whole-record references
```

```
=====
```

```
ENFORCE_ORPHANED_RECORD_GUARD = True
```

```
ORPHANED_RECORD_MARKER_TEMPLATE = (
```

```
 "*DB2-KEEP: {record} is an IDMS record with no DB2 layout after "
```

```
 "conversion - re-map this reference manually."
```

```
)
```

```
ORPHANED_LINE_PREFIX = "*DB2-KEEP "
```

```
=====
```

```
A4 - unreachable statements
```

```
=====
```

```
ENFORCE_UNREACHABLE_STATEMENT_GUARD = True
```

```
UNREACHABLE_MARKER = (
```

```
 "*DB2-KEEP: statement follows a paragraph EXIT and is unreachable."
```

```
)
```

```
PARAGRAPH_EXIT_WORDS = (
```

```
 "EXIT",
```

```
 "GOBACK",
```

```
 "STOP RUN",
```

```
)
```

```

---- blast radius cap
#
Real unreachable code after a paragraph EXIT is one or two stray lines.
A guard that fires dozens of times has lost sync with the paragraph
structure - a converter defect, not a program defect. When the cap is
exceeded the WHOLE pass is reverted and reported, so a tracking bug can
never comment out working code.
MAX_UNREACHABLE_STATEMENTS = 3

=====
A5 - counter group placement
=====
A counter may only join an existing 01 group when that group is provably
a work area. Anything written, read into, or moved as a whole is a
record layout and must never gain a field.
ENFORCE_SAFE_COUNTER_GROUP = True

Reusing a business group is technically safe but semantically noisy:
a row counter inside 01 WS-DATUMS reads as a date field. The generated
group is deterministic and reviewable, so it is preferred outright.
PREFER_GENERATED_COUNTER_GROUP = True

GENERATED_COUNTER_GROUP_NAME = "WS-DB2-COUNTERS"
GENERATED_COUNTER_GROUP_TEMPLATE = "01 {name}."
GENERATED_COUNTER_GROUP_MARKER = "DB2 GENERATED ROW COUNTERS"

Body indent for children of a generated 01 group.
4 spaces from column 8 lands the level number in Area B (column 12).
GENERATED_COUNTER_CHILD_INDENT = " "

Verbs that prove an 01 group is a record layout, not a work area.
RECORD_LAYOUT_VERBS = (
 "WRITE",
 "READ",
 "REWRITE",
 "RELEASE",
 "RETURN",
)

=====
Diagnostics
=====
STRUCTURAL_SAFETY_MESSAGES = {
 # ---- A1
 "extraction_refused": (
 "Structural safety: output write extraction refused - the guard "
 "condition spans {count} lines and cannot be split."
),
 # ---- A2
 "orphaned_record_commented": (
 "Structural safety: commented whole-record reference to {record}; "
 "the IDMS layout no longer exists."
),
 # ---- A4
 "unreachable_commented": (
 "Structural safety: commented {count} unreachable statement(s) "
 "after a paragraph EXIT."
),
 "unreachable_aborted": (
 "Structural safety: unreachable-statement guard matched {count} "
 "lines, above the limit of {limit}; the pass was reverted and no "
 "line was commented."
),
 # ---- A5
 "counter_group_created": (
 "Structural safety: created {group} for generated row counters."
),
 "counter_group_reused": (
 "Structural safety: reused work group {group} for row counters."
),
 "counter_group_rejected": (
 "Structural safety: {group} is a record layout; counters were not "
 "declared inside it."
),
}

```

```

=====
FILE: rules\test_runner_rules.py
ENCODING: utf-8
=====

LOCATION: rules/test_runner_rules.py
ACTION: CREATE NEW FILE

"""Test runner rules.

Constants only. No regex, no runtime logic, no program / record / table
names.

The runner shells out to pytest and reads its JUnit XML report. JUnit XML
is built into pytest core, so no extra plugin is added to the project.
"""

from __future__ import annotations

=====
Discovery
=====
TESTS_FOLDER = "tests"
TEST_FILE_GLOB = "test_*.py"
ALL_TESTS_LABEL = "All tests"

=====
Invocation
=====
PYTEST_MODULE = "pytest"
PYTEST_BASE_ARGS = (
 "-q",
 "--tb=short",
 "--no-header",
 "-p", "no:cacheprovider",
)
JUNIT_ARG_TEMPLATE = "--junit-xml={path}"
KEYWORD_ARG = "-k"
VERBOSE_ARG = "-vv"

Seconds before the run is abandoned. A converter test suite that hangs
is a defect in its own right; the UI must not hang with it.
RUN_TIMEOUT_SECONDS = 600

PYTHONPATH handed to the subprocess. Both entries are required: the
converter lives under src/, while rules/, patterns/ and catalogs/ sit
at the project root.
PYTHONPATH_ENTRIES = ("src", ".")
PYTHONPATH_SEPARATOR_KEY = "PYTHONPATH"

=====
Outcomes
=====
OUTCOME_PASSED = "PASSED"
OUTCOME_FAILED = "FAILED"
OUTCOME_ERROR = "ERROR"
OUTCOME_SKIPPED = "SKIPPED"

FAILING_OUTCOMES = (OUTCOME_FAILED, OUTCOME_ERROR)

OUTCOME_ICONS = {
 OUTCOME_PASSED: "PASS",
 OUTCOME_FAILED: "FAIL",
 OUTCOME_ERROR: "ERROR",
 OUTCOME_SKIPPED: "SKIP",
}

=====
UI labels
=====
UI_TAB_TITLE = "Tests"
UI_HEADER = "## Python Test Suite"
UI_INTRO = (
 "Run the converter's pytest suite and review the outcome per test. "
 "Results stay on screen until the next run."
)

```

```

)
UI_RUN_BUTTON = "Run Tests"
UI_RUNNING = "Running pytest..."
UI_TARGET_LABEL = "Test target"
UI_KEYWORD_LABEL = "Filter by keyword (pytest -k)"
UI_KEYWORD_PLACEHOLDER = "for example: reflow or counter"
UI_VERBOSE_LABEL = "Verbose output"
UI_FAILURES_ONLY_LABEL = "Show failures only"

UI_METRIC_TOTAL = "Total"
UI_METRIC_PASSED = "Passed"
UI_METRIC_FAILED = "Failed"
UI_METRIC_SKIPPED = "Skipped"
UI_METRIC_DURATION = "Duration (s)"

UI_TABLE_HEADER = "### Results"
UI_DETAIL_HEADER = "### Failure detail"
UI_OUTPUT_HEADER = "### Raw pytest output"
UI_COMMAND_HEADER = "### Command"

UI_NO_RESULT = (
 "No test run yet. Press Run Tests to execute the suite."
)
UI_NO_TESTS_FOUND = (
 "No test files found in the tests folder."
)
UI_ALL_PASSED = "All tests passed."
UI_SOME_FAILED = "{failed} test(s) failed."
UI_TIMED_OUT = (
 "The test run exceeded {seconds} seconds and was stopped."
)
UI_RUN_ERROR = "The test run could not be started: {reason}"

UI_DOWNLOAD_CSV = "Download results (csv)"
UI_DOWNLOAD_LOG = "Download raw output (txt)"
UI_FILE_CSV = "test_results_{stamp}.csv"
UI_FILE_LOG = "test_output_{stamp}.txt"
UI_MIME_CSV = "text/csv"
UI_MIME_TEXT = "text/plain"

TIMESTAMP_FORMAT = "%d-%m-%Y %H:%M:%S"
FILE_TIMESTAMP_FORMAT = "%d-%m-%Y_%H%M%S"

=====
Table columns
=====
COLUMN_OUTCOME = "Outcome"
COLUMN_MODULE = "Module"
COLUMN_TEST = "Test"
COLUMN_DURATION = "Seconds"
COLUMN_MESSAGE = "Message"

TABLE_COLUMNS = (
 COLUMN_OUTCOME,
 COLUMN_MODULE,
 COLUMN_TEST,
 COLUMN_DURATION,
 COLUMN_MESSAGE,
)

MESSAGE_PREVIEW_LENGTH = 160

=====
FILE: rules\timestamp_audit_rules.py
ENCODING: utf-8
=====

LOCATION: rules/timestamp_audit_rules.py
ACTION: REPLACE ENTIRE FILE

"""
Timestamp and audit rules.

This file contains timestamp/audit rule constants and generation templates
only. No parser, service, transformer, generator, resolver, or composer logic

```

belongs here.

No program name, table name, DCLGEN group name, or business field name is hardcoded. The generator supplies only the dynamic program-id value.

```
""
TIMESTAMP_AUDIT_RULES = [
 "Sheet Mapping decides DB2 table and column names.",
 "DCLGEN supplies COBOL host variable spelling and group names.",
 "DCLGEN must not introduce audit fields absent from Sheet Mapping.",
 "For UPDATE flows, generate TS_UPDATE and USER-ID moves only.",
 "TS_CREATE is insert-only and is not generated for update-only flows.",
 "Timestamp paragraph must be a safe paragraph with a terminating boundary.",
]

AUDIT_COLUMN_PREFIXES = [
 "TS_CREATE",
 "TS_UPDATE",
 "ID_USERID",
 "NR_USERID",
 "ID_USER",
 "NR_USER",
 "NS_IDMSKEY",
]

UPDATE_AUDIT_COLUMN_PREFIXES = [
 "TS_UPDATE",
 "ID_USERID",
 "NR_USERID",
 "ID_USER",
 "NR_USER",
]

INSERT_EXCLUDE_AUDIT_PREFIXES = [
 "TS_UPDATE",
]

DATE_COLUMN_PREFIXES = [
 "DA_",
 "DT_",
]

--- Timestamp generation templates (manual standard) ---
#
TS-SYSTEM receives FUNCTION CURRENT-DATE output.
TS-TIMESTAMP is a flat formatted timestamp with separator markers.
#
The generator supplies only the dynamic value:
- program_id (for CS-PROGRAM)

TIMESTAMP_WS_MARKER = "DB2 GENERATED TIMESTAMP AND AUDIT WORKING STORAGE"

TIMESTAMP_CS_PROGRAM_TEMPLATE = (
 "01 CS-PROGRAM PIC X(8) VALUE '{program_id:<8}'."
)

TIMESTAMP_WS_LINES = [
 "01 WS-TIMESTAMP-FIELDS.",
 " 05 TS-SYSTEM.",
 " 10 CC PIC X(2).",
 " 10 YY PIC X(2).",
 " 10 MM PIC X(2).",
 " 10 DD PIC X(2).",
 " 10 HH PIC X(2).",
 " 10 MI PIC X(2).",
 " 10 SS PIC X(2).",
 " 10 TT PIC X(2).",
 " 10 FILLER PIC X(13).",
 " 05 TS-TIMESTAMP.",
 " 10 CC PIC X(2).",
 " 10 YY PIC X(2).",
 " 10 TE-MARKER1 PIC X VALUE '-'.",
 " 10 MM PIC X(2).",
 " 10 TE-MARKER2 PIC X VALUE '-'.",
 " 10 DD PIC X(2).",
 " 10 TE-MARKER3 PIC X VALUE '-'.",
]
```

```

" 10 HH PIC X(2).",
" 10 TE-MARKER4 PIC X VALUE '."',
" 10 MI PIC X(2).",
" 10 TE-MARKER5 PIC X VALUE '."',
" 10 SS PIC X(2).",
" 10 TE-MARKER6 PIC X VALUE '."',
" 10 TT PIC X(2).",
" 10 NNNN PIC 9(04) VALUE 0.",
]

Field pairs moved from TS-SYSTEM into TS-TIMESTAMP (manual standard).
TIMESTAMP_MOVE_FIELDS = (
 "CC",
 "YY",
 "MM",
 "DD",
 "HH",
 "MI",
 "SS",
 "TT",
)

Paragraph body templates. {paragraph_name} is the only dynamic value.
TIMESTAMP_PARAGRAPH_HEADER_TEMPLATE = "{paragraph_name}."
TIMESTAMP_PARAGRAPH_SOURCE_MOVE = (
 " MOVE FUNCTION CURRENT-DATE TO TS-SYSTEM."
)
TIMESTAMP_PARAGRAPH_FIELD_MOVE_TEMPLATE = (
 " MOVE {field} OF TS-SYSTEM{pad}TO {field} OF TS-TIMESTAMP"
)
TIMESTAMP_PARAGRAPH_DISPLAY = " DISPLAY 'TIMESTAMP: ' TS-TIMESTAMP"
TIMESTAMP_PARAGRAPH_TERMINATOR = " ."

Column width at which the 'TO' part aligns in generated timestamp moves.
TIMESTAMP_MOVE_ALIGN_COLUMN = 38

LOCATION: rules/timestamp_audit_rules.py
ACTION: APPEND these two constants

Fallback program-id used only when no target PROGRAM-ID is supplied.
Kept in rules/ so no program name is hardcoded inside generator logic.
TIMESTAMP_DEFAULT_PROGRAM_ID = "DB2PGM"

COBOL PIC X(8) program-id field length (CS-PROGRAM width).
TIMESTAMP_PROGRAM_ID_LENGTH = 8

```

```

=====
FILE: rules\update_business_sql_rules.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

Update business SQL standardizer constants.
Constants only. No regex, no runtime logic, no program/record/table names.

--- Fixed-format column geometry ---
FIXED_BODY_WIDTH = 65
SEQUENCE_AREA_WIDTH = 6 # columns 1-6 (left sequence)
INDICATOR_COLUMN = 7 # column 7 (indicator area)
BODY_START_COLUMN = 7 # body begins after indicator (0-based slice at 7)
BODY_END_COLUMN = 72 # columns 8-72 are the body
FULL_LINE_WIDTH = 72 # a full fixed-format line reaches at least col 72
TOTAL_LINE_WIDTH = 80 # columns 73-80 are the right sequence
RIGHT_SEQUENCE_WIDTH = 8

--- Generated indentation ---
AREA_B_INDENT = " " # 4 spaces -> physical column 12
SQL_BODY_INDENT = " " # 7 spaces -> nested SQL body
NESTED_INDENT = " " # 8 spaces -> nested EVALUATE/IF body
WHEN_INDENT = " " # WHEN clause indent inside EVALUATE

--- Comment / indicator characters ---
COMMENT_INDICATORS = ("*", "/")

```

```

ACTIVE_INDICATOR = " "

--- COBOL keyword tokens used for line classification ---
TOKEN_IF = "IF "
TOKEN_PERFORM = "PERFORM "
TOKEN_WHEN = "WHEN "
TOKEN_END_IF = "END-IF"
TOKEN_INITIALIZE_DCL = "INITIALIZE DCL"
TOKEN_INITIALIZE = "INITIALIZE"
TOKEN_EXEC_SQL = "EXEC SQL"
TOKEN_END_EXEC = "END-EXEC"
TOKEN_EVALUATE_SQLCODE = "EVALUATE SQLCODE"
TOKEN_END_EVALUATE = "END-EVALUATE"
TOKEN_STAR = "*"

--- Fixed process/error paragraph names (COBOL-shop standard) ---
READ_FLAT_FILE_PARAGRAPH = "READ-FLAT-FILE"
SQL_ERROR_PARAGRAPH_NAMES = ("SQLERROR", "SQL-ERROR")
SQL_ERROR_ROUTINE_MARKER = "SQLERROR ROUTINE"

--- Generated body templates ---
PERFORM_TEMPLATE = "PERFORM {paragraph}."
COMMIT_CHECK_IF_TEMPLATE = "IF {counter} > {threshold}"
COMMIT_CHECK_PERFORM_TEMPLATE = "{indent}PERFORM {commit_paragraph}"
END_IF_STATEMENT = "END-IF."
ADD_UPDATE_COUNTER_TEMPLATE = "ADD 1 TO {counter}"
INITIALIZE_DCL_GROUP_TEMPLATE = "INITIALIZE {group}."
FINAL_PERIOD_LINE = " ."
LONE_PERIOD = "."

--- Diagnostics (skip messages not already in UPDATE_RESTART_DIAGNOSTICS) ---
DIAG_PROCESS_PARAGRAPH_UNRESOLVED = (
 "Business update SQL standardization skipped. Processing paragraph "
 "could not be resolved."
)
DIAG_PROCESS_PARAGRAPH_NOT_FOUND_TEMPLATE = (
 "Business update SQL standardization skipped. Processing paragraph not "
 "found: {paragraph}."
)

=====
FILE: rules\update_char_literal_rules.py
ENCODING: utf-8
=====

from __future__ import annotations

Rules for quoting numeric literals moved into character DB2 host variables.
#
Constants only. No regex patterns, parser logic, service logic, program names,
DB2 table names, DCLGEN names, or host variable names belong here.

UPDATE_CHAR_LITERAL_RULES = [
 "When a bare numeric literal is moved to a CHAR/VARCHAR DB2 host variable, "
 "quote the literal (manual standard).",
 "CHAR/VARCHAR detection is DCLGEN-driven (db2_type or COBOL PIC X).",
 "Do not quote literals moved to numeric hosts.",
 "Do not touch SQL host references inside EXEC SQL blocks.",
 "Do not hardcode program names, DB2 tables, columns, DCLGEN groups, "
 "or host variables.",
]

DB2 datatype prefixes that indicate a character column.
CHAR_DB2_TYPE_PREFIXES = ("CHAR", "VARCHAR", "GRAPHIC", "VARGRAPHIC")

COBOL picture prefix that indicates a character (alphanumeric) field.
CHAR_PICTURE_PREFIX = "X"

Diagnostic emitted when a literal is quoted.
CHAR_LITERAL_QUOTED_DIAGNOSTIC = (
 "Update CHAR literal: quoted numeric literal moved to CHAR host variable."
)

```



```
=====
FILE: rules\update_cobol_final_cleanup_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
"""
```

```
Rules for update COBOL final cleanup.
```

```
This module contains constants only.
```

```
No regex patterns, parser logic, service logic, program names, DB2 table names,
DCLGEN names, or host variable names belong here.
```

```
"""
```

```
UPDATE_FINAL_CLEANUP_RULES = [
 "Apply only generic final cleanup after COBOL conversion.",
 "Do not change business logic.",
 "Do not hardcode program names, DB2 tables, DB2 columns, DCLGEN groups, or host variables.",
 "Do not rewrite SQL DCLGEN dot references.",
 "Only rewrite non-SQL DCLGROUP.HOST references to COBOL OF format.",
 "Only initialize nearby resolved DCLGEN groups when local context is clear.",
 "Only insert audit moves for audit columns already present in generated UPDATE SET assignments.",
 "Only rewrite diagnostic labels when a nearby SQL table can be resolved.",
]
```

```
UPDATE_FINAL_LOOKAHEAD_LIMIT = 25
```

```
UPDATE_FINAL_LOOKBACK_DUPLICATE_LIMIT = 80
```

```
PROTECTED_BARE_TARGET_PREFIXES = (
```

```
 "WS-",
 "SW-",
 "WK-",
 "W-",
 "UIT-",
 "OUT-",
 "ES-",
 "SQL",
 "DCL",
 "ERROR-",
 "USER",
 "CS-",
 "TS-",
 "HR-",
 "HELP-",
 "PROGRAM-",
 "DA-",
 "DT-",
```

```
)
```

```
TIMESTAMP_AUDIT_PREFIXES = (
```

```
 "TS_UPDATE",
```

```
)
```

```
USER_AUDIT_PREFIXES = (
```

```
 "ID_USERID",
 "NR_USERID",
 "ID_USER",
 "NR_USER",
```

```
)
```

```
AUDIT_SOURCE_BY_KIND = {
 "timestamp": "TS-TIMESTAMP",
 "user": "CS-PROGRAM",
}
```

```
--- Update final cleanup fixed-format geometry & tokens (appended) ---
```

```
Fixed-format column geometry.
```

```
FINAL_CLEANUP_BODY_WIDTH = 65
```

```

FINAL_CLEANUP_SEQUENCE_WIDTH = 6 # columns 1-6
FINAL_CLEANUP_INDICATOR_COLUMN = 7 # column 7
FINAL_CLEANUP_BODY_START = 7
FINAL_CLEANUP_BODY_END = 72
FINAL_CLEANUP_FULL_WIDTH = 72
FINAL_CLEANUP_TOTAL_WIDTH = 80
FINAL_CLEANUP_RIGHT_SEQUENCE_WIDTH = 8

Generated indentation.
FINAL_CLEANUP_AREA_B = " " # 4 spaces -> column 12
FINAL_CLEANUP_SQL_BODY = " " # 7 spaces -> nested SQL body
FINAL_CLEANUP_WHEN_INDENT = " " # 8 spaces
FINAL_CLEANUP_ACTION_INDENT = " " # 10 spaces

Comment / debug indicator characters.
FINAL_CLEANUP_COMMENT_INDICATORS = ("*", "/", "D", "d")
FINAL_CLEANUP_COMMENT_CHARS = ("*", "/")
FINAL_CLEANUP_DEBUG_PREFIX = "D "

COBOL keyword tokens.
FINAL_CLEANUP_TOKEN_MOVE = "MOVE "
FINAL_CLEANUP_TOKEN_TO = " TO "
FINAL_CLEANUP_TOKEN_EXEC_SQL = "EXEC SQL"
FINAL_CLEANUP_TOKEN_END_EXEC = "END-EXEC"
FINAL_CLEANUP_TOKEN_EVALUATE_SQLCODE = "EVALUATE SQLCODE"
FINAL_CLEANUP_TOKEN_END_EVALUATE = "END-EVALUATE"
FINAL_CLEANUP_TOKEN_END_IF = "END-IF"
FINAL_CLEANUP_TOKEN_STOP_RUN = "STOP RUN."
FINAL_CLEANUP_TOKEN_END_IF_DOT = "END-IF."
FINAL_CLEANUP_TOKEN_PROCEDURE_DIVISION = "PROCEDURE DIVISION"
FINAL_CLEANUP_DCL_PREFIX = "DCL"
FINAL_CLEANUP_LONE_PERIOD = "."
FINAL_CLEANUP_DOT = "."

Rewrite templates.
FINAL_CLEANUP_MOVE_TEMPLATE = "MOVE {source}"
FINAL_CLEANUP_FIELD_OF_GROUP_TEMPLATE = "{field} OF {group}"
FINAL_CLEANUP_TO_FIELD_OF_GROUP_TEMPLATE = "TO {field} OF {group}"

Blank-line collapsing.
FINAL_CLEANUP_MAX_CONSECUTIVE_BLANKS = 2

=====
FILE: rules\update_program_postprocess_rules.py
ENCODING: utf-8
=====

"""
Rules for update-program COBOL postprocess.

This file contains configurable rule values only.
No regex patterns and no runtime logic belong here.
"""

from __future__ import annotations

UPDATE_POSTPROCESS_RULES = [
 "Apply only to update batch conversion.",
 "Do not change retrieval conversion.",
 "Use parsed COBOL, Copybook, Sheet Mapping, and DCLGEN metadata.",
 "Do not hardcode program-specific copybook or DCLGEN names in logic.",
 "If restart DCLGEN is unavailable, do not fabricate restart SQL.",
 "When restart DCLGEN is available, generate restart SQL from DCLGEN metadata.",
 "Preserve old EOF switch if source program still uses it.",
]

RESTART_PARAGRAPH_NAMES = {
 "control": "700-RESTART-CONTROL",
 "restart_found": "710-JOB-IS-RESTART",
 "write_restart": "720-WRITE-RESTART-REC",
 "commit": "800-PROCESS-COMMIT",
 "select": "700-SELECT-DZ01RSTV",
 "update": "700-UPDATE-DZ01RSTV",
 "insert": "700-INSERT-DZ01RSTV",

```

```

 "abend": "810000-CALL-USERABEN",
}

RESTART_DEFAULTS = {
 "phase": "1",
 "status_incomplete": "0",
 "status_complete": "1",
 "retention_days": "7",
}

UPDATE_WORKING_STORAGE_NAMES = {
 "switches_group": "WS-SWITCHES",
 "help_group": "HELP-VARIABLE",
 "restart_record": "WS-RESTART-REC",
 "read_count": "NB-READ",
 "restart_key": "KY-RESTART",
 "input_save_area": "WS-INPUT-KASBONS",
 "commit_counter": "WS-TELLER",
 "input_counter": "WS-NB-INPUT-UPD-I",
 "update_counter": "WS-NB-BFAR-UPD-I",
 "restart_length": "WS-RESTART-LEN",
}

RESTART_DCLGEN_ROLE_TOKENS = {
 "program": ["PROGRAM"],
 "program_fallback": ["NM", "ID", "RS", "PROGRAM"],
 "phase": ["PHASE"],
 "date": ["DA", "CR"],
 "date_fallback": ["DATE"],
 "status": ["CO", "ID"],
 "status_fallback": ["STATUS"],
 "retention": ["DAY"],
 "payload": ["DATA"],
 "payload_length": ["DATA", "LEN"],
 "payload_text": ["DATA", "TEXT"],
}

RESTART_DCLGEN_MIN_SCORE = 4

RESTART_INSERT_QUERYNO = "371"
RESTART_SELECT_QUERYNO = "376"
RESTART_UPDATE_QUERYNO = "398"

GENERATED_UPDATE_COMMENT = "*DB2: Update restart handling generated from DCLGEN."

=====
FILE: rules\update_restart_rules.py
ENCODING: utf-8
=====

from __future__ import annotations

"""
Rules for update-program restart postprocess.

This module contains configurable rule constants only.
No parser logic, generator logic, service logic, file paths, program names,
copybook names, DB2 table names, DCLGEN names, or host variables should be
hardcoded here.
"""

UPDATE_RESTART_RULES = [
 "Apply only in update-program postprocess.",
 "Do not touch retrieval conversion.",
 "Use parsed COBOL source, parsed Copybook fields, and parsed DCLGEN columns.",
 "Do not hardcode program-specific copybook names.",
 "Do not hardcode program-specific DCLGEN names.",
 "Do not hardcode DB2 restart table names in parser, service, or enhancer logic.",
 "Do not hardcode restart host variables in parser, service, or enhancer logic.",
 "Do not generate restart SQL unless restart DCLGEN metadata is available.",
 "Replace legacy IDMS restart-control flow only when restart DCLGEN is resolved.",
 "Preserve COBOL business logic unless update restart flow is explicitly replaced.",
 "For VARCHAR restart payload fields, derive generated LEN and TEXT subfields from the parsed payload group when needed.",
]

```

```

RESTART_PARAGRAPH_NAMES = {
 "control": "700-RESTART-CONTROL",
 "restart_found": "710-JOB-IS-RESTART",
 "write_restart": "720-WRITE-RESTART-REC",
 "commit": "800-PROCESS-COMMIT",
 "select": "700-SELECT-DZ01RSTV",
 "update": "700-UPDATE-DZ01RSTV",
 "insert": "700-INSERT-DZ01RSTV",
 "abend": "810000-CALL-USERABEN",
}

RESTART_STATUS_INCOMPLETE = "0"
RESTART_STATUS_COMPLETE = "1"
RESTART_DEFAULT_PHASE = "1"
RESTART_RETENTION_DAYS = "7"

RESTART_SELECT_QUERYNO = "376"
RESTART_UPDATE_QUERYNO = "398"
RESTART_INSERT_QUERYNO = "413"

RESTART_DCLGEN_MIN_SCORE = 10

RESTART_DCLGEN_ROLE_TOKENS = {
 "program": ["NM", "ID", "RS", "PROGRAM"],
 "program_fallback": ["PROGRAM"],
 "phase": ["NB", "ID", "RS", "PHASE"],
 "phase_fallback": ["PHASE"],
 "date": ["DA", "CR", "RS"],
 "date_fallback": ["DATE"],
 "status": ["CO", "ID", "RS"],
 "status_fallback": ["STATUS"],
 "retention": ["NB", "DAY", "RS"],
 "retention_fallback": ["DAY"],
 "payload": ["TE", "CH", "RS", "DATA"],
 "payload_fallback": ["DATA"],
 "payload_length": ["TE", "CH", "RS", "DATA", "LEN"],
 "payload_length_fallback": ["DATA", "LEN"],
 "payload_text": ["TE", "CH", "RS", "DATA", "TEXT"],
 "payload_text_fallback": ["DATA", "TEXT"],
}

RESTART_VARCHAR_SUFFIXES = {
 "length": "-LEN",
 "text": "-TEXT",
}

UPDATE_RESTART_WS_NAMES = {
 "switch_group": "WS-SWITCHES",
 "help_group": "HELP-VARIABLE",
 "restart_record": "WS-RESTART-REC",
 "read_count": "NB-READ",
 "restart_key": "KY-RESTART",
 "input_save_area": "WS-INPUT-KASBONS",
 "commit_counter": "WS-TELLER",
 "input_counter": "WS-NB-INPUT-UPD-I",
 "update_counter": "WS-NB-BFAR-UPD-I",
 "restart_len": "WS-RESTART-LEN",
}

UPDATE_LEGACY_RESTART_WS_NAMES = (
 "CTR-REC",
 "SW-EOF",
 "SW-RECAP",
 "STOP01",
)

UPDATE_STANDARD_DATE_WS_LINES = [
 "01 WS-DATUMVELDEN.",
 " 03 DA-YYMMDD.",
 " 05 YY PIC 99 VALUE ZERO.",
 " 05 MM PIC 99 VALUE ZERO.",
 " 05 DD PIC 99 VALUE ZERO.",
 " 03 DA-DDMMCCYY.",
 " 05 DD PIC 99 VALUE ZERO.",
 " 05 MM PIC 99 VALUE ZERO.",

```

```

" 05 CC PIC 99 VALUE ZERO.",
" 05 YY PIC 99 VALUE ZERO.",
" 03 DA-CCYYMMDD.",
" 05 CC PIC 99 VALUE ZERO.",
" 05 YY PIC 99 VALUE ZERO.",
" 05 MM PIC 99 VALUE ZERO.",
" 05 DD PIC 99 VALUE ZERO.",
" 03 DA-CCYYMMDD-R REDEFINES DA-CCYYMMDD.",
" 05 CCYY PIC 9999.",
" 05 MM PIC 99.",
" 05 DD PIC 99.",
" 02 DA-DD-MM-CCYY.",
" 03 DD PIC 9(02).",
" 03 TE-MARKER5 PIC X VALUE '.'.",
" 03 MM PIC 9(02).",
" 03 TE-MARKER6 PIC X VALUE '.'.",
" 03 CCYY PIC 9(04).",
]

UPDATE_MAIN_DATE_INITIALIZATION_LINES = [
 "MOVE ZEROES TO DA-CCYYMMDD.",
 "MOVE DATE-YMD8 TO DA-CCYYMMDD.",
 "MOVE CORR DA-CCYYMMDD-R TO DA-DD-MM-CCYY.",
]

UPDATE_SUMMARY_DISPLAY_TEMPLATES = [
 "DISPLAY 'NUMBER OF RECORDS PROCESSED: ' {input_counter}.",
 "DISPLAY 'NUMBER OF UPDATE RECORDS : ' {update_counter}.",
]

UPDATE_SQL_PARAGRAPH_PREFIX = "1100-UPDATE"
UPDATE_COMMIT_THRESHOLD = 99

UPDATE_RESTART_DIAGNOSTICS = {
 "start": "START UPDATE PROGRAM POST-PROCESS ENHANCEMENT",
 "end": "END UPDATE PROGRAM POST-PROCESS ENHANCEMENT",
 "context_failed": (
 "Postprocess context resolution failed. Generated COBOL left unchanged."
),
 "ws_exists": "Working-storage support already exists.",
 "ws_injected": "Injected COBOL-standard update restart working-storage.",
 "copy_exists": "Input copybook include already exists.",
 "copy_injected": "Injected input copybook include.",
 "dclgen_exists": "Restart DCLGEN include already exists.",
 "dclgen_injected": "Injected restart DCLGEN include.",
 "referenced_dclgen_injected": "Injected referenced DCLGEN include.",
 "date_ws_exists": "Manual-style date working-storage already exists.",
 "date_ws_injected": "Injected manual-style date working-storage.",
 "legacy_ws_removed": "Removed legacy update restart working-storage declarations.",
 "program_name_normalized": "Normalized PROGRAM-NAME move to resolved PROGRAM-ID.",
 "main_init_added": "Added standard update main initialization.",
 "summary_display_added": "Added standard update summary DISPLAY statements.",
 "read_replaced": "Replaced READ-FLAT-FILE with copybook-standard implementation.",
 "read_added": "Added READ-FLAT-FILE copybook-standard implementation.",
 "restart_missing": "Restart DCLGEN unresolved. Restart paragraphs skipped.",
 "restart_inserted": "Injected standard restart-table paragraphs.",
 "legacy_replaced": "Replaced legacy active restart main flow with standard update restart flow.",
 "legacy_not_found": "Legacy active restart main flow not found.",
 "malformed_sqlerror_fixed": "Fixed malformed PERFORM SQLERROR.END-EVALUATE line.",
 "business_update_extracted": "Extracted inline business UPDATE SQL into standard update paragraph.",
 "business_update_not_found": "Inline business UPDATE SQL block not found for extraction.",
}

__all__ = [
 "UPDATE_RESTART_RULES",
 "RESTART_PARAGRAPH_NAMES",
 "RESTART_STATUS_INCOMPLETE",
 "RESTART_STATUS_COMPLETE",
 "RESTART_DEFAULT_PHASE",
 "RESTART_RETENTION_DAYS",
 "RESTART_SELECT_QUERYNO",
 "RESTART_UPDATE_QUERYNO",
 "RESTART_INSERT_QUERYNO",
 "RESTART_DCLGEN_MIN_SCORE",
 "RESTART_DCLGEN_ROLE_TOKENS",
 "RESTART_VARCHAR_SUFFIXES",

```

```

 "UPDATE_RESTART_WS_NAMES",
 "UPDATE_LEGACY_RESTART_WS_NAMES",
 "UPDATE_STANDARD_DATE_WS_LINES",
 "UPDATE_MAIN_DATE_INITIALIZATION_LINES",
 "UPDATE_SUMMARY_DISPLAY_TEMPLATES",
 "UPDATE_SQL_PARAGRAPH_PREFIX",
 "UPDATE_COMMIT_THRESHOLD",
 "UPDATE_RESTART_DIAGNOSTICS",
]

--- Update main flow rewriter constants (appended) ---

Fixed-format geometry.
MAIN_FLOW_FIXED_BODY_WIDTH = 65
MAIN_FLOW_SEQUENCE_WIDTH = 6
MAIN_FLOW_INDICATOR_COLUMN = 7
MAIN_FLOW_BODY_START = 7
MAIN_FLOW_BODY_END = 72
MAIN_FLOW_FULL_WIDTH = 72
MAIN_FLOW_TOTAL_WIDTH = 80
MAIN_FLOW_RIGHT_SEQUENCE_WIDTH = 8
MAIN_FLOW_AREA_B_INDENT = " "
MAIN_FLOW_COMMENT_INDICATORS = ("*", "/", "D")

COBOL tokens.
MAIN_FLOW_READ_FLAT_FILE_PARAGRAPH = "READ-FLAT-FILE"
MAIN_FLOW_TOKEN_CLOSE = "CLOSE "
MAIN_FLOW_LONE_PERIOD = "."

Generated bodies.
MAIN_FLOW_PERFORM_READ_BODY = "PERFORM READ-FLAT-FILE"
MAIN_FLOW_PERFORM_READ_BODY_DOT = "PERFORM READ-FLAT-FILE."
MAIN_FLOW_SET_NOT_EOF_TEMPLATE = "SET {record}-NOT-EOF TO TRUE."
MAIN_FLOW_INITIALIZE_HELP_TEMPLATE = "INITIALIZE {help_group}."

Main restart flow template lines (dynamic values via .format()).
MAIN_FLOW_RESTART_TEMPLATE_LINES = [
 "",
 " PERFORM {control}.",
 "",
 " PERFORM READ-FLAT-FILE.",
 " PERFORM {process_paragraph} UNTIL {record}-EOF.",
 "",
 " PERFORM {commit}.",
 "",
 " CLOSE {file_name}.",
 "",
]

Legacy restart detection tokens.
MAIN_FLOW_LEGACY_RESTART_TOKENS = (
 "SW-RECAP",
 "CTR-REC",
 "FFRECAP",
 "NS-REC-FF-IN",
 "READ-FLAT-FILE VARYING",
 "INSERT INTO",
 "DZ01RST",
)

Restart / utility paragraph-number prefixes to skip when resolving the
business processing paragraph.
MAIN_FLOW_RESTART_PARAGRAPH_PREFIXES = (
 "600-",
 "700-",
 "710-",
 "720-",
 "800-",
 "810000-",
)

Region look-back windows.
MAIN_FLOW_OPEN_LOOKBACK = 20
MAIN_FLOW_CLOSE_LOOKBACK = 10

--- Main flow skip/status diagnostics (appended, not already present) ---

```

```

MAIN_FLOW_DIAGNOSTICS = {
 "dclgen_unresolved": (
 "Restart DCLGEN unresolved. Legacy restart main flow left unchanged."
),
 "open_not_found": (
 "OPEN INPUT line not found. Legacy restart main flow replacement "
 "skipped."
),
 "stop_not_found": (
 "STOP RUN line not found. Legacy restart main flow replacement "
 "skipped."
),
 "legacy_not_found": (
 "Legacy restart statements not found between OPEN INPUT and STOP RUN. "
 "Main flow replacement skipped."
),
 "process_unresolved": (
 "Processing paragraph could not be resolved. Legacy restart main flow "
 "replacement skipped."
),
 "process_empty": (
 "Processing paragraph is empty. Skipped READ-FLAT-FILE insertion into "
 "process paragraph."
),
 "process_not_found": (
 "Processing paragraph not found: {paragraph}. Skipped READ-FLAT-FILE "
 "insertion into process paragraph."
),
 "already_reads_next": (
 "Processing paragraph already reads next input record: {paragraph}."
),
 "no_insertion_point": (
 "Safe insertion point not found in processing paragraph: {paragraph}."
),
 "read_inserted": (
 "Inserted READ-FLAT-FILE at end of processing paragraph: {paragraph}."
),
}
--- Restart DCLGEN role-key spec (appended) ---
#
Each role that resolves a (column, field) pair from the same token set.
Format: role_name -> (token_key, fallback_token_key)
The builder loops over this map so no per-role code repetition is needed.

RESTART_ROLE_COLUMN_FIELD_SPEC = {
 "program": ("program", "program_fallback"),
 "phase": ("phase", "phase_fallback"),
 "date": ("date", "date_fallback"),
 "status": ("status", "status_fallback"),
 "retention": ("retention", "retention_fallback"),
}

Payload resolves only a column (no field pair).
RESTART_PAYLOAD_TOKEN_KEY = "payload"
RESTART_PAYLOAD_FALLBACK_TOKEN_KEY = "payload_fallback"

Payload child field specs: (token_key, fallback_token_key, suffix_key).
RESTART_PAYLOAD_LENGTH_SPEC = (
 "payload_length",
 "payload_length_fallback",
 "length",
)
RESTART_PAYLOAD_TEXT_SPEC = (
 "payload_text",
 "payload_text_fallback",
 "text",
)
LEGACY_ABEND_MARKER_LITERAL_PREFIX = "##&&"

now delegates to SqlErrorGenerator rather than carrying its own copy.
USE_SHARED_SQL_ERROR_GENERATOR = True

Retained only so an existing import does not break. Do not emit.
SQLERROR_PARAGRAPH_TEMPLATE: list[str] = []

```

```
=====
FILE: rules\update_restart_skip_rules.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

Update restart/control skip cleanup constants. Constants only.
No regex, no runtime logic, no program/record/table names.

Markers that indicate a generated missing-mapping block.
MISSING_MAPPING_MARKERS = (
 "CONVERSION SKIPPED",
 "MISSING SHEET MAPPING",
 "MISSING DCLGEN",
 "MISSING SELECT",
 "MISSING INSERT",
 "MISSING UPDATE",
 "MISSING DELETE",
 "MISSING MAPPING",
 "MISSING KEY COLUMN METADATA",
 "INCOMPLETE CONSERVATIVE UPDATE METADATA",
 "MISSING INSERT COLUMNS",
)

Substrings that identify a restart/control-like record name.
RESTART_CONTROL_HINTS = (
 "RECAP",
 "RESTART",
 "RST",
 "CONTROL",
 "CTRL",
 "CHECKPOINT",
 "CHKPT",
)

Generated replacement comment block for an unmapped restart/control record.
REPLACEMENT_BLOCK_TEMPLATES = (
 "* DB2: IDMS record {record} was not converted automatically.",
 "* DB2: Missing Sheet Mapping and DCLGEN metadata.",
 "* DB2: Restart/control logic requires manual DB2 redesign.",
 "CONTINUE.",
)

Advisory message (suppressed by default; the update post-process now
generates the real DB2 restart-table flow, so this advisory is obsolete).
read it from one place and the two cannot drift apart.
EMIT_MANUAL_REDESIGN_MESSAGE = False

MANUAL_REDESIGN_MESSAGE_TEMPLATE = (
 "Update restart skip: {record} has no usable Sheet Mapping entry; "
 "block replaced with a manual-redesign comment."
)
```

```
=====
FILE: rules\update_runner_messages.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

Update runner message templates. Text only. No regex, no logic.

---- Validation labels ----
LABEL_MAPPING_SHEET = "Mapping Sheet"
LABEL_DCLGEN_FOLDER = "DCLGen"
LABEL_UPDATE_PROGRAM = "Update Program"
LABEL_SHEET_MAPPING = "Sheet Mapping"
LABEL_DCLGEN_TEMPLATE = "DCLGEN {index}"
LABEL_COPYBOOK_TEMPLATE = "Copybook {index}"
LABEL_UPDATE_PROGRAM_TEMPLATE = "Update Program {index}"

---- Validation errors ----
```



```

FILE_NOT_FOUND_TEMPLATE = "{label} file not found: {path}"
PATH_NOT_FILE_TEMPLATE = "{label} path is not a file: {path}"
FOLDER_NOT_FOUND_TEMPLATE = "{label} folder not found: {path}"
PATH_NOT_FOLDER_TEMPLATE = "{label} path is not a folder: {path}"
NO_MAPPING_FILE_TEMPLATE = "No mapping sheet file found in: {folder}"
NO_PROGRAM_FILE_TEMPLATE = "No update program file found in: {folder}"
MULTIPLE_MAPPING_TEMPLATE = (
 "Multiple mapping sheet files found. Keep exactly one mapping sheet in "
 "{folder}. Found: {names}"
)
NO_DCLGEN_CANDIDATE_TEMPLATE = (
 "At least one DCLGEN file path is required. No DCLGEN candidate file was "
 "found. Searched:\n{searched}"
)

---- Shared input diagnostics ----
DIAG_START_LOAD_INPUTS = "START LOAD UPDATE SHARED INPUTS"
DIAG_MAPPING_FOLDER_TEMPLATE = "Sheet Mapping folder: {folder}"
DIAG_MAPPING_SELECTED_TEMPLATE = "Sheet Mapping selected: {path}"
DIAG_MAPPING_ROWS_TEMPLATE = "Sheet Mapping rows: {count}"
DIAG_DCLGEN_FOLDER_TEMPLATE = "DCLGEN folder: {folder}"
DIAG_DCLGEN_FILE_COUNT_TEMPLATE = "DCLGEN file count: {count}"
DIAG_DCLGEN_FILE_TEMPLATE = "DCLGEN {index}: {path}"
DIAG_DCLGEN_TEXT_LEN_TEMPLATE = "DCLGEN {index} text length: {length}"
DIAG_DCLGEN_TOTAL_COLUMNS_TEMPLATE = "DCLGEN total columns: {count}"
DIAG_COPYBOOK_FILE_COUNT_TEMPLATE = "Copybook file count: {count}"
DIAG_COPYBOOK_FILE_TEMPLATE = "Copybook {index}: {path}"
DIAG_COPYBOOK_TEXT_LEN_TEMPLATE = "Copybook {index} text length: {length}"
DIAG_COPYBOOK_TOTAL_FIELDS_TEMPLATE = "Copybook total fields: {count}"

COPYBOOK_SOURCE_LABEL = "Copybook files"
DCLGEN_SOURCE_LABEL_TEMPLATE = "DCLGEN file {index}"

---- Log format strings (printf-style) ----
LOG_MAPPING_SELECTED = "Sheet Mapping selected: %s"
LOG_MAPPING_ROWS = "Sheet Mapping rows: %s"
LOG_DCLGEN_FILE_COUNT = "DCLGEN file count: %s"
LOG_DCLGEN_FILE = "DCLGEN %s: %s"
LOG_DCLGEN_TEXT_LEN = "DCLGEN %s text length: %s"
LOG_DCLGEN_TOTAL_COLUMNS = "DCLGEN total columns: %s"
LOG_COPYBOOK_FILE_COUNT = "Copybook file count: %s"
LOG_COPYBOOK_FILE = "Copybook %s: %s"
LOG_COPYBOOK_TEXT_LEN = "Copybook %s text length: %s"
LOG_COPYBOOK_TOTAL_FIELDS = "Copybook total fields: %s"

---- Per-program conversion ----
DIAG_SOURCE_FILE_TEMPLATE = "IDMS COBOL source file: {path}"
DIAG_SOURCE_LEN_TEMPLATE = "IDMS COBOL source text length: {length}"
DIAG_DERIVED_PROGRAM_ID_TEMPLATE = "Derived target PROGRAM-ID: {value}"

LOG_SOURCE_FILE = "IDMS COBOL source file: %s"
LOG_SOURCE_LEN = "IDMS COBOL source text length: %s"
LOG_DERIVED_PROGRAM_ID = "Derived target PROGRAM-ID: %s"
LOG_GENERATION_COMPLETED = "Update DB2 COBOL generation completed."
LOG_OUTPUT_CREATED = "Output file created: %s"

PRESERVE_SOURCE_LABEL = "(preserve source)"

---- Update postprocess diagnostics ----
DIAG_UPDATE_ENHANCEMENT_DONE = "Update postprocess enhancement completed."
LOG_UPDATE_ENHANCEMENT_DONE = "Update postprocess enhancement completed."
DIAG_UPDATE_AUDIT_MOVES = (
 "Update audit moves: inserted TS_UPDATE / USER-ID host moves before "
 "UPDATE SQL."
)
DIAG_UPDATE_SKIPPED_TEMPLATE = (
 "Update postprocess enhancement skipped. Original converted COBOL will "
 "be written. Reason: {reason}"
)

---- Console output ----
PRINT_GENERATION_COMPLETED = "Update DB2 COBOL generation completed."
PRINT_INPUT_PROGRAM_TEMPLATE = "Input program: {path}"
PRINT_OUTPUT_CREATED_TEMPLATE = "Output file created: {path}"

HEADER_INPUT_SUMMARY = "Input Summary"

```

```

HEADER_SELECTED_INPUT_FILES = "Selected Input Files"
HEADER_VALIDATION_MESSAGES = "Validation Messages"
HEADER_DIAGNOSTICS = "Diagnostics"
HEADER_UPDATE_BATCH_SUMMARY = "Update Batch Summary"

RULE_INPUT_SUMMARY = "-----"
RULE_SELECTED_INPUT_FILES = "-----"
RULE_VALIDATION_MESSAGES = "-----"
RULE_DIAGNOSTICS = "-----"
RULE_UPDATE_BATCH_SUMMARY = "-----"

SUMMARY_PROJECT_ROOT_TEMPLATE = "Project Root : {value}"
SUMMARY_SRC_DIR_TEMPLATE = "SRC Directory : {value}"
SUMMARY_SHEET_ROWS_TEMPLATE = "Sheet Mapping Rows : {value}"
SUMMARY_DCLGEN_COLUMNS_TEMPLATE = "DCLGEN Columns : {value}"
SUMMARY_COPYBOOK_FIELDS_TEMPLATE = "Copybook Fields : {value}"
SUMMARY_COBOL_LEN_TEMPLATE = "COBOL Text Length : {value}"
SUMMARY_TARGET_PROGRAM_ID_TEMPLATE = "Target PROGRAM-ID : {value}"

SELECTED_SHEET_MAPPING_TEMPLATE = "Sheet Mapping : {value}"
SELECTED_COBOL_SOURCE_TEMPLATE = "IDMS COBOL Source : {value}"
SELECTED_DCLGEN_HEADER = "DCLGEN Files:"
SELECTED_COPYBOOK_HEADER = "Copybook Files:"
SELECTED_COPYBOOK_NONE = "Copybook Files : None"
SELECTED_PATH_BULLET_TEMPLATE = " - {value}"

BATCH_PROGRAM_FOLDER_TEMPLATE = "Program Folder : {value}"
BATCH_PROGRAM_COUNT_TEMPLATE = "Program Count : {value}"
BATCH_OUTPUT_FOLDER_TEMPLATE = "Output Folder : {value}"

NO_VALIDATION_MESSAGES = "No validation messages."
BULLET_TEMPLATE = "- {text}"

=====
FILE: rules\update_runner_rules.py
ENCODING: utf-8
=====

from __future__ import annotations

Update runner rule constants. Constants only. No regex, no logic.

UPDATE_RUNNER_RULES = [
 "Derive the target PROGRAM-ID from the source (VM...BD... -> VMDZ...).",
 "Preserve the source PROGRAM-ID when no PROGRAM-ID is detected.",
 "Load Sheet Mapping, DCLGEN, and Copybook inputs once and share them.",
 "Exactly one mapping sheet must be present for update runs.",
 "Do not hardcode program names, DB2 tables, columns, or host variables.",
]

AUTO_FIX_PIC_LENGTH_MISMATCHES_DEFAULT = False
OUTPUT_TIMESTAMP_FORMAT = "%d-%m-%Y_%H%M%S"
OUTPUT_FILE_EXTENSION = ".cbl"
OUTPUT_FILE_NAME_TEMPLATE = "{stem}_{timestamp}{extension}"
COBOL_TEXT_ENCODING = "utf-8"
UPDATE_LOGGER_NAME = "idms_db2_update_conversion"

=====
FILE: rules\update_sql_cleanup_rules.py
ENCODING: utf-8
=====

LOCATION: rules/update_sql_cleanup_rules.py
ACTION: REPLACE ENTIRE FILE
import re
"""
Update-program SQL cleanup constant tuples.

Constants only. No regex, no runtime logic.
"""

PROTECTED_BARE_TARGET_PREFIXES = (
 "WS-",
 "SW-",

```

```

 "WK-",
 "W-",
 "UIT-",
 "OUT-",
 "ES-",
 "SQL",
 "DCL",
 "ERROR-",
 "USER",
 "CS-",
 "TS-",
 "HR-",
 "HELP-",
 "PROGRAM-",
)

 UPDATE_AUDIT_PREFIXES = (
 "TS_UPDATE",
 "ID_USERID",
 "NR_USERID",
 "ID_USER",
 "NR_USER",
)

 INSERT_ONLY_AUDIT_PREFIXES = (
 "TS_CREATE",
)

 IDENTITY_KEY_PREFIXES = (
 "NS_ID",
 "NR_ID",
 "ID_",
 "CO_ID",
 "NR_IS",
)

 # --- Category E: manual-standard SQL block ---

 # QUERYNO value appended to generated INSERT/UPDATE/DELETE statements
 # (manual standard) so DB2 EXPLAIN can identify the generated statement.
 UPDATE_QUERYNO = "442"

 # When True, generated SQL uses EVALUATE SQLCODE (manual standard) instead
 # of the older IF SQLCODE NOT = 0 form.
 USE_EVALUATE_SQLCODE = True

 # LOCATION: composers/update_sql_cleanup/update_sql_cleanup_patterns.py
 # ACTION: APPEND these patterns to the existing file (add at the end)

 # --- SQLCODE handling block detection (IF and EVALUATE forms) ---

 SQLCODE_EVALUATE_PATTERN = re.compile(
 r"^\s*EVALUATE\s+SQLCODE\b",
 flags=re.IGNORECASE,
)

 END_EVALUATE_PATTERN = re.compile(
 r"^\s*END-EVALUATE\.\?\s*$",
 flags=re.IGNORECASE,
)

 # A lone period line that closes an EVALUATE block (COBOL scope terminator).
 LONE_PERIOD_PATTERN = re.compile(
 r"^\s*\.\s*$",
 flags=re.IGNORECASE,
)

 """
 Update SQL generation rules.
 """
 FILE: rules\update_sql_rules.py
 ENCODING: utf-8
 """

```

This file contains rules/constants only.  
No parser, formatter, generator, resolver, or transformer logic belongs here.

The goal is to keep UPDATE generation conservative and manual-style:

- Do not update every DCLGEN column.
  - Update only fields proven to be changed by the COBOL business flow.
  - For update flows, include only update audit fields.
  - For CALC-style IDMS records, use Sheet Mapping key metadata for WHERE.
- ""

```
CONSERVATIVE_UPDATE_RULES = [
 "UPDATE generation is conservative/manual-style, not broad all-column update.",
 "Only columns changed by original COBOL business logic should be updated.",
 "UPDATE audit columns may be added only when present in Sheet Mapping and DCLGEN.",
 "For UPDATE flows, generate TS_UPDATE and USER-ID moves only.",
 "TS_CREATE is insert-only and must not be generated for update-only flows.",
 "For IDMS CALC records, WHERE clause must use Sheet Mapping key metadata only.",
 "Do not add relationship foreign keys to CALC-record WHERE clauses unless marked as key metadata.",
]
```

```
UPDATE_AUDIT_COLUMN_PREFIXES = [
 "TS_UPDATE",
 "ID_USERID",
 "NR_USERID",
 "ID_USER",
 "NR_USER",
]
```

```
INSERT_ONLY_AUDIT_COLUMN_PREFIXES = [
 "TS_CREATE",
]
```

```
KEY_TEXT_MARKERS = [
 "PRIMARY",
 "PRIMARY KEY",
 "KEY",
 "CALC",
]
```

```
DATE_COLUMN_PREFIXES = [
 "DA_",
 "DT_",
]
```

```
=====
```

FILE: rules\update\_standard\_generator\_templates.py

ENCODING: utf-8

```
=====
```

```
from __future__ import annotations
```

# Update standard generator COBOL templates and literals.

#

# This module owns ALL generated COBOL layout text so the generator holds

# no COBOL literals. Only dynamic values (record, field, table, host names)

# are substituted at fill time. No program/record/table names are hardcoded;

# those arrive as .format() kwargs from resolved metadata.

#

# Layout rules enforced here:

# - Area A starts at column 8, Area B at column 12. A template line's leading

# spaces are relative to column 8, so a level-02 item needs 4 spaces.

# - Every EXEC SQL block is preceded by a MOVE to SQL-LOCATION, so a failure

# reports the paragraph that actually failed (CHK-20.07).

# - No period terminates a statement inside EVALUATE. Only END-EVALUATE

# carries one (cobol\_standards.ENFORCE\_NO\_PERIOD\_IN\_EVALUATE).

# --- Numeric / literal constants used in generated COBOL ---

RESTART\_LENGTH\_BASE = 15 # added to record length for WS-RESTART-LEN

EOF\_NOT\_SET\_VALUE = "N"

EOF\_SET\_VALUE = "Y"

COMMIT\_COUNTER\_RESET\_VALUE = "0"

# --- Shared generated symbol names (not program-specific) ---

CS\_PROGRAM\_FIELD = "CS-PROGRAM"

DA\_DD\_MM\_CCYY\_FIELD = "DA-DD-MM-CCYY"

```

COMMIT_COUNTER_LEGACY = "WS-TELLER"
SQL_ERROR_PARAGRAPH = "SQLERROR"
SQL_LOCATION_FIELD = "SQL-LOCATION"
ABEND_CALL_STATEMENT = "CALL USERABEN"

--- Display strings ---
DISPLAY_PREVIOUS_RUN_COMPLETE = "DISPLAY ' PREVIOUS RUN COMPLETE '"
DISPLAY_ERROR_REC_TEMPLATE = "DISPLAY 'ERROR REC NUMBER : ' {counter}"

--- Include templates ---
COPYBOOK_INCLUDE_TEMPLATE = "COPY {include}."
DCLGEN_INCLUDE_TEMPLATE = "EXEC SQL INCLUDE {include} END-EXEC."

--- Working-storage template ---
#
Indentation is relative to column 8. Level 02 therefore needs 4 spaces to
land in Area B (column 12); 3 spaces put it at column 11, inside Area A.
Nested levels step by 3 (02 -> col 12, 03/88 -> col 15).
WORKING_STORAGE_TEMPLATE = ""01 {switch_group}.
 02 SW-{record} PIC X VALUE '{eof_not}'.
 88 {record}-NOT-EOF VALUE '{eof_not}'.
 88 {record}-EOF VALUE '{eof_set}'.

01 {help_group}.
 02 {restart_record}.
 03 {read_count} PIC 9(15).
 03 {restart_key} PIC X({length}).

 02 {input_save_area} PIC X({length}).

 02 {commit_counter} PIC 9(9) COMP-3 VALUE ZERO.
 02 {input_counter} PIC 9(7) COMP-3 VALUE ZERO.
 02 {update_counter} PIC S9(11) COMP-3 VALUE +0.
 02 {restart_len} PIC 9(5) VALUE {restart_length}.""

--- READ flat file template ---
READ_FLAT_FILE_TEMPLATE = ""READ-FLAT-FILE.

 INITIALIZE {record}.

 READ {file_name} INTO {record}
 AT END
 SET {record}-EOF TO TRUE
 END-READ.

 IF {record}-NOT-EOF
 ADD +1 TO {commit_counter}
 ADD +1 TO {input_counter}
 MOVE {record} TO {input_save_area}
 END-IF.""

--- Restart control template ---
RESTART_CONTROL_TEMPLATE = ""{control}.

 MOVE {cs_program}
 TO {program_field} OF {host_record}.

 MOVE {da_field}
 TO {date_field} OF {host_record}.

 MOVE {default_phase}
 TO {phase_field} OF {host_record}.

 PERFORM {select}.

 EVALUATE SQLCODE
 WHEN 0
 PERFORM {restart_found}
 WHEN 100
 PERFORM {write_restart}
 WHEN OTHER
 PERFORM {sql_error}
 END-EVALUATE.

 MOVE {commit_reset} TO {commit_counter_legacy}.""

```

```

--- Restart found template ---
#
The counter MOVE sits AFTER END-PERFORM. Inside the loop it executed once
per skipped record and was overwritten every iteration; the final value was
correct only by coincidence.
RESTART_FOUND_TEMPLATE = ""{restart_found}.

 MOVE {payload_text_field} OF {host_record}
 TO {restart_record}.

 IF {status_field} OF {host_record} = '{status_complete}'
 {display_complete}
 PERFORM {abend}
 ELSE
 PERFORM {read_count} OF {restart_record} TIMES
 PERFORM READ-FLAT-FILE
 END-PERFORM
 MOVE {read_count} OF {restart_record}
 TO {input_counter}
 END-IF.""

--- Write restart template ---
WRITE_RESTART_TEMPLATE = ""{write_restart}.

 MOVE '{status_incomplete}'
 TO {status_field} OF {host_record}.

 MOVE {retention_days}
 TO {retention_field} OF {host_record}.

 MOVE {restart_len}
 TO {payload_len_field} OF {host_record}.

 MOVE 0
 TO {input_counter}.

 MOVE {input_counter}
 TO {read_count} OF {restart_record}.

 INITIALIZE {restart_key} OF {restart_record}.

 MOVE {restart_record}
 TO {payload_text_field} OF {host_record}.

 PERFORM {insert}.""

--- Process commit template ---
PROCESS_COMMIT_TEMPLATE = ""{commit}.

 MOVE 0 TO {commit_counter}.

 MOVE {input_counter}
 TO {read_count} OF {restart_record}.

 MOVE {input_save_area}
 TO {restart_key} OF {restart_record}.

 IF {record}-EOF
 MOVE '{status_complete}'
 TO {status_field} OF {host_record}
 ELSE
 MOVE '{status_incomplete}'
 TO {status_field} OF {host_record}
 END-IF.

 MOVE {restart_len}
 TO {payload_len_field} OF {host_record}.

 MOVE {restart_record}
 TO {payload_text_field} OF {host_record}.

 PERFORM {update}.

 MOVE '{commit}' TO SQL-LOCATION.

EXEC SQL

```

```

 COMMIT
 END-EXEC.

EVALUATE SQLCODE
 WHEN 0
 CONTINUE
 WHEN OTHER
 {display_error}
 PERFORM {sql_error}
END-EVALUATE. ""

--- Restart SQL template (SELECT / UPDATE / INSERT / ABEND) ---
#
Each statement sets SQL-LOCATION first. Without it a restart failure
reports whatever location the previous business statement left behind,
sending the operator to the wrong paragraph (CHK-20.07).
RESTART_SQL_TEMPLATE = ""{select}.

 MOVE '{select}' TO SQL-LOCATION.

EXEC SQL
 SELECT {program_column}
 , {phase_column}
 , {date_column}
 , {status_column}
 , {retention_column}
 , {payload_column}
 INTO :{host_record}.{program_field}
 , :{host_record}.{phase_field}
 , :{host_record}.{date_field}
 , :{host_record}.{status_field}
 , :{host_record}.{retention_field}
 , :{host_record}.{payload_group}
 FROM {table_name}
 WHERE {program_column} = :{host_record}.{program_field}
 AND {phase_column} = :{host_record}.{phase_field}
 AND {date_column} = :{host_record}.{date_field}
 QUERYNO {select_queryno}
END-EXEC.

EVALUATE SQLCODE
 WHEN 0
 CONTINUE
 WHEN 100
 CONTINUE
 WHEN OTHER
 {display_error}
 PERFORM {sql_error}
END-EVALUATE.

{update}.

 MOVE '{update}' TO SQL-LOCATION.

EXEC SQL
 UPDATE {table_name}
 SET {payload_column} = :{host_record}.{payload_group}
 , {status_column} = :{host_record}.{status_field}
 WHERE {program_column} = :{host_record}.{program_field}
 AND {phase_column} = :{host_record}.{phase_field}
 AND {date_column} = :{host_record}.{date_field}
 QUERYNO {update_queryno}
END-EXEC.

EVALUATE SQLCODE
 WHEN 0
 CONTINUE
 WHEN OTHER
 {display_error}
 PERFORM {sql_error}
END-EVALUATE.

{insert}.

 MOVE '{insert}' TO SQL-LOCATION.

```

```

EXEC SQL
 INSERT INTO {table_name}
 VALUES (:{host_record})
 QUERYNO {insert_queryno}
END-EXEC.

EVALUATE SQLCODE
 WHEN 0
 CONTINUE
 WHEN OTHER
 {display_error}
 PERFORM {sql_error}
END-EVALUATE.

{abend}.

 {abend_call}.""

Standard SQLERROR paragraph. Every generated "WHEN OTHER" branch does
"PERFORM SQLERROR", so this target paragraph must be declared once at the
end of PROCEDURE DIVISION. Minimal manual-standard body: report location +
SQLCODE, then abend via USERABEN.
#
NOTE: SqlErrorGenerator.paragraph_block() emits a different body
(EXEC SQL INCLUDE SQLERROR END-EXEC). Two SQLERROR forms exist in the
codebase. See open decision D-1 and CHK-05.08.
SQLERROR_PARAGRAPH_TEMPLATE = [
 "SQLERROR.",
 " DISPLAY 'SQL ERROR AT : ' SQL-LOCATION.",
 " DISPLAY 'SQLCODE : ' SQLCODE.",
 " CALL USERABEN.",
]

importer; this flag decides which body is actually emitted.
USE_SHARED_SQL_ERROR_GENERATOR = True

Token proving the copybook already supplies the routine.
SQL_ERROR_INCLUDE_TOKEN = "INCLUDE SQLERROR"

MSG_SQLERROR_INJECTED = "Injected standard SQLERROR routine."
MSG_SQLERROR_FROM_COPYBOOK = (
 "SQLERROR routine already supplied by the copybook; not injected."
)

=====
FILE: rules\update_storage_include_rules.py
ENCODING: utf-8
=====

from __future__ import annotations

Update storage/include manager constants. Constants only.
No regex, no runtime logic, no program/record/table names.

Includes that must never be auto-injected as DCLGEN includes.
EXCLUDED_INCLUDE_NAMES = frozenset({"GEN", "SQLCA", "SQLERRWS", "SQLERROR"})

DCLGEN host-group prefix.
DCL_GROUP_PREFIX = "DCL"

Working-storage anchor lines (checked before injecting).
WS_ANCHOR_01_TEMPLATE = "01 {group}."
DATE_WS_ANCHORS = ("01 WS-DATUMVELDEN.", "01 WS-DATUMVELDEN.")

CS-PROGRAM 77-level replacement body (program-id substituted).
CS_PROGRAM_77_TEMPLATE = (
 "{prefix}77 CS-PROGRAM PIC X(8) VALUE '{program_id}'."
)
PROGRAM_NAME_MOVE_REPLACEMENT_TEMPLATE = "MOVE '{program_id}' TO PROGRAM-NAME."

--- Diagnostics / skip messages ---
SKIP_DATA_DIVISION_WS = (
 "Safe DATA DIVISION insertion point not found. Skipped update restart "
 "working-storage injection."
)

```



```

SKIP_DATA_DIVISION_DATE_WS = (
 "Safe DATA DIVISION insertion point not found. Skipped manual-style date "
 "working-storage injection."
)
SKIP_DATA_DIVISION_DCLGEN_TEMPLATE = (
 "Safe DATA DIVISION insertion point not found. Skipped DCLGEN include: "
 "{include}"
)
SKIP_DATA_DIVISION_REFERENCED = (
 "Safe DATA DIVISION insertion point not found. Skipped referenced DCLGEN "
 "include injection."
)
SKIP_DATA_DIVISION_COPYBOOK_TEMPLATE = (
 "Safe DATA DIVISION insertion point not found. Skipped input copybook "
 "include: {include}"
)
RESTART_DCLGEN_UNAVAILABLE = (
 "Restart DCLGEN not available. Skipped restart include and restart SQL "
 "generation."
)
DCLGEN_INCLUDE_EXISTS_TEMPLATE = "DCLGEN include already exists: {include}"
DCLGEN_INCLUDE_INJECTED_TEMPLATE = "Injected DCLGEN include: {include}"
REFERENCED_DCLGEN_INJECTED_TEMPLATE = "{base}: {include}"
COPY_EXISTS_TEMPLATE = "{base}: {include}"
COPY_EXISTS_BY_RECORD_TEMPLATE = (
 "Input copybook include already exists by record name: {record}"
)
COPY_INJECTED_TEMPLATE = "{base}: {include}"

```

```

=====
FILE: rules\validation_display_rules.py
ENCODING: utf-8
=====

```

```

LOCATION: rules/validation_display_rules.py
ACTION: REPLACE ENTIRE FILE

```

""""Validation and diagnostics presentation rules.

Constants only. No regex, no runtime logic, no program / record / table names.

WHY THIS EXISTS

-----  
`validation\_messages` is a single flat list that every generator and composer appends to. It mixes three very different things:

- |                               |                                    |
|-------------------------------|------------------------------------|
| 1. INPUT and MAPPING failures | - the run produced unusable output |
| 2. SKIPS and GAPS             | - the run needs a human decision   |
| 3. PASS NARRATION             | - a composer reporting normal work |

The Validation tab rendered all three through st.warning(), so eighty lines of "a pass did its job" drowned the two lines that mattered.

Classification is PREFIX-FIRST and keyword-second, so a new composer that follows the existing "Prefix: text" convention is categorised correctly without touching this file.

""""

```

from __future__ import annotations

```

```

=====
Severities
=====
SEVERITY_ERROR = "Error"
SEVERITY_WARNING = "Warning"
SEVERITY_INFO = "Info"

```

```

SEVERITY_ORDER = (SEVERITY_ERROR, SEVERITY_WARNING, SEVERITY_INFO)

```

```

SEVERITY_ICONS = {
 SEVERITY_ERROR: "ERROR",
 SEVERITY_WARNING: "WARN",
 SEVERITY_INFO: "INFO",
}

```

```

=====
Prefix -> severity
#
Matched against the text before the first colon, case-insensitively.
A source that always reports failures is classified once here rather
than by scanning its wording.
=====
ERROR_PREFIXES = (
 "mapping validation",
 "production validation",
 "input validation",
 "validation error",
)

WARNING_PREFIXES = (
 "update restart skip",
 "db2 warning",
)

INFO_PREFIXES = (
 "cleanup",
 "safety",
 "counters",
 # MappingScopeFilter rewrites a workbook gap on a table THIS program
 # does not reference. It is a note about the Sheet Mapping, not a
 # conversion failure, so it belongs in the log rather than the Errors
 # band. Without this entry the "Mapping scope" prefix would fall
 # through to the keyword pass and be caught by "no usable".
 "mapping scope",
 "output write",
 "procedure indent",
 "sequence artifact",
 "db2 infrastructure",
 "db2 cursor paragraphs",
 "cursor declaration",
 "cursor join",
 "cursor flow",
 "timestamp generator",
 "sqlerror",
 "program flow analyzer",
 "update sql cleanup",
 "update structure feedback",
 "field reference",
)

=====
Keyword -> severity
#
Applied only when the prefix does not decide. Order matters: the first
matching tuple wins.
=====
ERROR_KEYWORDS = (
 "is required",
 "must contain",
 "failed",
 "cannot ",
 "does not exist",
 "no valid",
 "is empty",
)

WARNING_KEYWORDS = (
 "skipped",
 "warning",
 "missing",
 "unresolved",
 "manual redesign",
 "manual mapping required",
 "no usable",
 "left undeclared",
 "not relocated",
 "untouched",
 "unable to",
 "not found",
 "no deterministic",
)

```

```

)

=====
Category
=====
UNCATEGORISED = "General"
CATEGORY_SEPARATOR = ":"
MAX_CATEGORY_WORDS = 4

Messages carrying no prefix are grouped here rather than each becoming
its own single-item category.
FALLBACK_CATEGORY = "Converter"

=====
UI labels - Validation tab
=====
UI_VALIDATION_HEADER = "## Validation"
UI_VALIDATION_INTRO = (
 "Errors and warnings need action. Info entries are the conversion "
 "log: each one records a pass that ran normally."
)

UI_METRIC_ERRORS = "Errors"
UI_METRIC_WARNINGS = "Warnings"
UI_METRIC_INFO = "Info"
UI_METRIC_TOTAL = "Total"

UI_CLEAN_RUN = "No errors or warnings. The conversion completed cleanly."
UI_NO_MESSAGES = (
 "No validation messages yet. Generate DB2 COBOL from the Main tab."
)
UI_ERRORS_HEADER = "### Errors"
UI_WARNINGS_HEADER = "### Warnings"
UI_INFO_HEADER = "### Conversion log"
UI_INFO_CAPTION = (
 "{count} entries. Each records a pass that completed normally."
)

UI_FILTER_SEVERITY = "Severity"
UI_FILTER_CATEGORY = "Category"
UI_FILTER_SEARCH = "Search text"
UI_FILTER_SEARCH_PLACEHOLDER = "for example: cursor, counter, date"
UI_NO_MATCH = "No message matches the current filter."

=====
UI labels - Diagnostics tab
=====
UI_DIAGNOSTICS_HEADER = "## Diagnostics"
UI_DIAGNOSTICS_INTRO = (
 "What the parsers read from each uploaded file, and what the "
 "converter derived from it."
)

UI_FILES_HEADER = "### Uploaded files"
UI_FILES_EMPTY = "No files loaded. Use Load and Analyze Inputs first."
UI_PARSER_HEADER = "### Parser diagnostics"
UI_PARSER_EMPTY = (
 "No diagnostics available. Click Load and Analyze Inputs first."
)
UI_RAW_HEADER = "Raw log"

COLUMN_ROLE = "Input role"
COLUMN_FILE = "File"
COLUMN_SEVERITY = "Severity"
COLUMN_CATEGORY = "Category"
COLUMN_MESSAGE = "Message"
COLUMN_COUNT = "Entries"

MESSAGE_TABLE_COLUMNS = (
 COLUMN_SEVERITY,
 COLUMN_CATEGORY,
 COLUMN_MESSAGE,
)

=====
Downloads

```

```
=====
UI_DOWNLOAD_VALIDATION = "Download validation report (csv)"
UI_DOWNLOAD_DIAGNOSTICS = "Download diagnostics (txt)"
UI_FILE_VALIDATION = "validation_report.csv"
UI_FILE_DIAGNOSTICS = "diagnostics.txt"
UI_MIME_CSV = "text/csv"
UI_MIME_TEXT = "text/plain"
```

```
WARNING_KEYWORDS = tuple(WARNING_KEYWORDS) + (
 "refused rather than",
 "left for manual review",
 "left unchanged",
 "no readable pic",
 "is not wired into this build",
 "Ø movable field",
)
```

```
=====
FILE: rules\validation_rules.py
ENCODING: utf-8
=====
```

```
"""
Validation rule messages.
Validators import these constants so validation message text stays centralized.
"""
INPUT_VALIDATION_MESSAGES = {
 "missing_sheet_mapping": "Sheet Mapping is required and must contain rows.",
 "missing_dclgen": "At least one DCLGEN file is required.",
 "missing_dolgen": "At least one DCLGEN file is required.",
 "missing_idms_cobol": "IDMS COBOL source text file is required and must contain readable COBOL code.",
}
MAPPING_VALIDATION_MESSAGES = {
 "missing_mapping_rows": "Mapping validation: Sheet Mapping rows are missing.",
 "missing_dclgen_columns": "Mapping validation: DCLGEN columns are missing.",
 "missing_record_mapping": "Mapping validation: DB2 table mapping is missing.",
 "missing_column_mapping": "Mapping validation: DB2 column mapping is missing.",
}
PRODUCTION_VALIDATION_MESSAGES = {
 "todo_host_variable": "Production validation: TODO host variable remains in generated COBOL.",
 "todo_db2": "Production validation: TODO DB2 conversion marker remains in generated COBOL.",
 "error_db2": "Production validation: generated DB2 error marker remains in generated COBOL.",
 "unable_to_declare_cursor": "Production validation: unable-to-declare-cursor marker remains in generated COBOL.",
 "no_fetch_host_variables": "Production validation: cursor FETCH has no host variables.",
 "no_dclgen_hosts": "Production validation: no valid DCLGEN host variables are available for validation.",
}
```

```
=====
FILE: src\idms_db2_phase2__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\analyzers__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\analyzers\field_usage_analyzer.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.analyzers.field_usage_capture_service import (
 FieldUsageCaptureService,
)
```

```

from idms_db2_phase2.analyzers.field_usage_context_mapper import (
 FieldUsageContextMapper,
)
from idms_db2_phase2.analyzers.field_usage_models import (
 FieldUsage,
 FieldUsageAnalysis,
)
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from patterns.field_usage_analysis_patterns import (
 COMMENT_PATTERN,
 CONDITION_PATTERN,
 DIVISION_PATTERN,
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
)
from patterns.sequence_patterns import strip_sequence_numbers

```

```

class FieldUsageAnalyzer:
 """
 Generic COBOL field-usage analyzer.

 Detects:
 - IDMS qualified references: FIELD OF RECORD / FIELD IN RECORD
 - DCLGEN qualified references: FIELD OF DCLGROUP
 - DB2 host references: :DCLGROUP.FIELD / :FIELD OF DCLGROUP
 - MOVE source fields used for output
 - condition fields in IF / WHEN / UNTIL / EVALUATE

 This analyzer does not rewrite COBOL.
 """

 def __init__(
 self,
 mapping_repository: MappingRepository,
 table_name_resolver: TableNameResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.table_name_resolver = table_name_resolver

 self.context_mapper = FieldUsageContextMapper(
 mapping_repository=mapping_repository,
 table_name_resolver=table_name_resolver,
)

 self.mapping_records = self.context_mapper.mapping_records()
 self.group_to_record = self.context_mapper.group_to_record_map()

 self.capture_service = FieldUsageCaptureService(
 mapping_records=self.mapping_records,
 group_to_record=self.group_to_record,
)

 def analyze(
 self,
 cobol_text: str,
) -> FieldUsageAnalysis:
 result = FieldUsageAnalysis()
 inside_procedure = False
 inside_exec_sql = False

 for line_number, raw_line in enumerate(
 str(cobol_text or "").splitlines(),
 start=1,
):
 logical = strip_sequence_numbers(raw_line).strip()

 if not logical:
 continue

 if COMMENT_PATTERN.match(logical):
 continue

 division_match = DIVISION_PATTERN.match(logical)

```

```

 if division_match:
 inside_procedure = division_match.group(1).upper() == "PROCEDURE"
 continue

 if EXEC_SQL_START_PATTERN.match(logical):
 inside_exec_sql = True

 if inside_exec_sql:
 self.capture_service.capture_dclgen_references(
 logical=logical,
 result=result,
 is_condition=False,
 is_output=False,
)

 if EXEC_SQL_END_PATTERN.match(logical):
 inside_exec_sql = False

 continue

 if not inside_procedure:
 continue

 is_condition = bool(CONDITION_PATTERN.match(logical))

 self.capture_service.capture_idms_qualified_references(
 logical=logical,
 result=result,
 is_condition=is_condition,
)

 self.capture_service.capture_dclgen_references(
 logical=logical,
 result=result,
 is_condition=is_condition,
 is_output=False,
)

 self.capture_service.capture_move_usage(
 logical=logical,
 result=result,
)

 self._finalize_all_fields(result)
 self._append_diagnostics(result)

 return result

def _finalize_all_fields(
 self,
 result: FieldUsageAnalysis,
) -> None:
 for usage in result.usage_by_record.values():
 usage.all_fields.update(usage.condition_fields)
 usage.all_fields.update(usage.output_fields)
 usage.all_fields.update(usage.move_source_fields)
 usage.all_fields.update(usage.move_target_fields)
 usage.all_fields.update(usage.dclgen_host_fields)

def _append_diagnostics(
 self,
 result: FieldUsageAnalysis,
) -> None:
 result.diagnostics.append(
 "Field usage analyzer: records with usage detected: "
 f"{len(result.usage_by_record)}"
)

 for record_name, usage in sorted(result.usage_by_record.items()):
 result.diagnostics.append(
 "Field usage analyzer: "
 f"record={record_name}, "
 f"condition={len(usage.condition_fields)}, "
 f"output={len(usage.output_fields)}, "
 f"move_source={len(usage.move_source_fields)}, "
 f"move_target={len(usage.move_target_fields)}, "

```

```

 f"dclgen={len(usage.dclgen_host_fields)}"
)

__all__ = [
 "FieldUsage",
 "FieldUsageAnalysis",
 "FieldUsageAnalyzer",
]

=====
FILE: src\idms_db2_phase2\analyzers\field_usage_capture_service.py
ENCODING: utf-8
=====

from __future__ import annotations

from idms_db2_phase2.analyzers.field_usage_models import (
 FieldUsage,
 FieldUsageAnalysis,
)
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.field_usage_analysis_patterns import (
 DCLGEN_DOT_PATTERN,
 DCLGEN_OF_PATTERN,
 MOVE_PATTERN,
 QUALIFIED_REFERENCE_PATTERN,
)
from rules.field_usage_rules import (
 DCLGEN_GROUP_PREFIX,
 OUTPUT_TARGET_PREFIXES,
)

class FieldUsageCaptureService:
 """
 Captures field usage from individual COBOL logical statements.

 This service does not manage Procedure Division state. It only receives a
 logical line and updates FieldUsageAnalysis.
 """

 def __init__(
 self,
 *,
 mapping_records: set[str],
 group_to_record: dict[str, str],
) -> None:
 self.mapping_records = mapping_records
 self.group_to_record = group_to_record

 def capture_idms_qualified_references(
 self,
 *,
 logical: str,
 result: FieldUsageAnalysis,
 is_condition: bool,
) -> None:
 for match in QUALIFIED_REFERENCE_PATTERN.finditer(logical):
 field_name = NameNormalizer.to_cobol(match.group("field"))
 record_name = NameNormalizer.normalize(match.group("record"))

 if not field_name or not record_name:
 continue

 if record_name.startswith(DCLGEN_GROUP_PREFIX):
 continue

 if record_name not in self.mapping_records:
 continue

 usage = self.usage_for_record(result, record_name)
 usage.all_fields.add(field_name)

 if is_condition:

```

```

 usage.condition_fields.add(field_name)

def capture_dclgen_references(
 self,
 *,
 logical: str,
 result: FieldUsageAnalysis,
 is_condition: bool,
 is_output: bool,
) -> None:
 self._capture_dclgen_of_references(
 logical=logical,
 result=result,
 is_condition=is_condition,
 is_output=is_output,
)
 self._capture_dclgen_dot_references(
 logical=logical,
 result=result,
 is_condition=is_condition,
 is_output=is_output,
)

def capture_move_usage(
 self,
 *,
 logical: str,
 result: FieldUsageAnalysis,
) -> None:
 match = MOVE_PATTERN.search(logical)

 if not match:
 return

 source_text = match.group("source")
 target_text = match.group("target")
 is_output = self._is_output_target(target_text)

 self._capture_move_source_usage(
 source_text=source_text,
 result=result,
 is_output=is_output,
)
 self._capture_move_target_usage(
 target_text=target_text,
 result=result,
)

def usage_for_record(
 self,
 result: FieldUsageAnalysis,
 record_name: str,
) -> FieldUsage:
 record = NameNormalizer.normalize(record_name)

 if record not in result.usage_by_record:
 result.usage_by_record[record] = FieldUsage(record_name=record)

 return result.usage_by_record[record]

def _capture_dclgen_of_references(
 self,
 *,
 logical: str,
 result: FieldUsageAnalysis,
 is_condition: bool,
 is_output: bool,
) -> None:
 for match in DCLGEN_OF_PATTERN.finditer(logical):
 field_name = NameNormalizer.to_cobol(match.group("field"))
 group_name = NameNormalizer.normalize(match.group("group"))
 record_name = self.group_to_record.get(group_name, "")

 if not field_name or not record_name:
 continue

```



```

 usage = self.usage_for_record(result, record_name)
 usage.dclgen_host_fields.add(field_name)

 if is_condition:
 usage.condition_fields.add(field_name)

 if is_output:
 usage.output_fields.add(field_name)

def _capture_dclgen_dot_references(
 self,
 *,
 logical: str,
 result: FieldUsageAnalysis,
 is_condition: bool,
 is_output: bool,
) -> None:
 for match in DCLGEN_DOT_PATTERN.finditer(logical):
 field_name = NameNormalizer.to_cobol(match.group("field"))
 group_name = NameNormalizer.normalize(match.group("group"))
 record_name = self.group_to_record.get(group_name, "")

 if not field_name or not record_name:
 continue

 usage = self.usage_for_record(result, record_name)
 usage.dclgen_host_fields.add(field_name)

 if is_condition:
 usage.condition_fields.add(field_name)

 if is_output:
 usage.output_fields.add(field_name)

def _capture_move_source_usage(
 self,
 *,
 source_text: str,
 result: FieldUsageAnalysis,
 is_output: bool,
) -> None:
 for source_match in QUALIFIED_REFERENCE_PATTERN.finditer(source_text):
 field_name = NameNormalizer.to_cobol(source_match.group("field"))
 record_name = NameNormalizer.normalize(source_match.group("record"))

 if not field_name or not record_name:
 continue

 if record_name.startswith(DCLGEN_GROUP_PREFIX):
 continue

 if record_name not in self.mapping_records:
 continue

 usage = self.usage_for_record(result, record_name)
 usage.move_source_fields.add(field_name)

 if is_output:
 usage.output_fields.add(field_name)

 self.capture_dclgen_references(
 logical=source_text,
 result=result,
 is_condition=False,
 is_output=is_output,
)

def _capture_move_target_usage(
 self,
 *,
 target_text: str,
 result: FieldUsageAnalysis,
) -> None:
 for target_match in QUALIFIED_REFERENCE_PATTERN.finditer(target_text):
 field_name = NameNormalizer.to_cobol(target_match.group("field"))
 record_name = NameNormalizer.normalize(target_match.group("record"))

```

```

 if not field_name or not record_name:
 continue

 if record_name.startswith(DCLGEN_GROUP_PREFIX):
 continue

 if record_name not in self.mapping_records:
 continue

 usage = self.usage_for_record(result, record_name)
 usage.move_target_fields.add(field_name)

 def _is_output_target(
 self,
 target_text: str,
) -> bool:
 upper_target = str(target_text or "").upper()

 return any(
 prefix in upper_target
 for prefix in OUTPUT_TARGET_PREFIXES
)

```

```

=====
FILE: src\idms_db2_phase2\analyzers\field_usage_context_mapper.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.field_usage_rules import DCLGEN_GROUP_PREFIX

class FieldUsageContextMapper:
 """
 Builds reusable record and DCLGEN-group context for field usage analysis.
 """

 def __init__(
 self,
 *,
 mapping_repository: MappingRepository,
 table_name_resolver: TableNameResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.table_name_resolver = table_name_resolver

 def mapping_records(
 self,
) -> set[str]:
 try:
 return {
 NameNormalizer.normalize(record)
 for record in self.mapping_repository.records()
 if NameNormalizer.normalize(record)
 }
 except Exception:
 return set()

 def group_to_record_map(
 self,
) -> dict[str, str]:
 output: dict[str, str] = {}

 for record in self.mapping_records():
 normalized_record = NameNormalizer.normalize(record)
 table = self.table_name_resolver.table_for_record(normalized_record)

 if not table:
 continue

```

```

 group = DCLGEN_GROUP_PREFIX + NameNormalizer.to_cobol(table)
 output[NameNormalizer.normalize(group)] = normalized_record

```

```

 return output

```

```

=====
FILE: src\idms_db2_phase2\analyzers\field_usage_models.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from dataclasses import dataclass, field

```

```

@dataclass
class FieldUsage:
 record_name: str
 condition_fields: set[str] = field(default_factory=set)
 output_fields: set[str] = field(default_factory=set)
 move_source_fields: set[str] = field(default_factory=set)
 move_target_fields: set[str] = field(default_factory=set)
 dclgen_host_fields: set[str] = field(default_factory=set)
 all_fields: set[str] = field(default_factory=set)

```

```

@dataclass
class FieldUsageAnalysis:
 usage_by_record: dict[str, FieldUsage] = field(default_factory=dict)
 diagnostics: list[str] = field(default_factory=list)

```

```

=====
FILE: src\idms_db2_phase2\analyzers\metadata_service.py
ENCODING: utf-8
=====

```

```

from collections import defaultdict

```

```

from idms_db2_phase2.domain.models import (
 RelationshipSummary,
 RecordSummary,
 SheetMappingRow,
)
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class MetadataService:
 """
 Builds metadata summaries from Sheet Mapping rows.

 Used by UI and diagnostics to show:
 - Record summaries
 - Column rows
 - Relationship summaries
 - Sheet Mapping preview rows
 """

 def record_summaries(
 self,
 rows: list[SheetMappingRow],
) -> list[RecordSummary]:
 grouped: dict[str, list[SheetMappingRow]] = defaultdict(list)

 for row in rows:
 record = NameNormalizer.normalize(row.cobol_record_idms)

 if not record:
 continue

 grouped[record].append(row)

 output: list[RecordSummary] = []

 for record_name, record_rows in sorted(grouped.items()):

```

```

 db2_table = self._first_db2_table(record_rows)

 columns = [
 NameNormalizer.normalize(row.new_db2_field_name)
 for row in record_rows
 if NameNormalizer.normalize(row.new_db2_field_name)
]

 key_columns = [
 NameNormalizer.normalize(row.new_db2_field_name)
 for row in record_rows
 if NameNormalizer.normalize(row.new_db2_field_name)
 and (
 NameNormalizer.normalize(row.idms_key)
 or NameNormalizer.normalize(row.db2_key)
)
]

 output.append(
 RecordSummary(
 record_name=record_name,
 db2_table=db2_table,
 column_count=len(set(columns)),
 key_columns=sorted(set(key_columns)),
)
)

 return output

def column_rows(
 self,
 rows: list[SheetMappingRow],
) -> list[dict[str, str]]:
 output: list[dict[str, str]] = []

 for row in rows:
 record = NameNormalizer.normalize(row.cobol_record_idms)
 table = NameNormalizer.normalize(row.new_db2_record)
 column = NameNormalizer.normalize(row.new_db2_field_name)

 if not record and not table and not column:
 continue

 output.append(
 {
 "IDMS Record": record,
 "COBOL Zone": row.cobol_zone,
 "IDMS Key": row.idms_key,
 "IDMS PIC": row.idms_pic_clause,
 "DB2 Key": row.db2_key,
 "DB2 Table": table,
 "DB2 Column": column,
 "DB2 Type": row.new_db2_data_type,
 "Hopex Expression Type": row.hopex_expression_type_remark,
 "Remarks": row.remarks,
 "Relation": row.relation,
 "Reference Field": row.reference_field_name_copybook,
 "Reference PIC": row.reference_field_pic_clause,
 "Cross App DB2 Table": row.cross_application_db2_table,
 "Cross App DB2 Field": row.cross_application_db2_field_name,
 "Cross App DB2 Type": row.cross_application_db2_data_type,
 "Basetype": row.basetype,
 }
)

 return output

def relationship_summaries(
 self,
 rows: list[SheetMappingRow],
) -> list[RelationshipSummary]:
 output: list[RelationshipSummary] = []
 seen: set[tuple[str, str, str, str]] = set()

 for row in rows:
 relation = NameNormalizer.normalize(row.relation)

```

```

 if not relation:
 continue

 child_record = NameNormalizer.normalize(row.cobol_record_idms)
 child_key = NameNormalizer.normalize(row.new_db2_field_name)
 parent_key = NameNormalizer.normalize(
 row.cross_application_db2_field_name
 or row.new_db2_field_name
)
 parent_record = self._infer_parent_record_from_reference(row)

 key = (
 relation,
 parent_record,
 child_record,
 child_key,
)

 if key in seen:
 continue

 seen.add(key)

 output.append(
 RelationshipSummary(
 relation=relation,
 parent_record=parent_record,
 child_record=child_record,
 parent_key=parent_key,
 child_key=child_key,
)
)

return output

def mapping_preview_rows(
 self,
 rows: list[SheetMappingRow],
) -> list[dict[str, str]]:
 output: list[dict[str, str]] = []

 for row in rows:
 output.append(
 {
 "Cobol Record IDMS": row.cobol_record_idms,
 "Cobol Zone": row.cobol_zone,
 "IDMS Key": row.idms_key,
 "IDMS PIC Clause": row.idms_pic_clause,
 "Length of Field Bytes": row.length_of_field_bytes,
 "Field end position": row.field_end_position,
 "DB2 Key": row.db2_key,
 "New DB2 Record": row.new_db2_record,
 "New DB2 Field name": row.new_db2_field_name,
 "New DB2 Data Type": row.new_db2_data_type,
 "Hopex Expression Type": row.hopex_expression_type_remark,
 "Remarks": row.remarks,
 "Relation": row.relation,
 "Reference Field Name": row.reference_field_name_copybook,
 "Reference Field PIC Clause": row.reference_field_pic_clause,
 "Cross Application DB2 Table": row.cross_application_db2_table,
 "Cross Application DB2 Field Name": row.cross_application_db2_field_name,
 "Cross Appln DB2 Data Type": row.cross_application_db2_data_type,
 "Basetype": row.basetype,
 }
)

 return output

def _first_db2_table(
 self,
 rows: list[SheetMappingRow],
) -> str:
 for row in rows:
 table = NameNormalizer.normalize(row.new_db2_record)

```

```

 if table:
 return table

 return ""

def _infer_parent_record_from_reference(
 self,
 row: SheetMappingRow,
) -> str:
 if row.cross_application_db2_table:
 return NameNormalizer.normalize(row.cross_application_db2_table)

 value = row.cross_application_db2_field_name

 if not value:
 return ""

 normalized = NameNormalizer.normalize(value)
 parts = normalized.split("_")

 if len(parts) >= 2:
 return parts[0]

 return ""

```

```

=====
FILE: src\idms_db2_phase2\analyzers\program_flow_analyzer.py
ENCODING: utf-8
=====

```

```

from dataclasses import dataclass, field

```

```

from idms_db2_phase2.domain.models import DclgenColumn, IdmsOperation, SheetMappingRow
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.cobol_patterns import PARAGRAPH_PATTERN, PROCEDURE_DIVISION_PATTERN
from patterns.sequence_patterns import strip_sequence_numbers

```

```

@dataclass
class ParagraphSpan:
 name: str
 start_line: int
 end_line: int
 lines: list[str] = field(default_factory=list)

```

```

@dataclass
class CursorLoop:
 record_name: str = ""
 set_name: str = ""
 operation: str = ""
 operation_line: int = 0
 process_paragraph: str = ""
 perform_line: int = 0
 until_line: int = 0
 loop_type: str = "unknown"
 cursor_name: str = ""
 open_paragraph: str = ""
 fetch_paragraph: str = ""
 close_paragraph: str = ""

```

```

@dataclass
class OutputWrite:
 output_record: str = ""
 paragraph_name: str = ""
 write_line: int = 0
 move_lines: list[str] = field(default_factory=list)

```

```

@dataclass
class DateUsage:
 host_field: str = ""
 dclgen_group: str = ""

```

```

idms_record: str = ""
line_number: int = 0
line_text: str = ""
usage_type: str = ""

```

```
@dataclass
```

```
class ProgramFlowAnalysis:
```

```

 paragraphs: list[ParagraphSpan] = field(default_factory=list)
 cursor_loops: list[CursorLoop] = field(default_factory=list)
 output_writes: list[OutputWrite] = field(default_factory=list)
 date_usages: list[DateUsage] = field(default_factory=list)
 diagnostics: list[str] = field(default_factory=list)

```

```

def process_paragraph_for_cursor(
 self,
 cursor_name: str,
) -> str:
 cursor = NameNormalizer.to_cobol(NameNormalizer.normalize(cursor_name))

 for loop in self.cursor_loops:
 if NameNormalizer.to_cobol(NameNormalizer.normalize(loop.cursor_name)) == cursor:
 return loop.process_paragraph

 return ""

```

```

def loop_for_set(
 self,
 set_name: str,
) -> CursorLoop | None:
 target = NameNormalizer.normalize(set_name)

 for loop in self.cursor_loops:
 if NameNormalizer.normalize(loop.set_name) == target:
 return loop

 return None

```

```

def loop_for_record(
 self,
 record_name: str,
) -> CursorLoop | None:
 target = NameNormalizer.normalize(record_name)

 for loop in self.cursor_loops:
 if NameNormalizer.normalize(loop.record_name) == target:
 return loop

 return None

```

```
class ProgramFlowAnalyzer:
```

```

 """
 Generic flow analyzer for diagnostics.

```

```

 This analyzer does not rewrite COBOL. It detects:
 - procedure paragraphs
 - cursor-like operations from parsed IDMS operations
 - output write statements
 - basic date usage hints
 """

```

```

CURSOR_OPERATIONS = {
 "OBTAIN_FIRST",
 "OBTAIN_NEXT",
 "FIND_FIRST",
}

```

```

WRITE_TOKENS = {
 "WRITE",
 "STORE",
 "MODIFY",
 "ERASE",
}

```

```
DATE_TOKENS = {
```

```

 "DATE",
 "TIME",
 "TIMESTAMP",
 "TS-",
 }

def __init__(
 self,
 mapping_rows: list[SheetMappingRow] | None = None,
 dclgen_columns: list[DclgenColumn] | None = None,
) -> None:
 self.mapping_rows = mapping_rows or []
 self.dclgen_columns = dclgen_columns or []
 self.cursor_name_resolver = CursorNameResolver()

def analyze(
 self,
 cobol_text: str,
 operations: list[IdmsOperation] | None = None,
) -> ProgramFlowAnalysis:
 diagnostics: list[str] = []

 if not str(cobol_text or "").strip():
 return ProgramFlowAnalysis(
 diagnostics=["Program flow analyzer: COBOL text is empty."]
)

 logical_lines = self._logical_lines_with_numbers(cobol_text)
 paragraphs = self._paragraph_spans(logical_lines)
 cursor_loops = self._cursor_loops(operations or [])
 output_writes = self._output_writes(logical_lines, paragraphs)
 date_usages = self._date_usages(logical_lines)

 diagnostics.append(
 f"Program flow analyzer: Sheet Mapping rows received: {len(self.mapping_rows)}"
)
 diagnostics.append(
 f"Program flow analyzer: DCLGEN columns received: {len(self.dclgen_columns)}"
)
 diagnostics.append(
 f"Program flow analyzer: paragraphs detected: {len(paragraphs)}"
)
 diagnostics.append(
 f"Program flow analyzer: cursor loops detected: {len(cursor_loops)}"
)
 diagnostics.append(
 f"Program flow analyzer: output writes detected: {len(output_writes)}"
)
 diagnostics.append(
 f"Program flow analyzer: DB2 date usages detected: {len(date_usages)}"
)

 for loop in cursor_loops[:5]:
 diagnostics.append(
 "Program flow analyzer: loop sample: "
 f"record={loop.record_name}, set={loop.set_name}, "
 f"type={loop.loop_type}, cursor={loop.cursor_name}"
)

 return ProgramFlowAnalysis(
 paragraphs=paragraphs,
 cursor_loops=cursor_loops,
 output_writes=output_writes,
 date_usages=date_usages,
 diagnostics=diagnostics,
)

def _logical_lines_with_numbers(
 self,
 cobol_text: str,
) -> list[tuple[int, str, str]]:
 output: list[tuple[int, str, str]] = []

 for line_number, raw_line in enumerate(cobol_text.splitlines(), start=1):
 logical = strip_sequence_numbers(raw_line)
 output.append((line_number, logical, raw_line.rstrip()))

```



```

return output

def _paragraph_spans(
 self,
 logical_lines: list[tuple[int, str, str]],
) -> list[ParagraphSpan]:
 paragraphs: list[ParagraphSpan] = []
 current: ParagraphSpan | None = None
 inside_procedure = False

 for line_number, logical, raw_line in logical_lines:
 if PROCEDURE_DIVISION_PATTERN.match(logical):
 inside_procedure = True

 if not inside_procedure:
 continue

 match = PARAGRAPH_PATTERN.match(logical.strip())

 if match:
 if current is not None:
 current.end_line = line_number - 1
 paragraphs.append(current)

 current = ParagraphSpan(
 name=match.group("name").upper(),
 start_line=line_number,
 end_line=line_number,
 lines=[raw_line],
)
 continue

 if current is not None:
 current.lines.append(raw_line)
 current.end_line = line_number

 if current is not None:
 paragraphs.append(current)

 return paragraphs

def _cursor_loops(
 self,
 operations: list[IdmsOperation],
) -> list[CursorLoop]:
 output: list[CursorLoop] = []
 cursor_order_by_name: dict[str, int] = {}

 for operation in operations:
 operation_name = str(operation.operation or "").upper()

 if operation_name not in self.CURSOR_OPERATIONS:
 continue

 record_name = NameNormalizer.normalize(operation.record_name)
 set_name = NameNormalizer.normalize(operation.set_name)

 if not record_name:
 continue

 cursor_name = self.cursor_name_resolver.cursor_name_from_table(record_name)

 if cursor_name not in cursor_order_by_name:
 cursor_order_by_name[cursor_name] = len(cursor_order_by_name) + 1

 paragraph_spec = self.cursor_name_resolver.paragraph_names(
 cursor_order=cursor_order_by_name[cursor_name],
 cursor_name=cursor_name,
)

 output.append(
 CursorLoop(
 record_name=record_name,
 set_name=set_name,
 operation=operation_name,

```

```

 operation_line=operation.line_number,
 loop_type="cursor",
 cursor_name=cursor_name,
 open_paragraph=paragraph_spec["open"],
 fetch_paragraph=paragraph_spec["fetch"],
 close_paragraph=paragraph_spec["close"],
)
)

return output

def _output_writes(
 self,
 logical_lines: list[tuple[int, str, str]],
 paragraphs: list[ParagraphSpan],
) -> list[OutputWrite]:
 output: list[OutputWrite] = []
 paragraph_by_line = self._paragraph_name_by_line(paragraphs)

 for line_number, logical, _raw_line in logical_lines:
 upper = logical.strip().upper()

 if not any(upper.startswith(token) for token in self.WRITE_TOKENS):
 continue

 output.append(
 OutputWrite(
 output_record=self._first_word_after_operation(upper),
 paragraph_name=paragraph_by_line.get(line_number, ""),
 write_line=line_number,
)
)

 return output

def _date_usages(
 self,
 logical_lines: list[tuple[int, str, str]],
) -> list[DateUsage]:
 output: list[DateUsage] = []

 for line_number, logical, raw_line in logical_lines:
 upper = logical.upper()

 if not any(token in upper for token in self.DATE_TOKENS):
 continue

 output.append(
 DateUsage(
 line_number=line_number,
 line_text=raw_line,
 usage_type="date-token",
)
)

 return output

def _paragraph_name_by_line(
 self,
 paragraphs: list[ParagraphSpan],
) -> dict[int, str]:
 output: dict[int, str] = {}

 for paragraph in paragraphs:
 for line_number in range(paragraph.start_line, paragraph.end_line + 1):
 output[line_number] = paragraph.name

 return output

def _first_word_after_operation(
 self,
 upper_line: str,
) -> str:
 parts = upper_line.split()

 if len(parts) < 2:

```

```

 return ""

 return NameNormalizer.normalize(parts[1].rstrip("."))

=====
FILE: src\idms_db2_phase2\app.py
ENCODING: utf-8
=====

from pathlib import Path
import sys

import streamlit as st

CURRENT_FILE = Path(__file__).resolve()
SRC_DIR = CURRENT_FILE.parents[1]
PROJECT_ROOT = CURRENT_FILE.parents[2]

for path in [PROJECT_ROOT, SRC_DIR]:
 if str(path) not in sys.path:
 sys.path.insert(0, str(path))

from idms_db2_phase2.ui.main_page import render_main_page

def main() -> None:
 st.set_page_config(
 page_title="IDMS > DB2 Phase 2 Converter",
 layout="wide",
)

 st.title("IDMS > DB2 Phase 2 Converter")

 st.caption(
 "Upload Sheet Mapping, DCLGEN, optional Copybook files, "
 "and IDMS COBOL source code to generate DB2 embedded SQL COBOL."
)

 render_main_page()

if __name__ == "__main__":
 main()

=====
FILE: src\idms_db2_phase2\composers__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_db2_phase2\composers\cleanup\child_fetch_early_stop_cleanup.py
ENCODING: utf-8
=====

"""
Child fetch early-stop cleanup.

Adds an SW-STATUS-D early-stop condition to nested child cursor fetch loops,
keeping the stop condition in the manual-style two-line form.
"""

from idms_db2_phase2.composers.cleanup.cleanup_message_collector import (
 CleanupMessageCollector,
)
from idms_db2_phase2.composers.cleanup.cobol_cleanup_line_utils import (
 CobolCleanupLineUtils,
)
from patterns.cobol_cleanup_patterns import (
 FETCH_PARAGRAPH_NUMBER_PATTERN,
 FETCH_UNTIL_EOC_FULL_SW_PATTERN,

```

```

 FETCH_UNTIL_EOC_OR_SW_PATTERN,
 FETCH_UNTIL_EOC_PATTERN,
 SW_STATUS_D_MOVE_N_PATTERN,
 SW_STATUS_Y_CONTINUATION_PATTERN,
)
from rules.cobol_cleanup_rules import (
 CHILD_FETCH_MINIMUM_NUMBER,
 MOVE_N_LOOKBACK_WINDOW,
)

SW_STATUS_Y_CONTINUATION_INDENT = "

class ChildFetchEarlyStopCleanup:
 def __init__(
 self,
 messages: CleanupMessageCollector,
 line_utils: CobolCleanupLineUtils | None = None,
) -> None:
 self.messages = messages
 self.line_utils = line_utils or CobolCleanupLineUtils()

 def _logical(self, line: str) -> str:
 return self.line_utils.logical(line)

 def _leading_spaces(self, line: str) -> str:
 return self.line_utils.leading_spaces(line)

 def ensure_child_fetch_early_stop(self, text: str) -> str:
 if "SW-STATUS-D" not in text:
 return text

 lines = text.splitlines()
 output: list[str] = []
 changed = False
 index = 0

 while index < len(lines):
 line = lines[index]
 logical = self._logical(line)

 full_match = FETCH_UNTIL_EOC_FULL_SW_PATTERN.match(logical)
 if full_match and self._is_child_or_nested_fetch(
 full_match.group("fetch")
):
 self._append_early_stop(output, line, full_match)
 changed = True
 index += 1
 continue

 or_match = FETCH_UNTIL_EOC_OR_SW_PATTERN.match(logical)
 if or_match and self._is_child_or_nested_fetch(
 or_match.group("fetch")
):
 self._append_early_stop(output, line, or_match)
 changed = True

 if self._next_line_is_sw_status_y_continuation(lines, index):
 index += 2
 else:
 index += 1
 continue

 match = FETCH_UNTIL_EOC_PATTERN.match(logical)
 if match and self._is_child_or_nested_fetch(match.group("fetch")):
 self._append_early_stop(output, line, match)
 changed = True
 index += 1
 continue

 output.append(line)
 index += 1

 if changed:
 self.messages.add("child_fetch_early_stop")

```

```

 return "\n".join(output).rstrip() + "\n"

def _append_early_stop(
 self,
 output: list[str],
 line: str,
 match,
) -> None:
 leading = self._leading_spaces(line)
 fetch_paragraph = match.group("fetch")
 eoc_flag = match.group("eoc")

 if not self._previous_output_has_move_n(output):
 output.append(f"{leading}MOVE 'N' TO SW-STATUS-D")

 output.append(
 f"{leading}PERFORM {fetch_paragraph} UNTIL {eoc_flag} OR"
)
 output.append(
 f"{leading}{SW_STATUS_Y_CONTINUATION_INDENT}SW-STATUS-D = 'Y'"
)

def _next_line_is_sw_status_y_continuation(
 self,
 lines: list[str],
 index: int,
) -> bool:
 if index + 1 >= len(lines):
 return False

 logical = self._logical(lines[index + 1])
 return bool(SW_STATUS_Y_CONTINUATION_PATTERN.match(logical))

def _previous_output_has_move_n(self, output: list[str]) -> bool:
 lookback = output[-MOVE_N_LOOKBACK_WINDOW:]
 return any(
 SW_STATUS_D_MOVE_N_PATTERN.match(self._logical(line))
 for line in lookback
)

def _is_child_or_nested_fetch(self, fetch_paragraph: str) -> bool:
 paragraph = str(fetch_paragraph or "").upper()
 match = FETCH_PARAGRAPH_NUMBER_PATTERN.match(paragraph)

 if not match:
 return False

 try:
 number = int(match.group("number"))
 except ValueError:
 return False

 return number >= CHILD_FETCH_MINIMUM_NUMBER

```

```

=====
FILE: src\idms_db2_phase2\composers\cleanup\cleanup_message_collector.py
ENCODING: utf-8
=====

```

```

"""

```

Cleanup message collector.

A tiny shared container that accumulates COBOL cleanup messages. Sharing one collector across all cleanup passes preserves the composer contract, where `conversion_service` reads `composer.messages` after `compose()`.

```

"""

```

```

from rules.cobol_cleanup_rules import CLEANUP_MESSAGES

```

```

class CleanupMessageCollector:
 def __init__(self) -> None:
 self.messages: list[str] = []

 def reset(self) -> None:

```

```

self.messages = []

def add(self, key: str, **fields: str) -> None:
 template = CLEANUP_MESSAGES[key]
 self.messages.append(template.format(**fields))

=====
FILE: src\idms_db2_phase2\composers\cleanup\cobol_cleanup_line_utils.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/cleanup/cobol_cleanup_line_utils.py
ACTION: REPLACE ENTIRE FILE

"""Shared line helpers for the COBOL cleanup passes.

CORRECTION

is_comment_or_blank() previously classified a line by inspecting the
STRIPPED text:

 return stripped.startswith("*") or stripped.startswith("/")

That only works when strip_sequence_numbers() has already removed the
left sequence number. When it had not - which was every comment and
page-eject line, because the old LEFT_SEQUENCE_PATTERN required
whitespace in the indicator column - a body such as "001690/" was
classified as EXECUTABLE COBOL and the terminator pass added a period.

Classification now reads column 7 of the RAW line, which is where the
COBOL standard puts the indicator, and never depends on stripping having
worked first.
"""

from __future__ import annotations

from patterns.sequence_patterns import (
 is_comment_or_control,
 is_fixed_format_line,
 is_sequence_artifact,
 strip_sequence_numbers,
)

class CobolCleanupLineUtils:
 """Line inspection helpers shared by every cleanup pass."""

 # -----
 # Text extraction
 # -----
 def logical(self, line: str) -> str:
 return strip_sequence_numbers(str(line or "")).strip()

 def leading_spaces(self, line: str) -> str:
 value = str(line or "")
 return value[: len(value) - len(value.lstrip())]

 # -----
 # Classification
 # -----
 def is_comment_or_blank(self, logical: str) -> bool:
 """Legacy signature: classify from already-stripped text.

 Retained because existing passes call it with self._logical(line).
 Prefer is_comment_or_blank_line(), which reads the raw line.
 """
 stripped = str(logical or "").strip()

 if not stripped:
 return True

 if stripped.startswith("*") or stripped.startswith("/"):
 return True

 # A leaked sequence number is not executable COBOL either. Without

```

```

 # this, the terminator pass punctuates it into '001690/.'.
 return bool(is_sequence_artifact(striped))

def is_comment_or_blank_line(self, line: str) -> bool:
 """Classify from the RAW line by reading column 7."""
 text = str(line or "").rstrip("\n")

 if not text.strip():
 return True

 if is_comment_or_control(text):
 return True

 return self.is_sequence_artifact(text)

def is_fixed_line(self, line: str) -> bool:
 return is_fixed_format_line(line)

def is_sequence_artifact(self, line: str) -> bool:
 return is_sequence_artifact(line)

=====
FILE: src\idms_db2_phase2\composers\cleanup\db2_date_output_cleanup.py
ENCODING: utf-8
=====

"""
DB2 date output cleanup.

Converts a DB2 date host move (MOVE DA-/DT- OF DCLxxx TO UIT-/OUT- date
field) into the shared output-date realignment/conversion block before a
WRITE. Supports one-line and two-line MOVE forms.
"""

from idms_db2_phase2.composers.cleanup.cleanup_message_collector import (
 CleanupMessageCollector,
)
from idms_db2_phase2.composers.cleanup.cobol_cleanup_line_utils import (
 CobolCleanupLineUtils,
)
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.cobol_cleanup_patterns import (
 DB2_DATE_MOVE_ONE_LINE_PATTERN,
 DB2_DATE_MOVE_START_PATTERN,
 TO_UIT_DATE_FIELD_PATTERN,
)
from rules.db2_output_date_conversion_rules import (
 DB2_OUTPUT_DATE_CONVERSION_LINE_TEMPLATES,
 DB2_OUTPUT_DATE_HIGH_NUMERIC_LITERAL,
 DB2_OUTPUT_DATE_HIGH_VALUE_LITERAL,
 DB2_OUTPUT_DATE_LOW_VALUE_LITERAL,
)

class Db2DateOutputCleanup:
 def __init__(
 self,
 messages: CleanupMessageCollector,
 line_utils: CobolCleanupLineUtils | None = None,
) -> None:
 self.messages = messages
 self.line_utils = line_utils or CobolCleanupLineUtils()

 def _logical(self, line: str) -> str:
 return self.line_utils.logical(line)

 def _leading_spaces(self, line: str) -> str:
 return self.line_utils.leading_spaces(line)

 def convert_db2_date_moves_to_output_dates(self, text: str) -> str:
 lines = text.splitlines()
 output: list[str] = []
 changed = False
 index = 0

```

```

while index < len(lines):
 line = lines[index]
 logical = self._logical(line)

 one_line = DB2_DATE_MOVE_ONE_LINE_PATTERN.match(logical)

 if one_line:
 field = one_line.group("field")
 group = one_line.group("group")
 target = one_line.group("target")

 if self._should_generate_output_date_conversion(
 field_name=field,
 target_name=target,
):
 output.extend(
 self._date_conversion_lines(
 leading=self._leading_spaces(line),
 field=field,
 group=group,
 target=target,
)
)
 changed = True
 index += 1
 continue

 start_match = DB2_DATE_MOVE_START_PATTERN.match(logical)

 if start_match and index + 1 < len(lines):
 next_logical = self._logical(lines[index + 1])
 target_match = TO_UIT_DATE_FIELD_PATTERN.match(next_logical)

 if target_match:
 field = start_match.group("field")
 group = start_match.group("group")
 target = target_match.group("target")

 if self._should_generate_output_date_conversion(
 field_name=field,
 target_name=target,
):
 output.extend(
 self._date_conversion_lines(
 leading=self._leading_spaces(line),
 field=field,
 group=group,
 target=target,
)
)
 changed = True
 index += 2
 continue

 output.append(line)
 index += 1

if changed:
 self.messages.add("converted_db2_date_move")

return "\n".join(output).rstrip() + "\n"

def _should_generate_output_date_conversion(
 self,
 field_name: str,
 target_name: str,
) -> bool:
 """
 Return True only for DB2 date-host to output-date-field conversion.

 This prevents date ZEROES/SPACES logic from being generated for:
 - non-date output fields
 - DB2 DCLGEN host variables
 - update host moves
 - unrelated MOVE statements
 """

```



```

field = str(field_name or "").strip().upper()
target = str(target_name or "").strip().upper()

if not field or not target:
 return False

if not field.startswith(("DA-", "DT-")):
 return False

if not target.startswith(
 ("UIT-DA-", "UIT-DT-", "OUT-DA-", "OUT-DT-")
):
 return False

if " OF DCL" in target:
 return False

if target.startswith("DCL"):
 return False

return True

def _date_conversion_lines(
 self,
 leading: str,
 field: str,
 group: str,
 target: str,
) -> list[str]:
 field_name = NameNormalizer.to_cobol(field)
 group_name = NameNormalizer.to_cobol(group)
 target_name = NameNormalizer.to_cobol(target)

 if not self._should_generate_output_date_conversion(
 field_name=field_name,
 target_name=target_name,
):
 return []

 output: list[str] = []

 for template in DB2_OUTPUT_DATE_CONVERSION_LINE_TEMPLATES:
 line = template.format(
 indent=leading,
 field_name=field_name,
 group_name=group_name,
 target_name=target_name,
 low_value=DB2_OUTPUT_DATE_LOW_VALUE_LITERAL,
 high_value=DB2_OUTPUT_DATE_HIGH_VALUE_LITERAL,
 high_numeric=DB2_OUTPUT_DATE_HIGH_NUMERIC_LITERAL,
)

 if not line.strip():
 continue

 output.append(line.rstrip())

 return output

```

```

=====
FILE: src\idms_db2_phase2\composers\cleanup\dclgen_include_cleanup.py
ENCODING: utf-8
=====

```

""DCLGEN include cleanup.

Ensures an EXEC SQL INCLUDE exists for every referenced DCLGEN group that the DCLGEN repository knows about but that is not already included.

CORRECTION 1 - third include shape

-----  
This pass emitted its own three-line block with a single leading space, placing EXEC SQL at column 9. CHK-06.07 requires Area B, column 12.

CORRECTION 2 - single-shape duplicate detection

```

_existing_includes recognised one shape only, so an include already
emitted elsewhere was added again. A duplicated DCLGEN include duplicates
every data-name in the group, which the compiler rejects. CHK-08.02
reported this as DZBFARTV x2.

```

#### CORRECTION 3 - borrowed capture group

```

_referenced_dclgen_tables read match.group("group") from
patterns.cobol_cleanup_patterns.DCL_GROUP_PATTERN, which does not declare
that name, raising "IndexError: no such group". It now uses
DCLGEN_GROUP_PATTERN from patterns/db2_include_patterns.py, owned
alongside this feature.

```

#### CORRECTION 4 - include injected outside WORKING-STORAGE

```

_db2_include_insert_index scanned the WHOLE program for the last include
block, then inserted after it.

```

That was safe while every include lived in WORKING-STORAGE. It stopped being safe once the SQLERROR routine adopted the copybook form:

```

* SQLERROR ROUTINE.
 EXEC SQL
 INCLUDE SQLERROR
 END-EXEC.

```

That is an INCLUDE inside an EXEC SQL block, sitting in the PROCEDURE DIVISION at the end of the program. The scan found it as "the last include" and injected DZBFARTV after it, past the end of WORKING-STORAGE. CHK-08.03 reported the include as not sitting inside WORKING-STORAGE.

The scan is now bounded: it stops at LINKAGE SECTION or PROCEDURE DIVISION, whichever comes first, and falls back to inserting at that boundary.

```

from __future__ import annotations

from catalogs.output_sections import DCLGEN_GROUP_PREFIX
from idms_db2_phase2.composers.cleanup.cleanup_message_collector import (
 CleanupMessageCollector,
)
from idms_db2_phase2.composers.cleanup.cobol_cleanup_line_utils import (
 CobolCleanupLineUtils,
)
from idms_db2_phase2.generators.db2_infrastructure.include_renderer import (
 IncludeRenderer,
)
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.db2_include_patterns import DCLGEN_GROUP_PATTERN

```

```

TOKEN_EXEC_SQL = "EXEC SQL"
TOKEN_END_EXEC = "END-EXEC"
TOKEN_INCLUDE = "INCLUDE "

```

```

Boundaries of the area a DCLGEN include may occupy. Matched against the
uppercased logical line, so no regex dependency is introduced.
TOKEN_LINKAGE_SECTION = "LINKAGE SECTION"
TOKEN_PROCEDURE_DIVISION = "PROCEDURE DIVISION"

```

```

DATA_AREA_BOUNDARIES = (
 TOKEN_LINKAGE_SECTION,
 TOKEN_PROCEDURE_DIVISION,
)

```

```

class DclgenIncludeCleanup:
 """Adds a missing DCLGEN include, exactly once, inside WORKING-STORAGE."""

 def __init__(
 self,
 dclgen_repository: DclgenRepository,
 messages: CleanupMessageCollector,
 line_utils: CobolCleanupLineUtils | None = None,
):

```

```

) -> None:
 self.dclgen_repository = dclgen_repository
 self.messages = messages
 self.line_utils = line_utils or CobolCleanupLineUtils()

 # Shared with InfrastructureBlockBuilder and the update storage
 # injector, so shape, indent and duplicate test live in one place.
 self.includes = IncludeRenderer(self.line_utils)

=====
Public entry point
=====
def apply(self, text: str) -> str:
 if not text:
 return ""

 lines = str(text).splitlines()
 existing = self.includes.existing(lines)

 missing = [
 table
 for table in self._referenced_dclgen_tables(lines)
 if table not in existing
 and self.dclgen_repository.has_table(table)
]

 if not missing:
 return text

 include_lines: list[str] = []
 for table in missing:
 include_lines.extend(self.includes.render(table))
 self.messages.add("added_dclgen_include", table=table)

 insert_index = self._db2_include_insert_index(lines)

 if insert_index < 0:
 return text.rstrip() + "\n" + "\n".join(include_lines) + "\n"

 updated = lines[:insert_index] + include_lines + lines[insert_index:]
 return "\n".join(updated).rstrip() + "\n"

=====
Scanning
=====
def _referenced_dclgen_tables(self, lines: list[str]) -> list[str]:
 """Table names implied by DCLGEN host group references.

 DCLDZBFARTV -> DZBFARTV. Order preserved, de-duplicated.
 """
 output: list[str] = []
 seen: set[str] = set()

 for line in lines:
 logical = self._logical(line)
 if not logical:
 continue

 for match in DCLGEN_GROUP_PATTERN.finditer(logical):
 group = NameNormalizer.normalize(
 str(match.group("group") or "")
).upper()

 if not group.startswith(DCLGEN_GROUP_PREFIX):
 continue

 table = group[len(DCLGEN_GROUP_PREFIX):]
 if not table or table in seen:
 continue

 # DCLGEN itself strips to GEN, which is infrastructure,
 # not a table. So do SQLCA, SQLERRWS and SQLERROR.
 if self.includes.is_infrastructure(table):
 continue

 seen.add(table)

```

```

 output.append(table)

 return output

def _existing_includes(self, text: str) -> set[str]:
 """Every include already present, whichever shape carries it.

 Retained for callers outside this class.
 """
 return self.includes.existing(str(text or "").splitlines())

=====
Placement
=====
def _data_area_boundary(self, lines: list[str]) -> int:
 """Index of the first line that ends the DATA DIVISION area.

 A DCLGEN include must sit inside WORKING-STORAGE. Anything from
 LINKAGE SECTION onward is out of bounds, and so is the whole
 PROCEDURE DIVISION.
 """
 for index, line in enumerate(lines):
 logical = self._logical(line).upper()
 if not logical:
 continue

 if logical.startswith(DATA_AREA_BOUNDARIES):
 return index

 return len(lines)

def _db2_include_insert_index(self, lines: list[str]) -> int:
 """Insert after the last include INSIDE the data area.

 Bounded deliberately. The SQLERROR routine carries an
 EXEC SQL INCLUDE SQLERROR block in the PROCEDURE DIVISION, so an
 unbounded scan reported that as the last include and injected the
 DCLGEN include after it, outside WORKING-STORAGE.

 Handles both the single-line and the three-line block form. The
 original tracked only multi-line blocks, so once includes became
 single-line it could not find the end of the group.
 """
 boundary = self._data_area_boundary(lines)

 last_include_end = -1
 in_exec_sql = False
 include_seen_in_block = False

 for index in range(boundary):
 logical = self._logical(lines[index]).upper()
 if not logical:
 continue

 if logical.startswith(TOKEN_EXEC_SQL):
 in_exec_sql = True
 include_seen_in_block = TOKEN_INCLUDE in logical

 if TOKEN_END_EXEC in logical:
 if include_seen_in_block:
 last_include_end = index + 1
 in_exec_sql = False
 include_seen_in_block = False
 continue

 if in_exec_sql and TOKEN_INCLUDE in logical:
 include_seen_in_block = True

 if in_exec_sql and logical.startswith(TOKEN_END_EXEC):
 if include_seen_in_block:
 last_include_end = index + 1
 in_exec_sql = False
 include_seen_in_block = False

 if last_include_end >= 0:
 return last_include_end

```

```

No include yet: place the block immediately before LINKAGE
SECTION or PROCEDURE DIVISION, whichever ends the data area.
if boundary < len(lines):
 return boundary

return -1

=====
Helpers
=====
def _logical(self, line: str) -> str:
 return self.line_utils.logical(line)

=====
FILE: src\idms_db2_phase2\composers\cleanup\error_status_flag_cleanup.py
ENCODING: utf-8
=====

"""
ERROR-STATUS flag cleanup.

Replaces residual IDMS ERROR-STATUS loop control with a DB2 SW-STATUS-D flag
and declares SW-STATUS-D in Working-Storage only when it is needed.
"""

from idms_db2_phase2.composers.cleanup.cleanup_message_collector import (
 CleanupMessageCollector,
)
from idms_db2_phase2.composers.cleanup.cobol_cleanup_line_utils import (
 CobolCleanupLineUtils,
)
from patterns.cobol_cleanup_patterns import (
 ERROR_STATUS_MOVE_PATTERN,
 SW_STATUS_D_PIC_PATTERN,
 WORKING_STORAGE_PATTERN,
 WS_STATUS_PATTERN,
)
from rules.cobol_cleanup_rules import (
 SW_STATUS_D_DECLARATION,
 WS_DB2_FLAGS_GROUP,
)

class ErrorStatusFlagCleanup:
 def __init__(
 self,
 messages: CleanupMessageCollector,
 line_utils: CobolCleanupLineUtils | None = None,
) -> None:
 self.messages = messages
 self.line_utils = line_utils or CobolCleanupLineUtils()

 def _logical(self, line: str) -> str:
 return self.line_utils.logical(line)

 def _leading_spaces(self, line: str) -> str:
 return self.line_utils.leading_spaces(line)

 def replace_error_status_with_flag(self, text: str) -> str:
 output_lines: list[str] = []
 replaced = False

 for line in text.splitlines():
 logical = self._logical(line)

 if ERROR_STATUS_MOVE_PATTERN.match(logical):
 leading = self._leading_spaces(line)
 output_lines.append(f"{leading}MOVE 'Y' TO SW-STATUS-D")
 replaced = True
 continue

 output_lines.append(line)

 if replaced:

```

```

 self.messages.add("replaced_error_status")

 return "\n".join(output_lines).rstrip() + "\n"

def ensure_sw_status_d_declaration(self, text: str) -> str:
 if "SW-STATUS-D" not in text:
 return text

 if SW_STATUS_D_PIC_PATTERN.search(text):
 return text

 lines = text.splitlines()
 output: list[str] = []
 inserted = False

 for line in lines:
 output.append(line)

 if inserted:
 continue

 logical = self._logical(line)

 if WS_STATUS_PATTERN.match(logical):
 output.append(SW_STATUS_D_DECLARATION)
 inserted = True
 self.messages.add("declared_sw_status_d_ws")

 if inserted:
 return "\n".join(output).rstrip() + "\n"

 return self._insert_sw_status_d_after_working_storage(text)

def _insert_sw_status_d_after_working_storage(self, text: str) -> str:
 lines = text.splitlines()
 output: list[str] = []
 inserted = False

 for line in lines:
 output.append(line)

 if inserted:
 continue

 logical = self._logical(line)

 if WORKING_STORAGE_PATTERN.match(logical):
 output.append(WS_DB2_FLAGS_GROUP)
 output.append(SW_STATUS_D_DECLARATION)
 inserted = True
 self.messages.add("declared_sw_status_d_block")

 if inserted:
 return "\n".join(output).rstrip() + "\n"

 return text

```

```

=====
FILE: src\idms_db2_phase2\composers\cleanup\initialize_before_output_cleanup.py
ENCODING: utf-8
=====

```

```

"""

```

Initialize-before-output cleanup.

Ensures an INITIALIZE of the output record appears immediately before the first output-field population MOVE preceding a WRITE, removing any duplicate INITIALIZE for that record inside the population block.

```

"""

```

```

from idms_db2_phase2.composers.cleanup.cleanup_message_collector import (
 CleanupMessageCollector,
)
from idms_db2_phase2.composers.cleanup.cobol_cleanup_line_utils import (
 CobolCleanupLineUtils,
)

```

```

)
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.cobol_cleanup_patterns import (
 INITIALIZE_PATTERN,
 MOVE_START_PATTERN,
 MOVE_TO_OUTPUT_FIELD_PATTERN,
 TO_OUTPUT_FIELD_PATTERN,
 WRITE_PATTERN,
)
)
from rules.cobol_cleanup_rules import (
 INITIALIZE_LOOKBACK_WINDOW,
 OUTPUT_MOVE_SCAN_WINDOW,
 STOP_BACKWARD_SCAN_WORDS,
)
)

class InitializeBeforeOutputCleanup:
 def __init__(
 self,
 messages: CleanupMessageCollector,
 line_utils: CobolCleanupLineUtils | None = None,
) -> None:
 self.messages = messages
 self.line_utils = line_utils or CobolCleanupLineUtils()

 def _logical(self, line: str) -> str:
 return self.line_utils.logical(line)

 def _leading_spaces(self, line: str) -> str:
 return self.line_utils.leading_spaces(line)

 def _is_comment_or_blank(self, logical: str) -> bool:
 return self.line_utils.is_comment_or_blank(logical)

 def ensure_initialize_before_output_population(self, text: str) -> str:
 lines = text.splitlines()
 output = list(lines)
 changed = False

 write_blocks = self._output_write_blocks(output)

 for block in reversed(write_blocks):
 write_index = int(block["write_index"])
 record = str(block["record"])
 first_move_index = int(block["first_move_index"])

 if write_index < 0 or first_move_index < 0:
 continue

 output, removed_count = (
 self._remove_initialize_for_record_in_range(
 lines=output,
 record=record,
 start_index=first_move_index,
 end_index=write_index,
)
)

 if removed_count:
 changed = True
 write_index -= removed_count

 if self._has_initialize_immediately_before(
 lines=output,
 index=first_move_index,
 record=record,
):
 continue

 leading = self._leading_spaces(output[first_move_index])
 output.insert(first_move_index, f"{{leading}}INITIALIZE {{record}}")
 changed = True
 self.messages.add("moved_initialize", record=record)

 if changed:
 return "\n".join(output).rstrip() + "\n"

```

```

 return text

def _output_write_blocks(
 self,
 lines: list[str],
) -> list[dict[str, object]]:
 output: list[dict[str, object]] = []

 for index, line in enumerate(lines):
 logical = self._logical(line)
 match = WRITE_PATTERN.match(logical)

 if not match:
 continue

 record = match.group("record")
 first_move_index = (
 self._first_output_move_block_start_before_write(
 lines=lines,
 write_index=index,
)
)

 output.append(
 {
 "write_index": index,
 "record": record,
 "first_move_index": first_move_index,
 }
)

 return output

def _first_output_move_block_start_before_write(
 self,
 lines: list[str],
 write_index: int,
) -> int:
 start = max(0, write_index - OUTPUT_MOVE_SCAN_WINDOW)
 first_move_start = -1

 for index in range(start, write_index):
 if self._line_targets_output_field(lines[index]):
 move_start = self._move_block_start(
 lines=lines,
 target_index=index,
 search_start=start,
)
 if move_start >= 0:
 if first_move_start < 0 or move_start < first_move_start:
 first_move_start = move_start

 return first_move_start

def _line_targets_output_field(self, line: str) -> bool:
 logical = self._logical(line)

 if MOVE_TO_OUTPUT_FIELD_PATTERN.match(logical):
 return True

 if TO_OUTPUT_FIELD_PATTERN.match(logical):
 return True

 return False

def _move_block_start(
 self,
 lines: list[str],
 target_index: int,
 search_start: int,
) -> int:
 logical = self._logical(lines[target_index])

 if MOVE_TO_OUTPUT_FIELD_PATTERN.match(logical):
 return target_index

```



```

for index in range(target_index, search_start - 1, -1):
 candidate = self._logical(lines[index])
 upper = candidate.upper()

 if MOVE_START_PATTERN.match(candidate):
 return index

 if index == target_index:
 continue

 if self._is_comment_or_blank(candidate):
 continue

 if upper.startswith(STOP_BACKWARD_SCAN_WORDS):
 break

return target_index

def _remove_initialize_for_record_in_range(
 self,
 lines: list[str],
 record: str,
 start_index: int,
 end_index: int,
) -> tuple[list[str], int]:
 output: list[str] = []
 removed_count = 0

 for index, line in enumerate(lines):
 if start_index <= index < end_index:
 logical = self._logical(line)
 match = INITIALIZE_PATTERN.match(logical)

 if match:
 current_record = NameNormalizer.normalize(
 match.group("record")
)
 target_record = NameNormalizer.normalize(record)

 if current_record == target_record:
 removed_count += 1
 continue

 output.append(line)

 return output, removed_count

def _has_initialize_immediately_before(
 self,
 lines: list[str],
 index: int,
 record: str,
) -> bool:
 lookback_start = max(0, index - INITIALIZE_LOOKBACK_WINDOW)
 target_record = NameNormalizer.normalize(record)

 for prior_index in range(lookback_start, index):
 logical = self._logical(lines[prior_index])
 match = INITIALIZE_PATTERN.match(logical)

 if not match:
 continue

 current_record = NameNormalizer.normalize(match.group("record"))

 if current_record == target_record:
 return True

 return False

```

```
=====
FILE: src\idms_db2_phase2\composers\cleanup\output_write_placement_cleanup.py
ENCODING: utf-8
=====
```

"""Output write placement cleanup.

An IDMS child-set loop converts to:

```
PERFORM <open-child>
PERFORM <fetch-child> UNTIL <eoc> OR SW-STATUS-D = 'Y'
PERFORM <close-child>
```

The per-row paragraph performed from the child FETCH paragraph decides the row status only. A guarded output WRITE belongs in the PARENT paragraph, after the child cursor has been closed, because the decision is not final until every child row has been seen.

Left inside the per-row paragraph, a guarded WRITE fires again for every later matching child row and multiplies the output file.

Manual reference shape produced by this pass:

```
PERFORM <close-child>

IF NOT SW-STATUS-D = 'Y'
 IF <status-field> = '<value>'
 PERFORM <write-paragraph>
 END-IF
END-IF
```

This pass moves lines only. It never rewrites a condition, a MOVE or a WRITE, and it does nothing at all when the shape is not recognised:

1. locate child FETCH paragraphs (number >= CHILD\_FETCH\_MINIMUM\_NUMBER),
2. locate the row paragraph each one performs under WHEN ZERO,
3. locate the PERFORM <nnn>-CLOSE-<cursor> line in the parent paragraph,
4. lift a trailing IF ... WRITE ... END-IF block out of the row paragraph,
5. re-indent it and insert it after the child CLOSE, wrapped in the early-stop guard when the parent loop uses the early-stop flag.

No program, paragraph, record, table, cursor or host variable name is hardcoded. Regex lives in patterns/cobol\_cleanup\_patterns.py, constants in rules/cobol\_cleanup\_rules.py.

```
from idms_db2_phase2.composers.cleanup.cleanup_message_collector import (
 CleanupMessageCollector,
)
from idms_db2_phase2.composers.cleanup.cobol_cleanup_line_utils import (
 CobolCleanupLineUtils,
)
from patterns.cobol_cleanup_patterns import (
 END_IF_PATTERN,
 FETCH_PARAGRAPH_NUMBER_PATTERN,
 IF_START_PATTERN,
 PARAGRAPH_HEADER_PATTERN,
 PERFORM_CLOSE_CURSOR_PATTERN,
 PERFORM_CURSOR_PARAGRAPH_PATTERN,
 PERFORM_PARAGRAPH_PATTERN,
 WHEN_ZERO_PATTERN,
 WRITE_STATEMENT_PATTERN,
)
from rules.cobol_cleanup_rules import (
 CHILD_FETCH_MINIMUM_NUMBER,
 EARLY_STOP_FLAG,
 EARLY_STOP_GUARD_CLOSE,
 EARLY_STOP_GUARD_OPEN,
 EARLY_STOP_VALUE,
 ENFORCE_OUTPUT_WRITE_AFTER_CHILD_LOOP,
 FETCH_PARAGRAPH_CURSOR_SEPARATOR,
 NESTED_INDENT_STEP,
 NON_PARAGRAPH_SINGLE_WORDS,
 OUTPUT_WRITE_MOVE_PASS_LIMIT,
 WHEN_ZERO_PERFORM_SCAN_LIMIT,
)
```

```

class OutputWritePlacementCleanup:
 """Moves a guarded output write out of a child-row paragraph."""

 def __init__(
 self,
 messages: CleanupMessageCollector,
 line_utils: CobolCleanupLineUtils | None = None,
) -> None:
 self.messages = messages
 self.line_utils = line_utils or CobolCleanupLineUtils()

 def _logical(self, line: str) -> str:
 return self.line_utils.logical(line)

 def _leading_spaces(self, line: str) -> str:
 return self.line_utils.leading_spaces(line)

 def _is_comment_or_blank(self, logical: str) -> bool:
 return self.line_utils.is_comment_or_blank(logical)

 # -----
 # Public entry point
 # -----
 def move_output_write_after_child_loop(self, text: str) -> str:
 """Relocate every misplaced guarded output write block."""
 if not text or not ENFORCE_OUTPUT_WRITE_AFTER_CHILD_LOOP:
 return text or ""

 lines = (
 str(text)
 .replace("\r\n", "\n")
 .replace("\r", "\n")
 .split("\n")
)

 for _pass in range(OUTPUT_WRITE_MOVE_PASS_LIMIT):
 moved = self._move_one_block(lines)
 if not moved:
 break

 return "\n".join(lines).rstrip() + "\n"

 # -----
 # One relocation
 # -----
 def _move_one_block(self, lines: list[str]) -> bool:
 """Move at most one block. Returns True when the list was changed."""
 spans = self._paragraph_spans(lines)
 if not spans:
 return False

 for fetch_paragraph, fetch_span in spans.items():
 if not self._is_child_fetch(fetch_paragraph):
 continue

 row_paragraph = self._row_paragraph(lines, fetch_span)
 if not row_paragraph or row_paragraph not in spans:
 continue

 cursor = self._cursor_of(fetch_paragraph)
 if not cursor:
 continue

 close_index = self._close_perform_index(
 lines=lines,
 spans=spans,
 cursor=cursor,
)
 if close_index < 0:
 continue

 block = self._trailing_write_block(lines, spans[row_paragraph])
 if not block:
 continue

```

```

 start, end = block

 if self._index_in_span(close_index, (start, end)):
 continue

 self._relocate(
 lines=lines,
 start=start,
 end=end,
 close_index=close_index,
)

 self.messages.add(
 "output_write_moved",
 source=row_paragraph,
 target=self._owner_paragraph(spans, close_index),
)
 return True

 return False

def _relocate(
 self,
 lines: list[str],
 start: int,
 end: int,
 close_index: int,
) -> None:
 block = lines[start : end + 1]
 base_indent = self._leading_spaces(lines[close_index])
 guarded = self._uses_early_stop(lines, close_index)

 rendered = self._render(
 block=block,
 base_indent=base_indent,
 guarded=guarded,
)

 del lines[start : end + 1]

 removed = end - start + 1
 anchor = close_index if close_index < start else close_index - removed

 lines[anchor + 1 : anchor + 1] = rendered

Rendering

def _render(
 self,
 block: list[str],
 base_indent: str,
 guarded: bool,
) -> list[str]:
 """Re-indent the block and optionally wrap it in the early-stop guard."""
 body_indent = (
 base_indent + NESTED_INDENT_STEP if guarded else base_indent
)
 shifted = self._shift_indent(block, body_indent)

 if not guarded:
 return [""] + shifted

 guard_open = EARLY_STOP_GUARD_OPEN.format(
 flag=EARLY_STOP_FLAG,
 value=EARLY_STOP_VALUE,
)

 return (
 [""]
 + [f"{base_indent}{guard_open}"]
 + shifted
 + [f"{base_indent}{EARLY_STOP_GUARD_CLOSE}"]
)

```

```

def _shift_indent(
 self,
 block: list[str],
 target_indent: str,
) -> list[str]:
 """Re-indent a block, preserving its internal nesting."""
 original = self._minimum_indent(block)
 output: list[str] = []

 for line in block:
 logical = self._logical(line)

 if not logical:
 output.append("")
 continue

 current = self._leading_spaces(self._body_of(line))
 extra = current[len(original) :] if len(current) > len(original) else ""
 output.append(f"{target_indent}{extra}{logical}")

 return output

def _minimum_indent(self, block: list[str]) -> str:
 widest = None

 for line in block:
 logical = self._logical(line)
 if not logical:
 continue
 indent = self._leading_spaces(self._body_of(line))
 if widest is None or len(indent) < len(widest):
 widest = indent

 return widest or ""

def _body_of(self, line: str) -> str:
 """The line with sequence numbers stripped but indentation kept."""
 text = str(line or "").rstrip()
 logical = self._logical(text)

 if not logical:
 return ""

 position = text.find(logical)
 return text[position - self._indent_width(text, position) :] if False else (
 text[:position].replace("\t", " ")[-self._indent_width(text, position) :]
 + logical
 if position > 0
 else logical
)

@staticmethod
def _indent_width(text: str, position: int) -> int:
 prefix = text[:position]
 return len(prefix) - len(prefix.rstrip(" "))

Paragraph discovery

def _paragraph_spans(self, lines: list[str]) -> dict[str, tuple[int, int]]:
 """Paragraph name -> (first line index, last line index)."""
 spans: dict[str, tuple[int, int]] = {}
 current = ""
 start = -1

 for index, line in enumerate(lines):
 logical = self._logical(line)

 if self._is_comment_or_blank(logical):
 continue

 if not self._is_paragraph_header(logical):
 continue

 if current:
 spans[current] = (start, index - 1)

```

```

 current = logical.rstrip(".").upper()
 start = index

 if current and start >= 0:
 spans[current] = (start, len(lines) - 1)

 return spans

def _is_paragraph_header(self, logical: str) -> bool:
 text = str(logical or "").strip()

 if not text.endswith("."):
 return False

 if not PARAGRAPH_HEADER_PATTERN.match(text):
 return False

 word = text.rstrip(".").upper()
 return bool(word) and word not in NON_PARAGRAPH_SINGLE_WORDS

def _owner_paragraph(
 self,
 spans: dict[str, tuple[int, int]],
 index: int,
) -> str:
 for name, span in spans.items():
 if self._index_in_span(index, span):
 return name
 return "(unnamed)"

@staticmethod
def _index_in_span(index: int, span: tuple[int, int]) -> bool:
 start, end = span
 return start <= index <= end

Cursor correlation

def _is_child_fetch(self, paragraph: str) -> bool:
 match = FETCH_PARAGRAPH_NUMBER_PATTERN.match(str(paragraph or "").upper())

 if not match:
 return False

 try:
 number = int(match.group("number"))
 except ValueError:
 return False

 return number >= CHILD_FETCH_MINIMUM_NUMBER

@staticmethod
def _cursor_of(paragraph: str) -> str:
 text = str(paragraph or "").upper()
 separator = FETCH_PARAGRAPH_CURSOR_SEPARATOR

 if separator not in text:
 return ""

 return text.split(separator, 1)[1].strip()

def _row_paragraph(
 self,
 lines: list[str],
 fetch_span: tuple[int, int],
) -> str:
 """The business paragraph performed under WHEN ZERO of a FETCH."""
 start, end = fetch_span
 seen_when_zero = False
 scanned = 0

 for index in range(start + 1, min(end + 1, len(lines))):
 logical = self._logical(lines[index])

 if self._is_comment_or_blank(logical):

```

```

 continue

 if WHEN_ZERO_PATTERN.match(logical):
 seen_when_zero = True
 scanned = 0
 continue

 if not seen_when_zero:
 continue

 scanned += 1
 if scanned > WHEN_ZERO_PERFORM_SCAN_LIMIT:
 return ""

 if PERFORM_CURSOR_PARAGRAPH_PATTERN.match(logical):
 continue

 match = PERFORM_PARAGRAPH_PATTERN.match(logical)
 if match:
 return match.group("paragraph").upper()

 return ""

def _close_perform_index(
 self,
 lines: list[str],
 spans: dict[str, tuple[int, int]],
 cursor: str,
) -> int:
 """Index of PERFORM <nnn>-CLOSE-<cursor> in a non-cursor paragraph."""
 for index, line in enumerate(lines):
 logical = self._logical(line)

 match = PERFORM_CLOSE_CURSOR_PATTERN.match(logical)
 if not match:
 continue

 if match.group("cursor").upper() != cursor:
 continue

 owner = self._owner_paragraph(spans, index)
 if self._is_generated_cursor_paragraph(owner):
 continue

 return index

 return -1

def _is_generated_cursor_paragraph(self, paragraph: str) -> bool:
 return bool(
 FETCH_PARAGRAPH_NUMBER_PATTERN.match(str(paragraph or "").upper())
)

def _uses_early_stop(self, lines: list[str], close_index: int) -> bool:
 """True when the loop above the CLOSE uses the early-stop flag."""
 for index in range(close_index - 1, -1, -1):
 logical = self._logical(lines[index]).upper()

 if self._is_comment_or_blank(logical):
 continue

 if EARLY_STOP_FLAG in logical:
 return True

 if self._is_paragraph_header(logical):
 return False

 return False

Write block discovery

def _trailing_write_block(
 self,
 lines: list[str],
 span: tuple[int, int],

```

```

) -> tuple[int, int] | None:
 """The last IF ... WRITE ... END-IF construct of a paragraph."""
 start, end = span

 last = self._last_code_index(lines, start, end)
 if last < 0:
 return None

 if not END_IF_PATTERN.match(self._logical(lines[last])):
 return None

 opener = self._matching_if_index(lines, start, last)
 if opener < 0:
 return None

 block = lines[opener : last + 1]

 if not any(
 WRITE_STATEMENT_PATTERN.match(self._logical(line)) for line in block
):
 return None

 if any(
 PERFORM_CURSOR_PARAGRAPH_PATTERN.match(self._logical(line))
 for line in block
):
 return None

 return opener, last

def _last_code_index(
 self,
 lines: list[str],
 start: int,
 end: int,
) -> int:
 for index in range(min(end, len(lines) - 1), start, -1):
 logical = self._logical(lines[index])
 if self._is_comment_or_blank(logical):
 continue
 return index
 return -1

def _matching_if_index(
 self,
 lines: list[str],
 start: int,
 end_if_index: int,
) -> int:
 """Walk back from an END-IF to its own IF, honouring nesting."""
 depth = 0

 for index in range(end_if_index, start, -1):
 logical = self._logical(lines[index])

 if self._is_comment_or_blank(logical):
 continue

 if END_IF_PATTERN.match(logical):
 depth += 1
 continue

 if IF_START_PATTERN.match(logical):
 depth -= 1
 if depth == 0:
 return index

 return -1

```

```

=====
FILE: src\idms_db2_phase2\composers\cleanup\paragraph_terminator_cleanup.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/cleanup/paragraph_terminator_cleanup.py

```



```
ACTION: REPLACE ENTIRE FILE
```

```
"""
```

Paragraph sentence termination cleanup.

A COBOL paragraph is one or more sentences. When an earlier cleanup pass inserts or lifts a block, the paragraph can be left ending on a bare scope terminator:

```
WRITE UITRECORD
END-IF <- no period
/
710-OPEN-DZBEFFC1.
```

The sentence then runs into the next paragraph header and the compiler rejects the program.

This pass finds the last executable line of every paragraph in the PROCEDURE DIVISION and appends the sentence terminator when it is missing. Comment, page-eject and blank lines are skipped, and a line that already ends the sentence is never touched.

Runs last in CobolCleanupComposer, so it also covers blocks relocated by OutputWritePlacementCleanup.

No program, paragraph, record, table, cursor or host variable name is hardcoded. Regex lives in patterns/cobol\_cleanup\_patterns.py, constants in rules/cobol\_cleanup\_rules.py.

```
"""
```

```
from idms_db2_phase2.composers.cleanup.cleanup_message_collector import (
 CleanupMessageCollector,
)
from idms_db2_phase2.composers.cleanup.cobol_cleanup_line_utils import (
 CobolCleanupLineUtils,
)
from patterns.cobol_cleanup_patterns import (
 PARAGRAPH_HEADER_PATTERN,
 PROCEDURE_DIVISION_LINE_PATTERN,
 SECTION_HEADER_PATTERN,
)
from patterns.sequence_patterns import strip_sequence_numbers
from rules.cobol_cleanup_rules import (
 ENFORCE_PARAGRAPH_TERMINATION,
 NON_PARAGRAPH_SINGLE_WORDS,
 PARAGRAPH_TERMINATOR,
)
```

```
class ParagraphTerminatorCleanup:
 def __init__(
 self,
 messages: CleanupMessageCollector,
 line_utils: CobolCleanupLineUtils | None = None,
) -> None:
 self.messages = messages
 self.line_utils = line_utils or CobolCleanupLineUtils()

 def _logical(self, line: str) -> str:
 return self.line_utils.logical(line)

 def _is_comment_or_blank(self, logical: str) -> bool:
 return self.line_utils.is_comment_or_blank(logical)

 # -----
 # Public entry point
 # -----
 def ensure_paragraph_termination(self, text: str) -> str:
 if not text or not ENFORCE_PARAGRAPH_TERMINATION:
 return text or ""

 lines = (
 str(text)
 .replace("\r\n", "\n")
 .replace("\r", "\n")
 .split("\n")
```

```

)

procedure_index = self._procedure_division_index(lines)
if procedure_index < 0:
 return text

output = list(lines)
current_paragraph = ""
last_code_index = -1
changed = False

for index in range(procedure_index + 1, len(output)):
 logical = self._logical(output[index])

 if self._is_comment_or_blank(logical):
 continue

 if self._is_paragraph_header(logical):
 changed = (
 self._terminate(output, last_code_index, current_paragraph)
 or changed
)
 current_paragraph = logical.rstrip(PARAGRAPH_TERMINATOR)
 last_code_index = -1
 continue

 last_code_index = index

Close the final paragraph of the program.
changed = (
 self._terminate(output, last_code_index, current_paragraph)
 or changed
)

if not changed:
 return text

return "\n".join(output).rstrip() + "\n"

Termination

def _terminate(
 self,
 output: list[str],
 index: int,
 paragraph: str,
) -> bool:
 """Append the sentence terminator when it is missing."""
 if index < 0:
 return False

 line = output[index]
 logical = self._logical(line)

 if not logical:
 return False

 if logical.endswith(PARAGRAPH_TERMINATOR):
 return False

 body = strip_sequence_numbers(str(line or "")).rstrip()

 if not body.strip():
 return False

 output[index] = body + PARAGRAPH_TERMINATOR

 self.messages.add(
 "paragraph_terminated",
 paragraph=paragraph or "(unnamed)",
)
 return True

Structure detection

```

```

def _is_paragraph_header(self, logical: str) -> bool:
 text = str(logical or "").strip()

 if not text.endswith(PARAGRAPH_TERMINATOR):
 return False

 if SECTION_HEADER_PATTERN.match(text):
 return True

 if not PARAGRAPH_HEADER_PATTERN.match(text):
 return False

 word = text.rstrip(PARAGRAPH_TERMINATOR).upper()
 return bool(word) and word not in NON_PARAGRAPH_SINGLE_WORDS

def _procedure_division_index(self, lines: list[str]) -> int:
 for index, line in enumerate(lines):
 logical = self._logical(line)

 if self._is_comment_or_blank(logical):
 continue

 if PROCEDURE_DIVISION_LINE_PATTERN.match(logical):
 return index

 return -1

=====
FILE: src\idms_db2_phase2\composers\cleanup\residual_idms_comment_cleanup.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/cleanup/residual_idms_comment_cleanup.py
ACTION: CREATE NEW FILE

"""
Residual IDMS comment cleanup.

Removes advisory comment noise left by the IDMS->DB2 transformer, such as:
* DB2: Removed residual IDMS control statement: IDMS-CONTROL
* SECTION.
* DB2: Removed IDMS FIND CURRENT statement: FIND CURRENT FFRECAB
* DB2: IDMS record FFRECAB was not converted automatically.
* DB2: Missing Sheet Mapping and DCLGEN metadata.
* DB2: Restart/control logic requires manual DB2 redesign.

Safety:
- Only comment lines are removed (never executable code).
- A CONTINUE. left after a removed IDMS statement is removed only when it is
 redundant (another executable statement follows in the same block and the
 CONTINUE. is not the sole statement of an IF/ELSE/WHEN/EVALUATE branch).
"""

from __future__ import annotations

from idms_db2_phase2.composers.cleanup.cleanup_message_collector import (
 CleanupMessageCollector,
)
from idms_db2_phase2.composers.cleanup.cobol_cleanup_line_utils import (
 CobolCleanupLineUtils,
)
from patterns.cobol_cleanup_patterns import (
 BLOCK_OPENER_PATTERN,
 LONE_CONTINUE_PATTERN,
 RESIDUAL_IDMS_COMMENT_CONTINUATION_PATTERN,
 RESIDUAL_IDMS_COMMENT_PATTERN,
)

UPDATE the import block at the top to include the new pattern:
from patterns.cobol_cleanup_patterns import (
 BLOCK_OPENER_PATTERN,
 LONE_CONTINUE_PATTERN,
 REMOVED_OBTAIN_CALC_COMMENT_PATTERN,
 RESIDUAL_IDMS_COMMENT_CONTINUATION_PATTERN,
)

```

```

 RESIDUAL_IDMS_COMMENT_PATTERN,
)

class ResidualIdmsCommentCleanup:
 def __init__(
 self,
 messages: CleanupMessageCollector,
 line_utils: CobolCleanupLineUtils | None = None,
) -> None:
 self.messages = messages
 self.line_utils = line_utils or CobolCleanupLineUtils()

 def _logical(self, line: str) -> str:
 return self.line_utils.logical(line)

 def remove_residual_idms_comments(self, text: str) -> str:
 lines = text.splitlines()
 output: list[str] = []
 removed_comment = False
 index = 0

 while index < len(lines):
 line = lines[index]
 logical = self._logical(line)

 if (
 RESIDUAL_IDMS_COMMENT_PATTERN.match(logical)
 or REMOVED_OBTAIN_CALC_COMMENT_PATTERN.match(logical)
):
 index = self._skip_comment_group(lines, index)
 removed_comment = True
 index = self._maybe_skip_redundant_continue(
 lines=lines,
 index=index,
 output=output,
)
 continue

 output.append(line)
 index += 1

 if removed_comment:
 self.messages.add("removed_residual_idms_comment")

 return "\n".join(output).rstrip() + "\n"

 def _skip_comment_group(
 self,
 lines: list[str],
 index: int,
) -> int:
 """
 Skip the residual DB2 comment line plus any immediate continuation or
 related DB2 comment lines (residual, OBTAIN CALC removal, or wrapped).
 """
 index += 1
 while index < len(lines):
 logical = self._logical(lines[index])
 if RESIDUAL_IDMS_COMMENT_PATTERN.match(logical):
 index += 1
 continue
 if REMOVED_OBTAIN_CALC_COMMENT_PATTERN.match(logical):
 index += 1
 continue
 if RESIDUAL_IDMS_COMMENT_CONTINUATION_PATTERN.match(logical):
 index += 1
 continue
 break
 return index

 def _maybe_skip_redundant_continue(
 self,
 lines: list[str],
 index: int,
 output: list[str],
) -> int:

```

```

"""
If the next executable line is a lone CONTINUE., remove it only when
it is safe: it must NOT be the sole statement of an IF/ELSE/WHEN/
EVALUATE branch, and another executable statement (or END-IF/END-
EVALUATE/period) must not depend on it.
"""
if index >= len(lines):
 return index

logical = self._logical(lines[index])
if not LONE_CONTINUE_PATTERN.match(logical):
 return index

Look at the previous emitted executable line.
prev_opener = self._previous_is_block_opener(output)

Look ahead to the next non-blank, non-comment logical line.
next_logical = self._next_executable_logical(lines, index + 1)
next_upper = next_logical.upper()

closes_block = (
 next_upper.startswith("END-IF")
 or next_upper.startswith("END-EVALUATE")
 or next_upper.startswith("ELSE")
 or next_upper.startswith("WHEN")
 or next_upper == "."
)

If CONTINUE. is the only body of a branch, KEEP it (branch must
not be empty).
if prev_opener and closes_block:
 return index

Otherwise it is redundant -> skip it.
self.messages.add("removed_orphan_continue")
return index + 1

def _previous_is_block_opener(self, output: list[str]) -> bool:
 for line in reversed(output):
 logical = self._logical(line)
 if not logical:
 continue
 if logical.startswith("*"):
 continue
 return bool(BLOCK_OPENER_PATTERN.match(logical))
 return False

def _next_executable_logical(
 self,
 lines: list[str],
 start_index: int,
) -> str:
 for index in range(start_index, len(lines)):
 logical = self._logical(lines[index])
 if not logical:
 continue
 if logical.startswith("*"):
 continue
 return logical
 return ""

```

```

=====
FILE: src\idms_db2_phase2\composers\cobol_cleanup_composer.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/cobol_cleanup_composer.py
ACTION: REPLACE ENTIRE FILE

```

```

"""
COBOL post-conversion cleanup composer.

```

```

Applies only safe post-conversion COBOL cleanup passes:
- Remove residual IDMS conversion comment noise (+ redundant CONTINUE).
- Replace residual IDMS ERROR-STATUS loop control with DB2 flag control.

```

- Declare SW-STATUS-D only when needed.
- Add SW-STATUS-D early stop to nested child cursor fetch loops.
- Move a guarded output write out of the child-row paragraph to the parent paragraph, after the child cursor CLOSE.
- Add INITIALIZE before output-record population.
- Convert DB2 date host fields to output numeric date format before WRITE.
- Ensure DCLGEN INCLUDE exists when a DCLGEN group is referenced.
- Close any paragraph sentence left unterminated by an earlier pass.

No program names, table names, record names, or business-specific fields are hardcoded. Exposes the accumulated messages list for conversion\_service.

Pass ordering notes

-----  
 `OutputWritePlacementCleanup` must run AFTER `ChildFetchEarlyStopCleanup`, because it reads the early-stop flag from the child fetch loop to decide whether the relocated block needs the `IF NOT SW-STATUS-D = 'Y'` guard, and BEFORE `InitializeBeforeOutputCleanup` and `Db2DateOutputCleanup`, so those two passes see the write block in its final position.

`ParagraphTerminatorCleanup` runs LAST. Any pass that inserts or lifts a block can leave a paragraph ending on a bare scope terminator, which makes the sentence run into the next paragraph header and the compiler reject the program. This pass is the final guard for that.

"""

```
from idms_db2_phase2.composers.cleanup.child_fetch_early_stop_cleanup import (
 ChildFetchEarlyStopCleanup,
)
from idms_db2_phase2.composers.cleanup.cleanup_message_collector import (
 CleanupMessageCollector,
)
from idms_db2_phase2.composers.cleanup.cobol_cleanup_line_utils import (
 CobolCleanupLineUtils,
)
from idms_db2_phase2.composers.cleanup.db2_date_output_cleanup import (
 Db2DateOutputCleanup,
)
from idms_db2_phase2.composers.cleanup.dclgen_include_cleanup import (
 DclgenIncludeCleanup,
)
from idms_db2_phase2.composers.cleanup.error_status_flag_cleanup import (
 ErrorStatusFlagCleanup,
)
from idms_db2_phase2.composers.cleanup.initialize_before_output_cleanup import (
 InitializeBeforeOutputCleanup,
)
from idms_db2_phase2.composers.cleanup.output_write_placement_cleanup import (
 OutputWritePlacementCleanup,
)
from idms_db2_phase2.composers.cleanup.paragraph_terminator_cleanup import (
 ParagraphTerminatorCleanup,
)
from idms_db2_phase2.composers.cleanup.residual_idms_comment_cleanup import (
 ResidualIdmsCommentCleanup,
)
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
```

```
class CobolCleanupComposer:
 def __init__(
 self,
 dclgen_repository: DclgenRepository,
) -> None:
 self.dclgen_repository = dclgen_repository
 self._collector = CleanupMessageCollector()
 line_utils = CobolCleanupLineUtils()

 self._residual_comment = ResidualIdmsCommentCleanup(
 messages=self._collector,
 line_utils=line_utils,
)
 self._dclgen_include = DclgenIncludeCleanup(
 dclgen_repository=dclgen_repository,
 messages=self._collector,
 line_utils=line_utils,
```

```

)
 self._error_status = ErrorStatusFlagCleanup(
 messages=self._collector,
 line_utils=line_utils,
)
 self._child_fetch = ChildFetchEarlyStopCleanup(
 messages=self._collector,
 line_utils=line_utils,
)
 self._output_write_placement = OutputWritePlacementCleanup(
 messages=self._collector,
 line_utils=line_utils,
)
 self._initialize_output = InitializeBeforeOutputCleanup(
 messages=self._collector,
 line_utils=line_utils,
)
 self._db2_date_output = Db2DateOutputCleanup(
 messages=self._collector,
 line_utils=line_utils,
)
 self._paragraph_terminator = ParagraphTerminatorCleanup(
 messages=self._collector,
 line_utils=line_utils,
)
)

@property
def messages(self) -> list[str]:
 return self._collector.messages

def compose(self, text: str) -> str:
 self._collector.reset()
 output = str(text or "")

 if not output.strip():
 return output

 # Category G: strip residual IDMS comment noise first.
 output = self._residual_comment.remove_residual_idms_comments(output)

 output = self._dclgen_include.apply(output)
 output = self._error_status.replace_error_status_with_flag(output)
 output = self._error_status.ensure_sw_status_d_declaration(output)
 output = self._child_fetch.ensure_child_fetch_early_stop(output)

 # Must follow the early-stop pass: the guard around the relocated
 # block is derived from the child fetch loop condition.
 output = (
 self._output_write_placement
 .move_output_write_after_child_loop(output)
)

 output = (
 self._initialize_output
 .ensure_initialize_before_output_population(output)
)
 output = self._db2_date_output.convert_db2_date_moves_to_output_dates(
 output
)

 # Final guard: no paragraph may end on an unterminated sentence.
 output = self._paragraph_terminator.ensure_paragraph_termination(
 output
)

 return output.rstrip() + "\n"

```

```

=====
FILE: src\idms_db2_phase2\composers\cobol_formatter.py
ENCODING: utf-8
=====

```

```

class CobolFormatter:
 """
 Preserve-first COBOL formatter.

```

This formatter intentionally does NOT re-indent or restructure the full COBOL program.

Reason:

- The original IDMS COBOL source is the authority for manual style.
- The converter must preserve existing indentation, comments, sequence-like lines, paragraph spacing, and business-flow layout.
- Only generated DB2 blocks should be inserted or normalized by generators and composers.

This class therefore performs only safe text cleanup:

- Normalize Windows/Unix line endings.
- Remove trailing spaces.
- Limit excessive blank lines.
- Ensure one final newline.

It must not:

- Strip sequence numbers.
- Re-indent PROCEDURE DIVISION.
- Reformat original business logic.
- Move paragraphs.
- Add or remove periods.
- Change field references.

"""

```
def format(
 self,
 text: str,
) -> str:
 if not text:
 return ""

 normalized = self._normalize_line_endings(text)
 normalized = self._rstrip_lines(normalized)
 normalized = self._normalize_excess_blank_lines(normalized)

 return normalized.rstrip() + "\n"

def _normalize_line_endings(
 self,
 text: str,
) -> str:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

def _rstrip_lines(
 self,
 text: str,
) -> str:
 return "\n".join(
 line.rstrip()
 for line in str(text or "").splitlines()
)

def _normalize_excess_blank_lines(
 self,
 text: str,
) -> str:
 output = str(text or "")

 while "\n\n\n\n" in output:
 output = output.replace("\n\n\n\n", "\n\n\n")

 return output
```

```
=====
FILE: src\idms_db2_phase2\composers\counter_declaration__init__.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/counter_declaration/__init__.py
ACTION: CREATE NEW FILE
"""Placement and rendering helpers for generated row counters.
```

CounterDeclarationComposer decides WHAT to declare.



This package decides WHERE it goes and HOW it is rendered.

"""

```
from idms_db2_phase2.composers.counter_declaration.counter_anchor import (
 CounterAnchor,
 CounterAnchorFinder,
)
from idms_db2_phase2.composers.counter_declaration.counter_anchor_resolver import (
 CounterAnchorResolver,
)
from idms_db2_phase2.composers.counter_declaration.counter_line_factory import (
 CounterLineFactory,
)
from idms_db2_phase2.composers.counter_declaration.counter_totals_writer import (
 CounterTotalsWriter,
)
```

```
__all__ = [
 "CounterAnchor",
 "CounterAnchorFinder",
 "CounterAnchorResolver",
 "CounterLineFactory",
 "CounterTotalsWriter",
]
```

```
=====
FILE: src\idms_db2_phase2\composers\counter_declaration\counter_anchor.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/counter_declaration/counter_anchor.py
ACTION: CREATE NEW FILE
"""The place a counter declaration will be written.
```

```
CORRECTION - level number landed in Area A
```

```

```

The anchor previously cloned the 01 HEADER. A header sits in Area A with a body indent of 0, so the generated line read

```
10 WS-NB-OUTPUT-COUNT
```

at column 8 while every sibling sat at column 12. CHK-06.05 flagged it.

The template is now the group's FIRST SUBORDINATE entry, so the level number AND the indent are both inherited from a real sibling.

"""

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
from idms_db2_phase2.composers.counter_declaration.counter_line_factory import (
 CounterLineFactory,
)
from patterns.counter_declaration_patterns import (
 GROUP_HEADER_PATTERN,
 SECTION_OR_DIVISION_PATTERN,
 SUBORDINATE_ENTRY_PATTERN,
)
```

```
@dataclass(frozen=True)
```

```
class CounterAnchor:
```

```
 """Where and how a counter declaration is rendered."""
```

```
 insert_index: int = -1
```

```
 template_line: str = ""
```

```
 level: str = ""
```

```
 indent: str = ""
```

```
 group: str = ""
```

```
 @property
```

```
 def is_usable(self) -> bool:
```

```
 return self.insert_index >= 0 and bool(self.level)
```

```

class CounterAnchorFinder:
 """Locates the first subordinate entry of an 01 group."""

 def __init__(
 self,
 line_factory: CounterLineFactory | None = None,
) -> None:
 self.lines = line_factory or CounterLineFactory()

 def first_child(
 self,
 lines: list[str],
 header_index: int,
) -> tuple[int, str] | None:
 """(index, level) of the group's first subordinate entry.

 Returns None when the group is empty, or when the next
 declaration belongs to another group or another section.
 """
 for index in range(header_index + 1, len(lines)):
 line = lines[index]

 if self.lines.is_skippable(line):
 continue

 logical = self.lines.logical(line)
 if not logical:
 continue

 if GROUP_HEADER_PATTERN.match(logical):
 return None

 if SECTION_OR_DIVISION_PATTERN.match(logical):
 return None

 match = SUBORDINATE_ENTRY_PATTERN.match(logical)
 if match:
 return index, match.group("level")

 return None

 return None

 def anchor_for(
 self,
 lines: list[str],
 header_index: int,
 group_name: str,
) -> CounterAnchor | None:
 child = self.first_child(lines, header_index)
 if child is None:
 return None

 child_index, level = child
 template = lines[child_index]

 return CounterAnchor(
 insert_index=header_index + 1,
 template_line=template,
 level=level,
 indent=self.lines.body_indent(template),
 group=group_name,
)

__all__ = ["CounterAnchor", "CounterAnchorFinder"]

=====
FILE: src\idms_db2_phase2\composers\counter_declaration\counter_anchor_resolver.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/counter_declaration/counter_anchor_resolver.py
ACTION: CREATE NEW FILE

```

""Safety-first placement for generated row counters.

CORRECTION - counter declared inside an output record

-----

Placement previously took the FIRST 01 group in WORKING-STORAGE. On a program whose first group is the output record

```
01 REC-FORM.
05 F-FORM PIC X(478).
```

that produced

```
01 REC-FORM.
05 WS-NB-OUTPUT-COUNT PIC 9(7) COMP-3 VALUE ZEROES.
05 F-FORM PIC X(478).
```

shifting every byte of a fixed-length record. Every written record was corrupted, silently.

Resolution order

-----

1. reuse the generated counter group when a previous run made it,
2. otherwise reuse the LAST 01 group that is provably a work area,
3. otherwise create a dedicated 01 group at the WORKING-STORAGE boundary.

A record layout can never gain a field, in any branch.

""

```
from __future__ import annotations
```

```
from idms_db2_phase2.composers.counter_declaration.counter_anchor import (
 CounterAnchor,
 CounterAnchorFinder,
)
from idms_db2_phase2.composers.counter_declaration.counter_line_factory import (
 CounterLineFactory,
)
from idms_db2_phase2.services.working_storage_group_selector import (
 WorkingStorageGroupSelector,
)
from rules.counter_declaration_rules import FALLBACK_CHILD_LEVEL
from rules.structural_safety_rules import (
 ENFORCE_SAFE_COUNTER_GROUP,
 GENERATED_COUNTER_CHILD_INDENT,
 GENERATED_COUNTER_GROUP_MARKER,
 GENERATED_COUNTER_GROUP_NAME,
 GENERATED_COUNTER_GROUP_TEMPLATE,
 PREFER_GENERATED_COUNTER_GROUP,
 STRUCTURAL_SAFETY_MESSAGES,
)
```

```
CREATED_GROUP_OFFSET = 3
```

```
class CounterAnchorResolver:
```

```
 """Chooses a safe anchor, creating a group when none is safe."""
```

```
 def __init__(
 self,
 line_factory: CounterLineFactory | None = None,
 selector: WorkingStorageGroupSelector | None = None,
 finder: CounterAnchorFinder | None = None,
) -> None:
 self.lines = line_factory or CounterLineFactory()
 self.selector = selector or WorkingStorageGroupSelector(
 fixed_format=self.lines.fixed_format,
)
 self.finder = finder or CounterAnchorFinder(self.lines)
 self.messages: list[str] = []
```

```
----- public
```

```
def resolve(
 self,
 lines: list[str],
) -> tuple[list[str], CounterAnchor | None]:
 self.messages = []
```

```

layout = self.selector.analyze(lines)

if not ENFORCE_SAFE_COUNTER_GROUP:
 group = layout.groups[0] if layout.groups else None
 if group is None:
 return lines, None
 return lines, self.finder.anchor_for(
 lines, group.header_index, group.name
)

anchor = self._reuse_generated_group(lines, layout)
if anchor is not None:
 return lines, anchor

A business group such as 01 WS-DATUMS passes the record-layout
test but reads wrong: a row counter is not a date field. The
dedicated group is preferred unless the site opts out.
if not PREFER_GENERATED_COUNTER_GROUP:
 anchor = self._reuse_safe_group(lines, layout)
 if anchor is not None:
 return lines, anchor

self._report_rejected(layout)
return self._create_group(lines, layout.boundary_index)

----- 1
def _reuse_generated_group(self, lines, layout) -> CounterAnchor | None:
 for group in layout.groups:
 if group.name != GENERATED_COUNTER_GROUP_NAME:
 continue
 anchor = self.finder.anchor_for(
 lines, group.header_index, group.name
)
 if anchor is not None:
 self._log("counter_group_reused", group=group.name)
 return anchor
 return None

----- 2
def _reuse_safe_group(self, lines, layout) -> CounterAnchor | None:
 for group in reversed(layout.safe_groups()):
 anchor = self.finder.anchor_for(
 lines, group.header_index, group.name
)
 if anchor is not None:
 self._log("counter_group_reused", group=group.name)
 return anchor
 return None

----- 3
def _create_group(
 self,
 lines: list[str],
 boundary_index: int,
) -> tuple[list[str], CounterAnchor]:
 insert_at = boundary_index
 if insert_at < 0 or insert_at > len(lines):
 insert_at = len(lines)

 reference = lines[max(0, insert_at - 1)]
 marker = self.lines.comment(reference, GENERATED_COUNTER_GROUP_MARKER)
 header = self.lines.statement(
 reference,
 GENERATED_COUNTER_GROUP_TEMPLATE.format(
 name=GENERATED_COUNTER_GROUP_NAME,
),
)

 updated = lines[:insert_at] + [marker, header] + lines[insert_at:]
 self._log("counter_group_created", group=GENERATED_COUNTER_GROUP_NAME)

 return updated, CounterAnchor(
 insert_index=insert_at + CREATED_GROUP_OFFSET,
 template_line=header,
 level=FALLBACK_CHILD_LEVEL,
 indent=GENERATED_COUNTER_CHILD_INDENT,
)

```

```

 group=GENERATED_COUNTER_GROUP_NAME,
)

----- helpers
def _report_rejected(self, layout) -> None:
 for group in layout.groups:
 if group.is_record_layout:
 self._log("counter_group_rejected", group=group.name)

def _log(self, key: str, **values) -> None:
 template = STRUCTURAL_SAFETY_MESSAGES.get(key, "")
 if template:
 self.messages.append(template.format(**values))

__all__ = ["CounterAnchorResolver"]

=====
FILE: src\idms_db2_phase2\composers\counter_declaration\counter_line_factory.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/counter_declaration/counter_line_factory.py
ACTION: CREATE NEW FILE
"""Fixed-format line rendering for generated counter lines.

Owns no rules and no regex. Every generated line borrows the sequence
area of a template line; FinalSequenceResequencerService rewrites
columns 1-6 and 73-80 afterwards, so borrowed numbers are placeholders.
"""

from __future__ import annotations

from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)

FALLBACK_COMMENT_PREFIX = " *"

class CounterLineFactory:
 """Builds statement, comment and blank lines from a template."""

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()

 # ----- public
 def statement(self, template: str, body: str) -> str:
 """An executable line carrying `body` in columns 8-72."""
 left, indicator, _body, right = self.fixed_format.split(template)
 built = self.fixed_format.build_or_none(
 left,
 indicator or " ",
 body,
 right,
)
 return built if built is not None else body

 def comment(self, template: str, body: str) -> str:
 """A comment line: '*' in column 7, text from column 8."""
 left, _indicator, _body, right = self.fixed_format.split(template)
 text = str(body or "").rstrip("*").strip()
 built = self.fixed_format.build_or_none(left, "*", text, right)
 return built if built is not None else f"{FALLBACK_COMMENT_PREFIX}{text}"

 def blank(self, template: str) -> str:
 return self.statement(template, "")

 def body_indent(self, template: str) -> str:
 """Leading spaces of the template's body, as a string."""
 try:
 return self.fixed_format.body_indent(template)

```

```

 except Exception: # noqa: BLE001
 return ""

 def logical(self, line: str) -> str:
 try:
 return str(self.fixed_format.logical(line) or "").strip()
 except Exception: # noqa: BLE001
 return str(line or "").strip()

 def is_skippable(self, line: str) -> bool:
 """Comment, page-eject, debug or continuation line."""
 try:
 return bool(self.fixed_format.is_comment_or_control_line(line))
 except Exception: # noqa: BLE001
 return False

__all__ = ["CounterLineFactory"]

=====
FILE: src\idms_db2_phase2\composers\counter_declaration\counter_totals_writer.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/counter_declaration/counter_totals_writer.py
ACTION: CREATE NEW FILE
"""End-of-run counter totals.

CORRECTION - totals suppressed by an unrelated DISPLAY

The idempotency test was

 all(f"DISPLAY" in joined and name in joined for name in counters)

a placeholder-less f-string asking whether the word DISPLAY appeared
ANYWHERE in the program. One unrelated DISPLAY suppressed every total.

The test is now per counter, per line: a counter counts as already
displayed only when a single line carries DISPLAY, TOTAL and that
counter's name.
"""

from __future__ import annotations

from idms_db2_phase2.composers.counter_declaration.counter_line_factory import (
 CounterLineFactory,
)
from patterns.cobol_patterns import GOBACK_PATTERN, STOP_RUN_PATTERN
from rules.counter_declaration_rules import (
 COUNTER_DECLARATION_MESSAGES,
 COUNTER_DISPLAY_TEMPLATE,
 COUNTER_LABEL_TEMPLATE,
 COUNTER_LABEL_WIDTH,
 COUNTER_NAME_PREFIX,
 COUNTER_NAME_SUFFIX,
 COUNTER_OUTPUT_TOKEN,
 TOTALS_ANCHOR_STATEMENT,
 TOTALS_DISPLAY_VERB,
 TOTALS_INDENT,
 TOTALS_LABEL_TOKEN,
)

class CounterTotalsWriter:
 """Appends one DISPLAY per counter immediately before STOP RUN."""

 def __init__(
 self,
 line_factory: CounterLineFactory | None = None,
) -> None:
 self.lines = line_factory or CounterLineFactory()
 self.messages: list[str] = []

 # ----- public
 def append(self, lines: list[str], counters: list[str]) -> list[str]:

```

```

self.messages = []

if not counters:
 return lines

anchor = self._stop_run_index(lines)
if anchor < 0:
 self.messages.append(
 COUNTER_DECLARATION_MESSAGES["skipped_no_anchor"].format(
 anchor=TOTALS_ANCHOR_STATEMENT,
)
)
 return lines

if self._already_present(lines, counters):
 return lines

template = lines[anchor]
block = [
 self.lines.statement(template, self._display_body(name))
 for name in self._ordered(counters)
]
block.append(self.lines.blank(template))

self.messages.append(
 COUNTER_DECLARATION_MESSAGES["totals_added"].format(
 count=len(block) - 1,
)
)
return lines[:anchor] + block + lines[anchor:]

----- helpers
def _display_body(self, name: str) -> str:
 label = COUNTER_LABEL_TEMPLATE.format(
 token=self.token_of(name),
).ljust(COUNTER_LABEL_WIDTH)
 return TOTALS_INDENT + COUNTER_DISPLAY_TEMPLATE.format(
 label=label,
 name=name,
)

def _stop_run_index(self, lines: list[str]) -> int:
 """First program-termination statement: STOP RUN or GOBACK.

 CORRECTION - totals silently skipped.
 The anchor was STOP RUN only. A program CALLED with a parameter
 list ends with GOBACK, so every counter was declared and
 incremented but never displayed.
 """
 for index, line in enumerate(lines):
 if self.lines.is_skippable(line):
 continue

 logical = self.lines.logical(line)
 if STOP_RUN_PATTERN.match(logical):
 return index
 if GOBACK_PATTERN.match(logical):
 return index

 return -1

def _already_present(
 self,
 lines: list[str],
 counters: list[str],
) -> bool:
 displayed: set[str] = set()

 for line in lines:
 if self.lines.is_skippable(line):
 continue

 logical = self.lines.logical(line).upper()
 if TOTALS_DISPLAY_VERB not in logical:
 continue
 if TOTALS_LABEL_TOKEN not in logical:

```

```

 continue

 for name in counters:
 if name.upper() in logical:
 displayed.add(name.upper())

 return all(name.upper() in displayed for name in counters)

@staticmethod
def _ordered(counters: list[str]) -> list[str]:
 """Fetch counters first, the output counter last."""
 output_name = (
 f"{COUNTER_NAME_PREFIX}{COUNTER_OUTPUT_TOKEN}"
 f"{COUNTER_NAME_SUFFIX}"
).upper()
 output = [c for c in counters if c.upper() == output_name]
 others = [c for c in counters if c.upper() != output_name]
 return others + output

@staticmethod
def token_of(name: str) -> str:
 """WS-NB-OUTPUT-COUNT -> OUTPUT."""
 token = str(name or "").upper()
 if token.startswith(COUNTER_NAME_PREFIX):
 token = token[len(COUNTER_NAME_PREFIX):]
 if token.endswith(COUNTER_NAME_SUFFIX):
 token = token[: -len(COUNTER_NAME_SUFFIX)]
 return token or name

```

```
__all__ = ["CounterTotalsWriter"]
```

```

=====
FILE: src\idms_db2_phase2\composers\counter_declaration_composer.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/counter_declaration_composer.py
ACTION: REPLACE ENTIRE FILE

```

```
"""Counter declaration composer.
```

Declares every row counter the PROCEDURE DIVISION increments but  
WORKING-STORAGE does not define, and appends the end-of-run totals.

Why this exists

-----

OutputWriteParagraphComposer emits

```
 ADD 1 TO WS-NB-OUTPUT-COUNT
```

at the write call site, matching the manual reference. Nothing declared  
the field, so the generated program did not compile.

Reference-driven, not table-driven

-----

The composer collects the counters actually referenced, subtracts the  
names already declared anywhere in the DATA DIVISION, and declares only  
the remainder. It therefore cannot declare an unused field, and cannot  
miss one that a later generator starts incrementing.

Responsibility split

-----

This class decides WHAT to declare. CounterDeclarationWriter decides  
WHERE to put it, which is where the record-corruption defect lived.

Clean Architecture

-----

- Constants live in rules/counter\_declaration\_rules.py and  
rules/structural\_safety\_rules.py.
- Regex lives in patterns/counter\_declaration\_patterns.py.
- No program, record, table, cursor or host variable name is hardcoded.

```
"""
```

```
from __future__ import annotations
```



```

from idms_db2_phase2.composers.counter_declaration_writer import (
 CounterDeclarationWriter,
)
from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)
from patterns.counter_declaration_patterns import (
 COUNTER_INCREMENT_PATTERN,
 DATA_ENTRY_PATTERN,
)
from rules.counter_declaration_rules import (
 EMIT_COUNTER_TOTALS,
 ENFORCE_COUNTER_DECLARATION,
)

class CounterDeclarationComposer:
 """Declares referenced row counters and emits end-of-run totals."""

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
 writer: CounterDeclarationWriter | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()
 self.writer = writer or CounterDeclarationWriter(
 fixed_format=self.fixed_format,
)
 self.messages: list[str] = []

 # =====
 # Public entry point
 # =====
 def compose(self, text: str) -> str:
 self.messages = []

 if not text or not ENFORCE_COUNTER_DECLARATION:
 return str(text or "")

 lines = (
 str(text)
 .replace("\r\n", "\n")
 .replace("\r", "\n")
 .split("\n")
)

 referenced = self._referenced_counters(lines)
 if not referenced:
 return "\n".join(lines).rstrip() + "\n"

 declared = self._declared_names(lines)
 missing = [name for name in referenced if name not in declared]

 if missing:
 lines = self.writer.declare(lines=lines, missing=missing)
 self.messages.extend(self.writer.messages)

 if EMIT_COUNTER_TOTALS:
 lines = self.writer.append_totals(
 lines=lines,
 counters=referenced,
)
 self.messages.extend(self.writer.messages)

 return "\n".join(lines).rstrip() + "\n"

 # =====
 # Collection
 # =====
 def _referenced_counters(self, lines: list[str]) -> list[str]:
 """Counter names incremented anywhere in the program, in order."""
 out: list[str] = []
 seen: set[str] = set()

 for line in lines:

```

```

 if self.fixed_format.is_comment_or_control_line(line):
 continue

 match = COUNTER_INCREMENT_PATTERN.match(
 self.fixed_format.logical(line)
)
 if not match:
 continue

 name = match.group("name").upper()
 if name not in seen:
 seen.add(name)
 out.append(name)

 return out

def _declared_names(self, lines: list[str]) -> set[str]:
 """Every data name declared anywhere in the program."""
 out: set[str] = set()

 for line in lines:
 if self.fixed_format.is_comment_or_control_line(line):
 continue

 match = DATA_ENTRY_PATTERN.match(self.fixed_format.logical(line))
 if match:
 out.add(match.group("name").upper())

 return out

__all__ = ["CounterDeclarationComposer"]

=====
FILE: src\idms_db2_phase2\composers\counter_declaration_writer.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/counter_declaration_writer.py
ACTION: REPLACE ENTIRE FILE
"""Facade over the counter declaration package.

Keeps a single import path for CounterDeclarationComposer while the
placement, rendering and totals logic lives in focused modules:

 counter_declaration/counter_line_factory.py rendering
 counter_declaration/counter_anchor.py anchor model + lookup
 counter_declaration/counter_anchor_resolver.py safety-first placement
 counter_declaration/counter_totals_writer.py end-of-run totals

This class contains no COBOL knowledge of its own.
"""

from __future__ import annotations

from idms_db2_phase2.composers.counter_declaration import (
 CounterAnchorResolver,
 CounterLineFactory,
 CounterTotalsWriter,
)
from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)
from rules.counter_declaration_rules import (
 COUNTER_DECLARATION_MESSAGES,
 COUNTER_DECLARATION_TEMPLATE,
 COUNTER_PICTURE,
)

class CounterDeclarationWriter:
 """Writes counter declarations and end-of-run totals."""

 def __init__(
 self,

```

```

 fixed_format: FixedFormatLineService | None = None,
 resolver: CounterAnchorResolver | None = None,
 totals_writer: CounterTotalsWriter | None = None,
) -> None:
 self.line_factory = CounterLineFactory(
 fixed_format=fixed_format or FixedFormatLineService(),
)
 self.resolver = resolver or CounterAnchorResolver(
 line_factory=self.line_factory,
)
 self.totals_writer = totals_writer or CounterTotalsWriter(
 line_factory=self.line_factory,
)
 self.messages: list[str] = []

----- public
def declare(self, lines: list[str], missing: list[str]) -> list[str]:
 self.messages = []

 if not missing:
 return lines

 lines, anchor = self.resolver.resolve(lines)
 self.messages.extend(self.resolver.messages)

 if anchor is None or not anchor.is_usable:
 self.messages.append(
 COUNTER_DECLARATION_MESSAGES["skipped_no_group"].format(
 count=len(missing),
)
)
 return lines

 block: list[str] = []
 for name in missing:
 body = anchor.indent + COUNTER_DECLARATION_TEMPLATE.format(
 level=anchor.level,
 name=name,
 picture=COUNTER_PICTURE,
)
 block.append(
 self.line_factory.statement(anchor.template_line, body)
)
 self.messages.append(
 COUNTER_DECLARATION_MESSAGES["declared"].format(
 name=name,
 group=anchor.group,
)
)

 index = anchor.insert_index
 return lines[:index] + block + lines[index:]

def append_totals(
 self,
 lines: list[str],
 counters: list[str],
) -> list[str]:
 self.messages = []
 lines = self.totals_writer.append(lines, counters)
 self.messages.extend(self.totals_writer.messages)
 return lines

```

```
__all__ = ["CounterDeclarationWriter"]
```

```

=====
FILE: src\idms_db2_phase2\composers\cursor_close_guarantee__init__.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/cursor_close_guarantee/__init__.py
ACTION: CREATE NEW FILE

```

```
"""Helpers for the cursor close-guarantee pass.
```

CursorCloseGuaranteeComposer decides WHICH repair path a cursor needs.  
This package owns everything else.

models.py	value objects, no logic
message_log.py	one shared diagnostic collector
line_utils.py	logical line read / write, condition helpers
paragraph_index.py	paragraph discovery and navigation
cursor_scanner.py	generated cursor paragraphs and PERFORMs
loop_finder.py	business driving-loop discovery
condition_normaliser.py	legacy exit test -> cursor EOC flag
close_guarantor.py	inserts the missing PERFORM <close>
loop_synthesiser.py	builds the manual loop shape from scratch

Import order below is dependency order, so a circular import would fail  
loudly here rather than intermittently at call time.  
"""

```
from idms_db2_phase2.composers.cursor_close_guarantee.models import (
 UNTIL_KEYWORD,
 CursorParagraphSet,
 CursorPerform,
 LoopShape,
 ParagraphHeader,
)
from idms_db2_phase2.composers.cursor_close_guarantee.message_log import (
 MessageLog,
)
from idms_db2_phase2.composers.cursor_close_guarantee.line_utils import (
 COMMENT_INDICATORS,
 CursorGuaranteeLineUtils,
)
from idms_db2_phase2.composers.cursor_close_guarantee.paragraph_index import (
 ParagraphIndex,
)
from idms_db2_phase2.composers.cursor_close_guarantee.cursor_scanner import (
 PARAGRAPH_NAME_TEMPLATE,
 CursorScanner,
)
from idms_db2_phase2.composers.cursor_close_guarantee.loop_finder import (
 BusinessLoopFinder,
)
from idms_db2_phase2.composers.cursor_close_guarantee.condition_normaliser import (
 ConditionNormaliser,
)
from idms_db2_phase2.composers.cursor_close_guarantee.close_guarantor import (
 CloseGuarantor,
)
from idms_db2_phase2.composers.cursor_close_guarantee.loop_synthesiser import (
 LoopSynthesiser,
)

__all__ = [
 # constants
 "COMMENT_INDICATORS",
 "PARAGRAPH_NAME_TEMPLATE",
 "UNTIL_KEYWORD",
 # models
 "CursorParagraphSet",
 "CursorPerform",
 "LoopShape",
 "ParagraphHeader",
 # services
 "BusinessLoopFinder",
 "CloseGuarantor",
 "ConditionNormaliser",
 "CursorGuaranteeLineUtils",
 "CursorScanner",
 "LoopSynthesiser",
 "MessageLog",
 "ParagraphIndex",
]
```

```
=====
FILE: src\idms_db2_phase2\composers\cursor_close_guarantee\close_guarantor.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/cursor_close_guarantee/close_guarantor.py
ACTION: CREATE NEW FILE
```

```
"""Guarantees that a generated CLOSE paragraph is actually performed.
```

```
A declared-but-never-performed CLOSE paragraph leaks the cursor. DB2
holds the result set until the thread ends, and with WITH HOLD it
survives a COMMIT, so a long batch accumulates open cursors until it
abends on resource limits.
"""
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.composers.cursor_close_guarantee.line_utils import (
 CursorGuaranteeLineUtils,
)
from idms_db2_phase2.composers.cursor_close_guarantee.message_log import (
 MessageLog,
)
from idms_db2_phase2.composers.cursor_close_guarantee.models import (
 CursorParagraphSet,
 CursorPerform,
)
from rules.cursor_close_guarantee_rules import PERFORM_CLOSE_TEMPLATE
```

```
class CloseGuarantor:
```

```
 """Inserts the missing PERFORM of a cursor CLOSE paragraph."""
```

```
 def __init__(
 self,
 line_utils: CursorGuaranteeLineUtils | None = None,
 log: MessageLog | None = None,
) -> None:
 self.lines_utils = line_utils or CursorGuaranteeLineUtils()
 self.log = log or MessageLog()
```

```
 def guarantee(
 self,
 lines: list[str],
 after_index: int,
 cursor_set: CursorParagraphSet,
 close_performs: list[CursorPerform],
) -> list[str]:
 """Insert PERFORM <close>. after `after_index` when it is absent."""
 if close_performs:
 self.log.log("close_present", close=cursor_set.close_name)
 return lines
```

```
 if not 0 <= after_index < len(lines):
 return lines
```

```
 close_line = self.lines_utils.format_like(
 reference_line=lines[after_index],
 body=PERFORM_CLOSE_TEMPLATE.format(close=cursor_set.close_name),
)
```

```
 output = list(lines)
 output.insert(after_index + 1, close_line)
```

```
 self.log.log(
 "close_inserted",
 close=cursor_set.close_name,
 cursor=cursor_set.cursor,
)
```

```
 return output
```

```
__all__ = ["CloseGuarantor"]
```

```
=====
FILE: src\idms_db2_phase2\composers\cursor_close_guarantee\condition_normaliser.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/cursor_close_guarantee/condition_normaliser.py
ACTION: CREATE NEW FILE
```

```
"""Rewrites a legacy loop exit test to the cursor end-of-cursor flag.
```

```
 UNTIL SQLCODE = 100 -> UNTIL DZBFASC1-EOC
 UNTIL DB-END-OF-SET -> UNTIL DZBFASC1-EOC
```

```
A condition that is NOT in LEGACY_EOC_CONDITIONS is left exactly as it
is. Rewriting an unrecognised business condition would change working
program logic, which the business-flow preservation rule forbids.
"""
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.composers.cursor_close_guarantee.line_utils import (
 CursorGuaranteeLineUtils,
)
from idms_db2_phase2.composers.cursor_close_guarantee.message_log import (
 MessageLog,
)
from idms_db2_phase2.composers.cursor_close_guarantee.models import LoopShape
from rules.cursor_close_guarantee_rules import (
 STATEMENT_TERMINATOR,
 UNTIL_TEMPLATE,
)
```

```
class ConditionNormaliser:
```

```
 """Normalises loop exit conditions onto the cursor EOC flag."""
```

```
 def __init__(
 self,
 line_utils: CursorGuaranteeLineUtils | None = None,
 log: MessageLog | None = None,
) -> None:
 self.lines_utils = line_utils or CursorGuaranteeLineUtils()
 self.log = log or MessageLog()
```

```
 #
 # PERFORM <fetch> UNTIL <cond>.
 #
```

```
 def normalise_perform_line(
 self,
 lines: list[str],
 index: int,
 condition: str,
 cursor: str,
) -> list[str]:
 if not 0 <= index < len(lines):
 return lines
```

```
 logical = self.lines_utils.logical(lines[index])
 head, keyword, tail = self.lines_utils.partition_until(logical)
```

```
 if not keyword or not tail:
 return lines
```

```
 current = tail.rstrip(STATEMENT_TERMINATOR).strip()
```

```
 if not self._needs_rewrite(current, condition):
 return lines
```

```
 body = f"{head.rstrip()} {UNTIL_TEMPLATE.format(condition=condition)}"
```

```
 return self._write(
 lines=lines,
 index=index,
 body=body,
 terminated=self.lines_utils.is_terminated(logical),
)
```

```

 cursor=cursor,
 condition=condition,
)

#
A business driving loop
#
def normalise_loop(
 self,
 lines: list[str],
 loop: LoopShape,
 condition: str,
 cursor: str,
) -> list[str]:
 index = loop.until_index

 if not 0 <= index < len(lines):
 return lines

 if not self._needs_rewrite(loop.condition, condition):
 return lines

 logical = self.lines_utils.logical(lines[index])

 if self.lines_utils.is_until_only(logical):
 body = UNTIL_TEMPLATE.format(condition=condition)
 else:
 head, _keyword, _tail = self.lines_utils.partition_until(logical)
 body = (
 f"{head.rstrip()} "
 f"{UNTIL_TEMPLATE.format(condition=condition)}"
)

 return self._write(
 lines=lines,
 index=index,
 body=body,
 terminated=loop.terminated,
 cursor=cursor,
 condition=condition,
)

#
Helpers
#
def _needs_rewrite(self, current: str, condition: str) -> bool:
 """Only a recognised legacy test is ever rewritten."""
 if self.lines_utils.compress(current) == self.lines_utils.compress(
 condition
):
 return False

 return self.lines_utils.is_legacy_condition(current)

def _write(
 self,
 lines: list[str],
 index: int,
 body: str,
 terminated: bool,
 cursor: str,
 condition: str,
) -> list[str]:
 if terminated:
 body = f"{body}{STATEMENT_TERMINATOR}"

 output = list(lines)
 output[index] = self.lines_utils.format_like(
 reference_line=output[index],
 body=body,
)

 self.log.log("condition_rewritten", cursor=cursor, condition=condition)

 return output

```

```
__all__ = ["ConditionNormaliser"]
```

```
=====
FILE: src\idms_db2_phase2\composers\cursor_close_guarantee\cursor_scanner.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/cursor_close_guarantee/cursor_scanner.py
ACTION: CREATE NEW FILE
```

```
"""Read-only discovery of generated cursor paragraphs and their PERFORMs.
```

```
Knows how to recognise a generated cursor paragraph header and a PERFORM
of one. Knows nothing about loops, conditions or repairs.
"""
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.composers.cursor_close_guarantee.line_utils import (
 CursorGuaranteeLineUtils,
)
from idms_db2_phase2.composers.cursor_close_guarantee.models import (
 CursorParagraphSet,
 CursorPerform,
)
from patterns.cursor_close_guarantee_patterns import (
 CURSOR_PARAGRAPH_HEADER_PATTERN,
 PERFORM_CURSOR_PARAGRAPH_PATTERN,
)
```

```
PARAGRAPH_NAME_TEMPLATE = "{number:03d}-{operation}-{cursor}"
```

```
class CursorScanner:
```

```
 """Finds generated cursor paragraphs and every PERFORM of them."""
```

```
 def __init__(
 self,
 line_utils: CursorGuaranteeLineUtils | None = None,
) -> None:
 self.lines_utils = line_utils or CursorGuaranteeLineUtils()
```

```
 #
 # Paragraph headers
 #
```

```
 def paragraph_sets(self, lines: list[str]) -> dict:
 """Map every cursor to its generated OPEN / FETCH / CLOSE names."""
 found: dict = {}
```

```
 for line in lines:
 logical = self.lines_utils.logical(line)
```

```
 if not logical:
 continue
```

```
 match = CURSOR_PARAGRAPH_HEADER_PATTERN.match(logical)
```

```
 if not match:
 continue
```

```
 cursor = match.group("cursor").upper()
 operation = match.group("operation").upper()
 name = PARAGRAPH_NAME_TEMPLATE.format(
 number=int(match.group("number")),
 operation=operation,
 cursor=cursor,
)
```

```
 entry = found.setdefault(
 cursor,
 CursorParagraphSet(cursor=cursor, names={}),
)
 entry.remember(operation, name)
```



```

 return found

#
PERFORM statements
#
def performs(
 self,
 lines: list[str],
 cursor: str,
) -> list[CursorPerform]:
 """Every PERFORM of a generated paragraph belonging to one cursor."""
 target = str(cursor or "").upper()
 output: list[CursorPerform] = []

 if not target:
 return output

 for index, line in enumerate(lines):
 logical = self.lines_utils.logical(line)

 if not logical or self.lines_utils.is_comment(logical):
 continue

 match = PERFORM_CURSOR_PARAGRAPH_PATTERN.match(logical)

 if not match:
 continue

 if match.group("cursor").upper() != target:
 continue

 operation = match.group("operation").upper()

 output.append(
 CursorPerform(
 index=index,
 operation=operation,
 paragraph=PARAGRAPH_NAME_TEMPLATE.format(
 number=int(match.group("number")),
 operation=operation,
 cursor=target,
),
 tail=str(match.group("tail") or "").strip(),
)
)

 return output

@staticmethod
def by_operation(
 performs: list[CursorPerform],
 operation: str,
) -> list[CursorPerform]:
 return [p for p in performs if p.operation == operation]

```

```
__all__ = ["PARAGRAPH_NAME_TEMPLATE", "CursorScanner"]
```

```

=====
FILE: src\idms_db2_phase2\composers\cursor_close_guarantee\line_utils.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/cursor_close_guarantee/line_utils.py
ACTION: CREATE NEW FILE

```

```
"""Line inspection and formatting helpers.
```

```
Wraps CursorFlowLineFormatter so every helper in this package shares one
formatter instance and therefore one definition of "logical line" and one
definition of "format like this reference line".
```

```
Owns no regex definitions beyond what it imports, and no business rules.
"""
```

```

from __future__ import annotations

from idms_db2_phase2.composers.cursor_flow_line_formatter import (
 CursorFlowLineFormatter,
)
from patterns.cursor_close_guarantee_patterns import UNTIL_ONLY_PATTERN
from rules.cursor_close_guarantee_rules import (
 LEGACY_EOC_CONDITIONS,
 STATEMENT_TERMINATOR,
)

COMMENT_INDICATORS = ("*", "/")
UNTIL_KEYWORD = "UNTIL"

class CursorGuaranteeLineUtils:
 """Physical line helpers shared by every close-guarantee helper."""

 def __init__(
 self,
 line_formatter: CursorFlowLineFormatter | None = None,
) -> None:
 self.line_formatter = line_formatter or CursorFlowLineFormatter()

 #
 # Reading
 #
 def logical(self, line: str) -> str:
 """The COBOL body of a line, sequence areas stripped."""
 return self.line_formatter.logical(line)

 def is_blank_or_comment(self, line: str) -> bool:
 logical = self.logical(line)
 return not logical or self.is_comment(logical)

 @staticmethod
 def is_comment(logical: str) -> bool:
 text = str(logical or "").strip()
 return bool(text) and text[0] in COMMENT_INDICATORS

 @staticmethod
 def is_terminated(logical: str) -> bool:
 return str(logical or "").rstrip().endswith(STATEMENT_TERMINATOR)

 #
 # Writing
 #
 def format_like(self, reference_line: str, body: str) -> str:
 """Render `body` with the layout of `reference_line`."""
 return self.line_formatter.format_like_line(
 reference_line=reference_line,
 replacement_body=body,
)

 #
 # Condition helpers
 #
 @staticmethod
 def partition_until(logical: str) -> tuple[str, str, str]:
 """Split a logical line on UNTIL, preserving the original case.

 Returns (head, keyword, tail). `keyword` is empty when the line
 carries no UNTIL clause.
 """
 text = str(logical or "")
 position = text.upper().find(UNTIL_KEYWORD)

 if position < 0:
 return text, "", ""

 head = text[:position]
 tail = text[position + len(UNTIL_KEYWORD) :].strip()

 return head, UNTIL_KEYWORD, tail

 @staticmethod

```

```

def is_until_only(logical: str) -> bool:
 """True when the line carries nothing but the UNTIL clause."""
 return bool(UNTIL_ONLY_PATTERN.match(str(logical or "")))

def is_legacy_condition(self, condition: str) -> bool:
 """True when the exit test is still the raw IDMS / SQLCODE form."""
 current = self.compress(condition)

 if not current:
 return False

 return any(
 current == self.compress(candidate)
 for candidate in LEGACY_EOC_CONDITIONS
)

#
Text helpers
#
@staticmethod
def compress(text: str) -> str:
 """Whitespace-insensitive, case-insensitive comparison key."""
 return " ".join(str(text or "").split()).upper()

@staticmethod
def normalize_line_endings(text: str) -> str:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

__all__ = ["COMMENT_INDICATORS", "UNTIL_KEYWORD", "CursorGuaranteeLineUtils"]

=====
FILE: src\idms_db2_phase2\composers\cursor_close_guarantee\loop_finder.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/cursor_close_guarantee/loop_finder.py
ACTION: CREATE NEW FILE

"""Finds the business loop that drives a given paragraph.

Four idioms are recognised, in priority order:

 PERFORM <para> THRU <exit> UNTIL <cond>.
 PERFORM <para> UNTIL <cond>.
 PERFORM <para> THRU <exit> + UNTIL on a later line
 PERFORM <para> + UNTIL on a later line

A PERFORM of a GENERATED cursor paragraph is never a business driving
loop and is skipped, otherwise the pass would correlate the cursor with
its own fetch call.
"""

from __future__ import annotations

from idms_db2_phase2.composers.cursor_close_guarantee.line_utils import (
 CursorGuaranteeLineUtils,
)
from idms_db2_phase2.composers.cursor_close_guarantee.models import LoopShape
from patterns.cursor_close_guarantee_patterns import (
 PERFORM_CURSOR_PARAGRAPH_PATTERN,
 PERFORM_INLINE_WITH_UNTIL_PATTERN,
 PERFORM_PARAGRAPH_PATTERN,
 PERFORM_SPAN_PATTERN,
 PERFORM_SPAN_WITH_UNTIL_PATTERN,
 UNTIL_ONLY_PATTERN,
)
from rules.cursor_close_guarantee_rules import UNTIL_LOOKAHEAD_LIMIT

SAME_LINE_PATTERNS = (
 PERFORM_SPAN_WITH_UNTIL_PATTERN,
 PERFORM_INLINE_WITH_UNTIL_PATTERN,
)

LOOKAHEAD_PATTERNS = (

```

```

PERFORM_SPAN_PATTERN,
PERFORM_PARAGRAPH_PATTERN,
)

class BusinessLoopFinder:
 """Locates the driving loop of a business paragraph, if one exists."""

 def __init__(
 self,
 line_utils: CursorGuaranteeLineUtils | None = None,
) -> None:
 self.lines_utils = line_utils or CursorGuaranteeLineUtils()

 #
 # Public entry point
 #
 def find(
 self,
 lines: list[str],
 business: str,
) -> LoopShape | None:
 target = str(business or "").upper()

 if not target:
 return None

 for index, line in enumerate(lines):
 logical = self.lines_utils.logical(line)

 if not logical or self.lines_utils.is_comment(logical):
 continue

 # A generated cursor PERFORM is never a business driving loop.
 if PERFORM_CURSOR_PARAGRAPH_PATTERN.match(logical):
 continue

 shape = self._same_line_loop(logical, index, target)

 if shape is not None:
 return shape

 shape = self._lookahead_loop(lines, logical, index, target)

 if shape is not None:
 return shape

 return None

 #
 # Idiom handlers
 #
 def _same_line_loop(
 self,
 logical: str,
 index: int,
 target: str,
) -> LoopShape | None:
 """PERFORM ... UNTIL <cond> all on one line."""
 for pattern in SAME_LINE_PATTERNS:
 match = pattern.match(logical)

 if not match:
 continue

 if match.group("paragraph").upper() != target:
 continue

 return LoopShape(
 perform_index=index,
 until_index=index,
 end_index=index,
 condition=match.group("condition").strip(),
 terminated=self.lines_utils.is_terminated(logical),
)

```

```

 return None

 def _lookahead_loop(
 self,
 lines: list[str],
 logical: str,
 index: int,
 target: str,
) -> LoopShape | None:
 """PERFORM ... with the UNTIL clause on a continuation line."""
 for pattern in LOOKAHEAD_PATTERNS:
 match = pattern.match(logical)

 if not match:
 continue

 if match.group("paragraph").upper() != target:
 continue

 return self._until_on_following_line(lines, index)

 return None

 def _until_on_following_line(
 self,
 lines: list[str],
 perform_index: int,
) -> LoopShape | None:
 scanned = 0
 index = perform_index + 1

 while index < len(lines) and scanned < UNTIL_LOOKAHEAD_LIMIT:
 logical = self.lines_utils.logical(lines[index])

 if not logical or self.lines_utils.is_comment(logical):
 index += 1
 continue

 scanned += 1
 match = UNTIL_ONLY_PATTERN.match(logical)

 if match:
 return LoopShape(
 perform_index=perform_index,
 until_index=index,
 end_index=index,
 condition=match.group("condition").strip(),
 terminated=self.lines_utils.is_terminated(logical),
)

 index += 1

 return None

```

```
__all__ = ["BusinessLoopFinder"]
```

```

=====
FILE: src\idms_db2_phase2\composers\cursor_close_guarantee\loop_synthesiser.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/cursor_close_guarantee/loop_synthesiser.py
ACTION: CREATE NEW FILE

```

```
"""Builds the manual OPEN / FETCH-UNTIL / CLOSE shape from scratch.
```

Used only when no driving loop exists at all. Three edits, applied so that no earlier index is invalidated by a later one:

1. bind the FETCH paragraph WHEN ZERO branch to the business paragraph that used to carry the OBTAIN NEXT,
2. delete every orphan repeat PERFORM <fetch>, highest index first,
3. replace the priming PERFORM <fetch>. with the driving loop and, when needed, the CLOSE.

Step 2 runs before step 3 because deleting a line below the priming fetch cannot move the priming fetch, while inserting above it would move every repeat.

"""

```
from __future__ import annotations

from idms_db2_phase2.composers.cursor_close_guarantee.line_utils import (
 CursorGuaranteeLineUtils,
)
from idms_db2_phase2.composers.cursor_close_guarantee.message_log import (
 MessageLog,
)
from idms_db2_phase2.composers.cursor_close_guarantee.models import (
 CursorParagraphSet,
 CursorPerform,
)
from idms_db2_phase2.composers.cursor_close_guarantee.paragraph_index import (
 ParagraphIndex,
)
from patterns.cursor_close_guarantee_patterns import (
 CONTINUE_PATTERN,
 WHEN_ZERO_PATTERN,
)
from rules.cursor_close_guarantee_rules import (
 ENFORCE_FETCH_DRIVES_BUSINESS_PARAGRAPH,
 EXIT_PARAGRAPH_SUFFIX,
 PERFORM_BUSINESS_TEMPLATE,
 PERFORM_BUSINESS_THRU_TEMPLATE,
 PERFORM_CLOSE_TEMPLATE,
 PERFORM_FETCH_UNTIL_TEMPLATE,
 PREFER_THRU_EXIT_PARAGRAPH,
 STATEMENT_TERMINATOR,
)
```

class LoopSynthesiser:

"""Creates the manual driving-loop shape for an undriven cursor."""

```
def __init__(
 self,
 line_utils: CursorGuaranteeLineUtils | None = None,
 paragraph_index: ParagraphIndex | None = None,
 log: MessageLog | None = None,
) -> None:
 self.line_utils = line_utils or CursorGuaranteeLineUtils()
 self.paragraphs = paragraph_index or ParagraphIndex(self.line_utils)
 self.log = log or MessageLog()

#
Public entry point
#
def synthesise(
 self,
 lines: list[str],
 cursor_set: CursorParagraphSet,
 priming: CursorPerform,
 repeats: list[CursorPerform],
 business: str,
 close_performs: list[CursorPerform],
 eoc_condition: str,
) -> list[str]:
 output = list(lines)

 # 1. Bind the FETCH paragraph to the business paragraph.
 if business and ENFORCE_FETCH_DRIVES_BUSINESS_PARAGRAPH:
 output = self._bind_fetch_to_business(
 lines=output,
 fetch_paragraph=cursor_set.fetch_name,
 business=business,
)
 elif not business:
 self.log.log(
 "business_paragraph_unknown",
 cursor=cursor_set.cursor,
```

```

)

2. Remove every orphan repeat fetch, highest index first.
for repeat in sorted(repeats, key=lambda item: item.index, reverse=True):
 output = self._remove_statement(output, repeat.index)
 self.log.log("repeat_fetch_removed", fetch=repeat.paragraph)

3. Replace the priming fetch with the loop plus the CLOSE.
return self._write_driving_loop(
 lines=output,
 cursor_set=cursor_set,
 priming_index=priming.index,
 close_performs=close_performs,
 eoc_condition=eoc_condition,
)

#
Step 1
#
def _bind_fetch_to_business(
 self,
 lines: list[str],
 fetch_paragraph: str,
 business: str,
) -> list[str]:
 """Rewrite WHEN ZERO / CONTINUE inside the FETCH paragraph."""
 headers = self.paragraphs.headers(lines)
 span = self.paragraphs.span(headers, fetch_paragraph, len(lines))

 if span is None:
 return lines

 start, end = span
 output = list(lines)
 body = self._business_body(headers, business)

 index = start
 while index < end:
 if not WHEN_ZERO_PATTERN.match(self.lines_utils.logical(output[index])):
 index += 1
 continue

 target = self.paragraphs.next_executable_index(
 output, index + 1, end
)

 if target < 0:
 index += 1
 continue

 if not CONTINUE_PATTERN.match(self.lines_utils.logical(output[target])):
 index += 1
 continue

 output[target] = self.lines_utils.format_like(
 reference_line=output[target],
 body=body,
)

 self.log.log(
 "business_paragraph_bound",
 fetch=fetch_paragraph,
 paragraph=business,
)

 index = target + 1

 return output

def _business_body(self, headers: list, business: str) -> str:
 """PERFORM <para>, or PERFORM <para> THRU <para>-EXIT when it exists."""
 if PREFER_THRU_EXIT_PARAGRAPH:
 exit_name = f"{business}{EXIT_PARAGRAPH_SUFFIX}"

 if self.paragraphs.exists(headers, exit_name):
 return PERFORM_BUSINESS_THRU_TEMPLATE.format(

```

```

 paragraph=business,
 through=exit_name,
)

 return PERFORM_BUSINESS_TEMPLATE.format(paragraph=business)

#
Step 2
#
def _remove_statement(
 self,
 lines: list[str],
 index: int,
) -> list[str]:
 """Delete one statement without orphaning its COBOL sentence."""
 if not 0 <= index < len(lines):
 return lines

 carried_period = self.lines_utils.is_terminated(
 self.lines_utils.logical(lines[index])
)

 output = list(lines)
 del output[index]

 if not carried_period:
 return output

 previous = self.paragraphs.previous_executable_index(output, index - 1)

 if previous < 0:
 return output

 previous_logical = self.lines_utils.logical(output[previous])

 if self.lines_utils.is_terminated(previous_logical):
 return output

 output[previous] = self.lines_utils.format_like(
 reference_line=output[previous],
 body=f"{previous_logical.rstrip()}{STATEMENT_TERMINATOR}",
)

 return output

#
Step 3
#
def _write_driving_loop(
 self,
 lines: list[str],
 cursor_set: CursorParagraphSet,
 priming_index: int,
 close_performs: list[CursorPerform],
 eoc_condition: str,
) -> list[str]:
 if not 0 <= priming_index < len(lines):
 return lines

 output = list(lines)
 reference = output[priming_index]

 replacement = [
 self.lines_utils.format_like(
 reference_line=reference,
 body=PERFORM_FETCH_UNTIL_TEMPLATE.format(
 fetch=cursor_set.fetch_name,
 condition=eoc_condition,
),
),
]

 if close_performs:
 self.log.log("close_present", close=cursor_set.close_name)
 else:
 replacement.append(

```



```

 self.lines_utils.format_like(
 reference_line=reference,
 body=PERFORM_CLOSE_TEMPLATE.format(
 close=cursor_set.close_name,
),
)
)

output[priming_index : priming_index + 1] = replacement

self.log.log(
 "loop_synthesised",
 cursor=cursor_set.cursor,
 fetch=cursor_set.fetch_name,
 condition=eoc_condition,
 close=cursor_set.close_name,
)

return output

__all__ = ["LoopSynthesiser"]

=====
FILE: src\idms_db2_phase2\composers\cursor_close_guarantee\message_log.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/cursor_close_guarantee/message_log.py
ACTION: CREATE NEW FILE

"""Shared diagnostic collector for the cursor close-guarantee pass.

Every helper writes into ONE log instance so the composer can expose a
single `messages` list to conversion_service without each helper owning
its own list and drifting out of order.

Templates live in rules/cursor_close_guarantee_rules.py. This class never
formats a message it does not find there, so an unknown key is silently
ignored rather than raising in the middle of a conversion run.
"""

from __future__ import annotations

from rules.cursor_close_guarantee_rules import (
 CURSOR_CLOSE_GUARANTEE_MESSAGES,
)

class MessageLog:
 """Ordered diagnostic messages for one conversion run."""

 def __init__(self) -> None:
 self.messages: list[str] = []

 def reset(self) -> None:
 self.messages = []

 def log(self, key: str, **values) -> None:
 template = CURSOR_CLOSE_GUARANTEE_MESSAGES.get(key, "")

 if not template:
 return

 try:
 self.messages.append(template.format(**values))
 except (KeyError, IndexError):
 # A template/caller mismatch must never abort a conversion.
 self.messages.append(template)

__all__ = ["MessageLog"]

```

```
=====
FILE: src\idms_db2_phase2\composers\cursor_close_guarantee\models.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/cursor_close_guarantee/models.py
ACTION: CREATE NEW FILE
```

```
"""Value objects for the cursor close-guarantee pass.
```

```
Data only. No logic, no regex, no constants beyond the shape of the
records themselves.
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass, field
```

```
from rules.cursor_close_guarantee_rules import (
 OPERATION_CLOSE,
 OPERATION_FETCH,
 OPERATION_OPEN,
 REQUIRED_OPERATIONS,
)
```

```
UNTIL_KEYWORD = "UNTIL"
```

```
@dataclass
```

```
class ParagraphHeader:
```

```
 """One paragraph header line in the generated program."""
```

```
 index: int = -1
```

```
 name: str = ""
```

```
@dataclass
```

```
class CursorParagraphSet:
```

```
 """The generated OPEN / FETCH / CLOSE paragraph names of one cursor."""
```

```
 cursor: str = ""
```

```
 names: dict = field(default_factory=dict)
```

```
 @property
```

```
 def is_complete(self) -> bool:
```

```
 return all(
 operation in self.names for operation in REQUIRED_OPERATIONS
)
```

```
 @property
```

```
 def open_name(self) -> str:
```

```
 return self.names.get(OPERATION_OPEN, "")
```

```
 @property
```

```
 def fetch_name(self) -> str:
```

```
 return self.names.get(OPERATION_FETCH, "")
```

```
 @property
```

```
 def close_name(self) -> str:
```

```
 return self.names.get(OPERATION_CLOSE, "")
```

```
 def remember(self, operation: str, name: str) -> None:
```

```
 """First occurrence wins; a duplicate header is never honoured."""
 self.names.setdefault(operation, name)
```

```
@dataclass
```

```
class CursorPerform:
```

```
 """One PERFORM of a generated cursor paragraph."""
```

```
 index: int = -1
```

```
 operation: str = ""
```

```
 paragraph: str = ""
```

```
 tail: str = ""
```

```
 @property
```

```

def has_until(self) -> bool:
 return UNTIL_KEYWORD in str(self.tail or "").upper()

@dataclass
class LoopShape:
 """A business driving loop found in the generated flow."""

 perform_index: int = -1
 until_index: int = -1
 end_index: int = -1
 condition: str = ""
 terminated: bool = False

 @property
 def has_condition(self) -> bool:
 return bool(str(self.condition or "").strip())

__all__ = [
 "UNTIL_KEYWORD",
 "ParagraphHeader",
 "CursorParagraphSet",
 "CursorPerform",
 "LoopShape",
]

=====
FILE: src\idms_db2_phase2\composers\cursor_close_guarantee\paragraph_index.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/cursor_close_guarantee/paragraph_index.py
ACTION: CREATE NEW FILE

"""Paragraph discovery and navigation over generated COBOL lines.

Deliberately STATELESS: every method takes the current `lines` list. The
repair passes insert and delete lines, so a cached index would go stale
and silently address the wrong statement.
"""

from __future__ import annotations

from idms_db2_phase2.composers.cursor_close_guarantee.line_utils import (
 CursorGuaranteeLineUtils,
)
from idms_db2_phase2.composers.cursor_close_guarantee.models import (
 ParagraphHeader,
)
from patterns.cursor_close_guarantee_patterns import (
 PARAGRAPH_HEADER_PATTERN,
 SECTION_HEADER_PATTERN,
)
from rules.cursor_close_guarantee_rules import NON_PARAGRAPH_SINGLE_WORDS

class ParagraphIndex:
 """Finds paragraph headers and navigates between statements."""

 def __init__(
 self,
 line_utils: CursorGuaranteeLineUtils | None = None,
) -> None:
 self.lines_utils = line_utils or CursorGuaranteeLineUtils()

 #
 # Discovery
 #
 def headers(self, lines: list[str]) -> list[ParagraphHeader]:
 """Every real paragraph header, cursor paragraphs included.

 END-EXEC. and CONTINUE. look identical to a paragraph header at
 the token level, so NON_PARAGRAPH_SINGLE_WORDS filters them out.
 Treating one as a header would truncate the paragraph body at the

```

```

first EXEC SQL block.
"""
output: list[ParagraphHeader] = []

for index, line in enumerate(lines):
 logical = self.lines_utils.logical(line)

 if not logical or self.lines_utils.is_comment(logical):
 continue

 if SECTION_HEADER_PATTERN.match(logical):
 continue

 match = PARAGRAPH_HEADER_PATTERN.match(logical)

 if not match:
 continue

 name = match.group("name").upper()

 if name in NON_PARAGRAPH_SINGLE_WORDS:
 continue

 output.append(ParagraphHeader(index=index, name=name))

return output

#
Navigation
#
@staticmethod
def owner_paragraph(
 headers: list[ParagraphHeader],
 index: int,
) -> str:
 """Name of the paragraph that physically contains a line index."""
 owner = ""

 for header in headers:
 if header.index >= index:
 break
 owner = header.name

 return owner

@staticmethod
def span(
 headers: list[ParagraphHeader],
 name: str,
 total: int,
) -> tuple[int, int] | None:
 """(first body line, first line after the paragraph)."""
 target = str(name or "").upper()

 if not target:
 return None

 for position, header in enumerate(headers):
 if header.name != target:
 continue

 start = header.index + 1
 end = (
 headers[position + 1].index
 if position + 1 < len(headers)
 else total
)
 return start, min(end, total)

 return None

def exists(self, headers: list[ParagraphHeader], name: str) -> bool:
 target = str(name or "").upper()
 return any(header.name == target for header in headers)

#

```

```

Statement stepping
#
def next_executable_index(
 self,
 lines: list[str],
 start: int,
 end: int,
) -> int:
 for index in range(max(0, start), min(end, len(lines))):
 if self.lines_utils.is_blank_or_comment(lines[index]):
 continue
 return index

 return -1

def previous_executable_index(
 self,
 lines: list[str],
 start: int,
) -> int:
 """Previous statement, refusing to cross a paragraph boundary."""
 if not lines:
 return -1

 for index in range(min(start, len(lines) - 1), -1, -1):
 logical = self.lines_utils.logical(lines[index])

 if not logical or self.lines_utils.is_comment(logical):
 continue

 if PARAGRAPH_HEADER_PATTERN.match(logical):
 return -1

 return index

 return -1

```

```
__all__ = ["ParagraphIndex"]
```

```

=====
FILE: src\idms_db2_phase2\composers\cursor_close_guarantee_composer.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/cursor_close_guarantee_composer.py
ACTION: REPLACE ENTIRE FILE

```

```
"""Guarantees that every generated cursor is looped and closed.
```

Runs AFTER CursorFlowComposer. It is a no-op for any cursor that the INLINE or SPAN strategy already handled correctly, and a last-resort repair for every cursor they could not correlate.

WHY THIS EXISTS

-----  
CursorFlowComposer recognises two driving-loop idioms. An LRF-expanded program matches neither, so it logged `no\_loop\_for\_cursor` as a WARNING and shipped:

```

PERFORM 710-OPEN-DZBFASC1.
PERFORM 720-FETCH-DZBFASC1. <- primes once, never loops
...
BEHANDLING.
...
PERFORM 720-FETCH-DZBFASC1. <- orphan repeat fetch

```

with 730-CLOSE-DZBFASC1 declared but never performed. The cursor is opened, read twice, and leaked. The manual reference is:

```

PERFORM 710-OPEN-DZBFASC1.
PERFORM 720-FETCH-DZBFASC1 UNTIL DZBFASC1-EOC.
PERFORM 730-CLOSE-DZBFASC1.

```

with the FETCH paragraph driving the business paragraph:

```
EVALUATE SQLCODE
WHEN ZERO
 PERFORM BEHADELING THRU BEHADELING-EXIT
```

### THREE REPAIR PATHS

- A. A fetch PERFORM already carries UNTIL -> INLINE already ran.  
Only normalise a legacy exit condition and guarantee the CLOSE.
- B. A business SPAN loop drives the paragraph that held the OBTAIN NEXT  
-> SPAN shape. Normalise its exit condition and guarantee the CLOSE.  
The in-loop fetch is REQUIRED here and is never removed.
- C. No loop at all -> synthesise the manual shape, bind the FETCH  
paragraph to the business paragraph, and remove the orphan repeat  
fetch.

### ORCHESTRATION ONLY

```
Every decision helper lives in composers/cursor_close_guarantee/.
Regex lives in patterns/cursor_close_guarantee_patterns.py.
Constants live in rules/cursor_close_guarantee_rules.py.
No program, paragraph, record, table, cursor or host variable name is
hardcoded here.
"""
```

```
from __future__ import annotations

from idms_db2_phase2.composers.cursor_close_guarantee import (
 BusinessLoopFinder,
 CloseGuarantor,
 ConditionNormaliser,
 CursorGuaranteeLineUtils,
 CursorParagraphSet,
 CursorPerform,
 CursorScanner,
 LoopSynthesiser,
 MessageLog,
 ParagraphIndex,
)
from idms_db2_phase2.composers.cursor_flow_line_formatter import (
 CursorFlowLineFormatter,
)
from rules.cursor_close_guarantee_rules import (
 ENFORCE_CURSOR_CLOSE_GUARANTEE,
 EOC_CONDITION_TEMPLATE,
 OPERATION_CLOSE,
 OPERATION_FETCH,
 OPERATION_OPEN,
 SYNTHESISE_MISSING_DRIVING_LOOP,
)

class CursorCloseGuaranteeComposer:
 """Last-resort guarantee that a generated cursor loops and closes."""

 def __init__(
 self,
 line_formatter: CursorFlowLineFormatter | None = None,
) -> None:
 self.line_utils = CursorGuaranteeLineUtils(line_formatter)
 self.log = MessageLog()

 self.paragraphs = ParagraphIndex(self.line_utils)
 self.scanner = CursorScanner(self.line_utils)
 self.loop_finder = BusinessLoopFinder(self.line_utils)

 self.conditions = ConditionNormaliser(
 line_utils=self.line_utils,
 log=self.log,
)
 self.close_guarantor = CloseGuarantor(
 line_utils=self.line_utils,
 log=self.log,
)
```

```

 self.synthesiser = LoopSynthesiser(
 line_utils=self.line_utils,
 paragraph_index=self.paragraphs,
 log=self.log,
)

#
conversion_service reads this after compose()
#
@property
def messages(self) -> list[str]:
 return self.log.messages

#
Public entry point
#
def compose(self, text: str) -> str:
 self.log.reset()

 if not text:
 return ""

 if not ENFORCE_CURSOR_CLOSE_GUARANTEE:
 return str(text)

 lines = self.line_utils.normalize_line_endings(text).splitlines()

 if not lines:
 return ""

 cursor_sets = self.scanner.paragraph_sets(lines)

 for cursor in sorted(cursor_sets):
 cursor_set = cursor_sets[cursor]

 if not cursor_set.is_complete:
 self.log.log("incomplete_paragraph_set", cursor=cursor)
 continue

 lines = self._repair_cursor(lines, cursor_set)

 return "\n".join(lines).rstrip() + "\n"

#
Path selection
#
def _repair_cursor(
 self,
 lines: list[str],
 cursor_set: CursorParagraphSet,
) -> list[str]:
 cursor = cursor_set.cursor
 eoc_condition = EOC_CONDITION_TEMPLATE.format(cursor=cursor)

 performs = self.scanner.performs(lines, cursor)

 opens = self.scanner.by_operation(performs, OPERATION_OPEN)
 fetches = self.scanner.by_operation(performs, OPERATION_FETCH)
 closes = self.scanner.by_operation(performs, OPERATION_CLOSE)

 if not opens or not fetches:
 self.log.log("no_open_perform", cursor=cursor)
 return lines

 # ---- Path A: an INLINE loop already exists.
 driving = next((p for p in fetches if p.has_until), None)

 if driving is not None:
 return self._repair_inline(
 lines=lines,
 cursor_set=cursor_set,
 driving=driving,
 closes=closes,
 eoc_condition=eoc_condition,
)

```

```

priming = fetches[0]
repeats = fetches[1:]
business = self._business_paragraph(lines, repeats)

---- Path B: a business loop already drives that paragraph.
loop = self.loop_finder.find(lines, business) if business else None

if loop is not None:
 return self._repair_span(
 lines=lines,
 cursor_set=cursor_set,
 loop=loop,
 closes=closes,
 eoc_condition=eoc_condition,
)

---- Path C: nothing drives the cursor.
if not SYNTHESISE_MISSING_DRIVING_LOOP:
 return lines

return self.synthesiser.synthesise(
 lines=lines,
 cursor_set=cursor_set,
 priming=priming,
 repeats=repeats,
 business=business,
 close_performs=closes,
 eoc_condition=eoc_condition,
)

#
Path A
#
def _repair_inline(
 self,
 lines: list[str],
 cursor_set: CursorParagraphSet,
 driving: CursorPerform,
 closes: list[CursorPerform],
 eoc_condition: str,
) -> list[str]:
 lines = self.conditions.normalise_perform_line(
 lines=lines,
 index=driving.index,
 condition=eoc_condition,
 cursor=cursor_set.cursor,
)
 return self.close_guarantor.guarantee(
 lines=lines,
 after_index=driving.index,
 cursor_set=cursor_set,
 close_performs=closes,
)

#
Path B
#
def _repair_span(
 self,
 lines: list[str],
 cursor_set: CursorParagraphSet,
 loop,
 closes: list[CursorPerform],
 eoc_condition: str,
) -> list[str]:
 lines = self.conditions.normalise_loop(
 lines=lines,
 loop=loop,
 condition=eoc_condition,
 cursor=cursor_set.cursor,
)
 return self.close_guarantor.guarantee(
 lines=lines,
 after_index=loop.end_index,
 cursor_set=cursor_set,
 close_performs=closes,

```



```

)

#
Helpers
#
def _business_paragraph(
 self,
 lines: list[str],
 repeats: list[CursorPerform],
) -> str:
 """The paragraph that used to carry the converted OBTAIN NEXT."""
 if not repeats:
 return ""

 headers = self.paragraphs.headers(lines)

 return self.paragraphs.owner_paragraph(headers, repeats[0].index)

__all__ = ["CursorCloseGuaranteeComposer"]

=====
FILE: src\idms_db2_phase2\composers\cursor_flow__init__.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/cursor_flow/__init__.py
ACTION: CREATE NEW FILE
"""Cursor driving-loop analysis and span-loop rewriting."""

=====
FILE: src\idms_db2_phase2\composers\cursor_flow\loop_shape.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/cursor_flow/loop_shape.py
ACTION: CREATE NEW FILE
"""Classifies every driving loop in a program, whatever the idiom.

Read-only. Produces LoopShape values; rewrites nothing.
"""

from __future__ import annotations

from dataclasses import dataclass

from idms_db2_phase2.composers.cursor_flow_line_formatter import (
 CursorFlowLineFormatter,
)
from patterns.cursor_loop_patterns import (
 PERFORM_CURSOR_PARAGRAPH_PATTERN,
 PERFORM_INLINE_WITH_UNTIL_PATTERN,
 PERFORM_SIMPLE_PATTERN,
 PERFORM_SPAN_PATTERN,
 PERFORM_SPAN_WITH_UNTIL_PATTERN,
 UNTIL_LINE_PATTERN,
)
from rules.cursor_loop_rules import (
 LEGACY_EOC_CONDITIONS,
 LOOP_KIND_INLINE,
 LOOP_KIND_SPAN,
)

UNTIL_CONTINUATION_WINDOW = 4
COMMENT_PREFIXES = ("*", "/")

@dataclass
class LoopShape:
 """One driving loop, normalised across idioms."""

 kind: str = ""
 paragraph: str = ""
 through: str = ""

```

```

condition: str = ""
perform_index: int = -1
until_index: int = -1
end_index: int = -1
terminated: bool = False

@property
def is_span(self) -> bool:
 return self.kind == LOOP_KIND_SPAN

@property
def has_condition(self) -> bool:
 return bool(self.condition)

@property
def condition_is_legacy(self) -> bool:
 probe = self.condition.upper().replace(" ", "")
 return any(
 legacy.replace(" ", "") in probe
 for legacy in LEGACY_EOC_CONDITIONS
)

class LoopShapeScanner:
 """Finds every driving loop, in both idioms, anywhere in the program."""

 def __init__(
 self,
 line_formatter: CursorFlowLineFormatter | None = None,
) -> None:
 self.line_formatter = line_formatter or CursorFlowLineFormatter()

 # ----- public
 def scan(self, lines: list[str]) -> list[LoopShape]:
 shapes: list[LoopShape] = []
 index = 0

 while index < len(lines):
 logical = self._logical(lines[index])

 if not logical or self._is_comment(logical):
 index += 1
 continue

 # A generated cursor PERFORM is never a business driving loop.
 if PERFORM_CURSOR_PARAGRAPH_PATTERN.match(logical):
 index += 1
 continue

 shape = self._shape_at(lines, index, logical)
 if shape is None:
 index += 1
 continue

 shapes.append(shape)
 index = shape.end_index + 1

 return shapes

 @staticmethod
 def inline_loops(shapes: list[LoopShape]) -> list[LoopShape]:
 return [s for s in shapes if s.kind == LOOP_KIND_INLINE]

 @staticmethod
 def span_loops(shapes: list[LoopShape]) -> list[LoopShape]:
 return [s for s in shapes if s.kind == LOOP_KIND_SPAN]

 # ----- shapes
 def _shape_at(
 self,
 lines: list[str],
 index: int,
 logical: str,
) -> LoopShape | None:
 # 1. PERFORM a THRU b UNTIL cond.
 match = PERFORM_SPAN_WITH_UNTIL_PATTERN.match(logical)

```

```

if match:
 return LoopShape(
 kind=LOOP_KIND_SPAN,
 paragraph=match.group("paragraph").upper(),
 through=match.group("through").upper(),
 condition=match.group("condition").strip(),
 perform_index=index,
 until_index=index,
 end_index=index,
 terminated=logical.rstrip().endswith("."),
)

2. PERFORM a THRU b / UNTIL cond.
match = PERFORM_SPAN_PATTERN.match(logical)
if match:
 until_index, condition, terminated = self._until_after(
 lines, index + 1
)
 return LoopShape(
 kind=LOOP_KIND_SPAN,
 paragraph=match.group("paragraph").upper(),
 through=match.group("through").upper(),
 condition=condition,
 perform_index=index,
 until_index=until_index if until_index >= 0 else index,
 end_index=until_index if until_index >= 0 else index,
 terminated=terminated,
)

3. PERFORM a UNTIL cond.
match = PERFORM_INLINE_WITH_UNTIL_PATTERN.match(logical)
if match:
 return LoopShape(
 kind=LOOP_KIND_INLINE,
 paragraph=match.group("paragraph").upper(),
 condition=match.group("condition").strip(),
 perform_index=index,
 until_index=index,
 end_index=index,
 terminated=logical.rstrip().endswith("."),
)

4. PERFORM a / UNTIL cond.
match = PERFORM_SIMPLE_PATTERN.match(logical)
if match:
 until_index, condition, terminated = self._until_after(
 lines, index + 1
)
 if until_index < 0:
 return None
 return LoopShape(
 kind=LOOP_KIND_INLINE,
 paragraph=match.group("paragraph").upper(),
 condition=condition,
 perform_index=index,
 until_index=until_index,
 end_index=until_index,
 terminated=terminated,
)

return None

def _until_after(
 self,
 lines: list[str],
 start_index: int,
) -> tuple[int, str, bool]:
 end = min(len(lines), start_index + UNTIL_CONTINUATION_WINDOW)

 for index in range(start_index, end):
 logical = self._logical(lines[index])
 if not logical or self._is_comment(logical):
 continue

 match = UNTIL_LINE_PATTERN.match(logical)
 if match:

```

```

 return (
 index,
 match.group("condition").strip(),
 bool(match.group("terminator")),
)
 break

 return -1, "", False

----- helpers
def _logical(self, line: str) -> str:
 return self.line_formatter.logical(line)

@staticmethod
def _is_comment(logical: str) -> bool:
 return logical.startswith(COMMENT_PREFIXES)

__all__ = ["LoopShape", "LoopShapeScanner"]

=====
FILE: src\idms_db2_phase2\composers\cursor_flow\span_loop_plan.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/cursor_flow/span_loop_plan.py
ACTION: CREATE NEW FILE
"""Correlates a generated cursor with a SPAN driving loop.

Search is whole-program and bidirectional: a span loop legitimately
precedes the paragraph that opens the cursor.
"""

from __future__ import annotations

from dataclasses import dataclass

from idms_db2_phase2.composers.cursor_flow.loop_shape import (
 LoopShape,
 LoopShapeScanner,
)
from idms_db2_phase2.composers.cursor_flow_line_formatter import (
 CursorFlowLineFormatter,
)
from patterns.cursor_loop_patterns import PERFORM_CURSOR_PARAGRAPH_PATTERN
from rules.cursor_loop_rules import (
 CURSOR_LOOP_MESSAGES,
 EOC_CONDITION_TEMPLATE,
 MAX_CORRELATION_DISTANCE,
)

FETCH_OFFSET = 10
CLOSE_OFFSET = 20

@dataclass
class SpanLoopPlan:
 """A cursor whose driving loop is a paragraph span."""

 cursor_name: str = ""
 open_index: int = -1
 open_number: int = 0
 fetch_paragraph: str = ""
 close_paragraph: str = ""
 eoc_condition: str = ""
 loop: LoopShape | None = None

 @property
 def is_usable(self) -> bool:
 return bool(
 self.cursor_name
 and self.close_paragraph
 and self.loop is not None
 and self.loop.has_condition
)

```

```

class SpanLoopPlanBuilder:
 """Builds SpanLoopPlan values from scanned loop shapes."""

 def __init__(
 self,
 line_formatter: CursorFlowLineFormatter | None = None,
 scanner: LoopShapeScanner | None = None,
) -> None:
 self.line_formatter = line_formatter or CursorFlowLineFormatter()
 self.scanner = scanner or LoopShapeScanner(self.line_formatter)
 self.messages: list[str] = []

 # ----- public
 def build(
 self,
 lines: list[str],
 shapes: list[LoopShape],
 claimed_cursors: set[str] | None = None,
) -> list[SpanLoopPlan]:
 """Plans for cursors NOT already handled by the inline strategy."""
 self.messages = []

 span_loops = self.scanner.span_loops(shapes)
 if not span_loops:
 return []

 already = {str(name).upper() for name in (claimed_cursors or set())}
 plans: list[SpanLoopPlan] = []
 used_loops: set[int] = set()

 for cursor, open_index, open_number in self._cursor_opens(lines):
 if cursor in already:
 continue

 loop = self._nearest_loop(span_loops, open_index, used_loops)
 if loop is None:
 self._log("no_loop_for_cursor", cursor=cursor)
 continue

 used_loops.add(loop.perform_index)

 plan = SpanLoopPlan(
 cursor_name=cursor,
 open_index=open_index,
 open_number=open_number,
 fetch_paragraph=(
 f"{open_number + FETCH_OFFSET:03d}-FETCH-{cursor}"
),
 close_paragraph=(
 f"{open_number + CLOSE_OFFSET:03d}-CLOSE-{cursor}"
),
 eoc_condition=EOC_CONDITION_TEMPLATE.format(cursor=cursor),
 loop=loop,
)

 if not plan.is_usable:
 continue

 plans.append(plan)
 self._log(
 "span_plan",
 cursor=cursor,
 paragraph=loop.paragraph,
 through=loop.through,
)

 return plans

 # ----- helpers
 def _cursor_opens(self, lines: list[str]):
 """(cursor, index, number) for every PERFORM nnn-OPEN-<cursor>."""
 seen: set[str] = set()

 for index, line in enumerate(lines):

```

```

 logical = self.line_formatter.logical(line)
 match = PERFORM_CURSOR_PARAGRAPH_PATTERN.match(logical)

 if not match:
 continue
 if match.group("operation").upper() != "OPEN":
 continue

 cursor = match.group("cursor").upper()
 if cursor in seen:
 continue

 seen.add(cursor)
 yield cursor, index, int(match.group("number"))

 @staticmethod
 def _nearest_loop(
 span_loops: list[LoopShape],
 open_index: int,
 used_loops: set[int],
) -> LoopShape | None:
 """Closest unused span loop, searched in BOTH directions.

 A loop still carrying the legacy end test is always preferred: it
 is the one that has not been converted yet.
 """
 candidates = [
 loop for loop in span_loops
 if loop.perform_index not in used_loops and loop.has_condition
]
 if not candidates:
 return None

 legacy = [loop for loop in candidates if loop.condition_is_legacy]
 pool = legacy or candidates

 if MAX_CORRELATION_DISTANCE > 0:
 pool = [
 loop for loop in pool
 if abs(loop.perform_index - open_index)
 <= MAX_CORRELATION_DISTANCE
]
 if not pool:
 return None

 return min(pool, key=lambda loop: abs(loop.perform_index - open_index))

 def _log(self, key: str, **values) -> None:
 template = CURSOR_LOOP_MESSAGES.get(key, "")
 if template:
 self.messages.append(template.format(**values))

```

```

__all__ = ["SpanLoopPlan", "SpanLoopPlanBuilder"]

```

```

=====
FILE: src\idms_db2_phase2\composers\cursor_flow\span_loop_rewriter.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/cursor_flow/span_loop_rewriter.py
ACTION: CREATE NEW FILE
"""Conservative rewrite for a SPAN driving loop.

```

Does exactly two things:

1. replaces the legacy exit condition with the cursor end-of-cursor flag,
2. guarantees the cursor CLOSE paragraph is performed after the loop.

It NEVER restructures the loop, never moves the business paragraph and never deletes the in-loop FETCH. Restructuring a working span loop would violate the business-flow preservation rule.

```

"""

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.composers.cursor_flow.span_loop_plan import SpanLoopPlan
from idms_db2_phase2.composers.cursor_flow_line_formatter import (
 CursorFlowLineFormatter,
)
from patterns.cursor_loop_patterns import (
 PERFORM_CURSOR_PARAGRAPH_PATTERN,
 UNTIL_LINE_PATTERN,
)
from rules.cursor_loop_rules import (
 CLOSE_PRESENCE_WINDOW,
 CURSOR_LOOP_MESSAGES,
 PERFORM_CLOSE_TEMPLATE,
 STATEMENT_TERMINATOR,
 UNTIL_TEMPLATE,
 UNTIL_TERMINATED_TEMPLATE,
)

```

```

class SpanLoopRewriter:
 def __init__(
 self,
 line_formatter: CursorFlowLineFormatter | None = None,
) -> None:
 self.line_formatter = line_formatter or CursorFlowLineFormatter()
 self.messages: list[str] = []

 # ----- public
 def rewrite(
 self,
 lines: list[str],
 plans: list[SpanLoopPlan],
) -> list[str]:
 self.messages = []

 if not plans:
 return lines

 output = list(lines)

 # Descending order so earlier indexes stay valid after insertion.
 for plan in sorted(
 plans,
 key=lambda item: item.loop.end_index,
 reverse=True,
):
 output = self._rewrite_condition(output, plan)
 output = self._ensure_close(output, plan)

 return output

 # ----- condition
 def _rewrite_condition(
 self,
 lines: list[str],
 plan: SpanLoopPlan,
) -> list[str]:
 loop = plan.loop
 if loop is None or not loop.condition_is_legacy:
 return lines

 index = loop.until_index
 if not 0 <= index < len(lines):
 return lines

 logical = self.line_formatter.logical(lines[index])

 if UNTIL_LINE_PATTERN.match(logical):
 template = (
 UNTIL_TERMINATED_TEMPLATE if loop.terminated
 else UNTIL_TEMPLATE
)
 body = template.format(condition=plan.eoc_condition)
 else:
 # UNTIL is part of the PERFORM line - replace the tail only.
 head, _separator, _tail = logical.upper().partition("UNTIL")

```

```

 body = (
 f"{logical[:len(head)].rstrip()} "
 f"{UNTIL_TEMPLATE.format(condition=plan.eoc_condition)}"
)
 if loop.terminated:
 body = f"{body}{STATEMENT_TERMINATOR}"

 lines[index] = self.line_formatter.format_like_line(
 reference_line=lines[index],
 replacement_body=body,
)
 self._log("condition_rewritten", condition=plan.eoc_condition)
 return lines

----- close
def _ensure_close(
 self,
 lines: list[str],
 plan: SpanLoopPlan,
) -> list[str]:
 if self._close_already_performed(lines, plan):
 self._log("close_present", paragraph=plan.close_paragraph)
 return lines

 insert_at = plan.loop.end_index + 1
 if insert_at > len(lines):
 insert_at = len(lines)

 reference = lines[plan.loop.end_index]
 close_line = self.line_formatter.format_like_line(
 reference_line=reference,
 replacement_body=PERFORM_CLOSE_TEMPLATE.format(
 paragraph=plan.close_paragraph,
),
),
)

 lines.insert(insert_at, close_line)
 self._log("close_inserted", paragraph=plan.close_paragraph)
 return lines

def _close_already_performed(
 self,
 lines: list[str],
 plan: SpanLoopPlan,
) -> bool:
 wanted = plan.close_paragraph.upper()

 start = max(0, plan.loop.end_index - CLOSE_PRESENCE_WINDOW)
 end = min(len(lines), plan.loop.end_index + CLOSE_PRESENCE_WINDOW)

 for index in range(start, end):
 logical = self.line_formatter.logical(lines[index])
 match = PERFORM_CURSOR_PARAGRAPH_PATTERN.match(logical)
 if not match:
 continue
 if match.group("operation").upper() != "CLOSE":
 continue
 name = (
 f"{int(match.group('number')):03d}-CLOSE-"
 f"{match.group('cursor').upper()}"
)
 if name == wanted:
 return True

 return False

----- helpers
def _log(self, key: str, **values) -> None:
 template = CURSOR_LOOP_MESSAGES.get(key, "")
 if template:
 self.messages.append(template.format(**values))

```

```
__all__ = ["SpanLoopRewriter"]
```



```
=====
FILE: src\idms_db2_phase2\composers\cursor_flow_composer.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/cursor_flow_composer.py
ACTION: REPLACE ENTIRE FILE
"""Cursor flow composer.
```

Two strategies, selected by the shape of the driving loop.

INLINE (unchanged, proven)

```

PERFORM 710-OPEN-CURSOR. PERFORM 710-OPEN-CURSOR.
PERFORM 720-FETCH-CURSOR. -> PERFORM 720-FETCH-CURSOR
PERFORM BUSINESS-PARA UNTIL CURSOR-EOC.
 UNTIL SQLCODE = 100. PERFORM 730-CLOSE-CURSOR.
```

SPAN (older and LRF programs)

```

PERFORM BEHADELING THRU BEHADELING-EXIT
 UNTIL SQLCODE = 100. -> UNTIL DZBFASC1-EOC.
 PERFORM 730-CLOSE-DZBFASC1.
```

CORRECTION 1 - ungated OBTAIN NEXT cleanup

remove\_leftover\_obtain\_next\_cursor\_calls() previously ran even when NO plan had been produced, deleting the only FETCH inside the loop.

CORRECTION 2 - SPAN must not authorise the cleanup

The first gate accepted ANY plan, including a SPAN plan. A SPAN rewrite keeps the loop body untouched and never emits PERFORM <fetch> UNTIL <eoc>, so its in-loop fetch is REQUIRED. Gating on a SPAN plan therefore reproduced the original defect: the loop ran forever because SQLCODE never changed. Only an INLINE rewrite authorises the cleanup.

Orchestration only. Regex lives in patterns/, constants in rules/.

```
"""
from __future__ import annotations

from idms_db2_phase2.composers.cursor_flow.loop_shape import LoopShapeScanner
from idms_db2_phase2.composers.cursor_flow.span_loop_plan import (
 SpanLoopPlanBuilder,
)
from idms_db2_phase2.composers.cursor_flow.span_loop_rewriter import (
 SpanLoopRewriter,
)
from idms_db2_phase2.composers.cursor_flow_line_formatter import (
 CursorFlowLineFormatter,
)
from idms_db2_phase2.composers.cursor_flow_obtain_next_cleanup import (
 CursorFlowObtainNextCleanup,
)
from idms_db2_phase2.composers.cursor_flow_plan_discoverer import (
 CursorFlowPlanDiscoverer,
)
from idms_db2_phase2.composers.cursor_flow_rewriter import CursorFlowRewriter
from rules.cursor_loop_rules import (
 CURSOR_LOOP_MESSAGES,
 ENFORCE_SPAN_LOOP_REWRITE,
 REQUIRE_PLAN_FOR_OBTAIN_NEXT_CLEANUP,
)
```

```
class CursorFlowComposer:
 def __init__(self) -> None:
 self.line_formatter = CursorFlowLineFormatter()
 self.plan_discoverer = CursorFlowPlanDiscoverer(
 line_formatter=self.line_formatter,
)
 self.rewriter = CursorFlowRewriter(
 line_formatter=self.line_formatter,
)
 self.obtain_next_cleanup = CursorFlowObtainNextCleanup()
```

```

 line_formatter=self.line_formatter,
)
 self.loop_scanner = LoopShapeScanner(
 line_formatter=self.line_formatter,
)
 self.span_plan_builder = SpanLoopPlanBuilder(
 line_formatter=self.line_formatter,
 scanner=self.loop_scanner,
)
 self.span_rewriter = SpanLoopRewriter(
 line_formatter=self.line_formatter,
)
 self.messages: list[str] = []

----- public
def compose(self, text: str) -> str:
 self.messages = []

 if not text:
 return ""

 lines = self.line_formatter.normalize_line_endings(text).splitlines()

 # ---- Strategy 1: INLINE (unchanged behaviour)
 plans = self.plan_discoverer.discover(lines)

 # Only an INLINE rewrite replaces the loop with
 # PERFORM <fetch> UNTIL <eoc>. Nothing else may authorise the
 # OBTAIN NEXT cleanup.
 converted_inline = bool(plans)

 if plans:
 lines = self.rewriter.rewrite_main_flow(lines=lines, plans=plans)
 lines = self.rewriter.rewrite_fetch_paragraphs(
 lines=lines,
 plans=plans,
)

 # ---- Strategy 2: SPAN
 if ENFORCE_SPAN_LOOP_REWRITE:
 shapes = self.loop_scanner.scan(lines)
 self._log(
 "scanned",
 inline=len(self.loop_scanner.inline_loops(shapes)),
 span=len(self.loop_scanner.span_loops(shapes)),
)
 span_plans = self.span_plan_builder.build(
 lines=lines,
 shapes=shapes,
 claimed_cursors={plan.cursor_name for plan in plans},
)
 self.messages.extend(self.span_plan_builder.messages)

 if span_plans:
 lines = self.span_rewriter.rewrite(
 lines=lines,
 plans=span_plans,
)
 self.messages.extend(self.span_rewriter.messages)
 # DELIBERATELY does not set converted_inline: the span
 # loop still needs its own in-loop fetch.

 # ---- Cleanup: only after an INLINE conversion
 if converted_inline or not REQUIRE_PLAN_FOR_OBTAIN_NEXT_CLEANUP:
 lines = (
 self.obtain_next_cleanup
 .remove_leftover_obtain_next_cursor_calls(lines)
)
 else:
 self._log("obtain_next_retained")

 return "\n".join(lines).rstrip() + "\n"

----- helpers
def _log(self, key: str, **values) -> None:
 template = CURSOR_LOOP_MESSAGES.get(key, "")

```

```

 if template:
 self.messages.append(template.format(**values))

__all__ = ["CursorFlowComposer"]

=====
FILE: src\idms_db2_phase2\composers\cursor_flow_line_formatter.py
ENCODING: utf-8
=====

"""
Cursor flow line formatter.

Formats generated cursor-flow lines so they match the indentation and
fixed-format layout of a reference line. This helper owns no regex and no
business rules; it only performs safe physical formatting.
"""

from patterns.sequence_patterns import strip_sequence_numbers

class CursorFlowlineFormatter:
 FIXED_FORMAT_MIN_WIDTH = 80
 BODY_WIDTH = 65

 def logical(self, line: str) -> str:
 return strip_sequence_numbers(str(line or "")).strip()

 def normalize_line_endings(self, text: str) -> str:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

 def format_like_line(
 self,
 reference_line: str,
 replacement_body: str,
) -> str:
 line = str(reference_line or "").rstrip()

 if self._is_fixed_format_line(line):
 left = line[:6]
 indicator = line[6]
 right = line[72:80]
 body = self._body_with_reference_indent(
 reference_line=line,
 replacement_body=replacement_body,
)
 return (
 f"{left}{indicator}"
 f"{body[: self.BODY_WIDTH].ljust(self.BODY_WIDTH)}"
 f"{right}"
)

 indent = self.leading_spaces(reference_line, default=" ")
 return f"{indent}{replacement_body}"

 def _body_with_reference_indent(
 self,
 reference_line: str,
 replacement_body: str,
) -> str:
 body = reference_line[7:72].rstrip()
 indent = self.leading_spaces(body, default=" ")
 return f"{indent}{replacement_body}"

 def _is_fixed_format_line(self, line: str) -> bool:
 if len(line) < self.FIXED_FORMAT_MIN_WIDTH:
 return False
 if not line[:6].isdigit():
 return False
 if not line[72:80].isdigit():
 return False
 return True

 def leading_spaces(self, text: str, default: str = "") -> str:

```

```

value = str(text or "")
if not value:
 return default
count = len(value) - len(value.lstrip(" "))
if count <= 0:
 return default
return value[:count]

```

```

=====
FILE: src\idms_db2_phase2\composers\cursor_flow_models.py
ENCODING: utf-8
=====

```

```

"""
Cursor flow domain models.

Holds the data structure describing one discovered cursor execution flow.
This module contains no logic, only the plan definition.
"""

```

```

from dataclasses import dataclass

```

```

@dataclass
class CursorFlowPlan:
 cursor_name: str
 open_number: int
 fetch_number: int
 close_number: int
 open_paragraph: str
 fetch_paragraph: str
 close_paragraph: str
 eoc_condition: str
 business_paragraph: str
 open_index: int
 fetch_index: int
 business_index: int
 until_index: int

```

```

=====
FILE: src\idms_db2_phase2\composers\cursor_flow_obtain_next_cleanup.py
ENCODING: utf-8
=====

```

```

"""
Cursor flow OBTAIN NEXT cleanup.

Removes leftover generated OBTAIN NEXT cursor calls after a cursor has
already been converted to FETCH UNTIL EOC flow, replacing them with a
manual redesign comment and CONTINUE.
"""

```

```

from idms_db2_phase2.composers.cursor_flow_line_formatter import (
 CursorFlowLineFormatter,
)
from patterns.cursor_flow_patterns import (
 CONVERTED_OBTAIN_COMMENT_PATTERN,
 CONVERTED_OBTAIN_NEXT_COMMENT_PATTERN,
 PERFORM_CURSOR_PATTERN,
)
from rules.cursor_flow_rules import REMOVED_OBTAIN_NEXT_COMMENT

```

```

class CursorFlowObtainNextCleanup:
 def __init__(
 self,
 line_formatter: CursorFlowLineFormatter | None = None,
) -> None:
 self.line_formatter = line_formatter or CursorFlowLineFormatter()

 def _logical(self, line: str) -> str:
 return self.line_formatter.logical(line)

 def remove_leftover_obtain_next_cursor_calls(

```

```

 self,
 lines: list[str],
) -> list[str]:
 output: list[str] = []
 index = 0

 while index < len(lines):
 logical = self._logical(lines[index])

 if not CONVERTED_OBTAIN_NEXT_COMMENT_PATTERN.match(logical):
 output.append(lines[index])
 index += 1
 continue

 skipped_lines = [lines[index]]
 index = self._skip_obtain_next_block(
 lines=lines,
 start_index=index + 1,
 skipped_lines=skipped_lines,
)

 output.append(
 self.line_formatter.format_like_line(
 reference_line=skipped_lines[0],
 replacement_body=REMOVED_OBTAIN_NEXT_COMMENT,
)
)
 output.append(
 self.line_formatter.format_like_line(
 reference_line=skipped_lines[0],
 replacement_body="CONTINUE.",
)
)

 return output

```

```

def _skip_obtain_next_block(
 self,
 lines: list[str],
 start_index: int,
 skipped_lines: list[str],
) -> int:
 index = start_index

 while index < len(lines):
 next_logical = self._logical(lines[index])

 if not next_logical:
 skipped_lines.append(lines[index])
 index += 1
 continue

 if next_logical.startswith("*") and not (
 CONVERTED_OBTAIN_COMMENT_PATTERN.match(next_logical)
):
 break

 if CONVERTED_OBTAIN_COMMENT_PATTERN.match(next_logical):
 break

 if PERFORM_CURSOR_PATTERN.match(next_logical):
 skipped_lines.append(lines[index])
 index += 1
 continue

 break

 return index

```

```

=====
FILE: src\idms_db2_phase2\composers\cursor_flow_plan_discoverer.py
ENCODING: utf-8
=====
"""

```

Cursor flow plan discoverer.

Scans generated COBOL lines and correlates each cursor OPEN with its matching FETCH paragraph and business processing loop, producing a list of CursorFlowPlan objects. Discovery logic only.

```
"""
from idms_db2_phase2.composers.cursor_flow_line_formatter import (
 CursorFlowLineFormatter,
)
from idms_db2_phase2.composers.cursor_flow_models import CursorFlowPlan
from patterns.cursor_flow_patterns import (
 PERFORM_BUSINESS_PATTERN,
 PERFORM_CURSOR_PATTERN,
 UNTIL_SQLCODE_100_PATTERN,
)
from rules.cursor_flow_rules import (
 LOOKAHEAD_LIMIT,
 UNTIL_LOOKAHEAD_LIMIT,
)

class CursorFlowPlanDiscoverer:
 def __init__(
 self,
 line_formatter: CursorFlowLineFormatter | None = None,
) -> None:
 self.line_formatter = line_formatter or CursorFlowLineFormatter()

 def _logical(self, line: str) -> str:
 return self.line_formatter.logical(line)

 def discover(self, lines: list[str]) -> list[CursorFlowPlan]:
 plans: list[CursorFlowPlan] = []
 index = 0

 while index < len(lines):
 logical = self._logical(lines[index])
 open_match = PERFORM_CURSOR_PATTERN.match(logical)

 if not open_match:
 index += 1
 continue

 if open_match.group("operation").upper() != "OPEN":
 index += 1
 continue

 plan = self._build_plan(
 lines=lines,
 open_index=index,
 open_match=open_match,
)

 if plan is None:
 index += 1
 continue

 plans.append(plan)
 index = plan.until_index + 1

 return plans

 def _build_plan(
 self,
 lines: list[str],
 open_index: int,
 open_match,
) -> CursorFlowPlan | None:
 cursor_name = open_match.group("cursor").upper()
 open_number = int(open_match.group("number"))
 fetch_number = open_number + 10
 close_number = open_number + 20

 fetch_index = self._find_fetch_after_open(
 lines=lines,
```

```

 start_index=open_index + 1,
 cursor_name=cursor_name,
 fetch_number=fetch_number,
)
 if fetch_index < 0:
 return None

 business_info = self._find_business_loop_after_fetch(
 lines=lines,
 start_index=fetch_index + 1,
)
 if business_info is None:
 return None

 business_index, until_index, business_paragraph = business_info

 return CursorFlowPlan(
 cursor_name=cursor_name,
 open_number=open_number,
 fetch_number=fetch_number,
 close_number=close_number,
 open_paragraph=f"{open_number:03d}-OPEN-{cursor_name}",
 fetch_paragraph=f"{fetch_number:03d}-FETCH-{cursor_name}",
 close_paragraph=f"{close_number:03d}-CLOSE-{cursor_name}",
 eoc_condition=f"{cursor_name}-EOC",
 business_paragraph=business_paragraph,
 open_index=open_index,
 fetch_index=fetch_index,
 business_index=business_index,
 until_index=until_index,
)

def _find_fetch_after_open(
 self,
 lines: list[str],
 start_index: int,
 cursor_name: str,
 fetch_number: int,
) -> int:
 end_index = min(len(lines), start_index + LOOKAHEAD_LIMIT)

 for index in range(start_index, end_index):
 logical = self._logical(lines[index])
 match = PERFORM_CURSOR_PATTERN.match(logical)

 if not match:
 continue
 if match.group("operation").upper() != "FETCH":
 continue
 if match.group("cursor").upper() != cursor_name:
 continue
 if int(match.group("number")) != fetch_number:
 continue

 return index

 return -1

def _find_business_loop_after_fetch(
 self,
 lines: list[str],
 start_index: int,
) -> tuple[int, int, str] | None:
 end_index = min(len(lines), start_index + LOOKAHEAD_LIMIT)

 for index in range(start_index, end_index):
 logical = self._logical(lines[index])

 if not logical:
 continue
 if logical.startswith("*") or logical.startswith("/"):
 continue
 if PERFORM_CURSOR_PATTERN.match(logical):
 continue

 business_match = PERFORM_BUSINESS_PATTERN.match(logical)

```

```

 if not business_match:
 continue

 business_paragraph = business_match.group("paragraph").upper()
 until_index = self._find_until_sqlcode_100(
 lines=lines,
 start_index=index + 1,
)
 if until_index < 0:
 continue

 return index, until_index, business_paragraph

 return None

def _find_until_sqlcode_100(
 self,
 lines: list[str],
 start_index: int,
) -> int:
 end_index = min(len(lines), start_index + UNTIL_LOOKAHEAD_LIMIT)

 for index in range(start_index, end_index):
 logical = self._logical(lines[index])
 if UNTIL_SQLCODE_100_PATTERN.match(logical):
 return index

 return -1

```

```

=====
FILE: src\idms_db2_phase2\composers\cursor_flow_rewriter.py
ENCODING: utf-8
=====

```

```

"""

```

```

Cursor flow rewriter.

```

```

Rewrites the mechanical cursor execution flow into manual-style flow and
converts generated FETCH paragraphs so that WHEN ZERO / CONTINUE performs
the business paragraph. Rewriting logic only.
"""

```

```

from idms_db2_phase2.composers.cursor_flow_line_formatter import (
 CursorFlowLineFormatter,
)
from idms_db2_phase2.composers.cursor_flow_models import CursorFlowPlan
from patterns.cursor_flow_patterns import (
 ANY_PARAGRAPH_HEADER_PATTERN,
 CONTINUE_PATTERN,
 CURSOR_PARAGRAPH_HEADER_PATTERN,
 WHEN_ZERO_PATTERN,
)
from rules.cursor_flow_rules import (
 NON_PARAGRAPH_DOTTED_LINES,
 NON_PARAGRAPH_HEADER_PREFIXES,
)

```

```

class CursorFlowRewriter:
 def __init__(
 self,
 line_formatter: CursorFlowLineFormatter | None = None,
) -> None:
 self.line_formatter = line_formatter or CursorFlowLineFormatter()

 def _logical(self, line: str) -> str:
 return self.line_formatter.logical(line)

 def rewrite_main_flow(
 self,
 lines: list[str],
 plans: list[CursorFlowPlan],
) -> list[str]:
 plan_by_open_index = {plan.open_index: plan for plan in plans}

```



```

indexes_to_skip: set[int] = set()
for plan in plans:
 indexes_to_skip.add(plan.fetch_index)
 indexes_to_skip.add(plan.business_index)
 indexes_to_skip.add(plan.until_index)

output: list[str] = []

for index, line in enumerate(lines):
 if index in indexes_to_skip:
 continue

 plan = plan_by_open_index.get(index)
 if plan is None:
 output.append(line)
 continue

 output.append(line)
 output.append(
 self.line_formatter.format_like_line(
 reference_line=line,
 replacement_body=(
 f"PERFORM {plan.fetch_paragraph} "
 f"UNTIL {plan.eoc_condition}."
),
)
)
 output.append(
 self.line_formatter.format_like_line(
 reference_line=line,
 replacement_body=f"PERFORM {plan.close_paragraph}.",
)
)

return output

def rewrite_fetch_paragraphs(
 self,
 lines: list[str],
 plans: list[CursorFlowPlan],
) -> list[str]:
 business_by_fetch_paragraph = {
 plan.fetch_paragraph.upper(): plan.business_paragraph
 for plan in plans
 }

 output: list[str] = []
 index = 0

 while index < len(lines):
 logical = self._logical(lines[index])
 header_match = CURSOR_PARAGRAPH_HEADER_PATTERN.match(logical)

 if not header_match:
 output.append(lines[index])
 index += 1
 continue

 if header_match.group("operation").upper() != "FETCH":
 output.append(lines[index])
 index += 1
 continue

 fetch_paragraph = (
 f"{header_match.group('number')}-"
 f"FETCH-"
 f"{header_match.group('cursor')}"
).upper()

 business_paragraph = business_by_fetch_paragraph.get(
 fetch_paragraph
)
 if not business_paragraph:
 output.append(lines[index])
 index += 1
 continue

```

```

 paragraph_lines, next_index = self._collect_paragraph(
 lines=lines,
 start_index=index,
)
 output.extend(
 self._replace_when_zero_continue(
 paragraph_lines=paragraph_lines,
 business_paragraph=business_paragraph,
)
)
 index = next_index

 return output

def _collect_paragraph(
 self,
 lines: list[str],
 start_index: int,
) -> tuple[list[str], int]:
 output = [lines[start_index]]
 index = start_index + 1

 while index < len(lines):
 logical = self._logical(lines[index])

 if CURSOR_PARAGRAPH_HEADER_PATTERN.match(logical):
 break
 if self._is_non_cursor_paragraph_header(logical):
 break

 output.append(lines[index])
 index += 1

 return output, index

def _replace_when_zero_continue(
 self,
 paragraph_lines: list[str],
 business_paragraph: str,
) -> list[str]:
 output: list[str] = []
 index = 0

 while index < len(paragraph_lines):
 line = paragraph_lines[index]
 logical = self._logical(line)
 output.append(line)

 if not WHEN_ZERO_PATTERN.match(logical):
 index += 1
 continue

 next_index = self._next_non_blank_index(
 lines=paragraph_lines,
 start_index=index + 1,
)
 if next_index < 0:
 index += 1
 continue

 next_line = paragraph_lines[next_index]
 if not CONTINUE_PATTERN.match(self._logical(next_line)):
 index += 1
 continue

 output.extend(paragraph_lines[index + 1 : next_index])
 output.append(
 self.line_formatter.format_like_line(
 reference_line=next_line,
 replacement_body=f"PERFORM {business_paragraph}",
)
)
 index = next_index + 1

 return output

```

```

def _next_non_blank_index(
 self,
 lines: list[str],
 start_index: int,
) -> int:
 for index in range(start_index, len(lines)):
 if self._logical(lines[index]):
 return index
 return -1

def _is_non_cursor_paragraph_header(self, logical: str) -> bool:
 normalized = str(logical or "").strip().upper()

 if not normalized:
 return False
 if normalized in NON_PARAGRAPH_DOTTED_LINES:
 return False
 if normalized.startswith(NON_PARAGRAPH_HEADER_PREFIXES):
 return False
 if not ANY_PARAGRAPH_HEADER_PATTERN.match(normalized):
 return False
 if CURSOR_PARAGRAPH_HEADER_PATTERN.match(normalized):
 return False

 return True

```

```

=====
FILE: src\idms_db2_phase2\composers\cursor_order_cleanup_composer.py
ENCODING: utf-8
=====

```

```

"""

```

Cursor ORDER BY cleanup composer.

This composer removes unintended ORDER BY clauses from parent/root cursor declarations.

Generic rule:

- A cursor declaration without a WHERE clause is treated as a parent/root cursor.
- Parent/root cursor ORDER BY is removed unless explicit ordering is already handled upstream.
- A cursor declaration with a WHERE clause is treated as a child/dependent cursor and keeps ORDER BY.

This class does not hardcode cursor names, table names, record names, or business fields.

```

"""

```

```

from patterns.sequence_patterns import strip_sequence_numbers

```

```

class CursorOrderCleanupComposer:
 def compose(
 self,
 text: str,
) -> str:
 if not text:
 return ""

 lines = self._normalize_line_endings(text).splitlines()

 output: list[str] = []
 index = 0

 while index < len(lines):
 if not self._is_exec_sql_start(lines[index]):
 output.append(lines[index])
 index += 1
 continue

 block, next_index = self._collect_exec_sql_block(
 lines=lines,
 start_index=index,
)

```

```

 if not self._is_cursor_declare_block(block):
 output.extend(block)
 index = next_index
 continue

 output.extend(
 self._cleanup_cursor_declare_block(block)
)
 index = next_index

 return "\n".join(output).rstrip() + "\n"

def _normalize_line_endings(
 self,
 text: str,
) -> str:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

def _logical(
 self,
 line: str,
) -> str:
 return strip_sequence_numbers(str(line or "")).strip()

def _logical_upper(
 self,
 line: str,
) -> str:
 return self._logical(line).upper()

def _is_exec_sql_start(
 self,
 line: str,
) -> bool:
 return self._logical_upper(line) == "EXEC SQL"

def _is_exec_sql_end(
 self,
 line: str,
) -> bool:
 logical = self._logical_upper(line).rstrip(".")

 return logical == "END-EXEC"

def _collect_exec_sql_block(
 self,
 lines: list[str],
 start_index: int,
) -> tuple[list[str], int]:
 block: list[str] = []
 index = start_index

 while index < len(lines):
 block.append(lines[index])

 if self._is_exec_sql_end(lines[index]):
 return block, index + 1

 index += 1

 return block, index

def _is_cursor_declare_block(
 self,
 block: list[str],
) -> bool:
 for line in block:
 logical = self._logical_upper(line)

 if "DECLARE " in logical and "CURSOR " in logical:
 return True

 return False

def _cleanup_cursor_declare_block(

```

```

 self,
 block: list[str],
) -> list[str]:
 if self._has_where_clause(block):
 return list(block)

 if not self._has_order_by_clause(block):
 return list(block)

 return self._remove_order_by_block(block)

def _has_where_clause(
 self,
 block: list[str],
) -> bool:
 for line in block:
 logical = self._logical_upper(line)

 if logical == "WHERE":
 return True

 if logical.startswith("WHERE "):
 return True

 return False

def _has_order_by_clause(
 self,
 block: list[str],
) -> bool:
 for line in block:
 logical = self._logical_upper(line)

 if logical == "ORDER BY":
 return True

 if logical.startswith("ORDER BY "):
 return True

 return False

def _remove_order_by_block(
 self,
 block: list[str],
) -> list[str]:
 output: list[str] = []
 skipping_order_by = False

 for line in block:
 logical = self._logical_upper(line)

 if self._is_order_by_start(logical):
 skipping_order_by = True
 continue

 if skipping_order_by:
 if self._is_order_by_end(logical):
 skipping_order_by = False
 output.append(line)

 continue

 output.append(line)

 return output

def _is_order_by_start(
 self,
 logical: str,
) -> bool:
 if logical == "ORDER BY":
 return True

 if logical.startswith("ORDER BY "):
 return True

```

```

 return False

 def _is_order_by_end(
 self,
 logical: str,
) -> bool:
 if logical == "FOR READ ONLY":
 return True

 if logical.startswith("FOR READ ONLY"):
 return True

 if logical == "END-EXEC":
 return True

 if logical == "END-EXEC.":
 return True

 return False

```

```

=====
FILE: src\idms_db2_phase2\composers\db2_date_comparison_composer.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/db2_date_comparison_composer.py
ACTION: REPLACE ENTIRE FILE

```

```

"""DB2 date comparison composer.

```

Realigns DB2 date host fields before they are compared with a numeric COBOL date field, and declares the shared date helper Working-Storage (DA-CCYYMMDD, DA-CCYYMMDD-R, DA-DD-MM-CCYY) plus one HELP- field per converted host.

No program, table, DCLGEN group or business field name is hardcoded.

This class owns no regex, constants or templates. It only orchestrates:

Db2DateConditionAssembler	joins one IF across physical lines
Db2DateFieldDetector	finds DATE operands in a condition
Db2DateWorkingStorageManager	declares the helper fields
Db2DateComparisonRewriter	emits realignment + rewrites the IF

CORRECTION 1 - the pass was silent when it did nothing

-----

compose() returned the text unchanged and exposed no `messages`, so a program whose date comparison could not be detected looked exactly like a program with no dates at all. VMDZ7200 shipped comparing a 10-byte DD.MM.CCYY host against an 8-byte CCYYMMDD field and nothing in the conversion log said so. `messages` is now populated on EVERY path, including the no-op path, and conversion\_service.\_component\_messages picks it up automatically.

CORRECTION 2 - detection could not see the real condition

-----

The detector matched one physical line against a pattern that required the DATE host to be the first token after IF AND required the literal token PARMDATE. Both constraints are gone; see patterns/db2\_date\_patterns.py.

ORDERING

-----

Working-Storage is ensured BEFORE the rewrite. The rewrite inserts lines into the PROCEDURE DIVISION and re-scans from scratch, so the earlier DATA DIVISION insertion cannot invalidate its indexes.

```

from __future__ import annotations

```

```

from idms_db2_phase2.composers.db2_date_comparison_rewriter import (
 Db2DateComparisonRewriter,
)
from idms_db2_phase2.composers.db2_date_condition_assembler import (
 Db2DateConditionAssembler,
)

```

```

)
from idms_db2_phase2.composers.db2_date_field_detector import (
 Db2DateFieldDetector,
)
from idms_db2_phase2.composers.db2_date_line_utils import Db2DateLineUtils
from idms_db2_phase2.composers.db2_date_working_storage_manager import (
 Db2DateWorkingStorageManager,
)
from rules.db2_date_conversion_rules import (
 DB2_DATE_COMPARISON_WS_MARKER,
 DB2_DATE_MESSAGES,
 ENFORCE_DATE_COMPARISON_CONVERSION,
)

class Db2DateComparisonComposer:
 """Declares date helpers and converts every DATE host comparison."""

 WS_MARKER = DB2_DATE_COMPARISON_WS_MARKER

 def __init__(self) -> None:
 self.line_utils = Db2DateLineUtils()

 self.condition_assembler = Db2DateConditionAssembler(
 line_utils=self.line_utils,
)
 self.field_detector = Db2DateFieldDetector(
 line_utils=self.line_utils,
 assembler=self.condition_assembler,
)
 self.working_storage_manager = Db2DateWorkingStorageManager(
 line_utils=self.line_utils,
)
 self.comparison_rewriter = Db2DateComparisonRewriter(
 line_utils=self.line_utils,
 assembler=self.condition_assembler,
 detector=self.field_detector,
)

 self.messages: list[str] = []

 #
 # Public entry point
 #
 def compose(self, text: str) -> str:
 self.messages = []

 if not text:
 return ""

 if not ENFORCE_DATE_COMPARISON_CONVERSION:
 return str(text)

 lines = self.line_utils.normalize_line_endings(text).splitlines()

 if not lines:
 return ""

 date_fields = self.field_detector.date_fields_used_in_comparisons(
 lines
)
 shared_helpers_used = (
 self.field_detector.shared_date_helpers_used_in_procedure(lines)
)

 #
 # Nothing to do. Say so, rather than returning in silence.
 #
 if not date_fields and not shared_helpers_used:
 self.messages.append(DB2_DATE_MESSAGES["nothing_found"])
 return "\n".join(lines).rstrip() + "\n"

 #
 # 1. Working-Storage.
 #
 if date_fields or shared_helpers_used:

```

```

 lines = self.working_storage_manager.ensure_date_working_storage(
 lines,
 date_fields,
)

 if date_fields:
 self.messages.append(
 DB2_DATE_MESSAGES["helpers_declared"].format(
 count=len(date_fields),
)
)

 #
 # 2. Realignment and condition rewrite.
 #
 if date_fields:
 lines = self.comparison_rewriter.rewrite_date_comparisons(lines)
 self.messages.extend(self._rewriter_messages())

 return "\n".join(lines).rstrip() + "\n"

#
Helpers
#
def _rewriter_messages(self) -> list[str]:
 """Drain the rewriter's diagnostics, blank entries removed."""
 raw = getattr(self.comparison_rewriter, "messages", None) or []

 return [
 str(message)
 for message in raw
 if str(message or "").strip()
]

__all__ = ["Db2DateComparisonComposer"]

```

```

=====
FILE: src\idms_db2_phase2\composers\db2_date_comparison_rewriter.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/db2_date_comparison_rewriter.py
ACTION: REPLACE ENTIRE FILE

```

```

"""DB2 date comparison rewriter.

```

Emits a realignment block for every DB2 DATE host that takes part in a COBOL comparison, then substitutes that host with its HELP- helper inside the condition.

```

IF (DA-CPTAFS-479BFAS OF DCLDZBFASTV < DA-ARCH-YMD
 AND DA-CPTAFS-479BFAS OF DCLDZBFASTV NOT = '00000000') OR ...

```

becomes

```

* DB2: realigned 2 DB2 DATE host(s) before comparison.
MOVE ZEROES TO DA-CCYYMMDD
MOVE ZEROES TO HELP-DA-CPTAFS-479BFAS
MOVE DA-CPTAFS-479BFAS OF DCLDZBFASTV TO DA-DD-MM-CCYY
EVALUATE TRUE
...
END-EVALUATE
MOVE DA-CCYYMMDD-R TO HELP-DA-CPTAFS-479BFAS
... (same block per host) ...
IF (HELP-DA-CPTAFS-479BFAS < DA-ARCH-YMD
 AND HELP-DA-CPTAFS-479BFAS NOT = '00000000') OR ...

```

FIXED-FORMAT GEOMETRY

-----  
A COBOL record is four fixed column ranges:

```

1-6 left sequence
7 indicator
8-72 body <- the ONLY part this pass may touch

```



73-80 right sequence

Substitution happens inside the body, the body is re-padded to 65 columns, and the record is reassembled. The author's line breaks and indentation survive exactly.

CORRECTION 1 - substitution on the raw line moved the sequence area

-----  
The first version ran the regex over the whole 80-column record.  
`DA-CPTAFS-479BFAS OF DCLDZBFASTV` is 32 characters and  
`HELP-DA-CPTAFS-479BFAS` is 22, so the line shrank by 10 and the right sequence slid from column 73 to column 63. ProcedureIndentNormalizer then read a body with sequence digits in it and wrapped it onto two lines.

CORRECTION 2 - the FixedFormatLineService fallback returned the raw line

-----  
The second version called fixed\_format.body(), guarded by

```
except Exception: return str(line).rstrip()
```

The guard fired and returned the ENTIRE record as if it were the body. That body did not fit 65 columns, replace\_body\_wrapped() wrapped it, and the right-sequence digits were split off as a trailing token, producing

```
002610 AND 0
002620 HELP-DA-CPTAFS-479BFAS NOT = '00000000') OR 0
```

Column arithmetic is now done here, directly, with no fallback that can return something other than a body. A substitution that cannot be represented in 65 columns leaves the line UNCHANGED: a cosmetic pass must never corrupt source.

Rewriting logic only. Templates live in rules/db2\_date\_conversion\_rules.py; regex lives in patterns/db2\_date\_patterns.py.

"""

```
from __future__ import annotations
```

```
import re
```

```
from idms_db2_phase2.composers.db2_date_condition_assembler import (
 Db2DateConditionAssembler,
)
from idms_db2_phase2.composers.db2_date_field_detector import (
 Db2DateFieldDetector,
)
from idms_db2_phase2.composers.db2_date_line_utils import Db2DateLineUtils
from rules.db2_date_conversion_rules import (
 CONDITION_SCAN_LIMIT,
 DB2_DATE_CONVERSION_BANNER_TEMPLATE,
 DB2_DATE_CONVERSION_LINE_TEMPLATES,
 DB2_DATE_HIGH_NUMERIC_LITERAL,
 DB2_DATE_HIGH_VALUE_LITERAL,
 DB2_DATE_LOW_VALUE_LITERAL,
 DB2_DATE_MESSAGES,
 DB2_DATE_REFUSAL_COMMENT_TEMPLATE,
 EMIT_DATE_REFUSAL_COMMENT,
 ENFORCE_DATE_COMPARISON_CONVERSION,
 MAX_DATE_OPERANDS_PER_CONDITION,
)
```

```
DEFAULT_INDENT = " "
```

```
Fixed-format column geometry, zero based, end exclusive.
```

```
LEFT_SEQUENCE_END = 6
```

```
INDICATOR_INDEX = 6
```

```
BODY_START = 7
```

```
BODY_END = 72
```

```
LINE_WIDTH = 80
```

```
BODY_WIDTH = BODY_END - BODY_START # 65
```

```
class Db2DateComparisonRewriter:
```

```
 def __init__(
 self,
```

```

 line_utils: Db2DateLineUtils | None = None,
 assembler: Db2DateConditionAssembler | None = None,
 detector: Db2DateFieldDetector | None = None,
) -> None:
 self.line_utils = line_utils or Db2DateLineUtils()
 self.assembler = assembler or Db2DateConditionAssembler(
 self.line_utils
)
 self.detector = detector or Db2DateFieldDetector(
 line_utils=self.line_utils,
 assembler=self.assembler,
)
 self.messages: list[str] = []

#
Public entry point
#
def rewrite_date_comparisons(self, lines: list[str]) -> list[str]:
 self.messages = []

 if not ENFORCE_DATE_COMPARISON_CONVERSION:
 return list(lines)

 output: list[str] = []
 index = 0
 converted_conditions = 0
 converted_hosts = 0

 while index < len(lines):
 span = self.assembler.span_at(lines, index)

 if span is None:
 output.append(lines[index])
 index += 1
 continue

 operands = self.detector.operands_in(span)

 if not operands:
 output.extend(lines[i] for i in span.line_indexes)
 index = span.end_index + 1
 continue

 refusal = self._refusal_reason(span, operands)

 if refusal:
 output.extend(self._refuse(span, operands, refusal))
 output.extend(lines[i] for i in span.line_indexes)
 index = span.end_index + 1
 continue

 indent = span.indent or DEFAULT_INDENT

 output.append(
 DB2_DATE_CONVERSION_BANNER_TEMPLATE.format(
 indent=indent,
 count=len(operands),
)
)

 for operand in operands:
 output.extend(self._conversion_block(indent, operand))

 output.extend(
 self._rewritten_condition_lines(lines, span, operands)
)

 converted_conditions += 1
 converted_hosts += len(operands)
 index = span.end_index + 1

 if converted_conditions:
 self.messages.append(
 DB2_DATE_MESSAGES["converted"].format(
 count=converted_hosts,
 conditions=converted_conditions,
)
)

```

```

)
)

 return output

#
Realignment block
#
def _conversion_block(self, indent: str, operand) -> list[str]:
 helper = self.line_utils.helper_name(operand.field)

 return [
 template.format(
 indent=indent,
 helper=helper,
 field_name=operand.field,
 group_name=operand.group,
 low_value=DB2_DATE_LOW_VALUE_LITERAL,
 high_value=DB2_DATE_HIGH_VALUE_LITERAL,
 high_numeric=DB2_DATE_HIGH_NUMERIC_LITERAL,
)
 for template in DB2_DATE_CONVERSION_LINE_TEMPLATES
]

#
Condition substitution
#
def _rewritten_condition_lines(
 self,
 lines: list[str],
 span,
 operands,
) -> list[str]:
 """Each physical line with `FIELD OF GROUP` replaced by its helper."""
 replacements = [
 (
 self._operand_pattern(operand),
 self.line_utils.helper_name(operand.field),
)
 for operand in operands
]

 return [
 self._substitute(lines[position], replacements)
 for position in span.line_indexes
]

def _substitute(self, line: str, replacements) -> str:
 """Rewrite columns 8-72 only, then reassemble the record.

 Total function. Returns the ORIGINAL line whenever the rewrite
 cannot be represented, so this pass can never corrupt source.
 """
 original = str(line or "").rstrip("\n")

 if not self._is_fixed_record(original):
 return self._apply(original, replacements)

 padded = original.ljust(LINE_WIDTH)

 left = padded[0:LEFT_SEQUENCE_END]
 indicator = padded[INDICATOR_INDEX:BODY_START]
 body = padded[BODY_START:BODY_END]
 right = padded[BODY_END:LINE_WIDTH]

 new_body = self._apply(body, replacements)

 if new_body == body:
 return original

 trimmed = new_body.rstrip()

 if len(trimmed) > BODY_WIDTH:
 # Not representable in 65 columns. Leave the line alone.
 return original

```

```

 return f"{{left}}{{indicator}}{{trimmed.ljust(BODY_WIDTH)}}{{right}}"

 @staticmethod
 def _is_fixed_record(line: str) -> bool:
 """Columns 1-6 numeric is the only reliable fixed-format marker."""
 text = str(line or "")

 if len(text) < BODY_START:
 return False

 return text[0:LEFT_SEQUENCE_END].isdigit()

 @staticmethod
 def _apply(text: str, replacements) -> str:
 output = str(text or "")

 for pattern, helper in replacements:
 output = pattern.sub(helper, output)

 return output

 @staticmethod
 def _operand_pattern(operand):
 """FIELD OF GROUP` with flexible whitespace, whole-token only."""
 return re.compile(
 r"\b"
 + re.escape(operand.field)
 + r"\s+OF\s+"
 + re.escape(operand.group)
 + r"\b",
 flags=re.IGNORECASE,
)

 #
 # Refusal
 #
 def _refusal_reason(self, span, operands) -> str:
 if not span.complete:
 self.messages.append(
 DB2_DATE_MESSAGES["condition_terminated"].format(
 line=span.start_index + 1,
 limit=CONDITION_SCAN_LIMIT,
)
)
 return "the condition was not closed within the scan limit"

 if len(operands) > MAX_DATE_OPERANDS_PER_CONDITION:
 self.messages.append(
 DB2_DATE_MESSAGES["too_many_operands"].format(
 line=span.start_index + 1,
 count=len(operands),
 limit=MAX_DATE_OPERANDS_PER_CONDITION,
)
)
 return "the condition carries too many DATE operands"

 return ""

 @staticmethod
 def _refuse(span, operands, reason: str) -> list[str]:
 """A pass that cannot convert must say so IN the generated COBOL."""
 if not EMIT_DATE_REFUSAL_COMMENT:
 return []

 indent = span.indent or DEFAULT_INDENT

 return [
 DB2_DATE_REFUSAL_COMMENT_TEMPLATE.format(
 indent=indent,
 field=operand.field,
 reason=f"{{reason}}; convert this comparison manually.",
)
 for operand in operands
]

```

```
__all__ = ["Db2DateComparisonRewriter"]
```

```
=====
FILE: src\idms_db2_phase2\composers\db2_date_condition_assembler.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/db2_date_condition_assembler.py
ACTION: CREATE NEW FILE
```

```
"""Assembles one COBOL IF condition out of its physical lines.
```

```
WHY THIS EXISTS
```

```

Db2DateFieldDetector and Db2DateComparisonRewriter both used to inspect a
single physical line. A hand-written COBOL condition routinely spans
several:
```

```
IF (DA-CPTAFS-479BFAS OF DCLDZBFASTV < DA-ARCH-YMD
 AND DA-CPTAFS-479BFAS OF DCLDZBFASTV NOT = '00000000') OR
 (DA-CRFMAS-479BFAS OF DCLDZBFASTV < DA-ARCH-YMD AND
 DA-CPTAFS-479BFAS OF DCLDZBFASTV = '00000000')
```

A per-line matcher structurally cannot see that, so the date pass was
skipped and the program shipped comparing a 10-byte DD.MM.CCY host
against an 8-byte CCYMMDD field.

```
WHERE A CONDITION ENDS
```

```

Scanning stops at the FIRST of:
```

- \* parentheses balanced AND the line does not end on AND / OR / NOT AND
the next line does not begin with AND / OR / (,
- \* a line that starts a COBOL statement verb,
- \* CONDITION\_SCAN\_LIMIT lines.

Hitting the limit is reported, never guessed around.

Read-only. Assembles and measures; rewrites nothing.

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass, field
```

```
from idms_db2_phase2.composers.db2_date_line_utils import Db2DateLineUtils
from patterns.db2_date_patterns import (
 CONDITION_CONTINUATION_PATTERN,
 END_EXEC_PATTERN,
 EXEC_SQL_START_PATTERN,
 IF_START_PATTERN,
 STATEMENT_START_PATTERN,
 TRAILING_BOOLEAN_PATTERN,
)
from rules.db2_date_conversion_rules import CONDITION_SCAN_LIMIT
```

```
OPEN_PAREN = "("
CLOSE_PAREN = ")"
```

```
@dataclass
```

```
class ConditionSpan:
```

```
 """One IF condition and the physical lines that carry it."""
```

```
 start_index: int = -1
 end_index: int = -1 # inclusive
 indent: str = ""
 text: str = "" # assembled, single space separated
 line_indexes: list[int] = field(default_factory=list)
 complete: bool = True
```

```
@property
```

```
 def line_count(self) -> int:
 return len(self.line_indexes)
```

```

class Db2DateConditionAssembler:
 """Finds IF conditions and joins their physical lines."""

 def __init__(
 self,
 line_utils: Db2DateLineUtils | None = None,
) -> None:
 self.line_utils = line_utils or Db2DateLineUtils()

 #
 # Public entry point
 #
 def span_at(
 self,
 lines: list[str],
 index: int,
) -> ConditionSpan | None:
 """The condition starting at `index`, or None when it is not an IF."""
 if not 0 <= index < len(lines):
 return None

 logical = self.line_utils.logical(lines[index])

 if self.line_utils.is_comment_or_blank(logical):
 return None

 match = IF_START_PATTERN.match(logical)

 if not match:
 return None

 indent = match.group("indent") or ""
 first = str(match.group("rest") or "").strip()

 parts: list[str] = [first] if first else []
 used: list[int] = [index]
 depth = self._depth(first)
 previous = first

 cursor = index + 1
 scanned = 0
 complete = self._is_complete(previous, depth, lines, cursor)

 while not complete and scanned < CONDITION_SCAN_LIMIT:
 if cursor >= len(lines):
 break

 body = self.line_utils.logical(lines[cursor])

 if self.line_utils.is_comment_or_blank(body):
 cursor += 1
 continue

 scanned += 1
 parts.append(body.strip())
 used.append(cursor)
 depth += self._depth(body)
 previous = body
 cursor += 1

 complete = self._is_complete(previous, depth, lines, cursor)

 return ConditionSpan(
 start_index=index,
 end_index=used[-1],
 indent=indent,
 text=" ".join(part for part in parts if part).strip(),
 line_indexes=used,
 complete=complete,
)

 def spans(self, lines: list[str]) -> list[ConditionSpan]:
 """Every IF condition in the program, EXEC SQL blocks excluded."""
 output: list[ConditionSpan] = []
 index = 0

```

```

in_exec_sql = False

while index < len(lines):
 logical = self.line_utils.logical(lines[index])

 if EXEC_SQL_START_PATTERN.match(logical):
 in_exec_sql = not END_EXEC_PATTERN.search(logical)
 index += 1
 continue

 if in_exec_sql:
 if END_EXEC_PATTERN.search(logical):
 in_exec_sql = False
 index += 1
 continue

 span = self.span_at(lines, index)

 if span is None:
 index += 1
 continue

 output.append(span)
 index = span.end_index + 1

return output

#
Termination
#
def _is_complete(
 self,
 previous: str,
 depth: int,
 lines: list[str],
 cursor: int,
) -> bool:
 if depth > 0:
 return False

 if TRAILING_BOOLEAN_PATTERN.search(previous or ""):
 return False

 return not self._next_continues(lines, cursor)

def _next_continues(self, lines: list[str], cursor: int) -> bool:
 index = cursor

 while index < len(lines):
 body = self.line_utils.logical(lines[index])

 if self.line_utils.is_comment_or_blank(body):
 index += 1
 continue

 if STATEMENT_START_PATTERN.match(body):
 return False

 return bool(CONDITION_CONTINUATION_PATTERN.match(body))

 return False

@staticmethod
def _depth(text: str) -> int:
 body = str(text or "")
 return body.count(OPEN_PAREN) - body.count(CLOSE_PAREN)

__all__ = ["ConditionSpan", "Db2DateConditionAssembler"]

```

```
=====
FILE: src\idms_db2_phase2\composers\db2_date_field_detector.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/db2_date_field_detector.py
ACTION: REPLACE ENTIRE FILE
```

```
"""Detects DB2 DATE hosts that take part in a COBOL comparison.
```

```
Detection logic only. Owns no regex (patterns live in
patterns/db2_date_patterns.py) and no templates.
```

```
CORRECTION - detection was single-line and name-bound

```

```
The old detector matched one physical line against a pattern that
required the DATE host to be the first token after IF AND required the
literal token PARMDATE somewhere in the condition. Any parenthesised,
compound, multi-line or differently-named comparison returned an empty
field list, so the composer's `if date_fields:` guard was False and the
whole pass was skipped without a word.
```

```
Detection now works on an ASSEMBLED condition and names no business
field: a DATE host is recognised by its DCLGEN-qualified shape, and it
counts only when the condition also carries a comparison operator.
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
from idms_db2_phase2.composers.db2_date_condition_assembler import (
 ConditionSpan,
 Db2DateConditionAssembler,
)
from idms_db2_phase2.composers.db2_date_line_utils import Db2DateLineUtils
from patterns.db2_date_patterns import (
 COMPARISON_OPERATOR_PATTERN,
 DB2_DATE_HOST_OPERAND_PATTERN,
 DB2_SHARED_DATE_HELPER_USAGE_PATTERN,
 PROCEDURE_DIVISION_PATTERN,
 SQL_HOST_OPERAND_PATTERN,
)
```

```
@dataclass(frozen=True)
class DateOperand:
 """One `DA-xxx OF DCLyyy` reference inside a condition."""
```

```
 field: str = ""
 group: str = ""
```

```
 @property
 def reference(self) -> str:
 return f"{self.field} OF {self.group}"
```

```
class Db2DateFieldDetector:
 def __init__(
 self,
 line_utils: Db2DateLineUtils | None = None,
 assembler: Db2DateConditionAssembler | None = None,
) -> None:
 self.line_utils = line_utils or Db2DateLineUtils()
 self.assembler = assembler or Db2DateConditionAssembler(
 self.line_utils
)

 #
 # Public entry points
 #
 def date_fields_used_in_comparisons(
 self,
 lines: list[str],
) -> list[str]:
 """Distinct DATE host field names compared anywhere, in order."""
```



```

output: list[str] = []
seen: set[str] = set()

for span in self.assembler.spans(lines):
 for operand in self.operands_in(span):
 name = self.line_utils.clean_cobol_name(operand.field)

 if not name or name in seen:
 continue

 seen.add(name)
 output.append(name)

return output

def operands_in(self, span: ConditionSpan) -> list[DateOperand]:
 """Every DATE operand of one condition, duplicates removed.

 Returns [] when the condition carries no comparison operator: a
 bare reference inside an IF is not a comparison and must not
 trigger a rewrite.
 """
 text = str(getattr(span, "text", "") or "")

 if not text:
 return []

 if not COMPARISON_OPERATOR_PATTERN.search(text):
 return []

 # An SQL host reference (:DCLGRP.FIELD) is never rewritten.
 scannable = SQL_HOST_OPERAND_PATTERN.sub(" ", text)

 output: list[DateOperand] = []
 seen: set[tuple[str, str]] = set()

 for found in DB2_DATE_HOST_OPERAND_PATTERN.finditer(scannable):
 field = self.line_utils.clean_cobol_name(found.group("field"))
 group = self.line_utils.clean_cobol_name(found.group("group"))

 if not field or not group:
 continue

 key = (field, group)

 if key in seen:
 continue

 seen.add(key)
 output.append(DateOperand(field=field, group=group))

 return output

def shared_date_helpers_used_in_procedure(
 self,
 lines: list[str],
) -> bool:
 in_procedure_division = False

 for line in lines:
 logical = self.line_utils.logical(line)

 if not logical:
 continue

 if PROCEDURE_DIVISION_PATTERN.match(logical):
 in_procedure_division = True
 continue

 if not in_procedure_division:
 continue

 if self.line_utils.is_comment_or_blank(logical):
 continue

 if DB2_SHARED_DATE_HELPER_USAGE_PATTERN.search(logical):

```

```

 return True

 return False

__all__ = ["DateOperand", "Db2DateFieldDetector"]

=====
FILE: src\idms_db2_phase2\composers\db2_date_line_utils.py
ENCODING: utf-8
=====

"""
DB2 date line utilities.

Shared, stateless helpers used by the DB2 date comparison composer and its
collaborating classes. This module owns no regex and no business rules; it
only performs safe text-level operations.
"""

from patterns.sequence_patterns import strip_sequence_numbers

class Db2DateLineUtils:
 def normalize_line_endings(self, text: str) -> str:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

 def logical(self, line: str) -> str:
 return strip_sequence_numbers(str(line or "")).strip()

 def is_comment_or_blank(self, logical: str) -> bool:
 stripped = str(logical or "").strip()

 if not stripped:
 return True

 return stripped.startswith("*") or stripped.startswith("/")

 def leading_spaces_from_line(
 self,
 line: str,
 default: str = "",
) -> str:
 value = str(line or "")

 if not value:
 return default

 count = len(value) - len(value.lstrip(" "))

 if count <= 0:
 return default

 return value[:count]

 def clean_cobol_name(self, value: str) -> str:
 text = str(value or "").strip().upper()
 text = text.replace("_", "-")
 text = text.rstrip(".")

 while " " in text:
 text = text.replace(" ", "")

 return text

 def helper_name(self, field_name: str) -> str:
 return f"HELP-{self.clean_cobol_name(field_name)}"

=====
FILE: src\idms_db2_phase2\composers\db2_date_working_storage_manager.py
ENCODING: utf-8
=====

"""

```

DB2 date Working-Storage manager.

Ensures the shared DB2 date Working-Storage block and required HELP-\* helper fields exist, without duplicating WS-DATUMVELDEN or any HELP-\* field.

Placement logic only. All templates come from rules/db2\_date\_conversion\_rules.py; all regex from patterns/db2\_date\_patterns.py.

```
from idms_db2_phase2.composers.db2_date_line_utils import Db2DateLineUtils
from patterns.db2_date_patterns import (
 DATE_WORKING_STORAGE_MARKER_PATTERN,
 DB2_DATE_WORKING_STORAGE_BASE_PATTERN,
 LINKAGE_SECTION_PATTERN,
 PROCEDURE_DIVISION_PATTERN,
)
from rules.db2_date_conversion_rules import (
 DB2_DATE_BASE_WORKING_STORAGE_LINES,
 DB2_DATE_HELPER_FIELD_TEMPLATE,
)

WS_DATUMVELDEN_PREFIX = "01 WS-DATUMVELDEN"
LEVEL_01_PREFIX = "01 "
```

```
class Db2DateWorkingStorageManager:
 def __init__(
 self,
 line_utils: Db2DateLineUtils | None = None,
) -> None:
 self.line_utils = line_utils or Db2DateLineUtils()

 def _logical(self, line: str) -> str:
 return self.line_utils.logical(line)

 def ensure_date_working_storage(
 self,
 lines: list[str],
 date_fields: list[str],
) -> list[str]:
 """
 Ensure DB2 date Working-Storage exists.

 - If no date Working-Storage exists, insert the full base block.
 - If it already exists, add only missing HELP-* fields.
 - Update-only flows normally pass no date_fields, so only the base
 block is required.
 - Retrieval/date comparison flows may require HELP-* fields.
 """
 if not self._has_date_working_storage(lines):
 block = self._date_working_storage_block(date_fields)
 insert_index = self._date_working_storage_insert_index(lines)

 if insert_index < 0:
 return block + [""] + lines

 return lines[:insert_index] + block + [""] + lines[insert_index:]

 return self._ensure_missing_date_helper_fields(
 lines=lines,
 date_fields=date_fields,
)

 def _ensure_missing_date_helper_fields(
 self,
 lines: list[str],
 date_fields: list[str],
) -> list[str]:
 """
 Add missing HELP-* date fields when WS-DATUMVELDEN already exists.

 This prevents a retrieval/date-comparison regression where the base
 date block exists but a newly required HELP-* field is missing.
 """
 if not date_fields:
 return lines
```

```

missing_helpers: list[str] = []

for field_name in date_fields:
 helper = self.line_utils.helper_name(field_name)

 if self._helper_declared(lines=lines, helper=helper):
 continue

 missing_helpers.append(helper)

if not missing_helpers:
 return lines

helper_lines = [
 DB2_DATE_HELPER_FIELD_TEMPLATE.format(helper=helper)
 for helper in missing_helpers
]

insert_index = self._date_helper_insert_index(lines)

if insert_index < 0:
 return lines + helper_lines

return lines[:insert_index] + helper_lines + lines[insert_index:]

def _helper_declared(
 self,
 lines: list[str],
 helper: str,
) -> bool:
 """
 Return True only when the HELP-* field is declared before PROCEDURE
 DIVISION. A PROCEDURE DIVISION usage must not count as a declaration,
 which avoids false positives on reruns.
 """
 helper_upper = str(helper or "").strip().upper()

 if not helper_upper:
 return True

 for line in lines:
 logical = self._logical(line)

 if PROCEDURE_DIVISION_PATTERN.match(logical):
 return False

 if helper_upper in logical.upper():
 return True

 return False

def _date_helper_insert_index(self, lines: list[str]) -> int:
 """
 Return index where missing HELP-* fields should be inserted.

 Preferred placement:
 - Inside the existing date Working-Storage area.
 - Before LINKAGE SECTION.
 - Before PROCEDURE DIVISION when LINKAGE SECTION is absent.
 - Before the next 01-level item after WS-DATUMVELDEN.
 """
 ws_datumvelden_seen = False

 for index, line in enumerate(lines):
 logical = self._logical(line)
 upper_logical = logical.upper()

 if upper_logical.startswith(WS_DATUMVELDEN_PREFIX):
 ws_datumvelden_seen = True
 continue

 if not ws_datumvelden_seen:
 continue

 if LINKAGE_SECTION_PATTERN.match(logical):

```

```

 return index

 if PROCEDURE_DIVISION_PATTERN.match(logical):
 return index

 if upper_logical.startswith(LEVEL_01_PREFIX):
 return index

 return -1

def _has_date_working_storage(self, lines: list[str]) -> bool:
 in_procedure_division = False

 for line in lines:
 logical = self._logical(line)

 if DATE_WORKING_STORAGE_MARKER_PATTERN.search(line):
 return True

 if PROCEDURE_DIVISION_PATTERN.match(logical):
 in_procedure_division = True

 if in_procedure_division:
 continue

 upper_logical = logical.upper()

 if upper_logical.startswith(WS_DATUMVELDEN_PREFIX):
 return True

 if DB2_DATE_WORKING_STORAGE_BASE_PATTERN.search(upper_logical):
 return True

 return False

def _date_working_storage_insert_index(self, lines: list[str]) -> int:
 for index, line in enumerate(lines):
 logical = self._logical(line)

 if LINKAGE_SECTION_PATTERN.match(logical):
 return index

 for index, line in enumerate(lines):
 logical = self._logical(line)

 if PROCEDURE_DIVISION_PATTERN.match(logical):
 return index

 return -1

def _date_working_storage_block(
 self,
 date_fields: list[str],
) -> list[str]:
 lines = list(DB2_DATE_BASE_WORKING_STORAGE_LINES)

 for field_name in date_fields:
 helper = self.line_utils.helper_name(field_name)

 lines.append(
 DB2_DATE_HELPER_FIELD_TEMPLATE.format(helper=helper)
)

 return lines

```

```

=====
FILE: src\idms_db2_phase2\composers\final_cobol_fix_composer.py
ENCODING: utf-8
=====

```

```

"""
Final COBOL fix composer.

```

Applies the remaining generic post-conversion COBOL fixes:

1. Program-name synchronization.
2. DB2 date format correction.
3. Cursor SELECT/FETCH minimization.
4. Update-program final cleanup.
5. Procedure Division Area B alignment.
6. Readable IF / ELSE / END-IF and EVALUATE indentation.
7. STRING block manual-style formatting.
8. Final manual-style resequencing.

This composer owns no regex or constants. It orchestrates focused services. It does not hardcode business names, DB2 tables, columns, DCLGEN groups, host variables, or program names.

"""

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
from rules.final_feedback_fix_rules import (
 DEFAULT_DB2_DATE_EXTERNAL_FORMAT,
 ORDER_BY_COLUMNS_IN_SELECT_DEFAULT,
)
```

```
from idms_db2_phase2.services.cobol_area_alignment_service import (
 CobolAreaAlignmentService,
)
```

```
from idms_db2_phase2.services.cursor_select_minimizer_service import (
 CursorSelectMinimizerService,
)
```

```
from idms_db2_phase2.services.db2_comment_readability_service import (
 Db2CommentReadabilityService,
)
```

```
from idms_db2_phase2.services.db2_date_format_service import (
 Db2DateFormatService,
)
```

```
from idms_db2_phase2.services.final_sequence_resequencer_service import (
 FinalSequenceResequencerService,
)
```

```
from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)
```

```
from idms_db2_phase2.services.procedure_block_indent_service import (
 ProcedureBlockIndentService,
)
```

```
from idms_db2_phase2.services.program_name_sync_service import (
 ProgramNameSyncService,
)
```

```
from idms_db2_phase2.services.string_block_format_service import (
 StringBlockFormatService,
)
```

```
from idms_db2_phase2.services.update_cobol_final_cleanup_service import (
 UpdateCobolFinalCleanupService,
)
```

```
@dataclass(frozen=True)
```

```
class FinalCobolFixComposerConfig:
```

```
 db2_date_external_format: str = DEFAULT_DB2_DATE_EXTERNAL_FORMAT
 require_order_by_columns_in_select: bool = (
 ORDER_BY_COLUMNS_IN_SELECT_DEFAULT
)
 resequence_final_output: bool = True
```

```
class FinalCobolFixComposer:
```

```
 def __init__(
 self,
 config: FinalCobolFixComposerConfig | None = None,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.config = config or FinalCobolFixComposerConfig()
 self.fixed_format = fixed_format or FixedFormatLineService()

 self.program_name_sync = ProgramNameSyncService()
 self.date_format_service = Db2DateFormatService(
 date_external_format=self.config.db2_date_external_format,
)
```

```

self.cursor_select_minimizer = CursorSelectMinimizerService(
 fixed_format=self.fixed_format,
 require_order_by_columns_in_select=(
 self.config.require_order_by_columns_in_select
),
)
self.update_final_feedback = UpdateCobolFinalCleanupService(
 fixed_format=self.fixed_format,
)
self.area_alignment = CobolAreaAlignmentService(
 fixed_format=self.fixed_format,
)
self.comment_readability = Db2CommentReadabilityService(
 fixed_format=self.fixed_format,
)
self.procedure_indent = ProcedureBlockIndentService(
 fixed_format=self.fixed_format,
)
self.string_block_format = StringBlockFormatService(
 fixed_format=self.fixed_format,
)
self.sequence_resequencer = FinalSequenceResequencerService()

def compose(self, text: str) -> str:
 output = str(text or "")

 if not output:
 return ""

 output = self.program_name_sync.sync(output)
 output = self.date_format_service.apply(output)
 output = self.cursor_select_minimizer.minimize(output)
 output = self.update_final_feedback.apply(output)
 output = self.area_alignment.align(output)

 output = self.comment_readability.apply(output)
 output = self.procedure_indent.apply(output)

 # STRING block formatting must run after Area B alignment and after
 # generic IF/EVALUATE indentation.
 output = self.string_block_format.apply(output)

 if self.config.resequence_final_output:
 output = self.sequence_resequencer.resequence(output)

 return output.rstrip() + "\n"

def apply_final_cobol_fixes(
 text: str,
 db2_date_external_format: str | None = None,
 require_order_by_columns_in_select: bool = (
 ORDER_BY_COLUMNS_IN_SELECT_DEFAULT
),
 resequence_final_output: bool = True,
) -> str:
 composer = FinalCobolFixComposer(
 config=FinalCobolFixComposerConfig(
 db2_date_external_format=(
 db2_date_external_format or DEFAULT_DB2_DATE_EXTERNAL_FORMAT
),
 require_order_by_columns_in_select=(
 require_order_by_columns_in_select
),
 resequence_final_output=resequence_final_output,
)
)
 return composer.compose(text)

```

```

=====
FILE: src\idms_db2_phase2\composers\fixed_format_body_formatter.py
ENCODING: utf-8
=====

```

```

"""

```

Fixed-format COBOL body formatter.

Responsibilities:

- Decide Area A vs Area B.
- Preserve original indentation wherever possible.
- Keep original business COBOL indentation.
- Normalize generated SQL and generated procedure statements.

Regex patterns live in patterns/fixed\_format\_patterns.py; statement keyword groups in rules/cobol\_statement\_rules.py; layout constants in rules/fixed\_format\_rules.py.

"""

```
from __future__ import annotations
```

```
from patterns.fixed_format_patterns import (
 AREA_A_PREFIX_PATTERN,
 DATA_LEVEL_PATTERN,
 DIVISION_PATTERN,
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
 PARAGRAPH_PATTERN,
 SECTION_PATTERN,
)
```

```
from rules.cobol_statement_rules import (
 NON_PARAGRAPH_WORDS,
 PROCEDURE_VERBS,
 SQL_LEVEL_1_KEYWORDS,
 SQL_LEVEL_2_KEYWORDS,
)
```

```
from rules.fixed_format_rules import (
 AREA_A_DATA_LEVELS,
 AREA_B_INDENT,
 PRESERVE_VERBATIM_INDICATORS,
 PROCEDURE_DIVISION_NAME,
 SQL_INDENT,
)
```

```
class FixedFormatBodyFormatter:
```

```
 def area_body(
 self,
 body: str,
 logical: str,
 current_division: str,
 inside_exec_sql: bool,
 indicator: str,
 previous_procedure_indent: str,
) -> str:
 original = str(body or "").rstrip()
 clean_statement = str(logical or "").strip()
```

```
 if indicator in PRESERVE_VERBATIM_INDICATORS:
 return original
```

```
 if not clean_statement:
 return ""
```

```
 if original.startswith(" "):
 return original
```

```
 if self.is_area_a_statement(clean_statement):
 return clean_statement
```

```
 if inside_exec_sql:
 return self.sql_area_body(clean_statement)
```

```
 if current_division == PROCEDURE_DIVISION_NAME:
 return self.procedure_area_b_body(
 clean_statement=clean_statement,
 previous_procedure_indent=previous_procedure_indent,
)
```

```
 if DATA_LEVEL_PATTERN.match(clean_statement):
 return self.data_area_body(clean_statement)
```



```

 return clean_statement

def procedure_area_b_body(
 self,
 clean_statement: str,
 previous_procedure_indent: str,
) -> str:
 upper = clean_statement.upper()

 if self.is_area_a_statement(clean_statement):
 return clean_statement

 if upper.startswith(PROCEDURE_VERBS):
 return AREA_B_INDENT + clean_statement

 if previous_procedure_indent:
 return previous_procedure_indent + clean_statement

 return AREA_B_INDENT + clean_statement

def data_area_body(self, clean_statement: str) -> str:
 parts = clean_statement.split(maxsplit=1)
 if not parts:
 return clean_statement

 level = parts[0]
 if level in AREA_A_DATA_LEVELS:
 return clean_statement

 return AREA_B_INDENT + clean_statement

def sql_area_body(self, clean_statement: str) -> str:
 upper = clean_statement.upper()

 if EXEC_SQL_START_PATTERN.match(clean_statement):
 return AREA_B_INDENT + clean_statement

 if EXEC_SQL_END_PATTERN.match(clean_statement):
 return AREA_B_INDENT + clean_statement

 if upper.startswith(SQL_LEVEL_1_KEYWORDS):
 return SQL_INDENT + clean_statement

 if upper.startswith(SQL_LEVEL_2_KEYWORDS):
 return SQL_INDENT + clean_statement

 return SQL_INDENT + clean_statement

def is_area_a_statement(self, statement: str) -> bool:
 clean_statement = str(statement or "").strip()
 if not clean_statement:
 return False

 if DIVISION_PATTERN.match(clean_statement):
 return True
 if SECTION_PATTERN.match(clean_statement):
 return True
 if AREA_A_PREFIX_PATTERN.match(clean_statement):
 return True

 if DATA_LEVEL_PATTERN.match(clean_statement):
 first_word = clean_statement.split(maxsplit=1)[0]
 return first_word in AREA_A_DATA_LEVELS

 if not PARAGRAPH_PATTERN.match(clean_statement):
 return False

 paragraph_name = clean_statement.rstrip(".").strip().upper()
 if paragraph_name in NON_PARAGRAPH_WORDS:
 return False

 return True

def is_exec_sql_start(self, statement: str) -> bool:
 return bool(EXEC_SQL_START_PATTERN.match(str(statement or "").strip()))

```

```

def is_exec_sql_end(self, statement: str) -> bool:
 return bool(EXEC_SQL_END_PATTERN.match(str(statement or "").strip()))

def leading_spaces(self, text: str, default: str = "") -> str:
 value = str(text or "")
 if not value:
 return default

 count = len(value) - len(value.lstrip(" "))
 if count <= 0:
 return default

 return value[:count]

=====
FILE: src\idms_db2_phase2\composers\fixed_format_composer.py
ENCODING: utf-8
=====

"""
Fixed-format COBOL composer.

Orchestrates final physical COBOL formatting. Detailed responsibilities are
split into:
- fixed_format_line_parser.py
- fixed_format_body_formatter.py
- fixed_format_wrapper.py
- fixed_format_sequence_manager.py
- fixed_format_line_composer.py (per-line composition mixin)
- patterns/fixed_format_patterns.py
- rules/fixed_format_rules.py

Final physical layout:
- Columns 1-6 : left sequence number
- Column 7 : indicator area
- Columns 8-72 : COBOL body
- Columns 73-80 : right sequence number
"""

from idms_db2_phase2.composers.fixed_format_body_formatter import (
 FixedFormatBodyFormatter,
)
from idms_db2_phase2.composers.fixed_format_line_composer import (
 FixedFormatLineComposer,
)
from idms_db2_phase2.composers.fixed_format_line_parser import (
 FixedFormatLineParser,
)
from idms_db2_phase2.composers.fixed_format_sequence_manager import (
 FixedFormatSequenceManager,
)
from idms_db2_phase2.composers.fixed_format_wrapper import FixedFormatWrapper
from rules.fixed_format_rules import TOTAL_WIDTH, VALID_INDICATORS

class FixedFormatComposer(FixedFormatLineComposer):
 """Final fixed-format COBOL composer.

 Owns no regex patterns, layout constants, or sequence detection rules.
 It coordinates the helper classes; per-line composition is provided by
 FixedFormatLineComposer.

 Sequence numbering is generic:
 - Existing valid left sequence pattern is detected.
 - Existing valid manual-style right sequence pattern is detected.
 - Invalid small right sequence patterns are normalized by the manager.
 """

 def __init__(self) -> None:
 self.line_parser = FixedFormatLineParser()
 self.body_formatter = FixedFormatBodyFormatter()
 self.wrapper = FixedFormatWrapper()
 self.sequence_manager = FixedFormatSequenceManager()

 def format(

```

```

self,
text: str,
left_start: int | None = None,
left_step: int | None = None,
right_start: int | None = None,
right_step: int | None = None,
preserve_blank_lines: bool = True,
) -> str:
 if not text:
 return ""

 lines = self._normalize_line_endings(text).splitlines()
 lines = self._merge_dangling_boolean_lines(lines)

 state = self.sequence_manager.create_state(
 lines=lines,
 left_start=left_start,
 left_step=left_step,
 right_start=right_start,
 right_step=right_step,
)

 output_lines: list[str] = []
 current_division = ""
 inside_exec_sql = False
 previous_procedure_indent = " "

 for raw_line in lines:
 raw_text = str(raw_line or "").rstrip()

 if not raw_text.strip():
 self._append_blank(state, preserve_blank_lines, output_lines)
 continue

 (
 current_division,
 inside_exec_sql,
 previous_procedure_indent,
) = self._compose_body_lines(
 raw_text=raw_text,
 state=state,
 current_division=current_division,
 inside_exec_sql=inside_exec_sql,
 previous_procedure_indent=previous_procedure_indent,
 preserve_blank_lines=preserve_blank_lines,
 output_lines=output_lines,
)

 return "\n".join(output_lines).rstrip() + "\n"

def validate_fixed_format(self, text: str) -> list[str]:
 messages: list[str] = []

 for line_number, line in enumerate(
 str(text or "").splitlines(), start=1
):
 if not line:
 continue

 if len(line) != TOTAL_WIDTH:
 messages.append(
 f"Line {line_number}: expected 80 columns, found {len(line)}."
)
 continue

 left_seq = line[0:6]
 indicator = line[6:7]
 right_seq = line[72:80]

 if not left_seq.isdigit():
 messages.append(
 f"Line {line_number}: left sequence is not numeric."
)

 if indicator not in VALID_INDICATORS:
 messages.append(
 f"Line {line_number}: invalid indicator column value."
)

```

```

)
 if not right_seq.isdigit():
 messages.append(
 f"Line {line_number}: right sequence is not numeric."
)

 return messages

def _normalize_line_endings(self, text: str) -> str:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

def _merge_dangling_boolean_lines(self, lines: list[str]) -> list[str]:
 output: list[str] = []
 index = 0

 while index < len(lines):
 current = str(lines[index] or "").rstrip()

 if index + 1 >= len(lines):
 output.append(current)
 index += 1
 continue

 next_line = str(lines[index + 1] or "").rstrip()
 current_body = self.line_parser.body_for_boolean_merge(current)
 next_body = self.line_parser.body_for_boolean_merge(next_line)

 if not current_body.strip() or not next_body.strip():
 output.append(current)
 index += 1
 continue

 if self.wrapper.is_comment_or_page_line(current_body):
 output.append(current)
 index += 1
 continue

 if self.wrapper.is_comment_or_page_line(next_body):
 output.append(current)
 index += 1
 continue

 if self.wrapper.ends_with_boolean_operator(current_body):
 merged_body = f"{current_body.rstrip()} {next_body.strip()}"
 output.append(
 self.line_parser.replace_body_preserving_sequence(
 original_line=current,
 new_body=merged_body,
)
)
 index += 2
 continue

 output.append(current)
 index += 1

 return output

```

```

=====
FILE: src\idms_db2_phase2\composers\fixed_format_line_composer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from patterns.fixed_format_patterns import DIVISION_PATTERN, SEQUENCE_ONLY_PATTERN
from rules.fixed_format_rules import (
 BODY_WIDTH,
 COMMENT_INDICATOR,
 DEBUG_INDICATOR,
 DEBUG_INDICATOR_LOWER,
 PAGE_INDICATOR,
 PROCEDURE_DIVISION_NAME,
 SPACE_INDICATOR,
 TOTAL_WIDTH,
 VALID_INDICATORS,
)

```

```

class FixedFormatLineComposer:
 """Per-line composition: parse, area-body, wrap, sequence, compose.

 Depends on the host class for ``self.line_parser``, ``self.body_formatter``,
 and ``self.wrapper``. Layout constants and patterns are external.
 """

 def _compose_body_lines(
 self,
 *,
 raw_text: str,
 state,
 current_division: str,
 inside_exec_sql: bool,
 previous_procedure_indent: str,
 preserve_blank_lines: bool,
 output_lines: list[str],
) -> tuple[str, bool, str]:
 """Process one raw line, appending composed lines to output_lines.

 Returns the updated (current_division, inside_exec_sql,
 previous_procedure_indent) tuple.
 """
 if SEQUENCE_ONLY_PATTERN.match(raw_text.strip()):
 return current_division, inside_exec_sql, previous_procedure_indent

 parsed = self.line_parser.parse_line(raw_text)
 indicator = str(parsed["indicator"])
 body = str(parsed["body"])
 logical = body.strip()

 if not body.strip() and indicator == SPACE_INDICATOR:
 self._append_blank(state, preserve_blank_lines, output_lines)
 return current_division, inside_exec_sql, previous_procedure_indent

 division_match = DIVISION_PATTERN.match(logical)
 if division_match:
 current_division = division_match.group(1).upper()
 previous_procedure_indent = " "

 if self.body_formatter.is_exec_sql_start(logical):
 inside_exec_sql = True

 area_body = self.body_formatter.area_body(
 body=body,
 logical=logical,
 current_division=current_division,
 inside_exec_sql=inside_exec_sql,
 indicator=indicator,
 previous_procedure_indent=previous_procedure_indent,
)

 physical_bodies = self.wrapper.wrap_body(
 body=area_body,
 indicator=indicator,
 inside_exec_sql=inside_exec_sql,
 current_division=current_division,
 previous_procedure_indent=previous_procedure_indent,
)
 physical_bodies = self.wrapper.repair_boolean_operator_only_lines(
 physical_bodies
)

 for index, physical_body in enumerate(physical_bodies):
 physical_indicator = (
 self._continuation_indicator(indicator) if index > 0 else indicator
)
 output_lines.append(
 self._compose_line(
 left_seq=state.current_left(),
 indicator=physical_indicator,
 area_body=physical_body,
 right_seq=state.current_right(),
)
)

```

```

)
 state.advance()

 if (
 current_division == PROCEDURE_DIVISION_NAME
 and indicator == SPACE_INDICATOR
 and physical_bodies
 and not self.body_formatter.is_area_a_statement(logical)
):
 previous_procedure_indent = self.body_formatter.leading_spaces(
 physical_bodies[0],
 default=" ",
)

 if self.body_formatter.is_exec_sql_end(logical):
 inside_exec_sql = False

 return current_division, inside_exec_sql, previous_procedure_indent

def _append_blank(self, state, preserve_blank_lines, output_lines) -> None:
 if not preserve_blank_lines:
 return
 output_lines.append(
 self._compose_line(
 left_seq=state.current_left(),
 indicator=SPACE_INDICATOR,
 area_body="",
 right_seq=state.current_right(),
)
)
 state.advance()

def _continuation_indicator(self, indicator: str) -> str:
 if indicator in {COMMENT_INDICATOR, PAGE_INDICATOR}:
 return indicator
 if indicator in {DEBUG_INDICATOR, DEBUG_INDICATOR_LOWER}:
 return DEBUG_INDICATOR
 return SPACE_INDICATOR

def _compose_line(
 self,
 left_seq: str,
 indicator: str,
 area_body: str,
 right_seq: str,
) -> str:
 safe_left = str(left_seq or "").zfill(6)[-6:]
 safe_right = str(right_seq or "").zfill(8)[-8:]
 safe_indicator = str(indicator or SPACE_INDICATOR)[:1]

 if safe_indicator not in VALID_INDICATORS:
 safe_indicator = SPACE_INDICATOR
 if safe_indicator == DEBUG_INDICATOR_LOWER:
 safe_indicator = DEBUG_INDICATOR

 safe_body = str(area_body or "").rstrip()
 if len(safe_body) > BODY_WIDTH:
 safe_body = safe_body[:BODY_WIDTH]

 body_area = safe_body.ljust(BODY_WIDTH)
 line = f"{safe_left}{safe_indicator}{body_area}{safe_right}"

 if len(line) > TOTAL_WIDTH:
 return line[:TOTAL_WIDTH]
 if len(line) < TOTAL_WIDTH:
 return line.ljust(TOTAL_WIDTH)
 return line

```

```

=====
FILE: src\idms_db2_phase2\composers\fixed_format_line_parser.py
ENCODING: utf-8
=====

```

```

"""
Fixed-format COBOL line parser.

```

#### Responsibilities:

- Parse already sequenced 80-column COBOL lines.
- Parse unsequenced COBOL lines.
- Strip old left and right sequence numbers.
- Preserve COBOL body indentation.
- Detect indicator column values.

#### Important:

- A true left sequence number must start in column 1 and be exactly six digits.
- COBOL level numbers such as 01, 03, 05, 10, 20, 77, and 88 must never be treated as sequence numbers.
- Old right sequence numbers may appear separated by spaces or accidentally attached to the COBOL body. Both cases are handled conservatively.

Regex patterns live in `patterns/fixed_format_patterns.py`; layout constants and statement starters live in `rules/fixed_format_rules.py`. Loose-sequence stripping is provided by `FixedFormatSequenceStripper`.

```
"""
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.composers.fixed_format_sequence_stripper import (
 FixedFormatSequenceStripper,
```

```
)
```

```
from patterns.fixed_format_patterns import (
 DEBUG_LINE_PATTERN,
```

```
 TRUE_LEFT_SEQUENCE_PATTERN,
```

```
)
```

```
from rules.fixed_format_rules import (
```

```
 BODY_WIDTH,
```

```
 COMMENT_INDICATOR,
```

```
 DEBUG_INDICATOR,
```

```
 PAGE_INDICATOR,
```

```
 SPACE_INDICATOR,
```

```
 TOTAL_WIDTH,
```

```
 VALID_INDICATORS,
```

```
)
```

```
class FixedFormatLineParser(FixedFormatSequenceStripper):
```

```
 """Parses fixed-format and loose COBOL lines safely."""
```

```
 def parse_line(self, line: str) -> dict[str, str | bool]:
 text = str(line or "").rstrip()
```

```
 fixed = self.parse_fixed_80_line(text)
```

```
 if fixed is not None:
```

```
 return fixed
```

```
 body = self.remove_loose_sequence_numbers(text)
```

```
 indicator = self.indicator_for_body(body)
```

```
 body = self.strip_inline_indicator(body=body, indicator=indicator)
```

```
 return {
```

```
 "left_sequence": "",
```

```
 "indicator": indicator,
```

```
 "body": body,
```

```
 "right_sequence": "",
```

```
 "is_fixed_format": False,
```

```
 }
```

```
 def parse_fixed_80_line(self, line: str) -> dict[str, str | bool] | None:
```

```
 text = str(line or "")
```

```
 if not self._is_fixed_format_line(text):
```

```
 return None
```

```
 indicator = text[6:7]
```

```
 if indicator not in VALID_INDICATORS:
```

```
 indicator = SPACE_INDICATOR
```

```
 return {
```

```
 "left_sequence": text[:6],
```

```
 "indicator": indicator,
```

```
 "body": text[7:72].rstrip(),
```

```

 "right_sequence": text[72:80],
 "is_fixed_format": True,
 }

def body_for_boolean_merge(self, line: str) -> str:
 return str(self.parse_line(line).get("body", ""))

def indicator_for_body(self, body: str) -> str:
 text = str(body or "")
 if not text:
 return SPACE_INDICATOR

 stripped = text.lstrip()
 if stripped.startswith(COMMENT_INDICATOR):
 return COMMENT_INDICATOR
 if stripped.startswith(PAGE_INDICATOR):
 return PAGE_INDICATOR
 if DEBUG_LINE_PATTERN.match(text):
 return DEBUG_INDICATOR
 return SPACE_INDICATOR

def strip_inline_indicator(self, body: str, indicator: str) -> str:
 text = str(body or "")

 if indicator in {COMMENT_INDICATOR, PAGE_INDICATOR}:
 stripped = text.lstrip()
 if stripped.startswith(indicator):
 return stripped[1:].lstrip()

 if indicator == DEBUG_INDICATOR and DEBUG_LINE_PATTERN.match(text):
 stripped = text.lstrip()
 if stripped[:1].upper() == DEBUG_INDICATOR:
 return stripped[1:].lstrip()

 return text.rstrip()

def replace_body_preserving_sequence(
 self, original_line: str, new_body: str
) -> str:
 text = str(original_line or "").rstrip()
 parsed = self.parse_line(text)

 if bool(parsed.get("is_fixed_format")):
 left = str(parsed.get("left_sequence") or "").zfill(6)[-6:]
 indicator = str(parsed.get("indicator") or SPACE_INDICATOR)[:1]
 right = str(parsed.get("right_sequence") or "").zfill(8)[-8:]
 if indicator not in VALID_INDICATORS:
 indicator = SPACE_INDICATOR
 body = str(new_body or "").rstrip()
 return f"{left}{indicator}{body[:BODY_WIDTH].ljust(BODY_WIDTH)}{right}"

 match = TRUE_LEFT_SEQUENCE_PATTERN.match(text)
 if match:
 left = str(match.group("left") or "").zfill(6)[-6:]
 return f"{left} {new_body}"

 return str(new_body or "").rstrip()

def is_debug_line(self, stripped: str) -> bool:
 return bool(DEBUG_LINE_PATTERN.match(str(stripped or "")))

def _is_fixed_format_line(self, line: str) -> bool:
 text = str(line or "")
 if len(text) < TOTAL_WIDTH:
 return False
 if not text[:6].isdigit():
 return False
 if not text[72:80].isdigit():
 return False
 return True

```



```
=====
FILE: src\idms_db2_phase2\composers\fixed_format_sequence_manager.py
ENCODING: utf-8
=====
```

```
"""
```

```
Fixed-format COBOL sequence manager.
```

```
Responsibilities:
```

- Detect left sequence start and step from input when available.
- Detect manual-style right sequence start and step from input when available.
- Reject old/generated right sequence styles like 00000010, 00000020.
- Provide generic fallback manual sequence values when source cannot provide them.

```
"""
```

```
from __future__ import annotations
```

```
import re
```

```
from dataclasses import dataclass
```

```
try:
```

```
 from patterns.fixed_format_patterns import (
 LEFT_SEQUENCE_PATTERN,
 RIGHT_SEQUENCE_PATTERN,
)
```

```
except Exception:
```

```
 LEFT_SEQUENCE_PATTERN = re.compile(
 r"^\s*(?P<left>\d{6})\s+(?P<body>.*)$",
 flags=re.IGNORECASE,
)
```

```
 RIGHT_SEQUENCE_PATTERN = re.compile(
 r"^(?P<body>.*?)(?P<right>\d{8})\s*$",
 flags=re.IGNORECASE,
)
```

```
try:
```

```
 from rules.fixed_format_rules import (
 DEFAULT_LEFT_START,
 DEFAULT_LEFT_STEP,
 DEFAULT_RIGHT_START,
 DEFAULT_RIGHT_STEP,
 TOTAL_WIDTH,
)
```

```
except Exception:
```

```
 DEFAULT_LEFT_START = 10
 DEFAULT_LEFT_STEP = 10
 DEFAULT_RIGHT_START = 10000
 DEFAULT_RIGHT_STEP = 10000
 TOTAL_WIDTH = 80
```

```
@dataclass
```

```
class FixedFormatSequenceState:
```

```
 left_current: int
 left_step: int
 right_current: int
 right_step: int
```

```
 def current_left(
 self,
) -> str:
 return f"{self.left_current:06d}"
```

```
 def current_right(
 self,
) -> str:
 return f"{self.right_current:08d}"
```

```
 def advance(
 self,
) -> None:
 self.left_current += self.left_step
 self.right_current += self.right_step
```

```

class FixedFormatSequenceManager:
 """
 Generic sequence manager.

 Manual right sequence style:
 - 00010000
 - 00020000
 - 00030000

 Old/generated right sequence style:
 - 00000010
 - 00000020

 The old style is intentionally not preserved.
 """

 MIN_MANUAL_RIGHT_SEQUENCE = 10000
 MIN_MANUAL_RIGHT_STEP = 10000

 def create_state(
 self,
 lines: list[str],
 left_start: int | None = None,
 left_step: int | None = None,
 right_start: int | None = None,
 right_step: int | None = None,
) -> FixedFormatSequenceState:
 detected_left_values = self._left_sequence_values(lines)
 detected_right_values = self._right_sequence_values(lines)

 effective_left_start = (
 left_start
 if left_start is not None
 else self._first_positive(
 detected_left_values,
 DEFAULT_LEFT_START,
)
)

 effective_left_step = (
 left_step
 if left_step is not None
 else self._detect_step(
 detected_left_values,
 DEFAULT_LEFT_STEP,
)
)

 effective_right_start = (
 right_start
 if right_start is not None
 else self._detect_manual_right_start(
 detected_right_values,
)
)

 effective_right_step = (
 right_step
 if right_step is not None
 else self._detect_manual_right_step(
 detected_right_values,
)
)

 return FixedFormatSequenceState(
 left_current=effective_left_start,
 left_step=effective_left_step,
 right_current=effective_right_start,
 right_step=effective_right_step,
)

 def _left_sequence_values(
 self,
 lines: list[str],

```

```

) -> list[int]:
 values: list[int] = []

 for line in lines:
 text = str(line or "").rstrip()

 if len(text) >= TOTAL_WIDTH and text[:6].isdigit():
 values.append(int(text[:6]))
 continue

 match = LEFT_SEQUENCE_PATTERN.match(text)

 if match:
 left = str(match.group("left") or "")

 if left.isdigit():
 values.append(int(left))

 return values

def _right_sequence_values(
 self,
 lines: list[str],
) -> list[int]:
 values: list[int] = []

 for line in lines:
 text = str(line or "").rstrip()

 if len(text) >= TOTAL_WIDTH and text[72:80].isdigit():
 values.append(int(text[72:80]))
 continue

 match = RIGHT_SEQUENCE_PATTERN.match(text)

 if match:
 right = str(match.group("right") or "")

 if right.isdigit():
 values.append(int(right))

 return values

def _first_positive(
 self,
 values: list[int],
 default_value: int,
) -> int:
 for value in values:
 if value > 0:
 return value

 return default_value

def _detect_step(
 self,
 values: list[int],
 default_step: int,
) -> int:
 differences = self._positive_differences(values)

 if not differences:
 return default_step

 return self._most_common_difference(
 differences,
 default_step,
)

def _detect_manual_right_start(
 self,
 values: list[int],
) -> int:
 manual_values = [
 value
 for value in values

```

```

 if value >= self.MIN_MANUAL_RIGHT_SEQUENCE
]

 if not manual_values:
 return DEFAULT_RIGHT_START

 return manual_values[0]

def _detect_manual_right_step(
 self,
 values: list[int],
) -> int:
 manual_values = [
 value
 for value in values
 if value >= self.MIN_MANUAL_RIGHT_SEQUENCE
]

 differences = self._positive_differences(manual_values)

 if not differences:
 return DEFAULT_RIGHT_STEP

 step = self._most_common_difference(
 differences,
 DEFAULT_RIGHT_STEP,
)

 if step < self.MIN_MANUAL_RIGHT_STEP:
 return DEFAULT_RIGHT_STEP

 return step

def _positive_differences(
 self,
 values: list[int],
) -> list[int]:
 differences: list[int] = []

 for index in range(1, len(values)):
 difference = values[index] - values[index - 1]

 if difference > 0:
 differences.append(difference)

 return differences

def _most_common_difference(
 self,
 differences: list[int],
 default_step: int,
) -> int:
 if not differences:
 return default_step

 counts: dict[int, int] = {}

 for difference in differences:
 counts[difference] = counts.get(difference, 0) + 1

 return sorted(
 counts.items(),
 key=lambda item: (-item[1], item[0]),
)[0][0]

```

```

=====
FILE: src\idms_db2_phase2\composers\fixed_format_sequence_stripper.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from patterns.fixed_format_patterns import (
 COBOL_IDENTIFIER_TOKEN_PATTERN,
 RIGHT_SEQUENCE_WITH_SPACES_PATTERN,

```

```

 TRAILING_TIGHT_RIGHT_SEQUENCE_PATTERN,
 TRUE_LEFT_SEQUENCE_PATTERN,
)
from rules.fixed_format_rules import (
 COBOL_STATEMENT_STARTERS,
 RIGHT_SEQUENCE_GENERATED_PREFIX,
 TOTAL_WIDTH,
)

class FixedFormatSequenceStripper:
 """Strips loose left/right sequence numbers from non-fixed COBOL lines.

 Owns no regex definitions (patterns live in
 patterns/fixed_format_patterns.py) and no layout constants (rules).
 """

 def remove_loose_sequence_numbers(self, line: str) -> str:
 text = str(line or "").rstrip()
 text = self.remove_loose_right_sequence(text)
 text = self.remove_loose_left_sequence(text)
 return text.rstrip()

 def remove_loose_left_sequence(self, line: str) -> str:
 text = str(line or "")
 match = TRUE_LEFT_SEQUENCE_PATTERN.match(text)
 if not match:
 return text

 left = str(match.group("left") or "")
 body = str(match.group("body") or "")
 if not left.isdigit() or len(left) != 6:
 return text
 return body.lstrip()

 def remove_loose_right_sequence(self, line: str) -> str:
 text = str(line or "").rstrip()

 spaced = RIGHT_SEQUENCE_WITH_SPACES_PATTERN.match(text)
 if spaced:
 body = str(spaced.group("body") or "").rstrip()
 spaces = str(spaced.group("spaces") or "")
 right = str(spaced.group("right") or "")
 if self._looks_like_spaced_right_sequence(
 body=body, spaces=spaces, right=right, original=text
):
 return body

 tight = TRAILING_TIGHT_RIGHT_SEQUENCE_PATTERN.match(text)
 if tight:
 body = str(tight.group("body") or "").rstrip()
 right = str(tight.group("right") or "")
 if self._looks_like_tight_right_sequence(
 body=body, right=right, original=text
):
 return body

 return text

 def _looks_like_spaced_right_sequence(
 self, body: str, spaces: str, right: str, original: str
) -> bool:
 if not self._is_eight_digit_sequence(right):
 return False
 if not body:
 return False
 if len(original) >= TOTAL_WIDTH:
 return True
 if len(spaces) >= 2:
 return True
 return False

 def _looks_like_tight_right_sequence(
 self, body: str, right: str, original: str
) -> bool:
 if not self._is_eight_digit_sequence(right):

```

```

 return False
 if not body:
 return False
 if len(original) >= TOTAL_WIDTH and original[72:80].isdigit():
 return True

 body_upper = body.strip().upper()
 if not body_upper:
 return False
 if body_upper.startswith(COBOL_STATEMENT_STARTERS):
 return True
 if right.startswith(
 RIGHT_SEQUENCE_GENERATED_PREFIX
) and self._body_ends_like_cobol_identifier(body):
 return True
 return False

def _is_eight_digit_sequence(self, value: str) -> bool:
 text = str(value or "")
 return len(text) == 8 and text.isdigit()

def _body_ends_like_cobol_identifier(self, body: str) -> bool:
 text = str(body or "").rstrip()
 if not text:
 return False
 if text[-1].isdigit():
 return False
 if text[-1] in {'"', "'"}:
 return False
 last_token = text.split()[-1]
 return bool(COBOL_IDENTIFIER_TOKEN_PATTERN.fullmatch(last_token))

```

```

=====
FILE: src\idms_db2_phase2\composers\fixed_format_wrapper.py
ENCODING: utf-8
=====

```

```

"""

```

Fixed-format COBOL body wrapper.

Responsibilities:

- Split long comments.
- Split long MOVE statements.
- Split IF statements with AND/OR safely.
- Split generic long statements by word boundaries.
- Keep each body within columns 8-72.

Regex patterns live in patterns/fixed\_format\_patterns.py; layout constants and statement tokens in rules/fixed\_format\_rules.py.

```

"""

```

```

from __future__ import annotations

```

```

from patterns.fixed_format_patterns import (
 BOOLEAN_OPERATOR_END_PATTERN,
 IF_BOOLEAN_SPLIT_PATTERN,
 MOVE_STATEMENT_PATTERN,
)
from rules.fixed_format_rules import (
 BODY_WIDTH,
 BOOLEAN_OPERATOR_WORDS,
 COMMENT_INDICATOR,
 PAGE_INDICATOR,
 PROCEDURE_DIVISION_NAME,
 WRAP_IF_PREFIX,
 WRAP_MOVE_PREFIX,
 WRAP_TO_PREFIX,
)

```

```

class FixedFormatWrapper:
 def wrap_body(
 self,
 body: str,
 indicator: str,
)

```

```

 inside_exec_sql: bool,
 current_division: str,
 previous_procedure_indent: str,
) -> list[str]:
 text = str(body or "").rstrip()

 if len(text) <= BODY_WIDTH:
 return [text]

 if indicator in {COMMENT_INDICATOR, PAGE_INDICATOR}:
 return self.wrap_comment_body(text)

 return self.wrap_cobol_body(
 text=text,
 inside_exec_sql=inside_exec_sql,
 current_division=current_division,
 previous_procedure_indent=previous_procedure_indent,
)

def wrap_comment_body(self, text: str) -> list[str]:
 clean = str(text or "").strip()
 if not clean:
 return [""]
 return self.wrap_by_words(text=clean, continuation_indent="")

def wrap_cobol_body(
 self,
 text: str,
 inside_exec_sql: bool,
 current_division: str,
 previous_procedure_indent: str,
) -> list[str]:
 raw = str(text or "").rstrip()
 logical = raw.strip()

 indent = self.leading_spaces(
 raw,
 default=" " if current_division == PROCEDURE_DIVISION_NAME else "",
)
 if inside_exec_sql:
 indent = self.leading_spaces(raw, default=" ")

 if_lines = self.wrap_if_statement(logical=logical, indent=indent)
 if if_lines:
 return if_lines

 move_lines = self.wrap_move_statement(logical=logical, indent=indent)
 if move_lines:
 return move_lines

 return self.wrap_by_words(
 text=indent + logical,
 continuation_indent=indent + " ",
)

def wrap_if_statement(self, logical: str, indent: str) -> list[str]:
 text = str(logical or "").strip()

 if not text.upper().startswith(WRAP_IF_PREFIX):
 return []

 split_result = self.split_condition_on_boolean_operator(text)
 if not split_result:
 return self.wrap_by_words(
 text=indent + text,
 continuation_indent=indent + " ",
)

 left_part, operator, right_part = split_result
 first_line = f"{indent}{left_part.rstrip()}"
 second_line = f"{indent}{operator.strip().upper()} {right_part.strip()}"

 return self.force_lines_to_body_width(
 lines=[first_line, second_line],
 continuation_indent=indent + " ",
)

```

```

def split_condition_on_boolean_operator(
 self, logical: str
) -> tuple[str, str, str] | None:
 text = str(logical or "").strip()
 tokens = IF_BOOLEAN_SPLIT_PATTERN.split(text, maxsplit=1)

 if len(tokens) < 3:
 return None

 left_part = tokens[0].rstrip()
 operator = tokens[1].strip().upper()
 right_part = tokens[2].strip()

 if not left_part or not operator or not right_part:
 return None

 return left_part, operator, right_part

def wrap_move_statement(self, logical: str, indent: str) -> list[str]:
 match = MOVE_STATEMENT_PATTERN.match(str(logical or "").strip())
 if not match:
 return []

 src = match.group("src").strip()
 tgt = match.group("tgt").strip()

 first = f"{indent}{WRAP_MOVE_PREFIX}{src}"
 second = f"{indent}{WRAP_TO_PREFIX}{tgt}"

 if len(first) <= BODY_WIDTH and len(second) <= BODY_WIDTH:
 return [first, second]

 return self.wrap_by_words(
 text=indent + logical,
 continuation_indent=indent + " ",
)

def wrap_by_words(self, text: str, continuation_indent: str) -> list[str]:
 raw = str(text or "").rstrip()
 if len(raw) <= BODY_WIDTH:
 return [raw]

 leading_indent = self.leading_spaces(raw, default="")
 content = raw.strip()
 words = content.split()

 lines: list[str] = []
 current = leading_indent

 for word in words:
 if current.strip():
 candidate = current.rstrip() + " " + word
 else:
 candidate = leading_indent + word

 if len(candidate) <= BODY_WIDTH:
 current = candidate
 continue

 if current.strip():
 lines.append(current.rstrip())

 current = continuation_indent + word

 if current.strip():
 lines.append(current.rstrip())

 return self.force_lines_to_body_width(
 lines=lines,
 continuation_indent=continuation_indent,
)

def force_lines_to_body_width(
 self, lines: list[str], continuation_indent: str
) -> list[str]:

```



```

output: list[str] = []

for line in lines:
 text = str(line or "").rstrip()

 if len(text) <= BODY_WIDTH:
 output.append(text)
 continue

 indent = self.leading_spaces(text, default=continuation_indent)
 available_width = max(1, BODY_WIDTH - len(indent))
 content = text.strip()

 while content:
 part = content[:available_width].rstrip()
 if not part:
 part = content[:available_width]
 output.append(indent + part)
 content = content[len(part):].strip()

return self.repair_boolean_operator_only_lines(output)

def repair_boolean_operator_only_lines(
 self, lines: list[str]
) -> list[str]:
 output: list[str] = []
 index = 0

 while index < len(lines):
 current = str(lines[index] or "").rstrip()
 current_stripped = current.strip().upper()

 if current_stripped in BOOLEAN_OPERATOR_WORDS and index + 1 < len(lines):
 next_line = str(lines[index + 1] or "").rstrip()
 next_stripped = next_line.strip()

 if next_stripped:
 indent = self.leading_spaces(current, default=" ")
 repaired = f"{indent}{current_stripped} {next_stripped}"
 if len(repaired) <= BODY_WIDTH:
 output.append(repaired)
 index += 2
 continue

 output.append(current)
 index += 1

 return output

def ends_with_boolean_operator(self, body: str) -> bool:
 return bool(
 BOOLEAN_OPERATOR_END_PATTERN.search(str(body or "").strip())
)

def is_comment_or_page_line(self, logical: str) -> bool:
 stripped = str(logical or "").strip()
 return stripped.startswith(COMMENT_INDICATOR) or stripped.startswith(
 PAGE_INDICATOR
)

def leading_spaces(self, text: str, default: str = "") -> str:
 value = str(text or "")
 if not value:
 return default

 count = len(value) - len(value.lstrip(" "))
 if count <= 0:
 return default

 return value[:count]

```

```
=====
FILE: src\idms_db2_phase2\composers\late_db2_date_composer.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/late_db2_date_composer.py
ACTION: CREATE NEW FILE
```

```
"""Late DB2 date output conversion.
```

```
Re-runs Db2DateOutputCleanup after the output write block has reached its
FINAL location.
```

```
Why a second run is needed
```

```

Db2DateOutputCleanup lives inside CobolCleanupComposer. Its documented
ordering places it after OutputWritePlacementCleanup so it can see the
write block in its final position.
```

```
That held while the block merely moved between paragraphs. It no longer
holds: OutputWriteParagraphComposer now EXTRACTS the block into a new
WRITE-<record> paragraph much later in the pipeline. The date converter
has already finished by then, so a DB2 date host field is moved straight
into the output record without the CCYYMMDD realignment:
```

```
MOVE DA-UBSECIS-479BEFF OF DCLDZBEFFTV TO UIT-DA-UBSEC
```

```
UIT-DA-UBSEC then receives the raw DB2 external date form rather than the
numeric form the downstream file expects. That is silent data corruption,
not a formatting defect.
```

```
Idempotence
```

```

A second run is safe. The converter only matches a MOVE whose source is a
DB2 date host qualified by a DCLGEN group. After conversion the move
reads
```

```
MOVE DA-CCYYMMDD-R TO UIT-DA-UBSEC
```

```
which carries no DCLGEN qualifier and therefore cannot match again.
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.composers.cleanup.cleanup_message_collector import (
 CleanupMessageCollector,
)
from idms_db2_phase2.composers.cleanup.db2_date_output_cleanup import (
 Db2DateOutputCleanup,
)
from idms_db2_phase2.composers.cleanup.initialize_before_output_cleanup import (
 InitializeBeforeOutputCleanup,
)
```

```
class LateDb2DateComposer:
```

```
 """Applies the date and INITIALIZE passes to the relocated block."""
```

```
 def __init__(self) -> None:
 self.collector = CleanupMessageCollector()
 self.date_cleanup = Db2DateOutputCleanup(messages=self.collector)
 self.initialize_cleanup = InitializeBeforeOutputCleanup(
 messages=self.collector
)
 self.messages: list[str] = []
```

```
 def compose(self, text: str) -> str:
 self.messages = []
```

```
 if not text:
 return ""
```

```
 self.collector.reset()
 output = str(text)
```

```
 output = self.initialize_cleanup.ensure_initialize_before_output_population(
```

```

 output
)
 output = self.date_cleanup.convert_db2_date_moves_to_output_dates(
 output
)

 self.messages = list(self.collector.messages)
 return output.rstrip() + "\n"

```

```

=====
FILE: src\idms_db2_phase2\composers\manual_layout_composer.py
ENCODING: utf-8
=====

```

```
import re
```

```

from catalogs.output_sections import (
 DB2_CURSOR_PARAGRAPH_MARKER,
 DB2_INFRASTRUCTURE_MARKER,
)
from patterns.db2_patterns import END_PROGRAM_PATTERN

```

```
class ManualLayoutComposer:
```

```

 """
 Keeps generated DB2 blocks in a manual-style production layout.

```

```

 Required output order:
 1. Original COBOL header starts first.
 2. Original DATA and PROCEDURE structure remains.
 3. Generated DB2 infrastructure remains one complete block.
 4. Generated cursor OPEN/FETCH/CLOSE paragraphs are near the end.
 5. SQL-ERROR paragraph is near the end.
 6. END PROGRAM remains last.

```

```

 This composer is intentionally conservative. It normalizes spacing and
 keeps END PROGRAM as the final program boundary without rewriting business
 logic.
 """

```

```

def compose(
 self,
 text: str,
) -> str:
 if not text:
 return ""

 lines = self._normalize_line_endings(text).splitlines()
 lines = self._rstrip_blank_lines(lines)

 if not lines:
 return ""

 end_program_line = ""
 end_program_index = self._find_end_program_index(lines)

 if end_program_index >= 0:
 end_program_line = lines[end_program_index]
 lines = lines[:end_program_index] + lines[end_program_index + 1:]

 lines = self._normalize_generated_block_spacing(lines)

 if end_program_line:
 lines = self._append_end_program(lines, end_program_line)

 result = "\n".join(lines)
 result = self._normalize_blank_lines(result)

 return result.strip() + "\n"

```

```

def _normalize_line_endings(
 self,
 text: str,
) -> str:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

```

```

def _normalize_generated_block_spacing(
 self,
 lines: list[str],
) -> list[str]:
 output: list[str] = []

 for line in lines:
 logical = str(line or "").upper()

 if DB2_INFRASTRUCTURE_MARKER in logical:
 output = self._rstrip_blank_lines(output)
 output.append("")

 if DB2_CURSOR_PARAGRAPH_MARKER in logical:
 output = self._rstrip_blank_lines(output)
 output.append("")

 output.append(line)

 return output

def _append_end_program(
 self,
 lines: list[str],
 end_program_line: str,
) -> list[str]:
 if not end_program_line:
 return lines

 trimmed = self._rstrip_blank_lines(lines)

 if not trimmed:
 return [end_program_line]

 return trimmed + ["", end_program_line]

def _find_end_program_index(
 self,
 lines: list[str],
) -> int:
 for index, line in enumerate(lines):
 logical = self._logical_line(line)

 if END_PROGRAM_PATTERN.match(logical):
 return index

 return -1

def _logical_line(
 self,
 line: str,
) -> str:
 text = str(line or "").rstrip()

 text = self._remove_right_sequence(text)
 text = self._remove_left_sequence(text)

 return text.strip()

def _remove_left_sequence(
 self,
 line: str,
) -> str:
 text = str(line or "")

 if len(text) > 6 and text[:6].strip().isdigit():
 return text[6:].strip()

 return text

def _remove_right_sequence(
 self,
 line: str,
) -> str:
 text = str(line or "").rstrip()

```

```

match = re.match(
 r"^(?P<body>.*?)(?:\s+(?P<right>\d{8}))\s*$",
 text,
 flags=re.IGNORECASE,
)

if match:
 return match.group("body").rstrip()

return text

def _rstrip_blank_lines(
 self,
 lines: list[str],
) -> list[str]:
 index = len(lines)

 while index > 0 and not str(lines[index - 1] or "").strip():
 index -= 1

 return lines[:index]

def _lstrip_blank_lines(
 self,
 lines: list[str],
) -> list[str]:
 index = 0

 while index < len(lines) and not str(lines[index] or "").strip():
 index += 1

 return lines[index:]

def _trim_blank_edges(
 self,
 lines: list[str],
) -> list[str]:
 return self._lstrip_blank_lines(
 self._rstrip_blank_lines(lines)
)

def _normalize_blank_lines(
 self,
 text: str,
) -> str:
 normalized = str(text or "")

 normalized = re.sub(
 r"\n{4,}",
 "\n\n\n",
 normalized,
)

 normalized = re.sub(
 r"\n[\t]+\n",
 "\n\n",
 normalized,
)

 return normalized

```

```

=====
FILE: src\idms_db2_phase2\composers\manual_style_preserver.py
ENCODING: utf-8
=====

```

```

class ManualStylePreserver:
 """
 Preserves manual COBOL style by avoiding broad formatting changes.

 Requirement:
 - Output should remain close to original IDMS COBOL style.
 - Only DB2 conversion changes should be carried out.
 - Existing comments should remain where possible.
 """

```

```
"""
```

```
def preserve(
 self,
 original_text: str,
 converted_text: str,
) -> str:
 if not converted_text:
 return ""

 text = self._preserve_comment_density(converted_text)
 text = self._normalize_excess_blank_lines(text)

 return text.rstrip() + "\n"

def _preserve_comment_density(
 self,
 text: str,
) -> str:
 output_lines: list[str] = []

 for line in str(text or "").splitlines():
 output_lines.append(line.rstrip())

 return "\n".join(output_lines)

def _normalize_excess_blank_lines(
 self,
 text: str,
) -> str:
 while "\n\n\n\n" in text:
 text = text.replace("\n\n\n\n", "\n\n\n")

 return text
```

```
=====
FILE: src\idms_db2_phase2\composers\output_write__init__.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/output_write/__init__.py
ACTION: REPLACE ENTIRE FILE
"""Output write paragraph extraction.
```

```
 output_write_line_utils.py line inspection and cloning
 output_write_scanner.py divisions, IF/END-IF, WRITE, paragraphs
 output_write_guard.py structural safety gate
 output_write_block_finder.py candidate discovery
 output_write_eligibility.py accept / refuse decisions
 output_write_renderer.py call site and paragraph rendering
 output_write_extractor.py line replacement
```

```
Import order matters: every module below imports its collaborators from
the CONCRETE module path, never from this package, so nothing here can
create a cycle.
"""
```

```
from idms_db2_phase2.composers.output_write.output_write_line_utils import (
 OutputWriteLineUtils,
)
from idms_db2_phase2.composers.output_write.output_write_scanner import (
 OutputWriteScanner,
)
from idms_db2_phase2.composers.output_write.output_write_guard import (
 OutputWriteGuard,
)
from idms_db2_phase2.composers.output_write.output_write_renderer import (
 OutputWriteRenderer,
)
from idms_db2_phase2.composers.output_write.output_write_block_finder import (
 UNNAMED_PARAGRAPH,
 CandidateBlock,
 OutputWriteBlockFinder,
)
from idms_db2_phase2.composers.output_write.output_write_eligibility import (
```

```

 ExtractionPlan,
 OutputWriteEligibility,
 Refusal,
)
from idms_db2_phase2.composers.output_write.output_write_extractor import (
 OutputWriteExtractor,
)

```

```

__all__ = [
 "UNNAMED_PARAGRAPH",
 "CandidateBlock",
 "ExtractionPlan",
 "OutputWriteBlockFinder",
 "OutputWriteEligibility",
 "OutputWriteExtractor",
 "OutputWriteGuard",
 "OutputWriteLineUtils",
 "OutputWriteRenderer",
 "OutputWriteScanner",
 "Refusal",
]

```

```

=====
FILE: src\idms_db2_phase2\composers\output_write\output_write_block_finder.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/output_write/output_write_block_finder.py
ACTION: CREATE NEW FILE
"""Walks the PROCEDURE DIVISION and yields candidate IF blocks.

```

Read-only discovery. It knows how to find an IF / END-IF pair and which paragraph that pair sits in; it knows nothing about WRITE statements, paragraph naming or whether a block is worth extracting.

Candidates are yielded in source order, so the caller can try each in turn and stop at the first it accepts.

```

from __future__ import annotations

from collections.abc import Iterator
from dataclasses import dataclass

from idms_db2_phase2.composers.output_write.output_write_line_utils import (
 OutputWriteLineUtils,
)
from idms_db2_phase2.composers.output_write.output_write_scanner import (
 OutputWriteScanner,
)

```

```

UNNAMED_PARAGRAPH = "(unnamed)"

```

```

@dataclass(frozen=True)
class CandidateBlock:
 """One IF / END-IF pair inside a PROCEDURE DIVISION paragraph."""

 if_index: int = -1
 end_index: int = -1
 paragraph: str = ""

 @property
 def body_start(self) -> int:
 """First line of the guarded body.

 The IF line itself is never moved, which is why the guard must
 prove the condition does not span lines before any extraction
 is attempted.
 """
 return self.if_index + 1

 @property
 def source(self) -> str:
 return self.paragraph or UNNAMED_PARAGRAPH

```

```
def body(self, lines: list[str]) -> list[str]:
 return lines[self.body_start:self.end_index]
```

```
class OutputWriteBlockFinder:
 """Finds every IF / END-IF pair in the PROCEDURE DIVISION."""

 def __init__(
 self,
 line_utils: OutputWriteLineUtils | None = None,
 scanner: OutputWriteScanner | None = None,
) -> None:
 self.lines = line_utils or OutputWriteLineUtils()
 self.scanner = scanner or OutputWriteScanner(self.lines)

 # ----- public
 def candidates(self, lines: list[str]) -> Iterator[CandidateBlock]:
 """Every complete IF block, in source order.

 Stops at the first division boundary after the PROCEDURE
 DIVISION, so a LINKAGE or nested program can never be reached.
 """
 start = self.scanner.procedure_division_index(lines)
 if start < 0:
 return

 index = start + 1
 paragraph = ""

 while index < len(lines):
 logical = self.lines.logical(lines[index])

 if self.lines.is_division_boundary(logical):
 return

 header = self.lines.paragraph_name(lines[index])
 if header:
 paragraph = header
 index += 1
 continue

 if not self.scanner.is_condition_start(logical):
 index += 1
 continue

 end_index = self.scanner.matching_end_if(lines, index)
 if end_index >= 0:
 yield CandidateBlock(
 if_index=index,
 end_index=end_index,
 paragraph=paragraph,
)

 index += 1

__all__ = ["UNNAMED_PARAGRAPH", "CandidateBlock", "OutputWriteBlockFinder"]
```

```
=====
FILE: src\idms_db2_phase2\composers\output_write\output_write_eligibility.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/output_write/output_write_eligibility.py
ACTION: CREATE NEW FILE
"""Decides whether one candidate block may be extracted.
```

Every gate REFUSES rather than guesses. A refusal leaves working COBOL and, where it is worth reporting, a diagnostic; a guess produces source the compiler rejects or a WRITE that fires the wrong number of times.

THE GATES, IN ORDER

```
1 exactly one WRITE two writes is a business decision
```



```

2 enough executable lines a one-line body is not worth a paragraph
3 not already extracted an earlier pass may have done it
4 the paragraph name is free never shadow an existing paragraph
"""

from __future__ import annotations

from dataclasses import dataclass, field

from idms_db2_phase2.composers.output_write.output_write_block_finder import (
 CandidateBlock,
)
from idms_db2_phase2.composers.output_write.output_write_line_utils import (
 OutputWriteLineUtils,
)
from idms_db2_phase2.composers.output_write.output_write_scanner import (
 OutputWriteScanner,
)
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.output_write_paragraph_rules import (
 MINIMUM_BODY_LINES,
 REQUIRED_WRITE_COUNT,
 WRITE_PARAGRAPH_TEMPLATE,
)

@dataclass(frozen=True)
class Refusal:
 """A named diagnostic plus its template values.

 An empty key means 'refuse silently' - the block simply is not a
 candidate, and saying so on every IF in the program would drown the
 validation log.
 """

 key: str = ""
 values: dict = field(default_factory=dict)

 @property
 def is_reportable(self) -> bool:
 return bool(self.key)

@dataclass(frozen=True)
class ExtractionPlan:
 """An accepted block and the paragraph it will become."""

 block: CandidateBlock
 paragraph: str = ""
 record: str = ""
 body: list[str] = field(default_factory=list)

class OutputWriteEligibility:
 """All accept / refuse decisions for one candidate block."""

 def __init__(
 self,
 line_utils: OutputWriteLineUtils | None = None,
 scanner: OutputWriteScanner | None = None,
) -> None:
 self.lines = line_utils or OutputWriteLineUtils()
 self.scanner = scanner or OutputWriteScanner(self.lines)

 # ----- public
 def assess(
 self,
 lines: list[str],
 block: CandidateBlock,
 existing: set[str],
) -> tuple[ExtractionPlan | None, Refusal | None]:
 """(plan, refusal). Exactly one of the two is None."""
 body = block.body(lines)

 writes = self.scanner.write_records(body)
 if len(writes) != REQUIRED_WRITE_COUNT:

```

```

 return None, self._wrong_write_count(block, writes)

 if len(self.lines.executable_lines(body)) < MINIMUM_BODY_LINES:
 return None, Refusal(
 "skipped_too_small",
 {"source": block.source},
)

 if self.scanner.already_extracted(body):
 return None, Refusal()

 record = NameNormalizer.to_cobol(writes[0])
 paragraph = WRITE_PARAGRAPH_TEMPLATE.format(record=record)

 if paragraph in existing:
 return None, Refusal(
 "skipped_name_taken",
 {"paragraph": paragraph},
)

 return (
 ExtractionPlan(
 block=block,
 paragraph=paragraph,
 record=record,
 body=body,
),
 None,
)

----- helpers
@staticmethod
def _wrong_write_count(
 block: CandidateBlock,
 writes: list[str],
) -> Refusal:
 """Two WRITES is worth reporting; zero is just an ordinary IF."""
 if not writes:
 return Refusal()
 return Refusal(
 "skipped_multiple_writes",
 {"source": block.source, "count": len(writes)},
)

```

```
__all__ = ["ExtractionPlan", "OutputWriteEligibility", "Refusal"]
```

```
=====
FILE: src\idms_db2_phase2\composers\output_write\output_write_extractor.py
ENCODING: utf-8
=====
```

```

LOCATION: src/idms_db2_phase2/composers/output_write/output_write_extractor.py
ACTION: CREATE NEW FILE
"""Performs one accepted extraction.

```

Lines are MOVED and re-indented, never edited: no condition, MOVE or WRITE is ever rewritten. Two splices happen, in this order:

- 1 the guarded body is replaced by the call site,
- 2 the new paragraph is inserted after the host paragraph ends.

The second index must be corrected for the size change the first splice caused, which is the whole reason this lives in its own class.

```

from __future__ import annotations

from idms_db2_phase2.composers.output_write.output_write_eligibility import (
 ExtractionPlan,
)
from idms_db2_phase2.composers.output_write.output_write_line_utils import (
 OutputWriteLineUtils,
)
from idms_db2_phase2.composers.output_write.output_write_renderer import (

```

```

 OutputWriteRenderer,
)
from idms_db2_phase2.composers.output_write.output_write_scanner import (
 OutputWriteScanner,
)

```

```

class OutputWriteExtractor:
 """Splices the call site in and the new paragraph after."""

 def __init__(
 self,
 line_utils: OutputWriteLineUtils | None = None,
 renderer: OutputWriteRenderer | None = None,
 scanner: OutputWriteScanner | None = None,
) -> None:
 self.lines = line_utils or OutputWriteLineUtils()
 self.renderer = renderer or OutputWriteRenderer(self.lines)
 self.scanner = scanner or OutputWriteScanner(self.lines)

 # ----- public
 def apply(self, lines: list[str], plan: ExtractionPlan) -> None:
 """Rewrite `lines` in place for one accepted plan."""
 block = plan.block
 template = self.lines.template_line(lines, block.if_index)

 call_site = self.renderer.call_site_lines(
 template=template,
 indent=self.lines.call_indent(lines, block.body_start),
 paragraph=plan.paragraph,
)
 paragraph_block = self.renderer.paragraph_lines(
 template=template,
 paragraph=plan.paragraph,
 body=plan.body,
)

 # 1. Replace the guarded body with the PERFORM call site.
 lines[block.body_start:block.end_index] = call_site

 # 2. Insert the new paragraph after the host paragraph ends,
 # correcting for the size change step 1 just made.
 insert_at = self._insert_index(
 lines=lines,
 end_index=block.end_index,
 shift=len(call_site) - len(plan.body),
)
 lines[insert_at:insert_at] = paragraph_block

 # ----- helpers
 def _insert_index(
 self,
 *,
 lines: list[str],
 end_index: int,
 shift: int,
) -> int:
 """Index just past the paragraph that held the extracted block."""
 return self.scanner.paragraph_end_index(lines, end_index + shift)

```

```
__all__ = ["OutputWriteExtractor"]
```

```

=====
FILE: src\idms_db2_phase2\composers\output_write\output_write_guard.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/output_write/output_write_guard.py
ACTION: CREATE NEW FILE
"""Structural safety gate for the output write extraction.

```

```

DEFECT THIS CLOSES

```

```

Extraction splices the call site in place of the IF BODY, cutting at

```

if\_index + 1. That is correct only when the whole condition lives on the IF line. A hand-written guard frequently spans lines:

```
IF (DA-CPTAFS OF DCL... < DA-ARCH-YMD
 AND DA-CPTAFS OF DCL... NOT = '00000000') OR
 (DA-CRFMAS OF DCL... < DA-ARCH-YMD AND
 DA-CPTAFS OF DCL... = '00000000')
 PERFORM ...
```

Cutting at if\_index + 1 leaves the IF with no condition tail and moves the tail into the new paragraph, where it begins with a dangling AND. The program does not compile.

The gate is deliberately PROGRAM-WIDE and coarse: when any PROCEDURE DIVISION IF carries a multi-line condition, extraction is refused for the whole program and a diagnostic is emitted. Refusing leaves working COBOL; guessing produces source the compiler rejects.

```
from __future__ import annotations

from idms_db2_phase2.composers.output_write.output_write_line_utils import (
 OutputWriteLineUtils,
)
from idms_db2_phase2.services.cobol_condition_scanner import (
 CobolConditionScanner,
)
from patterns.output_write_paragraph_patterns import (
 PROCEDURE_DIVISION_PATTERN,
)

SINGLE_LINE_CONDITION = 1

class OutputWriteGuard:
 """Refuses extraction when a guard condition spans lines."""

 def __init__(
 self,
 line_utils: OutputWriteLineUtils | None = None,
 condition_scanner: CobolConditionScanner | None = None,
) -> None:
 self.lines = line_utils or OutputWriteLineUtils()
 self.condition_scanner = condition_scanner or CobolConditionScanner(
 fixed_format=self.lines.fixed_format,
)

 # ----- public
 def blocks_extraction(self, text: str) -> bool:
 return self.widest_condition(text) > SINGLE_LINE_CONDITION

 def widest_condition(self, text: str) -> int:
 """Line count of the longest PROCEDURE DIVISION IF condition."""
 lines = (
 str(text or "")
 .replace("\r\n", "\n")
 .replace("\r", "\n")
 .split("\n")
)

 in_procedure = False
 widest = 0

 for index, line in enumerate(lines):
 logical = self.lines.logical(line)
 if not logical:
 continue

 if PROCEDURE_DIVISION_PATTERN.match(logical):
 in_procedure = True
 continue

 if not in_procedure:
 continue

 if self.lines.is_skippable(line):
```

```

 continue

 if not self.condition_scanner.is_condition_start(line):
 continue

 size = self.condition_scanner.condition_line_count(lines, index)
 widest = max(widest, size)

 return widest

__all__ = ["OutputWriteGuard"]

=====
FILE: src\idms_db2_phase2\composers\output_write\output_write_line_utils.py
ENCODING: utf-8
=====

LOCATION: src\idms_db2_phase2\composers\output_write\output_write_line_utils.py
ACTION: CREATE NEW FILE
"""Line inspection and cloning for the output write extraction.

Two text views are deliberately distinct:

 logical(line) UPPERCASED - for pattern matching only
 body_text(line) ORIGINAL case - for text that is written back

Mixing them would upper-case business COBOL, which the business-flow
rule forbids.
"""

from __future__ import annotations

from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)
from patterns.output_write_paragraph_patterns import (
 DIVISION_PATTERN,
 PARAGRAPH_HEADER_PATTERN,
)
from rules.output_write_paragraph_rules import (
 IND_STATEMENT,
 SCOPE_TERMINATOR_WORDS,
)

END_SCOPE_PREFIX = "END-"

class OutputWriteLineUtils:
 """Shared line helpers. Owns no business decision."""

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()

 # ----- text views
 def logical(self, line: str) -> str:
 """Uppercased logical line. Matching only."""
 return self.fixed_format.logical(line).upper()

 def body_text(self, line: str) -> str:
 """Logical line in its ORIGINAL case. Rewriting only."""
 return self.fixed_format.logical(line)

 def is_skippable(self, line: str) -> bool:
 return self.fixed_format.is_comment_or_control_line(line)

 def body_indent(self, line: str) -> str:
 return self.fixed_format.body_indent(line)

 # ----- selection
 def executable_lines(self, body: list[str]) -> list[str]:
 return [

```

```

 line for line in body
 if self.body_text(line) and not self.is_skippable(line)
]

def last_executable_index(self, body: list[str]) -> int:
 for index in range(len(body) - 1, -1, -1):
 line = body[index]
 if self.is_skippable(line):
 continue
 if self.body_text(line):
 return index
 return -1

----- structure
def paragraph_name(self, line: str) -> str:
 if self.is_skippable(line):
 return ""

 match = PARAGRAPH_HEADER_PATTERN.match(self.logical(line))
 if not match:
 return ""

 name = match.group("name").upper()
 if name in SCOPE_TERMINATOR_WORDS or name.startswith(
 END_SCOPE_PREFIX
):
 return ""

 return name

def paragraph_names(self, lines: list[str]) -> set[str]:
 return {
 name for name in (self.paragraph_name(line) for line in lines)
 if name
 }

@staticmethod
def is_division_boundary(logical: str) -> bool:
 return bool(DIVISION_PATTERN.match(logical))

----- building
@staticmethod
def template_line(lines: list[str], index: int) -> str:
 return lines[index] if 0 <= index < len(lines) else ""

def clone(
 self,
 template: str,
 body: str,
 indicator: str | None = None,
) -> str:
 """Build a line borrowing the sequence area of `template`.

 FinalSequenceResequencerService rewrites columns 1-6 and 73-80
 afterwards, so borrowed numbers are placeholders only.
 """
 left, template_indicator, _body, right = self.fixed_format.split(
 template
)
 marker = (
 indicator if indicator is not None
 else (template_indicator or " ")
)
 built = self.fixed_format.build_or_none(left, marker, body, right)
 return built if built is not None else body

def replace_body_wrapped(self, line: str, body: str) -> list[str]:
 return self.fixed_format.replace_body_wrapped(line, body)

def call_indent(self, lines: list[str], body_start: int) -> str:
 for index in range(body_start, len(lines)):
 line = lines[index]
 if self.is_skippable(line):
 continue
 if self.body_text(line):
 return self.body_indent(line)

```

```

 return IND_STATEMENT

__all__ = ["OutputWriteLineUtils"]

=====
FILE: src\idms_db2_phase2\composers\output_write\output_write_renderer.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/output_write/output_write_renderer.py
ACTION: CREATE NEW FILE
"""Renders the call site and the extracted write paragraph.

Lines are MOVED and re-indented, never edited: no condition, MOVE or
WRITE is ever rewritten.
"""

from __future__ import annotations

from idms_db2_phase2.composers.output_write.output_write_line_utils import (
 OutputWriteLineUtils,
)
from rules.output_write_paragraph_rules import (
 BLANK_LINE,
 COUNTER_ADD_TEMPLATE,
 EMIT_OUTPUT_COUNTER_INCREMENT,
 EMIT_PAGE_EJECT_BEFORE_PARAGRAPH,
 IND_STATEMENT,
 OUTPUT_COUNTER_NAME,
 PAGE_EJECT_INDICATOR,
 PARAGRAPH_TERMINATOR,
 PERFORM_TEMPLATE,
 WRITE_PARAGRAPH_HEADER_TEMPLATE,
)

class OutputWriteRenderer:
 """Builds the replacement call site and the new paragraph."""

 def __init__(
 self,
 line_utils: OutputWriteLineUtils | None = None,
) -> None:
 self.lines = line_utils or OutputWriteLineUtils()

 # ----- call site
 def call_site_lines(
 self,
 template: str,
 indent: str,
 paragraph: str,
) -> list[str]:
 bodies = [indent + PERFORM_TEMPLATE.format(paragraph=paragraph)]

 if EMIT_OUTPUT_COUNTER_INCREMENT:
 bodies.append(
 indent
 + COUNTER_ADD_TEMPLATE.format(name=OUTPUT_COUNTER_NAME)
)

 return [self.lines.clone(template, body) for body in bodies]

 # ----- paragraph
 def paragraph_lines(
 self,
 template: str,
 paragraph: str,
 body: list[str],
) -> list[str]:
 out: list[str] = [self.lines.clone(template, BLANK_LINE)]

 if EMIT_PAGE_EJECT_BEFORE_PARAGRAPH:
 out.append(
 self.lines.clone(

```

```

 template,
 BLANK_LINE,
 indicator=PAGE_EJECT_INDICATOR,
)
)

out.append(
 self.lines.clone(
 template,
 WRITE_PARAGRAPH_HEADER_TEMPLATE.format(paragraph=paragraph),
)
)
out.extend(self.reindented_body(body))
return out

def reindented_body(self, body: list[str]) -> list[str]:
 """Re-anchor the lifted body to Area B, preserving relative depth.

 The block was nested inside two IFs, so every line carries extra
 indent. The shallowest executable line defines the baseline, and
 deeper lines keep their offset from it.
 """
 executable = self.lines.executable_lines(body)
 if not executable:
 return list(body)

 baseline = min(
 len(self.lines.body_indent(line)) for line in executable
)
 last_index = self.lines.last_executable_index(body)

 out: list[str] = []
 for position, line in enumerate(body):
 text = self.lines.body_text(line)

 if not text or self.lines.is_skippable(line):
 out.append(line)
 continue

 offset = max(len(self.lines.body_indent(line)) - baseline, 0)
 body_text = IND_STATEMENT + (" " * offset) + text

 if position == last_index and not body_text.endswith(
 PARAGRAPH_TERMINATOR
):
 body_text = body_text + PARAGRAPH_TERMINATOR

 out.extend(self.lines.replace_body_wrapped(line, body_text))

 return out

```

```
__all__ = ["OutputWriteRenderer"]
```

```

=====
FILE: src\idms_db2_phase2\composers\output_write\output_write_scanner.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/output_write/output_write_scanner.py
ACTION: CREATE NEW FILE
"""Block discovery for the output write extraction.

```

```

Read-only: locates the PROCEDURE DIVISION, matches IF / END-IF pairs,
finds WRITE records and paragraph boundaries. Rewrites nothing.
"""

```

```
from __future__ import annotations
```

```

from idms_db2_phase2.composers.output_write.output_write_line_utils import (
 OutputWriteLineUtils,
)
from patterns.output_write_paragraph_patterns import (
 END_IF_PATTERN,
 IF_START_PATTERN,

```



```

PERFORM_WRITE_PARAGRAPH_PATTERN,
PROCEDURE_DIVISION_PATTERN,
WRITE_STATEMENT_PATTERN,
)
from rules.output_write_paragraph_rules import IF_SCAN_LIMIT

class OutputWriteScanner:
 """Finds the boundaries the extraction needs."""

 def __init__(
 self,
 line_utils: OutputWriteLineUtils | None = None,
) -> None:
 self.lines = line_utils or OutputWriteLineUtils()

 # ----- divisions
 def procedure_division_index(self, lines: list[str]) -> int:
 for index, line in enumerate(lines):
 if PROCEDURE_DIVISION_PATTERN.match(self.lines.logical(line)):
 return index
 return -1

 # ----- scope
 @staticmethod
 def is_condition_start(logical: str) -> bool:
 return bool(IF_START_PATTERN.match(logical))

 def matching_end_if(self, lines: list[str], if_index: int) -> int:
 """Index of the END-IF that closes the IF at if_index."""
 depth = 0
 limit = min(len(lines), if_index + IF_SCAN_LIMIT)

 for index in range(if_index, limit):
 line = lines[index]

 if self.lines.is_skippable(line):
 continue

 logical = self.lines.logical(line)
 if not logical:
 continue

 if index > if_index and self.lines.paragraph_name(line):
 return -1

 if IF_START_PATTERN.match(logical):
 depth += 1
 continue

 if END_IF_PATTERN.match(logical):
 depth -= 1
 if depth == 0:
 return index

 return -1

 # ----- write
 def write_records(self, body: list[str]) -> list[str]:
 out: list[str] = []

 for line in body:
 if self.lines.is_skippable(line):
 continue
 match = WRITE_STATEMENT_PATTERN.match(self.lines.logical(line))
 if match:
 out.append(match.group("record"))

 return out

 def already_extracted(self, body: list[str]) -> bool:
 """True when an earlier pass already replaced this block."""
 return any(
 PERFORM_WRITE_PARAGRAPH_PATTERN.match(self.lines.logical(line))
 for line in body
)

```

```

----- paragraph
def paragraph_end_index(self, lines: list[str], start: int) -> int:
 """Index just past the paragraph containing `start`."""
 for index in range(start + 1, len(lines)):
 if self.lines.paragraph_name(lines[index]):
 return index
 if self.lines.is_division_boundary(
 self.lines.logical(lines[index])
):
 return index
 return len(lines)

__all__ = ["OutputWriteScanner"]

=====
FILE: src\idms_db2_phase2\composers\output_write_paragraph_composer.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/output_write_paragraph_composer.py
ACTION: REPLACE ENTIRE FILE
"""Output write paragraph composer.

Lifts an inline output-record population block into its own paragraph and
replaces it with a PERFORM, matching the COBOL team's manual reference.

 HOOFDVERWERKING.
 IF WS-STATUS = 'C'
 INITIALIZE UITRECORD ->
 MOVE ... TO ...
 WRITE UITRECORD
 END-IF.

 HOOFDVERWERKING.
 IF WS-STATUS = 'C'
 PERFORM WRITE-UITRECORD
 ADD 1 TO WS-NB-OUTPUT-COUNT
 END-IF.

 /
 WRITE-UITRECORD.
 INITIALIZE UITRECORD
 MOVE ... TO ...
 WRITE UITRECORD.

Scope and safety

- PROCEDURE DIVISION only.
- Exactly ONE WRITE per block. Two writes is a business decision.
- REFUSES for the whole program when any guard condition spans lines.
- Lines are moved and re-indented, never edited.

Ordering

AFTER composers["feedback_cleanup"] (relocates the guarded block) and
AFTER composers["fixed_format"] (so lines can clone a sequence area).
BEFORE FinalSequenceResequencerService.

Responsibility split

output_write/output_write_line_utils.py line inspection
output_write/output_write_scanner.py boundaries
output_write/output_write_guard.py structural safety gate
output_write/output_write_block_finder.py candidate discovery
output_write/output_write_eligibility.py accept / refuse
output_write/output_write_renderer.py rendering
output_write/output_write_extractor.py line replacement

This class owns iteration and diagnostics. Nothing else.
"""

from __future__ import annotations

Concrete module imports, not the package. A stale or partially updated
__init__.py then cannot break this composer:
#
ImportError: cannot import name 'OutputWriteBlockFinder' from
'idms_db2_phase2.composers.output_write'
#
from idms_db2_phase2.composers.output_write.output_write_block_finder import (
 OutputWriteBlockFinder,

```

```

)
from idms_db2_phase2.composers.output_write.output_write_eligibility import (
 OutputWriteEligibility,
)
from idms_db2_phase2.composers.output_write.output_write_extractor import (
 OutputWriteExtractor,
)
from idms_db2_phase2.composers.output_write.output_write_guard import (
 OutputWriteGuard,
)
from idms_db2_phase2.composers.output_write.output_write_line_utils import (
 OutputWriteLineUtils,
)
from idms_db2_phase2.composers.output_write.output_write_scanner import (
 OutputWriteScanner,
)
from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)
from rules.output_write_paragraph_rules import (
 EMIT_OUTPUT_COUNTER_INCREMENT,
 ENFORCE_OUTPUT_WRITE_PARAGRAPH,
 EXTRACTION_PASS_LIMIT,
 OUTPUT_COUNTER_NAME,
 OUTPUT_WRITE_PARAGRAPH_MESSAGES,
)
from rules.structural_safety_rules import (
 ENFORCE_CONDITION_INTEGRITY,
 STRUCTURAL_SAFETY_MESSAGES,
)

```

```

class OutputWriteParagraphComposer:
 """Extracts inline output write blocks into their own paragraphs."""

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.lines = OutputWriteLineUtils(
 fixed_format=fixed_format or FixedFormatLineService(),
)
 scanner = OutputWriteScanner(self.lines)

 self.guard = OutputWriteGuard(self.lines)
 self.finder = OutputWriteBlockFinder(self.lines, scanner)
 self.eligibility = OutputWriteEligibility(self.lines, scanner)
 self.extractor = OutputWriteExtractor(self.lines, scanner=scanner)
 self.messages: list[str] = []

 # =====
 # Public entry point
 # =====
 def compose(self, text: str) -> str:
 self.messages = []

 if not text or not ENFORCE_OUTPUT_WRITE_PARAGRAPH:
 return str(text or "")

 if self._refused_by_guard(text):
 return str(text)

 lines = (
 str(text)
 .replace("\r\n", "\n")
 .replace("\n", "\n")
 .split("\n")
)

 # Each success shifts every index after it, so the scan restarts.
 for _pass in range(EXTRACTION_PASS_LIMIT):
 if not self._extract_once(lines):
 break

 return "\n".join(lines).rstrip() + "\n"

```

```

=====
Structural safety
=====
def _refused_by_guard(self, text: str) -> bool:
 """Never cut between an IF keyword and the end of its condition.

 The gate is deliberately program-wide: one multi-line guard
 anywhere refuses every extraction in the program. Refusing
 leaves working COBOL; guessing produces source the compiler
 rejects.
 """
 if not ENFORCE_CONDITION_INTEGRITY:
 return False
 if not self.guard.blocks_extraction(text):
 return False

 self.messages.append(
 STRUCTURAL_SAFETY_MESSAGES["extraction_refused"].format(
 count=self.guard.widest_condition(text),
)
)
 return True

=====
One extraction
=====
def _extract_once(self, lines: list[str]) -> bool:
 """Extract the first eligible block. True when one was taken."""
 existing = self.lines.paragraph_names(lines)

 for block in self.finder.candidates(lines):
 plan, refusal = self.eligibility.assess(lines, block, existing)

 if plan is None:
 if refusal is not None and refusal.is_reportable:
 self._log(refusal.key, **refusal.values)
 continue

 self.extractor.apply(lines, plan)
 self._log_success(block.source, plan.paragraph)
 return True

 return False

=====
Diagnostics
=====
def _log_success(self, source: str, paragraph: str) -> None:
 self._log("extracted", source=source, paragraph=paragraph)

 if EMIT_OUTPUT_COUNTER_INCREMENT:
 self._log(
 "counter_added",
 counter=OUTPUT_COUNTER_NAME,
 paragraph=paragraph,
)

def _log(self, key: str, **values) -> None:
 template = OUTPUT_WRITE_PARAGRAPH_MESSAGES.get(key, "")
 if template:
 self.messages.append(template.format(**values))

__all__ = ["OutputWriteParagraphComposer"]

=====
FILE: src\idms_db2_phase2\composers\procedure_indent_normalizer.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/procedure_indent_normalizer.py
ACTION: REPLACE ENTIRE FILE

"""Procedure Division indentation normalizer.

```

Runs late (before final resequence) to give PROCEDURE DIVISION executable statements a consistent, DEPTH-AWARE Area-B indentation. Fixes irregular indents produced by different generators (SQL MOVES, EXEC SQL, timestamp MOVES, etc.) and indents IF / EVALUATE / PERFORM .. END-PERFORM bodies progressively to match the manual COBOL standard.

Safe scope

- Only PROCEDURE DIVISION.
- Only executable statement lines (space indicator in column 7).
- Never touches comments ('\*' / '/' ), debug lines ('D'), continuation lines ('-'), Area-A paragraph/section headers, or the DATA and other divisions.
- Preserves the 80-column frame: rewrites only columns 8-72 body indent; left seq (1-6) and right seq (73-80) untouched.
- EXEC SQL ... END-EXEC blocks keep the flat SQL body indent, not depth.

CORRECTION - silent truncation

This class previously carried a PRIVATE copy of the fixed-format geometry that had drifted from the shared service:

```
@staticmethod
def _is_fixed_line(line):
 return (len(text) >= 80 and text[:6].isdigit()
 and text[72:80].isdigit())

@staticmethod
def _rebuild(left, new_body, right):
 body_area = new_body[:65].ljust(65) # <-- silent truncation
 return f"{left[:6].zfill(6)}{' '}{body_area[:65]}{right[:8]}"
```

Two defects followed.

1. `_is_fixed_line` demanded a full 80 columns AND a numeric right sequence, so a short page-eject line such as '001690/' was misparsed and its sequence number could reach the COBOL body.
2. `_rebuild` truncated any body past column 72 without telling anyone. Because this normalizer is the LAST content pass in the pipeline, it is how

```
MOVE NR-IDGOOD-479EVEF OF DCLDZEVEFTV TO WS-NR-ID-GOOD
```

was written out as

```
MOVE NR-IDGOOD-479EVEF OF DCLDZEVEFTV TO WS-NR-ID-GO
```

producing an undefined data name that the compiler rejects.

Geometry is now delegated entirely to `FixedFormatLineService`. A re-indent that no longer fits is WRAPPED onto a continuation line; if even that is impossible the original line is kept unchanged. A cosmetic pass must never destroy a statement.

TEMPORARY INSTRUMENTATION

A four-line IF condition reached the generated file merged into two truncated lines. This pass processes ONE line at a time and has no join path, so the merge must happen upstream. `DUMP_INTERMEDIATE` writes the text this pass receives and the text it returns, which settles the question in one run. Set it to False, or delete the three marked blocks, once the culprit is fixed.

Clean Architecture

- Constants live in `rules/procedure_indent_rules.py`.
- Regex lives in `patterns/procedure_indent_patterns.py`.
- No program, paragraph, record, table, cursor or host variable name is hardcoded.

```
from __future__ import annotations
```

```
from dataclasses import dataclass
from pathlib import Path # TEMP DEBUG
```

```

from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)
from patterns.procedure_indent_patterns import (
 ELSE_PATTERN,
 END_EXEC_PATTERN,
 EVALUATE_START_PATTERN,
 EXEC_SQL_START_PATTERN,
 IF_START_PATTERN,
 INLINE_PERFORM_PATTERN,
 LONE_PERIOD_PATTERN,
 OTHER_DIVISION_PATTERN,
 PARAGRAPH_HEADER_PATTERN,
 PROCEDURE_DIVISION_PATTERN,
 SCOPE_TERMINATOR_PATTERN,
 SCOPED_OPENER_PATTERN,
 SECTION_HEADER_PATTERN,
 WHEN_PATTERN,
)
from rules.procedure_indent_rules import (
 AREA_A_INDENT,
 AREA_B_INDENT,
 BLOCK_KIND_EVALUATE,
 BLOCK_KIND_IF,
 BLOCK_KIND_OTHER,
 BLOCK_KIND_PERFORM,
 BLOCK_KIND_SQL,
 ENFORCE_PROCEDURE_INDENT,
 LONE_PERIOD_INDENT,
 MAX_NESTING_DEPTH,
 NEST_STEP,
 NON_PARAGRAPH_SINGLE_WORDS,
 PROCEDURE_INDENT_MESSAGES,
 SCOPE_TERMINATORS,
 SQL_BODY_OFFSET,
 WHEN_BODY_OFFSET,
 WHEN_OFFSET,
)

TEMP DEBUG - set False or delete the marked blocks once the upstream
merge is fixed.
DUMP_INTERMEDIATE = True
DUMP_IN = "debug_indent_in.txt"
DUMP_OUT = "debug_indent_out.txt"

```

```

@dataclass
class _Frame:
 """One open block on the indentation stack."""

 kind: str
 own_indent: int

 @property
 def body_indent(self) -> int:
 if self.kind == BLOCK_KIND_EVALUATE:
 return self.own_indent + WHEN_BODY_OFFSET
 if self.kind == BLOCK_KIND_SQL:
 return self.own_indent + SQL_BODY_OFFSET
 return self.own_indent + NEST_STEP

 @property
 def when_indent(self) -> int:
 return self.own_indent + WHEN_OFFSET

class ProcedureIndentNormalizer:
 """Depth-aware Area-B re-indentation for the PROCEDURE DIVISION."""

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()
 self.messages: list[str] = []

```

```

=====
Public entry point
=====
def compose(self, text: str) -> str:
 self.messages = []

 self._dump(DUMP_IN, text) # TEMP DEBUG

 if not text or not ENFORCE_PROCEDURE_INDENT:
 return str(text or "")

 lines = (
 str(text)
 .replace("\r\n", "\n")
 .replace("\r", "\n")
 .split("\n")
)

 output: list[str] = []
 stack: list[_Frame] = []
 in_procedure = False
 normalized = 0
 wrapped = 0

 for number, line in enumerate(lines, start=1):
 # --- lines this pass must never touch -----
 if self._is_untouchable(line):
 output.append(line)
 continue

 logical = self.fixed_format.logical(line)
 if not logical:
 output.append(line)
 continue

 upper = logical.upper()

 # --- division boundaries -----
 if PROCEDURE_DIVISION_PATTERN.match(upper):
 in_procedure = True
 stack = []
 output.append(line)
 continue

 if OTHER_DIVISION_PATTERN.match(upper):
 in_procedure = False
 stack = []
 output.append(line)
 continue

 if not in_procedure:
 output.append(line)
 continue

 # --- Area A headers reset the stack -----
 if self._is_area_a_header(upper):
 stack = []
 rendered = self._render(line, AREA_A_INDENT, logical)
 output.extend(rendered)
 if len(rendered) > 1:
 wrapped += len(rendered) - 1
 if rendered[0] != line:
 normalized += 1
 continue

 # --- depth guard -----
 if len(stack) > MAX_NESTING_DEPTH:
 self.messages.append(
 PROCEDURE_INDENT_MESSAGES["skipped_depth"].format(
 limit=MAX_NESTING_DEPTH,
 line=number,
)
)
 output.append(line)
 continue

```

```

 indent, stack = self._resolve(upper, stack)

 rendered = self._render(line, indent, logical)
 output.extend(rendered)

 if len(rendered) > 1:
 wrapped += len(rendered) - 1
 if rendered[0] != line:
 normalized += 1

 if normalized:
 self.messages.append(
 PROCEDURE_INDENT_MESSAGES["normalized"].format(
 count=normalized
)
)
 if wrapped:
 self.messages.append(
 PROCEDURE_INDENT_MESSAGES["wrapped"].format(count=wrapped)
)

 result = "\n".join(output).rstrip() + "\n"

 self._dump(DUMP_OUT, result) # TEMP DEBUG

 return result

=====
Temporary instrumentation
=====
@staticmethod
def _dump(name: str, text: str) -> None: # TEMP DEBUG
 """Write one pipeline snapshot. Never raises, never blocks."""
 if not DUMP_INTERMEDIATE:
 return

 try:
 Path(name).write_text(str(text or ""), encoding="utf-8")
 except Exception: # noqa: BLE001
 pass

=====
Indent resolution
=====
def _resolve(
 self,
 upper: str,
 stack: list[_Frame],
) -> tuple[int, list[_Frame]]:
 """Return the body indent for this line and the updated stack."""
 current = stack[-1].body_indent if stack else AREA_B_INDENT

 # --- END-EXEC: close the SQL frame -----
 if END_EXEC_PATTERN.match(upper):
 frame = self._pop(stack, BLOCK_KIND_SQL)
 return (frame.own_indent if frame else current), stack

 # --- inside EXEC SQL: flat body, no depth -----
 if stack and stack[-1].kind == BLOCK_KIND_SQL:
 return stack[-1].body_indent, stack

 # --- scope terminators -----
 terminator = SCOPE_TERMINATOR_PATTERN.match(upper)
 if terminator:
 token = terminator.group("token").upper()
 kind = SCOPE_TERMINATORS.get(token, BLOCK_KIND_OTHER)
 frame = self._pop(stack, kind)
 return (frame.own_indent if frame else AREA_B_INDENT), stack

 # --- lone period closes the paragraph sentence -----
 if LONE_PERIOD_PATTERN.match(upper):
 stack.clear()
 return LONE_PERIOD_INDENT, stack

 # --- ELSE aligns with its IF -----

```



```

if ELSE_PATTERN.match(upper):
 frame = self._peek(stack, BLOCK_KIND_IF)
 return (frame.own_indent if frame else AREA_B_INDENT), stack

--- WHEN aligns against its EVALUATE -----
if WHEN_PATTERN.match(upper):
 frame = self._peek(stack, BLOCK_KIND_EVALUATE)
 return (
 frame.when_indent if frame else AREA_B_INDENT + WHEN_OFFSET
), stack

--- openers render at the current indent, then push -----
if EXEC_SQL_START_PATTERN.match(upper):
 stack.append(_Frame(BLOCK_KIND_SQL, current))
 return current, stack

if EVALUATE_START_PATTERN.match(upper):
 stack.append(_Frame(BLOCK_KIND_EVALUATE, current))
 return current, stack

if IF_START_PATTERN.match(upper):
 if not self._is_self_closing(upper):
 stack.append(_Frame(BLOCK_KIND_IF, current))
 return current, stack

if INLINE_PERFORM_PATTERN.match(upper):
 if not self._is_self_closing(upper):
 stack.append(_Frame(BLOCK_KIND_PERFORM, current))
 return current, stack

if SCOPED_OPENER_PATTERN.match(upper):
 if self._opens_scope(upper):
 stack.append(_Frame(BLOCK_KIND_OTHER, current))
 return current, stack

--- ordinary statement -----
return current, stack

=====
Stack helpers
=====
@staticmethod
def _pop(stack: list[_Frame], kind: str) -> _Frame | None:
 """Pop the nearest frame of `kind`, tolerating a mismatch."""
 for index in range(len(stack) - 1, -1, -1):
 if stack[index].kind == kind:
 frame = stack[index]
 del stack[index:]
 return frame

 if stack:
 return stack.pop()

 return None

@staticmethod
def _peek(stack: list[_Frame], kind: str) -> _Frame | None:
 for index in range(len(stack) - 1, -1, -1):
 if stack[index].kind == kind:
 return stack[index]
 return None

=====
Classification
=====
def _is_untouchable(self, line: str) -> bool:
 """Comment, page-eject, debug, continuation or blank."""
 if not str(line or "").strip():
 return True

 if self.fixed_format.is_comment_or_control_line(line):
 return True

 if self.fixed_format.is_continuation_line(line):
 return True

```

```

 return self.fixed_format.is_sequence_artifact(line)

 @staticmethod
 def _is_area_a_header(upper: str) -> bool:
 if SECTION_HEADER_PATTERN.match(upper):
 return True

 match = PARAGRAPH_HEADER_PATTERN.match(upper)
 if not match:
 return False

 name = match.group("name").upper()
 return name not in NON_PARAGRAPH_SINGLE_WORDS

 @staticmethod
 def _is_self_closing(upper: str) -> bool:
 """A one-line block that already carries its own terminator.

 Example: IF X = 1 MOVE A TO B END-IF
 """
 text = upper.rstrip(".")
 return text.endswith("END-IF") or text.endswith("END-PERFORM")

 @staticmethod
 def _opens_scope(upper: str) -> bool:
 """A READ / SEARCH / STRING that is not already terminated."""
 text = upper.rstrip(".")
 if text.endswith(("END-READ", "END-SEARCH", "END-STRING",
 "END-UNSTRING")):
 return False
 return not upper.endswith(".")

=====
Rendering
=====
def _render(
 self,
 line: str,
 indent: int,
 logical: str,
) -> list[str]:
 """Re-indent one statement, wrapping instead of truncating.

 Returns the ORIGINAL line unchanged when the new body cannot be
 represented at all, so a cosmetic pass can never destroy code.
 """
 new_body = (" " * max(indent, 0)) + logical

 if new_body == self.fixed_format.body(line).rstrip():
 return [line]

 if self.fixed_format.body_fits(new_body):
 return [self.fixed_format.replace_body(line, new_body)]

 rendered = self.fixed_format.replace_body_wrapped(line, new_body)
 return rendered if rendered else [line]

__all__ = ["ProcedureIndentNormalizer"]

=====
FILE: src\idms_db2_phase2\composers\record_materialisation__init__.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/record_materialisation/__init__.py
ACTION: REPLACE ENTIRE FILE
"""Whole-record MOVE materialisation.

record_field_plan.py plan model + builder
record_layout_generator.py DATA DIVISION rendering
record_move_generator.py PROCEDURE DIVISION rendering
record_line_utils.py fixed-format line I/O
record_move_scanner.py site discovery
record_materialisation_guard.py refusal decisions

```

```

record_block_writer.py line replacement
"""

from idms_db2_phase2.composers.record_materialisation.record_field_plan import (
 RecordFieldPlan,
 RecordMaterialisationPlan,
 RecordPlanBuilder,
)
from idms_db2_phase2.composers.record_materialisation.record_layout_generator import (
 RecordLayoutGenerator,
)
from idms_db2_phase2.composers.record_materialisation.record_move_generator import (
 RecordMoveGenerator,
)
from idms_db2_phase2.composers.record_materialisation.record_line_utils import (
 RecordLineUtils,
)
from idms_db2_phase2.composers.record_materialisation.record_move_scanner import (
 MoveSite,
 RecordMoveScanner,
 TargetSite,
)
from idms_db2_phase2.composers.record_materialisation.record_materialisation_guard import (
 RecordMaterialisationGuard,
 Refusal,
)
from idms_db2_phase2.composers.record_materialisation.record_block_writer import (
 RecordBlockWriter,
)

__all__ = [
 "MoveSite",
 "RecordBlockWriter",
 "RecordFieldPlan",
 "RecordLayoutGenerator",
 "RecordLineUtils",
 "RecordMaterialisationGuard",
 "RecordMaterialisationPlan",
 "RecordMoveGenerator",
 "RecordMoveScanner",
 "RecordPlanBuilder",
 "Refusal",
 "TargetSite",
]

```

```

=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_block_writer.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_block_writer.py
ACTION: CREATE NEW FILE
"""Rewrites the MOVE statement and expands the target field.

```

CORRECTION 1 - the expanded header lost its indent

Generated output read

```

000820 01 REC-FORM.
000830 05 F-FORM. <- column 8, Area A

```

Level 01 and 77 start in Area A; levels 02-49 must start in Area B.  
The target's own body indent is now reused for the header, the  
layout block and the remainder FILLER, so the expansion sits exactly  
where the field it replaces sat.

CORRECTION 2 - the MOVE continuation lost its indent

```

MOVE CT-RKTGDSV-479BFAS OF DCLDZBFASTV TO
CT-RK-TGDSV OF VMBFAS <- same column as MOVE

```

The generator's own leading spaces are preserved, so the statement  
reads as one MOVE.

CORRECTION 3 - the record is a PREFIX of the target

478 - 432 = 46 trailing bytes are declared as FILLER, so the record keeps its length and no following byte shifts.

"""

```
from __future__ import annotations

from idms_db2_phase2.composers.record_materialisation.record_line_utils import (
 RecordLineUtils,
)
from rules.record_materialisation_rules import (
 MOVE_INDENT,
 RECORD_GROUP_LEVEL_STEP,
)
```

```
class RecordBlockWriter:
 """Produces the replacement lines. Decides nothing."""

 def __init__(
 self,
 *,
 layout_generator,
 move_generator,
 line_utils: RecordLineUtils | None = None,
) -> None:
 self.layout_generator = layout_generator
 self.move_generator = move_generator
 self.lines = line_utils or RecordLineUtils()

 # ----- move
 def replace_move(
 self,
 lines: list[str],
 index: int,
 plan,
) -> list[str]:
 """Swap the whole-record MOVE for the field-by-field block."""
 template = lines[index]
 indent = self.lines.body_indent(template) or MOVE_INDENT

 rendered: list[str] = []
 for body in self.move_generator.generate(plan):
 stripped = body.lstrip()
 if stripped.startswith("*"):
 rendered.extend(
 self.lines.emit(template, stripped.lstrip("*"), "*")
)
 continue
 # CORRECTION 2: body keeps its own leading spaces, so a
 # continuation line stays indented under its MOVE.
 rendered.extend(self.lines.emit(template, indent + body))

 return lines[:index] + rendered + lines[index + 1:]

 # ----- target
 def expand_target(
 self,
 lines: list[str],
 index: int,
 target: str,
 plan,
 *,
 target_level: int = 0,
 declared_bytes: int = 0,
) -> list[str]:
 """Replace `05 F-FORM PIC X(478).` with the real layout."""
 template = lines[index]
 body = self.lines.logical(template)
 level = body.split()[0] if body.split() else ""

 # CORRECTION 1: reuse the target's own indent.
 indent = self.lines.body_indent(template) or MOVE_INDENT
 base_level = target_level or self.lines.level_of(body)

 rendered: list[str] = list(
```

```

 self.lines.emit(template, f"{indent}{level} {target}.")
)

 for text in self.layout_generator.generate(
 plan,
 base_level=base_level,
 base_indent=indent,
):
 rendered.extend(self.lines.emit_generated(template, text))

 # CORRECTION 3: close the record out to the declared length.
 for text in self.layout_generator.remainder_line(
 level=base_level + RECORD_GROUP_LEVEL_STEP,
 byte_count=plan.remainder_bytes(declared_bytes),
 base_indent=indent,
):
 rendered.extend(self.lines.emit(template, text))

 return lines[:index] + rendered + lines[index + 1:]

__all__ = ["RecordBlockWriter"]

=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_entry_renderer.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_entry_renderer.py
ACTION: CREATE NEW FILE
"""Renders a group entry and an elementary entry.

Templates plus geometry. Decides nothing about the record as a whole -
it is handed a level, a name, a picture and an indent, and returns one
finished body line.

 06 VMBFAS.
 07 CT-RK-TGDSV PIC X(4).
 07 FILLER PIC X(2).
"""

from __future__ import annotations

from idms_db2_phase2.composers.record_materialisation.record_layout_geometry import (
 RecordLayoutGeometry,
)
from rules.record_materialisation_rules import (
 ELEMENTARY_ITEM_TEMPLATE,
 GROUP_ITEM_TEMPLATE,
)

class RecordEntryRenderer:
 """One data description entry, group or elementary."""

 def __init__(self, geometry: RecordLayoutGeometry | None = None) -> None:
 self.geometry = geometry or RecordLayoutGeometry()

 # ----- group
 def group(self, *, level: str, name: str, indent: str) -> str:
 """06 VMBFAS.`"""
 body = GROUP_ITEM_TEMPLATE.format(
 level=level,
 name=self.clean_name(name),
)
 return self.geometry.cap(
 self.geometry.terminate(indent + body)
)

 # ----- elementary
 def elementary(
 self,
 *,
 level: str,
 name: str,

```

```

 picture: str,
 indent: str,
) -> str:
 """"`07 CT-RK-TGDSV          PIC X(4).`

A blank picture means the sheet declared a group, so the entry
falls back to the group shape rather than emitting a bare name.
""""

 clean_picture = str(picture or "").strip()
 clean_name = self.clean_name(name)

 if not clean_picture:
 return self.group(level=level, name=clean_name, indent=indent)

 width = self.geometry.name_width(
 indent=indent,
 level=level,
 name=clean_name,
 picture=clean_picture,
)
 body = ELEMENTARY_ITEM_TEMPLATE.format(
 level=level,
 name=clean_name,
 width=width,
 picture=clean_picture,
)
 return self.geometry.cap(
 self.geometry.terminate(indent + body)
)

----- helpers
@staticmethod
def picture_of(item) -> str:
 """"'PIC S9(13)V99' + 'COMP-3' -> 'PIC S9(13)V99 COMP-3'."""
 picture = str(getattr(item, "picture", "") or "")
 usage = str(getattr(item, "usage", "") or "")
 if not usage:
 return picture.strip()
 return f"{picture} {usage}".strip()

@staticmethod
def clean_name(name: str) -> str:
 return str(name or "").strip().upper()

```

```
__all__ = ["RecordEntryRenderer"]
```

```

=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_field.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_field.py
ACTION: CREATE NEW FILE
""""One COBOL data description entry of a materialised record.

```

Shape and policy only. This class never reads a Sheet Mapping row and never talks to a resolver - it answers questions ABOUT a field that has already been read.

ROW SHAPES (Sheet Mapping is a hierarchy, not a flat record)

1 elementary, mapped	PIC + column	declare + move
2 group, mapped (dates)	GROUP + column	declare + move
3 group, structural	GROUP, no column	declare only
4 subordinate of a group	PIC, no column	declare only
5 FILLER	PIC, FILLER cell	declare only
6 REDEFINES	PIC, no column	declare only
7 DB2-only audit/key row	no COBOL zone	neither

GOVERNING PRINCIPLE

No DB2 column -> COBOL filler or the breakdown of a mapped parent. Declare it, skip it, never block.

Column DCLGEN does

```
not know -> a genuine metadata gap. Refuse.
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
from rules.record_materialisation_rules import (
 DATE_DB2_TYPE_PREFIX,
 FILLER_FIELD_NAMES,
 NON_COLUMN_SENTINELS,
 NON_DATE_DB2_TYPE_PREFIXES,
)
```

```
DEFAULT_LEVEL = "05"
COMP_3_SYMBOL = "COMP-3"
```

```
@dataclass
```

```
class RecordFieldPlan:
```

```
 """One COBOL data description entry of the materialised record."""
```

```
 level: str = DEFAULT_LEVEL
 name: str = ""
 picture: str = ""
 usage: str = ""
 redefines: str = ""
 db2_column: str = ""
 db2_type: str = ""
 host_reference: str = ""
 declared_bytes: int = 0
```

```
----- shape
```

```
@property
```

```
def is_group(self) -> bool:
 return not self.picture
```

```
@property
```

```
def is_filler(self) -> bool:
 """Shape 5. Occupies bytes, carries no data."""
 return str(self.name or "").strip().upper() in FILLER_FIELD_NAMES
```

```
@property
```

```
def is_redefinition(self) -> bool:
 return bool(self.redefines)
```

```
----- column
```

```
@property
```

```
def has_db2_column(self) -> bool:
 """CORRECTION C.
```

```
 The literal word FILLER, a dash, N/A and an empty cell are all
 sentinels meaning 'no column'. FILLER was previously looked up
 in DCLGEN, missed, and blocked the entire record.
 """
```

```
 candidate = str(self.db2_column or "").strip().upper()
 return candidate not in NON_COLUMN_SENTINELS
```

```
@property
```

```
def is_date(self) -> bool:
 compact = str(self.db2_type or "").upper().replace(" ", "")
 if compact.startswith(NON_DATE_DB2_TYPE_PREFIXES):
 return False
 return compact.startswith(DATE_DB2_TYPE_PREFIX)
```

```
----- policy
```

```
@property
```

```
def requires_host(self) -> bool:
 """CORRECTION B.
```

```
 Only a REAL DB2 column obliges DCLGEN to supply a host variable.
 A subordinate of a mapped group, a FILLER and a REDEFINES carry
 no column, are never looked up, and can never be reported
 missing.
 """
```

```
 if self.is_filler or self.is_redefinition:
```

```

 return False
 return self.has_db2_column

@property
def is_missing_host(self) -> bool:
 return self.requires_host and not self.host_reference

@property
def is_declarable(self) -> bool:
 """Every shape except 7 occupies storage and must be declared."""
 return bool(self.name)

@property
def is_movable(self) -> bool:
 """CORRECTION D.

```

WAS: not is\_group and host\_reference and not redefines  
-> every mapped date GROUP was excluded and the most important  
moves were silently dropped.

A GROUP carrying a DB2 column IS movable; that is exactly how  
the manual reference populates every date:

```

 MOVE HULP-DA-VA-FORM-AS TO DA-VA-FORM-AS OF VMBFAS
 """

```

```

 if self.is_redefinition:
 return False
 if self.is_filler:
 return False
 if not self.has_db2_column:
 return False
 return bool(self.host_reference)

```

```

@property
def occupies_storage(self) -> bool:
 """Bytes are counted once, on elementary items only.

 A group's size is the sum of its children, and a REDEFINES
 re-uses storage already counted, so neither may be added again.
 """
 return (
 not self.is_group
 and not self.is_redefinition
 and self.declared_bytes > 0
)

```

```

__all__ = ["COMP_3_SYMBOL", "DEFAULT_LEVEL", "RecordFieldPlan"]

```

```

=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_field_plan.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_field_plan.py
ACTION: REPLACE ENTIRE FILE
"""Back-compatible import point for the record plan model.

```

The model was split so no file exceeds a reviewable size:

record_field.py	RecordFieldPlan	shape + policy
record_plan.py	RecordMaterialisationPlan	views + sizes
record_row_reader.py	Sheet Mapping row ->	field
record_host_resolver.py	table / group / host	lookups
record_plan_builder.py	orchestration	

Several modules already import the three public names from here  
(record\_layout\_generator, record\_move\_generator, the package \_\_init\_\_),  
so this module keeps working as their import point. Nothing else had to  
change.

```

from __future__ import annotations

```

```

from idms_db2_phase2.composers.record_materialisation.record_field import (

```



```

COMP_3_SYMBOL,
DEFAULT_LEVEL,
RecordFieldPlan,
)
from idms_db2_phase2.composers.record_materialisation.record_host_resolver import (
 RecordHostResolver,
)
from idms_db2_phase2.composers.record_materialisation.record_plan import (
 RecordMaterialisationPlan,
)
from idms_db2_phase2.composers.record_materialisation.record_plan_builder import (
 RecordPlanBuilder,
)
from idms_db2_phase2.composers.record_materialisation.record_row_reader import (
 RecordRowReader,
)

```

```

__all__ = [
 "COMP_3_SYMBOL",
 "DEFAULT_LEVEL",
 "RecordFieldPlan",
 "RecordHostResolver",
 "RecordMaterialisationPlan",
 "RecordPlanBuilder",
 "RecordRowReader",
]

```

```

=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_host_resolver.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_host_resolver.py
ACTION: CREATE NEW FILE
"""Adapter over the table and host-variable resolvers.

```

```

DCLGEN is the authority for host variable spelling. This class does not
decide whether a lookup SHOULD happen - it only performs one and
normalises the answer. Every method is total: a resolver that raises
yields an empty string, so one bad column can never abort the pass.
"""

```

```

from __future__ import annotations

HOST_PREFIX = ":"
GROUP_SEPARATOR = "."
HOST_OF_GROUP_TEMPLATE = "{host} OF {group}"

```

```

class RecordHostResolver:
 """Resolves record -> table -> DCLGEN group -> host reference."""

 def __init__(
 self,
 *,
 table_name_resolver,
 host_variable_resolver,
) -> None:
 self.table_name_resolver = table_name_resolver
 self.host_variable_resolver = host_variable_resolver

 # ----- table
 def table_for(self, record: str) -> str:
 if not record:
 return ""
 try:
 return str(
 self.table_name_resolver.table_for_record(record) or ""
)
 except Exception: # noqa: BLE001
 return ""

 # ----- group
 def group_for(self, table: str) -> str:
 if not table:

```

```

 return ""
 try:
 return str(
 self.host_variable_resolver.group_for_table(table) or ""
)
 except Exception: # noqa: BLE001
 return ""

----- host
def host_for(self, table: str, column: str) -> str:
 """'DA-CRFMAS-479BFAS OF DCLDZBFASTV', or '' when unresolved.

 DCLGEN resolvers report a host either as ':GROUP.HOST' or as a
 bare name. Both are normalised to the qualified COBOL form the
 generated MOVE needs.
 """
 if not table or not column:
 return ""

 try:
 reference = self.host_variable_resolver.host_reference_for_column(
 table_name=table,
 column_name=column,
)
 except Exception: # noqa: BLE001
 return ""

 return self.normalize(reference)

----- helpers
@staticmethod
def normalize(reference) -> str:
 text = str(reference or "").strip().lstrip(HOST_PREFIX)
 if not text:
 return ""

 if GROUP_SEPARATOR in text:
 group, _sep, host = text.partition(GROUP_SEPARATOR)
 return HOST_OF_GROUP_TEMPLATE.format(
 host=host.strip().upper(),
 group=group.strip().upper(),
)

 return text.upper()

__all__ = ["RecordHostResolver"]

=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_layout_generator.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_layout_generator.py
ACTION: REPLACE ENTIRE FILE
"""Renders the materialised record as COBOL data description entries.

Orchestration only. Every column rule lives in the geometry class and
every template in one of the two renderers:

 record_layout_geometry.py columns, indents, widths, levels
 record_entry_renderer.py group + elementary entries
 record_redefines_renderer.py REDEFINES, inline or wrapped

Public API is unchanged, so record_block_writer.py needs no edit.

CORRECTION 3 - the record is a PREFIX of the target field

05 F-FORM PIC X(478). <- the field being replaced
06 VMBFAS. <- 432 bytes
07 ...
06 FILLER PIC X(46). <- the remainder, declared here

Without the remainder FILLER the record silently shortens and every
following field shifts left.

```

```

"""

from __future__ import annotations

from idms_db2_phase2.composers.record_materialisation.record_entry_renderer import (
 RecordEntryRenderer,
)
from idms_db2_phase2.composers.record_materialisation.record_layout_geometry import (
 RecordLayoutGeometry,
)
from idms_db2_phase2.composers.record_materialisation.record_plan import (
 RecordMaterialisationPlan,
)
from idms_db2_phase2.composers.record_materialisation.record_redefines_renderer import (
 RecordRedefinesRenderer,
)
from rules.record_materialisation_rules import (
 EMIT_REMAINDER_FILLER,
 LAYOUT_MARKER_TEMPLATE,
 RECORD_GROUP_LEVEL_STEP,
 REMAINDER_FILLER_NAME,
 REMAINDER_FILLER_PICTURE_TEMPLATE,
)

COMMENT_INDICATOR = "*"
RECORD_GROUP_DEPTH = 1
FIELD_DEPTH_OFFSET = 2

```

```

class RecordLayoutGenerator:
 """Builds the COBOL body lines for a materialised record."""

 def __init__(
 self,
 *,
 geometry: RecordLayoutGeometry | None = None,
 entries: RecordEntryRenderer | None = None,
 redefines: RecordRedefinesRenderer | None = None,
) -> None:
 self.geometry = geometry or RecordLayoutGeometry()
 self.entries = entries or RecordEntryRenderer(self.geometry)
 self.redefines = redefines or RecordRedefinesRenderer(self.geometry)

 # =====
 # Public
 # =====
 def generate(
 self,
 plan: RecordMaterialisationPlan,
 *,
 base_level: int = 0,
 base_indent: str = "",
) -> list[str]:
 """Body lines only. The caller applies the fixed-format frame.

 base_level - level of the target field being expanded.
 base_indent - body indent of the target field, so the block sits
 under it instead of jumping to Area A.
 """
 fields = plan.declarable_fields
 if not fields:
 return []

 root = self.geometry.root(base_indent)
 group_level = int(base_level or 0) + RECORD_GROUP_LEVEL_STEP

 lines: list[str] = [
 COMMENT_INDICATOR
 + LAYOUT_MARKER_TEMPLATE.format(record=plan.record_name),
 self.group_line(
 level=group_level,
 name=plan.record_name,
 base_indent=root,
 depth=RECORD_GROUP_DEPTH,
),
]

```

```

shift = (group_level + 1) - plan.min_level
stack: list[str] = []

for item in fields:
 level = self.geometry.shift(item.level, shift)
 depth = self.geometry.depth(stack, level) + FIELD_DEPTH_OFFSET
 # An entry may render as more than one physical line.
 lines.extend(self._entry(item, level, root, depth))

return lines

def remainder_line(
 self,
 *,
 level: int,
 byte_count: int,
 base_indent: str = "",
) -> list[str]:
 """CORRECTION 3. FILLER for the unused tail of the target field.

 Emitted as a SIBLING of the record group, never inside it.
 """
 if not EMIT_REMAINDER_FILLER:
 return []
 if not byte_count or int(byte_count) <= 0:
 return []

 root = self.geometry.root(base_indent)
 picture = REMAINDER_FILLER_PICTURE_TEMPLATE.format(
 bytes=int(byte_count)
)
 return [
 self.entries.elementary(
 level=self.geometry.clamp(level),
 name=REMAINDER_FILLER_NAME,
 picture=picture,
 indent=self.geometry.indent_text(root, RECORD_GROUP_DEPTH),
)
]

def group_line(
 self,
 *,
 level: int,
 name: str,
 base_indent: str = "",
 depth: int = 0,
) -> str:
 root = self.geometry.root(base_indent)
 return self.entries.group(
 level=self.geometry.clamp(level),
 name=name,
 indent=self.geometry.indent_text(root, depth),
)

=====
One entry
=====
def _entry(self, item, level: str, root: str, depth: int) -> list[str]:
 indent = self.geometry.indent_text(root, depth)
 picture = self.entries.picture_of(item)

 if item.is_redefinition:
 return self.redefines.render(
 level=level,
 name=item.name,
 base=item.redefines,
 picture=picture,
 indent=indent,
)

 if item.is_group:
 return [
 self.entries.group(
 level=level,

```

```

 name=item.name,
 indent=indent,
)
]

return [
 self.entries.elementary(
 level=level,
 name=item.name,
 picture=picture,
 indent=indent,
)
]

__all__ = ["RecordLayoutGenerator"]

=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_layout_geometry.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_layout_geometry.py
ACTION: CREATE NEW FILE
"""Fixed-format geometry for generated data description entries.

Pure arithmetic. No templates, no COBOL text, no plan knowledge - every
method takes strings and numbers and returns strings and numbers, so the
column rules can be reasoned about and tested in one place.

THE RULES THIS CLASS ENFORCES

* levels 02-49 start in Area B, column 12 or later,
* a body never reaches column 72,
* a name is padded before its PICTURE, and the padding shrinks rather
 than letting the line overflow,
* an entry carries EXACTLY ONE terminating period,
* nesting indent is capped, so no depth can push a picture off the end.

CORRECTION 2 - nesting overflowed the body

A fixed 4-space step put a level-11 entry at column 29 and pushed
'PIC S9(3) COMP-3.' past column 72, so the emitter degraded to an
unaligned fallback. The step is 2 spaces and the depth is capped.

CORRECTION 4 - double period on every group entry

GROUP_ITEM_TEMPLATE ends in a period, ELEMENTARY_ITEM_TEMPLATE does
not. Appending one unconditionally produced '06 VMBFAS..', which
the compiler rejects. terminate() never assumes what a template did.
"""

from __future__ import annotations

from rules.record_materialisation_rules import (
 LAYOUT_BASE_INDENT,
 LAYOUT_BODY_WIDTH,
 LAYOUT_LEVEL_STEP,
 LAYOUT_MAX_INDENT_DEPTH,
 LAYOUT_MIN_NAME_GAP,
 LAYOUT_NAME_WIDTH,
 MAX_LEVEL,
 MIN_LEVEL,
)

PERIOD = "."

class RecordLayoutGeometry:
 """Column arithmetic for one data description entry."""

 BODY_WIDTH = LAYOUT_BODY_WIDTH
 PERIOD = PERIOD

 # ----- indent

```

```

@staticmethod
def root(base_indent: str = "") -> str:
 """The indent of the field being expanded, or Area B."""
 return str(base_indent) if base_indent else LAYOUT_BASE_INDENT

@staticmethod
def indent_text(root: str, depth: int) -> str:
 """CORRECTION 2. Nesting indent is capped, never unbounded."""
 steps = max(0, min(int(depth), LAYOUT_MAX_INDENT_DEPTH))
 return str(root) + (LAYOUT_LEVEL_STEP * steps)

----- width
@classmethod
def name_width(
 cls,
 *,
 indent: str,
 level: str,
 name: str,
 picture: str,
) -> int:
 """Padding that keeps the entry inside columns 8-72.

 Shrinks the gap before the PICTURE rather than letting the line
 overflow, so alignment degrades gracefully and never truncates.
 """
 prefix = len(indent) + len(level) + 2
 available = cls.BODY_WIDTH - prefix - len(picture) - len(PERIOD)
 minimum = len(name) + LAYOUT_MIN_NAME_GAP

 if available < minimum:
 return minimum
 return max(minimum, min(LAYOUT_NAME_WIDTH, available))

@classmethod
def gap_before(cls, head: str, picture: str) -> int:
 """Spaces between a head fragment and its PICTURE. Never zero."""
 gap = cls.BODY_WIDTH - len(head) - len(picture) - len(PERIOD)
 return max(LAYOUT_MIN_NAME_GAP, gap)

----- termination
@staticmethod
def terminate(body: str) -> str:
 """CORRECTION 4. Exactly one period, whatever the template did."""
 text = str(body or "").rstrip()
 if not text:
 return text
 return text.rstrip(PERIOD).rstrip() + PERIOD

----- cap
@classmethod
def cap(cls, body: str) -> str:
 """Last-resort guard against a body wider than columns 8-72."""
 text = str(body or "").rstrip()
 if len(text) <= cls.BODY_WIDTH:
 return text

 indent = text[: len(text) - len(text.lstrip(" "))]
 parts = text.strip().split()
 collapsed = indent + " ".join(parts)
 if len(collapsed) <= cls.BODY_WIDTH:
 return collapsed

 return " ".join(parts)[: cls.BODY_WIDTH]

----- levels
@classmethod
def shift(cls, level: str, amount: int) -> str:
 """Rebase a Sheet Mapping level under the record group."""
 try:
 return cls.clamp(int(str(level).strip()) + int(amount))
 except (TypeError, ValueError):
 return str(level)

@staticmethod
def clamp(value: int) -> str:

```

```

 bounded = max(MIN_LEVEL, min(MAX_LEVEL, int(value)))
 return f"{bounded:02d}"

 @staticmethod
 def depth(stack: list[str], level: str) -> int:
 """Nesting depth from the level stack, not a fixed table.

 Mutates `stack`, which is how a single left-to-right pass can
 track nesting without looking ahead.
 """
 while stack and stack[-1] >= level:
 stack.pop()
 found = len(stack)
 stack.append(level)
 return found

__all__ = ["PERIOD", "RecordLayoutGeometry"]

=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_line_utils.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_line_utils.py
ACTION: CREATE NEW FILE
"""Fixed-format line helpers for record materialisation.

Owns no regex and no business rules. Every method is total: a line the
service cannot parse is returned as-is rather than raising, so a single
odd line can never abort the pass.
"""

from __future__ import annotations

from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)

COMMENT_INDICATOR = "*"
BLANK_INDICATOR = " "

class RecordLineUtils:
 """Reads and writes fixed-format COBOL lines."""

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()

 # ----- read
 def logical(self, line: str) -> str:
 """Columns 8-72, stripped."""
 try:
 return str(self.fixed_format.logical(line) or "").strip()
 except Exception: # noqa: BLE001
 return str(line or "").strip()

 def body_indent(self, line: str) -> str:
 """Leading spaces of the body, as a string."""
 try:
 return str(self.fixed_format.body_indent(line) or "")
 except Exception: # noqa: BLE001
 return ""

 def is_skippable(self, line: str) -> bool:
 """Comment / page-eject / debug / continuation, read from col 7."""
 try:
 return bool(self.fixed_format.is_comment_or_control_line(line))
 except Exception: # noqa: BLE001
 return str(line or "").lstrip().startswith(("*", "/"))

 @staticmethod

```

```

def level_of(body: str) -> int:
 """Level number of a data description entry, or 0."""
 parts = str(body or "").strip().split()
 if parts and parts[0].isdigit():
 return int(parts[0])
 return 0

----- write
def emit(
 self,
 template: str,
 body: str,
 indicator: str = BLANK_INDICATOR,
) -> list[str]:
 """Render one logical body as one or more fixed-format lines.

 The template supplies the sequence areas, so a generated line
 carries a real sequence number instead of a placeholder.
 """
 text = str(body or "").rstrip()
 if not text:
 return []

 try:
 left, template_indicator, _old, right = self.fixed_format.split(
 template
)
 except Exception: # noqa: BLE001
 return [text]

 marker = (
 str(indicator or "")
 or str(template_indicator or "")
 or BLANK_INDICATOR
)[:1]

 emitter = getattr(self.fixed_format, "emit", None)
 if callable(emitter):
 try:
 return list(emitter(left, marker, text, right))
 except Exception: # noqa: BLE001
 pass

 try:
 built = self.fixed_format.build_or_none(
 left, marker, text, right
)
 except Exception: # noqa: BLE001
 built = None

 return [built] if built else [text]

def emit_generated(self, template: str, text: str) -> list[str]:
 """Emit a generated body, routing a leading '*' to column 7."""
 stripped = str(text or "").lstrip()
 if stripped.startswith(COMMENT_INDICATOR):
 return self.emit(
 template,
 stripped.lstrip(COMMENT_INDICATOR),
 COMMENT_INDICATOR,
)
 return self.emit(template, text, BLANK_INDICATOR)

```

```
__all__ = ["RecordLineUtils"]
```

```

=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_materialisation_guard.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_materialisation_guard.py
ACTION: CREATE NEW FILE
"""Decides whether one record may be materialised.

```



Every gate REFUSES rather than guesses. A refusal leaves working COBOL and a diagnostic; a guess corrupts a fixed-length output record.

CORRECTION A - the length rule was equality

The IDMS record is a PREFIX of the output field, never equal to it:

```
05 F-FORM PIC X(478).
06 VMBFAS. <- 432 bytes
```

$|432 - 478| = 46 > 0$  guaranteed refusal on every run. The layout may be SHORTER; only LONGER is fatal, because that shifts every byte after it.

CORRECTION B + C - unmapped subordinates and FILLER blocked

A field is missing ONLY when it carries a real DB2 column that DCLGEN does not define. No column at all means COBOL filler or the breakdown of a mapped parent.

"""

```
from __future__ import annotations
```

```
from dataclasses import dataclass, field
```

```
from rules.record_materialisation_rules import (
 ALLOW_LAYOUT_SHORTER_THAN_TARGET,
 ENFORCE_LENGTH_MATCH,
 LENGTH_TOLERANCE,
)
```

```
MISSING_HOST_SAMPLE = 5
```

```
@dataclass(frozen=True)
```

```
class Refusal:
```

```
 """A named diagnostic plus its template values."""
```

```
 key: str = ""
```

```
 values: dict = field(default_factory=dict)
```

```
class RecordMaterialisationGuard:
```

```
 """All refusal decisions for one record, in evaluation order."""
```

```
 def __init__(self, plan_builder) -> None:
 self.plan_builder = plan_builder
```

```
 # ----- plan
```

```
 def build_plan(self, record: str):
 """(plan, refusal). plan is None when the record is refused."""
 if not self.plan_builder.looks_like_record(record):
 return None, Refusal(
 "skipped_not_a_record", {"record": record}
)
```

```
 plan = self.plan_builder.build(record)
```

```
 if not plan.fields:
 return None, Refusal("skipped_no_mapping", {"record": record})
```

```
 if not plan.group_name:
 return None, Refusal("skipped_no_group", {"record": record})
```

```
 if plan.missing_hosts:
 return None, Refusal(
 "skipped_incomplete",
 {
 "record": record,
 "missing": len(plan.missing_hosts),
 "names": ", ".join(
 plan.missing_hosts[:MISSING_HOST_SAMPLE]
),
 },
)
```

```

 return plan, None

----- length
@staticmethod
def check_length(plan, target: str, declared_bytes: int):
 """Refusal when the layout is LONGER than the target field."""
 if not ENFORCE_LENGTH_MATCH or not declared_bytes:
 return None

 too_long = plan.overflows(declared_bytes, LENGTH_TOLERANCE)
 too_short = (
 not ALLOW_LAYOUT_SHORTER_THAN_TARGET
 and plan.remainder_bytes(declared_bytes) > LENGTH_TOLERANCE
)

 if not (too_long or too_short):
 return None

 return Refusal(
 "skipped_too_long",
 {
 "record": plan.record_name,
 "actual": plan.total_bytes,
 "target": target,
 "expected": declared_bytes,
 },
)

----- advice
@staticmethod
def check_movable(plan):
 """Not a refusal. A visible warning that nothing is populated."""
 if plan.movable_fields:
 return None
 return Refusal(
 "declared_only",
 {
 "record": plan.record_name,
 "fields": len(plan.declarable_fields),
 },
)

__all__ = ["Refusal", "RecordMaterialisationGuard"]

=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_move_generator.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_move_generator.py
ACTION: REPLACE ENTIRE FILE
"""Renders the field-by-field population of a materialised record.

A non-date column becomes a single MOVE. A DB2 DATE column becomes the
manual reference conversion block, using the shared date staging fields
that Db2DateComparisonComposer already declares.

CORRECTION - the continuation line lost its indent

 MOVE CT-RKTGDSV-479BFAS OF DCLDZBFASTV TO
 CT-RK-TGDSV OF VMBFAS <- same column as the MOVE

The indent used to live in MOVE_TARGET_TEMPLATE's leading spaces,
which an unapplied rules override or a later indent pass can flatten.
It now lives in CODE, applied here, so it cannot be lost.

A MOVE short enough to fit on one line is emitted on one line, which
removes the continuation problem entirely for most fields.
"""

from __future__ import annotations

from idms_db2_phase2.composers.record_materialisation.record_field_plan import (
 RecordMaterialisationPlan,

```

```

)
from rules.record_materialisation_rules import (
 DATE_CONVERSION_LINE_TEMPLATES,
 DATE_HIGH_NUMERIC_LITERAL,
 DATE_HIGH_VALUE_LITERAL,
 DATE_LOW_VALUE_LITERAL,
 EMIT_DATE_CONVERSION,
 MOVE_CONTINUATION_INDENT,
 MOVE_INLINE_LIMIT,
 MOVE_MARKER_TEMPLATE,
 MOVE_TARGET_TEMPLATE,
 MOVE_TEMPLATE,
)

class RecordMoveGenerator:
 """Builds the MOVE block that populates the materialised record."""

 def generate(self, plan: RecordMaterialisationPlan) -> list[str]:
 """Body lines only. The caller applies indent and the frame."""
 fields = plan.movable_fields
 if not fields:
 return []

 lines: list[str] = [
 MOVE_MARKER_TEMPLATE.format(
 record=plan.record_name,
 group=plan.group_name,
 count=len(fields),
)
]

 for item in fields:
 if EMIT_DATE_CONVERSION and item.is_date:
 lines.extend(self._date_block(plan.record_name, item))
 else:
 lines.extend(self._simple_move(plan.record_name, item))

 return lines

 # ----- moves
 @staticmethod
 def _simple_move(record: str, item) -> list[str]:
 """One line when it fits; head + indented continuation otherwise."""
 source = MOVE_TEMPLATE.format(host=item.host_reference)
 target = MOVE_TARGET_TEMPLATE.format(
 field=item.name,
 record=record,
).strip()

 single = f"{source} {target}"
 if len(single) <= MOVE_INLINE_LIMIT:
 return [single]

 return [source, MOVE_CONTINUATION_INDENT + target]

 @staticmethod
 def _date_block(record: str, item) -> list[str]:
 return [
 template.format(
 host=item.host_reference,
 field=item.name,
 record=record,
 low_value=DATE_LOW_VALUE_LITERAL,
 high_value=DATE_HIGH_VALUE_LITERAL,
 high_numeric=DATE_HIGH_NUMERIC_LITERAL,
)
 for template in DATE_CONVERSION_LINE_TEMPLATES
]

__all__ = ["RecordMoveGenerator"]

```

```
=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_move_scanner.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_move_scanner.py
ACTION: REPLACE ENTIRE FILE
"""Locates whole-record MOVE statements and their target fields.
```

```
Read-only. Finds sites; rewrites nothing. Sizing lives in
record_picture_sizer.py and the value objects in record_sites.py, so
this class is nothing but two scans.
```

```
CORRECTION 1 - first-match-wins
```

```
The original scanner returned the FIRST whole-record MOVE and the
composer abandoned the whole pass on any refusal. A LINKAGE date
field matched first, had no Sheet Mapping rows, and the real record
MOVE was never examined - it then reached the structural safety pass
and was commented out.
```

```
find_move now takes the set of pairs already visited and returns the
next unvisited one, so the caller can iterate and a refusal stays
local to one record.
```

```
"""
```

```
from __future__ import annotations
```

```
import re
```

```
from idms_db2_phase2.composers.record_materialisation.record_line_utils import (
 RecordLineUtils,
)
from idms_db2_phase2.composers.record_materialisation.record_picture_sizer import (
 RecordPictureSizer,
)
from idms_db2_phase2.composers.record_materialisation.record_sites import (
 MoveSite,
 TargetSite,
)
from patterns.record_materialisation_patterns import (
 PROCEDURE_DIVISION_PATTERN,
 TARGET_FIELD_PATTERN_TEMPLATE,
 WHOLE_RECORD_MOVE_PATTERN,
)
```

```
class RecordMoveScanner:
```

```
 """Finds whole-record MOVEs and the fields they write into."""
```

```
 def __init__(
 self,
 line_utils: RecordLineUtils | None = None,
 sizer: RecordPictureSizer | None = None,
) -> None:
 self.lines = line_utils or RecordLineUtils()
 self.sizer = sizer or RecordPictureSizer()
```

```
 # ----- move
```

```
 def find_move(
 self,
 lines: list[str],
 handled: set[tuple[str, str]],
) -> MoveSite | None:
 """Next whole-record MOVE not yet visited. None when exhausted."""
 in_procedure = False
```

```
 for index, line in enumerate(lines):
 if self.lines.is_skippable(line):
 continue
```

```
 body = self.lines.logical(line)
 if not body:
 continue
```

```
 if PROCEDURE_DIVISION_PATTERN.match(body):
```

```

 in_procedure = True
 continue

 if not in_procedure:
 continue

 match = WHOLE_RECORD_MOVE_PATTERN.match(body)
 if not match:
 continue

 site = MoveSite(
 index=index,
 record=match.group("record").upper(),
 target=match.group("target").upper(),
)
 if site.key in handled:
 continue

 return site

return None

----- target
def find_target(self, lines: list[str], target: str) -> TargetSite:
 """The DATA DIVISION entry for `target`, or an empty TargetSite."""
 pattern = self._target_pattern(target)

 for index, line in enumerate(lines):
 if self.lines.is_skippable(line):
 continue

 body = self.lines.logical(line)
 if not body:
 continue

 # The DATA DIVISION ends here; stop rather than match a
 # PROCEDURE DIVISION reference to the same name.
 if PROCEDURE_DIVISION_PATTERN.match(body):
 break

 match = pattern.match(body)
 if not match:
 continue

 return TargetSite(
 index=index,
 declared_bytes=self.sizer.declared_bytes(body, match),
 level=self.lines.level_of(body),
)

 return TargetSite()

----- helpers
@staticmethod
def _target_pattern(target: str):
 return re.compile(
 TARGET_FIELD_PATTERN_TEMPLATE.format(name=re.escape(target)),
 flags=re.IGNORECASE,
)

Preserved for callers that still import it from this module.
@classmethod
def picture_bytes(cls, picture: str) -> int:
 return RecordPictureSizer.size_of(picture, picture)

__all__ = ["MoveSite", "RecordMoveScanner", "RecordPictureSizer", "TargetSite"]

```

```

=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_picture_sizer.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_picture_sizer.py
ACTION: REPLACE ENTIRE FILE

```

""Sizes a COBOL PICTURE clause in stored bytes.

Pure measurement. Given a data description line it answers one question: how many bytes does this entry occupy? Getting that wrong silently shifts every field of a fixed-length record.

WHY THIS MODULE OWNS NO REGEX

-----  
The project rule puts regex in patterns/. This module is the documented exception, and the exception is earned:

```
PIC 9999 -> 4 correct
PIC 9(7) -> 1 wrong
PIC X(478) -> 1 wrong
```

PICTURE\_SYMBOL\_PATTERN captures the symbol and exposes no repeat group, so every clause written with (n) collapsed to the count of literal symbols. F-FORM measured 1 byte, the length guard refused a 432-byte VMBFAS layout as "longer than the 1 byte F-FORM declares", and roughly 400 lines of generated output were discarded on every run.

Three separate attempts to fix that through the shared pattern module produced identical wrong answers, because a pattern this module cannot see is a dependency this module cannot verify. Counting repeat groups is character arithmetic, not pattern matching, so it is done here, in full view, where a unit test pins every branch.

No regex. No imports. No hidden behaviour.

""

```
from __future__ import annotations
```

```
Picture symbols that occupy a stored character position.
S and V are sign and implied decimal point: neither is stored.
COUNTED_SYMBOLS = frozenset("X9AZ")
```

```
Packed decimal halves the stored length.
COMP_3_SYMBOLS = ("COMPUTATIONAL-3", "COMP-3", "PACKED-DECIMAL")
COMP_3_SYMBOL = "COMP-3"
```

```
The PICTURE keyword, longest form first.
PIC_KEYWORDS = ("PICTURE", "PIC")
```

```
Clauses that may follow a PICTURE on the same entry. Reading past one
of these counts the A of VALUE and the A of USERABEN in
```

```
#
77 USERABEN PIC X(8) VALUE 'USERABEN'.
#
```

```
and reports 10 bytes instead of 8.
```

```
STOP_WORDS = frozenset({
 "BLANK",
 "BINARY",
 "COMP",
 "COMP-1",
 "COMP-2",
 "COMP-3",
 "COMP-4",
 "COMPUTATIONAL",
 "COMPUTATIONAL-1",
 "COMPUTATIONAL-2",
 "COMPUTATIONAL-3",
 "COMPUTATIONAL-4",
 "DISPLAY",
 "INDEXED",
 "JUST",
 "JUSTIFIED",
 "OCCURS",
 "PACKED-DECIMAL",
 "REDEFINES",
 "SIGN",
 "SYNC",
 "SYNCHRONIZED",
 "USAGE",
 "VALUE",
 "VALUES",
```

```

})

Characters that may appear inside a COBOL data name. A keyword is only
a keyword when it is not part of a longer name, so WS-PIC-CODE never
looks like a PICTURE clause.
NAME_CHARACTERS = frozenset(
 "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789-_"
)

PICTURE_GROUP = "picture"
OPEN_PAREN = "("
CLOSE_PAREN = ")"
PERIOD = "."

```

```

class RecordPictureSizer:
 """Measures a PICTURE clause, never under-reporting."""

 #
 # Public entry points
 #
 @classmethod
 def declared_bytes(cls, body: str, match=None) -> int:
 """Stored byte count of one data description entry.

 `body` the full logical line, e.g. "05 F-FORM PIC X(478)."
 `match` the caller's regex match. Its 'picture' group is ONE
 candidate, never the answer: it has been observed to
 capture 'X' where the line says 'X(478)'.

 Every candidate is measured and the LARGEST wins. A PICTURE can
 never declare fewer positions than its most complete reading, so
 the maximum can only correct a truncated read.
 """
 text = str(body or "")
 packed = cls.is_comp_3(text)

 sizes = [
 cls.size_of(candidate, COMP_3_SYMBOL if packed else "")
 for candidate in cls.candidates(text, match)
]

 return max(sizes) if sizes else 0

 @classmethod
 def size_of(cls, picture: str, usage: str = "") -> int:
 """Character positions of a PICTURE, honouring repeat counts.

 X(10) -> 10
 XXX -> 3
 9(7) -> 7
 9999 -> 4
 X(4) -> 4 whitespace inside the parentheses
 S9(13)V99 -> 15 S and V are not stored
 S9(11) COMP-3 -> 6 (11 // 2) + 1
 """
 text = str(picture or "").upper()

 if not text:
 return 0

 digits = 0
 index = 0
 length = len(text)

 while index < length:
 if text[index] not in COUNTED_SYMBOLS:
 index += 1
 continue

 index += 1
 repeat, index = cls._repeat_at(text, index)
 digits += repeat if repeat else 1

 if not digits:
 return 0

```

```

 if cls.is_comp_3(usage) or cls.is_comp_3(text):
 return (digits // 2) + 1

 return digits

@classmethod
def picture_bytes(cls, picture: str) -> int:
 """Back-compatible entry point for callers that pass a clause."""
 return cls.size_of(picture, picture)

#
Candidates
#
@classmethod
def candidates(cls, body: str, match=None) -> list[str]:
 """Every readable form of this entry's PICTURE, unordered."""
 out: list[str] = []

 captured = cls._group(match, PICTURE_GROUP)
 if captured:
 out.append(captured)

 clause = cls.clause_of(body)
 if clause:
 out.append(clause)

 # A caller may hand over a bare clause with no PIC keyword.
 if not clause and body and not cls._has_pic_keyword(str(body)):
 out.append(str(body))

 return out

@classmethod
def clause_of(cls, body: str) -> str:
 """The PICTURE clause of a data description line, bounded.

 Returns '' when the line carries no PICTURE.
 """
 text = str(body or "")
 upper = text.upper()

 start = -1
 keyword_length = 0

 for keyword in PIC_KEYWORDS:
 position = cls._find_word(upper, keyword)

 if position >= 0:
 start = position
 keyword_length = len(keyword)
 break

 if start < 0:
 return ""

 tokens = text[start + keyword_length:].split()

 if tokens and tokens[0].upper() == "IS":
 tokens = tokens[1:]

 kept: list[str] = []

 for token in tokens:
 if token.upper().strip(PERIOD) in STOP_WORDS:
 break
 kept.append(token)

 return " ".join(kept).strip().rstrip(PERIOD)

@staticmethod
def is_comp_3(text: str) -> bool:
 upper = str(text or "").upper()
 return any(symbol in upper for symbol in COMP_3_SYMBOLS)

#

```



```

Character arithmetic
#
@staticmethod
def _repeat_at(text: str, index: int) -> tuple[int, int]:
 """Read an optional '(n)' starting at `index`.

 Returns (repeat, next_index). repeat is 0 when the symbol
 carries no parenthesised count, and `next_index` is then
 unchanged, so the caller charges one position for the symbol.
 """
 length = len(text)
 cursor = index

 while cursor < length and text[cursor].isspace():
 cursor += 1

 if cursor >= length or text[cursor] != OPEN_PAREN:
 return 0, index

 cursor += 1

 while cursor < length and text[cursor].isspace():
 cursor += 1

 digits = ""

 while cursor < length and text[cursor].isdigit():
 digits += text[cursor]
 cursor += 1

 while cursor < length and text[cursor].isspace():
 cursor += 1

 if cursor >= length or text[cursor] != CLOSE_PAREN or not digits:
 return 0, index

 return int(digits), cursor + 1

@staticmethod
def _find_word(upper: str, word: str) -> int:
 """Index of `word` in `upper` when it stands as a whole token."""
 start = 0

 while True:
 position = upper.find(word, start)

 if position < 0:
 return -1

 before_ok = (
 position == 0
 or upper[position - 1] not in NAME_CHARACTERS
)
 after = position + len(word)
 after_ok = (
 after >= len(upper)
 or upper[after] not in NAME_CHARACTERS
)

 if before_ok and after_ok:
 return position

 start = position + 1

@classmethod
def _has_pic_keyword(cls, body: str) -> bool:
 upper = str(body or "").upper()
 return any(
 cls._find_word(upper, keyword) >= 0 for keyword in PIC_KEYWORDS
)

@staticmethod
def _group(match, name: str) -> str:
 if match is None:
 return ""

```

```

try:
 return str(match.group(name) or "").strip()
except (IndexError, KeyError, AttributeError, TypeError):
 return ""

__all__ = ["COMP_3_SYMBOL", "COUNTED_SYMBOLS", "RecordPictureSizer"]

=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_plan.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_plan.py
ACTION: CREATE NEW FILE
"""Everything needed to materialise one record.

A value object. It aggregates RecordFieldPlan values and answers the
size questions the guard asks; it never builds itself.
"""

from __future__ import annotations

from dataclasses import dataclass, field

from idms_db2_phase2.composers.record_materialisation.record_field import (
 DEFAULT_LEVEL,
 RecordFieldPlan,
)
from rules.record_materialisation_rules import (
 REQUIRE_COMPLETE_FIELD_COVERAGE,
)

@dataclass
class RecordMaterialisationPlan:
 """Fields, diagnostics and sizes for one materialised record."""

 record_name: str = ""
 table_name: str = ""
 group_name: str = ""
 fields: list[RecordFieldPlan] = field(default_factory=list)
 missing_hosts: list[str] = field(default_factory=list)
 skipped_rows: int = 0

 # ----- views
 @property
 def declarable_fields(self) -> list[RecordFieldPlan]:
 return [item for item in self.fields if item.is_declarable]

 @property
 def movable_fields(self) -> list[RecordFieldPlan]:
 return [item for item in self.fields if item.is_movable]

 @property
 def filler_fields(self) -> list[RecordFieldPlan]:
 return [item for item in self.fields if item.is_filler]

 # ----- sizes
 @property
 def total_bytes(self) -> int:
 return sum(
 item.declared_bytes
 for item in self.fields
 if item.occupies_storage
)

 @property
 def min_level(self) -> int:
 """Shallowest level in the mapping, used to rebase the layout."""
 levels = [
 int(str(item.level).strip())
 for item in self.declarable_fields
 if str(item.level).strip().isdigit()
]

```

```

 return min(levels) if levels else int(DEFAULT_LEVEL)

----- checks
@property
def is_complete(self) -> bool:
 if not REQUIRE_COMPLETE_FIELD_COVERAGE:
 return True
 return not self.missing_hosts

def remainder_bytes(self, declared_bytes: int) -> int:
 """Unused trailing bytes of the target field. Never negative.

 The IDMS record is a PREFIX of the output field:

 05 F-FORM PIC X(478).
 06 VMBFAS. <- 432 bytes
 06 FILLER PIC X(46). <- this remainder

 Without the remainder the record silently shortens and every
 following field shifts.
 """
 if not declared_bytes or int(declared_bytes) <= 0:
 return 0
 return max(0, int(declared_bytes) - self.total_bytes)

def overflows(self, declared_bytes: int, tolerance: int = 0) -> bool:
 """True ONLY when the layout is LONGER than the target field.

 Shorter is normal and is closed with FILLER, never refused.
 """
 if not declared_bytes or int(declared_bytes) <= 0:
 return False
 return (self.total_bytes - int(declared_bytes)) > int(tolerance)

__all__ = ["RecordMaterialisationPlan"]

=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_plan_builder.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_plan_builder.py
ACTION: CREATE NEW FILE
"""Assembles a RecordMaterialisationPlan from Sheet Mapping rows.

Orchestration only:

 record_row_reader.py reads each row
 record_host_resolver.py resolves the DCLGEN host
 record_field.py answers the policy questions
 record_plan.py holds the result

CORRECTION B + C - what counts as 'missing'

 A field is missing ONLY when it carries a REAL DB2 column that
 DCLGEN does not define. No column at all means COBOL filler or the
 breakdown of a mapped parent, and must never block the record.
 """

from __future__ import annotations

from idms_db2_phase2.composers.record_materialisation.record_host_resolver import (
 RecordHostResolver,
)
from idms_db2_phase2.composers.record_materialisation.record_plan import (
 RecordMaterialisationPlan,
)
from idms_db2_phase2.composers.record_materialisation.record_row_reader import (
 RecordRowReader,
)
from rules.record_materialisation_rules import NON_RECORD_NAME_SUFFIXES

class RecordPlanBuilder:

```

```

"""Turns Sheet Mapping rows into a RecordMaterialisationPlan."""

def __init__(
 self,
 *,
 mapping_repository,
 table_name_resolver,
 host_variable_resolver,
 row_reader: RecordRowReader | None = None,
) -> None:
 self.mapping_repository = mapping_repository
 self.rows_reader = row_reader or RecordRowReader()
 self.hosts = RecordHostResolver(
 table_name_resolver=table_name_resolver,
 host_variable_resolver=host_variable_resolver,
)

----- public
def build(self, record_name: str) -> RecordMaterialisationPlan:
 record = self.cobol_name(record_name)
 plan = RecordMaterialisationPlan(record_name=record)

 rows = self._rows(record)
 if not rows:
 return plan

 plan.table_name = self.hosts.table_for(record)
 plan.group_name = self.hosts.group_for(plan.table_name)

 for row in rows:
 item = self.rows_reader.read(row)

 if item is None:
 plan.skipped_rows += 1
 continue

 # Only a REAL column is ever looked up. FILLER, a blank
 # cell, a REDEFINES and the unmapped subordinates of a
 # mapped group are never queried.
 if item.requires_host:
 item.host_reference = self.hosts.host_for(
 plan.table_name,
 item.db2_column,
)

 plan.fields.append(item)

 if item.is_missing_host:
 plan.missing_hosts.append(item.name)

 return plan

@staticmethod
def looks_like_record(name: str) -> bool:
 """Guard so a LINKAGE work field such as DATE8-LS is not treated
 as a mapped IDMS record. Suffix driven, so no name is hardcoded.
 """
 candidate = str(name or "").strip().upper()
 if not candidate:
 return False
 return not candidate.endswith(NON_RECORD_NAME_SUFFIXES)

----- helpers
def _rows(self, record: str) -> list:
 try:
 return list(
 self.mapping_repository.rows_for_record(record) or []
)
 except Exception: # noqa: BLE001
 return []

@staticmethod
def cobol_name(value: str) -> str:
 return str(value or "").strip().upper().replace("_", "-")

```

```
__all__ = ["RecordPlanBuilder"]
```

```
=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_redefines_renderer.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_redefines_renderer.py
ACTION: CREATE NEW FILE
"""Renders a REDEFINES entry, on one line or two.
```

CORRECTION 1 - REDEFINES ran into its picture

```
10 CT-RK-TGOOD REDEFINES CT-RK-TGDSVPIC 9(4).
 ^^^ compile error
```

The template carried no separator. A gap is now guaranteed and is computed from the remaining body width.

CORRECTION 5 - a REDEFINES entry filled the body exactly

```
10 CT-RK-TGOOD REDEFINES CT-RK-TGDSV PIC 9(4).00900000
```

65 characters, no gap before the sequence area, no margin for a longer data-name. The manual splits the entry:

```
10 CT-RK-TGOOD REDEFINES CT-RK-TGDSV
 PIC 9(4).
```

```
render() therefore returns a LIST of lines.
"""
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.composers.record_materialisation.record_layout_geometry import (
 RecordLayoutGeometry,
)
from rules.record_materialisation_rules import (
 LAYOUT_MIN_NAME_GAP,
 REDEFINES_CONTINUATION_GAP,
 REDEFINES_HEAD_TEMPLATE,
 REDEFINES_INLINE_LIMIT,
 REDEFINES_TEMPLATE,
 WRAP_REDEFINES_ENTRY,
)
```

```
REDEFINES_KEYWORD = "REDEFINES"
```

```
class RecordRedefinesRenderer:
```

```
 """A REDEFINES entry. One line when it fits, two when it does not."""
```

```
 def __init__(self, geometry: RecordLayoutGeometry | None = None) -> None:
 self.geometry = geometry or RecordLayoutGeometry()
```

```
 # ----- public
```

```
 def render(
 self,
 *,
 level: str,
 name: str,
 base: str,
 picture: str,
 indent: str,
) -> list[str]:
 clean_picture = str(picture or "").strip()
```

```
 if not clean_picture:
 body = f"{level} {name} {REDEFINES_KEYWORD} {base}"
 return [
 self.geometry.cap(self.geometry.terminate(indent + body))
]
```

```
 single = self.inline(
 level=level,
```

```

 name=name,
 base=base,
 picture=clean_picture,
 indent=indent,
)

 if not WRAP_REDEFINES_ENTRY:
 return [single]
 if len(single) <= REDEFINES_INLINE_LIMIT:
 return [single]

 return self.wrapped(
 level=level,
 name=name,
 base=base,
 picture=clean_picture,
 indent=indent,
)

----- inline
def inline(
 self,
 *,
 level: str,
 name: str,
 base: str,
 picture: str,
 indent: str,
) -> str:
 """CORRECTION 1. The gap before the picture is guaranteed."""
 head = f"{indent}{level} {name} {REDEFINES_KEYWORD} {base}"
 gap = self.geometry.gap_before(head, picture)

 body = REDEFINES_TEMPLATE.format(
 level=level,
 name=name,
 base=base,
 gap=" " * gap,
 picture=picture,
)
 return self.geometry.cap(self.geometry.terminate(indent + body))

----- wrapped
def wrapped(
 self,
 *,
 level: str,
 name: str,
 base: str,
 picture: str,
 indent: str,
) -> list[str]:
 """CORRECTION 5. Manual reference shape, two lines."""
 gap = " " * max(LAYOUT_MIN_NAME_GAP, REDEFINES_CONTINUATION_GAP)
 head = REDEFINES_HEAD_TEMPLATE.format(
 level=level,
 name=name,
 gap=gap,
 base=base,
)

 # The continuation aligns under the data-name.
 continuation = indent + (" " * (len(level) + 2))

 return [
 self.geometry.cap(indent + head),
 self.geometry.cap(
 self.geometry.terminate(continuation + picture)
),
]

```

```
__all__ = ["RecordRedefinesRenderer"]
```

```
=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_row_reader.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_row_reader.py
ACTION: CREATE NEW FILE
"""Turns one Sheet Mapping row into a RecordFieldPlan.
```

Sheet Mapping is the authority for the COBOL layout (cobol\_zone, idms\_pic\_clause) and the DB2 column. Nothing is invented: a cell the sheet does not supply becomes an empty string, never a guess.

CORRECTION - the byte fallback trusted one repeat pattern

\_bytes fell back to PICTURE\_REPEAT\_PATTERN, which only sees an explicit '(nnn)'. That reads 'X(10)' correctly but returns 3 for 'XXX' and 13 for 'S9(13)V99' (the trailing V99 is invisible).

The fallback now counts every stored symbol with its repeat, the same way RecordMoveScanner sizes a target PICTURE. The sheet's own length column still wins when it is present.

```
from __future__ import annotations
```

```
from idms_db2_phase2.composers.record_materialisation.record_field import (
 COMP_3_SYMBOL,
 DEFAULT_LEVEL,
 RecordFieldPlan,
)
from patterns.record_materialisation_patterns import (
 COBOL_ZONE_PATTERN,
 COBOL_ZONE_REDEFINES_PATTERN,
 PICTURE_PATTERN,
 PICTURE_SYMBOL_PATTERN,
)
from rules.record_materialisation_rules import GROUP_PICTURE_TOKENS
```

```
COUNTED_SYMBOLS = ("X", "9", "A", "Z")
```

```
class RecordRowReader:
```

```
 """Reads Sheet Mapping rows. Resolves nothing, decides nothing."""
```

```
 # ----- public
```

```
 def read(self, row) -> RecordFieldPlan | None:
```

```
 """A field, or None when the row declares no COBOL storage.
```

```
 None means SHAPE 7: a DB2-only audit or surrogate-key row
 (TS_CREATE, TS_UPDATE, ID_USERID, the Foreign Key rows). It
 exists in the table, not in the record.
 """
```

```
 zone = str(getattr(row, "cobol_zone", "") or "").strip()
 if not zone:
 return None
```

```
 level, name, base = self._split_zone(zone)
 if not name:
 return None
```

```
 picture, usage = self.picture(
 str(getattr(row, "idms_pic_clause", "") or "")
)
```

```
 return RecordFieldPlan(
 level=str(level).strip().zfill(2),
 name=name,
 picture=picture,
 usage=usage,
 redefines=base,
 db2_column=self.column(row),
 db2_type=str(getattr(row, "new_db2_data_type", "") or ""),
 declared_bytes=self.declared_bytes(row, picture, usage),
)
```

```

----- zone
@staticmethod
def _split_zone(zone: str) -> tuple[str, str, str]:
 """'05 CT-RK-TGOOD REDEFINES CT-RK-TGDSV' -> (level, name, base)."""
 redefines = COBOL_ZONE_REDEFINES_PATTERN.match(zone)
 if redefines:
 return (
 redefines.group("level") or DEFAULT_LEVEL,
 redefines.group("name").upper(),
 redefines.group("base").upper(),
)

 plain = COBOL_ZONE_PATTERN.match(zone)
 if not plain:
 return DEFAULT_LEVEL, "", ""

 return (
 plain.group("level") or DEFAULT_LEVEL,
 plain.group("name").upper(),
 "",
)

----- picture
@staticmethod
def picture(clause: str) -> tuple[str, str]:
 """'PIC S9(13)V99 COMP-3' -> ('PIC S9(13)V99', 'COMP-3')."""
 text = str(clause or "").strip().upper()
 if not text or text in GROUP_PICTURE_TOKENS:
 return "", ""

 match = PICTURE_PATTERN.search(text)
 if not match:
 return "", ""

 return (
 f"PIC {match.group('picture')}",
 str(match.group("usage") or "").strip(),
)

----- column
@staticmethod
def column(row) -> str:
 return str(
 getattr(row, "new_db2_field_name", "")
 or getattr(row, "cross_application_db2_field_name", "")
 or ""
).strip().upper()

----- bytes
@classmethod
def declared_bytes(cls, row, picture: str, usage: str) -> int:
 """The sheet's length column, or the PICTURE when it is blank."""
 declared = str(
 getattr(row, "length_of_field_bytes", "") or ""
).strip()
 if declared.isdigit():
 return int(declared)
 return cls.picture_bytes(picture, usage)

@staticmethod
def picture_bytes(picture: str, usage: str = "") -> int:
 """Character positions of a PICTURE, honouring repeat counts.

 X(10) -> 10
 XXX -> 3
 S9(13)V99 -> 15 (S and V are not stored)
 """
 text = str(picture or "").upper()
 if not text:
 return 0

 digits = 0
 for found in PICTURE_SYMBOL_PATTERN.finditer(text):
 symbol = str(found.group("symbol") or "").upper()
 if symbol not in COUNTED_SYMBOLS:
 continue

```



```

 repeat = found.group("repeat")
 digits += int(repeat) if repeat and repeat.isdigit() else 1

 if not digits:
 return 0
 if COMP_3_SYMBOL in str(usage or "").upper():
 return (digits // 2) + 1
 return digits

```

```
__all__ = ["RecordRowReader"]
```

```

=====
FILE: src\idms_db2_phase2\composers\record_materialisation\record_sites.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/record_materialisation/record_sites.py
ACTION: CREATE NEW FILE
"""Where a whole-record MOVE and its target field live.

```

```

Two frozen value objects. They carry no behaviour beyond the two
convenience properties the caller needs, so a scan result can be passed
around without anyone being able to mutate it halfway through a rewrite.
"""

```

```
from __future__ import annotations
```

```
from dataclasses import dataclass
```

```
NOT_FOUND = -1
```

```
@dataclass(frozen=True)
```

```
class MoveSite:
```

```
 """One whole-record MOVE found in the PROCEDURE DIVISION.

```

```

 MOVE VMBFAS TO F-FORM
 ^record ^target
 """

```

```

 index: int = NOT_FOUND
 record: str = ""
 target: str = ""

```

```
@property
```

```

def is_found(self) -> bool:
 return self.index > NOT_FOUND

```

```
@property
```

```

def key(self) -> tuple[str, str]:
 """Identity used to remember which moves have been visited.

```

```

 The composer iterates until every move has been seen once, so a
 refused record cannot be retried forever.
 """

```

```
 return (self.record, self.target)
```

```
@dataclass(frozen=True)
```

```
class TargetSite:
```

```
 """The DATA DIVISION entry a whole-record MOVE writes into.

```

```

 05 F-FORM PIC X(478).
 ^level ^index ^declared_bytes
 """

```

```

 index: int = NOT_FOUND
 declared_bytes: int = 0
 level: int = 0

```

```
@property
```

```

def is_found(self) -> bool:
 return self.index >= 0

```

```

@property
def has_length(self) -> bool:
 """False means the PICTURE could not be sized.

 The caller must treat that as a diagnostic, never as zero
 remaining bytes - a silent zero is what shortened the record by
 46 bytes with no warning.
 """
 return self.declared_bytes > 0

```

```
__all__ = ["NOT_FOUND", "MoveSite", "TargetSite"]
```

```

=====
FILE: src\idms_db2_phase2\composers\record_materialisation_composer.py
ENCODING: utf-8
=====

```

```

LOCATION: src\idms_db2_phase2\composers\record_materialisation_composer.py
ACTION: REPLACE ENTIRE FILE
"""Materialises IDMS records referenced by whole-record MOVE statements.

```

```

MOVE VMBFAS TO F-FORM
-> 05 F-FORM.
 06 VMBFAS.
 07 ...
 06 FILLER PIC X(46).
MOVE <host> TO <field> OF VMBFAS ... one per mapped field

```

CORRECTION E - first-match-wins (the defect that masked all the others)

find\_move returned the FIRST whole-record MOVE and compose() ran 'return str(text)' on any refusal, abandoning the entire pass. A LINKAGE date field (DATE8-LS) matched first, had no Sheet Mapping rows, and VMBFAS was never examined - it then reached the structural safety pass and was commented out.

Every whole-record MOVE is now visited, and a refusal is LOCAL.

CORRECTION F - a refusal was invisible

Each refusal already carried its reason, but the reason landed in the Info band among thirty-odd other lines, so several runs went by with nobody able to say WHY the record did not materialise.

Three changes close that:

- \* the measured target length is reported on every attempt, so a length refusal needs no code read to interpret,
- \* every refusal reason is collected and re-emitted as ONE summary line naming the records,
- \* rules/validation\_display\_rules.py lifts refusal wording into the Warnings band.

Orchestration only:

```

record_move_scanner.py finds the sites
record_materialisation_guard.py decides refusals
record_block_writer.py writes the replacement lines
"""

```

```
from __future__ import annotations
```

```

from idms_db2_phase2.composers.record_materialisation.record_block_writer import (
 RecordBlockWriter,
)
from idms_db2_phase2.composers.record_materialisation.record_layout_generator import (
 RecordLayoutGenerator,
)
from idms_db2_phase2.composers.record_materialisation.record_line_utils import (
 RecordLineUtils,
)
from idms_db2_phase2.composers.record_materialisation.record_materialisation_guard import (
 RecordMaterialisationGuard,
)

```

```

from idms_db2_phase2.composers.record_materialisation.record_move_generator import (
 RecordMoveGenerator,
)
from idms_db2_phase2.composers.record_materialisation.record_move_scanner import (
 RecordMoveScanner,
)
from idms_db2_phase2.composers.record_materialisation.record_plan_builder import (
 RecordPlanBuilder,
)
from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)
from rules.record_materialisation_rules import (
 EMIT_TARGET_MEASUREMENT,
 ENFORCE_RECORD_MATERIALISATION,
 MAX_RECORD_MOVES,
 RECORD_MATERIALISATION_MESSAGES,
)

```

```

DETAIL_SEPARATOR = "; "
DETAIL_TEMPLATE = "{record} ({reason})"
MAX_DETAILS = 6

```

```

class RecordMaterialisationComposer:
 """Expands every whole-record MOVE it can prove safe."""

 def __init__(
 self,
 *,
 mapping_repository,
 table_name_resolver,
 host_variable_resolver,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.lines = RecordLineUtils(
 fixed_format=fixed_format or FixedFormatLineService()
)
 self.scanner = RecordMoveScanner(self.lines)
 self.guard = RecordMaterialisationGuard(
 plan_builder=RecordPlanBuilder(
 mapping_repository=mapping_repository,
 table_name_resolver=table_name_resolver,
 host_variable_resolver=host_variable_resolver,
)
)
 self.writer = RecordBlockWriter(
 layout_generator=RecordLayoutGenerator(),
 move_generator=RecordMoveGenerator(),
 line_utils=self.lines,
)
 self.messages: list[str] = []
 self.refusals: list[tuple[str, str]] = []

 # =====
 # Public entry point
 # =====
 def compose(self, text: str) -> str:
 self.messages = []
 self.refusals = []

 if not text or not ENFORCE_RECORD_MATERIALISATION:
 return str(text or "")

 current = str(text).replace("\r\n", "\n").replace("\r", "\n")

 handled: set[tuple[str, str]] = set()
 examined = applied = 0

 for _ in range(MAX_RECORD_MOVES):
 source = current.split("\n")

 site = self.scanner.find_move(source, handled)
 if site is None:
 break

```

```

 handled.add(site.key)
 examined += 1

 rewritten = self._materialise(source, site)
 if rewritten is None:
 continue

 current = "\n".join(rewritten)
 applied += 1

 self._report(examined, applied)
 return current.rstrip() + "\n"

=====
One record - None when refused
=====
def _materialise(self, lines: list[str], site) -> list[str] | None:
 plan, refusal = self.guard.build_plan(site.record)
 if refusal is not None:
 self._refuse(site.record, refusal)
 return None

 target = self.scanner.find_target(lines, site.target)

 if not target.is_found:
 self._refuse_key(
 site.record,
 "skipped_no_target",
 "target field not found in the DATA DIVISION",
 target=site.target,
)
 return None

 # Two numbers that make a length refusal self-explanatory.
 if EMIT_TARGET_MEASUREMENT:
 self._log_key(
 "target_measured",
 target=site.target,
 bytes=target.declared_bytes,
 record=plan.record_name,
 actual=plan.total_bytes,
)

 refusal = self.guard.check_length(
 plan, site.target, target.declared_bytes
)
 if refusal is not None:
 self._refuse(site.record, refusal)
 return None

 advice = self.guard.check_movable(plan)
 if advice is not None:
 self._log(advice)

 updated = self.writer.replace_move(list(lines), site.index, plan)
 updated = self.writer.expand_target(
 updated,
 target.index,
 site.target,
 plan,
 target_level=target.level,
 declared_bytes=target.declared_bytes,
)

 self._log_success(plan, site.target, target.declared_bytes)
 return updated

=====
Diagnostics
=====
def _log_success(self, plan, target: str, declared_bytes: int) -> None:
 self._log_key(
 "materialised",
 target=target,
 record=plan.record_name,
 fields=len(plan.declarable_fields),
)

```

```

 bytes=plan.total_bytes,
 moves=len(plan.movable_fields),
)

 # A target whose PIC could not be sized silently disables the
 # remainder FILLER, so the record shortens and every following
 # byte shifts. Never let that happen without a diagnostic.
 if not declared_bytes:
 self._log_key(
 "no_declared_length",
 target=target,
 record=plan.record_name,
)
 return

remainder = plan.remainder_bytes(declared_bytes)
if remainder:
 self._log_key(
 "remainder_filler",
 record=plan.record_name,
 target=target,
 actual=plan.total_bytes,
 expected=declared_bytes,
 remainder=remainder,
)

def _report(self, examined: int, applied: int) -> None:
 """A no-op is always visible, and a refusal is never buried."""
 if examined == 0:
 self._log_key("no_moves")
 return

 self._log_key(
 "scan_summary",
 examined=examined,
 applied=applied,
 refused=len(self.refusals),
)

 if not self.refusals:
 return

 details = DETAIL_SEPARATOR.join(
 DETAIL_TEMPLATE.format(record=record, reason=reason)
 for record, reason in self.refusals[:MAX_DETAILS]
)
 self._log_key(
 "refusal_summary",
 count=len(self.refusals),
 details=details,
)

=====
Refusal recording
=====
def _refuse(self, record: str, refusal) -> None:
 """Log the reason AND remember it for the summary line."""
 self._log(refusal)
 self.refusals.append((record, self._reason_of(refusal)))

def _refuse_key(
 self,
 record: str,
 key: str,
 reason: str,
 **values,
) -> None:
 self._log_key(key, **values)
 self.refusals.append((record, reason))

@staticmethod
def _reason_of(refusal) -> str:
 """A short human reason, derived from the message key."""
 return str(getattr(refusal, "key", "") or "refused").replace(
 "skipped_", ""
).replace("_", " ")

```

```

=====
Message rendering
=====
def _log(self, refusal) -> None:
 values = dict(getattr(refusal, "values", {}) or {})
 self._log_key(getattr(refusal, "key", ""), **values)

def _log_key(self, key: str, **values) -> None:
 template = RECORD_MATERIALISATION_MESSAGES.get(key, "")
 if not template:
 return
 try:
 self.messages.append(template.format(**values))
 except (IndexError, KeyError):
 # A template/value mismatch must never lose the diagnostic.
 self.messages.append(f"{template} [{key}: {values}]")

__all__ = ["RecordMaterialisationComposer"]

```

```

=====
FILE: src\idms_db2_phase2\composers\sequence_artifact_cleanup_composer.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/sequence_artifact_cleanup_composer.py
ACTION: CREATE NEW FILE

```

```

"""Sequence artifact cleanup composer.

```

Removes lines whose COBOL body is nothing but a leaked source sequence number, and repairs the sentence the removal exposes.

Symptom

```

002480 STOP RUN.
002490 001690/. <-- not COBOL
002500 HOOFDVERWERKING.

```

The original source carried a page-eject line:

```

001690/

```

A formatting pass that did not recognise the short, unsequenced form treated the WHOLE line as COBOL text. The left sequence number and the indicator became the body, and the paragraph terminator pass then added a period, producing '001690/.'.

Consequences if left in place

```

- The compiler rejects the statement.
- OutputWritePlacementCleanup cannot match a trailing
 IF ... WRITE ... END-IF block, because the artifact sits between the
 END-IF and the paragraph boundary. The guarded output write is then
 never relocated out of the child-row paragraph, and the WRITE fires
 once per fetched child row instead of once per parent row.

```

Why this runs late

```

The artifact is produced AFTER CobolCleanupComposer, by one of the
layout passes. Dropping it inside ParagraphTerminatorCleanup is too
early. This composer therefore runs immediately after
composers["fixed_format"], where every line is sequenced and no further
pass will reintroduce the shape.

```

Repair

```

Removing the artifact can leave the preceding statement unterminated,
because the artifact was carrying the period:

```

```

003040 END-IF
003050 002190/. <-- removed

```

This composer therefore walks back to the last executable line of the

paragraph and restores the sentence terminator when it is missing.

#### Clean Architecture

```

- Constants live in rules/sequence_artifact_rules.py.
- Line geometry and artifact detection come from FixedFormatLineService.
- No regex is defined here.
- No program, paragraph, record, table, cursor or host variable name is
 hardcoded.
"""
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)
from rules.sequence_artifact_rules import (
 ENFORCE_SEQUENCE_ARTIFACT_CLEANUP,
 PARAGRAPH_TERMINATOR,
 REPAIR_EXPOSED_SENTENCE,
 SENTENCE_CARRIERS,
 SEQUENCE_ARTIFACT_MESSAGES,
)
```

```
class SequenceArtifactCleanupComposer:
```

```
 """Drops leaked sequence-number lines and repairs the sentence."""
```

```
 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()
 self.messages: list[str] = []

 # =====
 # Public entry point
 # =====
 def compose(self, text: str) -> str:
 self.messages = []

 if not text or not ENFORCE_SEQUENCE_ARTIFACT_CLEANUP:
 return str(text or "")

 lines = (
 str(text)
 .replace("\r\n", "\n")
 .replace("\n", "\r\n")
 .split("\n")
)

 output: list[str] = []
 removed = 0
 repaired = 0

 for line in lines:
 if not self.fixed_format.is_sequence_artifact(line):
 output.append(line)
 continue

 removed += 1
 self.messages.append(
 SEQUENCE_ARTIFACT_MESSAGES["removed"].format(
 body=self.fixed_format.logical(line)
)
)

 if REPAIR_EXPOSED_SENTENCE and self._terminate_previous(output):
 repaired += 1

 if removed:
 self.messages.append(
 SEQUENCE_ARTIFACT_MESSAGES["summary"].format(
 removed=removed,
 repaired=repaired,
)
)
```

```

)

 return "\n".join(output).rstrip() + "\n"

=====
Sentence repair
=====
def _terminate_previous(self, output: list[str]) -> bool:
 """Close the sentence the removed artifact was carrying.

 Walks back over blank, comment, page-eject and debug lines to the
 last executable statement. Adds a period only when the statement
 is a recognised sentence carrier and does not already end in one.
 """
 for index in range(len(output) - 1, -1, -1):
 line = output[index]

 if self.fixed_format.is_comment_or_control_line(line):
 continue

 logical = self.fixed_format.logical(line)
 if not logical:
 continue

 if logical.endswith(PARAGRAPH_TERMINATOR):
 return False

 first_word = logical.split()[0].upper() if logical.split() else ""
 if first_word not in SENTENCE_CARRIERS:
 return False

 new_body = self.fixed_format.body(line).rstrip()
 rebuilt = self.fixed_format.replace_body(
 line,
 new_body + PARAGRAPH_TERMINATOR,
)

 if rebuilt == line:
 return False

 output[index] = rebuilt
 self.messages.append(
 SEQUENCE_ARTIFACT_MESSAGES["terminated"].format(
 statement=logical
)
)
)
 return True

return False

```

```

=====
FILE: src\idms_db2_phase2\composers\sqlcode_wrapper_cleanup_composer.py
ENCODING: utf-8
=====

```

"""

SQLCODE wrapper cleanup composer.

Removes contradictory SQLCODE wrappers left after redundant OBTAIN NEXT cursor calls have been removed. Targets:

```

IF NOT SQLCODE = 100
 * DB2: Removed redundant OBTAIN NEXT after cursor EOC loop.
 CONTINUE.
IF SQLCODE = 100
 ...
END-IF
END-IF.

```

and rewrites it to a single cleanup comment plus the recovered inner body.

Detection strings and tokens live in rules/sqlcode\_wrapper\_rules.py; the MOVE regex in patterns/sqlcode\_wrapper\_patterns.py; line/wrap helpers in SqlcodeWrapperLineUtils.

"""



```

from __future__ import annotations

from idms_db2_phase2.composers.sqlcode_wrapper_line_utils import (
 SqlcodeWrapperLineUtils,
)
from rules.sqlcode_wrapper_rules import (
 BASE_INDENT,
 CHILD_INDENT,
 CLEANUP_COMMENT_TEXT,
 COMMENT_PREFIXES,
 INNER_IF_LOOKAHEAD,
 INNER_IF_TEXT,
 MARKER_LOOKAHEAD,
 OUTER_IF_TEXT,
 REMOVED_OBTAIN_NEXT_TEXT,
 TOKEN_END_IF,
 TOKEN_IF,
)

```

```

class SqlcodeWrapperCleanupComposer(SqlcodeWrapperLineUtils):
 def compose(self, text: str) -> str:
 if not text:
 return ""

 lines = self._normalize_line_endings(text).splitlines()
 output: list[str] = []
 index = 0

 while index < len(lines):
 logical = self._logical(lines[index]).upper()

 if OUTER_IF_TEXT not in logical:
 output.append(lines[index])
 index += 1
 continue

 block = self._try_extract_redundant_wrapper_block(
 lines=lines, start_index=index
)
 if block is None:
 output.append(lines[index])
 index += 1
 continue

 replacement_lines, next_index = block
 output.extend(replacement_lines)
 index = next_index

 return "\n".join(output).rstrip() + "\n"

 def _try_extract_redundant_wrapper_block(
 self, lines: list[str], start_index: int
) -> tuple[list[str], int] | None:
 outer_line = lines[start_index]

 marker_index = self._find_marker_after_outer_if(
 lines=lines, start_index=start_index + 1
)
 if marker_index < 0:
 return None

 inner_if_index = self._find_inner_if_sqlcode_100(
 lines=lines, start_index=marker_index + 1
)
 if inner_if_index < 0:
 return None

 inner_end_index = self._find_matching_end_if(
 lines=lines, if_index=inner_if_index
)
 if inner_end_index < 0:
 return None

 outer_end_index = self._find_matching_end_if(

```

```

 lines=lines, if_index=start_index
)
 if outer_end_index < 0 or outer_end_index <= inner_end_index:
 return None

 inner_body = lines[inner_if_index + 1 : inner_end_index]
 normalized_inner_body = self._normalize_inner_body(lines=inner_body)

 replacement = [
 self._format_comment_like_line(
 reference_line=outer_line,
 comment_text=CLEANUP_COMMENT_TEXT,
)
]
 replacement.extend(normalized_inner_body)

 return replacement, outer_end_index + 1

def _find_marker_after_outer_if(
 self, lines: list[str], start_index: int
) -> int:
 end_index = min(len(lines), start_index + MARKER_LOOKAHEAD)
 for index in range(start_index, end_index):
 if REMOVED_OBTAIN_NEXT_TEXT in self._logical(lines[index]).upper():
 return index
 return -1

def _find_inner_if_sqlcode_100(
 self, lines: list[str], start_index: int
) -> int:
 end_index = min(len(lines), start_index + INNER_IF_LOOKAHEAD)
 for index in range(start_index, end_index):
 if self._logical(lines[index]).upper().startswith(INNER_IF_TEXT):
 return index
 return -1

def _find_matching_end_if(self, lines: list[str], if_index: int) -> int:
 depth = 0
 for index in range(if_index, len(lines)):
 logical = self._logical(lines[index]).upper().rstrip(".")
 if logical.startswith(TOKEN_IF):
 depth += 1
 continue
 if logical == TOKEN_END_IF:
 depth -= 1
 if depth == 0:
 return index
 return -1

def _normalize_inner_body(self, lines: list[str]) -> list[str]:
 output: list[str] = []
 first_content_seen = False

 for line in lines:
 logical = self._logical(line)

 if not logical:
 output.append(line)
 continue

 if any(logical.startswith(p) for p in COMMENT_PREFIXES):
 output.append(line)
 continue

 if not first_content_seen:
 output.extend(
 self._normalized_lines_for_logical(
 original_line=line, logical=logical, indent=BASE_INDENT
)
)
 first_content_seen = True
 continue

 if logical.upper().rstrip(".") == TOKEN_END_IF:
 output.extend(
 self._normalized_lines_for_logical(

```

```

 original_line=line,
 logical=TOKEN_END_IF,
 indent=BASE_INDENT,
)
)
 continue

 output.extend(
 self._normalized_lines_for_logical(
 original_line=line, logical=logical, indent=CHILD_INDENT
)
)

 return output

def _normalized_lines_for_logical(
 self, original_line: str, logical: str, indent: str
) -> list[str]:
 wrapped_bodies = self._wrap_logical_body(logical=logical, indent=indent)
 return [
 self._replace_body_part(original_line=original_line, new_body=body)
 for body in wrapped_bodies
]

```

```

=====
FILE: src\idms_db2_phase2\composers\sqlcode_wrapper_line_utils.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from patterns.sqlcode_wrapper_patterns import MOVE_TO_PATTERN
from patterns.sequence_patterns import strip_sequence_numbers
from rules.sqlcode_wrapper_rules import (
 BASE_INDENT,
 BODY_END_COLUMN,
 BODY_START_COLUMN,
 BODY_WIDTH,
 CONTINUATION_INDENT,
 DB2_COMMENT_PREFIX,
 FULL_LINE_WIDTH,
 INDICATOR_COLUMN,
 RIGHT_SEQUENCE_END,
 RIGHT_SEQUENCE_START,
 SEQUENCE_AREA_WIDTH,
 TOKEN_MOVE,
 TOKEN_OF_DCL,
 TOKEN_TO,
)

class SqlcodeWrapperLineUtils:
 """Line normalization, MOVE/word wrapping, and fixed-format helpers.

 Owns no marker/detection strings (rules) and no MOVE regex (patterns).
 """

 def _normalize_line_endings(self, text: str) -> str:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

 def _logical(self, line: str) -> str:
 return strip_sequence_numbers(str(line or "")).strip()

 def _wrap_logical_body(self, logical: str, indent: str) -> list[str]:
 text = str(logical or "").strip()
 if not text:
 return [indent.rstrip()]

 move_lines = self._wrap_move_statement(logical=text, indent=indent)
 if move_lines:
 return move_lines

 body = indent + text
 if len(body) <= BODY_WIDTH:
 return [body]

```

```

 return self._wrap_by_words(
 body=body,
 continuation_indent=indent + CONTINUATION_INDENT,
)

def _wrap_move_statement(self, logical: str, indent: str) -> list[str]:
 match = MOVE_TO_PATTERN.match(str(logical or "").strip())
 if not match:
 return []

 source = str(match.group("source") or "").strip()
 target = str(match.group("target") or "").strip()
 if not source or not target:
 return []

 force_split = TOKEN_OF_DCL in source.upper()
 one_line = f"{indent}{TOKEN_MOVE}{source} {TOKEN_TO}{target}"

 if len(one_line) <= BODY_WIDTH and not force_split:
 return [one_line]

 first_line = f"{indent}{TOKEN_MOVE}{source}"
 second_line = f"{indent}{TOKEN_TO}{target}"
 output: list[str] = []

 if len(first_line) <= BODY_WIDTH:
 output.append(first_line)
 else:
 output.extend(
 self._wrap_by_words(
 body=first_line,
 continuation_indent=indent + CONTINUATION_INDENT,
)
)

 if len(second_line) <= BODY_WIDTH:
 output.append(second_line)
 else:
 output.extend(
 self._wrap_by_words(
 body=second_line,
 continuation_indent=indent + CONTINUATION_INDENT,
)
)

 return output

def _wrap_by_words(self, body: str, continuation_indent: str) -> list[str]:
 text = str(body or "").rstrip()
 if len(text) <= BODY_WIDTH:
 return [text]

 words = text.strip().split()
 output: list[str] = []
 current = ""

 for word in words:
 candidate = word if not current else current.rstrip() + " " + word
 if len(candidate) <= BODY_WIDTH:
 current = candidate
 continue
 if current:
 if output:
 output.append(continuation_indent + current.strip())
 else:
 output.append(current.strip())
 current = word

 if current:
 if output:
 output.append(continuation_indent + current.strip())
 else:
 output.append(current.strip())

 if not output:

```

```

 return [text[:BODY_WIDTH]]

normalized: list[str] = []
for index, line in enumerate(output):
 if index == 0:
 normalized.append(line[:BODY_WIDTH])
 continue
 if line.startswith(continuation_indent):
 normalized.append(line[:BODY_WIDTH])
 else:
 normalized.append(
 (continuation_indent + line.strip())[:BODY_WIDTH]
)

return normalized

def _replace_body_part(self, original_line: str, new_body: str) -> str:
 line = str(original_line or "").rstrip()

 if self._is_fixed_format_line(line):
 left = line[:SEQUENCE_AREA_WIDTH]
 indicator = line[INDICATOR_COLUMN]
 right = line[RIGHT_SEQUENCE_START:RIGHT_SEQUENCE_END]
 body = (
 str(new_body or "").rstrip()[:BODY_WIDTH].ljust(BODY_WIDTH)
)
 return f"{left}{indicator}{body}{right}"

 return str(new_body or "").rstrip()

def _format_comment_like_line(
 self, reference_line: str, comment_text: str
) -> str:
 line = str(reference_line or "").rstrip()
 clean_comment = str(comment_text or "").strip()

 if not clean_comment.upper().startswith(DB2_COMMENT_PREFIX):
 clean_comment = f"{DB2_COMMENT_PREFIX} {clean_comment}"

 body = f" {clean_comment}"

 if self._is_fixed_format_line(line):
 left = line[:SEQUENCE_AREA_WIDTH]
 right = line[RIGHT_SEQUENCE_START:RIGHT_SEQUENCE_END]
 body_area = body[:BODY_WIDTH].ljust(BODY_WIDTH)
 return f"{left}*{body_area}{right}"

 return f"*{body}"

def _is_fixed_format_line(self, line: str) -> bool:
 text = str(line or "")
 if len(text) < FULL_LINE_WIDTH:
 return False
 if not text[:SEQUENCE_AREA_WIDTH].isdigit():
 return False
 if not text[RIGHT_SEQUENCE_START:RIGHT_SEQUENCE_END].isdigit():
 return False
 return True

```

```

=====
FILE: src\idms_db2_phase2\composers\structural_safety_composer.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/structural_safety_composer.py
ACTION: REPLACE ENTIRE FILE
"""Final structural safety pass. COMMENT-ONLY, never deletes.

```

A2 - orphaned whole-record reference  
 MOVE VMBFAS TO F-FORM survives after COPY IDMS LR is removed, so  
 VMBFAS is an undefined data-name.

A4 - unreachable statement after a paragraph EXIT  
 A statement between EXIT. and the next paragraph header can never  
 run and sits outside any paragraph.

CORRECTION 1 - EXIT. matched the paragraph-header pattern  
The exit test now runs BEFORE the header test.

CORRECTION 2 - long markers lost their sequence numbers  
Comments are wrapped through emit().

CORRECTION 3 - comment lines were never recognised  
\_is\_comment() tested FixedFormatLineService.logical(), which returns  
columns 8-72 only. The '\*' indicator lives in column 7, so it was  
never in the string being tested and every comment was treated as  
executable. Classification now reads the RAW line.

CORRECTION 4 - a tracking bug could comment out the whole program  
With CORRECTION 1 and 3 missing, the A4 guard stayed armed past the  
generated cursor block and commented 60+ lines. A hard cap now  
reverts the entire pass when it matches more lines than real  
unreachable code ever spans.

"""

```
from __future__ import annotations
```

```
from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)
```

```
from patterns.structural_safety_patterns import (
 EXIT_STATEMENT_PATTERN,
 MOVE_BARE_SOURCE_PATTERN,
 MOVE_BARE_TARGET_PATTERN,
 PARAGRAPH_HEADER_PATTERN,
 PROCEDURE_DIVISION_PATTERN,
 QUALIFIER_PATTERN,
 SECTION_HEADER_PATTERN,
)
```

```
from rules.structural_safety_rules import (
 ENFORCE_ORPHANED_RECORD_GUARD,
 ENFORCE_UNREACHABLE_STATEMENT_GUARD,
 MAX_UNREACHABLE_STATEMENTS,
 ORPHANED_LINE_PREFIX,
 ORPHANED_RECORD_MARKER_TEMPLATE,
 STRUCTURAL_SAFETY_MESSAGES,
 UNREACHABLE_MARKER,
)
```

```
DCLGEN_PREFIX = "DCL"
```

```
FALLBACK_COMMENT_PREFIX = " *"
```

```
class StructuralSafetyComposer:
```

```
 """Comments references the compiler would reject."""
```

```
 def __init__(
 self,
 original_idms_text: str = "",
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.original_idms_text = str(original_idms_text or "")
 self.fixed_format = fixed_format or FixedFormatLineService()
 self.messages: list[str] = []
```

```
----- public
```

```
 def compose(self, text: str) -> str:
 self.messages = []
```

```
 if not text:
 return ""
```

```
 lines = (
 str(text).replace("\r\n", "\n").replace("\r", "\n").split("\n")
)
```

```
 if ENFORCE_ORPHANED_RECORD_GUARD:
 lines = self._guard_orphaned_records(lines)
```

```
 if ENFORCE_UNREACHABLE_STATEMENT_GUARD:
 lines = self._guard_unreachable_statements(lines)
```

```

 return "\n".join(lines).rstrip() + "\n"

----- A2
def _guard_orphaned_records(self, lines: list[str]) -> list[str]:
 records = self._idms_record_names()
 if not records:
 return lines

 declared = self._declared_names(lines)
 output: list[str] = []

 for line in lines:
 if self._is_skippable(line):
 output.append(line)
 continue

 body = self._logical(line)
 if not body:
 output.append(line)
 continue

 record = self._whole_record_reference(body, records, declared)
 if not record:
 output.append(line)
 continue

 output.extend(
 self._comment_lines(
 line,
 ORPHANED_RECORD_MARKER_TEMPLATE.format(record=record),
)
)
 output.extend(
 self._comment_lines(line, f"{ORPHANED_LINE_PREFIX}{body}")
)
 self.messages.append(
 STRUCTURAL_SAFETY_MESSAGES[
 "orphaned_record_commented"
].format(record=record)
)

 return output

def _idms_record_names(self) -> set[str]:
 names: set[str] = set()
 for match in QUALIFIER_PATTERN.finditer(self.original_idms_text):
 name = match.group("record").upper()
 if name and not name.startswith(DCLGEN_PREFIX):
 names.add(name)
 return names

def _declared_names(self, lines: list[str]) -> set[str]:
 declared: set[str] = set()
 for line in lines:
 if self._is_skippable(line):
 continue
 body = self._logical(line)
 if not body:
 continue
 if PROCEDURE_DIVISION_PATTERN.match(body):
 break
 parts = body.split()
 if len(parts) >= 2 and parts[0].rstrip(".").isdigit():
 declared.add(parts[1].rstrip(".").upper())
 return declared

@staticmethod
def _whole_record_reference(
 body: str,
 records: set[str],
 declared: set[str],
) -> str:
 for pattern in (MOVE_BARE_SOURCE_PATTERN, MOVE_BARE_TARGET_PATTERN):
 match = pattern.match(body)
 if not match:

```

```

 continue
 name = match.group("name").upper()
 if name in records and name not in declared:
 return name
 return ""

----- A4
def _guard_unreachable_statements(self, lines: list[str]) -> list[str]:
 output: list[str] = []
 after_exit = False
 in_procedure = False
 commented = 0

 for line in lines:
 # Comment, page-eject, debug and continuation lines are
 # classified from the RAW line: the indicator is column 7,
 # which logical() does not return.
 if self._is_skippable(line):
 output.append(line)
 continue

 body = self._logical(line)

 if PROCEDURE_DIVISION_PATTERN.match(body):
 in_procedure = True
 after_exit = False
 output.append(line)
 continue

 if not in_procedure or not body:
 output.append(line)
 continue

 # EXIT test FIRST: "EXIT." also matches the header pattern.
 if EXIT_STATEMENT_PATTERN.match(body):
 after_exit = True
 output.append(line)
 continue

 # Paragraph and section headers disarm the guard. The name
 # may start with a digit: 710-OPEN-<cursor>.
 if PARAGRAPH_HEADER_PATTERN.match(body) or (
 SECTION_HEADER_PATTERN.match(body)
):
 after_exit = False
 output.append(line)
 continue

 if not after_exit:
 output.append(line)
 continue

 output.extend(self._comment_lines(line, UNREACHABLE_MARKER))
 output.extend(
 self._comment_lines(line, f"{ORPHANED_LINE_PREFIX}{body}")
)
 commented += 1

 # Blast-radius cap: revert everything rather than trust a guard
 # that has clearly lost sync with the paragraph structure.
 if commented > MAX_UNREACHABLE_STATEMENTS:
 self.messages.append(
 STRUCTURAL_SAFETY_MESSAGES["unreachable_aborted"].format(
 count=commented,
 limit=MAX_UNREACHABLE_STATEMENTS,
)
)
 return lines

 if commented:
 self.messages.append(
 STRUCTURAL_SAFETY_MESSAGES["unreachable_commented"].format(
 count=commented,
)
)

```



```

return output

----- helpers
def _comment_lines(self, reference: str, body: str) -> list[str]:
 """Comment line(s), wrapped when the text exceeds columns 8-72."""
 text = str(body or "").lstrip("*").rstrip()
 try:
 if self.fixed_format.is_fixed_line(reference):
 left, _ind, _old, right = self.fixed_format.split(reference)
 return self.fixed_format.emit(left, "*", text, right)
 except Exception: # noqa: BLE001
 pass
 return [f"{FALLBACK_COMMENT_PREFIX}{text}"]

def _logical(self, line: str) -> str:
 try:
 return str(self.fixed_format.logical(line) or "").rstrip()
 except Exception: # noqa: BLE001
 return str(line or "").rstrip()

def _is_skippable(self, line: str) -> bool:
 """Comment / page-eject / debug / continuation, read from column 7."""
 try:
 return bool(self.fixed_format.is_comment_or_control_line(line))
 except Exception: # noqa: BLE001
 return str(line or "").lstrip().startswith(("*", "/"))

__all__ = ["StructuralSafetyComposer"]

```

```

=====
FILE: src\idms_db2_phase2\composers\unmapped_record_block_composer.py
ENCODING: utf-8
=====

```

```

"""
Record-level Option B composer (IDMS-source-seeded, column-confirmed).

A record is UNMAPPED when ColumnNameResolver.columns_for_record(record)
returns an empty list. Covers the case where a Sheet Mapping ROW/table
exists (e.g. FFRECAB -> DZ01RSTB) but has zero usable columns.

For an unmapped record every referencing line is commented (record-level):
- lines qualified with 'OF/IN <record>', AND
- bare lines that move into a field of the record's naming family
 (the '-FF' family for FFRECAB, e.g. NS-SET-FF, NS-REC-FF-IN, NR-KY-FF).

```

```

Production comment layout (each emitted line is exactly 80 columns):
- columns 1-6 : left sequence number (preserved from original line)
- column 7 : '*' comment indicator
- columns 8-72 : COBOL body (original indentation preserved)
- columns 73-80: right sequence number (preserved from original line)

```

```

If a comment body does not fit in columns 8-72 it is wrapped onto the next
comment line (standard COBOL comment continuation), each still 80 columns.
"""

```

```

from __future__ import annotations

```

```

import re

```

```

from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.idms_patterns import (
 ERASE_PATTERN,
 MODIFY_PATTERN,
 OBTAIN_CALC_PATTERN,
 OBTAIN_CALC_REVERSED_PATTERN,
 STORE_PATTERN,
)

```

```

BODY_WIDTH = 65 # columns 8-72
LEFT_WIDTH = 6 # columns 1-6
RIGHT_WIDTH = 8 # columns 73-80

```

```

class UnmappedRecordBlockComposer:

 OF_RECORD_PATTERN = re.compile(
 r"\b(?:OF|IN)\s+(?P<record>[A-Z][A-Z0-9-]*)\b",
 flags=re.IGNORECASE,
)
 STATEMENT_END = re.compile(r"\.s*$")
 LEFT_SEQ = re.compile(r"^(?P<left>\d{6})")
 RIGHT_SEQ = re.compile(r"(?P<seq>\d{8})s*$")

 # FFRECB restart-record field family: every FFRECB field ends in
 # '-FF' or '-FF-<suffix>'. A plain counter like CTR-REC does NOT match.
 FF_FIELD_PATTERN = re.compile(
 r"\bTO\s+[A-Z0-9-]*-FF(?:-[A-Z0-9-]+)?\b",
 flags=re.IGNORECASE,
)

 def __init__(
 self,
 table_name_resolver: TableNameResolver,
 column_name_resolver: ColumnNameResolver,
 original_idms_text: str = "",
) -> None:
 self.table_name_resolver = table_name_resolver
 self.column_name_resolver = column_name_resolver
 self.original_idms_text = original_idms_text or ""
 self.messages: list[str] = []

 def compose(self, text: str) -> str:
 if not text:
 return ""

 unmapped = self._discover_unmapped_records()
 if not unmapped:
 return text

 lines = text.replace("\r\n", "\n").split("\n")
 record_alt = "|".join(re.escape(r) for r in sorted(unmapped))
 record_ref = re.compile(
 r"\b(?:OF|IN)\s+(?:\" + record_alt + r\")\b",
 flags=re.IGNORECASE,
)
 ff_family_active = "FFRECB" in unmapped

 output: list[str] = []
 marked: set[str] = set()
 commented_any = False
 i = 0
 n = len(lines)

 while i < n:
 logical = self._logical(lines[i])

 if not logical or logical.startswith("#"):
 output.append(lines[i])
 i += 1
 continue

 # Full statement span (this line until a period line).
 span_end = i
 while span_end < n:
 span_logical = self._logical(lines[span_end])
 if self.STATEMENT_END.search(span_logical):
 break
 span_end += 1
 span = list(range(i, min(span_end + 1, n)))
 span_logicals = [self._logical(lines[j]) for j in span]

 references_record = any(
 record_ref.search(l) for l in span_logicals
)
 references_ff_field = ff_family_active and any(
 self.FF_FIELD_PATTERN.search(l) for l in span_logicals
)

```

```

touches_dcl = any("DCL" in l.upper() for l in span_logicals)

if (references_record or references_ff_field) and not touches_dcl:
 record = self._record_of(span_logicals, unmapped)
 if not record and references_ff_field:
 record = "FFRECEAB"

 if record and record not in marked:
 cobol_record = NameNormalizer.to_cobol(
 NameNormalizer.normalize(record)
)
 first_left, first_right = self._sequences(lines[span[0]])
 output.extend(
 self._comment_marker(
 cobol_record, first_left, first_right
)
)
 marked.add(record)

 for j in span:
 output.extend(self._comment_line(lines[j]))

 commented_any = True
 i = span[-1] + 1
 continue

output.append(lines[i])
i += 1

if commented_any:
 for record in sorted(marked):
 self.messages.append(
 "Unmapped record block: commented all lines for "
 f"{record} (record-level Option B).")
)

return "\n".join(output).rstrip() + "\n"

---- discovery from ORIGINAL IDMS source -----

def _discover_unmapped_records(self) -> set[str]:
 found: set[str] = set()
 text = self.original_idms_text
 if not text:
 return found

 patterns = (
 OBTAIN_CALC_PATTERN,
 OBTAIN_CALC_REVERSED_PATTERN,
 STORE_PATTERN,
 MODIFY_PATTERN,
 ERASE_PATTERN,
)

 for raw in text.replace("\r\n", "\n").split("\n"):
 logical = self._logical(raw)
 upper = logical.upper()
 if not upper or upper.startswith("*"):
 continue
 for pattern in patterns:
 match = pattern.search(upper)
 if not match:
 continue
 if "record" not in match.groupdict():
 continue
 token = match.group("record")
 if not token:
 continue
 normalized = NameNormalizer.normalize(token)
 if not normalized:
 continue
 columns = self.column_name_resolver.columns_for_record(
 normalized
)
 if columns:
 continue

```

```

 found.add(normalized.upper())

 return found

---- helpers -----

def _record_of(
 self,
 span_logicals: list[str],
 unmapped: set[str],
) -> str:
 for logical in span_logicals:
 for match in self.OF_RECORD_PATTERN.finditer(logical):
 token = NameNormalizer.normalize(
 match.group("record")
).upper()
 if token in unmapped:
 return token
 return ""

def _sequences(self, line: str) -> tuple[str, str]:
 """Return (left6, right8) sequence numbers from a line, padded."""
 raw = str(line or "").rstrip("\n")
 left_match = self.LEFT_SEQ.match(raw)
 left = left_match.group("left") if left_match else "000000"
 right_match = self.RIGHT_SEQ.search(raw)
 right = right_match.group("seq") if right_match else "00000000"
 return left, right

def _original_body(self, line: str) -> str:
 """Return columns 8-72 body of the original fixed-format line."""
 raw = str(line or "").rstrip("\n")
 text = re.sub(r"\s*\d{8}\s*$", "", raw) # drop right seq
 if len(text) >= 8:
 return text[7:72].rstrip()
 # Unsequenced line: drop a leading 6-digit left seq if present.
 if len(text) >= 6 and text[:6].isdigit():
 return text[6:].rstrip()
 return text.rstrip()

def _build_comment_lines(
 self,
 body: str,
 left: str,
 right: str,
) -> list[str]:
 """
 Build one or more 80-column comment lines for a body, wrapping at
 the 65-char body window (cols 8-72). Continuation lines repeat the
 '*' indicator and keep the same left/right sequence numbers.
 """
 clean = body.rstrip()
 if not clean:
 clean = ""

 chunks: list[str] = []
 if len(clean) <= BODY_WIDTH:
 chunks.append(clean)
 else:
 # Preserve leading indentation on the first chunk; wrap the
 # remainder with a continuation indent so it stays readable.
 indent_len = len(clean) - len(clean.lstrip(" "))
 indent = " " * min(indent_len, 8)
 remaining = clean
 first = True
 while remaining:
 if first:
 piece = remaining[:BODY_WIDTH]
 remaining = remaining[BODY_WIDTH:]
 first = False
 else:
 room = BODY_WIDTH - len(indent)
 piece = indent + remaining[:room]
 remaining = remaining[room:]
 chunks.append(piece)

```

```

out: list[str] = []
left6 = (left or "000000")[:LEFT_WIDTH].zfill(LEFT_WIDTH)
right8 = (right or "00000000")[:RIGHT_WIDTH].zfill(RIGHT_WIDTH)
for chunk in chunks:
 body_area = chunk[:BODY_WIDTH].ljust(BODY_WIDTH)
 out.append(f"{left6}*{body_area}{right8}")
return out

def _comment_line(self, line: str) -> list[str]:
 left, right = self._sequences(line)
 body = self._original_body(line)
 return self._build_comment_lines(body, left, right)

def _comment_marker(
 self,
 cobol_record: str,
 left: str,
 right: str,
) -> list[str]:
 body = f" {cobol_record} - no DB2 columns, map manually."
 return self._build_comment_lines(body, left, right)

@staticmethod
def _logical(line: str) -> str:
 text = str(line or "")
 text = re.sub(r"^s*\d{6}s?", "", text)
 text = re.sub(r"s*\d{8}s*$", "", text)
 return text.strip()

```

```

=====
FILE: src\idms_db2_phase2\composers\update_program_feedback_composer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.composers.update_sql_cleanup_composer import (
 UpdateSqlCleanupComposer,
)

from idms_db2_phase2.composers.update_program_structure_feedback_composer import (
 UpdateProgramStructureFeedbackComposer,
)

from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver

```

```

class UpdateProgramFeedbackComposer:
 """
 Public update-program feedback composer.

 Keeps a stable import point while delegating to focused smaller files.
 """

 def __init__(
 self,
 mapping_repository: MappingRepository,
 dclgen_repository: DclgenRepository,
 table_name_resolver: TableNameResolver,
 host_variable_resolver: HostVariableResolver,
) -> None:
 self.sql_feedback = UpdateSqlCleanupComposer(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_name_resolver,
 host_variable_resolver=host_variable_resolver,
)
 self.structure_feedback = UpdateProgramStructureFeedbackComposer(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_name_resolver,
 host_variable_resolver=host_variable_resolver,
)

```

```

 self.messages: list[str] = []

 def compose(
 self,
 text: str,
) -> str:
 self.messages = []
 output = str(text or "")

 if not output.strip():
 return output

 output = self.sql_feedback.compose(output)
 self.messages.extend(self.sql_feedback.messages)

 output = self.structure_feedback.compose(output)
 self.messages.extend(self.structure_feedback.messages)

 return output.rstrip() + "\n"
=====
FILE: src\idms_db2_phase2\composers\update_program_structure_feedback_composer.py
ENCODING: utf-8
=====

from __future__ import annotations

import re

from idms_db2_phase2.composers.update_sql_cleanup_base import (
 UpdateSqlCleanupBase,
)
from idms_db2_phase2.services.name_normalizer import NameNormalizer

class UpdateProgramStructureFeedbackComposer(UpdateSqlCleanupBase):
 """
 Fixes update-program structure feedback issues:
 - DCLGEN INCLUDE placement.
 - timestamp paragraph fall-through.
 """

 def compose(
 self,
 text: str,
) -> str:
 self.messages = []
 output = str(text or "")

 if not output.strip():
 return output

 output = self._move_dclgen_includes_to_infrastructure(output)
 output = self._prevent_timestamp_fallthrough(output)

 return output.rstrip() + "\n"

 def _move_dclgen_includes_to_infrastructure(
 self,
 text: str,
) -> str:
 lines = text.splitlines()
 output: list[str] = []
 includes_to_move: list[str] = []
 index = 0
 changed = False

 while index < len(lines):
 block, next_index = self._collect_exec_sql_block(lines, index)

 if not block:
 output.append(lines[index])
 index += 1
 continue

```

```

 include = self._include_name_from_block(block)

 if include and include not in {"SQLCA", "SQLERRWS", "SQLERROR"}:
 if self._is_after_procedure(lines, index):
 includes_to_move.append(include)
 changed = True
 index = next_index
 continue

 output.extend(block)
 index = next_index

 if not includes_to_move:
 return text

 output = self._insert_includes_near_sqlca(output, includes_to_move)

 if changed:
 self.messages.append(
 "Update structure feedback: moved DCLGEN includes into DB2 infrastructure."
)

 return "\n".join(output).rstrip() + "\n"

def _prevent_timestamp_fallthrough(
 self,
 text: str,
) -> str:
 lines = text.splitlines()
 timestamp_index = -1
 stop_index = -1

 for index, line in enumerate(lines):
 logical = self._logical(line)

 if timestamp_index < 0 and self.TIMESTAMP_PARAGRAPH_PATTERN.match(logical):
 timestamp_index = index

 if stop_index < 0 and self.STOP_RUN_PATTERN.match(logical):
 stop_index = index

 if timestamp_index < 0 or stop_index < 0:
 return text

 if stop_index < timestamp_index:
 return text

 stop_line = lines.pop(stop_index)

 if stop_index < timestamp_index:
 timestamp_index -= 1

 lines.insert(timestamp_index, stop_line)

 self.messages.append(
 "Update structure feedback: moved STOP RUN before timestamp paragraph."
)

 return "\n".join(lines).rstrip() + "\n"

def _collect_exec_sql_block(
 self,
 lines: list[str],
 index: int,
) -> tuple[list[str], int]:
 if index >= len(lines):
 return [], index

 if not self.EXEC_SQL_START_PATTERN.match(self._logical(lines[index])):
 return [], index

 output: list[str] = []

 while index < len(lines):
 output.append(lines[index])
 logical = self._logical(lines[index])

```

```

 index += 1

 if self.EXEC_SQL_END_PATTERN.match(logical):
 break

 return output, index

def _include_name_from_block(
 self,
 block: list[str],
) -> str:
 for line in block:
 logical = self._logical(line)
 match = self.INCLUDE_PATTERN.match(logical)

 if match:
 return NameNormalizer.normalize(match.group("include"))

 return ""

def _is_after_procedure(
 self,
 lines: list[str],
 index: int,
) -> bool:
 for prior in lines[:index]:
 if self._logical(prior).upper().startswith("PROCEDURE DIVISION"):
 return True

 return False

def _insert_includes_near_sqlca(
 self,
 lines: list[str],
 includes: list[str],
) -> list[str]:
 existing = self._existing_includes("\n".join(lines))
 unique: list[str] = []

 for include in includes:
 clean = NameNormalizer.normalize(include)

 if not clean:
 continue
 if clean in existing:
 continue
 if clean in unique:
 continue

 unique.append(clean)

 if not unique:
 return lines

 insert_index = self._include_insert_index(lines)
 include_lines: list[str] = []

 for include in unique:
 include_lines.extend(
 [
 " EXEC SQL",
 f" INCLUDE {include}",
 " END-EXEC.",
]
)

 if insert_index < 0:
 return include_lines + lines

 return lines[:insert_index] + include_lines + lines[insert_index:]

def _include_insert_index(
 self,
 lines: list[str],
) -> int:
 last_include_end = -1

```



```

index = 0

while index < len(lines):
 block, next_index = self._collect_exec_sql_block(lines, index)

 if block:
 include = self._include_name_from_block(block)

 if include and include != "SQLERROR":
 last_include_end = next_index

 index = next_index
 continue

 if self.SQL_LOCATION_PATTERN.match(self._logical(lines[index])):
 return index

 if self.LINKAGE_SECTION_PATTERN.match(self._logical(lines[index])):
 return index

 index += 1

return last_include_end

def _existing_includes(
 self,
 text: str,
) -> set[str]:
 output: set[str] = set()

 for match in re.finditer(
 r"\bINCLUDE\s+([A-Z0-9]+\b",
 text,
 flags=re.IGNORECASE,
):
 include = NameNormalizer.normalize(match.group(1))

 if include:
 output.add(include)

 return output

```

```

=====
FILE: src\idms_db2_phase2\composers\update_restart_skip_composer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from patterns.sequence_patterns import strip_sequence_numbers
from patterns.update_restart_skip_patterns import (
 CONTINUE_PATTERN,
 CONVERTED_FOR_PATTERN,
 DB2_COMMENT_PATTERN,
 END_IF_PATTERN,
 REMOVED_OBTAIN_CALC_PATTERN,
 SKIPPED_FOR_PATTERN,
 SQLCODE_IF_PATTERN,
)
from rules.update_restart_skip_rules import (
 EMIT_MANUAL_REDESIGN_MESSAGE,
 MANUAL_REDESIGN_MESSAGE_TEMPLATE,
 MISSING_MAPPING_MARKERS,
 REPLACEMENT_BLOCK_TEMPLATES,
 RESTART_CONTROL_HINTS,
)

class UpdateRestartSkipComposer:
 """Cleans generated missing-mapping blocks for unmapped restart/control
 records.

 Generic rule:
 - Do not hardcode any record to a DB2 restart table.
 - Do not invent DB2 restart SQL.
 - Replace skipped/converted-with-missing-mapping restart/control blocks

```

```

 with a manual-redesign comment and CONTINUE.
- Remove SQLCODE checks after skipped SQL (no SQL was executed).

Regex lives in patterns/update_restart_skip_patterns.py; markers, hints,
the replacement block, and messages in
rules/update_restart_skip_rules.py.
"""

Public mirror of rules.update_restart_skip_rules.
#
The manual-redesign comment written into the generated COBOL is the
authoritative signal. Repeating it as a validation message on every
run adds noise for an input-data gap that is not a converter
defect, so the diagnostic is opt-in.
#
Read this attribute rather than importing the constant directly, so
a per-run override is honoured.
EMIT_MANUAL_REDESIGN_MESSAGE = EMIT_MANUAL_REDESIGN_MESSAGE

def __init__(
 self,
 emit_manual_redesign_message: bool | None = None,
) -> None:
 self.messages: list[str] = []

 if emit_manual_redesign_message is not None:
 self.EMIT_MANUAL_REDESIGN_MESSAGE = bool(
 emit_manual_redesign_message
)

def compose(self, text: str) -> str:
 self.messages = []
 if not text:
 return ""

 lines = self._normalize_line_endings(text).splitlines()
 output: list[str] = []
 index = 0

 while index < len(lines):
 logical = self._logical(lines[index])

 removed_obtain_match = REMOVED_OBTAIN_CALC_PATTERN.match(logical)
 skipped_match = SKIPPED_FOR_PATTERN.match(logical)
 converted_match = CONVERTED_FOR_PATTERN.match(logical)

 if removed_obtain_match:
 index = self._handle_restart_block(
 lines=lines,
 index=index,
 output=output,
 record_name=removed_obtain_match.group("record"),
 require_missing_marker=False,
)
 continue

 if skipped_match:
 index = self._handle_restart_block(
 lines=lines,
 index=index,
 output=output,
 record_name=skipped_match.group("record"),
 require_missing_marker=False,
)
 continue

 if converted_match:
 index = self._handle_restart_block(
 lines=lines,
 index=index,
 output=output,
 record_name=converted_match.group("record"),
 require_missing_marker=True,
)
 continue

 output.append(lines[index])

```

```

 index += 1

 return "\n".join(output).rstrip() + "\n"

def _handle_restart_block(
 self,
 *,
 lines: list[str],
 index: int,
 output: list[str],
 record_name: str,
 require_missing_marker: bool,
) -> int:
 record = str(record_name or "").upper()

 if not self._looks_like_restart_or_control(record):
 output.append(lines[index])
 return index + 1

 block_end = self._block_end(lines=lines, start_index=index)

 if require_missing_marker and not self._contains_missing_mapping_marker(
 lines[index:block_end]
):
 output.append(lines[index])
 return index + 1

 output.extend(self._replacement_block(record))
 next_index = self._skip_following_sqlcode_blocks(
 lines=lines, start_index=block_end
)
 self._note_manual_redesign(record)
 return next_index

def _block_end(self, lines: list[str], start_index: int) -> int:
 """Advance past a generated comment block ending in CONTINUE.

 Shared by the removed-OBTAIN-CALC, skipped, and converted cases
 (their scanning logic is identical).
 """
 index = start_index + 1

 while index < len(lines):
 logical = self._logical(lines[index])

 if not logical:
 index += 1
 continue
 if DB2_COMMENT_PATTERN.match(logical):
 index += 1
 continue
 if CONTINUE_PATTERN.match(logical):
 index += 1
 break

 break

 return index

def _contains_missing_mapping_marker(self, lines: list[str]) -> bool:
 combined = "\n".join(self._logical(line).upper() for line in lines)
 return any(marker in combined for marker in MISSING_MAPPING_MARKERS)

def _note_manual_redesign(self, record_name: str) -> None:
 """Record that a restart block was replaced.

 Suppressed by default. The generated COBOL already carries the
 manual-redesign comment, which is the signal the COBOL team acts
 on; the validation message is duplicate noise unless a run is
 specifically auditing skipped records.
 """
 if not self.EMIT_MANUAL_REDESIGN_MESSAGE:
 return

 record = str(record_name or "").strip().upper()
 if not record:

```

```

 return

 message = MANUAL_REDESIGN_MESSAGE_TEMPLATE.format(record=record)
 if message not in self.messages:
 self.messages.append(message)

 def _replacement_block(self, record_name: str) -> list[str]:
 return [
 template.format(record=record_name)
 for template in REPLACEMENT_BLOCK_TEMPLATES
]

 def _looks_like_restart_or_control(self, record_name: str) -> bool:
 normalized = str(record_name or "").upper()
 return any(hint in normalized for hint in RESTART_CONTROL_HINTS)

 def _skip_following_sqlcode_blocks(
 self, lines: list[str], start_index: int
) -> int:
 index = start_index
 while index < len(lines):
 if not SQLCODE_IF_PATTERN.match(self._logical(lines[index])):
 break
 index = self._skip_if_block(lines=lines, start_index=index)
 return index

 def _skip_if_block(self, lines: list[str], start_index: int) -> int:
 index = start_index
 while index < len(lines):
 logical = self._logical(lines[index])
 index += 1
 if END_IF_PATTERN.match(logical):
 break
 return index

 def _logical(self, line: str) -> str:
 return strip_sequence_numbers(str(line or "")).strip()

 def _normalize_line_endings(self, text: str) -> str:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

```

```

=====
FILE: src\idms_db2_phase2\composers\update_sql_cleanup__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\composers\update_sql_cleanup\bare_move_rewriter.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/update_sql_cleanup/bare_move_rewriter.py
ACTION: CREATE NEW FILE

```

```

"""Rewrites bare 'MOVE ... TO field' targets before a MODIFY into DB2 host moves."""

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.composers.update_sql_cleanup.date_move_builder import (
 DateMoveBuilder,
)
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class BareMoveRewriter:
 def __init__(self, owner, scanner, date_builder: DateMoveBuilder) -> None:
 self.owner = owner
 self.scanner = scanner
 self.date_builder = date_builder

 def rewrite(self, text: str) -> str:
 lines = text.splitlines()

```

```

output: list[str] = []
changed = False

for index, line in enumerate(lines):
 logical = self.owner._logical(line)
 match = self.owner.MOVE_TO_BARE_FIELD_PATTERN.match(logical)

 if not match:
 output.append(line)
 continue

 target = NameNormalizer.to_cobol(match.group("tgt"))

 if self.owner._is_protected_bare_target(target):
 output.append(line)
 continue

 record = self.scanner.next_modify_record(
 lines, index, max_distance=15
)
 if not record:
 output.append(line)
 continue

 table = self.owner._table_for_record(record)
 column = self.owner._column_for_source_field(record, target)

 if not table or not column:
 output.append(line)
 continue

 host_key = self.owner._host_reference_key(table, column)
 if not host_key:
 output.append(line)
 continue

 leading = self.owner._leading_spaces(line)
 source_value = match.group("src").strip()

 if self.date_builder.is_db2_date_column(column):
 output.extend(
 self.date_builder.date_ymd8_to_db2_external_move(
 source_value=source_value,
 host_key=host_key,
 leading=leading,
)
)
 else:
 output.append(f"{leading}MOVE {source_value} TO {host_key}")

 changed = True

if changed:
 self.owner.messages.append(
 "Update SQL cleanup: resolved bare MOVE target through "
 "Sheet Mapping and DCLGEN."
)

return "\n".join(output).rstrip() + "\n"

```

```

=====
FILE: src\idms_db2_phase2\composers\update_sql_cleanup\column_mixin.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/update_sql_cleanup/column_mixin.py
ACTION: CREATE NEW FILE

```

```

"""

```

Column resolution mixin for update-program SQL cleanup.

Deterministic: delegates source-field -> DB2 column resolution to the  
Sheet Mapping repository (the authority). No fuzzy / similarity matching.  
"""

```

from __future__ import annotations

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.update_sql_cleanup_rules import (
 INSERT_ONLY_AUDIT_PREFIXES,
 UPDATE_AUDIT_PREFIXES,
)

class ColumnMixin:
 def _column_for_source_field(
 self,
 record: str,
 source_field: str,
) -> str:
 """
 Resolve source field to DB2 column via the Sheet Mapping repository
 (deterministic authority). On no match, fail loud so a human corrects
 the mapping instead of the tool guessing.
 """
 column = self.mapping_repository.column_for_source_field(
 record_name=record,
 source_field_name=source_field,
)

 if column:
 return NameNormalizer.normalize(column)

 self.messages.append(
 f"Mapping gap: source field '{source_field}' in record {record} "
 f"has no deterministic DB2 column. Manual mapping required."
)
 return ""

 def _update_audit_columns(
 self,
 record: str,
 table: str,
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for row in self.mapping_repository.rows_for_record(record):
 column = NameNormalizer.normalize(
 getattr(row, "new_db2_field_name", "")
)
 if not column:
 continue
 if column.startswith(INSERT_ONLY_AUDIT_PREFIXES):
 continue
 if not column.startswith(UPDATE_AUDIT_PREFIXES):
 continue
 if column in seen:
 continue
 seen.add(column)
 output.append(column)

 return self._filter_existing_dclgen_columns(table, output)

 def _filter_existing_dclgen_columns(
 self,
 table: str,
 columns: list[str],
) -> list[str]:
 valid_columns = set(
 self.dclgen_repository.column_names_for_table(table)
)
 output: list[str] = []
 seen: set[str] = set()

 for column in columns:
 normalized = NameNormalizer.normalize(column)
 if not normalized:
 continue
 if normalized not in valid_columns:
 continue

```

```

 if normalized in seen:
 continue
 if not self.host_variable_resolver.has_host_for_column(
 table_name=table,
 column_name=normalized,
):
 continue
 seen.add(normalized)
 output.append(normalized)

 return output

```

```

=====
FILE: src\idms_db2_phase2\composers\update_sql_cleanup\composer_patterns.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/update_sql_cleanup/composer_patterns.py
ACTION: CREATE NEW FILE

```

```

"""
Regex patterns used only by the update SQL cleanup composer passes.

(The shared base patterns live in update_sql_cleanup_patterns.py.)
"""

```

```
import re
```

```

MOVE_TO_DCL_DOT_HOST_PATTERN = re.compile(
 r"^\s*MOVE\s+(?P<src>.+?)\s+TO\s+(?P<group>DCL[A-Z0-9-]+)\.(?P<host>[A-Z][A-Z0-9-]+)\.?\s*$",
 flags=re.IGNORECASE,
)

```

```

STRING INTO_DCL_HOST_PATTERN = re.compile(
 r"^\s*INTO\s+(?P<host>[A-Z][A-Z0-9-]+)\s+OF\s+(?P<group>DCL[A-Z0-9-]+)\.?\s*$",
 flags=re.IGNORECASE,
)

```

```

STRING INTO_DCL_DOT_HOST_PATTERN = re.compile(
 r"^\s*INTO\s+(?P<group>DCL[A-Z0-9-]+)\.(?P<host>[A-Z][A-Z0-9-]+)\.?\s*$",
 flags=re.IGNORECASE,
)

```

```

ANY_DCL_OF_REFERENCE_PATTERN = re.compile(
 r"\b(?:P<host>[A-Z][A-Z0-9-]+)\s+OF\s+(?P<group>DCL[A-Z0-9-]+)\b",
 flags=re.IGNORECASE,
)

```

```

ANY_DCL_DOT_REFERENCE_PATTERN = re.compile(
 r"\b(?:P<group>DCL[A-Z0-9-]+)\.(?P<host>[A-Z][A-Z0-9-]+)\b",
 flags=re.IGNORECASE,
)

```

```

DATE_COLUMN_PREFIXES = (
 "DA_",
 "DT_",
)

```

```

=====
FILE: src\idms_db2_phase2\composers\update_sql_cleanup\date_move_builder.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/update_sql_cleanup/date_move_builder.py
ACTION: REPLACE ENTIRE FILE

```

```

"""
DB2 date move builder (update SQL cleanup).

```

```

Builds the COBOL move sequence that converts a CCYYMMDD source value into the
DB2 external date format DD.MM.CCYY before moving it into a DB2 date host
field. Uses the DB2 low-date sentinel (00010101) for null dates.
"""

```

```

from __future__ import annotations

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.sql_generation_rules import (
 DATE_COLUMN_PREFIXES,
 DATE_STAGING_FIELDS,
 DB2_DATE_NULL_SENTINEL,
)

class DateMoveBuilder:

 def is_db2_date_column(self, column_name: str) -> bool:
 column = NameNormalizer.normalize(column_name)
 return any(
 column.startswith(prefix) for prefix in DATE_COLUMN_PREFIXES
)

 @staticmethod
 def _is_staging_field(source: str) -> bool:
 """True when the source is already a DB2 date staging field.

 A staging field is this builder's OUTPUT, not a valid input. Passing
 DA-DD-MM-CCYY back in moves 10 bytes into an 8-byte group, truncating
 the year, and the CORR move back writes garbage into the DB2 host.
 """
 return str(source or "").strip().upper() in DATE_STAGING_FIELDS

 def date_ymd8_to_db2_external_move(
 self,
 source_value: str,
 host_key: str,
 leading: str,
) -> list[str]:
 """Convert CCYYMMDD into DB2 external date format DD.MM.CCYY.

 Manual standard:
 - Clear DA-CCYYMMDD.
 - Move the source date into DA-CCYYMMDD.
 - If the date is ZERO or SPACES, substitute the DB2 low-date sentinel
 (00010101) instead of moving SPACES to the target.
 - Realign via DA-CCYYMMDD-R -> DA-DD-MM-CCYY.

 Precondition: source_value must be a CCYYMMDD (8-position) field.
 """
 source = str(source_value or "").strip()
 indent = leading if leading else " "

 # Already converted. Re-running the block would destroy the value.
 if self._is_staging_field(source):
 return [
 f"{indent}MOVE DA-DD-MM-CCYY TO {host_key}",
]

 if not source:
 return [
 f"{indent}MOVE ZEROES TO DA-CCYYMMDD",
 f"{indent}MOVE {DB2_DATE_NULL_SENTINEL} TO DA-CCYYMMDD",
 f"{indent}MOVE CORR DA-CCYYMMDD-R TO DA-DD-MM-CCYY",
 f"{indent}MOVE DA-DD-MM-CCYY TO {host_key}",
]

 return [
 f"{indent}MOVE ZEROES TO DA-CCYYMMDD",
 f"{indent}MOVE {source} TO DA-CCYYMMDD",
 f"{indent}IF DA-CCYYMMDD = ZERO OR DA-CCYYMMDD = SPACES",
 f"{indent} MOVE {DB2_DATE_NULL_SENTINEL} TO DA-CCYYMMDD",
 f"{indent}END-IF",
 f"{indent}MOVE CORR DA-CCYYMMDD-R TO DA-DD-MM-CCYY",
 f"{indent}MOVE DA-DD-MM-CCYY TO {host_key}",
]

```



```
=====
FILE: src\idms_db2_phase2\composers\update_sql_cleanup\key_column_mixin.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/update_sql_cleanup/key_column_mixin.py
ACTION: CREATE NEW FILE
```

```
"""Primary-key column resolution mixin for update-program SQL cleanup."""
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.update_sql_cleanup_rules import IDENTITY_KEY_PREFIXES
```

```
class KeyColumnMixin:
 def _db2_primary_key_columns(
 self,
 record: str,
 table: str,
) -> list[str]:
 """
 Return DB2 primary-key columns for SQL WHERE.

 1. Use Sheet Mapping rows marked DB2 PRIMARY/KEY.
 2. Remove rows marked as foreign/relationship.
 3. If multiple primary columns exist and a DB2 identity-style key is
 present, narrow to identity-style keys.
 4. Otherwise fall back to repository key metadata.
 """
 strict_keys = self._strict_db2_primary_key_columns(record)
 strict_keys = self._filter_existing_dclgen_columns(table, strict_keys)

 if len(strict_keys) <= 1:
 return strict_keys

 identity_keys = [
 column
 for column in strict_keys
 if column.startswith(IDENTITY_KEY_PREFIXES)
]

 if identity_keys:
 return identity_keys

 return strict_keys

 def _strict_db2_primary_key_columns(
 self,
 record: str,
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for row in self.mapping_repository.rows_for_record(record):
 db2_key = NameNormalizer.normalize(getattr(row, "db2_key", ""))
 relation = NameNormalizer.normalize(getattr(row, "relation", ""))
 column = NameNormalizer.normalize(
 getattr(row, "new_db2_field_name", "")
)

 if not column:
 continue
 if "FOREIGN" in relation:
 continue
 if "FOREIGN" in db2_key:
 continue
 if "PRIMARY" not in db2_key and db2_key != "KEY":
 continue
 if column in seen:
 continue

 seen.add(column)
 output.append(column)
```

```

 if output:
 return output

 if hasattr(self.mapping_repository, "key_columns_for_record"):
 return self.mapping_repository.key_columns_for_record(record)

 return []

=====
FILE: src\idms_db2_phase2\composers\update_sql_cleanup\modify_update_rewriter.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/update_sql_cleanup/modify_update_rewriter.py
ACTION: CREATE NEW FILE

"""Rewrites generated MODIFY markers into conservative DB2 UPDATE blocks."""

from __future__ import annotations

from idms_db2_phase2.composers.update_sql_cleanup.composer_patterns import (
 ANY_DCL_DOT_REFERENCE_PATTERN,
 ANY_DCL_OF_REFERENCE_PATTERN,
 MOVE_TO_DCL_DOT_HOST_PATTERN,
 STRING_INTO_DCL_DOT_HOST_PATTERN,
 STRING_INTO_DCL_HOST_PATTERN,
)
from idms_db2_phase2.services.name_normalizer import NameNormalizer

ADD to the imports at the top of modify_update_rewriter.py:
from rules.update_sql_cleanup_rules import UPDATE_QUERYNO

class ModifyUpdateRewriter:
 def __init__(self, owner, scanner) -> None:
 self.owner = owner
 self.scanner = scanner

 def rewrite(self, text: str) -> str:
 lines = text.splitlines()
 output: list[str] = []
 index = 0
 changed = False

 while index < len(lines):
 logical = self.owner._logical(lines[index])
 match = self.owner.CONVERTED_MODIFY_PATTERN.match(logical)

 if not match:
 output.append(lines[index])
 index += 1
 continue

 record = NameNormalizer.normalize(match.group("record"))
 table = self.owner._table_for_record(record)
 key_columns = self.owner._db2_primary_key_columns(record, table)

 changed_columns = self._changed_columns_after_obtain_calc(
 record=record,
 table=table,
 lines=lines,
 modify_index=index,
)
 update_columns = self._update_columns(
 record=record,
 table=table,
 changed_columns=changed_columns,
)

 if not table or not key_columns or not update_columns:
 output.append(lines[index])
 index += 1
 continue

 block_end = self.scanner.skip_generated_sql_and_sqlcode(
 lines, index + 1

```

```

)
 output.extend(
 self._conservative_update_block(
 record=record,
 table=table,
 update_columns=update_columns,
 key_columns=key_columns,
 leading=self.owner._leading_spaces(lines[index]),
)
)
 changed = True
 index = block_end

if changed:
 self.owner.messages.append(
 "Update SQL cleanup: rewrote MODIFY UPDATE to changed field "
 "plus update audit fields."
)

return "\n".join(output).rstrip() + "\n"

def _conservative_update_block(
 self,
 record: str,
 table: str,
 update_columns: list[str],
 key_columns: list[str],
 leading: str,
) -> list[str]:
 lines = [
 f"{leading}*DB2: Converted MODIFY for "
 f"{NameNormalizer.to_cobol(record)}.",
 f"{leading}MOVE 'UPDATE-{NameNormalizer.to_cobol(record)}' "
 f"TO SQL-LOCATION.",
 f"{leading}EXEC SQL",
 f"{leading} UPDATE {table}",
 f"{leading} SET",
]
 lines.extend(
 self.owner._set_lines(table, update_columns, leading + " ")
)
 lines.append(f"{leading} WHERE")
 lines.extend(
 self.owner._where_lines(table, key_columns, leading + " ")
)
 lines.append(f"{leading} QUERYNO {UPDATE_QUERYNO}")
 lines.append(f"{leading}END-EXEC")

 # Manual-standard SQLCODE handling: EVALUATE SQLCODE.
 lines.append(f"{leading}EVALUATE SQLCODE")
 lines.append(f"{leading} WHEN 0")
 lines.append(f"{leading} CONTINUE")
 lines.append(f"{leading} WHEN OTHER")
 lines.append(
 f"{leading} DISPLAY 'ERROR ON UPDATE {table}'"
)
 lines.append(f"{leading} PERFORM SQLERROR")
 lines.append(f"{leading}END-EVALUATE")
 lines.append(f"{leading}.")

 return lines

def _changed_columns_after_obtain_calc(
 self,
 record: str,
 table: str,
 lines: list[str],
 modify_index: int,
) -> list[str]:
 start = self._post_select_scan_start(lines, modify_index)
 return self._changed_columns_in_range(
 record=record,
 table=table,
 lines=lines,
 start=start,
 end=modify_index,
)

```

```

)

def _post_select_scan_start(
 self,
 lines: list[str],
 modify_index: int,
) -> int:
 search_start = max(0, modify_index - 80)
 latest_boundary = -1

 for index in range(search_start, modify_index):
 logical = self.owner._logical(lines[index])
 upper = logical.upper()

 if self.owner.EXEC_SQL_END_PATTERN.match(logical):
 latest_boundary = index
 continue
 if "REMOVED OBTAIN CALC SELECT" in upper:
 latest_boundary = index
 continue
 if "DIRECT UPDATE WILL USE MAPPED COMPOSITE KEY" in upper:
 latest_boundary = index
 continue

 if latest_boundary >= 0:
 return latest_boundary + 1

 return max(0, modify_index - 20)

def _changed_columns_in_range(
 self,
 record: str,
 table: str,
 lines: list[str],
 start: int,
 end: int,
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()
 in_string_block = False

 for index in range(start, end):
 logical = self.owner._logical(lines[index])
 upper = logical.upper()

 if upper.startswith("STRING "):
 in_string_block = True

 column = self._changed_column_from_logical(
 record=record,
 table=table,
 logical=logical,
 in_string_block=in_string_block,
)
 if column and column not in seen:
 seen.add(column)
 output.append(column)

 if upper.startswith("END-STRING"):
 in_string_block = False

 return self.owner._filter_existing_dclgen_columns(table, output)

def _changed_column_from_logical(
 self,
 record: str,
 table: str,
 logical: str,
 in_string_block: bool = False,
) -> str:
 bare_match = self.owner.MOVE_TO_BARE_FIELD_PATTERN.match(logical)
 if bare_match:
 target = NameNormalizer.to_cobol(bare_match.group("tgt"))
 return self.owner._column_for_source_field(record, target)

 of_match = self.owner.MOVE_TO_DCL_HOST_PATTERN.match(logical)

```

```

if of_match:
 return self._column_for_host(
 table=table,
 group=NameNormalizer.normalize(of_match.group("group")),
 host=NameNormalizer.to_cobol(of_match.group("host")),
)

dot_match = MOVE_TO_DCL_DOT_HOST_PATTERN.match(logical)
if dot_match:
 return self._column_for_host(
 table=table,
 group=NameNormalizer.normalize(dot_match.group("group")),
 host=NameNormalizer.to_cobol(dot_match.group("host")),
)

string_into_match = STRING_INTTO_DCL_HOST_PATTERN.match(logical)
if string_into_match:
 return self._column_for_host(
 table=table,
 group=NameNormalizer.normalize(
 string_into_match.group("group")
),
 host=NameNormalizer.to_cobol(string_into_match.group("host")),
)

string_dot_match = STRING_INTTO_DCL_DOT_HOST_PATTERN.match(logical)
if string_dot_match:
 return self._column_for_host(
 table=table,
 group=NameNormalizer.normalize(
 string_dot_match.group("group")
),
 host=NameNormalizer.to_cobol(string_dot_match.group("host")),
)

if in_string_block and "INTO" in logical.upper():
 generic_of_match = ANY_DCL_OF_REFERENCE_PATTERN.search(logical)
 if generic_of_match:
 return self._column_for_host(
 table=table,
 group=NameNormalizer.normalize(
 generic_of_match.group("group")
),
 host=NameNormalizer.to_cobol(
 generic_of_match.group("host")
),
)

 generic_dot_match = ANY_DCL_DOT_REFERENCE_PATTERN.search(logical)
 if generic_dot_match:
 return self._column_for_host(
 table=table,
 group=NameNormalizer.normalize(
 generic_dot_match.group("group")
),
 host=NameNormalizer.to_cobol(
 generic_dot_match.group("host")
),
)

return ""

def _column_for_host(
 self,
 table: str,
 group: str,
 host: str,
) -> str:
 expected_group = NameNormalizer.normalize(
 self.owner.dclgen_repository.group_for_table(table)
)
 if group != expected_group:
 return ""

 target_host = NameNormalizer.to_cobol(host)

```

```

 for column in self.owner.dclgen_repository.columns_for_table(table):
 current_host = NameNormalizer.to_cobol(
 column.cobol_host_name or column.column_name
)
 if current_host == target_host:
 return NameNormalizer.normalize(column.column_name)

 return ""

def _update_columns(
 self,
 record: str,
 table: str,
 changed_columns: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()
 key_columns = set(
 self.owner._db2_primary_key_columns(record, table)
)

 for column in changed_columns:
 normalized = NameNormalizer.normalize(column)
 if not normalized or normalized in key_columns or normalized in seen:
 continue
 seen.add(normalized)
 output.append(normalized)

 for column in self.owner._update_audit_columns(record, table):
 normalized = NameNormalizer.normalize(column)
 if not normalized or normalized in key_columns or normalized in seen:
 continue
 seen.add(normalized)
 output.append(normalized)

 return self.owner._filter_existing_dclgen_columns(table, output)

```

```

=====
FILE: src\idms_db2_phase2\composers\update_sql_cleanup\obtain_calc_remover.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/update_sql_cleanup/obtain_calc_remover.py
ACTION: CREATE NEW FILE

```

```

"""Removes generated OBTAIN CALC SELECT blocks before direct UPDATE."""

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class ObtainCalcRemover:
 def __init__(self, owner, scanner) -> None:
 self.owner = owner
 self.scanner = scanner

 def rewrite(self, text: str) -> str:
 lines = text.splitlines()
 output: list[str] = []
 index = 0
 changed = False

 while index < len(lines):
 logical = self.owner._logical(lines[index])
 match = self.owner.CONVERTED_OBTAIN_CALC_PATTERN.match(logical)

 if not match:
 output.append(lines[index])
 index += 1
 continue

 record = NameNormalizer.normalize(match.group("record"))
 cobol_record = NameNormalizer.to_cobol(record)
 leading = self.owner._leading_spaces(lines[index])

```

```

table = self.owner._table_for_record(record)

block_end = self.scanner.skip_generated_sql_and_sqlcode(
 lines, index + 1
)

if not table:
 output.extend(
 [
 f"{leading}*DB2: OBTAIN CALC conversion skipped for "
 f"{cobol_record}.",
 f"{leading}*DB2: Missing Sheet Mapping or DCLGEN "
 f"table metadata.",
 f"{leading}CONTINUE.",
]
)
 changed = True
 index = block_end
 continue

key_columns = self.owner._db2_primary_key_columns(
 record=record,
 table=table,
)

if not key_columns:
 output.extend(
 [
 f"{leading}*DB2: OBTAIN CALC conversion skipped for "
 f"{cobol_record}.",
 f"{leading}*DB2: Missing key column metadata.",
 f"{leading}CONTINUE.",
]
)
 changed = True
 index = block_end
 continue

output.extend(
 [
 f"{leading}*DB2: Removed OBTAIN CALC SELECT for "
 f"{cobol_record}.",
 f"{leading}*DB2: Direct UPDATE will use mapped composite "
 f"key WHERE clause.",
 f"{leading}CONTINUE.",
]
)
changed = True
index = block_end

if changed:
 self.owner.messages.append(
 "Update SQL cleanup: removed unnecessary OBTAIN CALC SELECT "
 "before direct UPDATE."
)

return "\n".join(output).rstrip() + "\n"

```

```

=====
FILE: src\idms_db2_phase2\composers\update_sql_cleanup\scan_mixin.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/update_sql_cleanup/scan_mixin.py
ACTION: CREATE NEW FILE

```

```

"""Line/text helpers, block-scan helpers, and predicates."""

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.composers.update_sql_cleanup.update_sql_cleanup_patterns import (
 END_IF_PATTERN,
 EXEC_SQL_END_PATTERN,
)

```

```

from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

from patterns.sequence_patterns import strip_sequence_numbers
from rules.update_sql_cleanup_rules import PROTECTED_BARE_TARGET_PREFIXES

from idms_db2_phase2.composers.update_sql_cleanup.update_sql_cleanup_patterns import (
 END_EVALUATE_PATTERN,
 END_IF_PATTERN,
 EXEC_SQL_END_PATTERN,
 LONE_PERIOD_PATTERN,
)

```

```

class ScanMixin:
 def _logical(
 self,
 line: str,
) -> str:
 return strip_sequence_numbers(str(line or "")).strip()

 def _leading_spaces(
 self,
 line: str,
) -> str:
 value = str(line or "")
 return value[: len(value) - len(value.lstrip())]

 def _table_for_record(
 self,
 record: str,
) -> str:
 mapped_table = self.mapping_repository.db2_table_for_record(record)
 if not mapped_table:
 return ""
 return self.table_name_resolver.resolve_table(mapped_table)

 def _is_protected_bare_target(
 self,
 target: str,
) -> bool:
 clean = NameNormalizer.to_cobol(target).upper()
 return clean.startswith(PROTECTED_BARE_TARGET_PREFIXES)

 def _skip_exec_sql(
 self,
 lines: list[str],
 index: int,
) -> int:
 while index < len(lines):
 logical = self._logical(lines[index])
 index += 1
 if EXEC_SQL_END_PATTERN.match(logical):
 break
 return index

 def _skip_if_block(
 self,
 lines: list[str],
 index: int,
) -> int:
 while index < len(lines):
 logical = self._logical(lines[index])
 index += 1
 if END_IF_PATTERN.match(logical):
 break
 return index
 # ADD this method inside class ScanMixin (next to _skip_if_block):

 def _skip_evaluate_block(
 self,
 lines: list[str],
 index: int,
) -> int:
 """
 Skip an EVALUATE SQLCODE ... END-EVALUATE block, including a trailing
 lone period line (the COBOL scope terminator emitted after
 END-EVALUATE).
 """

```



```

while index < len(lines):
 logical = self._logical(lines[index])
 index += 1
 if END_EVALUATE_PATTERN.match(logical):
 break

Consume a following lone period line if present.
while index < len(lines):
 logical = self._logical(lines[index])
 if not logical:
 index += 1
 continue
 if LONE_PERIOD_PATTERN.match(logical):
 index += 1
 break

return index

```

```

=====
FILE: src\idms_db2_phase2\composers\update_sql_cleanup\sql_block_scanner.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/update_sql_cleanup/sql_block_scanner.py
ACTION: REPLACE ENTIRE FILE

```

```

"""
SQL block scanner.

```

```

Helpers to skip generated SQL / SQLCODE blocks and to find the next MODIFY
record marker. Uses the base class scan helpers via the owner instance.
"""

```

```

from __future__ import annotations

from idms_db2_phase2.composers.update_sql_cleanup.update_sql_cleanup_patterns import (
 SQLCODE_EVALUATE_PATTERN,
)
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class SqlBlockScanner:
 def __init__(self, owner) -> None:
 # owner exposes: _logical, _skip_exec_sql, _skip_if_block,
 # _skip_evaluate_block, EXEC_SQL_START_PATTERN, SQLCODE_IF_PATTERN,
 # CONVERTED_MODIFY_PATTERN
 self.owner = owner

 def next_modify_record(
 self,
 lines: list[str],
 start_index: int,
 max_distance: int,
) -> str:
 end = min(len(lines), start_index + max_distance + 1)
 for index in range(start_index + 1, end):
 logical = self.owner._logical(lines[index])
 match = self.owner.CONVERTED_MODIFY_PATTERN.match(logical)
 if match:
 return NameNormalizer.normalize(match.group("record"))
 return ""

 def skip_generated_sql_and_sqlcode(
 self,
 lines: list[str],
 start: int,
) -> int:
 index = start
 while index < len(lines):
 logical = self.owner._logical(lines[index])
 upper = logical.upper()

 if not logical:
 index += 1
 continue

```

```

 if upper.startswith("MOVE 'SELECT-") or upper.startswith(
 "MOVE 'UPDATE-"
):
 index += 1
 continue

 if self.owner.EXEC_SQL_START_PATTERN.match(logical):
 index = self.owner._skip_exec_sql(lines, index)
 continue

 # Skip an IF SQLCODE ... END-IF block.
 if self.owner.SQLCODE_IF_PATTERN.match(logical):
 index = self.owner._skip_if_block(lines, index)
 continue

 # Skip an EVALUATE SQLCODE ... END-EVALUATE block (+ period).
 if SQLCODE_EVALUATE_PATTERN.match(logical):
 index = self.owner._skip_evaluate_block(lines, index)
 continue

 break

return index

```

```

=====
FILE: src\idms_db2_phase2\composers\update_sql_cleanup\sql_line_mixin.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/update_sql_cleanup/sql_line_mixin.py
ACTION: CREATE NEW FILE

```

```

"""Host-reference and SET/WHERE/comma line-builder mixin."""

```

```

from __future__ import annotations

```

```

class SqlLineMixin:
 def _host_reference(
 self,
 table: str,
 column: str,
) -> str:
 return self.host_variable_resolver.host_reference_for_column(
 table_name=table,
 column_name=column,
)

 def _host_reference_key(
 self,
 table: str,
 column: str,
) -> str:
 if hasattr(self.host_variable_resolver, "host_reference_key"):
 return self.host_variable_resolver.host_reference_key(
 table_name=table,
 column_name=column,
)

 reference = self._host_reference(table, column)

 if reference.startswith(":"):
 return reference[1:].strip()

 return reference

 def _set_lines(
 self,
 table: str,
 columns: list[str],
 indent: str,
) -> list[str]:
 output: list[str] = []

```

```

 for index, column in enumerate(columns):
 host = self._host_reference(table, column)
 if not host:
 continue
 suffix = "," if index < len(columns) - 1 else ""
 output.append(f"{indent}{column} = {host}{suffix}")

 return output

def _where_lines(
 self,
 table: str,
 columns: list[str],
 indent: str,
) -> list[str]:
 output: list[str] = []

 for index, column in enumerate(columns):
 host = self._host_reference(table, column)
 if not host:
 continue
 prefix = "" if index == 0 else "AND "
 output.append(f"{indent}{prefix}{column} = {host}")

 return output

def _comma_lines(
 self,
 values: list[str],
 indent: str,
) -> list[str]:
 output: list[str] = []

 for index, value in enumerate(values):
 suffix = "," if index < len(values) - 1 else ""
 output.append(f"{indent}{value}{suffix}")

 return output

```

```

=====
FILE: src\idms_db2_phase2\composers\update_sql_cleanup\sqlerror_normalizer.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/composers/update_sql_cleanup/sqlerror_normalizer.py
ACTION: CREATE NEW FILE

```

```

"""Normalizes malformed 'PERFORM SQLERROR.END-IF.' lines."""

```

```

from __future__ import annotations

```

```

class SqlerrorNormalizer:
 def __init__(self, owner) -> None:
 self.owner = owner

 def normalize(self, text: str) -> str:
 output: list[str] = []
 changed = False

 for line in text.splitlines():
 logical = self.owner._logical(line)
 match = self.owner.MALFORMED_SQLERROR_ENDIF_PATTERN.match(logical)

 if not match:
 output.append(line)
 continue

 leading = self.owner._leading_spaces(line)
 output.append(f"{leading}PERFORM SQLERROR.")
 output.append(f"{leading}END-IF.")
 changed = True

 if changed:
 self.owner.messages.append(

```

```

 "Update SQL cleanup: normalized malformed "
 "PERFORM SQLERROR.END-IF."
)

 return "\n".join(output).rstrip() + "\n"

```

```

=====
FILE: src\idms_db2_phase2\composers\update_sql_cleanup\update_sql_cleanup_patterns.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/composers/update_sql_cleanup/update_sql_cleanup_patterns.py
ACTION: REPLACE ENTIRE FILE

"""
Update-program SQL cleanup regex patterns.

Regex belongs here, not inside the shared base class.
"""

import re

CONVERTED_OBTAIN_CALC_PATTERN = re.compile(
 r"^\s**\s*DB2:\s*Converted\s+OBTAIN\s+CALC\s+for\s+(?P<record>[A-Z0-9-]+)\.\s*$",
 flags=re.IGNORECASE,
)

CONVERTED_MODIFY_PATTERN = re.compile(
 r"^\s**\s*DB2:\s*Converted\s+MODIFY\s+for\s+(?P<record>[A-Z0-9-]+)\.\s*$",
 flags=re.IGNORECASE,
)

EXEC_SQL_START_PATTERN = re.compile(
 r"^\s*EXEC\s+SQL\b",
 flags=re.IGNORECASE,
)

EXEC_SQL_END_PATTERN = re.compile(
 r"^\s*END-EXEC\.\s*$",
 flags=re.IGNORECASE,
)

INCLUDE_PATTERN = re.compile(
 r"^\s*INCLUDE\s+(?P<include>[A-Z0-9]+)\b",
 flags=re.IGNORECASE,
)

SQLCODE_IF_PATTERN = re.compile(
 r"^\s*IF\s+SQLCODE\b",
 flags=re.IGNORECASE,
)

END_IF_PATTERN = re.compile(
 r"^\s*END-IF\.\s*$",
 flags=re.IGNORECASE,
)

MALFORMED_SQLERROR_ENDIF_PATTERN = re.compile(
 r"^(?P<indent>\s*)PERFORM\s+SQLERROR\.\s*END-IF\.\s*$",
 flags=re.IGNORECASE,
)

MOVE_TO_BARE_FIELD_PATTERN = re.compile(
 r"^\s*MOVE\s+(?P<src>.+?)\s+TO\s+(?P<tgt>[A-Z][A-Z0-9-]+)\.\s*$",
 flags=re.IGNORECASE,
)

MOVE_TO_DCL_HOST_PATTERN = re.compile(
 r"^\s*MOVE\s+(?P<src>.+?)\s+TO\s+(?P<host>[A-Z][A-Z0-9-]+)\s+OF\s+(?P<group>DCL[A-Z0-9-]+)\.\s*$",
 flags=re.IGNORECASE,
)

SQL_LOCATION_PATTERN = re.compile(
 r"^\s*\01\s+SQL-LOCATION\b",
 flags=re.IGNORECASE,
)

```

```

)

LINKAGE_SECTION_PATTERN = re.compile(
 r"^\\s*LINKAGE\\s+SECTION\\.",
 flags=re.IGNORECASE,
)

TIMESTAMP_PARAGRAPH_PATTERN = re.compile(
 r"^\\s*600-GET-TIMESTAMP\\.\\s*$",
 flags=re.IGNORECASE,
)

STOP_RUN_PATTERN = re.compile(
 r"^\\s*STOP\\s+RUN\\.\\s*$",
 flags=re.IGNORECASE,
)

--- SQLCODE handling block detection (IF and EVALUATE forms) ---

SQLCODE_EVALUATE_PATTERN = re.compile(
 r"^\\s*EVALUATE\\s+SQLCODE\\b",
 flags=re.IGNORECASE,
)

END_EVALUATE_PATTERN = re.compile(
 r"^\\s*END-EVALUATE\\.\\s*$",
 flags=re.IGNORECASE,
)

A lone period line that closes an EVALUATE block (COBOL scope terminator).
LONE_PERIOD_PATTERN = re.compile(
 r"^\\s*\\.\\s*$",
 flags=re.IGNORECASE,
)

=====
FILE: src\\idms_db2_phase2\\composers\\update_sql_cleanup_base.py
ENCODING: utf-8
=====

LOCATION: src\\idms_db2_phase2\\composers\\update_sql_cleanup_base.py
ACTION: CREATE NEW FILE (this replaces update_program_feedback_shared.py)

"""
Shared base for update-program SQL cleanup composers.

Generic rules:
- Sheet Mapping is authority for DB2 tables and columns.
- DCLGEN is authority for host variable spelling.
- Do not hardcode program names, table names, columns, or host variables.
- For CALC/update flows, prefer DB2 primary-key metadata.
- Avoid broad WHERE clauses when a narrower DB2 identity key is available.

Source-field resolution is deterministic (delegated to the Sheet Mapping
repository). No fuzzy / similarity matching.

This base composes focused mixins so subclasses keep the exact same inherited
interface (self._logical, self.db2_primary_key_columns, self._set_lines,
self._column_for_source_field, ...) and the same class-level regex/constant
attributes.
"""

from __future__ import annotations

from idms_db2_phase2.composers.update_sql_cleanup.column_mixin import (
 ColumnMixin,
)
from idms_db2_phase2.composers.update_sql_cleanup.key_column_mixin import (
 KeyColumnMixin,
)
from idms_db2_phase2.composers.update_sql_cleanup.scan_mixin import ScanMixin
from idms_db2_phase2.composers.update_sql_cleanup.sql_line_mixin import (
 SqlLineMixin,
)
from idms_db2_phase2.composers.update_sql_cleanup.update_sql_cleanup_patterns import (

```

```

 CONVERTED_MODIFY_PATTERN,
 CONVERTED_OBTAIN_CALC_PATTERN,
 END_IF_PATTERN,
 EXEC_SQL_END_PATTERN,
 EXEC_SQL_START_PATTERN,
 INCLUDE_PATTERN,
 LINKAGE_SECTION_PATTERN,
 MALFORMED_SQLERROR_ENDIF_PATTERN,
 MOVE_TO_BARE_FIELD_PATTERN,
 MOVE_TO_DCL_HOST_PATTERN,
 SQL_LOCATION_PATTERN,
 SQLCODE_IF_PATTERN,
 STOP_RUN_PATTERN,
 TIMESTAMP_PARAGRAPH_PATTERN,
)
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.host_variable_resolver import (
 HostVariableResolver,
)
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from rules.update_sql_cleanup_rules import (
 IDENTITY_KEY_PREFIXES,
 INSERT_ONLY_AUDIT_PREFIXES,
 PROTECTED_BARE_TARGET_PREFIXES,
 UPDATE_AUDIT_PREFIXES,
)

class UpdateSqlCleanupBase(
 ScanMixin,
 KeyColumnMixin,
 ColumnMixin,
 SqlLineMixin,
):
 # Preserved class-level regex attributes (backwards-compatible).
 CONVERTED_OBTAIN_CALC_PATTERN = CONVERTED_OBTAIN_CALC_PATTERN
 CONVERTED_MODIFY_PATTERN = CONVERTED_MODIFY_PATTERN
 EXEC_SQL_START_PATTERN = EXEC_SQL_START_PATTERN
 EXEC_SQL_END_PATTERN = EXEC_SQL_END_PATTERN
 INCLUDE_PATTERN = INCLUDE_PATTERN
 SQLCODE_IF_PATTERN = SQLCODE_IF_PATTERN
 END_IF_PATTERN = END_IF_PATTERN
 MALFORMED_SQLERROR_ENDIF_PATTERN = MALFORMED_SQLERROR_ENDIF_PATTERN
 MOVE_TO_BARE_FIELD_PATTERN = MOVE_TO_BARE_FIELD_PATTERN
 MOVE_TO_DCL_HOST_PATTERN = MOVE_TO_DCL_HOST_PATTERN
 SQL_LOCATION_PATTERN = SQL_LOCATION_PATTERN
 LINKAGE_SECTION_PATTERN = LINKAGE_SECTION_PATTERN
 TIMESTAMP_PARAGRAPH_PATTERN = TIMESTAMP_PARAGRAPH_PATTERN
 STOP_RUN_PATTERN = STOP_RUN_PATTERN

 # Preserved class-level constant attributes (backwards-compatible).
 PROTECTED_BARE_TARGET_PREFIXES = PROTECTED_BARE_TARGET_PREFIXES
 UPDATE_AUDIT_PREFIXES = UPDATE_AUDIT_PREFIXES
 INSERT_ONLY_AUDIT_PREFIXES = INSERT_ONLY_AUDIT_PREFIXES
 IDENTITY_KEY_PREFIXES = IDENTITY_KEY_PREFIXES

 def __init__(
 self,
 mapping_repository: MappingRepository,
 dclgen_repository: DclgenRepository,
 table_name_resolver: TableNameResolver,
 host_variable_resolver: HostVariableResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver
 self.host_variable_resolver = host_variable_resolver
 self.messages: list[str] = []

```

```
=====
FILE: src\idms_db2_phase2\composers\update_sql_cleanup_composer.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/composers/update_sql_cleanup_composer.py
ACTION: CREATE NEW FILE (this replaces update_program_sql_feedback_composer.py)
```

```
"""
```

```
Update SQL cleanup composer.
```

```
Fixes update-program SQL issues:
```

- malformed PERFORM SQLERROR.END-IF.
- bare changed field move before MODIFY.
- unnecessary OBTAIN CALC SELECT.
- broad MODIFY UPDATE / WHERE (rewritten to changed field + audit fields).

```
Composite key WHERE includes all PK / CALC key fields; FK / FOREIGN columns
are never used in WHERE. No program, table, DCLGEN group, or field name is
hardcoded. Sheet Mapping decides table/column mapping; DCLGEN decides host
variable spelling. Delegates each pass to a focused helper class.
```

```
"""
```

```
from __future__ import annotations
```

```
import re
```

```
from idms_db2_phase2.composers.update_sql_cleanup.bare_move_rewriter import (
 BareMoveRewriter,
)
from idms_db2_phase2.composers.update_sql_cleanup.composer_patterns import (
 ANY_DCL_DOT_REFERENCE_PATTERN,
 ANY_DCL_OF_REFERENCE_PATTERN,
 DATE_COLUMN_PREFIXES,
 MOVE_TO_DCL_DOT_HOST_PATTERN,
 STRING INTO_DCL_DOT_HOST_PATTERN,
 STRING INTO_DCL_HOST_PATTERN,
)
from idms_db2_phase2.composers.update_sql_cleanup.date_move_builder import (
 DateMoveBuilder,
)
from idms_db2_phase2.composers.update_sql_cleanup.modify_update_rewriter import (
 ModifyUpdateRewriter,
)
from idms_db2_phase2.composers.update_sql_cleanup.obtain_calc_remover import (
 ObtainCalcRemover,
)
from idms_db2_phase2.composers.update_sql_cleanup.sql_block_scanner import (
 SqlBlockScanner,
)
from idms_db2_phase2.composers.update_sql_cleanup.sqlerror_normalizer import (
 SqlerrorNormalizer,
)
from idms_db2_phase2.composers.update_sql_cleanup_base import (
 UpdateSqlCleanupBase,
)
from idms_db2_phase2.services.name_normalizer import NameNormalizer
```

```
class UpdateSqlCleanupComposer(UpdateSqlCleanupBase):
 # Composer-specific patterns exposed for helpers that read them via owner.
 MOVE_TO_DCL_DOT_HOST_PATTERN = MOVE_TO_DCL_DOT_HOST_PATTERN
 STRING INTO_DCL_HOST_PATTERN = STRING INTO_DCL_HOST_PATTERN
 STRING INTO_DCL_DOT_HOST_PATTERN = STRING INTO_DCL_DOT_HOST_PATTERN
 ANY_DCL_OF_REFERENCE_PATTERN = ANY_DCL_OF_REFERENCE_PATTERN
 ANY_DCL_DOT_REFERENCE_PATTERN = ANY_DCL_DOT_REFERENCE_PATTERN
 DATE_COLUMN_PREFIXES = DATE_COLUMN_PREFIXES

 def __init__(self, *args, **kwargs) -> None:
 super().__init__(*args, **kwargs)
 self._scanner = SqlBlockScanner(self)
 self._date_builder = DateMoveBuilder()
 self._sqlerror = SqlerrorNormalizer(self)
 self._bare_move = BareMoveRewriter(
 owner=self,
 scanner=self._scanner,
```

```

 date_builder=self._date_builder,
)
 self._obtain_calc = ObtainCalcRemover(self, self._scanner)
 self._modify_update = ModifyUpdateRewriter(self, self._scanner)

def compose(self, text: str) -> str:
 self.messages = []
 output = str(text or "")

 if not output.strip():
 return output

 output = self._sqlerror.normalize(output)
 output = self._bare_move.rewrite(output)
 output = self._obtain_calc.rewrite(output)
 output = self._modify_update.rewrite(output)

 return output.rstrip() + "\n"

def _db2_primary_key_columns(
 self,
 record: str,
 table: str,
) -> list[str]:
 """
 Composite PK / CALC key columns for WHERE.

 - Include all DB2 PRIMARY / KEY columns.
 - Include all IDMS CALC key columns.
 - Exclude FK / FOREIGN / relationship columns.
 - Preserve mapping order.
 """
 output: list[str] = []
 seen: set[str] = set()

 for row in self.mapping_repository.rows_for_record(record):
 idms_key = NameNormalizer.normalize(getattr(row, "idms_key", ""))
 db2_key = NameNormalizer.normalize(getattr(row, "db2_key", ""))
 relation = NameNormalizer.normalize(getattr(row, "relation", ""))
 column = NameNormalizer.normalize(
 getattr(row, "new_db2_field_name", "")
 or getattr(row, "cross_application_db2_field_name", "")
)
 if not column:
 continue

 combined_text = " ".join([idms_key, db2_key, relation])
 padded_text = f" {combined_text} "

 if "FOREIGN" in combined_text:
 continue
 if " FK " in padded_text:
 continue

 is_key_column = (
 "PRIMARY" in db2_key
 or db2_key == "KEY"
 or "PRIMARY" in idms_key
 or idms_key == "KEY"
 or "CALC" in idms_key
)
 if not is_key_column:
 continue
 if column in seen:
 continue

 seen.add(column)
 output.append(column)

 return self._filter_existing_dclgen_columns(table, output)

```



```
=====
FILE: src\idms_db2_phase2\config__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\config\app_settings.py
ENCODING: utf-8
=====
```

```
"""
Application settings.

Keep app-level settings outside parser, transformer, generator, and service
logic.
"""
```

```
APP_NAME = "IDMS DB2 Phase 2 Converter"
```

```
APP_DESCRIPTION = (
 "Standalone Phase 2 project for converting IDMS COBOL programs "
 "into DB2 embedded SQL COBOL."
)
```

```
DEFAULT_STREAMLIT_PORT = 8502
```

```
DEFAULT_AUTO_FIX_PIC_LENGTH_MISMATCHES = False
```

```
=====
FILE: src\idms_db2_phase2\config\logging_settings.py
ENCODING: utf-8
=====
```

```
"""
Logging settings.

Log files are written to the project logs folder with date-time names.
"""
```

```
LOG_FILE_PREFIX = "idms_db2_conversion"
```

```
LOG_DATE_TIME_FORMAT = "%d-%m-%Y_%H%M%S"
```

```
LOG_MESSAGE_FORMAT = "%(asctime)s | %(levelname)s | %(name)s | %(message)s"
```

```
LOG_DATE_FORMAT = "%d-%m-%Y %H:%M:%S"
```

```
DEFAULT_LOG_LEVEL = "INFO"
```

```
=====
FILE: src\idms_db2_phase2\config\path_config.py
ENCODING: utf-8
=====
```

```
"""
Path settings for local command-line runs.

These paths are intentionally centralized here instead of being embedded in
runner, parser, or conversion logic.

Update these values for your local environment.
"""
```

```
from pathlib import Path
```

```
PROJECT_ROOT = Path(__file__).resolve().parents[1]
```

```

SRC_DIR = PROJECT_ROOT / "src"

LOGS_DIR = PROJECT_ROOT / "logs"

DEFAULT_INPUT_DIR = Path(r"C:\S-Input")

DEFAULT_OUTPUT_DIR = DEFAULT_INPUT_DIR / "Output"

DEFAULT_SHEET_MAPPING_PATH = DEFAULT_INPUT_DIR / "Excel_Sheet_mapping.csv"

DEFAULT_RETRIEVAL_SOURCE_PATH = DEFAULT_INPUT_DIR / "Retrieval.txt"

DEFAULT_UPDATE_SOURCE_PATH = DEFAULT_INPUT_DIR / "Update.txt"

DEFAULT_DCLGEN_CANDIDATE_PATHS = [
 DEFAULT_INPUT_DIR / "DCLGENS_BEFF.txt",
 DEFAULT_INPUT_DIR / "DCLGENS_BFAR.txt",
 DEFAULT_INPUT_DIR / "DCLGENS_EVEF.txt",
]

DEFAULT_COPYBOOK_CANDIDATE_PATHS = [
 DEFAULT_INPUT_DIR / "Copybook.txt",
]

=====
FILE: src\idms_db2_phase2\config\path_settings.py
ENCODING: utf-8
=====

r"""
Path settings for local command-line runs.

These paths are intentionally centralized here instead of being embedded in
runner, parser, transformer, generator, or orchestration logic.

Folder-based local input layout:

C:\S-Input
 Mapping Sheet
 Excel_Sheet_mapping.csv
 DCLGen
 *.txt / *.cbl / *.cob / *.cpy
 Copybook
 *.txt / *.cbl / *.cob / *.cpy
 Program
 Retrieval
 *.txt / *.cbl / *.cob
 Update
 *.txt / *.cbl / *.cob
 Output
 Retrieval
 Update
"""

from __future__ import annotations

from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
SRC_DIR = PROJECT_ROOT / "src"
LOGS_DIR = PROJECT_ROOT / "logs"

DEFAULT_INPUT_DIR = Path(r"C:\S-Input")

```

```

DEFAULT_MAPPING_SHEET_DIR = DEFAULT_INPUT_DIR / "Mapping Sheet"
DEFAULT_DCLGEN_DIR = DEFAULT_INPUT_DIR / "DCLGen"
DEFAULT_COPYBOOK_DIR = DEFAULT_INPUT_DIR / "Copybook"
DEFAULT_PROGRAM_DIR = DEFAULT_INPUT_DIR / "Program"

DEFAULT_RETRIEVAL_PROGRAM_DIR = DEFAULT_PROGRAM_DIR / "Retrieval"
DEFAULT_UPDATE_PROGRAM_DIR = DEFAULT_PROGRAM_DIR / "Update"

DEFAULT_OUTPUT_DIR = DEFAULT_INPUT_DIR / "Output"
DEFAULT_RETRIEVAL_OUTPUT_DIR = DEFAULT_OUTPUT_DIR / "Retrieval"
DEFAULT_UPDATE_OUTPUT_DIR = DEFAULT_OUTPUT_DIR / "Update"

SUPPORTED_MAPPING_EXTENSIONS = (
 ".csv",
 ".xlsx",
)

SUPPORTED_TEXT_EXTENSIONS = (
 ".txt",
 ".cbl",
 ".cob",
 ".cpy",
)

DEFAULT_SHEET_MAPPING_PATH = DEFAULT_MAPPING_SHEET_DIR / "Excel_Sheet_mapping.csv"
DEFAULT_RETRIEVAL_SOURCE_PATH = DEFAULT_RETRIEVAL_PROGRAM_DIR / "Retrieval.txt"
DEFAULT_UPDATE_SOURCE_PATH = DEFAULT_UPDATE_PROGRAM_DIR / "Update.txt"

```

```

def existing_files(
 paths: list[Path],
) -> list[Path]:
 """
 Return only paths that exist and are files.
 """
 return [
 path
 for path in paths
 if path.exists() and path.is_file()
]

def files_in_folder(
 folder: Path,
 extensions: tuple[str, ...],
) -> list[Path]:
 """
 Return supported files from a folder in stable name order.
 """
 if not folder.exists() or not folder.is_dir():
 return []

 normalized_extensions = tuple(
 extension.lower()
 for extension in extensions
)

 return sorted(
 [
 path
 for path in folder.iterdir()
 if path.is_file()
 and path.suffix.lower() in normalized_extensions
],
 key=lambda item: item.name.lower(),
)

```

```

def mapping_sheet_paths() -> list[Path]:
 """
 Return mapping sheet files from the single mapping sheet folder.

 If Excel_Sheet_mapping.csv exists, it is preferred first.
 """
 paths = files_in_folder(
 DEFAULT_MAPPING_SHEET_DIR,
)

```

```

 SUPPORTED_MAPPING_EXTENSIONS,
)

 if DEFAULT_SHEET_MAPPING_PATH.exists() and DEFAULT_SHEET_MAPPING_PATH.is_file():
 preferred = DEFAULT_SHEET_MAPPING_PATH
 paths = [
 preferred,
 *[
 path
 for path in paths
 if path.resolve() != preferred.resolve()
],
]

 return paths

def dclgen_paths() -> list[Path]:
 """
 Return all DCLGEN files from the DCLGen folder.
 """
 return files_in_folder(
 DEFAULT_DCLGEN_DIR,
 SUPPORTED_TEXT_EXTENSIONS,
)

def copybook_paths() -> list[Path]:
 """
 Return all copybook files from the Copybook folder.
 """
 return files_in_folder(
 DEFAULT_COPYBOOK_DIR,
 SUPPORTED_TEXT_EXTENSIONS,
)

def retrieval_program_paths() -> list[Path]:
 """
 Return all retrieval COBOL program files.
 """
 return files_in_folder(
 DEFAULT_RETRIEVAL_PROGRAM_DIR,
 SUPPORTED_TEXT_EXTENSIONS,
)

def update_program_paths() -> list[Path]:
 """
 Return all update COBOL program files.
 """
 return files_in_folder(
 DEFAULT_UPDATE_PROGRAM_DIR,
 SUPPORTED_TEXT_EXTENSIONS,
)

def ensure_output_dirs() -> None:
 """
 Ensure output folders exist.
 """
 DEFAULT_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
 DEFAULT_RETRIEVAL_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
 DEFAULT_UPDATE_OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

DEFAULT_DCLGEN_CANDIDATE_PATHS = dclgen_paths()
DEFAULT_COPYBOOK_CANDIDATE_PATHS = copybook_paths()

=====
FILE: src\idms_db2_phase2\domain__init__.py
ENCODING: utf-8
=====

[Empty file]

```

```
=====
FILE: src\idms_db2_phase2\domain\models.py
ENCODING: utf-8
=====
```

```
from dataclasses import dataclass, field
```

```
@dataclass
```

```
class SheetMappingRow:
```

```
 cobol_record_idms: str = ""
 cobol_zone: str = ""
 idms_key: str = ""
 idms_pic_clause: str = ""
 length_of_field_bytes: str = ""
 field_end_position: str = ""
 db2_key: str = ""
 new_db2_record: str = ""
 new_db2_field_name: str = ""
 new_db2_data_type: str = ""
 hopex_expression_type_remark: str = ""
 remarks: str = ""
 relation: str = ""
 reference_field_name_copybook: str = ""
 reference_field_pic_clause: str = ""
 cross_application_db2_table: str = ""
 cross_application_db2_field_name: str = ""
 cross_application_db2_data_type: str = ""
 basetype: str = ""
```

```
@dataclass
```

```
class DclgenColumn:
```

```
 table_name: str = ""
 column_name: str = ""
 db2_type: str = ""
 cobol_host_name: str = ""
 cobol_picture: str = ""
 cobol_usage: str = ""
 nullable: bool = True
```

```
@dataclass
```

```
class CopybookField:
```

```
 level: str = ""
 name: str = ""
 picture: str = ""
 usage: str = ""
 occurs: str = ""
```

```
@dataclass
```

```
class IdmsOperation:
```

```
 operation: str
 record_name: str = ""
 set_name: str = ""
 line_number: int = 0
 raw_line: str = ""
```

```
@dataclass
```

```
class RelationshipSummary:
```

```
 relation: str = ""
 parent_record: str = ""
 child_record: str = ""
 parent_key: str = ""
 child_key: str = ""
```

```
@dataclass
```

```
class RecordSummary:
```

```
 record_name: str
 db2_table: str
 column_count: int
```

```

key_columns: list[str] = field(default_factory=list)

LOCATION: src/idms_db2_phase2/domain/models.py
ACTION: REPLACE the existing ConversionInput dataclass

@dataclass
class ConversionInput:
 sheet_mapping_rows: list[SheetMappingRow] = field(default_factory=list)
 dclgen_columns: list[DclgenColumn] = field(default_factory=list)
 copybook_fields: list[CopybookField] = field(default_factory=list)
 logical_records: list[LogicalRecord] = field(default_factory=list)
 idms_cobol_text: str = ""
 target_program_id: str = ""
 auto_fix_pic_length_mismatches: bool = False

@dataclass
class ConversionResult:
 converted_cobol: str = ""
 validation_messages: list[str] = field(default_factory=list)
 operations: list[IdmsOperation] = field(default_factory=list)

LOCATION: src/idms_db2_phase2/domain/models.py
ACTION: APPEND (place above ConversionInput)

@dataclass
class LrfPathCommand:
 """One command line inside a SELECT FOR KEYWORD block."""
 verb: str = "" # FIND / OBTAIN / ERASE / IF
 scope: str = "" # CURRENT / EACH / FIRST
 record_name: str = "" # VMBSIAS
 within_name: str = "" # AR-VMBFRM1 or VMBSIAS-VMBFAS
 where_clause: str = "" # CALCKEY EQ KY-SIFORM OF VMBSIAS OF LR
 status_actions: dict = field(default_factory=dict) # {"0307": "RETURN VMBFAS-EOA"}
 line_number: int = 0
 raw_line: str = ""

@dataclass
class LrfPath:
 """One SELECT FOR KEYWORD block."""
 keyword: str = "" # VMBFAS-BY-VMBSIAS
 path_group_verb: str = "" # OBTAIN / ERASE / MODIFY / STORE
 logical_record: str = "" # VMBTL03-R01
 commands: list[LrfPathCommand] = field(default_factory=list)

@dataclass
class LogicalRecord:
 """One ADD LOGICAL RECORD block plus every path that serves it."""
 logical_record_name: str = "" # VMBTL03-R01
 element_records: list[str] = field(default_factory=list) # VMBSIAS, VMBFAS
 comments: list[str] = field(default_factory=list)
 paths: list[LrfPath] = field(default_factory=list)
 subschema_name: str = "" # VMBTS03
 schema_name: str = "" # VMBTSCH

=====
FILE: src\idms_db2_phase2\generators__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_db2_phase2\generators\counter_generator.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/generators/counter_generator.py
ACTION: CREATE NEW FILE
"""Row counter and program-name constant generator.

```

Closes open decision D-2 in favour of AUTOMATE, per the COBOL team's manual reference program:

```
10 WS-NB-BEFF-COUNT PIC 9(7) COMP-3 VALUE ZEROES.
10 WS-NB-OUTPUT-COUNT PIC 9(7) COMP-3 VALUE ZEROES.
10 CS-PROGRAM PIC X(8) VALUE 'VMDZ4420'.
...
ADD 1 TO WS-NB-BEFF-COUNT
...
DISPLAY 'TOTAL BEFF RECORDS : ' WS-NB-BEFF-COUNT
DISPLAY 'TOTAL OUTPUT RECORDS : ' WS-NB-OUTPUT-COUNT
```

Constants live in rules/counter\_rules.py. No program, record, table or cursor name is hardcoded here.

```
from __future__ import annotations

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.counter_rules import (
 COUNTER_ADD_TEMPLATE,
 COUNTER_DECLARATION_TEMPLATE,
 COUNTER_DISPLAY_TEMPLATE,
 COUNTER_LABEL_FETCH_TEMPLATE,
 COUNTER_LABEL_OUTPUT,
 COUNTER_LABEL_WIDTH,
 COUNTER_MESSAGES,
 COUNTER_NAME_TEMPLATE,
 COUNTER_PICTURE,
 EMIT_COUNTER_TOTALS,
 EMIT_PROGRAM_NAME_CONSTANT,
 EMIT_ROW_COUNTERS,
 OUTPUT_COUNTER_NAME,
 PROGRAM_CONSTANT_NAME,
 PROGRAM_CONSTANT_TEMPLATE,
 PROGRAM_NAME_MOVE_TEMPLATE,
 PROGRAM_NAME_TARGET_FIELD,
)

class CounterGenerator:
 """Builds counter declarations, increments and end-of-run totals."""

 def __init__(self) -> None:
 self.messages: list[str] = []

 # -----
 # Naming
 # -----
 @staticmethod
 def counter_name_for_table(table_name: str) -> str:
 """Derive WS-NB-<token>-COUNT from a DB2 table name.

 DZBEFFTV -> BEFF -> WS-NB-BEFF-COUNT
 The token is the distinctive middle of the table name, with the
 two-character system prefix and the TV/TB suffix removed.
 """
 table = NameNormalizer.normalize(str(table_name or ""))
 if not table:
 return ""

 token = table
 for suffix in ("_TV", "_TB"):
 if token.endswith(suffix):
 token = token[: -len(suffix)]
 break
 else:
 if token.endswith("TV") or token.endswith("TB"):
 token = token[:-2]

 if len(token) > 2:
 token = token[2:]

 token = token.strip("_-")
 if not token:
```

```

 return ""

 return COUNTER_NAME_TEMPLATE.format(token=token)

WORKING-STORAGE declarations

def declarations(
 self,
 program_id: str,
 table_names: list[str],
) -> list[str]:
 """Lines to append inside the existing 01 work group."""
 self.messages = []
 lines: list[str] = []

 if EMIT_ROW_COUNTERS:
 for name in self._counter_names(table_names):
 lines.append(
 COUNTER_DECLARATION_TEMPLATE.format(
 name=name, picture=COUNTER_PICTURE,
)
)
 self.messages.append(
 COUNTER_MESSAGES["declared_counter"].format(name=name)
)

 if EMIT_PROGRAM_NAME_CONSTANT and program_id:
 lines.append(
 PROGRAM_CONSTANT_TEMPLATE.format(
 name=PROGRAM_CONSTANT_NAME,
 program_id=NameNormalizer.to_cobol(program_id),
)
)
 self.messages.append(
 COUNTER_MESSAGES["declared_program_constant"].format(
 name=PROGRAM_CONSTANT_NAME, program_id=program_id,
)
)

 return lines

Increments

@staticmethod
def increment(counter_name: str) -> str:
 if not EMIT_ROW_COUNTERS or not counter_name:
 return ""
 return COUNTER_ADD_TEMPLATE.format(name=counter_name)

Program-name move

@staticmethod
def program_name_move() -> str:
 if not EMIT_PROGRAM_NAME_CONSTANT:
 return ""
 return PROGRAM_NAME_MOVE_TEMPLATE.format(
 constant=PROGRAM_CONSTANT_NAME,
 target=PROGRAM_NAME_TARGET_FIELD,
)

End-of-run totals

def totals(self, table_names: list[str]) -> list[str]:
 if not EMIT_COUNTER_TOTALS:
 return []

 lines: list[str] = []
 for table in table_names:
 name = self.counter_name_for_table(table)
 if not name:
 continue
 token = name.replace("WS-NB-", "").replace("-COUNT", "")

```



```

 label = COUNTER_LABEL_FETCH_TEMPLATE.format(token=token)
 lines.append(
 COUNTER_DISPLAY_TEMPLATE.format(
 label=label.ljust(COUNTER_LABEL_WIDTH), name=name,
)
)

 lines.append(
 COUNTER_DISPLAY_TEMPLATE.format(
 label=COUNTER_LABEL_OUTPUT.ljust(COUNTER_LABEL_WIDTH),
 name=OUTPUT_COUNTER_NAME,
)
)

 self.messages.append(
 COUNTER_MESSAGES["counter_totals_added"].format(count=len(lines))
)
 return lines

```

```

Helpers

def _counter_names(self, table_names: list[str]) -> list[str]:
 out: list[str] = []
 seen: set[str] = set()

 for table in table_names:
 name = self.counter_name_for_table(table)
 if name and name not in seen:
 seen.add(name)
 out.append(name)

 if OUTPUT_COUNTER_NAME not in seen:
 out.append(OUTPUT_COUNTER_NAME)

 return out

```

```

=====
FILE: src\idms_db2_phase2\generators\cursor_declaration_generator.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/generators/cursor_declaration_generator.py
ACTION: REPLACE ENTIRE FILE

```

"""Generates DB2 cursor DECLARE statements.

Emits the COBOL team's manual reference layout exactly:

```

* DECLARE CURSOR FOR BEFF TABLE
* -----
*
EXEC SQL DECLARE DZBEFFC1 CURSOR WITH HOLD FOR
SELECT CO_IDPRKSK_479BEFF
 , NS_IRMOSTK_479BEFF
 , DA_UBSECIS_479BEFF
FROM DZBEFFTV
FOR READ ONLY
 QUERYNO 176
END-EXEC

```

Authority and safety rules honoured here:

- A cursor with NO WHERE clause is a parent/root cursor and never carries ORDER BY (matches composers/cursor\_order\_cleanup\_composer.py).
- A cursor WITH a WHERE clause is a child/dependent cursor and keeps its ORDER BY.
- The left operand of every WHERE predicate must be a column of the cursor's own FROM table. A parent column on the left binds as SQLCODE -206, so the declaration is refused and reported instead.
- Nothing is fabricated when Sheet Mapping or DCLGEN metadata is missing; a DB2 WARNING comment is emitted (CONVERSION\_RULES).

Clean Architecture:

- Every literal lives in rules/cursor\_declaration\_rules.py and

```

rules/manual_reference_rules.py.
- No regex is defined in this file.
- No program, record, table, cursor or host variable name is hardcoded.
"""

from __future__ import annotations

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.cursor_declaration_rules import (
 CHILD_CURSOR_KEEPS_ORDER_BY,
 DECLARATION_MESSAGES,
 DECLARE_LINE_TEMPLATE,
 DECLARE_ON_EXEC_SQL_LINE,
 EMIT_QUERYNO,
 END_EXEC,
 ENFORCE_WHERE_COLUMN_OWNERSHIP,
 FOR_READ_ONLY,
 FROM_TEMPLATE,
 IND_CLAUSE,
 IND_EXEC,
 IND_QUERYNO,
 IND_SELECT_FIRST,
 IND_SELECT_NEXT,
 IND_WHERE_NEXT,
 LAST_DECLARATION_TERMINATOR,
 ORDER_BY_TEMPLATE,
 PARENT_CURSOR_KEEPS_ORDER_BY,
 QUERYNO_BASE,
 QUERYNO_STEP,
 QUERYNO_TEMPLATE,
 SELECT_FIRST_TEMPLATE,
 SELECT_NEXT_TEMPLATE,
 UNRESOLVED_JOIN_KEY_TEMPLATE,
 WHERE_FIRST_TEMPLATE,
 WHERE_NEXT_TEMPLATE,
)
from rules.manual_reference_rules import (
 CURSOR_BANNER_BLANK,
 CURSOR_BANNER_TITLE_TEMPLATE,
 CURSOR_BANNER_UNDERLINE,
)

Warning text used when the spec itself is unusable.
MISSING_CURSOR_NAME = (
 """ DB2 WARNING: Unable to declare cursor; missing cursor name."""
)
MISSING_TABLE_TEMPLATE = (
 """ DB2 WARNING: Unable to declare cursor {cursor}; """
 "missing DB2 table mapping."
)
MISSING_COLUMNS_TEMPLATE = (
 """ DB2 WARNING: Unable to declare cursor {cursor}; """
 "no DB2 columns resolved for table {table}."
)

Table-name affixes stripped when deriving the short banner name.
TABLE_SUFFIXES = ("_TV", "_TB")
TABLE_SHORT_SUFFIXES = ("TV", "TB")
TABLE_PREFIX_LENGTH = 2

class CursorDeclarationGenerator:
 """Builds the WORKING-STORAGE cursor declaration block."""

 def __init__(
 self,
 column_name_resolver,
 diagnostics: list[str] | None = None,
) -> None:
 self.column_name_resolver = column_name_resolver
 self.messages: list[str] = diagnostics if diagnostics is not None else []

 # =====
 # Public entry point
 # =====
 def generate(

```

```

 self,
 specs: list[dict[str, object]],
) -> list[str]:
 """Render every cursor declaration, in cursor order.

 Returns a flat list of COBOL body lines (no sequence numbers).
 The fixed-format writer applies columns 1-6, 7 and 73-80.
 """
 self.messages = []

 if not specs:
 return []

 lines: list[str] = []
 total = len(specs)

 for index, spec in enumerate(specs):
 block = self._declaration(
 spec=spec,
 order=index,
 is_last=index == total - 1,
)
 if not block:
 continue
 if lines:
 lines.append("")
 lines.extend(block)

 return lines

=====
One declaration
=====
def _declaration(
 self,
 spec: dict[str, object],
 order: int,
 is_last: bool,
) -> list[str]:
 cursor = NameNormalizer.to_cobol(str(spec.get("cursor_name", "")))
 if not cursor:
 return [MISSING_CURSOR_NAME]

 table = NameNormalizer.normalize(str(spec.get("table_name", "")))
 if not table:
 return [MISSING_TABLE_TEMPLATE.format(cursor=cursor)]

 columns = self._columns(spec)
 if not columns:
 return [
 MISSING_COLUMNS_TEMPLATE.format(cursor=cursor, table=table)
]

 conditions = self._where_conditions(spec)

 if ENFORCE_WHERE_COLUMN_OWNERSHIP and conditions:
 unowned = self._unowned_condition(conditions, table)
 if unowned:
 self._log(
 DECLARATION_MESSAGES["join_key_unresolved"].format(
 column=unowned,
 table=table,
 cursor=cursor,
)
)
 return [
 UNRESOLVED_JOIN_KEY_TEMPLATE.format(
 cursor=cursor,
 column=unowned,
 table=table,
)
]

 order_by = self._order_by(
 spec=spec,
 has_where=bool(conditions),

```

```

 cursor=cursor,
)
 queryno = self._queryno(order)

 lines: list[str] = []
 lines.extend(self._banner(table))
 lines.extend(
 self._body(
 cursor=cursor,
 table=table,
 columns=columns,
 conditions=conditions,
 order_by=order_by,
 queryno=queryno,
 is_last=is_last,
)
)

 self._log(
 DECLARATION_MESSAGES["declared_cursor"].format(
 cursor=cursor,
 table=table,
 count=len(columns),
 queryno=queryno,
)
)
 return lines

=====
Banner
=====
def _banner(self, table: str) -> list[str]:
 return [
 CURSOR_BANNER_TITLE_TEMPLATE.format(
 short_name=self._short_table_name(table)
),
 CURSOR_BANNER_UNDERLINE,
 CURSOR_BANNER_BLANK,
]

@staticmethod
def _short_table_name(table: str) -> str:
 """DZBEFFTV -> BEFF. Prefix and TV/TB suffix removed."""
 token = NameNormalizer.normalize(str(table or ""))
 if not token:
 return ""

 for suffix in TABLE_SUFFIXES:
 if token.endswith(suffix):
 token = token[: -len(suffix)]
 break
 else:
 for suffix in TABLE_SHORT_SUFFIXES:
 if token.endswith(suffix):
 token = token[: -len(suffix)]
 break

 if len(token) > TABLE_PREFIX_LENGTH:
 token = token[:TABLE_PREFIX_LENGTH:]

 return token.strip("_-") or NameNormalizer.normalize(table)

=====
SQL body
=====
def _body(
 self,
 cursor: str,
 table: str,
 columns: list[str],
 conditions: list[str],
 order_by: list[str],
 queryno: int,
 is_last: bool,
) -> list[str]:
 lines: list[str] = []

```

```

--- EXEC SQL DECLARE -----
declare = DECLARE_LINE_TEMPLATE.format(cursor=cursor)
if DECLARE_ON_EXEC_SQL_LINE:
 lines.append(f"{IND_EXEC}{declare}")
else:
 lines.append(f"{IND_EXEC}EXEC SQL")
 lines.append(f"{IND_CLAUSE}DECLARE {cursor} CURSOR WITH HOLD FOR")

--- SELECT column list -----
for position, column in enumerate(columns):
 if position == 0:
 lines.append(
 f"{IND_SELECT_FIRST}"
 f"{SELECT_FIRST_TEMPLATE.format(column=column)}"
)
 else:
 lines.append(
 f"{IND_SELECT_NEXT}"
 f"{SELECT_NEXT_TEMPLATE.format(column=column)}"
)

--- FROM -----
lines.append(f"{IND_CLAUSE}{FROM_TEMPLATE.format(table=table)}")

--- WHERE / AND -----
for position, condition in enumerate(conditions):
 if position == 0:
 lines.append(
 f"{IND_CLAUSE[:-1]}"
 f"{WHERE_FIRST_TEMPLATE.format(condition=condition)}"
)
 else:
 lines.append(
 f"{IND_WHERE_NEXT}"
 f"{WHERE_NEXT_TEMPLATE.format(condition=condition)}"
)

--- ORDER BY -----
if order_by:
 lines.append(
 f"{IND_CLAUSE}"
 f"{ORDER_BY_TEMPLATE.format(columns=' '.join(order_by))}"
)

--- FOR READ ONLY -----
lines.append(f"{IND_CLAUSE}{FOR_READ_ONLY}")

--- QUERYNO -----
if queryno:
 lines.append(
 f"{IND_QUERYNO}{QUERYNO_TEMPLATE.format(number=queryno)}"
)

--- END-EXEC -----
terminator = LAST_DECLARATION_TERMINATOR if is_last else ""
lines.append(f"{IND_EXEC}{END_EXEC}{terminator}")

return lines

=====
Spec helpers
=====
@staticmethod
def _columns(spec: dict[str, object]) -> list[str]:
 """Normalised SELECT column list, order preserved, de-duplicated.

 Column order is authoritative: the FETCH INTO host list is built
 from the same sequence, so a mismatch here silently corrupts data.
 """
 out: list[str] = []
 seen: set[str] = set()

 for column in list(spec.get("select_columns", []) or []):
 value = NameNormalizer.normalize(str(column))
 if not value or value in seen:

```

```

 continue
 seen.add(value)
 out.append(value)

 return out

@staticmethod
def _where_conditions(spec: dict[str, object]) -> list[str]:
 """Join predicates, already built by CursorJoinResolver."""
 out: list[str] = []
 seen: set[str] = set()

 for condition in list(spec.get("where_conditions", []) or []):
 value = str(condition or "").strip()
 if not value or value in seen:
 continue
 seen.add(value)
 out.append(value)

 return out

def _order_by(
 self,
 spec: dict[str, object],
 has_where: bool,
 cursor: str,
) -> list[str]:
 """ORDER BY survives on child cursors only.

 A cursor with no WHERE clause is a parent/root cursor. The manual
 reference never orders a parent cursor, so any ORDER BY resolved
 upstream is dropped here and reported.
 """
 columns = [
 str(column or "").strip()
 for column in list(spec.get("order_by_columns", []) or [])
 if str(column or "").strip()
]
 if not columns:
 return []

 keep = (
 CHILD_CURSOR_KEEPS_ORDER_BY
 if has_where
 else PARENT_CURSOR_KEEPS_ORDER_BY
)
 if not keep:
 self._log(
 DECLARATION_MESSAGES["order_by_removed"].format(cursor=cursor)
)
 return []

 return columns

@staticmethod
def _queryno(order: int) -> int:
 """Stable QUERYNO so repeated runs are byte-identical."""
 if not EMIT_QUERYNO:
 return 0
 return QUERYNO_BASE + (order * QUERYNO_STEP)

=====
WHERE column ownership
=====
def _unowned_condition(
 self,
 conditions: list[str],
 table: str,
) -> str:
 """Return the first WHERE left operand not owned by `table`.

 A child cursor may only filter on columns of its own FROM table.
 A parent column on the left produces SQLCODE -206 at bind time.
 Returns "" when every predicate is valid, or when DCLGEN has no
 column list for the table (nothing to validate against).
 """

```

```

 owned = {
 NameNormalizer.normalize(str(column)).upper()
 for column in self.column_name_resolver.columns_for_table(table)
 }
 owned.discard("")

 if not owned:
 return ""

 for condition in conditions:
 left = condition.split("=", 1)[0].strip().upper()
 if left and left not in owned:
 return left

 return ""

=====
Diagnostics
=====
def _log(self, message: str) -> None:
 if message:
 self.messages.append(message)

=====
FILE: src\idms_db2_phase2\generators\cursor_paragraph__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_db2_phase2\generators\cursor_paragraph\cursor_paragraph_body_builder.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/generators/cursor_paragraph/cursor_paragraph_body_builder.py
ACTION: CREATE NEW FILE

"""Builds the COBOL body of a cursor OPEN / FETCH / CLOSE paragraph.

This class owns no COBOL layout literals: every token, indent and template
lives in rules/cursor_paragraph_rules.py. Only dynamic values (cursor name,
paragraph name, host variables) are substituted here.
"""

from __future__ import annotations

from rules.cursor_paragraph_rules import (
 CLOSE_STATEMENT_TEMPLATE,
 CURSOR_EOC_SUFFIX,
 CURSOR_NOT_EOC_SUFFIX,
 DISPLAY_ERROR_TEMPLATE,
 ERROR_CLOSE_TEMPLATE,
 ERROR_FETCH_TEMPLATE,
 ERROR_OPEN_TEMPLATE,
 FETCH_STATEMENT_TEMPLATE,
 IND_SQL_BODY,
 IND_SQL_INT0,
 IND_SQL_INT0_NEXT,
 IND_STATEMENT,
 IND_WHEN,
 IND_WHEN_BODY,
 LONE_PERIOD,
 OPEN_STATEMENT_TEMPLATE,
 PARAGRAPH_HEADER_TEMPLATE,
 PERFORM_TEMPLATE,
 SET_EOC_TEMPLATE,
 SET_NOT_EOC_TEMPLATE,
 SQL_LOCATION_FIELD,
 SQL_LOCATION_NAME_TEMPLATE,
 SQL_LOCATION_NUMBER_PAD,
 SQL_LOCATION_NUMBER_TEMPLATE,
 SQL_LOCATION_USES_PARAGRAPH_NUMBER,
 TOKEN_CONTINUE,

```

```

TOKEN_END_EVALUATE,
TOKEN_END_EXEC,
TOKEN_EVALUATE_SQLCODE,
TOKEN_EXEC_SQL,
TOKEN_HOST_SEPARATOR,
TOKEN_INT0,
TOKEN_WHEN_100,
TOKEN_WHEN_OTHER,
TOKEN_WHEN_ZERO,

```

```
)
```

```
class CursorParagraphBodyBuilder:
```

```

def open_paragraph(
 self,
 cursor_name: str,
 paragraph_name: str,
 sql_error_paragraph: str,
) -> list[str]:
 lines = self._paragraph_prologue(
 paragraph_name=paragraph_name,
 statement=OPEN_STATEMENT_TEMPLATE.format(cursor=cursor_name),
)

 lines.extend(
 [
 f"{IND_STATEMENT}{TOKEN_EVALUATE_SQLCODE}",
 f"{IND_WHEN}{TOKEN_WHEN_ZERO}",
 IND_WHEN_BODY
 + SET_NOT_EOC_TEMPLATE.format(
 cursor=cursor_name,
 suffix=CURSOR_NOT_EOC_SUFFIX,
),
]
)

 lines.extend(
 self.sqlcode_error_tail(
 error_text=ERROR_OPEN_TEMPLATE.format(cursor=cursor_name),
 sql_error_paragraph=sql_error_paragraph,
)
)

 return lines

def fetch_paragraph(
 self,
 cursor_name: str,
 paragraph_name: str,
 host_variables: list[str],
 sql_error_paragraph: str,
) -> list[str]:
 lines = self._paragraph_prologue(
 paragraph_name=paragraph_name,
 statement=FETCH_STATEMENT_TEMPLATE.format(cursor=cursor_name),
 close_exec=False,
)

 if host_variables:
 lines.extend(self.fetch_into_lines(host_variables))

 lines.extend(
 [
 f"{IND_STATEMENT}{TOKEN_END_EXEC}",
 "",
 f"{IND_STATEMENT}{TOKEN_EVALUATE_SQLCODE}",
 f"{IND_WHEN}{TOKEN_WHEN_ZERO}",
 f"{IND_WHEN_BODY}{TOKEN_CONTINUE}",
 f"{IND_WHEN}{TOKEN_WHEN_100}",
 IND_WHEN_BODY
 + SET_EOC_TEMPLATE.format(
 cursor=cursor_name,
 suffix=CURSOR_EOC_SUFFIX,
),
]
)

```



```

)

 lines.extend(
 self.sqlcode_error_tail(
 error_text=ERROR_FETCH_TEMPLATE.format(cursor=cursor_name),
 sql_error_paragraph=sql_error_paragraph,
)
)

 return lines

def close_paragraph(
 self,
 cursor_name: str,
 paragraph_name: str,
 sql_error_paragraph: str,
) -> list[str]:
 lines = self._paragraph_prologue(
 paragraph_name=paragraph_name,
 statement=CLOSE_STATEMENT_TEMPLATE.format(cursor=cursor_name),
)

 lines.extend(
 [
 f"{IND_STATEMENT}{TOKEN_EVALUATE_SQLCODE}",
 f"{IND_WHEN}{TOKEN_WHEN_ZERO}",
 f"{IND_WHEN_BODY}{TOKEN_CONTINUE}",
]
)

 lines.extend(
 self.sqlcode_error_tail(
 error_text=ERROR_CLOSE_TEMPLATE.format(cursor=cursor_name),
 sql_error_paragraph=sql_error_paragraph,
)
)

 return lines

def sqlcode_error_tail(
 self,
 error_text: str,
 sql_error_paragraph: str,
) -> list[str]:
 """WHEN OTHER branch, END-EVALUATE, then the lone period line.

 Nothing here carries a period. The single period on its own line
 terminates the whole paragraph sentence.
 """
 return [
 f"{IND_WHEN}{TOKEN_WHEN_OTHER}",
 IND_WHEN_BODY + DISPLAY_ERROR_TEMPLATE.format(text=error_text),
 IND_WHEN_BODY
 + PERFORM_TEMPLATE.format(paragraph=sql_error_paragraph),
 f"{IND_STATEMENT}{TOKEN_END_EVALUATE}",
 f"{IND_STATEMENT}{LONE_PERIOD}",
]

def sql_location_move(self, paragraph_name: str) -> str:
 """MOVE the paragraph identity into SQL-LOCATION. No period.

 Manual reference uses the bare paragraph number. Falls back to the
 quoted full name when the paragraph is not number-prefixed.
 """
 name = str(paragraph_name or "").strip()
 number = name.split("-", 1)[0]

 if SQL_LOCATION_USES_PARAGRAPH_NUMBER and number.isdigit():
 return IND_STATEMENT + SQL_LOCATION_NUMBER_TEMPLATE.format(
 number=number,
 pad=SQL_LOCATION_NUMBER_PAD,
 field=SQL_LOCATION_FIELD,
)

 return IND_STATEMENT + SQL_LOCATION_NAME_TEMPLATE.format(
 name=name,
)

```

```

 field=SQL_LOCATION_FIELD,
)

def fetch_into_lines(
 self,
 host_variables: list[str],
) -> list[str]:
 """INTO clause in manual form: first host on INTO, leading commas.

 INTO :DCLDZBEFFTV.CO-IDPRKSK-479BEFF
 , :DCLDZBEFFTV.NS-IRMOSTK-479BEFF
 """
 output: list[str] = []

 for index, host in enumerate(host_variables):
 if index == 0:
 output.append(f"{IND_SQL_INT0}{TOKEN_INT0} {host}")
 continue

 output.append(f"{IND_SQL_INT0_NEXT}{TOKEN_HOST_SEPARATOR}{host}")

 return output

def _paragraph_prologue(
 self,
 paragraph_name: str,
 statement: str,
 close_exec: bool = True,
) -> list[str]:
 """Header, SQL-LOCATION move and the EXEC SQL opening."""
 lines = [
 PARAGRAPH_HEADER_TEMPLATE.format(paragraph=paragraph_name),
 self.sql_location_move(paragraph_name),
 "",
 f"{IND_STATEMENT}{TOKEN_EXEC_SQL}",
 f"{IND_SQL_BODY}{statement}",
]

 if close_exec:
 lines.append(f"{IND_STATEMENT}{TOKEN_END_EXEC}")
 lines.append("")

 return lines

```

```

=====
FILE: src\idms_db2_phase2\generators\cursor_paragraph\cursor_perform_rewriter.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/generators/cursor_paragraph/cursor_perform_rewriter.py
ACTION: CREATE NEW FILE

```

```

"""Rewrites PERFORM targets onto generated cursor paragraph names.

```

```

Also renames legacy SQL-ERROR references to SQLERROR. This class owns no
regex definitions: every pattern lives in patterns/cursor_paragraph_patterns.py.
"""

```

```

from __future__ import annotations

```

```

import re

```

```

from patterns.cursor_paragraph_patterns import (
 LEGACY_PERFORM_PATTERN_TEMPLATE,
 LEGACY_SQL_ERROR_HEADER_PATTERN,
 LEGACY_SQL_ERROR_HEADER_REPLACEMENT,
 LEGACY_SQL_ERROR_PERFORM_PATTERN,
 NUMBERED_PERFORM_PATTERN_TEMPLATE,
 PERFORM_REPLACEMENT_TEMPLATE,
 UNNUMBERED_PERFORM_PATTERN_TEMPLATE,
)
from rules.cursor_paragraph_rules import (
 CURSOR_OPERATIONS,
 SQL_ERROR_PARAGRAPH_NAME,
)

```

```

LEGACY_SPEC_KEYS = (
 ("old_open_paragraph", "open_paragraph"),
 ("old_fetch_paragraph", "fetch_paragraph"),
 ("old_close_paragraph", "close_paragraph"),
)

SPEC_KEY_BY_OPERATION = {
 "OPEN": "open_paragraph",
 "FETCH": "fetch_paragraph",
 "CLOSE": "close_paragraph",
}

class CursorPerformRewriter:

 def rewrite(
 self,
 text: str,
 cursor_specs: list[dict[str, object]],
) -> str:
 updated = str(text or "")

 for spec in cursor_specs:
 cursor_name = str(spec.get("cursor_name", "")).strip()

 if not cursor_name:
 continue

 for operation in CURSOR_OPERATIONS:
 target_paragraph = str(
 spec.get(SPEC_KEY_BY_OPERATION[operation], "")
).strip()

 if not target_paragraph:
 continue

 updated = self._replace_unnumbered(
 text=updated,
 operation=operation,
 cursor_name=cursor_name,
 target_paragraph=target_paragraph,
)

 updated = self._replace_numbered(
 text=updated,
 operation=operation,
 cursor_name=cursor_name,
 target_paragraph=target_paragraph,
)

 updated = self._replace_legacy_spec(
 text=updated,
 spec=spec,
)

 return updated

 def normalize_sql_error_references(
 self,
 text: str,
) -> str:
 """Rename SQL-ERROR to SQLERROR without changing termination.

 The original terminator is carried through unchanged. Forcing a
 period breaks any PERFORM that sits inside an EVALUATE block.
 """
 updated = str(text or "")

 updated = LEGACY_SQL_ERROR_PERFORM_PATTERN.sub(
 lambda match: (
 f"PERFORM {SQL_ERROR_PARAGRAPH_NAME}"
 + (match.group("dot") or "")
),
 updated,
)

```

```

 updated = LEGACY_SQL_ERROR_HEADER_PATTERN.sub(
 LEGACY_SQL_ERROR_HEADER_REPLACEMENT,
 updated,
)

 return updated

def _replace_unnumbered(
 self,
 text: str,
 operation: str,
 cursor_name: str,
 target_paragraph: str,
) -> str:
 pattern = re.compile(
 UNNUMBERED_PERFORM_PATTERN_TEMPLATE.format(
 operation=re.escape(operation),
 cursor=re.escape(cursor_name),
),
 flags=re.IGNORECASE,
)

 return pattern.sub(
 PERFORM_REPLACEMENT_TEMPLATE.format(paragraph=target_paragraph),
 text,
)

def _replace_numbered(
 self,
 text: str,
 operation: str,
 cursor_name: str,
 target_paragraph: str,
) -> str:
 pattern = re.compile(
 NUMBERED_PERFORM_PATTERN_TEMPLATE.format(
 operation=re.escape(operation),
 cursor=re.escape(cursor_name),
),
 flags=re.IGNORECASE,
)

 return pattern.sub(
 PERFORM_REPLACEMENT_TEMPLATE.format(paragraph=target_paragraph),
 text,
)

def _replace_legacy_spec(
 self,
 text: str,
 spec: dict[str, object],
) -> str:
 updated = str(text or "")

 for old_key, new_key in LEGACY_SPEC_KEYS:
 old_paragraph = str(spec.get(old_key, "")).strip()
 new_paragraph = str(spec.get(new_key, "")).strip()

 if not old_paragraph or not new_paragraph:
 continue

 if old_paragraph == new_paragraph:
 continue

 pattern = re.compile(
 LEGACY_PERFORM_PATTERN_TEMPLATE.format(
 paragraph=re.escape(old_paragraph),
),
 flags=re.IGNORECASE,
)

 updated = pattern.sub(
 PERFORM_REPLACEMENT_TEMPLATE.format(
 paragraph=new_paragraph,
),

```

```

 updated,
)

 return updated

```

```

=====
FILE: src\idms_db2_phase2\generators\cursor_paragraph_generator.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/generators/cursor_paragraph_generator.py
ACTION: REPLACE ENTIRE FILE

```

```

"""Generates DB2 cursor OPEN / FETCH / CLOSE paragraphs.

```

```

This class is a FACADE. It owns orchestration only:

```

- resolve the cursor specs (from Db2InfrastructureGenerator)
- normalise legacy SQL-ERROR references and rewrite PERFORM targets (CursorPerformRewriter)
- render each paragraph body (CursorParagraphBodyBuilder)
- insert the assembled block into the program once

Paragraph LAYOUT belongs to CursorParagraphBodyBuilder, not here. Banners, debug trace lines, join-key diagnostics and counter increments are body concerns and must be added there.

Public interface consumed by orchestration/conversion/conversion\_pipeline.py:

```

 converted_cobol, messages = generator.apply(
 cobol_text=converted_cobol,
 operations=operations,
)

```

```

Do not rename or remove `apply`.
"""

```

```

from __future__ import annotations

```

```

from catalogs.output_sections import DB2_CURSOR_PARAGRAPH_MARKER
from idms_db2_phase2.domain.models import IdmsOperation
from idms_db2_phase2.generators.cursor_paragraph.cursor_paragraph_body_builder import (
 CursorParagraphBodyBuilder,
)
from idms_db2_phase2.generators.cursor_paragraph.cursor_perform_rewriter import (
 CursorPerformRewriter,
)
from idms_db2_phase2.generators.db2_infrastructure_generator import (
 Db2InfrastructureGenerator,
)
from idms_db2_phase2.generators.sql_error_generator import SqlErrorGenerator
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from patterns.db2_patterns import END_PROGRAM_PATTERN
from rules.cursor_paragraph_rules import (
 COMMENT_BLOCK_TEMPLATE,
 DIAG_BLOCK_EXISTS,
 DIAG_GENERATED_TEMPLATE,
 DIAG_NO_CURSOR_OPERATIONS,
 DIAG_NO_HOST_VARIABLES_TEMPLATE,
 SQL_ERROR_PARAGRAPH_NAME,
)

```

```

class CursorParagraphGenerator:
 """Builds and inserts the generated cursor paragraph block."""

```

```

 SQLERROR_PARAGRAPH_NAME = SQL_ERROR_PARAGRAPH_NAME

```

```

 def __init__(
 self,
 db2_infrastructure_generator: Db2InfrastructureGenerator,
 host_variable_resolver: HostVariableResolver,
 sql_error_generator: SqlErrorGenerator,
 body_builder: CursorParagraphBodyBuilder | None = None,
 perform_rewriter: CursorPerformRewriter | None = None,
):

```

```

) -> None:
 self.db2_infrastructure_generator = db2_infrastructure_generator
 self.host_variable_resolver = host_variable_resolver
 self.sql_error_generator = sql_error_generator
 self.body_builder = body_builder or CursorParagraphBodyBuilder()
 self.perform_rewriter = perform_rewriter or CursorPerformRewriter()
 self.messages: list[str] = []

=====
Public entry point - DO NOT RENAME
=====
def apply(
 self,
 cobol_text: str,
 operations: list[IdmsOperation],
) -> tuple[str, list[str]]:
 self.messages = []
 text = str(cobol_text or "")

 if not text:
 return text, self.messages

 cursor_specs = self._cursor_specs(operations=operations)

 if not cursor_specs:
 self.messages.append(DIAG_NO_CURSOR_OPERATIONS)
 return text, self.messages

 sql_error_paragraph = self._sql_error_paragraph_name()

 updated_text = self.perform_rewriter.normalize_sql_error_references(
 text
)
 updated_text = self.perform_rewriter.rewrite(
 text=updated_text,
 cursor_specs=cursor_specs,
)

 if DB2_CURSOR_PARAGRAPH_MARKER in updated_text:
 self.messages.append(DIAG_BLOCK_EXISTS)
 return updated_text, self.messages

 block = self.paragraph_block(
 cursor_specs=cursor_specs,
 sql_error_paragraph=sql_error_paragraph,
)
 updated_text = self._insert_paragraph_block(
 text=updated_text,
 block=block,
)

 self.messages.append(
 DIAG_GENERATED_TEMPLATE.format(count=len(cursor_specs))
)
 self.messages.extend(self._host_variable_warnings(cursor_specs))
 return updated_text, self.messages

=====
Block assembly
=====
def paragraph_block(
 self,
 cursor_specs: list[dict[str, object]],
 sql_error_paragraph: str,
) -> str:
 lines: list[str] = []
 lines.extend(self._comment_block(DB2_CURSOR_PARAGRAPH_MARKER))
 lines.append("")

 for spec in cursor_specs:
 cursor_name = str(spec.get("cursor_name", ""))
 if not cursor_name:
 continue

 host_variables = list(spec.get("host_variables", []) or [])

```

```

builders = (
 ("open_paragraph", self.open_paragraph, False),
 ("fetch_paragraph", self.fetch_paragraph, True),
 ("close_paragraph", self.close_paragraph, False),
)

for key, builder, needs_hosts in builders:
 paragraph_name = str(spec.get(key, ""))
 if not paragraph_name:
 continue

 if needs_hosts:
 lines.extend(
 builder(
 cursor_name=cursor_name,
 paragraph_name=paragraph_name,
 host_variables=host_variables,
 sql_error_paragraph=sql_error_paragraph,
)
)
 else:
 lines.extend(
 builder(
 cursor_name=cursor_name,
 paragraph_name=paragraph_name,
 sql_error_paragraph=sql_error_paragraph,
)
)

 lines.append("")

return "\n".join(lines).rstrip() + "\n"

=====
Body delegation
=====
def open_paragraph(
 self,
 cursor_name: str,
 paragraph_name: str,
 sql_error_paragraph: str,
) -> list[str]:
 return self.body_builder.open_paragraph(
 cursor_name=cursor_name,
 paragraph_name=paragraph_name,
 sql_error_paragraph=sql_error_paragraph,
)

def fetch_paragraph(
 self,
 cursor_name: str,
 paragraph_name: str,
 host_variables: list[str],
 sql_error_paragraph: str,
) -> list[str]:
 return self.body_builder.fetch_paragraph(
 cursor_name=cursor_name,
 paragraph_name=paragraph_name,
 host_variables=self._clean_host_variables(host_variables),
 sql_error_paragraph=sql_error_paragraph,
)

def close_paragraph(
 self,
 cursor_name: str,
 paragraph_name: str,
 sql_error_paragraph: str,
) -> list[str]:
 return self.body_builder.close_paragraph(
 cursor_name=cursor_name,
 paragraph_name=paragraph_name,
 sql_error_paragraph=sql_error_paragraph,
)

=====
Helpers

```

```

=====
def _sql_error_paragraph_name(self) -> str:
 """Single source of truth for the SQL error paragraph name."""
 name = str(
 getattr(self.sql_error_generator, "PARAGRAPH_NAME", "") or ""
).strip()
 return name or self.SQLError_Paragraph_Name

def _cursor_specs(
 self,
 operations: list[IdmsOperation],
) -> list[dict[str, object]]:
 """Reuse the specs the infrastructure generator already built."""
 cached_specs = getattr(
 self.db2_infrastructure_generator,
 "last_cursor_specs",
 [],
)
 if cached_specs:
 return cached_specs

 return self.db2_infrastructure_generator.cursor_specs(operations)

@staticmethod
def _clean_host_variables(host_variables: list[str]) -> list[str]:
 out: list[str] = []
 seen: set[str] = set()

 for host in host_variables or []:
 value = str(host or "").strip()
 if not value or value in seen:
 continue
 seen.add(value)
 out.append(value)

 return out

@staticmethod
def _comment_block(title: str) -> list[str]:
 return [COMMENT_BLOCK_TEMPLATE.format(title=title)]

def _host_variable_warnings(
 self,
 cursor_specs: list[dict[str, object]],
) -> list[str]:
 warnings: list[str] = []

 for spec in cursor_specs:
 if list(spec.get("host_variables", []) or []):
 continue
 warnings.append(
 DIAG_NO_HOST_VARIABLES_TEMPLATE.format(
 cursor=str(spec.get("cursor_name", "")),
 record=str(spec.get("record_name", "")),
 table=str(spec.get("table_name", "")),
)
)

 return warnings

@staticmethod
def _insert_paragraph_block(text: str, block: str) -> str:
 """Place the block before END PROGRAM, else append at the end."""
 match = END_PROGRAM_PATTERN.search(text)

 if match:
 insert_at = match.start()
 return f"{text[:insert_at]}\n{block}\n{text[insert_at:]}"

 return f"{text.rstrip()}\n\n{block}"

```



```
=====
FILE: src\idms_db2_phase2\generators\db2_infrastructure__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\generators\db2_infrastructure\block_inserter.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/generators/db2_infrastructure/block_inserter.py
ACTION: CREATE NEW FILE
```

```
"""Inserts the generated block into DATA DIVISION and adds DCLGEN INITIALIZE."""
```

```
from __future__ import annotations
```

```
import re
```

```
from idms_db2_phase2.generators.db2_infrastructure.infrastructure_patterns import (
 DATA_DIVISION_PATTERN,
 LINKAGE_SECTION_PATTERN,
 PROCEDURE_DIVISION_PATTERN,
 WORKING_STORAGE_PATTERN,
)
from idms_db2_phase2.services.name_normalizer import NameNormalizer
```

```
class BlockInserter:
```

```
 def __init__(self, line_utils) -> None:
 self.line_utils = line_utils
```

```
 def insert_in_data_division(self, text: str, block: str) -> str:
 for pattern in [LINKAGE_SECTION_PATTERN, PROCEDURE_DIVISION_PATTERN]:
 match = pattern.search(text)
 if match:
 return (
 text[: match.start()].rstrip()
 + "\n\n"
 + block.rstrip()
 + "\n\n"
 + text[match.start():].lstrip()
).rstrip() + "\n"
```

```
 working_storage_match = WORKING_STORAGE_PATTERN.search(text)
 if working_storage_match:
 return (
 text[: working_storage_match.end()].rstrip()
 + "\n\n"
 + block.rstrip()
 + "\n\n"
 + text[working_storage_match.end():].lstrip()
).rstrip() + "\n"
```

```
 data_division_match = DATA_DIVISION_PATTERN.search(text)
 if data_division_match:
 return (
 text[: data_division_match.end()].rstrip()
 + "\n\n"
 + "WORKING-STORAGE SECTION."
 + "\n\n"
 + block.rstrip()
 + "\n\n"
 + text[data_division_match.end():].lstrip()
).rstrip() + "\n"
```

```
 return block.rstrip() + "\n\n" + text.rstrip() + "\n"
```

```
 def ensure_dclgen_initialization(
 self,
 text: str,
 include_names: list[str],
) -> str:
```

```

clean_include_names = [
 self.line_utils.normalize_include_name(include_name)
 for include_name in include_names
 if self.line_utils.normalize_include_name(include_name)
]
if not clean_include_names:
 return text

missing_initialize_names = []
for include_name in clean_include_names:
 group_name = "DCL" + NameNormalizer.to_cobol(include_name)
 if re.search(
 rf"\bINITIALIZE\s+{re.escape(group_name)}\b",
 text,
 flags=re.IGNORECASE,
):
 continue
 missing_initialize_names.append(group_name)

if not missing_initialize_names:
 return text

initialize_lines = [
 "* DB2: Initialize generated DCLGEN host groups."
]
for group_name in missing_initialize_names:
 initialize_lines.append(f"INITIALIZE {group_name}.")

block = "\n".join(initialize_lines)
return self._insert_after_procedure_division(text=text, block=block)

def _insert_after_procedure_division(self, text: str, block: str) -> str:
 lines = str(text or "").splitlines()
 output: list[str] = []
 inserted = False

 for line in lines:
 output.append(line)
 if inserted:
 continue
 logical = self.line_utils.logical_line(line)
 if PROCEDURE_DIVISION_PATTERN.match(logical):
 output.append(block)
 inserted = True

 if inserted:
 return "\n".join(output).rstrip() + "\n"

 return str(text or "").rstrip() + "\n\n" + block + "\n"

```

```

=====
FILE: src\idms_db2_phase2\generators\db2_infrastructure\cobol_line_utils.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/generators/db2_infrastructure/cobol_line_utils.py
ACTION: CREATE NEW FILE

```

```

"""Stateless COBOL line helpers for DB2 infrastructure generation."""

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class CobolLineUtils:
 def logical_line(self, line: str) -> str:
 text = str(line or "").rstrip()

 if len(text) >= 80:
 left = text[:6]
 body = text[7:72]
 if left.strip().isdigit():
 return body.strip()

```

```

 if len(text) > 6 and text[:6].strip().isdigit():
 return text[6:].strip()

 return text.strip()

def comment_block(self, title: str) -> list[str]:
 return [f"* {title:<62}*"]

def comma_lines(self, items: list[str], indent: str) -> list[str]:
 clean_items = [
 str(item or "").strip()
 for item in items
 if str(item or "").strip()
]
 output: list[str] = []
 for index, item in enumerate(clean_items):
 if index == 0:
 output.append(f"{indent}{item}")
 else:
 output.append(f"{indent}, {item}")
 return output

def and_lines(self, items: list[str], indent: str) -> list[str]:
 clean_items = [
 str(item or "").strip()
 for item in items
 if str(item or "").strip()
]
 output: list[str] = []
 for index, item in enumerate(clean_items):
 prefix = "AND " if index > 0 else ""
 output.append(f"{indent}{prefix}{item}")
 return output

def normalize_include_name(self, value: str) -> str:
 return NameNormalizer.normalize(value)

```

```

=====
FILE: src\idms_db2_phase2\generators\db2_infrastructure\cursor_declare_builder.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/generators/db2_infrastructure/cursor_declare_builder.py
ACTION: CREATE NEW FILE

```

```

"""Builds a DECLARE CURSOR statement.

```

```

EXEC SQL
 DECLARE DZBEFFC1 CURSOR WITH HOLD FOR
 SELECT
 CO_IDPRKSK_479BEFF
 , NS_IRMOSTK_479BEFF
 FROM DZBEFFTV
 FOR READ ONLY
END-EXEC.

```

Every line sits in Area B. The block form is used here deliberately: a DECLARE is long enough that the single-line shape used for INCLUDE would not fit columns 8 to 72.

Nothing is fabricated. A spec without a cursor name or without a resolved DB2 table produces a DB2 WARNING comment rather than guessed SQL, per CONVERSION\_RULES.

```

"""

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.generators.db2_infrastructure.cursor_spec import CursorSpec
from rules.db2_infrastructure_rules import (
 DECLARE_TEMPLATE,
 FROM_TEMPLATE,
 IND_EXEC,
 IND_SELECT_FIRST,
 IND_SELECT_NEXT,
 IND_SQL_BODY,

```

```

IND_WHERE_NEXT,
MISSING_CURSOR_NAME,
MISSING_TABLE_TEMPLATE,
SELECT_ALL,
SELECT_ITEM_FIRST_TEMPLATE,
SELECT_ITEM_NEXT_TEMPLATE,
TOKEN_END_EXEC,
TOKEN_EXEC_SQL,
TOKEN_FOR_READ_ONLY,
TOKEN_ORDER_BY,
TOKEN_SELECT,
TOKEN_WHERE,
)

class CursorDeclareBuilder:
 """Renders one DECLARE CURSOR statement."""

 def __init__(self, line_utils) -> None:
 self.line_utils = line_utils

 # =====
 # Public entry point
 # =====
 def build(self, spec: CursorSpec) -> list[str]:
 if not spec.cursor_name:
 return [MISSING_CURSOR_NAME]

 if not spec.table_name:
 return [MISSING_TABLE_TEMPLATE.format(cursor=spec.cursor_name)]

 columns = spec.select_columns or [SELECT_ALL]

 lines: list[str] = [
 f"{IND_EXEC}{TOKEN_EXEC_SQL}",
 f"{IND_SQL_BODY}"
 f"{DECLARE_TEMPLATE.format(cursor=spec.cursor_name)}",
 f"{IND_SQL_BODY}{TOKEN_SELECT}",
]
 lines.extend(self._select_lines(columns))
 lines.append(
 f"{IND_SQL_BODY}{FROM_TEMPLATE.format(table=spec.table_name)}"
)
 lines.extend(self._where_lines(spec))
 lines.extend(self._order_by_lines(spec))
 lines.append(f"{IND_SQL_BODY}{TOKEN_FOR_READ_ONLY}")
 lines.append(f"{IND_EXEC}{TOKEN_END_EXEC}")

 return lines

 # =====
 # Clauses
 # =====
 @staticmethod
 def _select_lines(columns: list[str]) -> list[str]:
 """Comma-leading list, so the column names line up."""
 out: list[str] = []

 for index, column in enumerate(columns):
 if index == 0:
 out.append(
 IND_SELECT_FIRST
 + SELECT_ITEM_FIRST_TEMPLATE.format(column=column)
)
 else:
 out.append(
 IND_SELECT_NEXT
 + SELECT_ITEM_NEXT_TEMPLATE.format(column=column)
)

 return out

 def _where_lines(self, spec: CursorSpec) -> list[str]:
 if not spec.where_conditions:
 return []

```

```

 return [
 f"{IND_SQL_BODY}{TOKEN_WHERE}",
 *self.line_utils.and_lines(
 items=spec.where_conditions,
 indent=IND_WHERE_NEXT,
),
]

```

```

def _order_by_lines(self, spec: CursorSpec) -> list[str]:
 if not spec.order_by_columns:
 return []

 return [
 f"{IND_SQL_BODY}{TOKEN_ORDER_BY}",
 *self.line_utils.comma_lines(
 items=spec.order_by_columns,
 indent=IND_WHERE_NEXT,
),
]

```

```

=====
FILE: src\idms_db2_phase2\generators\db2_infrastructure\cursor_flag_builder.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/generators/db2_infrastructure/cursor_flag_builder.py
ACTION: CREATE NEW FILE

```

```

"""Builds the end-of-cursor flag group for each cursor.

```

```

 01 WS-DZBEFFC1-FLAG PIC X.
 88 DZBEFFC1-NOT-EOC VALUE 'N'.
 88 DZBEFFC1-EOC VALUE 'Y'.

```

```

The 01 level is Area A by definition. The 88 condition names are
subordinate entries and belong in Area B, which CHK-06.05 enforces.
"""

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.generators.db2_infrastructure.cursor_spec import CursorSpec
from rules.db2_infrastructure_rules import (
 CONDITION_TEMPLATE,
 EOC_TEMPLATE,
 FLAG_DECLARATION_TEMPLATE,
 FLAG_NAME_TEMPLATE,
 IND_88_LEVEL,
 NOT_EOC_TEMPLATE,
 VALUE_EOC,
 VALUE_NOT_EOC,
)

```

```

class CursorFlagBuilder:

```

```

 """Renders one flag group per cursor."""

```

```

 def build(self, specs: list[CursorSpec]) -> list[str]:
 lines: list[str] = []

```

```

 for spec in specs or []:
 lines.extend(self._group(spec))

```

```

 return lines

```

```

=====
One group
=====

```

```

def _group(self, spec: CursorSpec) -> list[str]:
 if not spec.cursor_name:
 return []

```

```

 cursor = spec.cursor_name

```

```

 return [
 FLAG_DECLARATION_TEMPLATE.format(

```

```

 name=FLAG_NAME_TEMPLATE.format(cursor=cursor)
),
 IND_88_LEVEL
+ CONDITION_TEMPLATE.format(
 name=NOT_EOC_TEMPLATE.format(cursor=cursor),
 value=VALUE_NOT_EOC,
),
 IND_88_LEVEL
+ CONDITION_TEMPLATE.format(
 name=EOC_TEMPLATE.format(cursor=cursor),
 value=VALUE_EOC,
),
 "",
]

```

```

=====
FILE: src\idms_db2_phase2\generators\db2_infrastructure\cursor_spec.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/generators/db2_infrastructure/cursor_spec.py
ACTION: CREATE NEW FILE

```

```

"""Typed, cleaned view of a cursor spec dictionary.

```

```

Db2InfrastructureGenerator builds specs as plain dicts. Every consumer
was re-normalising the same five keys, each with its own idea of what
"clean" meant. This value object does it once.

```

```

Column ORDER is authoritative. The FETCH INTO host list is built from
the same sequence, so a reordering here silently corrupts every fetched
row. De-duplication therefore preserves first-seen order and never
sorts.
"""

```

```

from __future__ import annotations

```

```

from dataclasses import dataclass, field

```

```

from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

@dataclass(frozen=True)

```

```

class CursorSpec:

```

```

 """One cursor, ready to render."""

```

```

 cursor_name: str = ""

```

```

 table_name: str = ""

```

```

 record_name: str = ""

```

```

 select_columns: list[str] = field(default_factory=list)

```

```

 where_conditions: list[str] = field(default_factory=list)

```

```

 order_by_columns: list[str] = field(default_factory=list)

```

```

 @property

```

```

 def is_declarable(self) -> bool:

```

```

 return bool(self.cursor_name and self.table_name)

```

```

 @property

```

```

 def is_child(self) -> bool:

```

```

 """A cursor with a WHERE clause is a child of another cursor."""

```

```

 return bool(self.where_conditions)

```

```

 @property

```

```

 def not_eoc_suffix_source(self) -> str:

```

```

 return self.cursor_name

```

```

=====

```

```

Construction

```

```

=====

```

```

@classmethod

```

```

def read(cls, spec: dict[str, object]) -> "CursorSpec":

```

```

 source = spec or {}

```

```

 return cls(

```

```

 cursor_name=NameNormalizer.to_cobol(

```

```

 str(source.get("cursor_name", ""))
),
 table_name=NameNormalizer.normalize(
 str(source.get("table_name", ""))
),
 record_name=NameNormalizer.normalize(
 str(source.get("record_name", ""))
),
 select_columns=cls._columns(source.get("select_columns")),
 where_conditions=cls._items(source.get("where_conditions")),
 order_by_columns=cls._items(source.get("order_by_columns")),
)

@classmethod
def read_all(cls, specs: list[dict[str, object]]) -> list["CursorSpec"]:
 return [cls.read(spec) for spec in specs or []]

=====
Cleaning
=====
@staticmethod
def _columns(value: object) -> list[str]:
 """Normalised column names, de-duplicated, order preserved."""
 out: list[str] = []
 seen: set[str] = set()

 for column in list(value or []):
 normalized = NameNormalizer.normalize(str(column))
 if not normalized or normalized in seen:
 continue
 seen.add(normalized)
 out.append(normalized)

 return out

@staticmethod
def _items(value: object) -> list[str]:
 """Trimmed free-text clause items, de-duplicated, order preserved."""
 out: list[str] = []
 seen: set[str] = set()

 for item in list(value or []):
 text = str(item or "").strip()
 if not text or text in seen:
 continue
 seen.add(text)
 out.append(text)

 return out

=====
FILE: src\idms_db2_phase2\generators\db2_infrastructure\cursor_spec_builder.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/generators/db2_infrastructure/cursor_spec_builder.py
ACTION: CREATE NEW FILE

"""Builds cursor specifications from IDMS operations."""

from __future__ import annotations

from idms_db2_phase2.analyzers.field_usage_analyzer import FieldUsageAnalyzer
from idms_db2_phase2.domain.models import IdmsOperation
from idms_db2_phase2.services.name_normalizer import NameNormalizer

CURSOR_OPERATIONS = {
 "OBTAIN_FIRST",
 "OBTAIN_NEXT",
 "FIND_FIRST",
}

class CursorSpecBuilder:
 def __init__(self, owner) -> None:

```

```

owner exposes resolvers, mapping_repository, cursor_column_resolver,
relationship_resolver, cursor_name_resolver, table_name_resolver,
host_variable_resolver, _cursor_paragraph_spec.
self.owner = owner

def build(
 self,
 operations: list[IdmsOperation],
 cobol_text: str = "",
 field_usage_analysis=None,
) -> list[dict[str, object]]:
 output: list[dict[str, object]] = []
 seen: set[tuple[str, str, str]] = set()
 cursor_order = 1

 if field_usage_analysis is None:
 field_usage_analysis = FieldUsageAnalyzer(
 mapping_repository=self.owner.mapping_repository,
 table_name_resolver=self.owner.table_name_resolver,
).analyze(cobol_text)

 for operation in operations or []:
 operation_name = str(operation.operation or "").upper()
 if operation_name not in CURSOR_OPERATIONS:
 continue

 record_name = NameNormalizer.normalize(operation.record_name)
 set_name = NameNormalizer.normalize(operation.set_name)
 if not record_name:
 continue

 table_name = self.owner.table_name_resolver.table_for_record(
 record_name
)
 if not table_name:
 continue

 key = (set_name, record_name, table_name)
 if key in seen:
 continue
 seen.add(key)

 cursor_name = (
 self.owner.cursor_name_resolver.cursor_name_from_table(
 table_name
)
)
 paragraph_spec = self.owner._cursor_paragraph_spec(
 cursor_order=cursor_order,
 table_name=table_name,
)
 select_columns = (
 self.owner.cursor_column_resolver.select_columns_for_record(
 record_name=record_name,
 field_usage_analysis=field_usage_analysis,
)
)
 relationship = (
 self.owner.relationship_resolver.resolve_for_child_record(
 record_name
)
)
 where_conditions = self._where_conditions_from_relationship(
 relationship.conditions
)
 order_by_columns = (
 self.owner.cursor_column_resolver.order_by_columns_for_record(
 record_name
)
)
 host_variables = (
 self.owner.host_variable_resolver.host_references_for_columns(
 table_name=table_name,
 columns=select_columns,
)
)

```



```

 output.append(
 {
 "set_name": set_name,
 "record_name": record_name,
 "table_name": table_name,
 "cursor_name": cursor_name,
 "select_columns": select_columns,
 "host_variables": host_variables,
 "where_conditions": where_conditions,
 "order_by_columns": order_by_columns,
 "open_paragraph": paragraph_spec["open_paragraph"],
 "fetch_paragraph": paragraph_spec["fetch_paragraph"],
 "close_paragraph": paragraph_spec["close_paragraph"],
 "cursor_order": cursor_order,
 }
)
 cursor_order += 1

 return output

def include_names(
 self,
 cursor_specs: list[dict[str, object]],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for spec in cursor_specs or []:
 table_name = NameNormalizer.normalize(
 str(spec.get("table_name", ""))
)
 if not table_name:
 continue
 include_name = self.owner.line_utils.normalize_include_name(
 table_name
)
 if not include_name or include_name in seen:
 continue
 seen.add(include_name)
 output.append(include_name)

 return output

def spec_messages(
 self,
 cursor_specs: list[dict[str, object]],
) -> list[str]:
 messages: list[str] = []

 for spec in cursor_specs:
 cursor_name = str(spec.get("cursor_name", ""))
 record_name = str(spec.get("record_name", ""))
 table_name = str(spec.get("table_name", ""))
 select_columns = list(spec.get("select_columns", []))
 host_variables = list(spec.get("host_variables", []))

 if not table_name:
 messages.append(
 f"DB2 infrastructure: cursor {cursor_name} has no "
 f"resolved DB2 table for record {record_name}."
)
 if not select_columns:
 messages.append(
 f"DB2 infrastructure: cursor {cursor_name} has no "
 f"resolved SELECT columns for record {record_name}."
)
 if not host_variables:
 messages.append(
 f"DB2 infrastructure: cursor {cursor_name} has no "
 f"resolved FETCH host variables for record {record_name}."
)

 return messages

def _where_conditions_from_relationship(self, conditions) -> list[str]:

```

```

output: list[str] = []
for condition in conditions or []:
 child_column = NameNormalizer.normalize(
 getattr(condition, "child_column", "")
)
 parent_host = str(
 getattr(condition, "parent_host_reference", "") or ""
).strip()
 if not child_column or not parent_host:
 continue
 output.append(f"{child_column} = {parent_host}")
return output

```

```

=====
FILE: src\idms_db2_phase2\generators\db2_infrastructure\include_renderer.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/generators/db2_infrastructure/include_renderer.py
ACTION: CREATE NEW FILE

```

```

"""Single source of truth for EXEC SQL INCLUDE emission and detection.

```

Every component that emits or looks for an include routes through here, so the shape, the indent and the duplicate test cannot drift apart again.

The detection side is deliberately SHAPE-AGNOSTIC: it recognises an include whether it was written on one line or across a three-line block. Duplicate detection that saw only one shape is exactly what let DZBFARTV be injected twice.

```

from __future__ import annotations

```

```

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.db2_include_patterns import (
 END_EXEC_PATTERN,
 EXEC_SQL_OPEN_PATTERN,
 INCLUDE_BODY_PATTERN,
 INCLUDE_NAME_PATTERN,
 INCLUDE_SINGLE_PATTERN,
)
from rules.db2_include_rules import (
 IND_INCLUDE,
 IND_INCLUDE_BODY,
 INCLUDE_BLOCK_BODY_TEMPLATE,
 INCLUDE_BLOCK_CLOSE,
 INCLUDE_BLOCK_OPEN,
 INCLUDE_SINGLE_LINE,
 INCLUDE_SINGLE_TEMPLATE,
 INFRASTRUCTURE_INCLUDES,
)

```

```

class IncludeRenderer:
 """Renders include statements and finds the ones already present."""

 def __init__(self, line_utils=None) -> None:
 self.line_utils = line_utils

 # =====
 # Rendering
 # =====
 def render(self, name: str) -> list[str]:
 """One include, in the site's canonical shape and indent."""
 clean = self.normalize(name)
 if not clean:
 return []

 if INCLUDE_SINGLE_LINE:
 return [
 IND_INCLUDE + INCLUDE_SINGLE_TEMPLATE.format(name=clean)
]

```

```

 return [
 IND_INCLUDE + INCLUDE_BLOCK_OPEN,
 IND_INCLUDE_BODY
 + INCLUDE_BLOCK_BODY_TEMPLATE.format(name=clean),
 IND_INCLUDE + INCLUDE_BLOCK_CLOSE,
]

def render_all(self, names: list[str]) -> list[str]:
 """Render each distinct include once, order preserved."""
 out: list[str] = []
 seen: set[str] = set()

 for name in names or []:
 clean = self.normalize(name)
 if not clean or clean in seen:
 continue
 seen.add(clean)
 out.extend(self.render(clean))

 return out

=====
Detection
=====
def existing(self, lines: list[str]) -> set[str]:
 """Every include already present, whichever shape carries it."""
 found: set[str] = set()

 for line in lines or []:
 logical = self._logical(line)
 if not logical:
 continue

 match = INCLUDE_SINGLE_PATTERN.match(logical)
 if match:
 found.add(self.normalize(match.group("name")))
 continue

 match = INCLUDE_BODY_PATTERN.match(logical)
 if match:
 found.add(self.normalize(match.group("name")))

 found.discard("")
 return found

def counts(self, lines: list[str]) -> dict[str, int]:
 """How many times each include appears. Duplicates show as > 1."""
 totals: dict[str, int] = {}
 inside_block = False

 for line in lines or []:
 logical = self._logical(line)
 if not logical:
 continue

 single = INCLUDE_SINGLE_PATTERN.match(logical)
 if single:
 name = self.normalize(single.group("name"))
 totals[name] = totals.get(name, 0) + 1
 continue

 if EXEC_SQL_OPEN_PATTERN.match(logical):
 inside_block = True
 continue

 if END_EXEC_PATTERN.match(logical):
 inside_block = False
 continue

 if not inside_block:
 continue

 body = INCLUDE_BODY_PATTERN.match(logical)
 if body:
 name = self.normalize(body.group("name"))
 totals[name] = totals.get(name, 0) + 1

```

```

totals.pop("", None)
return totals

def missing(
 self,
 lines: list[str],
 required: list[str],
) -> list[str]:
 """Required includes not already present, order preserved."""
 present = self.existing(lines)
 out: list[str] = []
 seen: set[str] = set()

 for name in required or []:
 clean = self.normalize(name)
 if not clean or clean in present or clean in seen:
 continue
 seen.add(clean)
 out.append(clean)

 return out

=====
Classification
=====
@staticmethod
def is_infrastructure(name: str) -> bool:
 """SQLCA, SQLERRWS, SQLERROR and GEN are not DCLGEN tables."""
 return NameNormalizer.normalize(name).upper() in INFRASTRUCTURE_INCLUDES

=====
Helpers
=====
@staticmethod
def normalize(name: str) -> str:
 return NameNormalizer.normalize(str(name or "")).upper()

def _logical(self, line: str) -> str:
 """Strip the sequence area, whatever helper the caller supplied."""
 if self.line_utils is not None:
 getter = getattr(self.line_utils, "logical", None)
 if callable(getter):
 return str(getter(line) or "").strip().upper()

 from patterns.sequence_patterns import strip_sequence_numbers

 return strip_sequence_numbers(str(line or "")).strip().upper()

```

```

=====
FILE: src\idms_db2_phase2\generators\db2_infrastructure\infrastructure_block_builder.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/generators/db2_infrastructure/infrastructure_block_builder.py
ACTION: REPLACE ENTIRE FILE

```

```

"""Assembles the generated DB2 infrastructure DATA DIVISION block.

```

Assembly order only. Every element is rendered by a focused builder:

- |                               |                                     |
|-------------------------------|-------------------------------------|
| 1. Infrastructure marker      | comment_block                       |
| 2. SQLERRWS / SQLCA / DCLGEN  | IncludeRenderer                     |
| 3. SQL-LOCATION               | SqlLocationBuilder (off by default) |
| 4. Cursor end-of-cursor flags | CursorFlagBuilder                   |
| 5. Cursor DECLARE statements  | CursorDeclareBuilder                |

CORRECTION - duplicate DZBFARTV include

-----

This builder rendered its own include statements as a three-line block, while the update postprocess rendered a single-line form and scanned for only that shape. It therefore did not recognise the block already present and injected DZBFARTV a second time. A duplicated DCLGEN include duplicates every data-name in the group, which the compiler rejects.

Include emission AND detection now go through IncludeRenderer, so one shape and one duplicate test serve every caller.

CORRECTION - generated blocks sat in Area A

-----  
Lines were emitted with no indent, so after fixed-format sequencing  
EXEC SQL and END-EXEC landed at column 8. CHK-06.07 requires column 12.  
Indents now come from rules/db2\_infrastructure\_rules.py.

CORRECTION - duplicate SQL-LOCATION

-----  
See sql\_location\_builder.py. SQLERRWS owns the field.

Clean Architecture

-----  
- Section titles and include names: catalogs/output\_sections.py  
- Indents, tokens and templates: rules/db2\_infrastructure\_rules.py  
- No regex here.  
- No program, record, table, cursor or host variable name hardcoded.  
"""

```
from __future__ import annotations

from catalogs.output_sections import (
 DB2_CURSOR_DECLARATIONS_MARKER,
 DB2_CURSOR_FLAGS_MARKER,
 DB2_INFRASTRUCTURE_MARKER,
 SQLCA_INCLUDE_NAME,
 SQLERRWS_INCLUDE_NAME,
)
from idms_db2_phase2.generators.db2_infrastructure.cursor_declare_builder import (
 CursorDeclareBuilder,
)
from idms_db2_phase2.generators.db2_infrastructure.cursor_flag_builder import (
 CursorFlagBuilder,
)
from idms_db2_phase2.generators.db2_infrastructure.cursor_spec import CursorSpec
from idms_db2_phase2.generators.db2_infrastructure.include_renderer import (
 IncludeRenderer,
)
from idms_db2_phase2.generators.db2_infrastructure.sql_location_builder import (
 SqlLocationBuilder,
)
```

```
class InfrastructureBlockBuilder:
 """Assembles the DB2 infrastructure block for WORKING-STORAGE."""

 def __init__(self, line_utils) -> None:
 self.line_utils = line_utils
 self.includes = IncludeRenderer(line_utils)
 self.sql_location = SqlLocationBuilder(line_utils)
 self.flags = CursorFlagBuilder()
 self.declares = CursorDeclareBuilder(line_utils)

 # =====
 # Public entry point
 # =====
 def build(
 self,
 include_names: list[str],
 cursor_specs: list[dict[str, object]],
) -> str:
 specs = CursorSpec.read_all(cursor_specs)

 lines: list[str] = []
 lines.extend(self._infrastructure_section(include_names))
 lines.extend(self.sql_location.build())

 if specs:
 lines.extend(self._flags_section(specs))
 lines.extend(self._declarations_section(specs))

 return "\n".join(lines).rstrip() + "\n"

 # =====
```

```

Sections
=====
def _infrastructure_section(self, include_names: list[str]) -> list[str]:
 """Marker comment, then SQLERRWS, SQLCA and each DCLGEN include.

 SQLERRWS leads because it declares SQL-LOCATION, which every
 generated paragraph moves into before issuing SQL.
 """
 return [
 *self.line_utils.comment_block(DB2_INFRASTRUCTURE_MARKER),
 *self.includes.render_all(
 [
 SQLERRWS_INCLUDE_NAME,
 SQLCA_INCLUDE_NAME,
 *(include_names or []),
]
),
]

def _flags_section(self, specs: list[CursorSpec]) -> list[str]:
 return [
 "",
 *self.line_utils.comment_block(DB2_CURSOR_FLAGS_MARKER),
 *self.flags.build(specs),
]

def _declarations_section(self, specs: list[CursorSpec]) -> list[str]:
 lines: list[str] = [
 "",
 *self.line_utils.comment_block(DB2_CURSOR_DECLARATIONS_MARKER),
]

 for spec in specs:
 lines.extend(self.declares.build(spec))
 lines.append("")

 return lines

```

```

=====
FILE: src\idms_db2_phase2\generators\db2_infrastructure\infrastructure_patterns.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/generators/db2_infrastructure/infrastructure_patterns.py
ACTION: CREATE NEW FILE

```

```

"""COBOL division/section regex patterns for DB2 infrastructure insertion."""

```

```

import re

```

```

PROCEDURE_DIVISION_PATTERN = re.compile(
 r"^\s*(?:\d{6}\s+)?PROCEDURE\s+DIVISION\b.*\s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

```

```

LINKAGE_SECTION_PATTERN = re.compile(
 r"^\s*(?:\d{6}\s+)?LINKAGE\s+SECTION\.\s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

```

```

WORKING_STORAGE_PATTERN = re.compile(
 r"^\s*(?:\d{6}\s+)?WORKING-STORAGE\s+SECTION\.\s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

```

```

DATA_DIVISION_PATTERN = re.compile(
 r"^\s*(?:\d{6}\s+)?DATA\s+DIVISION\.\s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

```

```
=====
FILE: src\idms_db2_phase2\generators\db2_infrastructure\sql_location_builder.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/generators/db2_infrastructure/sql_location_builder.py
ACTION: CREATE NEW FILE
```

```
"""Builds the SQL-LOCATION declaration, when the site wants one.
```

```
CORRECTION - duplicate data-name

```

```
The infrastructure block unconditionally emitted
```

```
01 SQL-LOCATION PIC X(40) VALUE SPACES.
```

```
while also emitting EXEC SQL INCLUDE SQLERRWS. SQLERRWS already declares
SQL-LOCATION, so the program carried a duplicate data-name and the
compiler rejected it.
```

```
The COBOL team's manual reference program has no local declaration: it
moves into the field and lets the copybook own it. Emission is therefore
off by default, controlled by
catalogs.output_sections.DECLARE_SQL_LOCATION_FIELD.
"""
```

```
from __future__ import annotations
```

```
from catalogs.output_sections import (
 DB2_SQL_ERROR_LOCATION_MARKER,
 DECLARE_SQL_LOCATION_FIELD,
 SQL_LOCATION_FIELD_NAME,
 SQL_LOCATION_PICTURE,
)
from rules.db2_infrastructure_rules import SQL_LOCATION_DECLARATION_TEMPLATE
```

```
class SqlLocationBuilder:
```

```
 """Renders the SQL-LOCATION block, or nothing at all."""
```

```
 def __init__(self, line_utils) -> None:
 self.line_utils = line_utils
```

```
 def build(self) -> list[str]:
 if not DECLARE_SQL_LOCATION_FIELD:
 return []
```

```
 return [
 "",
 *self.line_utils.comment_block(DB2_SQL_ERROR_LOCATION_MARKER),
 SQL_LOCATION_DECLARATION_TEMPLATE.format(
 name=SQL_LOCATION_FIELD_NAME,
 picture=SQL_LOCATION_PICTURE,
),
]
```

```
=====
FILE: src\idms_db2_phase2\generators\db2_infrastructure_generator.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/generators/db2_infrastructure_generator.py
ACTION: REPLACE ENTIRE FILE
```

```
"""
Generates DB2 infrastructure in DATA DIVISION.
```

```
Thin facade delegating to focused helpers. Public API preserved:
- apply(cobol_text, operations) -> (text, messages)
- cursor_specs(operations, cobol_text, field_usage_analysis)
- include_names(cursor_specs)
- infrastructure_block(include_names, cursor_specs)
- last_cursor_specs attribute (used by CursorParagraphGenerator)
"""
```

```

from __future__ import annotations

from idms_db2_phase2.analyzers.field_usage_analyzer import FieldUsageAnalyzer
from idms_db2_phase2.domain.models import IdmsOperation
from idms_db2_phase2.generators.db2_infrastructure.block_inserter import (
 BlockInserter,
)
from idms_db2_phase2.generators.db2_infrastructure.cobol_line_utils import (
 CobolLineUtils,
)
from idms_db2_phase2.generators.db2_infrastructure.cursor_spec_builder import (
 CursorSpecBuilder,
)
from idms_db2_phase2.generators.db2_infrastructure.infrastructure_block_builder import (
 InfrastructureBlockBuilder,
)
from catalogs.output_sections import DB2_INFRASTRUCTURE_MARKER
from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.cursor_column_resolver import (
 CursorColumnResolver,
)
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.resolvers.host_variable_resolver import (
 HostVariableResolver,
)
from idms_db2_phase2.resolvers.relationship_resolver import RelationshipResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver

class Db2InfrastructureGenerator:
 def __init__(
 self,
 table_name_resolver: TableNameResolver,
 column_name_resolver: ColumnNameResolver,
 host_variable_resolver: HostVariableResolver,
 cursor_name_resolver: CursorNameResolver,
) -> None:
 self.table_name_resolver = table_name_resolver
 self.column_name_resolver = column_name_resolver
 self.host_variable_resolver = host_variable_resolver
 self.cursor_name_resolver = cursor_name_resolver

 self.mapping_repository = table_name_resolver.mapping_repository

 self.relationship_resolver = RelationshipResolver(
 mapping_repository=self.mapping_repository,
 table_name_resolver=self.table_name_resolver,
 host_variable_resolver=self.host_variable_resolver,
)
 self.cursor_column_resolver = CursorColumnResolver(
 mapping_repository=self.mapping_repository,
 table_name_resolver=self.table_name_resolver,
 column_name_resolver=self.column_name_resolver,
 relationship_resolver=self.relationship_resolver,
)

 self.line_utils = CobolLineUtils()
 self._spec_builder = CursorSpecBuilder(self)
 self._block_builder = InfrastructureBlockBuilder(self.line_utils)
 self._inserter = BlockInserter(self.line_utils)

 self.last_cursor_specs: list[dict[str, object]] = []

 def apply(
 self,
 cobol_text: str,
 operations: list[IdmsOperation],
) -> tuple[str, list[str]]:
 messages: list[str] = []
 text = str(cobol_text or "")

 if not text:
 self.last_cursor_specs = []
 return text, messages

 field_usage_analysis = FieldUsageAnalyzer(

```



```

 mapping_repository=self.mapping_repository,
 table_name_resolver=self.table_name_resolver,
).analyze(text)
messages.extend(field_usage_analysis.diagnostics)

cursor_specs = self.cursor_specs(
 operations=operations,
 cobol_text=text,
 field_usage_analysis=field_usage_analysis,
)
self.last_cursor_specs = cursor_specs

include_names = self.include_names(cursor_specs)
if include_names:
 messages.append(
 "DB2 infrastructure: DCLGEN includes selected: "
 + ", ".join(include_names)
)
else:
 messages.append(
 "DB2 infrastructure: no operation-specific DCLGEN includes "
 "resolved."
)

messages.extend(self._spec_builder.spec_messages(cursor_specs))

if DB2_INFRASTRUCTURE_MARKER in text:
 messages.append(
 "DB2 infrastructure: existing generated DB2 infrastructure "
 "block detected; not inserted again."
)
 updated = text
else:
 block = self.infrastructure_block(
 include_names=include_names,
 cursor_specs=cursor_specs,
)
 updated = self._inserter.insert_in_data_division(
 text=text,
 block=block,
)
 messages.append(
 "DB2 infrastructure: generated infrastructure block inserted."
)

updated = self._inserter.ensure_dclgen_initialization(
 text=updated,
 include_names=include_names,
)
return updated, messages

def cursor_specs(
 self,
 operations: list[IdmsOperation],
 cobol_text: str = "",
 field_usage_analysis=None,
) -> list[dict[str, object]]:
 return self._spec_builder.build(
 operations=operations,
 cobol_text=cobol_text,
 field_usage_analysis=field_usage_analysis,
)

def include_names(
 self,
 cursor_specs: list[dict[str, object]],
) -> list[str]:
 return self._spec_builder.include_names(cursor_specs)

def infrastructure_block(
 self,
 include_names: list[str],
 cursor_specs: list[dict[str, object]],
) -> str:
 return self._block_builder.build(
 include_names=include_names,

```

```

 cursor_specs=cursor_specs,
)

def _cursor_paragraph_spec(
 self,
 cursor_order: int,
 table_name: str,
) -> dict[str, str]:
 if hasattr(self.cursor_name_resolver, "cursor_spec"):
 return self.cursor_name_resolver.cursor_spec(
 cursor_order=cursor_order,
 table_name=table_name,
)

 cursor_name = self.cursor_name_resolver.cursor_name_from_table(
 table_name
)
 base = 710 + ((cursor_order - 1) * 100)
 return {
 "cursor_name": cursor_name,
 "open_paragraph": f"{base}-OPEN-{cursor_name}",
 "fetch_paragraph": f"{base + 10}-FETCH-{cursor_name}",
 "close_paragraph": f"{base + 20}-CLOSE-{cursor_name}",
 }

```

```

=====
FILE: src\idms_db2_phase2\generators\sql__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\generators\sql\sql_column_resolver.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/generators/sql/sql_column_resolver.py
ACTION: CREATE NEW FILE

```

```

"""SQL column resolution: table, key columns, insert columns, DCLGEN filter."""

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.timestamp_audit_rules import INSERT_EXCLUDE_AUDIT_PREFIXES

```

```

class SqlColumnResolver:
 def __init__(
 self,
 mapping_repository,
 dclgen_repository,
 table_name_resolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver

 def resolved_table_for_record(self, record_name: str) -> str:
 mapped_table = self.mapping_repository.db2_table_for_record(
 record_name
)
 if not mapped_table:
 return ""
 return self.table_name_resolver.resolve_table(mapped_table)

 def key_columns_for_record(
 self,
 record_name: str,
 table_name: str,
) -> list[str]:
 """
 Key columns for SQL WHERE. Composite PK / CALC keys required; FK /

```

```

FOREIGN columns never included; Sheet Mapping order preserved.
"""
columns: list[str] = []

if hasattr(self.mapping_repository, "rows_for_record"):
 columns = self._key_columns_from_mapping_rows(record_name)

if not columns and hasattr(
 self.mapping_repository,
 "primary_key_columns_for_record",
):
 columns = self.mapping_repository.primary_key_columns_for_record(
 record_name
)

return self.filter_to_existing_dclgen_columns(
 table_name=table_name,
 columns=columns,
)

def _key_columns_from_mapping_rows(
 self,
 record_name: str,
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for row in self.mapping_repository.rows_for_record(record_name):
 idms_key = NameNormalizer.normalize(getattr(row, "idms_key", ""))
 db2_key = NameNormalizer.normalize(getattr(row, "db2_key", ""))
 relation = NameNormalizer.normalize(getattr(row, "relation", ""))
 column = NameNormalizer.normalize(
 getattr(row, "new_db2_field_name", "")
 or getattr(row, "cross_application_db2_field_name", "")
)
 if not column:
 continue

 combined_text = " ".join([idms_key, db2_key, relation])
 padded_text = f" {combined_text} "

 if "FOREIGN" in combined_text:
 continue
 if " FK " in padded_text:
 continue

 is_key_column = (
 "PRIMARY" in db2_key
 or db2_key == "KEY"
 or "PRIMARY" in idms_key
 or idms_key == "KEY"
 or "CALC" in idms_key
)
 if not is_key_column:
 continue
 if column in seen:
 continue

 seen.add(column)
 output.append(column)

 return output

def insert_columns_for_record(
 self,
 record_name: str,
 table_name: str,
) -> list[str]:
 columns = self.mapping_repository.db2_columns_for_table(table_name)
 columns = [
 column
 for column in columns
 if not self._is_insert_excluded_audit_column(column)
]
 return self.filter_to_existing_dclgen_columns(table_name, columns)

```

```

def minimal_select_columns(
 self,
 record_name: str,
 table_name: str,
 key_columns: list[str],
) -> list[str]:
 return self.filter_to_existing_dclgen_columns(
 table_name=table_name,
 columns=key_columns,
)

def filter_to_existing_dclgen_columns(
 self,
 table_name: str,
 columns: list[str],
) -> list[str]:
 resolved_table = self.table_name_resolver.resolve_table(table_name)
 if not resolved_table:
 return []

 dclgen_columns = set(
 self.dclgen_repository.column_names_for_table(resolved_table)
)
 if not dclgen_columns:
 return []

 output: list[str] = []
 for column in columns:
 normalized = NameNormalizer.normalize(column)
 if not normalized:
 continue
 if normalized not in dclgen_columns:
 continue
 output.append(normalized)

 return self._unique(output)

def _is_insert_excluded_audit_column(self, column_name: str) -> bool:
 column = NameNormalizer.normalize(column_name)
 return any(
 column.startswith(prefix)
 for prefix in INSERT_EXCLUDE_AUDIT_PREFIXES
)

def _unique(self, values: list[str]) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()
 for value in values:
 normalized = NameNormalizer.normalize(value)
 if not normalized or normalized in seen:
 continue
 seen.add(normalized)
 output.append(normalized)
 return output

```

```

=====
FILE: src\idms_db2_phase2\generators\sql\sql_date_move_builder.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/generators/sql/sql_date_move_builder.py
ACTION: REPLACE ENTIRE FILE

```

```

"""DB2 date column detection and CCYYMMDD -> DD.MM.CCYY move builder."""

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.sql_generation_rules import (
 DATE_COLUMN_PREFIXES,
 DATE_STAGING_FIELDS,
 DB2_DATE_NULL_SENTINEL,
)

```

```

class SqlDateMoveBuilder:

 def is_db2_date_column(self, column_name: str) -> bool:
 column = NameNormalizer.normalize(column_name)
 return any(
 column.startswith(prefix) for prefix in DATE_COLUMN_PREFIXES
)

 @staticmethod
 def _is_staging_field(source: str) -> bool:
 """True when the source is already a DB2 date staging field.

 A staging field is this builder's OUTPUT, not a valid input. Passing
 DA-DD-MM-CCYY back in moves 10 bytes into an 8-byte group, truncating
 the year, and the CORR move back writes garbage into the DB2 host.
 """
 return str(source or "").strip().upper() in DATE_STAGING_FIELDS

 def date_ymd8_to_db2_external_move(
 self,
 source_value: str,
 host_key: str,
) -> list[str]:
 """Convert a CCYYMMDD source value into DB2 external date DD.MM.CCYY,
 then move it into the DB2 date host field.

 Manual standard:
 - Clear DA-CCYYMMDD.
 - Move the source date into DA-CCYYMMDD.
 - If the date is ZERO or SPACES, substitute the DB2 low-date sentinel
 (00010101) instead of moving SPACES to the target.
 - Realign via DA-CCYYMMDD-R -> DA-DD-MM-CCYY.

 Precondition: source_value must be a CCYYMMDD (8-position) field.
 A staging field is rejected by _is_staging_field and moved directly,
 because it is already in DB2 external form.
 """
 source = str(source_value or "").strip()

 # Already converted. Re-running the block would destroy the value.
 if self._is_staging_field(source):
 return [
 f"MOVE DA-DD-MM-CCYY TO {host_key}",
]

 if not source:
 return [
 "MOVE ZEROES TO DA-CCYYMMDD",
 f"MOVE {DB2_DATE_NULL_SENTINEL} TO DA-CCYYMMDD",
 "MOVE CORR DA-CCYYMMDD-R TO DA-DD-MM-CCYY",
 f"MOVE DA-DD-MM-CCYY TO {host_key}",
]

 return [
 "MOVE ZEROES TO DA-CCYYMMDD",
 f"MOVE {source} TO DA-CCYYMMDD",
 "IF DA-CCYYMMDD = ZERO OR DA-CCYYMMDD = SPACES",
 f" MOVE {DB2_DATE_NULL_SENTINEL} TO DA-CCYYMMDD",
 "END-IF",
 "MOVE CORR DA-CCYYMMDD-R TO DA-DD-MM-CCYY",
 f"MOVE DA-DD-MM-CCYY TO {host_key}",
]

=====
FILE: src\idms_db2_phase2\generators\sql\sql_line_builder.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/generators/sql/sql_line_builder.py
ACTION: CREATE NEW FILE

"""SQL line builders: SET, WHERE, column list, host list, skipped block."""

from __future__ import annotations

```

```

from idms_db2_phase2.services.name_normalizer import NameNormalizer

class SqlLineBuilder:
 def __init__(self, host_variable_resolver) -> None:
 self.host_variable_resolver = host_variable_resolver

 def set_lines(
 self,
 table_name: str,
 columns: list[str],
 indent: str,
) -> list[str]:
 assignments: list[tuple[str, str]] = []

 for column in columns:
 normalized = NameNormalizer.normalize(column)
 if not normalized:
 continue
 host = self.host_variable_resolver.host_reference_for_column(
 table_name=table_name,
 column_name=normalized,
)
 if not host:
 continue
 assignments.append((normalized, host))

 output: list[str] = []
 for index, item in enumerate(assignments):
 column, host = item
 suffix = "," if index < len(assignments) - 1 else ""
 output.append(f"{indent}{column} = {host}{suffix}")

 return output

 def where_lines(
 self,
 table_name: str,
 key_columns: list[str],
 indent: str,
) -> list[str]:
 """
 Build SQL WHERE lines from composite key columns. The caller passes
 only PK / CALC key columns; FK exclusion is handled upstream.
 """
 conditions: list[tuple[str, str]] = []

 for column in key_columns:
 normalized = NameNormalizer.normalize(column)
 if not normalized:
 continue
 host = self.host_variable_resolver.host_reference_for_column(
 table_name=table_name,
 column_name=normalized,
)
 if not host:
 continue
 conditions.append((normalized, host))

 output: list[str] = []
 for index, item in enumerate(conditions):
 column, host = item
 prefix = "" if index == 0 else "AND "
 output.append(f"{indent}{prefix}{column} = {host}")

 return output

 def sql_column_list(
 self,
 columns: list[str],
 indent: str,
) -> list[str]:
 output: list[str] = []
 for index, column in enumerate(columns):
 suffix = "," if index < len(columns) - 1 else ""
 output.append(f"{indent}{column}{suffix}")

```

```

 return output

 def host_variable_list(
 self,
 host_variables: list[str],
 indent: str,
) -> list[str]:
 output: list[str] = []
 for index, host in enumerate(host_variables):
 suffix = "," if index < len(host_variables) - 1 else ""
 output.append(f"{indent}{host}{suffix}")
 return output

 def missing_sql(
 self,
 operation: str,
 record_name: str,
 reason: str,
) -> list[str]:
 record = NameNormalizer.to_cobol(record_name)
 return [
 f"*DB2: {operation} conversion skipped for {record}.",
 f"*DB2: {reason}.",
 "CONTINUE.",
]

=====
FILE: src\idms_db2_phase2\generators\sql\sql_statement_builder.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/generators/sql/sql_statement_builder.py
ACTION: REPLACE ENTIRE FILE

"""Builds DB2 statement bodies: INSERT / UPDATE / DELETE / SELECT-skip / MOVE."""

from __future__ import annotations

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.update_sql_cleanup_rules import UPDATE_QUERYNO

class SqlStatementBuilder:
 def __init__(
 self,
 column_resolver,
 line_builder,
 date_builder,
 host_variable_resolver,
 mapping_repository,
 update_plan_resolver,
) -> None:
 self.column_resolver = column_resolver
 self.line_builder = line_builder
 self.date_builder = date_builder
 self.host_variable_resolver = host_variable_resolver
 self.mapping_repository = mapping_repository
 self.update_plan_resolver = update_plan_resolver
 self.messages: list[str] = []

 def _sqlcode_evaluate_block(self, operation: str, table: str) -> list[str]:
 """Manual-standard SQLCODE handling shared by INSERT/UPDATE/DELETE."""
 return [
 f" QUERYNO {UPDATE_QUERYNO}",
 "END-EXEC",
 "EVALUATE SQLCODE",
 " WHEN 0",
 " CONTINUE",
 " WHEN OTHER",
 f" DISPLAY 'ERROR ON {operation} {table} '",
 " PERFORM SQLERROR",
 "END-EVALUATE",
 ".",
]

```

```

def select_by_key(self, record_name: str) -> list[str]:
 self.messages = []
 record = NameNormalizer.normalize(record_name)
 table = self.column_resolver.resolved_table_for_record(record)

 if not record or not table:
 return self.line_builder.missing_sql(
 operation="OBTAIN CALC",
 record_name=record_name,
 reason="missing Sheet Mapping or DCLGEN table metadata",
)

 key_columns = self.column_resolver.key_columns_for_record(
 record_name=record,
 table_name=table,
)
 if not key_columns:
 return self.line_builder.missing_sql(
 operation="OBTAIN CALC",
 record_name=record,
 reason="missing key column metadata",
)

 return [
 f"*DB2: Removed OBTAIN CALC SELECT for "
 f"{NameNormalizer.to_cobol(record}}.",
 f"*DB2: Direct UPDATE will use mapped composite key WHERE clause.",
 "CONTINUE.",
]

def insert(self, record_name: str) -> list[str]:
 self.messages = []
 record = NameNormalizer.normalize(record_name)
 table = self.column_resolver.resolved_table_for_record(record)

 if not record or not table:
 return self.line_builder.missing_sql(
 operation="INSERT",
 record_name=record_name,
 reason="missing Sheet Mapping or DCLGEN table metadata",
)

 columns = self.column_resolver.insert_columns_for_record(
 record_name=record,
 table_name=table,
)
 if not columns:
 return self.line_builder.missing_sql(
 operation="INSERT",
 record_name=record,
 reason="missing insert columns",
)

 host_variables = (
 self.host_variable_resolver.host_references_for_columns(
 table_name=table,
 columns=columns,
)
)
 if not host_variables:
 return self.line_builder.missing_sql(
 operation="INSERT",
 record_name=record,
 reason="missing INSERT host variables in DCLGEN",
)

 lines: list[str] = [
 f"*DB2: Converted STORE for {NameNormalizer.to_cobol(record}}.",
 f"MOVE 'INSERT-{NameNormalizer.to_cobol(record}}' "
 f"TO SQL-LOCATION.",
 "EXEC SQL",
 f" INSERT INTO {table}",
 " (",
]
 lines.extend(
 self.line_builder.sql_column_list(columns, indent=" ")
)

```



```

)
lines.extend([")", " VALUES", " (")
lines.extend(
 self.line_builder.host_variable_list(
 host_variables, indent=" "
)
)
lines.append(" ")
lines.extend(self._sqlcode_evaluate_block("INSERT", table))
return lines

def update(
 self,
 record_name: str,
 changed_source_fields: list[str] | None = None,
) -> list[str]:
 self.messages = []
 record = NameNormalizer.normalize(record_name)
 plan = self.update_plan_resolver.resolve(
 record_name=record,
 changed_source_fields=changed_source_fields or [],
)
 self.messages.extend(plan.diagnostics)

 if plan.table_name:
 plan.key_columns = self.column_resolver.key_columns_for_record(
 record_name=record,
 table_name=plan.table_name,
)

 if not plan.is_complete:
 return self.line_builder.missing_sql(
 operation="UPDATE",
 record_name=record,
 reason="incomplete conservative UPDATE metadata",
)

 set_lines = self.line_builder.set_lines(
 table_name=plan.table_name,
 columns=plan.update_columns,
 indent=" ",
)
 where_lines = self.line_builder.where_lines(
 table_name=plan.table_name,
 key_columns=plan.key_columns,
 indent=" ",
)
 if not set_lines or not where_lines:
 return self.line_builder.missing_sql(
 operation="UPDATE",
 record_name=record,
 reason="missing SET or WHERE host variables in DCLGEN",
)

 lines: list[str] = [
 f"*DB2: Converted MODIFY for {NameNormalizer.to_cobol(record)}.",
 f"MOVE 'UPDATE-{NameNormalizer.to_cobol(record)}' "
 f"TO SQL-LOCATION.",
 "EXEC SQL",
 f" UPDATE {plan.table_name}",
 " SET",
]
 lines.extend(set_lines)
 lines.append(" WHERE")
 lines.extend(where_lines)
 lines.extend(self._sqlcode_evaluate_block("UPDATE", plan.table_name))
 return lines

def delete(self, record_name: str) -> list[str]:
 self.messages = []
 record = NameNormalizer.normalize(record_name)
 table = self.column_resolver.resolved_table_for_record(record)

 if not record or not table:
 return self.line_builder.missing_sql(
 operation="DELETE",

```

```

 record_name=record_name,
 reason="missing Sheet Mapping or DCLGEN table metadata",
)

 key_columns = self.column_resolver.key_columns_for_record(
 record_name=record,
 table_name=table,
)
 where_lines = self.line_builder.where_lines(
 table_name=table,
 key_columns=key_columns,
 indent=" ",
)
 if not where_lines:
 return self.line_builder.missing_sql(
 operation="DELETE",
 record_name=record,
 reason="missing DELETE key host variables in DCLGEN",
)

 lines: list[str] = [
 f"*DB2: Converted ERASE for {NameNormalizer.to_cobol(record)}.",
 f"MOVE 'DELETE-{NameNormalizer.to_cobol(record)}' "
 f"TO SQL-LOCATION.",
 "EXEC SQL",
 f" DELETE FROM {table}",
 " WHERE",
]
 lines.extend(where_lines)
 lines.extend(self._sqlcode_evaluate_block("DELETE", table))
 return lines

def commit(self) -> list[str]:
 return [
 "MOVE 'COMMIT' TO SQL-LOCATION.",
 "EXEC SQL",
 " COMMIT",
 "END-EXEC.",
]

def rollback(self) -> list[str]:
 return [
 "MOVE 'ROLLBACK' TO SQL-LOCATION.",
 "EXEC SQL",
 " ROLLBACK",
 "END-EXEC.",
]

def changed_field_move(
 self,
 record_name: str,
 source_value: str,
 target_source_field: str,
) -> list[str]:
 record = NameNormalizer.normalize(record_name)
 table = self.column_resolver.resolved_table_for_record(record)
 column = self.mapping_repository.column_for_source_field(
 record_name=record,
 source_field_name=target_source_field,
)
 column = NameNormalizer.normalize(column)

 if not table or not column:
 return [
 f"*DB2: Conversion skipped for MOVE target "
 f"{target_source_field}.",
 f"*DB2: Missing Sheet Mapping or DCLGEN metadata.",
 "CONTINUE.",
]

 host_key = self.host_variable_resolver.host_reference_key(
 table_name=table,
 column_name=column,
)
 if not host_key:
 return [

```

```

 f"*DB2: Conversion skipped for MOVE target "
 f"{target_source_field}.",
 "*DB2: Missing DCLGEN host variable.",
 "CONTINUE.",
]

 if self.date_builder.is_db2_date_column(column):
 return self.date_builder.date_ymd8_to_db2_external_move(
 source_value=source_value,
 host_key=host_key,
)

 return [f"MOVE {source_value} TO {host_key}"]

def missing_mapping(
 self,
 record_name: str,
 reason: str,
) -> list[str]:
 return self.line_builder.missing_sql(
 operation="DB2",
 record_name=record_name,
 reason=reason,
)

=====
FILE: src\idms_db2_phase2\generators\sql_error_generator.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/generators/sql_error_generator.py
ACTION: REPLACE ENTIRE FILE

"""Generates and ensures the SQLERROR routine.

Generic rule:
- Use SQLERROR, not SQL-ERROR.
- Generated DB2 cursor paragraphs and SQL blocks perform SQLERROR.
- The SQLERROR routine wraps the standard DB2 SQLERROR include.

Generated routine (manual reference layout):

 * SQLERROR ROUTINE.
 EXEC SQL
 INCLUDE SQLERROR
 END-EXEC.

Paragraph-header policy (Decision D-1, CLOSED by the manual reference):
- The SQLERROR copybook supplies the paragraph label itself.
- Emitting a local "SQLERROR." header duplicates that label and the
 compiler rejects the program with a duplicate paragraph-name error.
- code_review/standards/cobol_standards.py records the same decision:
 SQL_ERROR_BODY = ("EXEC SQL", "INCLUDE SQLERROR", "END-EXEC")
 SQL_ERROR_INCLUDE = "INCLUDE SQLERROR"
 and CHK-05.01 accepts "declared OR included".
- Set EMIT_SQL_ERROR_PARAGRAPH_HEADER = True in rules/sql_error_rules.py
 if a site ever needs the old behaviour back.

COBOL termination rule:
- A period inside EVALUATE terminates the COBOL *sentence*, which closes
 the EVALUATE early and orphans END-EVALUATE (compile error).
- This class therefore renames SQL-ERROR to SQLERROR but NEVER adds,
 moves or removes a period on a PERFORM inside an EVALUATE block. The
 original terminator is captured by the pattern and carried through
 unchanged.

Clean Architecture:
- No regex is defined in this file. Every pattern lives in
 patterns/cursor_paragraph_patterns.py and patterns/db2_patterns.py.
- Every literal lives in rules/sql_error_rules.py.
"""

from __future__ import annotations

import re

```

```

from patterns.cursor_paragraph_patterns import (
 LEGACY_SQL_ERROR_HEADER_PATTERN,
 LEGACY_SQL_ERROR_HEADER_REPLACEMENT,
 LEGACY_SQL_ERROR_PERFORM_PATTERN,
)
from patterns.db2_patterns import END_PROGRAM_PATTERN
from patterns.sql_error_patterns import (
 END_PROGRAM_BOUNDARY_PATTERN,
 PARAGRAPH_HEADER_PATTERN,
 SQLERROR_HEADER_ONLY_PATTERN,
 SQLERROR_PARAGRAPH_PATTERN,
)
from rules.sql_error_rules import (
 EMIT_SQL_ERROR_PARAGRAPH_HEADER,
 IND_SQL_BODY,
 IND_STATEMENT,
 OLD_BLOCK_BODY_TOKENS,
 SQL_ERROR_BANNER,
 SQL_ERROR_INCLUDE_NAME,
 SQL_ERROR_MESSAGES,
 SQL_ERROR_PARAGRAPH_NAME,
)

LEFT_SEQUENCE_WIDTH = 6
RIGHT_SEQUENCE_WIDTH = 8
FIXED_LINE_WIDTH = 80

class SqlErrorGenerator:
 """Owns the SQLERROR routine block and legacy-name normalisation."""

 SQLERROR_PARAGRAPH_NAME = SQL_ERROR_PARAGRAPH_NAME

 # Public alias consumed by CursorParagraphGenerator so both generators
 # read the paragraph name from one place and cannot drift apart.
 PARAGRAPH_NAME = SQL_ERROR_PARAGRAPH_NAME
 DEFAULT_PARAGRAPH_NAME = SQL_ERROR_PARAGRAPH_NAME

 # ... existing __init__ and methods unchanged ...

 # -----
 # Pipeline entry point - DO NOT RENAME
 # -----
 def ensure_sql_error_paragraph(self, text: str) -> str:
 """Called by orchestration/conversion/conversion_pipeline.py."""
 return self.ensure_paragraph(text)

 SQLERROR_PARAGRAPH_NAME = SQL_ERROR_PARAGRAPH_NAME

 def __init__(self) -> None:
 self.messages: list[str] = []

 # =====
 # Routine block
 # =====
 def paragraph_block(self) -> str:
 """The generated SQLERROR routine as one text block."""
 return "\n".join(self.paragraph_lines())

 def paragraph_lines(self) -> list[str]:
 """The generated SQLERROR routine as body lines.

 Manual reference emits a comment banner and the include only.
 The paragraph label itself comes from the SQLERROR copybook.
 """
 lines: list[str] = [SQL_ERROR_BANNER]

 if EMIT_SQL_ERROR_PARAGRAPH_HEADER:
 lines.append(f"{SQL_ERROR_PARAGRAPH_NAME}.")

 lines.append(f"{IND_STATEMENT}EXEC SQL")
 lines.append(f"{IND_SQL_BODY}INCLUDE {SQL_ERROR_INCLUDE_NAME}")
 lines.append(f"{IND_STATEMENT}END-EXEC.")
 return lines

```

```

=====
Ensure the routine exists exactly once
=====
def ensure_paragraph(self, text: str) -> str:
 """Normalise legacy names, then guarantee one SQLERROR routine."""
 self.messages = []

 output = self.rename_legacy(str(text or ""))
 if not output.strip():
 return output

 # An existing routine is rewritten in place so stale bodies
 # (DISPLAY / CALL USERABEN form) are replaced by the include form.
 if self._has_sqlerror_paragraph(output):
 output = self._replace_existing_sqlerror_paragraph(output)
 self.messages.append(SQL_ERROR_MESSAGES["routine_replaced"])
 return output

 if self._has_sqlerror_include(output):
 self.messages.append(SQL_ERROR_MESSAGES["already_present"])
 return output

 output = self._append_routine(output)
 self.messages.append(SQL_ERROR_MESSAGES["routine_added"])
 return output

Backward-compatible alias for existing callers.
def ensure(self, text: str) -> str:
 return self.ensure_paragraph(text)

def _append_routine(self, text: str) -> str:
 block = self.paragraph_block()
 match = END_PROGRAM_PATTERN.search(text)

 if match:
 insert_at = match.start()
 return f"{text[:insert_at]}\n{block}\n{text[insert_at:]}"

 return f"{text.rstrip()}\n\n{block}\n"

=====
Legacy SQL-ERROR -> SQLERROR
=====
def rename_legacy(self, text: str) -> str:
 """Rename SQL-ERROR to SQLERROR without touching terminators."""
 output = str(text or "")
 if not output:
 return output

 # A paragraph header always ends in a period, so this rename is
 # safe and the replacement re-supplies the period.
 output, header_count = LEGACY_SQL_ERROR_HEADER_PATTERN.subn(
 LEGACY_SQL_ERROR_HEADER_REPLACEMENT,
 output,
)

 # The PERFORM terminator is captured and carried through. A fixed
 # "PERFORM SQLERROR." substitution would add a period to a
 # statement inside EVALUATE and orphan END-EVALUATE.
 def _perform(match: re.Match) -> str:
 dot = match.group("dot") or ""
 return f"PERFORM {SQL_ERROR_PARAGRAPH_NAME}{dot}"

 output, perform_count = LEGACY_SQL_ERROR_PERFORM_PATTERN.subn(
 _perform,
 output,
)

 if header_count or perform_count:
 self.messages.append(
 SQL_ERROR_MESSAGES["renamed_legacy"].format(
 headers=header_count,
 performs=perform_count,
)
)

```

```

 return output

=====
Detection
=====
def _has_sqlerror_paragraph(self, text: str) -> bool:
 return bool(SQLERROR_PARAGRAPH_PATTERN.search(str(text or "")))

@staticmethod
def _has_sqlerror_include(text: str) -> bool:
 return f"INCLUDE {SQL_ERROR_INCLUDE_NAME}".upper() in str(
 text or ""
).upper()

def _is_sqlerror_paragraph_header(self, line: str) -> bool:
 logical = self._logical_line(line)
 return bool(SQLERROR_HEADER_ONLY_PATTERN.fullmatch(logical))

def _is_next_paragraph_or_program_boundary(self, line: str) -> bool:
 """True when the scan has run past the old SQLERROR body."""
 logical = self._logical_line(line)
 if not logical:
 return False
 if END_PROGRAM_BOUNDARY_PATTERN.match(logical):
 return True
 return bool(PARAGRAPH_HEADER_PATTERN.fullmatch(logical))

def _line_belongs_to_old_sqlerror_block(self, line: str) -> bool:
 """True for a body line of a previously generated SQLERROR block."""
 logical = self._logical_line(line).upper()
 if not logical:
 return False
 return any(token in logical for token in OLD_BLOCK_BODY_TOKENS)

=====
In-place replacement of an existing routine
=====
def _replace_existing_sqlerror_paragraph(self, text: str) -> str:
 """Swap any existing SQLERROR body for the generated routine."""
 lines = str(text or "").splitlines()
 output: list[str] = []
 index = 0
 replaced = False

 while index < len(lines):
 line = lines[index]

 if not self._is_sqlerror_paragraph_header(line):
 output.append(line)
 index += 1
 continue

 output.extend(self.paragraph_lines())
 replaced = True
 index += 1

 # Consume the old body: generated body lines and blank lines
 # only. Stop at the next paragraph header or END PROGRAM.
 while index < len(lines):
 current = lines[index]

 if self._is_next_paragraph_or_program_boundary(current):
 break
 if self._line_belongs_to_old_sqlerror_block(current):
 index += 1
 continue
 if not current.strip():
 index += 1
 continue
 break

 if not replaced:
 return text

 return "\n".join(output).rstrip() + "\n"

```

```

=====
Fixed-format helpers
=====
@staticmethod
def _logical_line(line: str) -> str:
 """Strip sequence numbers and the indicator, return the body."""
 text = str(line or "").rstrip("\n").rstrip()
 if not text:
 return ""

 if (
 len(text) >= FIXED_LINE_WIDTH
 and text[:LEFT_SEQUENCE_WIDTH].isdigit()
 and text[
 FIXED_LINE_WIDTH - RIGHT_SEQUENCE_WIDTH : FIXED_LINE_WIDTH
].isdigit()
):
 return text[
 LEFT_SEQUENCE_WIDTH + 1 : FIXED_LINE_WIDTH - RIGHT_SEQUENCE_WIDTH
].strip()

 if text[:LEFT_SEQUENCE_WIDTH].isdigit():
 return text[LEFT_SEQUENCE_WIDTH:].strip()

 return text.strip()

=====
FILE: src\idms_db2_phase2\generators\sql_generator.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/generators/sql_generator.py
ACTION: REPLACE ENTIRE FILE

"""
Generates DB2 embedded SQL snippets.

Authority:
- Sheet Mapping determines intended DB2 table and column names.
- DCLGEN determines final table availability, host variable spelling.
- TableNameResolver resolves TB/TV mismatches using DCLGEN.

Thin facade delegating to focused SQL builders. Public API preserved:
select_by_key, insert, update, delete, commit, rollback,
changed_field_move, missing_mapping, and the .messages attribute.
"""

from __future__ import annotations

from idms_db2_phase2.generators.sql.sql_column_resolver import (
 SqlColumnResolver,
)
from idms_db2_phase2.generators.sql.sql_date_move_builder import (
 SqlDateMoveBuilder,
)
from idms_db2_phase2.generators.sql.sql_line_builder import SqlLineBuilder
from idms_db2_phase2.generators.sql.sql_statement_builder import (
 SqlStatementBuilder,
)
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.host_variable_resolver import (
 HostVariableResolver,
)
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.resolvers.update_sql_plan_resolver import (
 UpdateSqlPlanResolver,
)

class SqlGenerator:
 def __init__(
 self,
 mapping_repository: MappingRepository,

```

```

dclgen_repository: DclgenRepository,
table_name_resolver: TableNameResolver,
column_name_resolver: ColumnNameResolver,
host_variable_resolver: HostVariableResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver
 self.column_name_resolver = column_name_resolver
 self.host_variable_resolver = host_variable_resolver
 self.messages: list[str] = []

 self.update_plan_resolver = UpdateSqlPlanResolver(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_name_resolver,
 host_variable_resolver=host_variable_resolver,
)

 column_resolver = SqlColumnResolver(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_name_resolver,
)
 line_builder = SqlLineBuilder(
 host_variable_resolver=host_variable_resolver,
)
 date_builder = SqlDateMoveBuilder()

 self._statements = SqlStatementBuilder(
 column_resolver=column_resolver,
 line_builder=line_builder,
 date_builder=date_builder,
 host_variable_resolver=host_variable_resolver,
 mapping_repository=mapping_repository,
 update_plan_resolver=self.update_plan_resolver,
)

def select_by_key(self, record_name: str) -> list[str]:
 lines = self._statements.select_by_key(record_name)
 self.messages = self._statements.messages
 return lines

def insert(self, record_name: str) -> list[str]:
 lines = self._statements.insert(record_name)
 self.messages = self._statements.messages
 return lines

def update(
 self,
 record_name: str,
 changed_source_fields: list[str] | None = None,
) -> list[str]:
 lines = self._statements.update(
 record_name=record_name,
 changed_source_fields=changed_source_fields,
)
 self.messages = self._statements.messages
 return lines

def delete(self, record_name: str) -> list[str]:
 lines = self._statements.delete(record_name)
 self.messages = self._statements.messages
 return lines

def commit(self) -> list[str]:
 return self._statements.commit()

def rollback(self) -> list[str]:
 return self._statements.rollback()

def changed_field_move(
 self,
 record_name: str,
 source_value: str,
 target_source_field: str,

```



```

) -> list[str]:
 return self._statements.changed_field_move(
 record_name=record_name,
 source_value=source_value,
 target_source_field=target_source_field,
)

def missing_mapping(
 self,
 record_name: str,
 reason: str,
) -> list[str]:
 return self._statements.missing_mapping(
 record_name=record_name,
 reason=reason,
)

```

```

=====
FILE: src\idms_db2_phase2\generators\timestamp_generator.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/generators/timestamp_generator.py
ACTION: REPLACE ENTIRE FILE

```

```

from __future__ import annotations

import re

from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.host_variable_resolver import (
 HostVariableResolver,
)
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from rules.timestamp_audit_rules import (
 TIMESTAMP_CS_PROGRAM_TEMPLATE,
 TIMESTAMP_DEFAULT_PROGRAM_ID, # <-- add
 TIMESTAMP_MOVE_ALIGN_COLUMN,
 TIMESTAMP_MOVE_FIELDS,
 TIMESTAMP_PARAGRAPH_DISPLAY,
 TIMESTAMP_PARAGRAPH_FIELD_MOVE_TEMPLATE,
 TIMESTAMP_PARAGRAPH_HEADER_TEMPLATE,
 TIMESTAMP_PARAGRAPH_SOURCE_MOVE,
 TIMESTAMP_PARAGRAPH_TERMINATOR,
 TIMESTAMP_PROGRAM_ID_LENGTH, # <-- add
 TIMESTAMP_WS_LINES,
 TIMESTAMP_WS_MARKER,
)

class TimestampGenerator:
 """
 Generates timestamp and audit support.

 This generator is intentionally conservative.

 For retrieval-only programs, it does not inject timestamp working storage,
 timestamp paragraphs, or audit MOVE statements.

 For update/insert/delete programs, it adds a safe timestamp working-storage
 block and a safe timestamp paragraph only when DB2 write activity is found.

 This class owns no COBOL layout literals; all timestamp templates live in
 rules/timestamp_audit_rules.py. Only the program-id is dynamic.
 """

 WS_MARKER = TIMESTAMP_WS_MARKER
 PARAGRAPH_NAME = "600-GET-TIMESTAMP"
 PARAGRAPH_EXIT_NAME = "600-GET-TIMESTAMP-EXIT"

 PROCEDURE_DIVISION_PATTERN = re.compile(
 r"^\s*(?:\d{6}\s+)?PROCEDURE\s+DIVISION\b.*\s*(?:\d{8})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

```

```

WORKING_STORAGE_PATTERN = re.compile(
 r"^\\s*(?:\\d{6}\\s+)?WORKING-STORAGE\\s+SECTION\\.\\s*(?:\\d{8})?\\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

LINKAGE_SECTION_PATTERN = re.compile(
 r"^\\s*(?:\\d{6}\\s+)?LINKAGE\\s+SECTION\\.\\s*(?:\\d{8})?\\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

END_PROGRAM_PATTERN = re.compile(
 r"^\\s*(?:\\d{6}\\s+)?END\\s+PROGRAM\\b.*\\.\\s*(?:\\d{8})?\\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

STOP_RUN_PATTERN = re.compile(
 r"^\\s*(?:\\d{6}\\s+)?STOP\\s+RUN\\.\\s*(?:\\d{8})?\\s*$",
 flags=re.IGNORECASE,
)

EXEC_SQL_PATTERN = re.compile(
 r"^\\s*EXEC\\s+SQL\\b",
 flags=re.IGNORECASE,
)

END_EXEC_PATTERN = re.compile(
 r"^\\s*END-EXEC\\.\\s*$",
 flags=re.IGNORECASE,
)

DB2_WRITE_OPERATION_PATTERN = re.compile(
 r"^\\s*(INSERT|UPDATE|DELETE)\\b",
 flags=re.IGNORECASE,
)

COBOL_WRITE_OPERATION_PATTERN = re.compile(
 r"^\\s*(STORE|MODIFY|ERASE)\\b",
 flags=re.IGNORECASE,
)

def __init__(
 self,
 mapping_repository: MappingRepository,
 table_name_resolver: TableNameResolver,
 host_variable_resolver: HostVariableResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.table_name_resolver = table_name_resolver
 self.host_variable_resolver = host_variable_resolver
 self.messages: list[str] = []

def apply(
 self,
 cobol_text: str,
 target_program_id: str = "",
) -> tuple[str, list[str]]:
 self.messages = []

 if not cobol_text:
 return "", self.messages

 if not self._has_db2_write_activity(cobol_text):
 self.messages.append(
 "Timestamp generator: no DB2 write activity found; "
 "timestamp audit generation skipped."
)
 return cobol_text.rstrip() + "\\n", self.messages

 updated_text = cobol_text

 updated_text = self._ensure_timestamp_working_storage(
 text=updated_text,
 target_program_id=target_program_id,
)

 updated_text = self._ensure_timestamp_paragraph(

```

```

 text=updated_text,
)

 updated_text = self._ensure_timestamp_perform(
 text=updated_text,
)

 self.messages.append(
 "Timestamp generator: timestamp working storage and paragraph "
 "ensured."
)

 return updated_text.rstrip() + "\n", self.messages

def _has_db2_write_activity(
 self,
 text: str,
) -> bool:
 in_exec_sql = False

 for line in str(text or "").splitlines():
 logical = self._logical_line(line)

 if self.EXEC_SQL_PATTERN.match(logical):
 in_exec_sql = True
 continue

 if in_exec_sql and self.DB2_WRITE_OPERATION_PATTERN.search(
 logical
):
 return True

 if self.END_EXEC_PATTERN.match(logical):
 in_exec_sql = False
 continue

 if self.COBOL_WRITE_OPERATION_PATTERN.search(logical):
 return True

 return False

def _ensure_timestamp_working_storage(
 self,
 text: str,
 target_program_id: str,
) -> str:
 if self.WS_MARKER in text:
 return text

 block = self._timestamp_working_storage_block(
 target_program_id=target_program_id,
)

 lines = text.splitlines()
 insert_index = self._working_storage_insert_index(lines)

 if insert_index < 0:
 return block.rstrip() + "\n\n" + text.rstrip() + "\n"

 updated_lines = (
 lines[:insert_index]
 + [""]
 + block.splitlines()
 + [""]
 + lines[insert_index:]
)

 return "\n".join(updated_lines).rstrip() + "\n"

def _ensure_timestamp_paragraph(
 self,
 text: str,
) -> str:
 if re.search(
 rf"^\s*(?:\d{{6}}\s+)?{re.escape(self.PARAGRAPH_NAME)}\.",
 text,
)

```

```

 flags=re.IGNORECASE | re.MULTILINE,
):
 return text

 block = self._timestamp_paragraph_block()

 lines = text.splitlines()
 insert_index = self._timestamp_paragraph_insert_index(lines)

 if insert_index < 0:
 return text.rstrip() + "\n\n" + block + "\n"

 updated_lines = (
 lines[:insert_index]
 + block.splitlines()
 + [""]
 + lines[insert_index:]
)

 return "\n".join(updated_lines).rstrip() + "\n"

def _ensure_timestamp_perform(
 self,
 text: str,
) -> str:
 if re.search(
 rf"\bPERFORM{s+{re.escape(self.PARAGRAPH_NAME)}}\b",
 text,
 flags=re.IGNORECASE,
):
 return text

 lines = text.splitlines()
 procedure_index = self._find_pattern_index(
 lines=lines,
 pattern=self.PROCEDURE_DIVISION_PATTERN,
)

 if procedure_index < 0:
 return text

 insert_index = procedure_index + 1

 while insert_index < len(lines):
 logical = self._logical_line(lines[insert_index])

 if not logical:
 insert_index += 1
 continue

 if logical.startswith("*"):
 insert_index += 1
 continue

 break

 updated_lines = (
 lines[:insert_index]
 + [f" PERFORM {self.PARAGRAPH_NAME}."]
 + lines[insert_index:]
)

 return "\n".join(updated_lines).rstrip() + "\n"

def _timestamp_working_storage_block(
 self,
 target_program_id: str,
) -> str:
 """
 Build the timestamp working storage from rules/ templates. Only the
 program-id is dynamic; the fallback program-id and field width come
 from rules/ so no program name or length literal lives in generator
 logic.
 """
 program_id = self._resolve_program_id(target_program_id)

```

```

lines = [
 f"* {TIMESTAMP_WS_MARKER}",
 TIMESTAMP_CS_PROGRAM_TEMPLATE.format(program_id=program_id),
]
lines.extend(TIMESTAMP_WS_LINES)

return "\n".join(lines)

def _resolve_program_id(
 self,
 target_program_id: str,
) -> str:
 """
 Resolve the CS-PROGRAM value. Falls back to the configured default
 program-id (from rules/) only when no target PROGRAM-ID is supplied,
 and truncates to the configured COBOL field length.
 """
 program_id = str(target_program_id or "").strip().upper()

 if not program_id:
 program_id = TIMESTAMP_DEFAULT_PROGRAM_ID

 if len(program_id) > TIMESTAMP_PROGRAM_ID_LENGTH:
 program_id = program_id[:TIMESTAMP_PROGRAM_ID_LENGTH]

 return program_id

def _timestamp_paragraph_block(
 self,
) -> str:
 """
 Build the timestamp paragraph from rules/ templates. Uses
 FUNCTION CURRENT-DATE and field-by-field moves (manual standard).
 """
 lines = [
 TIMESTAMP_PARAGRAPH_HEADER_TEMPLATE.format(
 paragraph_name=self.PARAGRAPH_NAME
),
 TIMESTAMP_PARAGRAPH_SOURCE_MOVE,
]

 for field in TIMESTAMP_MOVE_FIELDS:
 lines.append(self._timestamp_field_move_line(field))

 lines.append(TIMESTAMP_PARAGRAPH_DISPLAY)
 lines.append(TIMESTAMP_PARAGRAPH_TERMINATOR)

 return "\n".join(lines)

def _timestamp_field_move_line(
 self,
 field: str,
) -> str:
 """
 Build one aligned MOVE line from the rules/ template. Padding keeps
 the TO column aligned regardless of field-name length.
 """
 source_part = f" MOVE {field} OF TS-SYSTEM"
 pad = " " * max(1, TIMESTAMP_MOVE_ALIGN_COLUMN - len(source_part))
 return TIMESTAMP_PARAGRAPH_FIELD_MOVE_TEMPLATE.format(
 field=field,
 pad=pad,
)

def _working_storage_insert_index(
 self,
 lines: list[str],
) -> int:
 linkage_index = self._find_pattern_index(
 lines=lines,
 pattern=self.LINKAGE_SECTION_PATTERN,
)

 if linkage_index >= 0:
 return linkage_index

```

```

 procedure_index = self._find_pattern_index(
 lines=lines,
 pattern=self.PROCEDURE_DIVISION_PATTERN,
)

 if procedure_index >= 0:
 return procedure_index

 working_storage_index = self._find_pattern_index(
 lines=lines,
 pattern=self.WORKING_STORAGE_PATTERN,
)

 if working_storage_index >= 0:
 return working_storage_index + 1

 return -1

 def _timestamp_paragraph_insert_index(
 self,
 lines: list[str],
) -> int:
 for index, line in enumerate(lines):
 logical = self._logical_line(line)

 if self.STOP_RUN_PATTERN.match(logical):
 return index

 if self.END_PROGRAM_PATTERN.match(logical):
 return index

 return -1

 def _find_pattern_index(
 self,
 lines: list[str],
 pattern: re.Pattern,
) -> int:
 for index, line in enumerate(lines):
 logical = self._logical_line(line)

 if pattern.match(logical):
 return index

 return -1

 def _logical_line(
 self,
 line: str,
) -> str:
 text = str(line or "").rstrip()

 if len(text) > 6 and text[:6].strip().isdigit():
 return text[6:].strip()

 return text.strip()

```

```

=====
FILE: src\idms_db2_phase2\infrastructure__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\infrastructure\file_loader.py
ENCODING: utf-8
=====

```

```

from io import BytesIO

import pdfplumber

```

```

class FileLoader:
 """
 Loads readable text from uploaded-file-like objects.

 Expected uploaded-file-like object:
 - .name
 - .getvalue()

 Supported:
 - PDF files through pdfplumber
 - Text-like files through UTF-8 decode with errors ignored
 """

 def read_uploaded_text(
 self,
 uploaded_file,
) -> str:
 if uploaded_file is None:
 return ""

 raw_bytes = uploaded_file.getvalue()
 file_name = str(uploaded_file.name or "").lower()

 if file_name.endswith(".pdf"):
 return self.read_pdf_bytes(raw_bytes)

 return raw_bytes.decode(
 "utf-8",
 errors="ignore",
)

 def read_pdf_bytes(
 self,
 raw_bytes: bytes,
) -> str:
 lines: list[str] = []

 with pdfplumber.open(BytesIO(raw_bytes)) as pdf:
 for page in pdf.pages:
 page_text = page.extract_text() or ""

 for raw_line in page_text.splitlines():
 cleaned_line = self._clean_line(raw_line)

 if cleaned_line:
 lines.append(cleaned_line)

 return "\n".join(lines).strip()

 def _clean_line(
 self,
 line: str | None,
) -> str:
 if line is None:
 return ""

 cleaned = str(line).rstrip()
 cleaned = cleaned.replace("\t", " ")
 cleaned = cleaned.replace("\u00a0", " ")

 while " " in cleaned:
 cleaned = cleaned.replace(" ", " ")

 return cleaned.strip()

```

```

=====
FILE: src\idms_db2_phase2\infrastructure\local_uploaded_file.py
ENCODING: utf-8
=====

```

```

from pathlib import Path

```

```

class LocalUploadedFile:
 """

```

Local file wrapper.

Existing parser flow expects uploaded-file-like objects with:

- .name
- .getvalue()

This wrapper allows local runner scripts to reuse the same parser logic used by the Streamlit file uploader flow.

"""

```
def __init__(
 self,
 file_path: Path,
) -> None:
 self.file_path = Path(file_path)
 self.name = self.file_path.name

def getvalue(
 self,
) -> bytes:
 return self.file_path.read_bytes()
```

```
=====
FILE: src\idms_db2_phase2\infrastructure\logger_factory.py
ENCODING: utf-8
=====
```

```
import logging
from datetime import datetime
from pathlib import Path
```

```
from config.logging_settings import (
 DEFAULT_LOG_LEVEL,
 LOG_DATE_FORMAT,
 LOG_DATE_TIME_FORMAT,
 LOG_FILE_PREFIX,
 LOG_MESSAGE_FORMAT,
)
from config.path_settings import LOGS_DIR
```

```
class LoggerFactory:
```

"""

Creates production-style loggers that write to logs folder.

One log file is created per run:

logs/idms\_db2\_conversion\_DD-MM-YYYY\_HHMMSS.log

"""

@staticmethod

```
def create_logger(
 name: str,
 logs_dir: Path | None = None,
 level: str = DEFAULT_LOG_LEVEL,
) -> logging.Logger:
 target_logs_dir = Path(logs_dir or LOGS_DIR)
 target_logs_dir.mkdir(
 parents=True,
 exist_ok=True,
)

 timestamp = datetime.now().strftime(LOG_DATE_TIME_FORMAT)
 log_file = target_logs_dir / f"{LOG_FILE_PREFIX}_{timestamp}.log"

 logger = logging.getLogger(name)
 logger.setLevel(LoggerFactory._level_value(level))
 logger.handlers.clear()
 logger.propagate = False

 formatter = logging.Formatter(
 fmt=LOG_MESSAGE_FORMAT,
 datefmt=LOG_DATE_FORMAT,
)

 file_handler = logging.FileHandler(
```



```

 log_file,
 encoding="utf-8",
)
 file_handler.setLevel(LoggerFactory._level_value(level))
 file_handler.setFormatter(formatter)

 console_handler = logging.StreamHandler()
 console_handler.setLevel(LoggerFactory._level_value(level))
 console_handler.setFormatter(formatter)

 logger.addHandler(file_handler)
 logger.addHandler(console_handler)

 logger.info("Logger initialized.")
 logger.info("Log file: %s", log_file)

 return logger

@staticmethod
def _level_value(
 level: str,
) -> int:
 return getattr(
 logging,
 str(level or DEFAULT_LOG_LEVEL).upper(),
 logging.INFO,
)

```

```

=====
FILE: src\idms_db2_phase2\infrastructure\runtime_context.py
ENCODING: utf-8
=====

```

```

from dataclasses import dataclass, field
from pathlib import Path

```

```

@dataclass
class RuntimeContext:
 """
 Runtime context for local executions.

 Keep runtime paths outside parser and service logic.
 Parser and service classes should receive data, not know where the files
 came from.
 """

 project_root: Path
 input_dir: Path
 output_dir: Path
 sheet_mapping_path: Path | None = None
 dclgen_paths: list[Path] = field(default_factory=list)
 copybook_paths: list[Path] = field(default_factory=list)
 idms_cobol_source_path: Path | None = None
 target_program_id: str = ""
 auto_fix_pic_length_mismatches: bool = False

 def ensure_output_dir(
 self,
) -> None:
 self.output_dir.mkdir(
 parents=True,
 exist_ok=True,
)

 def validate_required_files(
 self,
) -> None:
 if self.sheet_mapping_path is None:
 raise ValueError("Sheet Mapping path is required.")

 if self.idms_cobol_source_path is None:
 raise ValueError("IDMS COBOL source path is required.")

 self._validate_file_exists(

```

```

 self.sheet_mapping_path,
 "Sheet Mapping",
)

 self._validate_file_exists(
 self.idms_cobol_source_path,
 "IDMS COBOL Source",
)

 if not self.dclgen_paths:
 raise ValueError("At least one DCLGEN file path is required.")

 for index, dclgen_path in enumerate(self.dclgen_paths, start=1):
 self._validate_file_exists(
 dclgen_path,
 f"DCLGEN {index}",
)

 for index, copybook_path in enumerate(self.copybook_paths, start=1):
 self._validate_file_exists(
 copybook_path,
 f"Copybook {index}",
)

 self.ensure_output_dir()

 def _validate_file_exists(
 self,
 file_path: Path,
 label: str,
) -> None:
 if not file_path.exists():
 raise FileNotFoundError(f"{label} file not found: {file_path}")

 if not file_path.is_file():
 raise FileNotFoundError(f"{label} path is not a file: {file_path}")

```

```

=====
FILE: src\idms_db2_phase2\logging_utils__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\orchestration__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\orchestration\conversion__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\orchestration\conversion\conversion_component_factory.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/orchestration/conversion/conversion_component_factory.py
ACTION: REPLACE ENTIRE FILE

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.composers.cobol_cleanup_composer import CobolCleanupComposer
from idms_db2_phase2.composers.cobol_formatter import CobolFormatter
from idms_db2_phase2.composers.cursor_flow_composer import CursorFlowComposer
from idms_db2_phase2.composers.cursor_order_cleanup_composer import (

```

```

 CursorOrderCleanupComposer,
)
from idms_db2_phase2.composers.db2_date_comparison_composer import (
 Db2DateComparisonComposer,
)
from idms_db2_phase2.composers.final_cobol_fix_composer import (
 FinalCobolFixComposer,
 FinalCobolFixComposerConfig,
)
from idms_db2_phase2.composers.fixed_format_composer import FixedFormatComposer
from idms_db2_phase2.composers.manual_layout_composer import ManualLayoutComposer
from idms_db2_phase2.composers.manual_style_preserver import ManualStylePreserver
from idms_db2_phase2.composers.sqlcode_wrapper_cleanup_composer import (
 SqlcodeWrapperCleanupComposer,
)
from idms_db2_phase2.composers.update_program_feedback_composer import (
 UpdateProgramFeedbackComposer,
)
from idms_db2_phase2.composers.update_restart_skip_composer import (
 UpdateRestartSkipComposer,
)
from idms_db2_phase2.domain.models import ConversionInput
from idms_db2_phase2.generators.cursor_paragraph_generator import (
 CursorParagraphGenerator,
)
from idms_db2_phase2.generators.db2_infrastructure_generator import (
 Db2InfrastructureGenerator,
)
from idms_db2_phase2.generators.sql_error_generator import SqlErrorGenerator
from idms_db2_phase2.generators.sql_generator import SqlGenerator
from idms_db2_phase2.generators.timestamp_generator import TimestampGenerator
from idms_db2_phase2.repositories.copybook_repository import CopybookRepository
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.lrf_repository import LrfRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.lrf_path_resolver import LrfPathResolver
from idms_db2_phase2.resolvers.record_context_resolver import RecordContextResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.transformers.cobol_transformer import CobolTransformer
from idms_db2_phase2.transformers.field_reference_rewriter import (
 FieldReferenceRewriter,
)
from idms_db2_phase2.transformers.idms_statement_transformer import (
 IdmsStatementTransformer,
)
from idms_db2_phase2.transformers.lrf_path_expander import LrfPathExpander
from rules.final_feedback_fix_rules import (
 DEFAULT_DB2_DATE_EXTERNAL_FORMAT,
 ORDER_BY_COLUMNS_IN_SELECT_DEFAULT,
)
from idms_db2_phase2.composers.record_materialisation_composer import (
 RecordMaterialisationComposer,
)

class ConversionComponentFactory:
 """Builds repositories, resolvers, generators, transformers, composers.

 CORRECTION - module imported where an instance was required

 This file previously carried BOTH

 from idms_db2_phase2.resolvers import cursor_name_resolver
 from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver

 and returned the first of those under the "cursor_name" key. That put a
 MODULE into the resolver map, so Db2InfrastructureGenerator failed at
 runtime with

 AttributeError: module 'idms_db2_phase2.resolvers.cursor_name_resolver'
 has no attribute 'cursor_name_from_table'

 The module import is removed. "cursor_name" now holds a constructed

```

CursorNameResolver, like every other resolver in the map.

#### LRF (Logical Record Facility)

LRF metadata is OPTIONAL. With no subschema supplied, LrfRepository is empty, LrfPathResolver reports is\_empty(), and LrfPathExpander returns the program source unchanged - so a program without logical records converts exactly as it did before LRF support existed.

```
"""

Repositories

def _repositories(self, conversion_input: ConversionInput) -> dict[str, object]:
 return {
 "mapping": MappingRepository(conversion_input.sheet_mapping_rows),
 "dclgen": DclgenRepository(conversion_input.dclgen_columns),
 "copybook": CopybookRepository(conversion_input.copybook_fields),
 "lrf": LrfRepository(
 getattr(conversion_input, "logical_records", None) or []
),
 }

Resolvers

def _resolvers(self, repositories: dict[str, object]) -> dict[str, object]:
 mapping_repository = repositories["mapping"]
 dclgen_repository = repositories["dclgen"]

 table_name_resolver = TableNameResolver(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
)
 column_name_resolver = ColumnNameResolver(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_name_resolver,
)
 host_variable_resolver = HostVariableResolver(
 dclgen_repository=dclgen_repository,
 table_name_resolver=table_name_resolver,
)
 record_context_resolver = RecordContextResolver(
 mapping_repository=mapping_repository,
 dclgen_repository=dclgen_repository,
)

 # MUST be an instance. See the class docstring.
 cursor_name_resolver = CursorNameResolver()

 lrf_path_resolver = LrfPathResolver(
 lrf_repository=repositories.get("lrf"),
 mapping_repository=mapping_repository,
)

 return {
 "table_name": table_name_resolver,
 "column_name": column_name_resolver,
 "host_variable": host_variable_resolver,
 "record_context": record_context_resolver,
 "cursor_name": cursor_name_resolver,
 "lrf_path": lrf_path_resolver,
 }

Generators

def _generators(self, repositories, resolvers) -> dict[str, object]:
 sql_error_generator = SqlErrorGenerator()

 sql_generator = SqlGenerator(
 mapping_repository=repositories["mapping"],
 dclgen_repository=repositories["dclgen"],
 table_name_resolver=resolvers["table_name"],
 column_name_resolver=resolvers["column_name"],
)
```

```

 host_variable_resolver=resolvers["host_variable"],
)
 db2_infrastructure_generator = Db2InfrastructureGenerator(
 table_name_resolver=resolvers["table_name"],
 column_name_resolver=resolvers["column_name"],
 host_variable_resolver=resolvers["host_variable"],
 cursor_name_resolver=resolvers["cursor_name"],
)
 cursor_paragraph_generator = CursorParagraphGenerator(
 db2_infrastructure_generator=db2_infrastructure_generator,
 host_variable_resolver=resolvers["host_variable"],
 sql_error_generator=sql_error_generator,
)
 timestamp_generator = TimestampGenerator(
 mapping_repository=repositories["mapping"],
 table_name_resolver=resolvers["table_name"],
 host_variable_resolver=resolvers["host_variable"],
)

 return {
 "sql": sql_generator,
 "sql_error": sql_error_generator,
 "db2_infrastructure": db2_infrastructure_generator,
 "cursor_paragraph": cursor_paragraph_generator,
 "timestamp": timestamp_generator,
 }

Transformers

def _transformers(self, repositories, resolvers, generators) -> dict[str, object]:
 idms_statement_transformer = IdmsStatementTransformer(
 sql_generator=generators["sql"],
 sql_error_generator=generators["sql_error"],
 table_name_resolver=resolvers["table_name"],
 cursor_name_resolver=resolvers["cursor_name"],
)

 return {
 "idms_statement": idms_statement_transformer,
 "cobol": CobolTransformer(
 idms_statement_transformer=idms_statement_transformer
),
 "field_reference": FieldReferenceRewriter(
 mapping_repository=repositories["mapping"],
 table_name_resolver=resolvers["table_name"],
 host_variable_resolver=resolvers["host_variable"],
),
 # Runs FIRST in the pipeline. No-op when no LRF metadata or no
 # LRF syntax is present.
 "lrf_path_expander": LrfPathExpander(
 path_resolver=resolvers["lrf_path"],
),
 }

Composers

def _composers(self, repositories, resolvers) -> dict[str, object]:
 composers: dict[str, object] = {}
 composers.update(self._formatting_composers())
 composers.update(self._cursor_composers())
 composers.update(self._cleanup_composers(repositories, resolvers))
 composers.update(self._update_composers(repositories, resolvers))
 composers.update(self._final_fix_composers())
 return composers

def _formatting_composers(self) -> dict[str, object]:
 return {
 "formatter": CobolFormatter(),
 "fixed_format": FixedFormatComposer(),
 "manual_layout": ManualLayoutComposer(),
 "style_preserver": ManualStylePreserver(),
 }

def _cursor_composers(self) -> dict[str, object]:

```

```

 return {
 "cursor_flow": CursorFlowComposer(),
 "cursor_order_cleanup": CursorOrderCleanupComposer(),
 "sqlcode_cleanup": SqlcodeWrapperCleanupComposer(),
 "date_compare": Db2DateComparisonComposer(),
 }

def _cleanup_composers(self, repositories, resolvers) -> dict[str, object]:
 return {
 "feedback_cleanup": CobolCleanupComposer(
 dclgen_repository=repositories["dclgen"],
),
 "update_restart_skip": UpdateRestartSkipComposer(),
 # Expands a whole-record MOVE into the record layout plus
 # field-by-field moves. Consumed by the FINISHING phase, not
 # by the content phase, because it needs sequenced lines.
 "record_materialisation": RecordMaterialisationComposer(
 mapping_repository=repositories["mapping"],
 table_name_resolver=resolvers["table_name"],
 host_variable_resolver=resolvers["host_variable"],
),
 }

def _update_composers(self, repositories, resolvers) -> dict[str, object]:
 return {
 "update_program_feedback": UpdateProgramFeedbackComposer(
 mapping_repository=repositories["mapping"],
 dclgen_repository=repositories["dclgen"],
 table_name_resolver=resolvers["table_name"],
 host_variable_resolver=resolvers["host_variable"],
),
 }

def _final_fix_composers(self) -> dict[str, object]:
 return {
 "final_feedback_fix": FinalCobolFixComposer(
 config=FinalCobolFixComposerConfig(
 db2_date_external_format=DEFAULT_DB2_DATE_EXTERNAL_FORMAT,
 require_order_by_columns_in_select=(
 ORDER_BY_COLUMNS_IN_SELECT_DEFAULT
),
),
),
 }

```

```
__all__ = ["ConversionComponentFactory"]
```

```

=====
FILE: src\idms_db2_phase2\orchestration\conversion\conversion_layout_pipeline.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/orchestration/conversion/conversion_layout_pipeline.py
ACTION: REPLACE ENTIRE FILE

```

```
"""Finishing phase of the IDMS->DB2 conversion.
```

```

Takes converted COBOL whose CONTENT is final and decides how it is laid
out: sequencing, paragraph extraction, counters, comment blocks,
structural safety, Area-B indentation and the final resequence.

```

```

This class owns the ORDERING CONTRACT only. Each pass lives in
ConversionLayoutSteps, so a pass can be tested in isolation and the order
can be read in one screen.
"""

```

```
from __future__ import annotations
```

```
from collections.abc import Callable
```

```

from idms_db2_phase2.orchestration.conversion.conversion_layout_steps import (
 ConversionLayoutSteps,
)

```

```

class ConversionLayoutPipeline:
 """Runs the layout and finishing passes, in a fixed, load-bearing order.

 Pass-ordering contracts

 2. ``fixed_format`` sequences every line, including generator-inserted
 blocks. It runs BEFORE any comment is emitted, so comment lines
 cannot be collapsed - at this point they do not exist yet.

 3. ``OutputWriteParagraphComposer`` extracts the record-population body
 into its own ``WRITE-<record>`` paragraph and replaces it with a
 PERFORM plus the output counter increment. It MUST run after:

 - the content phase's OutputWritePlacementCleanup, otherwise it
 extracts from the wrong paragraph and bakes in the
 once-per-child-row defect;
 - ``fixed_format``, otherwise it has no sequence area to clone
 when building the new paragraph header.

 4. ``LateDb2DateComposer`` re-runs the DB2 date and INITIALIZE passes.
 Those live inside ``feedback_cleanup`` and have already finished by
 the time step 3 moves the block into a brand new paragraph, so
 without this second run a DB2 date host field is moved straight into
 the output record, skipping the CCYYMMDD realignment. That is silent
 data corruption, not a formatting defect. Idempotent: a converted
 move carries no DCLGEN qualifier and cannot match again.

 5. ``CounterDeclarationComposer`` declares every ``WS-NB-*-COUNT`` the
 PROCEDURE DIVISION increments but WORKING-STORAGE does not define,
 and appends the end-of-run totals before STOP RUN. It MUST run after
 step 3, which is what emits the increment, and BEFORE step 7 so the
 generated DISPLAY lines are indented with everything else.

 6. ``UnmappedRecordBlockComposer`` comments record-level blocks that
 have no usable DB2 column mapping. Runs after every content pass so
 it never comments code another pass still needs to read.

 6b. ``StructuralSafetyComposer`` is the LAST guard before indentation.
 It comments whole-record references whose IDMS layout no longer
 exists, and statements left unreachable after a paragraph EXIT.
 It MUST run after step 6 - an unmapped-record block is commented
 first, so the safety pass never re-comments the same lines - and
 BEFORE step 7, so its comment lines are indented with everything
 else.

 7. ``ProcedureIndentNormalizer`` re-anchors Area B, covering the
 paragraph created in step 3, the totals added in step 5 and the
 comments added in step 6b.

 8. ``FinalSequenceResequencerService`` rewrites columns 1-6 and 73-80,
 correcting the placeholder sequence numbers cloned in steps 3, 5
 and 6b.
 """

 def __init__(
 self,
 *,
 composers: dict,
 resolvers: dict,
 component_messages: Callable[[object], list[str]],
 steps: ConversionLayoutSteps | None = None,
) -> None:
 self.steps = steps or ConversionLayoutSteps(
 composers=composers,
 resolvers=resolvers,
 component_messages=component_messages,
)

 # -----
 # Entry point
 # -----
 def finish(
 self,
 *,
 converted_cobol: str,

```

```

 original_idms_text: str,
 validation_messages: list[str],
) -> str:
 steps = self.steps

 converted_cobol = steps.apply_layout(
 converted_cobol=converted_cobol,
 original_idms_text=original_idms_text,
)
 converted_cobol = steps.extract_output_write_paragraph(
 converted_cobol=converted_cobol,
 validation_messages=validation_messages,
)
 converted_cobol = steps.apply_late_date_conversion(
 converted_cobol=converted_cobol,
 validation_messages=validation_messages,
)
 converted_cobol = steps.materialise_records(
 converted_cobol=converted_cobol,
 validation_messages=validation_messages,
)
 converted_cobol = steps.declare_counters(
 converted_cobol=converted_cobol,
 validation_messages=validation_messages,
)
 converted_cobol = steps.comment_unmapped_blocks(
 converted_cobol=converted_cobol,
 original_idms_text=original_idms_text,
 validation_messages=validation_messages,
)
 converted_cobol = steps.apply_structural_safety(
 converted_cobol=converted_cobol,
 original_idms_text=original_idms_text,
 validation_messages=validation_messages,
)
 converted_cobol = steps.normalize_indentation(
 converted_cobol=converted_cobol,
 validation_messages=validation_messages,
)
 return steps.resequence(converted_cobol)

```

```
__all__ = ["ConversionLayoutPipeline"]
```

```

=====
FILE: src\idms_db2_phase2\orchestration\conversion\conversion_layout_steps.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/orchestration/conversion/conversion_layout_steps.py
ACTION: REPLACE ENTIRE FILE
"""Individual finishing passes of the IDMS->DB2 conversion.

```

One method per pass. Each takes the converted text, runs one composer, drains that composer's diagnostics into the shared message list, and returns the new text. No method knows about any other method, so the ordering contract lives entirely in ConversionLayoutPipeline.

Deliberately NOT a mixin: it receives the composers and resolvers it needs plus a message-collection callable, so every pass can be constructed and tested without the component factory.

ORDERING CONTRACT (enforced by ConversionLayoutPipeline)

2. ``apply\_layout`` runs the formatter, manual layout, style preserver and fixed-format passes. ALL lines are sequenced here, BEFORE any commenting pass, so generator-inserted blocks carry a real sequence area and comment lines cannot be collapsed - they do not exist yet.
3. ``OutputWriteParagraphComposer`` lifts the record-population body into its own WRITE paragraph. Runs AFTER fixed\_format so every generated line can clone a real sequence area.
4. ``LateDb2DateComposer`` re-runs the DB2 date and INITIALIZE passes. Those live inside ``feedback\_cleanup`` and have already finished by



the time step 3 moves the block into a brand new paragraph, so without this second run a DB2 date host field is moved straight into the output record, skipping the CCYYMMDD realignment. That is silent data corruption, not a formatting defect. Idempotent: a converted move carries no DCLGEN qualifier and cannot match again.

- 4b. ``RecordMaterialisationComposer`` expands every whole-record MOVE into a real layout plus field-by-field moves. Runs AFTER step 2 so generated lines can clone a real sequence area, BEFORE step 5 so the counter pass sees the final field list, and BEFORE steps 6 and 6b, which would otherwise comment the move as an undefined data-name.
5. ``CounterDeclarationComposer`` declares every ``WS-NB-\*-COUNT`` the PROCEDURE DIVISION increments but WORKING-STORAGE does not define, and appends the end-of-run totals before STOP RUN. It MUST run after step 3, which is what emits the increment, and BEFORE step 7 so the generated DISPLAY lines are indented with everything else.
6. ``UnmappedRecordBlockComposer`` comments record-level blocks that have no usable DB2 column mapping. Runs after every content pass so it never comments code another pass still needs to read.
- 6b. ``StructuralSafetyComposer`` is the LAST guard before indentation. It comments whole-record references whose IDMS layout no longer exists, and statements left unreachable after a paragraph EXIT. It MUST run after step 6 - an unmapped-record block is commented first, so the safety pass never re-comments the same lines - and BEFORE step 7, so its comment lines are indented with everything else.
7. ``ProcedureIndentNormalizer`` aligns every PROCEDURE DIVISION statement line to Area B.
8. ``FinalSequenceResequencerService`` re-sequences ONLY columns 1-6 and 73-80.

CORRECTION - step 4b was a silent no-op

materialise\_records returned the source unchanged and said nothing when the composer was absent, so an unwired feature was indistinguishable from a feature that ran and found nothing to do. The absence is now reported.

"""

```
from __future__ import annotations
```

```
from collections.abc import Callable
```

```
from idms_db2_phase2.composers.counter_declaration_composer import (
 CounterDeclarationComposer,
```

```
)
from idms_db2_phase2.composers.late_db2_date_composer import (
 LateDb2DateComposer,
```

```
)
from idms_db2_phase2.composers.output_write_paragraph_composer import (
 OutputWriteParagraphComposer,
```

```
)
from idms_db2_phase2.composers.procedure_indent_normalizer import (
 ProcedureIndentNormalizer,
```

```
)
from idms_db2_phase2.composers.structural_safety_composer import (
 StructuralSafetyComposer,
```

```
)
from idms_db2_phase2.composers.unmapped_record_block_composer import (
 UnmappedRecordBlockComposer,
```

```
)
from idms_db2_phase2.services.final_sequence_resequencer_service import (
 FinalSequenceResequencerService,
```

```
)
from rules.record_materialisation_rules import (
 RECORD_MATERIALISATION_MESSAGES,
```

```
)
```

```
class ConversionLayoutSteps:
```

```
 """The finishing passes, each independently runnable."""
```

```

def __init__(
 self,
 *,
 composers: dict,
 resolvers: dict,
 component_messages: Callable[[object], list[str]],
) -> None:
 self.composers = composers
 self.resolvers = resolvers
 self._component_messages = component_messages

=====
2. Layout and sequencing
=====
def apply_layout(
 self,
 *,
 converted_cobol: str,
 original_idms_text: str,
) -> str:
 converted_cobol = self.composers["formatter"].format(converted_cobol)
 converted_cobol = self.composers["manual_layout"].compose(
 converted_cobol
)
 converted_cobol = self.composers["style_preserver"].preserve(
 original_text=original_idms_text,
 converted_text=converted_cobol,
)
 # Fixed-format ALL lines BEFORE commenting so generator-inserted
 # blocks are sequenced and comment lines cannot be collapsed (they
 # do not exist yet at this point).
 return self.composers["fixed_format"].format(converted_cobol)

=====
3. Output write paragraph extraction
=====
def extract_output_write_paragraph(
 self,
 *,
 converted_cobol: str,
 validation_messages: list[str],
) -> str:
 """Lift the record-population body into its own WRITE paragraph.

 Replaces it with a PERFORM plus the output counter increment,
 matching the manual reference. Runs AFTER fixed_format so each
 generated line can clone a real sequence area.

 Refuses when the guard condition spans lines: cutting between the
 IF keyword and the end of its condition orphans the condition and
 the program will not compile.
 """
 composer = OutputWriteParagraphComposer()
 converted_cobol = composer.compose(converted_cobol)
 validation_messages.extend(self._component_messages(composer))
 return converted_cobol

=====
4. Late DB2 date / INITIALIZE re-run
=====
def apply_late_date_conversion(
 self,
 *,
 converted_cobol: str,
 validation_messages: list[str],
) -> str:
 """Re-run the date and INITIALIZE passes on the relocated block.

 Step 3 extracts the record-population body into a brand new
 WRITE paragraph AFTER feedback_cleanup has already finished, so
 without this second run a DB2 date host field is moved straight
 into the output record and skips the CCYYMMDD realignment. That
 is silent data corruption, not a formatting defect.

 Idempotent: a converted move carries no DCLGEN qualifier and

```

```

cannot match again.
"""

composer = LateDb2DateComposer()
converted_cobol = composer.compose(converted_cobol)
validation_messages.extend(self._component_messages(composer))
return converted_cobol

=====
4b. IDMS record materialisation
=====
def materialise_records(
 self,
 *,
 converted_cobol: str,
 validation_messages: list[str],
) -> str:
 """Expand every whole-record MOVE into layout + field moves.

 Ordering:
 AFTER step 2, so generated lines can clone a real sequence area.
 BEFORE step 5, so the counter pass sees the final field list.
 BEFORE steps 6 and 6b, which would otherwise comment the move
 as an undefined data-name.

 CORRECTION - silent no-op.
 A build without the composer returned the source unchanged and
 said nothing, so an unwired feature was indistinguishable from a
 feature that ran and found nothing to do. The absence is now
 reported as a validation message.
 """
 composer = self.composers.get("record_materialisation")

 if composer is None:
 validation_messages.append(
 RECORD_MATERIALISATION_MESSAGES["composer_absent"]
)
 return converted_cobol

 converted_cobol = composer.compose(converted_cobol)
 validation_messages.extend(self._component_messages(composer))
 return converted_cobol

=====
5. Counter declarations and end-of-run totals
=====
def declare_counters(
 self,
 *,
 converted_cobol: str,
 validation_messages: list[str],
) -> str:
 """Declare referenced counters and append end-of-run totals.

 Placement is safety-first: a record layout can never gain a field,
 so a dedicated 01 group is created when no work group is safe.
 """
 composer = CounterDeclarationComposer()
 converted_cobol = composer.compose(converted_cobol)
 validation_messages.extend(self._component_messages(composer))
 return converted_cobol

=====
6. Unmapped record blocks
=====
def comment_unmapped_blocks(
 self,
 *,
 converted_cobol: str,
 original_idms_text: str,
 validation_messages: list[str],
) -> str:
 composer = UnmappedRecordBlockComposer(
 table_name_resolver=self.resolvers["table_name"],
 column_name_resolver=self.resolvers["column_name"],
 original_idms_text=original_idms_text,
)

```

```

 converted_cobol = composer.compose(converted_cobol)
 validation_messages.extend(self._component_messages(composer))
 return converted_cobol

=====
6b. Structural safety
=====
def apply_structural_safety(
 self,
 *,
 converted_cobol: str,
 original_idms_text: str,
 validation_messages: list[str],
) -> str:
 """Comment references the compiler would reject.

 Two guards, both COMMENT-ONLY:

 * a whole-record MOVE of an IDMS record whose copybook was
 removed - an undefined data-name,
 * an executable statement after a paragraph EXIT - unreachable
 and outside any paragraph.

 Nothing is deleted and no logic is rewritten, so a false positive
 costs a comment, never working code.
 """
 composer = StructuralSafetyComposer(
 original_idms_text=original_idms_text,
)
 converted_cobol = composer.compose(converted_cobol)
 validation_messages.extend(self._component_messages(composer))
 return converted_cobol

=====
7. Procedure Division indentation
=====
def normalize_indentation(
 self,
 *,
 converted_cobol: str,
 validation_messages: list[str],
) -> str:
 normalizer = ProcedureIndentNormalizer()
 converted_cobol = normalizer.compose(converted_cobol)
 validation_messages.extend(self._component_messages(normalizer))
 return converted_cobol

=====
8. Final resequence
=====
@staticmethod
def resequence(converted_cobol: str) -> str:
 """Re-sequence ONLY columns 1-6 and 73-80.

 Corrects the placeholder sequence numbers cloned by the
 extraction, counter and safety passes.
 """
 return FinalSequenceResequencerService().resequence(converted_cobol)

Backwards-compatible aliases for step 4.
#
apply_late_date_conversion is the canonical name and is what
ConversionLayoutPipeline.finish() calls. The aliases keep any older
caller or test working without a second file having to change.

ConversionLayoutSteps.reapply_db2_dates = (
 ConversionLayoutSteps.apply_late_date_conversion
)
ConversionLayoutSteps.apply_late_db2_dates = (
 ConversionLayoutSteps.apply_late_date_conversion
)
ConversionLayoutSteps.late_db2_dates = (
 ConversionLayoutSteps.apply_late_date_conversion
)

```

```
__all__ = ["ConversionLayoutSteps"]
```

```
=====
FILE: src\idms_db2_phase2\orchestration\conversion\conversion_message_utils.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations
```

```
class ConversionMessageUtils:
```

```
 """Message collection helpers shared by the conversion pipeline."""
```

```
 def _component_messages(self, component: object) -> list[str]:
 """Return messages from components that optionally expose them.
```

```

 Some transformers/composers expose a ``messages`` attribute; others do
 not. This keeps orchestration generic without forcing all components
 to implement the same diagnostics interface.
 """
```

```
 messages = getattr(component, "messages", [])
 if callable(messages):
 messages = messages()
 if not messages:
 return []
 return [
 str(message)
 for message in messages
 if str(message or "").strip()
]
```

```
 def _unique_messages(self, messages: list[str]) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()
 for message in messages:
 clean_message = str(message or "").strip()
 if not clean_message or clean_message in seen:
 continue
 seen.add(clean_message)
 output.append(clean_message)
 return output
```

```
=====
FILE: src\idms_db2_phase2\orchestration\conversion\conversion_pipeline.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/orchestration/conversion/conversion_pipeline.py
ACTION: REPLACE ENTIRE FILE
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.analyzers.program_flow_analyzer import ProgramFlowAnalyzer
from idms_db2_phase2.domain.models import ConversionInput
from idms_db2_phase2.orchestration.conversion.conversion_layout_pipeline import (
 ConversionLayoutPipeline,
)
from idms_db2_phase2.validators.mapping_scope_filter import (
 MappingScopeFilter,
)
from idms_db2_phase2.validators.mapping_validator import MappingValidator
```

```
class ConversionPipeline:
```

```
 """Runs the CONTENT phase of the IDMS->DB2 conversion.
```

```

 Depends on the host class for the component factory methods and message
 helpers (via mixins). Returns (converted_cobol, operations,
 dclgen_repository); validation messages are appended to the passed-in
 list.
```

```

 The FINISHING phase - layout, paragraph extraction, counters, indent and
```

resequencing - lives in ConversionLayoutPipeline. That split keeps the two concerns testable in isolation: this class decides WHAT the program says, that class decides HOW it is laid out.

Pass-ordering contracts (content phase)

- 0a. ``LrfPathExpander`` rewrites Logical Record Facility syntax into the classic IDMS syntax the rest of the converter already understands::

```
OBTAIN FIRST <LR> WHERE <keyword> -> OBTAIN FIRST <rec> WITHIN <set>
LR-STATUS = 'X-EOA' -> DB-END-OF-SET
FIELD OF <rec> OF LR -> FIELD OF <rec>
```

It MUST run first, because MappingScopeFilter, CobolTransformer and ProgramFlowAnalyzer all read the source text. Every pass after this point sees classic IDMS and needs no logical-record knowledge.

- 0b. ``MappingScopeFilter`` rescopes the mapping validation report to the program being converted. MappingValidator audits the WHOLE Sheet Mapping workbook, so a restart-table gap was reported as a blocking error against every retrieval program that never touches it. A gap stays an error for a table this program references, and becomes a workbook note for any other.
1. ``feedback\_cleanup`` (CobolCleanupComposer) runs ``OutputWritePlacementCleanup``, which lifts a guarded output WRITE out of the child-row paragraph into the PARENT paragraph, after the child cursor CLOSE. Left in the child-row paragraph the WRITE fires once per fetched child row instead of once per parent row.

Steps 2 through 8 are owned by ConversionLayoutPipeline and documented there.

```
"""

Entry point

def _run_pipeline(
 self,
 *,
 conversion_input: ConversionInput,
 validation_messages: list[str],
) -> tuple[str, list, object]:
 repositories = self._repositories(conversion_input)
 resolvers = self._resolvers(repositories)
 generators = self._generators(
 repositories=repositories,
 resolvers=resolvers,
)
 transformers = self._transformers(
 repositories=repositories,
 resolvers=resolvers,
 generators=generators,
)
 composers = self._composers(
 repositories=repositories,
 resolvers=resolvers,
)

 # -----
 # LRF -> classic IDMS expansion
 #
 # Must precede mapping scope, statement transformation and flow
 # analysis: all three read the program source. A program without
 # LRF metadata, or without LRF syntax, gets its text back unchanged.
 # -----
 source_text = self._expand_logical_records(
 conversion_input=conversion_input,
 transformers=transformers,
 validation_messages=validation_messages,
)

 # -----
 # Input validation
 #
 # MappingValidator audits the ENTIRE Sheet Mapping workbook. A table
```

```

carrying rows but no usable DB2 columns is a genuine gap, but it
is a BLOCKER only for a program that references that table.
Reporting restart-table gaps against a read-only retrieval program
marked a clean conversion as Rejected, so the report is rescoped
here. Nothing is discarded: an out-of-scope gap is rewritten as a
workbook note rather than hidden.

mapping_validator = MappingValidator(
 mapping_repository=repositories["mapping"],
 dclgen_repository=repositories["dclgen"],
 table_name_resolver=resolvers["table_name"],
)

mapping_scope_filter = MappingScopeFilter(
 mapping_repository=repositories["mapping"],
 table_name_resolver=resolvers["table_name"],
)
validation_messages.extend(
 mapping_scope_filter.apply(
 messages=mapping_validator.validate(),
 source_text=source_text,
)
)
validation_messages.extend(
 self._component_messages(mapping_scope_filter)
)

IDMS -> DB2 statement transformation

converted_cobol, transform_messages, operations = transformers[
 "cobol"
].transform(
 cobol_text=source_text,
 target_program_id=conversion_input.target_program_id,
)
validation_messages.extend(transform_messages)

converted_cobol = composers["update_restart_skip"].compose(
 converted_cobol
)
validation_messages.extend(
 self._component_messages(composers["update_restart_skip"])
)

Program flow analysis (diagnostics only)

flow_analysis = ProgramFlowAnalyzer(
 mapping_rows=conversion_input.sheet_mapping_rows,
 dclgen_columns=conversion_input.dclgen_columns,
).analyze(
 cobol_text=source_text,
 operations=operations,
)
validation_messages.extend(flow_analysis.diagnostics)

Field references -> DCLGEN host variables

converted_cobol = transformers["field_reference"].rewrite(
 converted_cobol
)
validation_messages.extend(
 self._component_messages(transformers["field_reference"])
)

DB2 infrastructure, cursors

converted_cobol = self._apply_db2_infrastructure(
 converted_cobol=converted_cobol,
 operations=operations,
 generators=generators,
 composers=composers,
 validation_messages=validation_messages,

```

```

)

Timestamp / audit, SQLERROR routine

converted_cobol, timestamp_messages = generators["timestamp"].apply(
 cobol_text=converted_cobol,
 target_program_id=conversion_input.target_program_id,
)
validation_messages.extend(timestamp_messages)

converted_cobol = generators["sql_error"].ensure_sql_error_paragraph(
 converted_cobol
)

Safe cleanup passes

converted_cobol = self._apply_cleanup_composers(
 converted_cobol=converted_cobol,
 composers=composers,
 validation_messages=validation_messages,
)

FINISHING phase
#
original_idms_text is the EXPANDED source, not the raw upload.
The converted text derives from it line by line, so style
preservation and unmapped-record discovery must compare against
the same baseline.

converted_cobol = ConversionLayoutPipeline(
 composers=composers,
 resolvers=resolvers,
 component_messages=self._component_messages,
).finish(
 converted_cobol=converted_cobol,
 original_idms_text=source_text,
 validation_messages=validation_messages,
)

return converted_cobol, operations, repositories["dclgen"]

LRF expansion

def _expand_logical_records(
 self,
 *,
 conversion_input: ConversionInput,
 transformers: dict,
 validation_messages: list[str],
) -> str:
 """Rewrite LRF syntax into classic IDMS syntax.

 Returns the source text every later pass must use. Never raises:
 an expansion failure falls back to the original source and is
 reported as a validation message, because an unconverted LRF verb
 is caught downstream by the residual-IDMS checks anyway.
 """
 source_text = str(conversion_input.idms_cobol_text or "")

 expander = transformers.get("lrf_path_expander")
 if expander is None:
 return source_text

 try:
 expanded = expander.expand(source_text)
 except Exception as exc: # noqa: BLE001
 validation_messages.append(
 "LRF expansion: skipped, original IDMS source will be used. "
 f"Reason: {exc}"
)
 return source_text

```



```

validation_messages.extend(self._component_messages(expander))

return expanded or source_text

DB2 infrastructure and cursor passes

#
DB2 infrastructure and cursor passes
#
def _apply_db2_infrastructure(
 self,
 *,
 converted_cobol: str,
 operations: list,
 generators: dict,
 composers: dict,
 validation_messages: list[str],
) -> str:
 """Declare cursors, build OPEN/FETCH/CLOSE paragraphs, tidy flow.

 CORRECTION - two composers ran mute

 `cursor_flow` and `date_compare` were called for their return
 value only; their `messages` were never drained into
 validation_messages. A pass that converted nothing was therefore
 indistinguishable from a pass that had nothing to convert, which
 is precisely how a DB2 DATE comparison shipped unconverted and
 unreported. Every composer in this method now reports.
 """
 converted_cobol, infrastructure_messages = generators[
 "db2_infrastructure"
].apply(cobol_text=converted_cobol, operations=operations)
 validation_messages.extend(infrastructure_messages)

 converted_cobol = composers["cursor_order_cleanup"].compose(
 converted_cobol
)

 converted_cobol, cursor_messages = generators[
 "cursor_paragraph"
].apply(cobol_text=converted_cobol, operations=operations)
 validation_messages.extend(cursor_messages)

 #
 # Cursor driving flow.
 #
 converted_cobol = composers["cursor_flow"].compose(converted_cobol)
 validation_messages.extend(
 self._component_messages(composers["cursor_flow"])
)

 converted_cobol = composers["sqlcode_cleanup"].compose(converted_cobol)
 converted_cobol = composers["cursor_order_cleanup"].compose(
 converted_cobol
)

 #
 # DB2 DATE comparison realignment.
 #
 # A DB2 DATE host is DD.MM.CCYY, 10 bytes. A COBOL date field is
 # CCYYMMDD, PIC 9(8). Comparing them directly compares '0'
 # against '2' and selects the wrong rows, so this pass must be
 # visible in the log whether it fires or not.
 #
 date_compare = composers.get("date_compare")

 if date_compare is None:
 validation_messages.append(
 "DB2 date compare: composer is not registered; DATE host "
 "comparisons will not be realigned."
)
 else:
 converted_cobol = date_compare.compose(converted_cobol)
 validation_messages.extend(
 self._component_messages(date_compare)
)

```

```

)

 return converted_cobol

Cleanup passes

def _apply_cleanup_composers(
 self,
 *,
 converted_cobol: str,
 composers: dict,
 validation_messages: list[str],
) -> str:
 """Content-level cleanup, before any layout decision is taken.

 feedback_cleanup == CobolCleanupComposer. It runs
 OutputWritePlacementCleanup, which moves the guarded output WRITE
 out of the child-row paragraph into the parent paragraph. The
 paragraph EXTRACTION in the finishing phase depends on that
 relocation having already happened.
 """
 converted_cobol = composers["feedback_cleanup"].compose(
 converted_cobol
)
 validation_messages.extend(
 self._component_messages(composers["feedback_cleanup"])
)

 converted_cobol = composers["update_program_feedback"].compose(
 converted_cobol
)
 validation_messages.extend(
 self._component_messages(composers["update_program_feedback"])
)

 converted_cobol = composers["update_restart_skip"].compose(
 converted_cobol
)
 validation_messages.extend(
 self._component_messages(composers["update_restart_skip"])
)

 return converted_cobol

```

```
__all__ = ["ConversionPipeline"]
```

```

=====
FILE: src\idms_db2_phase2\orchestration\conversion_service.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.domain.models import ConversionInput, ConversionResult
from idms_db2_phase2.orchestration.conversion.conversion_component_factory import (
 ConversionComponentFactory,
)
from idms_db2_phase2.orchestration.conversion.conversion_message_utils import (
 ConversionMessageUtils,
)
from idms_db2_phase2.orchestration.conversion.conversion_pipeline import (
 ConversionPipeline,
)
from idms_db2_phase2.validators.input_validator import InputValidator
from idms_db2_phase2.validators.production_validator import ProductionValidator

```

```

class ConversionService(
 ConversionComponentFactory,
 ConversionPipeline,
 ConversionMessageUtils,
):
 """Main orchestration service for IDMS COBOL to DB2 COBOL conversion."""

```

```

def convert(self, conversion_input: ConversionInput) -> ConversionResult:
 validation_messages: list[str] = []

 input_messages = InputValidator().validate(conversion_input)
 validation_messages.extend(input_messages)
 if input_messages:
 return ConversionResult(
 converted_cobol="",
 validation_messages=self._unique_messages(validation_messages),
 operations=[],
)

 converted_cobol, operations, dclgen_repository = self._run_pipeline(
 conversion_input=conversion_input,
 validation_messages=validation_messages,
)

 validation_messages.extend(
 ProductionValidator(dclgen_repository=dclgen_repository).validate(
 converted_cobol
)
)

 return ConversionResult(
 converted_cobol=converted_cobol,
 validation_messages=self._unique_messages(validation_messages),
 operations=operations,
)

```

```

=====
FILE: src\idms_db2_phase2\parsers__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\parsers\base_text_parser.py
ENCODING: utf-8
=====

```

```

class BaseTextParser:
 """
 Shared parser helper methods.

 This base class intentionally avoids business rules, schema names, and
 regex definitions. It contains only generic text cleanup utilities.
 """

 def normalize_line_endings(
 self,
 text: str,
) -> str:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n")

 def normalize_spaces(
 self,
 text: str,
) -> str:
 output = str(text or "").replace("\t", " ")
 output = output.replace("\u00a0", " ")

 while " " in output:
 output = output.replace(" ", " ")

 return output

 def clean_text(
 self,
 text: str,
) -> str:
 output = self.normalize_line_endings(text)
 output = self.normalize_spaces(output)

```

```

 output = output.replace("'", "'").replace("'''", "'")
 output = output.replace("'", "'").replace("'", "'")

 return output

def clean_cell(
 self,
 value,
) -> str:
 if value is None:
 return ""

 text = str(value)
 text = text.replace("\uffff", "")
 text = text.replace("\xa0", " ")
 text = text.replace("\r\n", "\n")
 text = text.replace("\r", "\n")
 text = self.normalize_spaces(text)

 return text.strip()

def strip_sequence_area(
 self,
 line: str,
) -> str:
 text = str(line or "").rstrip()

 if len(text) > 6:
 indicator = text[6:7]

 if indicator in ("*", "/"):
 return indicator

 if text[:6].strip().isdigit():
 return text[6:]

 return text

```

```

=====
FILE: src\idms_db2_phase2\parsers\cobol_parser.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.domain.models import IdmsOperation
from idms_db2_phase2.parsers.base_text_parser import BaseTextParser
from patterns.cobol_patterns import PROGRAM_ID_PATTERN
from patterns.idms_patterns import (
 ERASE_PATTERN,
 FIND_FIRST_PATTERN,
 MODIFY_PATTERN,
 OBTAIN_CALC_PATTERN,
 OBTAIN_CALC_REVERSED_PATTERN,
 OBTAIN_FIRST_NEXT_PATTERN,
 OBTAIN_OWNER_PATTERN,
 READY_UPDATE_PATTERN,
 STORE_PATTERN,
)

```

```

class CobolParser(BaseTextParser):
 """
 Parses IDMS COBOL source and identifies IDMS database operations.

 This parser does not convert COBOL. It only identifies operations for
 later transformation, generation, diagnostics, and validation.

 Regex patterns are stored in patterns/cobol_patterns.py and
 patterns/idms_patterns.py.
 """

 def program_id(
 self,
 cobol_text: str,
) -> str:
 match = PROGRAM_ID_PATTERN.search(cobol_text or "")

```

```

 if not match:
 return ""

 return match.group(1).upper()

def analyze(
 self,
 cobol_text: str,
) -> list[IdmsOperation]:
 operations: list[IdmsOperation] = []

 for line_number, line in enumerate(
 str(cobol_text or "").splitlines(),
 start=1,
):
 clean_line = self._strip_sequence_area(line)
 upper = clean_line.upper()

 operation = self._operation_from_line(
 line=clean_line,
 upper=upper,
 line_number=line_number,
)

 if operation is not None:
 operations.append(operation)

 return operations

def _strip_sequence_area(
 self,
 line: str,
) -> str:
 """Remove COBOL fixed-format sequence areas from a raw line.

 Self-contained so this parser does not depend on an external helper
 being present on the base class. Handles:
 - columns 1-6 : left sequence number (six leading digits)
 - column 7 : comment/debug indicators ('*', '/')
 Falls back to the base-class helper when available, otherwise uses a
 conservative built-in implementation.
 """

 # Prefer the shared base-class helper if it exists (keeps behavior
 # identical to the rest of the project), but never fail if it does not.
 base_helper = getattr(super(), "strip_sequence_area", None)
 if callable(base_helper):
 try:
 return base_helper(line)
 except Exception:
 pass

 text = str(line or "").rstrip()

 if len(text) > 6:
 indicator = text[6:7]
 if indicator in ("*", "/"):
 return indicator
 if text[:6].strip().isdigit():
 return text[6:]

 return text

def _operation_from_line(
 self,
 line: str,
 upper: str,
 line_number: int,
) -> IdmsOperation | None:
 match = OBTAIN_CALC_PATTERN.search(upper)

 if match:
 return IdmsOperation(
 operation="OBTAIN_CALC",
 record_name=match.group("record").upper(),
 line_number=line_number,

```

```

 raw_line=line,
)

match = OBTAIN_CALC_REVERSED_PATTERN.search(upper)

if match:
 return IdmsOperation(
 operation="OBTAIN_CALC",
 record_name=match.group("record").upper(),
 line_number=line_number,
 raw_line=line,
)

match = OBTAIN_FIRST_NEXT_PATTERN.search(upper)

if match:
 return IdmsOperation(
 operation=f"OBTAIN_{match.group('mode').upper()}",
 record_name=match.group("record").upper(),
 set_name=match.group("set").upper(),
 line_number=line_number,
 raw_line=line,
)

match = OBTAIN_OWNER_PATTERN.search(upper)

if match:
 return IdmsOperation(
 operation="OBTAIN_OWNER",
 set_name=match.group("set").upper(),
 line_number=line_number,
 raw_line=line,
)

match = FIND_FIRST_PATTERN.search(upper)

if match:
 return IdmsOperation(
 operation="FIND_FIRST",
 record_name=(match.group("record") or "").upper(),
 set_name=match.group("set").upper(),
 line_number=line_number,
 raw_line=line,
)

match = STORE_PATTERN.search(upper)

if match:
 return IdmsOperation(
 operation="STORE",
 record_name=match.group("record").upper(),
 line_number=line_number,
 raw_line=line,
)

match = MODIFY_PATTERN.search(upper)

if match:
 return IdmsOperation(
 operation="MODIFY",
 record_name=match.group("record").upper(),
 line_number=line_number,
 raw_line=line,
)

match = ERASE_PATTERN.search(upper)

if match:
 return IdmsOperation(
 operation="ERASE",
 record_name=match.group("record").upper(),
 line_number=line_number,
 raw_line=line,
)

match = READY_UPDATE_PATTERN.search(upper)

```

```

 if match:
 return IdmsOperation(
 operation="READY_UPDATE",
 record_name=match.group("record").upper(),
 line_number=line_number,
 raw_line=line,
)

 return None

__all__ = [
 "CobolParser",
]

=====
FILE: src\idms_db2_phase2\parsers\copybook_parser.py
ENCODING: utf-8
=====

"""
Parses COBOL copybook fields.

Supports:
- unsequenced copybook text
- fixed-format copybooks with left sequence numbers
- fixed-format copybooks with right sequence numbers
- multi-line logical COBOL data declarations

Parser remains generic. It does not hardcode copybook names, program names,
record names, or business field names.
"""

from __future__ import annotations

import re

from idms_db2_phase2.domain.models import CopybookField
from idms_db2_phase2.parsers.base_text_parser import BaseTextParser
from patterns.copybook_patterns import (
 COPYBOOK_COMMENT_OR_SKIP_PATTERN,
 COPYBOOK_FIELD_PATTERN,
 COPYBOOK_OCCURS_PATTERN,
 COPYBOOK_PIC_PATTERN,
 COPYBOOK_USAGE_PATTERN,
)

class CopybookParser(BaseTextParser):
 RIGHT_SEQUENCE_PATTERN = re.compile(
 r"^(?P<body>.*?)(?:\s+(?P<right>\d{8}))\s*$",
 flags=re.IGNORECASE,
)

 def __init__(self) -> None:
 self.diagnostics: list[str] = []

 def parse(
 self,
 text: str,
 source_label: str = "Copybook",
) -> list[CopybookField]:
 self.diagnostics = []
 if not str(text or "").strip():
 self.diagnostics.append(f"{source_label}: empty copybook text.")
 return []

 logical_lines = self._logical_lines(text)
 output: list[CopybookField] = []

 # ADDED: level-breakdown counters for a self-explanatory diagnostic.
 data_field_count = 0 # real data fields (levels 01-49, 66, 77)
 condition_name_count = 0 # 88-level condition names
 filler_count = 0 # skipped FILLER lines

```

```

for line in logical_lines:
 match = COPYBOOK_FIELD_PATTERN.search(line)

 if not match:
 continue

 level = str(match.group("level") or "").strip()
 name = str(match.group("name") or "").strip().upper()
 rest = str(match.group("rest") or "")

 if not name:
 continue

 if name == "FILLER":
 filler_count += 1 # ADDED: track skipped FILLER
 continue

 # ADDED: classify by level for the breakdown diagnostic.
 if level == "88":
 condition_name_count += 1
 else:
 data_field_count += 1

 output.append(
 CopybookField(
 level=level,
 name=name,
 picture=self._picture(rest),
 usage=self._usage(rest),
 occurs=self._occurs(rest),
)
)

self.diagnostics.append(
 f"{source_label}: logical copybook lines parsed: {len(logical_lines)}"
)

CHANGED: total count now annotated with the level breakdown so the
UI number ("Copybook Fields") is self-explanatory. The returned list
length is UNCHANGED, so the UI metric behaves exactly as before.
self.diagnostics.append(
 f"{source_label}: copybook fields parsed: {len(output)} "
 f"(data fields: {data_field_count}, "
 f"88-condition names: {condition_name_count}, "
 f"FILLER skipped: {filler_count})"
)

return output

def _picture(
 self,
 text: str,
) -> str:
 match = COPYBOOK_PIC_PATTERN.search(str(text or ""))

 if not match:
 return ""

 return str(match.group("pic") or "").strip().upper().rstrip(".")

def _usage(
 self,
 text: str,
) -> str:
 match = COPYBOOK_USAGE_PATTERN.search(str(text or ""))

 if not match:
 return ""

 return str(match.group("usage") or "").strip().upper().rstrip(".")

def _occurs(
 self,
 text: str,
) -> str:

```



```

match = COPYBOOK_OCCURS_PATTERN.search(str(text or ""))

if not match:
 return ""

return str(match.group("occurs") or "").strip()

def _logical_lines(
 self,
 text: str,
) -> list[str]:
 output: list[str] = []
 buffer = ""

 for raw_line in str(text or "").splitlines():
 line = self._copybook_body_line(raw_line)
 if not line.strip():
 continue

 if COPYBOOK_COMMENT_OR_SKIP_PATTERN.search(line):
 continue

 if line.lstrip().startswith("*") or line.lstrip().startswith("/"):
 continue

 if buffer:
 buffer = f"{buffer} {line.strip()}"
 else:
 buffer = line.strip()

 if "." in line:
 parts = buffer.split(".")

 for part in parts[:-1]:
 clean = part.strip()
 if clean:
 output.append(clean + ".")

 buffer = parts[-1].strip()

 if buffer.strip():
 output.append(buffer.strip())

 return output

def _copybook_body_line(
 self,
 raw_line: str,
) -> str:
 text = str(raw_line or "").rstrip("\n").rstrip("\r")

 if not text.strip():
 return ""

 if len(text) > 6 and text[:6].strip().isdigit():
 indicator = text[6:7]

 if indicator in ("*", "/"):
 return ""

 body = text[7:] if len(text) > 7 else ""
 return self._remove_right_sequence(body).strip()

 return self._remove_right_sequence(text).strip()

def _remove_right_sequence(
 self,
 text: str,
) -> str:
 value = str(text or "").rstrip()

 match = self.RIGHT_SEQUENCE_PATTERN.match(value)
 if not match:
 return value

 body = str(match.group("body") or "").rstrip()

```

```

right = str(match.group("right") or "")
if not right:
 return value
if not right.isdigit():
 return value
return body

```

```

__all__ = [
 "CopybookParser",
]

```

```

=====
FILE: src\idms_db2_phase2\parsers\dclgen__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\parsers\dclgen\cobol_host_field_parser.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/parsers/dclgen/cobol_host_field_parser.py
ACTION: CREATE NEW FILE

```

```

"""Parses COBOL host fields (PIC / USAGE) from DCLGEN text."""

```

```

from __future__ import annotations

```

```

from patterns.dclgen_patterns import (
 COBOL_FIELD_PATTERN,
 COBOL_GROUP_PATTERN,
 DCLGEN_PIC_PATTERN,
 DCLGEN_USAGE_PATTERN,
)

```

```

class CobolHostFieldParser:

```

```

 def __init__(self, diagnostics: list[str]) -> None:
 self.diagnostics = diagnostics

```

```

 def parse(
 self,
 text: str,
 source_label: str,
) -> list[dict[str, str]]:
 fields: list[dict[str, str]] = []
 current_group = ""
 seen_group = False

```

```

 for raw_line in text.splitlines():
 line = raw_line.rstrip()
 group_match = COBOL_GROUP_PATTERN.match(line)

```

```

 if group_match:
 current_group = group_match.group(1).upper()
 seen_group = True
 continue

```

```

 if not seen_group:
 continue

```

```

 match = COBOL_FIELD_PATTERN.match(line)
 if not match:
 continue

```

```

 level = match.group(1)
 name = match.group(2).upper()
 body = match.group("body") or ""
 nullable = name.upper().endswith("-NULL")

```

```

 pic = self._picture(body)

```

```

 usage = self._usage(body)

 if not pic and not usage:
 continue

 fields.append(
 {
 "group": current_group,
 "level": level,
 "name": name,
 "picture": pic,
 "usage": usage,
 "nullable_indicator": "Y" if nullable else "N",
 }
)

 self.diagnostics.append(
 f"{source_label}: COBOL host field scan found "
 f"{len(fields)} field(s)."
)
 return fields

def _picture(self, text: str) -> str:
 match = DCLGEN_PIC_PATTERN.search(str(text or ""))
 if not match:
 return ""
 return match.group("pic").strip().upper()

def _usage(self, text: str) -> str:
 match = DCLGEN_USAGE_PATTERN.search(str(text or ""))
 if not match:
 return ""
 return match.group(1).strip().upper()

```

```

=====
FILE: src\idms_db2_phase2\parsers\dclgen\dclgen_column_merger.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/parsers/dclgen/dclgen_column_merger.py
ACTION: CREATE NEW FILE

```

```

"""Merges parsed SQL columns with COBOL host fields into DclgenColumn objects."""

```

```

from __future__ import annotations

from catalogs.dclgen_schema import DCLGEN_HOST_FIELD_SUFFIXES_TO_IGNORE
from idms_db2_phase2.domain.models import DclgenColumn
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class DclgenColumnMerger:
 def __init__(self, name_normalizer) -> None:
 self.names = name_normalizer

 def merge_sql_columns_with_cobol_hosts(
 self,
 sql_columns: list[dict[str, object]],
 cobol_fields: list[dict[str, str]],
 fallback_table_name: str,
) -> list[DclgenColumn]:
 output: list[DclgenColumn] = []
 usable_hosts = self._usable_host_fields(cobol_fields)

 for index, sql_column in enumerate(sql_columns):
 table_name = self.names.normalize_sql_name(
 str(
 sql_column.get("table_name", "")
 or fallback_table_name
 or ""
)
)
 column_name = self.names.normalize_sql_name(
 str(sql_column.get("column_name", ""))
)

```

```

 db2_type = str(sql_column.get("db2_type", "") or "").strip()
 nullable = bool(sql_column.get("nullable", True))

 host = self._best_host_for_column(
 column_name=column_name,
 index=index,
 usable_hosts=usable_hosts,
)

 output.append(
 DclgenColumn(
 table_name=table_name,
 column_name=column_name,
 db2_type=db2_type,
 cobol_host_name=host.get("name", column_name),
 cobol_picture=host.get("picture", ""),
 cobol_usage=host.get("usage", ""),
 nullable=nullable,
)
)

 return output

def fallback_columns_from_cobol_fields(
 self,
 table_name: str,
 cobol_fields: list[dict[str, str]],
) -> list[DclgenColumn]:
 output: list[DclgenColumn] = []

 for field in self._usable_host_fields(cobol_fields):
 host_name = field.get("name", "")
 if not host_name:
 continue

 output.append(
 DclgenColumn(
 table_name=table_name,
 column_name=self.names.normalize_cobol_name_to_db2(
 host_name
),
 db2_type="",
 cobol_host_name=host_name,
 cobol_picture=field.get("picture", ""),
 cobol_usage=field.get("usage", ""),
 nullable=True,
)
)

 return output

def _usable_host_fields(
 self,
 cobol_fields: list[dict[str, str]],
) -> list[dict[str, str]]:
 output: list[dict[str, str]] = []
 for field in cobol_fields:
 name = str(field.get("name", "")).upper()
 if not name:
 continue
 if any(
 name.endswith(suffix)
 for suffix in DCLGEN_HOST_FIELD_SUFFIXES_TO_IGNORE
):
 continue
 output.append(field)
 return output

def _best_host_for_column(
 self,
 column_name: str,
 index: int,
 usable_hosts: list[dict[str, str]],
) -> dict[str, str]:
 normalized_column = self.names.normalize_compare_name(column_name)

```

```

for host in usable_hosts:
 host_name = self.names.normalize_compare_name(
 host.get("name", "")
)
 if host_name == normalized_column:
 return host

for host in usable_hosts:
 host_name = self.names.normalize_compare_name(
 host.get("name", "")
)
 if normalized_column and normalized_column in host_name:
 return host
 if host_name and host_name in normalized_column:
 return host

if index < len(usable_hosts):
 return usable_hosts[index]

return {
 "name": NameNormalizer.to_cobol(column_name),
 "picture": "",
 "usage": "",
}

```

```

=====
FILE: src\idms_db2_phase2\parsers\dclgen\sql_declare_parser.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/parsers/dclgen/sql_declare_parser.py
ACTION: CREATE NEW FILE

```

```

"""Parses EXEC SQL DECLARE TABLE column definitions from DCLGEN text."""

```

```

from __future__ import annotations

```

```

from catalogs.db2_sql_types import DB2_SQL_SKIP_WORDS, DB2_SQL_TYPE_STARTERS
from patterns.dclgen_patterns import (
 DECLARE_TABLE_ALT_PATTERN,
 DECLARE_TABLE_PATTERN,
 DCLGEN_TABLE_COMMENT_PATTERN,
 VALID_SQL_COLUMN_NAME_PATTERN,
)

```

```

class SqlDeclareParser:
 def __init__(self, name_normalizer, diagnostics: list[str]) -> None:
 self.names = name_normalizer
 self.diagnostics = diagnostics

 def find_table_name(self, text: str) -> str:
 for pattern in [
 DECLARE_TABLE_PATTERN,
 DECLARE_TABLE_ALT_PATTERN,
 DCLGEN_TABLE_COMMENT_PATTERN,
]:
 match = pattern.search(text)
 if match:
 return self.names.normalize_sql_name(match.group(1))
 return ""

 def parse_columns(
 self,
 text: str,
 table_name: str,
 source_label: str,
) -> list[dict[str, object]]:
 sections = self._extract_declare_sections(text)

 if not sections:
 self.diagnostics.append(
 f"{source_label}: no EXEC SQL DECLARE TABLE section found."
)
 return []

```

```

output: list[dict[str, object]] = []

for section_index, section in enumerate(sections, start=1):
 section_table = self.find_table_name(section) or table_name
 body = self._extract_parenthesized_body(section)

 if not body:
 self.diagnostics.append(
 f"{source_label}: DECLARE section {section_index} has "
 f"no column body."
)
 continue

 parsed = self._parse_declare_body(
 body=body,
 table_name=section_table,
)
 self.diagnostics.append(
 f"{source_label}: DECLARE section {section_index} parsed "
 f"SQL columns: {len(parsed)}"
)
 output.extend(parsed)

return output

def _extract_declare_sections(self, text: str) -> list[str]:
 lines = text.splitlines()
 sections: list[str] = []
 current: list[str] = []
 inside = False

 for line in lines:
 upper = line.upper()

 if "DECLARE" in upper and "TABLE" in upper:
 inside = True
 current = [line]
 continue

 if inside:
 current.append(line)

 if "END-EXEC" in upper:
 sections.append("\n".join(current))
 current = []
 inside = False
 continue

 if upper.strip() == ")" or upper.strip().startswith(")"):
 sections.append("\n".join(current))
 current = []
 inside = False
 continue

 if inside and current:
 sections.append("\n".join(current))

 return sections

def _extract_parenthesized_body(self, section: str) -> str:
 start = section.find("(")
 if start < 0:
 return ""

 depth = 0
 body_chars: list[str] = []

 for index in range(start, len(section)):
 char = section[index]

 if char == "(":
 depth += 1
 if depth == 1:
 continue

```

```

 if char == ")":
 depth -= 1
 if depth == 0:
 break

 if depth >= 1:
 body_chars.append(char)

 return "".join(body_chars)

def _parse_declare_body(
 self,
 body: str,
 table_name: str,
) -> list[dict[str, object]]:
 logical_items = self._split_sql_items(body)

 inline_columns: list[dict[str, object]] = []
 column_names: list[str] = []
 db2_types: list[str] = []

 for item in logical_items:
 normalized_item = self._normalize_sql_item(item)
 if not normalized_item:
 continue

 inline = self._parse_inline_column_definition(
 normalized_item,
 table_name=table_name,
)
 if inline:
 inline_columns.append(inline)
 continue

 if self._looks_like_column_name(normalized_item):
 column_names.append(
 self.names.normalize_sql_name(normalized_item)
)
 continue

 if self._looks_like_db2_type(normalized_item):
 db2_types.append(self._normalize_db2_type(normalized_item))
 continue

 if inline_columns:
 return inline_columns

 output: list[dict[str, object]] = []
 for index, column_name in enumerate(column_names):
 db2_type = db2_types[index] if index < len(db2_types) else ""
 output.append(
 {
 "table_name": table_name,
 "column_name": column_name,
 "db2_type": db2_type,
 "nullable": self._is_nullable_db2_type(db2_type),
 }
)

 return output

def _split_sql_items(self, body: str) -> list[str]:
 items: list[str] = []
 current: list[str] = []
 depth = 0
 normalized = body.replace("\n", " ")

 for char in normalized:
 if char == "(":
 depth += 1
 current.append(char)
 continue
 if char == ")":
 if depth > 0:
 depth -= 1
 current.append(char)

```

```

 continue
 if char == "," and depth == 0:
 item = "".join(current).strip()
 if item:
 items.append(item)
 current = []
 continue
 current.append(char)

item = "".join(current).strip()
if item:
 items.append(item)

expanded: list[str] = []
for item in items:
 expanded.extend(
 self._split_possible_stacked_column_or_type_lines(item)
)

return expanded

def _split_possible_stacked_column_or_type_lines(
 self,
 item: str,
) -> list[str]:
 text = " ".join(str(item or "").split())
 if not text:
 return []

 inline = self._parse_inline_column_definition(text, table_name="")
 if inline:
 return [text]

 tokens = text.split()
 if len(tokens) <= 1:
 return [text]

 if self._looks_like_db2_type(text):
 return [text]

 if all(self._looks_like_column_name(token) for token in tokens):
 return tokens

 return [text]

def _parse_inline_column_definition(
 self,
 item: str,
 table_name: str,
) -> dict[str, object] | None:
 tokens = item.split()
 if len(tokens) < 2:
 return None

 first = self.names.normalize_sql_name(tokens[0])
 if not self._looks_like_column_name(first):
 return None

 remaining = " ".join(tokens[1:])
 if not self._looks_like_db2_type(remaining):
 return None

 db2_type = self._normalize_db2_type(remaining)
 return {
 "table_name": table_name,
 "column_name": first,
 "db2_type": db2_type,
 "nullable": self._is_nullable_db2_type(db2_type),
 }

def _looks_like_column_name(self, value: str) -> bool:
 text = self.names.normalize_sql_name(value)
 if not text:
 return False
 if text in DB2_SQL_SKIP_WORDS:
 return False

```



```

 if text.split(" ")[0] in DB2_SQL_TYPE_STARTERS:
 return False
 return bool(VALID_SQL_COLUMN_NAME_PATTERN.fullmatch(text))

def _looks_like_db2_type(self, value: str) -> bool:
 text = str(value or "").strip().upper()
 if not text:
 return False
 first = text.split()[0]
 first = first.split("(", 1)[0]
 return first in DB2_SQL_TYPE_STARTERS

def _normalize_sql_item(self, value: str) -> str:
 text = str(value or "").strip()
 text = text.strip(",").strip()
 text = " ".join(text.split())
 return text

def _normalize_db2_type(self, value: str) -> str:
 text = " ".join(str(value or "").replace(" ", ", ").split())
 text = text.rstrip(",")
 return text.upper()

def _is_nullable_db2_type(self, db2_type: str) -> bool:
 return "NOT NULL" not in str(db2_type or "").upper()

=====
FILE: src\idms_db2_phase2\parsers\dclgen\sql_name_normalizer.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2\parsers\dclgen\sql_name_normalizer.py
ACTION: CREATE NEW FILE

"""DCLGEN SQL / COBOL name normalization helpers."""

from __future__ import annotations

from patterns.dclgen_patterns import (
 MULTIPLE_UNDERSCORE_PATTERN,
 SQL_NAME_CLEANUP_PATTERN,
)

class DclgenNameNormalizer:
 def normalize_sql_name(self, value: str) -> str:
 text = str(value or "").strip()
 text = text.strip('\'').strip('"')
 text = text.strip(">").strip("[").strip("]")

 if "." in text:
 text = text.split(".")[1]

 text = text.replace("-", "_")
 text = SQL_NAME_CLEANUP_PATTERN.sub("_", text.upper())
 text = MULTIPLE_UNDERSCORE_PATTERN.sub("_", text)

 return text.strip("_")

 def normalize_cobol_name_to_db2(self, value: str) -> str:
 text = str(value or "").strip().upper()
 text = text.replace("-", "_")
 text = SQL_NAME_CLEANUP_PATTERN.sub("_", text)
 text = MULTIPLE_UNDERSCORE_PATTERN.sub("_", text)

 return text.strip("_")

 def normalize_compare_name(self, value: str) -> str:
 text = self.normalize_cobol_name_to_db2(value)

 if text.startswith("DCL_"):
 return text[4:]

 if text.startswith("DCL"):
 return text[3:]

```

```
return text
```

```
=====
FILE: src\idms_db2_phase2\parsers\dclgen_parser.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/parsers/dclgen_parser.py
ACTION: REPLACE ENTIRE FILE
```

```
"""
Parses DB2 DCLGEN text files.
```

```
Thin facade delegating to focused helpers. Public API preserved:
- parse_many_texts(texts) -> list[DclgenColumn]
- parse(text, source_label) -> list[DclgenColumn]
- diagnostics attribute
```

```
Static content stays externalized:
- SQL type names: catalogs/db2_sql_types.py
- DCLGEN suffix rules: catalogs/dclgen_schema.py
- Regex patterns: patterns/dclgen_patterns.py
"""
```

```
from __future__ import annotations

from idms_db2_phase2.domain.models import DclgenColumn
from idms_db2_phase2.parsers.base_text_parser import BaseTextParser
from idms_db2_phase2.parsers.dclgen.cobol_host_field_parser import (
 CobolHostFieldParser,
)
from idms_db2_phase2.parsers.dclgen.dclgen_column_merger import (
 DclgenColumnMerger,
)
from idms_db2_phase2.parsers.dclgen.sql_declare_parser import SqlDeclareParser
from idms_db2_phase2.parsers.dclgen.sql_name_normalizer import (
 DclgenNameNormalizer,
)
```

```
class DclgenParser(BaseTextParser):
 def __init__(self) -> None:
 self.diagnostics: list[str] = []
 self._names = DclgenNameNormalizer()
 self._merger = DclgenColumnMerger(self._names)

 def parse_many_texts(
 self,
 texts: list[str],
) -> list[DclgenColumn]:
 self.diagnostics = []
 output: list[DclgenColumn] = []

 self.diagnostics.append(f"DCLGEN input file count: {len(texts)}")

 for index, text in enumerate(texts, start=1):
 source_label = f"DCLGEN file {index}"
 self.diagnostics.append(
 f"{source_label} input length: {len(text or '')}"
)
 parsed = self.parse(text=text, source_label=source_label)
 self.diagnostics.append(
 f"{source_label} parsed columns: {len(parsed)}"
)
 output.extend(parsed)

 self.diagnostics.append(
 f"DCLGEN total parsed columns: {len(output)}"
)
 return output

 def parse(
 self,
 text: str,
```

```

 source_label: str = "DCLGEN",
) -> list[DclgenColumn]:
 if not text or not text.strip():
 self.diagnostics.append(f"{source_label}: empty text.")
 return []

 cleaned_text = self.clean_text(text)

 sql_declare_parser = SqlDeclareParser(
 name_normalizer=self._names,
 diagnostics=self.diagnostics,
)
 cobol_field_parser = CobolHostFieldParser(
 diagnostics=self.diagnostics,
)

 table_name = sql_declare_parser.find_table_name(cleaned_text)
 if table_name:
 self.diagnostics.append(
 f"{source_label}: resolved table name: {table_name}"
)
 else:
 self.diagnostics.append(
 f"{source_label}: table name not found."
)

 sql_columns = sql_declare_parser.parse_columns(
 text=cleaned_text,
 table_name=table_name,
 source_label=source_label,
)
 cobol_fields = cobol_field_parser.parse(
 text=cleaned_text,
 source_label=source_label,
)

 if sql_columns:
 output = self._merger.merge_sql_columns_with_cobol_hosts(
 sql_columns=sql_columns,
 cobol_fields=cobol_fields,
 fallback_table_name=table_name,
)
 self.diagnostics.append(
 f"{source_label}: SQL columns parsed: {len(sql_columns)}"
)
 self.diagnostics.append(
 f"{source_label}: COBOL host fields parsed: "
 f"{len(cobol_fields)}"
)
 self.diagnostics.append(
 f"{source_label}: merged DCLGEN columns: {len(output)}"
)
 return output

 if cobol_fields:
 self.diagnostics.append(
 f"{source_label}: no SQL DECLARE columns found; using "
 f"COBOL host fields as fallback."
)
 return self._merger.fallback_columns_from_cobol_fields(
 table_name=table_name,
 cobol_fields=cobol_fields,
)

 self.diagnostics.append(
 f"{source_label}: no SQL columns or COBOL host fields found."
)
 return []

```

```

=====
FILE: src\idms_db2_phase2\parsers\lrf_parser.py
ENCODING: utf-8
=====

```

```
LOCATION: src/idms_db2_phase2/parsers/lrf_parser.py
```

```

ACTION: CREATE NEW FILE
"""Parses IDMS subschema source into LogicalRecord metadata.

Reads only the LRF portion of the subschema:
 ADD LOGICAL RECORD ... ELEMENTS ARE ... COMMENTS ...
 ADD PATH-GROUP <verb> <lr> ... SELECT FOR KEYWORD ...

Area, record and set sections are recorded as context only.
No COBOL knowledge, no SQL, no Streamlit.
"""

from __future__ import annotations

from idms_db2_phase2.domain.models import (
 LogicalRecord,
 LrfPath,
 LrfPathCommand,
)
from patterns.lrf_patterns import (
 COMMENTS_PATTERN,
 ELEMENTS_ARE_PATTERN,
 ELEMENT_NAME_PATTERN,
 ERASE_PATTERN,
 FIND_OBTAIN_PATTERN,
 IF_SET_EMPTY_PATTERN,
 LOGICAL_RECORD_PATTERN,
 ON_STATUS_PATTERN,
 PATH_GROUP_PATTERN,
 QUOTED_TEXT_PATTERN,
 SELECT_KEYWORD_PATTERN,
 SUBSCHEMA_HEADER_PATTERN,
 WHERE_PATTERN,
)
from rules.lrf_rules import (
 DIAG_LRF_PATHS_TEMPLATE,
 DIAG_LRF_TOTAL_RECORDS_TEMPLATE,
)

COMMENT_PREFIXES = ("*", "-")

class LrfParser:
 def __init__(self) -> None:
 self.diagnostics: list[str] = []

 # ---- public API -----
 def parse(self, text: str, source_label: str = "") -> list[LogicalRecord]:
 self.diagnostics = []
 records: dict[str, LogicalRecord] = {}

 subschema_name = ""
 schema_name = ""
 mode = "" # ELEMENTS | COMMENTS | PATH
 current_lr: LogicalRecord | None = None
 current_group_verb = ""
 current_path: LrfPath | None = None
 current_command: LrfPathCommand | None = None

 for number, raw_line in enumerate(str(text or "").splitlines(), start=1):
 line = raw_line.rstrip()
 stripped = line.strip()

 if not stripped or stripped.startswith("*"):
 continue

 header = SUBSCHEMA_HEADER_PATTERN.match(line)
 if header:
 subschema_name = header.group("subschema").upper()
 schema_name = header.group("schema").upper()
 continue

 # ---- ADD LOGICAL RECORD
 lr_match = LOGICAL_RECORD_PATTERN.match(line)
 if lr_match:
 name = lr_match.group("lr").upper()
 current_lr = records.setdefault(

```

```

 name,
 LogicalRecord(
 logical_record_name=name,
 subschema_name=subschema_name,
 schema_name=schema_name,
),
)
 mode = ""
 current_path = None
 current_command = None
 continue

if ELEMENTS_ARE_PATTERN.match(line):
 mode = "ELEMENTS"
 continue

if COMMENTS_PATTERN.match(line):
 mode = "COMMENTS"
 continue

---- ADD PATH-GROUP <verb> <lr>
group = PATH_GROUP_PATTERN.match(line)
if group:
 name = group.group("lr").upper()
 current_lr = records.setdefault(
 name,
 LogicalRecord(
 logical_record_name=name,
 subschema_name=subschema_name,
 schema_name=schema_name,
),
)
 current_group_verb = group.group("verb").upper()
 mode = "PATH"
 current_path = None
 current_command = None
 continue

---- SELECT FOR KEYWORD <keyword>
keyword = SELECT_KEYWORD_PATTERN.match(line)
if keyword and current_lr is not None:
 current_path = LrfPath(
 keyword=keyword.group("keyword").upper(),
 path_group_verb=current_group_verb,
 logical_record=current_lr.logical_record_name,
)
 current_lr.paths.append(current_path)
 current_command = None
 continue

if stripped == ".":
 mode = ""
 current_path = None
 current_command = None
 continue

---- body lines
if mode == "ELEMENTS" and current_lr is not None:
 element = ELEMENT_NAME_PATTERN.match(stripped)
 if element:
 value = element.group("record").upper()
 if value not in current_lr.element_records:
 current_lr.element_records.append(value)
 continue

if mode == "COMMENTS" and current_lr is not None:
 for quoted in QUOTED_TEXT_PATTERN.finditer(stripped):
 current_lr.comments.append(quoted.group("text"))
 continue

if mode == "PATH" and current_path is not None:
 current_command = self._path_line(
 stripped=stripped,
 raw_line=raw_line,
 number=number,
 path=current_path,

```

```

 current_command=current_command,
)

 result = list(records.values())
 self.diagnostics.append(
 DIAG_LRF_TOTAL_RECORDS_TEMPLATE.format(count=len(result))
)
 for record in result:
 self.diagnostics.append(
 DIAG_LRF_PATHS_TEMPLATE.format(
 lr=record.logical_record_name,
 count=len(record.paths),
)
)
 return result

---- helpers -----
def _path_line(
 self,
 *,
 stripped: str,
 raw_line: str,
 number: int,
 path: LrfPath,
 current_command: LrfPathCommand | None,
) -> LrfPathCommand | None:
 status = ON_STATUS_PATTERN.match(stripped)
 if status and current_command is not None:
 current_command.status_actions[status.group("status")] = (
 status.group("action").strip().upper()
)
 return current_command

 where = WHERE_PATTERN.match(stripped)
 if where and current_command is not None:
 current_command.where_clause = where.group("clause").strip().upper()
 return current_command

 empty = IF_SET_EMPTY_PATTERN.match(stripped)
 if empty:
 command = LrfPathCommand(
 verb="IF",
 scope="NOT EMPTY" if empty.group("negate") else "EMPTY",
 within_name=empty.group("set_name").upper(),
 line_number=number,
 raw_line=raw_line,
)
 path.commands.append(command)
 return command

 erase = ERASE_PATTERN.match(stripped)
 if erase:
 command = LrfPathCommand(
 verb="ERASE",
 record_name=erase.group("record").upper(),
 line_number=number,
 raw_line=raw_line,
)
 path.commands.append(command)
 return command

 access = FIND_OBTAIN_PATTERN.match(stripped)
 if access:
 command = LrfPathCommand(
 verb=access.group("verb").upper(),
 scope=(access.group("scope") or "").upper(),
 record_name=access.group("record").upper(),
 within_name=(access.group("within") or "").upper(),
 line_number=number,
 raw_line=raw_line,
)
 path.commands.append(command)
 return command

 return current_command

```

```
=====
FILE: src\idms_db2_phase2\parsers\sheet_mapping__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\parsers\sheet_mapping\sheet_mapping_cell_utils.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from patterns.sheet_mapping_patterns import (
 CELL_WHITESPACE_PATTERN,
 HEADER_NON_ALPHANUMERIC_PATTERN,
 HEADER_WHITESPACE_PATTERN,
)

class SheetMappingCellUtils:
 """Cell and header text normalization helpers.

 Owns no schema and no regex definitions (patterns live in
 patterns/sheet_mapping_patterns.py).
 """

 def _cell_to_string(self, value) -> str:
 if value is None:
 return ""
 text = str(value)
 text = text.replace("\uffeff", "")
 text = text.replace("\xa0", " ")
 text = text.replace("\r\n", "\n")
 text = text.replace("\r", "\n")
 text = CELL_WHITESPACE_PATTERN.sub(" ", text)
 return text.strip()

 def _normalize_header(self, value: str) -> str:
 text = self._cell_to_string(value)
 text = text.upper()
 text = text.replace("_", " ")
 text = HEADER_NON_ALPHANUMERIC_PATTERN.sub("", text)
 text = HEADER_WHITESPACE_PATTERN.sub(" ", text)
 return text.strip()
```

```
=====
FILE: src\idms_db2_phase2\parsers\sheet_mapping\sheet_mapping_diagnostics.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.domain.models import SheetMappingRow
from rules.sheet_mapping_parser_rules import (
 POPULATION_DIAGNOSTIC_SPEC,
 USEFUL_CONTEXT_GROUPS,
 USEFUL_CONTEXT_LABEL,
)

class SheetMappingDiagnostics:
 """Population diagnostics driven by a spec (no per-metric repetition)."""

 def _add_population_diagnostics(self, rows: list[SheetMappingRow]) -> None:
 for label, attrs in POPULATION_DIAGNOSTIC_SPEC:
 count = sum(1 for row in rows if self._any_attr_set(row, attrs))
 self.diagnostics.append(f"{label}: {count}")

 useful = sum(
 1
 for row in rows
```

```

 if all(self._any_attr_set(row, group) for group in USEFUL_CONTEXT_GROUPS)
)
 self.diagnostics.append(f"{USEFUL_CONTEXT_LABEL}: {useful}")

 @staticmethod
 def _any_attr_set(row: SheetMappingRow, attrs: list[str]) -> bool:
 return any(str(getattr(row, attr, "") or "").strip() for attr in attrs)

=====
FILE: src\idms_db2_phase2\parsers\sheet_mapping\sheet_mapping_row_mapper.py
ENCODING: utf-8
=====

from __future__ import annotations

from catalogs.sheet_mapping_schema import (
 SHEET_MAPPING_CANONICAL_COLUMNS,
 SHEET_MAPPING_FIELD_ALIASES,
 SHEET_MAPPING_HEADER_DETECTION_GROUPS,
 SHEET_MAPPING_MODEL_FIELD_MAP,
)
from idms_db2_phase2.domain.models import SheetMappingRow
from rules.sheet_mapping_parser_rules import (
 DIAG_SHEET_NORMALIZED_CELLS_TEMPLATE,
 HEADER_DEBUG_ROW_LIMIT,
 HEADER_SCAN_ROW_LIMIT,
)

class SheetMappingRowMapper:
 """Header detection, row-to-dict, and dict-to-model mapping.

 Column names, aliases, and detection groups come from
 catalogs/sheet_mapping_schema.py.
 """

 def _row_to_dict(self, headers: list[str], row: tuple) -> dict[str, str]:
 raw_row: dict[str, str] = {}
 for index, header in enumerate(headers):
 clean_header = self._cell_to_string(header)
 if not clean_header:
 continue
 value = row[index] if index < len(row) else ""
 raw_row[clean_header] = self._cell_to_string(value)
 return raw_row

 def _find_header_row(self, rows: list[tuple], sheet_title: str) -> int:
 first_canonical = self._normalize_header(SHEET_MAPPING_CANONICAL_COLUMNS[0])

 for index, row in enumerate(rows[:HEADER_SCAN_ROW_LIMIT]):
 normalized_cells = {
 self._normalize_header(self._cell_to_string(value))
 for value in row
 if value is not None
 }

 if index < HEADER_DEBUG_ROW_LIMIT:
 self.diagnostics.append(
 DIAG_SHEET_NORMALIZED_CELLS_TEMPLATE.format(
 sheet=sheet_title,
 index=index,
 cells=sorted(normalized_cells),
)
)

 if first_canonical in normalized_cells:
 return index

 if self._row_has_any_header(
 normalized_cells=normalized_cells,
 aliases=SHEET_MAPPING_HEADER_DETECTION_GROUPS[0],
):
 return index

 if self._row_has_any_header(

```



```

 normalized_cells=normalized_cells,
 aliases=SHEET_MAPPING_HEADER_DETECTION_GROUPS[1],
) and self._row_has_any_header(
 normalized_cells=normalized_cells,
 aliases=SHEET_MAPPING_HEADER_DETECTION_GROUPS[2],
):
 return index

 return -1

def _row_has_any_header(
 self, normalized_cells: set[str], aliases: list[str]
) -> bool:
 return any(
 self._normalize_header(alias) in normalized_cells for alias in aliases
)

def _to_mapping_row(self, raw_row: dict[str, str]) -> SheetMappingRow:
 values = {
 model_field: self._get(row=raw_row, canonical_name=canonical_column)
 for model_field, canonical_column in SHEET_MAPPING_MODEL_FIELD_MAP.items()
 }
 return SheetMappingRow(**values)

def _get(self, row: dict[str, str], canonical_name: str) -> str:
 aliases = SHEET_MAPPING_FIELD_ALIASES.get(canonical_name, [canonical_name])
 normalized_lookup = {
 self._normalize_header(key): value for key, value in row.items()
 }
 for alias in aliases:
 value = normalized_lookup.get(self._normalize_header(alias))
 if value is not None:
 return str(value).strip()
 return ""

def _has_useful_content(self, row: SheetMappingRow) -> bool:
 return any(
 str(getattr(row, field_name) or "").strip()
 for field_name in SHEET_MAPPING_MODEL_FIELD_MAP
)

```

```

=====
FILE: src\idms_db2_phase2\parsers\sheet_mapping_parser.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import csv
from io import BytesIO, StringIO

from openpyxl import load_workbook

from idms_db2_phase2.domain.models import SheetMappingRow
from idms_db2_phase2.parsers.sheet_mapping.sheet_mapping_cell_utils import (
 SheetMappingCellUtils,
)
from idms_db2_phase2.parsers.sheet_mapping.sheet_mapping_diagnostics import (
 SheetMappingDiagnostics,
)
from idms_db2_phase2.parsers.sheet_mapping.sheet_mapping_row_mapper import (
 SheetMappingRowMapper,
)
from rules.sheet_mapping_parser_rules import (
 CSV_SAMPLE_LENGTH,
 CSV_TEXT_ENCODING,
 DIAG_CSV_DECODED_LEN_TEMPLATE,
 DIAG_CSV_EMPTY,
 DIAG_CSV_HEADERS_TEMPLATE,
 DIAG_CSV_NO_ROWS,
 DIAG_CSV_SAMPLE_TEMPLATE,
 DIAG_CSV_USEFUL_ROWS_TEMPLATE,
 DIAG_FILE_NAME_TEMPLATE,
 DIAG_FILE_SIZE_TEMPLATE,
 DIAG_NO_FILE,
)

```

```

DIAG_SHEET_NO_HEADER_TEMPLATE,
DIAG_SHEET_USEFUL_ROWS_TEMPLATE,
DIAG_UNSUPPORTED_XLS,
DIAG_XLSX_EMPTY,
DIAG_XLSX_USEFUL_ROWS_TEMPLATE,
LEGACY_XLS_SUFFIX,
XLSX_SUFFIX,
)

class SheetMappingParser(
 SheetMappingCellUtils,
 SheetMappingRowMapper,
 SheetMappingDiagnostics,
):
 """Parses Sheet Mapping files from CSV/text or XLSX.

 Column names, aliases, and header detection groups live in
 catalogs/sheet_mapping_schema.py; regex in patterns/sheet_mapping_patterns.py;
 messages and magic numbers in rules/sheet_mapping_parser_rules.py.
 Cell utils, row mapping, and diagnostics are provided by mixins.
 """

 def __init__(self) -> None:
 self.diagnostics: list[str] = []

 def parse_uploaded_file(self, uploaded_file) -> list[SheetMappingRow]:
 self.diagnostics = []

 if uploaded_file is None:
 self.diagnostics.append(DIAG_NO_FILE)
 return []

 file_name = str(uploaded_file.name or "").lower()
 raw_bytes = uploaded_file.getvalue()

 self.diagnostics.append(DIAG_FILE_NAME_TEMPLATE.format(name=file_name))
 self.diagnostics.append(DIAG_FILE_SIZE_TEMPLATE.format(size=len(raw_bytes)))

 if file_name.endswith(XLSX_SUFFIX):
 return self.parse_xlsx_bytes(raw_bytes)

 if file_name.endswith(LEGACY_XLS_SUFFIX):
 self.diagnostics.append(DIAG_UNSUPPORTED_XLS)
 return []

 text = raw_bytes.decode(CSV_TEXT_ENCODING, errors="ignore")
 self.diagnostics.append(
 DIAG_CSV_DECODED_LEN_TEMPLATE.format(length=len(text))
)

 if text:
 sample = (
 text[:CSV_SAMPLE_LENGTH].replace("\r", "\\r").replace("\n", "\\n")
)
 self.diagnostics.append(DIAG_CSV_SAMPLE_TEMPLATE.format(sample=sample))

 return self.parse_csv_text(text)

 def parse_csv_text(self, text: str) -> list[SheetMappingRow]:
 if not str(text or "").strip():
 self.diagnostics.append(DIAG_CSV_EMPTY)
 return []

 raw_rows = [tuple(row) for row in csv.reader(StringIO(text))]
 if not raw_rows:
 self.diagnostics.append(DIAG_CSV_NO_ROWS)
 return []

 headers = [self._cell_to_string(value) for value in raw_rows[0]]
 self.diagnostics.append(DIAG_CSV_HEADERS_TEMPLATE.format(headers=headers))

 output = self._parse_data_rows(headers=headers, data_rows=raw_rows[1:])

 self.diagnostics.append(
 DIAG_CSV_USEFUL_ROWS_TEMPLATE.format(count=len(output))
)

```

```

)
 self._add_population_diagnostics(output)
 return output

def parse_xlsx_bytes(self, raw_bytes: bytes) -> list[SheetMappingRow]:
 if not raw_bytes:
 self.diagnostics.append(DIAG_XLSX_EMPTY)
 return []

 workbook = load_workbook(
 BytesIO(raw_bytes), data_only=True, read_only=True
)
 output: list[SheetMappingRow] = []

 for worksheet in workbook.worksheets:
 rows = list(worksheet.iter_rows(values_only=True))
 if not rows:
 continue

 header_index = self._find_header_row(
 rows=rows, sheet_title=worksheet.title
)
 if header_index < 0:
 self.diagnostics.append(
 DIAG_SHEET_NO_HEADER_TEMPLATE.format(sheet=worksheet.title)
)
 continue

 headers = [
 self._cell_to_string(value) for value in rows[header_index]
]
 parsed_rows = self._parse_data_rows(
 headers=headers, data_rows=rows[header_index + 1:]
)
 self.diagnostics.append(
 DIAG_SHEET_USEFUL_ROWS_TEMPLATE.format(
 sheet=worksheet.title, count=len(parsed_rows)
)
)
 output.extend(parsed_rows)

 self.diagnostics.append(
 DIAG_XLSX_USEFUL_ROWS_TEMPLATE.format(count=len(output))
)
 self._add_population_diagnostics(output)
 return output

def _parse_data_rows(
 self, *, headers: list[str], data_rows
) -> list[SheetMappingRow]:
 output: list[SheetMappingRow] = []
 for row in data_rows:
 raw_row = self._row_to_dict(headers=headers, row=row)
 mapping_row = self._to_mapping_row(raw_row)
 if self._has_useful_content(mapping_row):
 output.append(mapping_row)
 return output

```

```

=====
FILE: src\idms_db2_phase2\parsers\text_loader.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.infrastructure.file_loader import FileLoader

```

```

class TextLoader(FileLoader):
 """
 Backward-compatible alias for FileLoader.

 Existing UI and testing code may import TextLoader from parsers.
 Clean architecture keeps actual file loading in infrastructure.
 """

 pass

```

```
=====
FILE: src\idms_db2_phase2\postprocess__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\postprocess\cobol_update_standard_generator.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.postprocess.dynamic_metadata_resolver import (
 DclgenRoleInfo,
 UpdateProgramContext,
)
from rules.update_restart_rules import (
 RESTART_DEFAULT_PHASE,
 RESTART_INSERT_QUERYNO,
 RESTART_PARAGRAPH_NAMES,
 RESTART_RETENTION_DAYS,
 RESTART_SELECT_QUERYNO,
 RESTART_STATUS_COMPLETE,
 RESTART_STATUS_INCOMPLETE,
 RESTART_UPDATE_QUERYNO,
 UPDATE_RESTART_WS_NAMES,
 UPDATE_STANDARD_DATE_WS_LINES,
)
from rules.update_standard_generator_templates import (
 ABEND_CALL_STATEMENT,
 COMMIT_COUNTER_LEGACY,
 COMMIT_COUNTER_RESET_VALUE,
 COPYBOOK_INCLUDE_TEMPLATE,
 CS_PROGRAM_FIELD,
 DA_DD_MM_CCYY_FIELD,
 DCLGEN_INCLUDE_TEMPLATE,
 DISPLAY_ERROR_REC_TEMPLATE,
 DISPLAY_PREVIOUS_RUN_COMPLETE,
 EOF_NOT_SET_VALUE,
 EOF_SET_VALUE,
 PROCESS_COMMIT_TEMPLATE,
 READ_FLAT_FILE_TEMPLATE,
 RESTART_CONTROL_TEMPLATE,
 RESTART_FOUND_TEMPLATE,
 RESTART_LENGTH_BASE,
 RESTART_SQL_TEMPLATE,
 SQL_ERROR_PARAGRAPH,
 WORKING_STORAGE_TEMPLATE,
 WRITE_RESTART_TEMPLATE,
)

class CobolUpdateStandardGenerator:
 """Generates update-program COBOL snippets from resolved metadata.

 The generator owns no COBOL layout literals: every template lives in
 rules/update_standard_generator_templates.py. Only dynamic metadata
 (record, field, table, host names) is substituted here. No source
 program, copybook, DB2 table, DCLGEN, or host variable name is hardcoded.
 """

 def copybook_include(self, context: UpdateProgramContext) -> str:
 return COPYBOOK_INCLUDE_TEMPLATE.format(
 include=context.copybook_include_name
)

 def dclgen_include(self, role: DclgenRoleInfo) -> str:
 return DCLGEN_INCLUDE_TEMPLATE.format(include=role.include_name)

 def dclgen_include_by_name(self, include_name: str) -> str:
 return DCLGEN_INCLUDE_TEMPLATE.format(
```

```

 include=str(include_name or "").strip().upper()
)

def working_storage(self, context: UpdateProgramContext) -> str:
 names = UPDATE_RESTART_WS_NAMES
 length = context.record_length
 return WORKING_STORAGE_TEMPLATE.format(
 switch_group=names["switch_group"],
 record=context.input_record_name,
 eof_not=EOF_NOT_SET_VALUE,
 eof_set=EOF_SET_VALUE,
 help_group=names["help_group"],
 restart_record=names["restart_record"],
 read_count=names["read_count"],
 restart_key=names["restart_key"],
 input_save_area=names["input_save_area"],
 commit_counter=names["commit_counter"],
 input_counter=names["input_counter"],
 update_counter=names["update_counter"],
 restart_len=names["restart_len"],
 length=length,
 restart_length=RESTART_LENGTH_BASE + length,
)

def date_working_storage(self) -> str:
 return "\n".join(UPDATE_STANDARD_DATE_WS_LINES)

def read_flat_file(self, context: UpdateProgramContext) -> str:
 names = UPDATE_RESTART_WS_NAMES
 return READ_FLAT_FILE_TEMPLATE.format(
 record=context.input_record_name,
 file_name=context.input_file.name,
 commit_counter=names["commit_counter"],
 input_counter=names["input_counter"],
 input_save_area=names["input_save_area"],
)

def restart_paragraphs(self, context: UpdateProgramContext) -> str:
 parts = [
 self.restart_control(context),
 self.restart_found(context),
 self.write_restart(context),
 self.process_commit(context),
 self.restart_sql(context),
]
 return "\n\n".join(part for part in parts if part.strip())

def restart_control(self, context: UpdateProgramContext) -> str:
 role = context.restart_dclgen
 if not role:
 return ""

 p = RESTART_PARAGRAPH_NAMES
 return RESTART_CONTROL_TEMPLATE.format(
 control=p["control"],
 cs_program=CS_PROGRAM_FIELD,
 da_field=DA_DD_MM_CCYY_FIELD,
 program_field=role.program_field,
 date_field=role.date_field,
 phase_field=role.phase_field,
 host_record=role.host_record_name,
 default_phase=RESTART_DEFAULT_PHASE,
 select=p["select"],
 restart_found=p["restart_found"],
 write_restart=p["write_restart"],
 sql_error=SQL_ERROR_PARAGRAPH,
 commit_reset=COMMIT_COUNTER_RESET_VALUE,
 commit_counter_legacy=COMMIT_COUNTER_LEGACY,
)

def restart_found(self, context: UpdateProgramContext) -> str:
 role = context.restart_dclgen
 if not role:
 return ""

 p = RESTART_PARAGRAPH_NAMES

```

```

names = UPDATE_RESTART_WS_NAMES
return RESTART_FOUND_TEMPLATE.format(
 restart_found=p["restart_found"],
 payload_text_field=role.payload_text_field,
 host_record=role.host_record_name,
 restart_record=names["restart_record"],
 status_field=role.status_field,
 status_complete=RESTART_STATUS_COMPLETE,
 display_complete=DISPLAY_PREVIOUS_RUN_COMPLETE,
 abend=p["abend"],
 read_count=names["read_count"],
 input_counter=names["input_counter"],
)

def write_restart(self, context: UpdateProgramContext) -> str:
 role = context.restart_dclgen
 if not role:
 return ""

 p = RESTART_PARAGRAPH_NAMES
 names = UPDATE_RESTART_WS_NAMES
 return WRITE_RESTART_TEMPLATE.format(
 write_restart=p["write_restart"],
 status_incomplete=RESTART_STATUS_INCOMPLETE,
 status_field=role.status_field,
 host_record=role.host_record_name,
 retention_days=RESTART_RETENTION_DAYS,
 retention_field=role.retention_field,
 restart_len=names["restart_len"],
 payload_len_field=role.payload_len_field,
 input_counter=names["input_counter"],
 read_count=names["read_count"],
 restart_record=names["restart_record"],
 restart_key=names["restart_key"],
 payload_text_field=role.payload_text_field,
 insert=p["insert"],
)

def process_commit(self, context: UpdateProgramContext) -> str:
 role = context.restart_dclgen
 if not role:
 return ""

 p = RESTART_PARAGRAPH_NAMES
 names = UPDATE_RESTART_WS_NAMES
 return PROCESS_COMMIT_TEMPLATE.format(
 commit=p["commit"],
 commit_counter=names["commit_counter"],
 input_counter=names["input_counter"],
 read_count=names["read_count"],
 restart_record=names["restart_record"],
 input_save_area=names["input_save_area"],
 restart_key=names["restart_key"],
 record=context.input_record_name,
 status_complete=RESTART_STATUS_COMPLETE,
 status_incomplete=RESTART_STATUS_INCOMPLETE,
 status_field=role.status_field,
 host_record=role.host_record_name,
 restart_len=names["restart_len"],
 payload_len_field=role.payload_len_field,
 payload_text_field=role.payload_text_field,
 update=p["update"],
 display_error=DISPLAY_ERROR_REC_TEMPLATE.format(
 counter=names["input_counter"]
),
 sql_error=SQL_ERROR_PARAGRAPH,
)

def restart_sql(self, context: UpdateProgramContext) -> str:
 role = context.restart_dclgen
 if not role:
 return ""

 p = RESTART_PARAGRAPH_NAMES
 names = UPDATE_RESTART_WS_NAMES
 display_error = DISPLAY_ERROR_REC_TEMPLATE.format(

```

```

 counter=names["input_counter"]
)
 return RESTART_SQL_TEMPLATE.format(
 select=p["select"],
 update=p["update"],
 insert=p["insert"],
 abend=p["abend"],
 program_column=role.program_column,
 phase_column=role.phase_column,
 date_column=role.date_column,
 status_column=role.status_column,
 retention_column=role.retention_column,
 payload_column=role.payload_column,
 host_record=role.host_record_name,
 program_field=role.program_field,
 phase_field=role.phase_field,
 date_field=role.date_field,
 status_field=role.status_field,
 retention_field=role.retention_field,
 payload_group=role.payload_group,
 table_name=role.table_name,
 select_queryno=RESTART_SELECT_QUERYNO,
 update_queryno=RESTART_UPDATE_QUERYNO,
 insert_queryno=RESTART_INSERT_QUERYNO,
 display_error=display_error,
 sql_error=SQL_ERROR_PARAGRAPH,
 abend_call=ABEND_CALL_STATEMENT,
)

```

```

=====
FILE: src\idms_db2_phase2\postprocess\dynamic_metadata_resolver.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from dataclasses import fields, is_dataclass
from typing import Any

```

```

from idms_db2_phase2.postprocess.update_cobol_file_resolver import (
 UpdateCobolFileResolver,
)
from idms_db2_phase2.postprocess.update_copybook_resolver import (
 UpdateCopybookResolver,
)
from idms_db2_phase2.postprocess.update_metadata_models import (
 CobolFileInfo,
 CopybookRecordInfo,
 DclgenRoleInfo,
 UpdateProgramContext,
)
from idms_db2_phase2.postprocess.update_restart_dclgen_resolver import (
 UpdateRestartDclgenResolver,
)
from idms_db2_phase2.services.name_derivation_resolver import (
 NameDerivationResolver,
)

```

```

class DynamicMetadataResolver:
 """

```

```

 Resolves update-program metadata from parsed COBOL, Copybook, and DCLGEN data.

```

```

 This resolver is used only by update postprocess.
 It does not hardcode program-specific copybook names, DCLGEN names,
 DB2 table names, or host variables.

```

```

 Restart role matching is delegated to UpdateRestartDclgenResolver and is
 token-driven through rules/update_restart_rules.py.
 """

```

```

 def __init__(self) -> None:
 self.cobol_resolver = UpdateCobolFileResolver()
 self.copybook_resolver = UpdateCopybookResolver()
 self.restart_dclgen_resolver = UpdateRestartDclgenResolver()
 self.name_derivation_resolver = NameDerivationResolver() # ADDED

```

```

def resolve(
 self,
 *,
 source_cobol: str,
 generated_cobol: str,
 copybook_fields: list[Any],
 dclgen_columns: list[Any],
 target_program_id: str = "",
 mapping_repository: Any | None = None, # ADDED
) -> UpdateProgramContext:
 diagnostics: list[str] = []

 program_id = self.cobol_resolver.resolve_program_id(
 generated_cobol=generated_cobol,
 source_cobol=source_cobol,
 target_program_id=target_program_id,
)
 diagnostics.append(f"Resolved PROGRAM-ID: {program_id}")

 input_files = self.cobol_resolver.parse_cobol_files(
 source_cobol=source_cobol,
 generated_cobol=generated_cobol,
)

 if not input_files:
 raise ValueError("Unable to resolve input file from COBOL SELECT/FD/READ.")

 input_file = self.cobol_resolver.choose_input_file(input_files)
 diagnostics.append(f"Resolved input file: {input_file.name}")
 diagnostics.append(f"Resolved FD record: {input_file.fd_record}")
 diagnostics.append(f"Resolved record length: {input_file.record_length}")
 diagnostics.append(f"Resolved READ target: {input_file.read_target}")

 copybooks = self.copybook_resolver.build_copybook_records(copybook_fields)
 diagnostics.append(f"Resolved copybook record candidates: {len(copybooks)}")

 source_refs = self.cobol_resolver.extract_source_field_references(
 source_cobol
)
 copy_statements = self.cobol_resolver.extract_copy_statements(
 source_cobol + "\n" + generated_cobol
)

 input_copybook = self.copybook_resolver.choose_copybook(
 input_file=input_file,
 copybooks=copybooks,
 copy_statements=copy_statements,
 source_refs=source_refs,
)
 # Apply the deterministic VM...BD... -> VMDZ... derivation to the
 # input record and copybook include names, matching the manual's
 # program-ID rule (e.g. VMBD205I -> VMDZ205I). Generic + rule-driven;
 # non-matching names are returned unchanged. If nothing changes, the
 # original copybook is kept as-is.
 derived_record = self.name_derivation_resolver.derive(
 input_copybook.record_name
)
 derived_include = self.name_derivation_resolver.derive(
 input_copybook.include_name
)

 if (
 derived_record != input_copybook.record_name
 or derived_include != input_copybook.include_name
):
 input_copybook = CopybookRecordInfo(
 include_name=derived_include,
 record_name=derived_record,
 source_label=input_copybook.source_label,
 fields=input_copybook.fields,
 estimated_length=input_copybook.estimated_length,
)
 diagnostics.append(
 "Derived input record/copybook name via VM...BD...->VMDZ "
 f"rule: {derived_record} (COPY {derived_include})."

```



```

)
 diagnostics.append(f"Resolved copybook include: {input_copybook.include_name}")
 diagnostics.append(f"Resolved copybook 01 record: {input_copybook.record_name}")
 diagnostics.append(
 f"Resolved copybook length: {input_copybook.estimated_length}"
)

 legacy_eof_switch = self.cobol_resolver.resolve_legacy_eof_switch(
 source_cobol=source_cobol,
 generated_cobol=generated_cobol,
)

 if legacy_eof_switch:
 diagnostics.append(f"Resolved legacy EOF switch: {legacy_eof_switch}")
 else:
 diagnostics.append("Legacy EOF switch not resolved.")

 restart_dclgen = self.restart_dclgen_resolver.resolve_restart_dclgen(
 dclgen_columns,
 mapping_repository=mapping_repository, # ADDED
)

 if restart_dclgen:
 diagnostics.append(f"Resolved restart table: {restart_dclgen.table_name}")
 diagnostics.append(
 f"Resolved restart include: {restart_dclgen.include_name}"
)
 diagnostics.append(
 "Resolved restart host record: "
 f"{self._safe_attr(restart_dclgen, 'host_record_name')}"
)
 diagnostics.append(
 "Resolved restart program column: "
 f"{self._safe_attr(restart_dclgen, 'program_column')}"
)
 diagnostics.append(
 "Resolved restart program field: "
 f"{self._safe_attr(restart_dclgen, 'program_field')}"
)
 diagnostics.append(
 "Resolved restart phase field: "
 f"{self._safe_attr(restart_dclgen, 'phase_field')}"
)
 diagnostics.append(
 "Resolved restart date field: "
 f"{self._safe_attr(restart_dclgen, 'date_field')}"
)
 diagnostics.append(
 "Resolved restart status field: "
 f"{self._safe_attr(restart_dclgen, 'status_field')}"
)
 diagnostics.append(
 "Resolved restart retention field: "
 f"{self._safe_attr(restart_dclgen, 'retention_field')}"
)
 diagnostics.append(
 "Resolved restart payload column: "
 f"{self._safe_attr(restart_dclgen, 'payload_column')}"
)
 diagnostics.append(
 "Resolved restart payload group: "
 f"{self._safe_attr(restart_dclgen, 'payload_group')}"
)
 diagnostics.append(
 "Resolved restart payload length field: "
 f"{self._safe_attr(restart_dclgen, 'payload_len_field')}"
)
 diagnostics.append(
 "Resolved restart payload text field: "
 f"{self._safe_attr(restart_dclgen, 'payload_text_field')}"
)
 else:
 # ADDED: distinguish Case 3 (mapping-confirmation failed) in logs
 if mapping_repository is not None and dclgen_columns:
 diagnostics.append(
 "Restart flow skipped: a DCLGEN restart group was found "

```

```

 "but the restart table is not present in the Sheet "
 "Mapping. AND-gate not satisfied (Case 3).\"
)
 diagnostics.append("Restart DCLGEN not resolved.")

 return self._make_context(
 input_file=input_file,
 input_copybook=input_copybook,
 restart_dclgen=restart_dclgen,
 program_id=program_id,
 legacy_eof_switch=legacy_eof_switch,
 source_program_text=source_cobol,
 diagnostics=diagnostics,
)

def _make_context(
 self,
 **values: Any,
) -> UpdateProgramContext:
 """
 Creates UpdateProgramContext while staying compatible with the actual
 dataclass in update_metadata_models.py.

 The current uploaded model requires:
 - input_file
 - input_copybook
 - restart_dclgen
 - program_id

 Optional values are passed only if the local dataclass supports them.
 """
 if is_dataclass(UpdateProgramContext):
 accepted = {field.name for field in fields(UpdateProgramContext)}
 kwargs = {
 key: value
 for key, value in values.items()
 if key in accepted
 }
 context = UpdateProgramContext(**kwargs)
 else:
 context = UpdateProgramContext()

 for key, value in values.items():
 if not hasattr(context, key):
 try:
 setattr(context, key, value)
 except Exception:
 pass

 return context

def _safe_attr(
 self,
 obj: Any,
 name: str,
) -> str:
 return str(getattr(obj, name, "")) or ""

__all__ = [
 "CobolFileInfo",
 "CopybookRecordInfo",
 "DclgenRoleInfo",
 "DynamicMetadataResolver",
 "UpdateProgramContext",
]

```

```

=====
FILE: src\idms_db2_phase2\postprocess\final_cleanup__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\postprocess\final_cleanup\final_cleanup_dcl_reference.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from patterns.update_final_cleanup_patterns import (
 END_EXEC_PATTERN,
 EXEC_SQL_PATTERN,
 MOVE_TO_DCL_DOT_REFERENCE_PATTERN,
 NON_SQL_DCL_DOT_REFERENCE_STRICT_PATTERN,
 TO_DCL_DOT_REFERENCE_PATTERN,
)
from rules.update_cobol_final_cleanup_rules import (
 FINAL_CLEANUP_AREA_B,
 FINAL_CLEANUP_BODY_WIDTH,
 FINAL_CLEANUP_DCL_PREFIX,
 FINAL_CLEANUP_FIELD_OF_GROUP_TEMPLATE,
 FINAL_CLEANUP_MOVE_TEMPLATE,
 FINAL_CLEANUP_TOKEN_END_EXEC,
 FINAL_CLEANUP_TOKEN_MOVE,
 FINAL_CLEANUP_TOKEN_TO,
 FINAL_CLEANUP_TO_FIELD_OF_GROUP_TEMPLATE,
)

class FinalCleanupDclReference:
 """Rewrites non-SQL DCLGROUP.FIELD references to FIELD OF DCLGROUP."""

 def _rewrite_non_sql_dcl_dot_references(self, lines: list[str]) -> list[str]:
 output: list[str] = []
 in_exec_sql = False

 for line in lines:
 logical = self._logical(line)
 upper = logical.upper()

 if EXEC_SQL_PATTERN.match(logical):
 if FINAL_CLEANUP_TOKEN_END_EXEC in upper:
 output.append(line)
 continue
 in_exec_sql = True
 output.append(line)
 continue

 if in_exec_sql:
 output.append(line)
 if END_EXEC_PATTERN.match(logical):
 in_exec_sql = False
 continue

 move_match = MOVE_TO_DCL_DOT_REFERENCE_PATTERN.match(logical)
 if move_match:
 source = str(move_match.group("source")).strip()
 field = move_match.group("field").upper()
 group = move_match.group("group").upper()
 output.append(
 self._replace_body(
 line, FINAL_CLEANUP_MOVE_TEMPLATE.format(source=source)
)
)
 output.append(
 self._replace_body(
 line,
 FINAL_CLEANUP_TO_FIELD_OF_GROUP_TEMPLATE.format(
 field=field, group=group
),
)
)
 continue

 to_match = TO_DCL_DOT_REFERENCE_PATTERN.match(logical)
 if to_match:
 field = to_match.group("field").upper()
```

```

 group = to_match.group("group").upper()
 output.append(
 self._replace_body(
 line,
 FINAL_CLEANUP_TO_FIELD_OF_GROUP_TEMPLATE.format(
 field=field, group=group
),
)
)
 continue

 move_rewrite = self._rewrite_move_to_dcl_dot_as_lines(original_line=line)
 if move_rewrite:
 output.extend(move_rewrite)
 continue

 body = self._body(line)
 rewritten_body = NON_SQL_DCL_DOT_REFERENCE_STRICT_PATTERN.sub(
 lambda match: FINAL_CLEANUP_FIELD_OF_GROUP_TEMPLATE.format(
 field=match.group("field").upper(),
 group=match.group("group").upper(),
),
 body,
)
 if rewritten_body != body:
 output.append(self._replace_body(line, rewritten_body))
 continue

 output.append(line)

return output

def _rewrite_move_to_dcl_dot_as_lines(self, original_line: str) -> list[str]:
 body = self._body(original_line).rstrip()
 stripped = body.strip()
 upper = stripped.upper()

 if not upper.startswith(FINAL_CLEANUP_TOKEN_MOVE):
 return []
 if FINAL_CLEANUP_TOKEN_TO not in upper:
 return []

 before_to, after_to = self._split_last_to(stripped)
 if not before_to or not after_to or "." not in after_to:
 return []

 group, field = after_to.split(".", 1)
 group = group.strip().upper()
 field = field.strip().upper()

 trailing_period = ""
 if field.endswith("."):
 trailing_period = "."
 field = field[:-1].strip()

 if not group.startswith(FINAL_CLEANUP_DCL_PREFIX) or not field:
 return []

 leading = body[: len(body) - len(body.lstrip(" "))]
 if not leading:
 leading = FINAL_CLEANUP_AREA_B

 first_body = f"{leading}{before_to}"
 second_body = f"{leading}TO {field} OF {group}{trailing_period}"
 one_line = f"{leading}{before_to} TO {field} OF {group}{trailing_period}"

 if len(one_line) <= FINAL_CLEANUP_BODY_WIDTH:
 return [self._replace_body(original_line, one_line)]

 return [
 self._replace_body(original_line, first_body),
 self._replace_body(original_line, second_body),
]

def _split_last_to(self, text: str) -> tuple[str, str]:
 upper = str(text or "").upper()

```

```

marker = FINAL_CLEANUP_TOKEN_TO
index = upper.rfind(marker)
if index < 0:
 return "", ""
return text[:index].strip(), text[index + len(marker):].strip()

```

```

=====
FILE: src\idms_db2_phase2\postprocess\final_cleanup\final_cleanup_line_utils.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from patterns.update_final_cleanup_patterns import PARAGRAPH_HEADER_PATTERN
from rules.update_cobol_final_cleanup_rules import (
 FINAL_CLEANUP_BODY_END,
 FINAL_CLEANUP_BODY_START,
 FINAL_CLEANUP_BODY_WIDTH,
 FINAL_CLEANUP_COMMENT_CHARS,
 FINAL_CLEANUP_COMMENT_INDICATORS,
 FINAL_CLEANUP_DEBUG_PREFIX,
 FINAL_CLEANUP_FULL_WIDTH,
 FINAL_CLEANUP_INDICATOR_COLUMN,
 FINAL_CLEANUP_RIGHT_SEQUENCE_WIDTH,
 FINAL_CLEANUP_SEQUENCE_WIDTH,
 FINAL_CLEANUP_TOTAL_WIDTH,
)

```

```

class FinalCleanupLineUtils:
 """Fixed-format line helpers + paragraph range detection.

```

```

 Owns no business rules. Column geometry comes from
 rules/update_cobol_final_cleanup_rules.py.
 """

```

```

def _replace_body(self, line: str, body: str) -> str:
 text = str(line or "").rstrip("\n")
 clean_body = str(body or "")[:FINAL_CLEANUP_BODY_WIDTH].ljust(
 FINAL_CLEANUP_BODY_WIDTH
)

```

```

 if (
 len(text) >= FINAL_CLEANUP_FULL_WIDTH
 and text[:FINAL_CLEANUP_SEQUENCE_WIDTH].strip().isdigit()
):
 left = text[:FINAL_CLEANUP_SEQUENCE_WIDTH]
 right = (
 text[FINAL_CLEANUP_BODY_END:FINAL_CLEANUP_TOTAL_WIDTH]
 if len(text) >= FINAL_CLEANUP_TOTAL_WIDTH
 else ""
)
 return f"{left} {clean_body} {right}"

```

```

 if (
 len(text) > FINAL_CLEANUP_SEQUENCE_WIDTH
 and text[:FINAL_CLEANUP_SEQUENCE_WIDTH].strip().isdigit()
):
 left = text[:FINAL_CLEANUP_SEQUENCE_WIDTH]
 return f"{left} {clean_body.rstrip()}"

```

```

 return clean_body.rstrip()

```

```

def _body(self, line: str) -> str:
 text = str(line or "").rstrip("\n")

```

```

 if (
 len(text) >= FINAL_CLEANUP_FULL_WIDTH
 and text[:FINAL_CLEANUP_SEQUENCE_WIDTH].strip().isdigit()
):
 return text[FINAL_CLEANUP_BODY_START:FINAL_CLEANUP_BODY_END].rstrip()

```

```

 if (
 len(text) > FINAL_CLEANUP_SEQUENCE_WIDTH
 and text[:FINAL_CLEANUP_SEQUENCE_WIDTH].strip().isdigit()

```

```

):
 body = (
 text[FINAL_CLEANUP_BODY_START:]
 if len(text) > FINAL_CLEANUP_BODY_START
 else ""
)
 if (
 len(body) >= FINAL_CLEANUP_RIGHT_SEQUENCE_WIDTH
 and body[-FINAL_CLEANUP_RIGHT_SEQUENCE_WIDTH:].strip().isdigit()
):
 body = body[:-FINAL_CLEANUP_RIGHT_SEQUENCE_WIDTH]
 return body.rstrip()

return text.rstrip()

def _logical(self, line: str) -> str:
 return self._body(line).strip()

def _is_comment_or_debug(self, line: str) -> bool:
 text = str(line or "").rstrip("\n")
 if (
 len(text) >= FINAL_CLEANUP_INDICATOR_COLUMN
 and text[:FINAL_CLEANUP_SEQUENCE_WIDTH].strip().isdigit()
):
 return (
 text[FINAL_CLEANUP_SEQUENCE_WIDTH:FINAL_CLEANUP_INDICATOR_COLUMN]
 in FINAL_CLEANUP_COMMENT_INDICATORS
)

 stripped = text.strip()
 return (
 any(stripped.startswith(c) for c in FINAL_CLEANUP_COMMENT_CHARS)
 or stripped.upper().startswith(FINAL_CLEANUP_DEBUG_PREFIX)
)

def _paragraph_ranges(self, lines: list[str]) -> list[tuple[int, int]]:
 output: list[tuple[int, int]] = []
 current_start = -1

 for index, line in enumerate(lines):
 if not PARAGRAPH_HEADER_PATTERN.match(self._logical(line)):
 continue
 if current_start >= 0:
 output.append((current_start, index))
 current_start = index

 if current_start >= 0:
 output.append((current_start, len(lines)))

 return output

def _paragraph_end_index(self, lines: list[str], start: int) -> int:
 index = start + 1
 while index < len(lines):
 logical = self._logical(lines[index])
 if index > start + 1 and PARAGRAPH_HEADER_PATTERN.match(logical):
 return index
 index += 1
 return len(lines)

```

```

=====
FILE: src\idms_db2_phase2\postprocess\final_cleanup\final_cleanup_normalizer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from patterns.update_final_cleanup_patterns import (
 END_EVALUATE_PATTERN,
 END_EXEC_PATTERN,
 EVALUATE_SQLCODE_PATTERN,
 EXEC_SQL_PATTERN,
 PARAGRAPH_HEADER_PATTERN,
 PERFORM_PATTERN,
 PROCESS_COMMIT_IF_PATTERN,

```

```

 STOP_RUN_PATTERN,
 UPDATE_PARAGRAPH_PATTERN,
 WHEN_PATTERN,
)
from rules.update_cobol_final_cleanup_rules import (
 FINAL_CLEANUP_ACTION_INDENT,
 FINAL_CLEANUP_AREA_B,
 FINAL_CLEANUP_DOT,
 FINAL_CLEANUP_SQL_BODY,
 FINAL_CLEANUP_TOKEN_END_EVALUATE,
 FINAL_CLEANUP_TOKEN_END_EXEC,
 FINAL_CLEANUP_TOKEN_END_IF,
 FINAL_CLEANUP_TOKEN_END_IF_DOT,
 FINAL_CLEANUP_TOKEN_EVALUATE_SQLCODE,
 FINAL_CLEANUP_TOKEN_EXEC_SQL,
 FINAL_CLEANUP_TOKEN_PROCEDURE_DIVISION,
 FINAL_CLEANUP_TOKEN_STOP_RUN,
 FINAL_CLEANUP_WHEN_INDENT,
)

class FinalCleanupNormalizer:
 """Normalizes 1100-UPDATE-* paragraphs and PROCEDURE simple blocks."""

 def _normalize_update_paragraphs(self, lines: list[str]) -> list[str]:
 output = list(lines)
 index = 0
 while index < len(output):
 if not UPDATE_PARAGRAPH_PATTERN.match(self._logical(output[index])):
 index += 1
 continue
 end = self._paragraph_end_index(output, index)
 normalized = self._normalize_update_paragraph(output[index:end])
 output = output[:index] + normalized + output[end:]
 index += len(normalized)
 return output

 def _normalize_update_paragraph(self, lines: list[str]) -> list[str]:
 output: list[str] = []
 in_exec_sql = False
 in_evaluate = False

 for index, line in enumerate(lines):
 logical = self._logical(line)

 if index == 0 or not logical or self._is_comment_or_debug(line):
 output.append(line)
 continue

 if EXEC_SQL_PATTERN.match(logical):
 in_exec_sql = True
 output.append(
 self._replace_body(line, FINAL_CLEANUP_AREA_B + FINAL_CLEANUP_TOKEN_EXEC_SQL)
)
 continue

 if in_exec_sql and END_EXEC_PATTERN.match(logical):
 in_exec_sql = False
 output.append(
 self._replace_body(line, FINAL_CLEANUP_AREA_B + FINAL_CLEANUP_TOKEN_END_EXEC)
)
 continue

 if in_exec_sql:
 output.append(
 self._replace_body(line, FINAL_CLEANUP_SQL_BODY + logical.rstrip("."))
)
 continue

 if EVALUATE_SQLCODE_PATTERN.match(logical):
 in_evaluate = True
 output.append(
 self._replace_body(
 line, FINAL_CLEANUP_AREA_B + FINAL_CLEANUP_TOKEN_EVALUATE_SQLCODE
)
)
 continue

```

```

 continue

 if in_evaluate and WHEN_PATTERN.match(logical):
 output.append(
 self._replace_body(line, FINAL_CLEANUP_WHEN_INDENT + logical.rstrip("."))
)
 continue

 if in_evaluate and END_EVALUATE_PATTERN.match(logical):
 in_evaluate = False
 output.append(
 self._replace_body(
 line, FINAL_CLEANUP_AREA_B + FINAL_CLEANUP_TOKEN_END_EVALUATE
)
)
 continue

 if in_evaluate:
 output.append(
 self._replace_body(line, FINAL_CLEANUP_ACTION_INDENT + logical.rstrip("."))
)
 continue

 if logical == FINAL_CLEANUP_DOT:
 output.append(self._replace_body(line, FINAL_CLEANUP_AREA_B + "."))
 continue

 output.append(
 self._replace_body(line, FINAL_CLEANUP_AREA_B + logical.rstrip("."))
)

return output

def _normalize_procedure_simple_blocks(self, lines: list[str]) -> list[str]:
 output: list[str] = []
 in_procedure = False
 inside_simple_if = False

 for line in lines:
 logical = self._logical(line)
 upper = logical.upper().rstrip(".")

 if upper.startswith(FINAL_CLEANUP_TOKEN_PROCEDURE_DIVISION):
 in_procedure = True
 output.append(line)
 continue

 if not in_procedure or self._is_comment_or_debug(line):
 output.append(line)
 continue

 if PARAGRAPH_HEADER_PATTERN.match(logical):
 inside_simple_if = False
 output.append(line)
 continue

 if STOP_RUN_PATTERN.match(logical):
 output.append(
 self._replace_body(line, FINAL_CLEANUP_AREA_B + FINAL_CLEANUP_TOKEN_STOP_RUN)
)
 continue

 if PROCESS_COMMIT_IF_PATTERN.match(logical):
 inside_simple_if = True
 output.append(
 self._replace_body(line, FINAL_CLEANUP_AREA_B + logical.rstrip("."))
)
 continue

 if inside_simple_if and PERFORM_PATTERN.match(logical):
 output.append(
 self._replace_body(line, FINAL_CLEANUP_WHEN_INDENT + logical.rstrip("."))
)
 continue

 if inside_simple_if and upper.startswith(FINAL_CLEANUP_TOKEN_END_IF):

```



```

 inside_simple_if = False
 output.append(
 self._replace_body(line, FINAL_CLEANUP_AREA_B + FINAL_CLEANUP_TOKEN_END_IF_DOT)
)
 continue

 output.append(line)

return output

```

```

=====
FILE: src\idms_db2_phase2\postprocess\main_flow__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\postprocess\main_flow\main_flow_line_utils.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from rules.update_restart_rules import (
 MAIN_FLOW_AREA_B_INDENT,
 MAIN_FLOW_BODY_END,
 MAIN_FLOW_BODY_START,
 MAIN_FLOW_COMMENT_INDICATORS,
 MAIN_FLOW_FIXED_BODY_WIDTH,
 MAIN_FLOW_FULL_WIDTH,
 MAIN_FLOW_RIGHT_SEQUENCE_WIDTH,
 MAIN_FLOW_SEQUENCE_WIDTH,
 MAIN_FLOW_TOTAL_WIDTH,
)

class MainFlowLineUtils:
 """Fixed-format line rendering + logical extraction.

 Owns no business rules. Column geometry comes from
 rules/update_restart_rules.py.
 """

 def _line_like(self, *, reference_line: str, body: str) -> str:
 text = str(reference_line or "").rstrip("\n")
 clean_body = str(body or "").strip()

 if (
 len(text) >= MAIN_FLOW_FULL_WIDTH
 and text[:MAIN_FLOW_SEQUENCE_WIDTH].strip().isdigit()
):
 left = text[:MAIN_FLOW_SEQUENCE_WIDTH]
 indicator = text[MAIN_FLOW_SEQUENCE_WIDTH] if len(text) > MAIN_FLOW_SEQUENCE_WIDTH else " "
 right = (
 text[MAIN_FLOW_BODY_END:MAIN_FLOW_TOTAL_WIDTH]
 if len(text) >= MAIN_FLOW_TOTAL_WIDTH
 else ""
)
 body_area = f"{MAIN_FLOW_AREA_B_INDENT}{clean_body}"
 return (
 f"{left}{indicator}"
 f"{body_area[:MAIN_FLOW_FIXED_BODY_WIDTH].ljust(MAIN_FLOW_FIXED_BODY_WIDTH)}"
 f"{right}"
)

 if (
 len(text) > MAIN_FLOW_SEQUENCE_WIDTH
 and text[:MAIN_FLOW_SEQUENCE_WIDTH].strip().isdigit()
):
 left = text[:MAIN_FLOW_SEQUENCE_WIDTH]
 indicator = text[MAIN_FLOW_SEQUENCE_WIDTH] if len(text) > MAIN_FLOW_SEQUENCE_WIDTH else " "
 body_area = f"{MAIN_FLOW_AREA_B_INDENT}{clean_body}"
 return f"{left}{indicator}{body_area[:MAIN_FLOW_FIXED_BODY_WIDTH].rstrip()}"

```

```

 return f"{MAIN_FLOW_AREA_B_INDENT}{clean_body}"

def _logical(self, line: str) -> str:
 text = str(line or "").rstrip("\n")

 if (
 len(text) >= MAIN_FLOW_FULL_WIDTH
 and text[:MAIN_FLOW_SEQUENCE_WIDTH].strip().isdigit()
):
 return text[MAIN_FLOW_BODY_START:MAIN_FLOW_BODY_END].strip()

 if (
 len(text) > MAIN_FLOW_SEQUENCE_WIDTH
 and text[:MAIN_FLOW_SEQUENCE_WIDTH].strip().isdigit()
):
 body = text[MAIN_FLOW_SEQUENCE_WIDTH:]
 if body[:1] in MAIN_FLOW_COMMENT_INDICATORS:
 return body.strip()
 if (
 len(body) >= MAIN_FLOW_RIGHT_SEQUENCE_WIDTH
 and body[-MAIN_FLOW_RIGHT_SEQUENCE_WIDTH:].strip().isdigit()
):
 body = body[:-MAIN_FLOW_RIGHT_SEQUENCE_WIDTH]
 return body.strip()

 return text.strip()

```

```

=====
FILE: src\idms_db2_phase2\postprocess\main_flow\main_flow_locators.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from patterns.update_main_flow_patterns import (
 OPEN_INPUT_LINE_PATTERN,
 STOP_RUN_LINE_PATTERN,
)
from rules.update_restart_rules import (
 MAIN_FLOW_LEGACY_RESTART_TOKENS,
 MAIN_FLOW_LONE_PERIOD,
 MAIN_FLOW_TOKEN_CLOSE,
)

class MainFlowLocators:
 """Index/region finders and insertion planning for the main flow."""

 def _find_open_input_index(self, *, lines, file_name):
 target = str(file_name or "").strip().upper()
 if not target:
 return -1

 for index, line in enumerate(lines):
 match = OPEN_INPUT_LINE_PATTERN.search(self._logical(line).upper())
 if not match:
 continue
 if str(match.group("file") or "").strip().upper() == target:
 return index
 return -1

 def _find_close_input_index(self, *, lines, file_name):
 target = str(file_name or "").strip().upper()
 if not target:
 return -1

 for index, line in enumerate(lines):
 logical = self._logical(line).upper()
 if not logical.startswith(MAIN_FLOW_TOKEN_CLOSE):
 continue
 if target in logical:
 return index
 return -1

```

```

def _find_stop_run_index(self, *, lines, start_index):
 for index in range(start_index, len(lines)):
 if STOP_RUN_LINE_PATTERN.search(self._logical(lines[index]).upper()):
 return index
 return -1

def _has_legacy_restart_between(self, *, lines, start_index, end_index):
 for index in range(start_index, end_index):
 upper = self._logical(lines[index]).upper()
 if any(token in upper for token in MAIN_FLOW_LEGACY_RESTART_TOKENS):
 return True
 return False

def _read_next_insert_plan(self, *, lines, start, end):
 for index in range(end - 1, start, -1):
 logical = self._logical(lines[index])
 if not logical:
 continue
 if logical == MAIN_FLOW_LONE_PERIOD:
 return index, True
 if logical.endswith("."):
 return index + 1, False
 return end, False

```

```

=====
FILE: src\idms_db2_phase2\postprocess\main_flow\main_flow_paragraph_resolver.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from patterns.update_main_flow_patterns import (
 DIVISION_LINE_PATTERN,
 EXEC_SQL_LINE_PATTERN,
 MOVE_LINE_PATTERN,
 PARAGRAPH_HEADER_LINE_PATTERN,
 PERFORM_LINE_PATTERN,
 PERFORM_READ_FLAT_FILE_LINE_PATTERN,
 PERFORM_UNTIL_LINE_PATTERN,
 PROCEDURE_DIVISION_LINE_PATTERN,
 SECTION_LINE_PATTERN,
)
from rules.update_restart_rules import (
 MAIN_FLOW_READ_FLAT_FILE_PARAGRAPH,
 MAIN_FLOW_RESTART_PARAGRAPH_PREFIXES,
)

class MainFlowParagraphResolver:
 """Resolves and classifies the business processing paragraph."""

 def _resolve_process_paragraph_from_main_flow(
 self, *, lines, start_index, end_index
):
 for index in range(start_index, end_index):
 logical = self._logical(lines[index]).upper().rstrip(".")
 match = PERFORM_UNTIL_LINE_PATTERN.search(logical)
 if not match:
 continue

 paragraph = str(match.group("paragraph") or "").strip().upper()
 if not paragraph or paragraph == MAIN_FLOW_READ_FLAT_FILE_PARAGRAPH:
 continue
 if self._is_restart_or_utility_paragraph(paragraph):
 continue
 return paragraph

 return ""

 def _resolve_first_business_paragraph(self, *, lines):
 inside_procedure = False

 for index, line in enumerate(lines):
 logical = self._logical(line).strip()
 upper = logical.upper().rstrip(".")

```

```

 if PROCEDURE_DIVISION_LINE_PATTERN.match(logical):
 inside_procedure = True
 continue
 if not inside_procedure:
 continue
 if DIVISION_LINE_PATTERN.match(logical):
 break
 if not self._is_paragraph_header(logical):
 continue

 paragraph_name = upper
 if self._is_restart_or_utility_paragraph(paragraph_name):
 continue
 if self._paragraph_has_business_sql_or_moves(lines=lines, start_index=index):
 return paragraph_name

 return ""

def _paragraph_has_business_sql_or_moves(self, *, lines, start_index):
 for index in range(start_index + 1, len(lines)):
 logical = self._logical(lines[index]).strip()
 upper = logical.upper()

 if index > start_index + 1 and self._is_paragraph_header(logical):
 return False
 if DIVISION_LINE_PATTERN.match(logical):
 return False
 if EXEC_SQL_LINE_PATTERN.search(upper):
 return True
 if MOVE_LINE_PATTERN.match(upper):
 return True
 if PERFORM_LINE_PATTERN.match(upper):
 return True

 return False

def _paragraph_contains_perform_read_flat_file(self, paragraph_lines):
 for line in paragraph_lines:
 logical = self._logical(line).upper().rstrip(".")
 if PERFORM_READ_FLAT_FILE_LINE_PATTERN.match(logical):
 return True
 return False

def _is_paragraph_header(self, logical):
 text = str(logical or "").strip()
 if not PARAGRAPH_HEADER_LINE_PATTERN.match(text):
 return False

 upper = text.upper().rstrip(".")
 if SECTION_LINE_PATTERN.match(text):
 return False
 if DIVISION_LINE_PATTERN.match(text):
 return False
 if self._is_restart_or_utility_paragraph(upper):
 return False
 return True

def _is_restart_or_utility_paragraph(self, paragraph_name):
 name = str(paragraph_name or "").strip().upper().rstrip(".")
 if not name:
 return True
 if name == MAIN_FLOW_READ_FLAT_FILE_PARAGRAPH:
 return True
 return any(
 name.startswith(prefix)
 for prefix in MAIN_FLOW_RESTART_PARAGRAPH_PREFIXES
)

```

```

=====
FILE: src\idms_db2_phase2\postprocess\restart_role__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\postprocess\restart_role\restart_role_factory.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from dataclasses import fields, is_dataclass

from idms_db2_phase2.postprocess.update_metadata_models import DclgenRoleInfo
```

```
class RestartRoleFactory:
 """Constructs DclgenRoleInfo while staying compatible with the model.

 Accepts the model's declared fields as kwargs and attaches any extra
 alias attributes after construction (best-effort).
 """

 def _make_role(self, **values: str) -> DclgenRoleInfo:
 if is_dataclass(DclgenRoleInfo):
 accepted = {field.name for field in fields(DclgenRoleInfo)}
 kwargs = {
 key: value for key, value in values.items() if key in accepted
 }
 role = DclgenRoleInfo(**kwargs)
 else:
 role = DclgenRoleInfo()

 for key, value in values.items():
 if not hasattr(role, key):
 try:
 setattr(role, key, value)
 except Exception:
 pass

 return role
```

```
=====
FILE: src\idms_db2_phase2\postprocess\restart_role\restart_role_field_resolver.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from rules.update_restart_rules import (
 RESTART_DCLGEN_ROLE_TOKENS,
 RESTART_PAYLOAD_FALLBACK_TOKEN_KEY,
 RESTART_PAYLOAD_TOKEN_KEY,
 RESTART_VARCHAR_SUFFIXES,
)

class RestartRoleFieldResolver:
 """Resolves restart columns, fields, and payload child fields.

 Uses the token-driven matcher; owns no hardcoded restart names.
 Depends on the host class for ``self.matcher`` and ``self.row_mapper``.
 """

 def _payload_exclude_tokens(self) -> list[str]:
 return [
 RESTART_VARCHAR_SUFFIXES["length"],
 RESTART_VARCHAR_SUFFIXES["text"],
]

 def _resolve_payload_group(self, rows: list[dict[str, str]]) -> str:
 exclude = self._payload_exclude_tokens()

 payload_group = self.matcher.find_optional_field(
 rows=rows,
 tokens=RESTART_DCLGEN_ROLE_TOKENS[RESTART_PAYLOAD_TOKEN_KEY],
 fallback_tokens=RESTART_DCLGEN_ROLE_TOKENS[
```

```

 RESTART_PAYLOAD_FALLBACK_TOKEN_KEY
],
 exclude_tokens=exclude,
)
if payload_group:
 return payload_group

payload_column = self.matcher.find_optional_col(
 rows=rows,
 tokens=RESTART_DCLGEN_ROLE_TOKENS[RESTART_PAYLOAD_TOKEN_KEY],
 fallback_tokens=RESTART_DCLGEN_ROLE_TOKENS[
 RESTART_PAYLOAD_FALLBACK_TOKEN_KEY
],
 exclude_tokens=exclude,
)
if payload_column and hasattr(self.row_mapper, "column_to_cobol_field"):
 return self.row_mapper.column_to_cobol_field(payload_column)
if payload_column:
 return payload_column.replace("_", "-")

return self.matcher.find_field(
 rows=rows,
 tokens=RESTART_DCLGEN_ROLE_TOKENS[RESTART_PAYLOAD_TOKEN_KEY],
 fallback_tokens=RESTART_DCLGEN_ROLE_TOKENS[
 RESTART_PAYLOAD_FALLBACK_TOKEN_KEY
],
 exclude_tokens=exclude,
)

def _resolve_payload_child_field(
 self,
 *,
 rows: list[dict[str, str]],
 payload_group: str,
 token_key: str,
 fallback_token_key: str,
 suffix_key: str,
) -> str:
 suffix = RESTART_VARCHAR_SUFFIXES[suffix_key]

 explicit_child = self.matcher.find_optional_field(
 rows=rows,
 tokens=RESTART_DCLGEN_ROLE_TOKENS[token_key],
 fallback_tokens=RESTART_DCLGEN_ROLE_TOKENS[fallback_token_key],
)

 if explicit_child:
 if explicit_child == payload_group:
 return self._derive_varchar_field(
 payload_group=payload_group, suffix_key=suffix_key
)
 if explicit_child.endswith(suffix):
 return explicit_child
 if payload_group and explicit_child.startswith(payload_group):
 return explicit_child

 return self._derive_varchar_field(
 payload_group=payload_group, suffix_key=suffix_key
)

def _derive_varchar_field(self, *, payload_group: str, suffix_key: str) -> str:
 if not payload_group:
 return ""
 suffix = RESTART_VARCHAR_SUFFIXES[suffix_key]
 if payload_group.endswith(suffix):
 return payload_group
 return f"{payload_group}{suffix}"

def _find_column(
 self, *, rows: list[dict[str, str]], token_key: str, fallback_token_key: str
) -> str:
 return self.matcher.find_col(
 rows=rows,
 tokens=RESTART_DCLGEN_ROLE_TOKENS[token_key],
 fallback_tokens=RESTART_DCLGEN_ROLE_TOKENS[fallback_token_key],
)

```

```
def _find_field(
 self, *, rows: list[dict[str, str]], token_key: str, fallback_token_key: str
) -> str:
 return self.matcher.find_field(
 rows=rows,
 tokens=RESTART_DCLGEN_ROLE_TOKENS[token_key],
 fallback_tokens=RESTART_DCLGEN_ROLE_TOKENS[fallback_token_key],
)
```

```
=====
FILE: src\idms_db2_phase2\postprocess\storage_include__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\postprocess\storage_include\storage_include_injector.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/postprocess/storage_include/storage_include_injector.py
ACTION: REPLACE ENTIRE FILE
```

"""DCLGEN + copybook include injection and reference scanning.

CORRECTION 1 - duplicate DCLGEN include

-----  
\_existing\_exec\_sql\_includes scanned the RAW program text with a pattern  
that expects the single-line form:

```
EXEC SQL INCLUDE DZBFARTV END-EXEC.
```

Two things defeated it.

First, InfrastructureBlockBuilder emits the three-line block form, and  
after fixed-format sequencing the SEQUENCE NUMBERS sit between the  
tokens:

001080	EXEC SQL	01080000
001090	INCLUDE DZBFARTV	01090000
001100	END-EXEC.	01100000

The pattern's `\\s+` cannot span `'01080000'` and `'001090'`, so the match  
failed and DZBFARTV was reported absent.

Second, even on clean text the pattern only ever recognised one of the  
two shapes this codebase emits.

The injector therefore added DZBFARTV a second time. A duplicated DCLGEN  
include duplicates every data-name in the group, which the compiler  
rejects. CHK-08.02 reported exactly this.

Detection now runs over LOGICAL lines through IncludeRenderer, which  
recognises both shapes.

CORRECTION 2 - injected includes sat in Area A

-----  
Injected includes came from `self.generator.dclgen_include*`, which  
rendered them with a single leading space:

```
EXEC SQL INCLUDE DZ01RSTV END-EXEC.
```

CHK-06.07 requires at least 4 body spaces, i.e. column 12. Emission now  
goes through IncludeRenderer too, so shape and indent are decided in one  
place for every emitter in the project.

Copybook COPY statements are untouched: they are not EXEC SQL includes  
and have their own placement rules.

"""

```
from __future__ import annotations
```

```

from idms_db2_phase2.generators.db2_infrastructure.include_renderer import (
 IncludeRenderer,
)
from patterns.update_storage_include_patterns import (
 DCL_HOST_REFERENCE_PATTERN,
)
from rules.update_restart_rules import UPDATE_RESTART_DIAGNOSTICS
from rules.update_storage_include_rules import (
 COPY_EXISTS_BY_RECORD_TEMPLATE,
 COPY_EXISTS_TEMPLATE,
 COPY_INJECTED_TEMPLATE,
 DCL_GROUP_PREFIX,
 DCLGEN_INCLUDE_EXISTS_TEMPLATE,
 DCLGEN_INCLUDE_INJECTED_TEMPLATE,
 EXCLUDED_INCLUDE_NAMES,
 REFERENCED_DCLGEN_INJECTED_TEMPLATE,
 RESTART_DCLGEN_UNAVAILABLE,
 SKIP_DATA_DIVISION_COPYBOOK_TEMPLATE,
 SKIP_DATA_DIVISION_DCLGEN_TEMPLATE,
 SKIP_DATA_DIVISION_REFERENCED,
)

```

```

class StorageIncludeInjector:
 """DCLGEN + copybook include injection and reference scanning."""

 # =====
 # Shared include renderer
 # =====
 @property
 def includes(self) -> IncludeRenderer:
 """Renderer and scanner shared with every other include emitter.

 Built lazily because this class is a mixin: the host supplies
 self.line_utils and self.generator, and there is no constructor
 of our own to hook.
 """
 renderer = getattr(self, "_include_renderer", None)
 if renderer is None:
 renderer = IncludeRenderer(self.line_utils)
 self._include_renderer = renderer
 return renderer

 # =====
 # Restart DCLGEN include
 # =====
 def ensure_dclgen_include(self, cobol_text, context, diagnostics):
 role = context.restart_dclgen
 if not role:
 diagnostics.append(RESTART_DCLGEN_UNAVAILABLE)
 return cobol_text

 include_name = self.includes.normalize(role.include_name)
 if not include_name:
 diagnostics.append(RESTART_DCLGEN_UNAVAILABLE)
 return cobol_text

 lines = self.line_utils.lines(cobol_text)

 if include_name in self.includes.existing(lines):
 diagnostics.append(
 DCLGEN_INCLUDE_EXISTS_TEMPLATE.format(include=include_name)
)
 return cobol_text

 insert_index = self.line_utils.data_division_insert_line_index(lines)
 if insert_index < 0:
 diagnostics.append(
 SKIP_DATA_DIVISION_DCLGEN_TEMPLATE.format(include=include_name)
)
 return cobol_text

 block = self.includes.render(include_name)
 updated_lines = (
 lines[:insert_index] + block + [""] + lines[insert_index:]
)
)

```



```

 diagnostics.append(
 DCLGEN_INCLUDE_INJECTED_TEMPLATE.format(include=include_name)
)
 return self.line_utils.join(updated_lines)

=====
Referenced DCLGEN includes
=====
def ensure_referenced_dclgen_includes(self, cobol_text, diagnostics):
 lines = self.line_utils.lines(cobol_text)
 referenced = self._referenced_dclgen_includes(cobol_text)

 # Shape-agnostic and sequence-number safe. An include already
 # emitted as a three-line block counts as present and is never
 # injected a second time.
 missing = [
 include_name
 for include_name in self.includes.missing(lines, referenced)
 if include_name not in EXCLUDED_INCLUDE_NAMES
]

 if not missing:
 return cobol_text

 insert_index = self.line_utils.data_division_insert_line_index(lines)
 if insert_index < 0:
 diagnostics.append(SKIP_DATA_DIVISION_REFERENCED)
 return cobol_text

 include_lines: list[str] = []
 for include_name in missing:
 include_lines.extend(self.includes.render(include_name))
 diagnostics.append(
 REFERENCED_DCLGEN_INJECTED_TEMPLATE.format(
 base=UPDATE_RESTART_DIAGNOSTICS[
 "referenced_dclgen_injected"
],
 include=include_name,
)
)

 if not include_lines:
 return cobol_text

 updated_lines = (
 lines[:insert_index] + include_lines + [""] + lines[insert_index:]
)
 return self.line_utils.join(updated_lines)

=====
Copybook COPY member
=====
def ensure_copybook_include(self, cobol_text, context, diagnostics):
 """A COPY member, not an EXEC SQL include.

 Left on its original path deliberately: COPY has different
 placement rules and is not part of the DB2 include family.
 """
 include_name = context.copybook_include_name
 record_name = context.input_record_name

 if self.line_utils.contains_copy(cobol_text, include_name):
 diagnostics.append(
 COPY_EXISTS_TEMPLATE.format(
 base=UPDATE_RESTART_DIAGNOSTICS["copy_exists"],
 include=include_name,
)
)
 return cobol_text

 if record_name != include_name and self.line_utils.contains_copy(
 cobol_text, record_name
):
 diagnostics.append(
 COPY_EXISTS_BY_RECORD_TEMPLATE.format(record=record_name)
)

```

```

 return cobol_text

 lines = self.line_utils.lines(cobol_text)
 insert_index = self.line_utils.data_division_insert_line_index(lines)
 if insert_index < 0:
 diagnostics.append(
 SKIP_DATA_DIVISION_COPYBOOK_TEMPLATE.format(
 include=include_name
)
)
 return cobol_text

 block = self.generator.copybook_include(context).splitlines()
 updated_lines = (
 lines[:insert_index] + block + [""] + lines[insert_index:]
)
 diagnostics.append(
 COPY_INJECTED_TEMPLATE.format(
 base=UPDATE_RESTART_DIAGNOSTICS["copy_injected"],
 include=include_name,
)
)
 return self.line_utils.join(updated_lines)

```

```

=====
Scanning
=====

```

```

def _existing_exec_sql_includes(self, cobol_text) -> set[str]:
 """Every include already present, WHICHEVER shape carries it.

 Retained for callers outside this class. The work is delegated to
 IncludeRenderer, which reads LOGICAL lines and so is immune both
 to the block form and to interleaved sequence numbers.
 """
 return self.includes.existing(self.line_utils.lines(cobol_text))

```

```

def _referenced_dclgen_includes(self, cobol_text) -> list[str]:
 """Include names implied by DCLGEN host group references.

 DCLDZBFARTV -> DZBFARTV. Order preserved, de-duplicated,
 infrastructure includes excluded.
 """
 output: list[str] = []
 seen: set[str] = set()

 for match in DCL_HOST_REFERENCE_PATTERN.finditer(cobol_text):
 group_name = str(match.group("group") or "").strip().upper()

 if not group_name.startswith(DCL_GROUP_PREFIX):
 continue

 include_name = group_name[len(DCL_GROUP_PREFIX):]

 if not include_name or include_name in EXCLUDED_INCLUDE_NAMES:
 continue
 if include_name in seen:
 continue

 seen.add(include_name)
 output.append(include_name)

 return output

```

```

=====
FILE: src\idms_db2_phase2\postprocess\storage_include\storage_ws_injector.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.postprocess.dynamic_metadata_resolver import UpdateProgramContext
from patterns.update_restart_patterns import (
 MALFORMED_SQLERROR_END_EVALUATE_PATTERN,
)
from patterns.update_storage_include_patterns import (

```

```

 CS_PROGRAM_01_PATTERN,
 PROGRAM_NAME_MOVE_PATTERN,
)
from rules.update_restart_rules import (
 UPDATE_RESTART_DIAGNOSTICS,
 UPDATE_RESTART_WS_NAMES,
)
from rules.update_storage_include_rules import (
 CS_PROGRAM_77_TEMPLATE,
 DATE_WS_ANCHORS,
 PROGRAM_NAME_MOVE_REPLACEMENT_TEMPLATE,
 SKIP_DATA_DIVISION_DATE_WS,
 SKIP_DATA_DIVISION_WS,
 WS_ANCHOR_01_TEMPLATE,
)

class StorageWsInjector:
 """Working-storage, date-WS, program-name and SQLERROR passes.

 Depends on the host class for ``self.line_utils`` and ``self.generator``.
 """

 def fix_malformed_sqlerror(self, cobol_text, diagnostics):
 updated = MALFORMED_SQLERROR_END_EVALUATE_PATTERN.sub(
 "PERFORM SQLERROR\nEND-EVALUATE",
 cobol_text,
)
 if updated != cobol_text:
 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["malformed_sqlerror_fixed"])
 return updated

 def normalize_program_name(self, cobol_text, context, diagnostics):
 program_id = str(context.program_id or "").strip().upper()
 if not program_id:
 return cobol_text

 updated = PROGRAM_NAME_MOVE_PATTERN.sub(
 PROGRAM_NAME_MOVE_REPLACEMENT_TEMPLATE.format(program_id=program_id),
 cobol_text,
)
 updated = CS_PROGRAM_01_PATTERN.sub(
 lambda match: CS_PROGRAM_77_TEMPLATE.format(
 prefix=match.group("prefix"),
 program_id=program_id,
),
 updated,
)
 if updated != cobol_text:
 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["program_name_normalized"])
 return updated

 def ensure_working_storage(self, cobol_text, context, diagnostics):
 names = UPDATE_RESTART_WS_NAMES
 switch_anchor = WS_ANCHOR_01_TEMPLATE.format(group=names["switch_group"])
 help_anchor = WS_ANCHOR_01_TEMPLATE.format(group=names["help_group"])

 if self.line_utils.logical_contains(
 cobol_text, switch_anchor
) and self.line_utils.logical_contains(cobol_text, help_anchor):
 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["ws_exists"])
 return cobol_text

 lines = self.line_utils.lines(cobol_text)
 insert_index = self.line_utils.data_division_insert_line_index(lines)
 if insert_index < 0:
 diagnostics.append(SKIP_DATA_DIVISION_WS)
 return cobol_text

 block = self.generator.working_storage(context).splitlines()
 updated_lines = lines[:insert_index] + block + [""] + lines[insert_index:]
 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["ws_injected"])
 return self.line_utils.join(updated_lines)

 def ensure_date_working_storage(self, cobol_text, diagnostics):
 if any(

```

```

 self.line_utils.logical_contains(cobol_text, anchor)
 for anchor in DATE_WS_ANCHORS
):
 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["date_ws_exists"])
 return cobol_text

 lines = self.line_utils.lines(cobol_text)
 insert_index = self.line_utils.data_division_insert_line_index(lines)
 if insert_index < 0:
 diagnostics.append(SKIP_DATA_DIVISION_DATE_WS)
 return cobol_text

 block = self.generator.date_working_storage().splitlines()
 updated_lines = lines[:insert_index] + block + [""] + lines[insert_index:]
 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["date_ws_injected"])
 return self.line_utils.join(updated_lines)

```

```

=====
FILE: src\idms_db2_phase2\postprocess\update_business_sql_init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\postprocess\update_business_sql\update_business_sql_line_renderer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from rules.update_business_sql_rules import (
 ACTIVE_INDICATOR,
 AREA_B_INDENT,
 BODY_END_COLUMN,
 BODY_START_COLUMN,
 COMMENT_INDICATORS,
 FIXED_BODY_WIDTH,
 FULL_LINE_WIDTH,
 INDICATOR_COLUMN,
 RIGHT_SEQUENCE_WIDTH,
 SEQUENCE_AREA_WIDTH,
 TOTAL_LINE_WIDTH,
)

class UpdateBusinessSqlLineRenderer:
 """Fixed-format line rendering + logical-line extraction.

 Owns no regex and no business rules. Column geometry and indents come
 from rules/update_business_sql_rules.py.
 """

 def _active_line_like(self, *, reference_line: str, body: str) -> str:
 return self._line_like(
 reference_line=reference_line,
 body=body,
 force_indicator=ACTIVE_INDICATOR,
)

 def _line_like(
 self,
 *,
 reference_line: str,
 body: str,
 force_indicator: str | None = None,
) -> str:
 text = str(reference_line or "").rstrip("\n")
 clean_body = str(body or "").strip()

 if len(text) >= FULL_LINE_WIDTH and text[:SEQUENCE_AREA_WIDTH].strip().isdigit():
 left = text[:SEQUENCE_AREA_WIDTH]
 indicator = self._indicator_for(text, force_indicator)
 right = (

```

```

 text[BODY_END_COLUMN:TOTAL_LINE_WIDTH]
 if len(text) >= TOTAL_LINE_WIDTH
 else ""
)
 body_area = f"{AREA_B_INDENT}{clean_body}"
 return (
 f"{left}{indicator}"
 f"{body_area[:FIXED_BODY_WIDTH].ljust(FIXED_BODY_WIDTH)}"
 f"{right}"
)

 if len(text) > SEQUENCE_AREA_WIDTH and text[:SEQUENCE_AREA_WIDTH].strip().isdigit():
 left = text[:SEQUENCE_AREA_WIDTH]
 indicator = self._indicator_for(text, force_indicator)
 body_area = f"{AREA_B_INDENT}{clean_body}"
 return f"{left}{indicator}{body_area[:FIXED_BODY_WIDTH].rstrip()}"

 return f"{AREA_B_INDENT}{clean_body}"

def _indicator_for(self, text: str, force_indicator: str | None) -> str:
 if force_indicator is not None:
 return force_indicator
 return text[SEQUENCE_AREA_WIDTH] if len(text) > SEQUENCE_AREA_WIDTH else " "

def _is_comment_line(self, line: str) -> bool:
 text = str(line or "").rstrip("\n")

 if len(text) >= INDICATOR_COLUMN and text[:SEQUENCE_AREA_WIDTH].strip().isdigit():
 return text[SEQUENCE_AREA_WIDTH:INDICATOR_COLUMN] in COMMENT_INDICATORS

 stripped = text.strip()
 return any(stripped.startswith(ind) for ind in COMMENT_INDICATORS)

def _logical(self, line: str) -> str:
 text = str(line or "").rstrip("\n")

 if len(text) >= FULL_LINE_WIDTH and text[:SEQUENCE_AREA_WIDTH].strip().isdigit():
 return text[BODY_START_COLUMN:BODY_END_COLUMN].strip()

 if len(text) > SEQUENCE_AREA_WIDTH and text[:SEQUENCE_AREA_WIDTH].strip().isdigit():
 body = text[BODY_START_COLUMN:] if len(text) > BODY_START_COLUMN else ""
 if (
 len(body) >= RIGHT_SEQUENCE_WIDTH
 and body[-RIGHT_SEQUENCE_WIDTH:].strip().isdigit()
):
 body = body[:-RIGHT_SEQUENCE_WIDTH]
 return body.strip()

 return text.strip()

```

```

=====
FILE: src\idms_db2_phase2\postprocess\update_business_sql\update_paragraph_normalizer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from patterns.update_business_sql_patterns import (
 END_EVALUATE_PATTERN,
 END_EXEC_PATTERN,
 EVALUATE_SQLCODE_PATTERN,
 EXEC_SQL_START_PATTERN,
)
from rules.update_business_sql_rules import (
 NESTED_INDENT,
 SQL_BODY_INDENT,
 TOKEN_END_EVALUATE,
 TOKEN_END_EXEC,
 TOKEN_EVALUATE_SQLCODE,
 TOKEN_EXEC_SQL,
 TOKEN_WHEN,
 WHEN_INDENT,
)
from rules.update_restart_rules import UPDATE_SQL_PARAGRAPH_PREFIX

```

```

class UpdateParagraphNormalizer:
 """Normalizes generated update-paragraph indentation.

 Owns no column geometry; delegates line rendering to the host class
 (via _active_line_like / _logical / _is_comment_line).
 """

 def _normalize_all_update_paragraphs(self, lines: list[str]) -> list[str]:
 output = list(lines)
 index = 0

 while index < len(output):
 logical = self._logical(output[index]).upper().rstrip(".")

 if not logical.startswith(f"{UPDATE_SQL_PARAGRAPH_PREFIX}-"):
 index += 1
 continue

 paragraph_range = self.line_utils.find_paragraph_range(output, logical)
 if not paragraph_range:
 index += 1
 continue

 start, end = paragraph_range
 normalized = self._normalize_update_paragraph_lines(output[start:end])
 output = output[:start] + normalized + output[end:]
 index = start + len(normalized)

 return output

 def _normalize_update_paragraph_lines(
 self,
 paragraph_lines: list[str],
) -> list[str]:
 output: list[str] = []
 in_exec_sql = False
 in_evaluate = False

 for index, line in enumerate(paragraph_lines):
 logical = self._logical(line).strip()

 if index == 0 or not logical or self._is_comment_line(line):
 output.append(line)
 continue

 if EXEC_SQL_START_PATTERN.match(logical):
 in_exec_sql = True
 output.append(self._active_line_like(reference_line=line, body=TOKEN_EXEC_SQL))
 continue

 if in_exec_sql and END_EXEC_PATTERN.match(logical):
 in_exec_sql = False
 output.append(self._active_line_like(reference_line=line, body=TOKEN_END_EXEC))
 continue

 if in_exec_sql:
 output.append(
 self._active_line_like(
 reference_line=line,
 body=f"{SQL_BODY_INDENT}{logical.rstrip('.')}",
)
)
 continue

 if EVALUATE_SQLCODE_PATTERN.match(logical):
 in_evaluate = True
 output.append(
 self._active_line_like(reference_line=line, body=TOKEN_EVALUATE_SQLCODE)
)
 continue

 if in_evaluate and logical.upper().startswith(TOKEN_WHEN):
 output.append(
 self._active_line_like(
 reference_line=line,

```

```

 body=f"{WHEN_INDENT}{logical.rstrip('.')}",
)
)
 continue

 if in_evaluate and END_EVALUATE_PATTERN.match(logical):
 in_evaluate = False
 output.append(
 self._active_line_like(reference_line=line, body=TOKEN_END_EVALUATE)
)
 continue

 if in_evaluate:
 output.append(
 self._active_line_like(
 reference_line=line,
 body=f"{NESTED_INDENT}{logical.rstrip('.')}",
)
)
 continue

 output.append(self._active_line_like(reference_line=line, body=logical))

return output

```

```

=====
FILE: src\idms_db2_phase2\postprocess\update_business_sql\update_sql_block_extractor.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from patterns.update_business_sql_patterns import (
 CONTINUE_PATTERN,
 CONVERTED_MODIFY_COMMENT_PATTERN,
 DCL_HOST_GROUP_PATTERN,
 END_EVALUATE_PATTERN,
 END_EXEC_PATTERN,
 EVALUATE_SQLCODE_PATTERN,
 EXEC_SQL_START_PATTERN,
 EXEC_SQL_UPDATE_PATTERN,
 PARAGRAPH_HEADER_PATTERN,
 SQL_LOCATION_UPDATE_PATTERN,
 WHEN_ZERO_PATTERN,
)
from rules.update_business_sql_rules import (
 ADD_UPDATE_COUNTER_TEMPLATE,
 COMMIT_CHECK_IF_TEMPLATE,
 COMMIT_CHECK_PERFORM_TEMPLATE,
 END_IF_STATEMENT,
 FINAL_PERIOD_LINE,
 INITIALIZE_DCL_GROUP_TEMPLATE,
 NESTED_INDENT,
 PERFORM_TEMPLATE,
 SQL_ERROR_PARAGRAPH_NAMES,
 SQL_ERROR_ROUTINE_MARKER,
 TOKEN_INITIALIZE_DCL,
 TOKEN_STAR,
)
from rules.update_restart_rules import (
 RESTART_PARAGRAPH_NAMES,
 UPDATE_COMMIT_THRESHOLD,
 UPDATE_RESTART_DIAGNOSTICS,
 UPDATE_RESTART_WS_NAMES,
)

class UpdateSqlBlockExtractor:
 """Detects and extracts the inline generated UPDATE SQL block.

 Update-postprocess only. Depends on the host class for line rendering
 (_active_line_like / _logical / _is_comment_line), name helpers
 (_update_paragraph_name / _paragraph_exists / _strip_existing_final_period /
 _normalize_update_paragraph_lines), and the shared line_utils.
 Owns no regex and no column geometry; all patterns/constants are external.
 """

```

"""

```
def _extract_inline_update_block(
 self, *, lines, process_paragraph, block_info, diagnostics
):
 block_start, block_end, record_name, dcl_group = block_info
 update_paragraph = self._update_paragraph_name(record_name)

 if self._paragraph_exists(lines, update_paragraph):
 return lines

 extracted_block = self._replace_update_success_continue(
 extracted_block=lines[block_start:block_end]
)
 replacement = self._replacement_in_process(
 reference_line=lines[block_start],
 update_paragraph=update_paragraph,
)
 updated_lines = lines[:block_start] + replacement + lines[block_end:]

 process_range = self.line_utils.find_paragraph_range(
 updated_lines, process_paragraph
)
 if process_range:
 p_start, p_end = process_range
 updated_lines = self._ensure_initialize_dcl_group(
 lines=updated_lines, start=p_start, end=p_end, dcl_group=dcl_group
)

 update_paragraph_lines = self._build_update_paragraph(
 paragraph_name=update_paragraph, extracted_block=extracted_block
)
 insert_index = self._update_paragraph_insert_index(updated_lines)
 updated_lines = (
 updated_lines[:insert_index]
 + [""]
 + update_paragraph_lines
 + [""]
 + updated_lines[insert_index:]
)
 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["business_update_extracted"])
 return updated_lines

def _find_inline_update_block(self, *, lines, start, end):
 for index in range(start + 1, end):
 logical = self._logical(lines[index])
 comment_match = CONVERTED_MODIFY_COMMENT_PATTERN.match(logical)
 location_match = SQL_LOCATION_UPDATE_PATTERN.match(logical)
 if not comment_match and not location_match:
 continue

 record_name = ""
 if comment_match:
 record_name = str(comment_match.group("record") or "").strip().upper()
 if not record_name and location_match:
 record_name = str(location_match.group("record") or "").strip().upper()
 if not record_name:
 continue

 block_start = index
 if location_match and index > start + 1:
 previous_logical = self._logical(lines[index - 1])
 if CONVERTED_MODIFY_COMMENT_PATTERN.match(previous_logical):
 block_start = index - 1

 block_end = self._find_sqlcode_evaluate_end(
 lines=lines, start=index, limit=end
)
 if block_end <= index:
 continue

 dcl_group = self._find_dcl_group(lines=lines[block_start:block_end])
 return block_start, block_end, record_name, dcl_group

 return None
```



```

def _find_sqlcode_evaluate_end(self, *, lines, start, limit):
 seen_exec_sql = seen_update = seen_end_exec = seen_evaluate = False

 for index in range(start, limit):
 logical = self._logical(lines[index])
 if EXEC_SQL_START_PATTERN.match(logical):
 seen_exec_sql = True
 continue
 if seen_exec_sql and EXEC_SQL_UPDATE_PATTERN.match(logical):
 seen_update = True
 continue
 if seen_update and END_EXEC_PATTERN.match(logical):
 seen_end_exec = True
 continue
 if seen_end_exec and EVALUATE_SQLCODE_PATTERN.match(logical):
 seen_evaluate = True
 continue
 if seen_evaluate and END_EVALUATE_PATTERN.match(logical):
 return index + 1

 return -1

def _replacement_in_process(self, *, reference_line, update_paragraph):
 names = UPDATE_RESTART_WS_NAMES
 return [
 self._active_line_like(
 reference_line=reference_line,
 body=PERFORM_TEMPLATE.format(paragraph=update_paragraph),
),
 "",
 self._active_line_like(
 reference_line=reference_line,
 body=COMMIT_CHECK_IF_TEMPLATE.format(
 counter=names["commit_counter"],
 threshold=UPDATE_COMMIT_THRESHOLD,
),
),
 self._active_line_like(
 reference_line=reference_line,
 body=COMMIT_CHECK_PERFORM_TEMPLATE.format(
 indent=NESTED_INDENT,
 commit_paragraph=RESTART_PARAGRAPH_NAMES["commit"],
),
),
 self._active_line_like(
 reference_line=reference_line, body=END_IF_STATEMENT
),
]

def _build_update_paragraph(self, *, paragraph_name, extracted_block):
 cleaned_block = self._strip_existing_final_period(extracted_block)
 output = [f"{paragraph_name}.", ""]
 output.extend(cleaned_block)
 output.append(FINAL_PERIOD_LINE)
 return self._normalize_update_paragraph_lines(output)

def _replace_update_success_continue(self, *, extracted_block):
 names = UPDATE_RESTART_WS_NAMES
 output: list[str] = []
 index = 0

 while index < len(extracted_block):
 line = extracted_block[index]
 logical = self._logical(line)

 if WHEN_ZERO_PATTERN.match(logical):
 output.append(line)
 if index + 1 < len(extracted_block):
 next_logical = self._logical(extracted_block[index + 1])
 if CONTINUE_PATTERN.match(next_logical):
 output.append(
 self._active_line_like(
 reference_line=extracted_block[index + 1],
 body=ADD_UPDATE_COUNTER_TEMPLATE.format(
 counter=names["update_counter"]
),
),

```

```

)
)
 index += 2
 continue
 index += 1
 continue

 output.append(line)
 index += 1

return output

def _ensure_initialize_dcl_group(self, *, lines, start, end, dcl_group):
 if not dcl_group:
 return lines

 target = f"{TOKEN_INITIALIZE_DCL[:-3]} {dcl_group}".strip()
 for index in range(start, end):
 if self._logical(lines[index]).upper().rstrip(".") == target:
 return lines

 insert_index = self._first_executable_line_index(
 lines=lines, start=start, end=end
)
 if insert_index < 0:
 return lines

 initialize_line = self._active_line_like(
 reference_line=lines[insert_index],
 body=INITIALIZE_DCL_GROUP_TEMPLATE.format(group=dcl_group),
)
 return lines[:insert_index] + [initialize_line, ""] + lines[insert_index:]

def _first_executable_line_index(self, *, lines, start, end):
 for index in range(start + 1, end):
 line = lines[index]
 logical = self._logical(line).strip()
 if not logical or self._is_comment_line(line):
 continue
 if logical.startswith(TOKEN_STAR):
 continue
 if PARAGRAPH_HEADER_PATTERN.match(logical):
 continue
 return index
 return -1

def _update_paragraph_insert_index(self, lines):
 sqlerror_index = self._find_sqlerror_routine_index(lines)
 if sqlerror_index >= 0:
 return sqlerror_index

 restart_abend_range = self.line_utils.find_paragraph_range(
 lines, RESTART_PARAGRAPH_NAMES["abend"]
)
 if restart_abend_range:
 return restart_abend_range[1]

 return len(lines)

def _find_sqlerror_routine_index(self, lines):
 for index, line in enumerate(lines):
 logical = self._logical(line).upper().rstrip(".")
 if logical in set(SQL_ERROR_PARAGRAPH_NAMES):
 return index
 if SQL_ERROR_ROUTINE_MARKER in logical:
 return index
 return -1

def _find_dcl_group(self, lines):
 for line in lines:
 logical = self._logical(line)
 for match in DCL_HOST_GROUP_PATTERN.finditer(logical):
 group = str(match.group(0) or "").strip().upper()
 if group:
 return group
 return ""

```

```

=====
FILE: src\idms_db2_phase2\postprocess\update_business_sql_standardizer.py
ENCODING: utf-8
=====

from __future__ import annotations

from idms_db2_phase2.postprocess.dynamic_metadata_resolver import UpdateProgramContext
from idms_db2_phase2.postprocess.update_business_sql.update_business_sql_line_renderer import (
 UpdateBusinessSqlLineRenderer,
)
from idms_db2_phase2.postprocess.update_business_sql.update_paragraph_normalizer import (
 UpdateParagraphNormalizer,
)
from idms_db2_phase2.postprocess.update_business_sql.update_sql_block_extractor import (
 UpdateSqlBlockExtractor,
)
from idms_db2_phase2.postprocess.update_postprocess_line_utils import (
 UpdatePostprocessLineUtils,
)
from patterns.update_business_sql_patterns import (
 CONVERTED_MODIFY_COMMENT_PATTERN,
 PERFORM_UPDATE_PARAGRAPH_PATTERN,
)
from patterns.update_main_flow_patterns import PERFORM_UNTIL_LINE_PATTERN
from rules.update_business_sql_rules import (
 DIAG_PROCESS_PARAGRAPH_NOT_FOUND_TEMPLATE,
 DIAG_PROCESS_PARAGRAPH_UNRESOLVED,
 LONE_PERIOD,
 NESTED_INDENT,
 READ_FLAT_FILE_PARAGRAPH,
 TOKEN_END_IF,
 TOKEN_IF,
 TOKEN_INITIALIZE_DCL,
 TOKEN_PERFORM,
)
from rules.update_restart_rules import (
 UPDATE_RESTART_DIAGNOSTICS,
 UPDATE_RESTART_WS_NAMES,
 UPDATE_SQL_PARAGRAPH_PREFIX,
)

class UpdateBusinessSqlStandardizer(
 UpdateBusinessSqlLineRenderer,
 UpdateParagraphNormalizer,
 UpdateSqlBlockExtractor,
):
 """Standardizes update-program business SQL structure.

 Update-postprocess only; does not touch retrieval. This facade holds the
 orchestration and small paragraph helpers. Fixed-format rendering,
 paragraph normalization, and block extraction are provided by mixins.
 All COBOL tokens and column geometry live in
 rules/update_business_sql_rules.py.
 """

 def __init__(self, *, line_utils: UpdatePostprocessLineUtils) -> None:
 self.line_utils = line_utils

 def standardize(
 self,
 cobol_text: str,
 context: UpdateProgramContext,
 diagnostics: list[str],
) -> str:
 if not context.restart_dclgen:
 return cobol_text

 lines = self.line_utils.lines(cobol_text)

 process_paragraph = self._resolve_process_paragraph(
 lines=lines,
 input_record_name=context.input_record_name,

```

```

)
if not process_paragraph:
 diagnostics.append(DIAG_PROCESS_PARAGRAPH_UNRESOLVED)
 return cobol_text

paragraph_range = self.line_utils.find_paragraph_range(lines, process_paragraph)
if not paragraph_range:
 diagnostics.append(
 DIAG_PROCESS_PARAGRAPH_NOT_FOUND_TEMPLATE.format(
 paragraph=process_paragraph
)
)
 return cobol_text

start, end = paragraph_range
block_info = self._find_inline_update_block(lines=lines, start=start, end=end)

if block_info:
 lines = self._extract_inline_update_block(
 lines=lines,
 process_paragraph=process_paragraph,
 block_info=block_info,
 diagnostics=diagnostics,
)
elif not self._paragraph_contains_update_perform(
 lines=lines, start=start, end=end
):
 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["business_update_not_found"])

lines = self._cleanup_process_paragraph(
 lines=lines, process_paragraph=process_paragraph
)
lines = self._normalize_all_update_paragraphs(lines)

return self.line_utils.join(lines)

def _cleanup_process_paragraph(self, *, lines, process_paragraph):
 paragraph_range = self.line_utils.find_paragraph_range(lines, process_paragraph)
 if not paragraph_range:
 return lines

 start, end = paragraph_range
 output: list[str] = []

 for index, line in enumerate(lines):
 if not (start < index < end):
 output.append(line)
 continue

 logical = self._logical(line).strip()
 upper = logical.upper().rstrip(".")

 if upper.startswith(TOKEN_INITIALIZE_DCL) and self._is_comment_line(line):
 output.append(
 self._active_line_like(reference_line=line, body=upper + ".")
)
 continue

 if CONVERTED_MODIFY_COMMENT_PATTERN.match(logical):
 continue

 output.append(line)

 paragraph_range = self.line_utils.find_paragraph_range(output, process_paragraph)
 if not paragraph_range:
 return output

 start, end = paragraph_range
 return self._normalize_process_commit_block(lines=output, start=start, end=end)

def _normalize_process_commit_block(self, *, lines, start, end):
 output = list(lines)
 index = start + 1

 while index < end:
 logical = self._logical(output[index]).upper().rstrip(".")

```

```

 if not logical.startswith(TOKEN_IF):
 index += 1
 continue
 if UPDATE_RESTART_WS_NAMES["commit_counter"] not in logical:
 index += 1
 continue

 block_end = self._find_end_if(lines=output, start=index, limit=end)
 if block_end < 0:
 index += 1
 continue

 for block_index in range(index, block_end + 1):
 body = self._logical(output[block_index]).strip()
 if not body:
 continue
 if body.upper().startswith(TOKEN_PERFORM):
 output[block_index] = self._active_line_like(
 reference_line=output[block_index],
 body=f"{NESTED_INDENT}{body.rstrip('.')}",
)
 else:
 output[block_index] = self._active_line_like(
 reference_line=output[block_index], body=body
)

 index = block_end + 1

 return output

def _find_end_if(self, *, lines, start, limit):
 for index in range(start + 1, limit):
 logical = self._logical(lines[index]).upper().rstrip(".")
 if logical.startswith(TOKEN_END_IF):
 return index
 return -1

def _resolve_process_paragraph(self, *, lines, input_record_name):
 target_eof = f"{str(input_record_name or '').strip().upper()}-EOF"

 for line in lines:
 logical = self._logical(line).upper().rstrip(".")
 match = PERFORM_UNTIL_LINE_PATTERN.search(logical)
 if not match:
 continue
 if target_eof and target_eof not in logical:
 continue

 paragraph = str(match.group("paragraph") or "").strip().upper()
 if not paragraph or paragraph == READ_FLAT_FILE_PARAGRAPH:
 continue
 return paragraph

 return ""

def _paragraph_contains_update_perform(self, *, lines, start, end):
 for index in range(start + 1, end):
 if PERFORM_UPDATE_PARAGRAPH_PATTERN.match(self._logical(lines[index])):
 return True
 return False

def _paragraph_exists(self, lines, paragraph_name):
 target = str(paragraph_name or "").strip().upper().rstrip(".")
 for line in lines:
 if self._logical(line).upper().rstrip(".") == target:
 return True
 return False

def _update_paragraph_name(self, record_name):
 clean = str(record_name or "").strip().upper().replace("_", "-")
 return f"{UPDATE_SQL_PARAGRAPH_PREFIX}-{clean}"

def _strip_existing_final_period(self, lines):
 output = list(lines)
 while output and not self._logical(output[-1]):

```

```

 output.pop()
 if output and self._logical(output[-1]) == LONE_PERIOD:
 output.pop()
 return output

```

```

=====
FILE: src\idms_db2_phase2\postprocess\update_char_literal_quoter.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.update_char_literal_patterns import (
 END_EXEC_PATTERN,
 EXEC_SQL_START_PATTERN,
 MOVE_NUMLIT_ONLY_PATTERN,
 MOVE_NUMLIT_TO_HOST_OF_GROUP_PATTERN,
 TO_HOST_OF_GROUP_PATTERN,
)
from rules.update_char_literal_rules import (
 CHAR_DB2_TYPE_PREFIXES,
 CHAR_LITERAL_QUOTED_DIAGNOSTIC,
 CHAR_PICTURE_PREFIX,
)

```

```

class UpdateCharLiteralQuoter:
 """Quotes bare numeric literals moved into CHAR/VARCHAR DB2 host variables.

 Update-postprocess only. Does not touch retrieval.

 Generic + DCLGEN-driven:
 - Builds a {host_name: is_char} map from parsed DCLGEN columns.
 - CHAR is detected from the DCLGEN db2_type (CHAR/VARCHAR...) or the COBOL
 picture (PIC X...).
 - Rewrites non-SQL "MOVE <digits> TO <host> OF <DCLGROUP>" (one-line and
 wrapped forms) to "MOVE '<digits>' ..." only when the host is CHAR.
 - Never touches SQL host references inside EXEC SQL ... END-EXEC blocks.
 - No hardcoded program, table, column, DCLGEN group, or host variable names.
 """

```

```

 def __init__(self, dclgen_columns: list) -> None:
 self._char_hosts = self._build_char_host_map(dclgen_columns)

 # ----- #
 # Public entry point
 # ----- #
 def apply(
 self,
 text: str,
 diagnostics: list[str] | None = None,
) -> str:
 if not text or not self._char_hosts:
 return text or ""

 lines = (
 str(text)
 .replace("\r\n", "\n")
 .replace("\r", "\n")
 .split("\n")
)

 output: list[str] = []
 in_exec_sql = False
 changed = False
 index = 0

 while index < len(lines):
 line = lines[index]
 logical = self._logical(line)
 upper = logical.upper()

 # Track EXEC SQL blocks; never modify inside them.
 if EXEC_SQL_START_PATTERN.match(logical):

```

```

 # One-line EXEC SQL ... END-EXEC. does not open a block.
 if "END-EXEC" not in upper:
 in_exec_sql = True
 output.append(line)
 index += 1
 continue

 if in_exec_sql:
 output.append(line)
 if END_EXEC_PATTERN.match(logical):
 in_exec_sql = False
 index += 1
 continue

 # (1) One-line: MOVE <digits> TO <host> OF <DCLGROUP>
 one = MOVE_NUMLIT_TO_HOST_OF_GROUP_PATTERN.match(logical)
 if one and self._is_char_host(one.group("host")):
 literal = one.group("literal")
 host = one.group("host").upper()
 group = one.group("group").upper()
 new_body = f"MOVE '{literal}' TO {host} OF {group}"
 output.append(self._replace_body(line, new_body))
 changed = True
 index += 1
 continue

 # (2) Wrapped: current line "MOVE <digits>", next line
 # "TO <host> OF <DCLGROUP>".
 move_only = MOVE_NUMLIT_ONLY_PATTERN.match(logical)
 if move_only and index + 1 < len(lines):
 next_logical = self._logical(lines[index + 1])
 nxt = TO_HOST_OF_GROUP_PATTERN.match(next_logical)
 if nxt and self._is_char_host(nxt.group("host")):
 literal = move_only.group("literal")
 output.append(
 self._replace_body(line, f"MOVE '{literal}'")
)
 output.append(lines[index + 1])
 changed = True
 index += 2
 continue

 output.append(line)
 index += 1

 if changed and diagnostics is not None:
 diagnostics.append(CHAR_LITERAL_QUOTED_DIAGNOSTIC)

 return "\n".join(output).rstrip() + "\n"

CHAR host map (DCLGEN-driven)

def _build_char_host_map(self, dclgen_columns: list) -> dict[str, bool]:
 mapping: dict[str, bool] = {}
 for column in dclgen_columns or []:
 host = NameNormalizer.normalize(
 getattr(column, "cobol_host_name", "")
)
 if not host:
 continue
 db2_type = str(getattr(column, "db2_type", "") or "")
 picture = str(getattr(column, "cobol_picture", "") or "")
 if self._is_char_datatype(db2_type) or self._is_char_picture(
 picture
):
 mapping[host.upper()] = True
 return mapping

def _is_char_host(self, host: str) -> bool:
 key = NameNormalizer.normalize(host).upper()
 return self._char_hosts.get(key, False)

def _is_char_datatype(self, db2_type: str) -> bool:
 compact = str(db2_type or "").upper().replace(" ", "")
 return compact.startswith(CHAR_DB2_TYPE_PREFIXES)

```

```

def _is_char_picture(self, picture: str) -> bool:
 cleaned = str(picture or "").strip().upper().lstrip("(")
 # Drop a leading "PIC " if present.
 if cleaned.startswith("PIC "):
 cleaned = cleaned[4:].strip()
 return cleaned.startswith(CHAR_PICTURE_PREFIX)

Fixed-format line helpers (80-col aware)

BODY_WIDTH = 65

def _logical(self, line: str) -> str:
 text = str(line or "").rstrip("\n")
 if len(text) >= 72 and text[:6].strip().isdigit():
 return text[7:72].strip()
 if len(text) > 6 and text[:6].strip().isdigit():
 body = text[7:] if len(text) > 7 else ""
 if len(body) >= 8 and body[-8:].strip().isdigit():
 body = body[:-8]
 return body.strip()
 return text.strip()

def _replace_body(self, line: str, body: str) -> str:
 text = str(line or "").rstrip("\n")
 clean = str(body or "")[: self.BODY_WIDTH].ljust(self.BODY_WIDTH)
 if len(text) >= 72 and text[:6].strip().isdigit():
 left = text[:6]
 right = text[72:80] if len(text) >= 80 else ""
 return f"{left} {clean} {right}"
 if len(text) > 6 and text[:6].strip().isdigit():
 left = text[:6]
 return f"{left} {clean.rstrip()}"
 return clean.rstrip()

```

```

=====
FILE: src\idms_db2_phase2\postprocess\update_cobol_file_resolver.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from typing import Optional

from idms_db2_phase2.postprocess.update_metadata_models import (
 CobolFileInfo,
 norm_name,
 upper,
)

from patterns.dynamic_metadata_patterns import (
 COPY_RE,
 EOF_IF_RE,
 EOF_MOVE_RE,
 EOF_UNTIL_RE,
 FD_LINE_RE,
 FD_RECORD_01_RE,
 FIELD_USE_RE,
 MOVE_FIELD_RE,
 PROGRAM_RE,
 READ_LINE_RE,
 READ_RE,
 RECORD_CONTAINS_RE,
 SECTION_END_RE,
 SELECT_LINE_RE,
 SELECT_RE,
)

```

```

class UpdateCobolFileResolver:
 def resolve_program_id(
 self,
 *,
 generated_cobol: str,

```



```

 source_cobol: str,
 target_program_id: str,
) -> str:
 for text in [generated_cobol, source_cobol]:
 logical_text = self.logical_statement_text(text)
 match = PROGRAM_RE.search(logical_text)

 if match:
 return upper(match.group(1))

 return upper(target_program_id)

def parse_cobol_files(
 self,
 *,
 source_cobol: str,
 generated_cobol: str,
) -> dict[str, CobolFileInfo]:
 files: dict[str, CobolFileInfo] = {}

 for raw_text in [source_cobol, generated_cobol]:
 self._parse_from_logical_text(
 files=files,
 raw_text=raw_text,
)
 self._parse_from_physical_lines(
 files=files,
 raw_text=raw_text,
)

 return files

def choose_input_file(
 self,
 files: dict[str, CobolFileInfo],
) -> CobolFileInfo:
 read_files = [item for item in files.values() if item.read_target]
 if read_files:
 return read_files[0]

 fd_files = [item for item in files.values() if item.fd_record]
 if fd_files:
 return fd_files[0]

 return next(iter(files.values()))

def extract_copy_statements(
 self,
 text: str,
) -> set[str]:
 logical_text = self.logical_statement_text(text)

 return {
 norm_name(match.group(1))
 for match in COPY_RE.finditer(logical_text)
 }

def extract_source_field_references(
 self,
 source_cobol: str,
) -> set[str]:
 refs: set[str] = set()
 logical_text = self.logical_statement_text(source_cobol)

 for match in FIELD_USE_RE.finditer(logical_text):
 refs.add(norm_name(match.group(1)))

 for match in MOVE_FIELD_RE.finditer(logical_text):
 refs.add(norm_name(match.group(1)))

 return refs

def resolve_legacy_eof_switch(
 self,
 *,
 source_cobol: str,

```

```

 generated_cobol: str,
) -> str:
 combined = "\n".join(
 [
 self.logical_statement_text(source_cobol),
 self.logical_statement_text(generated_cobol),
]
)

 candidates: dict[str, int] = {}

 for pattern in [EOF_UNTIL_RE, EOF_IF_RE, EOF_MOVE_RE]:
 for match in pattern.finditer(combined):
 name = norm_name(match.group(1))

 if not name:
 continue

 if "EOF" not in name:
 continue

 candidates[name] = candidates.get(name, 0) + 1

 if not candidates:
 return ""

 return sorted(
 candidates.items(),
 key=lambda item: item[1],
 reverse=True,
)[0][0]

def _parse_from_logical_text(
 self,
 *,
 files: dict[str, CobolFileInfo],
 raw_text: str,
) -> None:
 text = self.logical_statement_text(raw_text)

 for match in SELECT_RE.finditer(text):
 name = upper(match.group(1))
 assignment = upper(match.group(2))
 files.setdefault(name, CobolFileInfo(name=name)).assignment = assignment

 for match in READ_RE.finditer(text):
 file_name = upper(match.group(1))
 read_target = upper(match.group(2))

 if read_target:
 files.setdefault(
 file_name,
 CobolFileInfo(name=file_name),
).read_target = read_target

def _parse_from_physical_lines(
 self,
 *,
 files: dict[str, CobolFileInfo],
 raw_text: str,
) -> None:
 current_fd = ""

 for raw_line in str(raw_text or "").splitlines():
 line = self.fixed_format_body_line(raw_line).strip()

 if not line:
 continue

 select_match = SELECT_LINE_RE.search(line)

 if select_match:
 file_name = upper(select_match.group(1))
 assignment = upper(select_match.group(2))
 files.setdefault(
 file_name,

```

```

 CobolFileInfo(name=file_name),
).assignment = assignment
 continue

fd_match = FD_LINE_RE.search(line)

if fd_match:
 current_fd = upper(fd_match.group(1))
 files.setdefault(current_fd, CobolFileInfo(name=current_fd))
 continue

if current_fd:
 length_match = RECORD_CONTAINS_RE.search(line)

 if length_match:
 files[current_fd].record_length = int(length_match.group(1))
 continue

 record_match = FD_RECORD_01_RE.search(line)

 if record_match:
 files[current_fd].fd_record = upper(record_match.group(1))
 continue

 if SECTION_END_RE.search(line):
 current_fd = ""

read_match = READ_LINE_RE.search(line)

if read_match:
 file_name = upper(read_match.group(1))
 read_target = upper(read_match.group(2))

 if read_target:
 files.setdefault(
 file_name,
 CobolFileInfo(name=file_name),
).read_target = read_target

def fixed_format_body_text(
 self,
 text: str,
) -> str:
 output: list[str] = []

 for raw_line in str(text or "").splitlines():
 body = self.fixed_format_body_line(raw_line)

 if not body:
 continue

 if body.lstrip().startswith("*") or body.lstrip().startswith("/"):
 continue

 output.append(body)

 return "\n".join(output)

def fixed_format_body_line(
 self,
 line: str,
) -> str:
 text = str(line or "").rstrip()

 if not text.strip():
 return ""

 if len(text) >= 72 and text[:6].strip().isdigit():
 indicator = text[6:7]

 if indicator in ("*", "/"):
 return ""

 return text[7:72].strip()

 if len(text) > 6 and text[:6].strip().isdigit():

```

```

 body = text[6:]

 if len(body) >= 8 and body[-8:].strip().isdigit():
 body = body[:-8]

 if body[:1] in ("*", "/", "D"):
 return body.strip()

 return body.strip()

 return text.strip()

def logical_statement_text(
 self,
 text: str,
) -> str:
 body_text = self.fixed_format_body_text(text)
 statements: list[str] = []
 buffer = ""

 for raw_line in body_text.splitlines():
 line = raw_line.strip()

 if not line:
 continue

 if buffer:
 buffer = f"{buffer} {line}"
 else:
 buffer = line

 if "." in line:
 parts = buffer.split(".")

 for part in parts[:-1]:
 clean = part.strip()

 if clean:
 statements.append(clean + ".")

 buffer = parts[-1].strip()

 if buffer:
 statements.append(buffer)

 return "\n".join(statements)

```

```

=====
FILE: src\idms_db2_phase2\postprocess\update_copybook_resolver.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from typing import Any

from idms_db2_phase2.postprocess.update_metadata_models import (
 CobolFileInfo,
 CopybookRecordInfo,
 include_from_label,
 norm_name,
 upper,
 value,
)

from patterns.dynamic_metadata_patterns import PIC_LENGTH_RE

class UpdateCopybookResolver:
 def build_copybook_records(
 self,
 copybook_fields: list[Any],
) -> list[CopybookRecordInfo]:
 records: list[CopybookRecordInfo] = []
 current: CopybookRecordInfo | None = None

```

```

for item in copybook_fields:
 level = self._field_level(item)
 name = norm_name(
 value(
 item,
 "name",
 "field_name",
 "cobol_name",
 "copybook_field_name",
 "field",
)
)

 if not name:
 continue

 source_label = upper(
 value(
 item,
 "source_label",
 "copybook_name",
 "source",
 "file_name",
 default="",
)
)

 if level == 1 or current is None:
 include_name = include_from_label(source_label) or name
 current = CopybookRecordInfo(
 include_name=include_name,
 record_name=name,
 source_label=source_label,
)
 records.append(current)

 current.fields.add(name)

 pic = value(item, "pic", "pic_clause", "picture", default="")
 occurs = value(item, "occurs", "occurs_times", default=1)

 try:
 occurs_count = int(occurs or 1)
 except (TypeError, ValueError):
 occurs_count = 1

 current.estimated_length += (
 self.estimate_pic_length(pic) * occurs_count
)

return records

def choose_copybook(
 self,
 *,
 input_file: CobolFileInfo,
 copybooks: list[CopybookRecordInfo],
 copy_statements: set[str],
 source_refs: set[str],
) -> CopybookRecordInfo:
 if not copybooks:
 raise ValueError("No copybook fields were parsed from uploaded copybook input.")

 scored: list[tuple[int, CopybookRecordInfo]] = []

 read_target = norm_name(input_file.read_target)
 fd_record = norm_name(input_file.fd_record)

 for copybook in copybooks:
 score = 0

 if read_target and read_target in {
 copybook.record_name,
 copybook.include_name,
 }:

```

```

 score += 100

 if fd_record and fd_record in {
 copybook.record_name,
 copybook.include_name,
 }:
 score += 20

 if copybook.include_name in copy_statements:
 score += 50

 if copybook.record_name in copy_statements:
 score += 40

 if (
 input_file.record_length
 and copybook.estimated_length == input_file.record_length
):
 score += 30

 score += len(copybook.fields.intersection(source_refs))

 scored.append((score, copybook))

scored.sort(key=lambda item: item[0], reverse=True)

if scored[0][0] <= 0 and len(copybooks) > 1:
 raise ValueError(
 "Unable to resolve input copybook safely. "
 "Multiple copybooks exist and no reliable match was found."
)

return scored[0][1]

def _field_level(
 self,
 item: Any,
) -> int:
 raw = value(
 item,
 "level",
 "level_number",
 "cobol_level",
 default=0,
)

 try:
 return int(raw)
 except (TypeError, ValueError):
 return 0

def estimate_pic_length(
 self,
 pic: Any,
) -> int:
 text = upper(pic)

 if not text:
 return 0

 text = text.replace(" ", "").replace(".", "")
 digits = 0

 for match in PIC_LENGTH_RE.finditer(text):
 digits += int(match.group(1)) if match.group(1) else 1

 if "COMP-3" in text or "COMP_3" in text:
 return int((digits + 2) / 2)

 return digits

```

```
=====
FILE: src\idms_db2_phase2\postprocess\update_date_host_converter.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.update_date_host_patterns import (
 END_EXEC_PATTERN,
 EXEC_SQL_START_PATTERN,
 MOVE_SOURCE_ONLY_PATTERN,
 MOVE_TO_HOST_OF_GROUP_PATTERN,
 TO_HOST_OF_GROUP_PATTERN,
)
from rules.sql_generation_rules import DB2_DATE_NULL_SENTINEL

class UpdateDateHostConverter:
 """Converts raw moves into DB2 DATE host variables to the manual date form.

 Update-postprocess only. Does not touch retrieval.

 A DB2 DATE column (DCLGEN db2_type == DATE, stored as PIC X(10)) must
 receive its value through the DD.MM.CCYY realignment, not a raw MOVE.
 TIMESTAMP columns are NOT date columns (they receive a formatted
 TS-TIMESTAMP) and are excluded.

 Finds:
 MOVE <source> TO <DATE-host> OF <DCLGROUP>
 where <source> is a plain field (not DA-DD-MM-CCYY, not TS-TIMESTAMP,
 not a literal) and <DATE-host> is a DCLGEN DATE column, and replaces it
 with the manual conversion block.

 Guards:
 - Only DATE hosts (DCLGEN db2_type == DATE) are converted.
 - TIMESTAMP / TIME columns are excluded.
 - Audit hosts (TS-UPDATE, TS-CREATE, ID-USERID, NR-USERID) are excluded.
 - Sources TS-TIMESTAMP / CS-PROGRAM are excluded.
 - Already-realigned moves (source == DA-DD-MM-CCYY) are skipped.
 - EXEC SQL host references are never touched.
 - No program, record, table, DCLGEN, or host name is hardcoded.
 """

 _ALREADY_REALIGNED_SOURCE = "DADMMCCYY"
 _PROTECTED_SOURCES = ("TS_TIMESTAMP", "CS_PROGRAM")
 _AUDIT_HOST_PREFIXES = ("TS_UPDATE", "TS_CREATE", "ID_USERID", "NR_USERID")

 def __init__(self, dclgen_columns: list) -> None:
 self._date_hosts = self._build_date_host_map(dclgen_columns)

 # ----- #
 # Public entry point
 # ----- #
 def apply(
 self,
 text: str,
 diagnostics: list[str] | None = None,
) -> str:
 if not text or not self._date_hosts:
 return text or ""

 lines = (
 str(text)
 .replace("\r\n", "\n")
 .replace("\n", "\n")
 .split("\n")
)

 output: list[str] = []
 in_exec_sql = False
 changed = False
 index = 0

 while index < len(lines):
 line = lines[index]
```

```

logical = self._logical(line)
upper = logical.upper()

if EXEC_SQL_START_PATTERN.match(logical):
 if "END-EXEC" not in upper:
 in_exec_sql = True
 output.append(line)
 index += 1
 continue

if in_exec_sql:
 output.append(line)
 if END_EXEC_PATTERN.match(logical):
 in_exec_sql = False
 index += 1
 continue

(1) One-line: MOVE <source> TO <host> OF <DCLGROUP>
one = MOVE_TO_HOST_OF_GROUP_PATTERN.match(logical)
if one:
 source = one.group("source").strip()
 host = one.group("host").upper()
 group = one.group("group").upper()
 if self._should_convert(host, source):
 output.extend(
 self._conversion_lines(line, source, host, group)
)
 changed = True
 index += 1
 continue

(2) Wrapped: "MOVE <source>" + next "TO <host> OF <DCLGROUP>"
move_only = MOVE_SOURCE_ONLY_PATTERN.match(logical)
if move_only and index + 1 < len(lines):
 next_logical = self._logical(lines[index + 1])
 nxt = TO_HOST_OF_GROUP_PATTERN.match(next_logical)
 if nxt:
 source = move_only.group("source").strip()
 host = nxt.group("host").upper()
 group = nxt.group("group").upper()
 if self._should_convert(host, source):
 output.extend(
 self._conversion_lines(line, source, host, group)
)
 changed = True
 index += 2
 continue

output.append(line)
index += 1

if changed and diagnostics is not None:
 diagnostics.append(
 "Update date host: converted raw move into DB2 DATE host "
 "to DD.MM.CCYY realignment form."
)

return "\n".join(output).rstrip() + "\n"

Decision + generation

def _should_convert(self, host: str, source: str) -> bool:
 if not self._is_date_host(host):
 return False

 host_key = NameNormalizer.normalize(host).upper().replace("-", "")
 if host_key.startswith(self._AUDIT_HOST_PREFIXES):
 return False

 src_key = NameNormalizer.normalize(source).upper().replace("-", "")
 if src_key == self._ALREADY_REALIGNED_SOURCE:
 return False
 if src_key in self._PROTECTED_SOURCES:
 return False
 if not source or source[0].isdigit() or source.startswith("'"):

```



```

 return False
 return True

def _conversion_lines(
 self,
 reference_line: str,
 source: str,
 host: str,
 group: str,
) -> list[str]:
 host_ref = f"{host} OF {group}"
 bodies = [
 "MOVE ZEROES TO DA-CCYYMMDD",
 f"MOVE {source} TO DA-CCYYMMDD",
 "IF DA-CCYYMMDD = ZERO OR DA-CCYYMMDD = SPACES",
 f" MOVE {DB2_DATE_NULL_SENTINEL} TO DA-CCYYMMDD",
 "END-IF",
 "MOVE CORR DA-CCYYMMDD-R TO DA-DD-MM-CCYY",
 f"MOVE DA-DD-MM-CCYY TO {host_ref}",
]
 return [self._replace_body(reference_line, b) for b in bodies]

DATE host map (DCLGEN-driven)

def _build_date_host_map(self, dclgen_columns: list) -> dict[str, bool]:
 mapping: dict[str, bool] = {}
 for column in dclgen_columns or []:
 host = NameNormalizer.normalize(
 getattr(column, "cobol_host_name", "")
)
 if not host:
 continue
 db2_type = str(getattr(column, "db2_type", "") or "")
 if self._is_date_datatype(db2_type):
 mapping[host.upper()] = True
 return mapping

def _is_date_host(self, host: str) -> bool:
 key = NameNormalizer.normalize(host).upper()
 return self._date_hosts.get(key, False)

def _is_date_datatype(self, db2_type: str) -> bool:
 compact = str(db2_type or "").upper().replace(" ", "")
 # DATE only. TIMESTAMP/TIME columns receive a formatted timestamp,
 # NOT a DD.MM.CCYY date, so they must be excluded.
 if compact.startswith("TIMESTAMP") or compact.startswith("TIME"):
 return False
 return compact.startswith("DATE")

Fixed-format helpers

BODY_WIDTH = 65

def _logical(self, line: str) -> str:
 text = str(line or "").rstrip("\n")
 if len(text) >= 72 and text[:6].strip().isdigit():
 return text[7:72].strip()
 if len(text) > 6 and text[:6].strip().isdigit():
 body = text[7:] if len(text) > 7 else ""
 if len(body) >= 8 and body[-8:].strip().isdigit():
 body = body[:-8]
 return body.strip()
 return text.strip()

def _replace_body(self, line: str, body: str) -> str:
 text = str(line or "").rstrip("\n")
 clean = str(body or "")[: self.BODY_WIDTH].ljust(self.BODY_WIDTH)
 if len(text) >= 72 and text[:6].strip().isdigit():
 left = text[:6]
 right = text[72:80] if len(text) >= 80 else ""
 return f"{left} {clean} {right}"
 if len(text) > 6 and text[:6].strip().isdigit():
 left = text[:6]
 return f"{left} {clean.rstrip()}"

```

```
return clean.rstrip()
```

```
=====
FILE: src\idms_db2_phase2\postprocess\update_enhancement_result.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from dataclasses import dataclass, field
```

```
@dataclass
class EnhancementResult:
 converted_cobol: str
 diagnostics: list[str] = field(default_factory=list)
```

```
=====
FILE: src\idms_db2_phase2\postprocess\update_final_cleanup.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.postprocess.final_cleanup.final_cleanup_dcl_reference import (
 FinalCleanupDclReference,
)
from idms_db2_phase2.postprocess.final_cleanup.final_cleanup_line_utils import (
 FinalCleanupLineUtils,
)
from idms_db2_phase2.postprocess.final_cleanup.final_cleanup_normalizer import (
 FinalCleanupNormalizer,
)
from patterns.update_final_cleanup_patterns import (
 CONVERTED_MODIFY_COMMENT_PATTERN,
 DCL_REFERENCE_PATTERN,
 INITIALIZE_DCL_PATTERN,
 MOVE_SPACES_TO_BARE_RECORD_PATTERN,
)
from rules.update_cobol_final_cleanup_rules import (
 FINAL_CLEANUP_MAX_CONSECUTIVE_BLANKS,
)
```

```
class UpdateFinalCleanup(
 FinalCleanupLineUtils,
 FinalCleanupDclReference,
 FinalCleanupNormalizer,
):
```

```
 """Final update-only cleanup after fixed-format resequencing.
```

```
 Orchestration only. Fixed-format helpers, DCL reference rewriting, and
 paragraph/procedure normalization are provided by mixins. All COBOL tokens
 and column geometry live in rules/update_cobol_final_cleanup_rules.py.
```

```
 Narrow, generic fixes only; does not rename variables/paragraphs and does
 not touch retrieval.
 """
```

```
 def apply(self, text: str) -> str:
 if not text:
 return ""
```

```
 lines = (
 str(text or "")
 .replace("\r\n", "\n")
 .replace("\r", "\n")
 .splitlines()
)
```

```
 lines = self._remove_leftover_modify_comments(lines)
 lines = self._rewrite_non_sql_dcl_dot_references(lines)
 lines = self._remove_obsolete_bare_record_initializes(lines)
 lines = self._normalize_update_paragraphs(lines)
```

```

lines = self._normalize_procedure_simple_blocks(lines)
lines = self._normalize_excess_blank_lines(lines)

return "\n".join(lines).rstrip() + "\n"

def _remove_leftover_modify_comments(self, lines: list[str]) -> list[str]:
 output: list[str] = []
 for line in lines:
 logical = self._body(line).strip()
 raw = str(line or "")
 if CONVERTED_MODIFY_COMMENT_PATTERN.search(logical):
 continue
 if CONVERTED_MODIFY_COMMENT_PATTERN.search(raw):
 continue
 output.append(line)
 return output

def _remove_obsolete_bare_record_initializes(
 self, lines: list[str]
) -> list[str]:
 remove_indexes: set[int] = set()

 for start, end in self._paragraph_ranges(lines):
 paragraph = lines[start:end]

 has_dcl_initialize = any(
 INITIALIZE_DCL_PATTERN.match(self._logical(line))
 for line in paragraph
)
 has_dcl_usage = any(
 DCL_REFERENCE_PATTERN.search(self._logical(line))
 for line in paragraph
)
 if not has_dcl_initialize or not has_dcl_usage:
 continue

 for index in range(start + 1, end):
 if MOVE_SPACES_TO_BARE_RECORD_PATTERN.match(self._logical(lines[index])):
 remove_indexes.add(index)

 if not remove_indexes:
 return lines

 return [
 line for index, line in enumerate(lines) if index not in remove_indexes
]

def _normalize_excess_blank_lines(self, lines: list[str]) -> list[str]:
 output: list[str] = []
 blank_count = 0

 for line in lines:
 if self._logical(line):
 blank_count = 0
 output.append(line)
 continue

 blank_count += 1
 if blank_count <= FINAL_CLEANUP_MAX_CONSECUTIVE_BLANKS:
 output.append(line)

 return output

```

```

=====
FILE: src\idms_db2_phase2\postprocess\update_input_record_renamer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import re

from patterns.update_record_rename_patterns import (
 COPY_RECORD_PATTERN_TEMPLATE,
 WORD_RECORD_PATTERN_TEMPLATE,

```

)

```
class UpdateInputRecordRenamer:
 """Renames original source references to the input record.

 Update-postprocess only. Does not touch retrieval.

 Given the original input-record name and its derived name (e.g.
 VMBD205I -> VMDZ205I from the VM...BD...->VMDZ rule), this pass:
 - renames non-comment executable references (COPY, OF/IN, INITIALIZE,
 READ INTO, MOVE, etc.) from old -> new,
 - removes duplicate COPY <new> lines (keeps the first),
 - never modifies comment lines (indicator '*' in column 7) so REMARKS
 history stays byte-for-byte identical.

 No program, record, or copybook name is hardcoded; old/new come from the
 resolved context.
 """

 def apply(
 self,
 text: str,
 old_name: str,
 new_name: str,
 diagnostics: list[str] | None = None,
) -> str:
 if not text:
 return text or ""

 old = str(old_name or "").strip().upper()
 new = str(new_name or "").strip().upper()

 if not old or not new or old == new:
 return text

 word_pattern = re.compile(
 WORD_RECORD_PATTERN_TEMPLATE.format(name=re.escape(old)),
 flags=re.IGNORECASE,
)
 copy_new_pattern = re.compile(
 COPY_RECORD_PATTERN_TEMPLATE.format(name=re.escape(new)),
 flags=re.IGNORECASE,
)

 lines = (
 str(text)
 .replace("\r\n", "\n")
 .replace("\r", "\n")
 .split("\n")
)

 output: list[str] = []
 changed = False
 seen_copy_new = False

 for line in lines:
 # Never touch comment / debug / page lines.
 if self._is_comment_or_debug(line):
 output.append(line)
 continue

 body = self._body(line)

 # Rename whole-word occurrences of the old record name.
 new_body = word_pattern.sub(new, body)

 if new_body != body:
 changed = True
 line = self._replace_body(line, new_body)
 body = new_body

 # Deduplicate COPY <new> lines (keep the first only).
 if copy_new_pattern.search(body):
 if seen_copy_new:
 changed = True
```

```

 continue # drop the duplicate COPY line
 seen_copy_new = True

 output.append(line)

 if changed and diagnostics is not None:
 diagnostics.append(
 f"Renamed input record references {old} -> {new} "
 "and removed duplicate COPY."
)

 return "\n".join(output).rstrip() + "\n"

Fixed-format helpers

def _is_comment_or_debug(self, line: str) -> bool:
 text = str(line or "").rstrip("\n")
 if len(text) >= 7 and text[:6].strip().isdigit():
 return text[6:7] in {"*", "/", "D", "d"}
 stripped = text.strip()
 return (
 stripped.startswith("*")
 or stripped.startswith("/")
 or stripped.upper().startswith("D ")
)

def _body(self, line: str) -> str:
 text = str(line or "").rstrip("\n")
 if len(text) >= 72 and text[:6].strip().isdigit():
 return text[7:72].rstrip()
 if len(text) > 6 and text[:6].strip().isdigit():
 body = text[7:] if len(text) > 7 else ""
 if len(body) >= 8 and body[-8:].strip().isdigit():
 body = body[:-8]
 return body.rstrip()
 return text.rstrip()

def _replace_body(self, line: str, body: str) -> str:
 text = str(line or "").rstrip("\n")
 clean = str(body or "")[:65].ljust(65)
 if len(text) >= 72 and text[:6].strip().isdigit():
 left = text[:6]
 right = text[72:80] if len(text) >= 80 else ""
 return f"{left} {clean} {right}"
 if len(text) > 6 and text[:6].strip().isdigit():
 left = text[:6]
 return f"{left} {clean.rstrip()}"
 return clean.rstrip()

=====
FILE: src\idms_db2_phase2\postprocess\update_main_flow_rewriter.py
ENCODING: utf-8
=====

from __future__ import annotations

from idms_db2_phase2.postprocess.dynamic_metadata_resolver import UpdateProgramContext
from idms_db2_phase2.postprocess.main_flow.main_flow_line_utils import (
 MainFlowLineUtils,
)
from idms_db2_phase2.postprocess.main_flow.main_flow_locators import MainFlowLocators
from idms_db2_phase2.postprocess.main_flow.main_flow_paragraph_resolver import (
 MainFlowParagraphResolver,
)
from idms_db2_phase2.postprocess.update_postprocess_line_utils import (
 UpdatePostprocessLineUtils,
)
from rules.update_restart_rules import (
 MAIN_FLOW_CLOSE_LOOKBACK,
 MAIN_FLOW_DIAGNOSTICS,
 MAIN_FLOW_INITIALIZE_HELP_TEMPLATE,
 MAIN_FLOW_OPEN_LOOKBACK,
 MAIN_FLOW_PERFORM_READ_BODY,
 MAIN_FLOW_PERFORM_READ_BODY_DOT,

```

```

MAIN_FLOW_RESTART_TEMPLATE_LINES,
MAIN_FLOW_SET_NOT_EOF_TEMPLATE,
RESTART_PARAGRAPH_NAMES,
UPDATE_MAIN_DATE_INITIALIZATION_LINES,
UPDATE_RESTART_DIAGNOSTICS,
UPDATE_RESTART_WS_NAMES,
UPDATE_SUMMARY_DISPLAY_TEMPLATES,
)

class UpdateMainFlowRewriter(
 MainFlowLineUtils,
 MainFlowParagraphResolver,
 MainFlowLocators,
):
 """Rewrites update-program main restart flow only.

 Update-postprocess only; does not touch retrieval. Orchestration and
 injection live here; line rendering, paragraph resolution, and locators
 are provided by mixins. All COBOL tokens, templates, and geometry live in
 rules/update_restart_rules.py.
 """

 def __init__(self, *, line_utils: UpdatePostprocessLineUtils) -> None:
 self.line_utils = line_utils

 def replace_legacy_restart_main_flow(
 self, cobol_text, context, diagnostics
):
 if not context.restart_dclgen:
 diagnostics.append(MAIN_FLOW_DIAGNOSTICS["dclgen_unresolved"])
 return cobol_text

 lines = self.line_utils.lines(cobol_text)
 open_index = self._find_open_input_index(
 lines=lines, file_name=context.input_file.name
)
 if open_index < 0:
 diagnostics.append(MAIN_FLOW_DIAGNOSTICS["open_not_found"])
 return cobol_text

 stop_index = self._find_stop_run_index(lines=lines, start_index=open_index + 1)
 if stop_index < 0:
 diagnostics.append(MAIN_FLOW_DIAGNOSTICS["stop_not_found"])
 return cobol_text

 if not self._has_legacy_restart_between(
 lines=lines, start_index=open_index + 1, end_index=stop_index
):
 diagnostics.append(MAIN_FLOW_DIAGNOSTICS["legacy_not_found"])
 return cobol_text

 process_paragraph = self._resolve_process_paragraph_from_main_flow(
 lines=lines, start_index=open_index + 1, end_index=stop_index
) or self._resolve_first_business_paragraph(lines=lines)

 if not process_paragraph:
 diagnostics.append(MAIN_FLOW_DIAGNOSTICS["process_unresolved"])
 return cobol_text

 lines = self._ensure_main_initialization_before_open(
 lines=lines, open_index=open_index, context=context, diagnostics=diagnostics
)

 open_index = self._find_open_input_index(
 lines=lines, file_name=context.input_file.name
)
 stop_index = self._find_stop_run_index(lines=lines, start_index=open_index + 1)

 replacement = self._main_restart_flow_lines(
 context=context, process_paragraph=process_paragraph
)
 updated_lines = lines[: open_index + 1] + replacement + lines[stop_index:]

 updated_lines = self._ensure_summary_display_before_close(
 lines=updated_lines, file_name=context.input_file.name, diagnostics=diagnostics

```

```

)
diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["legacy_replaced"])

updated_text = self.line_utils.join(updated_lines)
return self.ensure_processing_paragraph_reads_next(
 cobol_text=updated_text,
 process_paragraph=process_paragraph,
 diagnostics=diagnostics,
)

def ensure_processing_paragraph_reads_next(
 self, *, cobol_text, process_paragraph, diagnostics
):
 if not process_paragraph:
 diagnostics.append(MAIN_FLOW_DIAGNOSTICS["process_empty"])
 return cobol_text

 lines = self.line_utils.lines(cobol_text)
 paragraph_range = self.line_utils.find_paragraph_range(lines, process_paragraph)
 if not paragraph_range:
 diagnostics.append(
 MAIN_FLOW_DIAGNOSTICS["process_not_found"].format(
 paragraph=process_paragraph
)
)
 return cobol_text

 start, end = paragraph_range
 if self._paragraph_contains_perform_read_flat_file(lines[start:end]):
 diagnostics.append(
 MAIN_FLOW_DIAGNOSTICS["already_reads_next"].format(
 paragraph=process_paragraph
)
)
 return cobol_text

 insert_index, use_existing_period = self._read_next_insert_plan(
 lines=lines, start=start, end=end
)
 if insert_index < 0:
 diagnostics.append(
 MAIN_FLOW_DIAGNOSTICS["no_insertion_point"].format(
 paragraph=process_paragraph
)
)
 return cobol_text

 body = (
 MAIN_FLOW_PERFORM_READ_BODY
 if use_existing_period
 else MAIN_FLOW_PERFORM_READ_BODY_DOT
)
 reference_index = insert_index - 1 if insert_index > start else start
 read_line = self._line_like(reference_line=lines[reference_index], body=body)

 updated_lines = lines[:insert_index] + [read_line] + lines[insert_index:]
 diagnostics.append(
 MAIN_FLOW_DIAGNOSTICS["read_inserted"].format(paragraph=process_paragraph)
)
 return self.line_utils.join(updated_lines)

def _ensure_main_initialization_before_open(
 self, *, lines, open_index, context, diagnostics
):
 names = UPDATE_RESTART_WS_NAMES
 required_bodies = [
 MAIN_FLOW_SET_NOT_EOF_TEMPLATE.format(record=context.input_record_name),
 MAIN_FLOW_INITIALIZE_HELP_TEMPLATE.format(help_group=names["help_group"]),
 *UPDATE_MAIN_DATE_INITIALIZATION_LINES,
]

 existing_region = "\n".join(
 self._logical(line).upper()
 for line in lines[max(0, open_index - MAIN_FLOW_OPEN_LOOKBACK): open_index + 1]
)
 missing_bodies = [

```

```

 body
 for body in required_bodies
 if body.upper().rstrip(".") not in existing_region.replace(".", "")
]
 if not missing_bodies:
 return lines

 reference_line = lines[open_index]
 new_lines = [
 self._line_like(reference_line=reference_line, body=body)
 for body in missing_bodies
]
 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["main_init_added"])
 return lines[:open_index] + new_lines + [""] + lines[open_index:]

def _ensure_summary_display_before_close(
 self, *, lines, file_name, diagnostics
):
 names = UPDATE_RESTART_WS_NAMES
 close_index = self._find_close_input_index(lines=lines, file_name=file_name)
 if close_index < 0:
 return lines

 existing_region = "\n".join(
 self._logical(line).upper()
 for line in lines[max(0, close_index - MAIN_FLOW_CLOSE_LOOKBACK): close_index + 1]
)
 display_bodies = [
 template.format(
 input_counter=names["input_counter"],
 update_counter=names["update_counter"],
)
 for template in UPDATE_SUMMARY_DISPLAY_TEMPLATES
]
 missing_bodies = [
 body
 for body in display_bodies
 if body.upper().rstrip(".") not in existing_region.replace(".", "")
]
 if not missing_bodies:
 return lines

 reference_line = lines[close_index]
 new_lines = [
 self._line_like(reference_line=reference_line, body=body)
 for body in missing_bodies
]
 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["summary_display_added"])
 return lines[:close_index] + new_lines + lines[close_index:]

def _main_restart_flow_lines(self, *, context, process_paragraph):
 return [
 template.format(
 control=RESTART_PARAGRAPH_NAMES["control"],
 commit=RESTART_PARAGRAPH_NAMES["commit"],
 process_paragraph=process_paragraph,
 record=context.input_record_name,
 file_name=context.input_file.name,
)
 for template in MAIN_FLOW_RESTART_TEMPLATE_LINES
]

```

```

=====
FILE: src\idms_db2_phase2\postprocess\update_metadata_models.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import re
from dataclasses import dataclass, field
from typing import Any, Optional

```

```

def value(

```



```

 item: Any,
 *names: str,
 default: Any = None,
) -> Any:
 if item is None:
 return default

 if isinstance(item, dict):
 for name in names:
 item_value = item.get(name)
 if item_value not in (None, ""):
 return item_value

 for name in names:
 if hasattr(item, name):
 item_value = getattr(item, name)
 if item_value not in (None, ""):
 return item_value

 return default

def upper(value: Any) -> str:
 return str(value or "").strip().upper()

def norm_name(value: Any) -> str:
 text = upper(value)
 text = text.replace("_", "-")
 text = re.sub(r"^[A-Z0-9-]", "", text)
 return text

def sql_name(value: Any) -> str:
 text = upper(value)
 text = text.replace("-", "_")
 text = re.sub(r"^[A-Z0-9]", "", text)
 text = re.sub(r"_+", "_", text)
 return text.strip("_")

def compact(value: Any) -> str:
 return re.sub(r"^[A-Z0-9]", "", upper(value))

def include_from_label(source_label: str) -> str:
 if not source_label:
 return ""

 clean = source_label.replace("\\", "/").split("/")[-1]
 clean = clean.split(".")[0]
 return norm_name(clean)

@dataclass
class CobolFileInfo:
 name: str
 assignment: str = ""
 fd_record: str = ""
 record_length: int = 0
 read_target: str = ""

@dataclass
class CopybookRecordInfo:
 include_name: str
 record_name: str
 source_label: str = ""
 fields: set[str] = field(default_factory=set)
 estimated_length: int = 0

@dataclass
class DclgenRoleInfo:
 table_name: str
 include_name: str

```

```

host_record_name: str
program_column: str
phase_column: str
date_column: str
status_column: str
retention_column: str
payload_column: str
program_field: str
phase_field: str
date_field: str
status_field: str
retention_field: str
payload_group: str
payload_len_field: str
payload_text_field: str

```

```
@dataclass
```

```
class UpdateProgramContext:
```

```

 input_file: CobolFileInfo
 input_copybook: CopybookRecordInfo
 restart_dclgen: Optional[DclgenRoleInfo]
 program_id: str
 legacy_eof_switch: str = ""
 source_program_text: str = ""
 diagnostics: list[str] = field(default_factory=list)

```

```
@property
```

```

def input_record_name(self) -> str:
 return self.input_copybook.record_name or self.input_copybook.include_name

```

```
@property
```

```

def copybook_include_name(self) -> str:
 return self.input_copybook.include_name or self.input_record_name

```

```
@property
```

```

def record_length(self) -> int:
 if self.input_file.record_length:
 return self.input_file.record_length

 if self.input_copybook.estimated_length:
 return self.input_copybook.estimated_length

 return 1

```

```

=====
FILE: src\idms_db2_phase2\postprocess\update_postprocess_line_utils.py
ENCODING: utf-8
=====

```

```
from __future__ import annotations
```

```
import re
```

```

from patterns.update_restart_patterns import (
 COPY_INCLUDE_TOKEN_PATTERN_TEMPLATE,
 END_PROGRAM_PATTERN,
 LINKAGE_SECTION_PATTERN,
 PARAGRAPH_HEADER_PATTERN,
 PROCEDURE_DIVISION_PATTERN,
 WORKING_STORAGE_SECTION_PATTERN,
)

```

```
class UpdatePostprocessLineUtils:
```

```

 def lines(
 self,
 text: str,
) -> list[str]:
 return str(text or "").replace("\r\n", "\n").replace("\r", "\n").splitlines()

 def join(
 self,
 lines: list[str],
) -> str:

```

```

 return "\n".join(lines).rstrip() + "\n"

def logical(
 self,
 line: str,
) -> str:
 text = str(line or "").rstrip()

 if len(text) >= 72 and text[:6].strip().isdigit():
 return text[7:72].strip()

 if len(text) > 6 and text[:6].strip().isdigit():
 body = text[6:]

 if len(body) >= 8 and body[-8:].strip().isdigit():
 body = body[:-8]

 if body[:1] in ("*", "/", "D"):
 return body.strip()

 return body.strip()

 return text.strip()

def logical_text(
 self,
 cobol_text: str,
) -> str:
 return "\n".join(
 self.logical(line)
 for line in self.lines(cobol_text)
)

def logical_contains(
 self,
 cobol_text: str,
 expected: str,
) -> bool:
 target = str(expected or "").strip().upper()

 if not target:
 return False

 for line in self.lines(cobol_text):
 logical = self.logical(line).upper()

 if target in logical:
 return True

 return False

def contains_copy(
 self,
 cobol_text: str,
 include_name: str,
) -> bool:
 target = str(include_name or "").strip().upper()

 if not target:
 return False

 pattern = re.compile(
 COPY_INCLUDE_TOKEN_PATTERN_TEMPLATE.format(
 include_name=re.escape(target),
),
 flags=re.IGNORECASE,
)

 return bool(pattern.search(self.logical_text(cobol_text)))

def data_division_insert_line_index(
 self,
 lines: list[str],
) -> int:
 for index, line in enumerate(lines):
 logical = self.logical(line)

```

```

 if LINKAGE_SECTION_PATTERN.match(logical):
 return index

 for index, line in enumerate(lines):
 logical = self.logical(line)

 if PROCEDURE_DIVISION_PATTERN.match(logical):
 return index

 for index, line in enumerate(lines):
 logical = self.logical(line)

 if WORKING_STORAGE_SECTION_PATTERN.match(logical):
 return index + 1

 return -1

def procedure_paragraph_insert_line_index(
 self,
 lines: list[str],
) -> int:
 for index, line in enumerate(lines):
 logical = self.logical(line)

 if END_PROGRAM_PATTERN.match(logical):
 return index

 for line in lines:
 logical = self.logical(line)

 if PROCEDURE_DIVISION_PATTERN.match(logical):
 return len(lines)

 return -1

def find_paragraph_range(
 self,
 lines: list[str],
 paragraph_name: str,
) -> tuple[int, int] | None:
 target = str(paragraph_name or "").upper().rstrip(".")

 for index, line in enumerate(lines):
 logical = self.logical(line).upper().rstrip(".")

 if logical != target:
 continue

 end = index + 1

 while end < len(lines):
 next_logical = self.logical(lines[end]).strip()

 if end > index + 1 and PARAGRAPH_HEADER_PATTERN.match(next_logical):
 break

 end += 1

 return index, end

 return None

```

```

=====
FILE: src\idms_db2_phase2\postprocess\update_program_enhancer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.postprocess.cobol_update_standard_generator import (
 CobolUpdateStandardGenerator,
)
from idms_db2_phase2.postprocess.dynamic_metadata_resolver import (
 DynamicMetadataResolver,
)

```

```

)
from idms_db2_phase2.postprocess.update_business_sql_standardizer import (
 UpdateBusinessSqlStandardizer,
)
)
from idms_db2_phase2.postprocess.update_enhancement_result import (
 EnhancementResult,
)
)
from idms_db2_phase2.postprocess.update_main_flow_rewriter import (
 UpdateMainFlowRewriter,
)
)
from idms_db2_phase2.postprocess.update_postprocess_line_utils import (
 UpdatePostprocessLineUtils,
)
)
from idms_db2_phase2.postprocess.update_restart_paragraph_manager import (
 UpdateRestartParagraphManager,
)
)
from idms_db2_phase2.postprocess.update_storage_include_manager import (
 UpdateStorageIncludeManager,
)
)
from rules.update_restart_rules import UPDATE_RESTART_DIAGNOSTICS
from idms_db2_phase2.repositories.mapping_repository import MappingRepository # ADDED

```

```

class UpdateProgramEnhancer:

```

```

 """

```

```

 Update-program-only postprocess enhancer.

```

```

 This class is deliberately not used by retrieval.

```

```

 It uses metadata already parsed by the normal application flow and applies
 only update restart/input handling fixes.

```

```

 """

```

```

def __init__(self) -> None:

```

```

 self.resolver = DynamicMetadataResolver()

```

```

 self.generator = CobolUpdateStandardGenerator()

```

```

 self.line_utils = UpdatePostprocessLineUtils()

```

```

 self.storage_include_manager = UpdateStorageIncludeManager(
 generator=self.generator,
 line_utils=self.line_utils,
)

```

```

 self.main_flow_rewriter = UpdateMainFlowRewriter(
 line_utils=self.line_utils,
)

```

```

 self.restart_paragraph_manager = UpdateRestartParagraphManager(
 generator=self.generator,
 line_utils=self.line_utils,
)

```

```

 self.business_sql_standardizer = UpdateBusinessSqlStandardizer(
 line_utils=self.line_utils,
)

```

```

def enhance(

```

```

 self,

```

```

 *,

```

```

 source_cobol: str,

```

```

 converted_cobol: str,

```

```

 copybook_fields: list,

```

```

 dclgen_columns: list,

```

```

 target_program_id: str = "",

```

```

 sheet_mapping_rows: list | None = None, # ADDED: Sheet Mapping authority

```

```

) -> EnhancementResult:

```

```

 diagnostics: list[str] = [UPDATE_RESTART_DIAGNOSTICS["start"]]

```

```

 # ADDED: Sheet Mapping authority for the restart AND-gate.

```

```

 # Sheet Mapping is the authority for DB2 table names (authority_rules).

```

```

 # It is required to prevent DCLGEN-only fabrication of restart SQL

```

```

 # (Case 3). Defaults to an empty repository when rows are not supplied,

```

```

 # which keeps the AND-gate closed (safe skip) rather than fabricating.

```

```

 mapping_repository = MappingRepository(sheet_mapping_rows or [])

```

```

 try:

```

```

 context = self.resolver.resolve(

```

```

 source_cobol=source_cobol,
 generated_cobol=converted_cobol,
 copybook_fields=copybook_fields,
 dclgen_columns=dclgen_columns,
 target_program_id=target_program_id,
 mapping_repository=mapping_repository, # ADDED
)
except Exception as exc:
 diagnostics.append(
 f"{UPDATE_RESTART_DIAGNOSTICS['context_failed']} Reason: {exc}"
)
 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["end"])

 return EnhancementResult(
 converted_cobol=converted_cobol,
 diagnostics=diagnostics,
)

diagnostics.extend(context.diagnostics)

--- Split Case 2 vs Case 3 warnings -----
When restart_dclgen is None, no restart SQL is generated and the
legacy IDMS restart/control flow is preserved unchanged. We split
the diagnostic so operators can tell WHY it was skipped:
Case 3: DCLGEN restart group existed but Sheet Mapping did not
confirm the restart table (AND-gate blocked fabrication).
Case 2/4: restart DCLGEN itself was not resolved.
if getattr(context, "restart_dclgen", None) is None:
 has_dclgen_restart_candidate = bool(dclgen_columns)
 mapping_has_restart_hint = any(
 mapping_repository.has_table(t)
 for t in mapping_repository.tables()
)

 if has_dclgen_restart_candidate and mapping_has_restart_hint:
 # Case 3: DCLGEN present, but mapping did not confirm the table.
 diagnostics.append(
 "WARNING (Case 3): Restart flow NOT generated. A DCLGEN "
 "restart group was found, but the restart table is not "
 "confirmed in the Sheet Mapping. Legacy IDMS restart flow "
 "PRESERVED UNCHANGED. Add the restart table to the Sheet "
 "Mapping to auto-convert."
)
 else:
 # Case 2 / Case 4: restart DCLGEN itself was not resolved.
 diagnostics.append(
 "WARNING (Case 2/4): Restart-table DCLGEN not resolved. "
 "Legacy IDMS restart/control flow PRESERVED UNCHANGED and "
 "must be redesigned manually. Add the restart table to the "
 "DCLGEN inputs to auto-convert."
)

output = converted_cobol

output = self.storage_include_manager.fix_malformed_sqlerror(
 output,
 diagnostics,
)

output = self.storage_include_manager.normalize_program_name(
 output,
 context,
 diagnostics,
)

output = self.storage_include_manager.ensure_working_storage(
 output,
 context,
 diagnostics,
)

output = self.storage_include_manager.ensure_date_working_storage(
 output,
 diagnostics,
)

```

```

 output = self.storage_include_manager.ensure_dclgen_include(
 output,
 context,
 diagnostics,
)

 output = self.storage_include_manager.ensure_referenced_dclgen_includes(
 output,
 diagnostics,
)

 output = self.storage_include_manager.ensure_copybook_include(
 output,
 context,
 diagnostics,
)

 output = self.main_flow_rewriter.replace_legacy_restart_main_flow(
 output,
 context,
 diagnostics,
)

 output = self.restart_paragraph_manager.ensure_read_flat_file(
 output,
 context,
 diagnostics,
)

 output = self.restart_paragraph_manager.ensure_restart_paragraphs(
 output,
 context,
 diagnostics,
)

 output = self.business_sql_standardizer.standardize(
 output,
 context,
 diagnostics,
)

 output = self.storage_include_manager.remove_legacy_restart_working_storage(
 output,
 diagnostics,
)
 output = self.storage_include_manager.ensure_sqlerror_paragraph(# ADDED
 output, diagnostics,
)

 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["end"])

 return EnhancementResult(
 converted_cobol=output,
 diagnostics=diagnostics,
)

__all__ = [
 "EnhancementResult",
 "UpdateProgramEnhancer",
]

=====
FILE: src\idms_db2_phase2\postprocess\update_restart_dclgen_matcher.py
ENCODING: utf-8
=====

from __future__ import annotations

from typing import Optional

from idms_db2_phase2.postprocess.update_metadata_models import compact

class UpdateRestartDclgenMatcher:

```

"""

Finds restart columns or COBOL host fields using token matching.

This class contains generic matching logic only.

Business tokens are supplied from rules/update\_restart\_rules.py.

Important:

- Restart role matching must be strict enough to avoid resolving one role to another field.
- Example: NM-ID-RS-PROGRAM must not match status just because it contains ID and RS.
- Therefore all configured tokens must be present for a match.

"""

```
def find_optional_col(
 self,
 *,
 rows: list[dict[str, str]],
 tokens: list[str],
 fallback_tokens: Optional[list[str]] = None,
 exclude_tokens: Optional[list[str]] = None,
) -> str:
 try:
 return self.find_col(
 rows=rows,
 tokens=tokens,
 fallback_tokens=fallback_tokens,
 exclude_tokens=exclude_tokens,
)
 except ValueError:
 return ""
```

```
def find_optional_field(
 self,
 *,
 rows: list[dict[str, str]],
 tokens: list[str],
 fallback_tokens: Optional[list[str]] = None,
 exclude_tokens: Optional[list[str]] = None,
) -> str:
 try:
 return self.find_field(
 rows=rows,
 tokens=tokens,
 fallback_tokens=fallback_tokens,
 exclude_tokens=exclude_tokens,
)
 except ValueError:
 return ""
```

```
def find_col(
 self,
 *,
 rows: list[dict[str, str]],
 tokens: list[str],
 fallback_tokens: Optional[list[str]] = None,
 exclude_tokens: Optional[list[str]] = None,
) -> str:
 return self.find_by_key(
 rows=rows,
 key="column",
 tokens=tokens,
 fallback_tokens=fallback_tokens,
 exclude_tokens=exclude_tokens,
)
```

```
def find_field(
 self,
 *,
 rows: list[dict[str, str]],
 tokens: list[str],
 fallback_tokens: Optional[list[str]] = None,
 exclude_tokens: Optional[list[str]] = None,
) -> str:
 return self.find_by_key(
 rows=rows,
```



```

 key="field",
 tokens=tokens,
 fallback_tokens=fallback_tokens,
 exclude_tokens=exclude_tokens,
)

def find_by_key(
 self,
 *,
 rows: list[dict[str, str]],
 key: str,
 tokens: list[str],
 fallback_tokens: Optional[list[str]] = None,
 exclude_tokens: Optional[list[str]] = None,
) -> str:
 fallback_tokens = fallback_tokens or []
 exclude_tokens = exclude_tokens or []

 scored: list[tuple[int, int, str]] = []

 for index, row in enumerate(rows):
 value = str(row.get(key, "") or "").strip()

 if not value:
 continue

 searchable = self._searchable_row_text(row=row, primary_key=key)

 if self._has_excluded_token(
 searchable=searchable,
 exclude_tokens=exclude_tokens,
):
 continue

 score = self._match_score(
 searchable=searchable,
 tokens=tokens,
 fallback_tokens=fallback_tokens,
)

 if score > 0:
 scored.append((score, index, value))

 if not scored:
 raise ValueError(f"Unable to resolve restart field using tokens {tokens}")

 scored.sort(
 key=lambda item: (-item[0], item[1]),
)

 return scored[0][2]

def score_rows_for_restart(
 self,
 *,
 rows: list[dict[str, str]],
 token_sets: list[list[str]],
) -> int:
 score = 0

 for row in rows:
 searchable = self._searchable_row_text(row=row, primary_key="field")

 for tokens in token_sets:
 compact_tokens = [compact(token) for token in tokens if compact(token)]

 if self._token_set_found(
 searchable=searchable,
 tokens=tokens,
):
 score += len(compact_tokens)

 return score

def _searchable_row_text(
 self,

```

```

*,
row: dict[str, str],
primary_key: str,
) -> str:
 parts = [
 row.get(primary_key, ""),
 row.get("field", ""),
 row.get("column", ""),
 row.get("table_name", ""),
 row.get("include_name", ""),
 row.get("host_record", ""),
 row.get("source_label", ""),
]

 return compact(" ".join(str(part or "") for part in parts))

def _match_score(
 self,
 *,
 searchable: str,
 tokens: list[str],
 fallback_tokens: list[str],
) -> int:
 primary_tokens = [compact(token) for token in tokens if compact(token)]

 if self._token_set_found(
 searchable=searchable,
 tokens=tokens,
):
 return len(primary_tokens) * 10

 fallback_compact_tokens = [
 compact(token)
 for token in fallback_tokens
 if compact(token)
]

 if self._token_set_found(
 searchable=searchable,
 tokens=fallback_compact_tokens,
):
 return len(fallback_compact_tokens)

 return 0

def _token_set_found(
 self,
 *,
 searchable: str,
 tokens: list[str],
) -> bool:
 compact_tokens = [compact(token) for token in tokens if compact(token)]

 if not compact_tokens:
 return False

 return all(token in searchable for token in compact_tokens)

def _has_excluded_token(
 self,
 *,
 searchable: str,
 exclude_tokens: list[str],
) -> bool:
 compact_tokens = [compact(token) for token in exclude_tokens if compact(token)]

 if not compact_tokens:
 return False

 return any(token in searchable for token in compact_tokens)

```

```
=====
FILE: src\idms_db2_phase2\postprocess\update_restart_dclgen_resolver.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from typing import Any, Optional

from idms_db2_phase2.postprocess.update_metadata_models import (
 DclgenRoleInfo,
 upper,
)
from idms_db2_phase2.postprocess.update_restart_dclgen_matcher import (
 UpdateRestartDclgenMatcher,
)
from idms_db2_phase2.postprocess.update_restart_dclgen_role_builder import (
 UpdateRestartDclgenRoleBuilder,
)
from idms_db2_phase2.postprocess.update_restart_dclgen_row_mapper import (
 UpdateRestartDclgenRowMapper,
)

Substrings that identify a restart/control table by its DB2 table name.
The restart table is a known DB2 object (e.g. DZ01RSTV / DZ01RSTB); its
name always contains the "RST" control marker. This is a deterministic,
rule-based identifier – no fuzzy scoring, no threshold.
RESTART_TABLE_NAME_HINTS = ("RST", "RESTART")

class UpdateRestartDclgenResolver:
 """
 Selects the restart DCLGEN group by TABLE NAME (deterministic) and
 delegates role-building.

 Rule-based design:
 - The DCLGEN parser already resolves the DB2 table name for each group.
 - The restart table is identified by its name (contains 'RST'/'RESTART').
 - No token scoring and no MIN_SCORE threshold are used.

 This class does not hardcode a specific restart table name; it matches the
 generic restart-control naming marker, so any DZxxRSTx table is detected.
 """

 def __init__(
 self,
 *,
 row_mapper: UpdateRestartDclgenRowMapper | None = None,
 role_builder: UpdateRestartDclgenRoleBuilder | None = None,
 matcher: UpdateRestartDclgenMatcher | None = None,
) -> None:
 self.row_mapper = row_mapper or UpdateRestartDclgenRowMapper()
 self.matcher = matcher or UpdateRestartDclgenMatcher()
 self.role_builder = role_builder or UpdateRestartDclgenRoleBuilder(
 row_mapper=self.row_mapper,
 matcher=self.matcher,
)

 def resolve_restart_dclgen(
 self,
 dclgen_columns: list[Any],
 mapping_repository: Any | None = None, # kept for signature compatibility
) -> Optional[DclgenRoleInfo]:
 groups = self._group_columns_by_dclgen(dclgen_columns)

 restart_group = self._find_restart_group_by_table_name(groups)
 if not restart_group:
 return None

 group_key, columns = restart_group
 return self.role_builder.build_restart_roles(
 group_key=group_key,
 columns=columns,
)

 def _group_columns_by_dclgen(
```

```

 self,
 dclgen_columns: list[Any],
) -> dict[str, list[Any]]:
 """Group parsed DCLGEN columns by their resolved DB2 table name."""
 groups: dict[str, list[Any]] = {}

 for item in dclgen_columns:
 row = self.row_mapper.column_row(item)
 group_key = upper(row.get("table_name", ""))
 if not group_key:
 continue
 groups.setdefault(group_key, []).append(item)

 return groups

 def _find_restart_group_by_table_name(
 self,
 groups: dict[str, list[Any]],
) -> Optional[tuple[str, list[Any]]]:
 """Deterministically pick the restart DCLGEN group by table name.

 A group IS the restart table when its DB2 table name contains a
 restart-control marker ('RST' / 'RESTART'). No scoring, no threshold.
 """
 for group_key, columns in groups.items():
 table_name = upper(group_key)
 if any(hint in table_name for hint in RESTART_TABLE_NAME_HINTS):
 return group_key, columns

 return None

__all__ = [
 "UpdateRestartDclgenResolver",
 "RESTART_TABLE_NAME_HINTS",
]

```

```

=====
FILE: src\idms_db2_phase2\postprocess\update_restart_dclgen_role_builder.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from typing import Any

from catalogs.dclgen_schema import DCLGEN_GROUP_PREFIX
from idms_db2_phase2.postprocess.restart_role.restart_role_factory import (
 RestartRoleFactory,
)
from idms_db2_phase2.postprocess.restart_role.restart_role_field_resolver import (
 RestartRoleFieldResolver,
)
from idms_db2_phase2.postprocess.update_metadata_models import DclgenRoleInfo
from idms_db2_phase2.postprocess.update_restart_dclgen_matcher import (
 UpdateRestartDclgenMatcher,
)
from idms_db2_phase2.postprocess.update_restart_dclgen_row_mapper import (
 UpdateRestartDclgenRowMapper,
)
from rules.update_restart_rules import (
 RESTART_PAYLOAD_FALLBACK_TOKEN_KEY,
 RESTART_PAYLOAD_LENGTH_SPEC,
 RESTART_PAYLOAD_TEXT_SPEC,
 RESTART_PAYLOAD_TOKEN_KEY,
 RESTART_ROLE_COLUMN_FIELD_SPEC,
)

class UpdateRestartDclgenRoleBuilder(
 RestartRoleFieldResolver,
 RestartRoleFactory,
):
 """Builds DclgenRoleInfo from the selected restart DCLGEN column group.

```

Generic: no restart table names, DCLGEN names, or host variable names are hardcoded. VARCHAR LEN/TEXT child fields are derived from the resolved payload group. Field/column resolution and role construction are provided by mixins; the role-key spec lives in rules/update\_restart\_rules.py.

```

"""
def __init__(
 self,
 *,
 row_mapper: UpdateRestartDclgenRowMapper | None = None,
 matcher: UpdateRestartDclgenMatcher | None = None,
) -> None:
 self.row_mapper = row_mapper or UpdateRestartDclgenRowMapper()
 self.matcher = matcher or UpdateRestartDclgenMatcher()

def build_restart_roles(
 self,
 *,
 group_key: str,
 columns: list[Any],
) -> DclgenRoleInfo:
 rows = [self.row_mapper.column_row(item) for item in columns]

 table_name = self.row_mapper.first_value(rows, "table_name") or group_key
 include_name = (
 self.row_mapper.first_value(rows, "include_name") or table_name
)
 host_record = self.row_mapper.first_value(rows, "host_record")
 if not host_record and table_name:
 host_record = f"{DCLGEN_GROUP_PREFIX}{table_name}"

 payload_group = self._resolve_payload_group(rows)
 payload_len_field = self._resolve_payload_child_field(
 rows=rows,
 payload_group=payload_group,
 token_key=RESTART_PAYLOAD_LENGTH_SPEC[0],
 fallback_token_key=RESTART_PAYLOAD_LENGTH_SPEC[1],
 suffix_key=RESTART_PAYLOAD_LENGTH_SPEC[2],
)
 payload_text_field = self._resolve_payload_child_field(
 rows=rows,
 payload_group=payload_group,
 token_key=RESTART_PAYLOAD_TEXT_SPEC[0],
 fallback_token_key=RESTART_PAYLOAD_TEXT_SPEC[1],
 suffix_key=RESTART_PAYLOAD_TEXT_SPEC[2],
)

 # Resolve every (column, field) role pair from the spec in one loop.
 resolved: dict[str, str] = {}
 for role, (token_key, fallback_key) in RESTART_ROLE_COLUMN_FIELD_SPEC.items():
 resolved[f"{role}_col"] = self._find_column(
 rows=rows, token_key=token_key, fallback_token_key=fallback_key
)
 resolved[f"{role}_field"] = self._find_field(
 rows=rows, token_key=token_key, fallback_token_key=fallback_key
)

 payload_col = self._find_column(
 rows=rows,
 token_key=RESTART_PAYLOAD_TOKEN_KEY,
 fallback_token_key=RESTART_PAYLOAD_FALLBACK_TOKEN_KEY,
)

 return self._make_role(
 table_name=table_name,
 include_name=include_name,
 host_record_name=host_record,
 host_record=host_record,
 program_column=resolved["program_col"],
 program_col=resolved["program_col"],
 program_field=resolved["program_field"],
 phase_column=resolved["phase_col"],
 phase_col=resolved["phase_col"],
 phase_field=resolved["phase_field"],
 date_column=resolved["date_col"],
 date_col=resolved["date_col"],

```

```

 date_field=resolved["date_field"],
 status_column=resolved["status_col"],
 status_col=resolved["status_col"],
 status_field=resolved["status_field"],
 retention_column=resolved["retention_col"],
 retention_col=resolved["retention_col"],
 retention_field=resolved["retention_field"],
 payload_column=payload_col,
 payload_col=payload_col,
 payload_group=payload_group,
 payload_group_field=payload_group,
 payload_len_field=payload_len_field,
 payload_length_field=payload_len_field,
 payload_text_field=payload_text_field,
)

```

```

=====
FILE: src\idms_db2_phase2\postprocess\update_restart_dclgen_row_mapper.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from typing import Any

```

```

from catalogs.dclgen_schema import DCLGEN_GROUP_PREFIX
from idms_db2_phase2.postprocess.update_metadata_models import (
 include_from_label,
 upper,
 value,
)

```

```

class UpdateRestartDclgenRowMapper:

```

```

 """

```

```

 Converts parsed DCLGEN column objects into normalized row dictionaries.

```

```

 This class does not decide restart roles.

```

```

 It only normalizes metadata from parser output into consistent keys used
 by the restart resolver.

```

```

 The DCLGEN parser commonly exposes:

```

```

 - table_name
 - column_name
 - cobol_host_name
 - cobol_picture
 - cobol_usage

```

```

 This mapper must therefore read cobol_host_name as the COBOL host field.

```

```

 """

```

```

 def column_row(
 self,
 item: Any,
) -> dict[str, str]:
 table_name = upper(
 value(
 item,
 "table_name",
 "db2_table_name",
 "table",
 default="",
)
)

```

```

 source_label = value(
 item,
 "source_label",
 "file_name",
 "filename",
 default="",
)

```

```

 include_name = upper(
 value(

```

```

 item,
 "include_name",
 "dclgen_name",
 "dclgen_include",
 default="",
)
)

if not include_name:
 include_name = upper(include_from_label(source_label))

column_name = upper(
 value(
 item,
 "column_name",
 "db2_column_name",
 "column",
 "name",
 default="",
)
)

cobol_host_name = upper(
 value(
 item,
 "cobol_host_name",
 "host_variable",
 "host_name",
 "host_field",
 "field",
 "cobol_name",
 default="",
)
)

host_record = upper(
 value(
 item,
 "host_record_name",
 "host_record",
 "host_group",
 "cobol_host_group",
 "cobol_group_name",
 "dclgen_record_name",
 "dclgen_record",
 "record_name",
 default="",
)
)

if not host_record and table_name:
 host_record = f"{DCLGEN_GROUP_PREFIX}{table_name}"

cobol_picture = upper(
 value(
 item,
 "cobol_picture",
 "picture",
 "pic",
 default="",
)
)

cobol_usage = upper(
 value(
 item,
 "cobol_usage",
 "usage",
 default="",
)
)

return {
 "table_name": table_name,
 "include_name": include_name,
 "host_record": host_record,

```

```

 "column": column_name,
 "field": cobol_host_name,
 "picture": cobol_picture,
 "usage": cobol_usage,
 "source_label": upper(source_label),
 }

def first_value(
 self,
 rows: list[dict[str, str]],
 key: str,
) -> str:
 for row in rows:
 text = str(row.get(key, "") or "").strip()
 if text:
 return text

 return ""

=====
FILE: src\idms_db2_phase2\postprocess\update_restart_paragraph_manager.py
ENCODING: utf-8
=====

from __future__ import annotations

from idms_db2_phase2.postprocess.cobol_update_standard_generator import (
 CobolUpdateStandardGenerator,
)
from idms_db2_phase2.postprocess.dynamic_metadata_resolver import UpdateProgramContext
from idms_db2_phase2.postprocess.update_postprocess_line_utils import (
 UpdatePostprocessLineUtils,
)
from rules.update_restart_rules import (
 RESTART_PARAGRAPH_NAMES,
 UPDATE_RESTART_DIAGNOSTICS,
 UPDATE_RESTART_WS_NAMES,
)

class UpdateRestartParagraphManager:
 def __init__(
 self,
 *,
 generator: CobolUpdateStandardGenerator,
 line_utils: UpdatePostprocessLineUtils,
) -> None:
 self.generator = generator
 self.line_utils = line_utils

 def ensure_read_flat_file(
 self,
 cobol_text: str,
 context: UpdateProgramContext,
 diagnostics: list[str],
) -> str:
 switch_group = UPDATE_RESTART_WS_NAMES["switch_group"]

 if not self.line_utils.logical_contains(
 cobol_text,
 f"01 {switch_group}.",
):
 diagnostics.append(
 "WS-SWITCHES not present. "
 "Skipped READ-FLAT-FILE enhancement."
)
 return cobol_text

 paragraph = self.generator.read_flat_file(context)
 lines = self.line_utils.lines(cobol_text)

 existing_range = self.line_utils.find_paragraph_range(
 lines,
 "READ-FLAT-FILE",
)

```



```

if existing_range:
 start, end = existing_range
 updated_lines = lines[:start] + paragraph.splitlines() + [""] + lines[end:]
 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["read_replaced"])
 return self.line_utils.join(updated_lines)

insert_index = self.line_utils.procedure_paragraph_insert_line_index(lines)

if insert_index < 0:
 diagnostics.append(
 "Safe PROCEDURE DIVISION insertion point not found. "
 "Skipped READ-FLAT-FILE insertion."
)
 return cobol_text

updated_lines = (
 lines[:insert_index]
 + [""]
 + paragraph.splitlines()
 + [""]
 + lines[insert_index:]
)
diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["read_added"])
return self.line_utils.join(updated_lines)

def ensure_restart_paragraphs(
 self,
 cobol_text: str,
 context: UpdateProgramContext,
 diagnostics: list[str],
) -> str:
 if not context.restart_dclgen:
 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["restart_missing"])
 return cobol_text

 switch_group = UPDATE_RESTART_WS_NAMES["switch_group"]
 help_group = UPDATE_RESTART_WS_NAMES["help_group"]

 if not self.line_utils.logical_contains(
 cobol_text,
 f"01 {switch_group}.",
) or not self.line_utils.logical_contains(
 cobol_text,
 f"01 {help_group}.",
):
 diagnostics.append(
 "Restart working-storage not present. "
 "Skipped restart paragraph injection."
)
 return cobol_text

 restart_code = self.generator.restart_paragraphs(context)

 if not restart_code.strip():
 diagnostics.append("Restart generator produced no code. Skipped restart injection.")
 return cobol_text

 lines = self.line_utils.lines(cobol_text)

 updated_lines = self._remove_existing_restart_paragraphs(
 lines=lines,
)

 insert_index = self.line_utils.procedure_paragraph_insert_line_index(
 updated_lines
)

 if insert_index < 0:
 diagnostics.append(
 "Safe PROCEDURE DIVISION insertion point not found. "
 "Skipped restart paragraph injection."
)
 return cobol_text

 final_lines = (

```

```

 updated_lines[:insert_index]
 + [""]
 + restart_code.splitlines()
 + [""]
 + updated_lines[insert_index:]
)

 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["restart_inserted"])
 return self.line_utils.join(final_lines)

def _remove_existing_restart_paragraphs(
 self,
 *,
 lines: list[str],
) -> list[str]:
 paragraph_names = [
 RESTART_PARAGRAPH_NAMES["control"],
 RESTART_PARAGRAPH_NAMES["restart_found"],
 RESTART_PARAGRAPH_NAMES["write_restart"],
 RESTART_PARAGRAPH_NAMES["commit"],
 RESTART_PARAGRAPH_NAMES["select"],
 RESTART_PARAGRAPH_NAMES["update"],
 RESTART_PARAGRAPH_NAMES["insert"],
 RESTART_PARAGRAPH_NAMES["abend"],
]

 updated = list(lines)

 for paragraph_name in paragraph_names:
 existing_range = self.line_utils.find_paragraph_range(
 updated,
 paragraph_name,
)

 if not existing_range:
 continue

 start, end = existing_range
 updated = updated[:start] + updated[end:]

 return updated

```

```

=====
FILE: src\idms_db2_phase2\postprocess\update_storage_include_manager.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/postprocess/update_storage_include_manager.py
ACTION: REPLACE ENTIRE FILE

from __future__ import annotations

import re

from idms_db2_phase2.generators.sql_error_generator import SqlErrorGenerator
from idms_db2_phase2.postprocess.cobol_update_standard_generator import (
 CobolUpdateStandardGenerator,
)
from idms_db2_phase2.postprocess.storage_include.storage_include_injector import (
 StorageIncludeInjector,
)
from idms_db2_phase2.postprocess.storage_include.storage_ws_injector import (
 StorageWsInjector,
)
from idms_db2_phase2.postprocess.update_postprocess_line_utils import (
 UpdatePostprocessLineUtils,
)
from patterns.update_storage_include_patterns import (
 LEGACY_77_DECLARATION_PATTERN_TEMPLATE,
)
from rules.update_restart_rules import (
 UPDATE_LEGACY_RESTART_WS_NAMES,
 UPDATE_RESTART_DIAGNOSTICS,
)
from rules.update_standard_generator_templates import (

```

```

MSG_SQLERROR_FROM_COPYBOOK,
MSG_SQLERROR_INJECTED,
SQL_ERROR_INCLUDE_TOKEN,
SQLERROR_PARAGRAPH_TEMPLATE,
USE_SHARED_SQL_ERROR_GENERATOR,
)

Self-contained legacy abend-marker patterns.
#
CORRECTION - two of these patterns could never match.
#
They previously read:
#
PIC\s+X$\d+$
#
'$' is the END-OF-STRING ANCHOR, not a literal parenthesis. "X$\d+$"
means "literal X, then end of string, then digits, then end of string",
which is unsatisfiable. The intended expression is "X$\d+$", i.e.
PIC X(08).
#
The corruption came from a PDF/OCR round trip, and the comment that
claimed "with correct $\d+$ escaping" carried it forward. Cases (b),
(d) and (e) in remove_legacy_restart_working_storage therefore never
fired, and legacy abend markers were silently left in the output.
#
All patterns match the SEQUENCE-STRIPPED logical line.

_ABEND_ONELINE = re.compile(
 r"^\s*77\s+[A-Z0-9-]+\s+PIC\s+X$\d+$\s+VALUE\s+'##&&[^']*'\s*\.\?\s*$",
 flags=re.IGNORECASE,
)
_ABEND_VALUE_LINE = re.compile(
 r"^\s*VALUE\s+'##&&[^']*'\s*\.\?\s*$",
 flags=re.IGNORECASE,
)
_ABEND_77_HEADER_NAMED = re.compile(
 r"^\s*77\s+(?P<name>[A-Z0-9-]+)\s+PIC\s+X$\d+$\s*\.\?\s*$",
 flags=re.IGNORECASE,
)

SQLERROR detection, matched against the whole program text.
_PERFORM_SQLERROR = re.compile(
 r"\bPERFORM\s+SQLERROR\b",
 flags=re.IGNORECASE,
)
_SQLERROR_HEADER = re.compile(
 r"^\s*(?:\{6\}\s*)?SQLERROR\s*\.\s*(?:\{8\})?\s*$",
 flags=re.IGNORECASE | re.MULTILINE,
)

Fixed-format geometry, used by the self-contained _logical helper.
_LEFT_SEQUENCE_WIDTH = 6
_RIGHT_SEQUENCE_START = 72
_RIGHT_SEQUENCE_END = 80
_FIXED_LINE_WIDTH = 80

class UpdateStorageIncludeManager(
 StorageWsInjector,
 StorageIncludeInjector,
):
 """Manages update-program storage and include injection.

 Orchestration + legacy WS removal live here. Working-storage/program-name
 passes and include injection are provided by mixins.
 """

 def __init__(
 self,
 *,
 generator: CobolUpdateStandardGenerator,
 line_utils: UpdatePostprocessLineUtils,
) -> None:
 self.generator = generator
 self.line_utils = line_utils

```

```

Self-contained helpers (do not rely on line_utils internals).

@staticmethod
def _logical(line: str) -> str:
 """Strip the fixed-format sequence areas, then strip whitespace.

 CORRECTION - the right-sequence test was content-based.

 It read:

 if len(text) >= 8 and text[-8:].strip().isdigit():

 which removes the last eight characters of ANY line ending in
 eight digits, including a genuine statement such as

 MOVE 12345678 TO WS-FIELD

 The test is now COLUMN-based: columns 73-80 are only treated as a
 right sequence on a line that is actually 80 columns wide.
 """
 text = str(line or "").rstrip("\n")

 if (
 len(text) >= _FIXED_LINE_WIDTH
 and text[_RIGHT_SEQUENCE_START:_RIGHT_SEQUENCE_END].isdigit()
):
 text = text[:_RIGHT_SEQUENCE_START]

 if (
 len(text) >= _LEFT_SEQUENCE_WIDTH
 and text[:_LEFT_SEQUENCE_WIDTH].isdigit()
):
 return text[_LEFT_SEQUENCE_WIDTH:].strip()

 # Free-form fallback: a leading run of six digits after indent.
 stripped = text.lstrip()
 if (
 len(stripped) >= _LEFT_SEQUENCE_WIDTH
 and stripped[:_LEFT_SEQUENCE_WIDTH].isdigit()
):
 return stripped[_LEFT_SEQUENCE_WIDTH:].strip()

 return text.strip()

def _field_reference_count(self, field_name: str, cobol_text: str) -> int:
 pattern = re.compile(
 r"(?![A-Z0-9-])" + re.escape(field_name) + r"(?![A-Z0-9-])",
 flags=re.IGNORECASE,
)
 return len(pattern.findall(cobol_text))

Legacy restart / abend-marker WS removal.

def remove_legacy_restart_working_storage(self, cobol_text, diagnostics):
 cobol_text = str(cobol_text or "")
 lines = cobol_text.replace("\r\n", "\n").replace("\r", "\n").split("\n")

 output: list[str] = []
 removed = False

 legacy_patterns = [
 re.compile(
 LEGACY_77_DECLARATION_PATTERN_TEMPLATE.format(
 name=re.escape(name)
),
 flags=re.IGNORECASE,
)
 for name in UPDATE_LEGACY_RESTART_WS_NAMES
]

 index = 0
 total = len(lines)

```

```

while index < total:
 line = lines[index]
 logical = self._logical(line)

 # (a) legacy named 77 fields (CTR-REC, SW-EOF, SW-RECAP)
 if any(pattern.match(logical) for pattern in legacy_patterns):
 removed = True
 index += 1
 continue

 # (b) single-line abend marker (header + ##&& value together)
 if _ABEND_ONELINE.match(logical):
 removed = True
 index += 1
 continue

 # (c) orphaned ##&& VALUE line on its own
 if _ABEND_VALUE_LINE.match(logical):
 removed = True
 index += 1
 continue

 # (d)+(e) a "77 name PIC X(nn)" header with NO inline VALUE.
 named = _ABEND_77_HEADER_NAMED.match(logical)
 if named:
 field_name = named.group("name")

 # (d) wrapped: next non-blank logical line is a ##&& VALUE.
 look = index + 1
 while look < total and not self._logical(lines[look]).strip():
 look += 1

 if look < total and _ABEND_VALUE_LINE.match(
 self._logical(lines[look])
):
 removed = True
 index = look + 1 # drop header .. value inclusive
 continue

 # (e) dead orphan: value already stripped in a prior run.
 # Remove only if the field name is unused elsewhere.
 if self._field_reference_count(field_name, cobol_text) <= 1:
 removed = True
 index += 1
 continue

 output.append(line)
 index += 1

if removed and diagnostics is not None:
 diagnostics.append(UPDATE_RESTART_DIAGNOSTICS["legacy_ws_removed"])

return "\n".join(output).rstrip() + "\n"

```

```

SQLERROR routine

```

```

def ensure_sqlerror_paragraph(self, cobol_text, diagnostics=None):
 """Append the SQLERROR routine once, if the program needs one.

```

CORRECTION - this injected the WRONG body, and injected it even when the copybook already supplied the routine.

The method looked only for a local 'SQLERROR.' paragraph header. Under the site standard there is none: the routine arrives through

```

EXEC SQL
 INCLUDE SQLERROR
END-EXEC.

```

so the header was correctly absent, and this method then appended the legacy DISPLAY / CALL USERABEN body on top of it. That duplicated the copybook routine and made the update output diverge from the retrieval output. CHK-05.08 reported it.

Two changes:

```

- the INCLUDE form now counts as "already present";
- the body comes from the shared SqlErrorGenerator, so both
 program families emit one routine.
"""
text = str(cobol_text or "")

Only needed if something calls PERFORM SQLERROR.
if not _PERFORM_SQLERROR.search(text):
 return text

A local paragraph header already defines it.
if _SQLERROR_HEADER.search(text):
 return text

The copybook already supplies it. Appending a second body here
is what produced the duplicate routine.
if SQL_ERROR_INCLUDE_TOKEN.upper() in text.upper():
 if diagnostics is not None:
 diagnostics.append(MSG_SQLERROR_FROM_COPYBOOK)
 return text

lines = text.replace("\r\n", "\n").replace("\r", "\n").split("\n")

Drop trailing blank lines so the block appends cleanly.
while lines and not lines[-1].strip():
 lines.pop()

lines.append("")
lines.extend(self._sql_error_lines())

if diagnostics is not None:
 diagnostics.append(MSG_SQLERROR_INJECTED)

return "\n".join(lines).rstrip() + "\n"

def _sql_error_lines(self) -> list[str]:
 """The SQLERROR routine body, from the single shared generator.

 SQLERROR_PARAGRAPH_TEMPLATE is retained only as a fallback, so a
 site that has not yet adopted the copybook form can switch back
 by setting USE_SHARED_SQL_ERROR_GENERATOR to False.
 """
 if USE_SHARED_SQL_ERROR_GENERATOR:
 return list(SqlErrorGenerator().paragraph_lines())

 return list(SQLERROR_PARAGRAPH_TEMPLATE)

```

```

=====
FILE: src\idms_db2_phase2\repositories__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\repositories\copybook_repository.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.domain.models import CopybookField
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class CopybookRepository:
 """
 Repository wrapper for optional copybook fields.
 """

 def __init__(
 self,
 fields: list[CopybookField] | None = None,
) -> None:
 self.fields = fields or []

```

```

def all(
 self,
) -> list[CopybookField]:
 return list(self.fields)

def count(
 self,
) -> int:
 return len(self.fields)

def names(
 self,
) -> list[str]:
 values = {
 NameNormalizer.normalize(field.name)
 for field in self.fields
 if NameNormalizer.normalize(field.name)
 }

 return sorted(values)

def find(
 self,
 field_name: str,
) -> CopybookField | None:
 target = NameNormalizer.normalize(field_name)

 if not target:
 return None

 for field in self.fields:
 if NameNormalizer.normalize(field.name) == target:
 return field

 return None

def picture_for(
 self,
 field_name: str,
) -> str:
 field = self.find(field_name)

 if field is None:
 return ""

 return field.picture

def usage_for(
 self,
 field_name: str,
) -> str:
 field = self.find(field_name)

 if field is None:
 return ""

 return field.usage

def occurs_for(
 self,
 field_name: str,
) -> str:
 field = self.find(field_name)

 if field is None:
 return ""

 return field.occurs

def has_field(
 self,
 field_name: str,
) -> bool:
 return self.find(field_name) is not None

```

```
=====
FILE: src\idms_db2_phase2\repositories\dclgen_repository.py
ENCODING: utf-8
=====
```

```
from idms_db2_phase2.domain.models import DclgenColumn
from idms_db2_phase2.services.name_normalizer import NameNormalizer
```

```
class DclgenRepository:
 """
 Repository wrapper for DCLGEN columns.

 DCLGEN is the authority for COBOL host-variable spelling, group names,
 and PIC clauses.
 """

 def __init__(
 self,
 columns: list[DclgenColumn] | None = None,
) -> None:
 self.columns = columns or []

 def all(
 self,
) -> list[DclgenColumn]:
 return list(self.columns)

 def count(
 self,
) -> int:
 return len(self.columns)

 def tables(
 self,
) -> list[str]:
 values = {
 NameNormalizer.normalize(column.table_name)
 for column in self.columns
 if NameNormalizer.normalize(column.table_name)
 }

 return sorted(values)

 def columns_for_table(
 self,
 table_name: str,
) -> list[DclgenColumn]:
 target = NameNormalizer.normalize(table_name)

 if not target:
 return []

 return [
 column
 for column in self.columns
 if NameNormalizer.normalize(column.table_name) == target
]

 def column_names_for_table(
 self,
 table_name: str,
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for column in self.columns_for_table(table_name):
 name = NameNormalizer.normalize(column.column_name)

 if not name:
 continue

 if name in seen:
 continue
```



```

 seen.add(name)
 output.append(name)

 return output

def has_table(
 self,
 table_name: str,
) -> bool:
 return bool(self.columns_for_table(table_name))

def has_column(
 self,
 table_name: str,
 column_name: str,
) -> bool:
 target_column = NameNormalizer.normalize(column_name)

 if not target_column:
 return False

 return target_column in self.column_names_for_table(table_name)

def host_for_column(
 self,
 table_name: str,
 column_name: str,
) -> str:
 target_column = NameNormalizer.normalize(column_name)

 if not target_column:
 return ""

 for column in self.columns_for_table(table_name):
 if NameNormalizer.normalize(column.column_name) == target_column:
 return NameNormalizer.to_cobol(
 column.cobol_host_name or column.column_name
)

 return ""

def group_for_table(
 self,
 table_name: str,
) -> str:
 table = NameNormalizer.normalize(table_name)

 if not table:
 return ""

 return "DCL" + NameNormalizer.to_cobol(table)

def host_reference_for_column(
 self,
 table_name: str,
 column_name: str,
) -> str:
 group = self.group_for_table(table_name)
 host = self.host_for_column(
 table_name=table_name,
 column_name=column_name,
)

 if not group or not host:
 return ""

 return f":{host} OF {group}"

def valid_host_references(
 self,
) -> set[str]:
 output: set[str] = set()

 for column in self.columns:
 table = NameNormalizer.normalize(column.table_name)
 group = self.group_for_table(table)

```

```

 host = NameNormalizer.to_cobol(
 column.cobol_host_name or column.column_name
)

 if group and host:
 output.add(f"{group}.{host}")
 output.add(f"{group} {host}")
 output.add(host)

 return output

```

```

=====
FILE: src\idms_db2_phase2\repositories\lrf_repository.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/repositories/lrf_repository.py
ACTION: CREATE NEW FILE
"""Read-only lookups over parsed LRF metadata."""

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.domain.models import LogicalRecord, LrfPath
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.lrf_rules import PATH_GROUP_ERASE, PATH_GROUP_OBTAIN

```

```

class LrfRepository:
 def __init__(self, logical_records: list[LogicalRecord] | None = None) -> None:
 self._records = list(logical_records or [])
 self._by_name = {
 NameNormalizer.normalize(r.logical_record_name): r
 for r in self._records
 }

 def is_empty(self) -> bool:
 return not self._records

 def names(self) -> list[str]:
 return [r.logical_record_name for r in self._records]

 def get(self, logical_record_name: str) -> LogicalRecord | None:
 return self._by_name.get(NameNormalizer.normalize(logical_record_name))

 def element_records(self, logical_record_name: str) -> list[str]:
 record = self.get(logical_record_name)
 return list(record.element_records) if record else []

 def path_for_keyword(self, keyword: str) -> LrfPath | None:
 wanted = NameNormalizer.normalize(keyword)
 for record in self._records:
 for path in record.paths:
 if NameNormalizer.normalize(path.keyword) == wanted:
 return path
 return None

 def obtain_paths(self, logical_record_name: str) -> list[LrfPath]:
 return self._paths(logical_record_name, PATH_GROUP_OBTAIN)

 def erase_paths(self, logical_record_name: str) -> list[LrfPath]:
 return self._paths(logical_record_name, PATH_GROUP_ERASE)

 def owns_record(self, record_name: str) -> list[str]:
 """Which logical records include this IDMS record as an element."""
 wanted = NameNormalizer.normalize(record_name)
 return [
 r.logical_record_name
 for r in self._records
 if any(NameNormalizer.normalize(e) == wanted for e in r.element_records)
]

 def _paths(self, logical_record_name: str, verb: str) -> list[LrfPath]:
 record = self.get(logical_record_name)
 if not record:
 return []

```

```
return [p for p in record.paths if p.path_group_verb == verb]
```

```
=====
FILE: src\idms_db2_phase2\repositories\mapping_column_resolver.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/repositories/mapping_column_resolver.py
ACTION: CREATE NEW FILE
```

```
"""
```

```
Sheet Mapping column resolution.
```

```
Deterministic source-field -> DB2 column resolution using direct row
pairing (COBOL-side identifier -> new_db2_field_name), plus audit and
update-candidate column logic. No fuzzy / similarity matching.
"""
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.repositories.mapping_key_resolver import (
 MappingKeyResolver,
)
from idms_db2_phase2.repositories.mapping_row_query import MappingRowQuery
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.field_naming_rules import (
 INSERT_ONLY_AUDIT_PREFIXES,
 UPDATE_AUDIT_PREFIXES,
 canonical_field_key,
)
```

```
class MappingColumnResolver:
```

```
 def __init__(
 self,
 query: MappingRowQuery,
 key_resolver: MappingKeyResolver,
) -> None:
 self.query = query
 self.key_resolver = key_resolver
```

```
 def column_for_source_field(
 self,
 record_name: str,
 source_field_name: str,
) -> str:
```

```
 """
 Resolve a COBOL source field to its DB2 column via the Sheet Mapping
 row directly. Each row pairs the COBOL-side identifier with the DB2
 column, so this is an exact canonical-key lookup. Deterministic only.
 """
```

```
 source_key = canonical_field_key(source_field_name)
 if not source_key:
 return ""
```

```
 for row in self.query.rows_for_record(record_name):
 column = NameNormalizer.normalize(
 row.new_db2_field_name or row.cross_application_db2_field_name
)
 if not column:
 continue
```

```
 for candidate in (
 row.cobol_zone,
 row.reference_field_name_copybook,
 row.new_db2_field_name,
 row.cross_application_db2_field_name,
):
 if canonical_field_key(candidate) == source_key:
 return column
```

```
 # No deterministic mapping found. Never guess.
 return ""
```

```
 def update_audit_columns_for_record(
```

```

 self,
 record_name: str,
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for row in self.query.rows_for_record(record_name):
 column = NameNormalizer.normalize(row.new_db2_field_name)
 if not column:
 continue
 if column.startswith(INSERT_ONLY_AUDIT_PREFIXES):
 continue
 if not column.startswith(UPDATE_AUDIT_PREFIXES):
 continue
 if column in seen:
 continue
 seen.add(column)
 output.append(column)

 return output

 def update_candidate_columns_for_record(
 self,
 record_name: str,
 changed_source_fields: list[str] | None = None,
) -> list[str]:
 """
 Return conservative UPDATE candidate columns.

 - Resolve changed source fields through Sheet Mapping to DB2 columns.
 - Exclude composite key columns.
 - Exclude TS_CREATE (insert-only).
 - Add update audit columns only if present in Sheet Mapping.
 - Do not invent DB2 columns.
 """
 output: list[str] = []
 seen: set[str] = set()

 key_columns = set(
 self.key_resolver.primary_key_columns_for_record(record_name)
)

 for source_field in changed_source_fields or []:
 column = NameNormalizer.normalize(
 self.column_for_source_field(
 record_name=record_name,
 source_field_name=source_field,
)
)
 if not column:
 continue
 if column in key_columns:
 continue
 if column.startswith(INSERT_ONLY_AUDIT_PREFIXES):
 continue
 if column in seen:
 continue
 seen.add(column)
 output.append(column)

 for column in self.update_audit_columns_for_record(record_name):
 column = NameNormalizer.normalize(column)
 if not column or column in key_columns or column in seen:
 continue
 seen.add(column)
 output.append(column)

 return output

```

```

=====
FILE: src\idms_db2_phase2\repositories\mapping_key_resolver.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/repositories/mapping_key_resolver.py

```

```
ACTION: CREATE NEW FILE
```

```
"""
```

```
Sheet Mapping key-column resolution.
```

```
Deterministic primary / foreign / non-key column logic.
```

```
"""
```

```
from __future__ import annotations
```

```
from idms_db2_phase2.repositories.mapping_row_query import MappingRowQuery
```

```
from idms_db2_phase2.services.name_normalizer import NameNormalizer
```

```
class MappingKeyResolver:
```

```
 def __init__(
 self,
 query: MappingRowQuery,
) -> None:
 self.query = query
```

```
 def primary_key_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 """
```

```
 Return all primary-key-style columns for DB2 WHERE clauses.
```

```
 - If key is composite, include all mandatory PK / CALC key fields.
 - Never include FK / FOREIGN / relationship fields.
 - Preserve Sheet Mapping order.
 - Do not return duplicate columns.
 """
```

```
 output: list[str] = []
 seen: set[str] = set()
```

```
 for row in self.query.rows_for_record(record_name):
 idms_key = NameNormalizer.normalize(getattr(row, "idms_key", ""))
 db2_key = NameNormalizer.normalize(getattr(row, "db2_key", ""))
 relation = NameNormalizer.normalize(getattr(row, "relation", ""))
 column = NameNormalizer.normalize(
 getattr(row, "new_db2_field_name", "")
 or getattr(row, "cross_application_db2_field_name", "")
)
```

```
 if not column:
 continue
```

```
 key_text = " ".join([idms_key, db2_key, relation])
 padded_key_text = f" {key_text} "
```

```
 if "FOREIGN" in key_text:
 continue
```

```
 if " FK " in padded_key_text:
 continue
```

```
 is_key_column = (
 "PRIMARY" in db2_key
 or db2_key == "KEY"
 or "PRIMARY" in idms_key
 or idms_key == "KEY"
 or "CALC" in idms_key
)
```

```
 if not is_key_column:
 continue
```

```
 if column in seen:
 continue
```

```
 seen.add(column)
 output.append(column)
```

```
 return output
```

```
 def foreign_key_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
```

```

"""
Return FK / FOREIGN / relationship columns for a record.

Ensures FK fields are never treated as UPDATE / DELETE / SELECT
WHERE identity keys.
"""
output: list[str] = []
seen: set[str] = set()

for row in self.query.rows_for_record(record_name):
 db2_key = NameNormalizer.normalize(getattr(row, "db2_key", ""))
 relation = NameNormalizer.normalize(getattr(row, "relation", ""))
 text = " ".join([db2_key, relation])
 padded_text = f" {text} "

 is_foreign = "FOREIGN" in text or " FK " in padded_text
 if not is_foreign:
 continue

 column = NameNormalizer.normalize(
 getattr(row, "new_db2_field_name", "")
 or getattr(row, "cross_application_db2_field_name", "")
)
 if not column or column in seen:
 continue

 seen.add(column)
 output.append(column)

return output

def non_key_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 key_columns = set(
 self.primary_key_columns_for_record(record_name)
)
 output: list[str] = []
 seen: set[str] = set()

 for row in self.query.rows_for_record(record_name):
 column = NameNormalizer.normalize(row.new_db2_field_name)
 if not column or column in key_columns or column in seen:
 continue
 seen.add(column)
 output.append(column)

 return output

=====
FILE: src\idms_db2_phase2\repositories\mapping_repository.py
ENCODING: utf-8
=====

"""
Repository wrapper for Sheet Mapping rows.

Authority:
- Sheet Mapping provides DB2 record/table names and column names.
- Source-field resolution is deterministic (COBOL identifier ->
 new_db2_field_name on the same row). No fuzzy / similarity matching.

This class is a thin facade that delegates to focused helpers:
- MappingRowQuery (row filtering / listing)
- MappingKeyResolver (primary / foreign / non-key columns)
- MappingColumnResolver (source->column, audit, update candidates)
"""

from __future__ import annotations

from catalogs.db2_naming_catalog import DB2_TABLE_SUFFIX_EQUIVALENTS # ADDED
from idms_db2_phase2.domain.models import SheetMappingRow
from idms_db2_phase2.repositories.mapping_column_resolver import (
 MappingColumnResolver,

```

```

)
from idms_db2_phase2.repositories.mapping_key_resolver import (
 MappingKeyResolver,
)
from idms_db2_phase2.repositories.mapping_row_query import MappingRowQuery
from idms_db2_phase2.services.name_normalizer import NameNormalizer # ADDED

```

```

class MappingRepository:
 def __init__(
 self,
 rows: list[SheetMappingRow] | None = None,
) -> None:
 self.rows = rows or []
 self._query = MappingRowQuery(self.rows)
 self._keys = MappingKeyResolver(self._query)
 self._columns = MappingColumnResolver(self._query, self._keys)

 # --- row query ---
 def all(self) -> list[SheetMappingRow]:
 return self._query.all()

 def count(self) -> int:
 return self._query.count()

 def records(self) -> list[str]:
 return self._query.records()

 def tables(self) -> list[str]:
 return self._query.tables()

 def rows_for_record(
 self,
 record_name: str,
) -> list[SheetMappingRow]:
 return self._query.rows_for_record(record_name)

 def rows_for_table(
 self,
 table_name: str,
) -> list[SheetMappingRow]:
 return self._query.rows_for_table(table_name)

 def db2_columns_for_table(
 self,
 table_name: str,
) -> list[str]:
 return self._query.db2_columns_for_table(table_name)

 def db2_table_for_record(
 self,
 record_name: str,
) -> str:
 return self._query.db2_table_for_record(record_name)

 def has_record(self, record_name: str) -> bool:
 return bool(self._query.rows_for_record(record_name))

 def has_table(self, table_name: str) -> bool:
 return bool(self._query.rows_for_table(table_name))

 # --- key resolution ---
 def key_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 return self._keys.primary_key_columns_for_record(record_name)

 def primary_key_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 return self._keys.primary_key_columns_for_record(record_name)

 def foreign_key_columns_for_record(
 self,

```

```

 record_name: str,
) -> list[str]:
 return self._keys.foreign_key_columns_for_record(record_name)

def non_key_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 return self._keys.non_key_columns_for_record(record_name)

--- column resolution ---
def column_for_source_field(
 self,
 record_name: str,
 source_field_name: str,
) -> str:
 return self._columns.column_for_source_field(
 record_name=record_name,
 source_field_name=source_field_name,
)

def update_audit_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 return self._columns.update_audit_columns_for_record(record_name)

def update_candidate_columns_for_record(
 self,
 record_name: str,
 changed_source_fields: list[str] | None = None,
) -> list[str]:
 return self._columns.update_candidate_columns_for_record(
 record_name=record_name,
 changed_source_fields=changed_source_fields,
)

--- ADDED: restart AND-gate support (TB/TV aware) ---
def _restart_table_candidates(self, table_name: str) -> list[str]:
 """All equivalent DB2 table spellings (TB/TV) for confirmation.

 Mirrors TableNameResolver.table_candidates so a DCLGEN 'TV' name is
 confirmed against a Sheet Mapping 'TB' name (and vice versa).
 """
 normalized = NameNormalizer.normalize(table_name)
 if not normalized:
 return []

 candidates = [normalized]
 for source_suffix, target_suffix in DB2_TABLE_SUFFIX_EQUIVALENTS:
 if normalized.endswith(source_suffix):
 candidates.append(
 normalized[: -len(source_suffix)] + target_suffix
)

 # de-duplicate, preserve order
 seen: set[str] = set()
 output: list[str] = []
 for candidate in candidates:
 if candidate and candidate not in seen:
 seen.add(candidate)
 output.append(candidate)
 return output

def confirms_restart_table(self, table_name: str) -> bool:
 """True when the Sheet Mapping contains this DB2 table under ANY
 equivalent TB/TV spelling.

 Sheet Mapping is the authority for DB2 table names (authority_rules),
 but it stores the intended (often 'TB') spelling while DCLGEN carries
 the final ('TV') spelling. We therefore confirm across equivalents to
 avoid falsely blocking valid restart generation (Case 1).
 """
 for candidate in self._restart_table_candidates(table_name):
 if self.has_table(candidate):
 return True

```



```

 return False

__all__ = [
 "MappingRepository",
]

=====
FILE: src\idms_db2_phase2\repositories\mapping_row_query.py
ENCODING: utf-8
=====

LOCATION: src/idms_db2_phase2/repositories/mapping_row_query.py
ACTION: CREATE NEW FILE

"""
Sheet Mapping row query helpers.

Pure row filtering / listing. No column or key resolution logic.
"""

from __future__ import annotations

from idms_db2_phase2.domain.models import SheetMappingRow
from idms_db2_phase2.services.name_normalizer import NameNormalizer

class MappingRowQuery:
 def __init__(
 self,
 rows: list[SheetMappingRow],
) -> None:
 self.rows = rows

 def all(self) -> list[SheetMappingRow]:
 return list(self.rows)

 def count(self) -> int:
 return len(self.rows)

 def records(self) -> list[str]:
 values = {
 NameNormalizer.normalize(row.cobol_record_idms)
 for row in self.rows
 if NameNormalizer.normalize(row.cobol_record_idms)
 }
 return sorted(values)

 def tables(self) -> list[str]:
 values = {
 NameNormalizer.normalize(
 row.new_db2_record or row.cross_application_db2_table
)
 for row in self.rows
 if NameNormalizer.normalize(
 row.new_db2_record or row.cross_application_db2_table
)
 }
 return sorted(values)

 def rows_for_record(
 self,
 record_name: str,
) -> list[SheetMappingRow]:
 target = NameNormalizer.normalize(record_name)

 if not target:
 return []

 return [
 row
 for row in self.rows
 if NameNormalizer.normalize(row.cobol_record_idms) == target
]

```

```

def rows_for_table(
 self,
 table_name: str,
) -> list[SheetMappingRow]:
 target = NameNormalizer.normalize(table_name)

 if not target:
 return []

 return [
 row
 for row in self.rows
 if NameNormalizer.normalize(row.new_db2_record) == target
 or NameNormalizer.normalize(row.cross_application_db2_table)
 == target
]

def db2_columns_for_table(
 self,
 table_name: str,
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for row in self.rows_for_table(table_name):
 column = NameNormalizer.normalize(
 row.new_db2_field_name or row.cross_application_db2_field_name
)
 if not column or column in seen:
 continue
 seen.add(column)
 output.append(column)

 return output

def db2_table_for_record(
 self,
 record_name: str,
) -> str:
 rows = self.rows_for_record(record_name)

 for row in rows:
 table = NameNormalizer.normalize(row.new_db2_record)
 if table:
 return table

 for row in rows:
 table = NameNormalizer.normalize(row.cross_application_db2_table)
 if table:
 return table

 return ""

```

```

=====
FILE: src\idms_db2_phase2\resolvers__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\resolvers\column_name_resolver.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class ColumnNameResolver:
 """
 Resolves DB2 column names.
 """

```

Authority:

- Sheet Mapping provides DB2 column names.
  - DCLGEN validates whether a column exists for the resolved table.
- """

```
def __init__(
 self,
 mapping_repository: MappingRepository,
 dclgen_repository: DclgenRepository,
 table_name_resolver: TableNameResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver

def columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 table_name = self.table_name_resolver.table_for_record(record_name)

 if not table_name:
 return []

 mapped_columns = self.mapping_repository.db2_columns_for_table(table_name)

 if not mapped_columns:
 mapped_table = self.mapping_repository.db2_table_for_record(record_name)
 mapped_columns = self.mapping_repository.db2_columns_for_table(mapped_table)

 return self._filter_to_existing_dclgen_columns(
 table_name=table_name,
 columns=mapped_columns,
)

def columns_for_table(
 self,
 table_name: str,
) -> list[str]:
 resolved_table = self.table_name_resolver.resolve_table(table_name)

 if not resolved_table:
 return []

 mapped_columns = self.mapping_repository.db2_columns_for_table(table_name)

 if mapped_columns:
 return self._filter_to_existing_dclgen_columns(
 table_name=resolved_table,
 columns=mapped_columns,
)

 return self.dclgen_repository.column_names_for_table(resolved_table)

def key_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 table_name = self.table_name_resolver.table_for_record(record_name)

 if not table_name:
 return []

 mapped_keys = self.mapping_repository.key_columns_for_record(record_name)

 return self._filter_to_existing_dclgen_columns(
 table_name=table_name,
 columns=mapped_keys,
)

def has_column(
 self,
 table_name: str,
 column_name: str,
) -> bool:
```

```

 resolved_table = self.table_name_resolver.resolve_table(table_name)
 normalized_column = NameNormalizer.normalize(column_name)

 if not resolved_table or not normalized_column:
 return False

 return self.dclgen_repository.has_column(
 table_name=resolved_table,
 column_name=normalized_column,
)

def _filter_to_existing_dclgen_columns(
 self,
 table_name: str,
 columns: list[str],
) -> list[str]:
 resolved_table = self.table_name_resolver.resolve_table(table_name)

 if not resolved_table:
 return []

 dclgen_columns = set(
 self.dclgen_repository.column_names_for_table(resolved_table)
)

 if not dclgen_columns:
 return self._unique(columns)

 output: list[str] = []

 for column in columns:
 normalized = NameNormalizer.normalize(column)

 if not normalized:
 continue

 if normalized not in dclgen_columns:
 continue

 output.append(normalized)

 return self._unique(output)

def _unique(
 self,
 values: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for value in values:
 normalized = NameNormalizer.normalize(value)

 if not normalized:
 continue

 if normalized in seen:
 continue

 seen.add(normalized)
 output.append(normalized)

 return output

```

```

=====
FILE: src\idms_db2_phase2\resolvers\cursor_column_filter.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.timestamp_audit_rules import AUDIT_COLUMN_PREFIXES

```

```

class CursorColumnFilter:
 """
 Filters cursor columns to valid non-audit DB2 columns.

 This helper owns validation and de-duplication only.
 It does not choose business columns.
 """

 def __init__(
 self,
 table_name_resolver: TableNameResolver,
 column_name_resolver: ColumnNameResolver,
) -> None:
 self.table_name_resolver = table_name_resolver
 self.column_name_resolver = column_name_resolver

 def valid_non_audit_columns(
 self,
 record_name: str,
 columns: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for column in columns:
 normalized = NameNormalizer.normalize(column)

 if not normalized:
 continue

 if normalized in seen:
 continue

 if self.is_audit_column(normalized):
 continue

 if not self.column_exists_for_record(record_name, normalized):
 continue

 seen.add(normalized)
 output.append(normalized)

 return output

 def column_exists_for_record(
 self,
 record_name: str,
 column_name: str,
) -> bool:
 table = self.table_name_resolver.table_for_record(record_name)

 if not table:
 return False

 return self.column_name_resolver.has_column(
 table_name=table,
 column_name=column_name,
)

 def is_audit_column(
 self,
 column_name: str,
) -> bool:
 column = NameNormalizer.normalize(column_name)

 return any(
 column.startswith(prefix)
 for prefix in AUDIT_COLUMN_PREFIXES
)

 def unique(
 self,
 values: list[str],
) -> list[str]:

```

```

output: list[str] = []
seen: set[str] = set()

for value in values:
 normalized = str(value or "").strip()

 if not normalized:
 continue

 key = NameNormalizer.normalize(normalized)

 if key in seen:
 continue

 seen.add(key)
 output.append(normalized)

return output

```

```

=====
FILE: src\idms_db2_phase2\resolvers\cursor_column_resolver.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.analyzers.field_usage_analyzer import FieldUsageAnalysis
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.cursor_column_filter import CursorColumnFilter
from idms_db2_phase2.resolvers.cursor_field_usage_column_selector import (
 CursorFieldUsageColumnSelector,
)
from idms_db2_phase2.resolvers.cursor_order_by_resolver import CursorOrderByResolver
from idms_db2_phase2.resolvers.relationship_resolver import RelationshipResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class CursorColumnResolver:
 """
 Resolves minimal cursor SELECT and ORDER BY columns generically.

 SELECT priority:
 1. DCLGEN host fields used in procedure logic.
 2. Fields used in procedure conditions.
 3. Fields used as MOVE sources for output writes.
 4. Parent key columns required by child cursor relationships.
 5. Child order key columns.
 6. Fallback to mapped non-audit columns only if usage is unavailable.

 ORDER BY rule:
 - Parent/root cursors do not get ORDER BY unless explicit order metadata
 exists in Sheet Mapping.
 - Child cursors may order by non-FK primary/sequence key.
 - DESC is added for child sequence/order key where metadata indicates
 sequence/event/latest-first semantics.
 """

```

```

def __init__(
 self,
 mapping_repository: MappingRepository,
 table_name_resolver: TableNameResolver,
 column_name_resolver: ColumnNameResolver,
 relationship_resolver: RelationshipResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.table_name_resolver = table_name_resolver
 self.column_name_resolver = column_name_resolver
 self.relationship_resolver = relationship_resolver

 self.column_filter = CursorColumnFilter(
 table_name_resolver=table_name_resolver,
 column_name_resolver=column_name_resolver,
)

```

```

self.usage_column_selector = CursorFieldUsageColumnSelector(
 mapping_repository=mapping_repository,
)
self.order_by_resolver = CursorOrderByResolver(
 mapping_repository=mapping_repository,
 relationship_resolver=relationship_resolver,
 column_filter=self.column_filter,
)

def select_columns_for_record(
 self,
 record_name: str,
 field_usage_analysis: FieldUsageAnalysis | None = None,
) -> list[str]:
 record = NameNormalizer.normalize(record_name)

 if not record:
 return []

 selected: list[str] = []

 selected.extend(
 self.usage_column_selector.columns_from_field_usage(
 record_name=record,
 field_usage_analysis=field_usage_analysis,
)
)

 selected.extend(
 self.relationship_resolver.parent_key_columns_required_by_children(record)
)

 if self.relationship_resolver.has_foreign_keys(record):
 selected.extend(
 self.order_by_resolver.order_by_base_columns_for_record(record)
)

 selected = self.column_filter.valid_non_audit_columns(
 record_name=record,
 columns=selected,
)

 if selected:
 return selected

 fallback = self.column_name_resolver.columns_for_record(record)

 return self.column_filter.valid_non_audit_columns(
 record_name=record,
 columns=fallback,
)

def order_by_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 return self.order_by_resolver.order_by_columns_for_record(record_name)

```

```

=====
FILE: src\idms_db2_phase2\resolvers\cursor_field_usage_column_selector.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.analyzers.field_usage_analyzer import FieldUsageAnalysis
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class CursorFieldUsageColumnSelector:

```

```

 """

```

```

 Resolves cursor SELECT columns from procedure field usage.

```

```

 This helper maps COBOL source fields or DCLGEN host fields back to DB2

```

columns using Sheet Mapping metadata.

"""

```
def __init__(
 self,
 mapping_repository: MappingRepository,
) -> None:
 self.mapping_repository = mapping_repository

def columns_from_field_usage(
 self,
 record_name: str,
 field_usage_analysis: FieldUsageAnalysis | None,
) -> list[str]:
 if field_usage_analysis is None:
 return []

 usage = field_usage_analysis.usage_by_record.get(
 NameNormalizer.normalize(record_name)
)

 if usage is None:
 return []

 output: list[str] = []

 # Sort the field set so SELECT/FETCH column order is deterministic
 # run-to-run (a set iterates in a hash-seed-dependent order).
 for field_name in sorted(usage.all_fields):
 column = self.column_for_source_or_host_field(
 record_name=record_name,
 source_or_host_field=field_name,
)

 if column:
 output.append(column)

 return self._unique(output)

def column_for_source_or_host_field(
 self,
 record_name: str,
 source_or_host_field: str,
) -> str:
 record = NameNormalizer.normalize(record_name)
 source = NameNormalizer.normalize(source_or_host_field)

 if not record or not source:
 return ""

 rows = self.mapping_repository.rows_for_record(record)

 for row in rows:
 target_column = NameNormalizer.normalize(
 getattr(row, "new_db2_field_name", "")
 or getattr(row, "cross_application_db2_field_name", "")
)

 if not target_column:
 continue

 source_candidates = {
 NameNormalizer.normalize(
 self.extract_source_field(getattr(row, "cobol_zone", ""))
),
 NameNormalizer.normalize(
 self.extract_source_field(
 getattr(row, "reference_field_name_copybook", "")
)
),
 target_column,
 NameNormalizer.normalize(NameNormalizer.to_cobol(target_column)),
 }

 source_candidates = {
 candidate
```



```

 for candidate in source_candidates:
 if candidate:
 }

 if source in source_candidates:
 return target_column

 return ""

def extract_source_field(
 self,
 value: str,
) -> str:
 text = str(value or "").strip()

 if not text:
 return ""

 text = text.replace(".", " ")
 tokens = text.split()

 if tokens and tokens[0].isdigit() and len(tokens) > 1:
 return tokens[1]

 if tokens:
 return tokens[0]

 return ""

def _unique(
 self,
 values: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for value in values:
 normalized = NameNormalizer.normalize(value)

 if not normalized:
 continue

 if normalized in seen:
 continue

 seen.add(normalized)
 output.append(normalized)

 return output

```

```

=====
FILE: src\idms_db2_phase2\resolvers\cursor_join_resolver.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/resolvers/cursor_join_resolver.py
ACTION: CREATE NEW FILE
"""Builds child-cursor WHERE predicates from IDMS set relationships.

```

The left operand is always a column of the CHILD table.  
The right operand is always a host reference into the PARENT DCLGEN group.

```
<child column> = :<parent group>.<parent host field>
```

Anything that cannot be resolved on the child side is dropped and reported,  
never guessed. This honours CONVERSION\_RULES:  
"Do not fabricate DB2 SQL when Sheet Mapping metadata is missing."  
"""

```
from __future__ import annotations
```

```

from idms_db2_phase2.resolvers.relationship_resolver import RelationshipResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.cursor_declaration_rules import CHILD_JOIN_PREDICATE_TEMPLATE

```

```

class CursorJoinResolver:
 """Resolves the WHERE clause of a child cursor."""

 def __init__(
 self,
 relationship_resolver: RelationshipResolver,
 column_name_resolver,
 host_variable_resolver,
) -> None:
 self.relationship_resolver = relationship_resolver
 self.column_name_resolver = column_name_resolver
 self.host_variable_resolver = host_variable_resolver
 self.diagnostics: list[str] = []

 # -----
 # Public entry point
 # -----
 def where_conditions(
 self,
 child_record: str,
) -> list[str]:
 """Return the join predicates for one child record."""
 self.diagnostics = []

 resolution = self.relationship_resolver.resolve(child_record)
 if resolution is None:
 return []

 self.diagnostics.extend(list(resolution.diagnostics or []))

 child_table = NameNormalizer.normalize(resolution.child_table)
 owned = {
 NameNormalizer.normalize(str(column)).upper()
 for column in self.column_name_resolver.columns_for_table(
 child_table
)
 }

 conditions: list[str] = []
 seen: set[str] = set()

 for condition in list(resolution.conditions or []):
 predicate = self._predicate(condition, owned)
 if not predicate or predicate in seen:
 continue
 seen.add(predicate)
 conditions.append(predicate)

 return conditions

 # -----
 # One predicate
 # -----
 def _predicate(
 self,
 condition,
 owned: set[str],
) -> str:
 child_column = NameNormalizer.normalize(
 getattr(condition, "child_column", "")
)
 if not child_column:
 return ""

 if owned and child_column.upper() not in owned:
 self.diagnostics.append(
 f"Cursor join: {child_column} is not a column of "
 f"{condition.child_table}; predicate skipped."
)
 return ""

 group, host = self._parent_reference(condition)
 if not group or not host:
 self.diagnostics.append(
 f"Cursor join: no DCLGEN host variable for parent column "

```

```

 f"{getattr(condition, 'parent_column', '')}); predicate skipped."
)
 return ""

 return CHILD_JOIN_PREDICATE_TEMPLATE.format(
 child_column=child_column,
 parent_group=group,
 parent_host=host,
)

Parent host reference -> (group, host)

def _parent_reference(self, condition) -> tuple[str, str]:
 """Split the canonical ':DCLGROUP.HOST-FIELD' form into parts."""
 reference = str(
 getattr(condition, "parent_host_reference", "") or ""
).strip()

 if not reference:
 reference = self.host_variable_resolver.host_reference_for_column(
 table_name=getattr(condition, "parent_table", ""),
 column_name=getattr(condition, "parent_column", ""),
)

 reference = str(reference or "").strip().rstrip(":")
 if "." not in reference:
 return "", ""

 group, _, host = reference.partition(".")
 return (
 NameNormalizer.to_cobol(group.strip()),
 NameNormalizer.to_cobol(host.strip()),
)

```

```

=====
FILE: src\idms_db2_phase2\resolvers\cursor_name_resolver.py
ENCODING: utf-8
=====

```

```

from catalogs.db2_naming_catalog import DB2_CURSOR_SUFFIX, DB2_DEFAULT_CURSOR_NAME
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.cursor_rules import cursor_paragraph_number

```

```

class CursorNameResolver:
 """
 Resolves cursor names and cursor paragraph names.

 Authority:
 - Cursor names are derived from DB2 table or record names.
 - Cursor names are not derived from IDMS set names.
 """

 def cursor_name_from_table(
 self,
 table_name: str,
) -> str:
 table = NameNormalizer.normalize(table_name)

 if not table:
 return DB2_DEFAULT_CURSOR_NAME

 base = self._remove_table_suffix(table)

 if not base:
 return DB2_DEFAULT_CURSOR_NAME

 return NameNormalizer.to_cobol(base + DB2_CURSOR_SUFFIX)

 def paragraph_names(
 self,
 cursor_order: int,
 cursor_name: str,
) -> dict[str, str]:

```

```

clean_cursor = NameNormalizer.to_cobol(cursor_name)

return {
 "open": f"{cursor_paragraph_number(cursor_order, 'open')}-OPEN-{clean_cursor}",
 "fetch": f"{cursor_paragraph_number(cursor_order, 'fetch')}-FETCH-{clean_cursor}",
 "close": f"{cursor_paragraph_number(cursor_order, 'close')}-CLOSE-{clean_cursor}",
}

def paragraph_numbers(
 self,
 cursor_order: int,
) -> dict[str, int]:
 return {
 "open": cursor_paragraph_number(cursor_order, "open"),
 "fetch": cursor_paragraph_number(cursor_order, "fetch"),
 "close": cursor_paragraph_number(cursor_order, "close"),
 }

def cursor_spec(
 self,
 cursor_order: int,
 table_name: str,
) -> dict[str, str | int]:
 cursor_name = self.cursor_name_from_table(table_name)
 paragraph_names = self.paragraph_names(
 cursor_order=cursor_order,
 cursor_name=cursor_name,
)
 paragraph_numbers = self.paragraph_numbers(cursor_order)

 return {
 "cursor_name": cursor_name,
 "open_paragraph": paragraph_names["open"],
 "fetch_paragraph": paragraph_names["fetch"],
 "close_paragraph": paragraph_names["close"],
 "open_number": paragraph_numbers["open"],
 "fetch_number": paragraph_numbers["fetch"],
 "close_number": paragraph_numbers["close"],
 }

def _remove_table_suffix(
 self,
 table_name: str,
) -> str:
 table = NameNormalizer.normalize(table_name)

 if table.endswith("_IV") or table.endswith("_TB"):
 return table[:-3] + "_"

 if table.endswith("TV") or table.endswith("TB"):
 return table[:-2]

 return table + "_"

```

```

=====
FILE: src\idms_db2_phase2\resolvers\cursor_order_by_resolver.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.cursor_column_filter import CursorColumnFilter
from idms_db2_phase2.resolvers.relationship_resolver import RelationshipResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from rules.cursor_column_rules import (
 DESC_HINT_WORDS,
 ORDER_HINT_WORDS,
)

```

```

class CursorOrderByResolver:
 """
 Resolves cursor ORDER BY columns.

```

Parent cursors only receive ORDER BY when Sheet Mapping explicitly indicates order semantics. Child cursors use relationship-derived order columns.  
 """

```
def __init__(
 self,
 mapping_repository: MappingRepository,
 relationship_resolver: RelationshipResolver,
 column_filter: CursorColumnFilter,
) -> None:
 self.mapping_repository = mapping_repository
 self.relationship_resolver = relationship_resolver
 self.column_filter = column_filter

def order_by_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 record = NameNormalizer.normalize(record_name)

 if not record:
 return []

 is_child = self.relationship_resolver.has_foreign_keys(record)

 if not is_child:
 return self.explicit_parent_order_columns(record)

 order_columns = self.order_by_base_columns_for_record(record)
 output: list[str] = []

 for column in order_columns:
 expression = column

 if self.should_order_desc(
 record_name=record,
 column_name=column,
):
 expression = f"{column} DESC"

 output.append(expression)

 return self.column_filter.unique(output)

def explicit_parent_order_columns(
 self,
 record_name: str,
) -> list[str]:
 """
 Parent/root cursor gets ORDER BY only if Sheet Mapping explicitly
 indicates order/sort/rank semantics.
 """

 rows = self.mapping_repository.rows_for_record(record_name)
 output: list[str] = []

 for row in rows:
 text = " ".join(
 [
 str(getattr(row, "remarks", "")) or "",
 str(getattr(row, "hopex_expression_type_remark", "")) or "",
 str(getattr(row, "relation", "")) or "",
 str(getattr(row, "basetype", "")) or "",
]
).upper()

 if not any(word in text for word in ORDER_HINT_WORDS):
 continue

 column = NameNormalizer.normalize(
 getattr(row, "new_db2_field_name", "")
)

 if not column:
 continue
```

```

 if not self.column_filter.column_exists_for_record(record_name, column):
 continue

 if self.column_filter.is_audit_column(column):
 continue

 output.append(column)

 return self.column_filter.unique(output)

def order_by_base_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 return self.relationship_resolver.order_by_columns_for_record(record_name)

def should_order_desc(
 self,
 record_name: str,
 column_name: str,
) -> bool:
 record = NameNormalizer.normalize(record_name)
 column = NameNormalizer.normalize(column_name)

 rows = self.mapping_repository.rows_for_record(record)

 for row in rows:
 row_column = NameNormalizer.normalize(
 getattr(row, "new_db2_field_name", "")
)

 if row_column != column:
 continue

 combined_text = " ".join(
 [
 str(getattr(row, "db2_key", "") or ""),
 str(getattr(row, "hopex_expression_type_remark", "") or ""),
 str(getattr(row, "remarks", "") or ""),
 str(getattr(row, "new_db2_data_type", "") or ""),
 str(getattr(row, "basetype", "") or ""),
 column,
]
).upper()

 if any(word in combined_text for word in DESC_HINT_WORDS):
 return True

 return False

```

```

=====
FILE: src\idms_db2_phase2\resolvers\host_variable_resolver.py
ENCODING: utf-8
=====

```

```
import re
```

```

from catalogs.db2_naming_catalog import DB2_HOST_REFERENCE_PREFIX
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```
class HostVariableResolver:
```

```
 """
```

```
 Resolves DCLGEN host variables.
```

```
 Authority:
```

- DCLGEN determines COBOL host-variable spelling.
- DCLGEN determines DCLGEN group names.
- Final DB2 embedded SQL host reference format is:

```
 :DCLGROUP.HOST-FIELD
```

```
 Example:
```

:DCLDZBEFFTV.NR-IDSTOCK-479BEFF

This resolver also normalizes older generated forms:

```
:HOST-FIELD OF DCLGROUP
HOST-FIELD OF DCLGROUP
:DCLGROUP.HOST-FIELD
DCLGROUP.HOST-FIELD
```

into the canonical DB2 style:

```
:DCLGROUP.HOST-FIELD
"""
```

```
HOST_OF_GROUP_PATTERN = re.compile(
 r"^:?\s*"
 r"(?P<host>[A-Z][A-Z0-9-]*)"
 r"\s+OF\s+"
 r"(?P<group>DCL[A-Z0-9-]+)"
 r"\s*$",
 flags=re.IGNORECASE,
)

GROUP_DOT_HOST_PATTERN = re.compile(
 r"^:?\s*"
 r"(?P<group>DCL[A-Z0-9-]+)"
 r"\s*\.\s*"
 r"(?P<host>[A-Z][A-Z0-9-]*)"
 r"\s*$",
 flags=re.IGNORECASE,
)

DOUBLE_COLON_PATTERN = re.compile(
 r"^:+",
 flags=re.IGNORECASE,
)

def __init__(
 self,
 dclgen_repository: DclgenRepository,
 table_name_resolver: TableNameResolver,
) -> None:
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver

def host_for_column(
 self,
 table_name: str,
 column_name: str,
) -> str:
 resolved_table = self.table_name_resolver.resolve_table(table_name)

 if not resolved_table:
 return ""

 return self.dclgen_repository.host_for_column(
 table_name=resolved_table,
 column_name=column_name,
)

def group_for_table(
 self,
 table_name: str,
) -> str:
 resolved_table = self.table_name_resolver.resolve_table(table_name)

 if not resolved_table:
 return ""

 group = self.dclgen_repository.group_for_table(resolved_table)

 if group:
 return NameNormalizer.to_cobol(group)

 return "DCL" + NameNormalizer.to_cobol(resolved_table)
```

```

def host_reference_for_column(
 self,
 table_name: str,
 column_name: str,
) -> str:
 group = self.group_for_table(table_name)
 host = self.host_for_column(
 table_name=table_name,
 column_name=column_name,
)

 if not group or not host:
 return ""

 return self.normalize_host_reference(
 f"{DB2_HOST_REFERENCE_PREFIX}{group}.{host}"
)

def host_references_for_columns(
 self,
 table_name: str,
 columns: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for column in columns:
 reference = self.host_reference_for_column(
 table_name=table_name,
 column_name=column,
)

 if not reference:
 continue

 key = reference.upper()

 if key in seen:
 continue

 seen.add(key)
 output.append(reference)

 return output

def normalize_host_reference(
 self,
 value: str,
) -> str:
 text = str(value or "").strip()

 if not text:
 return ""

 text = text.replace("::", ":")
 text = self.DOUBLE_COLON_PATTERN.sub(":", text)

 of_match = self.HOST_OF_GROUP_PATTERN.match(text)

 if of_match:
 group = NameNormalizer.to_cobol(of_match.group("group"))
 host = NameNormalizer.to_cobol(of_match.group("host"))

 if not group or not host:
 return ""

 return f"{DB2_HOST_REFERENCE_PREFIX}{group}.{host}"

 dot_match = self.GROUP_DOT_HOST_PATTERN.match(text)

 if dot_match:
 group = NameNormalizer.to_cobol(dot_match.group("group"))
 host = NameNormalizer.to_cobol(dot_match.group("host"))

 if not group or not host:

```



```

 return ""

 return f"{DB2_HOST_REFERENCE_PREFIX}{group}.{host}"

 if text.startswith(DB2_HOST_REFERENCE_PREFIX):
 text = text[1:].strip()

 if "." in text:
 parts = text.split(".", 1)
 group = NameNormalizer.to_cobol(parts[0])
 host = NameNormalizer.to_cobol(parts[1])

 if group and host and group.upper().startswith("DCL"):
 return f"{DB2_HOST_REFERENCE_PREFIX}{group}.{host}"

 if text.upper().startswith("DCL") and " " in text:
 parts = text.split()

 if len(parts) >= 2:
 group = NameNormalizer.to_cobol(parts[0])
 host = NameNormalizer.to_cobol(parts[1])

 if group and host:
 return f"{DB2_HOST_REFERENCE_PREFIX}{group}.{host}"

 if text.upper().startswith("DCL"):
 return f"{DB2_HOST_REFERENCE_PREFIX}{NameNormalizer.to_cobol(text)}"

 return f"{DB2_HOST_REFERENCE_PREFIX}{NameNormalizer.to_cobol(text)}"

def valid_host_references(
 self,
) -> set[str]:
 output: set[str] = set()

 for reference in self.dclgen_repository.valid_host_references():
 normalized = self.normalize_host_reference(reference)

 if normalized:
 output.add(normalized)
 output.add(normalized[1:])

 return output

def has_host_for_column(
 self,
 table_name: str,
 column_name: str,
) -> bool:
 return bool(
 self.host_for_column(
 table_name=table_name,
 column_name=column_name,
)
)

def host_reference_key(
 self,
 table_name: str,
 column_name: str,
) -> str:
 reference = self.host_reference_for_column(
 table_name=table_name,
 column_name=column_name,
)

 if not reference:
 return ""

 return reference[1:]

```

```
=====
FILE: src\idms_db2_phase2\resolvers\lrf_path_resolver.py
ENCODING: utf-8
=====
```

```
LOCATION: src/idms_db2_phase2/resolvers/lrf_path_resolver.py
ACTION: REPLACE ENTIRE FILE
"""Turns an LRF path keyword into a concrete access plan.
```

```
NAME FORM
```

```

Every name this module returns is rendered in COBOL form (UPPERCASE with
hyphens), because the expander writes them straight back into COBOL
source. NameNormalizer.normalize() is deliberately NOT used: it produces
the DB2 column form (underscores) and would emit VMBTL03_R01.
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass, field
```

```
from rules.lrf_conversion_rules import (
 STATUS_END_OF_SET,
 STATUS_NOT_FOUND,
 STATUS_SET_EMPTY,
)
```

```
RETURN_PREFIX = "RETURN "
```

```
def cobol_name(value: str) -> str:
 """UPPERCASE COBOL/IDMS form: hyphens, never underscores."""
 return str(value or "").strip().upper().replace("_", "-")
```

```
@dataclass
```

```
class LrfAccessStep:
 """One renderable IDMS access line."""
```

```
 verb: str = ""
 scope: str = ""
 record_name: str = ""
 within_name: str = ""
 where_clause: str = ""
```

```
@property
```

```
def is_renderable(self) -> bool:
 return bool(self.record_name and self.within_name)
```

```
@dataclass
```

```
class LrfAccessPlan:
 """Everything the expander needs for one path keyword."""
```

```
 keyword: str = ""
 logical_record: str = ""
 path_group_verb: str = ""
 parent: LrfAccessStep | None = None
 driving: LrfAccessStep | None = None
 end_of_set_literals: set[str] = field(default_factory=set)
 not_found_literals: set[str] = field(default_factory=set)
 element_records: list[str] = field(default_factory=list)
```

```
@property
```

```
def is_usable(self) -> bool:
 return bool(self.driving and self.driving.is_renderable)
```

```
class LrfPathResolver:
```

```
 """Resolves a path keyword into an LrfAccessPlan."""
```

```
 def __init__(
 self,
 *,
 lrf_repository=None,
 mapping_repository=None,
```

```

) -> None:
 self.lrf_repository = lrf_repository
 self.mapping_repository = mapping_repository
 self.messages: list[str] = []

----- public
def is_empty(self) -> bool:
 if self.lrf_repository is None:
 return True
 try:
 return bool(self.lrf_repository.is_empty())
 except Exception: # noqa: BLE001
 return True

def plan_for_keyword(self, keyword: str) -> LrfAccessPlan | None:
 if self.is_empty():
 return None

 try:
 path = self.lrf_repository.path_for_keyword(keyword)
 except Exception: # noqa: BLE001
 return None

 if path is None:
 return None

 plan = LrfAccessPlan(
 keyword=cobol_name(path.keyword),
 logical_record=cobol_name(path.logical_record),
 path_group_verb=str(path.path_group_verb or "").upper(),
 element_records=[
 cobol_name(name)
 for name in self._element_records(path.logical_record)
],
)

 steps: list[LrfAccessStep] = []

 for command in path.commands or []:
 self._collect_literals(command, plan)

 verb = str(command.verb or "").upper()
 if verb not in ("OBTAIN", "FIND", "ERASE"):
 continue

 steps.append(
 LrfAccessStep(
 verb=verb,
 scope=str(command.scope or "").upper(),
 record_name=cobol_name(command.record_name),
 within_name=cobol_name(command.within_name),
 where_clause=str(command.where_clause or "").strip(),
)
)

 if not steps:
 return plan

 renderable = [step for step in steps if step.is_renderable]
 if renderable:
 plan.driving = renderable[-1]
 if len(renderable) > 1:
 plan.parent = renderable[0]
 else:
 plan.driving = steps[-1]

 return plan

def record_is_mapped(self, record_name: str) -> bool:
 """True when Sheet Mapping knows this IDMS record."""
 if self.mapping_repository is None:
 return True

 record = cobol_name(record_name)
 if not record:
 return False

```

```

 for probe in (
 "db2_table_for_record",
 "table_for_record",
 "has_record",
 "record_exists",
):
 method = getattr(self.mapping_repository, probe, None)
 if method is None:
 continue
 try:
 result = method(record)
 except Exception: # noqa: BLE001
 continue
 if isinstance(result, bool):
 return result
 if isinstance(result, str):
 return bool(result.strip())

 return True

----- helpers
def _element_records(self, logical_record_name: str) -> list:
 try:
 return self.lrf_repository.element_records(logical_record_name) or []
 except Exception: # noqa: BLE001
 return []

@staticmethod
def _collect_literals(command, plan: LrfAccessPlan) -> None:
 actions = getattr(command, "status_actions", None) or {}
 for status, action in actions.items():
 literal = LrfPathResolver._return_literal(action)
 if not literal:
 continue
 if str(status) == STATUS_END_OF_SET:
 plan.end_of_set_literals.add(literal)
 elif str(status) in (STATUS_NOT_FOUND, STATUS_SET_EMPTY):
 plan.not_found_literals.add(literal)

@staticmethod
def _return_literal(action: str) -> str:
 text = str(action or "").strip().upper()
 if not text.startswith(RETURN_PREFIX):
 return ""
 return cobol_name(text[len(RETURN_PREFIX):])

```

```
__all__ = ["LrfAccessStep", "LrfAccessPlan", "LrfPathResolver", "cobol_name"]
```

```

=====
FILE: src\idms_db2_phase2\resolvers\record_context_resolver.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class RecordContextResolver:
 """
 Resolves active source record context.

 This helper is used by transformers and field rewriters to avoid guessing
 the active record directly inside transformation logic.
 """

 def __init__(
 self,
 mapping_repository: MappingRepository,
 dclgen_repository: DclgenRepository,
) -> None:
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository

```

```

def known_records(
 self,
) -> list[str]:
 return self.mapping_repository.records()

def record_from_text(
 self,
 text: str,
) -> str:
 normalized_text = NameNormalizer.normalize(text)

 if not normalized_text:
 return ""

 for record in self.known_records():
 if record and record in normalized_text:
 return record

 return ""

def record_from_paragraph_name(
 self,
 paragraph_name: str,
) -> str:
 return self.record_from_text(paragraph_name)

def record_from_move_target(
 self,
 move_target: str,
) -> str:
 return self.record_from_text(move_target)

def record_from_initialize_target(
 self,
 initialize_target: str,
) -> str:
 text = NameNormalizer.normalize(initialize_target)

 if not text:
 return ""

 if text.startswith("DCL"):
 table = text[3:]
 return self.record_from_table(table)

 return self.record_from_text(text)

def record_from_table(
 self,
 table_name: str,
) -> str:
 table = NameNormalizer.normalize(table_name)

 if not table:
 return ""

 for record in self.known_records():
 record_table = self.mapping_repository.db2_table_for_record(record)

 if NameNormalizer.normalize(record_table) == table:
 return record

 return ""

def record_from_dclgen_group(
 self,
 group_name: str,
) -> str:
 text = NameNormalizer.normalize(group_name)

 if not text:
 return ""

 if text.startswith("DCL"):
 return self.record_from_table(text[3:])

```

```

 return ""

 def context_strength(
 self,
 record_name: str,
) -> int:
 if not record_name:
 return 0

 if self.mapping_repository.has_record(record_name):
 return 100

 return 0

=====
FILE: src\idms_db2_phase2\resolvers\relationship_child_resolver.py
ENCODING: utf-8
=====

from __future__ import annotations

from typing import Any

from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.relationship_key_classifier import (
 RelationshipKeyClassifier,
)
from idms_db2_phase2.resolvers.relationship_key_column_resolver import (
 RelationshipKeyColumnResolver,
)
from idms_db2_phase2.resolvers.relationship_models import (
 RelationshipCondition,
 RelationshipResolution,
)
from idms_db2_phase2.resolvers.relationship_table_lookup import RelationshipTableLookup
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer

class RelationshipChildResolver:
 """
 Resolves full parent-child relationship conditions for one child record.
 """

 def __init__(
 self,
 *,
 mapping_repository: MappingRepository,
 table_name_resolver: TableNameResolver,
 host_variable_resolver: HostVariableResolver,
 key_column_resolver: RelationshipKeyColumnResolver,
 table_lookup: RelationshipTableLookup,
 classifier: RelationshipKeyClassifier | None = None,
) -> None:
 self.mapping_repository = mapping_repository
 self.table_name_resolver = table_name_resolver
 self.host_variable_resolver = host_variable_resolver
 self.key_column_resolver = key_column_resolver
 self.table_lookup = table_lookup
 self.classifier = classifier or RelationshipKeyClassifier()

 def resolve_for_child_record(
 self,
 child_record: str,
) -> RelationshipResolution:
 child = NameNormalizer.normalize(child_record)
 result = RelationshipResolution(child_record=child)

 if not child:
 result.diagnostics.append("Relationship resolver: child record is empty.")
 return result

 child_table = self.table_name_resolver.table_for_record(child)

```

```

result.child_table = child_table

rows = self.mapping_repository.rows_for_record(child)
fk_rows = [
 row
 for row in rows
 if self.classifier.is_foreign_key_row(row)
]

if not fk_rows:
 result.diagnostics.append(
 f"Relationship resolver: no foreign-key rows for child record {child}."
)
 return result

parent_record = self._infer_parent_record(
 child_record=child,
 fk_rows=fk_rows,
)

parent_table = ""

if parent_record:
 parent_table = self.table_name_resolver.table_for_record(parent_record)

result.parent_record = parent_record
result.parent_table = parent_table

self._append_conditions_from_fk_rows(
 result=result,
 child=child,
 child_table=child_table,
 fk_rows=fk_rows,
 parent_record=parent_record,
 parent_table=parent_table,
)

result.diagnostics.append(
 "Relationship resolver: "
 f"child={child}, parent={result.parent_record}, "
 f"conditions={len(result.conditions)}"
)

return result

def _append_conditions_from_fk_rows(
 self,
 *,
 result: RelationshipResolution,
 child: str,
 child_table: str,
 fk_rows: list[Any],
 parent_record: str,
 parent_table: str,
) -> None:
 resolved_parent_record = parent_record

 for row in fk_rows:
 child_column = NameNormalizer.normalize(
 getattr(row, "new_db2_field_name", "")
 or getattr(row, "cross_application_db2_field_name", "")
)

 if not child_column:
 continue

 explicit_parent_table = self.table_name_resolver.resolve_table(
 getattr(row, "cross_application_db2_table", "")
)

 explicit_parent_column = NameNormalizer.normalize(
 getattr(row, "cross_application_db2_field_name", "")
)

 effective_parent_table = explicit_parent_table or parent_table
 effective_parent_column = explicit_parent_column or child_column

```

```

 if not effective_parent_table:
 effective_parent_table = self.table_lookup.find_parent_table_by_column(
 child_record=child,
 child_column=child_column,
 primary_key_columns_for_record=(
 self.key_column_resolver.primary_key_columns_for_record
),
)

 if not effective_parent_table:
 result.diagnostics.append(
 f"Relationship resolver: no parent table found for "
 f"{child}.{child_column}."
)
 continue

 if not resolved_parent_record:
 resolved_parent_record = self.table_lookup.record_for_table(
 effective_parent_table
)
 result.parent_record = resolved_parent_record

 if not result.parent_table:
 result.parent_table = effective_parent_table

 self._append_condition(
 result=result,
 child=child,
 child_table=child_table,
 child_column=child_column,
 parent_record=resolved_parent_record,
 parent_table=effective_parent_table,
 parent_column=effective_parent_column,
)

def _append_condition(
 self,
 *,
 result: RelationshipResolution,
 child: str,
 child_table: str,
 child_column: str,
 parent_record: str,
 parent_table: str,
 parent_column: str,
) -> None:
 parent_host = self.host_variable_resolver.host_reference_for_column(
 table_name=parent_table,
 column_name=parent_column,
)

 if not parent_host:
 result.diagnostics.append(
 "Relationship resolver: missing parent host for "
 f"child={child}, child_column={child_column}, "
 f"parent_table={parent_table}, parent_column={parent_column}."
)
 return

 result.conditions.append(
 RelationshipCondition(
 child_record=child,
 child_table=child_table,
 child_column=child_column,
 parent_record=parent_record,
 parent_table=parent_table,
 parent_column=parent_column,
 parent_host_reference=parent_host,
)
)

def _infer_parent_record(
 self,
 *,
 child_record: str,

```



```

 fk_rows: list[Any],
) -> str:
 for row in fk_rows:
 parent_table = self.table_name_resolver.resolve_table(
 getattr(row, "cross_application_db2_table", "")
)

 if parent_table:
 record = self.table_lookup.record_for_table(parent_table)
 if record:
 return record

 best_record = ""
 best_count = 0

 child_fk_columns = {
 NameNormalizer.normalize(
 getattr(row, "new_db2_field_name", "")
 or getattr(row, "cross_application_db2_field_name", "")
)
 for row in fk_rows
 }

 for record in self.mapping_repository.records():
 record = NameNormalizer.normalize(record)

 if record == child_record:
 continue

 parent_keys = set(
 self.key_column_resolver.primary_key_columns_for_record(record)
)

 if not parent_keys:
 continue

 match_count = len(parent_keys.intersection(child_fk_columns))

 if match_count > best_count:
 best_count = match_count
 best_record = record

 return best_record

```

```

=====
FILE: src\idms_db2_phase2\resolvers\relationship_key_classifier.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from typing import Any

```

```

from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class RelationshipKeyClassifier:

```

```

 """
 Classifies Sheet Mapping rows as primary-key or foreign-key rows.
 """

```

```

 def is_primary_key_row(
 self,
 row: Any,
) -> bool:
 db2_key = NameNormalizer.normalize(getattr(row, "db2_key", ""))
 idms_key = NameNormalizer.normalize(getattr(row, "idms_key", ""))

 return (
 "PRIMARY" in db2_key
 or db2_key == "KEY"
 or "CALC" in idms_key
)

```

```

 def is_foreign_key_row(

```

```

 self,
 row: Any,
) -> bool:
 db2_key = NameNormalizer.normalize(getattr(row, "db2_key", ""))
 relation = NameNormalizer.normalize(getattr(row, "relation", ""))

 return (
 "FOREIGN" in db2_key
 or "FK" in db2_key
 or "FOREIGN" in relation
)

```

```

=====
FILE: src\idms_db2_phase2\resolvers\relationship_key_column_resolver.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.relationship_key_classifier import (
 RelationshipKeyClassifier,
)
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class RelationshipKeyColumnResolver:
 """
 Resolves primary-key, foreign-key, and order-by columns for records.
 """

 def __init__(
 self,
 *,
 mapping_repository: MappingRepository,
 classifier: RelationshipKeyClassifier | None = None,
) -> None:
 self.mapping_repository = mapping_repository
 self.classifier = classifier or RelationshipKeyClassifier()

 def foreign_key_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 record = NameNormalizer.normalize(record_name)
 rows = self.mapping_repository.rows_for_record(record)
 output: list[str] = []
 seen: set[str] = set()

 for row in rows:
 if not self.classifier.is_foreign_key_row(row):
 continue

 column = NameNormalizer.normalize(
 getattr(row, "new_db2_field_name", "")
 or getattr(row, "cross_application_db2_field_name", "")
)

 if not column or column in seen:
 continue

 seen.add(column)
 output.append(column)

 return output

 def primary_key_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 record = NameNormalizer.normalize(record_name)
 rows = self.mapping_repository.rows_for_record(record)
 output: list[str] = []
 seen: set[str] = set()

```

```

for row in rows:
 if not self.classifier.is_primary_key_row(row):
 continue

 column = NameNormalizer.normalize(
 getattr(row, "new_db2_field_name", "")
)

 if not column or column in seen:
 continue

 seen.add(column)
 output.append(column)

return output

def order_by_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 record = NameNormalizer.normalize(record_name)
 primary_keys = self.primary_key_columns_for_record(record)
 foreign_keys = set(self.foreign_key_columns_for_record(record))
 output: list[str] = []

 for column in primary_keys:
 normalized = NameNormalizer.normalize(column)

 if not normalized:
 continue

 if normalized in foreign_keys:
 continue

 if normalized not in output:
 output.append(normalized)

 return output

def has_foreign_keys(
 self,
 record_name: str,
) -> bool:
 return bool(self.foreign_key_columns_for_record(record_name))

```

```

=====
FILE: src\idms_db2_phase2\resolvers\relationship_models.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from dataclasses import dataclass, field

```

```

@dataclass
class RelationshipCondition:
 child_record: str
 child_table: str
 child_column: str
 parent_record: str
 parent_table: str
 parent_column: str
 parent_host_reference: str

```

```

@dataclass
class RelationshipResolution:
 child_record: str
 parent_record: str = ""
 child_table: str = ""
 parent_table: str = ""
 conditions: list[RelationshipCondition] = field(default_factory=list)
 diagnostics: list[str] = field(default_factory=list)

```

```
=====
FILE: src\idms_db2_phase2\resolvers\relationship_resolver.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.relationship_child_resolver import (
 RelationshipChildResolver,
)
from idms_db2_phase2.resolvers.relationship_key_classifier import (
 RelationshipKeyClassifier,
)
from idms_db2_phase2.resolvers.relationship_key_column_resolver import (
 RelationshipKeyColumnResolver,
)
from idms_db2_phase2.resolvers.relationship_models import (
 RelationshipCondition,
 RelationshipResolution,
)
from idms_db2_phase2.resolvers.relationship_table_lookup import RelationshipTableLookup
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
```

```
class RelationshipResolver:
 """
 Resolves parent-child DB2 relationships using Sheet Mapping metadata.

 Public facade preserved for existing callers.
 """

 def __init__(
 self,
 mapping_repository: MappingRepository,
 table_name_resolver: TableNameResolver,
 host_variable_resolver: HostVariableResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.table_name_resolver = table_name_resolver
 self.host_variable_resolver = host_variable_resolver

 self.classifier = RelationshipKeyClassifier()
 self.key_column_resolver = RelationshipKeyColumnResolver(
 mapping_repository=mapping_repository,
 classifier=self.classifier,
)
 self.table_lookup = RelationshipTableLookup(
 mapping_repository=mapping_repository,
 table_name_resolver=table_name_resolver,
)
 self.child_resolver = RelationshipChildResolver(
 mapping_repository=mapping_repository,
 table_name_resolver=table_name_resolver,
 host_variable_resolver=host_variable_resolver,
 key_column_resolver=self.key_column_resolver,
 table_lookup=self.table_lookup,
 classifier=self.classifier,
)

 def resolve_for_child_record(
 self,
 child_record: str,
) -> RelationshipResolution:
 return self.child_resolver.resolve_for_child_record(child_record)

 def foreign_key_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 return self.key_column_resolver.foreign_key_columns_for_record(record_name)

 def primary_key_columns_for_record(
```

```

 self,
 record_name: str,
) -> list[str]:
 return self.key_column_resolver.primary_key_columns_for_record(record_name)

 def parent_key_columns_required_by_children(
 self,
 parent_record: str,
) -> list[str]:
 parent = NameNormalizer.normalize(parent_record)
 output: list[str] = []
 seen: set[str] = set()

 for child in self.mapping_repository.records():
 child = NameNormalizer.normalize(child)

 if child == parent:
 continue

 relation = self.resolve_for_child_record(child)

 if relation.parent_record != parent:
 continue

 for condition in relation.conditions:
 column = NameNormalizer.normalize(condition.parent_column)

 if not column or column in seen:
 continue

 seen.add(column)
 output.append(column)

 return output

 def order_by_columns_for_record(
 self,
 record_name: str,
) -> list[str]:
 return self.key_column_resolver.order_by_columns_for_record(record_name)

 def has_foreign_keys(
 self,
 record_name: str,
) -> bool:
 return self.key_column_resolver.has_foreign_keys(record_name)

__all__ = [
 "RelationshipCondition",
 "RelationshipResolution",
 "RelationshipResolver",
]

```

```

=====
FILE: src\idms_db2_phase2\resolvers\relationship_table_lookup.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer

class RelationshipTableLookup:
 """
 Resolves records from DB2 tables and parent tables from key columns.
 """

 def __init__(
 self,
 *,
 mapping_repository: MappingRepository,
) -> None:
 pass

```

```

 table_name_resolver: TableNameResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.table_name_resolver = table_name_resolver

 def record_for_table(
 self,
 table_name: str,
) -> str:
 table = self.table_name_resolver.resolve_table(table_name)

 if not table:
 return ""

 for record in self.mapping_repository.records():
 record_table = self.table_name_resolver.table_for_record(record)

 if record_table == table:
 return NameNormalizer.normalize(record)

 return ""

 def find_parent_table_by_column(
 self,
 *,
 child_record: str,
 child_column: str,
 primary_key_columns_for_record,
) -> str:
 column = NameNormalizer.normalize(child_column)

 if not column:
 return ""

 child_table = self.table_name_resolver.table_for_record(child_record)

 for record in self.mapping_repository.records():
 normalized_record = NameNormalizer.normalize(record)

 if normalized_record == child_record:
 continue

 table = self.table_name_resolver.table_for_record(normalized_record)

 if not table:
 continue

 if table == child_table:
 continue

 if column in primary_key_columns_for_record(normalized_record):
 return table

 return ""

```

```

=====
FILE: src\idms_db2_phase2\resolvers\table_name_resolver.py
ENCODING: utf-8
=====

```

```

from catalogs.db2_naming_catalog import DB2_TABLE_SUFFIX_EQUIVALENTS
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

class TableNameResolver:
 """
 Resolves DB2 table names.

 Authority:
 - Sheet Mapping provides intended DB2 table names.
 - DCLGEN provides the final available DB2 table name.
 - If Sheet Mapping says TB but DCLGEN has TV, resolve to DCLGEN TV.
 """

```

Example:

- Sheet Mapping: DZBFARTB
- DCLGEN table: DZBFARTV
- Final SQL table: DZBFARTV

"""

```
def __init__(
 self,
 mapping_repository: MappingRepository,
 dclgen_repository: DclgenRepository,
) -> None:
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository

def resolve_table(
 self,
 table_name: str,
) -> str:
 normalized = NameNormalizer.normalize(table_name)

 if not normalized:
 return ""

 for candidate in self.table_candidates(normalized):
 if self.dclgen_repository.has_table(candidate):
 return candidate

 return normalized

def table_for_record(
 self,
 record_name: str,
) -> str:
 mapped_table = self.mapping_repository.db2_table_for_record(record_name)

 if not mapped_table:
 return ""

 return self.resolve_table(mapped_table)

def table_candidates(
 self,
 table_name: str,
) -> list[str]:
 normalized = NameNormalizer.normalize(table_name)

 if not normalized:
 return []

 output: list[str] = [normalized]

 for source_suffix, target_suffix in DB2_TABLE_SUFFIX_EQUIVALENTS:
 if normalized.endswith(source_suffix):
 output.append(
 normalized[: -len(source_suffix)] + target_suffix
)

 cobol_name = NameNormalizer.to_cobol(normalized)
 compact_name = NameNormalizer.compact(normalized)

 if cobol_name:
 output.append(cobol_name)

 if compact_name:
 output.append(compact_name)

 final: list[str] = []

 for item in output:
 normalized_item = NameNormalizer.normalize(item)

 if not normalized_item:
 continue

 if normalized_item in final:
 continue
```

```

 final.append(normalized_item)

 return final

def known_dclgen_tables(
 self,
) -> list[str]:
 return self.dclgen_repository.tables()

def known_mapping_tables(
 self,
) -> list[str]:
 return self.mapping_repository.tables()

def has_resolved_table(
 self,
 table_name: str,
) -> bool:
 resolved = self.resolve_table(table_name)

 if not resolved:
 return False

 return self.dclgen_repository.has_table(resolved)

def context_strength(
 self,
 record_name: str,
) -> int:
 if not record_name:
 return 0

 if self.mapping_repository.has_record(record_name):
 return 100

 return 0

```

```

=====
FILE: src\idms_db2_phase2\resolvers\update_sql_plan_resolver.py
ENCODING: utf-8
=====

```

```

from dataclasses import dataclass, field

from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer

```

```

@dataclass
class UpdateSqlPlan:
 record_name: str = ""
 table_name: str = ""
 key_columns: list[str] = field(default_factory=list)
 update_columns: list[str] = field(default_factory=list)
 diagnostics: list[str] = field(default_factory=list)

 @property
 def is_complete(self) -> bool:
 return bool(
 self.record_name
 and self.table_name
 and self.key_columns
 and self.update_columns
)

```

```

class UpdateSqlPlanResolver:
 """
 Resolves conservative UPDATE SQL plans.

 Generic rules:

```



- Sheet Mapping decides DB2 table and column names.
  - DCLGEN decides whether host variables exist.
  - DB2 PRIMARY KEY metadata decides WHERE columns.
  - Composite primary keys are supported by returning all PK columns.
  - FOREIGN KEY / relationship columns must not be included in WHERE.
  - UPDATE columns come only from changed source fields and update audit fields.
- """

```

def __init__(
 self,
 mapping_repository: MappingRepository,
 dclgen_repository: DclgenRepository,
 table_name_resolver: TableNameResolver,
 host_variable_resolver: HostVariableResolver,
) -> None:
 self.mapping_repository = mapping_repository
 self.dclgen_repository = dclgen_repository
 self.table_name_resolver = table_name_resolver
 self.host_variable_resolver = host_variable_resolver

def resolve(
 self,
 record_name: str,
 changed_source_fields: list[str] | None = None,
) -> UpdateSqlPlan:
 record = NameNormalizer.normalize(record_name)
 plan = UpdateSqlPlan(record_name=record)

 if not record:
 plan.diagnostics.append("Update SQL plan: record name is empty.")
 return plan

 mapped_table = self.mapping_repository.db2_table_for_record(record)
 resolved_table = self.table_name_resolver.resolve_table(mapped_table)

 if not resolved_table:
 plan.diagnostics.append(
 f"Update SQL plan: no resolved DB2 table for record {record}."
)
 return plan

 plan.table_name = resolved_table

 key_columns = self._primary_key_columns_for_record(record)
 key_columns = self._filter_to_dclgen_columns(
 table_name=resolved_table,
 columns=key_columns,
)

 if not key_columns:
 plan.diagnostics.append(
 f"Update SQL plan: no valid primary key columns resolved for record {record}."
)

 update_columns = self.mapping_repository.update_candidate_columns_for_record(
 record_name=record,
 changed_source_fields=changed_source_fields or [],
)
 update_columns = self._filter_to_dclgen_columns(
 table_name=resolved_table,
 columns=update_columns,
)

 if not update_columns:
 plan.diagnostics.append(
 f"Update SQL plan: no conservative update columns resolved for record {record}."
)

 plan.key_columns = self._unique(key_columns)
 plan.update_columns = self._unique(update_columns)

 return plan

def _primary_key_columns_for_record(
 self,
 record_name: str,

```

```

) -> list[str]:
 """
 Return DB2 primary-key columns only.

 This intentionally does not use key_columns_for_record(), because that
 method can include broader IDMS CALC/key metadata.

 WHERE clause rule:
 - Include PRIMARY / KEY columns.
 - Exclude FOREIGN KEY columns.
 - Exclude relationship columns.
 - If multiple PK columns exist, keep all of them for composite PK WHERE.
 """
 if not hasattr(self.mapping_repository, "primary_key_columns_for_record"):
 return []

 return self.mapping_repository.primary_key_columns_for_record(record_name)

def _filter_to_dclgen_columns(
 self,
 table_name: str,
 columns: list[str],
) -> list[str]:
 dclgen_columns = set(
 self.dclgen_repository.column_names_for_table(table_name)
)

 if not dclgen_columns:
 return []

 output: list[str] = []

 for column in columns:
 normalized = NameNormalizer.normalize(column)

 if not normalized:
 continue

 if normalized not in dclgen_columns:
 continue

 if not self.host_variable_resolver.has_host_for_column(
 table_name=table_name,
 column_name=normalized,
):
 continue

 output.append(normalized)

 return self._unique(output)

def _unique(
 self,
 values: list[str],
) -> list[str]:
 output: list[str] = []
 seen: set[str] = set()

 for value in values:
 normalized = NameNormalizer.normalize(value)

 if not normalized:
 continue

 if normalized in seen:
 continue

 seen.add(normalized)
 output.append(normalized)

 return output

```

```
=====
FILE: src\idms_db2_phase2\services__init__.py
ENCODING: utf-8
=====
```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\services\cobol_area_alignment_classifier.py
ENCODING: utf-8
=====
```

```
"""
COBOL Area A / Area B alignment classification helpers.

This module contains decision helpers only. It does not rewrite COBOL lines.
"""

from __future__ import annotations

from patterns.final_feedback_fix_patterns import (
 DIVISION_SECTION_HEADER_PATTERN,
 PARAGRAPH_HEADER_PATTERN,
)
from rules.cobol_statement_rules import NON_PARAGRAPH_SINGLE_WORDS

class CobolAreaAlignmentClassifier:
 """
 Classifies Procedure Division logical lines for safe Area A / continuation
 decisions.
 """

 def is_area_a(
 self,
 logical: str,
) -> bool:
 text = str(logical or "").strip()
 upper = text.upper()

 if not text:
 return False

 if DIVISION_SECTION_HEADER_PATTERN.match(text):
 return True

 if not PARAGRAPH_HEADER_PATTERN.match(text):
 return False

 statement_name = upper.rstrip(".")

 if statement_name in NON_PARAGRAPH_SINGLE_WORDS:
 return False

 return True

 def looks_like_continuation(
 self,
 *,
 current_logical: str,
 next_logical: str,
) -> bool:
 """
 Detect generic continuation lines.

 This deliberately avoids hardcoded field/cursor names.

 Continuation is likely when:
 - current line ends with an operator or boolean connector
 - next line starts with a literal
 - next line starts with a comparison operand, not a COBOL verb/header
 """

 current = str(current_logical or "").strip().upper()
 next_text = str(next_logical or "").strip()
```

```

next_upper = next_text.upper()

if not current or not next_text:
 return False

if current.endswith(("=", ">", "<", ">=", "<=", "OR", "AND", "NOT")):
 return True

if next_text.startswith(("'", '"')):
 return True

first_word = next_upper.split()[0] if next_upper.split() else ""

if first_word in NON_PARAGRAPH_SINGLE_WORDS:
 return False

if DIVISION_SECTION_HEADER_PATTERN.match(next_text):
 return False

if PARAGRAPH_HEADER_PATTERN.match(next_text):
 return False

return True

```

```

=====
FILE: src\idms_db2_phase2\services\cobol_area_alignment_reflow.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/services/cobol_area_alignment_reflow.py
ACTION: REPLACE ENTIRE FILE

```

""""COBOL Area B safe two-line reflow helper.

This module only handles long-line reflow using an existing continuation line. It does not decide Procedure Division state or SQL state.

#### WHAT A CONTINUATION LINE IS

-----  
A fixed-format COBOL statement may continue onto the next line with NO continuation indicator, provided the break falls on a space between two complete words. The '-' indicator in column 7 is required only when a word or literal is split mid-token.

This helper therefore breaks on word boundaries ONLY, and never emits a continuation indicator.

#### CORRECTION - combined-budget truncation

-----  
The previous version joined the statement with its continuation fragment and re-split the pair on EVERY call, against a fixed combined budget, slicing the overflow off the tail:

```

 MOVE AM-CNSTK-479BEFF OF DCLDZBEFFTV
 TO UIT-AM- <- UIT-AM-CN-STOCK

```

Every affected pair measured exactly 60 combined body characters, with the remainder discarded, producing undefined data names.

A two-line statement is correct COBOL. Re-splitting it was unnecessary work, and truncating the result was destructive.

This version:

- PRESERVES the existing break. The pair is re-indented in place; the words are not moved.
- REBALANCES at a word boundary only when the FIRST line alone no longer fits columns 8-72 at its new indent.
- REFUSES - returns [] so the caller leaves both lines exactly as they were - whenever the pair cannot be represented.
- NEVER truncates, never splits a data name, never collapses the statement onto one line.
- Delegates all column arithmetic to FixedFormatLineService. No private copy of the geometry is kept here.

Contract preserved: try\_reflow\_with\_next\_line returns either an empty list, or EXACTLY TWO lines replacing the two it was given. It never adds or removes physical lines.

"""

```
from __future__ import annotations

from idms_db2_phase2.services.cobol_area_alignment_classifier import (
 CobolAreaAlignmentClassifier,
)
from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)
from rules.final_feedback_fix_rules import AREA_B_CONTINUATION_BODY_INDENT

Statement-starting keywords. A next line beginning with one of these is
a NEW statement, not a continuation fragment, and must never be joined
to the line above.
STATEMENT_KEYWORDS = (
 "ACCEPT", "ADD", "ALTER", "CALL", "CANCEL", "CLOSE", "COMPUTE",
 "CONTINUE", "DELETE", "DISPLAY", "DIVIDE", "ELSE", "EVALUATE",
 "EXEC", "EXIT", "GO", "GOBACK", "IF", "INITIALIZE", "INSPECT",
 "MERGE", "MOVE", "MULTIPLY", "OPEN", "PERFORM", "READ", "RELEASE",
 "RETURN", "REWRITE", "SEARCH", "SET", "SORT", "START", "STOP",
 "STRING", "SUBTRACT", "UNSTRING", "WHEN", "WRITE",
)

DIVISION_TOKENS = ("DIVISION", "SECTION")
```

```
class CobolAreaAlignmentReflow:
 """Re-indent a two-line statement without ever truncating it."""

 def __init__(
 self,
 *,
 fixed_format: FixedFormatLineService,
 classifier: CobolAreaAlignmentClassifier,
) -> None:
 self.fixed_format = fixed_format
 self.classifier = classifier

 # =====
 # Public entry point
 # =====
 def try_reflow_with_next_line(
 self,
 *,
 current_line: str,
 next_line: str,
 first_indent: str,
) -> list[str]:
 """Re-indent a statement that spans the current and next lines.

 Returns:
 [] leave both lines untouched
 [first, second] replacements for the two input lines
 """
 if not current_line or not next_line:
 return []

 if not self.fixed_format.is_fixed_line(current_line):
 return []

 if not self.fixed_format.is_fixed_line(next_line):
 return []

 if not self._is_continuation_fragment(next_line):
 return []

 current_logical = self.fixed_format.logical(current_line)
 next_logical = self.fixed_format.logical(next_line)

 if not current_logical or not next_logical:
 return []
```

```

indent = str(first_indent or "")
continuation = indent + AREA_B_CONTINUATION_BODY_INDENT

first_body = indent + current_logical
second_body = continuation + next_logical

--- Case 1: the author's break still works -----
#
Preferred path. The words stay exactly where the generator or
the programmer put them; only the indent changes.
if self.fixed_format.body_fits(first_body) and self.fixed_format.body_fits(
 second_body
):
 return self._render(current_line, next_line, first_body, second_body)

--- Case 2: rebalance at a word boundary -----
#
The first line no longer fits at its new indent. Move whole
words - never part of a word - onto the continuation line.
joined = f"{current_logical} {next_logical}".strip()
bodies = self.fixed_format.wrap_body_lines(
 indent + joined,
 continuation_indent=continuation,
 max_lines=2,
)

if len(bodies) != 2:
 # Cannot be represented in two lines. Refuse rather than cut.
 return []

return self._render(current_line, next_line, bodies[0], bodies[1])

=====
Rendering
=====
def _render(
 self,
 current_line: str,
 next_line: str,
 first_body: str,
 second_body: str,
) -> list[str]:
 """Rebuild both lines, or refuse if either will not assemble."""
 first_left, first_indicator, _b1, first_right = self.fixed_format.split(
 current_line
)
 next_left, next_indicator, _b2, next_right = self.fixed_format.split(
 next_line
)

 first = self.fixed_format.build_or_none(
 first_left,
 first_indicator or " ",
 first_body,
 first_right,
)
 second = self.fixed_format.build_or_none(
 next_left,
 next_indicator or " ",
 second_body,
 next_right,
)

 if first is None or second is None:
 return []

 return [first, second]

=====
Guards
=====
def _is_continuation_fragment(self, line: str) -> bool:
 """True when `line` is the tail of the statement above it.

 A paragraph header, section or division header, scope terminator,
 sentence terminator, comment, page eject or a NEW statement is

```

not a continuation. Joining any of those to the line above would corrupt the program.

```
"""
if self.fixed_format.is_comment_or_control_line(line):
 return False

if self.fixed_format.is_sequence_artifact(line):
 return False

logical = self.fixed_format.logical(line)
if not logical:
 return False

upper = logical.upper()
words = upper.split()

Lone sentence terminator.
if upper == ".":
 return False

Paragraph header: a single word ending in a period.
if len(words) == 1 and upper.endswith("."):
 return False

Scope terminator.
if upper.startswith("END-"):
 return False

Division or section header.
if any(token in words for token in DIVISION_TOKENS):
 return False

A new statement.
if words and words[0].rstrip(".") in STATEMENT_KEYWORDS:
 return False

return True
```

```
=====
FILE: src\idms_db2_phase2\services\cobol_area_alignment_service.py
ENCODING: utf-8
=====
```

```
"""
COBOL Area A / Area B alignment service.
```

Generic behavior:

- Does not change 80-column sequencing.
- Does not change business logic.
- Only realigns Procedure Division executable statements to Area B when safe.
- Never truncates a fixed-format COBOL body to force alignment.
- Safely reflows long two-line Procedure Division statements by using the existing continuation line when available.

No program names, cursor names, DB2 tables, DB2 columns, DCLGEN groups, or host variables are hardcoded.

```
"""
```

```
from __future__ import annotations

from patterns.final_feedback_fix_patterns import (
 END_EXEC_PATTERN,
 EXEC_SQL_START_PATTERN,
 PROCEDURE_DIVISION_TOKEN_PATTERN,
)
from rules.final_feedback_fix_rules import (
 AREA_B_BODY_INDENT,
 SQL_BODY_INDENT,
)
from idms_db2_phase2.services.cobol_area_alignment_classifier import (
 CobolAreaAlignmentClassifier,
)
from idms_db2_phase2.services.cobol_area_alignment_reflow import (
 CobolAreaAlignmentReflow,
)
```

```

from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)

class CobolAreaAlignmentService:
 """
 Aligns generated Procedure Division executable statements to Area B.

 Area B starts at physical column 12.
 Since COBOL body starts at physical column 8, four body spaces are used.

 Safety:
 - If adding the indent would exceed the fixed body width, the line is not
 blindly truncated.
 - If the next line is a continuation, this service combines both logical
 fragments and re-wraps them back into the same two physical lines.
 - If safe reflow is not possible, the original line is preserved.
 """

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()
 self.classifier = CobolAreaAlignmentClassifier()
 self.reflow = CobolAreaAlignmentReflow(
 fixed_format=self.fixed_format,
 classifier=self.classifier,
)

 def align(self, text: str) -> str:
 if not text:
 return ""

 lines = str(text).splitlines()
 output: list[str] = []

 in_procedure_division = False
 in_exec_sql = False
 index = 0

 while index < len(lines):
 line = lines[index]
 logical = self.fixed_format.logical(line)

 if PROCEDURE_DIVISION_TOKEN_PATTERN.search(logical):
 in_procedure_division = True
 output.append(line)
 index += 1
 continue

 if not in_procedure_division:
 output.append(line)
 index += 1
 continue

 if self.fixed_format.is_comment_or_control_line(line):
 output.append(line)
 index += 1
 continue

 if not logical:
 output.append(line)
 index += 1
 continue

 if EXEC_SQL_START_PATTERN.match(logical):
 in_exec_sql = True
 output.append(
 self._replace_body_when_safe(
 line=line,
 new_body=SQL_BODY_INDENT + logical,
)
)
 index += 1

```



```

 continue

 if in_exec_sql:
 output.append(
 self._replace_body_when_safe(
 line=line,
 new_body=SQL_BODY_INDENT + logical,
)
)

 if END_EXEC_PATTERN.search(logical):
 in_exec_sql = False

 index += 1
 continue

 if self.classifier.is_area_a(logical):
 output.append(
 self._replace_body_when_safe(
 line=line,
 new_body=logical,
)
)
 index += 1
 continue

 aligned_body = AREA_B_BODY_INDENT + logical

 if self._body_fits(aligned_body):
 output.append(
 self.fixed_format.replace_body(
 line,
 aligned_body,
)
)
 index += 1
 continue

 reflowed_lines = self.reflow.try_reflow_with_next_line(
 current_line=line,
 next_line=lines[index + 1] if index + 1 < len(lines) else "",
 first_indent=AREA_B_BODY_INDENT,
)

 if reflowed_lines:
 output.extend(reflowed_lines)
 index += 2
 continue

 output.append(line)
 index += 1

 return "\n".join(output)

def _replace_body_when_safe(
 self,
 *,
 line: str,
 new_body: str,
) -> str:
 """
 Replace fixed-format body only when it will not truncate.

 This prevents corruption such as losing operators at the end of long
 Procedure Division statements.
 """

 if not self.fixed_format.is_fixed_line(line):
 return self.fixed_format.replace_body(line, new_body)

 if not self._body_fits(new_body):
 return line

 return self.fixed_format.replace_body(line, new_body)

def _body_fits(

```

```

 self,
 body: str,
) -> bool:
 return len(str(body or "")) <= self.fixed_format.BODY_WIDTH

```

```

=====
FILE: src\idms_db2_phase2\services\cobol_condition_scanner.py
ENCODING: utf-8
=====

```

```

LOCATION: src/idms_db2_phase2/services/cobol_condition_scanner.py
ACTION: CREATE NEW FILE
"""Finds where a COBOL conditional expression ends.

```

A hand-written IF frequently spans lines:

```

IF (DA-CPTAFS OF DCL... < DA-ARCH-YMD
 AND DA-CPTAFS OF DCL... NOT = '00000000') OR
 (DA-CRFMAS OF DCL... < DA-ARCH-YMD AND
 DA-CPTAFS OF DCL... = '00000000')
PERFORM ...

```

Any pass that extracts a block MUST NOT cut between the IF keyword and the first statement, or the condition is orphaned and the program will not compile. Read-only: this service rewrites nothing.

```

from __future__ import annotations

```

```

from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)
from patterns.structural_safety_patterns import IF_START_PATTERN
from rules.structural_safety_rules import (
 CONDITION_CONTINUATION_TOKENS,
 CONDITION_SCAN_LIMIT,
 CONDITION_TERMINATING_VERBS,
)

```

```

COMMENT_PREFIXES = ("*", "/")

```

```

class CobolConditionScanner:

```

```

 """Locates the last line of a conditional expression."""

```

```

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()

```

```

 # ----- public

```

```

 def is_condition_start(self, line: str) -> bool:
 return bool(IF_START_PATTERN.match(self._logical(line)))

```

```

 def condition_end_index(self, lines: list[str], if_index: int) -> int:
 """Index of the LAST line belonging to the IF condition.

```

```

 Returns if_index when the whole condition fits on the IF line.
 Returns -1 when the condition cannot be resolved, which callers
 must treat as "refuse to extract".
 """

```

```

 if not 0 <= if_index < len(lines):
 return -1

```

```

 logical = self._logical(lines[if_index])
 if not IF_START_PATTERN.match(logical):
 return -1

```

```

 remainder = logical[2:].strip()
 if self._holds_statement(remainder) and self._balanced(remainder):
 return if_index

```

```

 depth = self._depth(remainder)
 end = min(len(lines), if_index + 1 + CONDITION_SCAN_LIMIT)

```

```

 for index in range(if_index + 1, end):
 body = self._logical(lines[index])

 if not body or self._is_comment(body):
 continue

 depth += self._depth(body)

 if depth <= 0 and self._starts_statement(body):
 return index - 1

 if depth <= 0 and not self._continues_condition(body):
 return index - 1

 return -1

def spans_multiple_lines(self, lines: list[str], if_index: int) -> bool:
 end = self.condition_end_index(lines, if_index)
 return end < 0 or end > if_index

def condition_line_count(self, lines: list[str], if_index: int) -> int:
 end = self.condition_end_index(lines, if_index)
 if end < 0:
 return CONDITION_SCAN_LIMIT
 return (end - if_index) + 1

----- helpers
def _logical(self, line: str) -> str:
 try:
 return str(self.fixed_format.logical(line) or "").strip()
 except Exception: # noqa: BLE001
 return str(line or "").strip()

@staticmethod
def _is_comment(body: str) -> bool:
 return body.startswith(COMMENT_PREFIXES)

@staticmethod
def _depth(body: str) -> int:
 return body.count("(") - body.count(")")

@staticmethod
def _balanced(body: str) -> bool:
 return body.count("(") == body.count(")")

@staticmethod
def _first_word(body: str) -> str:
 parts = str(body or "").strip().split()
 return parts[0].upper() if parts else ""

def _starts_statement(self, body: str) -> bool:
 return self._first_word(body) in CONDITION_TERMINATING_VERBS

def _continues_condition(self, body: str) -> bool:
 word = self._first_word(body)
 if word in CONDITION_CONTINUATION_TOKENS:
 return True
 return body.startswith("(")

def _holds_statement(self, remainder: str) -> bool:
 """True when the IF line already carries its first statement."""
 for verb in CONDITION_TERMINATING_VERBS:
 if f" {verb} " in f" {remainder.upper()} ":
 return True
 return False

__all__ = ["CobolConditionScanner"]

```

```
=====
FILE: src\idms_db2_phase2\services\conversion_service.py
ENCODING: utf-8
=====
```

```
from idms_db2_phase2.orchestration.conversion_service import ConversionService
__all__ = [
 "ConversionService",
]
```

```
=====
FILE: src\idms_db2_phase2\services\cursor_select_column_parser.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)
from patterns.final_feedback_fix_patterns import (
 ASC_DESC_PATTERN,
 END_EXEC_PATTERN,
 FOR_READ_ONLY_PATTERN,
 FROM_PATTERN,
 ORDER_BY_PATTERN,
 SELECT_ITEM_PATTERN,
 SELECT_KEYWORD_PATTERN,
 SQL_NAME_TOKEN_PATTERN,
)
from rules.cursor_select_minimizer_rules import SQL_NON_COLUMN_KEYWORDS

class CursorSelectColumnParser:
 """
 Parses SELECT columns and ORDER BY columns from a DECLARE CURSOR block.

 This parser does not remove or rewrite lines.
 """

 def __init__(
 self,
 fixed_format: FixedFormatLineService,
) -> None:
 self.fixed_format = fixed_format

 def select_columns(
 self,
 block: list[str],
) -> list[str]:
 columns: list[str] = []
 in_select = False

 for line in block:
 logical = self.fixed_format.logical(line)

 if SELECT_KEYWORD_PATTERN.match(logical):
 in_select = True
 continue

 if in_select and FROM_PATTERN.search(logical):
 break

 if in_select:
 column = self.select_column_from_logical(logical)

 if column:
 columns.append(column)

 return columns

 def select_column_from_logical(
 self,
```

```

 logical: str,
) -> str:
 text = str(logical or "").strip()

 if not text:
 return ""

 if self.is_sql_non_column_keyword(text):
 return ""

 match = SELECT_ITEM_PATTERN.match(text)

 if not match:
 return ""

 column = match.group("column").upper()

 if self.is_sql_non_column_keyword(column):
 return ""

 return column

def order_by_columns(
 self,
 block: list[str],
) -> list[str]:
 columns: list[str] = []
 in_order_by = False

 for line in block:
 logical = self.fixed_format.logical(line)

 if ORDER_BY_PATTERN.search(logical):
 in_order_by = True
 remainder = ORDER_BY_PATTERN.sub("", logical).strip()

 if remainder:
 columns.extend(self.parse_order_by_columns(remainder))

 continue

 if in_order_by:
 if FOR_READ_ONLY_PATTERN.search(logical) or END_EXEC_PATTERN.search(logical):
 break

 columns.extend(self.parse_order_by_columns(logical))

 return columns

def parse_order_by_columns(
 self,
 text: str,
) -> list[str]:
 columns: list[str] = []

 for item in str(text or "").split(","):
 token = item.strip()
 token = ASC_DESC_PATTERN.sub("", token).strip()
 token = token.strip(",")

 if SQL_NAME_TOKEN_PATTERN.match(token):
 columns.append(token.upper())

 return columns

def is_sql_non_column_keyword(
 self,
 value: str,
) -> bool:
 text = str(value or "").strip().upper().lstrip(",").strip()
 return text in SQL_NON_COLUMN_KEYWORDS

def normalize_name(
 self,
 value: str,
) -> str:

```

```
return str(value or "").strip().upper().replace("-", "_")
```

```
=====
FILE: src\idms_db2_phase2\services\cursor_select_fetch_rewriter.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.services.cursor_select_column_parser import (
 CursorSelectColumnParser,
)
from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)
from patterns.final_feedback_fix_patterns import (
 END_EXEC_PATTERN,
 FROM_PATTERN,
 HOST_REFERENCE_PATTERN,
 SELECT_KEYWORD_PATTERN,
)
from rules.cursor_select_minimizer_rules import (
 FETCH_HOST_BODY_PREFIX,
 SELECT_FIRST_ITEM_BODY_PREFIX,
 SELECT_NEXT_ITEM_BODY_PREFIX,
)

class CursorSelectFetchRewriter:
 """
 Removes SELECT and FETCH INTO items by position and normalizes commas only
 within their respective scopes.
 """

 def __init__(
 self,
 *,
 fixed_format: FixedFormatLineService,
 column_parser: CursorSelectColumnParser,
) -> None:
 self.fixed_format = fixed_format
 self.column_parser = column_parser

 def remove_select_positions(
 self,
 block: list[str],
 positions: set[int],
) -> list[str]:
 output: list[str] = []
 in_select = False
 position = -1

 for line in block:
 logical = self.fixed_format.logical(line)

 if SELECT_KEYWORD_PATTERN.match(logical):
 in_select = True
 output.append(line)
 continue

 if in_select and FROM_PATTERN.search(logical):
 in_select = False
 output = self.normalize_select_commas(output)
 output.append(line)
 continue

 if in_select:
 column = self.column_parser.select_column_from_logical(logical)

 if column:
 position += 1

 if position in positions:
 continue
```

```

 output.append(line)

 return self.normalize_select_commas(output)

def remove_fetch_positions(
 self,
 block: list[str],
 positions: set[int],
) -> list[str]:
 output: list[str] = []
 in_into = False
 position = -1

 for line in block:
 logical = self.fixed_format.logical(line)

 if logical.upper() == "INTO":
 in_into = True
 output.append(line)
 continue

 if in_into and END_EXEC_PATTERN.search(logical):
 in_into = False
 output = self.normalize_fetch_commas(output)
 output.append(line)
 continue

 if in_into and HOST_REFERENCE_PATTERN.search(logical):
 position += 1

 if position in positions:
 continue

 output.append(line)

 return self.normalize_fetch_commas(output)

def normalize_select_commas(
 self,
 lines: list[str],
) -> list[str]:
 result = list(lines)
 indexes: list[int] = []
 in_select = False

 for index, line in enumerate(result):
 logical = self.fixed_format.logical(line)

 if SELECT_KEYWORD_PATTERN.match(logical):
 in_select = True
 continue

 if in_select and FROM_PATTERN.search(logical):
 break

 if not in_select:
 continue

 column = self.column_parser.select_column_from_logical(logical)

 if column:
 indexes.append(index)

 for item_number, line_index in enumerate(indexes):
 line = result[line_index]
 logical = self.fixed_format.logical(line)
 column = logical.lstrip(",").strip()

 if item_number == 0:
 body = f"{SELECT_FIRST_ITEM_BODY_PREFIX}{column}"
 else:
 body = f"{SELECT_NEXT_ITEM_BODY_PREFIX}{column}"

 result[line_index] = self.fixed_format.replace_body(line, body)

 return result

```

```

def normalize_fetch_commas(
 self,
 lines: list[str],
) -> list[str]:
 result = list(lines)
 indexes: list[int] = []
 in_into = False

 for index, line in enumerate(result):
 logical = self.fixed_format.logical(line)

 if logical.upper() == "INTO":
 in_into = True
 continue

 if in_into and END_EXEC_PATTERN.search(logical):
 break

 if not in_into:
 continue

 if HOST_REFERENCE_PATTERN.search(logical):
 indexes.append(index)

 for item_number, line_index in enumerate(indexes):
 line = result[line_index]
 logical = self.fixed_format.logical(line).rstrip(",")

 if item_number < len(indexes) - 1:
 logical = logical + ","

 result[line_index] = self.fixed_format.replace_body(
 line,
 FETCH_HOST_BODY_PREFIX + logical,
)

 return result

```

```

=====
FILE: src\idms_db2_phase2\services\cursor_select_minimizer_service.py
ENCODING: utf-8
=====

```

```

"""
Cursor SELECT minimizer service.

```

Generic behavior:

- Removes columns selected/fetched only for ORDER BY when safe.
- Does not hardcode cursor names, table names, columns, or host variables.
- Keeps SELECT and FETCH INTO synchronized by position.

Important:

- Comma normalization is scoped only to SELECT item lines between SELECT and FROM.
- FETCH comma normalization is scoped only to host variables between INTO and END-EXEC.
- SQL keywords such as WHERE, FROM, ORDER BY, and FOR READ ONLY are never treated as selected columns.

```

"""

```

```

from __future__ import annotations

from idms_db2_phase2.services.cursor_select_column_parser import (
 CursorSelectColumnParser,
)
from idms_db2_phase2.services.cursor_select_fetch_rewriter import (
 CursorSelectFetchRewriter,
)
from idms_db2_phase2.services.cursor_select_usage_guard import (
 CursorSelectUsageGuard,
)
from idms_db2_phase2.services.cursor_sql_block_scanner import CursorSqlBlockScanner
from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)
from rules.final_feedback_fix_rules import ORDER_BY_COLUMNS_IN_SELECT_DEFAULT

```



```

class CursorSelectMinimizerService:
 """
 Removes order-by-only columns from cursor SELECT/FETCH when safe.

 Safety:
 - If the SQL column appears elsewhere outside the cursor declare block,
 keep it.
 - If the host-like COBOL name appears elsewhere outside the matching fetch
 block, keep it.
 - SELECT and FETCH are modified by position only when both are structurally
 safe.
 """

 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
 require_order_by_columns_in_select: bool = ORDER_BY_COLUMNS_IN_SELECT_DEFAULT,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()
 self.require_order_by_columns_in_select = require_order_by_columns_in_select

 self.block_scanner = CursorSqlBlockScanner(self.fixed_format)
 self.column_parser = CursorSelectColumnParser(self.fixed_format)
 self.usage_guard = CursorSelectUsageGuard(self.fixed_format)
 self.rewriter = CursorSelectFetchRewriter(
 fixed_format=self.fixed_format,
 column_parser=self.column_parser,
)

 def minimize(
 self,
 text: str,
) -> str:
 source = str(text or "")

 if not source:
 return ""

 if self.require_order_by_columns_in_select:
 return source

 lines = source.splitlines()
 declare_blocks = self.block_scanner.find_declare_blocks(lines)
 fetch_blocks = self.block_scanner.find_fetch_blocks(lines)

 removals_by_cursor = self._removal_positions_by_cursor(
 lines=lines,
 declare_blocks=declare_blocks,
 fetch_blocks=fetch_blocks,
)

 if not removals_by_cursor:
 return source

 updated = self._apply_select_removals(
 lines=lines,
 declare_blocks=declare_blocks,
 removals_by_cursor=removals_by_cursor,
)

 fetch_blocks = self.block_scanner.find_fetch_blocks(updated)

 updated = self._apply_fetch_removals(
 lines=updated,
 fetch_blocks=fetch_blocks,
 removals_by_cursor=removals_by_cursor,
)

 return "\n".join(updated)

 def _removal_positions_by_cursor(
 self,
 *,
 lines: list[str],

```

```

 declare_blocks: list[tuple[str, int, int]],
 fetch_blocks: list[tuple[str, int, int]],
) -> dict[str, set[int]]:
 removals_by_cursor: dict[str, set[int]] = {}

 for cursor, start, end in declare_blocks:
 block = lines[start : end + 1]
 select_columns = self.column_parser.select_columns(block)
 order_columns = self.column_parser.order_by_columns(block)

 if not select_columns or not order_columns:
 continue

 order_set = {
 self.column_parser.normalize_name(item)
 for item in order_columns
 }
 removable: set[int] = set()

 for index, column in enumerate(select_columns):
 if self.column_parser.normalize_name(column) not in order_set:
 continue

 if self.usage_guard.has_non_order_usage(
 lines=lines,
 declare_range=(start, end),
 fetch_blocks=fetch_blocks,
 cursor=cursor,
 column=column,
):
 continue

 removable.add(index)

 if removable:
 removals_by_cursor[cursor] = removable

 return removals_by_cursor

def _apply_select_removals(
 self,
 *,
 lines: list[str],
 declare_blocks: list[tuple[str, int, int]],
 removals_by_cursor: dict[str, set[int]],
) -> list[str]:
 updated = list(lines)

 for cursor, start, end in sorted(
 declare_blocks,
 key=lambda item: item[1],
 reverse=True,
):
 removable = removals_by_cursor.get(cursor)

 if not removable:
 continue

 block = updated[start : end + 1]
 updated[start : end + 1] = self.rewriter.remove_select_positions(
 block,
 removable,
)

 return updated

def _apply_fetch_removals(
 self,
 *,
 lines: list[str],
 fetch_blocks: list[tuple[str, int, int]],
 removals_by_cursor: dict[str, set[int]],
) -> list[str]:
 updated = list(lines)

 for cursor, start, end in sorted(

```

```

 fetch_blocks,
 key=lambda item: item[1],
 reverse=True,
):
 removable = removals_by_cursor.get(cursor)

 if not removable:
 continue

 block = updated[start : end + 1]
 updated[start : end + 1] = self.rewriter.remove_fetch_positions(
 block,
 removable,
)

 return updated

```

```

=====
FILE: src\idms_db2_phase2\services\cursor_select_usage_guard.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)

```

```

class CursorSelectUsageGuard:
 """
 Checks whether a candidate removable SELECT column is used outside its
 DECLARE CURSOR block and matching FETCH block.
 """

 def __init__(
 self,
 fixed_format: FixedFormatLineService,
) -> None:
 self.fixed_format = fixed_format

 def has_non_order_usage(
 self,
 *,
 lines: list[str],
 declare_range: tuple[int, int],
 fetch_blocks: list[tuple[str, int, int]],
 cursor: str,
 column: str,
) -> bool:
 host_like = column.replace("_", "-").upper()
 column_upper = column.upper()

 fetch_range = self.matching_fetch_range(fetch_blocks, cursor)

 for index, line in enumerate(lines):
 if self.index_in_range(index, declare_range):
 continue

 if fetch_range and self.index_in_range(index, fetch_range):
 continue

 logical = self.fixed_format.logical(line).upper()

 if column_upper in logical:
 return True

 if host_like in logical:
 return True

 return False

 def matching_fetch_range(
 self,
 fetch_blocks: list[tuple[str, int, int]],
) -> tuple[int, int]:

```

```

 cursor: str,
) -> tuple[int, int] | None:
 for fetch_cursor, start, end in fetch_blocks:
 if fetch_cursor == cursor:
 return start, end

 return None

 def index_in_range(
 self,
 index: int,
 value_range: tuple[int, int],
) -> bool:
 start, end = value_range
 return start <= index <= end

```

```

=====
FILE: src\idms_db2_phase2\services\cursor_sql_block_scanner.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)
from patterns.final_feedback_fix_patterns import (
 DECLARE_CURSOR_PATTERN,
 END_EXEC_PATTERN,
 FETCH_CURSOR_PATTERN,
)

```

```

class CursorSqlBlockScanner:
 """
 Finds DECLARE CURSOR and FETCH cursor EXEC SQL blocks.

 This scanner does not parse SELECT columns or rewrite SQL.
 """

 def __init__(
 self,
 fixed_format: FixedFormatLineService,
) -> None:
 self.fixed_format = fixed_format

 def find_declare_blocks(
 self,
 lines: list[str],
) -> list[tuple[str, int, int]]:
 blocks: list[tuple[str, int, int]] = []
 in_exec_sql = False
 start = -1
 cursor = ""

 for index, line in enumerate(lines):
 logical = self.fixed_format.logical(line)

 if logical.upper().startswith("EXEC SQL"):
 in_exec_sql = True
 start = index
 cursor = ""
 continue

 if in_exec_sql and not cursor:
 match = DECLARE_CURSOR_PATTERN.search(logical)

 if match:
 cursor = match.group("cursor").upper()

 if in_exec_sql and END_EXEC_PATTERN.search(logical):
 if cursor:
 blocks.append((cursor, start, index))

 in_exec_sql = False

```

```

 start = -1
 cursor = ""

 return blocks

def find_fetch_blocks(
 self,
 lines: list[str],
) -> list[tuple[str, int, int]]:
 blocks: list[tuple[str, int, int]] = []
 in_exec_sql = False
 start = -1
 cursor = ""

 for index, line in enumerate(lines):
 logical = self.fixed_format.logical(line)

 if logical.upper().startswith("EXEC SQL"):
 in_exec_sql = True
 start = index
 cursor = ""
 continue

 if in_exec_sql and not cursor:
 match = FETCH_CURSOR_PATTERN.search(logical)

 if match:
 cursor = match.group("cursor").upper()

 if in_exec_sql and END_EXEC_PATTERN.search(logical):
 if cursor:
 blocks.append((cursor, start, index))

 in_exec_sql = False
 start = -1
 cursor = ""

 return blocks

```

```

=====
FILE: src\idms_db2_phase2\services\db2_comment_readability_service.py
ENCODING: utf-8
=====

```

```

"""

```

```

DB2 comment readability service.

```

```

Improves the readability of generated DB2 comments only, preserving original
business comments exactly. Extracted from the former final fix composer.
"""

```

```

from __future__ import annotations

```

```

from patterns.final_feedback_fix_patterns import (
 DB2_COLON_COMMENT_BODY_PATTERN,
 DB2_SPACE_COMMENT_BODY_PATTERN,
)
from idms_db2_phase2.services.fixed_format_line_service import (
 FixedFormatLineService,
)

```

```

class Db2CommentReadabilityService:
 def __init__(
 self,
 fixed_format: FixedFormatLineService | None = None,
) -> None:
 self.fixed_format = fixed_format or FixedFormatLineService()

 def apply(self, text: str) -> str:
 if not text:
 return ""

 lines = str(text or "").splitlines()
 output: list[str] = []

```

```

db2_colon_continuation_pending = False

for line in lines:
 if not line:
 output.append(line)
 db2_colon_continuation_pending = False
 continue

 if self.fixed_format.is_fixed_line(line):
 left, indicator, body, right = self.fixed_format.split(line)

 if indicator != "*":
 output.append(line)
 db2_colon_continuation_pending = False
 continue

 clean_body = str(body or "").rstrip().strip()

 if not clean_body:
 output.append(line)
 db2_colon_continuation_pending = False
 continue

 is_colon = bool(
 DB2_COLON_COMMENT_BODY_PATTERN.match(clean_body)
)
 is_space = bool(
 DB2_SPACE_COMMENT_BODY_PATTERN.match(clean_body)
)

 if is_colon:
 normalized = DB2_COLON_COMMENT_BODY_PATTERN.sub(
 "DB2:", clean_body, count=1
).strip()
 updated = self._try_build_fixed_comment_line(
 left=left, right=right, comment_body=normalized
)
 output.append(updated or line)
 db2_colon_continuation_pending = True
 continue

 if is_space:
 normalized = DB2_SPACE_COMMENT_BODY_PATTERN.sub(
 "DB2 ", clean_body, count=1
).strip()
 updated = self._try_build_fixed_comment_line(
 left=left, right=right, comment_body=normalized
)
 output.append(updated or line)
 db2_colon_continuation_pending = False
 continue

 if db2_colon_continuation_pending:
 updated = self._try_build_fixed_comment_line(
 left=left, right=right, comment_body=clean_body
)
 output.append(updated or line)
 db2_colon_continuation_pending = False
 continue

 output.append(line)
 db2_colon_continuation_pending = False
 continue

 stripped = str(line or "").strip()

 if not stripped:
 output.append(line)
 db2_colon_continuation_pending = False
 continue

 is_colon = bool(DB2_COLON_COMMENT_BODY_PATTERN.match(stripped))
 is_space = bool(DB2_SPACE_COMMENT_BODY_PATTERN.match(stripped))

 if is_colon:
 normalized = DB2_COLON_COMMENT_BODY_PATTERN.sub(

```

```

 "DB2:", stripped, count=1
).strip()
 output.append(f"* {normalized}")
 db2_colon_continuation_pending = True
 continue

 if is_space:
 normalized = DB2_SPACE_COMMENT_BODY_PATTERN.sub(
 "DB2 ", stripped, count=1
).strip()
 output.append(f"* {normalized}")
 db2_colon_continuation_pending = False
 continue

 if db2_colon_continuation_pending and stripped.startswith("*"):
 continuation = stripped[1:].lstrip()
 output.append(f"* {continuation}")
 db2_colon_continuation_pending = False
 continue

 output.append(line)
 db2_colon_continuation_pending = False

 return "\n".join(output).rstrip() + "\n"

def _try_build_fixed_comment_line(
 self,
 left: str,
 right: str,
 comment_body: str,
) -> str:
 clean_body = str(comment_body or "").strip()

 if not clean_body:
 return ""

 body = f" {clean_body}"

 if len(body) > self.fixed_format.BODY_WIDTH:
 return ""

 return self.fixed_format.build(left, "*", body, right)

```

```

=====
FILE: src\idms_db2_phase2\services\db2_date_format_service.py
ENCODING: utf-8
=====

```

```

"""

```

```

DB2 date format service.

```

```

Generic behavior:

```

- Does not hardcode program, table, field, or DCLGEN names.
  - Keeps current European format unless configured otherwise.
  - Adjusts generated DB2 date helper references and sentinel literals.
- ```

"""

```

```

from __future__ import annotations

```

```

from patterns.final_feedback_fix_patterns import (
    DATE_HELPER_EUR_PATTERN,
    DATE_HIGH_EUR_LITERAL_PATTERN,
    DATE_LOW_EUR_LITERAL_PATTERN,
)
from rules.final_feedback_fix_rules import (
    DATE_FORMAT_ALIASES,
    DB2_DATE_HELPER_BY_FORMAT,
    DB2_DATE_HIGH_LITERAL_BY_FORMAT,
    DB2_DATE_LOW_LITERAL_BY_FORMAT,
    DEFAULT_DB2_DATE_EXTERNAL_FORMAT,
)

```

```

class Db2DateFormatService:
    """

```

Applies configured DB2 external date format to generated date conversion.

This service intentionally does not infer business semantics. It only transforms generated generic date helper references when a site/DCLGEN external date format is provided.

```
"""

def __init__(
    self,
    date_external_format: str | None = None,
) -> None:
    self.date_external_format = self.normalize_format(
        date_external_format or DEFAULT_DB2_DATE_EXTERNAL_FORMAT
    )

def apply(self, text: str) -> str:
    source = str(text or "")

    if not source:
        return ""

    if self.date_external_format == "DD.MM.YYYY":
        return source

    target_helper = DB2_DATE_HELPER_BY_FORMAT[self.date_external_format]
    target_low = DB2_DATE_LOW_LITERAL_BY_FORMAT[self.date_external_format]
    target_high = DB2_DATE_HIGH_LITERAL_BY_FORMAT[self.date_external_format]

    updated = DATE_HELPER_EUR_PATTERN.sub(target_helper, source)
    updated = DATE_LOW_EUR_LITERAL_PATTERN.sub(target_low, updated)
    updated = DATE_HIGH_EUR_LITERAL_PATTERN.sub(target_high, updated)

    return updated

def normalize_format(self, value: str) -> str:
    text = str(value or "").strip().upper()

    if not text:
        return DEFAULT_DB2_DATE_EXTERNAL_FORMAT

    return DATE_FORMAT_ALIASES.get(
        text,
        DEFAULT_DB2_DATE_EXTERNAL_FORMAT,
    )


```

```
=====
FILE: src\idms_db2_phase2\services\final_sequence_resequencer_service.py
ENCODING: utf-8
=====
```

```
"""
Final sequence resequencer service.
```

Generic behavior:

- Resequences already fixed-format 80-column COBOL lines.
- Preserves column 7 indicator.
- Preserves columns 8-72 COBOL body.
- Rewrites only columns 1-6 and 73-80.
- Does not hardcode program names, tables, columns, cursors, DCLGEN names, host variables, or business fields.

Purpose:

- Remove sequence gaps introduced by safe generated-line cleanup.

```
"""

from __future__ import annotations
```

```
from rules.fixed_format_rules import (
    DEFAULT_LEFT_START,
    DEFAULT_LEFT_STEP,
    DEFAULT_RIGHT_START,
    DEFAULT_RIGHT_STEP,
    TOTAL_WIDTH,
)
```



```

class FinalSequenceResequencerService:
    """
    Resequences final manual-style COBOL output.

    Physical layout:
    - Columns 1-6   : left sequence number
    - Column 7      : indicator area
    - Columns 8-72  : COBOL body
    - Columns 73-80 : right sequence number

    This service assumes the final output is already fixed-format.
    It does not reformat or wrap COBOL statements.
    """

    LEFT_START = 0
    LEFT_END = 6
    RIGHT_START = 72
    RIGHT_END = 80

    def resequence(
        self,
        text: str,
        left_start: int = DEFAULT_LEFT_START,
        left_step: int = DEFAULT_LEFT_STEP,
        right_start: int = DEFAULT_RIGHT_START,
        right_step: int = DEFAULT_RIGHT_STEP,
    ) -> str:
        source = str(text or "")

        if not source:
            return ""

        output: list[str] = []
        left_value = int(left_start)
        right_value = int(right_start)

        for line in source.splitlines():
            if not self._is_fixed_format_line(line):
                output.append(line)
                continue

            output.append(
                self._replace_sequence(
                    line=line,
                    left_value=left_value,
                    right_value=right_value,
                )
            )

            left_value += int(left_step)
            right_value += int(right_step)

        return "\n".join(output).rstrip() + "\n"

    def _is_fixed_format_line(
        self,
        line: str,
    ) -> bool:
        text = str(line or "")

        if len(text) != TOTAL_WIDTH:
            return False

        if not text[self.LEFT_START:self.LEFT_END].isdigit():
            return False

        if not text[self.RIGHT_START:self.RIGHT_END].isdigit():
            return False

        return True

    def _replace_sequence(
        self,
        line: str,
        left_value: int,

```

```

        right_value: int,
    ) -> str:
        text = str(line or "")

        left = f"{{left_value:06d}}"
        right = f"{{right_value:08d}}"

        return (
            left
            + text[self.LEFT_END:self.RIGHT_START]
            + right
        )

```

```

=====
FILE: src\idms_db2_phase2\services\fixed_format_line_service.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/services/fixed_format_line_service.py
# ACTION: REPLACE ENTIRE FILE

```

"""Fixed-format COBOL line helper service.

No regex.
No business hardcoding.

Purpose:

- Preserve existing left sequence, indicator, body, and right sequence layout.
- Allow services to update only the COBOL body area safely.

CORRECTION 1 - silent truncation

build() previously did:

```
safe_body = str(body or "")[: self.BODY_WIDTH].ljust(self.BODY_WIDTH)
```

which cut any body longer than columns 8-72 without telling anyone, so

```
MOVE NR-IDGOOD-479EVEF OF DCLDZEVEFTV    TO WS-NR-ID-GOOD
```

was written out as

```
MOVE NR-IDGOOD-479EVEF OF DCLDZEVEFTV    TO WS-NR-ID-GO
```

Truncation is never a correct outcome for COBOL source:

- build() raises BodyOverflowError instead of truncating.
- build_or_none() returns None instead of raising.
- replace_body() is total: returns the ORIGINAL line unchanged when the new body does not fit.
- replace_body_wrapped() returns one or more physical lines.

CORRECTION 2 - sequence numbers leaking into the body

is_fixed_line() previously required a full 80 columns AND a numeric right sequence:

```
len(text) >= 80 and text[0:6].isdigit() and text[72:80].isdigit()
```

A hand-maintained member often carries a short line such as

```
001690/
```

(7 characters, page eject, no right sequence). That failed the test, so split() returned the WHOLE line as the body. The left sequence and the indicator then became COBOL text, and a later terminator pass produced

```
001690/.
```

which is not a valid COBOL statement.

A line is now fixed-format when columns 1-6 are numeric and column 7 holds a valid indicator. The right sequence is optional. split() pads short lines so column access is always safe.

CORRECTION 3 - partial sequence dropped

```

-----
build() assembled 80 columns only when BOTH sequences were present. With the
relaxed parser a short line yields a left sequence and an empty right
sequence, so the left sequence was being discarded. build() now assembles
whenever EITHER sequence is present and pads the missing side.

```

CORRECTION 4 - two-line wrap ceiling

```

-----
wrap_body() refused any statement needing more than two physical lines, so
long generated bodies were silently skipped. wrap_body_lines() wraps into N
lines. wrap_body() keeps its original two-line contract for existing callers.

```

```

Migration note: any caller that relied on build() truncating must switch to
replace_body_wrapped() or pre-check with body_fits().
"""

```

```

from __future__ import annotations

```

```

class BodyOverflowError(ValueError):
    """Raised when a COBOL body would not fit columns 8-72."""

```

```

class FixedFormatLineService:
    """Utility for fixed-format COBOL lines.

```

```

    Physical layout:
    - Columns 1-6 : left sequence
    - Column 7 : indicator
    - Columns 8-72 : COBOL body
    - Columns 73-80: right sequence
    """

```

```

    LEFT_SEQUENCE_START = 0
    LEFT_SEQUENCE_END = 6
    INDICATOR_INDEX = 6
    BODY_START = 7
    BODY_END = 72
    RIGHT_SEQUENCE_START = 72
    RIGHT_SEQUENCE_END = 80
    BODY_WIDTH = 65
    LEFT_SEQUENCE_WIDTH = 6
    RIGHT_SEQUENCE_WIDTH = 8

```

```

    # Shortest line on which columns 1-6 and column 7 can be read.
    MIN_ADDRESSABLE_LENGTH = 7

```

```

    COMMENT_INDICATORS = ("*", "/")
    DEBUG_INDICATORS = ("D", "d")
    CONTINUATION_INDICATOR = "-"
    BLANK_INDICATOR = " "

```

```

    # Every indicator the COBOL standard permits in column 7.
    VALID_INDICATORS = (" ", "*", "/", "D", "d", "-")

```

```

    # Extra indent applied to a continuation physical line when a statement
    # has to be wrapped. Relative to the first line's own indent.
    WRAP_CONTINUATION_INDENT = " "

```

```

    # Upper bound on physical lines produced by one logical statement.
    # A statement needing more than this is almost certainly malformed.
    WRAP_MAX_LINES = 4

```

```

    # -----
    # Parsing
    # -----

```

```

    def is_fixed_line(self, line: str) -> bool:
        """True when columns 1-6 are numeric and column 7 is a valid indicator.

```

```

        The right sequence (73-80) is deliberately NOT required. Many
        hand-maintained members leave it blank or truncate the line, and
        demanding it caused the left sequence to be parsed as COBOL text.
        """

```

```

        text = str(line or "").rstrip("\n")

```

```

        if len(text) < self.MIN_ADDRESSABLE_LENGTH:

```

```

        return False

    if not text[
        self.LEFT_SEQUENCE_START : self.LEFT_SEQUENCE_END
    ].isdigit():
        return False

    return text[self.INDICATOR_INDEX] in self.VALID_INDICATORS

def is_strict_fixed_line(self, line: str) -> bool:
    """The original, stricter test: full 80 columns, both sequences numeric.

    Kept for callers that specifically need a fully sequenced line,
    for example the final resequencer.
    """
    text = str(line or "").rstrip("\n")
    return (
        len(text) >= self.RIGHT_SEQUENCE_END
        and text[
            self.LEFT_SEQUENCE_START : self.LEFT_SEQUENCE_END
        ].isdigit()
        and text[
            self.RIGHT_SEQUENCE_START : self.RIGHT_SEQUENCE_END
        ].isdigit()
    )

def split(self, line: str) -> tuple[str, str, str, str]:
    """Return (left_sequence, indicator, body, right_sequence).

    Short lines are padded to 80 columns first, so every slice is safe
    and a missing right sequence comes back as spaces rather than
    corrupting the body.
    """
    text = str(line or "").rstrip("\n")

    if self.is_fixed_line(text):
        padded = text.ljust(self.RIGHT_SEQUENCE_END)
        left = padded[self.LEFT_SEQUENCE_START : self.LEFT_SEQUENCE_END]
        indicator = padded[self.INDICATOR_INDEX]
        body = padded[self.BODY_START : self.BODY_END]
        right = padded[
            self.RIGHT_SEQUENCE_START : self.RIGHT_SEQUENCE_END
        ]
        return left, indicator, body, right

    return "", "", text, ""

def body(self, line: str) -> str:
    return self.split(line)[2]

def logical(self, line: str) -> str:
    return self.body(line).strip()

def indicator(self, line: str) -> str:
    return self.split(line)[1]

def left_sequence(self, line: str) -> str:
    return self.split(line)[0]

def right_sequence(self, line: str) -> str:
    return self.split(line)[3]

def leading_spaces(self, text: str, default: str = "") -> str:
    value = str(text or "")
    if not value:
        return default
    count = len(value) - len(value.lstrip(" "))
    return value[:count] if count > 0 else default

def body_indent(self, line: str) -> str:
    return self.leading_spaces(self.body(line))

# -----
# Classification
# -----
def is_comment_or_control_line(self, line: str) -> bool:

```

```

"""True for comment, page-eject and debug lines.

These must never be re-indented, re-wrapped or terminated.
"""
text = str(line or "").rstrip("\n")

if not text.strip():
    return False

if self.is_fixed_line(text):
    marker = text[self.INDICATOR_INDEX]
    return marker in self.COMMENT_INDICATORS + self.DEBUG_INDICATORS

stripped = text.lstrip()
return bool(stripped) and stripped[0] in self.COMMENT_INDICATORS

def is_comment_line(self, line: str) -> bool:
    return self.indicator(line) in self.COMMENT_INDICATORS

def is_debug_line(self, line: str) -> bool:
    return self.indicator(line) in self.DEBUG_INDICATORS

def is_continuation_line(self, line: str) -> bool:
    return self.indicator(line) == self.CONTINUATION_INDICATOR

def is_blank(self, line: str) -> bool:
    return not self.logical(line)

def is_sequence_artifact(self, line: str) -> bool:
    """True when the BODY is nothing but a leaked source sequence number.

    Produced by the old parser, for example:

        002520      001690/.

    The body here is '001690/.' - six digits, an optional indicator
    character and an optional period. Such a line is not COBOL and must
    be dropped, never terminated or re-indented.

    No regex: plain character inspection only.
    """
    candidate = self.logical(line)

    if not candidate:
        return False

    if candidate.endswith("."):
        candidate = candidate[:-1]

    if candidate and candidate[-1] in self.COMMENT_INDICATORS:
        candidate = candidate[:-1]

    candidate = candidate.strip()

    return (
        len(candidate) == self.LEFT_SEQUENCE_WIDTH
        and candidate.isdigit()
    )

# -----
# Width
# -----
def body_fits(self, body: str) -> bool:
    return len(str(body or "").rstrip()) <= self.BODY_WIDTH

# -----
# Building
# -----
def build(
    self,
    left_sequence: str,
    indicator: str,
    body: str,
    right_sequence: str,
) -> str:
    """Assemble one 80-column line.

```

Raises BodyOverflowError rather than truncating. Silent truncation destroys COBOL source; a caller must decide to skip or to wrap.

The line is assembled whenever EITHER sequence is present. A missing side is padded, so a short source line keeps its left sequence instead of losing it. The final resequencer rewrites columns 1-6 and 73-80 afterwards.

```
"""
clean_body = str(body or "").rstrip()

if not self.body_fits(clean_body):
    raise BodyOverflowError(
        f"COBOL body needs {len(clean_body)} columns, "
        f"only {self.BODY_WIDTH} are available: {clean_body!r}"
    )

left = str(left_sequence or "")
right = str(right_sequence or "")

if not left and not right:
    return clean_body

marker = str(indicator or "")[:1] or self.BLANK_INDICATOR
if marker not in self.VALID_INDICATORS:
    marker = self.BLANK_INDICATOR

padded_left = left.rjust(self.LEFT_SEQUENCE_WIDTH)[
    : self.LEFT_SEQUENCE_WIDTH
]
padded_right = right.ljust(self.RIGHT_SEQUENCE_WIDTH)[
    : self.RIGHT_SEQUENCE_WIDTH
]
padded_body = clean_body.ljust(self.BODY_WIDTH)

return f"{padded_left}{marker}{padded_body}{padded_right}"

def build_or_none(
    self,
    left_sequence: str,
    indicator: str,
    body: str,
    right_sequence: str,
) -> str | None:
    try:
        return self.build(left_sequence, indicator, body, right_sequence)
    except BodyOverflowError:
        return None

def emit(
    self,
    left_sequence: str,
    indicator: str,
    body: str,
    right_sequence: str,
) -> list[str]:
    """Render a logical body as one or more physical lines.

    Never truncates and never refuses: the body is wrapped as many times
    as needed, up to WRAP_MAX_LINES.
    """
    clean_body = str(body or "").rstrip()

    if self.body_fits(clean_body):
        return [
            self.build(left_sequence, indicator, clean_body, right_sequence)
        ]

    indent = self.leading_spaces(clean_body, default="")
    bodies = self.wrap_body_lines(
        clean_body,
        continuation_indent=indent + self.WRAP_CONTINUATION_INDENT,
    )

    if not bodies:
        raise BodyOverflowError(
```

```

        f"COBOL body cannot be wrapped within "
        f"{self.WRAP_MAX_LINES} lines: {clean_body!r}"
    )

    return [
        self.build(left_sequence, indicator, part, right_sequence)
        for part in bodies
    ]

# -----
# Body replacement
# -----
def replace_body(self, line: str, new_body: str) -> str:
    """Replace the body, or return the line unchanged when it will not fit.

    Total function: never raises, never truncates. A rewrite that cannot
    be represented is simply not applied.
    """
    left, indicator, _old_body, right = self.split(line)
    rebuilt = self.build_or_none(left, indicator, new_body, right)
    return rebuilt if rebuilt is not None else str(line or "").rstrip("\n")

def replace_body_strict(self, line: str, new_body: str) -> str:
    """Replace the body, raising BodyOverflowError when it will not fit."""
    left, indicator, _old_body, right = self.split(line)
    return self.build(left, indicator, new_body, right)

def replace_body_wrapped(self, line: str, new_body: str) -> list[str]:
    """Replace the body, wrapping onto further physical lines if needed.

    Returns one line when the body fits, several lines when it has to be
    split at word boundaries, and the original single line when even a
    wrapped form cannot be represented.

    Callers must EXTEND their output list, not append.
    """
    original = str(line or "").rstrip("\n")
    left, indicator, old_body, right = self.split(original)
    clean_body = str(new_body or "").rstrip()

    if self.body_fits(clean_body):
        return [self.replace_body(original, clean_body)]

    if not self.is_fixed_line(original):
        return [clean_body]

    indent = self.leading_spaces(old_body, default=" ")
    bodies = self.wrap_body_lines(
        clean_body,
        continuation_indent=indent + self.WRAP_CONTINUATION_INDENT,
    )

    if not bodies:
        return [original]

    rendered: list[str] = []
    for part in bodies:
        built = self.build_or_none(left, indicator, part, right)
        if built is None:
            return [original]
        rendered.append(built)

    return rendered

# -----
# Wrapping
# -----
def wrap_body(
    self,
    body: str,
    continuation_indent: str,
) -> list[str]:
    """Split one logical body into at most TWO fixed-format bodies.

    Original contract preserved for existing callers. Returns an empty
    list when the statement cannot be represented in two lines. Use

```

```

wrap_body_lines() when more than two lines are acceptable.
"""
bodies = self.wrap_body_lines(
    body,
    continuation_indent=continuation_indent,
    max_lines=2,
)
return bodies if len(bodies) == 2 else []

def wrap_body_lines(
    self,
    body: str,
    continuation_indent: str,
    max_lines: int | None = None,
) -> list[str]:
    """Split one logical body into N fixed-format bodies.

    Splits on word boundaries only, so a data-name is never cut in half.
    Returns an empty list when the statement still does not fit within
    max_lines, so the caller can skip the rewrite rather than emit
    corrupt source.
    """
    limit = max_lines if max_lines is not None else self.WRAP_MAX_LINES
    clean_body = str(body or "").rstrip()
    indent = self.leading_spaces(clean_body, default="")
    words = clean_body.split()

    if not words or limit < 1:
        return []

    first_limit = self.BODY_WIDTH - len(indent)
    next_limit = self.BODY_WIDTH - len(continuation_indent)

    if first_limit <= 0 or next_limit <= 0:
        return []

    # A single word longer than the window can never be wrapped.
    if any(len(word) > max(first_limit, next_limit) for word in words):
        return []

    bodies: list[str] = []
    remaining = list(words)
    is_first = True

    while remaining:
        if len(bodies) >= limit:
            return []

        prefix = indent if is_first else continuation_indent
        window = first_limit if is_first else next_limit

        taken: list[str] = []
        for word in remaining:
            candidate = " ".join(taken + [word])
            if len(candidate) <= window:
                taken.append(word)
                continue
            break

        if not taken:
            return []

        bodies.append(prefix + " ".join(taken))
        remaining = remaining[len(taken) :]
        is_first = False

    #if len(bodies) < 2:
    #    return []

    return bodies

```



```
=====
FILE: src\idms_db2_phase2\services\fixed_format_statement_wrapper.py
ENCODING: utf-8
=====
```

```
# LOCATION: src/idms_db2_phase2/services/fixed_format_statement_wrapper.py
# ACTION: CREATE NEW FILE
```

```
"""Wrap-aware fixed-format body replacement.
```

```
Any transformer that lengthens a statement in place must route through
this service. Replacing the body directly truncates at column 72, which
produces an undefined data name such as WS-NR-ID-GO.
```

```
Mirrors the proven splitter in SqlcodeWrapperLineUtils, but is reusable
from business-paragraph transformers rather than generated SQL blocks only.
"""
```

```
from __future__ import annotations
```

```
from patterns.fixed_format_wrap_patterns import MOVE_TO_PATTERN
from rules.fixed_format_wrap_rules import (
    BODY_WIDTH,
    CONTINUATION_INDENT,
    DIAG_WRAPPED_TEMPLATE,
    BODY_END_COLUMN,
    TOKEN_MOVE,
    TOKEN_OF_DCL,
    TOKEN_TO,
)
```

```
class FixedFormatStatementWrapper:
```

```
    def __init__(self, fixed_format) -> None:
        self.fixed_format = fixed_format
        self.messages: list[str] = []
```

```
    def replace_body(
        self,
        line: str,
        new_body: str,
    ) -> list[str]:
        """Replace a fixed-format body, wrapping when it no longer fits.
```

```
        Returns one or more physical lines. Callers must extend, not append.
        """
```

```
        body = str(new_body or "").rstrip()
```

```
        if not self.fixed_format.is_fixed_line(line):
            return [body]
```

```
        indent = self._leading_spaces(body)
        fragments = self._wrap(body=body, indent=indent)
```

```
        if len(fragments) > 1:
            self.messages.append(
                DIAG_WRAPPED_TEMPLATE.format(column=BODY_END_COLUMN)
            )
```

```
        return [
            self.fixed_format.replace_body(line, fragment)
            for fragment in fragments
        ]
```

```
    def _wrap(self, body: str, indent: str) -> list[str]:
        if len(body) <= BODY_WIDTH:
            return [body]
```

```
        move_lines = self._wrap_move(body=body, indent=indent)
        if move_lines:
            return move_lines
```

```
        return self._wrap_by_words(
            body=body,
            continuation_indent=indent + CONTINUATION_INDENT,
```

```

)

def _wrap_move(self, body: str, indent: str) -> list[str]:
    """Split MOVE <source> TO <target> on the TO boundary.

    A source qualified with OF DCL is always split, matching the
    generated-block convention, so the target can never be clipped.
    """
    match = MOVE_TO_PATTERN.match(body.strip())
    if not match:
        return []

    source = str(match.group("source") or "").strip()
    target = str(match.group("target") or "").strip()
    dot = str(match.group("dot") or "").strip()

    if not source or not target:
        return []

    one_line = f"{indent}{TOKEN_MOVE} {source} {TOKEN_TO} {target}{dot}"
    force_split = TOKEN_OF_DCL in source.upper()

    if len(one_line) <= BODY_WIDTH and not force_split:
        return [one_line]

    return [
        f"{indent}{TOKEN_MOVE} {source}",
        f"{indent}{TOKEN_TO} {target}{dot}",
    ]

def _wrap_by_words(
    self,
    body: str,
    continuation_indent: str,
) -> list[str]:
    words = body.split()
    if not words:
        return [body]

    indent = self._leading_spaces(body)
    output: list[str] = []
    current = indent

    for word in words:
        candidate = f"{current}{word}" if not current.strip() else f"{current} {word}"
        if len(candidate) <= BODY_WIDTH:
            current = candidate
            continue

        output.append(current.rstrip())
        current = f"{continuation_indent}{word}"

    if current.strip():
        output.append(current.rstrip())

    return output or [body]

@staticmethod
def _leading_spaces(text: str) -> str:
    raw = str(text or "")
    return raw[: len(raw) - len(raw.lstrip())]

```

```

=====
FILE: src\idms_db2_phase2\services\metadata_service.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.analyzers.metadata_service import MetadataService

```

```

__all__ = [
    "MetadataService",
]

```

```
=====
FILE: src\idms_db2_phase2\services\name_derivation_resolver.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from rules.name_derivation_rules import (
    NAME_DERIVATION_TARGET_MARKER,
    VM_BD_NAME_PATTERN,
)

class NameDerivationResolver:
    """Derives DB2-era COBOL names from the VM...BD... source shape.

    Generic and deterministic:
    VM <site>? BD <tail> -> VMDZ <site>? <tail>

    Examples:
    VM1BD567 -> VMDZ1567
    VMBD205I -> VMDZ205I

    Does not hardcode any program, record, or copybook name. If a name does
    not match the VM...BD... shape, it is returned unchanged.
    """

    def derive(self, name: str) -> str:
        source = str(name or "").strip()
        if not source:
            return source

        match = VM_BD_NAME_PATTERN.match(source)
        if not match:
            return source

        prefix = match.group("prefix")
        site = match.group("site") or ""
        tail = match.group("tail")

        # Preserve the original case of prefix; DZ marker uppercase.
        return f"{prefix}{NAME_DERIVATION_TARGET_MARKER}{site}{tail}".upper()

    def is_derivable(self, name: str) -> bool:
        return bool(VM_BD_NAME_PATTERN.match(str(name or "").strip()))
```

```
=====
FILE: src\idms_db2_phase2\services\name_normalizer.py
ENCODING: utf-8
=====
```

```
# LOCATION: src/idms_db2_phase2/services/name_normalizer.py
# ACTION: REPLACE ENTIRE FILE

"""Generic name normalization helper.

This service contains name transformation logic only.
Regex patterns are stored in patterns/naming_patterns.py.

DEFECT FIX
-----
`normalize()` produces a DB2 identifier (hyphen -> underscore). It was
being used to render IDMS SET and RECORD names into generated COBOL
comments, which emitted

    *DB2: Converted OBTAIN FIRST VMBFAS WITHIN AR_VMBFRM1.

instead of the IDMS spelling `AR-VMBFRM1`. An IDMS set name is never a
DB2 identifier, so it must never pass through `normalize()` on its way
to output.

`to_idms_name()` is the explicit, self-documenting entry point for that
case. It is a thin alias of `to_cobol()` so the two can never drift.
"""
```

```

from patterns.naming_patterns import (
    FOUR_DIGIT_RECORD_SUFFIX_PATTERN,
    MULTIPLE_UNDERSCORE_PATTERN,
    NON_COMPACT_NAME_CHARACTER_PATTERN,
    NON_DB2_NAME_CHARACTER_PATTERN,
)

class NameNormalizer:

    @staticmethod
    def normalize(
        value: str | None,
    ) -> str:
        """Canonical DB2 identifier: upper case, underscore separated."""
        if value is None:
            return ""

        text = str(value).strip().upper()

        if not text:
            return ""

        text = text.replace("-", "_")
        text = text.replace(" ", "_")
        text = NON_DB2_NAME_CHARACTER_PATTERN.sub("_", text)
        text = MULTIPLE_UNDERSCORE_PATTERN.sub("_", text)

        return text.strip("_")

    @staticmethod
    def to_cobol(
        value: str | None,
    ) -> str:
        """Canonical COBOL identifier: upper case, hyphen separated."""
        return NameNormalizer.normalize(value).replace("_", "-")

    @staticmethod
    def to_idms_name(
        value: str | None,
    ) -> str:
        """IDMS SET / RECORD / AREA name as it must appear in output.

        Use this for anything that is written back into generated COBOL
        text as an IDMS name (comments, diagnostics, DB2-KEEP markers).
        Never use `normalize()` for that purpose: it emits a DB2
        identifier and corrupts the IDMS spelling.
        """
        return NameNormalizer.to_cobol(value)

    @staticmethod
    def compact(
        value: str | None,
    ) -> str:
        text = NameNormalizer.normalize(value)
        return NON_COMPACT_NAME_CHARACTER_PATTERN.sub("", text)

    @staticmethod
    def remove_record_suffix(
        value: str | None,
    ) -> str:
        text = NameNormalizer.normalize(value)

        if not text:
            return ""

        return FOUR_DIGIT_RECORD_SUFFIX_PATTERN.sub("", text)

__all__ = ["NameNormalizer"]

```

```
=====
FILE: src\idms_db2_phase2\services\procedure_block_indent_service.py
ENCODING: utf-8
=====
```

```
"""
```

```
Procedure block indent service.
```

```
Improves readability of Procedure Division blocks by indenting IF/ELSE/END-IF
and EVALUATE/WHEN/END-EVALUATE. Extracted from the former final fix composer.
"""
```

```
from __future__ import annotations
```

```
from patterns.final_feedback_fix_patterns import (
    DIVISION_PATTERN,
    ELSE_PATTERN,
    END_EVALUATE_PATTERN,
    END_IF_PATTERN,
    EVALUATE_START_PATTERN,
    EXEC_SQL_END_LINE_PATTERN,
    EXEC_SQL_START_PATTERN,
    IF_START_PATTERN,
    PARAGRAPH_PATTERN,
    PROCEDURE_DIVISION_PATTERN,
    SECTION_PATTERN,
    WHEN_PATTERN,
)
from rules.cobol_statement_rules import NON_PARAGRAPH_SINGLE_WORDS
from rules.final_feedback_fix_rules import (
    FINAL_FIX_AREA_B_INDENT,
    FINAL_FIX_NESTED_INDENT,
)
from idms_db2_phase2.services.fixed_format_line_service import (
    FixedFormatLineService,
)
```

```
class ProcedureBlockIndentService:
    def __init__(
        self,
        fixed_format: FixedFormatLineService | None = None,
    ) -> None:
        self.fixed_format = fixed_format or FixedFormatLineService()

    def apply(self, text: str) -> str:
        if not text:
            return ""

        lines = str(text or "").splitlines()
        output: list[str] = []

        in_procedure_division = False
        in_exec_sql = False
        block_depth = 0
        evaluate_stack: list[bool] = []

        for line in lines:
            logical = self.fixed_format.logical(line)
            stripped = str(logical or "").strip()
            upper = stripped.upper()

            if not stripped:
                output.append(line)
                continue

            if PROCEDURE_DIVISION_PATTERN.match(upper):
                in_procedure_division = True
                block_depth = 0
                evaluate_stack = []
                output.append(line)
                continue

            if not in_procedure_division:
                output.append(line)
                continue
```

```

if DIVISION_PATTERN.match(upper) and not (
    PROCEDURE_DIVISION_PATTERN.match(upper)
):
    in_procedure_division = False
    block_depth = 0
    evaluate_stack = []
    output.append(line)
    continue

if self.fixed_format.is_comment_or_control_line(line):
    output.append(line)
    continue

if self._is_area_a_header(upper):
    output.append(line)
    continue

if EXEC_SQL_START_PATTERN.match(upper):
    in_exec_sql = True
    output.append(
        self._replace_body_when_fits(
            line=line,
            body=self._readable_body(
                logical=stripped,
                block_depth=self._effective_depth(
                    block_depth, evaluate_stack
                ),
            ),
        ),
    )
    continue

if in_exec_sql:
    output.append(
        self._replace_body_when_fits(
            line=line,
            body=self._readable_body(
                logical=stripped,
                block_depth=self._effective_depth(
                    block_depth, evaluate_stack
                ),
            ),
        ),
    )
    if EXEC_SQL_END_LINE_PATTERN.match(upper):
        in_exec_sql = False
        continue

if END_IF_PATTERN.match(upper):
    block_depth = max(0, block_depth - 1)
    output.append(
        self._replace_body_when_fits(
            line=line,
            body=self._readable_body(
                logical=stripped,
                block_depth=self._effective_depth(
                    block_depth, evaluate_stack
                ),
            ),
        ),
    )
    continue

if END_EVALUATE_PATTERN.match(upper):
    if evaluate_stack:
        evaluate_stack.pop()
    block_depth = max(0, block_depth - 1)
    output.append(
        self._replace_body_when_fits(
            line=line,
            body=self._readable_body(
                logical=stripped,
                block_depth=block_depth,
            ),
        ),
    )

```

```

    )
    continue

    if ELSE_PATTERN.match(upper):
        output.append(
            self._replace_body_when_fits(
                line=line,
                body=self._readable_body(
                    logical=stripped,
                    block_depth=max(
                        0,
                        self._effective_depth(
                            block_depth, evaluate_stack
                        )
                        - 1,
                    ),
                ),
            )
        )
        continue

    if WHEN_PATTERN.match(upper):
        if evaluate_stack:
            evaluate_stack[-1] = True
        output.append(
            self._replace_body_when_fits(
                line=line,
                body=self._readable_body(
                    logical=stripped,
                    block_depth=block_depth,
                ),
            )
        )
        continue

    output.append(
        self._replace_body_when_fits(
            line=line,
            body=self._readable_body(
                logical=stripped,
                block_depth=self._effective_depth(
                    block_depth, evaluate_stack
                ),
            ),
        )
    )

    if IF_START_PATTERN.match(upper):
        block_depth += 1
        continue

    if EVALUATE_START_PATTERN.match(upper):
        block_depth += 1
        evaluate_stack.append(False)
        continue

    return "\n".join(output).rstrip() + "\n"

def _effective_depth(
    self,
    block_depth: int,
    evaluate_stack: list[bool],
) -> int:
    if evaluate_stack and evaluate_stack[-1]:
        return block_depth + 1
    return block_depth

def _readable_body(
    self,
    logical: str,
    block_depth: int,
) -> str:
    clean_logical = str(logical or "").strip()
    depth = max(0, block_depth)
    return (
        FINAL_FIX_AREA_B_INDENT

```

```

        + (FINAL_FIX_NESTED_INDENT * depth)
        + clean_logical
    )

def _is_area_a_header(self, logical: str) -> bool:
    upper = str(logical or "").strip().upper()

    if not upper:
        return False

    if SECTION_PATTERN.match(upper):
        return True

    if PARAGRAPH_PATTERN.match(upper):
        word = upper.rstrip(".")
        return word not in NON_PARAGRAPH_SINGLE_WORDS

    return False

def _replace_body_when_fits(self, line: str, body: str) -> str:
    clean_body = str(body or "").rstrip()

    if not self.fixed_format.is_fixed_line(line):
        return clean_body

    if len(clean_body) > self.fixed_format.BODY_WIDTH:
        return line

    return self.fixed_format.replace_body(line, clean_body)

```

```

=====
FILE: src\idms_db2_phase2\services\production_validator.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\services\program_cursor_loop_analyzer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.domain.models import IdmsOperation
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from idms_db2_phase2.services.program_flow_models import CursorLoop
from rules.program_flow_rules import CURSOR_OPERATIONS

class ProgramCursorLoopAnalyzer:
    """
    Builds cursor-loop diagnostics from parsed IDMS operations.

    This class does not inspect or rewrite COBOL text.
    """

    def __init__(
        self,
        cursor_name_resolver: CursorNameResolver | None = None,
    ) -> None:
        self.cursor_name_resolver = cursor_name_resolver or CursorNameResolver()

    def cursor_loops(
        self,
        operations: list[IdmsOperation],
    ) -> list[CursorLoop]:
        output: list[CursorLoop] = []
        cursor_order_by_name: dict[str, int] = {}

        for operation in operations:
            operation_name = str(operation.operation or "").upper()

```



```

        if operation_name not in CURSOR_OPERATIONS:
            continue

        record_name = NameNormalizer.normalize(operation.record_name)
        set_name = NameNormalizer.normalize(operation.set_name)

        if not record_name:
            continue

        cursor_name = self.cursor_name_resolver.cursor_name_from_table(record_name)

        if cursor_name not in cursor_order_by_name:
            cursor_order_by_name[cursor_name] = len(cursor_order_by_name) + 1

        paragraph_spec = self.cursor_name_resolver.paragraph_names(
            cursor_order=cursor_order_by_name[cursor_name],
            cursor_name=cursor_name,
        )

        output.append(
            CursorLoop(
                record_name=record_name,
                set_name=set_name,
                operation=operation_name,
                operation_line=operation.line_number,
                loop_type="cursor",
                cursor_name=cursor_name,
                open_paragraph=paragraph_spec["open"],
                fetch_paragraph=paragraph_spec["fetch"],
                close_paragraph=paragraph_spec["close"],
            )
        )

    return output

```

```

=====
FILE: src\idms_db2_phase2\services\program_date_usage_analyzer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.services.program_flow_models import DateUsage
from rules.program_flow_rules import DATE_TOKENS

```

```

class ProgramDateUsageAnalyzer:
    """
    Detects basic date/time/timestamp usage hints for diagnostics.

    This class does not rewrite COBOL.
    """

    def date_usages(
        self,
        logical_lines: list[tuple[int, str, str]],
    ) -> list[DateUsage]:
        output: list[DateUsage] = []

        for line_number, logical, raw_line in logical_lines:
            upper = logical.upper()

            if not any(token in upper for token in DATE_TOKENS):
                continue

            output.append(
                DateUsage(
                    line_number=line_number,
                    line_text=raw_line,
                    usage_type="date-token",
                )
            )

        return output

```

```
=====
FILE: src\idms_db2_phase2\services\program_flow_analyzer.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.domain.models import DclgenColumn, IdmsOperation, SheetMappingRow
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.services.program_cursor_loop_analyzer import (
    ProgramCursorLoopAnalyzer,
)
from idms_db2_phase2.services.program_date_usage_analyzer import (
    ProgramDateUsageAnalyzer,
)
from idms_db2_phase2.services.program_flow_line_collector import (
    ProgramFlowLineCollector,
)
from idms_db2_phase2.services.program_flow_models import (
    CursorLoop,
    DateUsage,
    OutputWrite,
    ParagraphSpan,
    ProgramFlowAnalysis,
)
from idms_db2_phase2.services.program_output_write_analyzer import (
    ProgramOutputWriteAnalyzer,
)
from idms_db2_phase2.services.program_paragraph_analyzer import (
    ProgramParagraphAnalyzer,
)
```

```
class ProgramFlowAnalyzer:
    """
    Generic flow analyzer for diagnostics.

    This analyzer does not rewrite COBOL. It detects:
    - procedure paragraphs
    - cursor-like operations from parsed IDMS operations
    - output write statements
    - basic date usage hints
    """

    def __init__(
        self,
        mapping_rows: list[SheetMappingRow] | None = None,
        dclgen_columns: list[DclgenColumn] | None = None,
    ) -> None:
        self.mapping_rows = mapping_rows or []
        self.dclgen_columns = dclgen_columns or []

        self.cursor_name_resolver = CursorNameResolver()
        self.line_collector = ProgramFlowLineCollector()
        self.paragraph_analyzer = ProgramParagraphAnalyzer()
        self.cursor_loop_analyzer = ProgramCursorLoopAnalyzer(
            cursor_name_resolver=self.cursor_name_resolver,
        )
        self.output_write_analyzer = ProgramOutputWriteAnalyzer(
            paragraph_analyzer=self.paragraph_analyzer,
        )
        self.date_usage_analyzer = ProgramDateUsageAnalyzer()

    def analyze(
        self,
        cobol_text: str,
        operations: list[IdmsOperation] | None = None,
    ) -> ProgramFlowAnalysis:
        if not str(cobol_text or "").strip():
            return ProgramFlowAnalysis(
                diagnostics=["Program flow analyzer: COBOL text is empty."]
            )

        logical_lines = self.line_collector.logical_lines_with_numbers(cobol_text)
        paragraphs = self.paragraph_analyzer.paragraph_spans(logical_lines)
```

```

        cursor_loops = self.cursor_loop_analyzer.cursor_loops(operations or [])
        output_writes = self.output_write_analyzer.output_writes(
            logical_lines=logical_lines,
            paragraphs=paragraphs,
        )
        date_usages = self.date_usage_analyzer.date_usages(logical_lines)

        diagnostics = self._diagnostics(
            paragraphs=paragraphs,
            cursor_loops=cursor_loops,
            output_writes=output_writes,
            date_usages=date_usages,
        )

        return ProgramFlowAnalysis(
            paragraphs=paragraphs,
            cursor_loops=cursor_loops,
            output_writes=output_writes,
            date_usages=date_usages,
            diagnostics=diagnostics,
        )

def _diagnostics(
    self,
    *,
    paragraphs: list[ParagraphSpan],
    cursor_loops: list[CursorLoop],
    output_writes: list[OutputWrite],
    date_usages: list[DateUsage],
) -> list[str]:
    diagnostics: list[str] = []

    diagnostics.append(
        f"Program flow analyzer: Sheet Mapping rows received: {len(self.mapping_rows)}"
    )
    diagnostics.append(
        f"Program flow analyzer: DCLGEN columns received: {len(self.dclgen_columns)}"
    )
    diagnostics.append(
        f"Program flow analyzer: paragraphs detected: {len(paragraphs)}"
    )
    diagnostics.append(
        f"Program flow analyzer: cursor loops detected: {len(cursor_loops)}"
    )
    diagnostics.append(
        f"Program flow analyzer: output writes detected: {len(output_writes)}"
    )
    diagnostics.append(
        f"Program flow analyzer: DB2 date usages detected: {len(date_usages)}"
    )

    for loop in cursor_loops[:5]:
        diagnostics.append(
            "Program flow analyzer: loop sample: "
            f"record={loop.record_name}, set={loop.set_name}, "
            f"type={loop.loop_type}, cursor={loop.cursor_name}"
        )

    return diagnostics

__all__ = [
    "DateUsage",
    "CursorLoop",
    "OutputWrite",
    "ParagraphSpan",
    "ProgramFlowAnalysis",
    "ProgramFlowAnalyzer",
]

```

```
=====
FILE: src\idms_db2_phase2\services\program_flow_line_collector.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from patterns.sequence_patterns import strip_sequence_numbers

class ProgramFlowLineCollector:
    """
    Converts COBOL source text into line-numbered logical lines.

    Output tuple format:
    - line number
    - logical line without sequence area
    - original raw line with trailing newline removed
    """

    def logical_lines_with_numbers(
        self,
        cobol_text: str,
    ) -> list[tuple[int, str, str]]:
        output: list[tuple[int, str, str]] = []

        for line_number, raw_line in enumerate(
            str(cobol_text or "").splitlines(),
            start=1,
        ):
            logical = strip_sequence_numbers(raw_line)
            output.append((line_number, logical, raw_line.rstrip()))

        return output
```

```
=====
FILE: src\idms_db2_phase2\services\program_flow_models.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from dataclasses import dataclass, field

from idms_db2_phase2.services.name_normalizer import NameNormalizer
```

```
@dataclass
class ParagraphSpan:
    name: str
    start_line: int
    end_line: int
    lines: list[str] = field(default_factory=list)
```

```
@dataclass
class CursorLoop:
    record_name: str = ""
    set_name: str = ""
    operation: str = ""
    operation_line: int = 0
    process_paragraph: str = ""
    perform_line: int = 0
    until_line: int = 0
    loop_type: str = "unknown"
    cursor_name: str = ""
    open_paragraph: str = ""
    fetch_paragraph: str = ""
    close_paragraph: str = ""
```

```
@dataclass
class OutputWrite:
    output_record: str = ""
    paragraph_name: str = ""
```

```

write_line: int = 0
move_lines: list[str] = field(default_factory=list)

```

```
@dataclass
```

```

class DateUsage:
    host_field: str = ""
    dclgen_group: str = ""
    idms_record: str = ""
    line_number: int = 0
    line_text: str = ""
    usage_type: str = ""

```

```
@dataclass
```

```

class ProgramFlowAnalysis:
    paragraphs: list[ParagraphSpan] = field(default_factory=list)
    cursor_loops: list[CursorLoop] = field(default_factory=list)
    output_writes: list[OutputWrite] = field(default_factory=list)
    date_usages: list[DateUsage] = field(default_factory=list)
    diagnostics: list[str] = field(default_factory=list)

    def process_paragraph_for_cursor(
        self,
        cursor_name: str,
    ) -> str:
        cursor = NameNormalizer.to_cobol(NameNormalizer.normalize(cursor_name))

        for loop in self.cursor_loops:
            loop_cursor = NameNormalizer.to_cobol(
                NameNormalizer.normalize(loop.cursor_name)
            )

            if loop_cursor == cursor:
                return loop.process_paragraph

        return ""

    def loop_for_set(
        self,
        set_name: str,
    ) -> CursorLoop | None:
        target = NameNormalizer.normalize(set_name)

        for loop in self.cursor_loops:
            if NameNormalizer.normalize(loop.set_name) == target:
                return loop

        return None

    def loop_for_record(
        self,
        record_name: str,
    ) -> CursorLoop | None:
        target = NameNormalizer.normalize(record_name)

        for loop in self.cursor_loops:
            if NameNormalizer.normalize(loop.record_name) == target:
                return loop

        return None

```

```

=====
FILE: src\idms_db2_phase2\services\program_name_sync_service.py
ENCODING: utf-8
=====

```

```

"""
Program name synchronization service.

```

```

Generic behavior:
- Read final PROGRAM-ID.
- Rewrite only safe MOVE literal TO PROGRAM-NAME statements.
- Do not globally replace program names.
"""

```

```

from __future__ import annotations

from patterns.final_feedback_fix_patterns import (
    MOVE_TO_PROGRAM_NAME_PATTERN,
    PROGRAM_ID_PATTERN,
)

class ProgramNameSyncService:
    """
    Synchronizes PROGRAM-NAME from final PROGRAM-ID.

    No program name is hardcoded.
    """

    def sync(self, text: str) -> str:
        source = str(text or "")

        program_match = PROGRAM_ID_PATTERN.search(source)
        if not program_match:
            return source

        final_program_id = program_match.group("program").upper()

        def replace_program_name(match) -> str:
            return (
                match.group("prefix")
                + final_program_id
                + match.group("suffix")
            )

        return MOVE_TO_PROGRAM_NAME_PATTERN.sub(replace_program_name, source)

=====
FILE: src\idms_db2_phase2\services\program_output_write_analyzer.py
ENCODING: utf-8
=====

from __future__ import annotations

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from idms_db2_phase2.services.program_flow_models import OutputWrite, ParagraphSpan
from idms_db2_phase2.services.program_paragraph_analyzer import (
    ProgramParagraphAnalyzer,
)
from rules.program_flow_rules import WRITE_TOKENS

class ProgramOutputWriteAnalyzer:
    """
    Detects basic output write statements for diagnostics.

    This class does not rewrite COBOL.
    """

    def __init__(
        self,
        paragraph_analyzer: ProgramParagraphAnalyzer | None = None,
    ) -> None:
        self.paragraph_analyzer = paragraph_analyzer or ProgramParagraphAnalyzer()

    def output_writes(
        self,
        logical_lines: list[tuple[int, str, str]],
        paragraphs: list[ParagraphSpan],
    ) -> list[OutputWrite]:
        output: list[OutputWrite] = []
        paragraph_by_line = self.paragraph_analyzer.paragraph_name_by_line(paragraphs)

        for line_number, logical, _raw_line in logical_lines:
            upper = logical.strip().upper()

            if not any(upper.startswith(token) for token in WRITE_TOKENS):
                continue

```

```

        output.append(
            OutputWrite(
                output_record=self.first_word_after_operation(upper),
                paragraph_name=paragraph_by_line.get(line_number, ""),
                write_line=line_number,
            )
        )

    return output

def first_word_after_operation(
    self,
    upper_line: str,
) -> str:
    parts = upper_line.split()

    if len(parts) < 2:
        return ""

    return NameNormalizer.normalize(parts[1].rstrip("."))

```

```

=====
FILE: src\idms_db2_phase2\services\program_paragraph_analyzer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.services.program_flow_models import ParagraphSpan
from patterns.cobol_patterns import PARAGRAPH_PATTERN, PROCEDURE_DIVISION_PATTERN

```

```

class ProgramParagraphAnalyzer:
    """
    Detects Procedure Division paragraph spans.

    This analyzer does not rewrite COBOL.
    """

    def paragraph_spans(
        self,
        logical_lines: list[tuple[int, str, str]],
    ) -> list[ParagraphSpan]:
        paragraphs: list[ParagraphSpan] = []
        current: ParagraphSpan | None = None
        inside_procedure = False

        for line_number, logical, raw_line in logical_lines:
            if PROCEDURE_DIVISION_PATTERN.match(logical):
                inside_procedure = True

            if not inside_procedure:
                continue

            match = PARAGRAPH_PATTERN.match(logical.strip())

            if match:
                if current is not None:
                    current.end_line = line_number - 1
                    paragraphs.append(current)

                current = ParagraphSpan(
                    name=match.group("name").upper(),
                    start_line=line_number,
                    end_line=line_number,
                    lines=[raw_line],
                )
                continue

            if current is not None:
                current.lines.append(raw_line)
                current.end_line = line_number

        if current is not None:

```

```

        paragraphs.append(current)

    return paragraphs

def paragraph_name_by_line(
    self,
    paragraphs: list[ParagraphSpan],
) -> dict[int, str]:
    output: dict[int, str] = {}

    for paragraph in paragraphs:
        for line_number in range(paragraph.start_line, paragraph.end_line + 1):
            output[line_number] = paragraph.name

    return output

```

```

=====
FILE: src\idms_db2_phase2\services\sql_generator.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\services\string_block_format_service.py
ENCODING: utf-8
=====

```

```

"""
STRING block format service.

Reformats COBOL STRING blocks into manual style after Area B alignment.
Extracted from the former final fix composer.
"""

```

```

from __future__ import annotations

from patterns.final_feedback_fix_patterns import (
    STRING_END_PATTERN,
    STRING_INTRO_PATTERN,
    STRING_START_PATTERN,
)
from rules.final_feedback_fix_rules import (
    FINAL_FIX_STRING_CONTINUATION_BODY_INDENT,
    FINAL_FIX_STRING_INTRO_BODY_INDENT,
    FINAL_FIX_STRING_START_BODY_INDENT,
)
from idms_db2_phase2.services.fixed_format_line_service import (
    FixedFormatLineService,
)

class StringBlockFormatService:
    def __init__(
        self,
        fixed_format: FixedFormatLineService | None = None,
    ) -> None:
        self.fixed_format = fixed_format or FixedFormatLineService()

    def apply(self, text: str) -> str:
        if not text:
            return ""

        lines = str(text or "").splitlines()
        output: list[str] = []
        inside_string = False

        for line in lines:
            logical = self.fixed_format.logical(line)
            stripped = str(logical or "").strip()

            if not stripped:
                output.append(line)
                continue

```



```

        if self.fixed_format.is_comment_or_control_line(line):
            output.append(line)
            continue

        if STRING_START_PATTERN.match(stripped):
            inside_string = True
            output.append(
                self._replace_body_when_fits(
                    line=line,
                    body=FINAL_FIX_STRING_START_BODY_INDENT + stripped,
                )
            )
            continue

        if inside_string and STRING_END_PATTERN.match(stripped):
            inside_string = False
            output.append(
                self._replace_body_when_fits(
                    line=line,
                    body=FINAL_FIX_STRING_START_BODY_INDENT + stripped,
                )
            )
            continue

        if inside_string and STRING_INTRO_PATTERN.match(stripped):
            output.append(
                self._replace_body_when_fits(
                    line=line,
                    body=FINAL_FIX_STRING_INTRO_BODY_INDENT + stripped,
                )
            )
            continue

        if inside_string:
            output.append(
                self._replace_body_when_fits(
                    line=line,
                    body=(
                        FINAL_FIX_STRING_CONTINUATION_BODY_INDENT
                        + stripped
                    ),
                )
            )
            continue

        output.append(line)

    return "\n".join(output).rstrip() + "\n"

def _replace_body_when_fits(self, line: str, body: str) -> str:
    clean_body = str(body or "").rstrip()

    if not self.fixed_format.is_fixed_line(line):
        return clean_body

    if len(clean_body) > self.fixed_format.BODY_WIDTH:
        return line

    return self.fixed_format.replace_body(line, clean_body)

```

```

=====
FILE: src\idms_db2_phase2\services\update_audit_move_inserter.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.services.fixed_format_line_service import (
    FixedFormatLineService,
)
from idms_db2_phase2.services.update_cobol_final_cleanup_utils import (
    UpdateCobolFinalCleanupUtils,
)
from patterns.update_final_feedback_patterns import (

```

```

EXEC_SQL_END_PATTERN,
EXEC_SQL_START_PATTERN,
MOVE_TO_DCL_OF_REFERENCE_PATTERN,
SET_KEYWORD_PATTERN,
SQL_SET_ASSIGNMENT_PATTERN,
UPDATE_STATEMENT_PATTERN,
WHERE_KEYWORD_PATTERN,
)
from rules.timestamp_audit_rules import UPDATE_AUDIT_COLUMN_PREFIXES
from rules.update_cobol_final_cleanup_rules import (
    AUDIT_SOURCE_BY_KIND,
    TIMESTAMP_AUDIT_PREFIXES,
    UPDATE_FINAL_LOOKBACK_DUPLICATE_LIMIT,
    USER_AUDIT_PREFIXES,
)

class UpdateAuditMoveInserter:
    """
    Inserts audit MOVE statements before UPDATE SQL blocks.

    Only inserts moves for audit host variables already present in generated
    UPDATE SET assignments.
    """

    def __init__(
        self,
        *,
        fixed_format: FixedFormatLineService,
        utils: UpdateCobolFinalCleanupUtils,
    ) -> None:
        self.fixed_format = fixed_format
        self.utils = utils

    def insert_update_audit_moves(
        self,
        text: str,
    ) -> str:
        lines = text.splitlines()
        update_blocks = self.find_update_exec_sql_blocks(lines)

        if not update_blocks:
            return text.rstrip() + "\n"

        output = list(lines)

        for start, end in sorted(update_blocks, reverse=True):
            block = output[start : end + 1]
            audit_targets = self.audit_targets_from_update_block(block)

            if not audit_targets:
                continue

            reference_line = output[start]
            insert_lines: list[str] = []

            for group, host, audit_kind in audit_targets:
                if self.has_recent_move_to_host(
                    lines=output,
                    insert_index=start,
                    group=group,
                    host=host,
                ):
                    continue

                source = self.source_for_audit_kind(audit_kind)

                if not source:
                    continue

                move_logical = f"MOVE {source} TO {host} OF {group}"
                insert_lines.append(
                    self.fixed_format.replace_body(
                        reference_line,
                        self.utils.body_with_existing_indent(
                            original_line=reference_line,

```

```

        new_logical=move_logical,
    ),
)

if not insert_lines:
    continue

output[start:start] = insert_lines

return "\n".join(output).rstrip() + "\n"

def find_update_exec_sql_blocks(
    self,
    lines: list[str],
) -> list[tuple[int, int]]:
    blocks: list[tuple[int, int]] = []
    in_exec_sql = False
    start = -1
    has_update = False

    for index, line in enumerate(lines):
        logical = self.fixed_format.logical(line)
        upper = logical.upper()

        if EXEC_SQL_START_PATTERN.match(logical):
            # One-line "EXEC SQL ... END-EXEC." (e.g. INCLUDE) must not
            # wedge the in_exec_sql flag open. If END-EXEC is on the same
            # line, this block opens and closes immediately and carries
            # no UPDATE, so skip it without entering multi-line mode.
            if "END-EXEC" in upper:
                in_exec_sql = False
                start = -1
                has_update = False
                continue
            in_exec_sql = True
            start = index
            has_update = False
            continue

        if in_exec_sql and UPDATE_STATEMENT_PATTERN.match(logical):
            has_update = True

        if in_exec_sql and EXEC_SQL_END_PATTERN.match(logical):
            if has_update:
                blocks.append((start, index))
            in_exec_sql = False
            start = -1
            has_update = False

    return blocks

def audit_targets_from_update_block(
    self,
    block: list[str],
) -> list[tuple[str, str, str]]:
    in_set = False
    targets: list[tuple[str, str, str]] = []

    for line in block:
        logical = self.fixed_format.logical(line)

        if SET_KEYWORD_PATTERN.match(logical):
            in_set = True
            continue

        if in_set and WHERE_KEYWORD_PATTERN.match(logical):
            break

        if not in_set:
            continue

        match = SQL_SET_ASSIGNMENT_PATTERN.match(logical.strip())

        if not match:
            continue

```

```

        column = str(match.group("column") or "").upper()
        group = str(match.group("group") or "").upper()
        host = str(match.group("host") or "").upper()

        audit_kind = self.audit_kind_for_column(column)

        if not audit_kind:
            continue

        targets.append((group, host, audit_kind))

    return self.unique_targets(targets)

def audit_kind_for_column(
    self,
    column: str,
) -> str:
    normalized = str(column or "").upper()

    if not self.has_update_audit_prefix(normalized):
        return ""

    if normalized.startswith(TIMESTAMP_AUDIT_PREFIXES):
        return "timestamp"

    if normalized.startswith(USER_AUDIT_PREFIXES):
        return "user"

    return ""

def has_update_audit_prefix(
    self,
    column: str,
) -> bool:
    normalized = str(column or "").upper()

    return any(
        normalized.startswith(str(prefix or "").upper())
        for prefix in UPDATE_AUDIT_COLUMN_PREFIXES
    )

def source_for_audit_kind(
    self,
    audit_kind: str,
) -> str:
    return AUDIT_SOURCE_BY_KIND.get(str(audit_kind or "").lower(), "")

def has_recent_move_to_host(
    self,
    *,
    lines: list[str],
    insert_index: int,
    group: str,
    host: str,
) -> bool:
    target_group = str(group or "").upper()
    target_host = str(host or "").upper()

    start = max(0, insert_index - UPDATE_FINAL_LOOKBACK_DUPLICATE_LIMIT)

    for index in range(start, insert_index):
        logical = self.fixed_format.logical(lines[index])
        match = MOVE_TO_DCL_OF_REFERENCE_PATTERN.match(logical)

        if not match:
            continue

        found_group = str(match.group("group") or "").upper()
        found_host = str(match.group("host") or "").upper()

        if found_group == target_group and found_host == target_host:
            return True

    return False

```

```

def unique_targets(
    self,
    values: list[tuple[str, str, str]],
) -> list[tuple[str, str, str]]:
    output: list[tuple[str, str, str]] = []
    seen: set[tuple[str, str, str]] = set()

    for group, host, audit_kind in values:
        key = (
            str(group or "").upper(),
            str(host or "").upper(),
            str(audit_kind or "").lower(),
        )

        if not key[0] or not key[1] or not key[2]:
            continue

        if key in seen:
            continue

        seen.add(key)
        output.append(key)

    return output

```

```

=====
FILE: src\idms_db2_phase2\services\update_cobol_final_cleanup_service.py
ENCODING: utf-8
=====

```

```

"""

```

Update COBOL final cleanup service.

Generic behavior:

- Converts non-SQL DCLGEN dot references to COBOL OF qualification.
- Replaces residual bare IDMS record initialization with resolved DCLGEN group initialization when there is strong local DCLGEN context.
- Populates update audit host variables before UPDATE SQL blocks.
- Replaces generated DB2 diagnostic labels that still contain IDMS record names with nearby resolved DB2 table names.
- Does not hardcode program names, records, DB2 tables, DB2 columns, DCLGEN groups, or host variables.

```

from __future__ import annotations

```

```

from idms_db2_phase2.services.fixed_format_line_service import (
    FixedFormatLineService,
)
from idms_db2_phase2.services.update_audit_move_inserter import (
    UpdateAuditMoveInserter,
)
from idms_db2_phase2.services.update_cobol_final_cleanup_utils import (
    UpdateCobolFinalCleanupUtils,
)
from idms_db2_phase2.services.update_cobol_reference_cleanup import (
    UpdateCobolReferenceCleanup,
)
from idms_db2_phase2.services.update_diagnostic_label_normalizer import (
    UpdateDiagnosticLabelNormalizer,
)

```

```

class UpdateCobolFinalCleanupService:

```

```

    """

```

Public service for generic update-program final cleanup.

```

    This class only orchestrates focused cleanup helpers.
    """

```

```

    def __init__(
        self,
        fixed_format: FixedFormatLineService | None = None,
    ) -> None:
        self.fixed_format = fixed_format or FixedFormatLineService()

```

```

self.utils = UpdateCobolFinalCleanupUtils(self.fixed_format)

self.reference_cleanup = UpdateCobolReferenceCleanup(
    fixed_format=self.fixed_format,
    utils=self.utils,
)
self.audit_move_inserter = UpdateAuditMoveInserter(
    fixed_format=self.fixed_format,
    utils=self.utils,
)
self.diagnostic_label_normalizer = UpdateDiagnosticLabelNormalizer(
    fixed_format=self.fixed_format,
    utils=self.utils,
)

def apply(
    self,
    text: str,
) -> str:
    output = str(text or "")

    if not output:
        return ""

    output = self.reference_cleanup.normalize_non_sql_dcl_dot_references(output)
    output = self.reference_cleanup.replace_bare_record_initialization(output)
    output = self.audit_move_inserter.insert_update_audit_moves(output)
    output = self.diagnostic_label_normalizer.normalize_db2_diagnostic_labels(
        output
    )

    return output.rstrip() + "\n"

```

```

=====
FILE: src\idms_db2_phase2\services\update_cobol_final_cleanup_utils.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.services.fixed_format_line_service import (
    FixedFormatLineService,
)
from rules.update_cobol_final_cleanup_rules import (
    PROTECTED_BARE_TARGET_PREFIXES,
)

class UpdateCobolFinalCleanupUtils:
    """
    Shared fixed-format helper methods for update COBOL final cleanup.
    """

    def __init__(
        self,
        fixed_format: FixedFormatLineService,
    ) -> None:
        self.fixed_format = fixed_format

    def body_with_existing_indent(
        self,
        *,
        original_line: str,
        new_logical: str,
    ) -> str:
        body = self.fixed_format.body(original_line)
        leading = body[: len(body) - len(body.lstrip(" "))]

        if not leading:
            leading = "    "

        return leading + str(new_logical or "").strip()

    def comment_body_for_line(
        self,

```

```

*,
original_line: str,
comment_text: str,
) -> str:
    """
    Return the correct body for a comment line.

    In fixed-format COBOL, the comment indicator is already in column 7.
    Therefore, if the replacement text starts with '*', remove it from
    the body to avoid producing '**DB2...' physically.
    """

    text = str(comment_text or "").strip()

    if text.startswith("*"):
        text = text[1:].lstrip()

    body = self.fixed_format.body(original_line)
    leading = body[: len(body) - len(body.lstrip(" "))]

    return leading + text

def is_protected_bare_target(
    self,
    target: str,
) -> bool:
    normalized = str(target or "").upper()

    return any(
        normalized.startswith(prefix)
        for prefix in PROTECTED_BARE_TARGET_PREFIXES
    )

```

```

=====
FILE: src\idms_db2_phase2\services\update_cobol_reference_cleanup.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from collections import Counter

from idms_db2_phase2.services.fixed_format_line_service import (
    FixedFormatLineService,
)
from idms_db2_phase2.services.update_cobol_final_cleanup_utils import (
    UpdateCobolFinalCleanupUtils,
)
from patterns.update_final_feedback_patterns import (
    DCLGEN_OF_REFERENCE_PATTERN,
    EXEC_SQL_END_PATTERN,
    EXEC_SQL_START_PATTERN,
    INITIALIZE_DCL_PATTERN,
    MOVE_TO_BARE_RECORD_PATTERN,
    NON_SQL_DCL_DOT_REFERENCE_PATTERN,
    SQL_LOCATION_DB_OPERATION_PATTERN,
)
from rules.update_cobol_final_cleanup_rules import (
    UPDATE_FINAL_LOOKAHEAD_LIMIT,
    UPDATE_FINAL_LOOKBACK_DUPLICATE_LIMIT,
)

class UpdateCobolReferenceCleanup:
    """
    Handles non-SQL DCLGEN reference cleanup and safe bare-record initialization.

    Responsibilities:
    - Convert non-SQL DCLGROUP.HOST references to HOST OF DCLGROUP.
    - Replace MOVE SPACES TO bare-record with INITIALIZE DCLGROUP only when
      nearby DCLGEN context is clear.
    """

    def __init__(
        self,

```

```

*,
fixed_format: FixedFormatLineService,
utils: UpdateCobolFinalCleanupUtils,
) -> None:
    self.fixed_format = fixed_format
    self.utils = utils

def normalize_non_sql_dcl_dot_references(
    self,
    text: str,
) -> str:
    lines = text.splitlines()
    output: list[str] = []
    in_exec_sql = False

    for line in lines:
        logical = self.fixed_format.logical(line)

        if EXEC_SQL_START_PATTERN.match(logical):
            in_exec_sql = True
            output.append(line)
            continue

        if in_exec_sql:
            output.append(line)

            if EXEC_SQL_END_PATTERN.match(logical):
                in_exec_sql = False

            continue

        updated_logical = NON_SQL_DCL_DOT_REFERENCE_PATTERN.sub(
            self._replace_dcl_dot_with_of,
            logical,
        )

        if updated_logical == logical:
            output.append(line)
            continue

        output.append(
            self.fixed_format.replace_body(
                line,
                self.utils.body_with_existing_indent(
                    original_line=line,
                    new_logical=updated_logical,
                ),
            )
        )

    return "\n".join(output).rstrip() + "\n"

def replace_bare_record_initialization(
    self,
    text: str,
) -> str:
    lines = text.splitlines()
    output: list[str] = []
    in_exec_sql = False

    for index, line in enumerate(lines):
        logical = self.fixed_format.logical(line)

        if EXEC_SQL_START_PATTERN.match(logical):
            in_exec_sql = True
            output.append(line)
            continue

        if in_exec_sql:
            output.append(line)

            if EXEC_SQL_END_PATTERN.match(logical):
                in_exec_sql = False

            continue

```



```

        if self.fixed_format.is_comment_or_control_line(line):
            output.append(line)
            continue

        match = MOVE_TO_BARE_RECORD_PATTERN.match(logical)

        if not match:
            output.append(line)
            continue

        target = str(match.group("target") or "").upper()

        if self.utils.is_protected_bare_target(target):
            output.append(line)
            continue

        group = self.nearest_dcl_group_after(
            lines=lines,
            start_index=index + 1,
        )

        if not group:
            output.append(line)
            continue

        if self.recent_output_has_initialize(output, group):
            output.append(line)
            continue

        output.append(
            self.fixed_format.replace_body(
                line,
                self.utils.body_with_existing_indent(
                    original_line=line,
                    new_logical=f"INITIALIZE {group}",
                ),
            )
        )

    return "\n".join(output).rstrip() + "\n"

def nearest_dcl_group_after(
    self,
    *,
    lines: list[str],
    start_index: int,
) -> str:
    groups: list[str] = []
    in_exec_sql = False

    for index in range(
        start_index,
        min(len(lines), start_index + UPDATE_FINAL_LOOKAHEAD_LIMIT),
    ):
        logical = self.fixed_format.logical(lines[index])

        if EXEC_SQL_START_PATTERN.match(logical):
            in_exec_sql = True
            continue

        if in_exec_sql:
            if EXEC_SQL_END_PATTERN.match(logical):
                in_exec_sql = False
            continue

        for match in DCLGEN_OF_REFERENCE_PATTERN.finditer(logical):
            group = str(match.group("group") or "").upper()

            if group:
                groups.append(group)

        for match in NON_SQL_DCL_DOT_REFERENCE_PATTERN.finditer(logical):
            group = str(match.group("group") or "").upper()

            if group:
                groups.append(group)

```

```

        if SQL_LOCATION_DB_OPERATION_PATTERN.match(logical):
            break

    if not groups:
        return ""

    return Counter(groups).most_common(1)[0][0]

def recent_output_has_initialize(
    self,
    output: list[str],
    group: str,
) -> bool:
    target = str(group or "").upper()
    start = max(0, len(output) - UPDATE_FINAL_LOOKBACK_DUPLICATE_LIMIT)

    for index in range(len(output) - 1, start - 1, -1):
        logical = self.fixed_format.logical(output[index])
        match = INITIALIZE_DCL_PATTERN.match(logical)

        if match and str(match.group("group") or "").upper() == target:
            return True

    return False

def _replace_dcl_dot_with_of(
    self,
    match,
) -> str:
    group = str(match.group("group") or "").upper()
    host = str(match.group("host") or "").upper()

    return f"{host} OF {group}"

```

```

=====
FILE: src\idms_db2_phase2\services\update_diagnostic_label_normalizer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.services.fixed_format_line_service import (
    FixedFormatLineService,
)
from idms_db2_phase2.services.update_cobol_final_cleanup_utils import (
    UpdateCobolFinalCleanupUtils,
)
from patterns.update_final_feedback_patterns import (
    CONVERTED_DB2_OPERATION_COMMENT_PATTERN,
    EXEC_SQL_END_PATTERN,
    EXEC_SQL_START_PATTERN,
    FROM_STATEMENT_PATTERN,
    SQL_LOCATION_DB_OPERATION_PATTERN,
    UPDATE_STATEMENT_PATTERN,
)
from rules.update_cobol_final_cleanup_rules import UPDATE_FINAL_LOOKAHEAD_LIMIT

class UpdateDiagnosticLabelNormalizer:
    """
    Rewrites generated DB2 diagnostic labels from IDMS record naming to nearby
    resolved DB2 table naming when SQL context is available.
    """

    def __init__(
        self,
        *,
        fixed_format: FixedFormatLineService,
        utils: UpdateCobolFinalCleanupUtils,
    ) -> None:
        self.fixed_format = fixed_format
        self.utils = utils

    def normalize_db2_diagnostic_labels(

```

```

        self,
        text: str,
    ) -> str:
        lines = text.splitlines()
        output: list[str] = []

        for index, line in enumerate(lines):
            logical = self.fixed_format.logical(line)

            sql_location_match = SQL_LOCATION_DB_OPERATION_PATTERN.match(logical)

            if sql_location_match:
                replacement = self._replacement_sql_location(
                    line=line,
                    lines=lines,
                    start_index=index + 1,
                    operation=str(sql_location_match.group("operation") or ""),
                )

                if replacement:
                    output.append(replacement)
                    continue

            comment_match = CONVERTED_DB2_OPERATION_COMMENT_PATTERN.match(logical)

            if comment_match:
                replacement = self._replacement_comment(
                    line=line,
                    lines=lines,
                    start_index=index + 1,
                    operation_text=str(comment_match.group("operation") or ""),
                    prefix=str(comment_match.group("prefix") or ""),
                    suffix=str(comment_match.group("suffix") or "."),
                )

                if replacement:
                    output.append(replacement)
                    continue

            output.append(line)

        return "\n".join(output).rstrip() + "\n"

def _replacement_sql_location(
    self,
    *,
    line: str,
    lines: list[str],
    start_index: int,
    operation: str,
) -> str:
    table = self.nearby_table_for_operation(
        lines=lines,
        start_index=start_index,
        operation=operation,
    )

    if not table:
        return ""

    operation_name = str(operation or "").upper()

    return self.fixed_format.replace_body(
        line,
        self.utils.body_with_existing_indent(
            original_line=line,
            new_logical=f"MOVE '{operation_name}-{table}' TO SQL-LOCATION.",
        ),
    )

def _replacement_comment(
    self,
    *,
    line: str,
    lines: list[str],
    start_index: int,

```

```

        operation_text: str,
        prefix: str,
        suffix: str,
    ) -> str:
        table = self.nearby_table_for_comment_operation(
            lines=lines,
            start_index=start_index,
            operation_text=operation_text,
        )

        if not table:
            return ""

        updated_comment = prefix + table + suffix

        return self.fixed_format.replace_body(
            line,
            self.utils.comment_body_for_line(
                original_line=line,
                comment_text=updated_comment,
            ),
        )

def nearby_table_for_comment_operation(
    self,
    *,
    lines: list[str],
    start_index: int,
    operation_text: str,
) -> str:
    normalized = str(operation_text or "").upper()

    if "MODIFY" in normalized:
        return self.nearby_table_for_operation(
            lines=lines,
            start_index=start_index,
            operation="UPDATE",
        )

    if "OBTAIN" in normalized or "FIND" in normalized:
        return self.nearby_table_for_operation(
            lines=lines,
            start_index=start_index,
            operation="SELECT",
        )

    if "STORE" in normalized:
        return self.nearby_table_for_operation(
            lines=lines,
            start_index=start_index,
            operation="INSERT",
        )

    if "ERASE" in normalized:
        return self.nearby_table_for_operation(
            lines=lines,
            start_index=start_index,
            operation="DELETE",
        )

    return ""

def nearby_table_for_operation(
    self,
    *,
    lines: list[str],
    start_index: int,
    operation: str,
) -> str:
    wanted_operation = str(operation or "").upper()
    in_exec_sql = False

    for index in range(
        start_index,
        min(len(lines), start_index + UPDATE_FINAL_LOOKAHEAD_LIMIT),
    ):

```

```

        logical = self.fixed_format.logical(lines[index])

        if EXEC_SQL_START_PATTERN.match(logical):
            in_exec_sql = True
            continue

        if in_exec_sql and EXEC_SQL_END_PATTERN.match(logical):
            in_exec_sql = False
            continue

        if wanted_operation == "UPDATE":
            update_match = UPDATE_STATEMENT_PATTERN.match(logical)

            if update_match:
                return str(update_match.group("table") or "").upper()

        if wanted_operation == "SELECT":
            from_match = FROM_STATEMENT_PATTERN.match(logical)

            if from_match:
                return str(from_match.group("table") or "").upper()

        if wanted_operation in {"INSERT", "DELETE"}:
            table = self.table_for_insert_or_delete(logical)

            if table:
                return table

    return ""

def table_for_insert_or_delete(
    self,
    logical: str,
) -> str:
    text = str(logical or "").strip().upper()
    parts = text.split()

    if len(parts) < 3:
        return ""

    if parts[0] == "INSERT" and parts[1] == "INTO":
        return parts[2]

    if parts[0] == "DELETE" and parts[1] == "FROM":
        return parts[2]

    return ""

```

```

=====
FILE: src\idms_db2_phase2\services\update_final_feedback_service.py
ENCODING: utf-8
=====

```

```

"""

```

Backward-compatible wrapper for update COBOL final cleanup.

The professional implementation lives in:

idms_db2_phase2.services.update_cobol_final_cleanup_service

Keep this wrapper because existing composers may still import
UpdateFinalFeedbackService from this module.

```

"""

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.services.update_cobol_final_cleanup_service import (
    UpdateCobolFinalCleanupService,
)

```

```

class UpdateFinalFeedbackService(UpdateCobolFinalCleanupService):

```

```

    """

```

Backward-compatible alias for UpdateCobolFinalCleanupService.

```

    """

```

```
pass
```

```
__all__ = [  
    "UpdateFinalFeedbackService",  
]
```

```
=====
```

```
FILE: src\idms_db2_phase2\services\validation_service.py
```

```
ENCODING: utf-8
```

```
=====
```

```
from idms_db2_phase2.domain.models import ConversionInput
```

```
class ValidationService:
```

```
    """
```

```
    Validates the minimum required conversion inputs.
```

```
    This service performs input-level validation only. Production validation  
    of converted COBOL output should remain in a separate production validator.  
    """
```

```
    def validate(  
        self,  
        conversion_input: ConversionInput,  
    ) -> list[str]:  
        messages: list[str] = []
```

```
        if not conversion_input.sheet_mapping_rows:  
            messages.append(  
                "Sheet Mapping is required and must contain rows."  
            )
```

```
        if not conversion_input.dclgen_columns:  
            messages.append(  
                "At least one DCLGEN file is required."  
            )
```

```
        if not conversion_input.idms_cobol_text.strip():  
            messages.append(  
                "IDMS COBOL source text file is required and must contain readable COBOL code."  
            )
```

```
        return messages
```

```
=====
```

```
FILE: src\idms_db2_phase2\services\working_storage_group_selector.py
```

```
ENCODING: utf-8
```

```
=====
```

```
# LOCATION: src/idms_db2_phase2/services/working_storage_group_selector.py
```

```
# ACTION: CREATE NEW FILE
```

```
"""Chooses a SAFE 01 group for generated working-storage fields.
```

```
A generated counter must never be added to a record layout. Adding a  
field to a fixed-length record shifts every byte after it, silently  
corrupting the output file.
```

```
An 01 group is a record layout when any of these hold:
```

- * it is declared under FD / FILE SECTION,
- * it is the operand of WRITE / READ / REWRITE / RELEASE / RETURN,
- * it appears as WRITE ... FROM <name> or READ ... INTO <name>,
- * it is the target of MOVE SPACES / ZEROES TO <name>.

```
When no safe group exists the selector reports that a dedicated group  
must be created. It never guesses.
```

```
"""
```

```
from __future__ import annotations
```

```
from dataclasses import dataclass, field
```

```

from idms_db2_phase2.services.fixed_format_line_service import (
    FixedFormatLineService,
)
from patterns.structural_safety_patterns import (
    FD_PATTERN,
    FILE_SECTION_PATTERN,
    LEVEL_01_PATTERN,
    LINKAGE_SECTION_PATTERN,
    MOVE_WHOLE_GROUP_PATTERN,
    PROCEDURE_DIVISION_PATTERN,
    READ_INTO_PATTERN,
    RECORD_VERB_PATTERN,
    SUBORDINATE_LEVEL_PATTERN,
    WORKING_STORAGE_PATTERN,
    WRITE_FROM_PATTERN,
)

COMMENT_PREFIXES = ("*", "/")

@dataclass
class WorkingStorageGroup:
    name: str = ""
    header_index: int = -1
    last_child_index: int = -1
    child_level: str = ""
    is_record_layout: bool = False

    @property
    def is_safe(self) -> bool:
        return bool(self.name) and not self.is_record_layout

    @property
    def insert_index(self) -> int:
        if self.last_child_index >= 0:
            return self.last_child_index + 1
        return self.header_index + 1

@dataclass
class WorkingStorageLayout:
    groups: list[WorkingStorageGroup] = field(default_factory=list)
    working_storage_index: int = -1
    boundary_index: int = -1

    def safe_groups(self) -> list[WorkingStorageGroup]:
        return [group for group in self.groups if group.is_safe]

class WorkingStorageGroupSelector:
    """Maps WORKING-STORAGE and classifies every 01 group."""

    def __init__(
        self,
        fixed_format: FixedFormatLineService | None = None,
    ) -> None:
        self.fixed_format = fixed_format or FixedFormatLineService()

    # ----- public
    def analyze(self, lines: list[str]) -> WorkingStorageLayout:
        layout = WorkingStorageLayout()
        record_names = self._record_layout_names(lines)

        in_working_storage = False
        in_file_section = False
        current: WorkingStorageGroup | None = None

        for index, line in enumerate(lines):
            body = self._logical(line)
            if not body or self._is_comment(body):
                continue

            if FILE_SECTION_PATTERN.match(body):
                in_file_section = True
                in_working_storage = False
                continue

```

```

    if WORKING_STORAGE_PATTERN.match(body):
        in_file_section = False
        in_working_storage = True
        layout.working_storage_index = index
        continue

    if LINKAGE_SECTION_PATTERN.match(body) or (
        PROCEDURE_DIVISION_PATTERN.match(body)
    ):
        layout.boundary_index = index
        break

    if in_file_section or FD_PATTERN.match(body):
        continue

    if not in_working_storage:
        continue

    match = LEVEL_01_PATTERN.match(body)
    if match:
        name = match.group("name").upper()
        current = WorkingStorageGroup(
            name=name,
            header_index=index,
            is_record_layout=name in record_names,
        )
        layout.groups.append(current)
        continue

    child = SUBORDINATE_LEVEL_PATTERN.match(body)
    if child and current is not None:
        current.last_child_index = index
        if not current.child_level:
            current.child_level = child.group("level")

    if layout.boundary_index < 0:
        layout.boundary_index = len(lines)

    return layout

def select(self, lines: list[str]) -> WorkingStorageGroup | None:
    """The LAST safe work group, or None when one must be created."""
    safe = self.analyze(lines).safe_groups()
    return safe[-1] if safe else None

# ----- helpers
def _record_layout_names(self, lines: list[str]) -> set[str]:
    names: set[str] = set()
    in_file_section = False

    for line in lines:
        body = self._logical(line)
        if not body or self._is_comment(body):
            continue

        if FILE_SECTION_PATTERN.match(body):
            in_file_section = True
        elif WORKING_STORAGE_PATTERN.match(body):
            in_file_section = False

        if in_file_section:
            match = LEVEL_01_PATTERN.match(body)
            if match:
                names.add(match.group("name").upper())

        match = RECORD_VERB_PATTERN.match(body)
        if match:
            names.add(match.group("name").upper())

    for pattern in (
        WRITE_FROM_PATTERN,
        READ_INTO_PATTERN,
        MOVE_WHOLE_GROUP_PATTERN,
    ):
        found = pattern.search(body)

```



```

        if found:
            names.add(found.group("name").upper())

    return names

def _logical(self, line: str) -> str:
    try:
        return str(self.fixed_format.logical(line) or "").strip()
    except Exception: # noqa: BLE001
        return str(line or "").strip()

    @staticmethod
    def _is_comment(body: str) -> bool:
        return body.startswith(COMMENT_PREFIXES)

__all__ = [
    "WorkingStorageGroup",
    "WorkingStorageLayout",
    "WorkingStorageGroupSelector",
]

```

```

=====
FILE: src\idms_db2_phase2\testing\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\testing\batch\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\testing\batch\batch_environment.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import os
import sys
from pathlib import Path

from rules.batch_runner_rules import TARGET_PROGRAM_ID_ENV_KEY

def bootstrap_sys_path(current_file: Path) -> tuple[Path, Path]:
    """Ensure PROJECT_ROOT and SRC_DIR are importable.

    Returns (src_dir, project_root).
    """
    resolved = current_file.resolve()
    src_dir = resolved.parents[2]
    project_root = resolved.parents[3]

    for path in (project_root, src_dir):
        if str(path) not in sys.path:
            sys.path.insert(0, str(path))

    return src_dir, project_root

def env_target_program_id() -> str:
    """Read the target PROGRAM-ID override from the environment."""
    return str(os.getenv(TARGET_PROGRAM_ID_ENV_KEY, "") or "").strip().upper()

```

```
=====
FILE: src\idms_db2_phase2\testing\batch\batch_models.py
ENCODING: utf-8
=====
```

```
# LOCATION: src/idms_db2_phase2/testing/batch/batch_models.py
# ACTION: REPLACE ENTIRE FILE
```

```
from __future__ import annotations

from dataclasses import dataclass, field
from pathlib import Path
```

```
@dataclass(frozen=True)
class BatchMode:
    """Describes one conversion mode (Retrieval or Update)."""

    name: str
    program_dir: Path
    output_dir: Path
    apply_update_postprocess: bool
    default_target_program_id: str = ""
```

```
@dataclass
class SharedInputs:
    """Inputs loaded once and shared across every program in a batch."""

    sheet_rows: list
    dclgen_columns: list
    copybook_fields: list
    diagnostics: list[str] = field(default_factory=list)
    logical_records: list = field(default_factory=list)
```

```
=====
FILE: src\idms_db2_phase2\testing\batch\batch_reporter.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from pathlib import Path

from rules.batch_runner_messages import (
    BULLET_TEMPLATE,
    HEADER_DIAGNOSTICS,
    HEADER_VALIDATION_MESSAGES,
    LOG_CONVERSION_COMPLETED_TEMPLATE,
    LOG_INPUT_PROGRAM_TEMPLATE,
    LOG_OUTPUT_FILE_TEMPLATE,
    NO_VALIDATION_MESSAGES,
    PRESERVE_SOURCE_LABEL,
    PRINT_GENERATION_COMPLETED_TEMPLATE,
    PRINT_INPUT_PROGRAM_TEMPLATE,
    PRINT_OUTPUT_CREATED_TEMPLATE,
    SUMMARY_MAPPING_FOLDER_TEMPLATE,
    SUMMARY_MODE_TEMPLATE,
    SUMMARY_OUTPUT_FOLDER_TEMPLATE,
    SUMMARY_PROGRAM_COUNT_TEMPLATE,
    SUMMARY_PROGRAM_FOLDER_TEMPLATE,
    SUMMARY_PROJECT_ROOT_TEMPLATE,
    SUMMARY_SRC_DIR_TEMPLATE,
    SUMMARY_TARGET_PROGRAM_ID_TEMPLATE,
    HEADER_BATCH_INPUT_SUMMARY,
)

from .batch_models import BatchMode
```

```
def log_program_result(
    *,
    mode: BatchMode,
    program_path: Path,
    output_path: Path,
```

```

    result,
    diagnostics: list[str],
    logger,
) -> None:
    logger.info(LOG_CONVERSION_COMPLETED_TEMPLATE.format(mode=mode.name))
    logger.info(LOG_INPUT_PROGRAM_TEMPLATE.format(path=program_path))
    logger.info(LOG_OUTPUT_FILE_TEMPLATE.format(path=output_path))

    print("")
    print(PRINT_GENERATION_COMPLETED_TEMPLATE.format(mode=mode.name))
    print(PRINT_INPUT_PROGRAM_TEMPLATE.format(path=program_path))
    print(PRINT_OUTPUT_CREATED_TEMPLATE.format(path=output_path))

    print("")
    print(HEADER_VALIDATION_MESSAGES)
    print("")

    if result.validation_messages:
        for message in result.validation_messages:
            print(BULLET_TEMPLATE.format(text=message))
            logger.warning(message)
    else:
        print(NO_VALIDATION_MESSAGES)

    print("")
    print(HEADER_DIAGNOSTICS)
    print("")

    for diagnostic in diagnostics:
        print(BULLET_TEMPLATE.format(text=diagnostic))
        logger.info(diagnostic)

```

```

def print_batch_summary(
    *,
    project_root,
    src_dir,
    mode: BatchMode,
    mapping_folder,
    program_count: int,
    target_program_id: str,
) -> None:
    print("")
    print(HEADER_BATCH_INPUT_SUMMARY)
    print("")
    print(SUMMARY_PROJECT_ROOT_TEMPLATE.format(value=project_root))
    print(SUMMARY_SRC_DIR_TEMPLATE.format(value=src_dir))
    print(SUMMARY_MODE_TEMPLATE.format(value=mode.name))
    print(SUMMARY_MAPPING_FOLDER_TEMPLATE.format(value=mapping_folder))
    print(SUMMARY_PROGRAM_FOLDER_TEMPLATE.format(value=mode.program_dir))
    print(SUMMARY_PROGRAM_COUNT_TEMPLATE.format(value=program_count))
    print(SUMMARY_OUTPUT_FOLDER_TEMPLATE.format(value=mode.output_dir))
    print(
        SUMMARY_TARGET_PROGRAM_ID_TEMPLATE.format(
            value=target_program_id or PRESERVE_SOURCE_LABEL
        )
    )

```

```

=====
FILE: src\idms_db2_phase2\testing\batch\batch_validator.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/testing/batch/batch_validator.py
# ACTION: REPLACE ENTIRE FILE
"""Batch input validation.

```

```

Mapping Sheet, DCLGen and Copybook folders are mandatory and raise.
The LRF folder is OPTIONAL metadata: a missing folder is created silently
and never blocks a batch, because programs that do not use COPY IDMS LR
convert exactly as before.
"""

```

```

from __future__ import annotations

```

```

from pathlib import Path

from config.path_settings import (
    DEFAULT_COPYBOOK_DIR,
    DEFAULT_DCLGEN_DIR,
    DEFAULT_LRF_DIR,
    DEFAULT_MAPPING_SHEET_DIR,
    ensure_lrf_dir,
    ensure_output_dirs,
    lrf_paths,
    mapping_sheet_paths,
    retrieval_program_paths,
    update_program_paths,
)
from rules.batch_runner_messages import (
    FOLDER_NOT_FOUND_TEMPLATE,
    LABEL_COPYBOOK,
    LABEL_DCLGEN,
    LABEL_MAPPING_SHEET,
    LABEL_PROGRAM_TEMPLATE,
    NO_MAPPING_FILES_TEMPLATE,
    NO_PROGRAM_FILES_TEMPLATE,
    PATH_NOT_FOLDER_TEMPLATE,
    UNSUPPORTED_MODE_TEMPLATE,
)
from rules.batch_runner_rules import BATCH_MODE_RETRIEVAL, BATCH_MODE_UPDATE
from rules.lrf_rules import LABEL_LRF

from .batch_models import BatchMode

# ----- folders
def validate_folder(folder: Path, label: str) -> None:
    """Mandatory folder. Missing or not-a-folder is a hard stop."""
    if not folder.exists():
        raise FileNotFoundError(
            FOLDER_NOT_FOUND_TEMPLATE.format(label=label, folder=folder)
        )
    if not folder.is_dir():
        raise FileNotFoundError(
            PATH_NOT_FOLDER_TEMPLATE.format(label=label, folder=folder)
        )

def optional_folder(folder: Path) -> bool:
    """Optional folder. Returns True only when it exists and is a folder."""
    return folder.exists() and folder.is_dir()

def lrf_folder_status() -> tuple[bool, int]:
    """(folder_present, file_count) for the optional LRF input.

    Never raises. Callers use this only to narrate the batch report.
    """
    if not optional_folder(DEFAULT_LRF_DIR):
        return False, 0
    return True, len(lrf_paths())

def validate_common_input_folders() -> None:
    validate_folder(DEFAULT_MAPPING_SHEET_DIR, LABEL_MAPPING_SHEET)
    validate_folder(DEFAULT_DCLGEN_DIR, LABEL_DCLGEN)
    validate_folder(DEFAULT_COPYBOOK_DIR, LABEL_COPYBOOK)

    # ---- LRF is optional metadata, not a blocker.
    # Create it once so the folder always exists and the operator can see
    # where the subschema source is expected to be dropped.
    if not optional_folder(DEFAULT_LRF_DIR):
        ensure_lrf_dir()

    ensure_output_dirs()

def validate_mapping_files() -> None:
    """At least one mapping sheet must exist in the mapping folder."""
    if not mapping_sheet_paths():

```

```

        raise FileNotFoundError(
            NO_MAPPING_FILES_TEMPLATE.format(folder=DEFAULT_MAPPING_SHEET_DIR)
        )

# ----- modes
def program_paths_for_mode(mode_name: str) -> list[Path]:
    normalized = str(mode_name or "").strip().upper()

    if normalized == BATCH_MODE_UPDATE:
        return update_program_paths()

    if normalized == BATCH_MODE_RETRIEVAL:
        return retrieval_program_paths()

    raise ValueError(UNSUPPORTED_MODE_TEMPLATE.format(mode=mode_name))

def validate_mode(mode: BatchMode) -> None:
    validate_folder(
        mode.program_dir,
        LABEL_PROGRAM_TEMPLATE.format(mode=mode.name),
    )
    mode.output_dir.mkdir(parents=True, exist_ok=True)

    programs = program_paths_for_mode(mode.name)
    if not programs:
        raise FileNotFoundError(
            NO_PROGRAM_FILES_TEMPLATE.format(
                mode=mode.name,
                folder=mode.program_dir,
            )
        )

__all__ = [
    "validate_folder",
    "optional_folder",
    "lrf_folder_status",
    "validate_common_input_folders",
    "validate_mapping_files",
    "program_paths_for_mode",
    "validate_mode",
    "LABEL_LRF",
]

=====
FILE: src\idms_db2_phase2\testing\batch\program_converter.py
ENCODING: utf-8
=====

# LOCATION: src/idms_db2_phase2/testing/batch/program_converter.py
# ACTION: REPLACE ENTIRE FILE

from __future__ import annotations

from datetime import datetime
from pathlib import Path

from idms_db2_phase2.domain.models import ConversionInput
from idms_db2_phase2.infrastructure.file_loader import FileLoader
from idms_db2_phase2.infrastructure.local_uploaded_file import LocalUploadedFile
from idms_db2_phase2.orchestration.conversion_service import ConversionService
from patterns.lrf_patterns import COPY_IDMS_LR_PATTERN
from rules.batch_runner_messages import (
    DIAG_SOURCE_FILE_TEMPLATE,
    DIAG_SOURCE_LEN_TEMPLATE,
)
from rules.batch_runner_rules import (
    AUTO_FIX_PIC_LENGTH_MISMATCHES_DEFAULT,
    COBOL_TEXT_ENCODING,
    OUTPUT_FILE_EXTENSION,
    OUTPUT_FILE_NAME_TEMPLATE,
    OUTPUT_TIMESTAMP_FORMAT,
)

```

```

from rules.lrf_rules import (
    DIAG_LRF_NO_RECORDS_IN_SCOPE,
    DIAG_LRF_RECORDS_IN_SCOPE_TEMPLATE,
    LOG_LRF_TOTAL_RECORDS,
    LRF_VALIDATION_MESSAGES,
)

from .batch_models import BatchMode, SharedInputs
from .batch_reporter import log_program_result
from .update_postprocess_pipeline import apply_update_postprocess

def build_output_path(*, mode: BatchMode, program_path: Path) -> Path:
    timestamp = datetime.now().strftime(OUTPUT_TIMESTAMP_FORMAT)
    file_name = OUTPUT_FILE_NAME_TEMPLATE.format(
        stem=program_path.stem,
        timestamp=timestamp,
        extension=OUTPUT_FILE_EXTENSION,
    )
    return mode.output_dir / file_name

# -----
# LRF (Logical Record Facility)
# -----
def _logical_records(shared_inputs: SharedInputs) -> list:
    """LRF logical records from the shared inputs.

    Read with getattr so a SharedInputs instance created before the LRF
    field existed still converts without raising AttributeError.
    """
    return list(getattr(shared_inputs, "logical_records", None) or [])

def _source_uses_logical_records(source_text: str) -> list[str]:
    """Logical record names the SOURCE program copies via COPY IDMS LR."""
    names: list[str] = []
    for raw_line in str(source_text or "").splitlines():
        match = COPY_IDMS_LR_PATTERN.search(raw_line)
        if not match:
            continue
        name = str(match.group("lr") or "").strip().upper()
        if name and name not in names:
            names.append(name)
    return names

def _narrate_lrf(
    *,
    source_text: str,
    logical_records: list,
    diagnostics: list[str],
    logger,
) -> None:
    """Record what LRF metadata this program run will see.

    Never raises and never blocks the batch. A program that copies a
    logical record without a matching LRF subschema is reported once, so
    the gap is visible in the run log instead of failing silently.
    """
    if logical_records:
        names = ", ".join(
            str(getattr(record, "logical_record_name", "") or "")
            for record in logical_records
        )
        diagnostics.append(
            DIAG_LRF_RECORDS_IN_SCOPE_TEMPLATE.format(
                count=len(logical_records),
                names=names,
            )
        )
    else:
        diagnostics.append(DIAG_LRF_NO_RECORDS_IN_SCOPE)

    logger.info(LOG_LRF_TOTAL_RECORDS, len(logical_records))

```

```

used = _source_uses_logical_records(source_text)
if used and not logical_records:
    message = LRF_VALIDATION_MESSAGES["missing_lrf"]
    diagnostics.append(message)
    logger.warning(message)

def convert_one_program(
    *,
    mode: BatchMode,
    program_path: Path,
    shared_inputs: SharedInputs,
    target_program_id: str,
    logger,
) -> Path:
    file_loader = FileLoader()
    source_text = file_loader.read_uploaded_text(LocalUploadedFile(program_path))

    diagnostics = list(shared_inputs.diagnostics)
    diagnostics.append(
        DIAG_SOURCE_FILE_TEMPLATE.format(mode=mode.name, path=program_path)
    )
    diagnostics.append(
        DIAG_SOURCE_LEN_TEMPLATE.format(mode=mode.name, length=len(source_text))
    )

    logical_records = _logical_records(shared_inputs)
    _narrate_lrf(
        source_text=source_text,
        logical_records=logical_records,
        diagnostics=diagnostics,
        logger=logger,
    )

    service = ConversionService()
    result = service.convert(
        ConversionInput(
            sheet_mapping_rows=shared_inputs.sheet_rows,
            dclgen_columns=shared_inputs.dclgen_columns,
            copybook_fields=shared_inputs.copybook_fields,
            logical_records=logical_records,
            idms_cobol_text=source_text,
            target_program_id=target_program_id,
            auto_fix_pic_length_mismatches=AUTO_FIX_PIC_LENGTH_MISMATCHES_DEFAULT,
        )
    )

    converted_cobol = result.converted_cobol or ""

    if mode.apply_update_postprocess:
        converted_cobol = apply_update_postprocess(
            source_text=source_text,
            converted_cobol=converted_cobol,
            copybook_fields=shared_inputs.copybook_fields,
            dclgen_columns=shared_inputs.dclgen_columns,
            target_program_id=target_program_id,
            diagnostics=diagnostics,
            logger=logger,
        )

    output_path = build_output_path(mode=mode, program_path=program_path)
    output_path.write_text(converted_cobol, encoding=COBOL_TEXT_ENCODING)

    log_program_result(
        mode=mode,
        program_path=program_path,
        output_path=output_path,
        result=result,
        diagnostics=diagnostics,
        logger=logger,
    )

    return output_path

```

```
=====
FILE: src\idms_db2_phase2\testing\batch\shared_input_loader.py
ENCODING: utf-8
=====
```

```
# LOCATION: src/idms_db2_phase2/testing/batch/shared_input_loader.py
# ACTION: REPLACE ENTIRE FILE
```

```
from __future__ import annotations

from config.path_settings import (
    DEFAULT_COPYBOOK_DIR,
    DEFAULT_DCLGEN_DIR,
    DEFAULT_LRF_DIR,
    DEFAULT_MAPPING_SHEET_DIR,
    copybook_paths,
    dclgen_paths,
    lrf_paths,
    mapping_sheet_paths,
)
from idms_db2_phase2.infrastructure.file_loader import FileLoader
from idms_db2_phase2.infrastructure.local_uploaded_file import LocalUploadedFile
from idms_db2_phase2.parsers.copybook_parser import CopybookParser
from idms_db2_phase2.parsers.dclgen_parser import DclgenParser
from idms_db2_phase2.parsers.lrf_parser import LrfParser
from idms_db2_phase2.parsers.sheet_mapping_parser import SheetMappingParser
from rules.batch_runner_messages import (
    COPYBOOK_SOURCE_LABEL,
    DCLGEN_SOURCE_LABEL_TEMPLATE,
    DIAG_COPYBOOK_FILE_COUNT_TEMPLATE,
    DIAG_COPYBOOK_FILE_TEMPLATE,
    DIAG_COPYBOOK_FOLDER_TEMPLATE,
    DIAG_COPYBOOK_TEXT_LEN_TEMPLATE,
    DIAG_COPYBOOK_TOTAL_FIELDS_TEMPLATE,
    DIAG_DCLGEN_FILE_COUNT_TEMPLATE,
    DIAG_DCLGEN_FILE_TEMPLATE,
    DIAG_DCLGEN_FOLDER_TEMPLATE,
    DIAG_DCLGEN_TEXT_LEN_TEMPLATE,
    DIAG_DCLGEN_TOTAL_COLUMNS_TEMPLATE,
    DIAG_MAPPING_FOLDER_TEMPLATE,
    DIAG_MAPPING_ROWS_TEMPLATE,
    DIAG_MAPPING_SELECTED_TEMPLATE,
    DIAG_MULTIPLE_MAPPING_TEMPLATE,
    DIAG_START_LOAD_SHARED_INPUTS,
    NO_DCLGEN_FILES_TEMPLATE,
    NO_MAPPING_IN_FOLDER_TEMPLATE,
)
from rules.lrf_rules import (
    DIAG_LRF_FILE_COUNT_TEMPLATE,
    DIAG_LRF_FILE_TEMPLATE,
    DIAG_LRF_FOLDER_TEMPLATE,
    DIAG_LRF_PARSED_ZERO,
    DIAG_LRF_PARSE_FAILED_TEMPLATE,
    DIAG_LRF_READ_FAILED_TEMPLATE,
    DIAG_LRF_TEXT_LEN_TEMPLATE,
    LOG_LRF_FILE_COUNT,
    LOG_LRF_TOTAL_RECORDS,
    LRF_SOURCE_LABEL,
)

from .batch_models import SharedInputs

# ----- mapping
def _load_mapping_rows(sheet_parser, diagnostics: list[str]):
    mapping_files = mapping_sheet_paths()

    if not mapping_files:
        raise FileNotFoundError(
            NO_MAPPING_IN_FOLDER_TEMPLATE.format(
                folder=DEFAULT_MAPPING_SHEET_DIR
            )
        )

    if len(mapping_files) > 1:
        names = ", ".join(path.name for path in mapping_files)
```



```

        diagnostics.append(
            DIAG_MULTIPLE_MAPPING_TEMPLATE.format(
                first=mapping_files[0].name,
                names=names,
            )
        )

    sheet_path = mapping_files[0]
    sheet_rows = sheet_parser.parse_uploaded_file(LocalUploadedFile(sheet_path))

    diagnostics.append(
        DIAG_MAPPING_FOLDER_TEMPLATE.format(folder=DEFAULT_MAPPING_SHEET_DIR)
    )
    diagnostics.append(DIAG_MAPPING_SELECTED_TEMPLATE.format(path=sheet_path))
    diagnostics.append(DIAG_MAPPING_ROWS_TEMPLATE.format(count=len(sheet_rows)))

    if hasattr(sheet_parser, "diagnostics"):
        diagnostics.extend(sheet_parser.diagnostics)

    return sheet_rows

# ----- dclgen
def _load_dclgen_columns(dclgen_parser, file_loader, diagnostics: list[str]):
    dclgen_files = dclgen_paths()

    diagnostics.append(
        DIAG_DCLGEN_FOLDER_TEMPLATE.format(folder=DEFAULT_DCLGEN_DIR)
    )
    diagnostics.append(
        DIAG_DCLGEN_FILE_COUNT_TEMPLATE.format(count=len(dclgen_files))
    )

    if not dclgen_files:
        raise FileNotFoundError(
            NO_DCLGEN_FILES_TEMPLATE.format(folder=DEFAULT_DCLGEN_DIR)
        )

    dclgen_columns: list = []

    for index, path in enumerate(dclgen_files, start=1):
        text = file_loader.read_uploaded_text(LocalUploadedFile(path))
        diagnostics.append(
            DIAG_DCLGEN_FILE_TEMPLATE.format(index=index, path=path)
        )
        diagnostics.append(
            DIAG_DCLGEN_TEXT_LEN_TEMPLATE.format(index=index, length=len(text))
        )

        try:
            columns = dclgen_parser.parse(
                text=text,
                source_label=DCLGEN_SOURCE_LABEL_TEMPLATE.format(index=index),
            )
        except TypeError:
            columns = dclgen_parser.parse(text)

        dclgen_columns.extend(columns)

    diagnostics.append(
        DIAG_DCLGEN_TOTAL_COLUMNS_TEMPLATE.format(count=len(dclgen_columns))
    )

    if hasattr(dclgen_parser, "diagnostics"):
        diagnostics.extend(dclgen_parser.diagnostics)

    return dclgen_columns

# ----- copybook
def _load_copybook_fields(copybook_parser, file_loader, diagnostics: list[str]):
    copybook_files = copybook_paths()
    text_parts: list[str] = []

    diagnostics.append(
        DIAG_COPYBOOK_FOLDER_TEMPLATE.format(folder=DEFAULT_COPYBOOK_DIR)
    )

```

```

)
diagnostics.append(
    DIAG_COPYBOOK_FILE_COUNT_TEMPLATE.format(count=len(copybook_files))
)

for index, path in enumerate(copybook_files, start=1):
    text = file_loader.read_uploaded_text(LocalUploadedFile(path))
    text_parts.append(text)
    diagnostics.append(
        DIAG_COPYBOOK_FILE_TEMPLATE.format(index=index, path=path)
    )
    diagnostics.append(
        DIAG_COPYBOOK_TEXT_LEN_TEMPLATE.format(
            index=index,
            length=len(text),
        )
    )
)

copybook_text = "\n".join(text_parts)

if copybook_text.strip():
    try:
        copybook_fields = copybook_parser.parse(
            text=copybook_text,
            source_label=COPYBOOK_SOURCE_LABEL,
        )
    except TypeError:
        copybook_fields = copybook_parser.parse(copybook_text)
else:
    copybook_fields = []

diagnostics.append(
    DIAG_COPYBOOK_TOTAL_FIELDS_TEMPLATE.format(count=len(copybook_fields))
)

if hasattr(copybook_parser, "diagnostics"):
    diagnostics.extend(copybook_parser.diagnostics)

return copybook_fields

# ----- LRF
def _load_logical_records(lrf_parser, file_loader, diagnostics: list[str], logger):
    """Load the optional LRF subschema folder.

    LRF is metadata, not a program. A missing folder or an empty folder is
    a note, never a blocker: programs that do not use COPY IDMS LR convert
    exactly as before.
    """
    lrf_files = lrf_paths()

    diagnostics.append(DIAG_LRF_FOLDER_TEMPLATE.format(folder=DEFAULT_LRF_DIR))
    diagnostics.append(
        DIAG_LRF_FILE_COUNT_TEMPLATE.format(count=len(lrf_files))
    )
    logger.info(LOG_LRF_FILE_COUNT, len(lrf_files))

    if not lrf_files:
        diagnostics.append(DIAG_LRF_PARSED_ZERO)
        return []

    text_parts: list[str] = []

    for index, path in enumerate(lrf_files, start=1):
        try:
            text = file_loader.read_uploaded_text(LocalUploadedFile(path))
        except Exception as exc: # noqa: BLE001
            diagnostics.append(
                DIAG_LRF_READ_FAILED_TEMPLATE.format(name=path, reason=exc)
            )
            continue

        text_parts.append(text)
        diagnostics.append(
            DIAG_LRF_FILE_TEMPLATE.format(index=index, path=path)
        )
    )

```

```

        diagnostics.append(
            DIAG_LRF_TEXT_LEN_TEMPLATE.format(index=index, length=len(text))
        )

lrf_text = "\n".join(text_parts)

if not lrf_text.strip():
    diagnostics.append(DIAG_LRF_PARSED_ZERO)
    return []

try:
    try:
        logical_records = lrf_parser.parse(
            text=lrf_text,
            source_label=LRF_SOURCE_LABEL,
        )
    except TypeError:
        logical_records = lrf_parser.parse(lrf_text)
except Exception as exc: # noqa: BLE001
    diagnostics.append(
        DIAG_LRF_PARSE_FAILED_TEMPLATE.format(
            name=LRF_SOURCE_LABEL,
            reason=exc,
        )
    )
    return []

if hasattr(lrf_parser, "diagnostics"):
    diagnostics.extend(lrf_parser.diagnostics)

logger.info(LOG_LRF_TOTAL_RECORDS, len(logical_records))

return logical_records

# ----- entry point
def load_shared_inputs(logger) -> SharedInputs:
    diagnostics: list[str] = []

    sheet_parser = SheetMappingParser()
    dclgen_parser = DclgenParser()
    copybook_parser = CopybookParser()
    lrf_parser = LrfParser()
    file_loader = FileLoader()

    diagnostics.append(DIAG_START_LOAD_SHARED_INPUTS)
    logger.info(DIAG_START_LOAD_SHARED_INPUTS)

    sheet_rows = _load_mapping_rows(sheet_parser, diagnostics)
    dclgen_columns = _load_dclgen_columns(dclgen_parser, file_loader, diagnostics)
    copybook_fields = _load_copybook_fields(
        copybook_parser,
        file_loader,
        diagnostics,
    )
    logical_records = _load_logical_records(
        lrf_parser,
        file_loader,
        diagnostics,
        logger,
    )

    return SharedInputs(
        sheet_rows=sheet_rows,
        dclgen_columns=dclgen_columns,
        copybook_fields=copybook_fields,
        diagnostics=diagnostics,
        logical_records=logical_records,
    )

```

```
=====
FILE: src\idms_db2_phase2\testing\batch\update_postprocess_pipeline.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.composers.fixed_format_composer import FixedFormatComposer
from idms_db2_phase2.composers.procedure_indent_normalizer import (
    ProcedureIndentNormalizer,
)
from idms_db2_phase2.postprocess.update_char_literal_quoter import (
    UpdateCharLiteralQuoter,
)
from idms_db2_phase2.postprocess.update_date_host_converter import (
    UpdateDateHostConverter,
)
from idms_db2_phase2.postprocess.update_final_cleanup import UpdateFinalCleanup
from idms_db2_phase2.postprocess.update_input_record_renamer import (
    UpdateInputRecordRenamer,
)
from idms_db2_phase2.postprocess.update_program_enhancer import UpdateProgramEnhancer
from idms_db2_phase2.services.fixed_format_line_service import FixedFormatLineService
from idms_db2_phase2.services.name_derivation_resolver import NameDerivationResolver
from idms_db2_phase2.services.update_audit_move_inserter import (
    UpdateAuditMoveInserter,
)
from idms_db2_phase2.services.update_cobol_final_cleanup_utils import (
    UpdateCobolFinalCleanupUtils,
)
from patterns.batch_runner_patterns import COPY_NAME_PATTERN
from rules.batch_runner_messages import (
    DIAG_UPDATE_AUDIT_MOVES,
    DIAG_UPDATE_ENHANCEMENT_DONE,
    DIAG_UPDATE_SKIPPED_TEMPLATE,
)
```

```
def derive_input_record_rename(converted_cobol: str) -> tuple[str, str]:
    """Recover (old, new) input-record names from COPY statements.
```

```
    Applies the VM...BD...->VMDZ derivation. Returns ("", "") when no
    derivable COPY name is found. No name is hardcoded.
    """
```

```
    deriver = NameDerivationResolver()
```

```
    copy_names = COPY_NAME_PATTERN.findall(str(converted_cobol or ""))
```

```
    for raw in copy_names:
        name = str(raw or "").strip().upper()
        if not name:
            continue
        if deriver.is_derivable(name):
            derived = deriver.derive(name)
            if derived != name:
                return name, derived
```

```
    return "", ""
```

```
def apply_update_postprocess(
```

```
    *,
    source_text: str,
    converted_cobol: str,
    copybook_fields: list,
    dclgen_columns: list,
    target_program_id: str,
    diagnostics: list[str],
    logger,
) -> str:
    try:
        enhancer = UpdateProgramEnhancer()
        enhanced = enhancer.enhance(
            source_cobol=source_text,
            converted_cobol=converted_cobol,
            copybook_fields=copybook_fields,
```

```

        dclgen_columns=dclgen_columns,
        target_program_id=target_program_id,
    )

    diagnostics.extend(enhanced.diagnostics)
    diagnostics.append(DIAG_UPDATE_ENHANCEMENT_DONE)

    fixed = FixedFormatComposer().format(enhanced.converted_cobol)
    cleaned = UpdateFinalCleanup().apply(fixed)
    formatted = FixedFormatComposer().format(cleaned)

    _ff = FixedFormatLineService()
    _audit = UpdateAuditMoveInsertter(
        fixed_format=_ff,
        utils=UpdateCobolFinalCleanupUtils(_ff),
    )
    with_audit = _audit.insert_update_audit_moves(formatted)

    if with_audit != formatted:
        diagnostics.append(DIAG_UPDATE_AUDIT_MOVES)

    with_char = UpdateCharLiteralQuoter(dclgen_columns).apply(
        with_audit,
        diagnostics,
    )
    with_dates = UpdateDateHostConverter(dclgen_columns).apply(
        with_char,
        diagnostics,
    )

    old_record, new_record = derive_input_record_rename(with_dates)

    if old_record and new_record and old_record != new_record:
        with_renamed = UpdateInputRecordRenamer().apply(
            with_dates,
            old_name=old_record,
            new_name=new_record,
            diagnostics=diagnostics,
        )
    else:
        with_renamed = with_dates

    formatted_final = FixedFormatComposer().format(with_renamed)

    _indent = ProcedureIndentNormalizer()
    normalized = _indent.compose(formatted_final)
    if getattr(_indent, "messages", None):
        diagnostics.extend(_indent.messages)

    return FixedFormatComposer().format(normalized)

except Exception as exc:
    message = DIAG_UPDATE_SKIPPED_TEMPLATE.format(reason=exc)
    diagnostics.append(message)
    logger.exception(message)
    return converted_cobol

```

```

=====
FILE: src\idms_db2_phase2\testing\batch_console.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/testing/batch_console.py
# ACTION: REPLACE ENTIRE FILE

```

```

"""Console renderer for batch execution.

```

Layout only. Capability detection lives in console/terminal.py and the animation in console/progress_bar.py, so this class decides WHAT a line says and those two decide whether it can be drawn.

Design constraints

- Standard library only. The project's dependency list is deliberately short and a console renderer does not justify adding to it.

- Works in cmd.exe, PowerShell and a piped file. When stdout is not a terminal the animation is skipped, so a captured log holds one static line per step.
- Colour is an enhancement, never a requirement: every status is also spelled out as a word.

CORRECTIONS

1. verdict() embedded escape sequences directly inside f-string expressions. That is a syntax error before Python 3.12 and, more importantly, put a colour code in the layout layer where the palette belongs. It now uses the declared COLOUR_GREEN and COLOUR_RED.
2. step_finish() cleared the animated line by writing spaces and returning the cursor. The spaces survived in the terminal buffer and appeared as a long whitespace tail in any copied transcript. ProgressBar.clear_line() now erases the line properly.
3. closing() printed the Finished stamp immediately under the verdict. A blank line separates them.

```

"""
from __future__ import annotations

import sys
from datetime import datetime

from idms_db2_phase2.testing.console.progress_bar import ProgressBar
from idms_db2_phase2.testing.console.terminal import (
    Palette,
    TerminalCapabilities,
)
from rules.batch_console_rules import (
    COLOUR_GREEN,
    COLOUR_RED,
    COUNT_FILE_PLURAL,
    COUNT_FILE_SINGULAR,
    COUNT_NONE,
    FINISHED_TEMPLATE,
    INDENT,
    LABEL_WIDTH,
    RULE_HEAVY,
    RULE_LIGHT,
    STARTED_TEMPLATE,
    STATUS_COLOURS,
    STATUS_RUNNING,
    STATUS_WIDTH,
    STEP_COUNTER_TEMPLATE,
    STEP_LABEL_WIDTH,
    SUMMARY_COLUMN_SECONDS,
    SUMMARY_COLUMN_STATUS,
    SUMMARY_COLUMN_STEP,
    SUMMARY_HEADER_TEMPLATE,
    SUMMARY_ROW_TEMPLATE,
    SUMMARY_TOTAL_TEMPLATE,
    TIMESTAMP_FORMAT,
    TITLE,
    VALUE_UNKNOWN,
    VERDICT_FAILED_PLURAL,
    VERDICT_FAILED_SINGULAR,
    VERDICT_PASSED,
)

SUMMARY_STATUS_WIDTH = 12
SECONDS_TEMPLATE = "{seconds:>6.1f}s"

class BatchConsole:
    """Renders the batch execution report to stdout."""

    def __init__(self, quiet: bool = False) -> None:
        self.quiet = quiet
        self.capabilities = TerminalCapabilities()
        self.palette = Palette(self.capabilities)
        self.bar = ProgressBar(self.capabilities)

```

```

@property
def animated(self) -> bool:
    return self.capabilities.interactive and not self.quiet

# =====
# Frame
# =====
def banner(self) -> None:
    stamp = datetime.now().strftime(TIMESTAMP_FORMAT)
    self._line(RULE_HEAVY)
    self._line(f"{INDENT}{self.palette.bold(TITLE)}")
    self._line(
        f"{INDENT}{self.palette.dim(STARTED_TEMPLATE.format(stamp=stamp))}"
    )
    self._line(RULE_HEAVY)
    self._line("")

def closing(self) -> None:
    stamp = datetime.now().strftime(TIMESTAMP_FORMAT)
    self._line("")
    self._line(
        f"{INDENT}{self.palette.dim(FINISHED_TEMPLATE.format(stamp=stamp))}"
    )
    self._line(RULE_HEAVY)

def section(self, title: str) -> None:
    self._line(f"{INDENT}{self.palette.bold(title)}")
    self._line(f"{INDENT}{RULE_LIGHT}")

def blank(self) -> None:
    self._line("")

# =====
# Key / value
# =====
def key_value(self, label: str, value: object) -> None:
    text = VALUE_UNKNOWN if value is None else str(value)
    self._line(f"{INDENT}{label:<{LABEL_WIDTH}}{text}")

def file_count(self, label: str, count: int) -> None:
    if count <= 0:
        value = COUNT_NONE
    elif count == 1:
        value = COUNT_FILE_SINGULAR.format(count=count)
    else:
        value = COUNT_FILE_PLURAL.format(count=count)

    self.key_value(label, value)

# =====
# Step progress
# =====
def step_start(self, index: int, total: int, label: str) -> None:
    """Print the step line and begin the animation."""
    prefix = self._step_prefix(index, total, label)
    running = self._status(STATUS_RUNNING)

    if not self.animated:
        self._line(f"{prefix}{self.bar.render(0.0)} {running}")
        return

    self.bar.start(prefix, running)

def step_finish(
    self,
    index: int,
    total: int,
    label: str,
    status: str,
    seconds: float,
    note: str = "",
) -> None:
    """Stop the animation and replace the line with the outcome."""
    self.bar.stop()

    prefix = self._step_prefix(index, total, label)

```

```

suffix = SECONDS_TEMPLATE.format(seconds=seconds)
if note:
    suffix = f"{suffix} {self.palette.dim(note)}"

if self.animated:
    self.bar.clear_line()

self._line(
    f"{prefix}{self.bar.render(1.0)} {self._status(status)} {suffix}"
)

# =====
# Summary
# =====
def summary_header(self) -> None:
    header = SUMMARY_HEADER_TEMPLATE.format(
        step=SUMMARY_COLUMN_STEP,
        status=SUMMARY_COLUMN_STATUS,
        seconds=SUMMARY_COLUMN_SECONDS,
    )
    self._line(INDENT + self.palette.dim(header))

def summary_row(self, label: str, status: str, seconds: float) -> None:
    self._line(
        INDENT
        + SUMMARY_ROW_TEMPLATE.format(
            step=label,
            status=self._status(status, width=SUMMARY_STATUS_WIDTH),
            seconds=f"{seconds:.1f}",
        )
    )

def summary_total(self, seconds: float, failures: int) -> None:
    self._line(f"{INDENT}{RULE_LIGHT}")
    self._line(
        INDENT
        + SUMMARY_TOTAL_TEMPLATE.format(
            seconds=seconds,
            failures=failures,
        )
    )

def verdict(self, failures: int) -> None:
    self._line("")

    if failures == 0:
        self._line(
            f"{INDENT}{self.palette.paint(VERDICT_PASSED, COLOUR_GREEN)}"
        )
        return

    template = (
        VERDICT_FAILED_SINGULAR if failures == 1
        else VERDICT_FAILED_PLURAL
    )
    self._line(
        INDENT
        + self.palette.paint(
            template.format(count=failures),
            COLOUR_RED,
        )
    )

def detail(self, title: str, body: str) -> None:
    if not str(body or "").strip():
        return

    self._line("")
    self._line(f"{INDENT}{self.palette.bold(title)}")
    self._line(f"{INDENT}{RULE_LIGHT}")

    for raw in body.strip().splitlines():
        self._line(f"{INDENT}{raw.rstrip()}")

# =====
# Internals

```



```

# =====
@staticmethod
def _step_prefix(index: int, total: int, label: str) -> str:
    counter = STEP_COUNTER_TEMPLATE.format(index=index, total=total)
    return f"{INDENT}{counter} {label:<{STEP_LABEL_WIDTH}}"
```

```

def _status(self, status: str, width: int = STATUS_WIDTH) -> str:
    """Pad BEFORE colouring, so escape codes never affect alignment."""
    return self.palette.paint(
        str(status).ljust(width),
        STATUS_COLOURS.get(status, ""),
    )

@staticmethod
def _line(text: str) -> None:
    sys.stdout.write(text + "\n")
    sys.stdout.flush()

=====
FILE: src\idms_db2_phase2\testing\batch_execution.py
ENCODING: utf-8
=====

# LOCATION: src/idms_db2_phase2/testing/batch_execution.py
# ACTION: REPLACE ENTIRE FILE

"""Batch execution entry point: convert and code review in one command.

    set PYTHONPATH=src
    python src\idms_db2_phase2\testing\batch_execution.py
    python src\idms_db2_phase2\testing\batch_execution.py --mode retrieval
    python src\idms_db2_phase2\testing\batch_execution.py --mode update
    python src\idms_db2_phase2\testing\batch_execution.py --no-review
    python src\idms_db2_phase2\testing\batch_execution.py --quiet

Exit codes: 0 all steps passed, 1 a step was rejected, 2 a step errored.

Command line only. Every behaviour lives in
idms_db2_phase2.testing.execution.
"""

from __future__ import annotations

# -----
# Path bootstrap.
#
# This block MUST stay above every project import. Running
# "python src\idms_db2_phase2\testing\batch_execution.py" puts only the
# testing folder on sys.path, so without this the first import fails
# with ModuleNotFoundError.
# -----
import sys
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[3]
SRC_DIR = PROJECT_ROOT / "src"

for _candidate in (PROJECT_ROOT, SRC_DIR):
    if str(_candidate) not in sys.path:
        sys.path.insert(0, str(_candidate))
# -----

import argparse # noqa: E402

from idms_db2_phase2.testing.execution.orchestrator import ( # noqa: E402
    BatchExecution,
)
from rules.batch_console_rules import ( # noqa: E402
    EXIT_ERROR,
    MODE_ALL,
    MODES,
)

__all__ = ["BatchExecution", "parse_arguments", "main"]

```

```

def parse_arguments() -> argparse.Namespace:
    parser = argparse.ArgumentParser(
        description=(
            "Convert and code review IDMS COBOL programs in one batch."
        )
    )
    parser.add_argument(
        "--mode",
        choices=list(MODES),
        default=MODE_ALL,
        help="Which program family to process. Default: all.",
    )
    parser.add_argument(
        "--no-review",
        dest="review",
        action="store_false",
        help="Convert only; skip the code review steps.",
    )
    parser.add_argument(
        "--quiet",
        action="store_true",
        help="Suppress the animated progress bar.",
    )
    parser.set_defaults(review=True)
    return parser.parse_args()

def main() -> None:
    arguments = parse_arguments()

    batch = BatchExecution(
        mode=arguments.mode,
        review=arguments.review,
        quiet=arguments.quiet,
    )

    try:
        raise SystemExit(batch.run())
    except SystemExit:
        raise
    except KeyboardInterrupt:
        raise SystemExit(EXIT_ERROR)
    except Exception as exc: # noqa: BLE001
        print(f"ERROR: {exc}")
        raise SystemExit(EXIT_ERROR)

if __name__ == "__main__":
    main()

=====
FILE: src\idms_db2_phase2\testing\batch_program_runner.py
ENCODING: utf-8
=====

from __future__ import annotations

from pathlib import Path

from idms_db2_phase2.testing.batch.batch_environment import (
    bootstrap_sys_path,
    env_target_program_id,
)

SRC_DIR, PROJECT_ROOT = bootstrap_sys_path(Path(__file__))

from config.path_settings import DEFAULT_MAPPING_SHEET_DIR, LOGS_DIR # noqa: E402
from idms_db2_phase2.infrastructure.logger_factory import ( # noqa: E402
    LoggerFactory,
)
from idms_db2_phase2.testing.batch.batch_models import ( # noqa: E402
    BatchMode,
    SharedInputs,
)

```

```

from idms_db2_phase2.testing.batch.batch_reporter import ( # noqa: E402
    print_batch_summary,
)
from idms_db2_phase2.testing.batch.batch_validator import ( # noqa: E402
    program_paths_for_mode,
    validate_common_input_folders,
    validate_mode,
)
from idms_db2_phase2.testing.batch.program_converter import ( # noqa: E402
    convert_one_program,
)
from idms_db2_phase2.testing.batch.shared_input_loader import ( # noqa: E402
    load_shared_inputs,
)

__all__ = [
    "BatchMode",
    "SharedInputs",
    "env_target_program_id",
    "program_paths_for_mode",
    "load_shared_inputs",
    "convert_one_program",
    "run_batch",
]

def run_batch(mode: BatchMode) -> None:
    validate_common_input_folders()
    validate_mode(mode)

    logger = LoggerFactory.create_logger(
        name=f"idms_db2_{mode.name.lower()}_conversion",
        logs_dir=LOGS_DIR,
    )

    shared_inputs = load_shared_inputs(logger)
    programs = program_paths_for_mode(mode.name)
    target_program_id = env_target_program_id() or mode.default_target_program_id

    print_batch_summary(
        project_root=PROJECT_ROOT,
        src_dir=SRC_DIR,
        mode=mode,
        mapping_folder=DEFAULT_MAPPING_SHEET_DIR,
        program_count=len(programs),
        target_program_id=target_program_id,
    )

    for program_path in programs:
        convert_one_program(
            mode=mode,
            program_path=program_path,
            shared_inputs=shared_inputs,
            target_program_id=target_program_id,
            logger=logger,
        )

```

```

=====
FILE: src\idms_db2_phase2\testing\console\__init__.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/testing/console/__init__.py
# ACTION: CREATE NEW FILE

```

```

"""Terminal capability detection and progress rendering."""

```

```

=====
FILE: src\idms_db2_phase2\testing\console\progress_bar.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/testing/console/progress_bar.py
# ACTION: CREATE NEW FILE

```

"""Progress bar rendering and the indeterminate pulse animation.

The runners report no intermediate progress: each is a subprocess that prints a summary when it finishes. A percentage would therefore be a fiction, so a moving segment is used instead. It says "still working" without claiming to know how far along it is.

The animation is skipped entirely when stdout is not a terminal, so a piped transcript contains one static line per step rather than hundreds of carriage returns.

```
from __future__ import annotations

import sys
import threading

from idms_db2_phase2.testing.console.terminal import TerminalCapabilities
from rules.batch_console_rules import (
    BAR_CLOSE,
    BAR_OPEN,
    BAR_WIDTH,
    CARRIAGE_RETURN,
    CLEAR_PAD_WIDTH,
    ERASE_LINE,
    PULSE_INTERVAL_SECONDS,
    PULSE_SEGMENT,
)

class ProgressBar:
    """Draws the bar and owns the animation thread."""

    def __init__(self, capabilities: TerminalCapabilities) -> None:
        self.capabilities = capabilities
        self._stop: threading.Event | None = None
        self._thread: threading.Thread | None = None

    # =====
    # Rendering
    # =====
    def render(self, fraction: float = 1.0) -> str:
        filled = max(0, min(BAR_WIDTH, round(BAR_WIDTH * fraction)))
        return (
            BAR_OPEN
            + self.capabilities.filled * filled
            + self.capabilities.empty * (BAR_WIDTH - filled)
            + BAR_CLOSE
        )

    def _segment(self, position: int) -> str:
        cells = [self.capabilities.empty] * BAR_WIDTH
        for offset in range(PULSE_SEGMENT):
            cells[(position + offset) % BAR_WIDTH] = self.capabilities.filled
        return BAR_OPEN + "".join(cells) + BAR_CLOSE

    # =====
    # Animation
    # =====
    @property
    def running(self) -> bool:
        return self._thread is not None

    def start(self, prefix: str, suffix: str) -> None:
        """Begin the pulse. No-op when the console is not interactive."""
        if not self.capabilities.interactive:
            return

        self._stop = threading.Event()
        self._thread = threading.Thread(
            target=self._pulse,
            args=(prefix, suffix, self._stop),
            daemon=True,
        )
        self._thread.start()
```

```

def stop(self) -> None:
    if self._stop is not None:
        self._stop.set()
    if self._thread is not None:
        self._thread.join(timeout=1.0)
    self._stop = None
    self._thread = None

def _pulse(
    self,
    prefix: str,
    suffix: str,
    stop: threading.Event,
) -> None:
    position = 0
    while not stop.is_set():
        sys.stdout.write(
            f"{CARRIAGE_RETURN}{prefix}{self._segment(position)} {suffix}"
        )
        sys.stdout.flush()
        position = (position + 1) % BAR_WIDTH
        stop.wait(PULSE_INTERVAL_SECONDS)

```

```

# =====
# Line control
# =====

```

```

def clear_line(self) -> None:
    """Erase the animated line before writing the final one.

    Writing spaces and returning is not enough: the spaces persist in
    the terminal buffer and end up in any copied transcript, which is
    why the first version left a long tail of whitespace after every
    completed step.
    """
    if not self.capabilities.interactive:
        return

    if self.capabilities.ansi:
        sys.stdout.write(f"{CARRIAGE_RETURN}{ERASE_LINE}")
    else:
        sys.stdout.write(
            CARRIAGE_RETURN + " " * CLEAR_PAD_WIDTH + CARRIAGE_RETURN
        )

    sys.stdout.flush()

```

```

=====
FILE: src\idms_db2_phase2\testing\console\terminal.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/testing/console/terminal.py
# ACTION: CREATE NEW FILE

```

```

"""Terminal capability detection and colour.

```

Answers three questions once, at startup, so no renderer has to guess:

- is stdout a terminal, or is the batch being piped to a file?
- can escape sequences be used?
- can the console encode block glyphs, or must the bar be ASCII?

Colour is always an enhancement, never a requirement. Every status the batch prints is also spelled out as a word, so a transcript with the escapes stripped remains complete.

```

from __future__ import annotations

```

```

import os
import sys

```

```

from rules.batch_console_rules import (
    BAR_EMPTY_ASCII,

```

```

BAR_EMPTY_UNICODE,
BAR_FILLED_ASCII,
BAR_FILLED_UNICODE,
COLOUR_BOLD,
COLOUR_DIM,
COLOUR_RESET,
ENABLE_COLOUR,
NO_COLOUR_ENV,
)

WINDOWS_STDOUT_HANDLE = -11
ENABLE_VIRTUAL_TERMINAL_PROCESSING = 7

class TerminalCapabilities:
    """What this console can actually do."""

    def __init__(self) -> None:
        self._enable_windows_ansi()
        self.interactive = self._detect_interactive()
        self.colour = self._detect_colour()
        self.filled, self.empty = self._detect_glyphs()

    @property
    def ansi(self) -> bool:
        """True when escape sequences can be emitted.

        Tied to interactivity rather than to colour, because the
        erase-line sequence is still wanted when NO_COLOR is set.
        """
        return self.interactive

    # =====
    # Detection
    # =====
    @staticmethod
    def _detect_interactive() -> bool:
        try:
            return bool(sys.stdout.isatty())
        except Exception: # noqa: BLE001
            return False

    def _detect_colour(self) -> bool:
        if not ENABLE_COLOUR:
            return False
        if os.environ.get(NO_COLOUR_ENV):
            return False
        return self.interactive

    @staticmethod
    def _detect_glyphs() -> tuple[str, str]:
        """Block glyphs when the console can encode them, else ASCII.

        A legacy Windows code page raises UnicodeEncodeError on the block
        characters, which would abort the batch over a cosmetic choice.
        """
        encoding = getattr(sys.stdout, "encoding", "") or ""
        try:
            BAR_FILLED_UNICODE.encode(encoding)
            return BAR_FILLED_UNICODE, BAR_EMPTY_UNICODE
        except (LookupError, UnicodeEncodeError, AttributeError):
            return BAR_FILLED_ASCII, BAR_EMPTY_ASCII

    @staticmethod
    def _enable_windows_ansi() -> None:
        """Turn on virtual terminal processing on legacy Windows consoles."""
        if os.name != "nt":
            return
        try:
            import ctypes

            kernel32 = ctypes.windll.kernel32
            kernel32.SetConsoleMode(
                kernel32.GetStdHandle(WINDOWS_STDOUT_HANDLE),
                ENABLE_VIRTUAL_TERMINAL_PROCESSING,
            )

```

```

except Exception: # noqa: BLE001
    pass

class Palette:
    """Applies colour, or does nothing when colour is unavailable."""

    def __init__(self, capabilities: TerminalCapabilities) -> None:
        self.capabilities = capabilities

    def paint(self, text: str, code: str) -> str:
        if not self.capabilities.colour or not code:
            return str(text)
        return f"{code}{text}{COLOUR_RESET}"

    def bold(self, text: str) -> str:
        return self.paint(text, COLOUR_BOLD)

    def dim(self, text: str) -> str:
        return self.paint(text, COLOUR_DIM)

=====
FILE: src\idms_db2_phase2\testing\execution\__init__.py
ENCODING: utf-8
=====

# LOCATION: src/idms_db2_phase2/testing/execution/__init__.py
# ACTION: CREATE NEW FILE

"""Batch execution: convert and code review in one command."""

=====
FILE: src\idms_db2_phase2\testing\execution\environment.py
ENCODING: utf-8
=====

# LOCATION: src/idms_db2_phase2/testing/execution/environment.py
# ACTION: CREATE NEW FILE

"""Paths and child-process environment for batch execution.

Owns the two facts every other execution module needs: where the project
root is, and what environment a runner must be launched with.
"""

from __future__ import annotations

import os
from pathlib import Path

# ../src/idms_db2_phase2/testing/execution/environment.py -> project root
PROJECT_ROOT = Path(__file__).resolve().parents[4]
SRC_DIR = PROJECT_ROOT / "src"

PYTHONPATH_KEY = "PYTHONPATH"
UNBUFFERED_KEY = "PYTHONUNBUFFERED"

# Both entries are required: the converter lives under src/, while
# rules/, patterns/ and catalogs/ sit at the project root.
PYTHONPATH_ENTRIES = ("src", ".")

def runner_path(relative: str) -> Path:
    """Absolute path to a runner declared in rules/batch_console_rules."""
    return PROJECT_ROOT / relative

def subprocess_environment() -> dict[str, str]:
    """Environment for a runner subprocess.

    PYTHONPATH is set explicitly rather than inherited, so the batch
    behaves identically whether or not the operator set it first.
    Output is unbuffered so a long step still streams its log.
    """

```

```

env = dict(os.environ)

entries = [
    str(PROJECT_ROOT / entry) if entry != "." else str(PROJECT_ROOT)
    for entry in PYTHONPATH_ENTRIES
]

existing = env.get(PYTHONPATH_KEY, "")
if existing:
    entries.append(existing)

env[PYTHONPATH_KEY] = os.pathsep.join(entries)
env[UNBUFFERED_KEY] = "1"
return env

```

```

=====
FILE: src\idms_db2_phase2\testing\execution\input_inspector.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/testing/execution/input_inspector.py
# ACTION: CREATE NEW FILE

```

```

"""Counts input files and locates artefact folders.

```

```

File counts only. Row, column and field TOTALS are reported by the
runners themselves and scraped from their output, so this module never
has to open a workbook or a DCLGEN.

```

```

Every lookup degrades to zero or '-' rather than raising: a missing
folder is worth reporting, not worth aborting a batch for.
"""

```

```

from __future__ import annotations

```

```

from pathlib import Path

```

```

from rules.batch_console_rules import VALUE_UNKNOWN

```

```

COUNT_KEYS = ("mapping", "dclgen", "copybook", "retrieval", "update")

```

```

class InputInspector:

```

```

    """Reports what is present in the configured input folders."""

```

```

    def counts(self) -> dict[str, int]:
        empty = dict.fromkeys(COUNT_KEYS, 0)

```

```

        try:
            from config.path_settings import (
                DEFAULT_COPYBOOK_DIR,
                DEFAULT_DCLGEN_DIR,
                DEFAULT_MAPPING_SHEET_DIR,
                DEFAULT_RETRIEVAL_PROGRAM_DIR,
                DEFAULT_UPDATE_PROGRAM_DIR,
            )
        except Exception: # noqa: BLE001
            return empty

        return {
            "mapping": self._count(DEFAULT_MAPPING_SHEET_DIR),
            "dclgen": self._count(DEFAULT_DCLGEN_DIR),
            "copybook": self._count(DEFAULT_COPYBOOK_DIR),
            "retrieval": self._count(DEFAULT_RETRIEVAL_PROGRAM_DIR),
            "update": self._count(DEFAULT_UPDATE_PROGRAM_DIR),
        }

```

```

    def folders(self) -> tuple[str, str]:
        """(output folder, logs folder)."""

```

```

        try:
            from config.path_settings import DEFAULT_OUTPUT_DIR, LOGS_DIR

            return str(DEFAULT_OUTPUT_DIR), str(LOGS_DIR)
        except Exception: # noqa: BLE001
            return VALUE_UNKNOWN, VALUE_UNKNOWN

```



```

@staticmethod
def _count(folder) -> int:
    try:
        return sum(1 for path in Path(folder).iterdir() if path.is_file())
    except Exception: # noqa: BLE001
        return 0

=====
FILE: src\idms_db2_phase2\testing\execution\models.py
ENCODING: utf-8
=====

# LOCATION: src/idms_db2_phase2/testing/execution/models.py
# ACTION: CREATE NEW FILE

"""Batch execution data models. Data only, no behaviour beyond derivation."""

from __future__ import annotations

from dataclasses import dataclass, field

from rules.batch_console_rules import (
    FAILING_STATUSES,
    MODE_ALL,
    MODE_RETRIEVAL,
    MODE_UPDATE,
    STATUS_FAILED,
    STEP_CONVERT_RETRIEVAL,
    STEP_CONVERT_UPDATE,
    STEP_DEFINITIONS,
    STEP_REVIEW_RETRIEVAL,
    STEP_REVIEW_UPDATE,
    VALUE_UNKNOWN,
)

RETRIEVAL_STEPS = (STEP_CONVERT_RETRIEVAL, STEP_REVIEW_RETRIEVAL)
UPDATE_STEPS = (STEP_CONVERT_UPDATE, STEP_REVIEW_UPDATE)

@dataclass(frozen=True)
class BatchStep:
    """One declared step, before it runs."""

    key: str
    label: str
    path: str
    is_review: bool

    def in_scope(self, mode: str) -> bool:
        if mode == MODE_ALL:
            return True
        if mode == MODE_RETRIEVAL:
            return self.key in RETRIEVAL_STEPS
        if mode == MODE_UPDATE:
            return self.key in UPDATE_STEPS
        return True

    @classmethod
    def declared(cls) -> list["BatchStep"]:
        return [
            cls(key=key, label=label, path=path, is_review=is_review)
            for key, label, path, is_review in STEP_DEFINITIONS
        ]

@dataclass
class StepOutcome:
    """One executed step."""

    key: str
    label: str
    status: str
    seconds: float = 0.0
    note: str = ""

```

```

stdout: str = ""
stderr: str = ""

@property
def is_failure(self) -> bool:
    return self.status in FAILING_STATUSES

@property
def is_error(self) -> bool:
    return self.status == STATUS_FAILED

@property
def combined_output(self) -> str:
    return f"{self.stdout or ''}\n{self.stderr or ''}"

@dataclass
class BatchOutcome:
    """Everything the summary needs."""

    steps: list[StepOutcome] = field(default_factory=list)
    mapping_rows: str = VALUE_UNKNOWN
    dclgen_columns: str = VALUE_UNKNOWN
    copybook_fields: str = VALUE_UNKNOWN
    report_folder: str = ""

    @property
    def failures(self) -> int:
        return sum(1 for step in self.steps if step.is_failure)

    @property
    def errors(self) -> int:
        return sum(1 for step in self.steps if step.is_error)

    @property
    def seconds(self) -> float:
        return sum(step.seconds for step in self.steps)

    @property
    def failing_steps(self) -> list[StepOutcome]:
        return [step for step in self.steps if step.is_failure]

    @property
    def has_metadata(self) -> bool:
        return any(
            value != VALUE_UNKNOWN
            for value in (
                self.mapping_rows,
                self.dclgen_columns,
                self.copybook_fields,
            )
        )

=====
FILE: src\idms_db2_phase2\testing\execution\orchestrator.py
ENCODING: utf-8
=====

# LOCATION: src/idms_db2_phase2/testing/execution/orchestrator.py
# ACTION: CREATE NEW FILE

"""Batch orchestrator: selects steps, runs them, reports the outcome."""

from __future__ import annotations

from idms_db2_phase2.testing.batch_console import BatchConsole
from idms_db2_phase2.testing.execution.models import (
    BatchOutcome,
    BatchStep,
    StepOutcome,
)
from idms_db2_phase2.testing.execution.output_reader import OutputReader
from idms_db2_phase2.testing.execution.report_renderer import ReportRenderer
from idms_db2_phase2.testing.execution.step_runner import StepRunner
from rules.batch_console_rules import (

```

```

EXIT_ERROR,
EXIT_OK,
EXIT_REJECTED,
MODE_ALL,
MODES,
VALUE_UNKNOWN,
)

class BatchExecution:
    """Runs conversion and code review as one reported batch."""

    def __init__(
        self,
        mode: str = MODE_ALL,
        review: bool = True,
        quiet: bool = False,
    ) -> None:
        self.mode = mode if mode in MODES else MODE_ALL
        self.review = review

        self.console = BatchConsole(quiet=quiet)
        self.reader = OutputReader()
        self.runner = StepRunner(console=self.console, reader=self.reader)
        self.renderer = ReportRenderer(
            console=self.console, reader=self.reader
        )
        self.outcome = BatchOutcome()

    # =====
    # Public entry point
    # =====
    def run(self) -> int:
        self.console.banner()
        self.renderer.inputs(self.mode)

        self._execute()

        self.renderer.metadata(self.outcome)
        self.renderer.summary(self.outcome)
        self.renderer.artefacts(self.outcome)
        self.renderer.failures(self.outcome)
        self.renderer.verdict(self.outcome)
        self.console.closing()

        return self._exit_code()

    # =====
    # Execution
    # =====
    def selected_steps(self) -> list[BatchStep]:
        """Steps that will actually run, in order.

        Scope is applied HERE rather than inside the loop, so the step
        counter reflects the work being done. Listing four steps and
        marking two SKIPPED told the reader nothing and made [2/4] a lie.
        """
        return [
            step
            for step in BatchStep.declared()
            if (self.review or not step.is_review) and step.in_scope(self.mode)
        ]

    def _execute(self) -> None:
        self.renderer.execution_header()

        steps = self.selected_steps()
        total = len(steps)

        for index, step in enumerate(steps, start=1):
            outcome = self.runner.run(index, total, step)
            self.outcome.steps.append(outcome)
            self._absorb(outcome)

        self.renderer.execution_footer()

```

```

# =====
# Metadata harvesting
# =====
def _absorb(self, outcome: StepOutcome) -> None:
    """Keep the first value any runner reported."""
    text = outcome.stdout or ""
    if not text:
        return

    if self.outcome.mapping_rows == VALUE_UNKNOWN:
        self.outcome.mapping_rows = self.reader.mapping_rows(text)

    if self.outcome.dclgen_columns == VALUE_UNKNOWN:
        self.outcome.dclgen_columns = self.reader.dclgen_columns(text)

    if self.outcome.copybook_fields == VALUE_UNKNOWN:
        self.outcome.copybook_fields = self.reader.copybook_fields(text)

    if not self.outcome.report_folder:
        self.outcome.report_folder = self.reader.report_folder(text)

# =====
# Exit code
# =====
def _exit_code(self) -> int:
    if self.outcome.errors:
        return EXIT_ERROR
    if self.outcome.failures:
        return EXIT_REJECTED
    return EXIT_OK

```

```

=====
FILE: src\idms_db2_phase2\testing\execution\output_reader.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/testing/execution/output_reader.py
# ACTION: CREATE NEW FILE

```

""""Reads meaning out of a runner's console output.

The batch launches each runner as a subprocess, so its output IS the contract. This module turns that text into a status, a one-line note, metadata counts, and a short failure explanation.

CORRECTION - failure detail showed INFO lines

The first version printed the last five stderr lines. LoggerFactory attaches a StreamHandler, which defaults to stderr, so a rejected review reported five INFO lines about copybook parsing instead of the blocking check id. Detail is now assembled in priority order and INFO and DEBUG are excluded outright.
""""

```
from __future__ import annotations
```

```

from patterns.batch_console_patterns import (
    BLOCKING_PATTERN,
    CONVERSION_FAILED_PATTERN,
    COPYBOOK_FIELDS_PATTERN,
    DCLGEN_COLUMNS_PATTERN,
    ERROR_LINE_PATTERN,
    LOG_LINE_PATTERN,
    MAPPING_ROWS_PATTERN,
    NOTHING_TO_DO_PATTERN,
    QUIET_VERDICT_PATTERN,
    REPORT_FOLDER_PATTERN,
    REVIEW_SUMMARY_PATTERN,
    TRACEBACK_PATTERN,
)
from rules.batch_console_rules import (
    CONVERT_EXIT_STATUS,
    DETAIL_LOG_LEVELS,
    DETAIL_MAX_LINES,
    MSG_BLOCKING_TEMPLATE,

```

```

MSG_VERDICT_TEMPLATE,
REVIEW_EXIT_STATUS,
REVIEW_NOTE_TEMPLATE,
STATUS_FAILED,
STATUS_NOTHING,
VALUE_UNKNOWN,
)

class OutputReader:
    """Interprets runner stdout and stderr."""

    # =====
    # Status
    # =====
    @staticmethod
    def status_for(code: int, stdout: str, is_review: bool) -> str:
        """Map an exit code to a status, honouring 'nothing to do'.

        A runner that finds no programs exits 2, the same code it uses
        for a genuine error. Its own wording disambiguates, so an empty
        program folder reports NOTHING rather than a false failure.
        """
        if NOTHING_TO_DO_PATTERN.search(stdout or ""):
            return STATUS_NOTHING

        table = REVIEW_EXIT_STATUS if is_review else CONVERT_EXIT_STATUS
        return table.get(code, STATUS_FAILED)

    @staticmethod
    def note_for(stdout: str, is_review: bool) -> str:
        """The one-line summary shown beside a completed step."""
        if not is_review:
            return ""

        match = REVIEW_SUMMARY_PATTERN.search(stdout or "")
        if not match:
            return ""

        return REVIEW_NOTE_TEMPLATE.format(
            reviewed=match.group("reviewed"),
            accepted=match.group("accepted"),
            rejected=match.group("rejected"),
        )

    # =====
    # Metadata
    # =====
    @classmethod
    def mapping_rows(cls, text: str) -> str:
        return cls._first(MAPPING_ROWS_PATTERN, text, "count")

    @classmethod
    def dclgen_columns(cls, text: str) -> str:
        return cls._first(DCLGEN_COLUMNS_PATTERN, text, "count")

    @classmethod
    def copybook_fields(cls, text: str) -> str:
        return cls._first(COPYBOOK_FIELDS_PATTERN, text, "count")

    @classmethod
    def report_folder(cls, text: str) -> str:
        value = cls._first(REPORT_FOLDER_PATTERN, text, "path")
        return "" if value == VALUE_UNKNOWN else value

    @staticmethod
    def _first(pattern, text: str, group: str) -> str:
        match = pattern.search(text or "")
        if not match:
            return VALUE_UNKNOWN
        return str(match.group(group)).strip() or VALUE_UNKNOWN

    # =====
    # Failure detail
    # =====
    @staticmethod

```

```

def failure_lines(combined: str) -> list[str]:
    """The few facts that explain a failure.

    Priority order:
    1. blocking check ids from a rejected code review
    2. the per-program verdict lines
    3. explicit ERROR: lines
    4. a conversion-failed list
    5. WARNING and above from the logger stream
    6. exception lines from a traceback

    INFO and DEBUG are deliberately excluded: they belong in the log
    file, not in a summary read at a glance.
    """
    text = combined or ""
    out: list[str] = []
    seen: set[str] = set()

    def add(value: str) -> None:
        clean = " ".join(str(value or "").split())
        if clean and clean not in seen:
            seen.add(clean)
            out.append(clean)

    for match in BLOCKING_PATTERN.finditer(text):
        add(MSG_BLOCKING_TEMPLATE.format(ids=match.group("ids").strip()))

    for match in QUIET_VERDICT_PATTERN.finditer(text):
        add(
            MSG_VERDICT_TEMPLATE.format(
                verdict=match.group("verdict").upper(),
                name=match.group("name"),
            )
        )

    for match in ERROR_LINE_PATTERN.finditer(text):
        add(match.group("message"))

    for match in CONVERSION_FAILED_PATTERN.finditer(text):
        add(f"Conversion failed: {match.group('names')}")

    for raw in text.splitlines():
        match = LOG_LINE_PATTERN.match(raw.strip())
        if match and match.group("level").upper() in DETAIL_LOG_LEVELS:
            add(match.group("message"))

    for match in TRACEBACK_PATTERN.finditer(text):
        add(match.group("line"))

    return out[:DETAIL_MAX_LINES]

```

```

=====
FILE: src\idms_db2_phase2\testing\execution\report_renderer.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/testing/execution/report_renderer.py
# ACTION: CREATE NEW FILE

```

```

"""Renders every batch section through BatchConsole.

```

```

Presentation orchestration only: decides WHAT appears in each section,
while BatchConsole decides HOW a line is drawn.
"""

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.testing.execution.environment import PROJECT_ROOT
from idms_db2_phase2.testing.execution.input_inspector import InputInspector
from idms_db2_phase2.testing.execution.models import BatchOutcome
from idms_db2_phase2.testing.execution.output_reader import OutputReader
from rules.batch_console_rules import (
    DEFAULT_REPORT_FOLDER,
    LABEL_COPYBOOK,
    LABEL_COPYBOOK_FIELDS,

```

```

LABEL_DCLGEN,
LABEL_DCLGEN_COLUMNS,
LABEL_LOG_FOLDER,
LABEL_MAPPING_ROWS,
LABEL_MAPPING_SHEET,
LABEL_MODE,
LABEL_OUTPUT_FOLDER,
LABEL_REPORT_FOLDER,
LABEL_RETRIEVAL_PROGRAMS,
LABEL_UPDATE_PROGRAMS,
MSG_DETAIL_HEADER,
MSG_HINT_REPORTS,
MSG_NO_DETAIL,
SECTION_ARTEFACTS,
SECTION_EXECUTION,
SECTION_INPUTS,
SECTION_METADATA,
SECTION_SUMMARY,
)

class ReportRenderer:
    """Builds the on-screen batch report."""

    def __init__(
        self,
        console,
        inspector: InputInspector | None = None,
        reader: OutputReader | None = None,
    ) -> None:
        self.console = console
        self.inspector = inspector or InputInspector()
        self.reader = reader or OutputReader()

    # =====
    # Sections
    # =====
    def inputs(self, mode: str) -> None:
        self.console.section(SECTION_INPUTS)
        self.console.key_value(LABEL_MODE, mode)

        counts = self.inspector.counts()
        self.console.file_count(LABEL_MAPPING_SHEET, counts["mapping"])
        self.console.file_count(LABEL_DCLGEN, counts["dclgen"])
        self.console.file_count(LABEL_COPYBOOK, counts["copybook"])
        self.console.key_value(LABEL_RETRIEVAL_PROGRAMS, counts["retrieval"])
        self.console.key_value(LABEL_UPDATE_PROGRAMS, counts["update"])
        self.console.blank()

    def execution_header(self) -> None:
        self.console.section(SECTION_EXECUTION)

    def execution_footer(self) -> None:
        self.console.blank()

    def metadata(self, outcome: BatchOutcome) -> None:
        if not outcome.has_metadata:
            return

        self.console.section(SECTION_METADATA)
        self.console.key_value(LABEL_MAPPING_ROWS, outcome.mapping_rows)
        self.console.key_value(LABEL_DCLGEN_COLUMNS, outcome.dclgen_columns)
        self.console.key_value(LABEL_COPYBOOK_FIELDS, outcome.copybook_fields)
        self.console.blank()

    def summary(self, outcome: BatchOutcome) -> None:
        self.console.section(SECTION_SUMMARY)
        self.console.summary_header()

        for step in outcome.steps:
            self.console.summary_row(step.label, step.status, step.seconds)

        self.console.summary_total(outcome.seconds, outcome.failures)
        self.console.blank()

    def artefacts(self, outcome: BatchOutcome) -> None:

```

```

output_folder, logs_folder = self.inspector.folders()
report_folder = outcome.report_folder or str(
    PROJECT_ROOT / DEFAULT_REPORT_FOLDER
)

self.console.section(SECTION_ARTEFACTS)
self.console.key_value(LABEL_OUTPUT_FOLDER, output_folder)
self.console.key_value(LABEL_REPORT_FOLDER, report_folder)
self.console.key_value(LABEL_LOG_FOLDER, logs_folder)

def failures(self, outcome: BatchOutcome) -> None:
    failing = outcome.failing_steps
    if not failing:
        return

    lines: list[str] = []

    for step in failing:
        lines.append(f"{step.label}: {step.status}")

        detail = self.reader.failure_lines(step.combined_output)
        if detail:
            lines.extend(f" {entry}" for entry in detail)
        else:
            lines.append(f" {MSG_NO_DETAIL}")

        lines.append("")

    lines.append(MSG_HINT_REPORTS)
    self.console.detail(MSG_DETAIL_HEADER, "\n".join(lines))

def verdict(self, outcome: BatchOutcome) -> None:
    self.console.verdict(outcome.failures)

```

```

=====
FILE: src\idms_db2_phase2\testing\execution\step_runner.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/testing/execution/step_runner.py
# ACTION: CREATE NEW FILE

```

```

"""Runs one batch step as a subprocess.

```

```

Each runner is launched as its own process rather than imported:

```

- a runner that calls sys.exit, as these do, would otherwise terminate the batch;
- each runner performs its own sys.path bootstrap, which conflicts when several are imported into one interpreter;
- a change inside a runner cannot break the batch, because the contract is the command line and the exit code, not a function signature.

```

"""

```

```

from __future__ import annotations

```

```

import subprocess
import sys
import time

```

```

from idms_db2_phase2.testing.execution.environment import (
    runner_path,
    subprocess_environment,
)
from idms_db2_phase2.testing.execution.models import BatchStep, StepOutcome
from idms_db2_phase2.testing.execution.output_reader import OutputReader
from idms_db2_phase2.testing.execution.environment import PROJECT_ROOT
from rules.batch_console_rules import (
    MSG_RUNNER_MISSING,
    REVIEW_QUIET_ARG,
    STATUS_FAILED,
    STEP_TIMEOUT_SECONDS,
)

```

```

TIMEOUT_NOTE_TEMPLATE = "exceeded {seconds}s"

```



```

class StepRunner:
    """Executes a single step and reports progress through the console."""

    def __init__(self, console, reader: OutputReader | None = None) -> None:
        self.console = console
        self.reader = reader or OutputReader()

    def run(self, index: int, total: int, step: BatchStep) -> StepOutcome:
        runner = runner_path(step.path)

        if not runner.exists():
            return self._failed(
                index,
                total,
                step,
                seconds=0.0,
                note=MSG_RUNNER_MISSING.format(path=step.path),
            )

        self.console.step_start(index, total, step.label)
        started = time.monotonic()

        try:
            completed = subprocess.run(
                self._command(runner, step.is_review),
                cwd=str(PROJECT_ROOT),
                env=subprocess_environment(),
                capture_output=True,
                text=True,
                timeout=STEP_TIMEOUT_SECONDS,
                check=False,
            )

        except subprocess.TimeoutExpired:
            return self._failed(
                index,
                total,
                step,
                seconds=time.monotonic() - started,
                note=TIMEOUT_NOTE_TEMPLATE.format(
                    seconds=STEP_TIMEOUT_SECONDS
                ),
            )

        except Exception as exc: # noqa: BLE001
            return self._failed(
                index,
                total,
                step,
                seconds=time.monotonic() - started,
                note=str(exc),
            )

        seconds = time.monotonic() - started
        stdout = completed.stdout or ""
        stderr = completed.stderr or ""

        status = self.reader.status_for(
            completed.returncode, stdout, step.is_review
        )
        note = self.reader.note_for(stdout, step.is_review)

        self.console.step_finish(
            index, total, step.label, status, seconds, note
        )

        return StepOutcome(
            key=step.key,
            label=step.label,
            status=status,
            seconds=seconds,
            note=note,
            stdout=stdout,
            stderr=stderr,

```

```

)

# =====
# Helpers
# =====
@staticmethod
def _command(runner, is_review: bool) -> list[str]:
    command = [sys.executable, str(runner)]
    if is_review:
        command.append(REVIEW_QUIET_ARG)
    return command

def _failed(
    self,
    index: int,
    total: int,
    step: BatchStep,
    seconds: float,
    note: str,
) -> StepOutcome:
    self.console.step_finish(
        index, total, step.label, STATUS_FAILED, seconds, note
    )
    return StepOutcome(
        key=step.key,
        label=step.label,
        status=STATUS_FAILED,
        seconds=seconds,
        note=note,
    )

=====
FILE: src\idms_db2_phase2\testing\line_preservation_audit.py
ENCODING: utf-8
=====

# LOCATION: src/idms_db2_phase2/testing/line_preservation_audit.py
# ACTION: REPLACE ENTIRE FILE

"""
Line-preservation audit tool.

Answers the core question:
"Did the converter change ONLY the IDMS/DB2 lines and leave every other
business line byte-for-byte identical?"

This is the CORRECT benchmark for a surgical IDMS->DB2 converter – NOT a
match against a human rewrite manual (which adds paragraph renaming,
counters, restructuring, etc. that the converter deliberately does not do).

Classification of each INPUT line:
    CONVERTIBLE -> allowed to change (IDMS statement / control / mapped ref)
    PRESERVE    -> must appear unchanged in the output
    COMMENT/BLANK -> ignored

The audit reports:
- PRESERVE lines that were dropped or modified -> POTENTIAL BUG
- CONVERTIBLE lines that changed -> expected
- A pass/fail summary

Usage (from project root, with PYTHONPATH=src):
python src\idms_db2_phase2\testing\line_preservation_audit.py ^
--input C:\S\S-Input\Update.txt ^
--output C:\S\S-Input\Output\Update_<timestamp>.cbl ^
[--verbose]
"""

from __future__ import annotations

import argparse
import re
import sys
from pathlib import Path

CURRENT_FILE = Path(__file__).resolve()

```

```

SRC_DIR = CURRENT_FILE.parents[2]
PROJECT_ROOT = CURRENT_FILE.parents[3]
for path in (PROJECT_ROOT, SRC_DIR):
    if str(path) not in sys.path:
        sys.path.insert(0, str(path))

# --- Fixed-format helpers ---

def _strip_sequence_area(line: str) -> str:
    """
    Return the COBOL body (columns 8-72 area), handling fixed-format:
    - cols 1-6 : sequence number
    - col 7 : indicator (* / D)
    - cols 8-72 : body
    - cols 73-80: right sequence
    """
    text = str(line or "").rstrip("\n").rstrip()

    # Full fixed-format line (>= 72 with digit left sequence).
    if len(text) >= 72 and text[:6].strip().isdigit():
        return text[7:72].strip()

    # Left sequence only.
    if len(text) > 6 and text[:6].strip().isdigit():
        body = text[6:]
        # Drop a trailing 8-digit right sequence if present.
        if len(body) >= 8 and body[-8:].strip().isdigit():
            body = body[:-8]
        # Column-7 indicator handling.
        if body[:1] in ("*", "/", "D"):
            return body.strip()
        return body.strip()

    return text.strip()

def _indicator(line: str) -> str:
    """Return the column-7 indicator char, or '' if not fixed-format."""
    text = str(line or "").rstrip("\n")
    if len(text) >= 7 and text[:6].strip().isdigit():
        return text[6:7]
    return ""

def _is_comment_or_blank(line: str) -> bool:
    ind = _indicator(line)
    if ind in ("*", "/"):
        return True
    body = _strip_sequence_area(line)
    if not body:
        return True
    if body.startswith("*") or body.startswith("/"):
        return True
    return False

def _norm(body: str) -> str:
    """
    Collapse internal whitespace and drop a trailing period so that
    re-indentation and period differences are not treated as changes.
    """
    text = re.sub(r"\s+", " ", str(body or "").strip()).upper()
    return text.rstrip(".")

# --- CONVERTIBLE patterns (allowed to change) ---

_CONVERTIBLE_PATTERNS = [
    # IDMS executable database verbs
    re.compile(r"^\s*OBTAIN\b", re.IGNORECASE),
    re.compile(r"^\s*FIND\s+CURRENT\b", re.IGNORECASE),
    re.compile(r"^\s*FIND\s+FIRST\b", re.IGNORECASE),
    re.compile(r"^\s*STORE\b", re.IGNORECASE),
    re.compile(r"^\s*MODIFY\b", re.IGNORECASE),
    re.compile(r"^\s*ERASE\b", re.IGNORECASE),

```

```

re.compile(r"^\s*CONNECT\b", re.IGNORECASE),
re.compile(r"^\s*DISCONNECT\b", re.IGNORECASE),
re.compile(r"^\s*FINISH\b", re.IGNORECASE),
re.compile(r"^\s*BIND\b", re.IGNORECASE),
re.compile(r"^\s*READY\b", re.IGNORECASE),
re.compile(r"\bUSAGE-MODE\s+IS\s+(UPDATE|RETRIEVAL)\b", re.IGNORECASE),
re.compile(r"\bIDMS-STATUS\b", re.IGNORECASE),
re.compile(r"\bIDMS-ABORT\b", re.IGNORECASE),
re.compile(r"^\s*ON\s+DB-REC-NOT-FOUND\b", re.IGNORECASE),
re.compile(r"\bDB-REC-NOT-FOUND\b", re.IGNORECASE),
re.compile(r"\bDB-END-OF-SET\b", re.IGNORECASE),
# IDMS declarative / control
re.compile(r"^\s*IDMS-CONTROL\s+SECTION\b", re.IGNORECASE),
re.compile(r"^\s*PROTOCOL\b", re.IGNORECASE),
re.compile(r"^\s*IDMS-RECORDS\s+WITHIN\b", re.IGNORECASE),
re.compile(r"^\s*SCHEMA\s+SECTION\b", re.IGNORECASE),
re.compile(r"^\s*DB\s+[A-Z0-9-]+\s+WITHIN\s+[A-Z0-9-]+\b", re.IGNORECASE),
re.compile(r"^\s*COPY\s+IDMS\b", re.IGNORECASE),
# DB2-related lines the converter may change
re.compile(r"^\s*COMMIT\b", re.IGNORECASE),
re.compile(r"^\s*CBL\b", re.IGNORECASE),
re.compile(r"^\s*PROGRAM-ID\b", re.IGNORECASE),
re.compile(r"\bTO\s+PROGRAM-NAME\b", re.IGNORECASE),
# A move whose TARGET is a date field DA-/DT- (gets date conversion).
re.compile(r"\bTO\s+(?:DA|DT)-[A-Z0-9-]+\b", re.IGNORECASE),
# A move/reference to an IDMS record (bare or qualified).
re.compile(r"\bTO\s+VMB[A-Z0-9-]*\s*\.\s*\$?", re.IGNORECASE),
re.compile(r"\bTO\s+FFRECAB\b", re.IGNORECASE),
re.compile(r"\bOF\s+FFRECAB\b", re.IGNORECASE),
re.compile(r"\bOF\s+VMB[A-Z0-9-]*\b", re.IGNORECASE),
]

_QUALIFIED_REF_PATTERN = re.compile(
    r"\b[A-Z][A-Z0-9-]*\s+(?:OF|IN)\s+[A-Z][A-Z0-9-]*\b",
    re.IGNORECASE,
)

```

```

def _is_convertible(body: str) -> bool:
    for pattern in _CONVERTIBLE_PATTERNS:
        if pattern.search(body):
            return True
    match = _QUALIFIED_REF_PATTERN.search(body)
    if match:
        upper = body.upper()
        if " OF DCL" not in upper and " IN DCL" not in upper:
            return True
    return False

```

```

def audit(input_text: str, output_text: str) -> dict:
    input_lines = input_text.splitlines()
    output_bodies = {
        _norm(_strip_sequence_area(line))
        for line in output_text.splitlines()
        if not _is_comment_or_blank(line)
    }

    preserved_ok: list[tuple[int, str]] = []
    preserved_missing: list[tuple[int, str]] = []
    convertible: list[tuple[int, str]] = []
    comments_blank = 0

    for index, raw in enumerate(input_lines, start=1):
        if _is_comment_or_blank(raw):
            comments_blank += 1
            continue

        body = _strip_sequence_area(raw)

        if _is_convertible(body):
            convertible.append((index, body))
            continue

        if _norm(body) in output_bodies:
            preserved_ok.append((index, body))

```

```

        else:
            preserved_missing.append((index, body))

    return {
        "preserved_ok": preserved_ok,
        "preserved_missing": preserved_missing,
        "convertible": convertible,
        "comments_blank": comments_blank,
        "total_preserve": len(preserved_ok) + len(preserved_missing),
        "passed": len(preserved_missing) == 0,
    }

def print_report(report: dict, verbose: bool = False) -> None:
    print("")
    print("=" * 70)
    print("LINE-PRESERVATION AUDIT")
    print("=" * 70)
    print(f"Comment/blank lines (ignored)      : {report['comments_blank']}")
    print(
        f"Convertible lines (allowed change) : "
        f"{len(report['convertible'])}"
    )
    print(f"Preserve lines (must stay same)      : {report['total_preserve']}")
    print(
        f"    -> preserved correctly              : "
        f"{len(report['preserved_ok'])}"
    )
    print(
        f"    -> MISSING / CHANGED (bugs)        : "
        f"{len(report['preserved_missing'])}"
    )
    print("-" * 70)

    if report["preserved_missing"]:
        print("BUSINESS lines dropped or changed (investigate each):")
        print("")
        for line_number, body in report["preserved_missing"]:
            print(f"    input line {line_number:>5}: {body}")
        print("")
    else:
        print("No business lines were dropped or modified.")
        print("Only IDMS/DB2 lines changed. Correct surgical conversion.")

    if verbose:
        print("-" * 70)
        print("Convertible lines (expected to change):")
        for line_number, body in report["convertible"]:
            print(f"    input line {line_number:>5}: {body}")

    print("=" * 70)
    print(f"RESULT: {'PASS' if report['passed'] else 'FAIL'}")
    print("=" * 70)

def main() -> None:
    parser = argparse.ArgumentParser(
        description=(
            "Audit whether the converter changed ONLY IDMS/DB2 lines and "
            "preserved all other business logic."
        )
    )
    parser.add_argument(
        "--input",
        required=True,
        help="Path to the original IDMS COBOL source (converter input).",
    )
    parser.add_argument(
        "--output",
        required=True,
        help="Path to the generated DB2 COBOL (converter output).",
    )
    parser.add_argument(
        "--verbose",
        action="store_true",
        help="Also list the convertible lines that were expected to change.",
    )

```

```

)
args = parser.parse_args()

input_text = Path(args.input).read_text(
    encoding="utf-8", errors="replace"
)
output_text = Path(args.output).read_text(
    encoding="utf-8", errors="replace"
)

report = audit(input_text=input_text, output_text=output_text)
print_report(report, verbose=args.verbose)
sys.exit(0 if report["passed"] else 1)

```

```

if __name__ == "__main__":
    main()

```

```

=====
FILE: src\idms_db2_phase2\testing\pytest_runner.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/testing/pytest_runner.py
# ACTION: CREATE NEW FILE

```

```

"""Runs the project's pytest suite and returns structured results.

```

Design notes

- pytest is invoked as a SUBPROCESS, never imported in-process. A test module that mutates global state, monkeypatches a shared service or calls sys.exit would otherwise corrupt the running Streamlit session.
- Results are read from pytest's JUnit XML report. That format is part of pytest core, so no plugin is added to the project's dependencies, and parsing is deterministic rather than scraping console text.
- PYTHONPATH is set explicitly for the child process. The converter lives under src/ while rules/, patterns/ and catalogs/ sit at the project root, so both entries are required.
- The run is bounded by a timeout. A hanging suite must not hang the UI.

This module contains no Streamlit code and no COBOL knowledge.

```

"""

```

```

from __future__ import annotations

```

```

import os
import subprocess
import sys
import tempfile
import xml.etree.ElementTree as ElementTree
from dataclasses import dataclass, field
from datetime import datetime
from pathlib import Path

```

```

from rules.test_runner_rules import (
    ALL_TESTS_LABEL,
    FAILING_OUTCOMES,
    JUNIT_ARG_TEMPLATE,
    KEYWORD_ARG,
    OUTCOME_ERROR,
    OUTCOME_FAILED,
    OUTCOME_PASSED,
    OUTCOME_SKIPPED,
    PYTEST_BASE_ARGS,
    PYTEST_MODULE,
    PYTHONPATH_ENTRIES,
    PYTHONPATH_SEPARATOR_KEY,
    RUN_TIMEOUT_SECONDS,
    TEST_FILE_GLOB,
    TESTS_FOLDER,
    TIMESTAMP_FORMAT,

```

```

    VERBOSE_ARG,
)

@dataclass
class TestCaseResult:
    """One pytest test case."""

    module: str
    name: str
    outcome: str
    duration: float
    message: str = ""
    detail: str = ""

    @property
    def is_failure(self) -> bool:
        return self.outcome in FAILING_OUTCOMES

@dataclass
class TestRunResult:
    """The outcome of one pytest invocation."""

    command: str
    exit_code: int
    duration: float
    started_at: str
    cases: list[TestCaseResult] = field(default_factory=list)
    stdout: str = ""
    stderr: str = ""
    timed_out: bool = False
    error: str = ""

    # -- counters -----
    @property
    def total(self) -> int:
        return len(self.cases)

    @property
    def passed(self) -> int:
        return sum(1 for c in self.cases if c.outcome == OUTCOME_PASSED)

    @property
    def failed(self) -> int:
        return sum(1 for c in self.cases if c.outcome == OUTCOME_FAILED)

    @property
    def errors(self) -> int:
        return sum(1 for c in self.cases if c.outcome == OUTCOME_ERROR)

    @property
    def skipped(self) -> int:
        return sum(1 for c in self.cases if c.outcome == OUTCOME_SKIPPED)

    @property
    def failing(self) -> list[TestCaseResult]:
        return [c for c in self.cases if c.is_failure]

    @property
    def is_green(self) -> bool:
        return (
            not self.timed_out
            and not self.error
            and self.failed == 0
            and self.errors == 0
        )

class PytestRunner:
    """Discovers and runs the project's pytest suite."""

    def __init__(self, project_root: Path | None = None) -> None:
        self.project_root = project_root or self._resolve_project_root()

    # =====

```

```

# Discovery
# =====
@property
def tests_dir(self) -> Path:
    return self.project_root / TESTS_FOLDER

def available_targets(self) -> list[str]:
    """Selectable targets: the whole suite, then each test module."""
    if not self.tests_dir.exists():
        return [ALL_TESTS_LABEL]

    modules = sorted(
        path.name
        for path in self.tests_dir.glob(TEST_FILE_GLOB)
        if path.is_file()
    )
    return [ALL_TESTS_LABEL, *modules]

def has_tests(self) -> bool:
    return len(self.available_targets()) > 1

# =====
# Execution
# =====
def run(
    self,
    target: str = ALL_TESTS_LABEL,
    keyword: str = "",
    verbose: bool = False,
) -> TestRunResult:
    started = datetime.now()
    report_path = Path(
        tempfile.gettempdir()
    ) / f"pytest_report_{started.strftime('%Y%m%d%H%M%S%f')}.xml"

    command = self._build_command(
        target=target,
        keyword=keyword,
        verbose=verbose,
        report_path=report_path,
    )

    result = TestRunResult(
        command=" ".join(command),
        exit_code=-1,
        duration=0.0,
        started_at=started.strftime(TIMESTAMP_FORMAT),
    )

    try:
        completed = subprocess.run(
            command,
            cwd=str(self.project_root),
            env=self._environment(),
            capture_output=True,
            text=True,
            timeout=RUN_TIMEOUT_SECONDS,
            check=False,
        )
        result.exit_code = completed.returncode
        result.stdout = completed.stdout or ""
        result.stderr = completed.stderr or ""

    except subprocess.TimeoutExpired:
        result.timed_out = True

    except Exception as exc: # noqa: BLE001
        result.error = str(exc)

    result.duration = (datetime.now() - started).total_seconds()

    if report_path.exists():
        result.cases = self._parse_report(report_path)
        self._discard(report_path)

    return result

```



```

# =====
# Command and environment
# =====
def _build_command(
    self,
    target: str,
    keyword: str,
    verbose: bool,
    report_path: Path,
) -> list[str]:
    command = [sys.executable, "-m", PYTEST_MODULE]

    selected = str(target or "").strip()
    if selected and selected != ALL_TESTS_LABEL:
        command.append(str(Path(TESTS_FOLDER) / selected))
    else:
        command.append(TESTS_FOLDER)

    command.extend(PYTEST_BASE_ARGS)

    if verbose:
        command.append(VERBOSE_ARG)

    clean_keyword = str(keyword or "").strip()
    if clean_keyword:
        command.extend([KEYWORD_ARG, clean_keyword])

    command.append(JUNIT_ARG_TEMPLATE.format(path=report_path))
    return command

def _environment(self) -> dict[str, str]:
    env = dict(os.environ)
    entries = [
        str(self.project_root / entry) if entry != "." else str(self.project_root)
        for entry in PYTHONPATH_ENTRIES
    ]
    existing = env.get(PYTHONPATH_SEPARATOR_KEY, "")
    if existing:
        entries.append(existing)
    env[PYTHONPATH_SEPARATOR_KEY] = os.pathsep.join(entries)
    return env

# =====
# Report parsing
# =====
def _parse_report(self, report_path: Path) -> list[TestCaseResult]:
    try:
        tree = ElementTree.parse(report_path)
    except ElementTree.ParseError:
        return []

    cases: list[TestCaseResult] = []

    for element in tree.iter("testcase"):
        outcome = OUTCOME_PASSED
        message = ""
        detail = ""

        for child in element:
            tag = child.tag.lower()
            if tag == "failure":
                outcome = OUTCOME_FAILED
            elif tag == "error":
                outcome = OUTCOME_ERROR
            elif tag == "skipped":
                outcome = OUTCOME_SKIPPED
            else:
                continue

        message = str(child.get("message", "") or "").strip()
        detail = str(child.text or "").strip()
        break

    cases.append(
        TestCaseResult(

```

```

        module=self._module_of(element.get("classname", "")),
        name=str(element.get("name", "") or ""),
        outcome=outcome,
        duration=self._float_of(element.get("time", "0")),
        message=message,
        detail=detail,
    )
)

return cases

@staticmethod
def _module_of(classname: str) -> str:
    """tests.test_reflow.TestClass -> tests.test_reflow"""
    text = str(classname or "").strip()
    if not text:
        return ""
    parts = text.split(".")
    if len(parts) > 1 and parts[-1][:1].isupper():
        return ".".join(parts[:-1])
    return text

@staticmethod
def _float_of(value: str) -> float:
    try:
        return round(float(value), 3)
    except (TypeError, ValueError):
        return 0.0

@staticmethod
def _discard(path: Path) -> None:
    try:
        path.unlink()
    except OSError:
        pass

# =====
# Helpers
# =====
@staticmethod
def _resolve_project_root() -> Path:
    # ../src/idms_db2_phase2/testing/pytest_runner.py -> project root
    return Path(__file__).resolve().parents[3]

```

```

=====
FILE: src\idms_db2_phase2\testing\retrieval\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\testing\retrieval\retrieval_environment.py
ENCODING: utf-8
=====

```

```
from __future__ import annotations
```

```
import sys
from pathlib import Path
```

```
def bootstrap_sys_path(current_file: Path) -> tuple[Path, Path]:
    """Ensure PROJECT_ROOT and SRC_DIR are importable.
```

```

    Returns (src_dir, project_root).
    """
```

```

    resolved = current_file.resolve()
    src_dir = resolved.parents[2]
    project_root = resolved.parents[3]
```

```

    for path in (project_root, src_dir):
        if str(path) not in sys.path:
            sys.path.insert(0, str(path))

```

```
return src_dir, project_root
```

```
=====
FILE: src\idms_db2_phase2\testing\retrieval\retrieval_input_loader.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from config.path_settings import DEFAULT_MAPPING_SHEET_DIR
from idms_db2_phase2.infrastructure.file_loader import FileLoader
from idms_db2_phase2.infrastructure.local_uploaded_file import LocalUploadedFile
from idms_db2_phase2.parsers.copybook_parser import CopybookParser
from idms_db2_phase2.parsers.dclgen_parser import DclgenParser
from idms_db2_phase2.parsers.sheet_mapping_parser import SheetMappingParser
from rules.retrieval_runner_messages import (
    COPYBOOK_SOURCE_LABEL,
    DCLGEN_SOURCE_LABEL_TEMPLATE,
    DIAG_COPYBOOK_FILE_COUNT_TEMPLATE,
    DIAG_COPYBOOK_FILE_TEMPLATE,
    DIAG_COPYBOOK_TEXT_LEN_TEMPLATE,
    DIAG_COPYBOOK_TOTAL_FIELDS_TEMPLATE,
    DIAG_DCLGEN_FILE_COUNT_TEMPLATE,
    DIAG_DCLGEN_FILE_TEMPLATE,
    DIAG_DCLGEN_TEXT_LEN_TEMPLATE,
    DIAG_DCLGEN_TOTAL_COLUMNS_TEMPLATE,
    DIAG_MAPPING_FOLDER_TEMPLATE,
    DIAG_MAPPING_ROWS_TEMPLATE,
    DIAG_MAPPING_SELECTED_TEMPLATE,
    DIAG_START_LOAD_INPUTS,
    LOG_COPYBOOK_FILE,
    LOG_COPYBOOK_FILE_COUNT,
    LOG_COPYBOOK_TEXT_LEN,
    LOG_COPYBOOK_TOTAL_FIELDS,
    LOG_DCLGEN_FILE,
    LOG_DCLGEN_FILE_COUNT,
    LOG_DCLGEN_TEXT_LEN,
    LOG_DCLGEN_TOTAL_COLUMNS,
    LOG_MAPPING_ROWS,
    LOG_MAPPING_SELECTED,
)

from .retrieval_input_selector import (
    selected_copybook_paths,
    selected_dclgen_paths,
    selected_mapping_sheet_path,
)

def _load_mapping_rows(sheet_parser, diagnostics, logger):
    sheet_path = selected_mapping_sheet_path()
    sheet_rows = sheet_parser.parse_uploaded_file(LocalUploadedFile(sheet_path))

    diagnostics.append(
        DIAG_MAPPING_FOLDER_TEMPLATE.format(folder=DEFAULT_MAPPING_SHEET_DIR)
    )
    diagnostics.append(DIAG_MAPPING_SELECTED_TEMPLATE.format(path=sheet_path))
    diagnostics.append(DIAG_MAPPING_ROWS_TEMPLATE.format(count=len(sheet_rows)))
    logger.info(LOG_MAPPING_SELECTED, sheet_path)
    logger.info(LOG_MAPPING_ROWS, len(sheet_rows))

    if hasattr(sheet_parser, "diagnostics"):
        diagnostics.extend(sheet_parser.diagnostics)

    return sheet_rows

def _load_dclgen_columns(dclgen_parser, file_loader, diagnostics, logger):
    dclgen_files = selected_dclgen_paths()
    dclgen_texts: list[str] = []

    diagnostics.append(
        DIAG_DCLGEN_FILE_COUNT_TEMPLATE.format(count=len(dclgen_files))
    )
```

```

logger.info(LOG_DCLGEN_FILE_COUNT, len(dclgen_files))

for index, path in enumerate(dclgen_files, start=1):
    text = file_loader.read_uploaded_text(LocalUploadedFile(path))
    dclgen_texts.append(text)
    diagnostics.append(DIAG_DCLGEN_FILE_TEMPLATE.format(index=index, path=path))
    diagnostics.append(
        DIAG_DCLGEN_TEXT_LEN_TEMPLATE.format(index=index, length=len(text))
    )
    logger.info(LOG_DCLGEN_FILE, index, path)
    logger.info(LOG_DCLGEN_TEXT_LEN, index, len(text))

if hasattr(dclgen_parser, "parse_many_texts"):
    dclgen_columns = dclgen_parser.parse_many_texts(dclgen_texts)
else:
    dclgen_columns = []
    for index, text in enumerate(dclgen_texts, start=1):
        dclgen_columns.extend(
            dclgen_parser.parse(
                text=text,
                source_label=DCLGEN_SOURCE_LABEL_TEMPLATE.format(index=index),
            )
        )

diagnostics.append(
    DIAG_DCLGEN_TOTAL_COLUMNS_TEMPLATE.format(count=len(dclgen_columns))
)
logger.info(LOG_DCLGEN_TOTAL_COLUMNS, len(dclgen_columns))

if hasattr(dclgen_parser, "diagnostics"):
    diagnostics.extend(dclgen_parser.diagnostics)

return dclgen_columns

def _load_copybook_fields(copybook_parser, file_loader, diagnostics, logger):
    copybook_files = selected_copybook_paths()
    text_parts: list[str] = []

    diagnostics.append(
        DIAG_COPYBOOK_FILE_COUNT_TEMPLATE.format(count=len(copybook_files))
    )
    logger.info(LOG_COPYBOOK_FILE_COUNT, len(copybook_files))

    for index, path in enumerate(copybook_files, start=1):
        text = file_loader.read_uploaded_text(LocalUploadedFile(path))
        text_parts.append(text)
        diagnostics.append(DIAG_COPYBOOK_FILE_TEMPLATE.format(index=index, path=path))
        diagnostics.append(
            DIAG_COPYBOOK_TEXT_LEN_TEMPLATE.format(index=index, length=len(text))
        )
        logger.info(LOG_COPYBOOK_FILE, index, path)
        logger.info(LOG_COPYBOOK_TEXT_LEN, index, len(text))

    copybook_text = "\n".join(text_parts)

    if copybook_text.strip():
        try:
            copybook_fields = copybook_parser.parse(
                text=copybook_text,
                source_label=COPYBOOK_SOURCE_LABEL,
            )
        except TypeError:
            copybook_fields = copybook_parser.parse(copybook_text)
    else:
        copybook_fields = []

    diagnostics.append(
        DIAG_COPYBOOK_TOTAL_FIELDS_TEMPLATE.format(count=len(copybook_fields))
    )
    logger.info(LOG_COPYBOOK_TOTAL_FIELDS, len(copybook_fields))

    if hasattr(copybook_parser, "diagnostics"):
        diagnostics.extend(copybook_parser.diagnostics)

    return copybook_fields

```

```

def load_shared_inputs(logger) -> tuple[list, list, list, list[str]]:
    diagnostics: list[str] = []

    sheet_parser = SheetMappingParser()
    dclgen_parser = DclgenParser()
    copybook_parser = CopybookParser()
    file_loader = FileLoader()

    diagnostics.append(DIAG_START_LOAD_INPUTS)
    logger.info(DIAG_START_LOAD_INPUTS)

    sheet_rows = _load_mapping_rows(sheet_parser, diagnostics, logger)
    dclgen_columns = _load_dclgen_columns(
        dclgen_parser, file_loader, diagnostics, logger
    )
    copybook_fields = _load_copybook_fields(
        copybook_parser, file_loader, diagnostics, logger
    )

    return sheet_rows, dclgen_columns, copybook_fields, diagnostics

```

```

=====
FILE: src\idms_db2_phase2\testing\retrieval\retrieval_input_selector.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from pathlib import Path

from config.path_settings import (
    DEFAULT_COPYBOOK_CANDIDATE_PATHS,
    DEFAULT_DCLGEN_CANDIDATE_PATHS,
    DEFAULT_MAPPING_SHEET_DIR,
    DEFAULT_RETRIEVAL_PROGRAM_DIR,
    DEFAULT_RETRIEVAL_SOURCE_PATH,
    DEFAULT_SHEET_MAPPING_PATH,
    copybook_paths,
    dclgen_paths,
    ensure_output_dirs,
    mapping_sheet_paths,
    retrieval_program_paths,
)
from rules.retrieval_runner_messages import (
    FILE_NOT_FOUND_TEMPLATE,
    FOLDER_NOT_FOUND_TEMPLATE,
    LABEL_MAPPING_SHEET,
    LABEL_RETRIEVAL_PROGRAM,
    LABEL_RETRIEVAL_PROGRAM_TEMPLATE,
    LABEL_SHEET_MAPPING,
    LABEL_COPYBOOK_TEMPLATE,
    LABEL_DCLGEN_TEMPLATE,
    NO_DCLGEN_CANDIDATE_TEMPLATE,
    NO_MAPPING_FILE_TEMPLATE,
    NO_PROGRAM_FILE_TEMPLATE,
    PATH_NOT_FILE_TEMPLATE,
    PATH_NOT_FOLDER_TEMPLATE,
)

def validate_file_exists(file_path: Path, label: str) -> None:
    if not file_path.exists():
        raise FileNotFoundError(
            FILE_NOT_FOUND_TEMPLATE.format(label=label, path=file_path)
        )
    if not file_path.is_file():
        raise FileNotFoundError(
            PATH_NOT_FILE_TEMPLATE.format(label=label, path=file_path)
        )

def validate_folder_exists(folder_path: Path, label: str) -> None:
    if not folder_path.exists():

```

```

        raise FileNotFoundError(
            FOLDER_NOT_FOUND_TEMPLATE.format(label=label, path=folder_path)
        )
    if not folder_path.is_dir():
        raise FileNotFoundError(
            PATH_NOT_FOLDER_TEMPLATE.format(label=label, path=folder_path)
        )

def selected_mapping_sheet_path() -> Path:
    mapping_paths = mapping_sheet_paths()

    if not mapping_paths and DEFAULT_SHEET_MAPPING_PATH.exists():
        return DEFAULT_SHEET_MAPPING_PATH

    if not mapping_paths:
        raise FileNotFoundError(
            NO_MAPPING_FILE_TEMPLATE.format(folder=DEFAULT_MAPPING_SHEET_DIR)
        )

    return mapping_paths[0]

def selected_dclgen_paths() -> list[Path]:
    paths = dclgen_paths()
    if paths:
        return paths
    return [
        path
        for path in DEFAULT_DCLGEN_CANDIDATE_PATHS
        if path.exists() and path.is_file()
    ]

def selected_copybook_paths() -> list[Path]:
    paths = copybook_paths()
    if paths:
        return paths
    return [
        path
        for path in DEFAULT_COPYBOOK_CANDIDATE_PATHS
        if path.exists() and path.is_file()
    ]

def selected_program_paths() -> list[Path]:
    paths = retrieval_program_paths()
    if paths:
        return paths

    if (
        DEFAULT_RETRIEVAL_SOURCE_PATH.exists()
        and DEFAULT_RETRIEVAL_SOURCE_PATH.is_file()
    ):
        return [DEFAULT_RETRIEVAL_SOURCE_PATH]

    return []

def validate_inputs() -> None:
    validate_folder_exists(DEFAULT_MAPPING_SHEET_DIR, LABEL_MAPPING_SHEET)
    validate_folder_exists(DEFAULT_RETRIEVAL_PROGRAM_DIR, LABEL_RETRIEVAL_PROGRAM)

    validate_file_exists(selected_mapping_sheet_path(), LABEL_SHEET_MAPPING)

    dclgen_files = selected_dclgen_paths()
    if not dclgen_files:
        searched = "\n".join(str(path) for path in DEFAULT_DCLGEN_CANDIDATE_PATHS)
        raise ValueError(NO_DCLGEN_CANDIDATE_TEMPLATE.format(searched=searched))

    for index, dclgen_path in enumerate(dclgen_files, start=1):
        validate_file_exists(
            dclgen_path,
            LABEL_DCLGEN_TEMPLATE.format(index=index),
        )

```

```

for index, copybook_path in enumerate(selected_copybook_paths(), start=1):
    validate_file_exists(
        copybook_path,
        LABEL_COPYBOOK_TEMPLATE.format(index=index),
    )

program_files = selected_program_paths()
if not program_files:
    raise FileNotFoundError(
        NO_PROGRAM_FILE_TEMPLATE.format(folder=DEFAULT_RETRIEVAL_PROGRAM_DIR)
    )

for index, program_path in enumerate(program_files, start=1):
    validate_file_exists(
        program_path,
        LABEL_RETRIEVAL_PROGRAM_TEMPLATE.format(index=index),
    )

ensure_output_dirs()

```

```

=====
FILE: src\idms_db2_phase2\testing\retrieval\retrieval_program_converter.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/testing/retrieval/retrieval_program_converter.py
# ACTION: REPLACE ENTIRE FILE

```

```

"""Converts one retrieval program and prints its report.

```

```

CORRECTION - LRF metadata was never supplied
-----

```

```

`ConversionInput` carries a `logical_records` field. This runner never
populated it, so `LrfRepository` was always empty, `LrfPathExpander`
returned the source unchanged, and every program using COPY IDMS LR
shipped with raw LRF syntax:

```

```

OBTAIN FIRST VMBTL03-R01 WHERE SWEEP-VMBFAS.

```

```

No expansion means no `OBTAIN ... WITHIN ...`, which means no cursor,
no OPEN/FETCH/CLOSE paragraphs and no SQLERROR anchor. The failure was
silent because LRF is optional by design.

```

```

The records are now loaded from the configured LRF folder when the
caller does not supply them, so run_retrieval.py needs no change.

```

```

READING
-----

```

```

The subschema is read with utf-8-sig, which transparently strips a UTF-8
byte order mark. A BOM left in the file makes the first token of
`ADD SUBSCHEMA ...` unmatchable and yields zero logical records with no
error at all.
"""

```

```

from __future__ import annotations

```

```

from datetime import datetime
from pathlib import Path

```

```

from config.path_settings import DEFAULT_LRF_DIR, lrf_paths
from idms_db2_phase2.domain.models import ConversionInput
from idms_db2_phase2.infrastructure.file_loader import FileLoader
from idms_db2_phase2.infrastructure.local_uploaded_file import LocalUploadedFile
from idms_db2_phase2.orchestration.conversion_service import ConversionService
from idms_db2_phase2.parsers.cobol_parser import CobolParser
from idms_db2_phase2.parsers.lrf_parser import LrfParser
from idms_db2_phase2.services.name_derivation_resolver import NameDerivationResolver
from patterns.lrf_patterns import COPY_IDMS_LR_PATTERN
from rules.lrf_rules import (
    DIAG_LRF_FILE_COUNT_TEMPLATE,
    DIAG_LRF_FILE_TEMPLATE,
    DIAG_LRF_FOLDER_TEMPLATE,
    DIAG_LRF_NO_RECORDS_IN_SCOPE,
    DIAG_LRF_PARSED_ZERO,
    DIAG_LRF_PARSE_FAILED_TEMPLATE,
)

```

```

    DIAG_LRF_READ_FAILED_TEMPLATE,
    DIAG_LRF_RECORDS_IN_SCOPE_TEMPLATE,
    DIAG_LRF_TEXT_LEN_TEMPLATE,
    DIAG_LRF_TOTAL_RECORDS_TEMPLATE,
    LABEL_LRF,
    LOG_LRF_FILE_COUNT,
    LOG_LRF_TOTAL_RECORDS,
    LRF_VALIDATION_MESSAGES,
)
from rules.retrieval_runner_messages import (
    BULLET_TEMPLATE,
    DIAG_DERIVED_PROGRAM_ID_TEMPLATE,
    DIAG_SOURCE_FILE_TEMPLATE,
    DIAG_SOURCE_LEN_TEMPLATE,
    HEADER_DIAGNOSTICS,
    HEADER_INPUT_SUMMARY,
    HEADER_SELECTED_INPUT_FILES,
    HEADER_VALIDATION_MESSAGES,
    LOG_DERIVED_PROGRAM_ID,
    LOG_GENERATION_COMPLETED,
    LOG_OUTPUT_CREATED,
    LOG_SOURCE_FILE,
    LOG_SOURCE_LEN,
    NO_VALIDATION_MESSAGES,
    PRESERVE_SOURCE_LABEL,
    PRINT_GENERATION_COMPLETED,
    PRINT_INPUT_PROGRAM_TEMPLATE,
    PRINT_OUTPUT_CREATED_TEMPLATE,
    RULE_DIAGNOSTICS,
    RULE_INPUT_SUMMARY,
    RULE_SELECTED_INPUT_FILES,
    RULE_VALIDATION_MESSAGES,
    SELECTED_COBOL_SOURCE_TEMPLATE,
    SELECTED_COPYBOOK_HEADER,
    SELECTED_COPYBOOK_NONE,
    SELECTED_DCLGEN_HEADER,
    SELECTED_PATH_BULLET_TEMPLATE,
    SELECTED_SHEET_MAPPING_TEMPLATE,
    SUMMARY_COBOL_LEN_TEMPLATE,
    SUMMARY_COPYBOOK_FIELDS_TEMPLATE,
    SUMMARY_DCLGEN_COLUMNS_TEMPLATE,
    SUMMARY_PROJECT_ROOT_TEMPLATE,
    SUMMARY_SHEET_ROWS_TEMPLATE,
    SUMMARY_SRC_DIR_TEMPLATE,
    SUMMARY_TARGET_PROGRAM_ID_TEMPLATE,
)
from rules.retrieval_runner_rules import (
    AUTO_FIX_PIC_LENGTH_MISMATCHES_DEFAULT,
    COBOL_TEXT_ENCODING,
    OUTPUT_FILE_EXTENSION,
    OUTPUT_FILE_NAME_TEMPLATE,
    OUTPUT_TIMESTAMP_FORMAT,
)
from .retrieval_input_selector import (
    selected_copybook_paths,
    selected_dclgen_paths,
    selected_mapping_sheet_path,
)

# Reading with utf-8-sig strips a leading byte order mark transparently.
LRF_TEXT_ENCODING = "utf-8-sig"
LRF_SOURCE_LABEL = "LRF files"

def resolve_target_program_id(source_text: str) -> str:
    """Derive the target PROGRAM-ID from the source (VM...BD... -> VMDZ...)."""
    source_id = CobolParser().program_id(source_text)
    if not source_id:
        return ""
    return NameDerivationResolver().derive(source_id)

def _build_output_path(output_dir: Path, program_path: Path) -> Path:
    timestamp = datetime.now().strftime(OUTPUT_TIMESTAMP_FORMAT)
    file_name = OUTPUT_FILE_NAME_TEMPLATE.format(

```



```

        stem=program_path.stem,
        timestamp=timestamp,
        extension=OUTPUT_FILE_EXTENSION,
    )
    return output_dir / file_name

#
# LRF (Logical Record Facility)
#
def _read_lrf_text(path: Path, file_loader: FileLoader) -> str:
    """Read one subschema, BOM tolerant.

    FileLoader is tried first so behaviour matches the Streamlit upload
    path. A direct utf-8-sig read is the fallback, because a BOM left in
    the file silently produces zero logical records.
    """
    try:
        text = file_loader.read_uploaded_text(LocalUploadedFile(path))
        if str(text or "").strip():
            return str(text).lstrip("\ufffd")
    except Exception: # noqa: BLE001
        pass

    return Path(path).read_text(
        encoding=LRF_TEXT_ENCODING,
        errors="replace",
    )

def _parse_lrf_text(parser: LrfParser, text: str) -> list:
    """Parse subschema text, tolerating either parse() signature."""
    try:
        return list(parser.parse(text, source_label=LRF_SOURCE_LABEL) or [])
    except TypeError:
        return list(parser.parse(text) or [])

def load_logical_records(diagnostics: list[str], logger) -> list:
    """Every logical record declared in the configured LRF folder.

    Never raises and never blocks a run: LRF is optional metadata. A
    folder that is missing, empty or unreadable yields an empty list and
    a diagnostic, exactly as the batch loader does.
    """
    diagnostics.append(DIAG_LRF_FOLDER_TEMPLATE.format(folder=DEFAULT_LRF_DIR))

    try:
        files = [Path(path) for path in (lrf_paths() or [])]
    except Exception as exc: # noqa: BLE001
        diagnostics.append(
            DIAG_LRF_READ_FAILED_TEMPLATE.format(
                name=DEFAULT_LRF_DIR,
                reason=exc,
            )
        )
    files = []

    diagnostics.append(DIAG_LRF_FILE_COUNT_TEMPLATE.format(count=len(files)))
    logger.info(LOG_LRF_FILE_COUNT, len(files))

    if not files:
        diagnostics.append(DIAG_LRF_PARSED_ZERO)
        return []

    file_loader = FileLoader()
    parser = LrfParser()
    text_parts: list[str] = []

    for index, path in enumerate(files, start=1):
        diagnostics.append(
            DIAG_LRF_FILE_TEMPLATE.format(index=index, path=path)
        )

        try:
            text = _read_lrf_text(path, file_loader)

```

```

except Exception as exc: # noqa: BLE001
    diagnostics.append(
        DIAG_LRF_READ_FAILED_TEMPLATE.format(name=path, reason=exc)
    )
    continue

diagnostics.append(
    DIAG_LRF_TEXT_LEN_TEMPLATE.format(index=index, length=len(text))
)
text_parts.append(text)

combined = "\n".join(text_parts)

if not combined.strip():
    diagnostics.append(DIAG_LRF_PARSED_ZERO)
    return []

try:
    records = _parse_lrf_text(parser, combined)
except Exception as exc: # noqa: BLE001
    diagnostics.append(
        DIAG_LRF_PARSE_FAILED_TEMPLATE.format(
            name=LRF_SOURCE_LABEL,
            reason=exc,
        )
    )
    return []

diagnostics.append(
    DIAG_LRF_TOTAL_RECORDS_TEMPLATE.format(count=len(records))
)
logger.info(LOG_LRF_TOTAL_RECORDS, len(records))

parser_diagnostics = getattr(parser, "diagnostics", None)
if parser_diagnostics:
    diagnostics.extend(str(item) for item in parser_diagnostics)

return records

def _source_uses_logical_records(source_text: str) -> list[str]:
    """Logical record names the SOURCE copies via COPY IDMS LR."""
    names: list[str] = []

    for raw_line in str(source_text or "").splitlines():
        match = COPY_IDMS_LR_PATTERN.search(raw_line)

        if not match:
            continue

        name = str(match.group("lr") or "").strip().upper()

        if name and name not in names:
            names.append(name)

    return names

def _narrate_lrf(
    *,
    source_text: str,
    logical_records: list,
    diagnostics: list[str],
    logger,
) -> None:
    """Record what LRF metadata this program run will actually see.

    A program that copies a logical record without a matching subschema
    is reported once, so the gap is visible in the run log instead of
    failing silently.
    """
    if logical_records:
        names = ", ".join(
            str(getattr(record, "logical_record_name", "") or "")
            for record in logical_records
        )

```

```

        diagnostics.append(
            DIAG_LRF_RECORDS_IN_SCOPE_TEMPLATE.format(
                count=len(logical_records),
                names=names,
            )
        )
    else:
        diagnostics.append(DIAG_LRF_NO_RECORDS_IN_SCOPE)

used = _source_uses_logical_records(source_text)

if used and not logical_records:
    message = LRF_VALIDATION_MESSAGES["missing_lrf"]
    diagnostics.append(message)
    logger.warning(message)

#
# Reporting
#
def _print_summary(
    *,
    program_path,
    sheet_rows,
    dclgen_columns,
    copybook_fields,
    logical_records,
    idms_len,
    target_program_id,
    project_root,
    src_dir,
):
    print("")
    print(HEADER_INPUT_SUMMARY)
    print(RULE_INPUT_SUMMARY)
    print(SUMMARY_PROJECT_ROOT_TEMPLATE.format(value=project_root))
    print(SUMMARY_SRC_DIR_TEMPLATE.format(value=src_dir))
    print(SUMMARY_SHEET_ROWS_TEMPLATE.format(value=len(sheet_rows)))
    print(SUMMARY_DCLGEN_COLUMNS_TEMPLATE.format(value=len(dclgen_columns)))
    print(SUMMARY_COPYBOOK_FIELDS_TEMPLATE.format(value=len(copybook_fields)))
    print(f"{LABEL_LRF} Records : {len(logical_records)}")
    print(SUMMARY_COBOL_LEN_TEMPLATE.format(value=idms_len))
    print(
        SUMMARY_TARGET_PROGRAM_ID_TEMPLATE.format(
            value=target_program_id or PRESERVE_SOURCE_LABEL
        )
    )

    print("")
    print(HEADER_SELECTED_INPUT_FILES)
    print(RULE_SELECTED_INPUT_FILES)
    print(SELECTED_SHEET_MAPPING_TEMPLATE.format(value=selected_mapping_sheet_path()))
    print(SELECTED_COBOL_SOURCE_TEMPLATE.format(value=program_path))

    print(SELECTED_DCLGEN_HEADER)
    for path in selected_dclgen_paths():
        print(SELECTED_PATH_BULLET_TEMPLATE.format(value=path))

    copybooks = selected_copybook_paths()
    if copybooks:
        print(SELECTED_COPYBOOK_HEADER)
        for path in copybooks:
            print(SELECTED_PATH_BULLET_TEMPLATE.format(value=path))
    else:
        print(SELECTED_COPYBOOK_NONE)

    try:
        lrf_files = list(lrf_paths() or [])
    except Exception: # noqa: BLE001
        lrf_files = []

    print(f"{LABEL_LRF} Files:")
    if lrf_files:
        for path in lrf_files:
            print(SELECTED_PATH_BULLET_TEMPLATE.format(value=path))
    else:

```

```

        print(SELECTED_PATH_BULLET_TEMPLATE.format(value="None"))

def _print_results(result, diagnostics, logger):
    print("")
    print(HEADER_VALIDATION_MESSAGES)
    print(RULE_VALIDATION_MESSAGES)

    if result.validation_messages:
        for message in result.validation_messages:
            print(BULLET_TEMPLATE.format(text=message))
            logger.warning(message)
    else:
        print(NO_VALIDATION_MESSAGES)

    print("")
    print(HEADER_DIAGNOSTICS)
    print(RULE_DIAGNOSTICS)

    for diagnostic in diagnostics:
        print(BULLET_TEMPLATE.format(text=diagnostic))
        logger.info(diagnostic)

#
# Public entry point
#
def convert_one_program(
    *,
    program_path: Path,
    sheet_rows: list,
    dclgen_columns: list,
    copybook_fields: list,
    shared_diagnostics: list[str],
    output_dir: Path,
    project_root,
    src_dir,
    logger,
    logical_records: list | None = None,
) -> Path:
    """Convert one retrieval program.

    `logical_records` is OPTIONAL. When the caller does not supply it the
    configured LRF folder is read here, so run_retrieval.py works with no
    change while a batch caller can still load the subschema once and
    share it across every program.
    """

    file_loader = FileLoader()
    idms_cobol_text = file_loader.read_uploaded_text(LocalUploadedFile(program_path))

    target_program_id = resolve_target_program_id(idms_cobol_text)
    display_id = target_program_id or PRESERVE_SOURCE_LABEL

    diagnostics = list(shared_diagnostics)
    diagnostics.append(DIAG_SOURCE_FILE_TEMPLATE.format(path=program_path))
    diagnostics.append(DIAG_SOURCE_LEN_TEMPLATE.format(length=len(idms_cobol_text)))
    diagnostics.append(DIAG_DERIVED_PROGRAM_ID_TEMPLATE.format(value=display_id))

    logger.info(LOG_SOURCE_FILE, program_path)
    logger.info(LOG_SOURCE_LEN, len(idms_cobol_text))
    logger.info(LOG_DERIVED_PROGRAM_ID, display_id)

    # LRF sits between Copybook metadata and the COBOL source.
    records = (
        list(logical_records)
        if logical_records is not None
        else load_logical_records(diagnostics, logger)
    )

    _narrate_lrf(
        source_text=idms_cobol_text,
        logical_records=records,
        diagnostics=diagnostics,
        logger=logger,
    )

```

```

result = ConversionService().convert(
    ConversionInput(
        sheet_mapping_rows=sheet_rows,
        dclgen_columns=dclgen_columns,
        copybook_fields=copybook_fields,
        logical_records=records,
        idms_cobol_text=idms_cobol_text,
        target_program_id=target_program_id,
        auto_fix_pic_length_mismatches=AUTO_FIX_PIC_LENGTH_MISMATCHES_DEFAULT,
    )
)

output_cobol_path = _build_output_path(output_dir, program_path)
output_cobol_path.write_text(
    result.converted_cobol or "",
    encoding=COBOL_TEXT_ENCODING,
)

logger.info(LOG_GENERATION_COMPLETED)
logger.info(LOG_OUTPUT_CREATED, output_cobol_path)

print("")
print(PRINT_GENERATION_COMPLETED)
print(PRINT_INPUT_PROGRAM_TEMPLATE.format(path=program_path))
print(PRINT_OUTPUT_CREATED_TEMPLATE.format(path=output_cobol_path))

_print_summary(
    program_path=program_path,
    sheet_rows=sheet_rows,
    dclgen_columns=dclgen_columns,
    copybook_fields=copybook_fields,
    logical_records=records,
    idms_len=len(idms_cobol_text),
    target_program_id=target_program_id,
    project_root=project_root,
    src_dir=src_dir,
)
_print_results(result, diagnostics, logger)

return output_cobol_path

```

```

__all__ = [
    "convert_one_program",
    "load_logical_records",
    "resolve_target_program_id",
]

```

```

=====
FILE: src\idms_db2_phase2\testing\run_retrieval.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from pathlib import Path

from idms_db2_phase2.testing.retrieval.retrieval_environment import (
    bootstrap_sys_path,
)

SRC_DIR, PROJECT_ROOT = bootstrap_sys_path(Path(__file__))

from config.path_settings import ( # noqa: E402
    DEFAULT_RETRIEVAL_OUTPUT_DIR,
    DEFAULT_RETRIEVAL_PROGRAM_DIR,
    LOGS_DIR,
)
from idms_db2_phase2.infrastructure.logger_factory import ( # noqa: E402
    LoggerFactory,
)
from idms_db2_phase2.testing.retrieval.retrieval_input_loader import ( # noqa: E402
    load_shared_inputs,
)
from idms_db2_phase2.testing.retrieval.retrieval_input_selector import ( # noqa: E402

```

```

        selected_program_paths,
        validate_inputs,
    )
from idms_db2_phase2.testing.retrieval.retrieval_program_converter import ( # noqa: E402
    convert_one_program,
    resolve_target_program_id,
)
from rules.retrieval_runner_messages import ( # noqa: E402
    BATCH_OUTPUT_FOLDER_TEMPLATE,
    BATCH_PROGRAM_COUNT_TEMPLATE,
    BATCH_PROGRAM_FOLDER_TEMPLATE,
    HEADER_RETRIEVAL_BATCH_SUMMARY,
    RULE_RETRIEVAL_BATCH_SUMMARY,
)
from rules.retrieval_runner_rules import RETRIEVAL_LOGGER_NAME # noqa: E402

OUTPUT_DIR = DEFAULT_RETRIEVAL_OUTPUT_DIR

__all__ = [
    "resolve_target_program_id",
    "validate_inputs",
    "load_shared_inputs",
    "convert_one_program",
    "run_conversion",
]

def run_conversion() -> None:
    validate_inputs()

    logger = LoggerFactory.create_logger(
        name=RETRIEVAL_LOGGER_NAME,
        logs_dir=LOGS_DIR,
    )

    sheet_rows, dclgen_columns, copybook_fields, diagnostics = load_shared_inputs(
        logger
    )

    program_files = selected_program_paths()

    print("")
    print(HEADER_RETRIEVAL_BATCH_SUMMARY)
    print(RULE_RETRIEVAL_BATCH_SUMMARY)
    print(BATCH_PROGRAM_FOLDER_TEMPLATE.format(value=DEFAULT_RETRIEVAL_PROGRAM_DIR))
    print(BATCH_PROGRAM_COUNT_TEMPLATE.format(value=len(program_files)))
    print(BATCH_OUTPUT_FOLDER_TEMPLATE.format(value=OUTPUT_DIR))

    for program_path in program_files:
        convert_one_program(
            program_path=program_path,
            sheet_rows=sheet_rows,
            dclgen_columns=dclgen_columns,
            copybook_fields=copybook_fields,
            shared_diagnostics=diagnostics,
            output_dir=OUTPUT_DIR,
            project_root=PROJECT_ROOT,
            src_dir=SRC_DIR,
            logger=logger,
        )

if __name__ == "__main__":
    run_conversion()

```

```

=====
FILE: src\idms_db2_phase2\testing\run_update.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from pathlib import Path

from idms_db2_phase2.testing.update.update_environment import bootstrap_sys_path

```

```

SRC_DIR, PROJECT_ROOT = bootstrap_sys_path(Path(__file__))

from config.path_settings import ( # noqa: E402
    DEFAULT_UPDATE_OUTPUT_DIR,
    DEFAULT_UPDATE_PROGRAM_DIR,
    LOGS_DIR,
)
from idms_db2_phase2.infrastructure.logger_factory import LoggerFactory # noqa: E402
from idms_db2_phase2.testing.update.update_input_loader import ( # noqa: E402
    load_shared_inputs,
)
from idms_db2_phase2.testing.update.update_input_selector import ( # noqa: E402
    selected_program_paths,
    validate_inputs,
)
from idms_db2_phase2.testing.update.update_postprocess_pipeline import ( # noqa: E402
    enhance_update_program,
)
from idms_db2_phase2.testing.update.update_program_converter import ( # noqa: E402
    convert_one_program,
    resolve_target_program_id,
)
from rules.update_runner_messages import ( # noqa: E402
    BATCH_OUTPUT_FOLDER_TEMPLATE,
    BATCH_PROGRAM_COUNT_TEMPLATE,
    BATCH_PROGRAM_FOLDER_TEMPLATE,
    HEADER_UPDATE_BATCH_SUMMARY,
    RULE_UPDATE_BATCH_SUMMARY,
)
from rules.update_runner_rules import UPDATE_LOGGER_NAME # noqa: E402

OUTPUT_DIR = DEFAULT_UPDATE_OUTPUT_DIR

__all__ = [
    "resolve_target_program_id",
    "validate_inputs",
    "load_shared_inputs",
    "convert_one_program",
    "enhance_update_program",
    "run_conversion",
]

def run_conversion() -> None:
    validate_inputs(OUTPUT_DIR)

    logger = LoggerFactory.create_logger(
        name=UPDATE_LOGGER_NAME,
        logs_dir=LOGS_DIR,
    )

    sheet_rows, dclgen_columns, copybook_fields, diagnostics = load_shared_inputs(
        logger
    )

    program_files = selected_program_paths()

    print("")
    print(HEADER_UPDATE_BATCH_SUMMARY)
    print(RULE_UPDATE_BATCH_SUMMARY)
    print(BATCH_PROGRAM_FOLDER_TEMPLATE.format(value=DEFAULT_UPDATE_PROGRAM_DIR))
    print(BATCH_PROGRAM_COUNT_TEMPLATE.format(value=len(program_files)))
    print(BATCH_OUTPUT_FOLDER_TEMPLATE.format(value=OUTPUT_DIR))

    for program_path in program_files:
        convert_one_program(
            program_path=program_path,
            sheet_rows=sheet_rows,
            dclgen_columns=dclgen_columns,
            copybook_fields=copybook_fields,
            shared_diagnostics=diagnostics,
            output_dir=OUTPUT_DIR,
            project_root=PROJECT_ROOT,
            src_dir=SRC_DIR,
            logger=logger,

```

```

)

if __name__ == "__main__":
    run_conversion()

=====
FILE: src\idms_db2_phase2\testing\update\__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_db2_phase2\testing\update\update_environment.py
ENCODING: utf-8
=====

from __future__ import annotations

import sys
from pathlib import Path

def bootstrap_sys_path(current_file: Path) -> tuple[Path, Path]:
    """Ensure PROJECT_ROOT and SRC_DIR are importable.

    Returns (src_dir, project_root).
    """
    resolved = current_file.resolve()
    src_dir = resolved.parents[2]
    project_root = resolved.parents[3]

    for path in (project_root, src_dir):
        if str(path) not in sys.path:
            sys.path.insert(0, str(path))

    return src_dir, project_root

=====
FILE: src\idms_db2_phase2\testing\update\update_input_loader.py
ENCODING: utf-8
=====

from __future__ import annotations

from config.path_settings import DEFAULT_DCLGEN_DIR, DEFAULT_MAPPING_SHEET_DIR
from idms_db2_phase2.infrastructure.file_loader import FileLoader
from idms_db2_phase2.infrastructure.local_uploaded_file import LocalUploadedFile
from idms_db2_phase2.parsers.copybook_parser import CopybookParser
from idms_db2_phase2.parsers.dclgen_parser import DclgenParser
from idms_db2_phase2.parsers.sheet_mapping_parser import SheetMappingParser
from rules.update_runner_messages import (
    COPYBOOK_SOURCE_LABEL,
    DCLGEN_SOURCE_LABEL_TEMPLATE,
    DIAG_COPYBOOK_FILE_COUNT_TEMPLATE,
    DIAG_COPYBOOK_FILE_TEMPLATE,
    DIAG_COPYBOOK_TEXT_LEN_TEMPLATE,
    DIAG_COPYBOOK_TOTAL_FIELDS_TEMPLATE,
    DIAG_DCLGEN_FILE_COUNT_TEMPLATE,
    DIAG_DCLGEN_FILE_TEMPLATE,
    DIAG_DCLGEN_FOLDER_TEMPLATE,
    DIAG_DCLGEN_TEXT_LEN_TEMPLATE,
    DIAG_DCLGEN_TOTAL_COLUMNS_TEMPLATE,
    DIAG_MAPPING_FOLDER_TEMPLATE,
    DIAG_MAPPING_ROWS_TEMPLATE,
    DIAG_MAPPING_SELECTED_TEMPLATE,
    DIAG_START_LOAD_INPUTS,
    LOG_COPYBOOK_FILE,
    LOG_COPYBOOK_FILE_COUNT,
    LOG_COPYBOOK_TEXT_LEN,
    LOG_COPYBOOK_TOTAL_FIELDS,
    LOG_DCLGEN_FILE,

```



```

LOG_DCLGEN_FILE_COUNT,
LOG_DCLGEN_TEXT_LEN,
LOG_DCLGEN_TOTAL_COLUMNS,
LOG_MAPPING_ROWS,
LOG_MAPPING_SELECTED,
)

from .update_input_selector import (
    selected_copybook_paths,
    selected_dclgen_paths,
    selected_mapping_sheet_path,
)

def _load_mapping_rows(sheet_parser, diagnostics, logger):
    sheet_path = selected_mapping_sheet_path()
    sheet_rows = sheet_parser.parse_uploaded_file(LocalUploadedFile(sheet_path))

    diagnostics.append(
        DIAG_MAPPING_FOLDER_TEMPLATE.format(folder=DEFAULT_MAPPING_SHEET_DIR)
    )
    diagnostics.append(DIAG_MAPPING_SELECTED_TEMPLATE.format(path=sheet_path))
    diagnostics.append(DIAG_MAPPING_ROWS_TEMPLATE.format(count=len(sheet_rows)))
    logger.info(LOG_MAPPING_SELECTED, sheet_path)
    logger.info(LOG_MAPPING_ROWS, len(sheet_rows))

    if hasattr(sheet_parser, "diagnostics"):
        diagnostics.extend(sheet_parser.diagnostics)

    return sheet_rows

def _load_dclgen_columns(dclgen_parser, file_loader, diagnostics, logger):
    dclgen_files = selected_dclgen_paths()
    dclgen_texts: list[str] = []

    diagnostics.append(DIAG_DCLGEN_FOLDER_TEMPLATE.format(folder=DEFAULT_DCLGEN_DIR))
    diagnostics.append(
        DIAG_DCLGEN_FILE_COUNT_TEMPLATE.format(count=len(dclgen_files))
    )
    logger.info(LOG_DCLGEN_FILE_COUNT, len(dclgen_files))

    for index, path in enumerate(dclgen_files, start=1):
        text = file_loader.read_uploaded_text(LocalUploadedFile(path))
        dclgen_texts.append(text)
        diagnostics.append(DIAG_DCLGEN_FILE_TEMPLATE.format(index=index, path=path))
        diagnostics.append(
            DIAG_DCLGEN_TEXT_LEN_TEMPLATE.format(index=index, length=len(text))
        )
        logger.info(LOG_DCLGEN_FILE, index, path)
        logger.info(LOG_DCLGEN_TEXT_LEN, index, len(text))

    if hasattr(dclgen_parser, "parse_many_texts"):
        dclgen_columns = dclgen_parser.parse_many_texts(dclgen_texts)
    else:
        dclgen_columns = []
        for index, text in enumerate(dclgen_texts, start=1):
            dclgen_columns.extend(
                dclgen_parser.parse(
                    text=text,
                    source_label=DCLGEN_SOURCE_LABEL_TEMPLATE.format(index=index),
                )
            )

    diagnostics.append(
        DIAG_DCLGEN_TOTAL_COLUMNS_TEMPLATE.format(count=len(dclgen_columns))
    )
    logger.info(LOG_DCLGEN_TOTAL_COLUMNS, len(dclgen_columns))

    if hasattr(dclgen_parser, "diagnostics"):
        diagnostics.extend(dclgen_parser.diagnostics)

    return dclgen_columns

def _load_copybook_fields(copybook_parser, file_loader, diagnostics, logger):

```

```

copybook_files = selected_copybook_paths()
text_parts: list[str] = []

diagnostics.append(
    DIAG_COPYBOOK_FILE_COUNT_TEMPLATE.format(count=len(copybook_files))
)
logger.info(LOG_COPYBOOK_FILE_COUNT, len(copybook_files))

for index, path in enumerate(copybook_files, start=1):
    text = file_loader.read_uploaded_text(LocalUploadedFile(path))
    text_parts.append(text)
    diagnostics.append(DIAG_COPYBOOK_FILE_TEMPLATE.format(index=index, path=path))
    diagnostics.append(
        DIAG_COPYBOOK_TEXT_LEN_TEMPLATE.format(index=index, length=len(text))
    )
    logger.info(LOG_COPYBOOK_FILE, index, path)
    logger.info(LOG_COPYBOOK_TEXT_LEN, index, len(text))

copybook_text = "\n".join(text_parts)

if copybook_text.strip():
    try:
        copybook_fields = copybook_parser.parse(
            text=copybook_text,
            source_label=COPYBOOK_SOURCE_LABEL,
        )
    except TypeError:
        copybook_fields = copybook_parser.parse(copybook_text)
else:
    copybook_fields = []

diagnostics.append(
    DIAG_COPYBOOK_TOTAL_FIELDS_TEMPLATE.format(count=len(copybook_fields))
)
logger.info(LOG_COPYBOOK_TOTAL_FIELDS, len(copybook_fields))

if hasattr(copybook_parser, "diagnostics"):
    diagnostics.extend(copybook_parser.diagnostics)

return copybook_fields

def load_shared_inputs(logger) -> tuple[list, list, list, list[str]]:
    diagnostics: list[str] = []

    sheet_parser = SheetMappingParser()
    dclgen_parser = DclgenParser()
    copybook_parser = CopybookParser()
    file_loader = FileLoader()

    diagnostics.append(DIAG_START_LOAD_INPUTS)
    logger.info(DIAG_START_LOAD_INPUTS)

    sheet_rows = _load_mapping_rows(sheet_parser, diagnostics, logger)
    dclgen_columns = _load_dclgen_columns(
        dclgen_parser, file_loader, diagnostics, logger
    )
    copybook_fields = _load_copybook_fields(
        copybook_parser, file_loader, diagnostics, logger
    )

    return sheet_rows, dclgen_columns, copybook_fields, diagnostics

=====
FILE: src\idms_db2_phase2\testing\update\update_input_selector.py
ENCODING: utf-8
=====

from __future__ import annotations

from pathlib import Path

from config.path_settings import (
    DEFAULT_COPYBOOK_CANDIDATE_PATHS,
    DEFAULT_DCLGEN_CANDIDATE_PATHS,

```

```

    DEFAULT_DCLGEN_DIR,
    DEFAULT_MAPPING_SHEET_DIR,
    DEFAULT_SHEET_MAPPING_PATH,
    DEFAULT_UPDATE_PROGRAM_DIR,
    DEFAULT_UPDATE_SOURCE_PATH,
    copybook_paths,
    dclgen_paths,
    ensure_output_dirs,
    mapping_sheet_paths,
    update_program_paths,
)
from rules.update_runner_messages import (
    FILE_NOT_FOUND_TEMPLATE,
    FOLDER_NOT_FOUND_TEMPLATE,
    LABEL_COPYBOOK_TEMPLATE,
    LABEL_DCLGEN_FOLDER,
    LABEL_DCLGEN_TEMPLATE,
    LABEL_MAPPING_SHEET,
    LABEL_SHEET_MAPPING,
    LABEL_UPDATE_PROGRAM,
    LABEL_UPDATE_PROGRAM_TEMPLATE,
    MULTIPLE_MAPPING_TEMPLATE,
    NO_DCLGEN_CANDIDATE_TEMPLATE,
    NO_MAPPING_FILE_TEMPLATE,
    NO_PROGRAM_FILE_TEMPLATE,
    PATH_NOT_FILE_TEMPLATE,
    PATH_NOT_FOLDER_TEMPLATE,
)

def validate_file_exists(file_path: Path, label: str) -> None:
    if not file_path.exists():
        raise FileNotFoundError(
            FILE_NOT_FOUND_TEMPLATE.format(label=label, path=file_path)
        )
    if not file_path.is_file():
        raise FileNotFoundError(
            PATH_NOT_FILE_TEMPLATE.format(label=label, path=file_path)
        )

def validate_folder_exists(folder_path: Path, label: str) -> None:
    if not folder_path.exists():
        raise FileNotFoundError(
            FOLDER_NOT_FOUND_TEMPLATE.format(label=label, path=folder_path)
        )
    if not folder_path.is_dir():
        raise FileNotFoundError(
            PATH_NOT_FOLDER_TEMPLATE.format(label=label, path=folder_path)
        )

def selected_mapping_sheet_path() -> Path:
    mapping_paths = mapping_sheet_paths()

    if not mapping_paths and DEFAULT_SHEET_MAPPING_PATH.exists():
        return DEFAULT_SHEET_MAPPING_PATH

    if not mapping_paths:
        raise FileNotFoundError(
            NO_MAPPING_FILE_TEMPLATE.format(folder=DEFAULT_MAPPING_SHEET_DIR)
        )

    if len(mapping_paths) > 1:
        names = ", ".join(path.name for path in mapping_paths)
        raise ValueError(
            MULTIPLE_MAPPING_TEMPLATE.format(
                folder=DEFAULT_MAPPING_SHEET_DIR,
                names=names,
            )
        )

    return mapping_paths[0]

def selected_dclgen_paths() -> list[Path]:

```

```

paths = dclgen_paths()
if paths:
    return paths
return [
    path
    for path in DEFAULT_DCLGEN_CANDIDATE_PATHS
    if path.exists() and path.is_file()
]

def selected_copybook_paths() -> list[Path]:
    paths = copybook_paths()
    if paths:
        return paths
    return [
        path
        for path in DEFAULT_COPYBOOK_CANDIDATE_PATHS
        if path.exists() and path.is_file()
    ]

def selected_program_paths() -> list[Path]:
    paths = update_program_paths()
    if paths:
        return paths

    if DEFAULT_UPDATE_SOURCE_PATH.exists() and DEFAULT_UPDATE_SOURCE_PATH.is_file():
        return [DEFAULT_UPDATE_SOURCE_PATH]

    return []

def validate_inputs(output_dir: Path) -> None:
    validate_folder_exists(DEFAULT_MAPPING_SHEET_DIR, LABEL_MAPPING_SHEET)
    validate_folder_exists(DEFAULT_DCLGEN_DIR, LABEL_DCLGEN_FOLDER)
    validate_folder_exists(DEFAULT_UPDATE_PROGRAM_DIR, LABEL_UPDATE_PROGRAM)

    validate_file_exists(selected_mapping_sheet_path(), LABEL_SHEET_MAPPING)

    dclgen_files = selected_dclgen_paths()
    if not dclgen_files:
        searched = "\n".join(str(path) for path in DEFAULT_DCLGEN_CANDIDATE_PATHS)
        raise ValueError(NO_DCLGEN_CANDIDATE_TEMPLATE.format(searched=searched))

    for index, dclgen_path in enumerate(dclgen_files, start=1):
        validate_file_exists(dclgen_path, LABEL_DCLGEN_TEMPLATE.format(index=index))

    for index, copybook_path in enumerate(selected_copybook_paths(), start=1):
        validate_file_exists(
            copybook_path, LABEL_COPYBOOK_TEMPLATE.format(index=index)
        )

    program_files = selected_program_paths()
    if not program_files:
        raise FileNotFoundError(
            NO_PROGRAM_FILE_TEMPLATE.format(folder=DEFAULT_UPDATE_PROGRAM_DIR)
        )

    for index, program_path in enumerate(program_files, start=1):
        validate_file_exists(
            program_path, LABEL_UPDATE_PROGRAM_TEMPLATE.format(index=index)
        )

    ensure_output_dirs()
    output_dir.mkdir(parents=True, exist_ok=True)

```

```

=====
FILE: src\idms_db2_phase2\testing\update\update_postprocess_pipeline.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.composers.fixed_format_composer import FixedFormatComposer
from idms_db2_phase2.composers.procedure_indent_normalizer import (

```

```

    ProcedureIndentNormalizer,
)
from idms_db2_phase2.postprocess.update_char_literal_quoter import (
    UpdateCharLiteralQuoter,
)
from idms_db2_phase2.postprocess.update_date_host_converter import (
    UpdateDateHostConverter,
)
from idms_db2_phase2.postprocess.update_final_cleanup import UpdateFinalCleanup
from idms_db2_phase2.postprocess.update_input_record_renamer import (
    UpdateInputRecordRenamer,
)
from idms_db2_phase2.postprocess.update_program_enhancer import UpdateProgramEnhancer
from idms_db2_phase2.services.fixed_format_line_service import FixedFormatLineService
from idms_db2_phase2.services.name_derivation_resolver import NameDerivationResolver
from idms_db2_phase2.services.update_audit_move_inserter import (
    UpdateAuditMoveInserter,
)
from idms_db2_phase2.services.update_cobol_final_cleanup_utils import (
    UpdateCobolFinalCleanupUtils,
)
from patterns.batch_runner_patterns import COPY_NAME_PATTERN
from rules.update_runner_messages import (
    DIAG_UPDATE_AUDIT_MOVES,
    DIAG_UPDATE_ENHANCEMENT_DONE,
    DIAG_UPDATE_SKIPPED_TEMPLATE,
    LOG_UPDATE_ENHANCEMENT_DONE,
)

```

```

def derive_input_record_rename(converted_cobol: str) -> tuple[str, str]:
    """Recover (old, new) input-record names from COPY statements.

```

```

    Applies the VM...BD...->VMDZ derivation. Returns ("", "") when no
    derivable COPY name is found. No name is hardcoded.
    """

```

```

    deriver = NameDerivationResolver()

```

```

    copy_names = COPY_NAME_PATTERN.findall(str(converted_cobol or ""))

```

```

    for raw in copy_names:
        name = str(raw or "").strip().upper()
        if not name:
            continue
        if deriver.is_derivable(name):
            derived = deriver.derive(name)
            if derived != name:
                return name, derived

```

```

    return "", ""

```

```

def enhance_update_program(

```

```

    *,
    source_cobol: str,
    converted_cobol: str,
    copybook_fields: list,
    dclgen_columns: list,
    target_program_id: str,
    diagnostics: list[str],
    logger,
) -> str:
    try:
        enhancer = UpdateProgramEnhancer()
        enhanced_result = enhancer.enhance(
            source_cobol=source_cobol,
            converted_cobol=converted_cobol,
            copybook_fields=copybook_fields,
            dclgen_columns=dclgen_columns,
            target_program_id=target_program_id,
        )

```

```

        diagnostics.extend(enhanced_result.diagnostics)
        diagnostics.append(DIAG_UPDATE_ENHANCEMENT_DONE)
        logger.info(LOG_UPDATE_ENHANCEMENT_DONE)

```

```

fixed = FixedFormatComposer().format(enhanced_result.converted_cobol)
cleaned = UpdateFinalCleanup().apply(fixed)
formatted = FixedFormatComposer().format(cleaned)

```

```

_ff = FixedFormatLineService()
_audit = UpdateAuditMoveInserter(
    fixed_format=_ff,
    utils=UpdateCobolFinalCleanupUtils(_ff),
)
with_audit = _audit.insert_update_audit_moves(formatted)

if with_audit != formatted:
    diagnostics.append(DIAG_UPDATE_AUDIT_MOVES)

with_char = UpdateCharLiteralQuoter(dclgen_columns).apply(
    with_audit,
    diagnostics,
)
with_dates = UpdateDateHostConverter(dclgen_columns).apply(
    with_char,
    diagnostics,
)

old_record, new_record = derive_input_record_rename(with_dates)

if old_record and new_record and old_record != new_record:
    with_renamed = UpdateInputRecordRenamer().apply(
        with_dates,
        old_name=old_record,
        new_name=new_record,
        diagnostics=diagnostics,
    )
else:
    with_renamed = with_dates

formatted_final = FixedFormatComposer().format(with_renamed)

_indent = ProcedureIndentNormalizer()
normalized = _indent.compose(formatted_final)
if getattr(_indent, "messages", None):
    diagnostics.extend(_indent.messages)

return FixedFormatComposer().format(normalized)

```

```

except Exception as exc:
    message = DIAG_UPDATE_SKIPPED_TEMPLATE.format(reason=exc)
    diagnostics.append(message)
    logger.exception(message)
    return converted_cobol

```

```

=====
FILE: src\idms_db2_phase2\testing\update\update_program_converter.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from datetime import datetime
from pathlib import Path

```

```

from idms_db2_phase2.domain.models import ConversionInput
from idms_db2_phase2.infrastructure.file_loader import FileLoader
from idms_db2_phase2.infrastructure.local_uploaded_file import LocalUploadedFile
from idms_db2_phase2.orchestration.conversion_service import ConversionService
from idms_db2_phase2.parsers.cobol_parser import CobolParser
from idms_db2_phase2.services.name_derivation_resolver import NameDerivationResolver
from rules.update_runner_messages import (
    BULLET_TEMPLATE,
    DIAG_DERIVED_PROGRAM_ID_TEMPLATE,
    DIAG_SOURCE_FILE_TEMPLATE,
    DIAG_SOURCE_LEN_TEMPLATE,
    HEADER_DIAGNOSTICS,
    HEADER_INPUT_SUMMARY,
    HEADER_SELECTED_INPUT_FILES,
    HEADER_VALIDATION_MESSAGES,
)

```

```

LOG_DERIVED_PROGRAM_ID,
LOG_GENERATION_COMPLETED,
LOG_OUTPUT_CREATED,
LOG_SOURCE_FILE,
LOG_SOURCE_LEN,
NO_VALIDATION_MESSAGES,
PRESERVE_SOURCE_LABEL,
PRINT_GENERATION_COMPLETED,
PRINT_INPUT_PROGRAM_TEMPLATE,
PRINT_OUTPUT_CREATED_TEMPLATE,
RULE_DIAGNOSTICS,
RULE_INPUT_SUMMARY,
RULE_SELECTED_INPUT_FILES,
RULE_VALIDATION_MESSAGES,
SELECTED_COBOL_SOURCE_TEMPLATE,
SELECTED_COPYBOOK_HEADER,
SELECTED_COPYBOOK_NONE,
SELECTED_DCLGEN_HEADER,
SELECTED_PATH_BULLET_TEMPLATE,
SELECTED_SHEET_MAPPING_TEMPLATE,
SUMMARY_COBOL_LEN_TEMPLATE,
SUMMARY_COPYBOOK_FIELDS_TEMPLATE,
SUMMARY_DCLGEN_COLUMNS_TEMPLATE,
SUMMARY_PROJECT_ROOT_TEMPLATE,
SUMMARY_SHEET_ROWS_TEMPLATE,
SUMMARY_SRC_DIR_TEMPLATE,
SUMMARY_TARGET_PROGRAM_ID_TEMPLATE,
)
from rules.update_runner_rules import (
    AUTO_FIX_PIC_LENGTH_MISMATCHES_DEFAULT,
    COBOL_TEXT_ENCODING,
    OUTPUT_FILE_EXTENSION,
    OUTPUT_FILE_NAME_TEMPLATE,
    OUTPUT_TIMESTAMP_FORMAT,
)

from .update_input_selector import (
    selected_copybook_paths,
    selected_dclgen_paths,
    selected_mapping_sheet_path,
)

from .update_postprocess_pipeline import enhance_update_program

def resolve_target_program_id(source_text: str) -> str:
    """Derive the target PROGRAM-ID from the source (VM...BD... -> VMDZ...)."""
    source_id = CobolParser().program_id(source_text)
    if not source_id:
        return ""
    return NameDerivationResolver().derive(source_id)

def _build_output_path(output_dir: Path, program_path: Path) -> Path:
    timestamp = datetime.now().strftime(OUTPUT_TIMESTAMP_FORMAT)
    file_name = OUTPUT_FILE_NAME_TEMPLATE.format(
        stem=program_path.stem,
        timestamp=timestamp,
        extension=OUTPUT_FILE_EXTENSION,
    )
    return output_dir / file_name

def _print_summary(*, program_path, sheet_rows, dclgen_columns, copybook_fields,
                  idms_len, target_program_id, project_root, src_dir):
    print("")
    print(HEADER_INPUT_SUMMARY)
    print(RULE_INPUT_SUMMARY)
    print(SUMMARY_PROJECT_ROOT_TEMPLATE.format(value=project_root))
    print(SUMMARY_SRC_DIR_TEMPLATE.format(value=src_dir))
    print(SUMMARY_SHEET_ROWS_TEMPLATE.format(value=len(sheet_rows)))
    print(SUMMARY_DCLGEN_COLUMNS_TEMPLATE.format(value=len(dclgen_columns)))
    print(SUMMARY_COPYBOOK_FIELDS_TEMPLATE.format(value=len(copybook_fields)))
    print(SUMMARY_COBOL_LEN_TEMPLATE.format(value=idms_len))
    print(
        SUMMARY_TARGET_PROGRAM_ID_TEMPLATE.format(
            value=target_program_id or PRESERVE_SOURCE_LABEL
        )
    )

```

```

    )
)

print("")
print(HEADER_SELECTED_INPUT_FILES)
print(RULE_SELECTED_INPUT_FILES)
print(SELECTED_SHEET_MAPPING_TEMPLATE.format(value=selected_mapping_sheet_path()))
print(SELECTED_COBOL_SOURCE_TEMPLATE.format(value=program_path))

print(SELECTED_DCLGEN_HEADER)
for path in selected_dclgen_paths():
    print(SELECTED_PATH_BULLET_TEMPLATE.format(value=path))

copybooks = selected_copybook_paths()
if copybooks:
    print(SELECTED_COPYBOOK_HEADER)
    for path in copybooks:
        print(SELECTED_PATH_BULLET_TEMPLATE.format(value=path))
else:
    print(SELECTED_COPYBOOK_NONE)

def _print_results(result, diagnostics, logger):
    print("")
    print(HEADER_VALIDATION_MESSAGES)
    print(RULE_VALIDATION_MESSAGES)

    if result.validation_messages:
        for message in result.validation_messages:
            print(BULLET_TEMPLATE.format(text=message))
            logger.warning(message)
    else:
        print(NO_VALIDATION_MESSAGES)

    print("")
    print(HEADER_DIAGNOSTICS)
    print(RULE_DIAGNOSTICS)

    for diagnostic in diagnostics:
        print(BULLET_TEMPLATE.format(text=diagnostic))
        logger.info(diagnostic)

def convert_one_program(
    *,
    program_path: Path,
    sheet_rows: list,
    dclgen_columns: list,
    copybook_fields: list,
    shared_diagnostics: list[str],
    output_dir: Path,
    project_root,
    src_dir,
    logger,
) -> Path:
    file_loader = FileLoader()
    idms_cobol_text = file_loader.read_uploaded_text(LocalUploadedFile(program_path))

    target_program_id = resolve_target_program_id(idms_cobol_text)
    display_id = target_program_id or PRESERVE_SOURCE_LABEL

    diagnostics = list(shared_diagnostics)
    diagnostics.append(DIAG_SOURCE_FILE_TEMPLATE.format(path=program_path))
    diagnostics.append(DIAG_SOURCE_LEN_TEMPLATE.format(length=len(idms_cobol_text)))
    diagnostics.append(DIAG_DERIVED_PROGRAM_ID_TEMPLATE.format(value=display_id))

    logger.info(LOG_SOURCE_FILE, program_path)
    logger.info(LOG_SOURCE_LEN, len(idms_cobol_text))
    logger.info(LOG_DERIVED_PROGRAM_ID, display_id)

    result = ConversionService().convert(
        ConversionInput(
            sheet_mapping_rows=sheet_rows,
            dclgen_columns=dclgen_columns,
            copybook_fields=copybook_fields,
            idms_cobol_text=idms_cobol_text,

```



```

        target_program_id=target_program_id,
        auto_fix_pic_length_mismatches=AUTO_FIX_PIC_LENGTH_MISMATCHES_DEFAULT,
    )
)

converted_cobol = enhance_update_program(
    source_cobol=idms_cobol_text,
    converted_cobol=result.converted_cobol or "",
    copybook_fields=copybook_fields,
    dclgen_columns=dclgen_columns,
    target_program_id=target_program_id,
    diagnostics=diagnostics,
    logger=logger,
)

output_cobol_path = _build_output_path(output_dir, program_path)
output_cobol_path.write_text(converted_cobol, encoding=COBOL_TEXT_ENCODING)

logger.info(LOG_GENERATION_COMPLETED)
logger.info(LOG_OUTPUT_CREATED, output_cobol_path)

print("")
print(PRINT_GENERATION_COMPLETED)
print(PRINT_INPUT_PROGRAM_TEMPLATE.format(path=program_path))
print(PRINT_OUTPUT_CREATED_TEMPLATE.format(path=output_cobol_path))

_print_summary(
    program_path=program_path,
    sheet_rows=sheet_rows,
    dclgen_columns=dclgen_columns,
    copybook_fields=copybook_fields,
    idms_len=len(idms_cobol_text),
    target_program_id=target_program_id,
    project_root=project_root,
    src_dir=src_dir,
)
_print_results(result, diagnostics, logger)

return output_cobol_path

```

```

=====
FILE: src\idms_db2_phase2\transformers\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\transformers\cobol_program_id_transformer.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import re

from idms_db2_phase2.transformers.cobol_transformer_line_utils import (
    CobolTransformerLineUtils,
)
from patterns.cobol_patterns import PROGRAM_ID_PATTERN

class CobolProgramIdTransformer:
    """
    Handles target PROGRAM-ID replacement and final PROGRAM-ID period cleanup.
    """

    def __init__(
        self,
        line_utils: CobolTransformerLineUtils | None = None,
    ) -> None:
        self.line_utils = line_utils or CobolTransformerLineUtils()

    def program_id_replacement(

```

```

        self,
        *,
        original_line: str,
        logical_line: str,
        target_program_id: str,
    ) -> str | None:
        if not target_program_id:
            return None

        if not PROGRAM_ID_PATTERN.search(logical_line):
            return None

        program_id = str(target_program_id or "").strip().upper()

        if not program_id:
            return None

        replacement_body = f"PROGRAM-ID. {program_id}."

        return self.line_utils.replace_logical_body(
            original_line=original_line,
            replacement_body=replacement_body,
        )

def fix_program_id_period(
    self,
    *,
    text: str,
    target_program_id: str,
) -> str:
    program_id = str(target_program_id or "").strip().upper()

    if program_id:
        pattern = re.compile(
            rf"PROGRAM-ID\. \s*{re.escape(program_id)}\.\?",
            flags=re.IGNORECASE,
        )

        return pattern.sub(
            f"PROGRAM-ID. {program_id}.",
            text,
            count=1,
        )

    return re.sub(
        r"(PROGRAM-ID\. \s*[A-Z0-9- ]+)\s*$",
        r"\1.",
        text,
        count=1,
        flags=re.IGNORECASE | re.MULTILINE,
    )

```

```

=====
FILE: src\idms_db2_phase2\transformers\cobol_transformed_line_merger.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.transformers.cobol_transformer_line_utils import (
    CobolTransformerLineUtils,
)
from rules.cobol_transformer_rules import GENERATED_LINE_PREFIXES

class CobolTransformedLineMerger:
    """
    Merges transformed statement output with original fixed-format style when
    the transformation is a safe one-line token replacement.
    """

    def __init__(
        self,
        line_utils: CobolTransformerLineUtils | None = None,
    ) -> None:

```

```

        self.line_utils = line_utils or CobolTransformerLineUtils()

def merge_transformed_lines_with_original_style(
    self,
    *,
    original_line: str,
    original_logical: str,
    transformed_lines: list[str],
) -> list[str]:
    if not transformed_lines:
        return [original_line]

    if len(transformed_lines) == 1:
        transformed = str(transformed_lines[0] or "").strip()

        if self.is_single_line_token_replacement(
            original_logical=original_logical,
            transformed_line=transformed,
        ):
            return [
                self.line_utils.replace_logical_body(
                    original_line=original_line,
                    replacement_body=transformed,
                )
            ]

        return [transformed]

    return transformed_lines

def is_single_line_token_replacement(
    self,
    *,
    original_logical: str,
    transformed_line: str,
) -> bool:
    original = str(original_logical or "").strip()
    transformed = str(transformed_line or "").strip()
    transformed_upper = transformed.upper()

    if not original or not transformed:
        return False

    if "SQLCODE = 100" in transformed_upper:
        return True

    if "SQLCODE NOT = 100" in transformed_upper:
        return True

    if "SQLCODE NOT = 0" in transformed_upper:
        return True

    if transformed_upper.startswith(GENERATED_LINE_PREFIXES):
        return False

    if transformed.startswith("*"):
        return False

    return False

def transformer_changed_line(
    self,
    *,
    original_logical: str,
    transformed_lines: list[str],
) -> bool:
    if not transformed_lines:
        return False

    if len(transformed_lines) != 1:
        return True

    transformed = str(transformed_lines[0] or "").strip()
    original = str(original_logical or "").strip()

    return transformed != original

```

```
=====
FILE: src\idms_db2_phase2\transformers\cobol_transformer.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.domain.models import IdmsOperation
from idms_db2_phase2.parsers.cobol_parser import CobolParser
from idms_db2_phase2.transformers.cobol_program_id_transformer import (
    CobolProgramIdTransformer,
)
from idms_db2_phase2.transformers.cobol_transformed_line_merger import (
    CobolTransformedLineMerger,
)
from idms_db2_phase2.transformers.cobol_transformer_line_utils import (
    CobolTransformerLineUtils,
)
from idms_db2_phase2.transformers.idms_residual_cleanup import IdmsResidualCleanup
from idms_db2_phase2.transformers.idms_statement_transformer import (
    IdmsStatementTransformer,
)
from patterns.cobol_patterns import DIVISION_PATTERN
from patterns.db2_patterns import SQL_ERROR_PARAGRAPH_PATTERN
from rules.cobol_transformer_rules import (
    DB2_COMPILER_OPTION_LINE,
    DEFAULT_SQL_ERROR_PARAGRAPH,
)
)
```

```
class CobolTransformer:
    """
    Converts IDMS COBOL statements to DB2-compatible COBOL.

    Responsibilities are delegated to focused helper classes while keeping the
    existing public API unchanged.
    """

    def __init__(
        self,
        idms_statement_transformer: IdmsStatementTransformer,
    ) -> None:
        self.idms_statement_transformer = idms_statement_transformer
        self.parser = CobolParser()

        self.line_utils = CobolTransformerLineUtils()
        self.program_id_transformer = CobolProgramIdTransformer(
            line_utils=self.line_utils,
        )
        self.residual_cleanup = IdmsResidualCleanup()
        self.line_merger = CobolTransformedLineMerger(
            line_utils=self.line_utils,
        )

    def transform(
        self,
        cobol_text: str,
        target_program_id: str = "",
    ) -> tuple[str, list[str], list[IdmsOperation]]:
        operations = self.parser.analyze(cobol_text)

        validation_messages: list[str] = []
        output_lines: list[str] = []
        current_division = ""
        sql_error_paragraph = self._detect_sql_error_paragraph(cobol_text)
        skip_orphan_idms_abort_exit = False

        for raw_line in str(cobol_text or "").splitlines():
            line = raw_line.rstrip()
            logical = self.line_utils.logical_line(line)
            logical_stripped = logical.strip()

            if skip_orphan_idms_abort_exit:
                if self.residual_cleanup.is_exit_line(logical_stripped):
```

```

        skip_orphan_idms_abort_exit = False
        continue

    skip_orphan_idms_abort_exit = False

    if not logical_stripped:
        output_lines.append(line)
        continue

    if self._is_cbl_line(logical_stripped):
        output_lines.append(
            self.line_utils.replace_logical_body(
                original_line=line,
                replacement_body=DB2_COMPILER_OPTION_LINE,
            )
        )
        continue

    if self.residual_cleanup.is_idms_abort_paragraph(logical_stripped):
        output_lines.append(
            "* DB2: Removed orphan IDMS-ABORT paragraph."
        )
        skip_orphan_idms_abort_exit = True
        continue

    program_id_line = self.program_id_transformer.program_id_replacement(
        original_line=line,
        logical_line=logical_stripped,
        target_program_id=target_program_id,
    )

    if program_id_line is not None:
        output_lines.append(program_id_line)
        continue

    division_match = DIVISION_PATTERN.match(logical_stripped)

    if division_match:
        current_division = division_match.group(1).upper()
        output_lines.append(line)
        continue

    if self.residual_cleanup.is_idms_declarative_or_control(logical_stripped):
        output_lines.extend(
            self.residual_cleanup.removed_declarative_lines(
                logical_stripped
            )
        )
        continue

    if self.residual_cleanup.is_idms_executable_cleanup(logical_stripped):
        output_lines.extend(
            self.residual_cleanup.removed_executable_lines(
                logical_line=logical_stripped,
                current_division=current_division,
            )
        )
        continue

    transformed_lines, _opened_set = (
        self.idms_statement_transformer.transform_line(
            line=logical_stripped,
            current_division=current_division,
            sql_error_paragraph=sql_error_paragraph,
        )
    )

    if self.line_merger.transformer_changed_line(
        original_logical=logical_stripped,
        transformed_lines=transformed_lines,
    ):
        output_lines.extend(
            self.line_merger.merge_transformed_lines_with_original_style(
                original_line=line,
                original_logical=logical_stripped,
                transformed_lines=transformed_lines,
            )
        )

```

```

        )
    else:
        output_lines.append(line)

    converted_text = "\n".join(output_lines).rstrip() + "\n"

    converted_text = self.program_id_transformer.fix_program_id_period(
        text=converted_text,
        target_program_id=target_program_id,
    )

    return converted_text, validation_messages, operations

def _is_cbl_line(
    self,
    logical_line: str,
) -> bool:
    return str(logical_line or "").strip().upper().startswith("CBL ")

def _detect_sql_error_paragraph(
    self,
    cobol_text: str,
) -> str:
    match = SQL_ERROR_PARAGRAPH_PATTERN.search(str(cobol_text or ""))

    if not match:
        return DEFAULT_SQL_ERROR_PARAGRAPH

    return match.group(1).upper()

__all__ = [
    "CobolTransformer",
]

=====
FILE: src\idms_db2_phase2\transformers\cobol_transformer_line_utils.py
ENCODING: utf-8
=====

from __future__ import annotations

from patterns.cobol_transformer_patterns import (
    LEFT_SEQUENCE_PATTERN,
    RIGHT_SEQUENCE_PATTERN,
    RIGHT_SEQUENCE_WITH_SPACES_PATTERN,
)

class CobolTransformerLineUtils:
    """
    Handles COBOL logical line extraction and fixed-format body replacement.

    This utility preserves left and right sequence areas when replacing a
    logical COBOL body.
    """

    def logical_line(
        self,
        line: str,
    ) -> str:
        text = str(line or "").rstrip()

        text = self.remove_right_sequence(text)
        text = self.remove_left_sequence(text)

        return text.strip()

    def remove_left_sequence(
        self,
        line: str,
    ) -> str:
        text = str(line or "")

```

```

        if len(text) >= 6 and text[:6].strip().isdigit():
            return text[6:].strip()

        return text

def remove_right_sequence(
    self,
    line: str,
) -> str:
    text = str(line or "").rstrip()
    match = RIGHT_SEQUENCE_PATTERN.match(text)

    if match:
        return match.group("body").rstrip()

    return text

def replace_logical_body(
    self,
    original_line: str,
    replacement_body: str,
) -> str:
    text = str(original_line or "").rstrip()

    left_sequence = ""
    body_with_possible_right = text

    left_match = LEFT_SEQUENCE_PATTERN.match(body_with_possible_right)

    if left_match:
        left_sequence = left_match.group("left")
        body_with_possible_right = left_match.group("body")

    right_match = RIGHT_SEQUENCE_WITH_SPACES_PATTERN.match(
        body_with_possible_right
    )

    if right_match:
        original_body = right_match.group("body")
        original_spaces = right_match.group("spaces")
        right_sequence = right_match.group("right")

        original_body_area_width = len(original_body) + len(original_spaces)
        replacement = str(replacement_body or "")

        if len(replacement) >= original_body_area_width:
            formatted_body = replacement + " "
        else:
            formatted_body = replacement.ljust(original_body_area_width)

        return f"{left_sequence}{formatted_body}{right_sequence}"

    if left_sequence:
        return f"{left_sequence}{replacement_body}"

    return str(replacement_body or "")

__all__ = [
    "CobolTransformerLineUtils",
]

```

```

=====
FILE: src\idms_db2_phase2\transformers\field_reference\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```
=====
FILE: src\idms_db2_phase2\transformers\field_reference\field_reference_field_extractor.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.field_reference_rewriter_patterns import (
    COBOL_IDENTIFIER_PATTERN,
    LEVEL_NUMBER_FIELD_PATTERN,
    REDEFINES_BASE_PATTERN,
    REDEFINES_FIELD_PATTERN,
)

class FieldReferenceFieldExtractor:
    """Pure text helpers for extracting COBOL field names and keys.

    This class does not access repositories or resolve DB2 metadata.
    Regex patterns live in patterns/field_reference_rewriter_patterns.py.
    """

    def extract_field_name(self, value: str) -> str:
        text = str(value or "").strip()
        if not text:
            return ""

        text = text.replace(".", " ")

        level_match = LEVEL_NUMBER_FIELD_PATTERN.match(text)
        if level_match:
            return level_match.group(2)

        redefines_match = REDEFINES_FIELD_PATTERN.match(text)
        if redefines_match:
            return redefines_match.group(1)

        tokens = COBOL_IDENTIFIER_PATTERN.findall(text)
        if not tokens:
            return ""

        return tokens[0]

    def extract_redefines_base(self, value: str) -> str:
        text = str(value or "")
        match = REDEFINES_BASE_PATTERN.search(text)
        if not match:
            return ""
        return match.group(1)

    def field_key(self, value: str) -> str:
        return NameNormalizer.to_cobol(value)

    def first_non_empty(self, *values: str) -> str:
        for value in values:
            text = str(value or "").strip()
            if text:
                return text
        return ""
```

```
=====
FILE: src\idms_db2_phase2\transformers\field_reference\field_reference_host_resolver.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.field_reference_rewriter_patterns import (
    HOST_REFERENCE_DOT_PATTERN,
    HOST_REFERENCE_OF_PATTERN,
    HOST_REFERENCE_OF_SPLIT_PATTERN,
)

)
```



```

from rules.field_reference_rewriter_rules import (
    HOST_GROUP_SEPARATOR,
    HOST_OF_GROUP_TEMPLATE,
    HOST_OF_SPLIT_PART_COUNT,
    HOST_REFERENCE_LEADING_CHAR,
)

class FieldReferenceHostResolver:
    """Resolves and formats a DCLGEN host reference for a DB2 column.

    Produces the canonical "HOST OF GROUP" form. No inline regex and no
    hardcoded separators live here; all patterns and literals are external.
    """

    def __init__(self, host_variable_resolver: HostVariableResolver) -> None:
        self.host_variable_resolver = host_variable_resolver

    def dclgen_reference_for_column(
        self,
        *,
        table_name: str,
        column_name: str,
    ) -> str:
        host_reference = self.host_variable_resolver.host_reference_for_column(
            table_name=table_name,
            column_name=column_name,
        )

        if not host_reference:
            return ""

        host_reference = str(host_reference or "").strip()

        of_match = HOST_REFERENCE_OF_PATTERN.search(host_reference)
        if of_match:
            return self._format(of_match.group("host"), of_match.group("group"))

        dot_match = HOST_REFERENCE_DOT_PATTERN.search(host_reference)
        if dot_match:
            return self._format(dot_match.group("host"), dot_match.group("group"))

        cleaned = host_reference.lstrip(HOST_REFERENCE_LEADING_CHAR).strip()

        if HOST_GROUP_SEPARATOR in cleaned.upper():
            parts = HOST_REFERENCE_OF_SPLIT_PATTERN.split(cleaned)
            if len(parts) == HOST_OF_SPLIT_PART_COUNT:
                return self._format(parts[0], parts[1])

        return NameNormalizer.to_cobol(cleaned)

    def _format(self, host: str, group: str) -> str:
        return HOST_OF_GROUP_TEMPLATE.format(
            host=NameNormalizer.to_cobol(host),
            group=NameNormalizer.to_cobol(group),
        )

=====
FILE: src\idms_db2_phase2\transformers\field_reference_context_builder.py
ENCODING: utf-8
=====

from __future__ import annotations

from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from idms_db2_phase2.transformers.field_reference.field_reference_field_extractor import (
    FieldReferenceFieldExtractor,
)
from idms_db2_phase2.transformers.field_reference.field_reference_host_resolver import (
    FieldReferenceHostResolver,
)
from rules.field_reference_rewriter_rules import DCLGEN_GROUP_PREFIX

```

```

class FieldReferenceContextBuilder:
    """Builds record-to-field and record-to-DCLGEN context maps from Sheet
    Mapping and DCLGEN metadata.

    Orchestration only. Field-name parsing is delegated to
    FieldReferenceFieldExtractor and host-reference formatting to
    FieldReferenceHostResolver.
    """

    def __init__(
        self,
        *,
        mapping_repository: MappingRepository,
        table_name_resolver: TableNameResolver,
        host_variable_resolver: HostVariableResolver,
    ) -> None:
        self.mapping_repository = mapping_repository
        self.table_name_resolver = table_name_resolver
        self.host_variable_resolver = host_variable_resolver
        self._extractor = FieldReferenceFieldExtractor()
        self._host_resolver = FieldReferenceHostResolver(host_variable_resolver)

    def build_reference_maps(
        self,
    ) -> tuple[
        dict[str, dict[str, str]],
        dict[str, str],
        dict[str, str],
        dict[str, str],
    ]:
        record_field_map: dict[str, dict[str, str]] = {}
        record_group_map: dict[str, str] = {}
        table_record_map: dict[str, str] = {}
        group_record_map: dict[str, str] = {}

        for record in self.mapping_repository.records():
            normalized_record = NameNormalizer.normalize(record)
            if not normalized_record:
                continue

            resolved_table = self.table_name_resolver.table_for_record(
                normalized_record
            )

            if resolved_table:
                table_key = NameNormalizer.normalize(resolved_table)
                group_name = DCLGEN_GROUP_PREFIX + NameNormalizer.to_cobol(table_key)

                record_group_map[normalized_record] = group_name
                table_record_map[table_key] = normalized_record
                group_record_map[NameNormalizer.normalize(group_name)] = (
                    normalized_record
                )

            record_field_map[normalized_record] = self.field_map_for_record(
                normalized_record
            )

        self.add_redefines_aliases(record_field_map)

        return (
            record_field_map,
            record_group_map,
            table_record_map,
            group_record_map,
        )

    def field_map_for_record(self, record_name: str) -> dict[str, str]:
        output: dict[str, str] = {}
        rows = self.mapping_repository.rows_for_record(record_name)

        for row in rows:
            source_candidates = self.source_candidates_from_row(row)
            if not source_candidates:

```

```

        continue

    target_table = self.table_name_resolver.resolve_table(
        self._extractor.first_non_empty(
            getattr(row, "new_db2_record", ""),
            getattr(row, "cross_application_db2_table", ""),
        )
    )
    target_column = NameNormalizer.normalize(
        self._extractor.first_non_empty(
            getattr(row, "new_db2_field_name", ""),
            getattr(row, "cross_application_db2_field_name", ""),
        )
    )

    if not target_table or not target_column:
        continue

    target_reference = self._host_resolver.dclgen_reference_for_column(
        table_name=target_table,
        column_name=target_column,
    )
    if not target_reference:
        continue

    for source_candidate in source_candidates:
        source_key = self._extractor.field_key(source_candidate)
        if not source_key:
            continue
        if source_key not in output:
            output[source_key] = target_reference

    return output

def source_candidates_from_row(self, row) -> list[str]:
    candidates: list[str] = []

    for value in [
        getattr(row, "cobol_zone", ""),
        getattr(row, "reference_field_name_copybook", ""),
    ]:
        field_name = self._extractor.extract_field_name(value)
        if not field_name:
            continue

        cobol_name = NameNormalizer.to_cobol(field_name)
        if cobol_name and cobol_name not in candidates:
            candidates.append(cobol_name)

        redefines_base = self._extractor.extract_redefines_base(value)
        if redefines_base:
            base_name = NameNormalizer.to_cobol(redefines_base)
            if base_name and base_name not in candidates:
                candidates.append(base_name)

    return candidates

def add_redefines_aliases(
    self,
    record_field_map: dict[str, dict[str, str]],
) -> None:
    for record in self.mapping_repository.records():
        normalized_record = NameNormalizer.normalize(record)
        rows = self.mapping_repository.rows_for_record(normalized_record)
        field_map = record_field_map.get(normalized_record, {})
        if not field_map:
            continue

        for row in rows:
            cobol_zone = getattr(row, "cobol_zone", "") or ""
            alias_name = self._extractor.extract_field_name(cobol_zone)
            base_name = self._extractor.extract_redefines_base(cobol_zone)
            if not alias_name or not base_name:
                continue

            alias_key = self._extractor.field_key(alias_name)

```

```

        base_key = self._extractor.field_key(base_name)
        if not alias_key or not base_key:
            continue
        if alias_key in field_map:
            continue
        if base_key in field_map:
            field_map[alias_key] = field_map[base_key]

```

```

=====
FILE: src\idms_db2_phase2\transformers\field_reference_line_utils.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

```

```

from patterns.field_reference_rewriter_patterns import (
    DCL_DOT_REFERENCE_PATTERN,
    HOST_OF_DCL_REFERENCE_PATTERN,
    STRING_PATTERN,
)

```

```

class FieldReferenceLineUtils:

```

```

    """

```

```

    Line-level helpers for field reference rewriting.

```

```

    This class does not access repositories or resolve DB2 metadata.
    """

```

```

    def logical_line(
        self,
        line: str,
    ) -> str:
        text = str(line or "").rstrip()

        if len(text) >= 80:
            left = text[:6]
            indicator = text[6:7]
            body = text[7:72]

            if left.strip().isdigit():
                if indicator in {"*", "/"}:
                    return indicator + body.rstrip()

            return body.strip()

        if len(text) > 6 and text[:6].strip().isdigit():
            return text[6:].strip()

        return text.strip()

```

```

    def is_comment_or_blank(
        self,
        line: str,
    ) -> bool:
        logical = self.logical_line(line)
        stripped = str(logical or "").strip()

        if not stripped:
            return True

        if stripped.startswith("*"):
            return True

        if stripped.startswith("/"):
            return True

        return False

```

```

    def has_existing_dclgen_reference(
        self,
        line: str,
    ) -> bool:
        upper = str(line or "").upper()

```

```

    if " OF DCL" in upper:
        return True

    if DCL_DOT_REFERENCE_PATTERN.search(upper):
        return True

    if HOST_OF_DCL_REFERENCE_PATTERN.search(upper):
        return True

    return False

def split_string_segments(
    self,
    line: str,
) -> list[tuple[str, bool]]:
    text = str(line or "")
    output: list[tuple[str, bool]] = []
    last_index = 0

    for match in STRING_PATTERN.finditer(text):
        if match.start() > last_index:
            output.append(
                (
                    text[last_index : match.start()],
                    False,
                )
            )

            output.append(
                (
                    match.group(0),
                    True,
                )
            )

            last_index = match.end()

    if last_index < len(text):
        output.append(
            (
                text[last_index:],
                False,
            )
        )

    return output

```

```

=====
FILE: src\idms_db2_phase2\transformers\field_reference_record_context.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.field_reference_rewriter_patterns import (
    GENERATED_OBTAIN_COMMENT_PATTERN,
    INITIALIZE_DCL_PATTERN,
    MOVE_SPACES_TO_RECORD_PATTERN,
    PARAGRAPH_PATTERN,
    QUALIFIED_REFERENCE_PATTERN,
)

```

```

class FieldReferenceRecordContext:
    """
    Detects active record context for safe bare field rewriting.
    """

    def __init__(
        self,
        *,
        record_field_map: dict[str, dict[str, str]],
        group_record_map: dict[str, str],
    ) -> None:

```

```

self.record_field_map = record_field_map
self.group_record_map = group_record_map

def detect_active_record_from_line(
    self,
    *,
    logical_line: str,
    current_active_record: str,
) -> str:
    logical = str(logical_line or "").strip()

    if not logical:
        return current_active_record

    for detector in [
        self.record_from_paragraph,
        self.record_from_initialize,
        self.record_from_generated_comment,
        self.record_from_move_target,
        self.record_from_qualified_reference,
    ]:
        record = detector(logical)

        if record:
            return record

    return current_active_record

def record_from_paragraph(
    self,
    logical_line: str,
) -> str:
    match = PARAGRAPH_PATTERN.match(logical_line)

    if not match:
        return ""

    paragraph = NameNormalizer.normalize(match.group("name"))

    for record in self.record_field_map:
        if record and record in paragraph:
            return record

    return ""

def record_from_initialize(
    self,
    logical_line: str,
) -> str:
    match = INITIALIZE_DCL_PATTERN.search(logical_line)

    if not match:
        return ""

    group = NameNormalizer.normalize(match.group("group"))

    return self.group_record_map.get(group, "")

def record_from_generated_comment(
    self,
    logical_line: str,
) -> str:
    match = GENERATED_OBTAIN_COMMENT_PATTERN.search(logical_line)

    if not match:
        return ""

    record = NameNormalizer.normalize(match.group("record"))

    if record in self.record_field_map:
        return record

    return ""

def record_from_move_target(
    self,

```

```

        logical_line: str,
    ) -> str:
        match = MOVE_SPACES_TO_RECORD_PATTERN.search(logical_line)

        if not match:
            return ""

        record = NameNormalizer.normalize(match.group("record"))

        if record in self.record_field_map:
            return record

        return ""

def record_from_qualified_reference(
    self,
    logical_line: str,
) -> str:
    match = QUALIFIED_REFERENCE_PATTERN.search(logical_line)

    if not match:
        return ""

    record = NameNormalizer.normalize(match.group("record"))

    if record in self.record_field_map:
        return record

    return ""

```

```

=====
FILE: src\idms_db2_phase2\transformers\field_reference_replacement_engine.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import re

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.field_reference_rewriter_patterns import (
    OF_IN_AFTER_PATTERN,
    OF_IN_BEFORE_PATTERN,
    QUALIFIED_REFERENCE_PATTERN,
)
from rules.field_reference_rewriter_rules import PROTECTED_BARE_PREFIXES

class FieldReferenceReplacementEngine:
    """
    Rewrites qualified and bare IDMS field references using resolved
    Sheet Mapping + DCLGEN host-reference maps.
    """

    def __init__(
        self,
        *,
        record_field_map: dict[str, dict[str, str]],
        rewrite_messages: list[str],
    ) -> None:
        self.record_field_map = record_field_map
        self.rewrite_messages = rewrite_messages

    def rewrite_qualified_references(
        self,
        text: str,
    ) -> str:
        def repl(match):
            source_field = NameNormalizer.to_cobol(match.group("field"))
            source_record = NameNormalizer.normalize(match.group("record"))

            target = self.target_for_record_field(
                record_name=source_record,
                source_field=source_field,
            )

```

```

        if not target:
            return match.group(0)

        self.rewrite_messages.append(
            f"Qualified field rewrite: {source_field} OF "
            f"{NameNormalizer.to_cobol(source_record)} -> {target}"
        )

        return target

    return QUALIFIED_REFERENCE_PATTERN.sub(
        repl,
        text,
    )

def rewrite_bare_references(
    self,
    *,
    text: str,
    active_record: str,
) -> str:
    record = NameNormalizer.normalize(active_record)

    if not record:
        return text

    field_map = self.record_field_map.get(record, {})

    if not field_map:
        return text

    updated = str(text or "")

    for source_key, target_reference in sorted(
        field_map.items(),
        key=lambda item: len(item[0]),
        reverse=True,
    ):
        source_name = NameNormalizer.to_cobol(source_key)

        if not source_name or not target_reference:
            continue

        if self.is_protected_bare_source_name(source_name):
            continue

        updated = self.replace_bare_identifier(
            text=updated,
            source_name=source_name,
            target_name=target_reference,
        )

    return updated

def replace_bare_identifier(
    self,
    *,
    text: str,
    source_name: str,
    target_name: str,
) -> str:
    source = NameNormalizer.to_cobol(source_name)
    target = str(target_name or "").strip()

    if not source or not target:
        return text

    pattern = re.compile(
        rf"(?![A-Z0-9-]){re.escape(source)}(?![A-Z0-9-])",
        flags=re.IGNORECASE,
    )

    def repl(match):
        before_text = text[: match.start()]
        after_text = text[match.end() :]

```



```

        if self.is_part_of_qualified_reference(
            before_text=before_text,
            after_text=after_text,
        ):
            return match.group(0)

        self.rewrite_messages.append(
            f"Bare field rewrite: {source} -> {target}"
        )

        return target

    return pattern.sub(
        repl,
        text,
    )

def target_for_record_field(
    self,
    *,
    record_name: str,
    source_field: str,
) -> str:
    record = NameNormalizer.normalize(record_name)
    field = self.field_key(source_field)

    if not record or not field:
        return ""

    return self.record_field_map.get(record, {}).get(field, "")

def field_key(
    self,
    value: str,
) -> str:
    return NameNormalizer.to_cobol(value)

def is_protected_bare_source_name(
    self,
    source_name: str,
) -> bool:
    source = NameNormalizer.to_cobol(source_name).upper()

    if not source:
        return True

    for prefix in PROTECTED_BARE_PREFIXES:
        if source.startswith(prefix):
            return True

    return False

def is_part_of_qualified_reference(
    self,
    *,
    before_text: str,
    after_text: str,
) -> bool:
    before = str(before_text or "").upper()
    after = str(after_text or "").upper()

    if OF_IN_BEFORE_PATTERN.search(before):
        return True

    if OF_IN_AFTER_PATTERN.match(after):
        return True

    return False

__all__ = [
    "FieldReferenceReplacementEngine",
]

```

```
=====
FILE: src\idms_db2_phase2\transformers\field_reference_rewriter.py
ENCODING: utf-8
=====
```

```
# LOCATION: src/idms_db2_phase2/transformers/field_reference_rewriter.py
# ACTION: REPLACE ENTIRE FILE
```

```
from __future__ import annotations

from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.services.fixed_format_line_service import (
    FixedFormatLineService,
)
from idms_db2_phase2.transformers.field_reference_context_builder import (
    FieldReferenceContextBuilder,
)
from idms_db2_phase2.transformers.field_reference_line_utils import (
    FieldReferenceLineUtils,
)
from idms_db2_phase2.transformers.field_reference_record_context import (
    FieldReferenceRecordContext,
)
from idms_db2_phase2.transformers.field_reference_replacement_engine import (
    FieldReferenceReplacementEngine,
)
from patterns.field_reference_rewriter_patterns import (
    DIVISION_PATTERN,
    END_EXEC_PATTERN,
    EXEC_SQL_PATTERN,
)
from rules.field_reference_rewriter_rules import (
    ALLOW_REWRITE_WRAPPING,
    ENFORCE_FIXED_FORMAT_REWRITE,
    FIELD_REFERENCE_GEOMETRY_MESSAGES,
)

PROCEDURE_DIVISION_NAME = "PROCEDURE"
BLANK_INDICATOR = " "
LINE_ENDINGS = "\r\n"
```

```
class FieldReferenceRewriter:
    """
    Generic IDMS field reference to DB2 DCLGEN reference rewriter.

    Public facade preserved for existing callers.

    Rules:
    - Qualified references are always eligible:
        FIELD OF RECORD
        FIELD IN RECORD
    - Bare references are eligible only when active record context is strong.
    - Date fields are protected only for bare references.

    CORRECTION 1 - the rewrite broke fixed-format geometry
    -----
    _rewrite_line substituted into the WHOLE record, sequence areas
    included. A qualified rewrite is almost always longer than what it
    replaces:

        DA-CPTA-FORM-AS OF VMBFAS          25 characters
        DA-CPTAFS-479BFAS OF DCLDZBFASTV  32 characters

    so an 80-column record became 87 and the right sequence 00002020
    moved from column 73 to column 80. Later passes read the first
    displaced digit as body text and emitted

        AND HELP-DA-CPTAFS-479BFAS NOT = '00000000') OR 0

    Substitution now happens inside columns 8-72 ONLY, and the record is
    reassembled from its own left sequence, indicator and right sequence.

    CORRECTION 2 - trailing blanks were stripped
```

```

-----
`line = raw_line.rstrip()` removed the body padding that positions
the right sequence at column 73. Only the line ending is stripped
now; columns 8-72 keep their blanks.

```

CORRECTION 3 - an over-long rewrite had nowhere to go

```

-----
A body that no longer fits 65 columns is WRAPPED at word boundaries
onto further physical lines. _rewrite_line therefore returns a LIST,
and the caller extends rather than appends. When even a wrapped form
cannot be represented the original line is kept and reported: a
rewrite that cannot be rendered must never corrupt the record.

```

CORRECTION 4 - reference-map building was a tail method

```

-----
_build_reference_maps was the last method before __all__ and was
lost in transit, producing

```

```

AttributeError: 'FieldReferenceRewriter' object has no
attribute '_build_reference_maps'

```

rewrite() now calls the context builder directly, so no helper is
required, and the helper itself is declared next to __init__.

```

"""
def __init__(
    self,
    mapping_repository: MappingRepository,
    table_name_resolver: TableNameResolver,
    host_variable_resolver: HostVariableResolver,
    fixed_format: FixedFormatLineService | None = None,
) -> None:
    self.mapping_repository = mapping_repository
    self.table_name_resolver = table_name_resolver
    self.host_variable_resolver = host_variable_resolver

    self.rewrite_messages: list[str] = []
    self.messages: list[str] = self.rewrite_messages

    self.record_field_map: dict[str, dict[str, str]] = {}
    self.record_group_map: dict[str, str] = {}
    self.table_record_map: dict[str, str] = {}
    self.group_record_map: dict[str, str] = {}

    self.line_utils = FieldReferenceLineUtils()
    self.fixed_format = fixed_format or FixedFormatLineService()
    self.context_builder = FieldReferenceContextBuilder(
        mapping_repository=mapping_repository,
        table_name_resolver=table_name_resolver,
        host_variable_resolver=host_variable_resolver,
    )

def _build_reference_maps(self) -> None:
    """Retained for any external caller. rewrite() does not need it."""
    (
        self.record_field_map,
        self.record_group_map,
        self.table_record_map,
        self.group_record_map,
    ) = self.context_builder.build_reference_maps()

#
# Public entry point
#
def rewrite(
    self,
    text: str,
) -> str:
    self.rewrite_messages = []
    self.messages = self.rewrite_messages

    if not text:
        return ""

# Inlined deliberately: a helper declared at the tail of the class
# has already gone missing once.

```

```

(
    self.record_field_map,
    self.record_group_map,
    self.table_record_map,
    self.group_record_map,
) = self.context_builder.build_reference_maps()

if not self.record_field_map:
    return str(text or "").rstrip() + "\n"

lines = str(text or "").splitlines()

has_any_division = any(
    DIVISION_PATTERN.match(self.line_utils.logical_line(line))
    for line in lines
)

record_context = FieldReferenceRecordContext(
    record_field_map=self.record_field_map,
    group_record_map=self.group_record_map,
)

replacement_engine = FieldReferenceReplacementEngine(
    record_field_map=self.record_field_map,
    rewrite_messages=self.rewrite_messages,
)

output_lines: list[str] = []
inside_exec_sql = False

current_division = (
    PROCEDURE_DIVISION_NAME if not has_any_division else ""
)
active_record = ""

for raw_line in lines:
    # Only the line ending is removed. Trailing blanks are part
    # of a fixed-format record: they position the right sequence
    # at column 73.
    line = raw_line.rstrip(LINE_ENDINGS)
    logical = self.line_utils.logical_line(line)

    division_match = DIVISION_PATTERN.match(logical)

    if division_match:
        current_division = division_match.group(1).upper()
        active_record = ""
        output_lines.append(line)
        continue

    detected_context = record_context.detect_active_record_from_line(
        logical_line=logical,
        current_active_record=active_record,
    )

    if detected_context:
        active_record = detected_context

    if EXEC_SQL_PATTERN.match(logical):
        inside_exec_sql = True
        output_lines.append(line)
        continue

    if inside_exec_sql:
        output_lines.append(line)

        if END_EXEC_PATTERN.match(logical):
            inside_exec_sql = False

        continue

    output_lines.extend(
        self._rewrite_line(
            line=line,
            current_division=current_division,
            active_record=active_record,

```

```

        replacement_engine=replacement_engine,
    )
)

return "\n".join(output_lines).rstrip() + "\n"

#
# One line
#
def _rewrite_line(
    self,
    *,
    line: str,
    current_division: str,
    active_record: str,
    replacement_engine: FieldReferenceReplacementEngine,
) -> list[str]:
    """Rewrite one line. Returns one line, or several when wrapped."""
    if self.line_utils.is_comment_or_blank(line):
        return [line]

    if current_division != PROCEDURE_DIVISION_NAME:
        return [line]

    if self.line_utils.has_existing_dclgen_reference(line):
        return [line]

    if not ENFORCE_FIXED_FORMAT_REWRITE or not self._is_fixed(line):
        return [
            self._rewrite_text(
                text=line,
                active_record=active_record,
                replacement_engine=replacement_engine,
            )
        ]

    left, indicator, body, right = self._split(line)

    new_body = self._rewrite_text(
        text=body,
        active_record=active_record,
        replacement_engine=replacement_engine,
    )

    if new_body == body:
        return [line]

    rebuilt = self._build(left, indicator, new_body, right)

    if rebuilt is not None:
        return [rebuilt]

    if ALLOW_REWRITE_WRAPPING:
        wrapped = self._wrap(line, new_body)

        if wrapped:
            self.rewrite_messages.append(
                FIELD_REFERENCE_GEOMETRY_MESSAGES["wrapped"].format(
                    count=len(wrapped),
                )
            )
            return list(wrapped)

    self.rewrite_messages.append(
        FIELD_REFERENCE_GEOMETRY_MESSAGES["refused"].format(
            body=new_body.strip(),
        )
    )

    return [line]

def _rewrite_text(
    self,
    *,
    text: str,
    active_record: str,

```

```

        replacement_engine: FieldReferenceReplacementEngine,
    ) -> str:
        """Apply both rewrites to one span, string literals protected."""
        segments = self.line_utils.split_string_segments(text)
        rewritten_segments: list[str] = []

        for segment, is_string in segments:
            if is_string:
                rewritten_segments.append(segment)
                continue

            rewritten = replacement_engine.rewrite_qualified_references(segment)
            rewritten = replacement_engine.rewrite_bare_references(
                text=rewritten,
                active_record=active_record,
            )

            rewritten_segments.append(rewritten)

        return "".join(rewritten_segments)

#
# Fixed-format access, all total functions
#
def _is_fixed(self, line: str) -> bool:
    try:
        return bool(self.fixed_format.is_fixed_line(line))
    except Exception: # noqa: BLE001
        return False

def _split(self, line: str) -> tuple[str, str, str, str]:
    try:
        left, indicator, body, right = self.fixed_format.split(line)
        return (
            str(left or ""),
            str(indicator or BLANK_INDICATOR),
            str(body or ""),
            str(right or ""),
        )
    except Exception: # noqa: BLE001
        return "", BLANK_INDICATOR, str(line or ""), ""

def _build(
    self,
    left: str,
    indicator: str,
    body: str,
    right: str,
) -> str | None:
    try:
        return self.fixed_format.build_or_none(
            left,
            indicator or BLANK_INDICATOR,
            body,
            right,
        )
    except Exception: # noqa: BLE001
        return None

def _wrap(self, line: str, body: str) -> list[str]:
    try:
        return list(
            self.fixed_format.replace_body_wrapped(line, body) or []
        )
    except Exception: # noqa: BLE001
        return []

__all__ = [
    "FieldReferenceRewriter",
]

```

```
=====
FILE: src\idms_db2_phase2\transformers\idms_residual_cleanup.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

from patterns.cobol_transformer_patterns import (
    BIND_STATEMENT_PATTERN,
    FIND_CURRENT_STATEMENT_PATTERN,
    FINISH_STATEMENT_PATTERN,
    IDMS_ABORT_PARAGRAPH_PATTERN,
    IDMS_DECLARATIVE_PATTERNS,
    IDMS_EXECUTABLE_PATTERNS,
    EXIT_LINE_PATTERN,
)

class IdmsResidualCleanup:
    """
    Detects and removes residual IDMS declarative/control/executable lines.

    This cleanup is conservative and preserves business flow by adding
    CONTINUE. when a removed executable statement appears in PROCEDURE DIVISION.
    """

    def is_idms_declarative_or_control(
        self,
        logical_line: str,
    ) -> bool:
        statement = str(logical_line or "").strip().rstrip(".")

        for pattern in IDMS_DECLARATIVE_PATTERNS:
            if pattern.search(statement):
                return True

        return False

    def is_idms_executable_cleanup(
        self,
        logical_line: str,
    ) -> bool:
        statement = str(logical_line or "").strip().rstrip(".")

        for pattern in IDMS_EXECUTABLE_PATTERNS:
            if pattern.search(statement):
                return True

        return False

    def is_idms_abort_paragraph(
        self,
        logical_line: str,
    ) -> bool:
        statement = str(logical_line or "").strip()

        return bool(IDMS_ABORT_PARAGRAPH_PATTERN.match(statement))

    def is_exit_line(
        self,
        logical_line: str,
    ) -> bool:
        statement = str(logical_line or "").strip()

        return bool(EXIT_LINE_PATTERN.match(statement))

    def removed_declarative_lines(
        self,
        logical_line: str,
    ) -> list[str]:
        return [
            f"* DB2: Removed residual IDMS control statement: {logical_line}",
        ]

    def removed_executable_lines(
        self,
```

```

*,
logical_line: str,
current_division: str,
) -> list[str]:
    statement = str(logical_line or "").strip()

    if FINISH_STATEMENT_PATTERN.search(statement):
        if current_division == "PROCEDURE":
            return [
                "* DB2: IDMS FINISH converted to COMMIT.",
                "MOVE 'COMMIT' TO SQL-LOCATION.",
                "EXEC SQL",
                "    COMMIT",
                "END-EXEC.",
            ]

            return [
                f"* DB2: Removed IDMS FINISH outside PROCEDURE DIVISION: {statement}",
            ]

    if BIND_STATEMENT_PATTERN.search(statement):
        return self.procedure_safe_removal(
            message=f"* DB2: Removed IDMS BIND statement: {statement}",
            current_division=current_division,
        )

    if FIND_CURRENT_STATEMENT_PATTERN.search(statement):
        return self.procedure_safe_removal(
            message=f"* DB2: Removed IDMS FIND CURRENT statement: {statement}",
            current_division=current_division,
        )

    return self.procedure_safe_removal(
        message=f"* DB2: Removed residual IDMS executable statement: {statement}",
        current_division=current_division,
    )

def procedure_safe_removal(
    self,
    *,
    message: str,
    current_division: str,
) -> list[str]:
    if current_division == "PROCEDURE":
        return [
            message,
            "CONTINUE.",
        ]

    return [
        message,
    ]

```

```

=====
FILE: src\idms_db2_phase2\transformers\idms_statement\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\transformers\idms_statement\control_statement_converter.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from patterns.idms_patterns import (
    BIND_STATEMENT_PATTERN,
    COMMIT_PATTERN,
    CONNECT_STATEMENT_PATTERN,
    DISCONNECT_STATEMENT_PATTERN,
    FINISH_PATTERN,
    IDMS_ABORT_PERFORM_PATTERN,

```



```

IDMS_STATUS_PERFORM_PATTERN,
READY_PATTERN,
USAGE_MODE_PATTERN,
)
from rules.idms_transformer_messages import (
    COMMIT_OUTSIDE_PROCEDURE_TEMPLATE,
    FINISH_CONVERTED_TO_COMMIT,
    FINISH_OUTSIDE_PROCEDURE_TEMPLATE,
    REMOVED_BIND_TEMPLATE,
    REMOVED_CONNECT_TEMPLATE,
    REMOVED_CONTROL_STATEMENT_TEMPLATE,
    REMOVED_DISCONNECT_TEMPLATE,
    REMOVED_STATUS_ABORT_TEMPLATE,
    REMOVED_USAGE_READY_TEMPLATE,
)
from rules.idms_transformer_rules import PROCEDURE_DIVISION_NAME

class ControlStatementConverterMixin:
    """Converts IDMS declarative/control statements to DB2 comments."""

    def _convert_declarative_or_control(self, upper, stripped_line):
        if not self._is_idms_declarative_or_control_statement(upper):
            return None
        return [
            REMOVED_CONTROL_STATEMENT_TEMPLATE.format(line=stripped_line),
        ]

    def _convert_finish_or_commit(self, upper, stripped_line, current_division):
        if FINISH_PATTERN.search(upper):
            if current_division != PROCEDURE_DIVISION_NAME:
                return [
                    FINISH_OUTSIDE_PROCEDURE_TEMPLATE.format(line=stripped_line)
                ]
            return [FINISH_CONVERTED_TO_COMMIT, *self.sql_generator.commit()]

        if COMMIT_PATTERN.search(upper) and "EXEC SQL" not in upper:
            if current_division != PROCEDURE_DIVISION_NAME:
                return [
                    COMMIT_OUTSIDE_PROCEDURE_TEMPLATE.format(line=stripped_line)
                ]
            return self.sql_generator.commit()

        return None

    def _convert_bind_ready_connect_disconnect(
        self, upper, stripped_line, current_division
    ):
        if BIND_STATEMENT_PATTERN.search(upper):
            return self._removed_idms_executable_lines(
                message=REMOVED_BIND_TEMPLATE.format(line=stripped_line),
                current_division=current_division,
            )

        if READY_PATTERN.search(upper) or USAGE_MODE_PATTERN.search(upper):
            return self._removed_idms_executable_lines(
                message=REMOVED_USAGE_READY_TEMPLATE.format(line=stripped_line),
                current_division=current_division,
            )

        if CONNECT_STATEMENT_PATTERN.search(upper):
            return self._removed_idms_executable_lines(
                message=REMOVED_CONNECT_TEMPLATE.format(line=stripped_line),
                current_division=current_division,
            )

        if DISCONNECT_STATEMENT_PATTERN.search(upper):
            return self._removed_idms_executable_lines(
                message=REMOVED_DISCONNECT_TEMPLATE.format(line=stripped_line),
                current_division=current_division,
            )

        return None

    def _convert_status_abort_perform(
        self, upper, stripped_line, current_division
    )

```

```

):
    if IDMS_STATUS_PERFORM_PATTERN.search(upper) or (
        IDMS_ABORT_PERFORM_PATTERN.search(upper)
    ):
        return self._removed_idms_executable_lines(
            message=REMOVED_STATUS_ABORT_TEMPLATE.format(line=stripped_line),
            current_division=current_division,
        )
    return None

=====
FILE: src\idms_db2_phase2\transformers\idms_statement\data_statement_converter.py
ENCODING: utf-8
=====

# LOCATION: src/idms_db2_phase2/transformers/idms_statement/data_statement_converter.py
# ACTION: REPLACE ENTIRE FILE

from __future__ import annotations

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.idms_patterns import (
    ERASE_PATTERN,
    FIND_CURRENT_PATTERN,
    FIND_FIRST_PATTERN,
    MODIFY_PATTERN,
    OBTAIN_CALC_PATTERN,
    OBTAIN_CALC_REVERSED_PATTERN,
    OBTAIN_FIRST_NEXT_PATTERN,
    ON_DB_REC_NOT_FOUND_PATTERN,
    STORE_PATTERN,
)
from rules.idms_transformer_messages import (
    CONVERTED_FIND_FIRST_TEMPLATE,
    CONVERTED_OBTAIN_FIRST_TEMPLATE,
    CONVERTED_OBTAIN_NEXT_TEMPLATE,
    DIRECT_UPDATE_COMPOSITE_KEY,
    FIND_FIRST_OUTSIDE_PROCEDURE_TEMPLATE,
    OBTAIN_CALC_OUTSIDE_PROCEDURE_TEMPLATE,
    OBTAIN_OUTSIDE_PROCEDURE_TEMPLATE,
    OPERATION_ERASE,
    OPERATION_MODIFY,
    OPERATION_STORE,
    PERFORM_FETCH_CURSOR_TEMPLATE,
    PERFORM_OPEN_CURSOR_TEMPLATE,
    REMOVED_FIND_CURRENT_TEMPLATE,
    REMOVED_OBTAIN_CALC_SELECT_TEMPLATE,
    STORE_MODIFY_ERASE_OUTSIDE_PROCEDURE_TEMPLATE,
)
from rules.idms_transformer_rules import (
    CONTINUE_STATEMENT,
    OBTAIN_FIRST_KEYWORD,
    PROCEDURE_DIVISION_NAME,
    SQLCODE_100_CONTINUE_BODY,
    SQLCODE_100_IF_CLOSE,
    SQLCODE_100_IF_OPEN,
)

# Indent applied to the statement lifted into the generated
# IF SQLCODE = 100 guard block.
GUARDED_STATEMENT_INDENT = "    "

NO_OPENED_SET = ""

class DataStatementConverterMixin:
    """Converts IDMS data-access statements to DB2/cursor COBOL.

    NAME HANDLING CONTRACT
    -----
    Two forms of every IDMS name are needed and they must never be mixed:

    *_key      NameNormalizer.normalize() -> DB2 identifier, underscore
               separated. Used ONLY for resolver / repository lookups
               and for the opened_set value returned to the caller,

```

which downstream passes re-normalize.

```
*_display NameNormalizer.to_cobol() -> IDMS / COBOL spelling,  
hyphen separated. Used ONLY inside generated *DB2:  
comment text.
```

CORRECTION 1 - IDMS set name emitted as a DB2 identifier

_convert_obtain_first_next and _convert_find_first passed the
normalized key straight into the comment template, so an LRF-expanded
program produced

```
*DB2: Converted OBTAIN FIRST VMBFAS WITHIN AR_VMBFRM1.
```

for the IDMS set AR-VMBFRM1. An IDMS set is not a DB2 identifier and
must keep its hyphens. _convert_obtain_calc already did this correctly
via cobol_record; the other two were never brought in line.

CORRECTION 2 - hyphenated record names were corrupted too

The same defect rendered record VMB-FAR as VMB_FAR. It went unnoticed
only because the reference program's record, VMBFAS, carries no hyphen.

CORRECTION 3 - no unmapped-record guard on the cursor paths

_convert_obtain_calc and _convert_store_modify_erase both refuse when
table_for_record() is empty. The two cursor converters did not, so an
unmapped record resolved to an empty table, then to an empty cursor
name, and the generated lines read

```
PERFORM OPEN-.  
PERFORM FETCH-.
```

which the compiler rejects. Both now keep and comment the statement.

CORRECTION 4 - FIND FIRST declares <record> as OPTIONAL

FIND_FIRST_PATTERN allows 'FIND FIRST WITHIN <set>', which yields
record=None -> empty record -> the same broken PERFORM. Guarded.
"""

```
#  
# ON DB-REC-NOT-FOUND  
#  
def _convert_on_db_rec_not_found(self, stripped_line):  
    match = ON_DB_REC_NOT_FOUND_PATTERN.match(stripped_line)  
  
    if not match:  
        return None  
  
    statement = str(match.group("statement") or "").strip()  
  
    if not statement:  
        return [  
            SQLCODE_100_IF_OPEN,  
            SQLCODE_100_CONTINUE_BODY,  
            SQLCODE_100_IF_CLOSE,  
        ]  
  
    return [  
        SQLCODE_100_IF_OPEN,  
        f"{GUARDED_STATEMENT_INDENT}{statement}",  
        SQLCODE_100_IF_CLOSE,  
    ]  
  
#  
# OBTAIN ... CALC  
#  
def _convert_obtain_calc(  
    self,  
    upper,  
    stripped_line,  
    current_division,  
    sql_error_paragraph,  
):  
    match = OBTAIN_CALC_PATTERN.search(upper) or (
```

```

        OBTAIN_CALC_REVERSED_PATTERN.search(upper)
    )

    if not match:
        return None

    record_key = NameNormalizer.normalize(match.group("record"))
    record_display = NameNormalizer.to_cobol(record_key)

    if current_division != PROCEDURE_DIVISION_NAME:
        return [
            OBTAIN_CALC_OUTSIDE_PROCEDURE_TEMPLATE.format(
                line=stripped_line
            )
        ]

    if not record_key:
        return self._keep_and_comment_unmapped(
            record=record_key,
            stripped_line=stripped_line,
        )

    if not self.table_name_resolver.table_for_record(record_key):
        return self._keep_and_comment_unmapped(
            record=record_key,
            stripped_line=stripped_line,
        )

    return [
        REMOVED_OBTAIN_CALC_SELECT_TEMPLATE.format(record=record_display),
        DIRECT_UPDATE_COMPOSITE_KEY,
        CONTINUE_STATEMENT,
    ]

#
# OBTAIN FIRST / NEXT <record> WITHIN <set>
#
def _convert_obtain_first_next(
    self,
    upper,
    stripped_line,
    current_division,
):
    match = OBTAIN_FIRST_NEXT_PATTERN.search(upper)

    if not match:
        return None, NO_OPENED_SET

    record_key = NameNormalizer.normalize(match.group("record"))
    set_key = NameNormalizer.normalize(match.group("set"))

    record_display = NameNormalizer.to_cobol(record_key)
    set_display = NameNormalizer.to_cobol(set_key)

    if current_division != PROCEDURE_DIVISION_NAME:
        return [
            OBTAIN_OUTSIDE_PROCEDURE_TEMPLATE.format(line=stripped_line)
        ], NO_OPENED_SET

    cursor_name = self._resolve_cursor_name(record_key)

    if not cursor_name:
        return self._keep_and_comment_unmapped(
            record=record_key,
            stripped_line=stripped_line,
        ), NO_OPENED_SET

    if OBTAIN_FIRST_KEYWORD in upper:
        return [
            CONVERTED_OBTAIN_FIRST_TEMPLATE.format(
                record=record_display,
                set_name=set_display,
            ),
            PERFORM_OPEN_CURSOR_TEMPLATE.format(cursor=cursor_name),
            PERFORM_FETCH_CURSOR_TEMPLATE.format(cursor=cursor_name),
        ], set_key

```

```

        return [
            CONVERTED_OBTAIN_NEXT_TEMPLATE.format(
                record=record_display,
                set_name=set_display,
            ),
            PERFORM_FETCH_CURSOR_TEMPLATE.format(cursor=cursor_name),
        ], set_key

#
# FIND CURRENT
#
def _convert_find_current(self, upper, stripped_line, current_division):
    if not FIND_CURRENT_PATTERN.search(upper):
        return None

    return self._removed_idms_executable_lines(
        message=REMOVED_FIND_CURRENT_TEMPLATE.format(line=stripped_line),
        current_division=current_division,
    )

#
# FIND FIRST <record> WITHIN <set>
#
def _convert_find_first(self, upper, stripped_line, current_division):
    match = FIND_FIRST_PATTERN.search(upper)

    if not match:
        return None, NO_OPENED_SET

    record_key = NameNormalizer.normalize(match.group("record"))
    set_key = NameNormalizer.normalize(match.group("set"))

    record_display = NameNormalizer.to_cobol(record_key)
    set_display = NameNormalizer.to_cobol(set_key)

    if current_division != PROCEDURE_DIVISION_NAME:
        return [
            FIND_FIRST_OUTSIDE_PROCEDURE_TEMPLATE.format(
                line=stripped_line
            ),
            NO_OPENED_SET
        ], NO_OPENED_SET

    cursor_name = self._resolve_cursor_name(record_key)

    if not cursor_name:
        return self._keep_and_comment_unmapped(
            record=record_key,
            stripped_line=stripped_line,
        ), NO_OPENED_SET

    return [
        CONVERTED_FIND_FIRST_TEMPLATE.format(
            record=record_display,
            set_name=set_display,
        ),
        PERFORM_OPEN_CURSOR_TEMPLATE.format(cursor=cursor_name),
        PERFORM_FETCH_CURSOR_TEMPLATE.format(cursor=cursor_name),
    ], set_key

#
# STORE / MODIFY / ERASE
#
def _convert_store_modify_erase(
    self,
    upper,
    stripped_line,
    current_division,
    sql_error_paragraph,
):
    for pattern, operation_name, generator_method in (
        (STORE_PATTERN, OPERATION_STORE, self.sql_generator.insert),
        (MODIFY_PATTERN, OPERATION_MODIFY, self.sql_generator.update),
        (ERASE_PATTERN, OPERATION_ERASE, self.sql_generator.delete),
    ):
        match = pattern.search(upper)

```

```

        if not match:
            continue

        record_key = NameNormalizer.normalize(match.group("record"))

        if current_division != PROCEDURE_DIVISION_NAME:
            return [
                STORE_MODIFY_ERASE_OUTSIDE_PROCEDURE_TEMPLATE.format(
                    operation=operation_name,
                    line=stripped_line,
                )
            ]

        if not record_key:
            return self._keep_and_comment_unmapped(
                record=record_key,
                stripped_line=stripped_line,
            )

        if not self.table_name_resolver.table_for_record(record_key):
            return self._keep_and_comment_unmapped(
                record=record_key,
                stripped_line=stripped_line,
            )

        return generator_method(record_key)

    return None

#
# Shared cursor resolution
#
def _resolve_cursor_name(self, record_key: str) -> str:
    """Cursor name for a record, or '' when anything is unresolved.

    Returning '' is the single signal the two cursor converters use to
    fall back to keep-and-comment. Every intermediate step is checked,
    because an empty table silently produced an empty cursor name and
    the generated PERFORM had no paragraph to call.
    """
    if not record_key:
        return ""

    table_name = self.table_name_resolver.table_for_record(record_key)

    if not table_name:
        return ""

    cursor_name = self.cursor_name_resolver.cursor_name_from_table(
        table_name
    )

    return str(cursor_name or "").strip()

```

```
__all__ = ["DataStatementConverterMixin"]
```

```

=====
FILE: src\idms_db2_phase2\transformers\idms_statement\transformer_shared_mixin.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.idms_patterns import (
    DB_END_OF_SET_TOKEN_PATTERN,
    DB_REC_NOT_FOUND_TOKEN_PATTERN,
    IDMS_DECLARATIVE_OR_CONTROL_PATTERNS,
)
from rules.conversion_rules import (
    COMMENTED_IDMS_LINE_PREFIX,
    UNMAPPED_RECORD_MARKER_TEMPLATE,
)

```

```

from rules.idms_transformer_rules import (
    CONTINUE_STATEMENT,
    PROCEDURE_DIVISION_NAME,
    SQLCODE_END_TOKEN_REPLACEMENT,
)

```

```

class TransformerSharedMixin:

```

```

    """Shared helpers used by control/data statement converters.

```

```

    Owns the single, correct unmapped-record handling (Option B) with
    record-level tracking, and the generic token-replacement helpers.
    """

```

```

    def _keep_and_comment_unmapped(
        self,
        record: str,
        stripped_line: str,
    ) -> list[str]:

```

```

        """Register the record as unmapped and comment the verb line.

```

```

        Record-level Option B: the record is added to ``unmapped_records``
        so a downstream composer can comment EVERY line touching this
        record (not only this single IDMS verb line).
        """

```

```

        cobol_record = NameNormalizer.to_cobol(record)
        self.unmapped_records.add(cobol_record)
        return [
            UNMAPPED_RECORD_MARKER_TEMPLATE.format(record=cobol_record),
            f"{COMMENTED_IDMS_LINE_PREFIX} {stripped_line}",
        ]

```

```

    def _removed_idms_executable_lines(
        self,
        message: str,
        current_division: str,
    ) -> list[str]:

```

```

        if current_division == PROCEDURE_DIVISION_NAME:
            return [message, CONTINUE_STATEMENT]
        return [message]

```

```

    def _replace_idms_condition_tokens(
        self,
        line: str,
    ) -> list[str]:

```

```

        updated = DB_REC_NOT_FOUND_TOKEN_PATTERN.sub(
            SQLCODE_END_TOKEN_REPLACEMENT, line
        )
        updated = DB_END_OF_SET_TOKEN_PATTERN.sub(
            SQLCODE_END_TOKEN_REPLACEMENT, updated
        )
        return [updated]

```

```

    def _is_idms_declarative_or_control_statement(
        self,
        upper: str,
    ) -> bool:

```

```

        return any(
            pattern.search(upper)
            for pattern in IDMS_DECLARATIVE_OR_CONTROL_PATTERNS
        )

```

```

=====
FILE: src\idms_db2_phase2\transformers\idms_statement_transformer.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/transformers/idms_statement_transformer.py
# ACTION: REPLACE ENTIRE FILE

```

```

from __future__ import annotations

```

```

from idms_db2_phase2.transformers.idms_statement.control_statement_converter import (
    ControlStatementConverterMixin,
)

```

```

from idms_db2_phase2.transformers.idms_statement.data_statement_converter import (
    DataStatementConverterMixin,
)
from idms_db2_phase2.transformers.idms_statement.transformer_shared_mixin import (
    TransformerSharedMixin,
)

# Converter call signatures. The mixins are deliberately left untouched,
# so the dispatcher adapts to them rather than the other way round.
ARITY_LINE = 1          # (stripped_line)
ARITY_UPPER_LINE = 2     # (upper, stripped_line)
ARITY_WITH_DIVISION = 3  # (upper, stripped_line, current_division)
ARITY_WITH_SQL_ERROR = 4 # (upper, stripped_line, division, sql_error_paragraph)

NO_OPENED_SET = ""

class IdmsStatementTransformer(
    TransformerSharedMixin,
    ControlStatementConverterMixin,
    DataStatementConverterMixin,
):
    """Converts executable IDMS statements to DB2-compatible COBOL.

    Conversion logic only. Regex patterns live in patterns/idms_patterns.py;
    all *DB2: message text lives in rules/idms_transformer_messages.py.

    Feedback-driven rules:
    - OBTAIN CALC does not generate DB2 SELECT for update flow.
    - Direct UPDATE is enough when the WHERE has all composite PK/CALC keys.
    - Original IDMS OBTAIN CALC is still removed from final COBOL.
    - SqlGenerator methods already generate SQLCODE checks where required.
    - This transformer must not add duplicate SQLCODE wrappers.

    CORRECTION 1 - state leaked between programs
    -----
    ConversionComponentFactory builds ONE instance, and the batch runners
    convert every program in the folder inside a single process. Because
    `messages` and `unmapped_records` were only initialised in `__init__`,
    program 1's unmapped records were still present when program 2 ran,
    and the record-level composer commented records program 2 never
    referenced. reset() now clears per-program state; call it once per
    program, before the first transform_line().

    CORRECTION 2 - an empty result deleted a statement
    -----
    The dispatcher accepted any non-None result. A converter that returned
    an empty list therefore removed the source line silently. Only a
    NON-EMPTY result now counts as a conversion; anything else falls
    through to the next converter and finally to token replacement, so a
    statement can never disappear without a *DB2: comment explaining it.

    CORRECTION 3 - the converter chain was rebuilt per line
    -----
    Seven closures were allocated for every source line. The chain is a
    fixed table of bound methods built once in `__init__`.
    """

    def __init__(
        self,
        sql_generator,
        sql_error_generator,
        table_name_resolver,
        cursor_name_resolver,
    ) -> None:
        self.sql_generator = sql_generator
        self.sql_error_generator = sql_error_generator
        self.table_name_resolver = table_name_resolver
        self.cursor_name_resolver = cursor_name_resolver

        self.messages: list[str] = []
        # Collect every record with no DB2 target so a composer can comment
        # the WHOLE record block at record level.
        self.unmapped_records: set[str] = set()

        # Converters that never open a cursor set, in dispatch order.

```



```

# Order matters: OBTAIN CALC must be offered the line before
# OBTAIN FIRST / NEXT, and every declarative form before any
# executable form.
self._plain_converters = (
    (self._convert_declarative_or_control, ARITY_UPPER_LINE),
    (self._convert_finish_or_commit, ARITY_WITH_DIVISION),
    (self._convert_bind_ready_connect_disconnect, ARITY_WITH_DIVISION),
    (self._convert_status_abort_perform, ARITY_WITH_DIVISION),
    (self._convert_on_db_rec_not_found, ARITY_LINE),
    (self._convert_obtain_calc, ARITY_WITH_SQL_ERROR),
    (self._convert_find_current, ARITY_WITH_DIVISION),
)

# Converters that may also open a cursor set.
self._cursor_converters = (
    self._convert_obtain_first_next,
    self._convert_find_first,
)

#
# Per-program lifecycle
#
def reset(self) -> None:
    """Clear per-program state.

    MUST be called once per program by CobolTransformer.transform(),
    before the first transform_line(). Without it a batch run carries
    one program's diagnostics and unmapped records into the next.
    """
    self.messages = []
    self.unmapped_records = set()

#
# Public entry point
#
def transform_line(
    self,
    line: str,
    current_division: str,
    sql_error_paragraph: str,
) -> tuple[list[str], str]:
    stripped_line = str(line or "").strip()

    if not stripped_line:
        return [line], NO_OPENED_SET

    upper = stripped_line.upper()

    plain = self._run_plain_converters(
        upper=upper,
        stripped_line=stripped_line,
        current_division=current_division,
        sql_error_paragraph=sql_error_paragraph,
    )

    if plain is not None:
        return plain, NO_OPENED_SET

    cursor_lines, opened_set = self._run_cursor_converters(
        upper=upper,
        stripped_line=stripped_line,
        current_division=current_division,
    )

    if cursor_lines is not None:
        return cursor_lines, opened_set

    store_lines = self._convert_store_modify_erase(
        upper,
        stripped_line,
        current_division,
        sql_error_paragraph,
    )

    if self._is_converted(store_lines):
        return store_lines, NO_OPENED_SET

```

```

        return self._replace_idms_condition_tokens(line), NO_OPENED_SET

#
# Dispatch
#
def _run_plain_converters(
    self,
    *,
    upper: str,
    stripped_line: str,
    current_division: str,
    sql_error_paragraph: str,
) -> list[str] | None:
    """First converter that returns a non-empty result wins."""
    for converter, arity in self._plain_converters:
        result = self._invoke(
            converter=converter,
            arity=arity,
            upper=upper,
            stripped_line=stripped_line,
            current_division=current_division,
            sql_error_paragraph=sql_error_paragraph,
        )

        if self._is_converted(result):
            return result

    return None

def _run_cursor_converters(
    self,
    *,
    upper: str,
    stripped_line: str,
    current_division: str,
) -> tuple[list[str] | None, str]:
    """Converters that may also report the IDMS set they opened."""
    for converter in self._cursor_converters:
        result, opened_set = converter(
            upper,
            stripped_line,
            current_division,
        )

        if self._is_converted(result):
            return result, str(opened_set or NO_OPENED_SET)

    return None, NO_OPENED_SET

@staticmethod
def _invoke(
    *,
    converter,
    arity: int,
    upper: str,
    stripped_line: str,
    current_division: str,
    sql_error_paragraph: str,
):
    if arity == ARITY_LINE:
        return converter(stripped_line)

    if arity == ARITY_UPPER_LINE:
        return converter(upper, stripped_line)

    if arity == ARITY_WITH_DIVISION:
        return converter(upper, stripped_line, current_division)

    return converter(
        upper,
        stripped_line,
        current_division,
        sql_error_paragraph,
    )

```

```

@staticmethod
def _is_converted(result) -> bool:
    """True only for a real replacement.

    None means "this converter does not handle the line".
    An EMPTY list used to mean the same thing by accident, but the old
    dispatcher accepted it and the source statement vanished with no
    *DB2: comment. Empty is now treated as "not handled".
    """
    return result is not None and len(result) > 0

__all__ = ["IdmsStatementTransformer"]

=====
FILE: src\idms_db2_phase2\transformers\lrf_path_expander.py
ENCODING: utf-8
=====

# LOCATION: src/idms_db2_phase2/transformers/lrf_path_expander.py
# ACTION: REPLACE ENTIRE FILE
"""Expands LRF syntax into the classic IDMS syntax the converter knows.

NO-LRF CONTRACT
-----
With no logical record metadata, or with metadata but no LRF syntax in the
program, expand() returns the source text byte-for-byte. The converter
therefore behaves exactly as it did before LRF support existed.

NAME FORM
-----
Generated names are COBOL form (UPPERCASE, hyphens). NameNormalizer is not
used here: normalize() yields DB2 column form and would emit VMBTL03_R01.

COMMENT FORM
-----
A generated comment line carries '*' in the indicator column and its
marker starts immediately at column 8, so the output reads

    *DB2: Expanded LRF ...
    *DB2-KEEP: ...

which matches the marker text declared in rules/.
"""

from __future__ import annotations

from idms_db2_phase2.services.fixed_format_line_service import (
    FixedFormatLineService,
)
from patterns.lrf_program_patterns import (
    BARE_OF_LR_PATTERN,
    LR_ERASE_WHERE_PATTERN,
    LR_OBTAIN_WHERE_PATTERN,
    LR_STATUS_CONDITION_PATTERN,
    LR_STATUS_TOKEN_PATTERN,
    OF_LR_SUFFIX_PATTERN,
)
from rules.lrf_conversion_rules import (
    ENABLE_LRF_PATH_EXPANSION,
    END_OF_SET_TOKEN,
    EXPANDED_OBTAIN_TEMPLATE,
    EXPANDED_PARENT_TEMPLATE,
    FIND_FIRST_TEMPLATE,
    LRF_EXPANSION_MESSAGES,
    OBTAIN_FIRST_TEMPLATE,
    OBTAIN_NEXT_TEMPLATE,
    RECORD_NOT_FOUND_TOKEN,
    REQUIRE_MAPPED_DRIVING_RECORD,
    SKIPPED_NO_PATH_TEMPLATE,
    SKIPPED_NO_RECORD_TEMPLATE,
    SKIPPED_UNMAPPED_TEMPLATE,
    STATEMENT_TERMINATOR,
)

```

```

PROCEDURE_DIVISION_TOKEN = "PROCEDURE DIVISION"
COMMENT_INDICATORS = ("*", "/", "D", "-")
COMMENT_PREFIX_COLUMNS = "      "
MODE_NEXT = "NEXT"
DEFAULT_INDENT = "      "

```

```

def cobol_name(value: str) -> str:
    """UPPERCASE COBOL/IDMS form: hyphens, never underscores."""
    return str(value or "").strip().upper().replace("_", "-")

```

```

class LrfPathExpander:
    """Rewrites LRF program syntax into classic IDMS syntax."""

    def __init__(
        self,
        *,
        path_resolver=None,
        fixed_format: FixedFormatLineService | None = None,
    ) -> None:
        self.path_resolver = path_resolver
        self.fixed_format = fixed_format or FixedFormatLineService()
        self.messages: list[str] = []

    # ----- public
    def expand(self, cobol_text: str) -> str:
        self.messages = []

        text = str(cobol_text or "")
        if not text.strip() or not ENABLE_LRF_PATH_EXPANSION:
            return text

        if self.path_resolver is None or self.path_resolver.is_empty():
            if self._contains_lrf_syntax(text):
                self._log("no_logical_records")
            return text

        if not self._contains_lrf_syntax(text):
            return text

        lines = text.replace("\r\n", "\n").replace("\r", "\n").split("\n")

        output: list[str] = []
        in_procedure = False
        expanded = 0
        skipped = 0
        status_rewrites = 0
        qualifier_strips = 0

        for line in lines:
            logical = self._logical(line)
            upper = logical.upper()

            if PROCEDURE_DIVISION_TOKEN in upper:
                in_procedure = True
                output.append(line)
                continue

            if not in_procedure or self._is_comment(line) or not logical:
                output.append(line)
                continue

            obtain = LR_OBTAIN_WHERE_PATTERN.match(logical)
            if obtain:
                rendered, ok = self._expand_obtain(line, obtain)
                output.extend(rendered)
                expanded += 1 if ok else 0
                skipped += 0 if ok else 1
                continue

            erase = LR_ERASE_WHERE_PATTERN.match(logical)
            if erase:
                rendered, ok = self._expand_erase(line, erase)
                output.extend(rendered)
                expanded += 1 if ok else 0

```

```

        skipped += 0 if ok else 1
        continue

    new_logical, hits = self._rewrite_lr_status(logical)
    status_rewrites += hits

    new_logical, strips = self._strip_of_lr(new_logical)
    qualifier_strips += strips

    if new_logical != logical:
        output.append(self._replace_body(line, new_logical))
    else:
        output.append(line)

    if status_rewrites:
        self._log("status_rewritten", count=status_rewrites)
    if qualifier_strips:
        self._log("qualifier_stripped", count=qualifier_strips)
    if expanded or skipped:
        self._log("summary", expanded=expanded, skipped=skipped)

    if not (expanded or skipped or status_rewrites or qualifier_strips):
        return text

    return "\n".join(output).rstrip() + "\n"

# ----- expansion
def _expand_obtain(self, line: str, match) -> tuple[list[str], bool]:
    mode = str(match.group("mode") or "").upper()
    lr_name = cobol_name(match.group("lr"))
    keyword = cobol_name(match.group("keyword"))

    plan = self.path_resolver.plan_for_keyword(keyword)

    if plan is None:
        self._log("keyword_not_found", keyword=keyword)
        return self._keep(
            line, SKIPPED_NO_PATH_TEMPLATE.format(keyword=keyword)
        ), False

    if not plan.is_usable:
        self._log("keyword_not_found", keyword=keyword)
        return self._keep(
            line, SKIPPED_NO_RECORD_TEMPLATE.format(keyword=keyword)
        ), False

    record = plan.driving.record_name
    within = plan.driving.within_name

    if REQUIRE_MAPPED_DRIVING_RECORD and not self.path_resolver.record_is_mapped(record):
        self._log("record_unmapped", record=record, keyword=keyword)
        return self._keep(
            line, SKIPPED_UNMAPPED_TEMPLATE.format(record=record)
        ), False

    rendered: list[str] = []
    rendered.extend(
        self._comment(line, EXPANDED_OBTAIN_TEMPLATE.format(
            mode=mode, lr=lr_name, keyword=keyword,
            record=record, within=within,
        ))
    )

    if plan.parent is not None and mode != MODE_NEXT:
        parent = plan.parent
        rendered.extend(
            self._comment(line, EXPANDED_PARENT_TEMPLATE.format(
                keyword=keyword, record=parent.record_name,
            ))
        )
    rendered.extend(
        self._statement(line, FIND_FIRST_TEMPLATE.format(
            record=parent.record_name,
            within=parent.within_name,
        ))
    )

```

```

        self._log(
            "parent_emitted",
            record=parent.record_name,
            within=parent.within_name,
            keyword=keyword,
        )

    template = (
        OBTAIN_NEXT_TEMPLATE if mode == MODE_NEXT
        else OBTAIN_FIRST_TEMPLATE
    )
    rendered.extend(
        self._statement(line, template.format(record=record, within=within))
    )

    self._log(
        "expanded",
        mode=mode, lr=lr_name, keyword=keyword,
        record=record, within=within,
    )
    return rendered, True

def _expand_erase(self, line: str, match) -> tuple[list[str], bool]:
    keyword = cobol_name(match.group("keyword"))
    plan = self.path_resolver.plan_for_keyword(keyword)

    if plan is None or not plan.is_usable:
        self._log("keyword_not_found", keyword=keyword)
        return self._keep(
            line, SKIPPED_NO_PATH_TEMPLATE.format(keyword=keyword)
        ), False

    record = plan.driving.record_name

    if REQUIRE_MAPPED_DRIVING_RECORD and not self.path_resolver.record_is_mapped(record):
        self._log("record_unmapped", record=record, keyword=keyword)
        return self._keep(
            line, SKIPPED_UNMAPPED_TEMPLATE.format(record=record)
        ), False

    rendered = self._comment(line, EXPANDED_OBTAIN_TEMPLATE.format(
        mode="ERASE", lr=plan.logical_record, keyword=keyword,
        record=record, within=plan.driving.within_name,
    ))
    rendered.extend(self._statement(line, f"ERASE {record}"))
    return rendered, True

# ----- status/qualifier
@staticmethod
def _rewrite_lr_status(logical: str) -> tuple[str, int]:
    hits = 0

    def _replace(match) -> str:
        nonlocal hits
        hits += 1
        operator = str(match.group("operator") or "").upper()
        literal = cobol_name(match.group("literal"))
        token = (
            RECORD_NOT_FOUND_TOKEN
            if "NOT-FND" in literal or "NOT-FOUND" in literal
            else END_OF_SET_TOKEN
        )
        return f"NOT {token}" if operator.startswith("NOT") else token

    return LR_STATUS_CONDITION_PATTERN.sub(_replace, logical), hits

@staticmethod
def _strip_of_lr(logical: str) -> tuple[str, int]:
    first, a = OF_LR_SUFFIX_PATTERN.subn(r"\g<head>", logical)
    second, b = BARE_OF_LR_PATTERN.subn("", first)
    return second, a + b

# ----- render
def _statement(self, template_line: str, body: str) -> list[str]:
    text = body.rstrip()
    if not text.endswith(STATEMENT_TERMINATOR):

```

```

        text = f"{text}{STATEMENT_TERMINATOR}"
        indent = self._body_indent(template_line)
        return self._emit(template_line, f"{indent}{text}", indicator=" ")

def _comment(self, template_line: str, body: str) -> list[str]:
    """Comment body starts at column 8 - no indent, no leading star."""
    text = str(body or "").lstrip("*").strip()
    return self._emit(template_line, text, indicator="*")

def _keep(self, line: str, marker: str) -> list[str]:
    rendered = self._comment(line, marker)
    rendered.extend(self._comment(line, self._logical(line)))
    return rendered

def _emit(self, template_line: str, body: str, indicator: str) -> list[str]:
    if not self._is_fixed(template_line):
        if indicator == "*":
            return [f"{COMMENT_PREFIX_COLUMNS}*{body}"]
        return [body]

    left, _ind, _old, right = self.fixed_format.split(template_line)
    try:
        return self.fixed_format.emit(left, indicator, body, right)
    except Exception: # noqa: BLE001
        return [template_line]

def _replace_body(self, line: str, new_body: str) -> str:
    indent = self._body_indent(line)
    text = f"{indent}{new_body.strip()}"
    if not self._is_fixed(line):
        return text
    return self.fixed_format.replace_body(line, text)

# ----- helpers
def _logical(self, line: str) -> str:
    try:
        return str(self.fixed_format.logical(line) or "").strip()
    except Exception: # noqa: BLE001
        return str(line or "").strip()

def _is_fixed(self, line: str) -> bool:
    try:
        return bool(self.fixed_format.is_fixed_line(line))
    except Exception: # noqa: BLE001
        return False

def _body_indent(self, line: str) -> str:
    if not self._is_fixed(line):
        text = str(line or "")
        return " " * (len(text) - len(text.lstrip(" ")))
    try:
        _l, _i, body, _r = self.fixed_format.split(line)
        return " " * (len(body) - len(body.lstrip(" ")))
    except Exception: # noqa: BLE001
        return DEFAULT_INDENT

@staticmethod
def _is_comment(line: str) -> bool:
    text = str(line or "")
    if len(text) >= 7 and text[:6].strip().isdigit():
        return text[6] in COMMENT_INDICATORS
    return text.lstrip().startswith(COMMENT_INDICATORS)

@staticmethod
def _contains_lrf_syntax(text: str) -> bool:
    upper = str(text or "").upper()
    if LR_STATUS_TOKEN_PATTERN.search(upper):
        return True
    if " OF LR" in upper:
        return True
    for line in upper.splitlines():
        if LR_OBTAIN_WHERE_PATTERN.match(line.strip()):
            return True
        if LR_ERASE_WHERE_PATTERN.match(line.strip()):
            return True
    return False

```

```

def _log(self, key: str, **values) -> None:
    template = LRF_EXPANSION_MESSAGES.get(key, "")
    if template:
        self.messages.append(template.format(**values))

__all__ = ["LrfPathExpander", "cobol_name"]

=====
FILE: src\idms_db2_phase2\transformers\qualified_reference_transformer.py
ENCODING: utf-8
=====

"""
Qualified IDMS field reference transformer.

Rewrites safe qualified references only:

- FIELD OF IDMS-RECORD
- FIELD IN IDMS-RECORD

Bare words are not globally rewritten.
"""

from patterns.field_usage_patterns import QUALIFIED_REFERENCE_PATTERN
from rules.field_mapping_rules import QUALIFIED_REFERENCE_REWRITE_RULES

class QualifiedReferenceTransformer:
    """
    Rewrites qualified IDMS field references to DB2 DCLGEN host references when
    mapping metadata exists.
    """

    def __init__(
        self,
        mapping_repository,
        dclgen_repository,
    ) -> None:
        self.mapping_repository = mapping_repository
        self.dclgen_repository = dclgen_repository

    def transform_line(
        self,
        line: str,
    ) -> str:
        if not line:
            return ""

        return QUALIFIED_REFERENCE_PATTERN.sub(
            self._replace_reference,
            line,
        )

    def _replace_reference(
        self,
        match,
    ) -> str:
        field_name = match.group("field")
        record_name = match.group("record")

        mapped_column = self.mapping_repository.find_column_mapping(
            record_name=record_name,
            idms_field_name=field_name,
        )

        if not mapped_column:
            return match.group(0)

        host_reference = self.dclgen_repository.find_host_reference(
            table_name=mapped_column.db2_table_name,
            column_name=mapped_column.db2_column_name,
        )

```



```

    if not host_reference:
        return match.group(0)

    return host_reference

```

```

=====
FILE: src\idms_db2_phase2\ui\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: src\idms_db2_phase2\ui\conversion_actions.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/ui/conversion_actions.py
# ACTION: REPLACE ENTIRE FILE

```

```

from __future__ import annotations

```

```

import re

```

```

import streamlit as st

```

```

from idms_db2_phase2.domain.models import ConversionInput
from idms_db2_phase2.orchestration.conversion_service import ConversionService
from idms_db2_phase2.parsers.cobol_parser import CobolParser
from idms_db2_phase2.services.name_derivation_resolver import NameDerivationResolver
from idms_db2_phase2.ui.file_name_utils import build_converted_cobol_file_name
from patterns.lrf_patterns import COPY_IDMS_LR_PATTERN
from rules.lrf_rules import (
    DIAG_LRF_NO_RECORDS_IN_SCOPE,
    DIAG_LRF_RECORDS_IN_SCOPE_TEMPLATE,
    LRF_VALIDATION_MESSAGES,
)

```

```

# Detects any IDMS write verb in the SOURCE program (STORE/MODIFY/ERASE).
# If present, this is an update program and needs the update post-process.
_IDMS_WRITE_PATTERN = re.compile(
    r"^\s*(?:\d{6}\s+)?(STORE|MODIFY|ERASE)\b",
    flags=re.IGNORECASE | re.MULTILINE,
)

```

```

def resolve_target_program_id(source_text: str) -> str:
    """Derive the target PROGRAM-ID from the uploaded COBOL source.

    Applies the project's VM...BD... -> VMDZ... rule (e.g. VM4BD420 -> VMDZ4420).
    Returns "" when no PROGRAM-ID is detected, so the converter preserves the
    source PROGRAM-ID instead of forcing a literal.
    """
    source_id = CobolParser().program_id(source_text)
    if not source_id:
        return ""
    return NameDerivationResolver().derive(source_id)

```

```

def _is_update_program(source_text: str) -> bool:
    """True when the source contains IDMS write verbs (STORE/MODIFY/ERASE)."""
    return bool(_IDMS_WRITE_PATTERN.search(str(source_text or "")))

```

```

# -----
# LRF (Logical Record Facility)
# -----

```

```

def _logical_records() -> list:
    """LRF logical records currently held in session state.

    Read with .get() so a session created before the LRF uploader existed
    still converts without raising a KeyError.
    """

```

```

return list(st.session_state.get("logical_records") or [])

def _source_uses_logical_records(source_text: str) -> list[str]:
    """Logical record names the SOURCE program copies via COPY IDMS LR."""
    names: list[str] = []
    for raw_line in str(source_text or "").splitlines():
        match = COPY_IDMS_LR_PATTERN.search(raw_line)
        if not match:
            continue
        name = str(match.group("lr") or "").strip().upper()
        if name and name not in names:
            names.append(name)
    return names

def _narrate_lrf(
    *,
    source_text: str,
    logical_records: list,
    diagnostics: list[str],
) -> None:
    """Record what LRF metadata this conversion will actually see.

    Never blocks the run. A program that copies a logical record without a
    matching LRF subschema is reported once, so the gap is visible in the
    Validation tab instead of failing silently.
    """
    if logical_records:
        names = ", ".join(
            str(getattr(record, "logical_record_name", "") or "")
            for record in logical_records
        )
        diagnostics.append(
            DIAG_LRF_RECORDS_IN_SCOPE_TEMPLATE.format(
                count=len(logical_records),
                names=names,
            )
        )
    else:
        diagnostics.append(DIAG_LRF_NO_RECORDS_IN_SCOPE)

    used = _source_uses_logical_records(source_text)
    if used and not logical_records:
        diagnostics.append(LRF_VALIDATION_MESSAGES["missing_lrf"])

def _apply_update_postprocess(
    *,
    source_text: str,
    converted_cobol: str,
    copybook_fields: list,
    dclgen_columns: list,
    target_program_id: str,
    diagnostics: list[str],
) -> str:
    """Run the update-only enhancement pipeline (restart flow, SQLERROR fix,
    DCLGEN includes, audit moves, CHAR/date host conversion, final cleanup).

    Mirrors testing/update/update_postprocess_pipeline.enhance_update_program
    but is import-light for the Streamlit context. Any failure returns the
    base converted COBOL unchanged.
    """
    try:
        from idms_db2_phase2.composers.fixed_format_composer import (
            FixedFormatComposer,
        )
        from idms_db2_phase2.composers.procedure_indent_normalizer import (
            ProcedureIndentNormalizer,
        )
        from idms_db2_phase2.postprocess.update_char_literal_quoter import (
            UpdateCharLiteralQuoter,
        )
        from idms_db2_phase2.postprocess.update_date_host_converter import (
            UpdateDateHostConverter,
        )

```

```

from idms_db2_phase2.postprocess.update_final_cleanup import (
    UpdateFinalCleanup,
)
from idms_db2_phase2.postprocess.update_input_record_renamer import (
    UpdateInputRecordRenamer,
)
from idms_db2_phase2.postprocess.update_program_enhancer import (
    UpdateProgramEnhancer,
)
from idms_db2_phase2.services.fixed_format_line_service import (
    FixedFormatLineService,
)
from idms_db2_phase2.services.update_audit_move_inserter import (
    UpdateAuditMoveInserter,
)
from idms_db2_phase2.services.update_cobol_final_cleanup_utils import (
    UpdateCobolFinalCleanupUtils,
)
from patterns.batch_runner_patterns import COPY_NAME_PATTERN
from idms_db2_phase2.services.name_derivation_resolver import (
    NameDerivationResolver,
)

enhanced = UpdateProgramEnhancer().enhance(
    source_cobol=source_text,
    converted_cobol=converted_cobol,
    copybook_fields=copybook_fields,
    dclgen_columns=dclgen_columns,
    target_program_id=target_program_id,
)
diagnostics.extend(enhanced.diagnostics)

fixed = FixedFormatComposer().format(enhanced.converted_cobol)
cleaned = UpdateFinalCleanup().apply(fixed)
formatted = FixedFormatComposer().format(cleaned)

_ff = FixedFormatLineService()
with_audit = UpdateAuditMoveInserter(
    fixed_format=_ff,
    utils=UpdateCobolFinalCleanupUtils(_ff),
).insert_update_audit_moves(formatted)

with_char = UpdateCharLiteralQuoter(dclgen_columns).apply(
    with_audit, diagnostics
)
with_dates = UpdateDateHostConverter(dclgen_columns).apply(
    with_char, diagnostics
)

# Rename original input-record refs to the derived VM...BD...->VMDZ name.
deriver = NameDerivationResolver()
old_record = new_record = ""
for raw in COPY_NAME_PATTERN.findall(str(with_dates or "")):
    name = str(raw or "").strip().upper()
    if name and deriver.is_derivable(name):
        derived = deriver.derive(name)
        if derived != name:
            old_record, new_record = name, derived
            break

if old_record and new_record and old_record != new_record:
    with_renamed = UpdateInputRecordRenamer().apply(
        with_dates,
        old_name=old_record,
        new_name=new_record,
        diagnostics=diagnostics,
    )
else:
    with_renamed = with_dates

formatted_final = FixedFormatComposer().format(with_renamed)

_indent = ProcedureIndentNormalizer()
normalized = _indent.compose(formatted_final)
if getattr(_indent, "messages", None):
    diagnostics.extend(_indent.messages)

```

```

        return FixedFormatComposer().format(normalized)

except Exception as exc: # noqa: BLE001
    diagnostics.append(
        "Update postprocess enhancement skipped. Base converted COBOL "
        f"will be used. Reason: {exc}"
    )
    return converted_cobol

def generate_db2_cobol() -> None:
    if not _validate_generation_inputs():
        return

    idms_cobol_text = st.session_state.idms_cobol_text
    target_program_id = resolve_target_program_id(idms_cobol_text)

    # LRF metadata is optional. Narrate it before conversion so the log
    # reads in the same order the pipeline consumed its inputs.
    logical_records = _logical_records()
    lrf_messages: list[str] = []
    _narrate_lrf(
        source_text=idms_cobol_text,
        logical_records=logical_records,
        diagnostics=lrf_messages,
    )

    service = ConversionService()
    result = service.convert(
        ConversionInput(
            sheet_mapping_rows=st.session_state.sheet_mapping_rows,
            dclgen_columns=st.session_state.dclgen_columns,
            copybook_fields=st.session_state.copybook_fields,
            logical_records=logical_records,
            idms_cobol_text=idms_cobol_text,
            target_program_id=target_program_id,
        )
    )

    converted_cobol = result.converted_cobol or ""
    validation_messages = lrf_messages + list(result.validation_messages or [])

    program_kind = "retrieval"
    if converted_cobol and _is_update_program(idms_cobol_text):
        program_kind = "update"
        converted_cobol = _apply_update_postprocess(
            source_text=idms_cobol_text,
            converted_cobol=converted_cobol,
            copybook_fields=st.session_state.copybook_fields,
            dclgen_columns=st.session_state.dclgen_columns,
            target_program_id=target_program_id,
            diagnostics=validation_messages,
        )

    st.session_state.converted_cobol = converted_cobol
    st.session_state.validation_messages = validation_messages
    st.session_state.operations = result.operations or []

    st.session_state.converted_cobol_file_name = build_converted_cobol_file_name(
        target_program_id=target_program_id,
        source_file_name=st.session_state.idms_cobol_source_name,
    )

    st.session_state.generated = bool(st.session_state.converted_cobol)

    if st.session_state.generated:
        st.success(
            f"DB2 COBOL generated ({program_kind} program, PROGRAM-ID: "
            f"{target_program_id or '(preserved from source)'}"
            f"{_lrf_suffix(logical_records)}). "
            "Download is now available in the Main tab."
        )
    else:
        st.warning(
            "DB2 COBOL was not generated. "

```

```

        "Review the Validation and Diagnostics tabs."
    )

def _lrf_suffix(logical_records: list) -> str:
    """Short LRF note appended to the success banner."""
    if not logical_records:
        return ""
    return f", LRF logical records: {len(logical_records)}"

def _validate_generation_inputs() -> bool:
    if not st.session_state.loaded:
        st.warning("Load and analyze valid inputs before generating DB2 COBOL.")
        return False

    if not st.session_state.sheet_mapping_rows:
        st.error("Sheet Mapping rows are empty. Reload Sheet Mapping input.")
        return False

    if not st.session_state.dclgen_columns:
        st.error("DCLGEN columns are empty. Reload DCLGEN input.")
        return False

    if not st.session_state.idms_cobol_text.strip():
        st.error("IDMS COBOL source is empty. Reload COBOL input.")
        return False

    return True

```

```

=====
FILE: src\idms_db2_phase2\ui\file_name_utils.py
ENCODING: utf-8
=====

```

```

from __future__ import annotations

import re
from pathlib import Path

def build_converted_cobol_file_name(
    target_program_id: str,
    source_file_name: str,
) -> str:
    target_stem = sanitize_file_stem(target_program_id)

    if target_stem:
        return f"{target_stem}_db2.cbl"

    source_name = str(source_file_name or "").strip()

    if source_name:
        source_stem = sanitize_file_stem(Path(source_name).stem)

        if source_stem:
            return f"{source_stem}_db2.cbl"

    return "converted_db2_cobol.cbl"

def sanitize_file_stem(value: str) -> str:
    text = str(value or "").strip()

    if not text:
        return ""

    text = re.sub(r"^[A-Za-z0-9_-]+", "_", text)
    text = re.sub(r"_+", "_", text)

    return text.strip("_")

```

```
=====
FILE: src\idms_db2_phase2\ui\input_loader.py
ENCODING: utf-8
=====
```

```
from __future__ import annotations

import streamlit as st

from idms_db2_phase2.infrastructure.file_loader import FileLoader
from idms_db2_phase2.parsers.copybook_parser import CopybookParser
from idms_db2_phase2.parsers.dclgen_parser import DclgenParser
from idms_db2_phase2.parsers.sheet_mapping_parser import SheetMappingParser
from idms_db2_phase2.ui.session_state import reset_generated_output_state

# LOCATION: src/idms_db2_phase2/ui/input_loader.py
# ACTION: ADD these imports below the existing imports

from idms_db2_phase2.parsers.lrf_parser import LrfParser
from rules.lrf_rules import (
    DIAG_LRF_FILE_COUNT_TEMPLATE,
    DIAG_LRF_FILE_TEMPLATE,
    DIAG_LRF_NOT_UPLOADED,
    DIAG_LRF_PARSED_ZERO,
    DIAG_LRF_PARSE_FAILED_TEMPLATE,
    DIAG_LRF_READ_FAILED_TEMPLATE,
    DIAG_LRF_TEXT_LEN_TEMPLATE,
    LRF_SOURCE_LABEL,
)

# LOCATION: src/idms_db2_phase2/ui/input_loader.py
# ACTION: REPLACE the whole load_and_analyze_inputs function

def load_and_analyze_inputs(
    sheet_mapping_file,
    dclgen_files,
    copybook_files,
    lrf_files=None,
    idms_cobol_source_file=None,
) -> None:
    diagnostics: list[str] = []
    uploaded_file_names: dict[str, object] = {}

    file_loader = FileLoader()
    sheet_mapping_parser = SheetMappingParser()
    dclgen_parser = DclgenParser()
    copybook_parser = CopybookParser()
    lrf_parser = LrfParser()

    diagnostics.append("START LOAD INPUTS")

    sheet_rows = _load_sheet_mapping(
        sheet_mapping_file=sheet_mapping_file,
        parser=sheet_mapping_parser,
        diagnostics=diagnostics,
        uploaded_file_names=uploaded_file_names,
    )
    dclgen_columns = _load_dclgen_files(
        dclgen_files=dclgen_files,
        file_loader=file_loader,
        parser=dclgen_parser,
        diagnostics=diagnostics,
        uploaded_file_names=uploaded_file_names,
    )
    copybook_fields = _load_copybook_files(
        copybook_files=copybook_files,
        file_loader=file_loader,
        parser=copybook_parser,
        diagnostics=diagnostics,
        uploaded_file_names=uploaded_file_names,
    )

    # ---- LRF sits between Copybook metadata and the COBOL source.
    logical_records = _load_lrf_files(
        lrf_files=lrf_files,
        file_loader=file_loader,
```

```

        parser=lrf_parser,
        diagnostics=diagnostics,
        uploaded_file_names=uploaded_file_names,
    )

    idms_cobol_text, idms_cobol_source_name = _load_idms_cobol_source(
        idms_cobol_source_file=idms_cobol_source_file,
        file_loader=file_loader,
        diagnostics=diagnostics,
        uploaded_file_names=uploaded_file_names,
    )

    st.session_state.sheet_mapping_rows = sheet_rows
    st.session_state.dclgen_columns = dclgen_columns
    st.session_state.copybook_fields = copybook_fields
    st.session_state.logical_records = logical_records
    st.session_state.idms_cobol_text = idms_cobol_text
    st.session_state.idms_cobol_source_name = idms_cobol_source_name

    reset_generated_output_state()

    st.session_state.diagnostics = diagnostics
    st.session_state.uploaded_file_names = uploaded_file_names

    required_inputs_loaded = (
        bool(sheet_rows)
        and bool(dclgen_columns)
        and bool(idms_cobol_text.strip())
    )

    st.session_state.loaded = required_inputs_loaded

    if required_inputs_loaded:
        st.success("Inputs loaded and analyzed.")
    else:
        st.warning(
            "Inputs were processed, but one or more required inputs are missing "
            "or parsed as empty. Review the Diagnostics tab."
        )

def _load_sheet_mapping(
    *,
    sheet_mapping_file,
    parser: SheetMappingParser,
    diagnostics: list[str],
    uploaded_file_names: dict[str, object],
) -> list:
    if sheet_mapping_file is None:
        diagnostics.append("Sheet Mapping file not uploaded.")
        return []

    uploaded_file_names["sheet_mapping_file"] = str(sheet_mapping_file.name or "")

    try:
        rows = parser.parse_uploaded_file(sheet_mapping_file)
        diagnostics.append(f"Sheet Mapping uploaded: {sheet_mapping_file.name}")
        diagnostics.append(f"Sheet Mapping parsed rows: {len(rows)}")

        if hasattr(parser, "diagnostics"):
            diagnostics.extend(parser.diagnostics)

        return rows

    except Exception as exc:
        diagnostics.append(f"Sheet Mapping parse failed: {exc}")
        return []

def _load_dclgen_files(
    *,
    dclgen_files,
    file_loader: FileLoader,
    parser: DclgenParser,
    diagnostics: list[str],
    uploaded_file_names: dict[str, object],
) -> list:

```

```

if not dclgen_files:
    diagnostics.append("DCLGEN file not uploaded.")
    return []

diagnostics.append(f"DCLGEN file count: {len(dclgen_files)}")

file_names: list[str] = []
texts: list[str] = []

for index, file in enumerate(dclgen_files, start=1):
    file_name = str(file.name or "")
    file_names.append(file_name)

    try:
        text = file_loader.read_uploaded_text(file)
        texts.append(text)
        diagnostics.append(f"DCLGEN {index}: {file_name}")
        diagnostics.append(f"DCLGEN {index} text length: {len(text)}")

    except Exception as exc:
        diagnostics.append(f"DCLGEN read failed for {file_name}: {exc}")

uploaded_file_names["dclgen_files"] = file_names

try:
    if hasattr(parser, "parse_many_texts"):
        columns = parser.parse_many_texts(texts)
    else:
        columns = []

    for index, text in enumerate(texts, start=1):
        parsed_columns = parser.parse(
            text=text,
            source_label=f"DCLGEN file {index}",
        )
        columns.extend(parsed_columns)

    diagnostics.append(f"DCLGEN parsed columns: {len(columns)}")

    if hasattr(parser, "diagnostics"):
        diagnostics.extend(parser.diagnostics)

    return columns

except Exception as exc:
    diagnostics.append(f"DCLGEN parse failed: {exc}")
    return []

def _load_copybook_files(
    *,
    copybook_files,
    file_loader: FileLoader,
    parser: CopybookParser,
    diagnostics: list[str],
    uploaded_file_names: dict[str, object],
) -> list:
    if not copybook_files:
        diagnostics.append("Copybook file not uploaded. Continuing without copybook.")
        return []

    diagnostics.append(f"Copybook file count: {len(copybook_files)}")

    file_names: list[str] = []
    text_parts: list[str] = []

    for index, file in enumerate(copybook_files, start=1):
        file_name = str(file.name or "")
        file_names.append(file_name)

        try:
            text = file_loader.read_uploaded_text(file)
            text_parts.append(text)
            diagnostics.append(f"Copybook {index}: {file_name}")
            diagnostics.append(f"Copybook {index} text length: {len(text)}")

```



```

        except Exception as exc:
            diagnostics.append(f"Copybook read failed for {file_name}: {exc}")

uploaded_file_names["copybook_files"] = file_names

copybook_text = "\n".join(text_parts)

if not copybook_text.strip():
    diagnostics.append("Copybook parsed fields: 0")
    return []

try:
    try:
        fields = parser.parse(
            text=copybook_text,
            source_label="Copybook files",
        )
    except TypeError:
        fields = parser.parse(copybook_text)

    diagnostics.append(f"Copybook parsed fields: {len(fields)}")

    if hasattr(parser, "diagnostics"):
        diagnostics.extend(parser.diagnostics)

    return fields

except Exception as exc:
    diagnostics.append(f"Copybook parse failed: {exc}")
    diagnostics.append("Copybook parsed fields: 0")
    return []

def _load_idms_cobol_source(
    *,
    idms_cobol_source_file,
    file_loader: FileLoader,
    diagnostics: list[str],
    uploaded_file_names: dict[str, object],
) -> tuple[str, str]:
    if idms_cobol_source_file is None:
        diagnostics.append("IDMS COBOL source file not uploaded.")
        return "", ""

    source_name = str(idms_cobol_source_file.name or "")
    uploaded_file_names["idms_cobol_source_file"] = source_name

    try:
        text = file_loader.read_uploaded_text(idms_cobol_source_file)
        diagnostics.append(f"IDMS COBOL source uploaded: {source_name}")
        diagnostics.append(f"IDMS COBOL source text length: {len(text)}")
        return text, source_name

    except Exception as exc:
        diagnostics.append(f"IDMS COBOL source read failed: {exc}")
        return "", source_name

# LOCATION: src/idms_db2_phase2/ui/input_loader.py
# ACTION: ADD NEW FUNCTION

def _load_lrf_files(
    *,
    lrf_files,
    file_loader: FileLoader,
    parser: LrfParser,
    diagnostics: list[str],
    uploaded_file_names: dict[str, object],
) -> list:
    if not lrf_files:
        diagnostics.append(DIAG_LRF_NOT_UPLOADED)
        return []

    diagnostics.append(DIAG_LRF_FILE_COUNT_TEMPLATE.format(count=len(lrf_files)))

    file_names: list[str] = []
    text_parts: list[str] = []

```

```

for index, file in enumerate(lrf_files, start=1):
    file_name = str(file.name or "")
    file_names.append(file_name)
    try:
        text = file_loader.read_uploaded_text(file)
        text_parts.append(text)
        diagnostics.append(
            DIAG_LRF_FILE_TEMPLATE.format(index=index, path=file_name)
        )
        diagnostics.append(
            DIAG_LRF_TEXT_LEN_TEMPLATE.format(index=index, length=len(text))
        )
    except Exception as exc: # noqa: BLE001
        diagnostics.append(
            DIAG_LRF_READ_FAILED_TEMPLATE.format(name=file_name, reason=exc)
        )

uploaded_file_names["lrf_files"] = file_names

lrf_text = "\n".join(text_parts)
if not lrf_text.strip():
    diagnostics.append(DIAG_LRF_PARSED_ZERO)
    return []

try:
    try:
        logical_records = parser.parse(
            text=lrf_text,
            source_label=LRF_SOURCE_LABEL,
        )
    except TypeError:
        logical_records = parser.parse(lrf_text)

    if hasattr(parser, "diagnostics"):
        diagnostics.extend(parser.diagnostics)

    return logical_records

except Exception as exc: # noqa: BLE001
    diagnostics.append(
        DIAG_LRF_PARSE_FAILED_TEMPLATE.format(
            name=LRF_SOURCE_LABEL,
            reason=exc,
        )
    )
    return []

```

```

=====
FILE: src\idms_db2_phase2\ui\main_page.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/ui/main_page.py
# ACTION: REPLACE ENTIRE FILE

```

```

from __future__ import annotations

import streamlit as st

from code_review.ui.review_panel import render_review_panel
from idms_db2_phase2.ui.main_tab import render_main_tab
from idms_db2_phase2.ui.output_tabs import (
    render_diagnostics_tab,
    render_generated_cobol_tab,
    render_sheet_mapping_rows_tab,
    render_validation_tab,
)
from idms_db2_phase2.ui.session_state import initialize_session_state
from idms_db2_phase2.ui.test_panel import render_test_panel
from rules.test_runner_rules import UI_TAB_TITLE as TESTS_TAB_TITLE

TAB_MAIN = "Main"
TAB_SHEET_MAPPING = "Sheet Mapping Rows"
TAB_GENERATED = "Generated DB2 COBOL"

```

```
TAB_VALIDATION = "Validation"
TAB_CODE_REVIEW = "Code Review"
TAB_DIAGNOSTICS = "Diagnostics"
```

```
def render_main_page() -> None:
    initialize_session_state()

    tabs = st.tabs(
        [
            TAB_MAIN,
            TAB_SHEET_MAPPING,
            TAB_GENERATED,
            TAB_VALIDATION,
            TAB_CODE_REVIEW,
            TAB_DIAGNOSTICS,
            TESTS_TAB_TITLE,
        ]
    )

    with tabs[0]:
        render_main_tab()

    with tabs[1]:
        render_sheet_mapping_rows_tab()

    with tabs[2]:
        render_generated_cobol_tab()

    with tabs[3]:
        render_validation_tab()

    with tabs[4]:
        render_review_panel()

    with tabs[5]:
        render_diagnostics_tab()

    with tabs[6]:
        render_test_panel()
```

```
=====
FILE: src\idms_db2_phase2\ui\main_tab.py
ENCODING: utf-8
=====
```

```
# LOCATION: src/idms_db2_phase2/ui/main_tab.py
# ACTION: REPLACE ENTIRE FILE
```

```
from __future__ import annotations

import streamlit as st

from idms_db2_phase2.ui.conversion_actions import generate_db2_cobol
from idms_db2_phase2.ui.input_loader import load_and_analyze_inputs
from idms_db2_phase2.ui.status_panel import (
    render_current_status,
    render_main_download_section,
)
from rules.lrf_rules import LRF_UPLOAD_HELP, LRF_UPLOAD_LABEL
```

```
def render_main_tab() -> None:
    st.markdown("## Upload Inputs")
    st.info(
        "Upload Sheet Mapping, one or more DCLGEN files, optional Copybook files, "
        "the optional LRF Subschema file, and the IDMS COBOL source text file "
        "to generate DB2 embedded SQL COBOL."
    )

    (
        sheet_mapping_file,
        dclgen_files,
        copybook_files,
        lrf_files,
```

```

        idms_cobol_source_file,
    ) = _render_uploaders()

    st.caption(
        "Target PROGRAM-ID is derived automatically from the uploaded COBOL "
        "source (VM...BD... -> VMDZ... rule). No manual entry required."
    )

    _render_action_buttons(
        sheet_mapping_file=sheet_mapping_file,
        dclgen_files=dclgen_files,
        copybook_files=copybook_files,
        lrf_files=lrf_files,
        idms_cobol_source_file=idms_cobol_source_file,
    )

    render_current_status()
    render_main_download_section()

def _render_uploaders():
    col1, col2 = st.columns(2)

    with col1:
        sheet_mapping_file = st.file_uploader(
            "Sheet Mapping Excel or CSV",
            type=["xlsx", "csv"],
            key="sheet_mapping_file",
            help=(
                "Upload Sheet Mapping as .xlsx or .csv. "
                "If your file is .xls, save it as .xlsx or .csv first."
            ),
        )
        dclgen_files = st.file_uploader(
            "DCLGEN text file or files",
            type=["txt", "cbl", "cpy"],
            accept_multiple_files=True,
            key="dclgen_files",
        )
        lrf_files = st.file_uploader(
            LRF_UPLOAD_LABEL,
            type=["txt", "sub", "sch", "cpy", "cbl"],
            accept_multiple_files=True,
            key="lrf_files",
            help=LRF_UPLOAD_HELP,
        )

    with col2:
        copybook_files = st.file_uploader(
            "Optional Copybook text file or files",
            type=["txt", "cbl", "cpy"],
            accept_multiple_files=True,
            key="copybook_files",
        )
        idms_cobol_source_file = st.file_uploader(
            "IDMS COBOL source code",
            type=["txt", "cbl", "cob"],
            key="idms_cobol_source_file",
        )

    return (
        sheet_mapping_file,
        dclgen_files,
        copybook_files,
        lrf_files,
        idms_cobol_source_file,
    )

def _render_action_buttons(
    *,
    sheet_mapping_file,
    dclgen_files,
    copybook_files,
    lrf_files,
    idms_cobol_source_file,

```

```

) -> None:
    col_load, col_generate = st.columns(2)

    with col_load:
        if st.button("Load and Analyze Inputs", type="primary"):
            load_and_analyze_inputs(
                sheet_mapping_file=sheet_mapping_file,
                dclgen_files=dclgen_files,
                copybook_files=copybook_files,
                lrf_files=lrf_files,
                idms_cobol_source_file=idms_cobol_source_file,
            )

    with col_generate:
        if st.button("Generate DB2 COBOL"):
            generate_db2_cobol()

```

```

=====
FILE: src\idms_db2_phase2\ui\message_classifier.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/ui/message_classifier.py
# ACTION: CREATE NEW FILE

```

```

"""Classifies pipeline messages into severity and category.

```

```

Pure logic: no Streamlit, no file access, no COBOL knowledge. The UI
layer renders whatever this returns.

```

```

Classification order

```

1. PREFIX - the text before the first colon. A source that only ever reports failures, or only ever reports normal work, is decided once here.
2. KEYWORD - applied only when the prefix does not decide.
3. DEFAULT - Info. Most of the list is pass narration, so an unmatched message is far more likely to be narration than a fault.

```

Deliberate default

```

```

-----
Defaulting to Info rather than Warning is the whole point of this class.
The previous UI treated every message as a warning, which buried the two
that mattered under eighty that did not.
"""

```

```

from __future__ import annotations

```

```

from dataclasses import dataclass

```

```

from rules.validation_display_rules import (
    CATEGORY_SEPARATOR,
    ERROR_KEYWORDS,
    ERROR_PREFIXES,
    FALLBACK_CATEGORY,
    INFO_PREFIXES,
    MAX_CATEGORY_WORDS,
    SEVERITY_ERROR,
    SEVERITY_INFO,
    SEVERITY_ORDER,
    SEVERITY_WARNING,
    WARNING_KEYWORDS,
    WARNING_PREFIXES,
)

```

```

@dataclass(frozen=True)

```

```

class ClassifiedMessage:

```

```

    """One pipeline message with its severity and category."""

```

```

    text: str
    severity: str
    category: str

```

```

    @property

```

```

def is_actionable(self) -> bool:
    return self.severity in (SEVERITY_ERROR, SEVERITY_WARNING)

class MessageClassifier:
    """Assigns a severity and a category to a pipeline message."""

    # =====
    # Public entry points
    # =====
    def classify(self, message: str) -> ClassifiedMessage:
        text = " ".join(str(message or "").split())

        if not text:
            return ClassifiedMessage("", SEVERITY_INFO, FALLBACK_CATEGORY)

        prefix = self._prefix(text)
        severity = self._severity_from_prefix(prefix)

        if severity is None:
            severity = self._severity_from_keywords(text)

        return ClassifiedMessage(
            text=text,
            severity=severity,
            category=self._category(prefix),
        )

    def classify_all(self, messages: list[str]) -> list[ClassifiedMessage]:
        """Classify a list, de-duplicated, severity-ordered."""
        seen: set[str] = set()
        out: list[ClassifiedMessage] = []

        for message in messages or []:
            item = self.classify(message)
            if not item.text or item.text in seen:
                continue
            seen.add(item.text)
            out.append(item)

        return self.sort(out)

    @staticmethod
    def sort(items: list[ClassifiedMessage]) -> list[ClassifiedMessage]:
        """Errors first, then warnings, then info. Stable within a band."""
        rank = {name: index for index, name in enumerate(SEVERITY_ORDER)}
        return sorted(
            items,
            key=lambda item: (rank.get(item.severity, len(rank)), item.category),
        )

    @staticmethod
    def counts(items: list[ClassifiedMessage]) -> dict[str, int]:
        result = {name: 0 for name in SEVERITY_ORDER}
        for item in items:
            if item.severity in result:
                result[item.severity] += 1
        return result

    @staticmethod
    def categories(items: list[ClassifiedMessage]) -> list[str]:
        return sorted({item.category for item in items if item.category})

    @staticmethod
    def of_severity(
        items: list[ClassifiedMessage],
        severity: str,
    ) -> list[ClassifiedMessage]:
        return [item for item in items if item.severity == severity]

    # =====
    # Internals
    # =====
    @staticmethod
    def _prefix(text: str) -> str:
        """Text before the first colon, when it reads like a source name.

```

A long run of words before the colon is prose, not a prefix, so it is rejected rather than becoming a category of one.

"""

```
if CATEGORY_SEPARATOR not in text:
    return ""
```

```
candidate = text.split(CATEGORY_SEPARATOR, 1)[0].strip()
if not candidate or len(candidate.split()) > MAX_CATEGORY_WORDS:
    return ""
```

```
return candidate
```

```
@staticmethod
```

```
def _severity_from_prefix(prefix: str) -> str | None:
```

```
    if not prefix:
        return None
```

```
    lowered = prefix.lower()
```

```
    if lowered.startswith(ERROR_PREFIXES):
        return SEVERITY_ERROR
```

```
    if lowered.startswith(WARNING_PREFIXES):
        return SEVERITY_WARNING
```

```
    if lowered.startswith(INFO_PREFIXES):
        return SEVERITY_INFO
```

```
    return None
```

```
@staticmethod
```

```
def _severity_from_keywords(text: str) -> str:
```

```
    lowered = text.lower()
```

```
    if any(keyword in lowered for keyword in ERROR_KEYWORDS):
        return SEVERITY_ERROR
```

```
    if any(keyword in lowered for keyword in WARNING_KEYWORDS):
        return SEVERITY_WARNING
```

```
    return SEVERITY_INFO
```

```
@staticmethod
```

```
def _category(prefix: str) -> str:
```

```
    if not prefix:
        return FALLBACK_CATEGORY
```

```
    return prefix[:1].upper() + prefix[1:]
```

```
=====
FILE: src\idms_db2_phase2\ui\output_tabs.py
ENCODING: utf-8
=====
```

```
# LOCATION: src/idms_db2_phase2/ui/output_tabs.py
# ACTION: REPLACE ENTIRE FILE
```

```
from __future__ import annotations
```

```
import csv
import io
```

```
import streamlit as st
```

```
from idms_db2_phase2.analyzers.metadata_service import MetadataService
from idms_db2_phase2.ui.message_classifier import (
    ClassifiedMessage,
    MessageClassifier,
)
```

```
from rules.validation_display_rules import (
    COLUMN_CATEGORY,
    COLUMN_COUNT,
    COLUMN_FILE,
    COLUMN_MESSAGE,
    COLUMN_ROLE,
```

```

COLUMN_SEVERITY,
MESSAGE_TABLE_COLUMNS,
SEVERITY_ERROR,
SEVERITY_INFO,
SEVERITY_ORDER,
SEVERITY_WARNING,
UI_CLEAN_RUN,
UI_DIAGNOSTICS_HEADER,
UI_DIAGNOSTICS_INTRO,
UI_DOWNLOAD_DIAGNOSTICS,
UI_DOWNLOAD_VALIDATION,
UI_ERRORS_HEADER,
UI_FILE_DIAGNOSTICS,
UI_FILE_VALIDATION,
UI_FILES_EMPTY,
UI_FILES_HEADER,
UI_FILTER_CATEGORY,
UI_FILTER_SEARCH,
UI_FILTER_SEARCH_PLACEHOLDER,
UI_FILTER_SEVERITY,
UI_INFO_CAPTION,
UI_INFO_HEADER,
UI_METRIC_ERRORS,
UI_METRIC_INFO,
UI_METRIC_TOTAL,
UI_METRIC_WARNINGS,
UI_MIME_CSV,
UI_MIME_TEXT,
UI_NO_MATCH,
UI_NO_MESSAGES,
UI_PARSER_EMPTY,
UI_PARSER_HEADER,
UI_RAW_HEADER,
UI_VALIDATION_HEADER,
UI_VALIDATION_INTRO,
UI_WARNINGS_HEADER,
)

_CLASSIFIER = MessageClassifier()

# =====
# Sheet Mapping tab
# =====
def render_sheet_mapping_rows_tab() -> None:
    st.markdown("## Sheet Mapping Rows")

    metadata_service = MetadataService()
    rows = metadata_service.mapping_preview_rows(
        st.session_state.sheet_mapping_rows,
    )

    if rows:
        st.dataframe(rows, use_container_width=True, hide_index=True)
        return

    st.info("No Sheet Mapping rows available. Upload Excel or CSV first.")

# =====
# Generated COBOL tab
# =====
def render_generated_cobol_tab() -> None:
    st.markdown("## Generated DB2 COBOL")

    if not st.session_state.converted_cobol:
        st.info("Generate DB2 COBOL from the Main tab.")
        return

    st.caption(
        f"Output file name: `{st.session_state.converted_cobol_file_name}`"
    )

    st.download_button(
        label="Download Generated COBOL",
        data=st.session_state.converted_cobol,

```



```

        file_name=st.session_state.converted_cobol_file_name,
        mime="text/plain",
        type="primary",
        key="generated_tab_download_generated_cobol",
    )

    st.text_area(
        "Final DB2 COBOL Code",
        value=st.session_state.converted_cobol,
        height=760,
        key="generated_tab_final_db2_cobol_code",
    )

# =====
# Validation tab
# =====
def render_validation_tab() -> None:
    """Severity-aware validation report.

    The previous version pushed every message through st.warning(), which
    painted eighty lines of normal pass narration the same colour as a
    genuine failure. Messages are now classified, counted, filtered and
    grouped, so the two lines that need action are visible immediately.
    """
    st.markdown(UI_VALIDATION_HEADER)
    st.caption(UI_VALIDATION_INTRO)

    raw = st.session_state.get("validation_messages") or []
    if not raw:
        st.info(UI_NO_MESSAGES)
        return

    items = _CLASSIFIER.classify_all(list(raw))
    counts = _CLASSIFIER.counts(items)

    _render_metrics(counts, len(items))

    if counts[SEVERITY_ERROR] == 0 and counts[SEVERITY_WARNING] == 0:
        st.success(UI_CLEAN_RUN)

    filtered = _render_filters(items)

    if not filtered:
        st.info(UI_NO_MATCH)
        return

    _render_band(
        items=_CLASSIFIER.of_severity(filtered, SEVERITY_ERROR),
        header=UI_ERRORS_HEADER,
        renderer=st.error,
    )
    _render_band(
        items=_CLASSIFIER.of_severity(filtered, SEVERITY_WARNING),
        header=UI_WARNINGS_HEADER,
        renderer=st.warning,
    )
    _render_info_band(_CLASSIFIER.of_severity(filtered, SEVERITY_INFO))

    st.download_button(
        label=UI_DOWNLOAD_VALIDATION,
        data=_validation_csv(items),
        file_name=UI_FILE_VALIDATION,
        mime=UI_MIME_CSV,
        key="validation_tab_download_csv",
    )

def _render_metrics(counts: dict[str, int], total: int) -> None:
    columns = st.columns(4)
    columns[0].metric(UI_METRIC_ERRORS, counts[SEVERITY_ERROR])
    columns[1].metric(UI_METRIC_WARNINGS, counts[SEVERITY_WARNING])
    columns[2].metric(UI_METRIC_INFO, counts[SEVERITY_INFO])
    columns[3].metric(UI_METRIC_TOTAL, total)

```

```

def _render_filters(
    items: list[ClassifiedMessage],
) -> list[ClassifiedMessage]:
    left, middle, right = st.columns([2, 2, 2])

    with left:
        severities = st.multiselect(
            UI_FILTER_SEVERITY,
            options=list(SEVERITY_ORDER),
            default=list(SEVERITY_ORDER),
            key="validation_tab_severity",
        )

    with middle:
        categories = st.multiselect(
            UI_FILTER_CATEGORY,
            options=_CLASSIFIER.categories(items),
            default=[],
            key="validation_tab_category",
        )

    with right:
        search = st.text_input(
            UI_FILTER_SEARCH,
            value="",
            placeholder=UI_FILTER_SEARCH_PLACEHOLDER,
            key="validation_tab_search",
        )

    needle = str(search or "").strip().lower()

    return [
        item
        for item in items
        if item.severity in severities
        and (not categories or item.category in categories)
        and (not needle or needle in item.text.lower())
    ]

def _render_band(
    items: list[ClassifiedMessage],
    header: str,
    renderer,
) -> None:
    if not items:
        return

    st.markdown(header)
    for item in items:
        renderer(item.text)

def _render_info_band(items: list[ClassifiedMessage]) -> None:
    """Narration goes in a collapsed table, never eighty coloured boxes."""
    if not items:
        return

    st.markdown(UI_INFO_HEADER)

    with st.expander(
        UI_INFO_CAPTION.format(count=len(items)),
        expanded=False,
    ):
        st.dataframe(
            [
                {
                    COLUMN_SEVERITY: item.severity,
                    COLUMN_CATEGORY: item.category,
                    COLUMN_MESSAGE: item.text,
                }
                for item in items
            ],
            use_container_width=True,
            hide_index=True,
            column_order=list(MESSAGE_TABLE_COLUMNS),
        )

```

```

)

def _validation_csv(items: list[ClassifiedMessage]) -> str:
    buffer = io.StringIO()
    writer = csv.writer(buffer, lineterminator="\n")
    writer.writerow(list(MESSAGE_TABLE_COLUMNS))

    for item in items:
        writer.writerow([item.severity, item.category, item.text])

    return buffer.getvalue()

# =====
# Diagnostics tab
# =====
def render_diagnostics_tab() -> None:
    """Parser diagnostics, grouped and searchable.

    Replaces a raw st.json dump plus one unscrollable code block.
    """
    st.markdown(UI_DIAGNOSTICS_HEADER)
    st.caption(UI_DIAGNOSTICS_INTRO)

    _render_uploaded_files()
    _render_parser_diagnostics()

def _render_uploaded_files() -> None:
    st.markdown(UI_FILES_HEADER)

    uploaded = st.session_state.get("uploaded_file_names") or {}
    if not uploaded:
        st.info(UI_FILES_EMPTY)
        return

    rows: list[dict[str, str]] = []
    for role, value in uploaded.items():
        label = str(role).replace("_", " ").title()

        if isinstance(value, (list, tuple, set)):
            for name in value:
                rows.append({COLUMN_ROLE: label, COLUMN_FILE: str(name)})
        elif value:
            rows.append({COLUMN_ROLE: label, COLUMN_FILE: str(value)})

    if not rows:
        st.info(UI_FILES_EMPTY)
        return

    st.dataframe(
        rows,
        use_container_width=True,
        hide_index=True,
        column_order=[COLUMN_ROLE, COLUMN_FILE],
    )

def _render_parser_diagnostics() -> None:
    st.markdown(UI_PARSER_HEADER)

    diagnostics = st.session_state.get("diagnostics") or []
    if not diagnostics:
        st.info(UI_PARSER_EMPTY)
        return

    items = _CLASSIFIER.classify_all(list(diagnostics))

    summary: dict[str, int] = {}
    for item in items:
        summary[item.category] = summary.get(item.category, 0) + 1

    st.dataframe(
        [
            {COLUMN_CATEGORY: category, COLUMN_COUNT: count}

```

```

        for category, count in sorted(summary.items())
    ],
    use_container_width=True,
    hide_index=True,
    column_order=[COLUMN_CATEGORY, COLUMN_COUNT],
)

search = st.text_input(
    UI_FILTER_SEARCH,
    value="",
    placeholder=UI_FILTER_SEARCH_PLACEHOLDER,
    key="diagnostics_tab_search",
)

needle = str(search or "").strip().lower()
visible = [
    item for item in items
    if not needle or needle in item.text.lower()
]

if not visible:
    st.info(UI_NO_MATCH)
else:
    st.dataframe(
        [
            {
                COLUMN_CATEGORY: item.category,
                COLUMN_MESSAGE: item.text,
            }
            for item in visible
        ],
        use_container_width=True,
        hide_index=True,
        column_order=[COLUMN_CATEGORY, COLUMN_MESSAGE],
    )

with st.expander(UI_RAW_HEADER, expanded=False):
    st.code("\n".join(str(line) for line in diagnostics), language="text")

st.download_button(
    label=UI_DOWNLOAD_DIAGNOSTICS,
    data="\n".join(str(line) for line in diagnostics),
    file_name=UI_FILE_DIAGNOSTICS,
    mime=UI_MIME_TEXT,
    key="diagnostics_tab_download",
)

```

```

=====
FILE: src\idms_db2_phase2\ui\session_state.py
ENCODING: utf-8
=====

```

```

# LOCATION: src/idms_db2_phase2/ui/session_state.py
# ACTION: REPLACE ENTIRE FILE

```

```

from __future__ import annotations

```

```

import streamlit as st

```

```

def initialize_session_state() -> None:
    defaults = {
        "sheet_mapping_rows": [],
        "dclgen_columns": [],
        "copybook_fields": [],
        "logical_records": [],
        "idms_cobol_text": "",
        "idms_cobol_source_name": "",
        "converted_cobol": "",
        "converted_cobol_file_name": "converted_db2_cobol.cb1",
        "validation_messages": [],
        "operations": [],
        "generated": False,
        "loaded": False,
        "diagnostics": [],
    }

```

```

        "uploaded_file_names": {},
    }

    for key, value in defaults.items():
        if key in st.session_state:
            continue
        if isinstance(value, list):
            st.session_state[key] = []
        elif isinstance(value, dict):
            st.session_state[key] = {}
        else:
            st.session_state[key] = value

def reset_generated_output_state() -> None:
    st.session_state.converted_cobol = ""
    st.session_state.converted_cobol_file_name = "converted_db2_cobol.cbl"
    st.session_state.validation_messages = []
    st.session_state.operations = []
    st.session_state.generated = False

=====
FILE: src\idms_db2_phase2\ui\status_panel.py
ENCODING: utf-8
=====

# LOCATION: src/idms_db2_phase2/ui/status_panel.py
# ACTION: REPLACE ENTIRE FILE

from __future__ import annotations

import streamlit as st

from rules.lrf_rules import LRF_METRIC_LABEL

def render_current_status() -> None:
    st.markdown("## Current Status")
    col1, col2, col3, col4, col5 = st.columns(5)

    with col1:
        st.metric("Sheet Mapping Rows", len(st.session_state.sheet_mapping_rows))

    with col2:
        st.metric("DCLGEN Columns", len(st.session_state.dclgen_columns))

    with col3:
        st.metric("Copybook Fields", len(st.session_state.copybook_fields))

    with col4:
        st.metric(
            LRF_METRIC_LABEL,
            len(st.session_state.get("logical_records") or []),
        )

    with col5:
        st.metric("IDMS COBOL Length", len(st.session_state.idms_cobol_text))

    if st.session_state.loaded:
        st.success("Inputs loaded. Review diagnostics before generating.")

    if st.session_state.generated:
        st.success("DB2 COBOL generated. Download is available below.")

def render_main_download_section() -> None:
    st.markdown("## Download Generated Output")

    if not st.session_state.generated or not st.session_state.converted_cobol:
        st.info("Generate DB2 COBOL to enable download.")
        return

    st.success("Generated DB2 COBOL is ready for download.")

    st.caption(

```

```

        f"Output file name: `{st.session_state.converted_cobol_file_name}`"
    )

    st.download_button(
        label="Download Generated DB2 COBOL",
        data=st.session_state.converted_cobol,
        file_name=st.session_state.converted_cobol_file_name,
        mime="text/plain",
        type="primary",
        key="main_download_generated_cobol",
    )

    with st.expander("Preview Generated DB2 COBOL", expanded=False):
        st.text_area(
            "Generated DB2 COBOL Preview",
            value=st.session_state.converted_cobol,
            height=500,
            key="main_generated_cobol_preview",
        )

=====
FILE: src\idms_db2_phase2\ui\test_panel.py
ENCODING: utf-8
=====

# LOCATION: src/idms_db2_phase2/ui/test_panel.py
# ACTION: CREATE NEW FILE

"""Streamlit panel for running the Python test suite.

Reads only from session state and writes only to it. Contains no test
logic of its own: discovery, invocation and report parsing all live in
idms_db2_phase2.testing.pytest_runner, so the panel stays
presentation-only and the runner stays testable.
"""

from __future__ import annotations

import csv
import io
from datetime import datetime

import streamlit as st

from idms_db2_phase2.testing.pytest_runner import (
    PytestRunner,
    TestRunResult,
)
from rules.test_runner_rules import (
    ALL_TESTS_LABEL,
    COLUMN_DURATION,
    COLUMN_MESSAGE,
    COLUMN_MODULE,
    COLUMN_OUTCOME,
    COLUMN_TEST,
    FILE_TIMESTAMP_FORMAT,
    MESSAGE_PREVIEW_LENGTH,
    OUTCOME_ICONS,
    RUN_TIMEOUT_SECONDS,
    TABLE_COLUMNS,
    UI_ALL_PASSED,
    UI_COMMAND_HEADER,
    UI_DETAIL_HEADER,
    UI_DOWNLOAD_CSV,
    UI_DOWNLOAD_LOG,
    UI_FAILURES_ONLY_LABEL,
    UI_FILE_CSV,
    UI_FILE_LOG,
    UI_HEADER,
    UI_INTRO,
    UI_KEYWORD_LABEL,
    UI_KEYWORD_PLACEHOLDER,
    UI_METRIC_DURATION,
    UI_METRIC_FAILED,
    UI_METRIC_PASSED,

```

```

    UI_METRIC_SKIPPED,
    UI_METRIC_TOTAL,
    UI_MIME_CSV,
    UI_MIME_TEXT,
    UI_NO_RESULT,
    UI_NO_TESTS_FOUND,
    UI_OUTPUT_HEADER,
    UI_RUN_BUTTON,
    UI_RUN_ERROR,
    UI_RUNNING,
    UI_SOME_FAILED,
    UI_TABLE_HEADER,
    UI_TARGET_LABEL,
    UI_TIMED_OUT,
    UI_VERBOSE_LABEL,
)

STATE_RESULT = "test_run_result"
STATE_STAMP = "test_run_stamp"

def render_test_panel() -> None:
    """Render the Tests tab."""
    st.markdown(UI_HEADER)
    st.caption(UI_INTRO)

    runner = PytestRunner()

    if not runner.has_tests():
        st.info(UI_NO_TESTS_FOUND)
        st.caption(f"Looked in: {runner.tests_dir}")
        return

    target, keyword, verbose = _render_controls(runner)

    if st.button(UI_RUN_BUTTON, type="primary", key="test_panel_run"):
        with st.spinner(UI_RUNNING):
            result = runner.run(
                target=target,
                keyword=keyword,
                verbose=verbose,
            )
            st.session_state[STATE_RESULT] = result
            st.session_state[STATE_STAMP] = datetime.now().strftime(
                FILE_TIMESTAMP_FORMAT
            )

    result = st.session_state.get(STATE_RESULT)

    if result is None:
        st.info(UI_NO_RESULT)
        return

    _render_result(result)

# =====
# Controls
# =====
def _render_controls(runner: PytestRunner) -> tuple[str, str, bool]:
    left, middle, right = st.columns([2, 2, 1])

    with left:
        target = st.selectbox(
            UI_TARGET_LABEL,
            options=runner.available_targets(),
            index=0,
            key="test_panel_target",
        )

    with middle:
        keyword = st.text_input(
            UI_KEYWORD_LABEL,
            value="",
            placeholder=UI_KEYWORD_PLACEHOLDER,
            key="test_panel_keyword",

```

```

    )

    with right:
        verbose = st.checkbox(
            UI_VERBOSE_LABEL,
            value=False,
            key="test_panel_verbose",
        )

    return target or ALL_TESTS_LABEL, keyword, verbose

# =====
# Result rendering
# =====
def _render_result(result: TestRunResult) -> None:
    if result.error:
        st.error(UI_RUN_ERROR.format(reason=result.error))
        return

    if result.timed_out:
        st.error(UI_TIMED_OUT.format(seconds=RUN_TIMEOUT_SECONDS))

    _render_metrics(result)
    _render_verdict(result)
    _render_table(result)
    _render_failures(result)
    _render_downloads(result)
    _render_raw_output(result)

def _render_metrics(result: TestRunResult) -> None:
    columns = st.columns(5)
    columns[0].metric(UI_METRIC_TOTAL, result.total)
    columns[1].metric(UI_METRIC_PASSED, result.passed)
    columns[2].metric(
        UI_METRIC_FAILED,
        result.failed + result.errors,
    )
    columns[3].metric(UI_METRIC_SKIPPED, result.skipped)
    columns[4].metric(UI_METRIC_DURATION, f"{result.duration:.1f}")

def _render_verdict(result: TestRunResult) -> None:
    if result.timed_out:
        return

    if result.is_green:
        st.success(UI_ALL_PASSED)
        return

    st.error(
        UI_SOME_FAILED.format(failed=result.failed + result.errors)
    )

def _render_table(result: TestRunResult) -> None:
    st.markdown(UI_TABLE_HEADER)

    failures_only = st.checkbox(
        UI_FAILURES_ONLY_LABEL,
        value=not result.is_green,
        key="test_panel_failures_only",
    )

    cases = result.failing if failures_only else result.cases
    if not cases:
        st.info(UI_ALL_PASSED)
        return

    rows = [
        {
            COLUMN_OUTCOME: OUTCOME_ICONS.get(case.outcome, case.outcome),
            COLUMN_MODULE: case.module,
            COLUMN_TEST: case.name,
            COLUMN_DURATION: case.duration,

```



```

        COLUMN_MESSAGE: _preview(case.message),
    }
    for case in cases
]

st.dataframe(
    rows,
    use_container_width=True,
    hide_index=True,
    column_order=list(TABLE_COLUMNS),
)

def _render_failures(result: TestRunResult) -> None:
    if not result.failing:
        return

    st.markdown(UI_DETAIL_HEADER)

    for index, case in enumerate(result.failing):
        label = f"{OUTCOME_ICONS.get(case.outcome, case.outcome)} | {case.module} | {case.name}"
        with st.expander(label, expanded=index == 0):
            if case.message:
                st.markdown(f"***{case.message}***")
            if case.detail:
                st.code(case.detail, language="text")

def _render_downloads(result: TestRunResult) -> None:
    stamp = st.session_state.get(STATE_STAMP, "")

    left, right = st.columns(2)

    with left:
        st.download_button(
            label=UI_DOWNLOAD_CSV,
            data=_as_csv(result),
            file_name=UI_FILE_CSV.format(stamp=stamp),
            mime=UI_MIME_CSV,
            key="test_panel_download_csv",
        )

    with right:
        st.download_button(
            label=UI_DOWNLOAD_LOG,
            data=_as_log(result),
            file_name=UI_FILE_LOG.format(stamp=stamp),
            mime=UI_MIME_TEXT,
            key="test_panel_download_log",
        )

def _render_raw_output(result: TestRunResult) -> None:
    with st.expander(UI_OUTPUT_HEADER.replace("### ", ""), expanded=False):
        st.markdown(UI_COMMAND_HEADER)
        st.code(result.command, language="text")

        if result.stdout.strip():
            st.code(result.stdout, language="text")

        if result.stderr.strip():
            st.code(result.stderr, language="text")

# =====
# Serialisation
# =====
def _as_csv(result: TestRunResult) -> str:
    buffer = io.StringIO()
    writer = csv.writer(buffer, lineterminator="\n")
    writer.writerow(list(TABLE_COLUMNS))

    for case in result.cases:
        writer.writerow(
            [
                case.outcome,

```

```

        case.module,
        case.name,
        case.duration,
        case.message.replace("\n", " ").strip(),
    ]
)

return buffer.getvalue()

def _as_log(result: TestRunResult) -> str:
    lines = [
        f"Command : {result.command}",
        f"Started : {result.started_at}",
        f"Exit code : {result.exit_code}",
        f"Duration : {result.duration:.2f}s",
        f"Total : {result.total}",
        f"Passed : {result.passed}",
        f"Failed : {result.failed}",
        f"Errors : {result.errors}",
        f"Skipped : {result.skipped}",
        "",
        "--- stdout ---",
        result.stdout,
        "",
        "--- stderr ---",
        result.stderr,
    ]
    return "\n".join(lines)

def _preview(text: str) -> str:
    clean = " ".join(str(text or "").split())
    if len(clean) <= MESSAGE_PREVIEW_LENGTH:
        return clean
    return clean[: MESSAGE_PREVIEW_LENGTH - 3] + "..."

=====
FILE: src\idms_db2_phase2\validators\__init__.py
ENCODING: utf-8
=====

[Empty file]

=====
FILE: src\idms_db2_phase2\validators\input_validator.py
ENCODING: utf-8
=====

from idms_db2_phase2.domain.models import ConversionInput
from rules.validation_rules import INPUT_VALIDATION_MESSAGES

class InputValidator:
    """
    Validates required conversion inputs.

    This validator checks only input availability. It does not validate
    generated COBOL production quality.
    """

    def validate(
        self,
        conversion_input: ConversionInput,
    ) -> list[str]:
        messages: list[str] = []

        if not conversion_input.sheet_mapping_rows:
            messages.append(
                INPUT_VALIDATION_MESSAGES["missing_sheet_mapping"]
            )

        if not conversion_input.dclgen_columns:
            messages.append(

```

```

        INPUT_VALIDATION_MESSAGES["missing_dclgen"]
    )

    if not conversion_input.idms_cobol_text.strip():
        messages.append(
            INPUT_VALIDATION_MESSAGES["missing_idms_cobol"]
        )

    return messages

=====
FILE: src\idms_db2_phase2\validators\mapping_scope_filter.py
ENCODING: utf-8
=====

# LOCATION: src/idms_db2_phase2/validators/mapping_scope_filter.py
# ACTION: CREATE NEW FILE

"""Scopes mapping validation messages to the program being converted.

A Sheet Mapping gap is an ERROR only for a table the program actually
references. For every other table it is a workbook note, and reporting it
as a blocker makes a clean retrieval conversion read as Rejected.

Reference detection is deliberately conservative: a table counts as
referenced when its own name, or the IDMS record name mapped to it,
appears anywhere in the SOURCE program. No message is ever discarded -
out-of-scope gaps are rewritten, not hidden.
"""

from __future__ import annotations

from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.mapping_scope_patterns import (
    MESSAGE_TABLE_PATTERN,
    SOURCE_TOKEN_PATTERN,
)
from rules.mapping_scope_rules import (
    DIAG_SCOPE_APPLIED,
    ENFORCE_MAPPING_SCOPE_FILTER,
    OUT_OF_SCOPE_TEMPLATE,
)

class MappingScopeFilter:
    """Rewrites mapping gaps for tables the program does not reference."""

    def __init__(
        self,
        mapping_repository,
        table_name_resolver,
    ) -> None:
        self.mapping_repository = mapping_repository
        self.table_name_resolver = table_name_resolver
        self.messages: list[str] = []

    # =====
    # Public entry point
    # =====
    def apply(
        self,
        messages: list[str],
        source_text: str,
    ) -> list[str]:
        self.messages = []

        if not messages or not ENFORCE_MAPPING_SCOPE_FILTER:
            return list(messages or [])

        referenced = self._referenced_tables(source_text)
        out: list[str] = []
        rewritten = 0
        gaps = 0

        for message in messages:

```

```

        table = self._table_of(message)

        if not table:
            out.append(message)
            continue

        gaps += 1

        if table in referenced:
            out.append(message)
            continue

        out.append(OUT_OF_SCOPE_TEMPLATE.format(table=table))
        rewritten += 1

    if rewritten:
        self.messages.append(
            DIAG_SCOPE_APPLIED.format(
                out_of_scope=rewritten,
                total=gaps,
            )
        )

    return out

# =====
# Reference detection
# =====
def _referenced_tables(self, source_text: str) -> set[str]:
    """Tables whose name or mapped record appears in the source."""
    tokens = {
        NameNormalizer.compact(match.group(0))
        for match in SOURCE_TOKEN_PATTERN.finditer(str(source_text or ""))
    }
    tokens.discard("")

    referenced: set[str] = set()

    for record in self._records():
        table = NameNormalizer.normalize(
            self.table_name_resolver.table_for_record(record)
        )
        if not table:
            continue

        if NameNormalizer.compact(table) in tokens:
            referenced.add(table)
            continue

        if NameNormalizer.compact(record) in tokens:
            referenced.add(table)

    return referenced

def _records(self) -> list[str]:
    try:
        return [
            NameNormalizer.normalize(record)
            for record in self.mapping_repository.records()
        ]
    except Exception: # noqa: BLE001
        return []

@staticmethod
def _table_of(message: str) -> str:
    match = MESSAGE_TABLE_PATTERN.search(str(message or ""))
    if not match:
        return ""
    return NameNormalizer.normalize(match.group("table"))

```

```
=====
FILE: src\idms_db2_phase2\validators\mapping_validator.py
ENCODING: utf-8
=====
```

```
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from rules.validation_rules import MAPPING_VALIDATION_MESSAGES
```

```
class MappingValidator:
```

```
    """
```

```
    Validates Sheet Mapping and DCLGEN consistency.
```

```
    This validator checks metadata availability and resolver alignment.
```

```
    """
```

```
    def __init__(
```

```
        self,
```

```
        mapping_repository: MappingRepository,
```

```
        dclgen_repository: DclgenRepository,
```

```
        table_name_resolver: TableNameResolver,
```

```
    ) -> None:
```

```
        self.mapping_repository = mapping_repository
```

```
        self.dclgen_repository = dclgen_repository
```

```
        self.table_name_resolver = table_name_resolver
```

```
    def validate(
```

```
        self,
```

```
    ) -> list[str]:
```

```
        messages: list[str] = []
```

```
        if self.mapping_repository.count() == 0:
```

```
            messages.append(
```

```
                MAPPING_VALIDATION_MESSAGES["missing_mapping_rows"]
```

```
            )
```

```
        if self.dclgen_repository.count() == 0:
```

```
            messages.append(
```

```
                MAPPING_VALIDATION_MESSAGES["missing_dclgen_columns"]
```

```
            )
```

```
        messages.extend(self._validate_record_table_resolution())
```

```
        messages.extend(self._validate_table_column_resolution())
```

```
        return messages
```

```
    def _validate_record_table_resolution(
```

```
        self,
```

```
    ) -> list[str]:
```

```
        messages: list[str] = []
```

```
        for record in self.mapping_repository.records():
```

```
            table = self.table_name_resolver.table_for_record(record)
```

```
            if table:
```

```
                continue
```

```
            messages.append(
```

```
                f"{MAPPING_VALIDATION_MESSAGES['missing_record_mapping']} Record={record}"
```

```
            )
```

```
        return messages
```

```
    def _validate_table_column_resolution(
```

```
        self,
```

```
    ) -> list[str]:
```

```
        messages: list[str] = []
```

```
        for table in self.mapping_repository.tables():
```

```
            resolved_table = self.table_name_resolver.resolve_table(table)
```

```
            if not resolved_table:
```

```
                continue
```

```

        columns = self.mapping_repository.db2_columns_for_table(table)

        if columns:
            continue

        messages.append(
            f"{MAPPING_VALIDATION_MESSAGES['missing_column_mapping']} Table={resolved_table}"
        )

    return messages

```

```

=====
FILE: src\idms_db2_phase2\validators\production_validator.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.services.name_normalizer import NameNormalizer
from patterns.sequence_patterns import strip_sequence_numbers
from patterns.sql_patterns import HOST_REFERENCE_PATTERN
from patterns.validation_patterns import (
    ERROR_DB2_PATTERN,
    FORBIDDEN_EXECUTABLE_IDMS_PATTERNS,
    FORBIDDEN_IDMS_DECLARATIVE_PATTERNS,
    NO_FETCH_HOST_VARIABLES_PATTERN,
    TODO_DB2_PATTERN,
    TODO_HOST_VARIABLE_PATTERN,
    UNABLE_TO_DECLARE_CURSOR_PATTERN,
)
from rules.db2_validation_rules import REQUIRED_DB2_TOKENS
from rules.validation_rules import PRODUCTION_VALIDATION_MESSAGES

```

```

class ProductionValidator:
    """
    Performs production-focused validation for generated DB2 COBOL.

    It detects:
    - Missing required DB2 constructs.
    - TODO host variables.
    - Generated DB2 error markers.
    - Residual executable IDMS statements.
    - Residual IDMS declarative/control statements.
    - Generated DCLGEN host variables not found in uploaded DCLGEN metadata.

    This validator must ignore commented fixed-format lines.
    """

    def __init__(
        self,
        dclgen_repository: DclgenRepository,
    ) -> None:
        self.dclgen_repository = dclgen_repository

    def validate(
        self,
        converted_cobol_text: str,
    ) -> list[str]:
        messages: list[str] = []
        text = str(converted_cobol_text or "")

        if not text.strip():
            return ["Production validation: converted COBOL text is empty."]

        self._validate_required_db2_tokens(text, messages)
        self._validate_no_todo_or_generated_error(text, messages)
        self._validate_forbidden_idms_patterns(text, messages)
        self._validate_forbidden_idms_declaratives(text, messages)
        self._validate_generated_dclgen_host_variables(text, messages)

        return messages

    def _validate_required_db2_tokens(
        self,
        text: str,

```

```

        messages: list[str],
    ) -> None:
        upper_text = text.upper()

        for token in REQUIRED_DB2_TOKENS:
            if token.upper() in upper_text:
                continue

            key = f"missing_{NameNormalizer.normalize(token).lower()}"

            if key in PRODUCTION_VALIDATION_MESSAGES:
                messages.append(PRODUCTION_VALIDATION_MESSAGES[key])
                continue

            messages.append(
                f"Production validation: required DB2 token missing: {token}"
            )

def _validate_no_todo_or_generated_error(
    self,
    text: str,
    messages: list[str],
) -> None:
    if TODO_HOST_VARIABLE_PATTERN.search(text):
        messages.append(
            PRODUCTION_VALIDATION_MESSAGES["todo_host_variable"]
        )

    if TODO_DB2_PATTERN.search(text):
        messages.append(
            PRODUCTION_VALIDATION_MESSAGES["todo_db2"]
        )

    if ERROR_DB2_PATTERN.search(text):
        messages.append(
            PRODUCTION_VALIDATION_MESSAGES["error_db2"]
        )

    if UNABLE_TO_DECLARE_CURSOR_PATTERN.search(text):
        messages.append(
            PRODUCTION_VALIDATION_MESSAGES["unable_to_declare_cursor"]
        )

    if NO_FETCH_HOST_VARIABLES_PATTERN.search(text):
        messages.append(
            PRODUCTION_VALIDATION_MESSAGES["no_fetch_host_variables"]
        )

def _validate_forbidden_idms_patterns(
    self,
    text: str,
    messages: list[str],
) -> None:
    for line_number, line in enumerate(text.splitlines(), start=1):
        logical = self._logical_line(line)

        if self._is_comment_or_blank(logical):
            continue

        for pattern in FORBIDDEN_EXECUTABLE_IDMS_PATTERNS:
            if pattern.search(logical):
                messages.append(
                    "Production validation: residual executable IDMS statement "
                    f"remains near line {line_number}: {logical}"
                )
                break

def _validate_forbidden_idms_declaratives(
    self,
    text: str,
    messages: list[str],
) -> None:
    for line_number, line in enumerate(text.splitlines(), start=1):
        logical = self._logical_line(line)

        if self._is_comment_or_blank(logical):

```

```

        continue

    for pattern in FORBIDDEN_IDMS_DECLARATIVE_PATTERNS:
        if pattern.search(logical):
            messages.append(
                "Production validation: residual IDMS declarative/control "
                f"statement remains near line {line_number}: {logical}"
            )
            break

def _validate_generated_dclgen_host_variables(
    self,
    converted_cobol_text: str,
    messages: list[str],
) -> None:
    generated_hosts = self._generated_dclgen_host_references(
        converted_cobol_text
    )

    if not generated_hosts:
        return

    valid_hosts = self._valid_dclgen_host_reference_keys()

    if not valid_hosts:
        messages.append(
            PRODUCTION_VALIDATION_MESSAGES["no_dclgen_hosts"]
        )
        return

    missing_hosts = sorted(
        host for host in generated_hosts if host not in valid_hosts
    )

    for host in missing_hosts:
        messages.append(
            f"Production validation: generated host variable {host} "
            "was not found in uploaded DCLGEN columns."
        )

def _generated_dclgen_host_references(
    self,
    converted_cobol_text: str,
) -> set[str]:
    output: set[str] = set()

    for match in HOST_REFERENCE_PATTERN.finditer(converted_cobol_text):
        group = NameNormalizer.to_cobol(match.group("group"))
        field = NameNormalizer.to_cobol(match.group("field"))

        key = self._host_key(group=group, field=field)

        if key:
            output.add(key)

    return output

def _valid_dclgen_host_reference_keys(
    self,
) -> set[str]:
    output: set[str] = set()

    for column in self.dclgen_repository.all():
        table = NameNormalizer.normalize(column.table_name)
        field = NameNormalizer.to_cobol(
            column.cobol_host_name or column.column_name
        )

        if not table or not field:
            continue

        group = self.dclgen_repository.group_for_table(table)
        key = self._host_key(group=group, field=field)

        if key:
            output.add(key)

```



```

        return output

def _host_key(
    self,
    group: str,
    field: str,
) -> str:
    clean_group = NameNormalizer.to_cobol(group)
    clean_field = NameNormalizer.to_cobol(field)

    if not clean_group or not clean_field:
        return ""

    return f"{clean_group}.{clean_field}"

def _logical_line(
    self,
    line: str,
) -> str:
    text = str(line or "").rstrip()

    if self._is_fixed_format_comment(text):
        return "*"

    return strip_sequence_numbers(text).strip()

def _is_fixed_format_comment(
    self,
    line: str,
) -> bool:
    text = str(line or "")

    if len(text) >= 7 and text[:6].isdigit() and text[6:7] in ("*", "/"):
        return True

    stripped = text.strip()

    if stripped.startswith("*") or stripped.startswith("/"):
        return True

    return False

def _is_comment_or_blank(
    self,
    line: str,
) -> bool:
    stripped = str(line or "").strip()

    if not stripped:
        return True

    if stripped.startswith("*") or stripped.startswith("/"):
        return True

    return False

```

```

=====
FILE: tests\__init__.py
ENCODING: utf-8
=====

```

[Empty file]

```

=====
FILE: tests\conftest.py
ENCODING: utf-8
=====

```

```

# LOCATION: tests/conftest.py
# ACTION: CREATE NEW FILE

```

```

"""Shared pytest configuration and fixtures.

```

WHY THIS FILE EXISTS

1. sys.path bootstrap.

The converter lives under src/ while rules/, patterns/ and catalogs/ sit at the project root, so BOTH must be importable. README documents \$env:PYTHONPATH = "src;" but a forgotten environment variable then reports as a mysterious collection error. Setting it here makes `python -m pytest tests -q` work from a clean shell.

2. One shared transformer context.

build_transformer_context() used to live inside a test module, and other test modules imported it with

```
from tests.test_idms_statement_transformer import build_transformer_context
```

Importing one test module from another depends on rootdir, on pytest's import mode and on collection order, and it breaks the moment the file is renamed. conftest.py is auto-discovered, so nothing imports anything.

Use either form:

```
def test_x(transformer_context):          # fixture
    ...

def test_y():                             # plain call
    context = build_transformer_context()
"""

from __future__ import annotations

import sys
from pathlib import Path

#
# Path bootstrap. MUST run before any idms_db2_phase2 / rules / patterns
# import below.
#
PROJECT_ROOT = Path(__file__).resolve().parents[1]
SRC_DIR = PROJECT_ROOT / "src"

for _candidate in (PROJECT_ROOT, SRC_DIR):
    if str(_candidate) not in sys.path:
        sys.path.insert(0, str(_candidate))

import pytest # noqa: E402

from idms_db2_phase2.domain.models import ( # noqa: E402
    DclgenColumn,
    SheetMappingRow,
)
from idms_db2_phase2.generators.sql_error_generator import ( # noqa: E402
    SqlErrorGenerator,
)
from idms_db2_phase2.generators.sql_generator import SqlGenerator # noqa: E402
from idms_db2_phase2.repositories.dclgen_repository import ( # noqa: E402
    DclgenRepository,
)
from idms_db2_phase2.repositories.mapping_repository import ( # noqa: E402
    MappingRepository,
)
from idms_db2_phase2.resolvers.column_name_resolver import ( # noqa: E402
    ColumnNameResolver,
)
from idms_db2_phase2.resolvers.cursor_name_resolver import ( # noqa: E402
    CursorNameResolver,
)
from idms_db2_phase2.resolvers.host_variable_resolver import ( # noqa: E402
    HostVariableResolver,
)
from idms_db2_phase2.resolvers.table_name_resolver import ( # noqa: E402
    TableNameResolver,
)
from idms_db2_phase2.transformers.idms_statement_transformer import ( # noqa: E402
    IdmsStatementTransformer,
)
```

```

def build_sheet_mapping_rows() -> list[SheetMappingRow]:
    """One mapped IDMS record, VMB-FAR, with a CALC primary key."""
    return [
        SheetMappingRow(
            cobol_record_idms="VMB-FAR",
            cobol_zone="NR-ID",
            idms_key="CALC",
            db2_key="PRIMARY KEY",
            new_db2_record="DZBFARTB",
            new_db2_field_name="NR_ID",
        ),
        SheetMappingRow(
            cobol_record_idms="VMB-FAR",
            cobol_zone="CD-NAME",
            new_db2_record="DZBFARTB",
            new_db2_field_name="CD_NAME",
        ),
    ]

def build_dclgen_columns() -> list[DclgenColumn]:
    """DCLGEN view DZBFARTV for the mapped record."""
    return [
        DclgenColumn(
            table_name="DZBFARTV",
            column_name="NR_ID",
            cobol_host_name="NR-ID",
        ),
        DclgenColumn(
            table_name="DZBFARTV",
            column_name="CD_NAME",
            cobol_host_name="CD-NAME",
        ),
    ]

def build_transformer_context() -> dict:
    """Fully wired IdmsStatementTransformer plus its collaborators.

    Returned keys:
        mapping_repository
        dclgen_repository
        table_resolver
        host_resolver
        statement_transformer
    """
    mapping_repository = MappingRepository(build_sheet_mapping_rows())
    dclgen_repository = DclgenRepository(build_dclgen_columns())

    table_resolver = TableNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
    )

    column_resolver = ColumnNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
    )

    host_resolver = HostVariableResolver(
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
    )

    cursor_resolver = CursorNameResolver()

    sql_generator = SqlGenerator(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
        column_name_resolver=column_resolver,
        host_variable_resolver=host_resolver,
    )

```

```

statement_transformer = IdmsStatementTransformer(
    sql_generator=sql_generator,
    sql_error_generator=SqlErrorGenerator(),
    table_name_resolver=table_resolver,
    cursor_name_resolver=cursor_resolver,
)

return {
    "mapping_repository": mapping_repository,
    "dclgen_repository": dclgen_repository,
    "table_resolver": table_resolver,
    "host_resolver": host_resolver,
    "statement_transformer": statement_transformer,
}

#
# Fixtures
#
@pytest.fixture
def transformer_context() -> dict:
    """Fresh transformer context per test. Never shared between tests."""
    return build_transformer_context()

@pytest.fixture
def statement_transformer(transformer_context) -> IdmsStatementTransformer:
    """The wired IdmsStatementTransformer on its own."""
    return transformer_context["statement_transformer"]

@pytest.fixture
def transform_line(statement_transformer):
    """Convert one IDMS line. Returns (joined_text, opened_set)."""

    def _transform(
        line: str,
        current_division: str = "PROCEDURE",
        sql_error_paragraph: str = "SQL-ERROR",
    ) -> tuple[str, str]:
        lines, opened_set = statement_transformer.transform_line(
            line=line,
            current_division=current_division,
            sql_error_paragraph=sql_error_paragraph,
        )
        return "\n".join(lines), opened_set

    return _transform

=====
FILE: tests\test_cobol_area_alignment_reflow.py
ENCODING: utf-8
=====

# LOCATION: tests/test_cobol_area_alignment_reflow.py
# ACTION: CREATE NEW FILE

from idms_db2_phase2.services.cobol_area_alignment_classifier import (
    CobolAreaAlignmentClassifier,
)
from idms_db2_phase2.services.cobol_area_alignment_reflow import (
    CobolAreaAlignmentReflow,
)
from idms_db2_phase2.services.fixed_format_line_service import (
    FixedFormatLineService,
)

def _fixed(body: str, seq: str = "002770", right: str = "02770000") -> str:
    return f"{seq} {body[:65].ljust(65)}{right}"

def _reflow() -> CobolAreaAlignmentReflow:
    return CobolAreaAlignmentReflow(
        fixed_format=FixedFormatLineService(),

```

```

        classifier=CobolAreaAlignmentClassifier(),
    )

def _body(line: str) -> str:
    return line[7:72].rstrip()

def test_two_line_move_keeps_both_lines_and_full_name():
    first = _fixed("        MOVE AM-CNSTK-479BEFF OF DCLDZBEFFTV")
    second = _fixed("        TO UIT-AM-CN-STOCK", seq="002780", right="02780000")

    out = _reflow().try_reflow_with_next_line(
        current_line=first,
        next_line=second,
        first_indent="    ",
    )

    assert len(out) == 2
    assert "UIT-AM-CN-STOCK" in _body(out[1])
    assert "MOVE AM-CNSTK-479BEFF OF DCLDZBEFFTV" in _body(out[0])

def test_continuation_is_indented_one_level_further():
    first = _fixed("        MOVE A OF B")
    second = _fixed("        TO C", seq="002780", right="02780000")

    out = _reflow().try_reflow_with_next_line(
        current_line=first,
        next_line=second,
        first_indent="    ",
    )

    assert _body(out[0]).startswith("        MOVE")
    assert _body(out[1]).startswith("        TO")

def test_never_truncates_a_data_name():
    long_target = "TO UIT-SOME-VERY-LONG-OUTPUT-FIELD-NAME-HERE"
    first = _fixed("        MOVE SOURCE-FIELD-NAME OF DCLDZBEFFTV")
    second = _fixed(f"        {long_target}", seq="002780", right="02780000")

    out = _reflow().try_reflow_with_next_line(
        current_line=first,
        next_line=second,
        first_indent="    ",
    )

    combined = " ".join(_body(line) for line in out) if out else ""
    assert out == [] or "UIT-SOME-VERY-LONG-OUTPUT-FIELD-NAME-HERE" in combined

def test_next_paragraph_header_is_never_joined():
    first = _fixed("        MOVE A TO B")
    second = _fixed("VERWERK-VMBEVEF.", seq="002780", right="02780000")

    out = _reflow().try_reflow_with_next_line(
        current_line=first,
        next_line=second,
        first_indent="    ",
    )

    assert out == []

def test_next_statement_is_never_joined():
    first = _fixed("        MOVE A TO B")
    second = _fixed("        PERFORM 710-OPEN-DZBEFFC1", seq="002780",
                    right="02780000")

    out = _reflow().try_reflow_with_next_line(
        current_line=first,
        next_line=second,
        first_indent="    ",
    )

```

```

assert out == []

def test_statement_is_not_collapsed_onto_one_line():
    first = _fixed("    MOVE A OF B")
    second = _fixed("    TO C", seq="002780", right="02780000")

    out = _reflow().try_reflow_with_next_line(
        current_line=first,
        next_line=second,
        first_indent="    ",
    )

    assert len(out) == 2
    assert _body(out[1]).strip() != ""

=====
FILE: tests\test_condition_is_never_truncated.py
ENCODING: utf-8
=====

# LOCATION: tests/test_condition_is_never_truncated.py
# ACTION: CREATE NEW FILE

"""No pass may lose characters from a PROCEDURE DIVISION statement.

REGRESSION
-----
A four-line IF condition was emitted as two lines truncated mid-name:

    IF (HELP-DA-CPTAFS-479BFAS < DA-ARCH-YMD AND HELP-DA-CPTAFS-
        (HELP-DA-CRFMAS-479BFAS < DA-ARCH-YMD AND HELP-DA-CPTAFS-4

losing "NOT = '00000000') OR" and "= '00000000'" entirely.
"""

from idms_db2_phase2.composers.procedure_indent_normalizer import (
    ProcedureIndentNormalizer,
)

QUOTE = chr(39)
ZERO8 = QUOTE + "00000000" + QUOTE

def fixed(seq: str, body: str, right: str) -> str:
    assert len(body) <= 65
    return seq + " " + body.ljust(65) + right

SOURCE = "\n".join(
    [
        fixed("001410", " PROCEDURE DIVISION.", "01410000"),
        fixed("001990", " BEHADELING.", "01990000"),
        fixed("002010", "     IF (HELP-DA-CPTAFS-479BFAS < DA-ARCH-YMD AND", "02010000"),
        fixed("002020", "         HELP-DA-CPTAFS-479BFAS NOT = " + ZERO8 + ") OR", "02020000"),
        fixed("002021", "         (HELP-DA-CRFMAS-479BFAS < DA-ARCH-YMD AND", "02021000"),
        fixed("002022", "         HELP-DA-CPTAFS-479BFAS = " + ZERO8 + ")", "02022000"),
        fixed("002030", "         ADD 1 TO WS-TELLER", "02030000"),
        fixed("002110", "     END-IF.", "02110000"),
    ]
) + "\n"

def _tokens(text: str) -> str:
    """Every non-blank character of every body, order preserved."""
    out = []

    for line in text.splitlines():
        body = line[7:72] if len(line) >= 72 else line
        out.append("".join(body.split()))

    return "".join(out)

def test_no_character_is_lost():

```

```

result = ProcedureIndentNormalizer().compose(SOURCE)

assert _tokens(result) == _tokens(SOURCE)

def test_no_identifier_is_cut_in_half():
    result = ProcedureIndentNormalizer().compose(SOURCE)

    assert "HELP-DA-CPTAFS-479BFAS" in result
    assert result.count("HELP-DA-CPTAFS-479BFAS") == 3
    assert ZER08 in result

def test_every_record_stays_eighty_columns():
    result = ProcedureIndentNormalizer().compose(SOURCE)

    for line in result.splitlines():
        if line[:6].isdigit():
            assert len(line) == 80, f"geometry broken: {line!r}"

=====
FILE: tests\test_db2_date_comparison.py
ENCODING: utf-8
=====

# LOCATION: tests/test_db2_date_comparison.py
# ACTION: CREATE NEW FILE

"""DB2 DATE comparison conversion.

REGRESSION CONTRACT
-----
A DB2 DATE host is stored as DD.MM.CCYY (10 bytes). A COBOL date field is
CCYYMMDD, PIC 9(8). Comparing them directly compares '0' against '2' and
selects the wrong rows. Every DATE host that takes part in a comparison
must therefore be realigned into its HELP- helper first.

The two historical blockers, each fatal on its own:

1. the host had to be the FIRST token after IF, so a parenthesised
   condition never matched,
2. the condition had to contain the literal token PARMDATE, a hardcoded
   host variable belonging to one program.
"""

import pytest

from idms_db2_phase2.composers.db2_date_comparison_composer import (
    Db2DateComparisonComposer,
)
from idms_db2_phase2.composers.db2_date_condition_assembler import (
    Db2DateConditionAssembler,
)
from idms_db2_phase2.composers.db2_date_field_detector import (
    Db2DateFieldDetector,
)

#
# Fixtures
#
@pytest.fixture
def composer() -> Db2DateComparisonComposer:
    return Db2DateComparisonComposer()

@pytest.fixture
def detector() -> Db2DateFieldDetector:
    return Db2DateFieldDetector()

@pytest.fixture
def assembler() -> Db2DateConditionAssembler:
    return Db2DateConditionAssembler()

```

```

def _program(body: str) -> str:
    return (
        "        WORKING-STORAGE SECTION.\n"
        "        01 WS-A                                PIC X.\n"
        "        PROCEDURE DIVISION.\n"
        f"{body}"
    )

#
# The exact VMDZ7200 condition
#
VMDZ7200_CONDITION = (
    "        IF (DA-CPTAFS-479BFAS OF DCLDZBFASTV < DA-ARCH-YMD\n"
    "            AND DA-CPTAFS-479BFAS OF DCLDZBFASTV NOT = '00000000') OR\n"
    "            (DA-CRFMAS-479BFAS OF DCLDZBFASTV < DA-ARCH-YMD AND\n"
    "            DA-CPTAFS-479BFAS OF DCLDZBFASTV = '00000000')\n"
    "        ADD 1 TO WS-TELLER\n"
    "        END-IF.\n"
)

#
# Assembler
#
def test_assembler_joins_a_four_line_parenthesised_condition(assembler):
    lines = _program(VMDZ7200_CONDITION).splitlines()
    start = next(
        index
        for index, line in enumerate(lines)
        if line.strip().startswith("IF (")
    )

    span = assembler.span_at(lines, start)

    assert span is not None
    assert span.complete is True
    assert span.line_count == 4
    assert "DA-CPTAFS-479BFAS" in span.text
    assert "DA-CRFMAS-479BFAS" in span.text

def test_assembler_stops_before_the_statement_body(assembler):
    lines = _program(VMDZ7200_CONDITION).splitlines()
    start = next(
        index
        for index, line in enumerate(lines)
        if line.strip().startswith("IF (")
    )

    span = assembler.span_at(lines, start)

    assert "ADD 1 TO WS-TELLER" not in span.text

#
# Detector
#
def test_detector_finds_both_hosts_in_a_compound_condition(detector):
    lines = _program(VMDZ7200_CONDITION).splitlines()

    fields = detector.date_fields_used_in_comparisons(lines)

    assert "DA-CPTAFS-479BFAS" in fields
    assert "DA-CRFMAS-479BFAS" in fields

def test_detector_no_longer_requires_the_token_parmdate(detector):
    body = (
        "        IF DA-CPTAFS-479BFAS OF DCLDZBFASTV < DA-ARCH-YMD\n"
        "        CONTINUE\n"
        "        END-IF.\n"
    )
    lines = _program(body).splitlines()

```



```

    assert detector.date_fields_used_in_comparisons(lines) == [
        "DA-CPTAFS-479BFAS"
    ]

def test_detector_ignores_a_reference_with_no_comparison(detector):
    body = (
        "            IF WS-A = 'Y'\n"
        "            MOVE DA-CPTAFS-479BFAS OF DCLDZBFASTV TO WS-A\n"
        "            END-IF.\n"
    )
    lines = _program(body).splitlines()

    assert detector.date_fields_used_in_comparisons(lines) == []

def test_detector_never_touches_an_exec_sql_host(detector):
    body = (
        "            EXEC SQL\n"
        "            FETCH DZBFASC1\n"
        "            INTO :DCLDZBFASTV.DA-CPTAFS-479BFAS\n"
        "            END-EXEC.\n"
    )
    lines = _program(body).splitlines()

    assert detector.date_fields_used_in_comparisons(lines) == []

#
# Composer - end to end
#
def test_compound_condition_is_rewritten_onto_helpers(composer):
    output = composer.compose(_program(VMDZ7200_CONDITION))

    assert "HELP-DA-CPTAFS-479BFAS" in output
    assert "HELP-DA-CRFMAS-479BFAS" in output
    assert "DA-CPTAFS-479BFAS OF DCLDZBFASTV <" not in output

def test_realignment_block_is_emitted_before_the_if(composer):
    output = composer.compose(_program(VMDZ7200_CONDITION))
    lines = output.splitlines()

    realign = next(
        index for index, line in enumerate(lines) if "DA-DD-MM-CCYY" in line
    )
    condition = next(
        index for index, line in enumerate(lines) if line.strip().startswith("IF")
    )

    assert realign < condition

def test_shared_date_working_storage_is_declared(composer):
    output = composer.compose(_program(VMDZ7200_CONDITION))

    assert "DA-CCYYMMDD" in output
    assert "DA-DD-MM-CCYY" in output

def test_original_line_breaks_are_preserved(composer):
    """The four physical lines of the condition stay four lines.

    Bounded precisely between the IF and the first statement of the
    body. An earlier version of this test counted every line mentioning
    HELP-DA-, which also caught the two Working-Storage declarations the
    pass legitimately generates, and reported 6.
    """
    output = composer.compose(_program(VMDZ7200_CONDITION))
    lines = output.splitlines()

    start = next(
        index
        for index, line in enumerate(lines)
        if line.strip().startswith("IF ")
    )

```

```

end = next(
    index
    for index, line in enumerate(lines)
    if "ADD 1 TO WS-TELLER" in line
)

condition_lines = [
    line for line in lines[start:end] if line.strip()
]

assert len(condition_lines) == 4
assert all("HELP-DA-" in line for line in condition_lines)

def test_one_helper_field_is_declared_per_converted_host(composer):
    """The realignment target must exist in Working-Storage."""
    output = composer.compose(_program(VMDZ7200_CONDITION))

    declarations = [
        line for line in output.splitlines()
        if "HELP-DA-" in line and "PIC" in line
    ]

    assert len(declarations) == 2
    assert any("HELP-DA-CPTAFS-479BFAS" in line for line in declarations)
    assert any("HELP-DA-CRFMAS-479BFAS" in line for line in declarations)

def test_literals_and_business_operands_are_untouched(composer):
    output = composer.compose(_program(VMDZ7200_CONDITION))

    assert "'00000000'" in output
    assert "DA-ARCH-YMD" in output
    assert "ADD 1 TO WS-TELLER" in output

#
# Diagnostics - the pass must never be silent
#
def test_conversion_is_reported(composer):
    composer.compose(_program(VMDZ7200_CONDITION))
    joined = " ".join(composer.messages)

    assert "DB2 date compare" in joined
    assert "realigned" in joined

def test_a_program_with_no_dates_says_so(composer):
    body = (
        "          IF WS-A = 'Y'\n"
        "          CONTINUE\n"
        "          END-IF.\n"
    )

    composer.compose(_program(body))

    assert any(
        "no DB2 DATE host is compared" in message
        for message in composer.messages
    )

def test_empty_text_is_safe(composer):
    assert composer.compose("") == ""
    assert composer.messages == []

#
# Fixed-format geometry
#
def _fixed(seq: str, body: str, right: str) -> str:
    """An 80-column line: 6 seq, 1 indicator, 65 body, 8 right seq."""
    return f"{seq} {body[:65].ljust(65)}{right}"

FIXED_CONDITION = [
    _fixed("002110", "          IF (DA-CPTAFS-479BFAS OF DCLDZBFAS TV < DA-ARCH-YMD", "02110000"),
    _fixed("002120", "          AND DA-CPTAFS-479BFAS OF DCLDZBFAS TV NOT = '00000000') OR", "02120000"),

```

```

    _fixed("002130", "      (DA-CRFMAS-479BFAS OF DCLDZBFASTV < DA-ARCH-YMD AND", "02130000"),
    _fixed("002140", "      DA-CPTAFS-479BFAS OF DCLDZBFASTV = '00000000')", "02140000"),
    _fixed("002150", "      ADD 1 TO WS-TELLER", "02150000"),
    _fixed("002160", "      END-IF.", "02160000"),
]

def test_substitution_keeps_every_line_at_eighty_columns():
    """The right sequence area must stay at column 73.

    Substituting into the raw line shortened it by 10 characters, slid
    the sequence number to column 63, and the indent normalizer then
    wrapped the corrupted body onto two lines.
    """
    from idms_db2_phase2.composers.db2_date_comparison_rewriter import (
        Db2DateComparisonRewriter,
    )

    header = [
        _fixed("001750", " PROCEDURE DIVISION.", "01750000"),
        _fixed("002290", " BEHANDELING.", "02290000"),
    ]
    output = Db2DateComparisonRewriter().rewrite_date_comparisons(
        header + FIXED_CONDITION
    )

    rewritten = [line for line in output if "HELP-DA-" in line and "MOVE" not in line]

    assert rewritten, "no condition line was rewritten"

    for line in rewritten:
        assert len(line) == 80, f"geometry broken: {line!r}"
        assert line[:6].isdigit(), f"left sequence lost: {line!r}"
        assert line[72:80].isdigit(), f"right sequence moved: {line!r}"

def test_a_four_line_condition_stays_four_lines_when_fixed_format():
    from idms_db2_phase2.composers.db2_date_comparison_rewriter import (
        Db2DateComparisonRewriter,
    )

    header = [
        _fixed("001750", " PROCEDURE DIVISION.", "01750000"),
        _fixed("002290", " BEHANDELING.", "02290000"),
    ]
    output = Db2DateComparisonRewriter().rewrite_date_comparisons(
        header + FIXED_CONDITION
    )

    rewritten = [line for line in output if "HELP-DA-" in line and "MOVE" not in line]

    assert len(rewritten) == 4

#
# Fixed-format geometry
#
def _fixed(seq: str, body: str, right: str) -> str:
    """An exact 80-column record: 6 seq + 1 indicator + 65 body + 8 seq."""
    assert len(body) <= 65
    return f"{seq} {body.ljust(65)}{right}"

FIXED_PROGRAM = [
    _fixed("001750", " PROCEDURE DIVISION.", "01750000"),
    _fixed("002290", " BEHANDELING.", "02290000"),
    _fixed("002110", "      IF (DA-CPTAFS-479BFAS OF DCLDZBFASTV < DA-ARCH-YMD", "02110000"),
    _fixed("002120", "      AND DA-CPTAFS-479BFAS OF DCLDZBFASTV NOT = '00000000') OR", "02120000"),
    _fixed("002130", "      (DA-CRFMAS-479BFAS OF DCLDZBFASTV < DA-ARCH-YMD AND", "02130000"),
    _fixed("002140", "      DA-CPTAFS-479BFAS OF DCLDZBFASTV = '00000000')", "02140000"),
    _fixed("002150", "      ADD 1 TO WS-TELLER", "02150000"),
    _fixed("002160", "      END-IF.", "02160000"),
]

def _rewrite_fixed():
    from idms_db2_phase2.composers.db2_date_comparison_rewriter import (

```

```

        Db2DateComparisonRewriter,
    )

    output = Db2DateComparisonRewriter().rewrite_date_comparisons(
        list(FIXED_PROGRAM)
    )
    return [line for line in output if "HELP-DA-" in line and "MOVE" not in line]

```

```

def test_every_rewritten_record_is_exactly_eighty_columns():
    for line in _rewrite_fixed():
        assert len(line) == 80, f"geometry broken: {line!r}"
        assert line[0:6].isdigit(), f"left sequence lost: {line!r}"
        assert line[72:80].isdigit(), f"right sequence moved: {line!r}"

```

```

def test_a_four_line_condition_stays_four_lines():
    assert len(_rewrite_fixed()) == 4

```

```

def test_no_stray_token_is_appended_to_the_body():
    """A wrapped pseudo-body once split the right sequence off as '0'."""
    for line in _rewrite_fixed():
        body = line[7:72].rstrip()
        assert not body.endswith(" 0"), f"stray token: {line!r}"
        assert "'" 0" not in body, f"stray token: {line!r}"
        assert not body.endswith("AND 0"), f"stray token: {line!r}"

```

```

def test_the_condition_text_is_intact():
    bodies = [line[7:72].rstrip() for line in _rewrite_fixed()]
    joined = " ".join(bodies)

    assert "HELP-DA-CPTAFS-479BFAS < DA-ARCH-YMD" in joined
    assert "HELP-DA-CPTAFS-479BFAS NOT = '00000000'" in joined
    assert "HELP-DA-CRFMAS-479BFAS < DA-ARCH-YMD" in joined
    assert "OF DCLDZBFASTV" not in joined

```

```

=====
FILE: tests\test_field_reference_geometry.py
ENCODING: utf-8
=====

```

```

# LOCATION: tests/test_field_reference_geometry.py
# ACTION: CREATE NEW FILE

```

```

"""Fixed-format geometry of the qualified field rewrite.

```

```

REGRESSION

```

```

-----
0002020          DA-CPTA-FORM-AS OF VMBFAS NOT = '00000000') OR          00002020

```

```

is 80 columns. The rewrite makes the text 7 characters longer, so
substituting into the WHOLE record produced 87 characters and pushed
00002020 from column 73 to column 80. Later passes read the first
displaced digit as body text and emitted

```

```

    AND HELP-DA-CPTAFS-479BFAS NOT = '00000000') OR 0
"""

```

```

import pytest

```

```

from idms_db2_phase2.services.fixed_format_line_service import (
    FixedFormatLineService,
)
from idms_db2_phase2.transformers.field_reference_rewriter import (
    FieldReferenceRewriter,
)

```

```

QUOTE = chr(39)
ZER08 = QUOTE + "00000000" + QUOTE

```

```

def fixed(seq: str, body: str, right: str) -> str:
    assert len(body) <= 65

```

```

return seq + " " + body.ljust(65) + right

SOURCE = "\n".join(
    [
        fixed("001410", " PROCEDURE DIVISION.", "00001410"),
        fixed("001990", " BEHANDLING.", "00001990"),
        fixed("002010", "      IF (DA-CPTA-FORM-AS OF VMBFAS < DA-ARCH-YMD AND", "00002010"),
        fixed("002020", "      DA-CPTA-FORM-AS OF VMBFAS NOT = " + ZERO8 + ") OR", "00002020"),
        fixed("002030", "      ADD 1 TO WS-TELLER", "00002030"),
        fixed("002110", "      END-IF.", "00002110"),
    ]
) + "\n"

@pytest.fixture
def rewriter(transformer_context):
    """Reuses the shared repositories from tests/conftest.py."""
    return FieldReferenceRewriter(
        mapping_repository=transformer_context["mapping_repository"],
        table_name_resolver=transformer_context["table_resolver"],
        host_variable_resolver=transformer_context["host_resolver"],
    )

def test_every_emitted_record_keeps_its_geometry(rewriter):
    output = rewriter.rewrite(SOURCE)

    for line in output.splitlines():
        if not line.strip():
            continue
        if not line[:6].isdigit():
            continue

        assert len(line) == 80, f"geometry broken: {line!r}"
        assert line[72:80].isdigit(), f"right sequence moved: {line!r}"

def test_no_stray_sequence_digit_leaks_into_the_body(rewriter):
    output = rewriter.rewrite(SOURCE)

    for line in output.splitlines():
        body = line[7:72].rstrip() if len(line) >= 72 else line
        assert not body.endswith(" 0"), f"stray digit: {line!r}"

def test_trailing_blanks_are_preserved(rewriter):
    """Blanks in columns 8-72 position the right sequence at column 73."""
    output = rewriter.rewrite(SOURCE)
    lines = [line for line in output.splitlines() if line[:6].isdigit()]

    assert lines
    assert all(len(line) == 80 for line in lines)

def test_a_line_with_no_reference_is_untouched(rewriter):
    output = rewriter.rewrite(SOURCE)

    assert any(
        "ADD 1 TO WS-TELLER" in line for line in output.splitlines()
    )

```

```

=====
FILE: tests\test_idms_statement_set_names.py
ENCODING: utf-8
=====

# LOCATION: tests/test_idms_statement_set_names.py
# ACTION: CREATE NEW FILE

"""IDMS name rendering in generated *DB2: comments.

No import from another test module. `transform_line` is a fixture
supplied by tests/conftest.py.

```

CONTRACT

An IDMS SET or RECORD name written into generated COBOL text must keep its IDMS spelling (hyphens). NameNormalizer.normalize() produces a DB2 identifier and must never reach a comment.
"""

```
def test_set_name_keeps_idms_spelling_in_the_comment(transform_line):
    text, _opened = transform_line("OBTAIN FIRST VMB-FAR WITHIN AR-VMBFRM1.")

    assert "WITHIN AR-VMBFRM1" in text
    assert "AR_VMBFRM1" not in text

def test_record_name_keeps_cobol_spelling_in_the_comment(transform_line):
    text, _opened = transform_line("OBTAIN FIRST VMB-FAR WITHIN AR-VMBFRM1.")

    assert "OBTAIN FIRST VMB-FAR" in text
    assert "VMB_FAR" not in text

def test_obtain_next_keeps_idms_spelling(transform_line):
    text, _opened = transform_line("OBTAIN NEXT VMB-FAR WITHIN AR-VMBFRM1.")

    assert "OBTAIN NEXT VMB-FAR WITHIN AR-VMBFRM1" in text

def test_opened_set_is_still_the_normalized_key(transform_line):
    _text, opened = transform_line("OBTAIN FIRST VMB-FAR WITHIN AR-VMBFRM1.")

    assert opened == "AR_VMBFRM1"

def test_unmapped_record_never_emits_an_empty_cursor_perform(transform_line):
    text, opened = transform_line("OBTAIN FIRST NOSUCHREC WITHIN AR-VMBFRM1.")

    assert "PERFORM OPEN-." not in text
    assert "PERFORM FETCH-." not in text
    assert opened == ""

def test_find_first_without_a_record_is_kept_and_commented(transform_line):
    text, opened = transform_line("FIND FIRST WITHIN AR-VMBFRM1.")

    assert "PERFORM OPEN-." not in text
    assert opened == ""

def test_obtain_outside_procedure_division_is_only_commented(transform_line):
    text, opened = transform_line(
        "OBTAIN FIRST VMB-FAR WITHIN AR-VMBFRM1.",
        current_division="DATA",
    )

    assert "PERFORM" not in text
    assert opened == ""
```

```
=====
FILE: tests\test_imports.py
ENCODING: utf-8
=====
```

```
def test_core_imports():
    from idms_db2_phase2.domain.models import ConversionInput
    from idms_db2_phase2.orchestration.conversion_service import ConversionService
    from idms_db2_phase2.parsers.sheet_mapping_parser import SheetMappingParser
    from idms_db2_phase2.parsers.dclgen_parser import DclgenParser
    from idms_db2_phase2.parsers.copybook_parser import CopybookParser
    from idms_db2_phase2.parsers.cobol_parser import CobolParser

    assert ConversionInput is not None
    assert ConversionService is not None
    assert SheetMappingParser is not None
    assert DclgenParser is not None
```

```

assert CopybookParser is not None
assert CobolParser is not None

```

```

def test_clean_architecture_imports():
    from catalogs.sheet_mapping_schema import SHEET_MAPPING_MODEL_FIELD_MAP
    from patterns.idms_patterns import OBTAIN_CALC_PATTERN
    from rules.authority_rules import INPUT_AUTHORITY_RULES

    assert SHEET_MAPPING_MODEL_FIELD_MAP
    assert OBTAIN_CALC_PATTERN
    assert INPUT_AUTHORITY_RULES

def test_layer_imports():
    from idms_db2_phase2.repositories.mapping_repository import MappingRepository
    from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
    from idms_db2_phase2.generators.sql_generator import SqlGenerator
    from idms_db2_phase2.transformers.cobol_transformer import CobolTransformer
    from idms_db2_phase2.validators.production_validator import ProductionValidator
    from idms_db2_phase2.composers.cobol_formatter import CobolFormatter

    assert MappingRepository is not None
    assert TableNameResolver is not None
    assert SqlGenerator is not None
    assert CobolTransformer is not None
    assert ProductionValidator is not None
    assert CobolFormatter is not None

```

```

=====
FILE: tests\test_lrf_path_expander.py
ENCODING: utf-8
=====

```

```

# LOCATION: tests/test_lrf_path_expander.py
# ACTION: REPLACE ENTIRE FILE
"""LRF path expansion tests.

```

```

ASSERTION CONTRACT
-----

```

```

The expander KEEPS an audit trail: every expanded statement is preceded by
a '*DB2:' comment quoting the original LRF statement. That comment is a
feature, not residue, so assertions about "no LRF verb remains" must look
at EXECUTABLE lines only. Asserting against the whole output text would
forbid the audit trail.
"""

```

```

from idms_db2_phase2.parsers.lrf_parser import LrfParser
from idms_db2_phase2.repositories.lrf_repository import LrfRepository
from idms_db2_phase2.resolvers.lrf_path_resolver import LrfPathResolver
from idms_db2_phase2.transformers.lrf_path_expander import LrfPathExpander

```

```

SUBSCHEMA = """
    ADD SUBSCHEMA NAME IS VMBTS03 OF SCHEMA NAME IS VMBTSCH
    .
    ADD LOGICAL RECORD VMBTL03-R01
        ELEMENTS ARE
            VMBSIAS
            VMBFAS
        .
    ADD PATH-GROUP OBTAIN VMBTL03-R01
        SELECT FOR KEYWORD SWEEP-VMBFAS
            OBTAIN EACH VMBFAS WITHIN AR-VMBFRM1
                ON 0307 RETURN VMBFAS-EOA
                ON 0000 NEXT
        SELECT FOR KEYWORD VMBFAS-BY-VMBSIAS
            FIND FIRST VMBSIAS WITHIN AR-VMBFRM1
                WHERE CALCKEY EQ KY-SIFORM OF VMBSIAS OF LR
                ON 0326 RETURN VMBSIAS-NOT-FND
                ON 0000 NEXT
            OBTAIN EACH VMBFAS WITHIN VMBSIAS-VMBFAS
                ON 0000 NEXT
                ON 0307 RETURN NO-MORE-VMBFAS
        .

```

```
"""
```

```
COMMENT_INDICATORS = ("*", "/" )
```

```
# ----- helpers
```

```
def _expander():
    records = LrfParser().parse(SUBSCHEMA)
    resolver = LrfPathResolver(
        lrf_repository=LrfRepository(records),
        mapping_repository=None,
    )
    return LrfPathExpander(path_resolver=resolver)
```

```
def _is_comment(line: str) -> bool:
    text = str(line or "")
    if len(text) >= 7 and text[:6].strip().isdigit():
        return text[6] in COMMENT_INDICATORS
    return text.lstrip().startswith(COMMENT_INDICATORS)
```

```
def _executable(output: str) -> str:
    """Only the executable lines - comments and audit trail removed."""
    return "\n".join(
        line for line in str(output or "").splitlines()
        if line.strip() and not _is_comment(line)
    )
```

```
def _comments(output: str) -> str:
    """Only the comment lines."""
    return "\n".join(
        line for line in str(output or "").splitlines()
        if _is_comment(line)
    )
```

```
# ----- parser
```

```
def test_parser_reads_logical_record_and_paths():
    records = LrfParser().parse(SUBSCHEMA)
    assert len(records) == 1

    record = records[0]
    assert record.logical_record_name == "VMBTL03-R01"
    assert record.element_records == ["VMBSIAS", "VMBFAS"]
    assert {path.keyword for path in record.paths} == {
        "SWEEP-VMBFAS",
        "VMBFAS-BY-VMBSIAS",
    }
```

```
# ----- expansion
```

```
def test_obtain_first_is_expanded_to_classic_idms():
    source = (
        "        PROCEDURE DIVISION.\n"
        "        OBTAIN FIRST VMBTL03-R01 WHERE SWEEP-VMBFAS.\n"
    )
    out = _expander().expand(source)
    code = _executable(out)

    assert "OBTAIN FIRST VMBFAS WITHIN AR-VMBFRM1." in code
    # No logical-record verb survives as EXECUTABLE code.
    assert "VMBTL03-R01" not in code
    assert "WHERE SWEEP-VMBFAS" not in code
```

```
def test_expansion_keeps_an_audit_trail():
    source = (
        "        PROCEDURE DIVISION.\n"
        "        OBTAIN FIRST VMBTL03-R01 WHERE SWEEP-VMBFAS.\n"
    )
    out = _expander().expand(source)
    notes = _comments(out)

    assert "*DB2:" in notes
```



```

assert "VMBTL03-R01" in notes
assert "SWEEP-VMBFAS" in notes

def test_obtain_next_is_expanded_without_parent_access():
    source = (
        "        PROCEDURE DIVISION.\n"
        "        OBTAIN NEXT VMBTL03-R01 WHERE VMBFAS-BY-VMBSIAS.\n"
    )
    code = _executable(_expander().expand(source))

    assert "OBTAIN NEXT VMBFAS WITHIN VMBSIAS-VMBFAS." in code
    assert "FIND FIRST VMBSIAS" not in code

def test_obtain_first_emits_parent_access_for_join_path():
    source = (
        "        PROCEDURE DIVISION.\n"
        "        OBTAIN FIRST VMBTL03-R01 WHERE VMBFAS-BY-VMBSIAS.\n"
    )
    code = _executable(_expander().expand(source))

    assert "FIND FIRST VMBSIAS WITHIN AR-VMBFRM1." in code
    assert "OBTAIN FIRST VMBFAS WITHIN VMBSIAS-VMBFAS." in code

    # Parent must precede the child access.
    assert code.index("FIND FIRST VMBSIAS") < code.index("OBTAIN FIRST VMBFAS")

def test_expanded_names_use_cobol_hyphens_never_underscores():
    source = (
        "        PROCEDURE DIVISION.\n"
        "        OBTAIN FIRST VMBTL03-R01 WHERE SWEEP-VMBFAS.\n"
    )
    out = _expander().expand(source)

    assert "_" not in out
    assert "AR-VMBFRM1" in out

# ----- LR-STATUS
def test_lr_status_condition_becomes_end_of_set_token():
    source = (
        "        PROCEDURE DIVISION.\n"
        "        PERFORM BEHADELING UNTIL LR-STATUS = 'VMBFAS-EOA'.\n"
    )
    code = _executable(_expander().expand(source))

    assert "DB-END-OF-SET" in code
    assert "LR-STATUS" not in code

def test_not_found_literal_becomes_record_not_found_token():
    source = (
        "        PROCEDURE DIVISION.\n"
        "        IF LR-STATUS = 'VMBSIAS-NOT-FND'\n"
    )
    code = _executable(_expander().expand(source))

    assert "DB-REC-NOT-FOUND" in code
    assert "LR-STATUS" not in code

# ----- OF LR suffix
def test_of_lr_qualifier_is_stripped():
    source = (
        "        PROCEDURE DIVISION.\n"
        "        MOVE KY-SIFORM OF VMBSIAS OF LR TO WS-KEY.\n"
    )
    code = _executable(_expander().expand(source))

    assert "KY-SIFORM OF VMBSIAS TO WS-KEY" in code
    assert "OF LR" not in code

# ----- safe refusal

```

```

def test_unknown_keyword_is_kept_and_commented():
    source = (
        "        PROCEDURE DIVISION.\n"
        "        OBTAIN FIRST VMBTL03-R01 WHERE UNKNOWN-KEYWORD.\n"
    )
    out = _expander().expand(source)
    notes = _comments(out)
    code = _executable(out)

    assert "*DB2-KEEP:" in notes
    assert "UNKNOWN-KEYWORD" in notes
    # Nothing was guessed: no expanded statement was emitted.
    assert "OBTAIN FIRST" not in code

# ----- no-LRF contract
def test_without_lrf_metadata_source_is_unchanged():
    resolver = LrfPathResolver(
        lrf_repository=LrfRepository([]),
        mapping_repository=None,
    )
    expander = LrfPathExpander(path_resolver=resolver)
    source = (
        "        PROCEDURE DIVISION.\n"
        "        OBTAIN FIRST VMBTL03-R01 WHERE SWEEP-VMBFAS.\n"
    )
    assert expander.expand(source) == source

def test_program_without_lrf_syntax_is_unchanged():
    source = (
        "        PROCEDURE DIVISION.\n"
        "        OBTAIN FIRST VMBFAS WITHIN AR-VMBFRM1.\n"
        "        MOVE NR-IS-FORM-AS OF VMBFAS TO W-NR.\n"
    )
    assert _expander().expand(source) == source

def test_no_path_resolver_returns_source_unchanged():
    expander = LrfPathExpander(path_resolver=None)
    source = (
        "        PROCEDURE DIVISION.\n"
        "        OBTAIN FIRST VMBTL03-R01 WHERE SWEEP-VMBFAS.\n"
    )
    assert expander.expand(source) == source

# ----- safe scope
def test_data_division_lines_are_never_touched():
    source = (
        "        DATA DIVISION.\n"
        "        01 LR-STATUS PIC X(16).\n"
    )
    out = _expander().expand(source)
    assert "01 LR-STATUS PIC X(16)." in out

def test_comment_lines_are_never_expanded():
    source = (
        "        PROCEDURE DIVISION.\n"
        "        * OBTAIN FIRST VMBTL03-R01 WHERE SWEEP-VMBFAS.\n"
    )
    out = _expander().expand(source)
    assert "OBTAIN FIRST VMBFAS" not in _executable(out)

```

```

=====
FILE: tests\test_name_derivation.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.services.name_derivation_resolver import (
    NameDerivationResolver,
)

```

```

def _r():
    return NameDerivationResolver()

def test_program_id_with_site_char():
    # VM<site>BD<tail> -> VMDZ<site><tail>
    assert _r().derive("VM1BD567") == "VMDZ1567"

def test_record_name_without_site_char():
    # VMBD<tail> -> VMDZ<tail>
    assert _r().derive("VMBD205I") == "VMDZ205I"

def test_generic_vm_bd_name():
    assert _r().derive("VMBD510X") == "VMDZ510X"

def test_non_matching_name_unchanged():
    assert _r().derive("OPERATIES") == "OPERATIES"

def test_empty_and_none_safe():
    assert _r().derive("") == ""
    assert _r().derive(None) == ""

def test_is_derivable_flags():
    r = _r()
    assert r.is_derivable("VM1BD567") is True
    assert r.is_derivable("VMBD205I") is True
    assert r.is_derivable("OPERATIES") is False

=====
FILE: tests\test_name_normalizer.py
ENCODING: utf-8
=====

from idms_db2_phase2.services.name_normalizer import NameNormalizer

def test_normalize_replaces_hyphen_with_underscore():
    assert NameNormalizer.normalize("VMB-FAR") == "VMB_FAR"

def test_to_cobol_replaces_underscore_with_hyphen():
    assert NameNormalizer.to_cobol("NR_ID_479_FAR") == "NR-ID-479-FAR"

def test_remove_record_suffix_removes_four_digit_suffix():
    assert NameNormalizer.remove_record_suffix("FIELD_NAME_0410") == "FIELD_NAME"

def test_compact_removes_non_alphanumeric_characters():
    assert NameNormalizer.compact("DZ-BFAR_TV") == "DZBFARTV"

def test_normalize_none_returns_empty_string():
    assert NameNormalizer.normalize(None) == ""

=====
FILE: tests\test_output_write_paragraph_composer.py
ENCODING: utf-8
=====

# LOCATION: tests/test_output_write_paragraph_composer.py
# ACTION: CREATE NEW FILE

"""Output write paragraph extraction tests."""

from idms_db2_phase2.composers.output_write_paragraph_composer import (
    OutputWriteParagraphComposer,
)

```

```

def _fixed(body: str, seq: str = "001000", right: str = "01000000") -> str:
    """One 80-column fixed-format line."""
    return f"{seq} {body[:65].ljust(65)}{right}"

def _program() -> str:
    return "\n".join([
        _fixed("PROCEDURE DIVISION."),
        _fixed("HOOFDVERWERKING."),
        _fixed("    PERFORM 830-CLOSE-DZEVEFC1"),
        _fixed("    IF NOT SW-STATUS-D = 'Y'"),
        _fixed("        IF WS-STATUS = 'C'"),
        _fixed("            INITIALIZE UITRECORD"),
        _fixed("            MOVE WS-NR-CIO-CRE TO UIT-NR-CIO-CRE"),
        _fixed("            WRITE UITRECORD"),
        _fixed("        END-IF"),
        _fixed("    END-IF."),
        _fixed("710-OPEN-DZBEFFC1."),
        _fixed("    CONTINUE."),
    ])

def test_write_block_is_extracted_into_its_own_paragraph():
    out = OutputWriteParagraphComposer().compose(_program())

    assert "WRITE-UITRECORD." in out
    assert "PERFORM WRITE-UITRECORD" in out
    assert out.count("WRITE UITRECORD") == 1

def test_population_statements_move_into_the_new_paragraph():
    out = OutputWriteParagraphComposer().compose(_program())
    tail = out.split("WRITE-UITRECORD.", 1)[1]

    assert "INITIALIZE UITRECORD" in tail
    assert "MOVE WS-NR-CIO-CRE TO UIT-NR-CIO-CRE" in tail

def test_counter_increment_is_added_at_the_call_site():
    out = OutputWriteParagraphComposer().compose(_program())
    call_site = out.split("WRITE-UITRECORD.", 1)[0]

    assert "ADD 1 TO WS-NB-OUTPUT-COUNT" in call_site

def test_guard_conditions_are_left_untouched():
    out = OutputWriteParagraphComposer().compose(_program())

    assert "IF NOT SW-STATUS-D = 'Y'" in out
    assert "IF WS-STATUS = 'C'" in out

def test_every_line_stays_eighty_columns():
    out = OutputWriteParagraphComposer().compose(_program())

    for line in out.splitlines():
        if line.strip():
            assert len(line) == 80, repr(line)

def test_diagnostics_report_the_extraction():
    composer = OutputWriteParagraphComposer()
    composer.compose(_program())
    joined = " ".join(composer.messages)

    assert "extracted record population" in joined
    assert "WRITE-UITRECORD" in joined

def test_two_writes_are_left_alone():
    text = _program().replace(
        _fixed("        WRITE UITRECORD"),
        _fixed("        WRITE UITRECORD")
        + "\n"
        + _fixed("        WRITE UITRECORD"),
    )

```

```

)
out = OutputWriteParagraphComposer().compose(text)

assert "PERFORM WRITE-UITRECORD" not in out

def test_extraction_is_idempotent():
    composer = OutputWriteParagraphComposer()
    once = composer.compose(_program())
    twice = composer.compose(once)

    assert once == twice

def test_program_without_a_write_is_unchanged():
    text = "\n".join([
        _fixed("PROCEDURE DIVISION."),
        _fixed("HOOFDVERWERKING."),
        _fixed("    IF WS-STATUS = 'C'"),
        _fixed("        MOVE 'X' TO WS-STATUS"),
        _fixed("    END-IF."),
    ])
    out = OutputWriteParagraphComposer().compose(text)

    assert "PERFORM WRITE-" not in out

=====
FILE: tests\test_parsers.py
ENCODING: utf-8
=====

from idms_db2_phase2.parsers.cobol_parser import CobolParser
from idms_db2_phase2.parsers.copybook_parser import CopybookParser
from idms_db2_phase2.parsers.dclgen_parser import DclgenParser
from idms_db2_phase2.parsers.sheet_mapping_parser import SheetMappingParser

def test_cobol_parser_detects_program_id():
    parser = CobolParser()

    assert parser.program_id("PROGRAM-ID. VMDZ1567.") == "VMDZ1567"

def test_cobol_parser_detects_obtain_calc():
    parser = CobolParser()

    operations = parser.analyze("OBTAIN VMB-FAR CALC.")

    assert len(operations) == 1
    assert operations[0].operation == "OBTAIN_CALC"
    assert operations[0].record_name == "VMB-FAR"

def test_cobol_parser_detects_store():
    parser = CobolParser()

    operations = parser.analyze("STORE VMB-FAR.")

    assert len(operations) == 1
    assert operations[0].operation == "STORE"
    assert operations[0].record_name == "VMB-FAR"

def test_copybook_parser_parses_field():
    parser = CopybookParser()

    fields = parser.parse(
        """
        10 NR-ID PIC 9(8).
        """
    )

    assert len(fields) == 1
    assert fields[0].name == "NR-ID"
    assert fields[0].picture == "9(8)"

```

```

def test_dclgen_parser_pares_inline_sql_declare():
    parser = DclgenParser()

    columns = parser.parse(
        """
        EXEC SQL DECLARE DZBFARTV TABLE
        (
            NR_ID DECIMAL(8,0) NOT NULL,
            CD_NAME CHAR(4)
        )
        END-EXEC.

        01 DCLDZBFARTV.
          10 NR-ID PIC S9(8) COMP-3.
          10 CD-NAME PIC X(4).
        """
    )

    assert len(columns) == 2
    assert columns[0].table_name == "DZBFARTV"
    assert columns[0].column_name == "NR_ID"
    assert columns[0].cobol_host_name == "NR-ID"

def test_sheet_mapping_parser_pares_csv_text():
    parser = SheetMappingParser()

    rows = parser.parse_csv_text(
        "Cobol Record IDMS,Cobol Zone,IDMS Key,DB2 Key,New DB2 Record,New DB2 Field name\n"
        "VMB-FAR,NR-ID,CALC,PRIMARY KEY,DZBFARTB,NR_ID\n"
    )

    assert len(rows) == 1
    assert rows[0].cobol_record_idms == "VMB-FAR"
    assert rows[0].new_db2_record == "DZBFARTB"
    assert rows[0].new_db2_field_name == "NR_ID"

=====
FILE: tests\test_patterns.py
ENCODING: utf-8
=====

from patterns.cobol_patterns import PROGRAM_ID_PATTERN
from patterns.dclgen_patterns import DECLARE_TABLE_PATTERN
from patterns.idms_patterns import OBTAIN_CALC_PATTERN, STORE_PATTERN
from patterns.sql_patterns import EXEC_SQL_PATTERN, UPDATE_SQL_PATTERN
from patterns.validation_patterns import TODO_DB2_PATTERN

def test_program_id_pattern_matches():
    assert PROGRAM_ID_PATTERN.search("PROGRAM-ID. VMDZ1567.")

def test_dclgen_declare_table_pattern_matches():
    text = "EXEC SQL DECLARE DZBFARTV TABLE"

    match = DECLARE_TABLE_PATTERN.search(text)

    assert match
    assert match.group(1).strip() == "DZBFARTV"

def test_obtain_calc_pattern_matches():
    match = OBTAIN_CALC_PATTERN.search("OBTAIN VMB-FAR CALC.")

    assert match
    assert match.group("record") == "VMB-FAR"

def test_store_pattern_matches():
    match = STORE_PATTERN.search("STORE VMB-FAR.")

```

```

assert match
assert match.group("record") == "VMB-FAR"

def test_exec_sql_pattern_matches():
    assert EXEC_SQL_PATTERN.match("EXEC SQL")

def test_update_sql_pattern_matches_table():
    match = UPDATE_SQL_PATTERN.search("UPDATE DZBFARTV")

    assert match
    assert match.group("table") == "DZBFARTV"

def test_todo_db2_validation_pattern_matches():
    assert TODO_DB2_PATTERN.search("TODO DB2 mapping missing")

```

```

=====
FILE: tests\test_record_picture_sizer.py
ENCODING: utf-8
=====

```

```

# LOCATION: tests/test_record_picture_sizer.py
# ACTION: CREATE NEW FILE

```

```

"""PICTURE sizing.

```

```

REGRESSION
-----

```

```

    05  F-FORM                PIC X(478).

```

```

was measured at 1 byte because only the symbol 'X' was read. The length
guard then refused a 432-byte VMBFAS layout as "longer than the 1 byte
F-FORM declares", discarding roughly 400 lines of generated output.
"""

```

```

import pytest

```

```

from idms_db2_phase2.composers.record_materialisation.record_picture_sizer import (
    RecordPictureSizer,
)

```

```

@pytest.mark.parametrize(
    "body, expected",
    [
        ("05  F-FORM                PIC X(478).", 478),
        ("05  F-NR-IS                PIC 9(7).", 7),
        ("05  F-SW-ARCH              PIC X.", 1),
        ("01  DAARCH-REC             PIC X(20).", 20),
        ("05  W-NR-ID-ISSUE          PIC 9(7).", 7),
        ("03  D-YEAR                 PIC 9(2).", 2),
        ("05  CCYY                   PIC 9999.", 4),
        ("05  F-TELLER               PIC S9(11) COMP-3.", 6),
    ],
)

```

```

def test_declared_bytes_reads_the_full_clause(body, expected):
    assert RecordPictureSizer.declared_bytes(body) == expected

```

```

def test_a_truncated_caller_capture_cannot_win():
    """The caller's group says 'X'; the line says 'X(478)'."""

```

```

    class _Match:
        def group(self, name):
            return "X"

```

```

    body = "05  F-FORM                PIC X(478)."

```

```

    assert RecordPictureSizer.declared_bytes(body, _Match()) == 478

```

```

def test_the_data_name_is_never_measured():
    """F-TARGET contains an A, which the symbol class matches."""

```

```

assert RecordPictureSizer.declared_bytes(
    "05 F-TARGET          PIC X(20)."
) == 20

def test_comp_3_is_halved():
    assert RecordPictureSizer.size_of("S9(11)", "COMP-3") == 6
    assert RecordPictureSizer.size_of("9(7)", "COMP-3") == 4

def test_empty_and_unreadable_are_zero():
    assert RecordPictureSizer.declared_bytes("") == 0
    assert RecordPictureSizer.declared_bytes("05 F-GROUP.") == 0
    assert RecordPictureSizer.size_of("") == 0

def test_repeat_counts_are_honoured():
    """The defect: (n) was ignored and every clause measured 1."""
    assert RecordPictureSizer.size_of("X(478)") == 478
    assert RecordPictureSizer.size_of("9(7)") == 7
    assert RecordPictureSizer.size_of("9999") == 4
    assert RecordPictureSizer.size_of("XXX") == 3
    assert RecordPictureSizer.size_of("S9(13)V99") == 15

def test_whitespace_inside_the_repeat_is_tolerated():
    """The mapping workbook contains 'PIC X(4 )'."""
    assert RecordPictureSizer.declared_bytes("05 A-FIELD  PIC X(4 ).") == 4

def test_a_value_clause_is_not_measured():
    assert RecordPictureSizer.declared_bytes(
        "77 USERABEN  PIC X(8)  VALUE 'USERABEN'."
    ) == 8

```

```

=====
FILE: tests\test_repositories.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.domain.models import CopybookField, DclgenColumn, SheetMappingRow
from idms_db2_phase2.repositories.copybook_repository import CopybookRepository
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository

```

```

def test_mapping_repository_returns_records():
    repository = MappingRepository(
        [
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                new_db2_record="DZBFARTB",
                new_db2_field_name="NR_ID",
            )
        ]
    )

    assert repository.records() == ["VMB_FAR"]

def test_mapping_repository_finds_rows_for_record():
    repository = MappingRepository(
        [
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                new_db2_record="DZBFARTB",
                new_db2_field_name="NR_ID",
            )
        ]
    )

    rows = repository.rows_for_record("VMB-FAR")

    assert len(rows) == 1
    assert rows[0].new_db2_record == "DZBFARTB"

```



```

def test_mapping_repository_returns_table_for_record():
    repository = MappingRepository(
        [
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                new_db2_record="DZBFARTB",
            )
        ]
    )

    assert repository.db2_table_for_record("VMB-FAR") == "DZBFARTB"

def test_dclgen_repository_has_table():
    repository = DclgenRepository(
        [
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="NR_ID",
                cobol_host_name="NR-ID",
            )
        ]
    )

    assert repository.has_table("DZBFARTV")

def test_dclgen_repository_host_for_column():
    repository = DclgenRepository(
        [
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="NR_ID",
                cobol_host_name="NR-ID",
            )
        ]
    )

    assert repository.host_for_column("DZBFARTV", "NR_ID") == "NR-ID"

def test_copybook_repository_finds_field():
    repository = CopybookRepository(
        [
            CopybookField(
                level="10",
                name="NR-ID",
                picture="9(8)",
            )
        ]
    )

    assert repository.has_field("NR-ID")
    assert repository.picture_for("NR-ID") == "9(8)"

```

```

=====
FILE: tests\test_transformers.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.domain.models import DclgenColumn, SheetMappingRow
from idms_db2_phase2.generators.sql_error_generator import SqlErrorGenerator
from idms_db2_phase2.generators.sql_generator import SqlGenerator
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.column_name_resolver import ColumnNameResolver
from idms_db2_phase2.resolvers.cursor_name_resolver import CursorNameResolver
from idms_db2_phase2.resolvers.host_variable_resolver import HostVariableResolver
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.transformers.cobol_transformer import CobolTransformer
from idms_db2_phase2.transformers.field_reference_rewriter import FieldReferenceRewriter
from idms_db2_phase2.transformers.idms_statement_transformer import (
    IdmsStatementTransformer,
)

```

```

def build_transformer_context():
    mapping_repository = MappingRepository(
        [
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                cobol_zone="NR-ID",
                idms_key="CALC",
                db2_key="PRIMARY KEY",
                new_db2_record="DZBFARTB",
                new_db2_field_name="NR_ID",
            ),
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                cobol_zone="CD-NAME",
                new_db2_record="DZBFARTB",
                new_db2_field_name="CD_NAME",
            ),
        ]
    )

    dclgen_repository = DclgenRepository(
        [
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="NR_ID",
                cobol_host_name="NR-ID",
            ),
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="CD_NAME",
                cobol_host_name="CD-NAME",
            ),
        ]
    )

    table_resolver = TableNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
    )

    column_resolver = ColumnNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
    )

    host_resolver = HostVariableResolver(
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
    )

    cursor_resolver = CursorNameResolver()

    sql_generator = SqlGenerator(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
        column_name_resolver=column_resolver,
        host_variable_resolver=host_resolver,
    )

    statement_transformer = IdmsStatementTransformer(
        sql_generator=sql_generator,
        sql_error_generator=SqlErrorGenerator(),
        table_name_resolver=table_resolver,
        cursor_name_resolver=cursor_resolver,
    )

    return {
        "mapping_repository": mapping_repository,
        "dclgen_repository": dclgen_repository,
        "table_resolver": table_resolver,
        "host_resolver": host_resolver,
        "statement_transformer": statement_transformer,
    }

```

```

}

def test_idms_statement_transformer_converts_obtain_calc():
    context = build_transformer_context()

    lines, _opened_set = context["statement_transformer"].transform_line(
        line="OBTAIN VMB-FAR CALC.",
        current_division="PROCEDURE",
        sql_error_paragraph="SQL-ERROR",
    )

    text = "\n".join(lines)

    assert "Removed OBTAIN CALC SELECT" in text
    assert "Direct UPDATE will use mapped composite key WHERE clause" in text
    assert "CONTINUE." in text

def test_idms_statement_transformer_converts_store_to_insert_or_skips_safely():
    context = build_transformer_context()

    lines, _opened_set = context["statement_transformer"].transform_line(
        line="STORE VMB-FAR.",
        current_division="PROCEDURE",
        sql_error_paragraph="SQL-ERROR",
    )

    text = "\n".join(lines)

    assert (
        "Converted STORE" in text
        or "INSERT conversion skipped" in text
    )

    assert (
        "INSERT INTO DZBFARTV" in text
        or "missing insert columns" in text
        or "Missing Sheet Mapping or DCLGEN" in text
    )

def test_idms_statement_transformer_removes_bind():
    context = build_transformer_context()

    lines, _opened_set = context["statement_transformer"].transform_line(
        line="BIND RUN-UNIT.",
        current_division="PROCEDURE",
        sql_error_paragraph="SQL-ERROR",
    )

    text = "\n".join(lines)

    assert "Removed IDMS BIND statement" in text
    assert "CONTINUE." in text

def test_cobol_transformer_converts_program_id_and_idms_statement():
    context = build_transformer_context()

    transformer = CobolTransformer(
        idms_statement_transformer=context["statement_transformer"],
    )

    source = """
IDENTIFICATION DIVISION.
PROGRAM-ID. OLDPROG.
PROCEDURE DIVISION.
OBTAIN VMB-FAR CALC.
END PROGRAM OLDPROG.
"""

    converted, messages, operations = transformer.transform(
        cobol_text=source,
        target_program_id="NEWPROG",
    )

```

```

assert "PROGRAM-ID. NEWPROG." in converted
assert "Removed OBTAIN CALC SELECT" in converted
assert "Direct UPDATE will use mapped composite key WHERE clause" in converted
assert len(operations) == 1
assert messages == []

```

```

def test_field_reference_rewriter_rewrites_qualified_reference():
    context = build_transformer_context()

    rewriter = FieldReferenceRewriter(
        mapping_repository=context["mapping_repository"],
        table_name_resolver=context["table_resolver"],
        host_variable_resolver=context["host_resolver"],
    )

    result = rewriter.rewrite("MOVE NR-ID OF VMB-FAR TO WS-NR-ID.")

    assert "NR-ID OF DCLDZBFARTV" in result
    assert "VMB-FAR" not in result

```

```

=====
FILE: tests\test_update_char_literal_quoter.py
ENCODING: utf-8
=====

```

```

from idms_db2_phase2.postprocess.update_final_cleanup import UpdateFinalCleanup

```

```

def _fixed(body: str) -> str:
    """Build an 80-col fixed-format line: left(6)+space+body(65)+space+right(8)."""
    return f"002430 {body[:65].ljust(65)} 02430000"

```

```

def test_non_sql_move_dot_reference_is_rewritten_to_of():
    line = _fixed("MOVE DA-DD-MM-CCYY TO DCLDZBFARTV.DA-INFSDGD-479BFAR")
    out = UpdateFinalCleanup().apply(line)
    assert "DA-INFSDGD-479BFAR OF DCLDZBFARTV" in out
    assert "DCLDZBFARTV.DA-INFSDGD" not in out

```

```

def test_one_line_exec_sql_include_does_not_wedge_flag():
    # A one-line EXEC SQL INCLUDE must NOT cause a later non-SQL dot-reference
    # line to be treated as inside EXEC SQL (the wedge regression).
    include = _fixed("EXEC SQL INCLUDE DZBFARTV END-EXEC.")
    move = _fixed("MOVE DA-DD-MM-CCYY TO DCLDZBFARTV.DA-INFSDGD-479BFAR")
    out = UpdateFinalCleanup().apply(include + "\n" + move)
    assert "DA-INFSDGD-479BFAR OF DCLDZBFARTV" in out
    assert "DCLDZBFARTV.DA-INFSDGD" not in out

```

```

def test_sql_host_reference_inside_exec_sql_preserved():
    block = "\n".join([
        _fixed("EXEC SQL"),
        _fixed("UPDATE DZBFARTV"),
        _fixed("SET"),
        _fixed("DA-INFSDGD_479BFAR = :DCLDZBFARTV.DA-INFSDGD-479BFAR"),
        _fixed("END-EXEC."),
    ])
    out = UpdateFinalCleanup().apply(block)
    # SQL host var must stay in :DCLGROUP.FIELD form.
    assert ":DCLDZBFARTV.DA-INFSDGD-479BFAR" in out

```

```

=====
FILE: tests\test_update_date_host_converter.py
ENCODING: utf-8
=====

```

```

from types import SimpleNamespace

```

```

from idms_db2_phase2.postprocess.update_date_host_converter import (
    UpdateDateHostConverter,

```

```

)

def _col(host, db2_type):
    return SimpleNamespace(
        cobol_host_name=host,
        db2_type=db2_type,
        cobol_picture="",
        column_name="",
        table_name="DZBFARTV",
    )

def _fixed(body: str) -> str:
    return f"002300 {body[:65].ljust(65)} 02300000"

def _converter():
    cols = [
        _col("DA-CRFMAS-479BFAR", "DATE NOT NULL"),
        _col("DA-INFSDGD-479BFAR", "DATE NOT NULL"),
        _col("TS-UPDATE-479BFAR", "TIMESTAMP NOT NULL"),
        _col("ID-USERID-479BFAR", "CHAR(8) NOT NULL"),
    ]
    return UpdateDateHostConverter(cols)

def test_raw_move_to_date_host_is_converted_one_line():
    line = _fixed("MOVE DA-CREA-FORM-AS-R TO DA-CRFMAS-479BFAR OF DCLDZBFARTV")
    out = _converter().apply(line)
    assert "MOVE ZEROES TO DA-CCYMMDD" in out
    assert "MOVE DA-CREA-FORM-AS-R TO DA-CCYMMDD" in out
    assert "MOVE 00010101 TO DA-CCYMMDD" in out
    assert "MOVE CORR DA-CCYMMDD-R TO DA-DD-MM-CCYY" in out
    assert "MOVE DA-DD-MM-CCYY TO DA-CRFMAS-479BFAR OF DCLDZBFARTV" in out
    # The raw direct move must be gone.
    assert "MOVE DA-CREA-FORM-AS-R TO DA-CRFMAS-479BFAR" not in out

def test_raw_move_to_date_host_is_converted_wrapped():
    block = "\n".join([
        _fixed("MOVE DA-CREA-FORM-AS-R"),
        _fixed("TO DA-CRFMAS-479BFAR OF DCLDZBFARTV"),
    ])
    out = _converter().apply(block)
    assert "MOVE DA-DD-MM-CCYY TO DA-CRFMAS-479BFAR OF DCLDZBFARTV" in out

def test_timestamp_audit_move_is_never_converted():
    # REGRESSION GUARD: a TIMESTAMP host receiving TS-TIMESTAMP must stay raw.
    line = _fixed("MOVE TS-TIMESTAMP TO TS-UPDATE-479BFAR OF DCLDZBFARTV")
    out = _converter().apply(line)
    assert "MOVE TS-TIMESTAMP TO TS-UPDATE-479BFAR OF DCLDZBFARTV" in out
    assert "MOVE DA-DD-MM-CCYY TO TS-UPDATE-479BFAR" not in out
    assert "MOVE ZEROES TO DA-CCYMMDD" not in out

def test_user_audit_char_move_is_never_converted():
    line = _fixed("MOVE CS-PROGRAM TO ID-USERID-479BFAR OF DCLDZBFARTV")
    out = _converter().apply(line)
    assert "MOVE CS-PROGRAM TO ID-USERID-479BFAR OF DCLDZBFARTV" in out
    assert "MOVE DA-DD-MM-CCYY TO ID-USERID-479BFAR" not in out

def test_exec_sql_host_reference_untouched():
    block = "\n".join([
        _fixed("EXEC SQL"),
        _fixed("UPDATE DZBFARTV"),
        _fixed("SET DA_CRFMAS_479BFAR = :DCLDZBFARTV.DA-CRFMAS-479BFAR"),
        _fixed("END-EXEC."),
    ])
    out = _converter().apply(block)
    assert ":DCLDZBFARTV.DA-CRFMAS-479BFAR" in out
    assert "MOVE ZEROES TO DA-CCYMMDD" not in out

```

```
=====
FILE: tests\test_update_final_cleanup_dcl_dot.py
ENCODING: utf-8
=====
```

```
from idms_db2_phase2.postprocess.update_final_cleanup import UpdateFinalCleanup
```

```
def _fixed(body: str) -> str:
    """Build an 80-col fixed-format line: left(6)+space+body(65)+space+right(8)."""
    return f"002430 {body[:65].ljust(65)} 02430000"
```

```
def test_non_sql_move_dot_reference_is_rewritten_to_of():
    line = _fixed("MOVE DA-DD-MM-CCYY TO DCLDZBFARTV.DA-INFSDGD-479BFAR")
    out = UpdateFinalCleanup().apply(line)
    assert "DA-INFSDGD-479BFAR OF DCLDZBFARTV" in out
    assert "DCLDZBFARTV.DA-INFSDGD" not in out
```

```
def test_one_line_exec_sql_include_does_not_wedge_flag():
    # A one-line EXEC SQL INCLUDE must NOT cause a later non-SQL dot-reference
    # line to be treated as inside EXEC SQL (the wedge regression).
    include = _fixed("EXEC SQL INCLUDE DZBFARTV END-EXEC.")
    move = _fixed("MOVE DA-DD-MM-CCYY TO DCLDZBFARTV.DA-INFSDGD-479BFAR")
    out = UpdateFinalCleanup().apply(include + "\n" + move)
    assert "DA-INFSDGD-479BFAR OF DCLDZBFARTV" in out
    assert "DCLDZBFARTV.DA-INFSDGD" not in out
```

```
def test_sql_host_reference_inside_exec_sql_preserved():
    block = "\n".join([
        _fixed("EXEC SQL"),
        _fixed("UPDATE DZBFARTV"),
        _fixed("SET"),
        _fixed("DA-INFSDGD-479BFAR = :DCLDZBFARTV.DA-INFSDGD-479BFAR"),
        _fixed("END-EXEC."),
    ])
    out = UpdateFinalCleanup().apply(block)
    # SQL host var must stay in :DCLGROUP.FIELD form.
    assert ":DCLDZBFARTV.DA-INFSDGD-479BFAR" in out
```

```
=====
FILE: tests\test_update_input_record_renamer.py
ENCODING: utf-8
=====
```

```
from idms_db2_phase2.postprocess.update_input_record_renamer import (
    UpdateInputRecordRenamer,
)
```

```
def _fixed(body: str, seq: str = "000750") -> str:
    return f"{seq} {body[:65].ljust(65)} 00750000"
```

```
def _comment(body: str, seq: str = "000260") -> str:
    # Column 7 = '*' comment indicator.
    return f"{seq}*{body[:65].ljust(65)} 00260000"
```

```
def test_executable_references_renamed():
    block = "\n".join([
        _fixed("COPY VMBD205I."),
        _fixed("INITIALIZE VMBD205I."),
        _fixed("READ OPERATIES INTO VMBD205I"),
        _fixed("MOVE NR-ID-FORM-AS OF VMBD205I"),
    ])
    out = UpdateInputRecordRenamer().apply(
        block, old_name="VMBD205I", new_name="VMDZ205I"
    )
    assert "VMBD205I" not in out
    assert "COPY VMDZ205I" in out
```

```

assert "INITIALIZE VMDZ205I" in out
assert "OF VMDZ205I" in out

def test_comment_line_preserved():
    block = "\n".join([
        _comment("DAISY SECURITIES KASBONS : VMBD510X --> VMBD205I"),
        _fixed("COPY VMBD205I."),
    ])
    out = UpdateInputRecordRenamer().apply(
        block, old_name="VMBD205I", new_name="VMDZ205I"
    )
    # The comment (historical REMARKS) must keep the original VMBD205I.
    assert "VMBD205I --" in out or "VMBD205I" in out.splitlines()[0]
    # But the executable COPY must be renamed.
    assert "COPY VMDZ205I" in out

def test_duplicate_copy_deduplicated():
    block = "\n".join([
        _fixed("COPY VMBD205I.", seq="000750"),
        _fixed("COPY VMDZ205I.", seq="001680"),
    ])
    out = UpdateInputRecordRenamer().apply(
        block, old_name="VMBD205I", new_name="VMDZ205I"
    )
    # After renaming the first COPY to VMDZ205I, only ONE COPY VMDZ205I remains.
    assert out.upper().count("COPY VMDZ205I") == 1

def test_no_change_when_old_equals_new():
    block = _fixed("COPY VMDZ205I.")
    out = UpdateInputRecordRenamer().apply(
        block, old_name="VMDZ205I", new_name="VMDZ205I"
    )
    assert "COPY VMDZ205I" in out

=====
FILE: tests\test_update_restart_skip_no_message.py
ENCODING: utf-8
=====

from idms_db2_phase2.composers.update_restart_skip_composer import (
    UpdateRestartSkipComposer,
)

def _fixed(body: str) -> str:
    return f"002690 {body[:65].ljust(65)} 02690000"

def test_manual_redesign_message_suppressed_by_default():
    text = "\n".join([
        _fixed("*DB2: UPDATE conversion skipped for FFRECA.B."),
        _fixed("*DB2: Missing Sheet Mapping and DCLGEN metadata."),
        _fixed("CONTINUE."),
    ])
    composer = UpdateRestartSkipComposer()
    composer.compose(text)
    assert not any(
        "manual DB2 redesign" in m for m in composer.messages
    )

def test_message_flag_is_opt_in():
    composer = UpdateRestartSkipComposer()
    assert composer.EMIT_MANUAL_REDESIGN_MESSAGE is False

# --- Manual-redesign diagnostic -----
#
# When a restart or control record has no usable Sheet Mapping entry,
# UpdateRestartSkipComposer replaces the generated block with a
# manual-redesign comment and CONTINUE. That replacement is visible in
# the generated COBOL and is the authoritative signal.

```

```

#
# Emitting a matching VALIDATION MESSAGE as well is opt-in, because it
# repeats on every run for a condition that is an input-data gap rather
# than a converter defect, and drowns out genuine diagnostics.
#
# Consumed by:
#   src/idms_db2_phase2/composers/update_restart_skip_composer.py
#
# Mirrored onto the composer as a class attribute so callers and tests
# read it from one place and the two cannot drift apart.
EMIT_MANUAL_REDESIGN_MESSAGE = False

MANUAL_REDESIGN_MESSAGE_TEMPLATE = (
    "Update restart skip: {record} has no usable Sheet Mapping entry; "
    "block replaced with a manual-redesign comment."
)

=====
FILE: tests\test_validators.py
ENCODING: utf-8
=====

from idms_db2_phase2.domain.models import ConversionInput, DclgenColumn, SheetMappingRow
from idms_db2_phase2.repositories.dclgen_repository import DclgenRepository
from idms_db2_phase2.repositories.mapping_repository import MappingRepository
from idms_db2_phase2.resolvers.table_name_resolver import TableNameResolver
from idms_db2_phase2.validators.input_validator import InputValidator
from idms_db2_phase2.validators.mapping_validator import MappingValidator
from idms_db2_phase2.validators.production_validator import ProductionValidator

def test_input_validator_detects_missing_inputs():
    validator = InputValidator()

    messages = validator.validate(ConversionInput())

    assert "Sheet Mapping is required and must contain rows." in messages
    assert "At least one DCLGEN file is required." in messages
    assert "IDMS COBOL source text file is required and must contain readable COBOL code." in messages

def test_input_validator_passes_with_required_inputs():
    validator = InputValidator()

    messages = validator.validate(
        ConversionInput(
            sheet_mapping_rows=[SheetMappingRow(cobol_record_idms="REC")],
            dclgen_columns=[DclgenColumn(table_name="TABLE")],
            idms_cobol_text="IDENTIFICATION DIVISION.",
        )
    )

    assert messages == []

def test_mapping_validator_detects_missing_metadata():
    mapping_repository = MappingRepository([])
    dclgen_repository = DclgenRepository([])
    table_resolver = TableNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
    )

    validator = MappingValidator(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
    )

    messages = validator.validate()

    assert "Mapping validation: Sheet Mapping rows are missing." in messages
    assert "Mapping validation: DCLGEN columns are missing." in messages

```



```

def test_mapping_validator_passes_basic_mapping():
    mapping_repository = MappingRepository(
        [
            SheetMappingRow(
                cobol_record_idms="VMB-FAR",
                new_db2_record="DZBFARTB",
                new_db2_field_name="NR_ID",
            )
        ]
    )
    dclgen_repository = DclgenRepository(
        [
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="NR_ID",
                cobol_host_name="NR-ID",
            )
        ]
    )
    table_resolver = TableNameResolver(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
    )

    validator = MappingValidator(
        mapping_repository=mapping_repository,
        dclgen_repository=dclgen_repository,
        table_name_resolver=table_resolver,
    )

    messages = validator.validate()

    assert "Mapping validation: Sheet Mapping rows are missing." not in messages
    assert "Mapping validation: DCLGEN columns are missing." not in messages

def test_production_validator_detects_missing_db2_tokens():
    dclgen_repository = DclgenRepository([])
    validator = ProductionValidator(dclgen_repository=dclgen_repository)

    messages = validator.validate("IDENTIFICATION DIVISION.")

    assert "Production validation: required DB2 token missing: EXEC SQL" in messages
    assert "Production validation: required DB2 token missing: SQLCA" in messages
    assert "Production validation: required DB2 token missing: END-EXEC" in messages

def test_production_validator_detects_residual_idms_statement():
    dclgen_repository = DclgenRepository([])
    validator = ProductionValidator(dclgen_repository=dclgen_repository)

    text = """
EXEC SQL
    INCLUDE SQLCA
END-EXEC.
PROCEDURE DIVISION.
OBTAIN VMB-FAR CALC.
"""

    messages = validator.validate(text)

    assert any("residual executable IDMS statement remains" in message for message in messages)

def test_production_validator_accepts_valid_host_reference():
    dclgen_repository = DclgenRepository(
        [
            DclgenColumn(
                table_name="DZBFARTV",
                column_name="NR_ID",
                cobol_host_name="NR-ID",
            )
        ]
    )
    validator = ProductionValidator(dclgen_repository=dclgen_repository)

```

```

text = """
EXEC SQL
    INCLUDE SQLCA
END-EXEC.
EXEC SQL
    SELECT NR_ID
    INTO :DCLDZBFARTV.NR-ID
    FROM DZBFARTV
END-EXEC.
"""

messages = validator.validate(text)

assert not any("was not found in uploaded DCLGEN columns" in message for message in messages)

=====
FILE: tools\check_record_materialisation.py
ENCODING: utf-8
=====

# LOCATION: tools/check_record_materialisation.py
# ACTION: CREATE NEW FILE
"""Pinpoints why record materialisation did not run.

    set PYTHONPATH=src;.
    python tools\check_record_materialisation.py

Exit 0 = the composer works in isolation; the problem is wiring.
Exit 1 = the composer itself is broken; the reason is printed.
"""

from __future__ import annotations

import sys
import traceback
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
for candidate in (ROOT, ROOT / "src"):
    if str(candidate) not in sys.path:
        sys.path.insert(0, str(candidate))

REQUIRED_RULES = [
    "ALLOW_LAYOUT_SHORTER_THAN_TARGET", "EMIT_REMAINDER_FILLER",
    "FILLER_FIELD_NAMES", "LAYOUT_BODY_WIDTH", "LAYOUT_MAX_INDENT_DEPTH",
    "LAYOUT_MIN_NAME_GAP", "MAX_RECORD_MOVES", "MIN_LEVEL", "MAX_LEVEL",
    "MOVE_CONTINUATION_INDENT", "MOVE_INLINE_LIMIT",
    "NON_COLUMN_SENTINELS", "NON_RECORD_NAME_SUFFIXES",
    "RECORD_GROUP_LEVEL_STEP", "REDEFINES_CONTINUATION_GAP",
    "REDEFINES_HEAD_TEMPLATE", "REDEFINES_INLINE_LIMIT",
    "REMAINDER_FILLER_NAME", "REMAINDER_FILLER_PICTURE_TEMPLATE",
    "WRAP_REDEFINES_ENTRY",
]
REQUIRED_PATTERNS = ["PICTURE_CLAUSE_PATTERN", "PICTURE_SYMBOL_PATTERN"]
REQUIRED_MESSAGES = [
    "materialised", "remainder_filler", "scan_summary", "no_moves",
    "composer_absent", "no_declared_length", "skipped_not_a_record",
]

def _fail(step: str, detail: str) -> None:
    print(f"\n FAIL {step}\n      {detail}")
    sys.exit(1)

def _ok(step: str, detail: str = "") -> None:
    print(f" OK      {step}" + (f" ({detail})" if detail else ""))

# ----- 1
print("\n1. rules / patterns constants")
try:
    import rules.record_materialisation_rules as R
    import patterns.record_materialisation_patterns as P
except Exception:

```

```

        _fail("import", traceback.format_exc(limit=3))

missing = [n for n in REQUIRED_RULES if not hasattr(R, n)]
if missing:
    _fail("rules constants", "missing: " + ", ".join(missing))
_ok("rules constants", f"{len(REQUIRED_RULES)} present")

missing = [n for n in REQUIRED_PATTERNS if not hasattr(P, n)]
if missing:
    _fail("patterns", "missing: " + ", ".join(missing))
_ok("patterns", f"{len(REQUIRED_PATTERNS)} present")

missing = [k for k in REQUIRED_MESSAGES
            if k not in R.RECORD_MATERIALISATION_MESSAGES]
if missing:
    _fail("message keys", "missing: " + ", ".join(missing))
_ok("message keys")

# ----- 2
print("\n2. module imports")
try:
    from idms_db2_phase2.composers.record_materialisation_composer import (
        RecordMaterialisationComposer,
    )
except Exception:
    _fail("composer import", traceback.format_exc(limit=5))
_ok("composer import")

# ----- 3
print("\n3. composer behaviour on a synthetic program")

class _Row:
    def __init__(self, zone, pic, length, column):
        self.cobol_zone = zone
        self.idms_pic_clause = pic
        self.length_of_field_bytes = length
        self.new_db2_field_name = column
        self.new_db2_data_type = ""
        self.cross_application_db2_field_name = ""

class _Mapping:
    def rows_for_record(self, record):
        if str(record).upper() != "TESTREC":
            return []
        return [
            _Row("02 F-ALPHA", "PIC X(10)", "10", "COL_ALPHA"),
            _Row("02 FILLER", "PIC X(5)", "5", "FILLER"),
        ]

class _Table:
    def table_for_record(self, record):
        return "TESTTB" if str(record).upper() == "TESTREC" else ""

class _Host:
    def group_for_table(self, table):
        return "DCLTESTTV"

    def host_reference_for_column(self, table_name, column_name):
        return f":DCLTESTTV.{str(column_name).replace('_', '-')}"

def _line(seq: str, body: str) -> str:
    return f"{seq} {body.ljust(65)}{seq}0"

SOURCE = "\n".join([
    _line("000100", "DATA DIVISION."),
    _line("000200", "WORKING-STORAGE SECTION."),
    _line("000300", "01 OUT-REC."),
    _line("000400", "    05 F-TARGET                PIC X(20)."),
    _line("000500", "PROCEDURE DIVISION."),
    _line("000600", "    MOVE TESTREC TO F-TARGET."),
])

```

```

]) + "\n"

composer = RecordMaterialisationComposer(
    mapping_repository=_Mapping(),
    table_name_resolver=_Table(),
    host_variable_resolver=_Host(),
)

try:
    result = composer.compose(SOURCE)
except Exception:
    _fail("compose()", traceback.format_exc(limit=8))

print("\n --- messages ---")
for message in composer.messages:
    print(f"      {message}")

print("\n --- output ---")
for row in result.splitlines():
    print(f"      {row.rstrip()}")

expanded = "06 TESTREC." in result
filler = "FILLER" in result and "X(5)" in result
remainder = "X(5)." in result and result.count("FILLER") >= 2

print()
if not expanded:
    _fail("expansion", "F-TARGET was not expanded - see messages above")
    _ok("expansion", "TESTREC layout emitted")
    _ok("filler handling" if filler else "filler handling")
print(f" { 'OK' if remainder else 'WARN ' } remainder FILLER "
      f"{'emitted' if remainder else 'NOT emitted - check declared_bytes'}")

print("\nComposer works in isolation. If the app still skips it, the")
print("problem is WIRING:")
print(" a) conversion_component_factory.py must register")
print("    composers['record_materialisation'] = RecordMaterialisationComposer(...)")
print(" b) conversion_layout_pipeline.finish() must call")
print("    steps.materialise_records(converted_cobol=..., validation_messages=...)")
sys.exit(0)

```

```

=====
FILE: tools\date_rewrite_probe.py
ENCODING: utf-8
=====

```

```

# LOCATION: tools/date_rewrite_probe.py
# ACTION: CREATE NEW FILE

```

```

"""Runs ONLY the DB2 date rewriter and verifies column geometry.

```

```

Three attempts at fixing a mangled condition produced byte-identical
output, which means the rewriter is either not running or not the cause.
This probe answers that question directly: it feeds the exact VMDZ7200
condition to the rewriter alone, with no pipeline around it, and checks
every emitted record.

```

```

$env:PYTHONPATH = "src;."
python tools\date_rewrite_probe.py

```

```

Exit 0: the rewriter is clean, the defect is downstream.
Exit 1: the rewriter is the culprit.
"""

```

```

from __future__ import annotations

```

```

import sys
from pathlib import Path

```

```

PROJECT_ROOT = Path(__file__).resolve().parents[1]
SRC_DIR = PROJECT_ROOT / "src"

```

```

for _candidate in (PROJECT_ROOT, SRC_DIR):
    if str(_candidate) not in sys.path:
        sys.path.insert(0, str(_candidate))

```

```

from idms_db2_phase2.composers.db2_date_comparison_rewriter import ( # noqa: E402
    Db2DateComparisonRewriter,
)

```

```

LINE_WIDTH = 80
BODY_START = 7
BODY_END = 72

```

```

def fixed(seq: str, body: str, right: str) -> str:
    return f"{seq} {body.ljust(65)}{right}"

```

```

PROGRAM = [
    fixed("001750", " PROCEDURE DIVISION.", "01750000"),
    fixed("002290", " BEHANDLING.", "02290000"),
    fixed("002110", "      IF (DA-CPTAFS-479BFAS OF DCLDZBFASTV < DA-ARCH-YMD", "02110000"),
    fixed("002120", "          AND DA-CPTAFS-479BFAS OF DCLDZBFASTV NOT = '00000000') OR", "02120000"),
    fixed("002130", "          (DA-CRFMAS-479BFAS OF DCLDZBFASTV < DA-ARCH-YMD AND", "02130000"),
    fixed("002140", "          DA-CPTAFS-479BFAS OF DCLDZBFASTV = '00000000')", "02140000"),
    fixed("002150", "          ADD 1 TO WS-TELLER", "02150000"),
    fixed("002160", "      END-IF.", "02160000"),
]

```

```

def main() -> None:
    rewriter = Db2DateComparisonRewriter()

    print("=" * 78)
    print("MODULE :", Db2DateComparisonRewriter.__module__)
    print("FILE   :", sys.modules[Db2DateComparisonRewriter.__module__].__file__)
    print("SLICING:", hasattr(rewriter, "_is_fixed_record"))
    print("=" * 78)

    output = rewriter.rewrite_date_comparisons(list(PROGRAM))

    problems: list[str] = []

    print("\nEMITTED RECORDS")
    print("-" * 78)

    for line in output:
        marker = " "
        # A GENERATED line is a plain body with no sequence area; the
        # fixed-format composer numbers it later. Only a line that
        # already carries a left sequence must be a full 80-column
        # record.
        carries_sequence = len(line) >= BODY_START and line[:6].isdigit()

        if carries_sequence and len(line) != LINE_WIDTH:
            marker = "!"
            problems.append(f"width {len(line)}: {line!r}")
        elif line[:6].isdigit() and not line[BODY_END:LINE_WIDTH].isdigit():
            marker = "!"
            problems.append(f"right sequence moved: {line!r}")

        body = line[BODY_START:BODY_END].rstrip() if len(line) >= BODY_END else line
        print(f"{marker} |{body}|")

    print("-" * 78)
    print("\nDIAGNOSTICS")
    for message in rewriter.messages:
        print(" -", message)

    print("\nRESULT")
    if problems:
        print(" REWRITER IS THE CULPRIT")
        for problem in problems:
            print("   ", problem)
        raise SystemExit(1)

    print(" Rewriter output is clean. The defect is in a LATER pass.")
    raise SystemExit(0)

```

```

if __name__ == "__main__":
    main()

=====
FILE: tools\downstream_probe.py
ENCODING: utf-8
=====

# LOCATION: tools/downstream_probe.py
# ACTION: CREATE NEW FILE

"""Finds which post-conversion pass corrupts a rewritten IF condition.

BACKGROUND
-----
tools/date_rewrite_probe.py proved Db2DateComparisonRewriter emits four
clean 80-column records:

    IF (HELP-DA-CPTAFS-479BFAS < DA-ARCH-YMD
        AND HELP-DA-CPTAFS-479BFAS NOT = '00000000') OR
        (HELP-DA-CRFMAS-479BFAS < DA-ARCH-YMD AND
        HELP-DA-CPTAFS-479BFAS = '00000000')

The generated file contains six records with a stray '0' on three of
them:

    AND 0
    HELP-DA-CPTAFS-479BFAS NOT = '00000000') OR 0
    = '00000000') 0

A bare '0' is the first character of the right sequence area 02610000,
so some pass is reading the body as columns 8-73 instead of 8-72, or is
re-wrapping a body that already carries sequence digits.

This probe feeds the KNOWN-GOOD records to each candidate pass in turn
and reports the first one that changes the line count or introduces a
stray token.

    $env:PYTHONPATH = "src;."
    python tools\downstream_probe.py
"""

from __future__ import annotations

import importlib
import sys
import traceback
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
SRC_DIR = PROJECT_ROOT / "src"

for _candidate in (PROJECT_ROOT, SRC_DIR):
    if str(_candidate) not in sys.path:
        sys.path.insert(0, str(_candidate))

BODY_START = 7
BODY_END = 72
LINE_WIDTH = 80

def fixed(seq: str, body: str, right: str) -> str:
    return f"{seq} {body.ljust(65)}{right}"

# Exactly what the rewriter emits, verified by date_rewrite_probe.
CLEAN = [
    fixed("001750", " PROCEDURE DIVISION.", "01750000"),
    fixed("002290", " BEHANDELING.", "02290000"),
    fixed("002600", "         IF (HELP-DA-CPTAFS-479BFAS < DA-ARCH-YMD", "02600000"),
    fixed("002610", "         AND HELP-DA-CPTAFS-479BFAS NOT = '00000000') OR", "02610000"),
    fixed("002620", "         (HELP-DA-CRFMAS-479BFAS < DA-ARCH-YMD AND", "02620000"),
    fixed("002630", "         HELP-DA-CPTAFS-479BFAS = '00000000')", "02630000"),
    fixed("002660", "         ADD 1                TO WS-TELLER", "02660000"),
    fixed("002770", "         END-IF.", "02770000"),

```

```

]

CLEAN_TEXT = "\n".join(CLEAN) + "\n"

# (label, [candidate module paths], class name, method name)
CANDIDATES = [
    (
        "FixedFormatComposer",
        [
            "idms_db2_phase2.composers.fixed_format_composer",
            "idms_db2_phase2.composers.fixed_format",
        ],
        "FixedFormatComposer",
        ("format", "compose"),
    ),
    (
        "ProcedureIndentNormalizer",
        [
            "idms_db2_phase2.services.procedure_indent_normalizer",
            "idms_db2_phase2.composers.procedure_indent_normalizer",
        ],
        "ProcedureIndentNormalizer",
        ("compose", "format"),
    ),
    (
        "CobolAreaAlignmentReflow",
        [
            "idms_db2_phase2.services.cobol_area_alignment_reflow",
        ],
        "CobolAreaAlignmentReflow",
        ("compose", "reflow"),
    ),
    (
        "SequenceArtifactCleanupComposer",
        [
            "idms_db2_phase2.composers.sequence_artifact_cleanup_composer",
        ],
        "SequenceArtifactCleanupComposer",
        ("compose",),
    ),
    (
        "FinalSequenceResequencerService",
        [
            "idms_db2_phase2.services.final_sequence_resequencer_service",
        ],
        "FinalSequenceResequencerService",
        ("resequence", "compose"),
    ),
    (
        "FinalCobolFixComposer",
        [
            "idms_db2_phase2.composers.final_cobol_fix_composer",
        ],
        "FinalCobolFixComposer",
        ("compose", "format"),
    ),
    (
        "StructuralSafetyComposer",
        [
            "idms_db2_phase2.composers.structural_safety_composer",
        ],
        "StructuralSafetyComposer",
        ("compose",),
    ),
]

```

```

STRAY_MARKERS = (" 0", "AND 0", "OR 0", "'') 0")

```

```

def load(paths: list[str], class_name: str):
    for path in paths:
        try:
            module = importlib.import_module(path)
        except Exception: # noqa: BLE001
            continue

```

```

        target = getattr(module, class_name, None)

        if target is not None:
            return target, path

    return None, ""

def condition_bodies(text: str) -> list[str]:
    output = []

    for line in str(text or "").splitlines():
        body = line[BODY_START:BODY_END] if len(line) >= BODY_END else line[BODY_START:]
        body = body.rstrip()

        if "HELP-DA-" in body and "MOVE" not in body:
            output.append(body)

    return output

BASELINE = condition_bodies(CLEAN_TEXT)

def report(label: str, module_path: str, text: str) -> bool:
    bodies = condition_bodies(text)
    broken = False

    print(f"\n{label}")
    print(f"  module: {module_path}")
    print(f"  condition records: {len(BASELINE)} -> {len(bodies)}")

    if len(bodies) != len(BASELINE):
        broken = True

    for body in bodies:
        flag = " "
        if body.rstrip().endswith(" 0") or "'" 0" in body:
            flag = "!"
            broken = True
        print(f"  {flag} |{body}|")

    print("  VERDICT:", "CORRUPTS" if broken else "clean")
    return broken

def main() -> None:
    print("=" * 78)
    print("BASELINE (verified clean output of the date rewriter)")
    print("=" * 78)
    for body in BASELINE:
        print(f"  |{body}|")

    print("\n" + "=" * 78)
    print("RUNNING EACH DOWNSTREAM PASS IN ISOLATION")
    print("=" * 78)

```

```

=====
FILE: tools\reflow_probe.py
ENCODING: utf-8
=====

```

```

# LOCATION: tools/reflow_probe.py
# ACTION: REPLACE ENTIRE FILE

```

```

"""Feeds the real VMDZ7200 condition to CobolAreaAlignmentReflow alone.

```

```

    $env:PYTHONPATH = "src;."
    python tools\\reflow_probe.py

```

```

A '!' on any emitted body names the reflow as the source of the stray
'0' and shows exactly which input pair triggers it.
"""

```



```

from __future__ import annotations

import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]

for _path in (ROOT, ROOT / "src"):
    if str(_path) not in sys.path:
        sys.path.insert(0, str(_path))

from idms_db2_phase2.services.cobol_area_alignment_classifier import ( # noqa: E402
    CobolAreaAlignmentClassifier,
)
from idms_db2_phase2.services.cobol_area_alignment_reflow import ( # noqa: E402
    CobolAreaAlignmentReflow,
)
from idms_db2_phase2.services.fixed_format_line_service import ( # noqa: E402
    FixedFormatLineService,
)

QUOTE = chr(39)
ZER08 = QUOTE + "00000000" + QUOTE
AREA_B = "    "

def fixed(seq: str, body: str, right: str) -> str:
    return seq + " " + body.ljust(65) + right

L1 = fixed("002110", "    IF (HELP-DA-CPTAFS-479BFAS < DA-ARCH-YMD", "02110000")
L2 = fixed("002120", "    AND HELP-DA-CPTAFS-479BFAS NOT = " + ZER08 + ") OR", "02120000")
L3 = fixed("002130", "    (HELP-DA-CRFMAS-479BFAS < DA-ARCH-YMD AND", "02130000")
L4 = fixed("002140", "    HELP-DA-CPTAFS-479BFAS = " + ZER08 + ")", "02140000")

def body_of(line: str) -> str:
    return line[7:72].rstrip() if len(line) >= 72 else str(line)

def main() -> None:
    fixed_format = FixedFormatLineService()

    reflow = CobolAreaAlignmentReflow(
        fixed_format=fixed_format,
        classifier=CobolAreaAlignmentClassifier(),
    )

    print("INPUT")
    for line in (L1, L2, L3, L4):
        print("    |" + body_of(line) + "|")

    print("\nlogical() readback")
    for line in (L1, L2, L3, L4):
        print("    " + repr(fixed_format.logical(line)))

    for name, first, second in (
        ("L1 + L2", L1, L2),
        ("L2 + L3", L2, L3),
        ("L3 + L4", L3, L4),
        ("L4 + END-IF", L4, fixed("002160", "    END-IF.", "02160000")),
    ):
        print("\n-- " + name)

        try:
            out = reflow.try_reflow_with_next_line(
                current_line=first,
                next_line=second,
                first_indent=AREA_B,
            )
        except Exception as exc: # noqa: BLE001
            print("    RAISED:", type(exc).__name__, exc)
            continue

        if not out:

```

```

        print("    (no reflow - pair left alone)")
        continue

    print("    reflowed into", len(out), "line(s)")

    for line in out:
        body = body_of(line)
        flag = "!" if body.rstrip().endswith(" 0") else " "
        print(" " + flag + " |" + body + "|" len=" + str(len(line)))

if __name__ == "__main__":
    main()

=====
FILE: tools\rules_probe.py
ENCODING: utf-8
=====

# LOCATION: tools/rules_probe.py
# ACTION: REPLACE ENTIRE FILE

"""Developer probe: verify every `from rules.X import (...)` name exists.

Scans the project for imports from rules/, patterns/ and catalogs/, then
checks that each imported name actually exists in the target module.

Catches the exact failure class that has cost this project several
round-trips: a constant referenced by a composer or generator but never
declared in its rules file, which only surfaces at runtime as

    ImportError: cannot import name 'COMMENT_BLOCK_TEMPLATE' from
    'rules.cursor_paragraph_rules'

BOM HANDLING
-----
Several source files in this project are saved as UTF-8 WITH a byte order
mark. Python imports them without complaint because the import machinery
uses utf-8-sig, but ast.parse() on text read as plain utf-8 fails with

    invalid non-printable character U+FEFF

That is a reader defect, not a source defect, so this probe reads with
utf-8-sig and reports a BOM separately as a hygiene note.

Run from the project root:

    $env:PYTHONPATH = "src;."
    python tools\rules_probe.py

Exit codes: 0 all names resolve, 1 one or more are missing.
"""

from __future__ import annotations

import ast
import importlib
import sys
from pathlib import Path

# -----
# Path bootstrap. Must run before any project import, because running
# "python tools\rules_probe.py" puts only tools\ on sys.path.
# -----
PROJECT_ROOT = Path(__file__).resolve().parents[1]
SRC_DIR = PROJECT_ROOT / "src"

for _candidate in (PROJECT_ROOT, SRC_DIR):
    if str(_candidate) not in sys.path:
        sys.path.insert(0, str(_candidate))

SCAN_DIRS = ("src", "rules", "patterns", "catalogs", "code_review", "tests")
WATCHED_PACKAGES = ("rules", "patterns", "catalogs")

SKIP_DIR_NAMES = {

```

```

    "__pycache__",
    ".venv",
    ".git",
    ".pytest_cache",
    ".mypy_cache",
    "node_modules",
}

# Reading with utf-8-sig strips a leading BOM transparently.
SOURCE_ENCODING = "utf-8-sig"
BOM = "\ufeff"

def _python_files() -> list[Path]:
    files: list[Path] = []

    for folder in SCAN_DIRS:
        base = PROJECT_ROOT / folder
        if not base.exists():
            continue

        for path in sorted(base.rglob("*.py")):
            if any(part in SKIP_DIR_NAMES for part in path.parts):
                continue
            files.append(path)

    return files

def _relative(path: Path) -> str:
    try:
        return str(path.relative_to(PROJECT_ROOT))
    except ValueError:
        return str(path)

def _read_source(path: Path) -> str:
    """Read a source file, stripping any byte order mark."""
    text = path.read_text(encoding=SOURCE_ENCODING, errors="replace")
    return text.lstrip(BOM)

def _has_bom(path: Path) -> bool:
    try:
        return path.read_bytes()[3] == b"\xef\xbb\xbf"
    except OSError:
        return False

def missing_names() -> tuple[list[str], list[str]]:
    """Return (problems, bom_files)."""
    problems: list[str] = []
    bom_files: list[str] = []
    cache: dict[str, object] = {}

    for path in _python_files():
        if _has_bom(path):
            bom_files.append(_relative(path))

        try:
            tree = ast.parse(_read_source(path), filename=str(path))
        except SyntaxError as exc:
            problems.append(f"{_relative(path)} -> syntax error: {exc}")
            continue

        for node in ast.walk(tree):
            if not isinstance(node, ast.ImportFrom):
                continue

            module = node.module or ""
            if not module.startswith(WATCHED_PACKAGES):
                continue

            if module not in cache:
                try:
                    cache[module] = importlib.import_module(module)

```

```

        except Exception as exc: # noqa: BLE001
            problems.append(
                f"{_relative(path)} -> cannot import {module}: {exc}"
            )
            cache[module] = None

    target = cache.get(module)
    if target is None:
        continue

    for alias in node.names:
        if alias.name == "*":
            continue
        if not hasattr(target, alias.name):
            problems.append(
                f"{_relative(path)} -> "
                f"{module}.{alias.name} does not exist"
            )

    return problems, bom_files

def unused_modules() -> list[str]:
    """Rules / patterns / catalogs modules nothing imports from.

    Informational. A brand new rules file that nothing consumes usually
    means the wiring step was missed.
    """
    imported: set[str] = set()

    for path in _python_files():
        try:
            tree = ast.parse(_read_source(path), filename=str(path))
        except SyntaxError:
            continue

        for node in ast.walk(tree):
            if isinstance(node, ast.ImportFrom) and node.module:
                imported.add(node.module)

    orphans: list[str] = []

    for package in WATCHED_PACKAGES:
        base = PROJECT_ROOT / package
        if not base.exists():
            continue

        for path in sorted(base.rglob("*.py")):
            if path.name == "__init__.py":
                continue
            if any(part in SKIP_DIR_NAMES for part in path.parts):
                continue

            module = (
                path.relative_to(PROJECT_ROOT)
                .with_suffix("")
                .as_posix()
                .replace("/", ".")
            )
            if module not in imported:
                orphans.append(module)

    return orphans

def misplaced_rules_modules() -> list[str]:
    """Files named *_rules.py sitting under patterns/.

    The Clean Architecture Rule puts regex in patterns/ and business or
    layout constants in rules/. A rules file under patterns/ is drift.
    """
    base = PROJECT_ROOT / "patterns"
    if not base.exists():
        return []

    return [

```

```

        _relative(path)
        for path in sorted(base.rglob("*_rules.py"))
        if not any(part in SKIP_DIR_NAMES for part in path.parts)
    ]

def main() -> None:
    problems, bom_files = missing_names()
    orphans = unused_modules()
    misplaced = misplaced_rules_modules()

    if orphans:
        print("Modules nothing imports from (informational):")
        for module in orphans:
            print(f" {module}")
        print("")

    if misplaced:
        print("Rules files under patterns/ (architecture drift):")
        for module in misplaced:
            print(f" {module}")
        print("")

    if bom_files:
        print("Files saved with a UTF-8 BOM (hygiene only, imports fine):")
        for module in bom_files:
            print(f" {module}")
        print("")

    if not problems:
        print("rules / patterns / catalogs imports OK")
        raise SystemExit(0)

    print("MISSING NAMES")
    print("-" * 60)
    for problem in problems:
        print(problem)
    print("-" * 60)
    print(f"{len(problems)} problem(s)")
    raise SystemExit(1)

if __name__ == "__main__":
    main()

=====
EXPORT SUMMARY
=====
Project folder: IDMS-DB2-Code-Conversion
Text/code files processed: 594

```